(4,8):(8,1)

(128,128):(1,128)

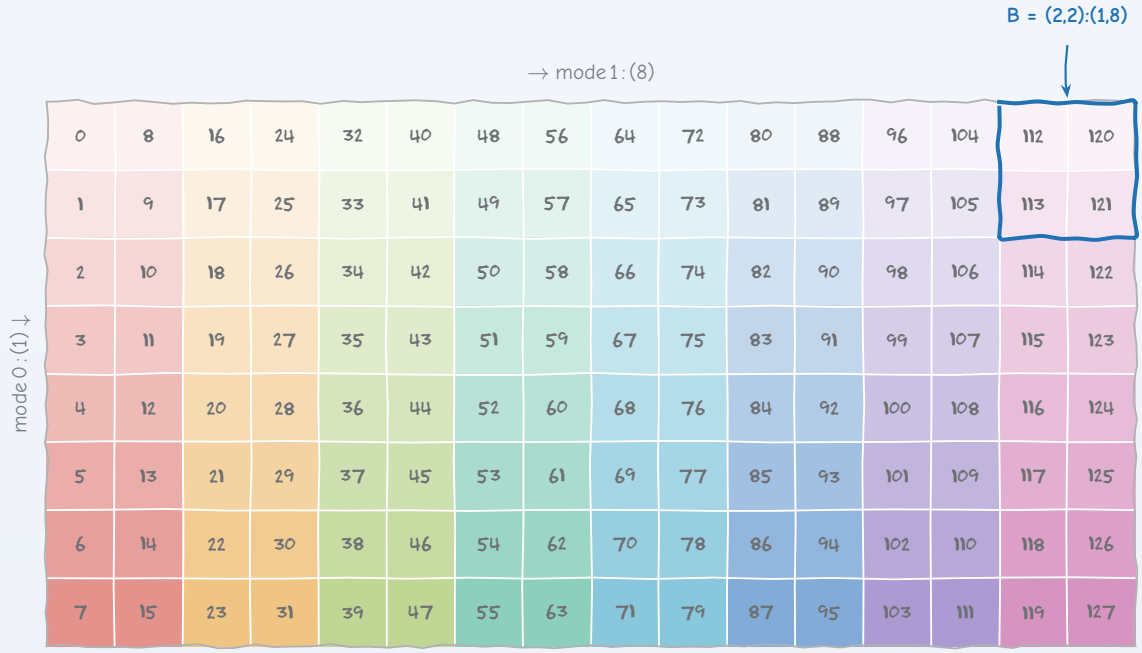
LEETCUDA

CUDA KERNEL 优化之路

从编程模型到 CUTE 与 FLASHATTENTION

((2,2),(2,2))

(3):(1)



L = (8,16):(1,8)

(16,8):(8,1)

(8,16):(1,8)

```
auto tiled_mma = make_tiled_mma(SM80_16x8x16_F32F16F16F32_TN{}, Layout<Shape<_2,_2,_1>>{});
```

郑 重 声 明
本 书 仅 用 于 开 源 学 习 与 技 术 交 流，
未 经 作 者 本 人 同 意， 不 得 用 于 任 何 商 业 用 途。

目录

第一部分　基础篇：编程模型、访存与归约 1

第一章　性能分析先于优化：nsys 与 ncu 2

1.1 本章导读 . . . 2
1.2 问题与动机：不 profile 就优化的代价 . . . 2
1.3 工具链总览与顺序方法论 . . . 2
1.3.1 两个工具回答两个不同的问题 . . . 2
1.3.2 顺序：先 nsys，后 ncu . . . 3
1.4 nsys 命令行实战 . . . 3
1.4.1 最小采集 . . . 3
1.4.2 stats 报表：两个最常用的 report . . . 4
1.4.3 体积控制：一次 500GB 事故 . . . 4
1.4.4 窗口化采集：cudaProfilerApi . . . 4
1.5 ncu 命令行实战 . . . 4
1.5.1 基础采集与 section . . . 4
1.5.2 --kernel-name 必须用 regex: 前缀 . . . 5
1.5.3 --page raw 提取原始 metric . . . 5
1.5.4 权限限制与同架构代理 GPU . . . 5
1.5.5 replay 的干扰与 A/B 规则 . . . 5
1.6 核心指标字典 . . . 5
1.6.1 SpeedOfLight：SM% 与 MEM%（roofline 定位） . . . 5
1.6.2 Occupancy：理论、达到与三类限制源 . . . 6
1.6.3 寄存器与 spill：255 墙 . . . 6
1.6.4 Warp State：stall reasons，好 stall 与坏 stall . . . 6
1.6.5 Memory Workload：L1/L2/DRAM 分层归因 . . . 7
1.6.6 bank conflict 与 smem 指标 . . . 7
1.6.7 指令 mix 与 tensor pipe 利用率 . . . 7
1.7 瓶颈判断决策树 . . . 8
1.8 实战三例：占比、寄存器与 stall . . . 9
1.8.1 例一：nsys 先行——占比决定优化预算 . . . 9
1.8.2 例二：寄存器压力——255 墙、spill 与 setmaxnreg 的取舍 . . . 9
1.8.3 例三：stall 证据链——好 stall 与坏 stall 的判读 . . . 9
1.9 常见坑 . . . 10
1.10 小结 . . . 10

第二章　GPU 架构、执行模型与 Roofline 11

2.1 本章导读 . . . 11
2.2 问题与动机 . . . 11
2.3 SM 内部结构与内存层级 . . . 12
2.3.1 SM：GPU 的基本计算单元 . . . 12

2.3.2 内存层级：带宽与容量的权衡 . . . 13
2.4 数学原理：Roofline 与算术强度 . . . 14
2.4.1 算术强度（Arithmetic Intensity, AI） . . . 14
2.4.2 三个典型算子的 AI 逐项推导 . . . 14
2.4.3 Roofline 模型与 ridge point . . . 14
2.4.4 Occupancy 与延迟隐藏 . . . 16
2.5 设计与实现 . . . 16
2.6 图示 . . . 16
2.7 性能分析 . . . 17
2.8 实践经验 . . . 17
2.9 常见坑 . . . 18
2.10 面试要点速查 . . . 18
2.11 最小测试与动手实验 . . . 18

第三章 归约原语：Warp/Block Reduce 与 Dot 20
3.1 本章导读 . . . 20
3.2 问题与动机 . . . 20
3.3 数学原理 . . . 21
3.3.1 warp shuffle 与蝶形归约 . . . 21
3.3.2 Block 两级归约的步数统计 . . . 22
3.3.3 跨 block 求和与浮点非结合性 . . . 22
3.4 设计与实现 . . . 22
3.4.1 第一层：warp 内蝶形归约 . . . 22
3.4.2 第二层：block 两级归约 . . . 23
3.4.3 第三层：跨 block 全局归约与点积 . . . 24
3.5 图示 . . . 26
3.6 性能分析 . . . 26
3.7 实践经验 . . . 27
3.8 常见坑 . . . 27
3.9 面试要点速查 . . . 28
3.10 最小测试与动手实验 . . . 28

第四章 向量化访存与原子操作 29
4.1 本章导读 . . . 29
4.2 问题与动机 . . . 29
4.3 数学原理 . . . 29
4.3.1 Coalescing：sector 与事务合并 . . . 29
4.3.2 向量化：一条指令 128 bit . . . 30
4.3.3 原子操作与直方图的冲突量 . . . 30
4.4 设计与实现 . . . 31
4.4.1 relu：标量基线与 vec4 对照 . . . 31
4.4.2 elementwise add：安全范式 . . . 32
4.4.3 histogram：原子操作的应用 . . . 33
4.5 图示 . . . 33
4.6 性能分析 . . . 34
4.7 实践经验 . . . 34
4.8 常见坑 . . . 35
4.9 面试要点速查 . . . 35
4.10 最小测试与动手实验 . . . 35

第五章 Softmax 三级递进：从 naive 到 online **37**
5.1 本章导读 . . . 37
5.2 问题与动机 . . . 37
5.3 数学原理 . . . 38
5.3.1 定义与平移不变性 . . . 38
5.3.2 为什么必须减 max：e^x 的溢出界 . . . 38
5.3.3 online 递推：(m, ℓ) 双状态单遍更新 . . . 38
5.3.4 二元合并：把递推广广成可归约的运算 . . . 39
5.4 设计与实现 . . . 40
5.4.1 Level 1：naive——按定义直译 . . . 40
5.4.2 Level 2：safe——两遍扫描的工业标配 . . . 40
5.4.3 Level 3：online——(m, d) 融合归约 . . . 41
5.5 图示 . . . 43
5.6 性能分析 . . . 43
5.7 实践经验 . . . 44
5.8 常见坑 . . . 44
5.9 面试要点速查 . . . 45
5.10 最小测试与动手实验 . . . 45

第六章 LSE 与分块合并：merge_attn_states **47**
6.1 本章导读 . . . 47
6.2 问题与动机 . . . 47
6.3 数学原理 . . . 47
6.3.1 分块状态与 LSE 的无损打包 . . . 47
6.3.2 合并公式：从真值形式到 LSE 形式 . . . 48
6.3.3 空段与 $\pm\infty$ 语义 . . . 49
6.4 设计与实现 . . . 49
6.5 图示 . . . 52
6.6 性能分析 . . . 52
6.7 实践经验 . . . 52
6.8 常见坑 . . . 53
6.9 面试要点速查 . . . 53
6.10 最小测试与动手实验 . . . 54

第七章 归一化：RMSNorm 与 LayerNorm **55**
7.1 本章导读 . . . 55
7.2 问题与动机 . . . 55
7.3 数学原理 . . . 55
7.3.1 LayerNorm：均值与方差的双重归约 . . . 55
7.3.2 RMSNorm：去掉中心化的单归约形式 . . . 56
7.3.3 RMSNorm 与 LayerNorm 的关系：去均值化简 . . . 56
7.3.4 1-pass 与 2-pass：归约结构的形式化 . . . 57
7.3.5 rsqrtf：倒数平方根的硬件路径 . . . 57
7.4 设计与实现 . . . 57
7.4.1 行划分：one-row-one-block . . . 57
7.4.2 rms_norm：单归约 + 单广播 . . . 58
7.4.3 rms_norm_vec4：一次搬运四个元素 . . . 59
7.4.4 layer_norm：两次归约 + 两次同步 . . . 60
7.4.5 layer_norm_vec4：寄存器复用的两遍结构 . . . 61

7.5 图示 . . . 62
7.6 性能分析 . . . 62
7.6.1 Roofline 定位 . . . 62
7.6.2 实测数据 . . . 63
7.7 实践经验 . . . 63
7.8 常见坑 . . . 63
7.9 面试要点速查 . . . 64
7.10 最小测试与动手实验 . . . 64

第八章 RoPE 与矩阵转置（Bank Conflict 专题） 65
8.1 本章导读 . . . 65
8.2 问题与动机 . . . 65
8.3 数学原理 . . . 65
8.3.1 二维旋转与复数形式 . . . 65
8.3.2 相对位置不变性：RoPE 的核心性质 . . . 66
8.3.3 高维形式：分块对角旋转矩阵 . . . 66
8.3.4 转置的「数学」 . . . 67
8.4 设计与实现 . . . 67
8.4.1 rope：每线程一对元素 . . . 67
8.4.2 mat_transpose：矛盾的直接呈现 . . . 68
8.4.3 mat_transpose_padded：shared memory 中转 + PAD . . . 70
8.5 图示：Bank Conflict 专题 . . . 71
8.5.1 shared memory 的物理结构 . . . 71
8.5.2 本 kernel 的精确冲突分析 . . . 71
8.5.3 模拟验证 . . . 72
8.6 性能分析 . . . 72
8.7 实践经验 . . . 73
8.8 常见坑 . . . 73
8.9 面试要点速查 . . . 73
8.10 最小测试与动手实验 . . . 74

第二部分 GEMM 篇：从 CUDA Core 到 Tensor Core 75

第九章 SGEMV：memory-bound 的三种划分 76
9.1 本章导读 . . . 76
9.2 问题与动机 . . . 76
9.3 数学原理 . . . 77
9.3.1 定义与算术强度：两种计数口径 . . . 77
9.3.2 warp-per-row 的覆盖数学 . . . 77
9.3.3 float4 的覆盖与对齐 . . . 77
9.3.4 K=16：双行复用与分段蝶形归约 . . . 77
9.4 设计与实现 . . . 78
9.4.1 sgemv_k32：基础 warp-per-row . . . 78
9.4.2 sgemv_k128：float4 向量化 . . . 79
9.4.3 sgemv_k16：K=16 双行复用 . . . 80
9.5 图示 . . . 81
9.6 性能分析 . . . 81
9.7 实践经验 . . . 81
9.8 常见坑 . . . 82

9.9 面试要点速查 . . . 82
9.10 最小测试与动手实验 . . . 83

第十章 SGEMM 阶梯一：Block Tile、Vec4 与 Thread Tile 84
10.1 本章导读 . . . 84
10.2 问题与动机 . . . 84
10.3 数学原理 . . . 85
10.3.1 记号与总 FLOPs . . . 85
10.3.2 naive：AI 与 K 无关 . . . 85
10.3.3 Block Tile：AI 与 K 切片深度无关，只随输出面增长 . . . 85
10.3.4 Thread Tile：换一层计数——smem 到寄存器 . . . 85
10.3.5 Vectorize：不改字节数，改指令数与事务 . . . 85
10.3.6 roofline 定位与两级同步的正确性 . . . 85
10.4 设计与实现 . . . 86
10.4.1 五层金字塔总览 . . . 86
10.4.2 Level 1：`sgemm`，Block Tile $32 \times 32 \times 32$ . . . 86
10.4.3 Level 1+：`sgemm_vec4` 的设计注释 . . . 88
10.4.4 Level 1+：`sgemm_vec4` 的实现 . . . 89
10.5 图示 . . . 91
10.6 性能分析 . . . 91
10.7 实践经验 . . . 92
10.8 常见坑 . . . 92
10.9 面试要点速查 . . . 93
10.10 最小测试与动手实验 . . . 93

第十一章 SGEMM 阶梯二：TF32 WMMA 与 cp.async 双缓冲 95
11.1 本章导读 . . . 95
11.2 问题与动机 . . . 95
11.3 数学原理 . . . 95
11.3.1 TF32：去掉 13 个尾数位的 fp32 . . . 95
11.3.2 舍入误差界：TF32 内积到底差多少 . . . 96
11.3.3 双缓冲的时序数学 . . . 97
11.4 设计与实现 . . . 97
11.4.1 配置金字塔：从 block 到 fragment . . . 97
11.4.2 FP32 到 TF32 的原地转换 . . . 97
11.4.3 WMMA 编程模型：fragment 三件套 . . . 98
11.4.4 shared memory 布局与 padding . . . 98
11.4.5 GMEM 到 SMEM：每线程一个 float4 . . . 99
11.4.6 cp.async 双缓冲流水线 . . . 100
11.5 图示 . . . 102
11.6 性能分析 . . . 102
11.7 实践经验 . . . 103
11.8 常见坑 . . . 103
11.9 面试要点速查 . . . 103
11.10 最小测试与动手实验 . . . 104

第十二章 HGEMM：mma.sync m16n8k16、ldmatrix 与多级流水 **106**
12.1 本章导读 . . . 106
12.2 问题与动机 . . . 106
12.3 数学原理 . . . 106
12.3.1 `mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16` 逐字段 . . . 106
12.3.2 A fragment：每 lane 8 个 f16，四个象限 . . . 107
12.3.3 B fragment：每 lane 4 个 f16，按列分布 . . . 108
12.3.4 C fragment：epilogue 的出发点 . . . 108
12.3.5 `ldmatrix`：warp 集体装载指令逐字段 . . . 109
12.3.6 kStages = 3 的多级流水 . . . 109
12.4 设计与实现 . . . 110
12.4.1 Tile 金字塔与 256 线程 . . . 110
12.4.2 smem 布局与 cp.async 预载 . . . 111
12.4.3 统一循环：预取、条件加载与自适应等待 . . . 112
12.4.4 ldmatrix 装载 A（x4）与 B（x2） . . . 113
12.4.5 计算核：每 warp 每迭代 16 条 mma . . . 114
12.4.6 Epilogue：shuffle 收拢 C fragment . . . 114
12.5 性能分析 . . . 115
12.6 实践经验 . . . 116
12.7 常见坑 . . . 116
12.8 面试要点速查 . . . 117
12.9 最小测试与动手实验 . . . 117

第十三章 HGEMM：XOR Swizzle、寄存器双缓冲与 Block Swizzle **119**
13.1 本章导读 . . . 119
13.2 问题与动机 . . . 119
13.3 数学原理 . . . 120
13.3.1 bank 模型：32 bank、4 B 宽、128 B 一行 . . . 120
13.3.2 无 swizzle 的冲突推导：BK=16（32 B 行宽）为 2-way . . . 120
13.3.3 无 swizzle 的冲突推导：BK=64（128 B 行宽）为 8-way . . . 121
13.3.4 XOR swizzle：v1 的位运算与逐 bit 解释 . . . 121
13.3.5 可逆性：swizzle 是对合，写读同函数即互逆 . . . 121
13.3.6 Swizzle$\langle B, M, S\rangle$ 一般形式与 v1/v2 等价 . . . 122
13.3.7 关键勘误：BK=64 下 per-slice swizzle 只到 4-way，不是 1-way . . . 122
13.3.8 Block Swizzle：坐标重映射与 L2 footprint 推导 . . . 123
13.4 设计与实现 . . . 123
13.4.1 swizzle 函数族：v1、v2 与派发器 . . . 123
13.4.2 kernel 骨架：模板参数、block swizzle 重映射与 smem 布局 . . . 126
13.4.3 G→S：swizzle 进地址的 cp.async . . . 127
13.4.4 寄存器双缓冲与 k_step 乒乓 . . . 128
13.5 图示 . . . 131
13.6 性能分析 . . . 133
13.7 实践经验 . . . 134
13.8 常见坑 . . . 134
13.9 面试要点速查 . . . 134
13.10 最小测试与动手实验 . . . 135

第十四章 Hopper 之路：TMA、mbarrier 与 WGMMA **137**
14.1 本章导读 . . . 137
14.2 问题与动机 . . . 137
14.3 数学原理 . . . 138
14.3.1 TMA 的 box 几何与坐标 . . . 138
14.3.2 mbarrier 的双计数代数 . . . 138
14.3.3 WGMMA 的形状与寄存器分布 . . . 139
14.4 设计与实现 . . . 140
14.4.1 warpgroup：Hopper 的最小执行单元 . . . 140
14.4.2 TMA descriptor：cuTensorMapEncodeTiled 逐参数 . . . 140
14.4.3 发起搬运：cp.async.bulk.tensor.2d . . . 141
14.4.4 mbarrier 协作协议：full/empty 双屏障 . . . 142
14.4.5 WGMMA smem descriptor：64-bit 逐字段编码 . . . 144
14.4.6 wgmma.fence / commit_group / wait_group 三件套 . . . 145
14.4.7 kernel 主体：consumer 四步与写回 . . . 146
14.5 图示 . . . 148
14.6 性能分析 . . . 148
14.7 实践经验与考据说明 . . . 149
14.8 常见坑 . . . 149
14.9 面试要点速查 . . . 150
14.10 最小测试与动手实验 . . . 151

第十五章 SM120：TMA + mma.sync + Warp Specialization **152**
15.1 本章导读 . . . 152
15.2 问题与动机 . . . 152
15.3 数学原理 . . . 153
15.3.1 SM120 与 Hopper 的能力差集 . . . 153
15.3.2 producer/consumer 到达数：逐 phase 计数 . . . 153
15.3.3 setmaxnreg 池数学与满池死锁 . . . 153
15.3.4 TMA box：回指与本章坐标 . . . 155
15.4 设计与实现 . . . 155
15.4.1 kernel 签名与 tile 层级 . . . 155
15.4.2 smem 布局与 1024B 对齐：本章最重要的差异点 . . . 156
15.4.3 barrier 初始化 . . . 157
15.4.4 producer：三步循环 . . . 157
15.4.5 consumer：C0 预热与无 ___syncthreads 的主循环 . . . 158
15.4.6 swizzle<64> 与 k_step 的位置：与 ch12 的关键分歧 . . . 159
15.4.7 B 的加载与 mma 发射、epilogue . . . 159
15.5 图示 . . . 161
15.6 性能分析 . . . 161
15.7 实践经验与考据说明 . . . 162
15.8 常见坑 . . . 162
15.9 面试要点速查 . . . 164
15.10 最小测试与动手实验 . . . 164

第三部分 Attention 篇：FlashAttention 2/3 与大 head_dim **166**

第十六章 Attention 数学基础与 FlashAttention 原理 **167**
16.1 本章导读 . . . 167

16.2 问题与动机 . . . 167
16.3 数学原理 . . . 168
16.3.1 Attention 的定义与逐项含义 . . . 168
16.3.2 为什么除以 $\sqrt{d}$：点积方差推导 . . . 168
16.3.3 朴素实现：FLOPs、显存与带宽三项开销 . . . 169
16.3.4 分块计算与 online rescale：逐块推导 . . . 170
16.3.5 分块循环不变量与归纳法证明 . . . 171
16.3.6 复杂度总结与 IO 分析 . . . 172
16.3.7 FA1 → FA2 → FA3：三代演进 . . . 172
16.4 设计与实现 . . . 173
16.4.1 flash_attn.cuh 的总体结构 . . . 173
16.4.2 头注释导读：五条要点 . . . 174
16.4.3 launch 配置与 tile 参数化 . . . 174
16.4.4 执行流程：九步与本章公式的对应 . . . 175
16.5 图示 . . . 176
16.6 性能分析 . . . 176
16.7 实践经验与考据说明 . . . 177
16.8 常见坑 . . . 178
16.9 面试要点速查 . . . 178
16.10 最小测试与动手实验 . . . 179

第十七章 FA2（一）：Split-Q + MMA 多级流水 181
17.1 本章导读 . . . 181
17.2 问题与动机 . . . 181
17.3 数学原理 . . . 182
17.3.1 记号与配置 . . . 182
17.3.2 Split-Q 行划分与状态的 warp 私有化 . . . 182
17.3.3 S 的 C fragment 如何变成 $P@V$ 的 A fragment . . . 182
17.3.4 行归约为什么是 4-lane 蝶形 . . . 183
17.4 设计与实现 . . . 184
17.4.1 模板参数、线程索引与 smem 布局 . . . 185
17.4.2 寄存器工作集：状态、操作数与累加器 . . . 187
17.4.3 Step 1/2：Q 一次性预载，K 预热 kStagesK−1 级 . . . 188
17.4.4 Step 3 外循环：3a 多级流水加载 . . . 189
17.4.5 3b：$S = QK^\top$ 的 ldmatrix + mma . . . 191
17.4.6 3c：online softmax——寄存器内的三遍扫描 . . . 192
17.4.7 3d：$P@V$——P 不出寄存器 . . . 194
17.4.8 3e：online rescale 与状态推进 . . . 196
17.4.9 Step 4/5：最终归一化与 epilogue 写回 . . . 197
17.5 图示 . . . 199
17.6 性能分析 . . . 199
17.7 实践经验与考据说明 . . . 200
17.8 常见坑 . . . 201
17.9 面试要点速查 . . . 201
17.10 最小测试与动手实验 . . . 202

第十八章 FA2（二）：TMA + Warp Specialization **203**
18.1 本章导读 . . . 203
18.2 问题与动机 . . . 203
18.3 数学原理 . . . 204
18.3.1 split-Q 网格与循环不变量 . . . 204
18.3.2 mbarrier arrive_count 协议：257 的代数 . . . 204
18.3.3 双流水的重叠窗口 . . . 206
18.3.4 smem 占用与 stage 上限 . . . 206
18.4 设计与实现 . . . 206
18.4.1 总体结构：从「warp 各自流水」到「角色分化」 . . . 206
18.4.2 模板参数与 tile 派生 . . . 207
18.4.3 smem 布局：1024B 对齐与 chunk-major . . . 207
18.4.4 barrier 初始化与线程编队 . . . 210
18.4.5 producer：P0/P1/P1b/P2 四个阶段 . . . 210
18.4.6 consumer：C0 预热 + 五个阶段主循环 . . . 212
18.4.7 epilogue：终归一化与集体写回 . . . 215
18.5 图示 . . . 217
18.6 性能分析 . . . 217
18.7 实践经验与考据说明 . . . 218
18.8 常见坑 . . . 219
18.9 面试要点速查 . . . 219
18.10 最小测试与动手实验 . . . 220

第十九章 FA3：双 Consumer Warpgroup **222**
19.1 本章导读 . . . 222
19.2 问题与动机 . . . 222
19.3 数学原理 . . . 223
19.3.1 split-KV 双流划分与合并不变量 . . . 223
19.3.2 $T_c = 1$ 的退化：$-\infty$ 参与的 merge . . . 223
19.3.3 每 WG 独立 stage：屏障计数从 257 缩到 129 . . . 223
19.3.4 setmaxnreg 池数学：三 warpgroup 的寄存器预算 . . . 224
19.4 设计与实现 . . . 225
19.4.1 总体结构：三分线程与奇偶路由 . . . 225
19.4.2 producer：单线程 TMA 与 cid 路由 . . . 226
19.4.3 consumer 主循环：五个阶段与两个 early release . . . 227
19.4.4 merge epilogue：scratch 复用与两步同步 . . . 228
19.4.5 与第 十五 章 SM120 WS 的对照：同思想，不同硬件 . . . 230
19.5 图示 . . . 231
19.6 性能分析 . . . 231
19.7 实践经验与考据说明 . . . 232
19.8 常见坑 . . . 233
19.9 面试要点速查 . . . 234
19.10 最小测试与动手实验 . . . 234

第二十章 FFPA Split-D：大 head_dim 分块注意力 **236**
20.1 本章导读 . . . 236
20.2 问题与动机 . . . 236
20.3 数学原理 . . . 237
20.3.1 记号 . . . 237

20.3.2 QK 的 chunk 分解：精确、无权重 . . . 237
20.3.3 分块 online softmax 的两条不变量 . . . 238
20.3.4 把 O 拆成 C 个 chunk 分量 . . . 238
20.3.5 与 Split-KV 的合并方式对照 . . . 239
20.3.6 chunk 数、stage 数与相位 . . . 239
20.4 设计与实现 . . . 240
20.4.1 traits：双 TiledMma 与 SW128 chunk 布局 . . . 240
20.4.2 smem 池：三段基址与 chunk 偏移 . . . 241
20.4.3 cp.async 版：128 线程全员搬 + 双流水 . . . 241
20.4.4 TMA+WS 版：单线程 TMA 与 (stage, phase) 计数 . . . 244
20.4.5 两实现对照 . . . 247
20.4.6 O 累加器的寄存器占用 . . . 247
20.5 图示 . . . 248
20.6 性能分析 . . . 248
20.7 实践经验与考据说明 . . . 249
20.8 常见坑 . . . 250
20.9 面试要点速查 . . . 250
20.10 最小测试与动手实验 . . . 251

第四部分 CuTe 篇：布局代数与现代化 Kernel 工程 253

导读：CuTe 布局表示与代数（白皮书中文译注） 254
1 引言与动机 . . . 254
1.1 相关工作 . . . 255
1.2 规范化循环与循环变换 . . . 256
1.3 张量与折叠 . . . 257
2 布局表示（Layout Representation） . . . 259
2.1 元组与层次元组（Tuple 与 HTuple） . . . 259
2.2 形状（Shape） . . . 260
2.3 步幅（Stride） . . . 262
2.4 布局 . . . 263
2.5 张量（Tensor） . . . 266
2.6 应用（Applications） . . . 269
3 布局代数（Layout Algebra） . . . 271
3.1 拼接（Concatenate） . . . 271
3.2 合并（Coalesce） . . . 271
3.3 复合（Composition） . . . 272
3.4 逆（Inverse） . . . 277
3.5 补 . . . 280
4 结论 . . . 284
致谢 . . . 284

第二十一章 CuTe（一）：Layout 基础与代数 285
21.1 本章导读 . . . 285
21.2 问题与动机 . . . 285
21.3 数学原理 . . . 286
21.3.1 Layout 是一个函数 . . . 286
21.3.2 五个运算，五类实际问题 . . . 287
21.4 设计与实现 . . . 288

21.4.1 先看「类型即布局」：hgemm.cuh 的 CuTe 类型清单 . . . 288
21.4.2 Compose：$R(c) = A(B(c))$ . . . 289
21.4.3 Inverse：右逆与左逆 . . . 289
21.4.4 Product：乘积 . . . 290
21.4.5 Divide：tile 的数学 . . . 291
21.4.6 Mode 操作：投影、合并、展平、化简 . . . 292
21.4.7 运算一览与 hgemm 的对应位置 . . . 292
21.4.8 Compose & Inverse 实战：FA2 的 P 矩阵布局转换 . . . 292
21.5 图示 . . . 293
21.6 性能分析 . . . 293
21.7 常见坑 . . . 298
21.8 面试要点速查 . . . 298
21.9 最小测试与动手实验 . . . 299

第二十二章 CuTe（二）：Tensor 与 TiledCopy 301
22.1 本章导读 . . . 301
22.2 问题与动机 . . . 301
22.3 数学原理 . . . 302
22.3.1 Tensor：engine 与 layout . . . 302
22.3.2 小例子：4 个线程搬一块 $(4, 4)$ . . . 302
22.3.3 (thread, value) 网格：TV-layout . . . 303
22.3.4 raked_product：从 Thr/Val 到 MN . . . 304
22.3.5 守恒律与重复因子 . . . 304
22.3.6 partition 的数学 . . . 305
22.3.7 retile：零拷贝重标记与 upcast . . . 306
22.4 设计与实现 . . . 306
22.4.1 kernel 骨架：Tensor 与 local_tile . . . 306
22.4.2 三种 TiledCopy 的构造方式 . . . 307
22.4.3 G→S：显式 Thr/Val 与两级切片 . . . 307
22.4.4 S→R：从 TiledMMA 推导，并用 retile_D 对齐目的 . . . 308
22.4.5 epilogue：R→S→G 与 group_modes . . . 309
22.5 图示 . . . 310
22.6 性能分析 . . . 310
22.7 常见坑 . . . 311
22.8 面试要点速查 . . . 312
22.9 最小测试与动手实验 . . . 312

第二十三章 CuTe（三）：TiledMMA 与 fragment 314
23.1 本章导读 . . . 314
23.2 问题与动机 . . . 314
23.3 数学原理 . . . 315
23.3.1 MMA 原子：一条指令的 thread-data 映射 . . . 315
23.3.2 TiledMMA：原子、排布、tile 三个参数 . . . 316
23.3.3 q 的推导链：从 tile 到 fragment . . . 316
23.3.4 守恒与 tile 循环 . . . 317
23.3.5 get_slice：线程坐标的三层解构 . . . 318
23.4 设计与实现 . . . 318
23.4.1 FFPA traits：两个 TiledMMA 的完整推导 . . . 318
23.4.2 fragment 的使用：三行代码的完整语义 . . . 319

23.4.3 QK→PV：fragment 的原地复用 . . . 320
23.5 图示 . . . 321
23.6 性能分析 . . . 321
23.7 常见坑 . . . 322
23.8 面试要点速查 . . . 323
23.9 最小测试与动手实验 . . . 324

第二十四章 CuTe（四）：Swizzle 与 TMA 326
24.1 本章导读 . . . 326
24.2 问题与动机 . . . 326
24.3 数学原理 . . . 327
24.3.1 现象：bank conflict 与两种解法 . . . 327
24.3.2 Swizzle 的位模型 . . . 327
24.3.3 从位模型到「chunk XOR 行号」 . . . 328
24.3.4 composition：Swizzle 与基础布局的合成 . . . 329
24.3.5 位置无关与位置相关：最容易出错的一处 . . . 329
24.3.6 tile_to_shape：从原子到多级 smem 布局 . . . 330
24.4 设计与实现 . . . 331
24.4.1 hgemm 的三个 smem 布局别名 . . . 331
24.4.2 为什么不能直接套 `GMMA::Layout_K_SW128_Atom` . . . 331
24.4.3 TMA 的两条构造路径 . . . 332
24.4.4 生产者与消费者：两套地址算术的对齐 . . . 333
24.4.5 对齐与 phase . . . 333
24.5 图示 . . . 334
24.6 性能分析 . . . 334
24.7 常见坑 . . . 335
24.8 面试要点速查 . . . 335
24.9 最小测试与动手实验 . . . 336

第二十五章 CuTe 应用（一）：HGEMM 338
25.1 本章导读 . . . 338
25.2 问题与动机 . . . 338
25.3 数学原理 . . . 339
25.3.1 问题定义：TN 约定与第一个 Tensor . . . 339
25.3.2 Tile 金字塔：从 CTA tile 到 MMA atom . . . 339
25.3.3 MMA atom 与 EURepeat . . . 340
25.3.4 三件事：数据在哪、谁算哪块、谁搬哪块 . . . 341
25.4 设计与实现 . . . 341
25.4.1 总览：kernel 签名与三步路线 . . . 341
25.4.2 第一步：切块、分区、`cute::gemm` . . . 342
25.4.3 第二步：smem 与流水线——让搬运和计算重叠 . . . 344
25.4.4 第三步之一：swizzle 布局与 `ldmatrix` 装载 . . . 346
25.4.5 第三步之二：寄存器占用与 epilogue 宽写 . . . 347
25.4.6 launch wrapper：类型即配置 . . . 348
25.5 图示 . . . 350
25.6 性能分析 . . . 350
25.7 常见坑 . . . 352
25.8 面试要点速查 . . . 352
25.9 最小测试与动手实验 . . . 353

第二十六章 CuTe 应用（二）：FlashAttention 三实现对照 **354**
26.1 本章导读 . . . 354
26.2 问题与动机 . . . 354
26.3 数学原理 . . . 356
26.3.1 两个 traits：编译期布局的「配置表」 . . . 356
26.3.2 TiledMMA 的类型推导链：从 atom 到 EURepeat . . . 356
26.3.3 rowcol 变换：把累加器重排成行 × 列 . . . 357
26.3.4 P 矩阵的布局变换：从 C-layout 到 A-layout . . . 358
26.3.5 两套流水的语义：cp.async 的组栈与 TMA 的事务计数 . . . 360
26.4 设计与实现 . . . 361
26.4.1 共享层：五个工具函数 . . . 361
26.4.2 实现 1：cp.async + 统一流水（256 线程） . . . 362
26.4.3 实现 2：TMA + Warp Specialization（384 线程） . . . 364
26.4.4 实现 3：FA3 式双 consumer（384 线程） . . . 366
26.4.5 实现 4：42 行的 TMA 自检 . . . 367
26.4.6 结构对照 . . . 368
26.5 图示 . . . 368
26.6 性能分析 . . . 369
26.7 常见坑 . . . 369
26.8 面试要点速查 . . . 370
26.9 最小测试与动手实验 . . . 370

第二十七章 CuTe 应用（三）：FFPA Split-D 的类型代数 **373**
27.1 本章导读 . . . 373
27.2 问题与动机 . . . 373
27.3 数学原理：类型与布局代数 . . . 374
27.3.1 两个 TiledMma 的推导链 . . . 374
27.3.2 fragment 的元素计数：两个恒等式 . . . 375
27.3.3 $S \to P$：一次零拷贝的布局重解释 . . . 376
27.3.4 D-chunk 循环的布局语义 . . . 377
27.4 设计与实现：逐相位读 kernel . . . 378
27.4.1 traits 与 chunk 常量 . . . 378
27.4.2 G2S `TiledCopy`：与 MMA 划分对齐 . . . 379
27.4.3 V 的转置视图与 fragment 预计算 . . . 380
27.4.4 寄存器占用：$32 \times C$ 与两个行向量 . . . 381
27.4.5 Phase 1：QK 的 chunk 流水 . . . 382
27.4.6 Phase 2：softmax 与 P 的生成 . . . 383
27.4.7 Phase 3：PV 的 chunk 循环与两个 TiledMma 的交接 . . . 384
27.4.8 Phase 4：归一化与写回 . . . 385
27.5 图示：双 TiledMma 的数据流 . . . 386
27.6 性能分析 . . . 386
27.7 常见坑 . . . 386
27.8 面试要点速查 . . . 387
27.9 最小测试与动手实验 . . . 387

第二十八章 CuTe 应用（四）：SM120 持久化 FlashAttention——超越 cuDNN **389**
28.1 本章导读 . . . 389
28.2 问题与动机：为什么专用 kernel 能赢厂商库 . . . 389
28.3 数学原理 . . . 390

28.3.1 log2 域 online softmax 回顾 . . . 390
28.3.2 FA-4 条件重缩放 . . . 390
28.3.3 softmax scale·$\log_2 e$ 融合 . . . 390
28.4 设计与实现 . . . 391
28.4.1 总体架构：WS 双 warpgroup 与寄存器池数学 . . . 391
28.4.2 smem 布局与 swizzle . . . 392
28.4.3 TMA 装载协议：descriptor、V-first 预取与 mbarrier 相位 . . . 393
28.4.4 Q 寄存器持久化与 `gemm_rs`：TiledMMA/TiledCopy 组合 . . . 395
28.4.5 persistent CTA：wave 量化尾损与线性分配 . . . 397
28.4.6 Epilogue：R→S(STSM)→TMA store 与尾部 R→G . . . 398
28.5 性能分析 . . . 399
28.6 专论：SM120 上 `setmaxnreg` 的生效条件 . . . 399
28.7 勘误与考据 . . . 400
28.8 常见坑 . . . 401
28.9 面试要点速查 . . . 401
28.10 最小测试与动手实验 . . . 401

第二十九章 CuTe 应用（五）：SM120 大 head_dim Split-D——non-WS 全员 MMA 流水 403
29.1 本章导读 . . . 403
29.2 问题与动机：WS 教学版慢在哪里 . . . 403
29.3 协议设计 . . . 404
29.3.1 tile 几何与 smem 预算：chunk 解耦 . . . 404
29.3.2 M8N1 TiledMma：warp 持整行，softmax 归约零跨 warp . . . 405
29.3.3 non-WS 执行模型：全员 MMA 与内联 TMA . . . 406
29.3.4 Phase 1：QK split-D 累加与领先预取 . . . 408
29.3.5 跨 tile 预取：barrier 集不相交的收益 . . . 409
29.3.6 Phase 2：边界 mask、lazy rescale 与 P 的零拷贝 . . . 410
29.3.7 Phase 3：PV split-D 与条件重缩放的落点 . . . 411
29.3.8 Phase 4：STSM 批量暂存与 TMA store bulk group . . . 412
29.3.9 launcher：BHND 压平与四方向 descriptor . . . 414
29.4 性能分析 . . . 415
29.4.1 主场景与 stages 解耦实验 . . . 415
29.4.2 机理：寄存器 spill 决定 stages 的最优值 . . . 415
29.4.3 与 WS 版的逐项对照收尾 . . . 416
29.5 常见坑 . . . 417
29.6 面试要点速查 . . . 417
29.7 最小测试与动手实验 . . . 418

第五部分 FP8/FP4 HGEMM 篇：量化矩阵乘的数学与 Kernel 工程 419

第三十章 FP8 GEMM（一）：e4m3 格式、per-row/per-block 量化与前处理 Kernel 420
30.1 本章导读 . . . 420
30.2 问题与动机 . . . 420
30.2.1 为什么 GEMM 量化比 attention 量化「容易」 . . . 420
30.2.2 约束与 API 约定 . . . 421
30.3 e4m3 回顾：GEMM 视角 . . . 421
30.4 量化数学与误差分析 . . . 421
30.4.1 量化算子与 scale 折叠恒等式 . . . 421
30.4.2 四种粒度组合 . . . 422

30.4.3 误差模型：能差到多少 . . . 422
30.4.4 amax 零块与数值安全 . . . 423
30.5 前处理 Kernel 工程 . . . 423
30.5.1 A 的 per-row 量化：warp-per-row . . . 424
30.5.2 A 的 per-block 量化：block 二级归约 . . . 425
30.5.3 B 的转置量化：双朝向 smem staging . . . 426
30.5.4 e4m3x2 打包与饱和语义 . . . 428
30.6 精度实测 . . . 428
30.7 小结 . . . 428
延伸阅读 . . . 428

第三十一章 FP8 GEMM（二）：CuTe 主 Kernel、在线反量化与 cuBLAS 对比 429
31.1 本章导读 . . . 429
31.2 总体架构 . . . 429
31.3 Traits：tile 几何、SW128 与 M8N1 TiledMma . . . 430
31.4 non-WS 主 kernel：全员 MMA 与内联 TMA . . . 431
31.5 Epilogue：在线反量化的四步 . . . 433
31.6 WS 进阶变体与 setmaxnreg . . . 434
31.7 C++ API 与 bench 接线 . . . 435
31.8 实测：精度与性能 vs cuBLAS BF16 . . . 437
31.8.1 对照基线剖析：cuBLAS BF16 为什么只有 163 T . . . 438
31.8.2 量化开销 . . . 439
31.8.3 权重 B 离线量化：部署形态 . . . 439
31.8.4 tile 几何与流水深度：128^3 并非最优 . . . 440
31.8.5 最快配置的 NCU 读数：为什么是它 . . . 442
31.8.6 精度判定 . . . 443
31.9 小结 . . . 443
延伸阅读 . . . 443

第三十二章 FP4 GEMM（一）：NVFP4 线格式、两级量化与误差模型 444
32.1 本章导读 . . . 444
32.2 问题与动机 . . . 444
32.2.1 e2m1 的格：每个 2 的幂区间只有两个码点 . . . 444
32.2.2 单级 per-group scale 的两个失效模式 . . . 445
32.2.3 两级折叠恒等式 . . . 445
32.3 NVFP4 线格式 . . . 446
32.3.1 e2m1 与 ue4m3 的位域 . . . 446
32.3.2 硬件转换指令：平局取偶与 satfinite . . . 446
32.3.3 SF 的 gmem 布局：512 B 原子 . . . 447
32.4 量化前处理 Kernel . . . 448
32.4.1 常数与模式枚举 . . . 448
32.4.2 A 侧：两趟 pass . . . 448
32.4.3 B 侧：列 amax 与转置量化分成两个 kernel . . . 450
32.4.4 B 侧离线量化：部署的真实形态 . . . 452
32.5 误差模型：从量化步长到 14.4% . . . 453
32.5.1 逐格累加：把 8 个格的误差贡献加起来 . . . 453
32.5.2 三项修正：max 归一、SF 舍入、双侧求和 . . . 454
32.5.3 level-1 真正解决的问题 . . . 454
32.6 精度实测 . . . 455

32.7 小结 . . . 455
延伸阅读 . . . 455

第三十三章 FP4 GEMM（二）：blockscaled MMA、SF 流水与 cuBLAS 对比 456
33.1 本章导读 . . . 456
33.2 总体架构 . . . 456
33.3 指令与 Traits . . . 457
33.3.1 block scale 形式的 `mma.sync` . . . 457
33.3.2 Traits：两个被固定的参数 . . . 458
33.4 SF 的 smem 原子与四路 TMA . . . 459
33.4.1 smem 原子的四段式子 . . . 459
33.4.2 一条可验证的结论：smem 原子与 gmem 原子逐字节同序 . . . 460
33.4.3 TMA box = 一个 SF 原子：两个固定值的来源 . . . 460
33.5 non-WS 主 kernel：四路装载与 zip 张量 . . . 461
33.6 Epilogue：只剩一项乘法 . . . 462
33.7 WS 变体与一个 FP8 里不存在的死锁 . . . 463
33.7.1 死锁与修复：empty 必须由 consumer 侧预置 . . . 463
33.7.2 命名屏障 . . . 464
33.8 C++ API 与 bench 接线 . . . 464
33.9 实测：精度与性能 vs cuBLAS BF16 . . . 466
33.9.1 对照基线：cuBLAS BF16 为什么还是 165 T . . . 467
33.9.2 精度 . . . 467
33.9.3 量化开销，以及一次重构 . . . 468
33.9.4 tile 几何与流水深度：128^3 不是最优，128×256 才是 . . . 468
33.10 小结 . . . 469
延伸阅读 . . . 470

第六部分 FP8/FP4 Attention 篇：量化注意力的数学与 Kernel 工程 471

第三十四章 量化注意力的数学基础 472
34.1 本章导读 . . . 472
34.2 问题与动机 . . . 472
34.2.1 硬件的精度阶梯 . . . 472
34.2.2 attention 的三个量化点与「P 无法离线」困难 . . . 473
34.2.3 精度挑战：为什么不能直接量化 . . . 473
34.3 数学原理 . . . 473
34.3.1 量化算子与记号 . . . 473
34.3.2 数值格式：从均匀到指数 . . . 473
34.3.3 量化粒度谱系 . . . 474
34.3.4 attention 的 scale 流向总览 . . . 475
34.3.5 SageAttention 谱系：1 → 2 → 2++ → 3 . . . 476
34.3.6 FA-4 lazy rescale 与量化域的交互 . . . 477
34.3.7 ESS 误差模型：causal 早行的绝对误差放大 . . . 478
34.3.8 per-stage 误差分解与工程结论 . . . 478
34.4 设计与实现 . . . 479
34.4.1 P 量化的三步协议 . . . 479
34.4.2 两种 p_{scale} 模式的发射路径 . . . 479
34.4.3 log2 域的 lazy rescale . . . 480
34.5 图示 . . . 480

34.6 性能分析 . . . 482
34.7 实践经验 . . . 482
34.8 常见坑 . . . 482
34.9 面试要点速查 . . . 482
34.10 最小测试与动手实验 . . . 483

第三十五章 FP8（一）：量化前处理链 484
35.1 本章导读 . . . 484
35.2 问题与动机 . . . 484
35.2.1 前处理链在端到端里的位置 . . . 484
35.2.2 一次 forward 的 aux kernel 清单 . . . 484
35.2.3 三条「不变变换」的设计哲学 . . . 485
35.3 数学原理 . . . 485
35.3.1 smooth-K：列均值平移与 LSE 修正 . . . 485
35.3.2 smooth-V：行和不变量与均值剥离 . . . 485
35.3.3 Hadamard 旋转：正交不变量与 outlier 扩散 . . . 486
35.3.4 量化粒度谱系：与 C-fragment 对齐 . . . 486
35.3.5 V 的预转置与列置换 . . . 487
35.3.6 NHD 零拷贝寻址 . . . 488
35.4 设计与实现 . . . 488
35.4.1 前处理编排：一个函数，三个家族共享 . . . 488
35.4.2 两阶段确定性列均值 . . . 488
35.4.3 块级 QK 量化：寄存器驻留 amax . . . 489
35.4.4 per-thread 量化：面向 fragment 的 scale 布局 . . . 490
35.4.5 $V^\top$ 转置量化：smem staging . . . 490
35.4.6 Hadamard butterfly kernel . . . 491
35.5 图示 . . . 492
35.6 性能分析 . . . 494
35.6.1 带宽需求：为什么 aux 链必须达到带宽峰值 . . . 494
35.6.2 每个 kernel 的实测要点 . . . 494
35.6.3 粒度的性能代价与 knob 实测 . . . 494
35.7 实践经验 . . . 494
35.8 常见坑 . . . 494
35.9 面试要点速查 . . . 495
35.10 最小测试与动手实验 . . . 495

第三十六章 FP8（二）：persist-D 主 Kernel 与 Scale 折叠代数 497
36.1 本章导读 . . . 497
36.2 问题与动机 . . . 497
36.2.1 主 kernel 的输入约定 . . . 497
36.2.2 为什么 scale 消去是核心矛盾 . . . 497
36.2.3 persist-D 的适用域与家族分工 . . . 498
36.3 数学原理 . . . 498
36.3.1 QK 反量化折进 log2 域 softmax . . . 498
36.3.2 P 量化三步协议与 v_s 精确消去 . . . 498
36.3.3 lazy rescale 的溢出约束与 fp8 的关闭 . . . 498
36.3.4 f16 PV 累加器的域约束 . . . 499
36.3.5 归约轴置换不变性与 reorg-free 打包 . . . 499
36.4 设计与实现 . . . 499

36.4.1 traits：把配置变成类型 . . . 499
36.4.2 WS 骨架与 smem 复用 . . . 500
36.4.3 主循环：五个 Phase 的数据通路 . . . 501
36.4.4 reorg-free 打包的布局推导 . . . 501
36.5 图示 . . . 502
36.6 性能分析 . . . 504
36.6.1 本机实测（RTX PRO 5000，B1 H32 N8192，bench 默认配置） . . . 504
36.6.2 微优化上限：证伪清单 . . . 504
36.6.3 带宽与算术的边界 . . . 504
36.7 实践经验 . . . 505
36.7.1 案例：D=192 正确性 bug 的定位（定位实录） . . . 505
36.7.2 考据：sm_89 的 cp.async 变体 . . . 505
36.8 常见坑 . . . 506
36.9 面试要点速查 . . . 506
36.10 最小测试与动手实验 . . . 506

第三十七章 FP8（三）：split-D 与 M4N2——大 D 的两个瓶颈 508
37.1 本章导读 . . . 508
37.2 问题与动机 . . . 508
37.2.1 两个瓶颈的数量级 . . . 508
37.2.2 split-D 与 M4N2 各解决一个瓶颈 . . . 509
37.2.3 causal 精度的已知边界 . . . 509
37.3 数学原理 . . . 509
37.3.1 split-D 分块恒等式：精确，不是近似 . . . 509
37.3.2 M4N2 的 N-warp 划分与 softmax 合并协议 . . . 510
37.3.3 寄存器压力模型：为什么只有减 M_w 有效 . . . 510
37.4 设计与实现 . . . 511
37.4.1 traits：两份预算表 . . . 511
37.4.2 M8N1 主循环：chunk 流水的相位规则 . . . 512
37.4.3 M4N2 的三项新代价 . . . 512
37.4.4 证伪记录与默认值 . . . 513
37.5 图示 . . . 513
37.6 性能分析 . . . 514
37.6.1 本机实测（RTX PRO 5000，B1 H32 N8192，bench 默认配置） . . . 514
37.6.2 交叉点与 M4N2 的引用数据（本机无 $D \geq 768$ 编译实例） . . . 514
37.6.3 roofline 视角 . . . 515
37.7 实践经验 . . . 515
37.7.1 案例：causal allclose 临界失败的定位与定性 . . . 515
37.7.2 什么时候不该学本章 . . . 515
37.8 常见坑 . . . 515
37.9 面试要点速查 . . . 516
37.10 最小测试与动手实验 . . . 516
延伸阅读 . . . 516

第三十八章 FP4（一）：NVFP4 格式与量化链 517
38.1 本章导读 . . . 517
38.2 问题与动机 . . . 517
38.3 数学原理 . . . 518
38.3.1 NVFP4：e2m1 数据 + ue4m3 块尺度 . . . 518

38.3.2 smooth-Q/K 强制与 delta_s 的 rank-1 恒等式 . . . 518
38.3.3 smooth-V 的恒等式 . . . 519
38.3.4 pad 零贡献与 hadamard 的线性性 . . . 519
38.4 设计与实现 . . . 519
38.4.1 Q/K 量化 kernel：4 线程一 token . . . 519
38.4.2 $V^\top$ 量化：smem 转置 + SF atom 布局 . . . 520
38.4.3 fused 单 launch（$D \le 128$）与 fused WHT . . . 520
38.4.4 delta_s：wmma 版与 CuTe/TMA 版 . . . 521
38.4.5 kv_perm32 列置换与 perm-aware masking . . . 521
38.4.6 subbyte 寻址的陷阱 . . . 522
38.5 图示 . . . 522
38.6 性能分析 . . . 524
38.6.1 量化链本身的收益 . . . 524
38.6.2 e2e：fp4 persist-D 的 D 扫描 . . . 524
38.7 实践经验 . . . 524
38.8 常见坑 . . . 525
38.9 面试要点速查 . . . 525
38.10 最小测试与动手实验 . . . 525
延伸阅读 . . . 526

第三十九章 FP4（二）：persist-D 主 Kernel 与两级 P 量化 527
39.1 本章导读 . . . 527
39.2 问题与动机 . . . 527
39.3 数学原理 . . . 527
39.3.1 两级 P 量化：SA3 §3.2 的方案 . . . 527
39.3.2 online softmax 下的退化：第一级变成常数 . . . 528
39.3.3 归一化也折进 exp2 . . . 528
39.3.4 lazy rescale 与 lse 复合公式 . . . 528
39.3.5 MXFP8 PV 变体的数值约定 . . . 528
39.4 设计与实现 . . . 528
39.4.1 骨架复用与 grid 调度约定 . . . 528
39.4.2 Q 的寄存器驻留与一个典型 bug . . . 529
39.4.3 主循环五步 . . . 529
39.4.4 epilogue：finalize、加回与批量 staging . . . 530
39.5 图示 . . . 530
39.6 性能分析 . . . 531
39.6.1 本机 e2e（fp4 vs SDPA） . . . 531
39.6.2 fp4 对 fp8 的相对收益 . . . 531
39.6.3 已落地与已证伪的微优化 . . . 531
39.7 实践经验 . . . 532
39.8 常见坑 . . . 532
39.9 面试要点速查 . . . 532
39.10 最小测试与动手实验 . . . 533
延伸阅读 . . . 533

第四十章 FP8/FP4 性能实战：Bench、精度与生态 534
40.1 本章导读 . . . 534
40.2 问题与动机 . . . 534
40.3 数学原理 . . . 535

40.3.1 speedup 与有效 TFLOPS . . . 535
40.3.2 端到端摊销 . . . 536
40.3.3 精度指标体系 . . . 536
40.4 设计与实现 . . . 536
40.4.1 两套 bench，两个层次 . . . 536
40.4.2 测量规则的代码化 . . . 537
40.4.3 CLI knob 全集 . . . 537
40.5 实测数据总览 . . . 537
40.6 性能分析 . . . 539
40.6.1 fp8 矩阵：D 的三段形态 . . . 539
40.6.2 fp4 矩阵：D≥128 稳定 2x 级 . . . 539
40.6.3 knob 矩阵：数值通路中性，粒度有代价 . . . 539
40.6.4 竞品对照 . . . 540
40.7 实践经验 . . . 540
40.8 常见坑 . . . 540
40.9 面试要点速查 . . . 541
40.10 最小测试与动手实验 . . . 541

第七部分 附录 543

附录 A 基础设施工具箱：common.cuh 解析 544
A.1 这个文件在全书的角色 . . . 544
A.2 搬运原语：向量化宏与 cp.async . . . 545
A.3 Tensor Core 入口：ldmatrix 与 HMMA 宏 . . . 545
A.4 smem swizzle 家族 . . . 545
A.5 TMA/mbarrier helpers 与 setmaxnreg . . . 546
A.6 WGMMA 家族与 host 侧 TensorMap . . . 547
A.7 写一个新教学 kernel 的最小 checklist . . . 547

附录 B 性能数据汇总 549
B.1 口径与标尺 . . . 549
B.2 全书核心演进主线 . . . 549
B.3 Part I：带宽型算子明细 . . . 550
B.4 Part II：GEMM 阶梯与 bank conflict . . . 550
B.5 Part III：Attention 演进 . . . 551
B.6 Part IV：CuTe 对照口径 . . . 551
B.7 Part V：FP8 GEMM 明细 . . . 551
B.8 Part VI：FP8/FP4 量化 attention 明细 . . . 552
B.9 四条性能主线 . . . 553

附录 C 构建、运行与最小测试指南 555
C.1 两套构建入口的分工 . . . 555
C.2 ffpa-attn（Part VI 源码仓）安装与 bench . . . 555
C.3 book/tests/build_tests.sh 用法 . . . 556
C.4 公共对拍头 common_test.h . . . 556
C.5 -arch 目标选择 . . . 556
C.6 常见编译与运行错误速查 . . . 557
C.7 全书 218 个测试：运行与期望输出 . . . 557

附录 D 源码索引 558

附录 E 参考文献与延伸阅读 **560**
E.1 官方文档与仓库 . . . 560
E.2 论文 . . . 560
E.3 知乎系列（按主题） . . . 561
E.3.1 GPU 架构与指令集（第 1 章） . . . 561
E.3.2 基础算子优化（第 2–9 章） . . . 561
E.3.3 Tensor Core 与 MMA 编程（第 10–11 章） . . . 562
E.3.4 Swizzle 与 bank conflict（第 12、23 章） . . . 562
E.3.5 异步搬运：cp.async / mbarrier / TMA / WGMMA（第 10、13、14、17 章） . . . 563
E.3.6 CuTe：Layout 与代数（第 20 章） . . . 563
E.3.7 CuTe：TiledCopy 与 TiledMMA（第 21、22 章） . . . 563
E.3.8 CuTe GEMM 与流水线（第 24、25 章） . . . 564
E.3.9 FlashAttention 与 FFPA（第 15–19、25、26 章） . . . 564

第一部分

基础篇：编程模型、访存与归约

第一章　性能分析先于优化：nsys 与 ncu

1.1　本章导读

在动手写第一个 kernel 之前，本书先约定一条工程规则：**profile 永远都是最重要的**。优化 GPU kernel 的正确循环不是「改了跑一下看快没快」，而是「先测量、再假设、改一处、再测量」。凭直觉的优化，快了不知道为什么快，慢了也找不到原因。这一章专门讲两件测量工具：**Nsight Systems**（nsys，全局时间线）与 **Nsight Compute**（ncu，单 kernel 计数器），从原理、命令行用法到核心指标的含义都会逐一拆开，并给出一张「症状 → 证据 → 结论 → 对策」的瓶颈判断决策树。

本章不含 kernel（与 ch01 及 Part IV 的原理章 ch20–22 一样，是纯理论/工具章）。没有新代码可讲，但有大量 **实战教训**，全部来自真实的 CUDA kernel 开发记录，涉及寄存器压力、spill、warp stall、Tensor Core 饱和度、L1/L2 命中率、bank conflict，每一个指标都配一个真实的「指标驱动优化」或「指标否决优化」案例。后续所有章节的「性能分析」节都默认读者已掌握本章的工具与词汇表。

前置：无（与 ch01 互为入口，roofline 词汇见 ch01）。**源码区间**：无（工具方法论章）。**编译宏与架构限定**：无。

1.2　问题与动机：不 profile 就优化的代价

先看三个真实的开发反例，说明「不 profile 就优化」会错在哪里。

反例一：感觉是装载瓶颈，其实是依赖链。一个 sm_89 上的 fp8 attention kernel（persist-D 结构）对标业界最快实现慢 28%（909.4μs vs 710μs）。直觉方案是「加深装载流水、减指令、缩 tile」，全部实测无效；NCU 的 stall 剖析结果直接给出结论：`math_pipe_throttle` 1.32 主导、`wait` 0.82、`long_scoreboard` 仅 0.04，**装载根本不是瓶颈**：瓶颈是单 CTA 内 softmax→PV 的串行依赖链，唯一有效的改动是拆链（2 CTA/SM），实测 −5%。

反例二：感觉主 kernel 慢，其实差距在别处。e2e 落后时先对主 kernel 跑 ncu，做完整套指标分析才发现主 kernel 已占 e2e 时间 86–92%、前处理链已优于对照，一次 nsys 的时间线本可以三分钟得出同样结论，避免在错误的 kernel 上浪费半天 ncu 分析。

反例三：感觉融合更快，其实更慢。把 softmax 的 bias 注入从「分离 kernel」改成「单 pass 融合 + smem tile」，直觉上少一次读写应该更快，实测反而慢 4.2ms：NCU 显示融合把 LDS（shared load）拉进了 consumer 关键路径，`sleeping` stall 从 3.5% 升到 22.3%。指标否决了这次「优化」。

三个反例的共同教训：GPU 上的性能瓶颈很少出现在直觉指向的位置。成熟的优化工作流会把它固化成一条硬规则：**没有 NCU 数据，不改 kernel**；每条优化结论必须引用具体 metric 数值作为证据。本章就把这套「证据从哪里来、怎么读」讲透。

1.3　工具链总览与顺序方法论

1.3.1　两个工具回答两个不同的问题

nsys 与 ncu 的采集原理完全不同：

- **nsys (Nsight Systems)**：**trace** 型。在驱动层记录 API 调用、kernel launch、内存拷贝、流与事件的时间戳，拼出一张全局时间线。它回答「**时间花在哪**」：哪个 kernel 占比最大、kernel 之间有没有空隙（gap）、CPU launch 是否跟不上、多流是否真的在重叠。代价低：不需要 GPU 性能计数器权限，任何环境都能跑，被测程序几乎不失真。

- **ncu (Nsight Compute)：counter replay** 型。要读一个 kernel 的成百上千个硬件计数器（SM 吞吐、cache 命中、warp stall 等），而一遍执行采不完这么多计数器，于是 ncu 把 kernel **回放多遍**，每遍采一组；为了数据可比还会锁定时钟。它回答「**这个 kernel 内部慢在哪**」。代价高：单 kernel 深挖、时间被串行化与放大，**ncu 报告里的 kernel 耗时不能当 bench 时间用**。

分工：nsys 定位「谁慢」，ncu 解释「单个 kernel 为什么慢」。

1.3.2 顺序：先 nsys，后 ncu

顺序不是习惯问题，是成本问题：

1. **nsys 先行**：一次采集拿到全部 kernel 的时间占比排序。如果主 kernel 占 86%，那么辅助 kernel 再慢也只有 14% 的优化上限：先确认「该优化谁」，再谈「怎么优化」。nsys 还能立刻暴露三类结构性问题：launch gap（CPU 跟不上，一个真实案例 wall-GPU 差 129μs vs 对照 50μs，原因在 dispatch 而非 kernel）、该并行的流没重叠、意外的 fallback kernel。
2. **ncu 跟进**：只对 nsys 排出的 Top-1/Top-2 kernel 做 `--set full` 深挖，读 roofline、occupancy、stall、memory 各维指标，形成假设。
3. **单变量修改 → 复测**：每次只改一处，re-profile 后用 `--diff` 对照前后指标；无效或负优化立即回退。

反着来的代价见本章开头的反例二：先 ncu 后 nsys，等于把半天的深挖时间花在一个可能不重要的 kernel 上。

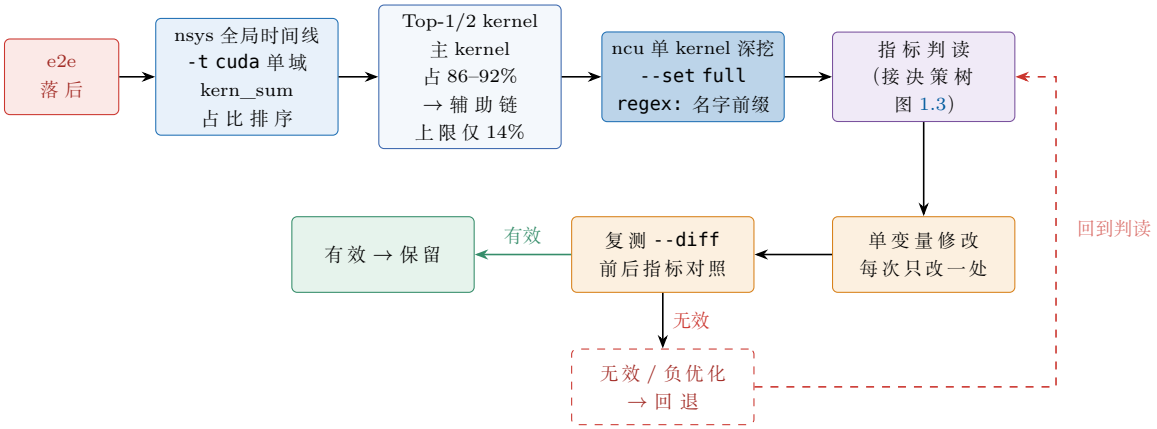


图 1.1: nsys → ncu → 单变量修改 → 复测 `--diff` 的闭环工作流：nsys 先给全局占比与优化预算（主 kernel 占 86–92%，辅助链上限仅 14%，对应 §1.8 例一）；ncu 只深挖 Top-1/2 kernel；每次只改一处，复测后无效立即回退、回到指标判读。配色：红 = 问题起点，蓝 = 采集/分析链，紫 = 判读决策点，橙 = 修改/复测动作，绿 = 有效保留，红虚线 = 无效回退。

1.4 nsys 命令行实战

1.4.1 最小采集

原则：**只采需要的域、只采最短的窗口**。对「看 kernel 占比」这个目标，一条 trace 域 `-t cuda` 就够，CPU 采样与上下文切换全部关掉，warmup 与 repeat 降到最小：

nsys 最小采集（kernel 时间占比场景）

```
nsys profile -t cuda --sample=none --cpuctxsw=none \
  -o .tmp/ch00/report.nsys-rep python bench.py
```

1.4.2　stats 报表：两个最常用的 report

采集产物是 `.nsys-rep`，用 `nsys stats` 导出统计：

nsys stats 两个高频 report

```
nsys stats -r cuda_gpu_kern_sum report.nsys-rep   # per-kernel 时间占比（最常用）
nsys stats -r cuda_gpu_trace   report.nsys-rep   # per-launch 时间线（看 gap/流重叠）
```

`cuda_gpu_kern_sum` 按 kernel 聚合总时长与占比，直接给出「该优化谁」的答案；`cuda_gpu_trace` 逐 launch 列时间戳，用来看相邻 kernel 间的空隙与多流 overlap。首次对旧 report 运行时报 `Existing SQLite export found` 时，加 `--force-export=true` 强制重新导出。

1.4.3　体积控制：一次 500GB 事故

nsys 的中间数据默认写到 `/tmp/nvidia/nsight_systems`，体积 ≈ 单事件字节数 × 事件数，而事件数随时长线性增长。对大模型推理（多模型链、大 seqlen、多次 repeat）不加限制地全量采集，曾把 `/tmp` 写到 **500GiB+** 直到磁盘写满系统卡死（真实事故，2026-09）。三条铁律：

1. **输出与临时文件落项目目录**：显式设 `TMPDIR` 指向项目 `.tmp/` 下的目录并用 `-o` 指定输出，事后清理。注意 TMPDIR 路径要短，过长的路径会超过 Unix domain socket 的 107 字节路径上限，导致 Ray 的 plasma store 构造失败。
2. **禁止对大模型 e2e 不加限制地全量采集**：repeat 降到 0–1，单域 `-t cuda`，关采样。
3. **优先 torch.profiler**：只想看 kernel 时间表时，`torch.profiler` 输出 chrome trace 落在 `.tmp/`，零外部临时文件风险；nsys 只在必须看时间线/同步事件时使用。

1.4.4　窗口化采集：cudaProfilerApi

只想采推理中的一小段（比如 50 个 denoise step 中的 1 步）时，用 `--capture-range=cudaProfilerApi` 配合代码里的开关，把采集窗口限定在目标区间：

窗口化采集（Python 侧）

```
nsys profile -t cuda --capture-range=cudaProfilerApi \
  --sample=none --cpuctxsw=none -o .tmp/ch00/win python bench.py
# bench.py 内：
#   torch.cuda.profiler.start()   # 窗口开始
#   ... 目标 step ...
#   torch.cuda.profiler.stop()    # 窗口结束
```

1.5　ncu 命令行实战

1.5.1　基础采集与 section

ncu 全量采集与 CSV 导出

```
ncu --set full -o .tmp/ch00/report_v1 --target-processes all ./kernel_app
ncu --import .tmp/ch00/report_v1.ncu-rep --page raw --csv > report_v1.csv
```

`--set full` 采全部 section（roofline、occupancy、warp state、memory workload、instruction mix 等），单 kernel 回放多遍、耗时可观；快速确认可用默认 basic set。交互式 `ncu-ui` 看报告，命令行用 `--page details`（rule-based 汇总）或 `--page raw`（原始 metric，见下）。

1.5.2 --kernel-name 必须用 regex: 前缀

C++ 模板 kernel 的完整名带一长串后缀（如 `persist_d_ws_fwd_cute_fp8_sm120`，实际实例化名比这更长）。`--kernel-name` 默认**精确匹配**，拿「名字的前缀」去匹配会报 `No kernels were profiled` 且生成空 report。必须加 `regex:` 前缀走部分匹配：

kernel 名过滤：regex 前缀

```
ncu --set full -o rep --kernel-name regex:persist_d_ws_fwd_cute_fp8 \
  --target-processes all ./app     # OK: 部分匹配
ncu --kernel-name persist_d_ws_fwd_cute_fp8_sm120 ...  # 错: 精确匹配, 0 命中
```

1.5.3 --page raw 提取原始 metric

ncu 的 `--csv` 默认输出是 **rule-based** 的 section 汇总（按诊断规则组织），不是逐个原始计数器。要拿具体 stall reason 的数值，必须走 `--page raw`。raw 输出里每个 metric 是「名字行 + 单位行 + 数值行」的多行结构，用 `grep -A3` 抓：

从 raw page 提取 stall 指标

```
ncu --import rep.ncu-rep --page raw 2>&1 | \
  grep -A3 "warps_issue_stalled_\(wait\|math_pipe_throttle\|sleeping\)"
```

1.5.4 权限限制与同架构代理 GPU

读性能计数器需要驱动授权。容器/云主机上常见 `/proc/driver/nvidia/params` 里 `RmProfilingAdminOnly` 标志为 1，此时 `--set full` 直接报 `ERR_NVGPUCTRPERM`，且容器内无法 reload 驱动。可行解法是**同架构代理**：找一块相同微架构的本地 GPU 跑 ncu，例如用本地 RTX PRO 5000（sm_120a）代理远端 RTX 5090（sm_120）。stall reasons、指令 mix、occupancy 这类**架构级**结论可以迁移，绝对性能数字（SM 数、频率、L2 容量）不可迁移；ncu 验证完架构结论后，回目标硬件跑 timing bench 收数。

1.5.5 replay 的干扰与 A/B 规则

ncu 锁频 + 多遍回放 + 串行执行，因此**报告内耗时只用于相对比较**。做优化 A/B 时回到真实 bench，并遵守两条规则：

- **校验公共参照**：许多 bench 会同时输出被测实现与公共参照（如 PyTorch SDPA）两组时间；对比前先核对两轮参照是否吻合（±1%）。被 GPU 级干扰（并发进程/时钟态）污染的一轮，两者会**同比例膨胀**（实测约 2×），直接对比会得出「−50% 全面加速」这种物理上不可能的假结论。
- **min-of-N 口径**：单轮抖动大的实现（如 tma 变体）取 N 次最小值对比；归因 race 类问题必须 stash → 重建 → 同 probe 的 HEAD 对照，单次「好结果」不可信。

1.6 核心指标字典

以下每个指标按「含义 / 怎么读 / 异常意味着什么 / 对策」展开。数值口径以 sm_120 上的实测为参照。

1.6.1 SpeedOfLight：SM% 与 MEM%（roofline 定位）

`sm__throughput`（SM%）与 `gpu__compute_memory_throughput`（MEM%）分别是计算管线与内存管线的峰值利用率，是 NCU 报告的第一行。两者把 kernel 放上 ch01 的 roofline：SM% 高 MEM% 低是 compute-bound，反之 memory-bound；**双低**（如 SM/MEM 都在 30% 附近）则是 latency-bound：两条管线都没有饱和，问题在并行度或依赖链，这是 roofline 图上看不出的第三态，必须回到 stall 指标找证据。

1.6.2　Occupancy：理论、达到与三类限制源

区分两个数：**理论 occupancy**（kernel 资源能驻留多少 warp，受寄存器、smem、block/warp 上限三类限制源钳制）与**达到 occupancy**（`sm__warps_active`，实际运行均值，受 tail effect 与负载不均拖累）。NCU 的 Occupancy section 会明确指出 `launch__occupancy_limit_registers` / `__shared_mem` / `__blocks` 哪个是当前限制源。一个 fp8 attention kernel 的实例：REG:255 直接把 `Block Limit Registers` 限制到 1：1 CTA/SM、12 warps（4 producer + 8 consumer，共 384 线程），理论 occupancy 25%（12/48），此时想提并行度只有降寄存器或换结构（见 §1.8 例二）。注意 ch01 的告诫：occupancy 是手段不是 KPI，GEMM 类 kernel 用大 tile + 低 occupancy + 高 ILP 换性能是常态。

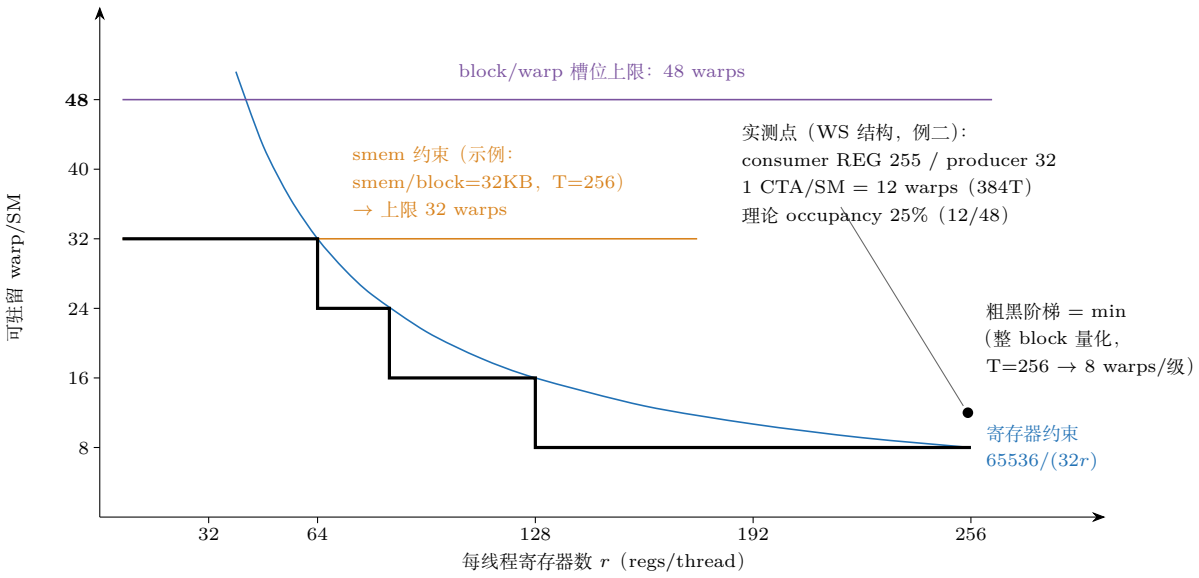


图 1.2: 理论 occupancy 的三限制源取 min（sm_120，48 warp 槽位口径）：寄存器约束 $65536/(32r)$（warp 粒度简化）随每线程寄存器数 r 下降；smem 与槽位为水平约束（示例：smem/block=32KB、T=256 → 32 warps）。粗黑阶梯为按整 block（T=256，8 warps/block）量化的 min 包络；实测点为 WS 结构（producer 32 / consumer 232 regs）1 CTA/SM = 12 warps（384T），理论 occupancy 25%（12/48）。

1.6.3　寄存器与 spill：255 墙

`launch__registers_per_thread` 给出每线程寄存器数，硬上限 **255**（每线程）。超限时编译器把变量溢出到 local memory（SASS 里表现为 `LDL/STL` 指令，NCU 里看 `l1tex__t_bytes_pipe_lsu_mem_local_op__ld/st.sum` 是否非零）。编译期排查用 `--ptxas-options=-v` 与 `cuobjdump --dump-resource-usage` 的 STACK 字节。两个实战要点：

- **spill 的代价主要不是流量**。一个 fp4 attention kernel 在 D=512 时 spill 总量 234MB，但 local 流量仅 12.8GB/s，带宽不是瓶颈；真正的代价是**延迟暴露**：8 warps/SM 意味着每 scheduler 只有 2 个 warp 可切换，LDL 的几百 cycle 延迟藏不住，同时指令数也变多。
- **寄存器预算可以算出来**。attention 的 O 累加器每线程占 $k_{Br} \cdot D/256$ 个 f32 寄存器（M8N1 布局，256 = 32 lanes ×8 个 warp，对应 consumer 全部 256 线程）；k_{Br}=128 时为 $D/2$，D=512 要 256 个，**超出 255 寄存器上限**。这正是大 D 必须换 M4N2 atom（列拆给两个 warp，每线程 $D/4$）的数学来源（ch19）。

1.6.4　Warp State：stall reasons，好 stall 与坏 stall

warp 每个周期若不能发射，必有一个「停滞原因」。sm_120 上的口径是 `smsp_average_warps_issue_stalled_*_per_issue_active.ratio`，即每个发射周期停在该原因上的平均 warp 数。表 1.1 是实战中最常读的八项：

读 stall 剖析结果的关键是**分好坏**：`math_pipe_throttle` 与 `not_selected` 高说明计算与调度资源在正常工作；`sleeping` 高几乎总是结构问题；`wait` 与 `long_scoreboard` 的**相对大小**直接区分「依赖链受限」与「装载受限」，例见 §1.8 例三。

表 1.1: warp stall 指标字典（sm_120 实测口径）

指标后缀	含义	高值时怎么办
`wait`	固定延迟依赖（寄存器/ALU 结果未就绪）	依赖链问题:拆链、增并行 warp、重排指令；attention 的 softmax→PV 串行链是典型来源
`long_scoreboard`	等 gmem/TMA 数据	装载不足：`cp.async`/TMA、加深 stage、prefetch
`short_scoreboard`	等 smem（MIO 队列）	LDS 链太长：swizzle 布局、减少 smem 往返
`math_pipe_throttle`	数学管线排队满	**好信号**：计算饱和。保持或进一步减 FLOPs
`sleeping`	warp 休眠（mbarrier 退避）	**纯浪费**：producer/consumer 失衡或空转自旋，查 WS 结构
`barrier`	等显式栅栏	同步过频：减少 `__syncthreads` 点、缩 block
`not_selected`	就绪但调度器选中了其他 warp	**健康**：有冗余并行度可隐藏延迟
`selected`	正在发射（非停滞）	基线，越大越好

1.6.5 Memory Workload：L1/L2/DRAM 分层归因

三层各看命中率与吞吐两个数:

- **L1/TEX**：`l1tex__t_sector_hit_rate.pct`。低于 20% 且数据本该复用时，考虑 tiling 进 smem 或改访问模式。注意 smem 与 L1 **共池**（GB202 实测每多占 16KB smem，kernel 约慢 4–5μs），「多分 smem」不是免费的。
- **L2**：`lts__t_sector_hit_rate.pct`。大 GEMM 上低命中可用 CTA swizzle / StreamK 提数据局部性；推理场景 K/V 重复读高命中是常态。
- **DRAM**：`dram__throughput`（峰值百分比）。接近饱和（>70%）说明带宽墙已到，优化方向只剩减流量（混合精度、融合、去物化）。一条量化前处理链实测达到饱和带宽 1.04TB/s（约为理论峰值 1344GB/s 的 77%）。此时「融合省 launch」已无杠杆，只有减流量才有，这个判断直接由 DRAM 指标给出。
- **load/store 与合并访问**：global load 的每请求扇区数（`l1tex` 前缀的 `average_t_sectors__per_request_...` 系列指标）衡量合并访问质量：32-bit 连续访问的理想值是 4（一个 warp 的 128B 恰好 4 个 32B sector）；明显超标说明访存发散，流量被放大，对策是内维连续、向量化（`float4` 128-bit 访存，ch03）与 AoS→SoA。IO 压力要从三个正交维度一起看：**吞吐**（哪层饱和）、**命中率**（数据该由哪层负责）、**合并度**（流量有没有被访问模式放大）。

分层归因顺序：DRAM 饱和先看是不是 L2 miss 推的（L2 hit 低 + DRAM 高 = 工作集溢出），L2 高命中但 L1 低命中 = smem/L1 复用不足。IO 压力的「层」定位错了，对策就全错。

1.6.6 bank conflict 与 smem 指标

`l1tex__data_bank_conflicts_pipe_lsu_mem__shared_op_ld/st.sum` 直接数冲突次数（ch12 表 12-1 的数据来源）。判读要点：**先看占比再动手**。曾有一个「消除 O 写回 bank conflict」的优化立项，NCU 一查 ld conflict 仅 0.39%，且 MUFU 等待被 MMA 掩盖，整个方向当场否决。冲突计数要换算成「每 warp 每 cycle 的额外重放」才能反映实际影响，绝对数大不等于值得修。

1.6.7 指令 mix 与 tensor pipe 利用率

各执行管线的活跃占比（`pipe_fma` / `pipe_alu` / `pipe_lsu` / `pipe_tensor`）回答「指令都花在哪了」。对本书 Part II–IV 的主线而言，**tensor pipe 利用率就是 Tensor Core 饱和度**：fp16 GEMM 类 kernel 若 `pipe_fma` 高

而 `pipe_tensor` 低，说明还在用 CUDA Core 算乘加，应该换 MMA 指令（ch10–11 的主题）。一个 fp8 kernel 的实例：tensor pipe 从 49% 提到 59.5% 的那次优化，判据正是「`math_pipe_throttle` 降了、tensor pipe 升了」，两条指标共同确认依赖链被拆开、Tensor Core 利用率提高。

1.7　瓶颈判断决策树

把上述指标串成一条判断流程（数值阈值为工程经验值，非硬边界）：

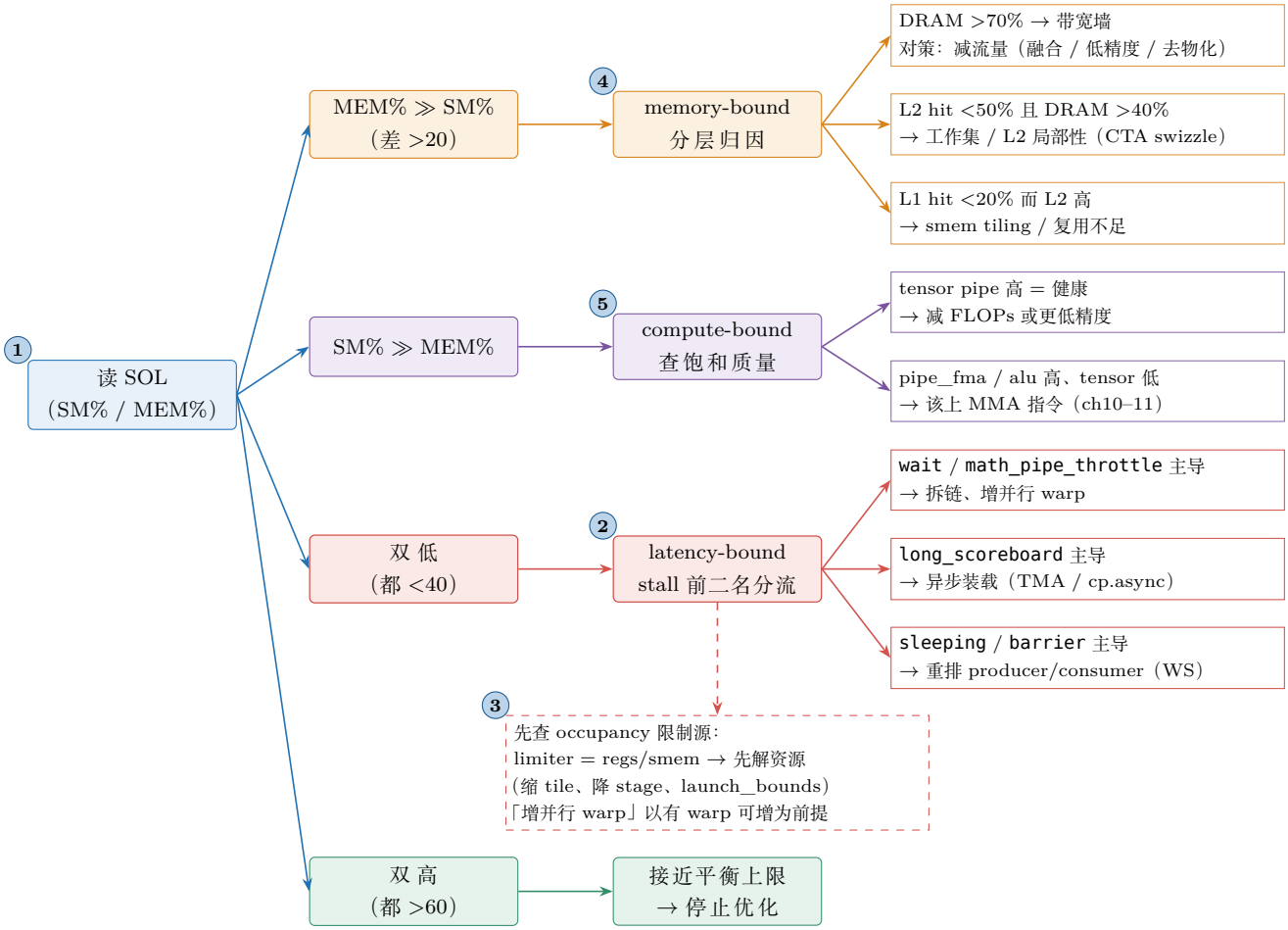


图 1.3: 瓶颈判断决策树：先读 SOL（SM%/MEM%）四分流——蓝为入口判断，橙为 memory-bound 链，紫为 compute-bound 链，红为 latency-bound 链，绿为双高（停止优化）；对策框白底、边框沿用所属分支色。节点角标 1–5 对应正文 enumerate 步骤（阈值为工程经验值）。

1. **先看 SOL**：MEM% ≫ SM%（差 >20）→ memory-bound，进第 4 步；SM% ≫ MEM% → compute-bound，进第 5 步；**双低**（都 <40）→ latency/依赖链问题，进第 2 步；双高（都 >60）→ 已接近平衡上限，停止优化。

2. **latency-bound 分流**（看 stall 前二名）：`wait` 或 `math_pipe_throttle` 主导 → 依赖链/发射受限，对策是拆链（拆 CTA、拆 warp 分工）或增并行 warp；`long_scoreboard` 主导 → 数据供给不足，对策是异步装载（`cp.async`/TMA）与加深流水；`sleeping`/`barrier` 主导 → 同步结构问题，对策是重排 producer/consumer 与栅栏。

3. **查 occupancy 是否是限制源**：若 achieved 低且 limiter 是 registers/smem，先解决资源（缩 tile、降 stage、`__launch_bounds__`），因为第 2 步的「增并行 warp」以有 warp 可增为前提。

4. **memory-bound 分层**：DRAM >70% → 带宽墙，减流量；L2 hit <50% 且 DRAM >40% → 工作集/L2 局部性问题；L1 hit <20% 而 L2 高 → smem tiling/复用不足。顺带查 coalescing 与 bank conflict。

5. **compute-bound 查饱和质量**：tensor pipe 高 = 健康，优化转向减 FLOPs 或换更低精度；tensor pipe 低而 fma/alu 高 = 该上 MMA；指令数本身异常大时查发散与编译器胶水指令。

表 1.2: 症状 → 证据 → 结论 → 对策速查

症状	指标证据	结论	对策
e2e 慢但不知谁慢	nsys kern_sum 占比	定位目标 kernel	只对 Top-1/2 跑 ncu
kernel 慢、带宽未饱和	MEM%≤40，wait/mpt 主导	依赖链受限	拆链、2 CTA/SM、增并行 warp
装载等数据	long_scoreboard 高	供给不足	TMA/cp.async、加深 stage
并行度上不去	occupancy 低 + limiter=regs	寄存器上限	缩 tile、M4N2 拆列、launch_bounds
local ld/st 非零	LDL/STL、local bytes	spill	降每线程状态、查 255 寄存器上限
空转严重	sleeping 高	WS 失衡	重排 producer/consumer
带宽饱和	DRAM >70%	带宽墙	减流量：融合/低精度/去物化
smem 访问慢	bank conflicts 高	冲突重放	swizzle/pad（ch12）
TC 没用上	pipe_tensor 低	CUDA Core 算矩阵	换 mma/wgmma 指令

1.8 实战三例：占比、寄存器与 stall

1.8.1 例一：nsys 先行——占比决定优化预算

场景：一个 fp8/fp4 量化 attention 库的 e2e（含量化前处理链）对标业界实现。e2e 一度落后，最初的直觉是「辅助链 kernel 太多（每 call 约 13 个）应该融合成一个 Mega Kernel」。先用全局视角（nsys + 每 kernel 带宽核算）确认：大 N 场景**主 kernel 占 e2e 时间 86–92%**，而 aux 链每个 kernel 都已达到实测饱和带宽 1.04TB/s：融合只能省 launch 不能减流量，大 N wall 差实测 <0.2%，方向证伪；真正的杠杆在主 kernel 与引擎层 overlap。**方法论**：占比数据先给「优化预算上限」（占 e2e 14% 的部分提不出 30% 的收益），带宽核算再判这部分还有没有优化空间。

1.8.2 例二：寄存器压力——255 墙、spill 与 setmaxnreg 的取舍

场景：fp8 persist-D kernel 采用 warp specialization（WS）：1 个 producer warp group（128T，专职 TMA）+ 1 个 consumer（256T，全计算），用 `setmaxnreg` 把寄存器从 producer（32 个）再分配给 consumer（232 个）。寄存器压力的证据链分三层：

- **静态**：`cuobjdump --dump-resource-usage`（D=768, sm_120f）：fp8 split_d REG:255 + STACK 1104–1944B（重 spill）；fp8 m4n2 REG:255 + STACK 0–264B（几乎无 spill）；fp16 split_d REG:196–254 + STACK 约 1536B。STACK 字节就是溢出到 local 的体积。
- **动态**：NCU 确认 `Block Limit Registers=1`（REG:255 的直接后果），理论 occupancy 25%（12 warps/48）且 smem 守卫恒空转，限制源唯一。
- **取舍**：consumer 需要更多寄存器（`setmaxnreg` 上限 232，实测 224–232 区间性能平坦），但 O 累加器 $D/2$（M8N1）在 D=512 时要 256 个，**超出 255 寄存器上限**，于是 split-D 的 WS 变体被整体禁用，大 D 改走 M4N2 把每线程压力降到 $D/4$。这是「寄存器文件利用率 vs occupancy vs 算法结构」三方权衡的标准案例：不是把寄存器降下来就一定有利，而是选一个让 tile、并行度与 spill 同时可接受的工作点。

（注意 `setmaxnreg` 是否被丢弃与 arch 后缀无关：`sm_120a` 与 `sm_120f` 都完整支持并保留该指令。真正决定它的是 TMA 的目的地址空间——写成 `shared::cluster` 会被 ptxas 以 C7506 静默丢弃，写成 `shared::cta` 则保留——以及 `__launch_bounds_` 的第二参数（缺则 C7508 丢弃）。见第 十五 章坑二。）

1.8.3 例三：stall 证据链——好 stall 与坏 stall 的判读

场景：sm_89 fp8 persist-D 909.4μs vs 业界实现 710μs （慢 28%）。NCU 剖析结果：

- `math_pipe_throttle` **1.32**（第一名）、`wait` 0.82、`long_scoreboard` **0.04**、tensor pipe 49%、255 regs、1 CTA/SM。
- 判读：`long_scoreboard` 近零 ⇒ 装载不是瓶颈，一切「加深流水/减指令/缩 tile」无效（后续 6 项实验全部证伪，原因就是没先读这组数）；`mpt` 高但 tensor pipe 只有 49% ⇒ 管线在排队却没排满 Tensor Core，即**依赖链/发射受限**。
- 对策与验证：kBr=64 + kBc=64 + 128T（2 CTA/SM 拆链）→ 864μs （−5%），ncu 确认 2 blocks/SM、每线程寄存器降到 250；**指标闭环**：mpt 1.32→0.72、tensor pipe 49%→59.5%，「排队变短、TC 利用率更高」与 wall 时间下降互为印证。

同族的「坏 stall」案例：bias 注入的 smem tile 版本 `sleeping` 从 3.5% 涨到 22.3%（SASS 采样里 `SYNCS.PHASECHK` mbarrier 自旋占 18%）：LDS 进了 consumer 关键路径、两级 QK 流水供不上，全 warp 自旋等 K，该版本被指标否决。

1.9　常见坑

1. **`--kernel-name` 精确匹配陷阱**。模板 kernel 名带长后缀，前缀匹配报 `No kernels were profiled` 且生成空 report；一律 `regex:` 前缀。
2. **ncu 辅助工具与 bench 配置不一致**。profiling 用的配置必须与 bench 默认逐 flag 对齐；显式传 flag 会覆盖 bench 内部的默认值，实测逐 flag 传参 vs 全默认可差 13%，指标分析对象可能根本不是 bench 跑的那个 kernel 形态。
3. **A/B 不校验公共参照**。GPU 级干扰让两轮**同比例膨胀**（约 2×），对比出假加速。先核对 bench 输出里的 SDPA 参照时间（±1% 内）再谈结论；抖动大的实现用 min-of-N。
4. **把 section 输出当 raw metric**。`--csv` 默认输出是 rule-based 汇总；要具体计数器数值必须 `--page raw`（配合 `grep -A3` 抓多行结构）。
5. **拿 ncu 的耗时报性能**。replay 多遍 + 锁频 + 串行化，报告内时间只做相对比较；性能数字一律回真实 bench 采。
6. **不标性能口径**。写结论必须标注 GPU 型号/架构/日期（PRO 5000 sm_120a 与 5090 sm_120 的架构级结论可互证，绝对数字不可）；fallback 也要查：小 seqlen（<512）时库可能自动 fallback 到 PyTorch SDPA，profile 到的根本不是目标 kernel。
7. **nsys 全量采集写满磁盘**。大模型 e2e 无限制采集曾写满 `/tmp`（500GiB+）。TMPDIR 落项目目录、单域、最小 repeat、事后必清（§1.4）。

1.10　小结

本章没有写一行 kernel，却给出了全书其余 26 章共同的工作流程：**nsys 定位（谁慢）→ ncu 定性（为什么慢）→ 单变量修改 → 复测 diff → 指标闭环**。指标字典里最值得带走的三句话：SOL 双低不等于没有问题，而是 latency-bound，答案在 stall 剖析结果里；stall 分好坏，`math_pipe_throttle` 高说明 Tensor Core 已饱和，`sleeping` 高是结构问题的信号；寄存器 255 墙、spill 与 occupancy 由同一份资源预算决定，读数时要一起读。工具路径：ncu/nsys 随 CUDA Toolkit 安装（通常在 `/usr/local/cuda/bin`），不在 PATH 时用绝对路径调用。

延伸阅读

- NVIDIA Nsight Systems 文档：developer.nvidia.com/nsight-systems
- NVIDIA Nsight Compute 文档（含 Profiling Guide 的完整指标索引）：developer.nvidia.com/nsight-compute
- 本书各章「性能分析」节：ch09–14（GEMM/TMA/WS 的指标判读）、ch12 （bank conflict 计数器实战）、ch19（大 D 寄存器压力模型）、ch01 （roofline 与 occupancy 的理论侧）。

第二章　GPU 架构、执行模型与 Roofline

2.1　本章导读

这是全书的第一章。动手写任何 kernel 之前，先回答一个更基本的问题：GPU 到底是怎么跑代码的，瓶颈到底在哪里？这一章把 base.cuh 开头的 Phase 0「面试速查表」(L1–86) 扩写成一篇完整的第一课：从 SM (Streaming Multiprocessor，流式多处理器）的内部结构讲起，穿过 SIMT 执行模型与内存层级，最后落到 **Roofline 模型**：只靠乘除法就能预估 kernel 的性能上限。

本章不引入任何新的编译宏与架构限定（全 arch 通用），也没有 device kernel，配套的最小测试 ch01_roofline.cu 是一个 host 侧演示程序：对一组典型算子计算算术强度并与解析期望值核对，同时打印本机 GPU 的资源与理论带宽。后续所有章节的「性能分析」节都会反复使用本章建立的词汇表：算术强度 (AI)、ridge point、memory-bound 与 compute-bound、occupancy 与延迟隐藏。

前置：无。**源码区间**：base.cuh L1–86。

2.2　问题与动机

先建立一个直觉：CPU 和 GPU 是两种设计哲学的产物。CPU 的目标是**尽快完成一个任务**（低延迟），于是它把晶体管花在大容量 cache、复杂的分支预测与乱序执行引擎上；GPU 的目标是**同时完成海量任务**（高吞吐），于是它把晶体管花在成千上万个算术逻辑单元（ALU）上，寄存器文件巨大而 cache 相对很小，分支预测几乎不存在：GPU 根本不打算「预测」分支，它靠成千上万个线程之间的快速切换把内存延迟藏起来。

直接后果是：**同一个算法在 GPU 上快慢与否，几乎完全由它的「访存-计算比例」决定**。一个向量加法（读两个数、加一次、写一个数）与一个矩阵乘法（读两个数要做几十上百次乘加）在 CPU 上差异不大，在 GPU 上差异极大：前者把大部分时间花在等数据上（memory-bound），后者才能让计算单元饱和（compute-bound）。base.cuh L22–30 把这种判断浓缩成三条判据，外加一条 occupancy 公式：

- **Memory-bound**：AI（Arithmetic Intensity，算术强度）低于机器的 FLOPS/ 带宽比值；
- **Compute-bound**：AI 足够大，受限于计算吞吐；
- **Latency-bound**：线程不够多，无法隐藏内存延迟。

$$\text{occupancy} = \frac{\text{active warps}}{\text{max warps per SM}},$$

受三类资源分别取下限：每线程寄存器数 → threads/SM；每 block 的 shared memory → blocks/SM；block 大小 → blocks/SM。

这三条判据就是本章的路线图。前两条给出判断标准，即比较算术强度与机器的「转折点」；第三条是容易被忽略的瓶颈：latency-bound，它甚至与算法无关，纯粹是并行度不够、延迟藏不住的问题。要把这三条判据看懂，我们需要先弄清两件事：机器的算力与带宽从哪里来（§2.3），以及「算术强度」怎么算（§2.4）。

2.3　SM 内部结构与内存层级

2.3.1　SM：GPU 的基本计算单元

GPU 是由大量 SM 组成的阵列。以本章测试机 RTX PRO 5000（Blackwell，sm_120）为例，一块芯片上有 110 个 SM；base.cuh L7–21 为 SM 内部结构与内存层级留了一份速查清单，部件规格如下：

部件	规格（速查口径）
Warp Scheduler × 4	每 SM 4 个 warp scheduler，每个每周期可发射 1 条指令
Register File	每 SM 65536 × 32-bit（4 bytes）= 256 KB
Shared Memory / L1	可配置划分，最大 shared memory ∼228 KB（Hopper）
Tensor Cores	Hopper 每 SM 4 个；Blackwell 数量随型号/定义不同，建议以官方 ISV guide 为准
Warp	32 threads，最小调度单元，SIMT 执行模型

清单的另一半是内存层级带宽数量级（H100 参考：HBM3 ∼3.35 TB/s 理论、L2 ∼12 TB/s、L1/SMEM ∼19 TB/s、Register ∼100+ TB/s），见下一小节的速率/容量金字塔。部件清单逐条展开：

- **Warp Scheduler × 4**：每个 SM 有 4 个 warp scheduler，每个每周期可以发射 1 条指令。这意味着一个 SM 每周期最多发出 4 条指令，「发射带宽」本身就是一种资源。
- **Register File**：每 SM 65536 × 32-bit = 256 KB。这是 GPU 上最快也最稀缺的存储：所有活跃线程的寄存器都从这里分配。寄存器分配以 warp 为粒度均分，若 kernel 每线程用 64 个寄存器，一个 256 线程的 block 就要占 $256 \times 64 \times 4\,\mathrm{B} = 64$ KB，SM 里最多驻留 4 个这样的 block。
- **Shared Memory / L1**：同一块片上 SRAM 中可配置划分出的共享内存部分（Hopper 上这块 SRAM 为 256 KB，SM120 本机为 128 KB 量级），Hopper 上最多约 228 KB/SM 可用作 smem（SM80 为 164 KB/SM，per-block opt-in 上限 163 KB；SM120 本机实测 opt-in 上限为 99 KB，见 §2.11）。block 内线程交换数据靠的就是它，后续章节 bank conflict（ch07）、swizzle（ch12）也都在这块 SRAM 上展开。
- **Tensor Cores**：Hopper 每 SM 4 个第四代 Tensor Core，专门执行矩阵乘加（MMA）；Blackwell 的数量与形态随型号有差异，源码注释提醒「以官方 ISV guide 为准」，本书 Part II–IV 的大量篇幅都在围绕如何充分利用这些 Tensor Core。
- **Warp = 32 threads**：warp 是**最小调度与指令发射单元**，32 个线程锁步执行同一条指令（SIMT）。注意区分两个概念：线程是**执行**的最小粒度，warp 是**调度与发射**的最小粒度。硬件把 32 条「线程通道」捆绑成一个 warp，用一套取指-译码-发射逻辑服务 32 个计算 lane，控制逻辑的开销因此被摊薄，能以少量控制电路驱动大量 ALU。

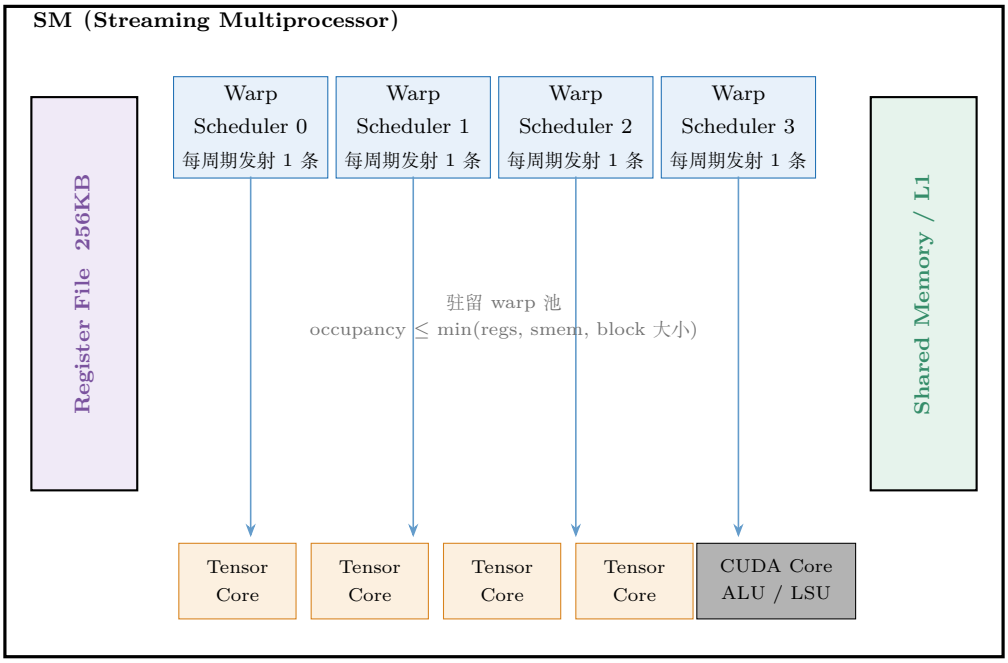


图 2.1: SM 内部结构示意（Blackwell sm_120 口径）：顶部 4 个 warp scheduler（各每周期发射 1 条指令），左侧 256KB 寄存器文件与右侧 smem/L1 共池 SRAM 共同钳制驻留 block 数，底部 4 个 Tensor Core 与 CUDA Core 阵列执行发射来的指令。

2.3.2 内存层级：带宽与容量的权衡

base.cuh L16–21 给出了 H100 参考下的带宽数量级：HBM3 约 3.35 TB/s（理论值，实际约 2.5–3.0 TB/s）、L2 约 12 TB/s、L1/SMEM 约 19 TB/s、寄存器等效带宽 100+ TB/s。把这些数字画成金字塔（图 ch01-1），层级规律直接呈现出来：**越靠近计算单元，带宽越高、容量越小**，相邻层级的带宽差从 1.5 倍到 5 倍不等；容量上寄存器与 smem 同量级，再往上每升一级差两个数量级以上。

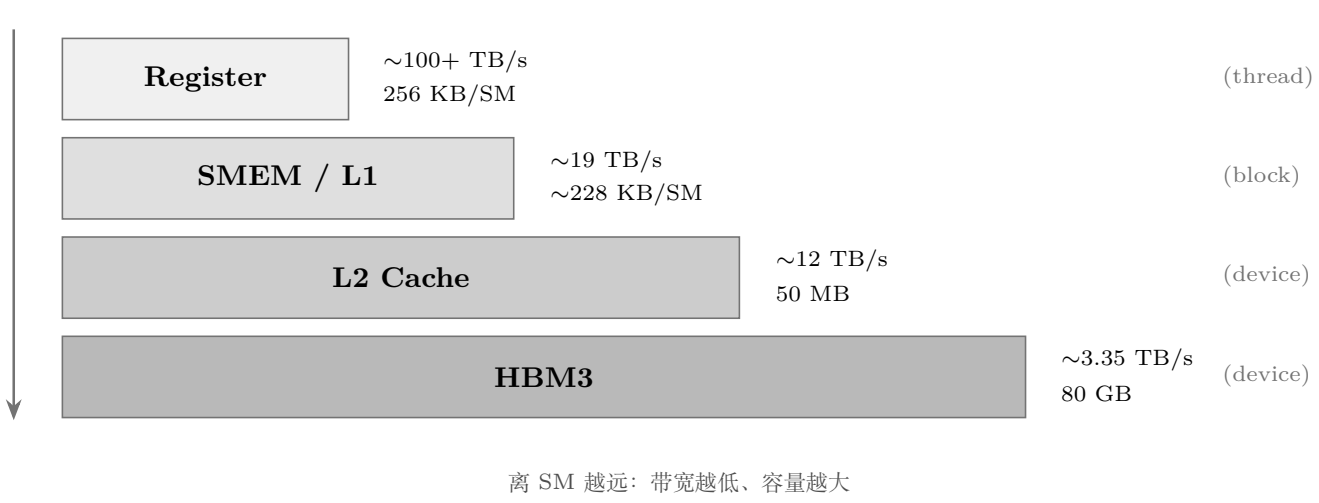


图 2.2: GPU 内存层级金字塔：离 SM 越远带宽越低、容量越大（H100 数量级）。

这张图给出 GPU 编程的一条基本规律：**数据每上移一层，带宽就高一级；kernel 优化就是让数据尽量驻留在高层级**。ch02 让归约的中间量全部驻留寄存器（warp shuffle），ch09–12 用 tiling 把矩阵块搬进 smem 复用，ch13–14 用 TMA+Warp Specialization 把「搬运」与「计算」解耦成流水线，走的都是同一条路。

2.4　数学原理：Roofline 与算术强度

2.4.1　算术强度（Arithmetic Intensity, AI）

定义一个算法（或一段 kernel）的算术强度：

$$AI \;=\; \frac{\text{FLOPs}}{\text{Bytes}}, \tag{2.1}$$

其中 FLOPs 是算法需要的浮点运算次数（**总量**，单位 FLOP，不是每秒），Bytes 是它必然要在**最外层存储**（通常是 HBM）上搬运的字节数，口径是「每个数据读一次、写一次」。AI 的单位是 FLOP/Byte，物理含义是：**每搬运 1 字节，能换来多少次浮点运算**。

三个注意点。第一，FLOPs 数「算法固有」的量：乘加各记 1 次，因此一个点积 $a \cdot b$（N 项）是 $2N$ 次。第二，Bytes 是**不可再省的下界**：若算法要读一遍 a 写一遍 y，无论 cache 命中率多高，HBM 流量都不会低于 $4(|a| + |y|)$ 字节（fp32）。第三，中间量不算：softmax 的逐元素 max、exp、sum 都在寄存器里完成，不出 HBM，所以不计入 Bytes。

2.4.2　三个典型算子的 AI 逐项推导

GEMM（$C = AB$，尺寸 $M{\times}K$ 乘 $K{\times}N$）。输出 C 有 $M \times N$ 个元素，每个元素 $c_{ij} = \sum_k a_{ik} b_{kj}$ 需要 K 次乘与 K 次加，共 $2K$ 次；所以

$$\text{FLOPs} = 2MNK. \tag{2.2}$$

Bytes 按「三个矩阵各读写一次」计：

$$\text{Bytes} = 4\,(MK + KN + MN). \tag{2.3}$$

当 $M = N = K$ 时有简洁的解析形式：

$$AI_{\text{GEMM}} = \frac{2K^3}{3K^2 \cdot 4} = \frac{K}{6}, \tag{2.4}$$

$K = 4096$ 时 $AI = 682.7$ FLOP/Byte，base.cuh L75–79 注释中「≈ 685」由此而来（注释取了 137G/200M 的粗略值，精确值是 682.67，见 §2.8）。AI 随 K **线性增长**：GEMM 天然是 compute-bound 算法，矩阵越大每字节访存对应的计算量越多，这也是它适合 GPU 与 Tensor Core 的根本原因。

GEMV（$y = Ax$）。A 是 $M{\times}K$，x 是 K 维向量，y 是 M 维输出。y 的每个元素要 K 次乘 K 次加，FLOPs $= 2MK$；而 Bytes $= 4(MK + K + M)$。大 MK 主导下：

$$AI_{\text{GEMV}} \approx \frac{2MK}{4MK} = 0.5 \text{ FLOP/Byte}. \tag{2.5}$$

矩阵 A 的每个元素只读取一次，没有任何复用，AI 被「读一次只用一次」固定在 0.5，比 GEMM 低三个数量级，是彻底的 memory-bound。ch08 将看到三种 warp 划分方式如何逼近这 0.5 对应的理论带宽。

Softmax（逐 token，宽度 N）。对长度 N 的行做 softmax：1 次 max 比较、N 次减法与 exp（计 $2N$）、N 次求和、N 次除法，共约 $5N$ FLOPs；访存是读入 x、写出 y 各一次，$4 \times 2N$ 字节：

$$AI_{\text{softmax}} = \frac{5N}{8N} = \frac{5}{8} = 0.625, \tag{2.6}$$

与 base.cuh L85 的口径一致。ch04 的三级递进（naive → safe → online）优化的是访存**趟数**（3-pass → 1-pass），而 AI 的上限 0.625 告诉我们：无论怎么优化，它始终是 memory-bound，最终性能用「离理论带宽多近」来衡量。

2.4.3　Roofline 模型与 ridge point

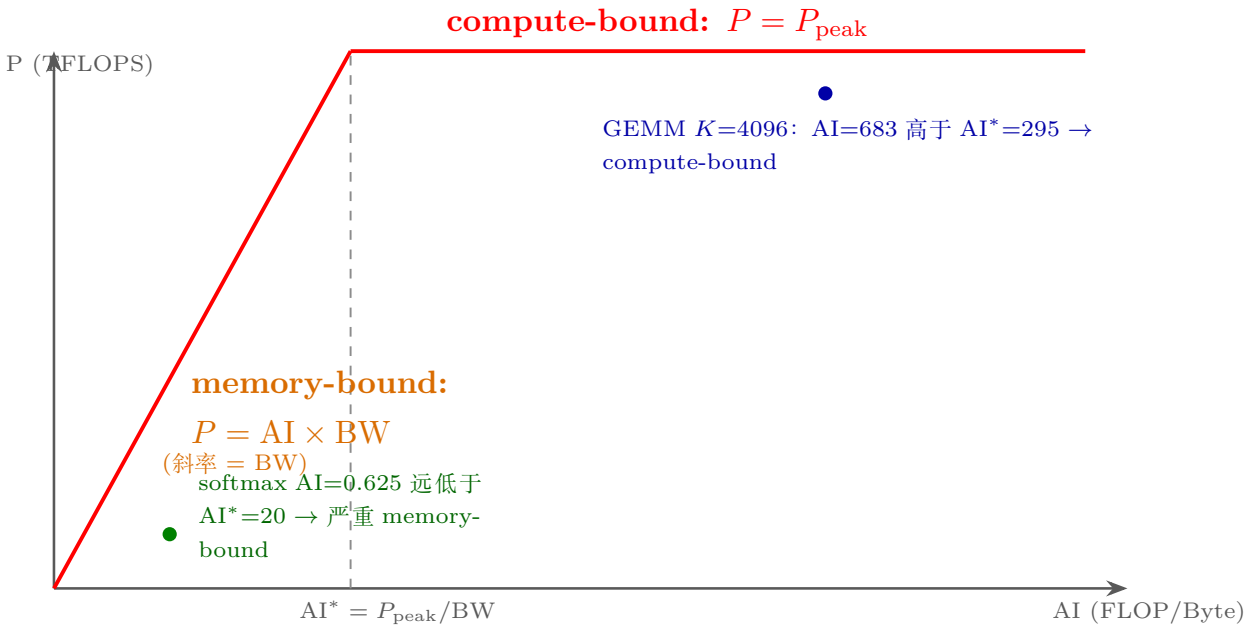
把峰值算力 P_{peak}（FLOP/s）与峰值带宽 BW（Byte/s）放在一起，Roofline 模型给出任意算子的可达性能上界：

$$P \;=\; \min\big(P_{peak},\; AI \times BW\big). \tag{2.7}$$

它的图像（图 ch01-2）是一整条「屋顶线」：左边是一段斜率为 BW 的斜坡（强度每多 1 FLOP/Byte，性能就多 BW，带宽是瓶颈），右边是一段水平的「天花板」（P_{peak}，算力是瓶颈）。两段交点的横坐标叫 **ridge point**：

$$AI^* \;=\; \frac{P_{peak}}{BW}. \tag{2.8}$$

Roofline 模型： $P = \min(P_{\text{peak}},\ \text{AI} \times \text{BW})$



例：H100 SXM（base.cuh L84–86 口径）FP16 TC dense P_{peak} 约 989 TFLOPS，BW=3.35 TB/s → AI^* 约 295；FP32 CUDA core 约 67 TFLOPS → AI^* 约 20

图 2.3: Roofline 模型：$P = \min(P_{\text{peak}}, \text{AI} \times \text{BW})$，脊点 $\text{AI}^* = P_{\text{peak}}/\text{BW}$。GEMM K=4096（AI= 683）落在计算屋顶，softmax（AI= 0.625）落在带宽斜线区。

判断规则由此而来：$AI < AI^*$ 则 memory-bound，$AI > AI^*$ 则 compute-bound。

同一个 GPU 有多条屋顶线。FP32 CUDA core 与 FP16 Tensor Core 的 P_{peak} 相差约 15 倍，于是 ridge point 也相差 15 倍：在 H100 上，AI=100 的算子用 FP32 跑是 compute-bound（$100 > 20$），切到 FP16 TC 反而是 memory-bound（$100 < 295$）。这解释了一个反直觉的现象：**把 kernel 迁移到 Tensor Core 上未必变快**：如果它本是 memory-bound，算力提高 15 倍也无法提升性能。同理，把 ridge point 算错（比如把 sparse 峰值当 dense 用，见 §2.9）会把 bound 类别判反，整个优化方向就全错了。

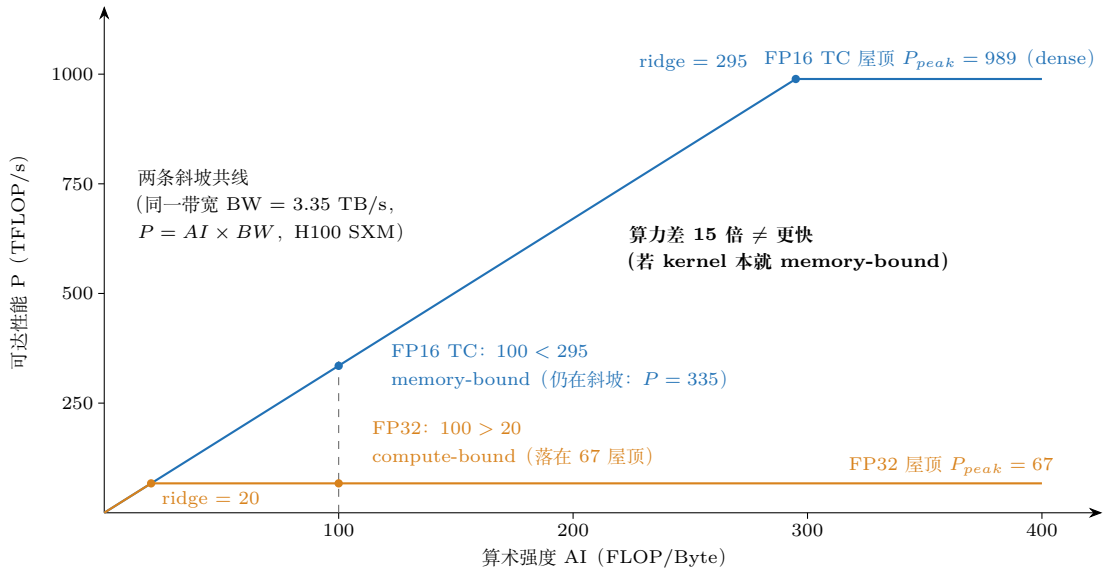


图 2.4: 同一 GPU 的两条屋顶线（H100 SXM）：FP32（$P_{peak} = 67$ TFLOP/s，ridge = 20）与 FP16 TC dense（$P_{peak} = 989$，ridge = 295）共用同一条带宽斜坡（BW = 3.35 TB/s），脊点相差约 15 倍。AI=100 的竖虚线在两条线上落点相反：FP32 侧已 compute-bound（$100 > 20$），FP16 TC 侧仍 memory-bound（$100 < 295$）——把 kernel 迁到 Tensor Core 未必变快；ridge 口径算错（如 sparse 当 dense）会把 bound 类别判反。

2.4.4　Occupancy 与延迟隐藏

Roofline 描述「算法-机器」的匹配，occupancy 描述「实现」的并行度。GPU 隐藏内存延迟靠的不是 cache（容量太小），而是 **warp 切换**：当 warp A 因等 HBM 数据而停顿时，scheduler 立刻切到 warp B 发射指令；等到 A 的数据到了再切回来。设单次 HBM 访问约 500 ∼ 800 个时钟周期，若要发射口持续有指令可发，SM 上必须常驻足够多的活跃 warp。活跃 warp 数与硬件上限之比就是 occupancy（base.cuh L27–30）：

$$\text{occupancy} = \frac{\text{active warps / SM}}{\text{max warps / SM}}, \tag{2.9}$$

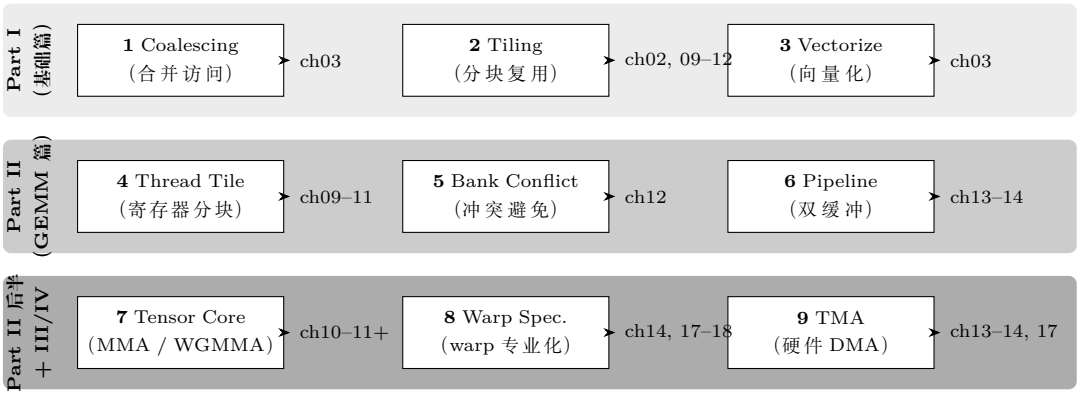
它同时受三类资源钳制，实际值取各约束下的最小值：每线程寄存器数（寄存器用得越多，能驻留的 warp 越少）、每 block 的 smem 用量（smem 用得多，block 数就少）、block 大小（block 太小则凑不满 SM 的线程槽位）。本书后续反复出现这类权衡：ch12 的寄存器双缓冲用更多寄存器换流水线，ch14 的 setmaxnreg 干脆在 producer/consumer 之间**动态重新分配**寄存器池，都是在 occupancy 这个维度上做取舍。

2.5　设计与实现

本章没有 device kernel，Phase 0 的资产是一张「速查表」，它的「实现」就是把 9 条优化手段清单（base.cuh L32–69）与全书章节对应起来，形成一张路线图。先把前三条摘出来：

- **Coalesced Memory Access（合并访问）**：同一 warp 的线程访问连续的 128B 对齐地址，只需 1 次内存事务；否则产生多次事务（最坏 32 次）。
- **Tiling（分块）**：将数据从 HBM 分块加载到 shared memory 复用，减少 HBM 访问；GEMM 的形态是 Block Tile（BM×BN）加 K Tile（BK）。
- **Vectorized Memory Access（向量化）**：使用 `float4`/ `half2` 等向量类型，减少 load/store 指令数；`float4` 是 128-bit，单条指令加载 16 bytes。

清单的第 1–3 条构成 Part I 前三章的骨架：coalescing 是 ch03 的主题，vectorize 也在 ch03，tiling 的初级形态（warp/block reduce 中的两级结构）在 ch02 就能见到。第 4–6 条（thread tile、bank conflict、pipeline）在 Part II GEMM 篇逐一兑现；第 7–9 条（Tensor Core、Warp Specialization、TMA）属于 Part II 后半与 Part III/IV。读到这里建议翻回本表：本书的章节顺序，就是这张清单逐条展开的顺序。



行分组沿用 base.cuh 清单 1–3 / 4–6 / 7–9：第 1–3 条是 Part I 前三章骨架；第 7–9 条的兑现章节横跨 Part II 后半（ch13–14）与 Part III/IV

图 2.5: base.cuh 九条优化手段与全书章节路线图：第 1–3 条构成 Part I 前三章的骨架（coalescing 与 vectorize 在 ch03，tiling 的初级形态在 ch02）；第 4–6 条在 Part II GEMM 篇兑现；第 7–9 条横跨 Part II 后半与 Part III/IV。泳道灰度随 Part 递增（15%/40%/65%）。

2.6　图示

图 ch01-1（内存层级金字塔）与图 ch01-2（Roofline 示意）已分别在 §2.3 与 §2.4 给出正式版。这里补充一张「一次 HBM 读的全程」，把本章的概念串成时间线：

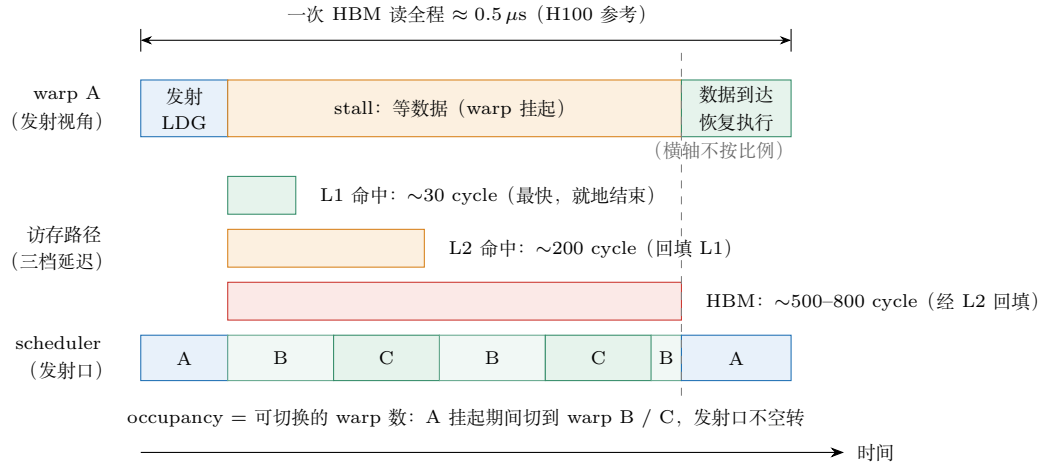


图 2.6: 一次访存的全程与延迟隐藏（蓝 = 发射/执行，橙 = 等待/过渡延迟，绿 = 命中与恢复，红 = 直达 HBM 的最长延迟）

2.7 性能分析

本章无 kernel，性能分析给两张「标尺」表，供全书各章对照。

表 1：H100 SXM 参考口径（取自 base.cuh L16–21 与 L84–86，官方 spec 与微基准口径混排，见 §2.8）：

层级	带宽	容量	作用域
Register	~100+ TB/s	256 KB/SM	thread
SMEM/L1	~19 TB/s	~228 KB/SM	block
L2	~12 TB/s	50 MB	device
HBM3	~3.35 TB/s（理论）	80 GB	device

ridge point：FP16 TC dense 989/3.35 ≈ 295，FP32 67/3.35 ≈ 20。对照三个算子：GEMM K=4096（AI 683）在两条线上都 compute-bound；GEMV （AI 0.5）与 softmax（AI 0.625）远低于 AI^*，天花板是带宽本身。

表 2：本机 RTX PRO 5000 (sm_120a, 2026-09-11 实测，来自 ch01_roofline.cu 属性查询)：110 SM，L2 96 MB，smem/block opt-in 99 KB，GDDR7 理论带宽 1344 GB/s（14 GHz × 384-bit × 2），FP32 峰值估算 66.9 TFLOPS（128 FMA/clk/SM 口径），ridge point ≈ 49.8 FLOP/Byte。后续各章 bench 都在这台机器上完成，请用这张表换算各章 kernel 「达到了理论带宽的百分之几」。特别注意：ch03 的 bench（§4.6）会在**只有 16 MB 数据**的情况下测出 1862 GB/s，超过 HBM 理论值，原因是数据集装进了 96 MB 的 L2，实测反映的是 L2 带宽。这里得出一条硬约束：**roofline 里的 BW 必须用「数据实际所在层级」的带宽**，小数据集的 bench 对 HBM 口径失真。

2.8 实践经验

本章对 base.cuh L1–86 注释逐条核查（F1 架构事实/F3 数学/F4 性能断言），以下两条记在此处，全量记录见 book/CHECKLOG.md：

实践经验 2.1 (F4)：L16–21 内存层级带宽数量级 「L2 约 12 TB/s、L1/SMEM 约 19 TB/s」属于微基准口径的数量级估计，随测量方法（读/写比例、跨 SM 通信模式）浮动明显；HBM 3.35 TB/s 则是 H100 SXM 的官方 spec 值。混排时建议在正文口径中分别标注「官方 spec」与「微基准量级」，避免读者按同一精度理解这组数字。

实践经验 2.2 (F3)：L89 GEMM 算例单位 注释「FLOPs = $2 \times 4096^3 \approx 137$ GFLOPS」中单位应为 GFLOP（运算量），FLOPS（每秒）是速率单位；该段实为 Roofline 的静态算例，与时间无关。精确值 $2 \times 4096^3/(3 \times 4096^2 \times 4) = 682.7$ FLOP/Byte，注释的 685 为四舍五入口径。

2.9　常见坑

1. **Bytes 漏算写回**。AI 分母必须包含输出写回（读 a, b 写 y 是 $3\times4N$ 字节，不是 $2\times4N$）；漏算会把 memory-bound 算成 compute-bound。

2. **sparse 峰值当 dense 用**。NVIDIA 规格表上带 sparsity 的 TFLOPS 是稠密口径的两倍；ridge point 差一倍，bound 类别可能判反（H100 FP16 TC：1979 sparse vs 989 dense，对应 AI^* 590 vs 295）。

3. **把 occupancy 当唯一 KPI**。occupancy 上限 100% 需要「少用寄存器/少用 smem」，但 GEMM 类 kernel 恰恰要用大 thread tile 与多级 smem 流水换性能；低 occupancy + 高 ILP 常优于高 occupancy + 低 ILP。occupancy 是手段，roofline 位置才是目标。

4. **用小数据集标定带宽**。16 MB 的输入永远命中 96 MB 的 L2，测出的「带宽」是 L2 的（ch03 实测 1862 GB/s > HBM 理论 1344 GB/s）。标定 HBM 带宽时数据集至少要数倍于 L2 容量。

5. **混淆 FLOP 与 FLOPS**。AI 的分子是运算量（FLOP），roofline 纵轴是速率（FLOP/s）；单位写错，量纲就全乱了（参见 §2.8 第二条）。

2.10　面试要点速查

- **Q：warp 是什么？为什么是 32？**
  A：最小调度与指令发射单元，32 线程锁步（SIMT）。32 与 warp scheduler 的每周期发射粒度、寄存器文件 bank 宽度匹配；发散（divergence）会让一个 warp 内的分支两条路径串行执行，等效指令数翻倍。

- **Q：怎么区分 memory-bound 与 compute-bound？**
  A：算 AI，与 ridge point $AI^* = P_{peak}/BW$ 比较：低于则带宽瓶颈，高于则算力瓶颈。优化方向完全不同：前者靠 coalescing/向量化/tiling，后者靠 Tensor Core/指令效率。

- **Q：为什么需要 occupancy？**
  A：延迟隐藏。GPU 不做分支预测与深级乱序，靠大量活跃 warp 之间的快速切换填满发射口；occupancy 低则切换不了，HBM 延迟（约 500–800 cycle）直接暴露。但 occupancy 受寄存器/smem/block 大小三重钳制，是权衡不是越大越好。

- **Q：寄存器溢出（spill）是什么？**
  A：每线程寄存器需求超过分配上限时，编译器把局部变量挪到 local memory（实际在 HBM/L1），访存量大幅增加、延迟暴露。用 `--ptxas-options=-v` 观察 spill 次数，出现即优先 reconsider thread tile 大小。

- **Q：L1 与 shared memory 的关系？**
  A：同一块片上 SRAM 的两种角色，可配置划分。L1 对所有访存透明缓存；smem 由程序员显式管理、block 内共享。Hopper 起 smem 上限约 228 KB/SM。

2.11　最小测试与动手实验

测试文件：book/tests/ch01_roofline.cu（host 侧演示，无 GPU kernel）。它做两件事：(1) 对 GEMM/GEMV/softmax/eadd/relu/dot 六个算子，按 §2.4 的口径计算 FLOPs、Bytes 与 AI，并与解析期望常数（682.67、0.5、0.625、1/12、1/8、0.25）核对；(2) 打印本机 GPU 的 SM 数、L2、smem 上限、GDDR7 理论带宽与 FP32 峰值估算，是全书 bench 的标尺来源。

编译与运行（CWD：book/tests/）：`./build_tests.sh --ch ch01`。

实测输出（RTX PRO 5000，sm_120a，2026-09-11；末行设备名照录自驱动返回，含架构代号 sm120）：

```
PASS GEMM     FLOPs=1.3744e+11 Bytes=2.0133e+08 AI= 682.6667
PASS GEMV     FLOPs=3.3554e+07 Bytes=6.7142e+07 AI=   0.4998
PASS softmax  FLOPs=2.0480e+04 Bytes=3.2768e+04 AI=   0.6250
PASS eadd     FLOPs=4.0960e+03 Bytes=4.9152e+04 AI=   0.0833
```

```
PASS relu     FLOPs=4.0960e+03 Bytes=3.2768e+04 AI=   0.1250
PASS dot      FLOPs=8.1920e+03 Bytes=3.2768e+04 AI=   0.2500
name=NVIDIA RTX PRO 5000 72GB Blackwell sm120 SMs=110
mem-theoretical-BW ~ 1344 GB/s   fp32-peak ~ 66.9 TFLOPS
ridge-point ~ 49.8 FLOP/Byte
```

动手实验：把 softmax 的 FLOPs 口径从 $5N$ 改成 $3N$（不含减法与除法的近似），观察 AI 变为 0.375，结论不变（仍然 memory-bound）：roofline 对 FLOPs 计数的容错性很高，AI 相差一倍通常不会翻转 bound 类别，但把分母的 Bytes 算漏一项就可能翻转。

延伸阅读

- @reed《GPU 指令集架构-前言》：zhuanlan.zhihu.com/p/686198447
- @reed《GPU 指令集架构-寄存器》：zhuanlan.zhihu.com/p/688616037
- @reed《GPU 指令集架构-Load 和 Cache》：zhuanlan.zhihu.com/p/692445145
- @ 紫气东来《CUDA（一）编程基础》：zhuanlan.zhihu.com/p/645330027
- @ 紫气东来《CUDA（二）内存体系》：zhuanlan.zhihu.com/p/654027980

第三章　归约原语：Warp/Block Reduce 与 Dot

3.1　本章导读

归约（reduce）是 GPU 上一切「多对一」计算的总称：求和、求 max、点积、softmax 的分母、LayerNorm 的均值方差、attention 里的 LSE 等，全部是归约的变体。归约写得好不好，直接决定上层算子的下限。本章拆解 base.cuh Phase 1（L87–305）的四类实现：warp 内蝶形归约（warp_reduce_sum / warp_reduce_max）、block 两级归约（block_reduce_sum / block_reduce_max）、跨 block 全局归约（block_reduce_all）以及它的直接应用点积（dot / dot_vec4）。

本章按「层层放大」的顺序来讲：先用 1 条 shuffle 指令在 warp 内归约 32 个数（§3.4.1），再用 shared memory 把 8 个 warp 合并成一个 block （§3.4.2），最后用 atomicAdd 把上万个 block 汇成一个全局结果（§3.4.3）。每一层引入新的硬件机制，也带出新的坑。

前置：第 二 章（warp/SIMT/延迟隐藏）。**源码区间**：base.cuh L87–305。**无编译宏限定**。

3.2　问题与动机

在串行世界里，归约是平凡的：一个 for 循环，N 个元素 $N-1$ 次加法。但这个循环是**严格串行的依赖链**：第 k 次加法必须等第 $k-1$ 次的结果。放进 GPU 的语境，问题就来了：无论你启动多少线程，一条长依赖链上的指令永远无法并行，延迟（数周期的 ALU + 数百周期的访存）完全无法隐藏。

归约优化的核心，就是把这条深链条**改造成浅树**：N 个数两两相加（$N/2$ 个并行加法），再对结果两两相加（$N/4$ 个），如此递推，树高 $\lceil \log_2 N \rceil$，1024 个数只要 10 层。树形化也有代价：「谁跟谁相加」变成了结构化的信息，硬件与代码都要按约定交换数据。本章的任务就是讲清这套约定在 GPU 三级存储（寄存器/shuffle、smem、global/atomic）上的三种实现。

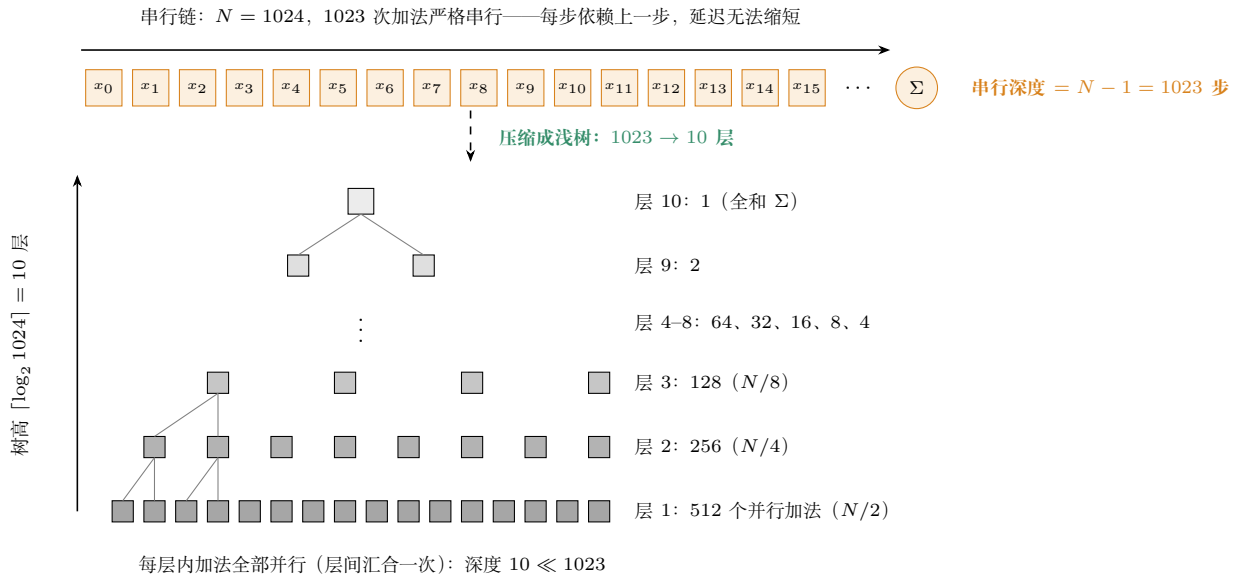


图 3.1: 串行依赖链与二叉归约树对照（$N = 1024$）：上半为串行链——1023 次加法严格依序，无论启动多少线程延迟都无法隐藏；下半为两两相加的归约树——树高 $\lceil \log_2 N \rceil = 10$ 层，每层内加法全部并行（灰度自底向上随节点数减半渐浅）。深度从 $N-1$ 压缩到 $\log_2 N$，代价是「谁跟谁相加」成为必须显式约定的结构信息。

3.3 数学原理

3.3.1 warp shuffle 与蝶形归约

CUDA 提供 warp 级寄存器交换指令 shuffle：同 warp 的线程可以**直接读对方寄存器**，不经过任何存储器。本章用到的两条：

- `__shfl_xor_sync(mask, var, laneMask, width)`：返回 srcLane = laneId $\oplus$ laneMask 线程的 `var` 值，其中 $\oplus$ 是按位异或；`width` 把交换限制在大小为 width 的 segment 内（lane 地址先对 width 取模再异或）。
- `__shfl_sync(mask, var, srcLane, width)`：所有线程读同一 srcLane 的值，用于**广播**。

蝶形归约（butterfly reduction）用 xor 指令构造两两相对的 lane 对。设 warp 宽度 $W = 2^R$（$R = 5$），第 r 轮（$r = 1, \dots, R$）的掩码为 $m_r = 2^{R-r}$，每个 lane 把自己的值与 lane $i \oplus m_r$ 的值相加。这个过程可以解析地刻画。定义 $S(r, i)$ 为「r 轮之后 lane i 持有的部分和」，则

$$S(r,i) \;=\; \sum_{j \bmod 2^{R-r} \,=\, i \bmod 2^{R-r}} x_j, \tag{3.1}$$

即 r 轮之后，lane i 持有**所有与它低 $R-r$ 位相同的原始元素之和**。用归纳法证明：$r = 0$ 时模 $2^R = W$ 的剩余类只含 i 自己，$S(0, i) = x_i$，成立。归纳步：第 r 轮掩码 $m_r = 2^{R-r}$，lane i 执行 $S(r, i) = S(r-1, i) + S(r-1, i \oplus m_r)$。按归纳假设，第一项是模 2^{R-r+1} 的剩余类 i 的和，第二项是剩余类 $i \oplus m_r$ 的和；这两个剩余类恰好是把模 2^{R-r+1} 的类 i 按第 $R-r$ 位切成两半，拼起来正是模 2^{R-r} 的剩余类 i。□

R 轮之后 $2^{R-R} = 1$，「模 1 相同」对所有 j 成立，于是 $S(R, i) = \sum_j x_j$，即**每个 lane 都持有完整的归约结果**（all-reduce 语义）。这个证明顺带给出 xor 成对交换的三条性质，源码注释（L144–147）把它们总结为：成对对称（$(i \oplus m) \oplus m = i$）、每轮一对一无冲突（$m \neq 0$ 时 $i \oplus m \neq i$）、距离逐轮减半而信息量翻倍。

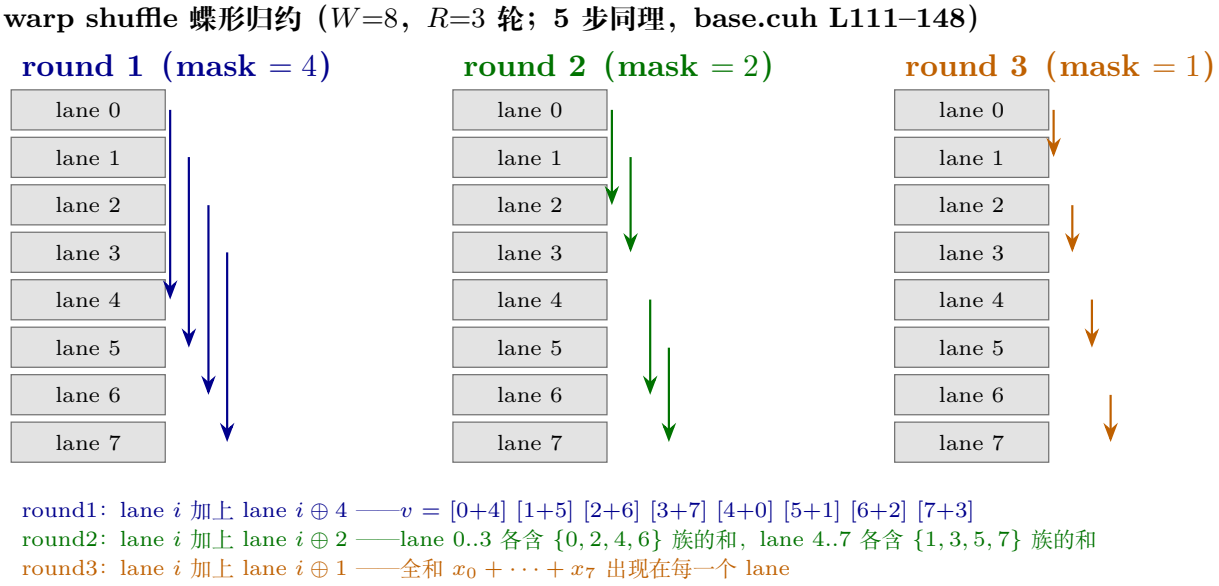


图 3.2: warp shuffle 蝶形归约：三轮 XOR 成对相加（mask $4 \to 2 \to 1$）后全和出现在每一个 lane（以 W=8 为例，5 步同理）。

为什么不用 `__shfl_down_sync`？down 模式下 lane i 每轮读 lane $i + m_r$，越界 lane 读自己（复制）。R 轮后只有 lane 0 凑齐全和，其余 lane 持有的是被「越界复制」污染的错误部分和；且每轮真正「有效」的 lane 数逐轮减半（第一轮只有一半 lane 的加法是有意义的），硬件却仍为全 warp 执行这些指令。xor 模式所有 lane 每轮都做有效加法、结束时全员拿到结果，正如 base.cuh L95 注释所说「xor 模式所有线程做相同工作量，更均衡」。后文 block 归约的广播步骤也因此免费：所有 lane 已经都有结果（sum 场景），或只需一次 shfl 广播（见 §3.4.2）。

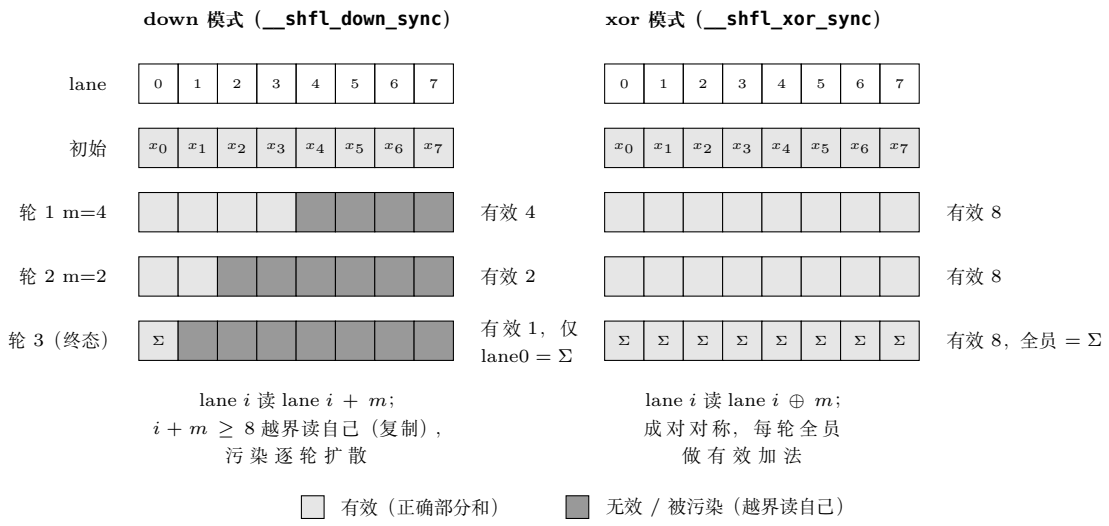


图 3.3: down 与 xor 两种 shuffle 逐轮对比（W=8，与图 3.2 同尺度，灰 20 = 有效 / 灰 80 = 无效污染）：down 模式 lane i 读 lane $i+m$，越界者读自己（复制），有效 lane 数逐轮减半（$4 \to 2 \to 1$），终态仅 lane0 持正确全和；xor 模式成对对称、每轮全员有效，终态 8 个 lane 全部持有 Σ（all-reduce）——block 归约的广播步骤因此不再需要。

3.3.2　Block 两级归约的步数统计

一个 block 有 T 个线程、$W = T/32$ 个 warp。两级归约的总开销：

$$\underbrace{\log_2 32}_{\text{warp 内 5 轮 shuffle}} + \underbrace{1}_{\text{smem 写 + sync}} + \underbrace{\log_2 W}_{\text{二次归约}} = 5 + \log_2 W \text{ 轮}, \tag{3.2}$$

$T = 256$（$W = 8$）时共 8 轮，外加一次 `__syncthreads()`。smem 流量只有 W 次写 + W 次读（各 4B）。对比「所有中间量走 smem」的朴素树形归约（$\log_2 T = 8$ 轮 smem 往返），两级结构把 smem 流量降了一个量级。这就是 ch01 图 ch01-1 那条规律的局部应用：**能在寄存器层完成的（warp 内 5 轮），绝不下放到 smem 层**。

3.3.3　跨 block 求和与浮点非结合性

$N \gg T$ 时需要多个 block 分摊元素，每个 block 产出部分和后，还差「把 G 个 block 的和汇成一个」的最后一步。base.cuh 用 `atomicAdd(y, val)`：G 次全局原子累加直接打进同一个地址 y。原子性保证结果**正确**，但有两个代价：其一，G 次原子操作在 L2 的原子单元上是**串行化**的（对同一地址的 read-modify-write 无法并行），§3.6 的 bench 显示这就是该 kernel 的真瓶颈；其二，浮点加法不满足结合律，G 个部分和的累加顺序取决于 block 完成顺序，**run-to-run 不确定**。这就是 ch02 测试用 CPU fp64 参考 + 容差（TOL_F32ACC）而不是 bitwise 比对的原因，也是所有「atomic 求和型」kernel （含 Part II 的 split-K GEMM）精度对齐时要记住的背景。

3.4　设计与实现

3.4.1　第一层：warp 内蝶形归约

先看入口。Phase 1 的开场注释（L87–95）把要点总结为三条：

- `warp_reduce`：用 `__shfl_xor_sync` 做蝶形归约，$O(\log N)$ 步，无需 shared memory；
- `block_reduce`：两级归约（warp → shared memory → warp0 broadcast），注意最后必须 broadcast 回所有线程（`__shfl_sync`），否则只有 warp0 知道结果；
- 为什么不用 `__shfl_down_sync`？xor 模式所有线程做相同工作量，更均衡。

再看实现：

base.cuh L105-110, L149-156（warp reduce sum：模板参数与实现）

```cpp
// Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
// 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
// 复杂度 O(logN)，N=32 时仅需 5 步
// 模板参数：T=数据类型，kWarpWidth=segment width（默认 32）
// kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width，限制 shuffle 在同一 segment 内
// 当 kWarpWidth < 32（如 FA 中 kWarpWidth=4）时，只有同 segment 的 lane 参与通信
template <const int kWarpWidth = kWarpSize, typename T = float>
__device__ __forceinline__ T warp_reduce_sum(T val) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
  }
  return val;
}
```

实现只有一行循环体，但模板参数的语义值得展开。`kWarpWidth` 既是循环起点（`mask = kWarpWidth >> 1`）也是 shuffle 的 `width` 实参：当它小于 32 时（如后续 FA 章节的 $W = 4$），异或被限制在 segment 内，相邻 segment 互不串扰，一个模板参数同时实现了「归约宽度」与「通信域」，这是本书第一处「用编译期参数把语义固化进指令」的示范。`mask` 从 $W/2$ 递减到 1，正是 §3.3 推导中 $m_r = 2^{R-r}$ 的代码形态；`#pragma unroll` 让 5 轮循环在编译期完全展开成 5 条独立 shuffle。warp reduce max（L158–166）与 sum 完全同构，只是把 `+` 换成 `max`，不再赘述。

源码注释里还附了一张以 $W = 8$ 为例的逐轮成对图（L111–148），它就是图 ch02-1 的底稿：初始时每个 lane 持有自己的值 $v_0 \dots v_7$，此后每一轮 lane i 与 lane $i \oplus \text{mask}$ 交换并累加，掩码 $4 \to 2 \to 1$ 逐轮减半，掩码归零后循环终止、归约完成。逐轮的成对相加与部分和演化如下（记 $\Sigma\{i_1, i_2, \dots\}$ 为括号内下标元素之和）：

轮次	mask	本轮成对元素	轮后各 lane 持有
初始	—	—	各持己值 $v_0, v_1, \dots, v_7$
1	4	(0,4) (1,5) (2,6) (3,7)	$v_0{+}v_4$, $v_1{+}v_5$, …, $v_7{+}v_3$（两两之和）
2	2	(0,2) (1,3) (4,6) (5,7)	偶数 lane 持 $\Sigma\{0,2,4,6\}$，奇数 lane 持 $\Sigma\{1,3,5,7\}$
3	1	(0,1) (2,3) (4,5) (6,7)	所有 lane 持全和 $v_0 + v_1 + \cdots + v_7$

注释末尾还总结了蝶形成对交换的关键性质：XOR 成对对称（$(i \oplus m) \oplus m = i$），每轮每个 lane 恰与 1 个其他 lane 通信，一对一无冲突；每轮信息传递距离减半（W=32 时 $16 \to 8 \to 4 \to 2 \to 1$）、信息量翻倍；全程 $O(\log_2 N)$ 步、纯寄存器操作、无需 shared memory；`__shfl_xor_sync` 的第四个参数 `kWarpWidth` 限制 segment width（如 W=4 时只有同 segment 的 lane 参与 shuffle），即前文模板参数所述的通信域行为。

3.4.2 第二层：block 两级归约

base.cuh L172-197（block reduce sum：注释与实现）

```cpp
// Block Reduce Sum — FP32（增强版，带 broadcast）
// 两级归约流程：
//   1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
//   2. warp leader (lane=0) 写入 shared memory
//   3. syncthreads 后，lane 0~kNumWarps-1 读取 shared memory
//   4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
//   5. __shfl_sync broadcast 到所有线程（关键！否则每个warp只有lane<kNumWarps 知道结果）
template <const int kNumThreads = 256>
__device__ float block_reduce_sum(float val) {
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  int warp = threadIdx.x / kWarpSize;
  int lane = threadIdx.x % kWarpSize;
  __shared__ float shared[kNumWarps];

  float value = warp_reduce_sum<kWarpSize>(val);
```

```
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < kNumWarps) ? shared[lane] : 0.0f;
  value = warp_reduce_sum<kNumWarps>(value);
  // 关键：broadcast 结果到所有线程，后续用 result
  // 做除法等操作时所有线程都能拿到
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}
```

对照 §3.3 的步数统计读这段代码：第 1 步 `warp_reduce_sum<kWarpSize>` 是 warp 内 5 轮；第 2 步 lane 0 把 warp 部分和写进 smem；`__syncthreads()` 划出读写边界：没有它，先写完的 warp 可能被后写的覆盖；第 3 步是本实现的关键：**不是让 warp 0 独自做二次归约，而是每个 warp 的前 W 个 lane 都读入全部 W 个部分和、各自再归约一遍**。多余的 $W-1$ 份计算是冗余的（只是寄存器加法），换来的是不需要第二次 smem 往返与第二次同步。第 4 步 `__shfl_sync(0xffffffff, value, 0, 32)` 把 lane 0 的结果广播给 **所有** 256 个线程。源码注释「关键！否则每个 warp 只有 lane 小于 kNumWarps 知道结果」说的就是这一步：归约函数的约定是「block 内全员可见」，后续 softmax 除以 ℓ、norm 除以 σ 都依赖它。block reduce max （L198–214）同构，初值从 0 换成 `-FLT_MAX`。

3.4.3　第三层：跨 block 全局归约与点积

base.cuh L216-247（block reduce all：多 block 求和 + atomicAdd）

```
// ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
// 多 block 各自做 warp→smem→warp0 reduce，然后 atomicAdd 到全局 y
// 跨 block 求和的常见模式，适合 N 较大时使用；
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/reduce/block_all_reduce.cu
template <const int kNumThreads = 256>
__global__ void block_reduce_all(float *a, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + tid;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float val = (idx < N) ? a[idx] : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  val = warp_reduce_sum<kWarpSize>(val);
  if (lane == 0)
    shared[warp] = val;
  __syncthreads();

  val = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0)
    val = warp_reduce_sum<kNumWarps>(val);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
    atomicAdd(y, val);
}

// ---- Dot Product: y = sum(a[i] * b[i]) ----
// 核心模式：elementwise 乘法 → block reduce → atomicAdd 全局累加
// Grid:  ((N + 255) / 256, 1, 1)
```

base.cuh L248-277（dot：elementwise 乘法 + block reduce + atomicAdd）

```
// Block: (256, 1, 1)
```

```cpp
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int kNumThreads = 256>
__global__ void dot(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + tid;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  prod = warp_reduce_sum<kWarpSize>(prod);
  if (lane == 0)
    shared[warp] = prod;
  __syncthreads();

  prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
    prod = warp_reduce_sum<kNumWarps>(prod);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
    atomicAdd(y, prod);
}

// Dot Product + float4
// Grid:  ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
// Block: (64, 1, 1), 256/4=64
// 注意：该版本默认输入地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/dot-product/dot_product.cu
```

两个 kernel 共享同一骨架：守卫式加载（`idx < N` 时取数、否则取加法单位元 0，这保证不满的最后一个 block 不污染求和；max 版对应 `-FLT_MAX`）→ warp 归约 → smem 汇合 → **warp 0** 二次归约 → **单个线程**写全局。与 block_reduce_sum 的两处差异值得逐条分析：其一，二次归约只留给 warp 0（`if (warp == 0)`），因为这里的结果只需被写出、不必广播全员；其二，写出者是 `tid == 0`，源码注释特意强调「tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加」：lane 0 是每个 warp 的第 0 线程（一个 block 里有 W 个），tid 0 才是全 block 唯一。多 block 之间则靠 `atomicAdd` 汇合，如 §3.3 所述。

dot 把「逐元素乘」并入加载阶段（`prod = a[idx] * b[idx]`），归约骨架保持不变。这也是本源码库反复出现的设计范式：**算子 = 元素级预变换 + 归约 + 后处理**，ch04 的 softmax（预变换 exp、后处理除法）与 ch06 的 norm（预变换平方）都是这个模板的具体实例。dot_vec4（L278–305）则把加载换成 `FLOAT4` 一次取 4 元素、每线程先局部乘加 4 项再进归约（ch03 详述向量化原理），线程数降为 $T/4$：

base.cuh L278-305（dot vec4：向量化加载版点积）

```cpp
template <const int kNumThreads = 256 / 4>
__global__ void dot_vec4(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * kNumThreads + tid) * 4;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float4 reg_a = FLOAT4(a[idx]);
  float4 reg_b = FLOAT4(b[idx]);
  float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
                            reg_a.z * reg_b.z + reg_a.w * reg_b.w)
                         : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  prod = warp_reduce_sum<kWarpSize>(prod);
  if (lane == 0)
```

```
    shared[warp] = prod;
  __syncthreads();

  prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0)
    prod = warp_reduce_sum<kNumWarps>(prod);
  if (tid == 0)
    atomicAdd(y, prod);
}
```

注意它的守卫只有 `idx < N` 而 FLOAT4 一次读 4 个元素，N 不是 4 的倍数时最后一个线程会越界读，这一隐患与处理策略见 §3.7 与 ch03。

3.5　图示

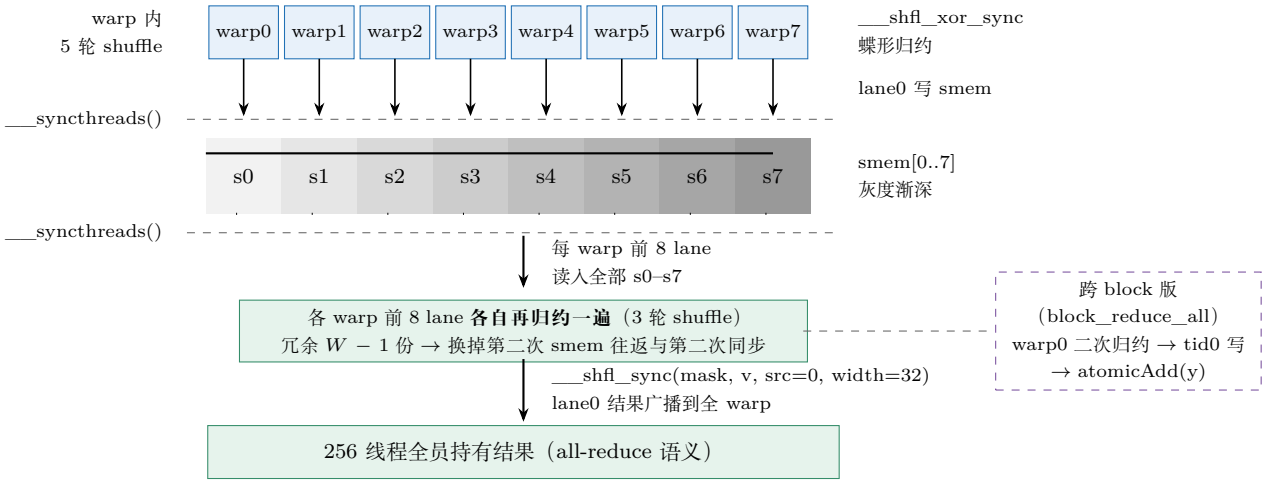


图 3.4: block 两级归约数据流

3.6　性能分析

Roofline 定性：归约每元素只有 1–2 FLOPs、读 4B（dot 读 a, b 共 8B），$AI \le 0.25$，明显落在 memory-bound 区；理想成绩 = 理论带宽。block_reduce_all 与 dot 额外带一个串行尾巴：G 次对同一地址的 atomicAdd。实测（RTX PRO 5000，sm_120a，2026-09-11，bench 口径：warmup 3 + 计时 10 次均值，N = 4,194,304，数据 16/32 MB 全部落在 96 MB L2 内，见 ch01 §2.7 关于 L2 口径的提醒）：

kernel	均值 (ms)	有效带宽
block_reduce_all	0.04164	402.93 GB/s
dot	0.04162	806.16 GB/s
dot_vec4	0.04155	807.53 GB/s

三行时间几乎完全相同（约 42 μs），说明瓶颈已经转移：32 MB 的 dot 数据流只要 32 MB/1344 GB/s ≈ 24 μs（HBM 口径）甚至更少（L2 口径），而实测 42 μs 由另一个因素决定：$G = N/256 = 16384$ 个 block 各做一次 `atomicAdd(y)`，对同一地址的全局原子串行化按 42 μs/16384 ≈ 2.6 ns/op 计，16384 次合计约 42 μs，恰好等于实测总时长。**结论：该实现的上限由全局原子的串行吞吐决定**；dot_vec4 把访存指令数降为 1/4（字节数不变），总时长却完全不变，正是因为访存早已不是短板。改进方向：把「每 block 一次全局原子」改成「两级 kernel：先每 block 写出自己的和（G 个地址，无冲突），再起一个小 kernel 归约 G 个数」，或 block 内先 smem 累积再少次原子，atomic 次数从 G 降到 $\lceil G/K \rceil$，时间随之线性下降。这也是面试里「跨 block 归约怎么优化」的标准答案骨架。

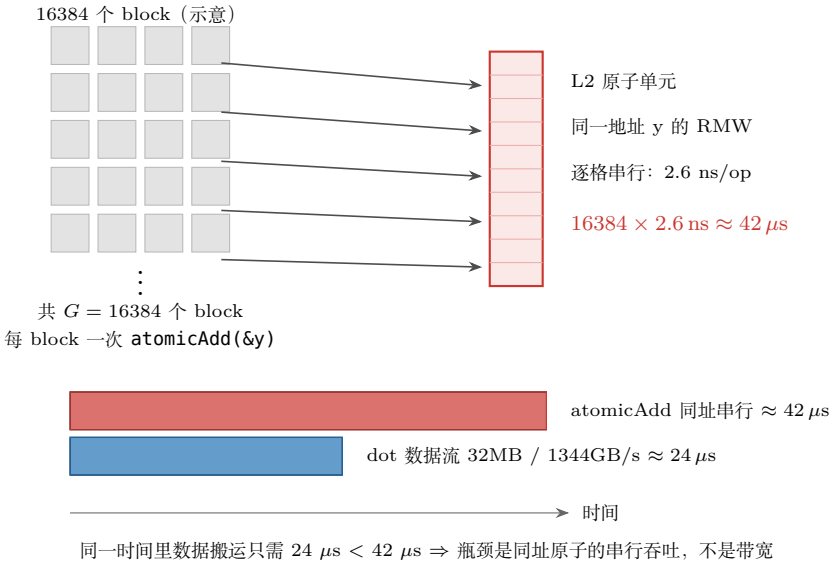


图 3.5: 全局原子串行化的量化证据：$G = 16384$ 个 block 的 atomicAdd 全部汇入同一地址 y 的 L2 原子单元窄队列逐格串行（2.6 ns/op，共约 42 μs），长于 dot 本身的数据流时间（约 24 μs）。

3.7 实践经验

实践经验 3.1 (F1)： L93–94 两级归约流程注释与实现不一致 面试要点注释写「block_reduce: 两级归约（warp → shared memory → warp0 broadcast）」。但 block_reduce_sum 的实现（L183–196）是**每个 warp 都**从 smem 读入全部部分和并各自二次归约，再用 `__shfl_sync` 从 lane0 广播回全员；「warp0 broadcast」更像 block_reduce_all 的流程（仅 warp0 归约、tid0 写出）。同文件 L172–181 的五步详细注释与实现一致，两处口径建议统一为「warp → smem → 各 warp 冗余二次归约 → lane0 经 shfl 广播」。

实践经验 3.2 (F1)： L278–284 dot_vec4 边界守卫不足（越界读隐患）注释「该版本默认输入地址满足 float4 对齐；最适合 N 按 4 对齐的场景」，但实现守卫只有 `idx < N`，而 `FLOAT4(a[idx])` 一次读 4 元素：N 非 4 倍数时最后一个线程读取 `a[idx..idx+3]` 的尾部越界（本测试 $N = 512$ 恰为 4 倍数故 PASS）。同文件 elementwise_add_vec4（L358–374）用 `(idx + 3) < N` 主路径 + 标量尾部处理，是安全范式，建议三处 vec4 kernel（dot/relu/eadd）统一。relu_vec4 有同样问题，详见 ch03。

3.8 常见坑

1. **漏掉 `__syncthreads()`**。smem 写与读之间没有同步，先完成的 warp 会读/覆盖到脏数据；且同步必须放在**所有线程都能到达**的位置（发散分支内的 syncthreads 会死锁）。

2. **归约后不广播**。block_reduce_sum 若删掉最后的 `__shfl_sync`，只有每个 warp 的前 W 个 lane 持有正确结果，后续「全员除以 sum」的代码会读到错误结果，而且这种错在小例子里可能不触发（warp0 的 lane 恰好是对的）。

3. **把 lane0 当 tid0 用**。跨 block 写出时用 lane0，一个 block 会写出 W 次，结果系统性偏大 W 倍（sum），源码注释专门用「tid == 0, not lane 0」提醒。

4. **二次归约的 width 传错**。第二次 warp_reduce 的模板参数是 `kNumWarps` 不是 `kWarpSize`；传错不会算错结果：smem 读取带 `lane < kNumWarps` 守卫，越界 lane 持单位元，但会多做 $\log_2 32 - \log_2 W$ 轮无效 shuffle，归约域也超出了有效数据范围。

5. **期望 atomic 求和的结果逐位可复现**。浮点加法非结合 + block 调度不定序 ⇒ run-to-run 抖动；对齐测试要么用容差，要么改用确定序的两级归约。

6. **warp 内归约后忘记处理 partial warp**。若活跃线程数不足 32 （如最后一个不满的 warp），参与归约的 lane 携带中性值即可（sum 用 0、max 用 `-FLT_MAX`），本实现的守卫式加载已处理；自行修改实现时别丢掉这一步。

3.9　面试要点速查

- **Q：warp reduce 为什么用 `__shfl_xor_sync` 而不是 down？**
  A：xor 成对对称、每轮全员有效、结束后 all-reduce（全员持有结果）；down 只有 lane0 有结果、每轮有效 lane 减半。见 §3.3 的不变量证明。

- **Q：写出一个 block 求和的完整步骤？**
  A：warp 归约（5 轮 shuffle）→ lane0 写 smem → syncthreads → 各 warp 前 W lane 读入再归约（$\log_2 W$ 轮）→ shfl 广播全员。$T = 256$ 共 8 轮 shuffle + 1 次同步。

- **Q：跨 block 归约有哪些方案，各自瓶颈？**
  A：atomicAdd 直写（瓶颈：同地址原子串行化，本机实测约 2.6 ns/op，见 §3.6）；两级 kernel（多一次 launch，无冲突）；网格级协作组或原子计数 + 最后 block 归约（threadfence 技巧）。

- **Q：`__shfl_xor_sync` 的 width 参数是干什么的？**
  A：限制 shuffle 的 segment 宽度，异或在 segment 内取模进行；用于子 warp 粒度的独立归约（FA 里 kWarpWidth=4 每组 4 lane 独立 softmax 统计量）。

- **Q：为什么归约测试不能用 bitwise 断言？**
  A：浮点加法非结合，原子汇合的加法顺序 run-to-run 不定；工程口径是 CPU fp64 参考 + 相对容差（本书 TOL_F32ACC=10^{-3}，实测该组 kernel 误差 10^{-7} 量级）。

3.10　最小测试与动手实验

测试文件：book/tests/ch02_reduce.cu，逻辑抽取自 bench/base.cu 的 test_block_reduce（L22）与 test_dot（L57），改用 common_test.h 的 CPU fp64 参考与 book_check 容差判定：$N = 512$ 随机 fp32，分别验证 block_reduce_all / dot / dot_vec4 对 CPU 求和与点积。

运行（CWD：book/tests/，`./build_tests.sh --ch ch02`）：

```
PASS block-reduce-all(N=512): max-err=7.161e-07 (tol=1.0e-03)
PASS dot(N=512):              max-err=4.480e-07 (tol=1.0e-03)
PASS dot-vec4(N=512):         max-err=9.248e-07 (tol=1.0e-03)
```

（输出 kernel 名按本书排版把下划线转写为连字符；实测 max_abs_err 均为 10^{-7} 量级，远低于 10^{-3} 容差，与 §3.3 的浮点抖动分析一致：512 项求和的 fp32 误差在 10^{-7}，跨 block 原子顺序抖动未放大误差。）

动手实验：(1) 把 block_reduce_all 的写回从 tid0 改成 lane0，观察结果放大约 8 倍（W 倍），体会「tid vs lane」；(2) 把 grid 减为 1 个 block 再跑 bench，观察时长从 42 μs 掉到远小于原值，验证瓶颈在 G 次全局原子而非归约本身；(3) 把 N 改成 510 再编译运行 dot_vec4，用 compute-sanitizer 的 memcheck 观察越界读（对应 §3.7 第二条）。

延伸阅读

- @ 懒蚂蚁呀不嘿《CUDA reduce 算子详解》：zhuanlan.zhihu.com/p/1905661893739283464
- @reed《GPU 指令集架构-Warp 级和 Uniform 操作》：zhuanlan.zhihu.com/p/712357647

第四章　向量化访存与原子操作

4.1　本章导读

逐元素（elementwise）算子是 GPU kernel 世界的「Hello World」：结构最简单，却浓缩了 GPU 性能工程的两条第一性原理：**coalescing**（访存合并）与 **向量化**（vectorized load/store）。前者决定「一次访存搬多少有效数据」，后者决定「一条指令搬多少数据」。本章以 base.cuh Phase 2（L306–386）的五个实现为线索：relu / relu_vec4（激活）、elementwise_add / elementwise_add_vec4（残差连接的原型）、histogram（原子操作）。

前四个 kernel 是「如何达到峰值带宽」的正反教材；histogram 则引入本章第二个关键机制 atomicAdd：它把访问模式从「读-算-写」扩展到「读-改-写」，ch02 末尾的跨 block 归约已经用到它，这里系统讨论它的代价。LLM 推理引擎里，激活、残差、直方图统计（如 MoE 路由计数）无处不在，这几个「小算子」在端到端延迟里的占比不小。

前置：第 二 章（roofline/带宽）、第 三 章（归约骨架，本章 vec4 版点积会回指）。**源码区间**：base.cuh L306–386。**无编译宏限定**。

4.2　问题与动机

先把这两个算子放上 roofline（口径沿用 ch01）：relu 每元素读 4B 写 4B、做 1 FLOP，$AI = 1/8 = 0.125$；elementwise_add 读 a, b 写 c 共 12B、做 1 FLOP，$AI = 1/12 \approx 0.083$。对照本机 ridge point（约 49.8 FLOP/Byte），它们是纯粹的 memory-bound 算子，性能上限就是带宽本身。于是问题转化为：怎么让每一个字节、每一条访存指令都发挥最大作用？

两个互补的杠杆。其一是 coalescing：硬件以 32B sector 为粒度搬运数据，若一个 warp 的 32 个线程访问的是连续地址，128B 恰好铺满 4 个 sector，零浪费；反之若地址发散，最坏要搬 32 个 sector 共 1024B，有效仅 128B，效率 12.5%。其二是向量化：用 float4 让一条指令搬 16B，指令数降为 1/4。这两个杠杆都不改变算法的 FLOPs 与 Bytes（roofline 位置不变），改变的是 **实际带宽向理论带宽的逼近程度**，这正是 memory-bound 算子优化的全部内容。

而 histogram 提出另一个问题：当多个线程要**写同一个位置**时怎么办？朴素方案会丢失更新（lost update）；atomicAdd 给出正确性保证，代价是冲突串行化。ch02 的 bench 已经量化过全局原子的代价（约 2.6 ns/op），本章在一个更极端的场景（256-bin 直方图、四百万次插入）里进一步量化它。

4.3　数学原理

4.3.1　Coalescing：sector 与事务合并

现代 NVIDIA GPU（Volta 起）的访存以 **32 字节 sector** 为最小搬运粒度，L2 与 HBM 之间以 32B sector、L1 与 L2 之间以更大粒度的 cache line 组织。当一条访存指令在一个 warp（32 线程）上执行时，硬件把 32 个线程的地址按 sector 归类合并（coalesce），每个被触及的 sector 生成一次搬运请求。定义一次 warp 访存的**有效率**：

$$\eta \;=\; \frac{\text{线程实际读写的字节数}}{\text{被搬运的 sector 总字节数}} \;=\; \frac{32 \times b}{S \times 32}, \tag{4.1}$$

其中 b 是每线程访问的字节数（fp32 标量 $b = 4$），S 是 32 个地址触及的 sector 数。三种典型情形（图 ch03-1）：

- **连续（coalesced）**：32 个 4B 地址恰好铺满 $[base, base + 128)$，$S = 4$，$\eta = 128/128 = 100\%$。
- **stride 为 2**：地址铺满 256B 跨度但只用一半，$S = 8$，$\eta = 50\%$，有效带宽直接减半。

- **大 stride**：每线程落在不同 sector，$S = 32$，$\eta = 128/1024 = 12.5\%$（若按 L1↔L2 的 128B cache line 口径则为 $128/4096 \approx 3\%$），矩阵转置的朴素版本（ch07 专题）正是这种形态。

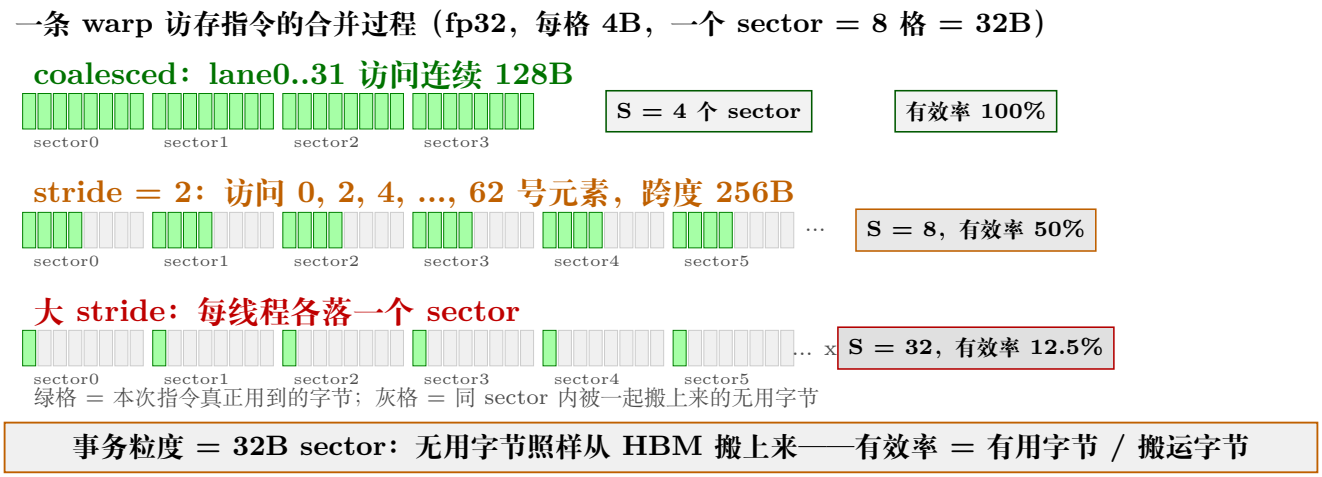


图 4.1: coalescing 与非合并访存对比（fp32，每格 4B，一个 sector 8 格 32B）：coalesced 一条指令落 4 个 sector 有效率 100%；stride=2 落 8 个、50%；大 stride 每线程各占一个 sector、仅 12.5%——事务粒度是 32B sector，无用字节照样从 HBM 搬上来。

一维排布的逐元素算子天然满足连续性：线程 t 处理元素 idx = blockIdx $\times T + t$，warp 内 32 个连续 idx 对应连续 4B 地址。这就是源码注释「grid/block 维度设计：grid(N/threads), block(threads)，一维即可」背后的全部数学：**只要 idx 映射是线性的，coalescing 是免费的**；它昂贵的时候，是高维张量的维度交错（ch07 转置、ch16 的 BHND 布局）。

4.3.2　向量化：一条指令 128 bit

向量化把每线程的处理粒度从 1 个标量提升到 4 个（float4）。对 N 个元素、block 大小 T：

$$\text{标量版：}\ \frac{N}{T}\ \text{线程} \times 2\ \text{条访存指令}, \qquad \text{vec4 版：}\ \frac{N}{4T}\ \text{线程} \times 2\ \text{条访存指令}, \tag{4.2}$$

warp 级访存指令数同为 $N/32$ 的倍数，但每元素摊得的访存指令数从 2 降到 1/2（线程数与总指令数降为 1/4，每线程仍是 1 条 128-bit load + 1 条 128-bit store），load/store 指令发射压力降为 1/4，同时每线程拿到 4 个独立的计算输入，ILP（指令级并行）翻两番。地址也依然连续：vec4 线程 t 处理 $[4t, 4t+3]$，warp 的 32 个线程覆盖连续 512B = 16 个 sector，η 仍是 100%。可见**向量化与 coalescing 正交**：前者减指令，后者减无效字节；最好的情况是两者兼得：一条 128-bit load 恰好铺满 4 个整 sector。

向量化的代价是**对齐约束**。float4 类型要求 16B 对齐：cudaMalloc 返回的基址满足 256B 对齐，因此只要**元素偏移 idx 是 4 的倍数**（vec4 版 idx = 4× 全局线程号，天然满足），地址即对齐。反过来，若 N 不是 4 的倍数，最后一组 `FLOAT4(a[idx])` 的 4 元素读取会越过数组末尾：源码里两个 vec4 kernel（relu/dot）只守卫 `idx < N`，存在越界读隐患，一个（elementwise_add_vec4）用 `idx + 3 < N` 加标量尾部兜底，见 §4.7 的逐条核查。

4.3.3　原子操作与直方图的冲突量

histogram 要维护 $y[a_i] \mathrel{+}= 1$：读 $y[a_i]$、加 1、写回，即 read-modify-write（RMW）。若两个线程同时 RMW 同一 bin 且无同步，两次加法会丢失一次（lost update）。atomicAdd 把 RMW 变成硬件级的不可分割操作：在 L2 的原子处理单元上排队执行，同地址操作**串行化**，不同地址并行。串行化的代价可以直接建模：设一次全局原子延迟约 τ（ch02 实测约 2.6 ns），若全部 N 次插入都落在同一个 bin，吞吐上限为 $1/\tau$；若 B 个 bin 均匀承压，理论上限约 B/τ op/s。真实情形介于两者之间，且 L1 侧的 atomic 路径与请求排队会进一步压低实测值：§4.6 的 bench 会看到，4M 次 256-bin 插入实测有效带宽只有 26.7 GB/s，比相邻的 elementwise_add（2654 GB/s）低**两个数量级**。这也解释了为什么生产级实现都要做 smem 私有化：先在 block 内用 smem 直方图（冲突域缩小到 block 内、且 smem 原子远快于全局），最后每个 block 对全局做 B 次合并写，全局原子总次数变为 $G \times B$（$G = N/T$ 为 block 数），仅当 $T > B$ 时才低于 N；$T \le B$ 时次数不变，收益来自 smem 原子远快于全局原子与写模式去热点。

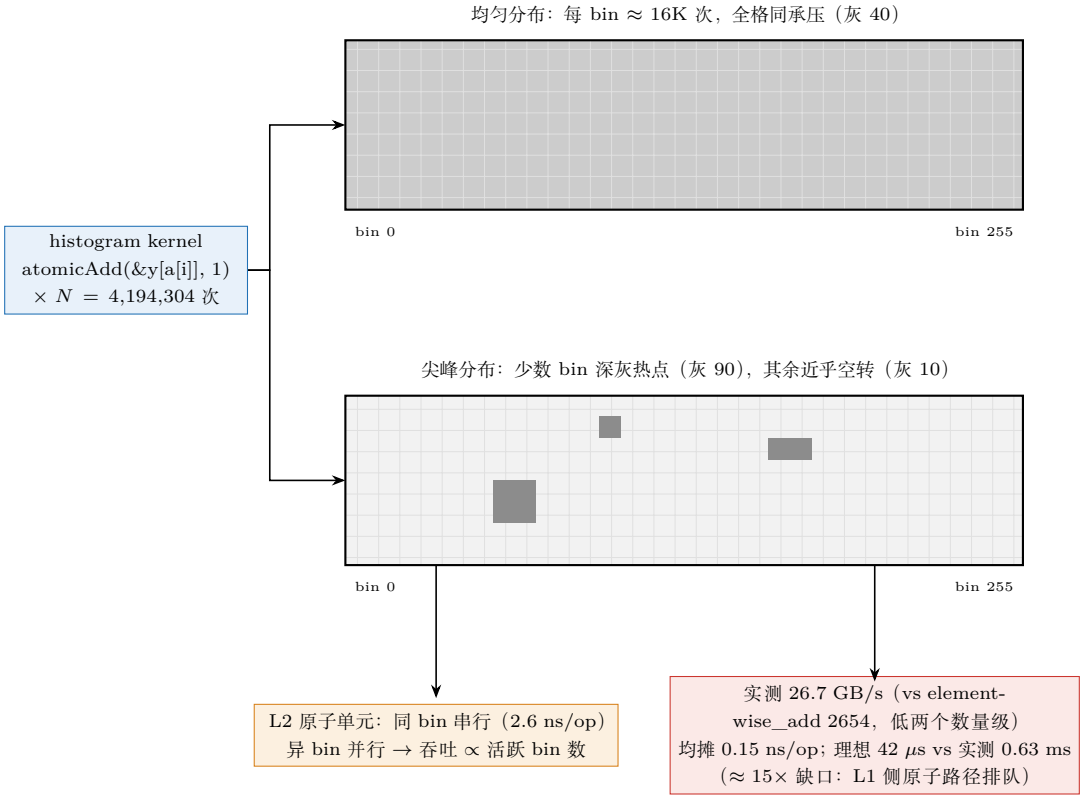


图 4.2: 256-bin 直方图的原子冲突热力图（bin 排成 8×32 网格示意）：4M 次 atomicAdd 从左侧注入；上格均匀分布时各 bin 等承压（灰 40），下格尖峰分布时少数 bin 形成深灰热点（灰 90）、其余近乎空转（灰 10）——bin 分布决定同址串行化程度。下方案框汇总实测：26.7 GB/s 比 elementwise_add 的 2654 GB/s 低两个数量级，均摊 0.15 ns/op；按「256 bin 理想并行 + 同址 2.6 ns 串行」估算理想 42 μs，实测 0.63 ms（约 15 倍缺口来自 L1 侧原子路径排队）。

4.4 设计与实现

Phase 2 的开场注释给出了本章的考点清单：

- 逐元素操作是最简单的 kernel，核心考点是 memory coalescing；
- `float4` 向量化可将内存事务数减为 1/4，大幅提升 bandwidth utilization；
- grid/block 维度设计：`grid(N/threads)`、`block(threads)`，一维即可。

4.4.1 relu：标量基线与 vec4 对照

base.cuh L314-322（relu：标量版）

```
// ---- ReLU: y = max(0, x) ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu(float *x, float *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    y[idx] = fmaxf(0.0f, x[idx]);
}
```

三行代码，两条考点：一维 idx 映射保证 warp 内连续 coalesced；`fmaxf(0.0f, x)` 表达的是「逐元素无分支」：若写成 `if (x > 0) ... else ...`，warp 会因 lane 间条件不同而发散（divergence），两条路径串行执行；`fmaxf` 编译为单条 `max.f32` 指令，无发散。**逐元素算子的分支要写进指令选择而不是控制流**，这个原则在 ch04 的 `__expf` 选择、ch06 的 rsqrt 中反复出现。

base.cuh L324-342（relu vec4：单条 128-bit load/store）

```cpp
// ReLU + float4 向量化：每个线程处理 4 个元素，减少 75% 的 load/store 指令
// block(64)×4(float4)=256 元素/block，与基础版吞吐相同
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1)
// 注意：该版本默认地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu_vec4(float *x, float *y, int N) {
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  if (idx < N) {
    float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
    float4 reg_y;
    reg_y.x = fmaxf(0.0f, reg_x.x);
    reg_y.y = fmaxf(0.0f, reg_x.y);
    reg_y.z = fmaxf(0.0f, reg_x.z);
    reg_y.w = fmaxf(0.0f, reg_x.w);
    FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
  }
}
```

`FLOAT4(x[idx])` 是 common.cuh 的宏：把任意标量引用重解释为 float4 引用，编译为一条 `LDG.E.128`；`FLOAT4(y[idx]) = reg_y` 同理是一条 `STG.E.128`。四个 fmaxf 是 4 条独立的寄存器指令，它们与 load/store 的比例恰好说明：**向量化后指令瓶颈从访存侧转移到了计算侧**，这正是「AI 低」算子的理想结构。

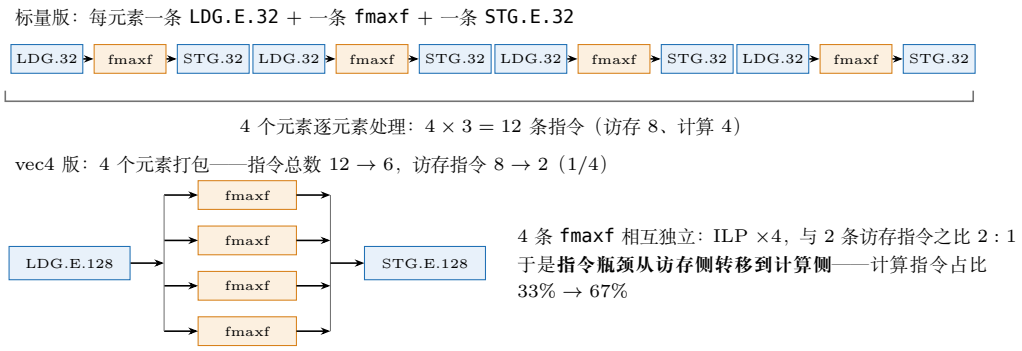


图 4.3: 标量与 vec4 的指令流对照(relu，以 4 个元素为例)：标量版每元素 3 条指令(`LDG.E.32` → `fmaxf` → `STG.E.32`)，4 元素共 12 条、其中访存 8 条；vec4 版用一条 `LDG.E.128` 取回 4 个元素，4 条 `fmaxf` 互不依赖（ILP ×4）后由一条 `STG.E.128` 写回，总数降到 6 条、访存只剩 2 条。访存 : 计算从 8 : 4 变为 2 : 4，指令瓶颈因此从访存侧移到计算侧。

4.4.2　elementwise add：安全范式

base.cuh L343-374（elementwise add：标量版与 vec4 安全版）

```cpp
// ---- Elementwise Add: c = a + b ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add(float *a, float *b, float *c, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    c[idx] = a[idx] + b[idx];
}

// Elementwise Add + float4 向量化
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
// 注意：主路径要求 4 元素对齐；尾部不足 4 个元素时回退到标量处理
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
```

```
  int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
  if ((idx + 3) < N) {
    float4 reg_a = FLOAT4(a[idx]);
    float4 reg_b = FLOAT4(b[idx]);
    float4 reg_c;
    reg_c.x = reg_a.x + reg_b.x;
    reg_c.y = reg_a.y + reg_b.y;
    reg_c.z = reg_a.z + reg_b.z;
    reg_c.w = reg_a.w + reg_b.w;
    FLOAT4(c[idx]) = reg_c;
  } else if (idx < N) {
    for (int i = 0; (idx + i) < N; i++) {
      c[idx + i] = a[idx + i] + b[idx + i];
    }
  }
}
```

vec4 版的守卫值得逐行读：主路径 `(idx + 3) < N` 一次性保证 4 个元素都在界内；否则进入标量尾部循环 `for (i = 0; idx + i < N; i++)` 逐个补齐。这个「主路径快、尾路径对」的二段式是向量化 kernel 的通用范式（Part II 的 GEMM 主 K 循环 + 尾部处理同构）。它同时示范了另一种工程判断：`else if (idx < N)` 里 idx 不足 4 时（如 N=510，idx=508）循环最多跑 2 次，正确但慢：**尾部牺牲性能换正确性**，主路径保住 99% 的流量。

4.4.3 histogram：原子操作的应用

base.cuh L376-385（histogram：atomicAdd 计数）

```
// ---- Histogram: y[a[i]]++ ----
// 演示 atomicAdd 的用法：多个线程可能同时更新同一个 bin
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/histogram/histogram.cu
__global__ void histogram(int *a, int *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    atomicAdd(&(y[a[idx]]), 1);
}
```

一行有效代码。`atomicAdd(&(y[a[idx]]), 1)` 把 RMW 交给硬件：地址相同则串行、不同则并行，正确性由硬件保证。按 §4.3 的冲突量估算：bin 分布越尖（少数 bin 承压越多），串行化越严重。调用约定也别忽略：输入值必须落在 $[0, B)$ 内，越界即写坏全局内存：kernel 层不检查（性能优先），这是调用方的责任，也是「kernel 信任调用方」设计哲学的一个例子。

4.5 图示

图 ch03-1（coalescing 对比）已见 §4.3。补充一张三条优化路线的「收益分解」图，把本章与 ch02 的实测串起来：

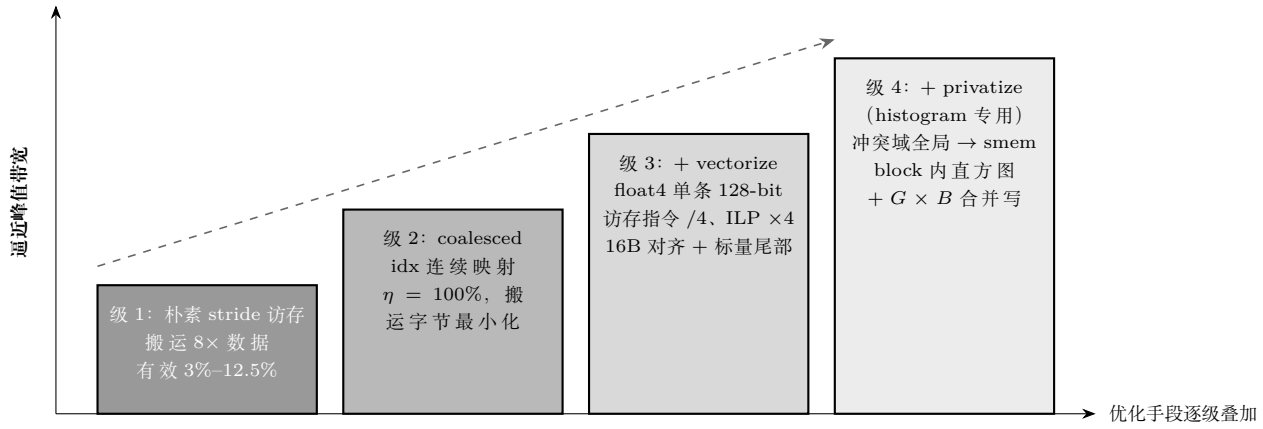


图 4.4: 逐元素算子的四级优化阶梯：朴素 stride → coalescing → 向量化 → smem 私有化，纵轴为逼近峰值带宽的程度，灰度越浅越接近上限（级 4 为 histogram 专用的原子冲突域收缩）。

4.6　性能分析

Roofline 定性：relu（AI 0.125）与 elementwise_add（AI 0.083）都是极深 memory-bound 区的算子，性能分数只看「实测带宽 / 数据所在层级的理论带宽」。实测（RTX PRO 5000，sm_120a，2026-09-11；bench 口径：warmup 3 + 计时 10 次均值，N = 4,194,304，输入 16 MB 全部落在 96 MB L2 内，故实测带宽是 **L2 口径**而非 HBM 口径，见 ch01 §2.7）：

kernel	均值 (ms)	有效带宽	口径
relu	0.01802	1862.15 GB/s	读 + 写 8B/元素
relu_vec4	0.01798	1866.46 GB/s	同上
elementwise_add	0.01897	2653.73 GB/s	两读一写 12B/元素
elementwise_add_vec4	0.01927	2611.86 GB/s	同上
histogram (256 bins)	0.62860	26.69 GB/s	按读 4B/元素计，原子另计

（以上为与 ch02 相同口径的 bench 复测结果。）三条观察：

其一，L2 命中让实测超过 HBM 理论值。1862 GB/s > 本机 GDDR7 理论 1344 GB/s：数据 16 MB 从未离开 96 MB 的 L2，实测是 L2 到 SM 的带宽。这是 ch01「BW 用数据实际所在层级」提醒的实证；若要测 HBM 口径，应把 N 放大到数倍于 L2。

其二，vec4 与标量在 L2 口径下几乎打平（1862 vs 1866、2654 vs 2612，差异在 ±2% 噪声内）。因为此时瓶颈在 L2/互联的搬运能力而非指令发射：4 倍的指令缩减没有释放空间。但不要据此断言 vec4 无用：当数据集大于 L2（HBM 直读）、或 block 数受 grid 限制（指令发射成为瓶颈）时，vec4 的 1/4 指令量与 4 倍 ILP 优势才会显现。工程结论是**先看清瓶颈层级，再选优化手段**。

其三，histogram 比 elementwise 慢两个数量级（26.7 vs 2654 GB/s，0.63 ms vs 0.019 ms）。4M 次原子均摊每次约 0.63 ms/4.2M ≈ 0.15 ns，单看均摊延迟并不差（256 个 bin 提供了并行度，反而低于 ch02 同址全局求和的 2.6 ns/op），慢在总量：按「256 bin 理想并行 + 同址 2.6 ns 串行」模型，理想耗时约 $4.2\text{M}/256 \times 2.6\,\text{ns} \approx 42\,\mu\text{s}$，实测 0.63 ms 慢了约 15 倍，缺口来自 4M 个原子请求在 L1 侧 atomic 路径的排队与串行化。优化方向即 §4.3 的 smem 私有化：全局原子次数 $N \to G \times B$（G 为 block 数、B 为 bin 数）。注意本配置 T=B=256 时 $G \times B = N$，次数并不下降，收益来自 smem 原子远快于全局原子与写热点消除，预期可把 0.63 ms 压低一个数量级以上，留作动手实验。

4.7　实践经验

实践经验 4.1 (F1)： L311「内存事务数减为 1/4」口径过时 注释称「float4 向量化可将内存事务数减为 1/4」。Volta 起访存以 32B sector 为粒度：coalesced 场景下搬运的 sector 数由总数据量决定（每元素都要搬一次），向量化并不减少 sector 事务数；它减少的是**每线程 load/store 指令数**（4 条 32-bit → 1 条 128-bit）。「事务数减为 1/4」只在 Fermi/Kepler 的 128B 事务口径下才成立，建议改为「访存指令数减为 1/4」。

实践经验 4.2 (F1)：L324–329 relu_vec4 边界守卫不足（越界读隐患）与 ch02 的 dot_vec4 有同样问题：守卫只有 `idx < N`，而 FLOAT4 一次读 4 元素，N 非 4 倍数时最后一个线程越界读 `x[idx..idx+3]`。注释「最适合 N 按 4 对齐的场景」是隐含免责，未明示越界后果。同文件 elementwise_add_vec4（L358–374）的 `(idx + 3) < N` 主路径 + 标量尾部是安全范式，建议统一。

实践经验 4.3 (F4)：L325「与基础版吞吐相同」表述不准 「block(64)×4(float4)=256 元素/block，与基础版吞吐相同」：相同的是**每 block 处理规模**（256 元素）；吞吐（带宽利用率）两者通常不同，取决于瓶颈层级（本机 L2 口径下两者差异在 ±2% 噪声内，见 §4.6）。建议改为「每 block 处理元素数相同」。

4.8 常见坑

1. **float4 未对齐**。基址非 16B 对齐时（例如对子数组 `a + 1` 做 FLOAT4）触发 CUDA misaligned address 运行时错误。约定：vec4 主路径的 idx 必须是 4 的倍数、指针必须来自整块分配的基址。
2. **N 非 4 倍数时用「浅守卫」vec4 kernel**。relu_vec4 / dot_vec4 的 `idx < N` 不覆盖 4 元素读取的尾部（见 §4.7）；生产代码应统一为 elementwise_add_vec4 的二段式守卫。用 compute-sanitizer memcheck 可以稳定复现越界。
3. **histogram 忘记清零**。atomicAdd 是「加在旧值上」，复用缓冲区时若不 memset，两轮结果会叠加，本测试每轮 bench 前显式 memset。
4. **把原子当免费**。全局原子吞吐有上限（同址串行化），热点地址会让 kernel 慢百倍（§4.6 实测 99 倍差）。先问「冲突域能不能缩小」：smem 私有化、分桶、thread-warp 内先归约再提交。
5. **原子还破坏确定性**。与 ch02 的浮点 atomicAdd 同理，冲突顺序不定 ⇒ 累加型原子的结果 run-to-run 抖动；int 计数（histogram）无此问题，浮点累加要留意。
6. **grid 尺寸截断**。grid 计算要向上取整 `(N + T - 1) / T`，配合 kernel 内 `idx < N` 守卫；漏守卫或漏取整都会越界。

4.9 面试要点速查

- **Q：什么是 coalescing？怎么保证？**
  A：warp 的 32 个地址按 32B sector 合并搬运；保证手段是让相邻线程访问相邻地址（一维线性 idx 映射），高维场景靠布局分析与 padding/swizzle （ch07/ch12）。
- **Q：float4 向量化的收益来源？**
  A：访存指令数减为 1/4（单条 128-bit load/store）、每线程 ILP 增至 4；前提是 16B 对齐。收益大小取决于当前瓶颈是发射带宽还是数据搬运带宽（本机 L2 口径下收益不足 2%，HBM 直读场景更大）。
- **Q：histogram 为什么慢？如何优化？**
  A：RMW 原子在冲突 bin 上串行化。优化：smem 私有直方图（冲突域缩到 block 内）+ 全局合并（每 block 每 bin 一次合并写，总次数 $(N/T) \times B$，仅当 $T > B$ 时下降）；进一步可按 bin 分块或用 warp 级聚合。
- **Q：atomicAdd 的硬件代价模型？**
  A：L2 原子单元排队 RMW，同地址严格串行（本机实测全局求和约 2.6 ns/op）、不同地址并行；吞吐上限约 B/τ（B 为活跃地址数）。
- **Q：逐元素 kernel 怎么打到峰值带宽？**
  A：连续 idx 映射（coalesced）+ 足够多的线程覆盖访存延迟（occupancy）+ vec4 降指令量 + 尾部安全处理；最后用「实测带宽/所在层级理论带宽」衡量（本机 L2 口径实测 1862–2654 GB/s）。

4.10 最小测试与动手实验

测试文件：book/tests/ch03_elementwise.cu，逻辑抽取自 bench/base.cu 的 test_relu（L108）/ test_elementwise（L153）/ test_histogram（L201），改用 common_test.h 判定：$N = 512$ 验证 relu/relu_vec4 与 CPU 逐元素精确一

致（误差 0）、elementwise_add 两版误差 6×10^{-8}、histogram（16 bins）与 CPU 计数完全一致；另附 5 个 kernel 的 cudaEvent bench（warmup 3 + 10 次均值）。

运行（CWD：book/tests/，`./build_tests.sh --ch ch03`；kernel 名按本书排版把下划线转写为连字符，实测 max_abs_err 见各行）：

```
PASS relu(N=512):              err=0.000e+00 (tol=1.0e-03)
PASS relu-vec4(N=512):         err=0.000e+00 (tol=1.0e-03)
PASS elementwise-add(N=512):   err=5.960e-08 (tol=1.0e-03)
PASS elementwise-add-vec4:     err=5.960e-08 (tol=1.0e-03)
PASS histogram(BINS=16,N=512): err=0.000e+00 (tol=1.0e-03)
```

动手实验：(1) 把 bench 的 N 从 2^{22} 放大到 2^{26}（256 MB > L2），观察 relu 带宽从 1862 GB/s 回落到 HBM 口径，亲手验证 ch01 的「层级带宽」提醒；(2) 给 histogram 加 smem 私有直方图（`__shared__ int cnt[B]` + block 内原子 + 合并），对比 0.63 ms 基线；(3) 把 relu_vec4 的 N 改成 510，用 compute-sanitizer memcheck 复现越界读（对应 §4.7 第二条）。

延伸阅读

- @ 懒蚂蚁呀不嘿《CUDA element-wise 算子详解》：zhuanlan.zhihu.com/p/1888630735520391519
- @ 紫气东来《ops(5) 激活与残差》：zhuanlan.zhihu.com/p/695703671
- @reed《GPU 指令集架构-程序控制和原子操作》：zhuanlan.zhihu.com/p/712357443

第五章　Softmax 三级递进：从 naive 到 online

5.1　本章导读

Softmax 是深度学习里使用最频繁的归一化算子：语言模型输出头的概率分布、注意力机制中的 softmax($QK^\top/\sqrt{d}$)（第 十六 章的核心算子）、MoE 的门控权重，都离不开它。它的数学定义只有一行，但在 GPU 上写对一个**又快又稳**的 softmax，要同时解决两个问题：**数值稳定性**（e^x 的溢出）与**归约遍数**（多一遍全局扫描就多一倍同步与访存）。本章沿着教学源码的三级递进，依次解决这两个问题：

- **Level 1 naive**（base.cuh L544，`softmax_per_token`）：直接按定义实现，演示溢出风险与最基本的 block 归约用法；
- **Level 2 safe**（base.cuh L564，`safe_softmax_per_token`）：减 max 的两遍扫描（2 次 block reduce），数值稳定的工业标配形态；
- **Level 3 online**：`online_safe_softmax_per_token`（base.cuh L632）：(m, ℓ) 双状态单遍联动更新（1 次融合归约），是 FlashAttention online rescale 的数学内核。

前置章节：第 三 章的 warp/block 归约原语，本章的三版实现分别建立在其 `block_reduce_sum`、`block_reduce_max` 与 XOR 蝶形归约之上；不熟悉 `__shfl_xor_sync` 蝶形归约的读者请先回看第 三 章。**源码区间**：base.cuh L520–665 (Phase 3a)，无编译宏限定，全部架构适用；配套最小测试为 `book/tests/ch04_softmax.cu`。**后继**：本章 4.3 节推导的 online 递推与二元合并公式，将在第 六 章以 LSE 的形式被「压缩」成一个标量状态，并在第 十六 章发展为 FlashAttention 的 online rescale。

5.2　问题与动机

动机一：e^x 会溢出。直觉上 softmax 的输入都是「分数」，量级应该有限，但 attention 里，logit 是 Q、K 行向量的内积，量级随训练状态与序列位置漂移；语言模型 logits 在训练早期破百也不罕见。而 fp32 能表示的最大正规数约 3.4×10^{38}，e^{89} 就已经越界变成 `inf`。一旦分子或分母出现 `inf`，输出就是 `inf/inf` 型的 `NaN`，模型输出错误结果。所以「减 max」不是可选的优化技巧，而是正确性的底线。

动机二：每多一遍扫描，就多一遍代价。教科书的 safe softmax 要对输入做三次遍历：求 max、求指数和、再除一遍写回。每次「遍历」在 GPU 上至少意味着一遍全局内存读取，或者一次 block 级同步归约。本章将看到：三版实现的差距就在**全局扫描次数**上（3-pass → 2-pass → 1-pass），图 5.3 给出三者的数据流对比。

动机三：softmax 的「行内自足性」。softmax 沿一行（一个 token、一个 query）独立计算，行与行之间零依赖。这意味着一行数据可以完整驻留在一个 block 的寄存器/共享内存里，为「一遍扫描」的 online 形态提供了条件：这也是它后来能被 FlashAttention 纳入「KV 分块流水线」的根本原因：分块后的部分结果可以用一个 (m, ℓ)（或 LSE）状态无损概括。

5.3　数学原理

5.3.1　定义与平移不变性

设一行输入 $x^{(1)}, x^{(2)}, \ldots, x^{(n)}$，softmax 的定义为

$$y^{(i)} \;=\; \mathrm{softmax}\big(x\big)^{(i)} \;=\; \frac{e^{x^{(i)}}}{\sum_{j=1}^{n} e^{x^{(j)}}}, \qquad i = 1, \ldots, n. \tag{5.1}$$

输出 $y^{(i)} \in (0,1)$ 且 $\sum_i y^{(i)} = 1$。对任意常数 c，softmax 具有**平移不变性**：

$$\frac{e^{x^{(i)}-c}}{\sum_j e^{x^{(j)}-c}} \;=\; \frac{e^{-c}\, e^{x^{(i)}}}{e^{-c}\sum_j e^{x^{(j)}}} \;=\; \frac{e^{x^{(i)}}}{\sum_j e^{x^{(j)}}}, \tag{5.2}$$

即 $\mathrm{softmax}(x - c\cdot\mathbf{1}) = \mathrm{softmax}(x)$：分子分母同乘 e^{-c}，严格约掉。平移不变性就是一切数值稳定手段的数学基础：**减 max 不改变结果，只改变中间量的量级**。

5.3.2　为什么必须减 max：e^x 的溢出界

fp32 的最大正规数（`FLT_MAX`）为

$$\mathrm{FLT_MAX} \;=\; \big(2 - 2^{-23}\big)\times 2^{127} \;\approx\; 3.4028 \times 10^{38}, \qquad \ln\big(\mathrm{FLT_MAX}\big) \;\approx\; 88.72. \tag{5.3}$$

因此 e^t 不溢出的充要条件是 $t \lesssim 88.72$。两个锚点数字：$e^{88} \approx 1.65\times 10^{38}$（尚可表示），$e^{89} \approx 4.49\times 10^{38} > \mathrm{FLT_MAX}$（已经是 `inf`）。源码注释 base.cuh L556 写的「溢出阈值约 88.7」正是这个界（核校无误）。一旦有元素溢出：若分子为 `inf` 则 $y^{(i)} = \texttt{inf}/s$；若分母 $s = \texttt{inf}$，则 $y^{(i)} = \texttt{inf}/\texttt{inf} = \texttt{NaN}$。本章测试 Case C 用 $x \in [80, 120]$ 复现了这一点：naive 版输出 `NaN`，safe/online 版结果正常。

利用平移不变性（式 5.2），取 $c = m = \max_j x^{(j)}$，得到 **safe softmax**：

$$m = \max_j x^{(j)}, \qquad y^{(i)} \;=\; \frac{e^{x^{(i)}-m}}{\sum_j e^{x^{(j)}-m}}. \tag{5.4}$$

减去 max 之后每个指数参数 $x^{(i)} - m \le 0$，故 $e^{x^{(i)}-m} \le 1$，分子绝不溢出，分母是 n 个不超过 1 的正数之和（最大为 n），也绝不溢出。下溢方向（极小值被舍入为 0）只会让本来就权重趋零的元素更趋零，不影响结果的相对结构。

5.3.3　online 递推：(m, ℓ) 双状态单遍更新

safe 版的问题是要**先**知道全行的 max 才能**再**算指数和，这强制了两遍扫描。online softmax（Milakov & Gimelshein, arXiv:1805.02867）的思路是：维护一对 **运行状态**，让它随时都等价于「已见元素构成的 safe softmax 中间量」：

$$m^{(k)} = \max_{j \le k} x^{(j)}, \qquad \ell^{(k)} = \sum_{j=1}^{k} e^{\,x^{(j)} - m^{(k)}}, \tag{5.5}$$

即 $\ell^{(k)}$ 永远是「以前 k 个元素自己的 max 为基准的指数和」。现在考虑第 $k+1$ 个元素 $x^{(k+1)}$，逐项推导新状态。第一步，新 max 是

$$m^{\mathrm{new}} = \max\big(m,\; x^{(k+1)}\big). \tag{5.6}$$

第二步，按定义展开 ℓ^{new} 并把求和拆成「旧元素 + 新元素」两段：

$$\begin{aligned} \ell^{\mathrm{new}} &= \sum_{j=1}^{k+1} e^{\,x^{(j)} - m^{\mathrm{new}}} \;=\; \sum_{j=1}^{k} e^{\,x^{(j)} - m^{\mathrm{new}}} \;+\; e^{\,x^{(k+1)} - m^{\mathrm{new}}} \\ &= \underbrace{\sum_{j=1}^{k} e^{\,x^{(j)} - m}}_{\text{这正是}\ell} \cdot\, e^{\,m - m^{\mathrm{new}}} \;+\; e^{\,x^{(k+1)} - m^{\mathrm{new}}}, \end{aligned} \tag{5.7}$$

其中第二个等号在旧元素一项上同时乘除 $e^{m-m^{\mathrm{new}}}$（$e^{m-m^{\mathrm{new}}}$ 是常数可以提到求和外）。于是得到 online 递推的核心公式：

$$m^{\mathrm{new}} = \max(m,\, x^{(k+1)}), \qquad \ell^{\mathrm{new}} = \ell\, e^{\,m - m^{\mathrm{new}}} + e^{\,x^{(k+1)} - m^{\mathrm{new}}}. \tag{5.8}$$

两种情形的直觉：

- 若 $x^{(k+1)} \le m$（新元素不刷新 max）：$m^{\text{new}} = m$，修正因子 $e^{m-m^{\text{new}}} = e^0 = 1$，旧的和保持不变，$\ell$ 只需累加一个新项；
- 若 $x^{(k+1)} > m$（新元素刷新 max）：旧的 ℓ 是「以旧 max 为基准」的，基准抬高了 $m^{\text{new}} - m$，所以整体乘 $e^{m-m^{\text{new}}} < 1$ 做 **rescale（缩放）**，再加新项。缩放是全局一次乘法，代价 $O(1)$。

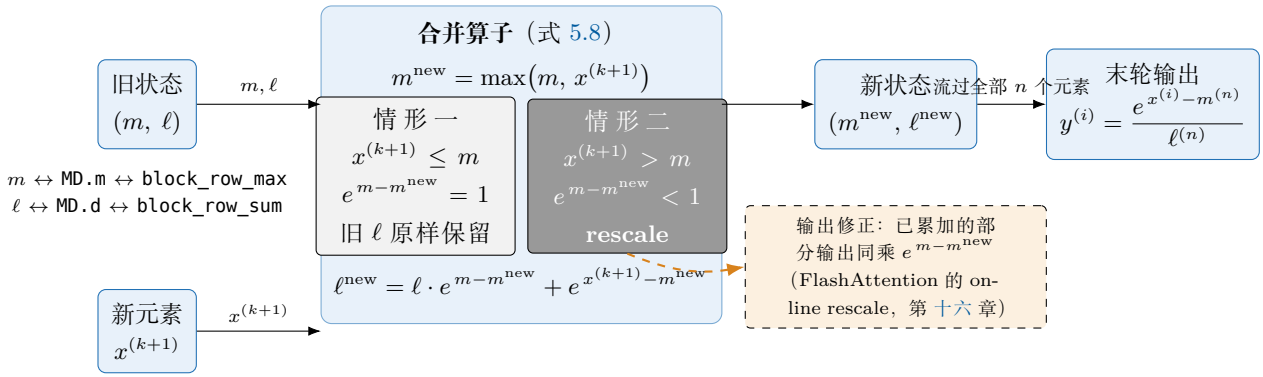


图 5.1: online softmax 单步递推的数据流：旧状态 (m, ℓ) 与新元素 $x^{(k+1)}$ 经合并算子（式 5.8）产出新状态。新元素不刷新 max 时修正因子为 1、旧 ℓ 原样累加；刷新 max 时旧 ℓ 需乘 $e^{m-m^{\text{new}}} < 1$ 做一次 rescale（最深灰），FlashAttention 中已写出的输出累加器 O 同乘该因子。源码对照：m/ℓ 即本章 Level 3 的 `MD.m`/`MD.d`，对应 FlashAttention 源码的 `block_row_max`/`block_row_sum`。

全部 n 个元素流过后，输出为 $y^{(i)} = e^{x^{(i)}-m^{(n)}}/\ell^{(n)}$。注意：真正的 online 实现（含 FlashAttention）里，x 之外还有加权对象 V，输出 O 也要跟着 rescale，那正是第 十六 章 online rescale 的内容，本章先只关注 (m, ℓ) 这对状态。符号约定上，本章的 m、ℓ 即全书记号表中的 running max / running sum，在 FlashAttention 实现里对应源码变量 `block_row_max` 与 `block_row_sum`（联动更新量为 `block_row_max_new`、`block_row_sum_new`）；本章 Level 3 实现中它们由 `MD` 结构体的 `m`、`d` 两个字段承载。

5.3.4 二元合并：把递推广广成可归约的运算

式 5.8 是「状态 + 单元素」的更新。GPU 归约需要的是「状态 + 状态」。设两段各自算好了部分状态 $(m^{(1)}, \ell^{(1)})$（覆盖集合 $S^{(1)}$）与 $(m^{(2)}, \ell^{(2)})$（覆盖集合 $S^{(2)}$），合并公式为

$$m = \max(m^{(1)}, m^{(2)}), \qquad \ell = \ell^{(1)} e^{m^{(1)}-m} + \ell^{(2)} e^{m^{(2)}-m}. \tag{5.9}$$

推导与式 5.7 同构：合并后的基准是两段 max 的较大者，两段的和各乘一个「基准抬升修正因子」后相加。当 $m^{(1)} \ge m^{(2)}$ 时退化为 $\ell = \ell^{(1)} + \ell^{(2)} e^{m^{(2)}-m^{(1)}}$，只有较小者需要 rescale，较大者原样保留。把式 5.8 看作式 5.9 的特例也很自然：单元素 $x^{(k+1)}$ 就是一个 $(x^{(k+1)}, 1)$ 状态（$e^0 = 1$）。

引理 5.1 (二元合并的结合律)**.** *定义运算 $(m^{(1)}, \ell^{(1)}) \oplus (m^{(2)}, \ell^{(2)})$ 为式 5.9。若各部分状态都满足不变量式 5.5（即 $\ell = \sum_{x \in S} e^{x-m}$ 且 $m = \max S$），则合并结果也满足同一不变量，且 $\oplus$ 满足结合律。*

证明. 合并后的 max 是 $\max(S^{(1)} \cup S^{(2)})$；而

$$\ell^{(1)} e^{m^{(1)}-m} + \ell^{(2)} e^{m^{(2)}-m} = \sum_{x \in S^{(1)}} e^{x-m^{(1)}} e^{m^{(1)}-m} + \sum_{x \in S^{(2)}} e^{x-m^{(2)}} e^{m^{(2)}-m} = \sum_{x \in S^{(1)} \cup S^{(2)}} e^{x-m},$$

不变量保持。有了不变量，三个状态无论先结合哪两个，结果都是「全集的 (max, 指数和)」，由全集唯一决定，故任意归约树（结合顺序）给出同一结果。 □

引理 5.1 是本章实现能直接套用第 三 章 XOR 蝶形归约的**合法性证明**：32 个 lane 携带各自的 (m, d)，按 XOR 蝶形任意两两合并，5 步后所有 lane 拿到的都是全行的正确状态，与归约顺序无关。

最后预告两个后继。其一是 **LSE**：对状态取对数压缩，

$$\text{LSE}(x) \;=\; \log \sum_j e^{x^{(j)}} \;=\; m + \log \ell, \tag{5.10}$$

它把 (m, ℓ) 两个数打包成一个标量（正式定义见第 六 章定义 6.1），是分块合并的核心状态。其二是 FlashAttention 的 online rescale：在 attention 里，与 (m, ℓ) 联动更新的还有输出累加器 $O^{\text{new}} = e^{m-m^{\text{new}}} O + e^{s-m^{\text{new}}} V$，分母 ℓ 留到最后统一除，第 十六 章展开。

5.4　设计与实现

三版实现的公共骨架都是第 三 章的 per-token 模式：一个 block 处理一行（一个 token），grid 维度携带 token 维度。注意一个容易踩坑的约定：kernel 形参 N 的语义是**总元素数**（原始工程入口传 $N = S \times H$，S 为 token 数、H 为每 token 长度；见 4.7 节勘误一）。

5.4.1　Level 1：naive——按定义直译

base.cuh L540-552（Level 1：naive softmax，无 max 减法）

```cpp
template <const int kNumThreads = 256>
// Grid:  (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024，一个 block 处理一个 token
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
  float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
  if (idx < N)
    y[idx] = exp_val / exp_sum;
}
```

数据流只有三步：每个线程算自己元素的 $e^{x^{(i)}}$；`block_reduce_sum` 把 2^8 个指数加起来（warp 内蝶形 + shared memory 二级 + 广播，见第 三 章）；每线程做一次除法写回。访存上是每元素读 4 B 写 4 B，归约开销是 1 次 `__syncthreads` 级的 block 同步。它就是式 5.1 的直译，也因此继承了定义的全部缺陷：输入一旦有元素超过 88.72，e^x 变 `inf`，输出全行 `NaN`。它的存在价值是充当「反面教材 + 归约骨架」：后两版只改数据流，不改这套归约基建。

5.4.2　Level 2：safe——两遍扫描的工业标配

base.cuh L563-578（Level 2：safe softmax，2 次 block reduce）

```cpp
template <const int kNumThreads = 256>
__global__ void safe_softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  // Pass 1: block reduce max — 找最大值
  float val = (idx < N) ? x[idx] : -FLT_MAX;
  float max_val = block_reduce_max<kNumThreads>(val);

  // Pass 2: exp(x - max) → block reduce sum
  float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
  float exp_sum = block_reduce_sum<kNumThreads>(exp_val);

  if (idx < N)
    y[idx] = exp_val / exp_sum;
}
```

对照式 5.4，实现是两次 block 归约的串联：

- **Pass 1（求 max）**：边界线程填 `-FLT_MAX`（求和的单位元换成了 max 的单位元），`block_reduce_max` 返回全行最大值并广播到所有线程；

- **Pass 2（指数和）**：每线程算 $e^{x^{(i)}-m}$（上界 1，永不溢出），`block_reduce_sum` 求和并广播；
- **写回**：除法与写回融合，不再单独占一遍扫描。

实现上有个细节：Pass 2 里 `expf(x[idx] - max_val)` 重新读了一次 `x[idx]`，x 实际被全局读了两次（第二次通常 L1 命中，代价远小于一次完整 DRAM 遍历，但不是零）。同步开销上是 2 次 block 级同步（两次归约各一次）。safe 版换来的保证是：**任何有限输入都不产生 `inf/NaN`**，这是它成为推理引擎默认形态的原因。

5.4.3 Level 3：online——(m, d) 融合归约

base.cuh L603-625（MD 状态与 warp_reduce_md 二元合并）

```
struct __align__(8) MD {
  float m; // running max
  float d; // running denominator (sum of exp(x - max))
};

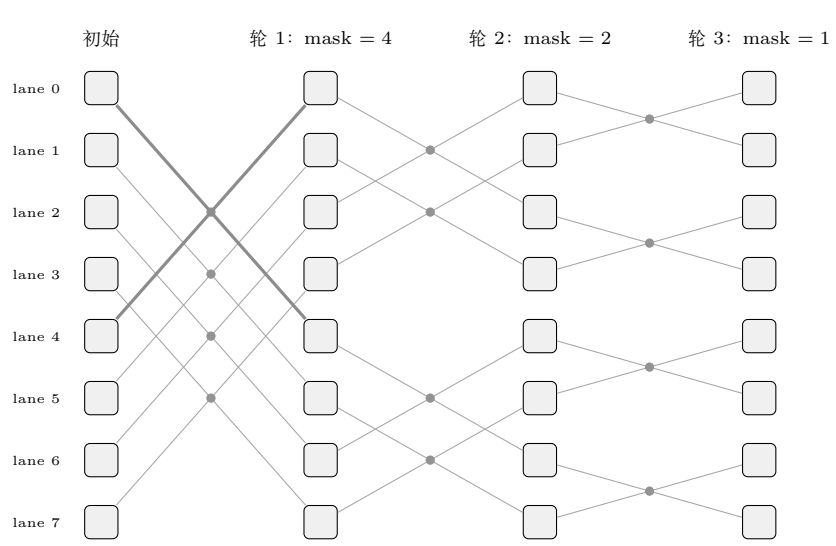
template <const int kWarpWidth = kWarpSize>
__device__ __forceinline__ MD warp_reduce_md(MD md1) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    MD md2;
    md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
    md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);

    MD b_m = (md1.m > md2.m) ? md1 : md2; // max
    MD s_m = (md1.m > md2.m) ? md2 : md1;

    // b_m.d 无需 rescale（其基准 max 已是全局 max），
    // s_m.d 需 rescale：exp(s_m.m - b_m.m)
    md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
    md1.m = b_m.m;
  }
  return md1;
}
```

`MD` 结构体就是式 5.5 的运行状态：`m` 对应 max、`d` 对应 ℓ（denominator，分母），8 字节对齐使它可以整体进 shared memory。`warp_reduce_md` 把式 5.9 装进了蝶形归约的骨架，逐行对照：

- 每轮 mask 下，lane i 与 lane $i \oplus \text{mask}$ 交换各自的 (m, d)，得到对称的两个输入（`md1` 与 `md2`）；
- `b_m`/`s_m` 分别是 max 较大/较小的那个状态（bigger/smaller 的缩写）；
- 合并时**只有较小者需要 rescale**：`b_m` 的基准已经是二者共同的 max，它的 d 原样保留；`s_m` 的 d 乘修正因子 `__expf(s_m.m - b_m.m)`。这正是式 5.9 退化形式 $m^{(1)} \geq m^{(2)}$ 的情形；
- 结合律由引理 5.1 保证，因此 XOR 蝶形的归约顺序不影响结果。



初始每 lane 打包 $(x^{(i)}, 1)$（边界 lane 填 $(-\texttt{FLT_MAX}, 0)$）；轮 3 后**每个 lane 都持有全行** (m, ℓ)
结合律（引理 5.1）⇒ 任意归约顺序结果相同；真实 warp 为 32 lane、5 步（mask = 16, 8, 4, 2, 1），本图以 8 lane 示意

图 5.2: MD 状态的 XOR 蝶形归约（8 lane 示意，mask $= 4 \to 2 \to 1$）：每轮 lane i 与 $i \oplus$ mask 互换 (m, d) 并按式 5.9 合并——m 较大一侧（中灰点）d 原样保留，较小一侧（深灰点）d 乘 `__expf(s_m.m - b_m.m)` 做 rescale（深粗线强调首对交换）。成对骨架同第 三 章图 3.2，本图聚焦 MD 合并语义；结合律（引理 5.1）保证归约顺序无关。

base.cuh L631-664（Level 3：online safe softmax 主 kernel）

```cpp
template <const int kNumThreads = 256>
__global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + threadIdx.x;
  const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ MD shared[kNumWarps];

  float val = (idx < N) ? x[idx] : -FLT_MAX;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  // 初始化：每个线程持有一个 (max, denom) 对
  MD md;
  md.m = idx < N ? val : -FLT_MAX;
  md.d = idx < N ? 1.0f : 0.0f;

  // 第一级规约：warp_reduce_md 在归约中自动更新 m 和 d
  md = warp_reduce_md<kWarpSize>(md);

  if (lane == 0)
    shared[warp] = md;
  __syncthreads();

  // 第二级归约：每个 warp 结果再做一次 warp_reduce_md（复用 block_reduce 模式）
  md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
  md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果

  // 用全局 max 和 denom 做最终 softmax
  float d_inv = __fdividef(1.0f, md.d);
  // 边界线程即使看到 d=0 的填充值，也不会走到写回路径
  if (idx < N) {
    y[idx] = __expf(val - md.m) * d_inv;
  }
}
```

主 kernel 的流程：（1）初始化：每个线程把**自己的那一个元素**打包成状态 $(x^{(i)}, 1)$（边界线程填 $(-\texttt{FLT_MAX}, 0)$：max 用单位元，d 用加法单位元 0，这样它们参与归约但不污染结果）；（2）第一级：warp 内 5 步 `warp_reduce_md` 蝶形；（3）warp leader 把 warp 状态写入 shared memory，`__syncthreads` 一次；（4）第二级：每个 warp 的所有 lane 重新读出 8 个 warp 状态再做一次 `warp_reduce_md`，注释里「用 kWarpSize 确保每个 lane 都拿到最终结果」的用意是：第二级不只在 warp 0 做，而是**每个 warp 都独立算一遍**，免去了第二次 `__syncthreads` 加广播；（5）写回 $y^{(i)} = e^{x^{(i)}-m} \cdot d^{-1}$，除法用近似倒数 `__fdividef` 配一次乘法完成，指数用快速数学 `__expf`。与 safe 版对比：扫描遍数从 2 降到 1（x 只读一次、一直保留在寄存器里），block 级同步从 2 次降到 1 次；代价是归约算子从「单个加法」变成「合并运算」，且写回阶段的 `__expf`/`__fdividef` 是近似指令（精度见 4.8 节坑三）。

5.5 图示

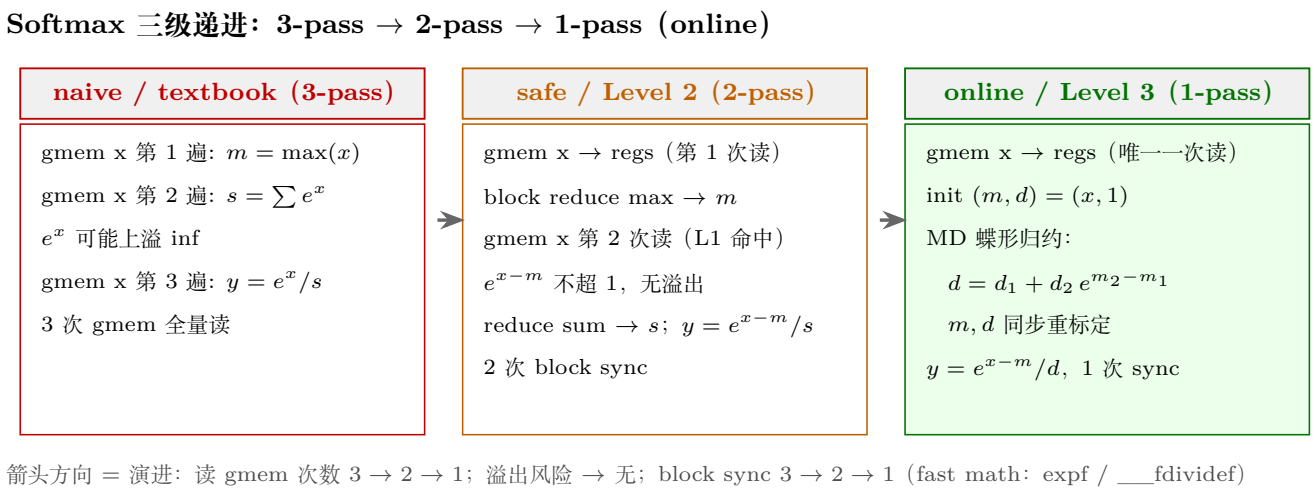


图 5.3: Softmax 三级递进的 3-pass → 2-pass → 1-pass 数据流：左为教科书三遍扫描（无 max 减法，e^x 可能上溢）；中为 safe 版两次归约；右为 online 版 (m, d) 状态一次融合归约。

图 5.3 的三条泳道对应本章三级实现。读图的关键是「全局扫描次数」这一维度：3-pass 版本每遍历一次 x 都要付一遍 DRAM 带宽；2-pass 版本把 x 装进寄存器后只需两次 block 归约；1-pass 版本连归约都只剩一次：(m, d) 在蝶形归约里同步更新。FlashAttention 把右列的 (m, d) 再加一列 O，就是第 十六 章的 online rescale。

5.6 性能分析

算术强度：per-token softmax 每个元素的有效访存是读 4 B + 写 4 B = 8 B；计算上除一次超越函数 e^x（SFU 指令，不占 FMA 流水）外只有 $O(1)$ 次乘加。按「每元素 2 FLOP」的乐观口径，$AI \approx 2/8 = 0.25$ FLOP/B，远低于现代 GPU 数十至上百 FLOP/B 的 roofline 拐点，softmax 是典型的 **memory-bound** 算子，优化重心在扫描遍数与同步次数，而不是 FLOP。

本章 bench 复测（RTX PRO 5000 72GB Blackwell，sm_120，2026-09-11；$S = 4096$ 个 token、每 token $H = 256$、block = 256；warmup 3 次 + 10 次取均值）：

表 5.1: ch04 三版 softmax 耗时对比（10 次均值，带宽按每元素 8 B 有效流量计）

实现	ms/iter	有效带宽 (GB/s)
naive（1 次归约，无 max 减法）	0.0121	690.6
safe（2 次归约）	0.0146	572.9
online（1 次 MD 融合归约）	0.0142	591.6

三个观察：（1）naive 最快：它比 safe 少一次 block 归约（少一次全局同步），比 online 的合并运算轻；在输入保证有界时它确实是三者中最便宜的，可惜这个前提在生产环境不可保证。（2）online 比 safe 略快：扫描遍数与同步次数的优势（1 次 vs 2 次）压过了更重的合并算子与快速数学指令；差距在小 N 下不明显，当 N 变大、第二次全局读取不再能被 L1 完全吸收时，online 的优势会拉大。（3）三者的绝对带宽都不高（数百 GB/s）：单次 kernel 只有十几个微秒，launch 与同步的固定开销占比大；附录 B 的汇总口径会注明这是「短 kernel 小形状」数据，不与 README 的大形状 GEMM/attention 带宽直接比较。

5.7　实践经验

实践经验 5.1 (F3)：base.cuh L538-539 与 L541-542：启动配置注释的两处含混 其一，L538 的「grid(S*h/h, h), block(h)」中 Sh/h 化简后就是 S，写法混乱；实际启动为 grid$(S,1,1)$、block$(H,1,1)$，与 L541-542 一致。其二，更实质的问题：三版 kernel 内的边界守卫都是全局索引 `idx < N`，但注释没有说明 N 的语义是**总元素数** $S\times H$（原始工程 `softmax.cu` 入口有 `const int N = S * H;`，base.cuh 精简版丢掉了这行）。本章测试曾因按「每 token 长度」理解 N 并以 $S=2$ 调用而复现故障：block 1 的全部线程被守卫剪掉，token 1 的输出**不写回**（旧值残留）。抽取源 bench/base.cu 的 test_softmax（L359）固定 grid(1)、$N=256$（单 token），恰好绕开了该歧义。建议读者牢记约定：grid(S) 时必须传 $N=S\times H$，且 H 恰等于 blockDim（一个 token 装满一个 block）。

实践经验 5.2 (F3 核校通过)：base.cuh L556：溢出阈值 88.7 ln(FLT_MAX) ≈ 88.72；$e^{88}\approx 1.65\times 10^{38}<$ FLT_MAX，$e^{89}\approx 4.49\times 10^{38}>$ FLT_MAX。注释数字与独立推导一致。

实践经验 5.3 (F5 核校通过)：base.cuh L590：算法出处 arXiv:1805.02867 Online normalizer calculation for softmax（M. Milakov, N. Gimelshein, 2018），arXiv 编号正确。该文同时给出了本文式 5.9 的二元合并形式。

实践经验 5.4 (F3)：base.cuh L583-585 与 L586-589：两套公式并存需辨析 注释同时给出「单元素增量更新」与「二元合并」两组公式，数学上等价（后者视为前者的推广，见式 5.8 与式 5.9 的关系），但注意实现细节：`warp_reduce_md` 中的指数用的是快速数学 `__expf`，与 safe 版的 `expf` 精度语义不同（见 4.8 节坑三），两版输出逐位对比会有 10^{-8} 量级差异，属预期现象（本章测试的一致性判定按此放宽）。

5.8　常见坑

坑一：N 传错语义，静默丢输出。见 4.7 节勘误一：grid(S) 且 $S>1$ 时必须传 $N=S\times H$，否则 block $1,\dots,S-1$ 整体被 `idx < N` 剪掉，输出缓冲残留旧值：不报错、不崩溃，纯静默。建议自测时先以 $S=2$ 跑一遍「输出缓冲先 memset 填充特定值」的检查。

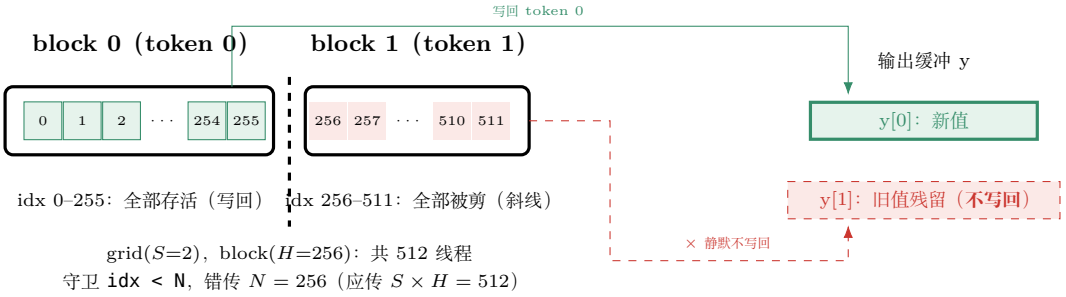


图 5.4: 坑一「N 传错语义」的静默丢输出：grid$(S{=}2)$、block$(H{=}256)$ 时正确应传 $N=S\times H=512$；若按「每 token 长度」错传 $N=256$，`idx < N` 守卫线落在 block 0/block 1 边界上，block 1 的全部线程（idx 256–511，斜线格）被剪掉，token 1 的输出缓冲残留旧值——不报错、不崩溃（见 4.7 节勘误一）。

坑二：边界填充值要和运算匹配。求和的填充是 0（加法单位元），求 max 的填充是 `-FLT_MAX`（max 单位元），online 版状态填充是 $(\texttt{-FLT_MAX},0)$。三者各自成对，混用（比如 max 归约填 0）会让 max 被钳在 0：基准被抬高，指数参数 $x-0$ 比正确的 $x-m$ 整体偏小（m 为真实行最大值；输出在不下溢时因平移不变性式 5.2 逐位不变），真正的风险是整行强负值（$x\lesssim -87$）时 e^x 全部下溢为 0，得到 0/0 的 NaN（naive 上溢故障的镜像）。若整行全为边界线程（空行），$d=0$ 会出现除零，per-token 场景下由调用方保证行非空。

坑三：`__expf`/`__fdividef` 的精度语义。`__expf` 是 SFU 硬件指数，最大误差约 $2+\lfloor|1.16x|\rfloor$ ulp（小参数下约 2 ulp）；`expf` 是软件精确版。推理路径用快速数学通常无损（softmax 输出本就有 fp32 舍入），但参考实现、精度回归、

数值实验必须用 `expf`。同理 online 版写回的 `__fdividef` 是近似除法。第 二十 章量化注意力对 LSE 的精度更敏感，届时这类选择要重新评估。

坑四：fp16 输入必须升 fp32 累加。half 的最大值约 65504，e^x 在 $x \gtrsim 11.1$（$\ln 65504 \approx 11.09$）越界。所以 fp16 softmax 的正确做法永远是：减 max 后用 fp32 做指数与累加（F32Acc），最后再落回 fp16。第 七 章的 RMSNorm/LayerNorm 同理，容差分级也沿用 F32Acc 档。

坑五：blockDim 必须与模板参数一致。三版 kernel 的模板参数 `kNumThreads` 决定二级归约的 shared 数组大小与参与线程数，launch 时 blockDim 不一致会越界或归约错位。这是第 三 章 block 归约的既有约束，本章沿用。

5.9 面试要点速查

Q1：为什么 softmax 要减 max？不减会怎样？

A：数学上平移不变（式 5.2），工程上 e^x 在 $x > 88.72$ 时溢出为 `inf`，输出变 `NaN`；减 max 后指数参数非正、指数值 ≤ 1，绝对安全。

Q2：safe 版比 naive 版多花了什么？

A：一次额外的 block 归约（一次全局同步点），以及写回阶段对 x 的第二次读取（通常 L1 命中）。本章实测约慢 21%（表 5.1），换来的是任意有限输入下的确定性。

Q3：online softmax 的不变量是什么？为什么能套 XOR 蝶形归约？

A：不变量是式 5.5；二元合并满足结合律（引理 5.1），所以任意归约树结果唯一，XOR 蝶形 5 步即可全 warp 收敛。

Q4：单元素更新和二元合并两套公式什么关系？

A：后者是前者的一般化：单元素 x 等价于状态 $(x, 1)$。$m^{(1)} \geq m^{(2)}$ 时合并退化为 $\ell = \ell^{(1)} + \ell^{(2)} e^{m^{(2)} - m^{(1)}}$，即「大者不动、小者 rescale」。

Q5：FlashAttention 为什么必须要 online softmax？

A：attention 若先物化整张分数矩阵 S 再 softmax，显存是 $O(N^2)$；online rescale 让 softmax 依附在 KV 分块循环上流式完成，S 从不被完整物化，显存 $O(N)$。这正是第 十六 章的主题，而它的状态打包形式 LSE 就是下一章的核心对象。

5.10 最小测试与动手实验

测试文件：`book/tests/ch04_softmax.cu`（逻辑抽取自 bench/base.cu 的 test_softmax，L359，对照改为 CPU fp64 参考）。三个 case：A（$S = 2$、$N = 256$，$x \in [-5, 5]$，三版正确性 + 两两一致性）；B（$S = 1$、$N = 100$、block=128，边界线程守卫路径）；C（$x \in [80, 120]$，naive 必然溢出 vs safe/online 稳定的演示）。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --ch ch04`（编译 + 运行一体）。本机运行输出（RTX PRO 5000，2026-09-11，PASS 行的误差数值见表 5.2）：

```
case A: S=2 N=256 block=256 x in [-5,5]
case B: S=1 N=100 block=128 x in [-5,5]
case C: S=1 N=256 block=256 x in [80,120]
INFO naive with large x: y[0]=nan (non-finite expected)
ALL OK
```

判据：三版对 CPU fp64 参考的误差都在 10^{-8} 量级（远小于 F32Acc 档 10^{-3}）；case C 中 naive 输出 `NaN` 而 safe/online 与参考吻合，直观展示了 4.3 节的溢出界。动手实验建议：（1）把 Case A 的 scale 从 5 逐步调到 30，观察 naive 从正确到溢出的临界点；（2）把 launch 改成 $S = 2$ 但只传 $N = 256$，复现勘误一的静默丢输出；（3）bench 用 `./build/ch04_softmax bench` 自行复现表 5.1 的数据。

延伸阅读

- @ 紫气东来《CUDA 算子系列（二）：SoftMax》，https://zhuanlan.zhihu.com/p/695307283（naive/safe/online 的 CUDA 实现对照）；
- @DefTruth《图解大模型训练之：推理加速——从 Online-Softmax 到 FlashAttention V1/V2/V3》，https://zhuanlan.zhihu.com/p/668888063（online softmax → FA 的完整推导链，本章 4.3 节的风格基线）；

表 5.2: ch04 测试 PASS 行数值摘录（完整行式输出见 `tests/.build/ch04_softmax` 运行日志；tol 为判定容差）

case	判定项	最大绝对误差	tol
A	softmax naive	3.914e-09	1.0e-03
A	softmax safe	3.624e-09	1.0e-03
A	softmax online	3.914e-09	1.0e-03
A	consistency naive vs safe	3.725e-09	2.0e-05
A	consistency safe vs online	3.725e-09	2.0e-05
A	consistency naive vs online	3.725e-09	2.0e-05
B	softmax naive / safe / online	$<$ 1.2e-08	1.0e-03
C	softmax safe	1.150e-08	1.0e-03
C	softmax online	8.146e-09	1.0e-03

- M. Milakov, N. Gimelshein. Online normalizer calculation for softmax. arXiv:1805.02867, 2018（online 递推与二元合并的原始出处）。

第六章　LSE 与分块合并：merge_attn_states

6.1　本章导读

第 五 章末尾我们预告了 LSE：它把 online softmax 的双状态 (m, ℓ) 压缩成一个标量。本章讲清楚两件事：**为什么这个压缩是无损的**，以及 **拿到两份分块的部分结果后，如何只靠 LSE 就把它们合并成与「一次性算完」逐元素相等的结果**。教学载体是 base.cuh 的 `merge_attn_states`（base.cuh L437 起）：它实现 FlashInfer 论文（arXiv 2501.01005）Section 2.2 描述的 attention 状态合并逻辑，也是 vLLM/FlashInfer 里 chunked prefill、FlashDecoding 等分块方案共用的最终合并环节。

前置章节：第 五 章（online 递推与二元合并式）。**源码区间**：base.cuh L387–519（Phase 2 末尾），无编译宏限定，全部架构适用。**后继**：第 十六 章 FlashAttention 的 online rescale 把本章的「事后合并」搬进主循环内部；第 二十 章 FFPA Split-D 则在大 head_dim 切分后用同一套公式跨 D-chunk 合并。

6.2　问题与动机

长序列推理有一条基本的工程规律：**注意力要沿 KV 维分块**。原因有二：其一，KV cache 随序列增长，一次性装不下或不想装（chunked prefill、PagedAttention 的分页调度）；其二，把 KV 沿序列切给多个 SM 并行（FlashDecoding 的 split-KV），用并行度换延迟。分块之后立刻产生一个新问题：**每个分块各自做了一次独立的 softmax，它们的输出不能直接相加**。softmax 不是线性算子，分母（指数和）被各自的 max 归一化过，直接相加等于把两份「各自除过各自分母」的概率混在一起。

好在第 五 章的 online 递推已经给出：只要每块额外输出一个状态 (m, ℓ)（或其打包形式 LSE），两块就能严格合并，且结果与「把两块拼起来一次算完」逐元素相等。于是整个分块体系拆成两层：

- **块内**：各块独立跑标准 attention kernel，输出已归一化的 O^A, O^B 与各自的 $\mathrm{LSE}^A, \mathrm{LSE}^B$；
- **块间**：一个极轻量的合并 kernel（本章的分析对象）按指数权重把两份 O 加权混合。它每个输出元素只有 3 次 FLOP、约 12 B 流量，完全并行、无任何同步，是一个纯粹 bandwidth-bound 的 elementwise kernel。

这个设计的关键在于**状态接口极窄**：块与块之间只传一个标量 LSE（外加那份本来就要传的 O），主 kernel 不需要为「可合并」做任何特殊化改造。本章测试将构造两个分块状态，验证合并结果与整体 softmax 直接计算逐元素相等。

6.3　数学原理

6.3.1　分块状态与 LSE 的无损打包

沿用第 五 章记号：某 query 行的打分为 $s^{(j)}$，KV 被切成两个分块 A（$j \in A$，共 n 个）与 B（$j \in B$，共 m 个）。每个分块独立跑完 attention 后持有四个量：

$$m^A = \max_{j \in A} s^{(j)}, \quad \ell^A = \sum_{j \in A} e^{\,s^{(j)} - m^A}, \quad O^A = \frac{1}{\ell^A} \sum_{j \in A} e^{\,s^{(j)} - m^A} v^{(j)}, \quad \mathrm{LSE}^A = m^A + \log \ell^A, \tag{6.1}$$

分块 B 同理（上标 A 换 B）。O^A 是**已归一化**的分块输出（除过自己的 ℓ^A），这正是 FlashAttention kernel 落盘的形态；LSE^A 是对数求和指数（log-sum-exp），它是 m 与 ℓ 的可逆打包：

定义 6.1 (LSE)**.** 对集合 S 上的分数 $\{s^{(j)}\}$，$\mathrm{LSE}(S)=\log\sum_{j\in S}e^{s^{(j)}}$。若以 $m=\max_{j\in S}s^{(j)}$ 与 $\ell=\sum_{j\in S}e^{s^{(j)}-m}$ 表示，则

$$\mathrm{LSE}(S)=m+\log\ell. \tag{6.2}$$

而对**任意**基准 $\tilde m$，$e^{\mathrm{LSE}-\tilde m}=\sum_{j\in S}e^{s^{(j)}-\tilde m}$ 都成立（取 $\tilde m=m$ 时即 ℓ），LSE 单个标量即可还原任意基准下的指数和，无损携带 (m,ℓ) 状态的全部信息。

6.3.2　合并公式：从真值形式到 LSE 形式

先写**真值形式**。把两块拼起来一次性做 safe softmax，基准取 $m=\max(m^A,m^B)$：

$$O=\frac{\sum_{j\in A\cup B}e^{s^{(j)}-m}v^{(j)}}{\ell},\qquad \ell=e^{m^A-m}\ell^A+e^{m^B-m}\ell^B, \tag{6.3}$$

其中分母正是第 五 章式 5.9 的二元合并。把分子的求和按 A、B 拆开并凑因子：

$$\sum_{j\in A}e^{s^{(j)}-m}v^{(j)}=\underbrace{e^{m^A-m}\ell^A}_{\triangleq W^A}\cdot\underbrace{\frac{1}{\ell^A}\sum_{j\in A}e^{s^{(j)}-m^A}v^{(j)}}_{=O^A}, \tag{6.4}$$

于是真值合并可以写成「未归一化权重」W^A,W^B 对已归一化输出的凸组合：

$$O=\frac{W^AO^A+W^BO^B}{W^A+W^B},\qquad W^A=e^{m^A-m}\ell^A,\ W^B=e^{m^B-m}\ell^B. \tag{6.5}$$

现在做**关键替换**：问题要求我们手上只有 $\mathrm{LSE}^A,\mathrm{LSE}^B$（没有分开的 m^A,ℓ^A）。利用定义 6.1 的还原性，对合并基准 $L\triangleq\max\big(\mathrm{LSE}^A,\mathrm{LSE}^B\big)$ 计算

$$w^A\triangleq e^{\mathrm{LSE}^A-L}=e^{m^A+\log\ell^A-L}=e^{m-L}\cdot\underbrace{e^{m^A-m}\ell^A}_{W^A}. \tag{6.6}$$

也就是说：**用 LSE 算出的权重 w^A 与真值权重 W^A 只差一个对两块完全相同的常数因子** e^{m-L}。在式 6.5 的分子分母里，这个常数同时出现、同时约掉。定义归一化权重

$$\alpha=\frac{w^A}{w^A+w^B},\qquad \beta=\frac{w^B}{w^A+w^B},\qquad \alpha+\beta=1, \tag{6.7}$$

则 $\alpha=W^A/(W^A+W^B)$、$\beta=W^B/(W^A+W^B)$，立刻得到本章的核心定理。

定理 6.2 (LSE 合并等价于整体 softmax 加权)**.** *设两分块状态由式 6.1 给出，$L=\max(\mathrm{LSE}^A,\mathrm{LSE}^B)$，$\alpha,\beta$ 由式 6.7 定义，则*

$$O=\alpha O^A+\beta O^B=\frac{\sum_{j\in A\cup B}e^{s^{(j)}-m}v^{(j)}}{\sum_{j\in A\cup B}e^{s^{(j)}-m}}, \tag{6.8}$$

即两步合并（先块内归一化、后 LSE 加权）与拼接后一次性 softmax 加权逐元素相等。

证明. 由式 6.6，$w^A/(w^A+w^B)=W^A/(W^A+W^B)$（常数因子 e^{m-L} 约去）。代入式 6.5：$\alpha O^A+\beta O^B=(W^AO^A+W^BO^B)/(W^A+W^B)$。再由式 6.4 反向展开，分子为 $\sum_{j\in A}e^{s^{(j)}-m}v^{(j)}+\sum_{j\in B}e^{s^{(j)}-m}v^{(j)}=\sum_{j\in A\cup B}e^{s^{(j)}-m}v^{(j)}$，分母为式 6.3 的 $\ell=\sum_{j\in A\cup B}e^{s^{(j)}-m}$。得证。 □

定理 6.2 可以拆成三个点。**其一**，shift 基准可以任选：用 $L=\max(\mathrm{LSE})$ 还是 $m=\max(m)$ 做基准只改变常数因子，不影响凸组合系数，所以 kernel 可以直接在 LSE 域上取 max，无需还原 m。同时 $w^A,w^B\le e^0=1$，数值安全性与 safe softmax 相同。**其二**，$\alpha+\beta=1$ 意味着合并是**凸组合**：O 落在 $[O^A,O^B]$ 的凸包内，不会放大任何一方的量级。**其三**，这套推导对「两块」成立，对任意多块也成立（逐对两两合并即可，对应第 十六 章 FA 主循环里的 online rescale 与第 二十 章跨 D-chunk 的两阶段合并）。

6.3.3 空段与 $\pm\infty$ 语义

causal mask 等机制会产生「空段」：某分块内全部打分被 mask 成 $-\infty$，此时 $\ell = 0$，LSE 依实现约定可能记为 $+\infty$ 或 $-\infty$。合并层的正确语义是**空段权重必须为 0**。源码的防御是：先把 $+\infty$ 的 LSE 清洗成 $-\infty$（base.cuh L484-485），此后 $w = e^{-\infty - L} = 0$，该块权重自然退化为 0（数学上严格成立，核校通过）。但要小心一个残留边界：**两段同时为空**时 $L = -\infty$，$w = e^{-\infty-(-\infty)} = e^{\mathrm{NaN}}$，输出 `NaN`，kernel 未处理；真实合并场景（prefix+suffix 至少一段非空）不会触发，但做单元测试与泛化改造时必须知道这条边界的存在。

6.4 设计与实现

base.cuh L437-457（merge_attn_states 签名与线程规划）

```cpp
__global__ void merge_attn_states(
    float *output,              // [num_tokens, num_heads, head_size]
    const float *prefix_output, // [num_tokens, num_heads, head_size]
    const float *prefix_lse,    // [num_heads, num_tokens]
    const float *suffix_output, // [num_tokens, num_heads, head_size]
    const float *suffix_lse,    // [num_heads, num_tokens]
    int num_tokens, int num_heads, int head_size) {

  constexpr int kNumThreads = 128;
  constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
  using pack_t = uint4;

  // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
  const int threads_per_head = head_size / kPackSize;
  // 实际有效总线程数，超过这个数的线程会被忽略，block是按照kNumThreads=128
  // 来分配的，那么最后一个block的线程数可能会超过实际需要的线程数
  const int total_threads = num_tokens * num_heads * threads_per_head;

  const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
  if (global_idx >= total_threads)
    return;
```

先看接口与布局约定，这里有本章最容易犯错的一处**转置陷阱**：

- O 相关的三个数组（`output`/`prefix_output`/`suffix_output`）布局是 [num_tokens, num_heads, head_size]，token 维在最外，与 attention 输出惯例一致；
- 而 LSE 的两个数组布局是 [num_heads, num_tokens]，head 维在最外（FlashAttention-2 惯例里 LSE 逐 head 连续存放便于后续逐 head 处理）。于是同一个逻辑对 (t, h) 在两边的寻址分别是 $t \cdot H \cdot D + h \cdot D$ 与 $h \cdot T + t$，一个是「行优先的块」一个是「转置的下标」，写测试时极易弄反（见 5.8 节坑二）。

线程规划是三维切分：`threads_per_head = head_size / kPackSize` 个线程覆盖一个 (token, head) 对的 D 个元素，`total_threads` 为全局有效线程数，超过它的线程直接返回（block 按 128 对齐分配，最后一个 block 会多出空闲线程）。

base.cuh L459-485（global_idx 三级解包与 LSE 读取、inf 清洗）

```cpp
  // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
  // token_head_idx: 第几个 (token, head) 对，行优先展平
  const int token_head_idx = global_idx / threads_per_head;
  // pack_idx: 该 (token, head) 对内第几个 pack，0 ~ threads_per_head-1
  const int pack_idx = global_idx % threads_per_head;

  // token_head_idx 分解为 (token_idx, head_idx)
  const int token_idx = token_head_idx / num_heads; // token 变化慢（外维）
  const int head_idx = token_head_idx % num_heads;  // head 变化快（内维）

```

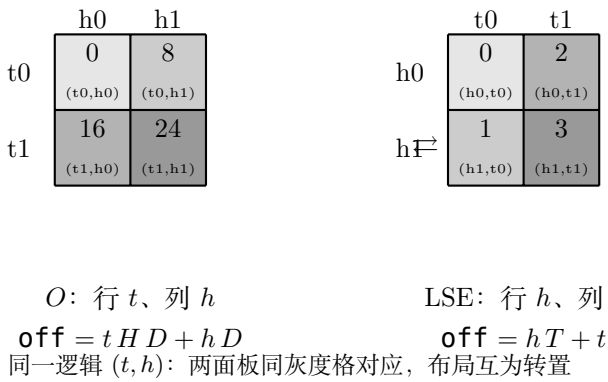


图 6.1: 坑二的布局转置陷阱 (T=2, H=2, D=8)：同一个逻辑对 (t,h)（同灰度）在 $O[T,H,D]$ 中的偏移是 $t\cdot H\cdot D+h\cdot D$（行优先的块），在 LSE$[H,T]$ 中的偏移是 $h\cdot T+t$（转置的下标）。索引公式写反不报任何错，但合并的是错配的 (token, head)。

```
  // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
  const int pack_offset = pack_idx * kPackSize;
  // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
  const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;

  // 定位到当前 (token, head) 的输出段起始
  const float *prefix_head = prefix_output + head_offset;
  const float *suffix_head = suffix_output + head_offset;
  float *output_head = output + head_offset;

  // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
  float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
  float s_lse = suffix_lse[head_idx * num_tokens + token_idx];

  // inf → -inf: 空 attention 段 LSE 可能为 +inf，替换后权重退化为 0
  p_lse = isinf(p_lse) ? -INFINITY : p_lse;
  s_lse = isinf(s_lse) ? -INFINITY : s_lse;
```

这段是一维线程号到三维数据坐标的解包：`global_idx` 先除/模 `threads_per_head` 得到（第几个 (token,head) 对，pack 内第几个 128-bit 包），再除/模 `num_heads` 分出 token（变化慢的外维）与 head（变化快的内维）。以头注释的示例（T=2, H=2, D=8）：8 个线程的 `token_head_idx` 依次为 $0,0,1,1,2,2,3,3$、`pack_idx` 为 $0,1,0,1,0,1,0,1$。同一个 warp 里相邻线程访问相邻 pack，两次 O 的读与一次写都是 128-bit 完全 coalesced，这是它达到 TB/s 级带宽的直接原因。LSE 读取只有每 (token, head) 2 次标量加载（`p_lse`、`s_lse`），随后立即做 $+\infty \to -\infty$ 清洗（5.3 节）。

base.cuh L487-497（Step 1–3：max 归一、指数还原、归一化权重）

```
  // Step 1: max 归一化 (与 safe softmax 同理，防 exp 溢出)
  const float max_lse = fmaxf(p_lse, s_lse);
  p_lse -= max_lse;
  s_lse -= max_lse;

  // Step 2-3: 指数还原 → 混合比例 alpha, beta
  const float p_se = expf(p_lse);
  const float s_se = expf(s_lse);
  const float out_se = p_se + s_se;
  const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
  const float s_scale = s_se / out_se; // beta  = w_2 / (w_1 + w_2)
```

对照定理 6.2 的推导链，加上最前面的清洗，实现共五步：

- **Step 0（清洗）**：`isinf ? -INFINITY : lse`，把空段的 $+\infty$ 约定归一到 $-\infty$；
- **Step 1（max 归一）**：$L=\max(\mathrm{LSE}^A,\mathrm{LSE}^B)$，两数各减 L，与 safe softmax 的减 max 完全同构，保证后续指数

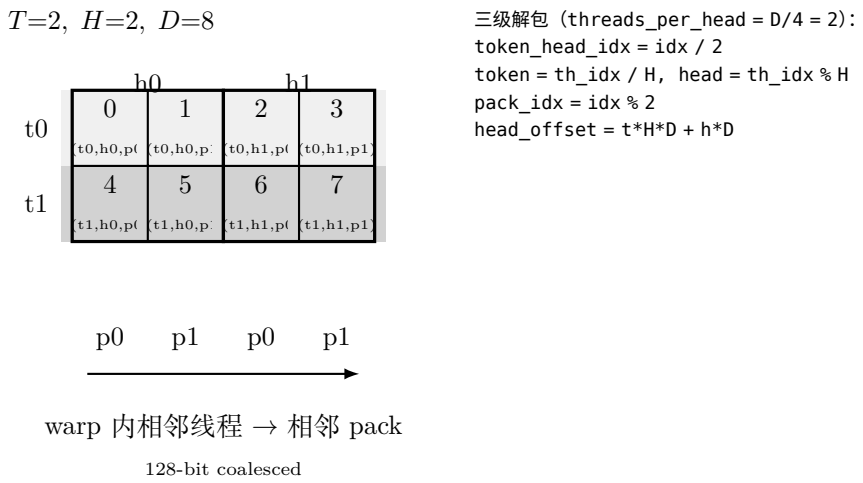


图 6.2: `global_idx` 三级解包（T=2, H=2, D=8，共 8 个线程）：行 = token（两级灰度区分），行内粗边框子块 = head，列 = pack（`threads_per_head = D/4 = 2`）；格内为 `global_idx`，小字为解包结果 (t, h, p)。warp 内相邻线程命中相邻 pack，O 的两次读与一次写均为 128-bit 完全 coalesced。

参数非正；

- **Step 2（指数还原）**：$w^A = e^{\mathrm{LSE}^A - L}$、$w^B = e^{\mathrm{LSE}^B - L}$（源码 `p_se`/`s_se`），即式 6.6；
- **Step 3（归一化）**：$\alpha = w^A/(w^A + w^B)$、$\beta = w^B/(w^A + w^B)$（源码 `p_scale`/`s_scale`），即式 6.7，常数因子 e^{m-L} 在此处被约掉；
- **Step 4（凸组合）**：$O = \alpha O^A + \beta O^B$，见下一段。

源码头注释写「Step 1–4」而概括为「5 步推导」，本书按「清洗 + Step 1–4」共 5 步展开（见 5.7 节勘误一）；另外 Step 0–3 由覆盖每个 (token, head) 对的全部线程各自冗余执行（输入相同、结果一致；LSE 同址加载经合并后实际流量仍为每对 8B），计算被该 head 的 D 个输出元素摊薄，这就是为什么合并层的计算开销可以忽略不计。

base.cuh L499-517（Step 4：128-bit 向量化凸组合）

```
  // Step 4: 逐元素加权合并 O = alpha * O_1 + beta * O_2
  // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
  if (pack_offset < head_size) {
    pack_t p_pack =
        reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
    pack_t s_pack =
        reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
    pack_t o_pack;

#pragma unroll
    for (int i = 0; i < kPackSize; ++i) {
      const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
      const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
      const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
      reinterpret_cast<float *>(&o_pack)[i] = o_v;
    }

    reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
  }
```

Step 4 的主体是 `uint4`（= 4× float，128-bit）打包加载：`prefix`/`suffix` 各取一个 pack，逐元素做一次乘加 $o = p \cdot \alpha + s \cdot \beta$（硬件可融合为 FMA），再整包写回。每个输出元素 3 次 FLOP、2 次读 1 次写；128-bit 粒度让每线程的 4 个元素共享同一份 α, β 与同一份 LSE 读取。收尾的 `if (pack_offset < head_size)` 在「D 是 4 的倍数」的约定下恒为真（见 5.7 节勘误三），属于防御性写法。

6.5　图示

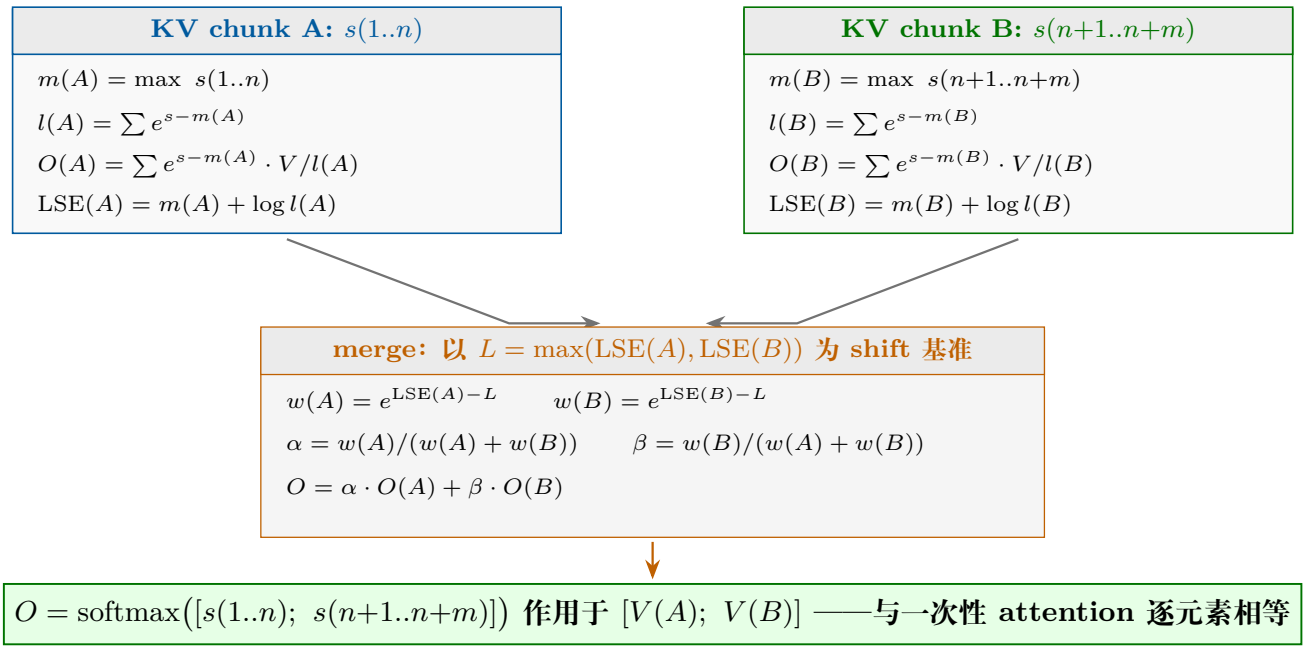


图 6.3: 两个分块 attention 状态的合并：块内各自输出（已归一化 O 与单标量 LSE），块间以 $L = \max(\text{LSE})$ 为 shift 基准计算凸组合系数 α, β，合并结果与整体 softmax 直接加权逐元素相等（定理 6.2）。

6.6　性能分析

算术强度：每个输出元素读 O^A（4 B）、读 O^B（4 B）、写 O（4 B），共 12 B；LSE 是每 (token, head) 8 B 的标量流量，摊到元素上是 $8/D$ B。计算是每元素 3 FLOP（2 乘 1 加）。$D = 128$ 时 $AI \approx 3/12.06 \approx 0.25$ FLOP/B，位于 roofline 拐点左侧，是典型的 **severely memory-bound** 算子，头注释的结论正确，但「20 bytes」的口径把 LSE 读按每元素计了（见 5.7 节勘误二）。优化方向也因此明确：向量化宽度（128-bit 已到位）、coalescing（三维解包保证）、以及避免任何分支/同步（本 kernel 无共享内存、无同步点）。

本章 bench 复测（RTX PRO 5000 72GB Blackwell，sm_120，2026-09-11；$T = 16384$、$H = 32$、$D = 128$；每次迭代净流量为两份 O 读取与一份 O 写入 3×268.4 MB，加两份 LSE 共 2×2.1 MB，合计约 809.5 MB；warmup 3 次 + 10 次取均值）：

表 6.1: ch05 merge_attn_states 耗时与带宽（10 次均值）

kernel	ms/iter	有效带宽 (GB/s)
merge_attn_states	0.7849	1031.4

单 kernel 0.78 ms 完成 809.5 MB 访存，有效带宽 1031 GB/s，已达到该卡理论带宽的约 77%，对「读两份写一份」的纯流式访存是合理水平（读写切换与 LSE 的零散标量访问会损失一部分）。作为对比参照：同卡上 ch04 的 softmax 小形状只有数百 GB/s，差距主要来自形状规模（本 kernel 每次迭代有 1600 万个线程、13 万个 block，launch 开销完全摊薄）。若要继续提升，可尝试的方向是 256-bit 访问或 LSE 的 warp 级广播复用，但收益预计在个位数百分比，该算子的带宽已接近硬件上限。

6.7　实践经验

实践经验 6.1 (F3)：base.cuh L393：「5 步推导」只列了 Step 1–4 头注释称「数学公式（5 步推导）」，随后仅列出 Step 1（max 归

一)、Step 2（指数还原)、Step 3（归一化混合比例)、Step 4（加权合并）四步。按实现数，第 5 步应是 L484-485 的 $+\infty \to -\infty$ 清洗（或把「$w_1 + w_2$ 归一化」与「逐元素加权」拆开计数)。本章 5.4 节按「清洗 + Step 1–4」共 5 步展开，与代码行一一对应；建议后续修订注释时明确第 5 步所指。

实践经验 6.2 (F4)：base.cuh L412：「AI ≈ 3 FLOP / 20 bytes ≈ 0.15」口径需注明 20 B 的口径把两次 LSE 读（8 B）按**每个输出元素**计入，仅在 $D = 1$ 量级成立；LSE 是每 (token, head) 一份的标量，摊销后每元素流量为 $12 + 8/D$ B（$D = 128$ 时约 12.06 B，$AI \approx 0.25$)。「severely memory-bound」的定性结论不受影响，数值口径建议标注为「LSE 不摊销的上界」。

实践经验 6.3 (F3)：base.cuh L450 与 L501：head_size 须为 4 的倍数是隐含约定 `threads_per_head = head_size / kPackSize` 是整除写法：若 head_size 不是 4 的倍数，尾部 head_size % 4 个元素**没有任何线程覆盖**，输出静默不写（无报错)。而 L501 的守卫 `if (pack_offset < head_size)` 在整除约定下恒为真，非整除时也覆盖不到尾部元素，属防御性死代码。调用方应保证 head_size % 4 == 0（现实中 attention 的 head_dim 均满足)，泛化改造则需补尾部标量路径。

实践经验 6.4 (F5 核校通过)： base.cuh L388：FlashInfer 论文引用 arXiv 2501.01005 即 FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving (2025-01), 编号正确；其 attention 合并逻辑（本章 5.3 节同款公式）出自该文 Section 2.2 附近，小节号未逐页复核，引用整体成立。另核校 L407-409：空段 $+\infty$ 清洗后 $e^{-\infty - L} = 0$ 的权重退化数学正确；补充边界：两段同时为空时 $L = -\infty$、出现 NaN（见 5.3 节末)，注释未提及，做泛化测试时需自行防护。

6.8 常见坑

坑一：输入 O 必须是已归一化的。本章推导（式 6.4）假设 O^A 已除过 ℓ^A（FA kernel 落盘形态)。若误用未归一化的分块输出（还带着自己的 ℓ)，合并结果会整体错一个 ℓ 因子，数值上看起来「量级差不多」，精度上已经全错。改造或复用该 kernel 前先确认上游约定。

坑二：O 与 LSE 的布局转置。O 是 $[T, H, D]$、LSE 是 $[H, T]$（源码 L479-481 的寻址 `lse[h * num_tokens + t]`)。两者转置关系，索引公式写反不会报任何错，但合并的是「错配的 (token, head)」，输出表现为与参考完全对不上。本章测试的 build_case 里两个布局分别独立排布，可作为对照。

坑三：head_size 必须是 4 的倍数。见 5.7 节勘误三。另注意 `uint4` 访问要求 16 B 对齐：`head_offset` 以元素计是 $(t \cdot H + h) \cdot D$，D 为 4 的倍数时自然对齐；若在张量切片视图上直接取指针（如从大 tensor 切出非对齐段)，会触发 misaligned address 崩溃。

坑四：两段同时为空会输出 NaN。见 5.3 节末。真实分块场景（prefix + suffix、split-KV 的各段）至少一段非空；但做泛化工具或单测时，应显式防护「双空段」或将其定义为约定俗成的输出（例如全零或直接透传)。

坑五：区分「事后合并」与「在线合并」。本 kernel 是分块计算**完成之后**的合并，需要落盘两份 O；FlashAttention 主循环内部（第 十六 章）与 FFPA 跨 D-chunk（第 二十 章）用的是同一套数学、但在寄存器里**在线** rescale，从不物化两份 O。两者数学同源（定理 6.2)，工程取舍不同：前者换来主 kernel 的零侵入，后者省一整轮显存往返。

6.9 面试要点速查

Q1：LSE 是什么？为什么它能代替 (m, ℓ) 两个状态？

A：$\mathrm{LSE} = m + \log \ell$（定义 6.1)。给定任意 shift 基准 L，$e^{\mathrm{LSE} - L}$ 同时还原出「以 L 为基准的未归一化指数和」，即 (m, ℓ) 的全部信息；一个 float 就够，这是分块方案接口极窄的原因。

Q2：写出两块合并的完整流程。

A：五步：清洗 $(+\infty \to -\infty) \to L = \max(\mathrm{LSE}^A, \mathrm{LSE}^B) \to w = e^{\mathrm{LSE} - L} \to \alpha = w^A/(w^A + w^B), \beta = w^B/(w^A + w^B) \to O = \alpha O^A + \beta O^B$，且 $\alpha + \beta = 1$（凸组合)。

Q3：为什么用 $\max(\mathrm{LSE})$ 而不是 $\max(m)$ 做 shift 基准也正确？

A：定理 6.2 的证明核心：LSE 域的权重 w 与真值权重 W 只差一个两块共享的常数因子 $e^{m - L}$，归一化后约掉；shift 基准只影响数值安全（保证指数参数非正)，不影响数学结果。

Q4：这个 kernel 为什么是 memory-bound？瓶颈怎么估？

A：每元素 12 B 流量对 3 FLOP，$AI \approx 0.25$ FLOP/B；实测 0.78 ms 搬 809.5 MB（表 6.1)，约 1.03 TB/s。优化空间在访存粒度与 coalescing（已做 128-bit)，而非计算。

Q5：空段的 LSE 该如何处理？

A：约定 $+\infty$（或 $-\infty$)的空段先统一清洗成 $-\infty$，使其权重 $e^{-\infty - L} = 0$；注意两段同时为空时会出现 $-\infty - (-\infty) = \mathrm{NaN}$，

需要上游保证至少一段非空或显式防护。

Q6：它与 FlashAttention 内部的 online rescale 什么关系？

A：同一套合并数学（比较式 6.5 与第 十六 章的 $O \leftarrow e^{m^{old}-m^{new}}O + PV$）：本 kernel 是事后、落盘、elementwise 的版本；FA 是在线、寄存器、随 KV 分块流水推进的版本。

6.10　最小测试与动手实验

测试文件：`book/tests/ch05_merge_attn.cu`（逻辑抽取自 bench/base.cu 的 test_merge_attn_states，L235；对照改为 CPU fp64 的「整体 softmax 直接加权」）。每个 case 为每个 (token, head) 构造两个 KV 分块（各 32 个 score），以 float 模拟 FA 分块输出（O^A, O^B 已归一化、LSE $= m + \log \ell$），验证两件事：合并结果 $\overset{!}{=}$ 全序列 softmax 直接计算（定理 6.2 的数值版）；prefix LSE 置 $+\infty$ 后该段输出精确退化为 suffix。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --ch ch05`（编译 + 运行一体）。本机运行输出（RTX PRO 5000，2026-09-11，PASS 行的误差数值见表 6.2）：

```
case A: T=2 H=2 D=8 kv=2x32
case B: T=3 H=4 D=128 kv=2x32
ALL OK
```

表 6.2: ch05 测试 PASS 行数值摘录（完整行式输出见 `tests/.build/ch05_merge_attn` 运行日志；tol 为判定容差）

case	判定项	最大绝对误差	tol
A	merge == direct full-softmax	9.624e-08	1.0e-03
A	inf-lse → suffix only	0.000e+00	1.0e-06
B	merge == direct full-softmax	2.049e-07	1.0e-03
B	inf-lse → suffix only	0.000e+00	1.0e-06

两处细节：其一，「合并 == 直接计算」的误差是 10^{-7} 量级（fp32 分块状态 + fp32 合并累积），远小于 F32Acc 档 10^{-3}，说明该合并路径的数值质量足以支撑真实 attention 的分块方案；其二，inf 段测试误差**精确为 0**：$\alpha = 0, \beta = 1$ 时 FMA 的 $0 \cdot p + 1 \cdot s$ 不引入任何舍入，权重退化是位精确的。动手实验建议：（1）把某一段的 score 整体抬高 20（模拟「一段压倒另一段」），观察 α 趋近 1、O 趋近 O^A；（2）故意把 LSE 布局按 $[T, H]$ 传（复现坑二），确认输出全面失配；（3）bench 用 `./build/ch05_merge_attn bench` 复现表 6.1。

延伸阅读

- @DefTruth《vLLM 源码解读 1.4：Merge Attention States 算子》：https://zhuanlan.zhihu.com/p/1904937907703243110（vLLM 中该合并算子的工程上下文与 CUDA/Triton 双实现）；
- @DefTruth《图解大模型训练之：推理加速——从 Online-Softmax 到 FlashAttention V1/V2/V3》，https://zhuanlan.zhihu.com/p/668888063（online softmax → 在线合并的推导链，本章定理的前置视角）；
- FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv:2501.01005, 2025（attention 状态合并的论文出处，源码头注释引用）。

第七章　归一化：RMSNorm 与 LayerNorm

7.1　本章导读

深度学习推理引擎里出现频率最高的两类归一化算子，是 RMSNorm 与 LayerNorm。本章讲它们的数学原理与 GPU kernel 实现。这两类算子与第 五 章的 softmax 同属一个计算家族：**一行（row）内所有元素协作计算出一个统计量，再把统计量广播回行内做逐元素缩放**。差别只在于统计量是 $\{\mu, \sigma\}$ 还是 RMS，以及由此带来的归约次数差异。

前置章节：第 三 章的 warp/block 归约原语（本章直接复用 `block_reduce_sum`）；第 四 章的向量化访存（本章 vec4 版本的前置）。

源码区间：`base.cuh` L666-801，共 4 个 kernel：`rms_norm`（L677）、`rms_norm_vec4`（L699）、`layer_norm`（L733）、`layer_norm_vec4`（L765）。**编译宏与架构限定**：无，fp32 纯 CUDA 实现，任意 arch 可编译运行。配套最小测试 `tests/ch06_norm.cu`。

7.2　问题与动机

Transformer 的每个 block 至少包含两个归一化层（pre-norm 结构中位于 attention 与 MLP 之前）。对 LLaMA 系列模型，这两处全部使用 RMSNorm；对 GPT-2、BERT 及大量多模态模型，两处是 LayerNorm。一次 7B 模型的前向推理，归一化层的调用次数与 attention 层同阶，是典型的高频算子。

从计算结构上看，归一化算子由三段组成：

1. **行内归约**：对长度为 K 的一行求统计量（和、平方和）；
2. **统计量广播**：把一个标量从「算出它的线程」传递给行内所有线程；
3. **逐元素变换**：读原值、减均值（LayerNorm）、乘缩放、加偏置、写回。

这个结构与 softmax 的「行内 max/sum 归约 + 广播」同构，因此第 五 章的分析手段可以原样迁移。而 RMSNorm 与 LayerNorm 之间的差异，恰好落在「归约几次、同步几次」上，**1-pass 与 2-pass 归约结构**就用这一对算子来讲解。

性能特征先给结论：归一化是典型的 memory-bound 算子。每个元素读 4B、写 4B，浮点运算只有 2-3 次，算术强度 $AI \approx 0.3$ FLOP/Byte，远低于现代 GPU Roofline 拐点（第 二 章）。优化的两条主线很清楚：其一，把访存做满（coalescing 与向量化）；其二，减少归约与同步的开销（1-pass 优于 2-pass）。

7.3　数学原理

7.3.1　LayerNorm：均值与方差的双重归约

LayerNorm（Ba et al., 2016）对一行输入 $x = (x_1, x_2, \ldots, x_K)$ 定义如下。首先计算行均值

$$\mu = \frac{1}{K} \sum_{k=1}^{K} x_k, \tag{7.1}$$

再计算**有偏**行方差（除以 K 而非 $K-1$，与 PyTorch `nn.LayerNorm` 口径一致）

$$\sigma^2 = \frac{1}{K} \sum_{k=1}^{K} (x_k - \mu)^2 . \tag{7.2}$$

归一化值与输出分别为

$$\hat{x}_k = \frac{x_k - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y_k = \hat{x}_k \cdot g + b, \tag{7.3}$$

其中 g（gain/scale）与 b（bias）在源码实现中是标量；生产实现里它们是逐通道可学习参数向量，逐元素乘加的形状不变。$\epsilon = 10^{-5}$ 是数值稳定项，**加在根号之内**，即分母是 $\sqrt{\sigma^2 + \epsilon}$ 而不是 $\sqrt{\sigma^2} + \epsilon$：若一行全零，$\sigma^2 = 0$，没有 ϵ 时除零得到 NaN。

7.3.2　RMSNorm：去掉中心化的单归约形式

RMSNorm（Zhang & Sennrich, 2019）只保留「缩放」而不做「中心化」。先定义均方根

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{K}\sum_{k=1}^{K} x_k^2 + \epsilon}, \tag{7.4}$$

输出为

$$y_k = \frac{x_k}{\mathrm{RMS}(x)} \cdot g. \tag{7.5}$$

7.3.3　RMSNorm 与 LayerNorm 的关系：去均值化简

两者不是孤立的设计。对 LayerNorm 的方差做逐项展开：

$$\begin{aligned}\sigma^2 &= \frac{1}{K}\sum_{k=1}^{K}(x_k - \mu)^2 = \frac{1}{K}\sum_{k=1}^{K}\left(x_k^2 - 2\mu x_k + \mu^2\right) \\ &= \underbrace{\frac{1}{K}\sum_{k=1}^{K} x_k^2}_{\text{平方均值}} - 2\mu \cdot \underbrace{\frac{1}{K}\sum_{k=1}^{K} x_k}_{=\mu} + \mu^2 = \frac{1}{K}\sum_{k=1}^{K} x_k^2 - \mu^2.\end{aligned} \tag{7.6}$$

移项即得恒等式 $\frac{1}{K}\sum_k x_k^2 = \sigma^2 + \mu^2$。代入式 (7.5) 的分母：

$$\mathrm{RMS}(x) = \sqrt{\sigma^2 + \mu^2 + \epsilon}. \tag{7.7}$$

对照式 (7.3) 可以读出三层关系：

1. 当输入近似零中心（$\mu \approx 0$）时，$\mathrm{RMS}(x) \approx \sqrt{\sigma^2 + \epsilon}$，RMSNorm 与 LayerNorm 几乎重合；神经网络中间激活的均值通常远小于二阶矩，这直观解释了 RMSNorm 去掉中心化后精度几乎不受影响。

2. RMSNorm 分母比 LayerNorm 分母多出 μ^2 项：LayerNorm 只除「波动幅度」，RMSNorm 除「总能量（波动 + 直流）」。

3. 由此产生一个重要差别：LayerNorm 具备**平移不变性**，$x \to x + c$ 时 $\mu \to \mu + c$、σ^2 不变，输出不变；RMSNorm 不具备（μ^2 项改变）。所以 LLaMA 系列把 RMSNorm 前面的 QKV 线性投影的 bias 直接去掉（bias 只会给分母注入无信息的 μ^2），这是架构层面的连锁设计。

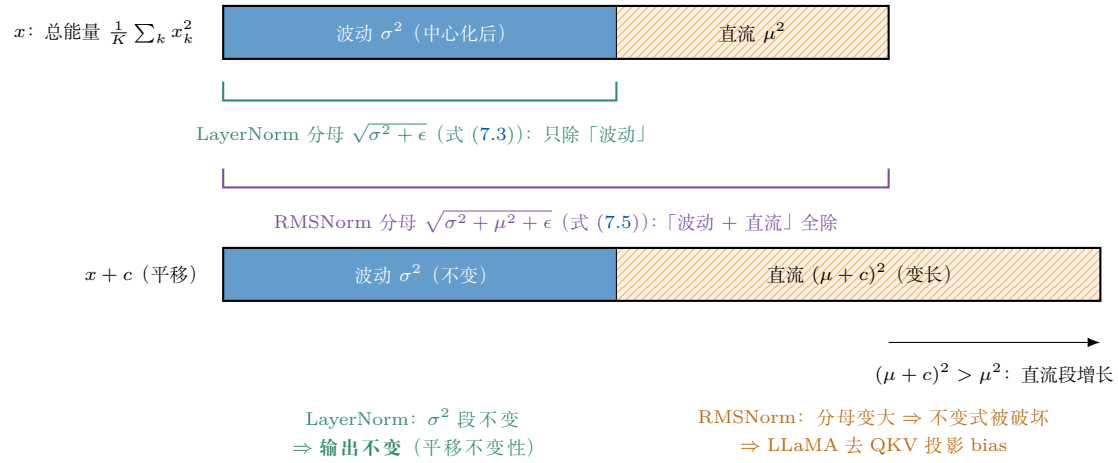


图 7.1: 分母的能量分解（恒等式 (7.6)：$\frac{1}{K}\sum_k x_k^2 = \sigma^2 + \mu^2$）：LayerNorm 只除「波动」$\sigma^2$，RMSNorm 除「总能量（波动 + 直流）」。平移 $x \to x + c$ 时 σ^2 段宽度不变而 μ^2 段变长——LayerNorm 输出不变，RMSNorm 分母增大，这就是去 bias 的架构连锁（6.3.3 节第 3 条）。

7.3.4 1-pass 与 2-pass：归约结构的形式化

从 kernel 视角重新审视上述公式。记两个基本归约量为 $S_1 = \sum_k x_k$、$S_2 = \sum_k x_k^2$：

- **RMSNorm 是 1-pass**：只需要 S_2 一个归约量，一次 block reduce 完成，随后 $\mathrm{RMS}^{-1} = \mathrm{rsqrtf}(S_2/K + \epsilon)$ 广播，行内每个元素乘回去。全程只有一次归约、两次块级同步。
- **LayerNorm 的定义式是 2-pass**：μ 依赖 S_1（第一次归约），而 σ^2 依赖 μ（定义式 $\frac{1}{K}\sum_k (x_k - \mu)^2$ 必须等 μ 广播后才能计算），依赖链强制两次归约。

LayerNorm 能否也做成 1-pass? 利用恒等式 (7.6)，$\sigma^2 = S_2/K - \mu^2$，可以在一次归约中同时求 S_1 与 S_2（warp shuffle 时打包两个 float，或在同一线程内做两次独立归约），从而把 LayerNorm 压成 1-pass。代价是数值风险：$S_2/K - \mu^2$ 是「大数减小数」，当 $|\mu| \gg \sigma$（分布整体远离零点，且数据动态范围大）时，两个相近的浮点数相减发生 catastrophic cancellation，fp32 精度下方差可能出现明显误差甚至为负。数值稳定且仍只过一遍数据的方案是 Welford 递推（增量维护 μ 与 $M_2 \equiv \sum_k (x_k - \bar{x})^2$，分块结果可用合并公式汇总），但它的归约合并逻辑复杂得多。工程权衡：fp32 输入下 2-pass 简单可靠；fp16/bf16 输入的生产 kernel 通常升 fp32 累加后沿用 2-pass，或采用 Welford。本章源码选择最直白的结构：RMSNorm 1-pass、LayerNorm 2-pass。

7.3.5 rsqrtf：倒数平方根的硬件路径

两个 kernel 的分母最终都写成

$$s = \frac{1}{\sqrt{v + \epsilon}} = \mathrm{rsqrtf}(v + \epsilon), \tag{7.8}$$

其中 v 是行内归约量。CUDA 的 `rsqrtf` 映射到 MUFU.RSQ 硬件近似指令，最大相对误差约 2 ulp；对归一化这种「得到 s 后还要乘一个 $O(1)$ 量级元素值」的场景，2 ulp 的相对误差远低于 fp32 输出的可分辨精度，是无损加速。对比手写 $1.0f / \mathtt{sqrtf}(v)$（`sqrtf` 加一次除法，两条慢指令），`rsqrtf` 一条指令完成。

7.4 设计与实现

7.4.1 行划分：one-row-one-block

四个 kernel 采用同一网格设计：`grid(N)`, `block(kNumThreads=128)`，N 是行数（batch × seq_len），$K = 128$ 是列数。一个 block 恰好负责一行的归约，行内 K 个元素由 block 内 K 个线程一一对应，天然 coalesced（warp 内 32 线程读连续 32 个 float）。

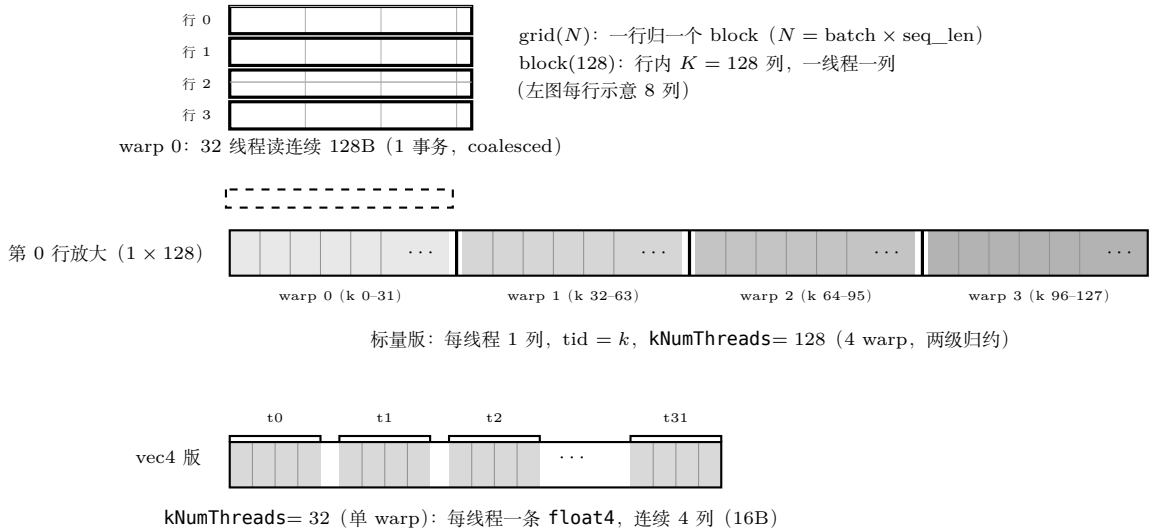


图 7.2: one-row-one-block 行划分：上，$N \times K$ 矩阵每行归一个 block（`grid(N)`）；中，单行放大为 1×128 条带，4 个 warp 各占 32 列（各一档灰度），warp 0 的 32 线程读连续 128B，单事务合并；下，vec4 版改为 32 线程、每线程一条 `float4`（连续 4 列），指令数降为 1/4。

归约本身直接复用第 三 章的 `block_reduce_sum`（base.cuh L179-196）：warp 内蝶形 shuffle 归约、warp leader 写 shared memory、`__syncthreads()`、第二级 warp 归约，最后 `__shfl_sync` 把结果广播到**每个**线程。这一步广播无法省略：行内所有线程都要拿到同一个 s 才能各自完成缩放。

base.cuh L179-196（block_reduce_sum：两级归约 + 全员广播，详见第 三 章）

```cpp
template <const int kNumThreads = 256>
__device__ float block_reduce_sum(float val) {
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  int warp = threadIdx.x / kWarpSize;
  int lane = threadIdx.x % kWarpSize;
  __shared__ float shared[kNumWarps];

  float value = warp_reduce_sum<kWarpSize>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < kNumWarps) ? shared[lane] : 0.0f;
  value = warp_reduce_sum<kNumWarps>(value);
  // 关键：broadcast 结果到所有线程，后续用 result
  // 做除法等操作时所有线程都能拿到
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}
```

再看模板参数的编译期特化：标量版传 `kNumThreads=128`（4 个 warp，两级归约）；vec4 版传 `kNumThreads=32`（单 warp，此时 `kNumWarps=1`，第二级归约的 shuffle 循环在编译期折叠为空，shared memory 中转退化成一次额外同步）。同一个模板，参数不同即得到两级或一级结构。

7.4.2　rms_norm：单归约 + 单广播

base.cuh L666-692（rms_norm：1-pass RMSNorm）

```cpp
// ============================================================================
// Phase 3b: RMS Normalization (1-pass reduce)
// ============================================================================
// 面试要点:
//   - RMS Norm: y = (x / rms(x)) * g, 1/rms(x) = rsqrt(mean(x²))
```

```
//   - 只需 1 次 block reduce (sum of squares)，比 Layer Norm 少 1 次同步
//   - Llama 系列使用 RMS Norm
//   - grid(N, K/K), block(K): 一行一个 block
// Grid:  (N, 1, 1), N=batch*seq_len, 每行一个 block
// Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int kNumThreads = 128>
__global__ void rms_norm(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;
  float variance = value * value;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
  __syncthreads();
  if (idx < N * K)
    y[idx] = (value * s_variance) * g;
}
```

逐段拆解：

1. **载入与越界 padding**：`value = (idx < N*K) ? x[idx] : 0.0f`。越界线程贡献 0，而 $0^2 = 0$ 不影响平方和。padding 值的选择取决于归约量的物理含义。对照第 五 章 softmax 的行 max 归约必须 pad $-$FLT_MAX，如果这里错 pad 成 FLT_MAX，平方和直接溢出。

2. **单次归约**：`block_reduce_sum<kNumThreads>(variance)` 一次得到整行平方和。变量名 `variance` 在此处尚不是方差，只是「平方和」的中间量，除以 K 才是均方。

3. **tid 0 写共享标量**：`if (tid == 0) s_variance = rsqrtf(variance / K + epsilon)`。先除 K 再加 ϵ 再开根号取倒数，与式 (7.5) 逐项对应；顺序若写成 rsqrtf$((v+\epsilon)/K)$，ϵ 的相对作用被缩小 K 倍（近乎失效），分母偏小使输出系统性偏大。

4. **第二次同步**：`__syncthreads()`。`block_reduce_sum` 内部的同步只保证「warp leader 写 `shared[]`」的可见性；`s_variance` 是另一个共享变量，写在 reduce 之后，必须再加一次屏障，否则其他线程可能读到未初始化的 `s_variance`。这是数据竞争，错误值非确定。

5. **缩放写回**：`y[idx] = (value * s_variance) * g`。所有线程拿到同一个 $s = 1/\text{RMS}$，乘各自的 x_k 与 gain。

7.4.3 rms_norm_vec4：一次搬运四个元素

base.cuh L693-721（rms_norm_vec4：float4 向量化访存）

```

// RMS Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐，且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int kNumThreads = 128 / 4>
__global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
```

```
  float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
                                    reg_x.z * reg_x.z + reg_x.w * reg_x.w)
                                 : 0.0f;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();
  float4 reg_y;
  reg_y.x = reg_x.x * s_variance * g;
  reg_y.y = reg_x.y * s_variance * g;
  reg_y.z = reg_x.z * s_variance * g;
  reg_y.w = reg_x.w * s_variance * g;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}
```

与标量版的差异全部围绕访存：

- 线程索引步长变为 4：`idx = (blockIdx * blockDim + tid) * 4`，block 线程数相应从 128 降到 32（$K/4$）。
- `FLOAT4` 宏把相邻 4 个 float 一次 16B 装入 `float4` 寄存器；写回同样一条 `FLOAT4` 存储。每线程的有效访存宽度翻 4 倍，load/store 指令数减为 1/4。
- 归约的输入改为局部 4 元素平方和 `reg_x.x*reg_x.x + ... + reg_x.w*reg_x.w`：先在寄存器内串行折叠 4 项（编译器可映射为 FMA 链），再交给 block reduce。归约树形状变化带来的舍入差异在 fp32 下可以忽略（本章测试实测 $\sim 4 \times 10^{-7}$）。
- **对齐前提**：`float4` 访问要求地址 16B 对齐，即行首地址对齐（$K\%4 = 0$ 保证行内不破对齐）加基址对齐（`cudaMalloc` 返回 256B 对齐，自然满足）。不满足时触发 misaligned address 错误而非静默出错。

7.4.4　layer_norm：两次归约 + 两次同步

base.cuh L723-758（layer_norm：2-pass LayerNorm）

```
// ================================================================================
// Phase 3c: Layer Normalization (2-pass reduce)
// ================================================================================
// 面试要点:
//   - Layer Norm: y = ((x - mean) / std) * g + b, std = sqrt(variance), variance = mean((x - mean)²)
//   - 需要 2 次 block reduce: 先 mean (sum/K) , 再 variance (sum((x-mean)²)/K)
//   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
// Grid:  (N, 1, 1), 一行一个 block
// Block: (128, 1, 1), kNumThreads=K=128
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int kNumThreads = 128>
__global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;

  // Pass 1: compute mean
  float sum = block_reduce_sum<kNumThreads>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads(); // 必须等待 s_mean 对所有线程可见

```

```
  // Pass 2: compute variance = (x - mean)²
  float variance = (value - s_mean) * (value - s_mean);
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
  __syncthreads(); // 必须等待 s_variance 对所有线程可见

  if (idx < N * K)
    y[idx] = ((value - s_mean) * s_variance) * g + b;
}
```

结构上相当于把 rms_norm 的归约-广播结构做两遍，两次归约的依赖关系与第 五 章 safe softmax「先 max 后 sum」的依赖链完全相同：

- **Pass 1**：归约 S_1 得 `s_mean = sum / K`，`__syncthreads()` 广播。这一次同步不可省：Pass 2 中每个线程都要读 `s_mean`。
- **Pass 2**：用广播后的均值计算 $\sum_k (x_k - \mu)^2$，再次 block reduce，tid 0 写 `s_variance = rsqrtf(variance / K + epsilon)`，再一次 `__syncthreads()` 广播。
- 输出 `y = ((value - s_mean) * s_variance) * g + b`，与式 (7.3) 逐项对应。

同步开销精确计数：layer_norm 全程 `__syncthreads()` 共 4 次（两次 block reduce 内部各 1 次 + 显式 2 次），rms_norm 只有 2 次，这就是源码注释「比 Layer Norm 少 1 次同步」的完整语境（少一次归约，连带少一次 reduce 内部同步与一次显式同步）。$K = 128$ 时每次块级同步的绝对延迟在微秒以下，但归一化算子本身访存量极小（一行 $128 \times 4\text{B} \times 2$ 的访存在 $\sim$ TB/s 带宽下不足 $1\,\mu s$），相对占比可观，本章第 6 节的实测数据会量化这一点。

7.4.5 layer_norm_vec4：寄存器复用的两遍结构

base.cuh L765-800（layer_norm_vec4：Vec4 的 2-pass 版本）

```
template <const int kNumThreads = 128 / 4>
__global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;

  float sum = block_reduce_sum<kNumThreads>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads();

  float4 reg_x_hat;
  reg_x_hat.x = reg_x.x - s_mean;
  reg_x_hat.y = reg_x.y - s_mean;
  reg_x_hat.z = reg_x.z - s_mean;
  reg_x_hat.w = reg_x.w - s_mean;
  float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
                   reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();

  float4 reg_y;
```

```
  reg_y.x = reg_x_hat.x * s_variance * g + b;
  reg_y.y = reg_x_hat.y * s_variance * g + b;
  reg_y.z = reg_x_hat.z * s_variance * g + b;
  reg_y.w = reg_x_hat.w * s_variance * g + b;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}
```

vec4 化带来一个标量版没有的实现细节：Pass 2 需要再次用到 x_k，但 gmem 里的值早已读入 `reg_x` 寄存器，重读 gmem 既慢又可能与其他线程的写入不一致。源码的做法是把 Pass 1 的减法结果直接物化到 `reg_x_hat`（4 个寄存器），Pass 2 的平方和与最后的缩放输出都从它出发，做到**一次读、全程寄存器复用**。中间量的生命周期管理（`reg_x` 用完即弃、`reg_x_hat` 跨 Pass 存活）是 2-pass 向量化 kernel 的通用模式，第 五 章 online softmax 的 $m/\ell/O$ 寄存器组是同一思想的更复杂形态。

7.5　图示

图 ch06-1 对比两类归一化的归约结构。RMSNorm 的关键在于归约量只有一个 S_2；LayerNorm 的 σ^2 依赖 μ，依赖链把结构拉长为两段。

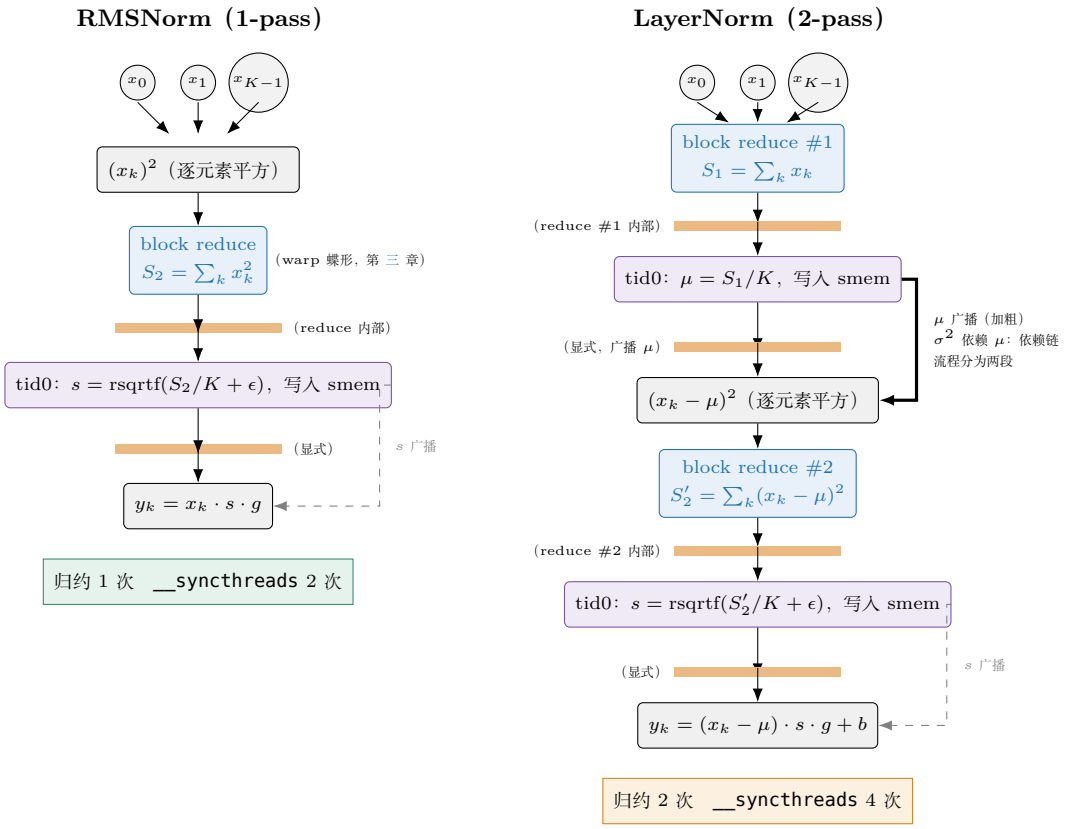


图 7.3: RMSNorm 1-pass 与 LayerNorm 2-pass 的归约结构对比

7.6　性能分析

7.6.1　Roofline 定位

每元素运算量：RMSNorm 约 3 FLOP（平方、累加、缩放；rsqrt 每行摊薄一次），LayerNorm 约 5 FLOP（多出减均值）。流量固定 8B（读 4B + 写 4B），$AI \approx 0.3\text{-}0.6$ FLOP/Byte。对照第 二 章的 Roofline 模型，这类算子远在拐点左侧的带宽受限区，优化目标就是逼近访存带宽上限，一切同步与指令开销都在与访存竞争关键路径。

7.6.2 实测数据

测试环境：NVIDIA RTX PRO 5000 72GB（Blackwell，sm_120a，110 SM，L2 96MB），CUDA 13，cudaEvent 计时取 20 次 min，warmup 3 次；$K = 128$，fp32；有效带宽按读 + 写 $2NK \cdot 4\,\mathrm{B}$ 计：

N（行）	rms_norm	rms_norm_vec4	layer_norm	layer_norm_vec4
8192	0.0104 ms / 809 GB/s	0.0101 ms / 832 GB/s	0.0109 ms / 771 GB/s	0.0107 ms / 783 GB/s
131072	0.1289 ms / 1041 GB/s	0.1107 ms / 1212 GB/s	0.1384 ms / 970 GB/s	0.1118 ms / 1201 GB/s
524288	0.5019 ms / 1070 GB/s	0.5059 ms / 1061 GB/s	0.5369 ms / 1000 GB/s	0.5084 ms / 1056 GB/s

三个规模对应两种访存口径（工作集按读集 $x + y$ 合计计，与第 八 章同口径）：$N = 8192$ 时合计约 8 MB 完全落在 96MB L2 之内，是 L2 口径；$N = 131072$ 时输入合计约 128 MB 已超出 L2：输入数组（64 MB）可基本驻留、写集引起部分逐出，实测带宽（1041–1212 GB/s）介于 HBM 饱和值与 L2 之间；$N = 524288$ 时读写流量 512 MB 远超 L2，是 HBM 口径。两个结构性结论：

1. **vec4 收益只在 L2 口径显著**：$N = 131072$ 行时 rms_norm 1041 → 1212 GB/s（+16%），layer_norm 970 → 1201 GB/s（+24%）。L2 带宽远高于 HBM，标量访存的指令发射与事务数成为瓶颈，向量化直接缩短指令路径。而 HBM 口径（$N = 524288$）下四个 kernel 全部收敛到 1000-1070 GB/s，带宽上限由 DRAM 决定，标量版也能达到该上限，vec4 的收益被吸收。

2. **1-pass 收益在两种口径下都稳定约** 7%：同精度对比标量版 rms_norm vs layer_norm：0.1289 vs 0.1384 ms（L2 口径，−6.9%）；0.5019 vs 0.5369 ms（HBM 口径，−6.5%）。减少的那一次 block reduce 与两次同步在两种口径下都没有被访存完全掩盖：归一化算子的访存量太小，结构开销始终在关键路径上。这正是 LLaMA 选择 RMSNorm 在 kernel 层面的直接收益。

7.7 实践经验

实践经验 7.1 (F3 | 模板参数与 K 的隐含约定)：源码注释只提示「kNumThreads=K=128 (K>128 时调整模板参数)」，但 $K < 128$ 且不改模板参数同样出错：网格按 `idx = blockIdx*128 + tid` 线性展开，一个 block 会覆盖到相邻行的数据，行归约被跨行污染（`idx < N*K` 只防全局越界，不防跨行）。正确约定是 K 必须等于模板参数；支持任意 K 需要行内 padding 或行内循环分块，当前实现未包含。

实践经验 7.2 (F3 | ϵ 的语义未在注释中写明)：源码 `rsqrtf(variance / (float)K + epsilon)` 的 ϵ 加在「均值之后、根号之内」，与 PyTorch LayerNorm/RMSNorm 的 $\sqrt{v + \epsilon}$ 口径一致；个别实现把 ϵ 加在根号外或除在 K 上（`variance/(K+eps)`），语义不同。本章正文式 (7.5) 已固定记号。

实践经验 7.3 (F1 | Llama 使用 RMSNorm 的陈述属实)：「Llama 系列使用 RMS Norm」与公开实现一致（LLaMA/LLaMA 2/LLaMA 3 及 Mistral 等）。补充考据：这些模型同时移除了 RMSNorm 前线性投影的 bias，原因正是 RMSNorm 无平移不变性（本章第 3 节推导的 μ^2 项），bias 无归一化收益。

7.8 常见坑

1. **模板参数与 K 不匹配**：标量版要求 `kNumThreads == K`；$K > 128$ 需调大模板参数，$K < 128$ 会跨行污染（见勘误框注）。

2. **vec4 对齐**：$K\%4 \neq 0$ 或张量基址非 16B 对齐时，`FLOAT4` 触发 misaligned address；此时回退标量版或做尾部标量处理。

3. **写共享标量后漏同步**：`s_mean`/`s_variance` 写后的 `__syncthreads()` 一旦遗漏，竞争窗口小、错误非确定性，表现为「偶尔个别行结果错误」，极难复现定位。

4. **越界 padding 值与归约量不匹配**：平方和 pad 0 正确，pad 大数则溢出；同理行 max 归约必须 pad −FLT_MAX（第 五 章）。

5. **fp16 输入直接用 fp16 累加**：K 较大时平方和可能溢出 fp16 上限（65504），且舍入误差快速累积；正确做法是升 fp32 累加（本章 fp32 演示天然规避，第 十二 章的 F32Acc/F16Acc 容差分级会再遇到它）。

6. **`rsqrtf` 精度预期**：MUFU.RSQ 是近似指令（约 2 ulp），推理场景下影响可忽略；训练反向中 norm 层梯度含 s^3 因子，对精度敏感的实现可在关键路径换 `1.0f/sqrtf()` 换取 IEEE 精度。

7.9　面试要点速查

1. **Q：LayerNorm 与 RMSNorm 的数学关系？** A：恒等式 $\frac{1}{K}\sum x_k^2 = \sigma^2 + \mu^2$；RMSNorm 等价于 LayerNorm 去掉中心化、分母换成 $\sqrt{\sigma^2+\mu^2+\epsilon}$；$\mu \approx 0$ 时两者近似相等。

2. **Q：为什么 RMSNorm 更快？** A：归约量从两个（依赖链）降到一个：1 次 block reduce + 2 次同步 vs 2 次 + 4 次；实测 fp32 下快约 7%（本章第 6 节）。

3. **Q：LayerNorm 能做 1-pass 吗？** A：可以（$\sigma^2 = S_2/K - \mu^2$ 或 Welford），但「大数减小数」有 catastrophic cancellation 风险；fp32 且 $|\mu| \ll \sigma$ 时可安全使用，否则 Welford。

4. **Q：GPU 上实现行归一化的标准结构？** A：one-row-one-block；block reduce 求统计量；tid 0 写 shared 标量；`__syncthreads()` 后全块读出广播；最后逐元素缩放。

5. **Q：为什么写 `s_variance` 后还需要一次 `__syncthreads()`？** A：reduce 内部的同步只覆盖 reduce 自己的 shared 数组；新共享变量的写读对必须另行建立块级可见性。

6. **Q：vec4 化的前提？** A：$K\%4 = 0$、基址与行首 16B 对齐；越界线程的 padding 值必须与归约量语义一致。

7.10　最小测试与动手实验

测试文件 `tests/ch06_norm.cu`（$\leq$ 300 行），逻辑抽取自 `bench/base.cu` 的 `test_rms_norm`（L432）与 `test_layer_norm`（L490），对照参考改为 CPU fp64 手算（`double` 累加求 S_1, S_2，再按式 (7.5) (7.3) 生成参考），容差取 `TOL_F32ACC` $= 10^{-3}$。规模 $N = 512, K = 128$，覆盖 4 个 kernel：

```
cd kernels/interview/book/tests && CUDA_VISIBLE_DEVICES=7 ./build_tests.sh --ch ch06
  PASS rms_norm: max_abs_err=4.392e-07 (tol=1.0e-03)
  PASS rms_norm_vec4: max_abs_err=4.392e-07 (tol=1.0e-03)
  PASS layer_norm: max_abs_err=4.991e-07 (tol=1.0e-03)
  PASS layer_norm_vec4: max_abs_err=5.318e-07 (tol=1.0e-03)
  ch06 ALL PASS
```

性能实验：`./.build/ch06_norm --bench` 复现本章第 6 节表格（cudaEvent min-of-20）。建议动手：把 `kNumThreads` 改成 256 而 $K = 128$ 不变，观察归约结果如何被跨行污染；删除写 `s_variance` 后的 `__syncthreads()` 并用 `compute-sanitizer --tool racecheck` 捕获竞争。

延伸阅读

- 紫气东来，《OneFlow 之 LayerNorm 的 CUDA 实现拆解》：https://zhuanlan.zhihu.com/p/694974164，生产级 LayerNorm kernel（逐通道 gain/beta 向量、fp32 累加）的工程化拆解，与本章教学版互补。

第八章　RoPE 与矩阵转置（Bank Conflict 专题）

8.1　本章导读

本章处理两个表面无关、内核相通的算子。**RoPE**（Rotary Position Embedding，旋转位置编码）是逐元素数学型算子：每对分量做一个 2D 旋转，难点在数学形式的等价变换与超越函数的计算策略。**矩阵转置**是访存型算子的典型教学载体：读合并则写不合并的矛盾要求引入 shared memory 中转，而中转本身又引出本章的重点问题：**bank conflict**。转置 kernel 中学到的 padding 思想，会在第 十三 章升级为 XOR swizzle。

前置章节：第 四 章 coalescing（合并访存事务的概念）；第 二 章 shared memory 层级。

源码区间：`base.cuh` L802-909，3 个 kernel：`rope`（L817）、`mat_transpose`（L858）、`mat_transpose_padded`（L871）。**编译宏与架构限定**：无，fp32，任意 arch。配套最小测试 `tests/ch07_rope_transpose.cu`。

8.2　问题与动机

RoPE 是当前 LLM 位置编码的事实标准（LLaMA、Qwen、PaLM、Mistral 等全部采用），每层 attention 的 Q/K 在进入 attention 计算之前都要经过一次 RoPE 变换。它的核心性质是**内积只依赖相对位置**，这让它天然兼容 KV cache 的增量推理。作为独立算子，RoPE 是逐元素访存 + 少量超越函数；作为融合对象，它常被并入 attention kernel（第 十七 章起会看到 attention kernel 的结构，RoPE 的预计算与融合策略是工程上的常见分叉点）。

矩阵转置则应用广泛：内存布局变换（NHWC ↔ NCHW）、attention 的 QKV 重排、GEMM 中把 B 矩阵预先转置成行主以获得合并访存（第 十 章起大量出现）。它的计算量为零，性能完全由访存模式决定，于是存在一个无法回避的矛盾：

> 按行读就是合并的，那么按列写必然是跨步的。

naive 转置一半访存（写）退化到 1/8 的有效利用率（32B sector 口径；若按 L1↔L2 的 128B cache line 口径则 1/32）；引入 shared memory 中转可以两边都合并，但「按列读 shared memory」又会撞上 shared memory 的物理结构限制：32 个 bank。如何量化冲突、如何用最小的改动消除主要冲突，就是本章 bank conflict 专题的全部内容。

8.3　数学原理

8.3.1　二维旋转与复数形式

RoPE 的基本操作是把一对分量 (x_1, x_2) 视作平面上的一个点，绕原点旋转角度 θ：

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = R(\theta) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \tag{8.1}$$

即 $x_1' = x_1\cos\theta - x_2\sin\theta$，$x_2' = x_1\sin\theta + x_2\cos\theta$。把 (x_1, x_2) 视作复数 $z = x_1 + ix_2$，旋转等价于乘一个模为 1 的复数：

$$R(\theta)\, z \;\equiv\; e^{i\theta} z = (\cos\theta + i\sin\theta)(x_1 + ix_2), \tag{8.2}$$

展开实部虚部与式 (8.1) 逐项一致。复数形式的好处是利用乘法结合律可直接推出相对位置性质（下一小节）。

8.3.2　相对位置不变性：RoPE 的核心性质

设位置 m 的 query 对 (q_1, q_2)、位置 n 的 key 对 (k_1, k_2)，分别旋转 $m\theta$ 与 $n\theta$。attention 打分用的内积（2D 情形即复数内积的实部）满足

$$\big(e^{im\theta} z_q\big)^* \big(e^{in\theta} z_k\big) = e^{-im\theta}\, z_q^*\, e^{in\theta}\, z_k = e^{i(n-m)\theta}\, z_q^* z_k = z_q^*\, R\big((n-m)\theta\big)\, z_k. \tag{8.3}$$

第一步用了共轭 $(e^{im\theta})^* = e^{-im\theta}$ 与乘法结合律；结果只含位置差 $n-m$，与绝对位置 m, n 无关，这就是「内积只依赖相对位置」。矩阵形式同样简洁：旋转矩阵是正交阵且 $R(a)^\top = R(-a)$，故

$$R(m\theta)^\top R(n\theta) = R(-m\theta)\, R(n\theta) = R\big((n-m)\theta\big). \tag{8.4}$$

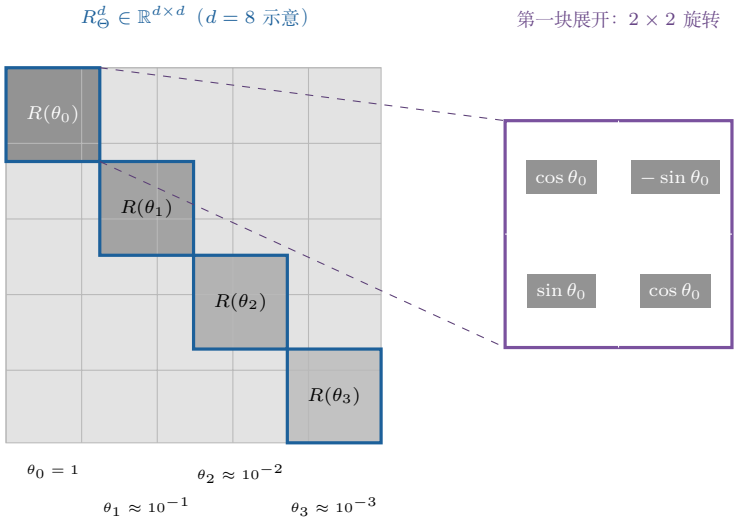
训练时模型看到的打分只随相对位移变化，位置信息以「旋转相位」的形式注入，RoPE 的外推性质就建立在这条数学性质上。

8.3.3　高维形式：分块对角旋转矩阵

d 维向量（d 为偶数）不整体做一个 d 维旋转，而是切成 $d/2$ 个独立的 2D 子空间，各自旋转一个角度：

$$R_\Theta^d = \mathrm{diag}\big(R(\theta_0),\, R(\theta_1),\, \ldots,\, R(\theta_{d/2-1})\big) \in \mathbb{R}^{d\times d}, \qquad \theta_i = 10000^{-2i/d}, \tag{8.5}$$

R_Θ^d 是 $d/2$ 个 2×2 旋转块排布在主对角线上的**分块对角**矩阵；位置 m 的变换是 $R_\Theta^d(m)$，即每个块的角度乘 m。频率序列 $\theta_i = 10000^{-2i/d}$ 是等比数列：$i=0$ 时 $\theta_0 = 1$（波长 2π，最高频，分辨局部），$i = d/2-1$ 时 $\theta_i \approx 10^{-4}$（波长 $\approx 2\pi\times 10^4$，最低频，分辨远距）。基 10000 与第 十六 章 attention 的 softmax 温度一样是经验超参，控制「波长谱」的覆盖范围。



非对角区全零（浅灰）：$d/2$ 个 2D 旋转互相独立；位置 m 的变换为 $R_\Theta^d(m)$——每块角度 $\times m$
灰度语义同图 8.2：同 θ_i 同灰度，θ_0 最深（最高频）→ 渐浅（最低频）

图 8.1: 分块对角旋转矩阵 R_Θ^d 的结构（$d=8$ 示意，即 4 个 2×2 旋转块；式 (8.5)）：第一块展开为 $\left(\begin{smallmatrix}\cos\theta_0 & -\sin\theta_0\\ \sin\theta_0 & \cos\theta_0\end{smallmatrix}\right)$，其余块同构、角度按 $\theta_i = 10000^{-2i/d}$ 等比衰减；非对角区全零，各对独立旋转。块灰度沿 i 渐变，与图 8.2 的分组灰度语义一致。

需要特别注意**分组约定**：式 (8.5) 只规定了「哪两个分量组成一对」，没有规定分组的枚举顺序。常见的两种是**相邻对**（interleaved：$(x_0,x_1),(x_2,x_3),\ldots$，原始论文与本源码采用）与**半分对**（half-split：$(x_0,x_{d/2}),(x_1,x_{d/2+1}),\ldots$，HuggingFace transformers 的 `rotate_half` 实现）。两者是同一数学对象在不同分量排列下的表现形式，两两重排即可互相转换；但 checkpoint 权重与 KV cache 布局如果不做对应重排，结果就是错的，对齐 HuggingFace 权重时这是经典坑（见本章实践经验）。

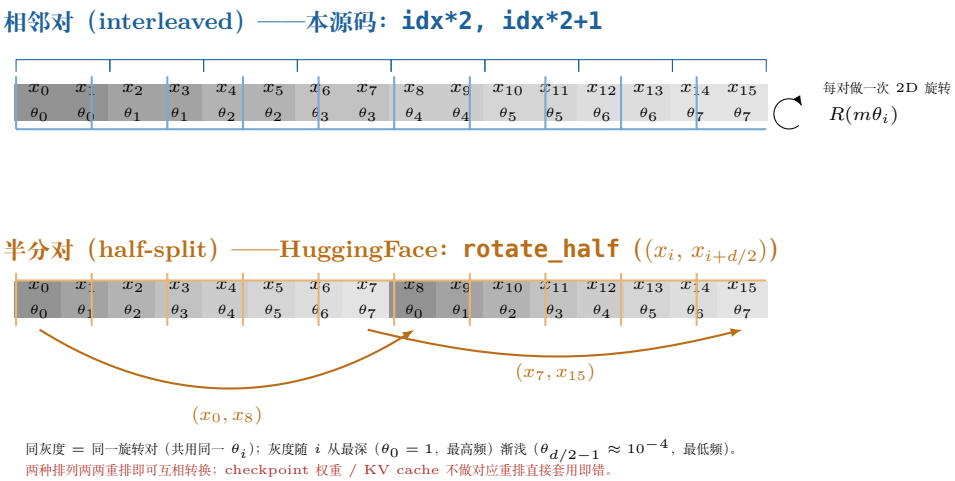


图 8.2: RoPE 两种分组约定（示意 d=16，即 8 对；源码测试为每 token N=64 对）：相邻对 interleaved（源码 `idx*2`/`idx*2+1`）与半分对 half-split（HuggingFace `rotate_half`）。同灰度格属于同一旋转对、共用同一 θ_i；两行是同一分块对角旋转在不同分量排列下的形态。

8.3.4 转置的「数学」

转置本身没有数学难度：$y_{ji} = x_{ij}$。需要形式化的是访存代价。行主存储下 x_{ij} 位于地址 $i \cdot \text{col} + j$。第 四 章的 coalescing 原则要求同 warp 的 32 线程访问连续地址；转置的读与写不可能同时满足：行主读合并意味着列主写跨步，反之亦然。这就是下一节用 shared memory 中转的动机。

8.4 设计与实现

8.4.1 rope：每线程一对元素

base.cuh L805-834（rope：按相邻对分组的 RoPE kernel）

```cpp
// 面试要点:
//   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
//     [x1']   [cos(θ)  -sin(θ)] [x1]
//     [x2'] = [sin(θ)   cos(θ)] [x2]
//   - θ_i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
//   - token_pos = idx / N: token 在序列中的位置
//   - token_idx = idx % N: token 内的维度对索引
//   - 输入 [seq_len, hidden_size], 输出同形状

// Grid:  ((seq_len * N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/rope/rope.cu
__global__ void rope(float *x, float *out, int seq_len, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  float x1 = x[idx * 2];
  float x2 = x[idx * 2 + 1];
  int token_pos = idx / N; // 序列位置
  int token_idx = idx % N; // 维度对索引

  // 频率计算: θ_i = 1 / (10000^(2i/d))
  float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
  float sin_v = sinf(token_pos * exp_v);
  float cos_v = cosf(token_pos * exp_v);

  // 2D 旋转
  float out1 = x1 * cos_v - x2 * sin_v;
  float out2 = x1 * sin_v + x2 * cos_v;
```

```
  out[idx * 2] = out1;
  out[idx * 2 + 1] = out2;
}
```

索引解码。`idx` 是「维度对」的全局编号，范围 $[0, \text{seq_len} \times N)$，其中参数 N 是**每 token 的对数**（$N = d/2$，hidden size 为 $2N$）。`token_pos = idx / N` 解出序列位置 m，`token_idx = idx % N` 解出 token 内的对号 i。一对分量占两个连续地址 `x[idx*2]` 与 `x[idx*2+1]`。

频率计算。`exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f))` 实现式 (8.5) 的 $\theta_i = 10000^{-2i/d}$：分子 `2 * token_idx` 是整数乘法，除以 `(N * 2.0f)`（即 d）时提升为浮点除法，不会截断。角度 `token_pos * exp_v` 即 $m\theta_i$。注意分母用的是 $d = N \times 2$ 而不是 N：若误用 N，频率谱整体偏移一个平方量级，这是把「对数」当「维度数」的经典笔误。

旋转与写回。两行乘加 $x_1' = x_1\cos\theta - x_2\sin\theta$、$x_2' = x_1\sin\theta + x_2\cos\theta$ 与式 (8.1) 逐项对应，编译器可映射为 FMA。

访存与计算策略。相邻线程访问相邻对，warp 的 32 线程读 64 个连续 float（256B，合并为 2 个事务），写同理，已接近最优但还差一步：一对元素本可用一条 `float2` load 完成。计算侧的问题更大：`powf/sinf/cosf` 三个超越函数在每个线程重算，而 θ_i 只依赖对号 i（整张 batch 复用）、$m\theta_i$ 只依赖 (m, i)（每层每次 forward 都相同）。生产实现普遍改为预计算 cos/sin 缓存表（`seq_len` $\times d/2$ 个元素，启动时一次生成）或把 RoPE 融合进 attention/elementwise 链。本章 kernel 保留「每线程重算」的教学形态，把减少重复计算的优化作为练习留给读者实验。

8.4.2　mat_transpose：矛盾的直接呈现

base.cuh L839-865（mat_transpose：naive 转置及其访存注释）

```
// 面试要点（Bank Conflict 专题）：
//   - Shared memory 有 32 个 bank，每个 bank 4 bytes（32-bit）
//   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
//   - n-way bank conflict：n 个线程冲突 → 访问串行化为 n 次
//   - 解决方案：PAD（在每行末尾加 1 个元素，打破地址对齐）
//
// 转置的四步演进：
//   naive：非合并写入（列优先写）→ 每个 warp 产生 32 次内存事务
//   shared：写入 smem（行优先）→ 从 smem 读取（列优先）→ 合并写入 gmem
//   BCF：   smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17]，PAD=1 加在第二维消除 bank conflict
//   merge_write：进一步将 4 次 separate store 合并为 1 次 float4 store
// 注：本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write)，Level 2/3 省略

// 每个线程处理 1 个元素，block(16,16)
// 读：x 按行优先访问，warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
// 写：y 按列优先写入，32 线程跨 row 行分散 → 非合并写入 ✗（32 次内存事务）
// Grid:  ((col + 15) / 16, (row + 15) / 16, 1)，每线程 1 元素
// Block: (16, 16, 1)
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose(float *x, float *y, const int row, const int col) {
  const int c = blockIdx.x * blockDim.x + threadIdx.x;  // col in x
  const int r = blockIdx.y * blockDim.y + threadIdx.y;  // row in x
  if (r < row && c < col) {
    // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
    y[c * row + r] = x[r * col + c];
  }
}
```

每线程搬运一个元素：`y[c * row + r] = x[r * col + c]`。读侧：warp 内 32 线程的 r 固定、c 连续，读 128B 连续段，合并。写侧：c 每加 1，地址跳一整行（`row` 个 float），32 线程写到 32 个互不相邻的 4B，每个 4B 落在各自的内存事务里，产生 32 次写事务（$32 \times 32\text{B} = 1024\text{B}$），只运了 128B 有效数据。源码注释「非合并写入 → 32 次内存事务」说的正是这个。读侧利用率 100%，写侧仅 1/8（32B sector 口径；按 128B cache line 口径则 1/32），总带宽主要受写侧限制（本章第 6 节实测 512 规模只有 ~ 350 GB/s，是 padded 版的 69%）。

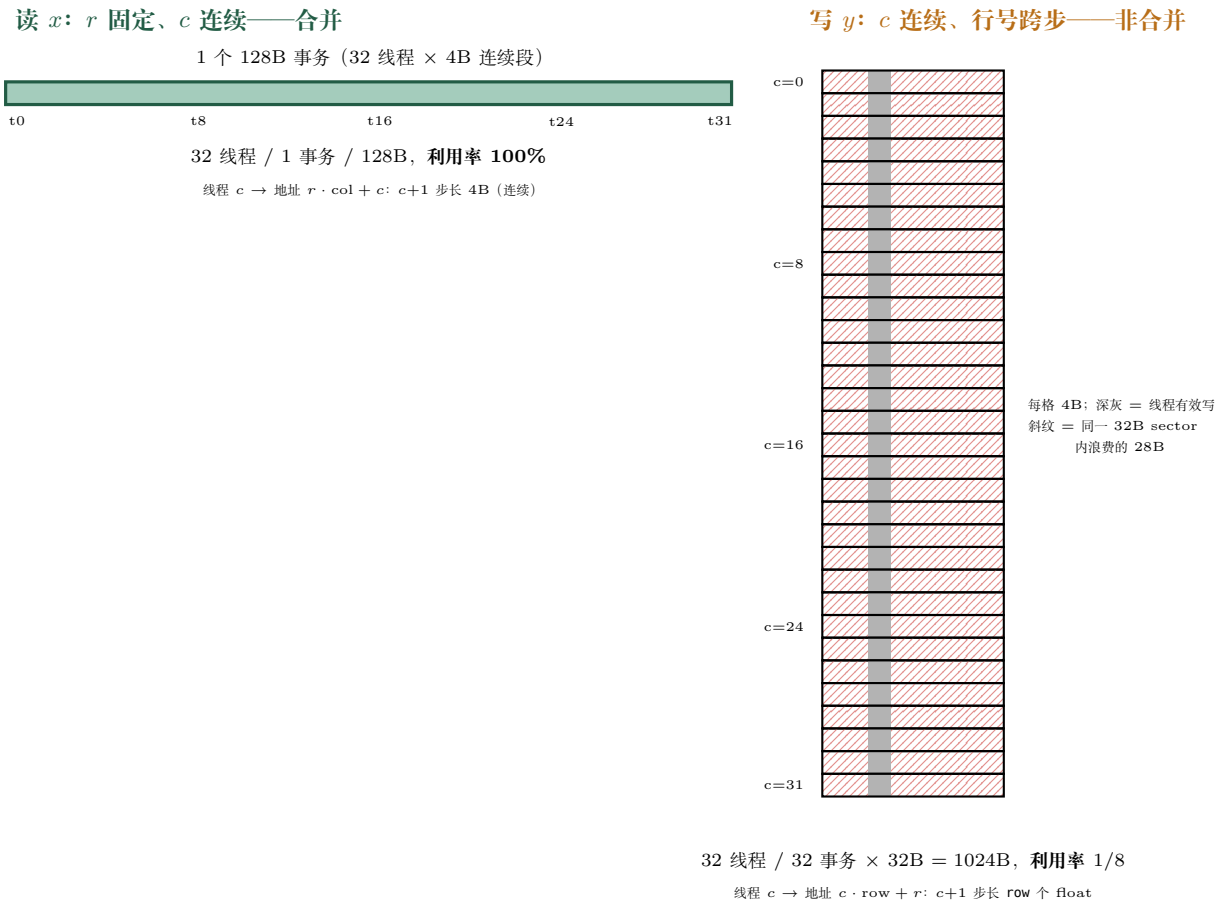


图 8.3: naive 转置（一线程一元素）的访存矛盾：读侧 warp 32 线程（r 固定、c 连续）合并为 1 个 128B 事务、利用率 100%（左）；写侧 32 个 4B 写分散到 y 的 32 行（c 固定列位置），各自命中独立的 32B sector，32 次事务共搬运 1024B 却只有 128B 有效，利用率 1/8（右，斜纹为浪费字节）。

base.cuh L871-907（mat_transpose_padded：smem 中转 + PAD = 1）

```
__global__ void mat_transpose_padded(
    float *x, float *y, const int row, const int col) {
  const int tx = threadIdx.x;
  const int ty = threadIdx.y;

  constexpr int TILE = 16;
  constexpr int PAD = 1;
  // Bank conflict fix: 每行多 1 个元素，打破 32-bank 对齐
  __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16

  // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
  const int x_c = blockIdx.x * TILE + tx;
  const int x_r = blockIdx.y * TILE + ty;

  if (x_r * 4 < row && x_c < col) {
    // Step 1: 4 rows × 1 col → smem (row-major);
#pragma unroll
    for (int i = 0; i < 4; ++i) {
      tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
    }
    __syncthreads();

    // Step 2: Transposed read from smem
    float4 reg_trans;
    reg_trans.x = tile[tx * 4 + 0][ty];
    reg_trans.y = tile[tx * 4 + 1][ty];
    reg_trans.z = tile[tx * 4 + 2][ty];
    reg_trans.w = tile[tx * 4 + 3][ty];

    // y 空间坐标: y_r = x 的列, y_c = x 的行
    const int y_r = blockIdx.x * TILE + ty;          // = x col
    const int y_c = (blockIdx.y * TILE + tx) * 4;    // = x row

    // Coalesced write: y[y_r][y_c .. y_c+3]
    FLOAT4(y[y_r * row + y_c]) = reg_trans;
  }
}
```

8.4.3　mat_transpose_padded：shared memory 中转 + PAD

整体思路是「两侧各自合并，中间经 shared memory 中转」：

1. **Step 1（gmem → smem，合并读）**：`tile[ty*4+i][tx] = x[(x_r*4+i)*col + x_c]`。每线程负责 4 行 × 1 列（`#pragma unroll` 展开 4 次读），warp 内 x_c 连续，读合并。写入 smem 是按行连续写。

2. **Step 2（smem 转置读 → gmem，合并写）**：`reg_trans` 的 4 个分量取 `tile[tx*4+j][ty]`（$j = 0..3$）：行号与列号互换，正是「转置读 smem」。随后 `FLOAT4(y[y_r * row + y_c]) = reg_trans` 把 4 个连续输出元素一条 16B 指令写回；同 warp 内 16 线程覆盖 64 个连续输出 float，合并写。

两个坐标换算值得展开细讲：y 的行号 $= x$ 的列号，故 `y_r = blockIdx.x * TILE + ty`；y 的列号 $= x$ 的行号 $\times 1$，但每线程覆盖 4 个连续行，故 `y_c = (blockIdx.y * TILE + tx) * 4`。`FLOAT4` 写回要求 `y_c` 起 4 个元素 16B 对齐，即 x 的行号是 4 的倍数，这由 `y_c` 的构造天然保证。

smem tile 的形状是 `[TILE*4][TILE+PAD]` = [64][17]：第二维多出的 `PAD = 1` 不是防越界，而是**消除 bank conflict** 的关键，下一节就讲它。

8.5 图示：Bank Conflict 专题

8.5.1 shared memory 的物理结构

shared memory 被切成 32 个 bank（对应一个 warp 的 32 线程），每个 bank 宽 4B；地址按每 4B 一次循环映射到 bank：

$$\text{bank}(addr) = \left(\frac{addr}{4\,\text{B}}\right) \bmod 32. \tag{8.6}$$

硬件每个周期允许每个 bank 服务一次访问。同一 warp 内多个线程访问**同一 bank 的不同地址**时，访问被迫串行成 n 次（n-way bank conflict，带宽除以 n）；访问**同一地址**则是广播（broadcast），一个周期完成，不冲突。分析任何 bank 冲突问题都是固定三步：第一步，确认 warp 到线程坐标的映射；第二步，写出每线程的地址表达式；第三步，对地址做 mod-32 直方图。

8.5.2 本 kernel 的精确冲突分析

`mat_transpose_padded` 的 block 是 $(16, 16)$，线程坐标 (tx, ty)，warp 内线性映射为 `tid = tx + 16*ty`，即 warp 0 覆盖 $ty \in \{0, 1\}$、$tx \in [0, 16)$ 的 32 线程。设 tile 列宽 W（无 PAD 时 $W = 16$，PAD=1 时 $W = 17$），按 4B 字计地址。

Step 2 转置读（每条 LD 指令，j 固定）：地址 $(4\,tx + j)\,W + ty$。

- 无 PAD（$W = 16$）：$64\,tx + 16j + ty$，对 32 取模后 tx 项整体消失（$64 \equiv 0 \pmod{32}$），bank 号 $= (16j + ty) \bmod 32$ **与 tx 无关**，32 个线程全部落在 2 个 bank，每个 bank 16 线程等待，**16-way conflict**，读 shared memory 的吞吐降为 1/16。
- PAD=1（$W = 17$）：地址 $68\,tx + 17j + ty$，而 $68 \equiv 4 \pmod{32}$：tx 每加 1，bank 号前进 4。16 个线程的 bank 号构成 $\{b, b{+}4, b{+}8, \ldots, b{+}60\} \bmod 32$，恰好覆盖 8 个 bank、每个 bank 2 线程，**16-way 降到 2-way**，事务数从 16 降到 2（读吞吐 ×8）。

Step 1 写 smem（每条 ST 指令，i 固定）：地址 $(4\,ty + i)\,W + tx$。无 PAD 时 bank 号 $= (16i + tx) \bmod 32$ 与 ty 无关：warp 的两排 ty 并行写相同 bank 序列，每 bank 2 线程，**2-way**；PAD=1 时行距变为 $68 \equiv 4 \pmod{32}$，两排 bank 序列错开 4，仍有部分重叠，模拟结果**仍是 2-way**（写侧 PAD 前后无改善）。

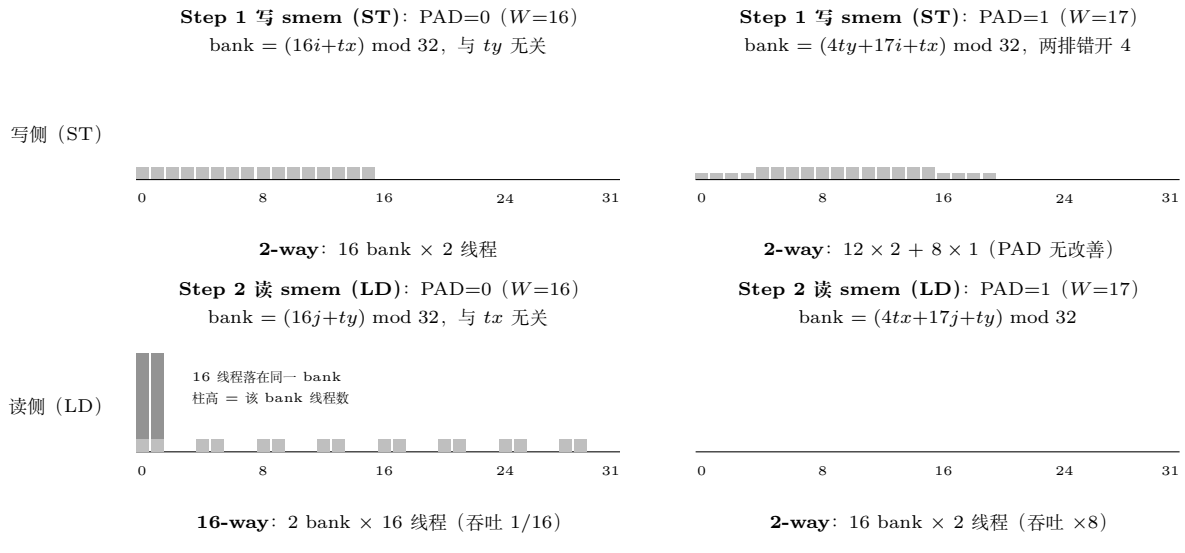


图 8.4: bank 占用直方图（2 指令 × 2 配置，每格为 32 bank 的线程计数，取 $i = j = 0$ 的一条指令）：写侧（ST）PAD=0 与 PAD=1 都是每 bank 至多 2 线程——写侧 PAD 前后无改善；读侧（LD）PAD=0 时 32 线程落在 2 个 bank（16-way，深灰柱），PAD=1 后 tx 每加 1 bank 前进 4，16 个 bank 各 2 线程（2-way）。图 8.5 讲「步长 64B→68B 为何打破对齐」，本图给出两指令两配置的完整占用结果。

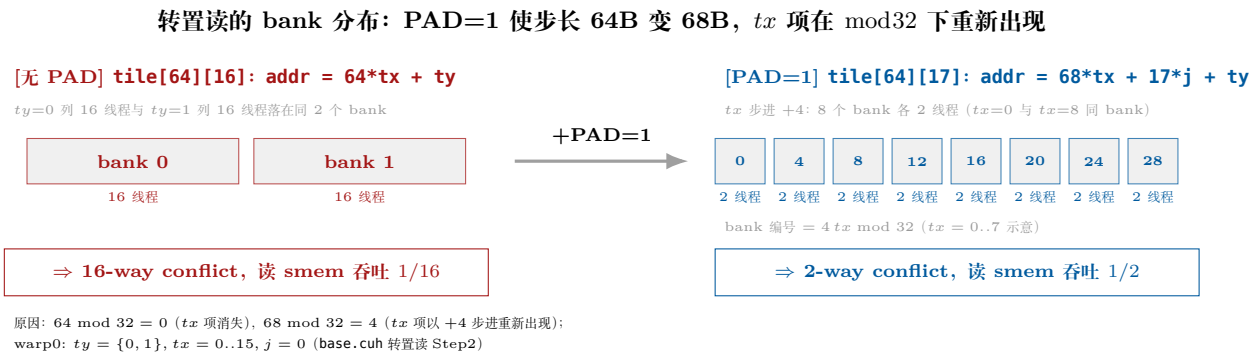


图 8.5：bank conflict vs padding——PAD=1 使步长 64B 变为 68B，tx 项在 mod 32 下重新出现，16-way 冲突降为 2-way（读 smem 吞吐 1/16 → 1/2）。

这里留两个思考题：（1）为什么经典「一线程一元素」的 32×32 tile 加 PAD=1 能做到零冲突，而本 kernel 的 4 行版本不能？（提示：经典版 warp 内 32 线程读同一列，地址步长 $33 \times 4\text{B} \equiv 4\text{B}$，32 线程 bank 号两两不同；本版每线程 4 行、warp 只跨 2 列，自由度更小。）（2）残留的 2-way 怎么彻底消除？把行距从 4 改成别的值、或直接重排 tile 内的物理布局，第 十三 章的 XOR swizzle 会给出系统性答案：用地址位异或代替加法 padding，在不浪费 smem 容量的前提下打散 bank 映射。

8.5.3　模拟验证

上述推导可以不依赖 GPU 复现：按 warp 映射逐指令统计 bank 直方图的模拟（几十行 Python 即可完成），输出与推导一致：

```
tile[64][16] (PAD=0):  Step1 write: 2-way   Step2 read: 16-way
tile[64][17] (PAD=1):  Step1 write: 2-way   Step2 read:  2-way
```

8.6　性能分析

转置是零计算算子（$AI \approx 0$），性能上限 = 带宽上限，优化就是把事务数降到理论最小：读 $M^2/32$ 个事务 + 写 $M^2/32$ 个事务（每 warp 每向一个 128B 段）。

测试环境与口径：NVIDIA RTX PRO 5000 72GB（Blackwell，sm_120a，L2 96MB），cudaEvent 取 20 次 min；$M \times M$ fp32 方阵；有效带宽按读 + 写 $2M^2 \cdot 4\,\text{B}$ 计：

M	mat_transpose (naive)	mat_transpose_padded	加速比
512	0.0060 ms / 351 GB/s	0.0041 ms / 508 GB/s	1.45×
1024	0.0112 ms / 751 GB/s	0.0068 ms / 1242 GB/s	1.65×
2048	0.0347 ms / 966 GB/s	0.0145 ms / 2320 GB/s	2.40×
8192	0.5909 ms / 909 GB/s	0.5318 ms / 1010 GB/s	1.11×

口径划分：$M \le 2048$ 时数据（最大 34MB）完全落在 96MB L2 工作集内，测的是 L2 口径；$M = 8192$ 时读写流量 512MB 远超 L2，是 HBM 口径。

1. **L2 口径下收益放大到 2.4×**：L2 延迟低、带宽高，事务数（而非 DRAM 流量）成为第一瓶颈。naive 的 32 次写事务在 L2 上全部计入开销；padded 的 float4 合并写加上 smem 读从 16-way 降到 2-way，两处共同压低了事务开销。$M = 2048$ 时 padded 有效带宽 2320 GB/s，这是 L2 量级的数字，不代表 HBM。

2. **HBM 口径下收益降至 1.11×**：$M = 8192$ 时数据穿透 L2 落到 DRAM，L2 的 write-back 聚合会把 naive 分散的 4B 写在写回 DRAM 前重新拼成整段，非合并写的惩罚被部分吸收；此时两者都逼近 DRAM 带宽，剩余差距主要来自 smem 2-way 与调度差异。结论：**转置优化的收益必须按数据规模/L2 行为分口径报告**，拿 L2 口径的 2.4× 去外推大矩阵会严重高估。

8.7 实践经验

实践经验 8.1 (F3 | 「PAD 消除 bank conflict」在本 kernel 中是大幅降低而非消除)**：** 源码注释把 PAD 表述为「Bank conflict fix：每行多 1 个元素，打破 32-bank 对齐」与「解决方案：PAD……打破地址对齐」。按本章第 5 节的逐指令 bank 直方图：PAD=1 把 Step 2 转置读从 16-way 降到 2-way（主要收益），Step 1 写侧 PAD 前后均为 2-way。即注释的结论在数量级上成立（读事务 16 → 2），但「消除」并不准确，要彻底消除需要第 十三 章的 swizzle 或改变每线程 4 行的分工。经典「一线程一元素 + 32×33 tile」布局才是 PAD 完全消除冲突的形态。

实践经验 8.2 (F5 | RoPE 的分组约定考据)**：** 本 kernel 按相邻对(interleaved)分组：`idx*2` 与 `idx*2+1` 成对，θ_i 作用于 (x_{2i}, x_{2i+1})。HuggingFace transformers 主流实现（`rotate_half`）按半分（half-split）分组：$(x_i, x_{i+d/2})$。两者经两两重排后数学等价，但直接套用权重不做重排会得到错误结果；社区若干「RoPE 精度不对」的 issue 根因即在此。

实践经验 8.3 (F3 | 参数 N 的语义易误读)**：** rope 的参数 N 是每 token 的**对数**（$d/2$），而非 hidden size；频率分母用的是 $N \times 2$。与第 七 章中 N 表示行数、本章 transpose 中 N 无出现等用法对照，读多文件时需逐处确认参数语义。

8.8 常见坑

1. **padded 版的对齐前提**：grid 按 $(col + 15)/16 \times (row + 63)/64$ 覆盖，分支只判 `x_r*4 < row`；若 row 不是 64 的倍数，边界 tile 的 4 行批读写会越界（例如 $row = 66$ 时访问第 66、67 行）。`FLOAT4` 写回还要求 `row % 4 == 0`（行首 16B 对齐）。测试与 bench 全部使用 64 的倍数规模。

2. **warp 到 (tx, ty) 的映射想当然**：bank 分析第一步就是确认映射，`(16,16)` block 是 `tid = tx + 16*ty`（x 优先）。映射记错，直方图全错，冲突度结论随之全错。

3. **同地址是 broadcast 不是 conflict**：同 bank 同地址（全 warp 读同一标量）一个周期完成；先判地址是否相同，再判 bank 是否相同。

4. **grid 尺寸复制粘贴**：naive 的 y 向是 $(row + 15)/16$，padded 是 $(row + 63)/64$，每线程覆盖行数变了 4 倍，漏改 grid 会让输出一半是旧值（本测试刻意先清零 y 再启动 padded kernel，防止「读到残值也算对」的假阳性）。

5. **超越函数精度**：`powf` 相对误差 $\sim 10^{-6}$，经 $m\theta_i$ 放大后（$m = 512, \theta_0 = 1$ 时角度绝对误差 $\sim 5 \times 10^{-4}$）仍在本章容差内；长序列（m 上万）时误差线性放大，生产实现用 fp64 预计算或查表规避。

6. **转置 kernel 的写侧假阳性**：若不清零输出缓冲，未覆盖区域的旧值可能与参考「碰巧一致」，掩盖 grid 覆盖不全的 bug。

8.9 面试要点速查

1. **Q：shared memory 的 bank 结构？** A：32 个 bank、每 bank 4B 宽、每周期每 bank 服务一次；连续 4B 字按 $(addr/4) \bmod 32$ 轮转映射。

2. **Q：什么是 n-way bank conflict？和 broadcast 的区别？** A：同 warp 多线程访问同 bank **不同**地址 → 串行 n 次，带宽 $1/n$；同地址 → 广播，无代价。

3. **Q：为什么 +1 padding 能打破冲突？** A：Step 2 中相邻 tx 相隔 4 行，字步长从 $4 \times 16 = 64 \equiv 0 \pmod{32}$（对 32 bank 周期整除，列访问全部命中同一 bank）变为 $4 \times 17 = 68 \equiv 4 \pmod{32}$（行距 $17 \times 4 = 68$B），bank 号每线程前进 4，不再对齐回绕。代价是每行多 4B smem（本例 64 行多 256B，约 6%）。

4. **Q：RoPE 为什么满足相对位置不变性？** A：旋转矩阵正交，$R(m\theta)^\top R(n\theta) = R((n-m)\theta)$；复数形式即 $e^{-im\theta} e^{in\theta} = e^{i(n-m)\theta}$。

5. **Q：$\theta_i = 10000^{-2i/d}$ 的设计意图？** A：波长 $\lambda_i = 2\pi/\theta_i$ 构成 2π 到 $\sim 2\pi \times 10^4$ 的等比谱，高频分量分辨局部、低频分量分辨远距，多尺度覆盖长程依赖。

6. **Q：interleaved 与 half-split 分组的区别？** A：同一分块对角旋转的不同分量排列，两两重排互相等价；权重/cache 对齐时必须按 checkpoint 约定处理。

7. **Q：转置为什么要经过 shared memory？** A：读合并与写合并不可兼得（行主存储下二者互斥）；smem 中转让 gmem 两侧都合并，代价是 smem 容量、一次 `__syncthreads()` 与潜在的 bank conflict。

8.10　最小测试与动手实验

测试文件 `tests/ch07_rope_transpose.cu`（≤ 300 行），逻辑抽取自 `bench/base.cu` 的 `test_rope`（L554）与 `test_mat_transpose`（L610）、`test_mat_transpose_padded`（L656）。RoPE 的 CPU 参考用 `double` 直接按旋转矩阵计算（`std::pow/std::cos/std::sin`）；转置的参考是纯数据搬运，两个 kernel 的 `max_abs_err` 应为 0，且测试显式断言两 kernel 输出逐元素 bitwise 一致。规模：RoPE `seq_len`=128、$N = 64$（对/token）；转置 512×512。

```
cd kernels/interview/book/tests && CUDA_VISIBLE_DEVICES=7 ./build_tests.sh --ch ch07
  PASS rope: max_abs_err=7.099e-06 (tol=1.0e-03)
  PASS mat_transpose: max_abs_err=0.000e+00 (tol=1.0e-03)
  PASS mat_transpose_padded: max_abs_err=0.000e+00 (tol=1.0e-03)
  ch07 ALL PASS
```

性能实验：`./.build/ch07_rope_transpose --bench` 复现本章第 6 节表格；bank 直方图可用本章第 5 节的模拟复现。建议动手：把 `PAD` 改成 2、4，观察 Step 2 的冲突度如何随步长变化（模拟脚本改一行即可）；把每线程 4 行改成 2 行（tile 变 [32][17]、grid 相应调整），看 2-way 残留是否消失，同时体会第 十三 章为什么需要 swizzle。

延伸阅读

- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021（RoPE 的第一出处：$\theta_i = 10000^{-2i/d}$ 等比频率谱、内积只依赖相对位置与原始 interleaved 分组约定的来源，对应本章式 (8.1) 与分组约定考据）。
- 懒蚂蚁呀不嘿，《CUDA matrix transpose 优化（bank conflict 消除）专题》：https://zhuanlan.zhihu.com/p/1899760505733756129，转置四步演进（naive → smem → padding → merge write）的完整实验，与本章源码的 Level 1/Level 4 对照阅读。
- frankshi，《GPU shared memory bank conflict 与 swizzle》：https://zhuanlan.zhihu.com/p/4746910252，swizzle 的位运算原理，本章 padding 残留问题的进阶答案，第 十三 章的预告篇。

第二部分

GEMM 篇：从 CUDA Core 到 Tensor Core

第九章　SGEMV：memory-bound 的三种划分

9.1　本章导读

GEMV（GEneral Matrix-Vector multiplication，矩阵向量乘）是 GEMM 在 $N = 1$ 时的退化形式，也是大模型推理 decode 阶段最基本的算子形态：每次前向只处理一个 token，权重矩阵乘以 activation 向量。它没有 GEMM 那种靠分块换来的复用收益：输出只剩一个向量，第 十 章金字塔的第一级（block tile 复用）在 $N = 1$ 时不再存在。本章沿教学源码 sgemv.cuh 的三个 kernel，把「纯 memory-bound 算子怎么写」这条线走完整：

- **sgemv_k32**（sgemv.cuh L24）——基础 **warp-per-row**：一个 warp 负责矩阵的一行，32 个 lane 沿 K 维铺开，每轮覆盖 32 个元素；
- **sgemv_k128**（sgemv.cuh L53）——**float4 向量化**：每个 lane 每轮用一条 128-bit load 处理 4 个元素，一个 warp 一轮覆盖 128 个 K 元素；
- **sgemv_k16**（sgemv.cuh L85）——$K < 32$ 时的**双行复用**：`kRowPerWarp=2`，前 16 个 lane 与后 16 个 lane 各算一行，用「分段宽度」的 XOR 蝶形归约各自收敛。

前置章节：第 二 章的 roofline 模型（算术强度决定优化方向，本章全篇都在用它）、第 三 章的 warp 归约原语（三个 kernel 的收尾都落在 `warp_reduce_sum` 上）、第 四 章的向量化访存与合并访问。**源码区间**：sgemv.cuh L1–102，无编译宏限定，全部架构适用；配套最小测试为 `book/tests/ch08_sgemv.cu`。**后继**：第 十 章把 N 维度放开，复用维度回归，算术强度从 $O(1)$ 变为随 tile 尺寸增长，重点也从带宽利用转向分块复用。

9.2　问题与动机

动机一：decode 阶段的形状就是 GEMV。语言模型自回归推理分两个阶段：prefill 一次处理整段 prompt，是大 M、大 N 的 GEMM；decode 每步只生成一个 token，投影矩阵 $W \in \mathbb{R}^{M\times K}$ 乘以单个 activation 向量 $x \in \mathbb{R}^{K}$，输出 $y \in \mathbb{R}^{M}$，$N = 1$，典型的 GEMV（工程上常把若干步 decode 的请求拼成 $N = 2, 4, \ldots$ 的 skinny GEMM，但单请求视角仍是 GEMV）。权重矩阵一遍推理要被读成百上千次，GEMV 的效率直接决定 decode 吞吐。

动机二：复用维度消失，AI 是常数。第 十 章会看到，GEMM 的核心机制在于「一个 B tile 服务 B_M 个输出行、一个 A tile 服务 B_N 个输出列」。$N = 1$ 时输出只有 M 个元素，y_m 需要 A 的第 m 行整行与 x 的完整拷贝；**行与行之间没有共享**（A 的每个元素只被用一次），唯一的复用是 x 被 M 行共享，这层复用交给 L2 缓存即可，kernel 层面无事可做。于是算术强度本身就是 $O(1)$ 量级的常数（8.3 节精确推导为 $0.25 \sim 0.5$ FLOP/B），比现代 GPU 的 roofline 拐点低两个数量级。**GEMV 的全部优化目标只有一个：让有效访存带宽逼近机器上限**。计算侧无论怎么安排都不是瓶颈。

动机三：K 的大小决定 32 个 lane 能否占满。warp-per-row 的自然形态是 32 个 lane 铺满一行。$K \gg 32$ 时，每个 lane 要串行累加多个元素，也就有了向量化（每 lane 每轮搬 4 个）的余地；K 恰为 32 的倍数时，一轮覆盖整行、循环次数为整数；而 $K < 32$（比如 head 维度、或某些量化场景下的 16）时，一个 warp 铺不满一行，直接写会浪费一半 lane，这时应该让一个 warp 同时负责**多行**，把 32 个 lane 切成两段各 16 个。三个 kernel 正是对这三种 K 形态的针对性设计，源码头注释（L9–12）把这套决策浓缩成了三行。

9.3 数学原理

9.3.1 定义与算术强度：两种计数口径

GEMV 的定义为

$$y_m = \sum_{k=0}^{K-1} a_{m,k}\, x_k, \qquad m = 0, 1, \ldots, M-1, \tag{9.1}$$

即 $A \in \mathbb{R}^{M\times K}$（row-major）、$x \in \mathbb{R}^K$、$y \in \mathbb{R}^M$。总 FLOPs 为 $2MK$（乘、加各计一次）；访存量：读 A 一遍 $4MK$ 字节，读 x、写 y 各 $4K$、$4M$ 字节。关键在 x 怎么计数：

$$\mathrm{AI}_{\text{乐观}} \approx \frac{2MK}{4MK + 4K + 4M} \approx \frac{1}{2} \qquad (x \text{ 只读一次：} M \text{ 大时 } x \text{ 常驻 L2，边际成本趋零}), \tag{9.2}$$

$$\mathrm{AI}_{\text{悲观}} = \frac{2MK}{4MK + 4MK} = \frac{1}{4} \qquad (\text{每个行 warp 都从显存重读一遍 } x)。 \tag{9.3}$$

两个口径都与 M、K **无关**，GEMV 的 AI 是常数，这就是源码头注释「AI ≈ O(1)，瓶颈在内存带宽」（sgemv.cuh L7）的精确含义（定性核校通过）。

放进第 二 章的 roofline：$P \le \min(P_{peak},\ \mathrm{AI}\cdot\mathrm{BW})$。以本章实测口径（RTX PRO 5000，fp32 峰值按运行时查询折算约 66.9 TFLOPS；可达带宽以 8.6 节 k128 实测的 934 GB/s 为参照），拐点约 $66.9/0.934 \approx 71.6$ FLOP/B。GEMV 的 AI $\approx 0.25 \sim 0.5$ 落在拐点左侧两个数量级：计算上限 $\approx 0.5 \times 934 \approx 0.47$ TFLOPS，只占峰值的 0.7%。**kernel 写得再「聪明」，FLOPs 侧也毫无收益；能优化的只有带宽**：减少指令数（向量化）、消灭闲置 lane（多行复用）、保证合并访问（coalescing），目标都是使 DRAM 带宽达到饱和。

9.3.2 warp-per-row 的覆盖数学

三版共同的启动形状是 `block(32,4)`、`grid(ceil(M/4))`。CUDA 把 block 内线程按 x 最快线性化，于是每个 warp 恰好是「`threadIdx.x` $\in [0,32)$、`threadIdx.y` 固定」的一组线程，**一个 warp 天然对应一行**：

$$m = \mathrm{blockIdx.x}\cdot 4 + \texttt{threadIdx.y}, \qquad k = 32\,w + \mathrm{lane}, \quad w = 0, \ldots, \lceil K/32\rceil - 1. \tag{9.4}$$

第 w 轮 32 个 lane 连续覆盖 $[32w,\ 32w+32)$，轮数 `NUM_ITERS` $= \lceil K/32\rceil$。每 lane 串行累加 `NUM_ITERS` 项后，做一次第 三 章的 5 步 XOR 蝶形归约，lane 0 写回 y_m。行守卫 `if (m < M)` 是 **warp 一致**的（同一 warp 内所有 lane 共享 m，要么全体执行要么全体跳过），因此蝶形归约里 `__shfl_xor_sync` 的掩码 `0xffffffff` 是安全的；这个「一致/不一致」的判别在 k16 那里会反过来，见 8.7 节勘误二。

9.3.3 float4 的覆盖与对齐

k128 把每 lane 每轮的工作量从 1 个元素提到 4 个：

$$k = 4(32\,w + \mathrm{lane}), \quad \texttt{NUM_ITERS} = \lceil\lceil K/32\rceil/4\rceil = \lceil K/128\rceil. \tag{9.5}$$

一条 `float4` load（128-bit）取回 $a_{m,k..k+3}$，与 $x_{k..k+3}$ 做 4 次 FMA 后累入同一个 `sum`。对齐需要三个条件同时成立：（1）K 是 128 的倍数，这同时保证每一行的起点 $a + mK$ 也是 16B 对齐；（2）基址本身 16B 对齐（`cudaMalloc` 返回 256B 对齐，天然满足）；（3）步进恒为 4 个 float（式 9.5 保证）。三个条件的交集就是源码注释「最适合 K 按 128 对齐」（L51）。

9.3.4 K=16：双行复用与分段蝶形归约

$K = 16 < 32$ 时，单行数据占不满一个 warp。k16 的做法是把 warp 沿行方向分成两半：

$$\begin{aligned} &\texttt{kNumLanePerRow} = \lceil 32/\texttt{kRowPerWarp}\rceil = 16, \\ &k = \mathrm{lane} \bmod 16, \qquad m = (\mathrm{blockIdx.x}\cdot 4 + \texttt{threadIdx.y})\cdot 2 + \lfloor \mathrm{lane}/16\rfloor. \end{aligned} \tag{9.6}$$

lane 0 … 15 算第 m 行、lane 16 … 31 算第 $m+1$ 行，每 lane 恰好一个元素。收尾的归约换成 `warp_reduce_sum<16>`：蝶形掩码序列退化为 $8, 4, 2, 1$，且 `__shfl_xor_sync` 的第 4 个参数（width= 16）把数据交换限制在 16-lane

segment 之内，前 16 个 lane 与后 16 个 lane 各自独立收敛、互不干扰。这正是第 三 章讲 `__shfl_xor_sync` 时「width 参数划分 segment」的实战用法。写回条件相应变成 `k == 0`（**不是** `lane == 0`）：满足条件的 lane 是 0 和 16 两个，分别写 y_m 与 y_{m+1}，逐步数据流见图 9.1。

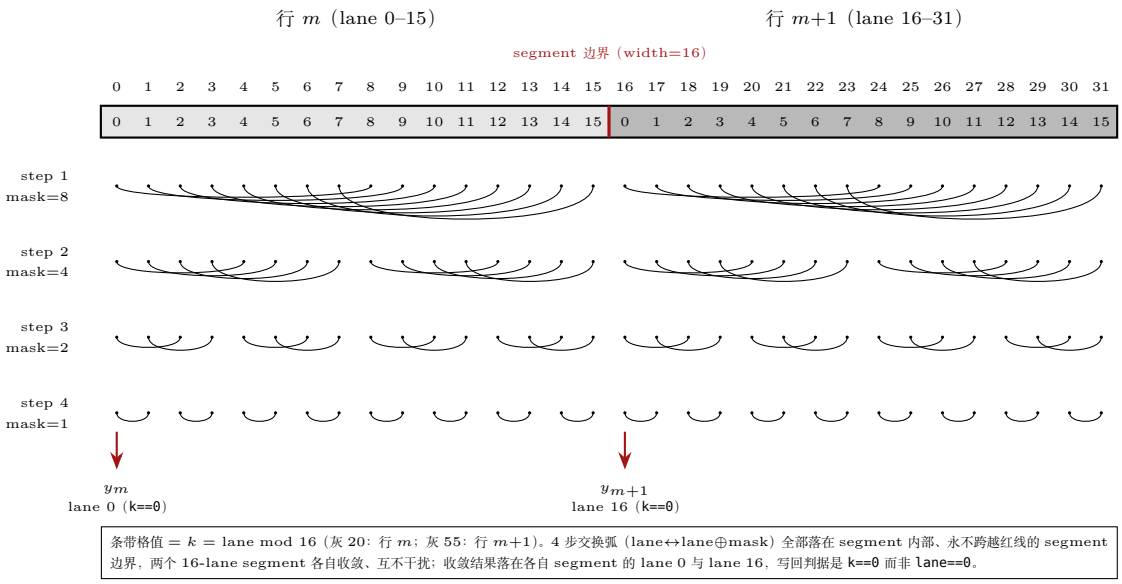


图 9.1: k16 分段蝶形归约逐步数据流：width=16 把一个 32-lane warp 切成两套独立归约（红线为 segment 边界）。上：lane 条带（格值 $k = \text{lane} \bmod 16$，前 16 个 lane 算行 m、后 16 个算行 $m{+}1$）；下：4 步 `__shfl_xor_sync`（mask 8/4/2/1）的交换弧，弧线均在 segment 内部；收敛后 lane 0 与 lane 16（`k==0`）分别写回 y_m、y_{m+1}（式 9.6）。

9.4　设计与实现

源码文件头的策略注释把 8.2 节的三个动机浓缩成了一张可执行的选型表：GEMV（$a \in \mathbb{R}^{M\times K}$，$x \in \mathbb{R}^{K}$，$y \in \mathbb{R}^{M}$，$y = a \cdot x$，$N = 1$）是典型的纯 memory-bound 算子：AI $\approx O(1)$，瓶颈在内存带宽；核心策略是 warp-per-row：每个 warp 负责矩阵的一行。不同 K 值对应不同分块策略：

- K 为 32 的倍数：一个 warp 的 32 线程恰好覆盖 K 维；
- K 为 128 的倍数：每个线程用 `float4` 处理 4 个元素，一个 warp 覆盖 128 个 K 元素；
- $K = 16 < 32$：一个 warp 用不满，用 `kRowPerWarp=2` 让每个 warp 处理 2 行。

9.4.1　sgemv_k32：基础 warp-per-row

sgemv.cuh L16-45（Level 1：warp-per-row 基础版）

```
// ---- SGEMV K32: 基础 warp-per-row ----
// 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖，否则内层循环 kNumWarps 次)
// grid(M/4), 每个 warp 负责一行
// K 为 32 的倍数时，warp 的 32 个线程恰好覆盖 K 维
// Grid:  ((M + 3) / 4, 1, 1), 每 block 处理 4 行
// Block: (32, 4, 1), 每行 1 warp
// 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int lane = tx % kWarpSize; // 0~31
  int m = blockIdx.x * blockDim.y + ty; // 全局行号
  if (m < M) {
    float sum = 0.0f;
```

```
    // 沿 K 维的迭代数 = ceil(K/32)，每个 warp 要累加完整的K，那么
    // 每个thread就要负责累加NUM_ITERS个元素，NUM_ITERS = ceil(K/32)
    const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      // 假设K是32的整倍数，m * K 本行的起始地址，x: Kx1
      int k = w * kWarpSize + lane;
      sum += a[m * K + k] * x[k];
    }
    sum = warp_reduce_sum<kWarpSize>(sum);
    // 每个 warp 处理一行，lane 0 写回结果
    if (lane == 0)
      y[m] = sum;
  }
}
```

对照式 9.4 逐行读：(1) `lane = tx % kWarpSize` 在 `blockDim.x=32` 时就是 `tx` 本身，取模是防御性写法；(2) 内层循环每轮做一次 `a[m*K+k] * x[k]`，`a` 的 32 个 lane 地址连续（合并访问，128B 事务），`x` 的 32 个 lane 地址同样连续；`x` 只有 16KB （$K = 4096$ 口径），第一轮之后基本驻留在 L1/L2，所以「重读 x」的悲观口径（式 9.3）在实际上是乐观口径；(3) `warp_reduce_sum` 收尾后只有 lane 0 写回，一个 warp 写出 4 字节，写流量占比可忽略。注意 kernel **没有** $k < K$ 的守卫：K 不是 32 的倍数时最后一轮会越界读（8.8 节坑一），这是「对齐换简洁」的明确约定。

9.4.2 sgemv_k128：float4 向量化

sgemv.cuh L47-75（Level 2：float4 向量化）

```
// ---- SGEMV K128: float4 向量化 ----
// 每个线程处理 4 个元素(float4)，一个 warp 覆盖 128 个元素
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意：该版本最适合 K 按 128 对齐，且 x/a 的地址满足 float4 对齐
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int lane = tx % kWarpSize; // 0~31
  int m = blockDim.y * blockIdx.x + ty;

  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/128)，每个 warp 每轮用 float4 覆盖 128 个 K 元素
    const int NUM_ITERS = (((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      int k = (w * kWarpSize + lane) * 4;
      float4 reg_x = FLOAT4(x[k]);
      float4 reg_a = FLOAT4(a[m * K + k]);
      sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
              reg_a.w * reg_x.w);
    }
    sum = warp_reduce_sum<kWarpSize>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}
```

三处值得停下来看。(1) `FLOAT4(v)` 宏（common.cuh L26）把任意左值 `reinterpret_cast` 成 `float4*` 再解引用，load 与 store 都能用，是全书向量化的标配工具；(2) `NUM_ITERS` 的双层取整（式 9.5）写成了 `ceil(ceil(K/32)/4)` 的展开式，语义等价于 `ceil(K/128)`；(3) 累加把 4 次 FMA 折成一行括号表达式，编译器会生成 4 条独立

的 FFMA，但对 memory-bound kernel 这不重要（计算只占峰值 1% 不到），真正的好处是 **load 指令数与事务数**：每 warp 每轮 32 条 128-bit load 替代 128 条 32-bit load，指令发射压力降为 1/4，内存管线更容易饱和。8.6 节实测它带来约 5% 带宽提升。

9.4.3　sgemv_k16：K=16 双行复用

sgemv.cuh L77-100（Level 3：kRowPerWarp=2 分段归约）

```
// ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
// 面试亮点：K=16 < 32，一个 warp 可以处理多行
// kRowPerWarp=2，kNumLanePerRow=16，前 16 个 lane 处理 row0，后 16 个 lane 处理 row1
// Grid:  ((M + 7) / 8, 1, 1)，NUM_ROWS=8
// Block: (32, 4, 1)
// 注意：这一版是面向 K=16 的专用写法；kRowPerWarp=2 时一个 warp 同时处理 2 行
// source: LeetCUDA/kernels/sgemv/sgemv.cu
template <const int kRowPerWarp = 2>
__global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
  constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int lane = tx % kWarpSize;
  int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
  int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
  if (m < M) {
    float sum = A[m * K + k] * x[k];
    // 按照kNumLanePerRow=16，分2组各自做 warp reduce sum，k==0的lane写回结果
    sum = warp_reduce_sum<kNumLanePerRow>(sum);
    // 注意：判断条件是 k == 0，不是 lane == 0!
    if (k == 0)
      y[m] = sum;
  }
}
```

对照式 9.6：（1）模板参数 kRowPerWarp 把「一个 warp 几行」做成编译期常量，kNumLanePerRow 随之推导为 16；（2）每个 lane 只做一次乘加（$K = 16$ 恰好等于 segment 宽度，无循环）；（3）warp_reduce_sum<16> 内部蝶形掩码从 8 开始递减，width 参数保证前 16 与后 16 两个 segment 互不干扰；（4）写回判据 k == 0 有两个命中 lane（0 与 16），分别写回两行的结果。一个附带的好处：row-major 下第 m 行与第 $m+1$ 行在内存里首尾相接，warp 的 32 个 lane 恰好拼出一段 128B 连续地址，**合并访问没有因为 K=16 而损失**。但要注意行守卫的粒度变了：if (m < M) 在 lane 0 . . . 15 与 16 . . . 31 之间可能分叉（M 为奇数时的最后一个 warp），这违反 shuffle 掩码的语义要求，8.7 节勘误二详细展开，测试也据此固定 $M\%8 = 0$。

9.5 图示

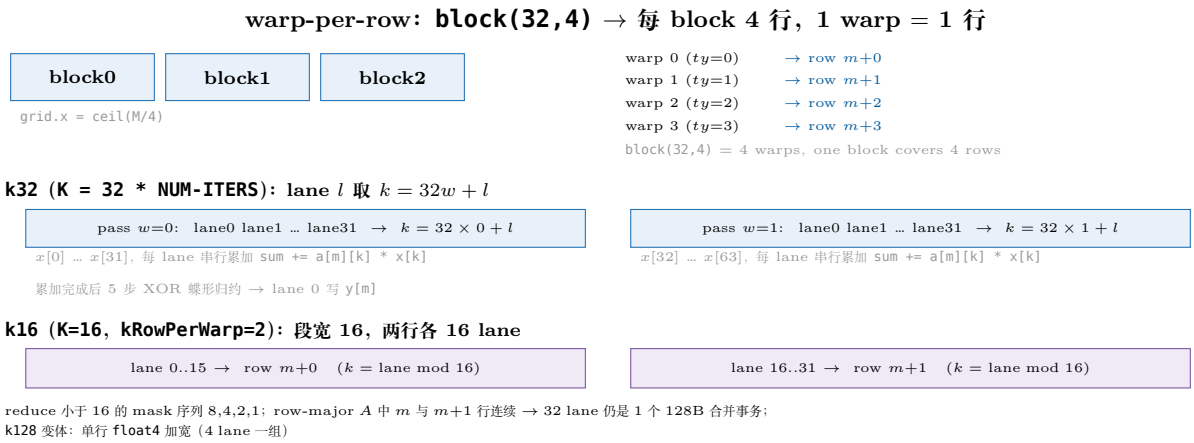


图 9.2: warp-per-row 的划分：k32 单行 32 lane 铺满、k16 双行各 16 lane 分段归约（mask 8/4/2/1）；k128 变体单行 float4 加宽。

图 9.2 的三段对应三个 kernel。读图的关键维度是「warp 的 32 个 lane 怎么分派给 (m, k)」：k32/k128 是一行独占一个 warp，差异只在每 lane 每轮搬几个 k；k16 则把行方向也用上了：当 K 维度不足以占满 32 个 lane 时，把多余的 lane 调去算下一行，而不是让它们空转。

9.6 性能分析

本章复测（RTX PRO 5000 72GB Blackwell，sm_120，110 SMs；时钟按运行时查询 2.38 GHz，fp32 峰值按每 SM 128 条 FMA 流水（256 FLOP/cycle）折算，约 66.9 TFLOPS；warmup 5 + 50 次均值（k16 为 100 次））：

表 9.1: ch08 三版 SGEMV 耗时与有效带宽（有效流量 $= 4MK + 4K + 4M$ 字节）

实现	形状	ms/iter	有效带宽 (GB/s)
k32	M=8192, K=4096	0.1512	888.3
k128	M=8192, K=4096	0.1437	934.2
k16	M=1048576, K=16	0.1078	661.4

四个观察：（1）**k128 就是本机可达带宽的参照**。A 矩阵 128MB 远大于 L2，每 iter 把它完整读一遍，934 GB/s 的有效带宽已经把 launch、写回等固定开销摊薄到可忽略，后续章节（如第 十 章的 roofline 定位）直接引用这个数作为「实测带宽口径」。（2）**float4 带来** +5.2%（934 vs 888）。对纯带宽型 kernel 这已是全部收益空间：指令数降为 1/4、事务利用率更高；它不改变字节数，所以不可能有数量级的变化。（3）**k16 是 661 GB/s**。原因有二：每 lane 只有一个元素可累加，没有任何 ILP 去藏访存延迟；每个 block 只有 128 线程、block 数高达 13 万，调度与收尾开销占比上升。仍能到参照值的 71%，说明双行复用的设计是有效的。（4）**roofline 定位**：AI ≈ 0.5（式 9.2）意味着计算侧上限仅 ≈ 0.47 TFLOPS（峰值的 0.7%），三版 kernel 的差距全部来自带宽利用是否充分，与计算速度无关。

9.7 实践经验

实践经验 9.1 (F3)： sgemv.cuh L17：「否则内层循环 kNumWarps 次」表述错误 该行注释描述 k32 在 K 非 32 倍数时的行为，写「否则内层循环 kNumWarps 次」。首先 `kNumWarps` 这个名字在本文件中不存在（block 内 warp 数实为 4，见 L21）；其次内层循环次数是 `NUM_ITERS` $= \lceil K/32 \rceil$（L33），与 warp 数无关，仅当 $K = 128$ 时 $\lceil K/32 \rceil = 4$ 与 warp 数巧合相等。建议按 L31–33 的代码事实理解：一轮覆盖 32 个元素，轮数由 K 唯一决定。

实践经验 9.2 (F2)： sgemv_k16：行守卫在半 warp 粒度分叉，掩码 0xffffffff 属未定义行为 k32/k128 的行守卫 `if (m < M)` 是 warp 一致的；k16 由于一个 warp 携带两行（式 9.6），当 M 为奇数且该 warp 恰好覆盖最后一行时，前 16 个 lane 通过守卫、后

16 个 lane 不通过，`warp_reduce_sum<16>` 里的 `__shfl_xor_sync(0xffffffff, ...)` 把未执行该指令的 lane 命名进了掩码。按 CUDA C++ Programming Guide 的规定，「掩码中命名的所有线程必须都执行同一条 shuffle」，否则行为未定义。实践中当前硬件通常「碰巧」正确（分段宽度限制了实际交换范围），但这是不能依赖的语义。规避方式：保证 $M\%8 = 0$（每 block 8 行，守卫 warp 一致），或改用协同组、或让掩码与活跃 lane 精确匹配。本章测试固定 $M\%8 = 0$ 以避开该路径。

实践经验 9.3 (F3 核校通过)**：** sgemv.cuh L7：「AI ≈ O(1)」的定性判断 精确推导见式 9.2 与式 9.3：AI 是与 M、K 无关的常数（0.25 ∼ 0.5 FLOP/B），「O(1)、瓶颈在带宽」的定性结论成立。另 L10–12 的三种 K 形态选型与三个 kernel 的实际设计一一对应。

9.8　常见坑

坑一：K 不对齐，静默越界。k32/k128 没有 $k < K$ 守卫：K 不是 32（或 128）的倍数时，最后一轮 `a[m*K+k]` 与 `x[k]` 直接越界读。它不报错、不崩溃，结果看起来还「基本对」，越是接近边界越是隐蔽。`compute-sanitizer` 一测即能发现。调用方必须保证对齐，或在外层把尾块切给通用路径。

坑二：float4 的 16B 对齐三条件。$K\%128 = 0$（保证行内偏移与行起点同时对齐）、基址 16B 对齐（`cudaMalloc` 天然满足；但若传入的是某大 buffer 的 `offset` 视图就要小心）、步进是 4 的倍数（式 9.5 自动满足）。任何一个不满足都是 misaligned access，kernel 直接异常终止；相比坑一的静默越界更容易暴露，但同样在生产环境不可接受。

坑三：k16 的 M 必须是 8 的倍数。不是正确性一定会坏，而是 8.7 节勘误二所述的掩码语义约束。另外：k32/k128 只要求调用方处理 $M\%4$（守卫内消化），两个 kernel 的边界约定粒度不同，混用时要分开记。

坑四：写回条件 `lane==0` 与 `k==0` 不可互换。k32/k128 每 warp 一个写者（lane 0）；k16 每个 segment 一个写者（k==0 命中 lane 0 与 16）。把 k16 的写回写成 `lane == 0` 会丢一半输出：第 $m+1$ 行永远不写，症状同样是「静默残留旧值」。

坑五：M 小 K 大的形状会使 GPU 大量空闲。warp-per-row 的并行度上限是 M：$M = 8$ 时 grid 只有 2 个 block，110 个 SM 中 108 个空闲。正确做法是沿 K 切分（split-K：多个 block 各算一段部分和，再原子累加或二次归并），教学源码未实现，读者可参照第 三 章的两级归约自行补全，这也是面试高频追问点。

9.9　面试要点速查

Q1：为什么 GEMV 是 memory-bound？给出数量级。

A：$AI = 2MK/(4MK + \ldots) \approx 0.25 \sim 0.5$ FLOP/B（x 计数口径不同），与 M、K 无关；对现代 GPU（拐点数十 FLOP/B）低两个数量级，计算上限不足峰值 1%，优化只剩带宽一件事。

Q2：warp-per-row 与 block-per-row 怎么选？

A：warp-per-row 用一次 warp 内蝶形归约（5 步、零同步）收尾，行间零依赖、扩展性好；block-per-row 让一个 block 跨 warp 归约一行，K 很大时每 block 累加更深、归约要两级（第 三 章），换来的好处是 M 极小时仍能提供 block 级并行度。常规形状选前者，M 小 K 大时后者或 split-K 更合适。

Q3：$K = 16 < 32$ 时怎么设计？

A：让一个 warp 服务 `kRowPerWarp` 行，lane 按 segment 分组（$k = \text{lane} \bmod 16$，行号 $\lfloor \text{lane}/16 \rfloor$），归约用 width= 16 的分段蝶形（掩码 $8, 4, 2, 1$），写回判据是 `k == 0`（每 segment 一个写者）。

Q4：`__shfl_xor_sync` 第四个参数（width）是什么语义？

A：把 warp 切成宽度为 width 的 segment，lane 只与本 segment 内的 lane $\oplus$ mask 交换；掩码序列相应从 width/2 开始递减。k16 用它实现「一个 warp、两套独立归约」。同时注意掩码命名的线程必须都执行该指令（8.7 节勘误二的 UB 边界）。

Q5：GEMV 与 GEMM 优化的本质区别？

A：GEMM 有 N 维，A、B 的 tile 都能被多个输出复用，AI 随 tile 尺寸增长（第 十 章的 $B_M B_N/(2(B_M + B_N))$）；GEMV 的 $N = 1$ 使 A 元素零复用、AI 是常数，优化只剩带宽利用一条路。GEMM 靠分块复用抬高 AI，GEMV 只能优化带宽利用。

9.10 最小测试与动手实验

测试文件：`book/tests/ch08_sgemv.cu`（逻辑抽取自 bench/sgemv.cu 的 test_sgemv，L21，对照改为 CPU fp64 参考）。三个 case：A（M=64、K=512，K 同时被 32/128 整除，k32 与 k128 同形状对比）；B（M=7、K=128，行守卫路径，grid 2 个 block、第二个 block 有 warp 被剪）；C（M=40、K=16，k16 双行分段归约，$M\%8=0$ 避开勘误二的 UB 路径）。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --ch ch08`（编译 + 运行一体）。本机运行输出（RTX PRO 5000，2026-09-11；PASS 行的 `max_abs_err` 数值见表 9.2）：

```
case A: M=64 K=512 (K%128==0)
case B: M=7 K=128 (M guard)
case C: M=40 K=16 (2 rows/warp)
ALL OK
```

表 9.2: ch08 测试 PASS 行数值摘录（GPU 输出 vs CPU fp64 参考；tol 为判定容差）

case	判定项	最大绝对误差	tol
A	sgemv-k32	2.043e-06	1.0e-03
A	sgemv-k128	2.550e-06	1.0e-03
B	sgemv-k32	2.458e-07	1.0e-03
B	sgemv-k128	3.744e-06	1.0e-03
C	sgemv-k16	2.566e-07	1.0e-03

判据：三版对 CPU fp64 参考的最大绝对误差均在 10^{-6} 量级（F32Acc 档 10^{-3} 的千分之一），行守卫路径（case B）与分段归约路径（case C）覆盖到位。注意 case B 的 k16 不可照抄：把 M=7 用于 k16 会落入勘误二的 UB 路径。动手实验建议：（1）把 case A 的 K 改成 4096 重跑，观察误差随 $\sqrt{K}$ 缓涨但仍远离容差；（2）用 `compute-sanitizer --tool memcheck` 跑一遍 $K=40$（非对齐）的调用，直接查看坑一的越界报告；（3）运行 `./.build/ch08_sgemv bench` 复现表 9.1，并把 k16 的 M 从 2^{20} 降到 2^{16}，观察 L2 驻留如何使有效带宽虚高。

延伸阅读

- @ 有了琦琦的棍子《深入浅出 GPU 优化系列：gemv 优化》，https://zhuanlan.zhihu.com/p/494144694（GEMV 从 naive 到向量化、多行复用的完整演进实验，与本章三个 kernel 对照）；
- @ 紫气东来《CUDA（二）：GPU 的内存体系及其优化指南》，https://zhuanlan.zhihu.com/p/654027980（内存层级与带宽利用的背景，本章带宽优化思路的底层依据）；
- @ 白牛《CUDA 入门的正确姿势：how-to-optimize-gemm》，https://zhuanlan.zhihu.com/p/478846788（GEMM 优化阶梯总览，下一章的主参考，可作为「复用维度回归」的预告阅读）。

第十章　SGEMM 阶梯一：Block Tile、Vec4 与 Thread Tile

10.1　本章导读

GEMM（GEneral Matrix Multiply）是 GPU 上最重要的算子：大模型里除 norm/softmax 外几乎所有的线性层、attention 里的 $QK^\top$ 与 PV，最终都落到 GEMM 上。它决定了 GPU 硬件的诸多设计取向，也是 kernel 工程师面试的核心考点。本章是 Part II GEMM 篇的第一章，把教学源码 sgemm.cuh 头部的「优化五层金字塔」里 **CUDA Core 路径**的前三级走完：

- **Level 1 —Block Tile**（`sgemm`，sgemm.cuh L33）：$32 \times 32 \times 32$ 分块 + shared memory，一个线程算一个 C 元素，是最小但完整的 tiling 骨架，两级 `__syncthreads` 的正确性语义在此详细展开；
- **Level 2 + Level 3 —Thread Tile 与 Vectorize**（`sgemm_vec4`，sgemm.cuh L108）：$128 \times 128 \times 32$ 分块 + 每线程 4×4 寄存器 tile + `float4` 加载，三个优化叠加后的工程实现。

前置章节：第 二 章 roofline（本章的每一级提升都要用 AI 定量核证）、第 四 章合并访问与向量化、第 九 章 GEMV（作为「复用维度消失」的对照组）。**源码区间**：sgemm.cuh L6–183；无编译宏限定，全部架构适用；配套最小测试为 `book/tests/ch09_sgemm.cu`。**后继**：第 十一 章阶梯二（TF32 WMMA + cp.async 双缓冲）消除「等数据」的空转；第 十二 章起进入 Tensor Core；第 十三 章解决本章留下的 bank conflict 与 swizzle 问题。

10.2　问题与动机

动机一：naive GEMM 的算术强度只有 0.25。按定义直译，$C_{m,n} = \sum_k A_{m,k} B_{k,n}$：每个输出元素要读 A 的一行片段（K 个 float）和 B 的一列片段（K 个 float），做 K 次乘加。算术强度 $\mathrm{AI} = 2K\ \mathrm{FLOP} / (8K\ \mathrm{B}) = 0.25\ \mathrm{FLOP/B}$，为 **与 K 无关的常数**，与第 九 章 GEMV 同一量级。K 越大只是把流量按比例放大，AI 保持不变。在本机 roofline（拐点约 71.6 FLOP/B，见 9.6 节）下，naive 的计算上限被限制在带宽 $\times 0.25$，离 fp32 峰值差近两个数量级。

动机二：复用一直存在，只是 naive 没有利用。观察一：同一行块 $A[m{:}m{+}B_M, :]$ 上的所有输出共享同一段 A；同一列块 $B[:, n{:}n{+}B_N]$ 上的所有输出共享同一段 B。把这两段搬进 shared memory，一次搬运、B_N 次（对 A）与 B_M 次（对 B）复用，这就是 **Block Tile**，金字塔第一级。观察二：进一步让**一个线程**负责多个输出，则它从 smem 取出的每个 A 元素能服务 T_N 个输出、每个 B 元素能服务 T_M 个输出，即 **Thread Tile**，复用从 smem 级下沉到寄存器级。观察三：搬运本身（gmem→smem）可以用 `float4` 把指令数降到 1/4，即 **Vectorize**。三级叠加后，本章的 `sgemm_vec4` 在 2048^3 上比 naive 快 4.76×（9.6 节实测），而它使用的仍然只是 CUDA Core 与 shared memory。

动机三：金字塔的每一级解决一个不同的瓶颈。Block Tile 解决 **gmem 带宽**（AI 从 0.25 到 32）；Thread Tile 解决 **smem 带宽与指令数**（每 FMA 的 smem 读从 2 次降到 0.5 次）；再往上，剩下的瓶颈依次是访存延迟（→ 双缓冲，第 十一 章）、CUDA Core 吞吐本身（→ Tensor Core，第 十一–十二 章）、生产者消费者同步（→ Warp Specialization + TMA，第 十四–十五 章）。

10.3 数学原理

10.3.1 记号与总 FLOPs

沿用全书记号表：M, N, K 为行/列/归约维，B_M, B_N, B_K 为 block tile 尺寸（源码 `BM/BN/BK`），T_M, T_N 为 thread tile 尺寸。$C = A \times B$ 共 $2MNK$ FLOPs。row-major 下 $\text{addr}(A_{m,k}) = mK + k$、$\text{addr}(B_{k,n}) = kN + n$（以 float 为单位）。

10.3.2 naive：AI 与 K 无关

每输出 $C_{m,n}$：读 K 个 A 元素 + K 个 B 元素 $= 8K$ 字节，做 $2K$ FLOPs，

$$\text{AI}_{naive} = \frac{2K}{8K} = 0.25 \text{ FLOP/B}, \tag{10.1}$$

常数。直觉上的误区是「K 大了 AI 会变好」：分子分母同乘 K，恰好相消。

10.3.3 Block Tile：AI 与 K 切片深度无关，只随输出面增长

一个 block 负责 C 的 $B_M \times B_N$ 子块。沿 K 按 B_K 切片循环，每个 k-slice 从 gmem 搬入 A 的 $B_M \times B_K$ 与 B 的 $B_K \times B_N$ 两块 smem，然后完成 $B_M \times B_N \times B_K$ 的乘加。按 block 计数：

$$\text{AI}_{tile} = \frac{2\,B_M B_N B_K}{4B_K(B_M + B_N)} = \frac{B_M B_N}{2(B_M + B_N)}, \tag{10.2}$$

B_K 在分子分母间相消，**加深 B_K 不提升 AI**，只增加 smem 驻留量与单次同步覆盖的计算量；AI 由 $B_M \times B_N$（「输出面」）唯一决定。代入本章两个实现：$32 \times 32 \to \text{AI} = 8$ FLOP/B；$128 \times 128 \to \text{AI} = 32$ FLOP/B，恰好是三级递进（naive $0.25 \to 8 \to 32$）。方形 tile 时式 10.2 简化为 $B/4$：每边翻倍，AI 翻倍。

10.3.4 Thread Tile：换一层计数——smem 到寄存器

Thread tile **不改变** gmem 流量（那是 block tile 决定的），它优化的是 smem→ 寄存器这一跳。每线程算 $T_M \times T_N$ 个输出，则沿 k 每一步：从 smem 取 T_M 个 A 与 T_N 个 B（$4(T_M + T_N)$ 字节），做 $2T_M T_N$ FLOPs：

$$\text{AI}_{smem} = \frac{2\,T_M T_N}{4\,(T_M + T_N)} = \frac{T_M T_N}{2\,(T_M + T_N)}. \tag{10.3}$$

1×1（naive，每线程一个输出）时 $= 0.25$；4×4 时 $= 1.0$，每 FMA 的 smem 读次数从 2 降到 0.5，**4 倍**。这些值看着仍小，但 smem 的可用带宽比 DRAM 高一个数量级以上，瓶颈的上限完全不同；另外 $4 \times 4 = 16$ 个输出给出 16 条互相独立的 FMA 链，ILP 充分。

10.3.5 Vectorize：不改字节数，改指令数与事务

`float4` 把 4 次 32-bit 访问合并成 1 次 128-bit 访问：load/store 指令数 1/4，内存管线事务利用率更高（一个 warp 一条指令发起 512B 连续请求）。它对式 10.2 的分子分母都不动，**AI 不变，但带宽更容易达到上限**。对第 九 章的 GEMV 实测是 +5%；对 GEMM 的 gmem→smem 搬运同样成立。

10.3.6 roofline 定位与两级同步的正确性

以第 九 章实测的 934 GB/s 为本机可达带宽口径、fp32 峰值 66.9 TFLOPS（运行时查询折算），拐点 ≈ 71.6 FLOP/B。于是：naive（AI 0.25）计算上限 ≈ 0.23 TFLOPS；`sgemm`（AI 8）上限 ≈ 7.5 TFLOPS，仍在 memory-bound 区；`sgemm_vec4`（AI 32）上限 ≈ 29.9 TFLOPS，逼近拐点，进入「带宽与算力都吃紧」的过渡区。这正是引入阶梯二（双缓冲 + Tensor Core）的理由。

形式化之外，tiling 循环的正确性依赖两道栅栏，缺一不可：

- **第一道 `__syncthreads`**（消费前）：保证本 k-slice 的 `s_a`、`s_b` 已被**全部**线程写完，才能开始读，否则会读到其他线程尚未写完的数据；

- **第二道 `__syncthreads`**（覆写前）：保证所有线程已读完本 slice，才允许下一轮加载覆写同一块 smem，否则慢的线程读到的数据被快的线程提前覆写。

两道栅栏之间是「计算窗口」，之外是「搬运窗口」。本章两版 kernel 都是串行的「搬一算一」，空转时序见图 10.1；第 十一 章的双缓冲会让两个窗口重叠起来。

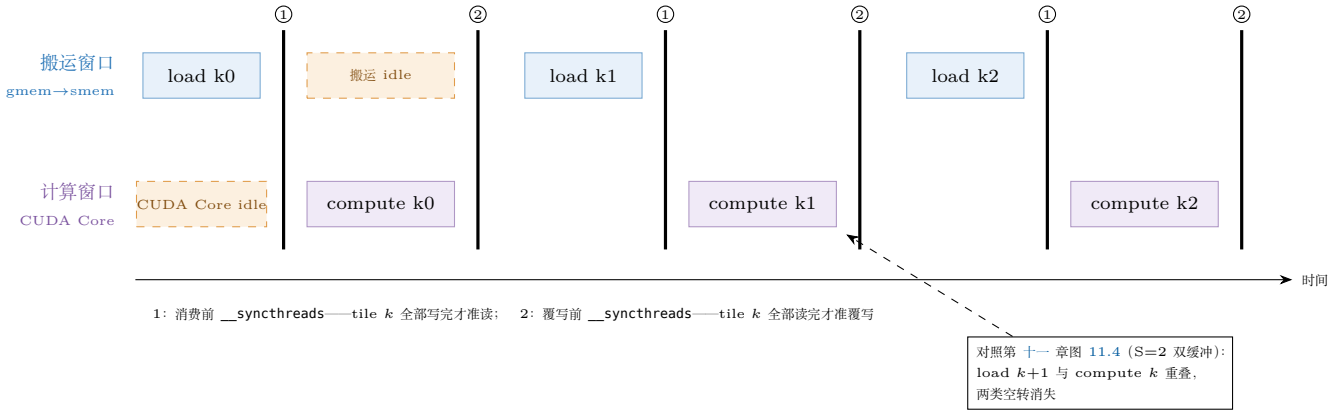


图 10.1: 单缓冲 k-slice 循环的双栅栏时序（3 个迭代）：load（蓝）→ 栅栏 1（消费前）→ compute（紫）→ 栅栏 2（覆写前）。load 期间 CUDA Core 空转、compute 期间搬运单元空转——「搬一算一」串行使两个窗口无法重叠；双缓冲（图 11.4）正是为消除这两类空转而设计。

10.4　设计与实现

10.4.1　五层金字塔总览

源码头部的金字塔注释是整个 Part II 的总纲：其优化路径是一张五层金字塔，各层的手段与计算密度递进见表 10.1。

表 10.1: GEMM 优化五层金字塔与计算密度递进

Level	手段	计算密度递进
1	Tiling（分块 + shared memory）：将数据从 HBM 搬到 SMEM 复用	AI $\approx B_K/(2\times\text{sizeof}) \approx 32/8 = 4$，仍是 memory-bound
2	Thread Tile（寄存器分块）：每个线程计算 TM × TN 个元素，提高计算密度	AI $\approx \text{TM}\times\text{TN}\times B_K/(2\times\text{sizeof}) \approx 8\times8\times8/8 = 64$，compute-bound
3	Vectorize（向量化访存）：`float4/half2`，减少 load/store 指令数	——
4	Tensor Core（MMA `m16n8k16`）：硬件矩阵乘单元，warp 级指令	提供硬件加速的 256 FMA/cycle/warp，提升吞吐
5	Warp Specialization + TMA（WGMMA `m64n128k16`）：Hopper 异步执行	——

五级中本章覆盖 Level 1/2/3（Tiling、Thread Tile 与 Vectorize）。表中 Level 1/2 两行的 AI 公式经核校存在量纲问题（AI 实际与 B_K 无关，正确值见式 10.2），定性结论不受影响，详见 9.7 节勘误一。

10.4.2　Level 1：`sgemm`，Block Tile $32 \times 32 \times 32$

sgemm.cuh L26-54（Level 1：设计注释、线程到 smem 的索引映射）

```
// ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
// 最基础的 tiling 实现，演示 shared memory 的核心用法
// C = A x B, C[M, N] = A[M, K] x B[K, N]
// BM=BN=32, BK=32, block(32, 32)，一个线程计算 c 的一个元素
```

```cpp
// Grid:  ((N + 31) / 32, (M + 31) / 32, 1)
// Block: (32, 32, 1), 1024 线程
// source: LeetCUDA/kernels/sgemm/sgemm.cu
__global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 32; // vec 版: 32x4 = 128
  constexpr int BN = 32; // vec 版: 32x4 = 128
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK], s_b[BK][BN]; //  32x32x4=4KB smem, float = 4 bytes

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx;

  // 线程到 smem 的映射: 32×32 线程，每个线程加载 a 和 b 各 1 个元素
  // 技巧: 一般来说 “/” 表示线程不是连续排布的，"%" 表示线程是连续排布的
  // 因此，在需要考虑连续访问的维度使用“%”，比如，连续的线程访问列方向连续的元素
  // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
  // 线程不连续访问 M 维度 → 用 /;
  int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
  int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
  int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
  int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
  int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
  int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
```

索引映射是这个 kernel 的全部「设计」。注释 L45–48 给出一条经验法则：「**/ 表示线程在该维不连续排布，% 表示连续排布**」：要把连续的 tid 映射到需要合并访问的维度（K 维，stride 1）就用 %，映射到跨步维度（M、N 维）就用 /。于是 tid → (tid/32, tid%32) 同时服务 A 的 (m,k) 与 B 的 (k,n)：一个 warp 的 32 个 lane 读 a 的一行连续 32 个 float（128B 合并事务），读 b 的一行连续 32 个 float，都是理想合并。两张网格全等、仅轴语义不同的「一图两读」见图 10.2。



图 10.2: Level 1 的 tid→(m,k)/(k,n) 映射双网格（格内数字为 tid 0–255，行 = tid/32、列 = tid%32，8 行即 8 个 warp，行间同色系深浅相间）。上：$s_a[m][k]$（蓝）；下：$s_b[k][n]$（紫）——两图数字排布完全相同，一条映射同时服务 A 与 B；warp 内 32 个列地址连续，构成 128B 合并事务。

sgemm.cuh L55-79（Level 1：k-slice 主循环与写回）

```

  float sum = 0.f; // 遍历完整的K，slice K;
  // 这里不用pragma unroll，因为K不是编译器常量，编译器无法展开循环
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
    __syncthreads(); // 确保整个 smem tile 加载完毕

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
      int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
      sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
    }
    __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
  }
  int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
  int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
  int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
  c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
}
```

主循环就是 9.3.6 节的两道栅栏夹一个计算窗口：加载 → 同步 → $B_K = 32$ 次「读一行 A、读一列 B、一次 FMA」→ 同步。计算阶段的 smem 访问天然无 bank conflict：warp 内所有 lane 的 tid/32 相同（读同一个 `s_a[m][k]`，广播），tid%32 连续（读 `s_b[k][n]`，stride 1）。写回同样合并。它慢在两处：AI 只有 8（带宽上限 7.5 TFLOPS），且每 FMA 要 2 次 smem 读（式 10.3 的 1×1 档）。

10.4.3　Level 1+：`sgemm_vec4` 的设计注释

`sgemm_vec4`（Block Tile 128×128 + K Tile 32 + Thread Tile 4×4）在 Level 1 基础上引入两层优化：其一，`float4` 向量化加载，A/B 各用 1 条 128-bit load 取代 4 条 32-bit load；其二，Thread Tile 4×4，每线程计算 16 个 C 元素，把计算/访存比从 $B_K/2 \approx 16$ 提升到 $\mathrm{TM} \cdot \mathrm{TN} \cdot B_K/2 \approx 256$，同时减少线程总数带来的同步开销。问题设定 $C = A \times B$，即 $C[M,N] = A[M,K] \times B[K,N]$（$A/B$ 均 row-major）；$B_M = B_N = 128$、$B_K = 32$，block $(32, 32)$ 共 1024 线程，每线程负责 $4 \times 4 = 16$ 个 C 元素，$1024 \times 16 = 16384 = 128 \times 128$，恰好铺满；Grid $((N+127)/128, (M+127)/128, 1)$。边界假设与 Level 1 naive 版一致：M/N 为 128 的倍数、K 为 32 的倍数。线程到 4×4 tile 的映射与加载映射**解耦**（独立计算更清晰）；注释还提示了计算阶段 `s_b` 的 bank conflict（4-way，生产代码可用 `s_b[BK][BN+1]` PAD 打散，这里保持最简布局便于讲解）。映射与提示下文逐一验算。

先独立验证注释里的两套映射（都能对上）：**加载映射**，A 块 128×32：每线程 4 个元素，8 线程铺满一行 32 列（tid/8 定行、(tid%8)·4 定列组），1024 线程恰好 128×32；B 块 32×128：32 线程铺满一行（tid/32 定行、(tid%32)·4 定列组），1024 线程恰好 32×128；row-major 下两条映射的 4 元素都连续，`float4` 合法。**计算映射**，与加载映射**完全解耦**（源码 L130–133 特意强调这一点）：线程 tid 负责输出的 4×4 子块，行基址 (tid/32)·4、列基址 (tid%32)·4；32×32 个线程 × 每人 16 个输出 $= 16384 = 128 \times 128$，不重不漏（vec4 落点见图 10.3）。



图 10.3: Vec4 加载映射双网格：上 s_a，行 m = tid/8、列组 k = (tid%8) · 4，8 线程铺满一行；下 s_b，行 k = tid/32、列组 n = (tid%32) · 4，32 线程铺满一行。每个 4 小格组 = 一个 tid 的一条 `float4`（row-major 下 4 元素连续，加载合法）；红框为 tid 11 在两 tile 中的落点。

注释 L99–102 的 bank conflict 提示也经推导属实：计算阶段读 `s_b[k][n]` 时，warp 内列地址步进 4 个 float（16B），32 个 lane 只命中 8 个 bank，4-way conflict；生产代码用 `s_b[BK][BN+1]` 加一列 PAD 打散，本章保持最简布局，把系统解法（swizzle）留给第 十三 章（bank 命中模式见图 10.4）。

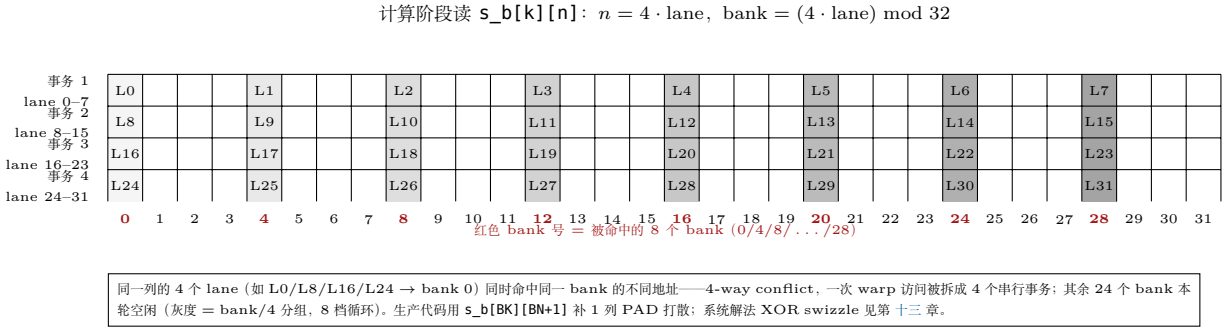


图 10.4: s_b 计算访问的 4-way bank conflict：warp 内 32 lane 按 n = 4·lane 步进 4 个 float（16B），bank = (4·lane) mod 32 只命中 0/4/8/. . . /28 共 8 个 bank——每列 4 个 lane（Lj/Lj+8/Lj+16/Lj+24）同 bank 不同地址，一次访问拆成 4 个事务。

10.4.4 Level 1+：`sgemm_vec4` 的实现

sgemm.cuh L108-137（Level 1+：常量、加载映射与 4x4 thread tile 基址）

```
__global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 128;
  constexpr int BN = 128;
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
  __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
```

```
  int tid = threadIdx.y * blockDim.x + tx; // 0~1023

  // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
  int load_smem_a_m = tid / 8;        // 0~127, 8 线程/行
  int load_smem_a_k = (tid % 8) * 4;  // 0,4,...,28
  // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
  int load_smem_b_k = tid / 32;       // 0~31, 32 线程/行
  int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124

  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  // 4×4 Thread Tile 基址（独立于加载映射），这里compute索引的计算逻辑要和
  // load索引的计算逻辑分开，load/compute是可以独立索引的，理解这点很重要。
  // 目标C Tile为[BM,BN]=[128x128]，有32x32线程，则每个线程处理4x4 tile
  // 那么，就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
  int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
  int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124

  float sum[4][4] = {0.f};
```

sgemm.cuh L138-167（Level 1+：float4 搬运、双栅栏与 4x4 FMA 内核）

```
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
    __syncthreads();

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      // 每次迭代加载 4 个 A 元素 + 4 个 B 元素，再做 4×4=16 次 FMA
      float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
                         s_a[comp_smem_a_m_base + 1][k],
                         s_a[comp_smem_a_m_base + 2][k],
                         s_a[comp_smem_a_m_base + 3][k]};
      float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
                         s_b[k][comp_smem_b_n_base + 1],
                         s_b[k][comp_smem_b_n_base + 2],
                         s_b[k][comp_smem_b_n_base + 3]};
#pragma unroll
      for (int i = 0; i < 4; ++i) {
#pragma unroll
        for (int j = 0; j < 4; ++j) {
          sum[i][j] += a_vals[i] * b_vals[j];
        }
      }
    }
    __syncthreads();
  }
```

计算内核逐行对照式 10.3：每个 k 取 4 个 A、4 个 B 进寄存器（`a_vals`/`b_vals`），做 $4 \times 4 = 16$ 次 FMA，`sum[4][4]` 是 16 条独立累加链，编译器可以充分乱序；16 次 FMA 只花 8 次 smem 读，这就是 Thread Tile 的全部含义。

sgemm.cuh L168-182（Level 1+：4x4 写回，每行一条 float4 store）

```

```

```
  // 存储 4×4：每行 4 个元素连续 → 可用 float4 store（要求 N 为 4 的倍数以保证对齐）
  int store_gmem_c_m = by * BM + comp_smem_a_m_base;
  int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
#pragma unroll
  for (int i = 0; i < 4; ++i) {
    int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
    float4 reg_c;
    reg_c.x = sum[i][0];
    reg_c.y = sum[i][1];
    reg_c.z = sum[i][2];
    reg_c.w = sum[i][3];
    FLOAT4(c[store_gmem_c_addr]) = reg_c;
  }
}
```

写回把每行 4 个连续输出拼成一条 `float4` store（要求 $N\%4=0$）。最后补一个源码没有显式写出来的观察：**隐式 warp tile**。warp 由 32 个连续 tid 组成，行基址 $(\text{tid}/32)\cdot 4$ 在 warp 内相同、列基址 $(\text{tid}\%32)\cdot 4$ 以 4 递增，于是**一个 warp 实际负责一条 4×128 的 C 横带**。金字塔的 warp 级在本章无需额外代码；第 十一 章的 WMMA 版会把它显式化为 4×2 的 warp 排布。

10.5 图示

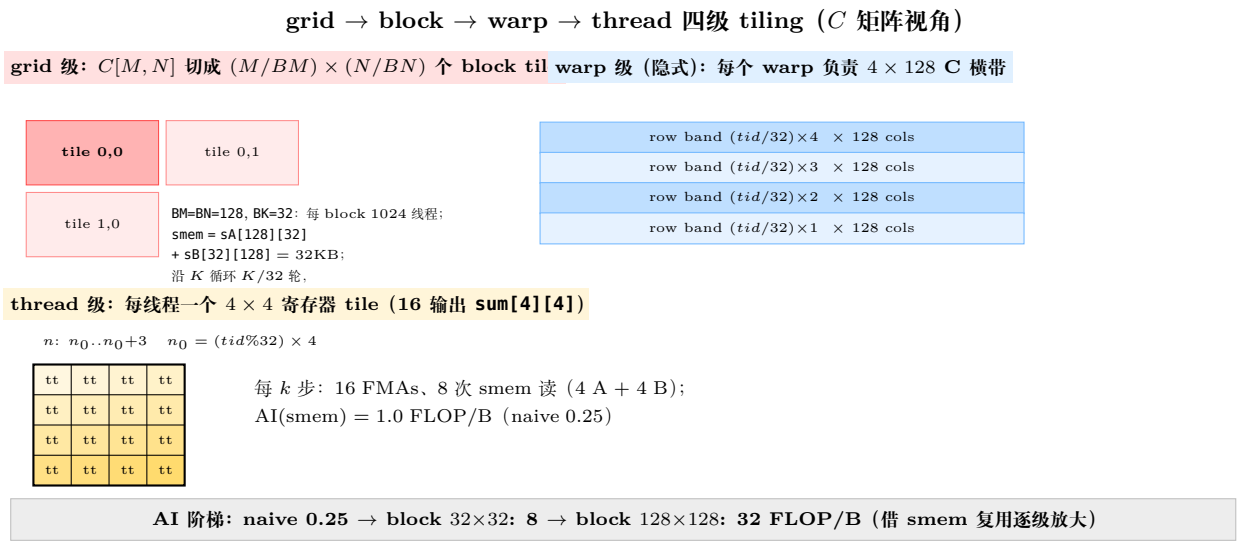


图 10.5: 四级 tiling 递进：grid 级切 C 为 block tile；block 级用 smem 承载 A/B 切片；warp 级是隐式的 4×128 横带；thread 级是 4×4 寄存器 tile。AI 阶梯：naive 0.25 → block 32×32: 8 → block 128×128: 32 FLOP/B。

图 10.5 把 9.3 节的三级公式落到同一张图上。读图顺序建议从下往上：thread tile 决定「每个线程负责多少输出」，warp 横带是它的自然分组，block tile 决定「smem 里驻留多少输入」，grid 决定「C 被切成几份」。第 十一 章起 warp 级会从隐式变为显式编排（4×2 warp 排布），图的骨架不变。

10.6 性能分析

本章复测（RTX PRO 5000 72GB Blackwell，sm_120，110 SMs，threads/SM = 1536，smem/SM = 100KB；时钟按运行时查询 2.38 GHz，fp32 峰值按 128 FMA/SM/cycle 折算 ≈ 66.9 TFLOPS；warmup 3 + 10 次均值）：

五个观察：（1）2048^3 **上 vec4 快** 4.76×（3.144 → 0.661 ms），且形状越大差距越稳定，小形状下两版都未达到各自的瓶颈。（2）**naive 被带宽限制在** ≈ 5.5 **TFLOPS**：AI = 8 对应的带宽隐含上限 $8\times 934\approx 7.5$ TFLOPS，实测拿到其中 73%（差值来自同步空转与访存延迟未被隐藏）；模型带宽 683 GB/s 也印证带宽是上限。（3）**vec4 到 25.97 TFLOPS**：AI = 32 的上限是 29.9，实测 87%，在「带宽上限」与「算力上限 66.9」之间，它已经把不带双缓冲、不用 Tensor Core 的 CUDA Core 写法发挥到了该路径的较高水平。距离峰值的 61% 缺口，正是后两章要解决的问题：访存

表 10.2: ch09 两版 SGEMM 实测（方阵 $M{=}N{=}K$；blocks 为 grid 尺寸；模型带宽按式 10.2 的流量模型折算）

实现	MNK	blocks	ms/iter	TFLOPS	%peak	模型带宽 (GB/s)
naive	512	256	0.065	4.13	6.2	516
vec4	512	16	0.057	4.68	7.0	147
naive	1024	1024	0.418	5.14	7.7	642
vec4	1024	64	0.114	18.92	28.3	589
naive	2048	4096	3.144	5.46	8.2	683
vec4	2048	256	0.661	25.97	38.8	813

延迟（cp.async 双缓冲）与算力（TF32/FP16 Tensor Core）。(4) **512 规模的 vec4 受并行度不足限制**：128^2 的 block tile 在 512^2 上只切出 $4\times4=16$ 个 block，110 个 SM 有 94 个空闲；naive 同形状有 256 个 block 反而铺得更满。tile 不是越大越好：「block 数 $\geq$ SM 数的若干倍」是形状适配的底线。(5) **占用率视角**：两版都是 1024 线程的 block，而本机 threads/SM = 1536，单个 SM 同时只能驻留 1 个 block，vec4 的 32KB smem 与 1024 线程都不再是第一瓶颈，瓶颈回到「每 SM 同时在飞（in-flight）的加载与 FMA 数」。

10.7　实践经验

实践经验 10.1 (F3)：sgemm.cuh L18-19：金字塔注释的两个 AI 公式量纲不一致 注释原文给出「Level 1: `AI ≈ B_K/(2×sizeof) ≈ 32/8=4`」与「Level 2: `AI ≈ TM×TN×B_K/(2×sizeof) ≈ 8×8×8/8=64`」。问题一：正确的 block tile 推导（式 10.2）中 B_K 在分子分母间 **相消**，AI 与 B_K 无关，把 B_K 放进分子会得出「加深 K 切片就能提升 AI」的错误直觉；数值上 32×32 tile 的 AI 是 8 而非 4。问题二：Level 2 公式代入 $T_M{=}T_N{=}B_K{=}8$ 得 64，与任何计数口径都对不上：thread tile 不改变 gmem AI（仍由 block tile 决定），它提升的是 smem 级计算密度（式 10.3，1×1 的 0.25 到 4×4 的 1.0，即 4 倍）。两个公式的**定性结论**（Level 1 仍 memory-bound、Level 2 起向 compute-bound 迈进）在本机 roofline 下均成立（9.6 节），但推导与数值应按本章 9.3 节修正。

实践经验 10.2 (F3)：sgemm.cuh L84-85：「AI 从 BK/2 提升到 TM*TN*BK/2」同族问题 `sgemm_vec4` 头注释写「AI 从 `BK/2` ≈16 提升到 `TM*TN*BK/2` ≈ 256」，与勘误一同源的量纲问题（分子含 B_K、分母是「2 个 float 的字节数」）。按本章口径改写应为：gmem AI 从 8（32^2 tile）升到 32（128^2 tile，来自 block tile 放大）；smem 级计算密度从 0.25 升到 1.0（来自 4×4 thread tile）；`float4` 不改 AI、只降指令数。三个数字各有归属，不宜混在一个公式里。

实践经验 10.3 (F5 排版)：sgemm.cuh L11-15：金字塔注释换行错位 L11–15 的物理换行把 Level 2 的描述与 Level 3/4/5 的内容挤在同一行（如「提高计算密度 Level 3 —Vectorize」），阅读时需按 Level 关键字自行断句。源码冻结不改，仅提示读者。

实践经验 10.4 (核校通过)：sgemm.cuh 三处关键断言 其一，L99–102 的 4-way bank conflict：计算阶段 `s_b[k][n]` 的 warp 内地址步进 4 float，32 lane 命中 $\{j, j{+}4, \dots\}$ 共 8 个 bank，是 4-way，推导成立；其二，L87–88 的覆盖算术 $1024\times16=16384=128\times128$ 成立（与 9.4.3 节映射推导一致）；其三，L106 的边界约定（M、N 为 128 倍数、K 为 32 倍数）与两个 kernel 的无守卫实现一致，naive 版相应为 32 倍数。

10.8　常见坑

坑一：省掉第二道 `__syncthreads`。最常见的「优化」笔误：以为计算完就能直接加载下一片。后果是 race：快线程覆写 smem、慢线程读到新数据，**症状随形状与调度变化，难以稳定复现**。本章测试的动手实验（1）演示复现方法。

坑二：边界约定由调用方保证。两个 kernel 对 M、N、K **均无守卫**：naive 要求三者都是 32 的倍数，vec4 要求 M、N 是 128 的倍数、K 是 32 的倍数。不满足时越界读**且越界写**，比第 九 章 GEMV 的只读越界更危险。生产代码要么切尾块专用路径，要么在 tile 边界加条件读（写 0）。

坑三：`/` 与 `%` 用反。把合并维度（K/N，stride 1）交给 `/`，warp 内地址立刻变成跨步 32/128 的散访，合并访问与 bank conflict 双重退化，性能会下降一个数量级。记住 9.4.2 节的法则：连续的维度用 `%`。

坑四：`float4` 的对齐。与第 九 章坑二同源：`vec4` 的加载/写回要求 16B 对齐。本章形状约束（M、N 为 128 倍数、K 为 32 倍数）自动保证；一旦放松（比如 K 只保证 4 的倍数），B 矩阵行起点 $b+kN$（N 为 4 倍数即可）仍对齐，但 `s_a` 行内偏移 $(\text{tid}\%8)\cdot4$ 恒对齐，逐项检查三处（基址、行起点、列偏移）再动手。

坑五：tile 并非越大越好。9.6 节观察（4）：512^2 上 128^2 tile 只有 16 个 block，SM 大量闲置；同时 1024 线程的 block 在 1536 threads/SM 的机器上每 SM 只驻留 1 个。tile 尺寸是「AI（越大越好）」与「并行度/占用（越多越小越好）」的折中，形状适配（split-K、尾块处理）是工程上必须处理的问题。

10.9 面试要点速查

Q1：为什么 tiling 能提升 GEMM 性能？给出 AI 公式。

A：block tile 让每份 gmem 数据被 B_N（或 B_M）个输出复用，AI $= B_M B_N/(2(B_M + B_N))$，与 B_K 无关；$32^2 \to 8$、$128^2 \to 32$ FLOP/B。naive 的 0.25 是常数，与 K 无关。

Q2：两道 `__syncthreads` 各防什么？

A：第一道防「读到没写完的 smem」（生产者/消费者），第二道防「读还没读完就被下一轮覆写」（消费者/生产者）。双缓冲（第 十一 章）用两份 smem 交替，可以把第二道栅栏变成流水线的一个同步点。

Q3：block tile 与 thread tile 分别优化哪一层的复用？

A：block tile 优化 gmem→smem（AI 式 10.2）；thread tile 优化 smem→ 寄存器（式 10.3，每 FMA 的 smem 读从 2 降到 0.5），并同时给出 16 条独立 FMA 链的 ILP。二者正交、可叠加。

Q4：vec4 的加载为什么是合并访问？

A：A 侧 8 个 lane 的 16B 请求拼出一行 32 float（128B）；B 侧 32 个 lane 的 16B 请求拼出 512B 连续段。warp 一条指令 512B，事务利用率最高档。

Q5：既然 AI 与 B_K 无关，为什么不把 B_K 取得尽量大？

A：加大 B_K 只增加 smem 驻留（占用率下降）与单次同步覆盖的计算量，AI 不变。B_K 的选择受 smem 容量与 k 循环指令开销摊薄两方面约束，32 或 64 是常见值；真正提升 AI 的是 $B_M \times B_N$。

Q6：本章的 warp tile 在哪里？

A：隐式的：32 个连续 tid 的 4×4 tile 恰好拼成 4×128 的 C 横带（行基址 warp 内相同）。显式 warp tile 排布（如 4×2）在第 十一 章 WMMA 版出现。

10.10 最小测试与动手实验

测试文件：`book/tests/ch09_sgemm.cu`（逻辑抽取自 bench/sgemm.cu 的 test_sgemm，L21，对照由 cuBLAS 改为 CPU fp64 参考）。两个 case：A（128^3：naive 16 blocks vs vec4 1 block）；B（256^3：64 vs 4 blocks）。两版 kernel 输出与参考逐元素比对，容差取 F32Acc 档（10^{-3}）。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --ch ch09`。本机运行输出（RTX PRO 5000，2026-09-11；PASS 行的 `max_abs_err` 数值见表 10.3）：

```
case A: M=N=K=128 (16 vs 1 blocks)
case B: M=N=K=256 (64 vs 4 blocks)
ALL OK
```

表 10.3: ch09 测试 PASS 行数值摘录（GPU 输出 vs CPU fp64 参考；tol 为判定容差）

case	判定项	最大绝对误差	tol
A	sgemm (32x32x32)	7.322e-06	1.0e-03
A	sgemm-vec4 (128x128x32)	7.322e-06	1.0e-03
B	sgemm (32x32x32)	1.362e-05	1.0e-03
B	sgemm-vec4 (128x128x32)	1.362e-05	1.0e-03

判据：两 kernel 对 CPU fp64 参考的误差在 10^{-5} 量级（$K \le 256$ 的串行 fp32 累加噪声），远小于 F32Acc 档容差；两 kernel 输出一致（同形状同累加顺序时误差完全相同，亦可作交叉印证）。注意测试形状必须满足倍数约束（坑二），否则越界写会直接破坏堆。动手实验建议：（1）注释掉 `sgemm` 的第二道 `__syncthreads`（在自己的副本上），用 $K = 512$ 反复跑 case A 100 次，统计错误率，观察坑一错误的间歇出现；（2）把 M 改成 33（不满足倍数约束）跑

`compute-sanitizer`，观察越界报告；(3) 运行 `./.build/ch09_sgemm bench` 复现表 10.2，并解释 512^3 时 vec4 相对 naive 只有 1.13× 的原因（9.6 节观察四）。

延伸阅读

- @ 白牛《CUDA 入门的正确姿势：how-to-optimize-gemm》，https://zhuanlan.zhihu.com/p/478846788（寄存器/共享内存分块的逐级推导与实验，本章 9.3 节 AI 阶梯的独立佐证）；
- @ 紫气东来《CUDA（三）：通用矩阵乘法：从入门到熟练》，https://zhuanlan.zhihu.com/p/657632577（naive → tiling → 向量化 → Tensor Core 的完整实现链，与本章两个 kernel 及第 十一 章衔接）；
- @ 紫气东来《CUDA（二）：GPU 的内存体系及其优化指南》，https://zhuanlan.zhihu.com/p/654027980（smem/L2/DRAM 层级与带宽背景，理解 9.6 节 roofline 定位的底层依据）。

第十一章　SGEMM 阶梯二：TF32 WMMA 与 cp.async 双缓冲

11.1　本章导读

第 十 章把 SGEMM 从 naive 的 Block Tile 一路优化到 Vec4 + Thread Tile，但无论怎么划分，乘加始终跑在 **CUDA Core** 的 FFMA 流水线上，这是 CUDA Core 路径的性能上限。本章完成两处架构切换：第一处把乘加搬进 **Tensor Core**，用 **TF32** 数据格式与 **WMMA** 编程模型重写 SGEMM；第二处用 **cp.async** 异步拷贝 + **双缓冲**（double buffering）把 global memory 到 shared memory 的搬运与计算重叠起来。两件事合起来，就是教学源码里的 `sgemm_tf32`（sgemm.cuh L229）。

本章源码区间为 sgemm.cuh L184–434（Phase 7a+），包含两个 kernel：`f32x4_tf32x4_kernel`（L205，FP32→TF32 原地转换）与 `sgemm_tf32`（L229，WMMA $m16n16k8$ + dynamic shared memory + 2 级流水）。无编译宏限定，WMMA TF32 路径要求 `sm_80` 及以上（Ampere 起）；配套最小测试为 `book/tests/ch10_sgemm_tf32.cu`。

前置章节：第 十 章的 tiling 金字塔（Block Tile / Thread Tile / 向量化加载），本章的 block/warp 划分直接建立在它之上；第 四 章的 coalescing 与 float4 向量化（本章加载路径全部是 16 B 的 `cp.async`）。**后继**：第 十二 章将抛弃 WMMA 的「布局黑盒」，直接手写 `mma.sync` PTX 指令并引入 `ldmatrix` 与更深的多级流水；第 十三 章在其上加 XOR swizzle。

11.2　问题与动机

动机一：CUDA Core 的 FFMA 吞吐上限。一条 FFMA 指令每个线程每周期至多完成 1 对乘加（2 FLOP）；一个 128 线程的 block 每 cycle 的理论峰值就是 256 FLOP 的量级。而 Tensor Core 是专为矩阵乘设计的矩阵乘累加单元：以 Ampere 为例，一条 warp 级的 TF32 MMA 指令（$m16n8k8$）一次完成 $16\times8\times8=1024$ 个乘加，是同宽度 FFMA 的 32 倍密度。想继续提升 GEMM 吞吐，就必须把循环核从「每线程一个 FMA」换成「每 warp 一条矩阵指令」。

动机二：舍弃 13 个尾数位换 8 倍吞吐。Tensor Core 不直接接受 fp32 输入。Ampere 为 fp32 输入准备的是 **TF32**（TensorFloat-32）：去掉 fp32 尾数的低 13 位、保留 8 位指数，得到 1+8+10 共 19 位、精度与 fp16 相同的格式。指数域不变意味着动态范围与 fp32 完全一致（约 $10^{\pm38}$），深度学习里绝大多数 fp32 GEMM 都能直接迁移。这是典型的「精度换吞吐」取舍，10.3 节会给出定量分析。

动机三：同步搬运制造的流水线空泡。第 十 章的加载路径是「`ld.global` 到寄存器，再 `st.shared`」，每搬一个 tile 都要全 block 同步等待：加载阶段 Tensor Core 空闲，计算阶段加载单元空闲。Ampere 起提供的 `cp.async` 让 global→shared 的搬运绕过寄存器、由异步单元完成，程序员可以「先发出下一块的拷贝，再计算当前块」，用双缓冲把搬运延迟掩盖在计算时间内。本章 kernel 的 `kStages`=2 正是两级缓冲。

11.3　数学原理

11.3.1　TF32：去掉 13 个尾数位的 fp32

一个 IEEE 754 fp32 数是 1 位符号 s、8 位指数 e、23 位尾数 m。TF32 保持 s 与 e 的位宽与语义不变，只把显式尾数位从 23 压到 10，共 19 位，存储时仍占用一个 32 位字（低 13 位填 0 或在硬件路径中直接忽略）。四种格式的对照：源码注释（sgemm.cuh L188）写「TF32 精度：mantissa 10bit，~3 位十进制有效数字」，对照表 11.1：$p=11$ 位二进制有效位对应 $11\log_{10}2\approx3.31$ 位十进制，注释核校通过。注意 TF32 与 fp16 的**尾数**完全相同（10 位），差别全在指

表 11.1: fp32 / TF32 / bf16 / fp16 位域对照

格式	符号	指数	尾数 (显式)	有效位 p	十进制有效数字
fp32	1	8	23	24	≈ 7.2
TF32	1	8	10	11	≈ 3.3
bf16	1	8	7	8	≈ 2.4
fp16	1	5	10	11	≈ 3.3

数：TF32 用 fp32 的 8 位指数换来与 fp32 相同的动态范围，代价是它通常只在 Tensor Core 数据路径里出现。四种格式按位宽比例的对照见图 11.1。

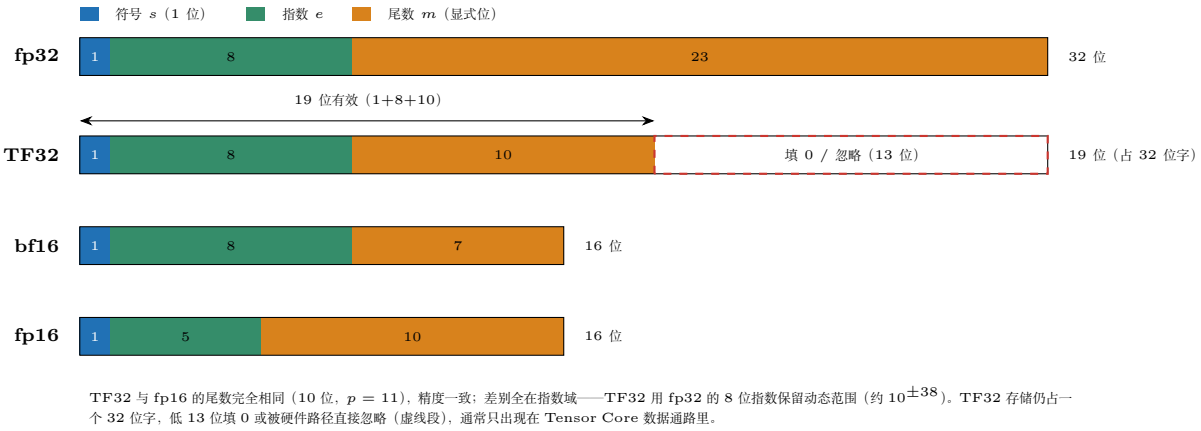


图 11.1: 四种浮点格式的位域条（与表 11.1 同序，按位宽比例绘制）：蓝 = 符号位、绿 = 指数、橙 = 显式尾数。TF32 去掉 fp32 尾数低 13 位（红色虚线框：填 0 / 忽略），保留 1+8+10 共 19 位；bf16/fp16 只有 16 位，指数域分别为 8 与 5。

11.3.2　舍入误差界：TF32 内积到底差多少

设 x 为 fp32 数，$fl_{\text{tf32}}(x)$ 表示把 x 舍入到最近的 TF32 （round-to-nearest）。TF32 有 $p = 11$ 位有效精度（1 隐含位 + 10 显式位），单位舍入误差（unit roundoff）

$$u \;=\; 2^{-p} \;=\; 2^{-11} \;\approx\; 4.88 \times 10^{-4}, \tag{11.1}$$

即对任意有限 fp32 数 x（不在舍入溢出的极端区间），

$$fl_{\text{tf32}}(x) = x\,(1+\delta), \qquad |\delta| \le u. \tag{11.2}$$

TF32 的乘法在 Tensor Core 内部以 fp32 精度执行（PTX 对 `.tf32` MMA 的规定是「元素乘按指明精度执行，累加至少单精度」，见第 十二 章 11.3.1 节的原文引述），因此误差只在**输入舍入**这一步发生。对内积 $s = \sum_{j=1}^{K} a_j b_j$，设 $\hat{a}_j = fl(a_j)$、$\hat{b}_j = fl(b_j)$，逐项展开：

$$\begin{aligned} \hat{a}_j\hat{b}_j &= a_j(1+\delta_j)\,b_j(\epsilon_j) = a_j b_j\,(1+\delta_j)(1+\epsilon_j) \\ &\approx a_j b_j\,\big(1+\delta_j+\epsilon_j\big), \qquad |\delta_j|, |\epsilon_j| \le u, \end{aligned} \tag{11.3}$$

其中二阶小量 $\delta_j\epsilon_j$ 已略去。把 K 项加总，得到 TF32 输入舍入带来的误差上界

$$\Big|\,\text{TF32-GEMM}(s) - s\,\Big| \;\le\; 2u \sum_{j=1}^{K} |a_j|\,|b_j| \;+\; O(Ku^2) \;+\; (\text{fp32 累加误差}). \tag{11.4}$$

两个直接推论：（1）误差按 $\sum |a_j b_j|$ 增长：对固定的元素幅值，它随 K **线性**增长；（2）**相对误差**约为 $2u \cdot \frac{\sum |a_j b_j|}{|\sum a_j b_j|}$，随机独立元素下分母量级 $\sqrt{K}\,\sigma$（σ 为单项标准差）而分子量级 $K\mu$（μ 为 $|a_j b_j|$ 均值），比值 $\propto \sqrt{K}$：内积越长，TF32 的相对误差越差，但 $\sqrt{K}$ 的增速远慢于 K。以 $[-1, 1]$ 均匀随机输入、$K = 512$ 为例：$\sum |a_j b_j| \approx K/4 = 128$，式 11.4 的界 $\approx 2 \times 4.88 \times 10^{-4} \times 128 \approx 0.125$，而输出自身量级 $\approx \sqrt{K/9} \approx 7.5$，相对误差界约 1.7%，与「~3 位十进制有效数字」的直觉一致。深度学习权重与激活的数值噪声通常远大于此，TF32 在生产环境被广泛接受，靠的就是这一点。

11.3.3 双缓冲的时序数学

设搬运一个 k-tile（global→shared）需要时间 T_{ld}，计算一个 k-tile 需要 T_{cp}。无流水（单缓冲）时每步总代价是两者**串行相加** $T_{\mathrm{ld}}+T_{\mathrm{cp}}$。双缓冲提供两份 smem 缓冲 S_0, S_1，稳态下第 k 步在「用 $S_{k \bmod 2}$ 计算 tile k」的同时「向 $S_{(k+1) \bmod 2}$ 发出 tile $k+1$ 的搬运」，两件事并行，稳态周期收敛到

$$T_{\mathrm{step}} \;=\; \max\!\big(T_{\mathrm{ld}},\, T_{\mathrm{cp}}\big), \tag{11.5}$$

即完全掩盖较短的一侧。图 11.4 画出了这个时序。推广到 S 级（$S=$kStages）流水，允许 in-flight 的搬运组数为 $S-2$（本章 S=2 时为 0 组、靠迭代末 `wait_group` 收口；第 十二 章 S=3 时为 1 组）。更深的流水能容忍更长的搬运抖动，但 smem 消耗按 S 线性增长，第 十二 章讨论 kStages 权衡时会回到这个事实。

11.4 设计与实现

11.4.1 配置金字塔：从 block 到 fragment

kernel 的 tile 配置由模板参数逐层相乘得到（sgemm.cuh L225–236）：

- **block tile**：$B_M \times B_N \times B_K = 128\times128\times8$，其中 $B_M =$ kWMma_M×kWMmaTile_M×kValTile_M $= 16\times4\times2$，$B_N = 16\times2\times4$，$B_K =$ kWMma_K $= 8$；
- **warp tile**：256 线程 = 8 warp，按 4×2 排布（`warp_m`=0–3、`warp_n`=0–1），每 warp 负责 32×64 的 C 子块；
- **fragment tile**：每 warp 内部再摆 $2\times4=8$ 个 $m16n16k8$ 的 WMMA fragment（源码命名 `kValTileM/N`），32×64 恰好被 16×16 的 fragment 阵列铺满。

参数命名即语义：`kWMmaTileM/N`（4×2）是 warp **间**的排布数，`kValTileM/N`（2×4）是 warp **内** fragment 的值重复数，与 `warp_m = warp_id / 2`、`warp_n = warp_id % 2`（L247–249）给出的真实几何一一对应，读代码时可按名字直接还原排布（历史注记见 10.7 节）。三层分解全景见图 11.2。

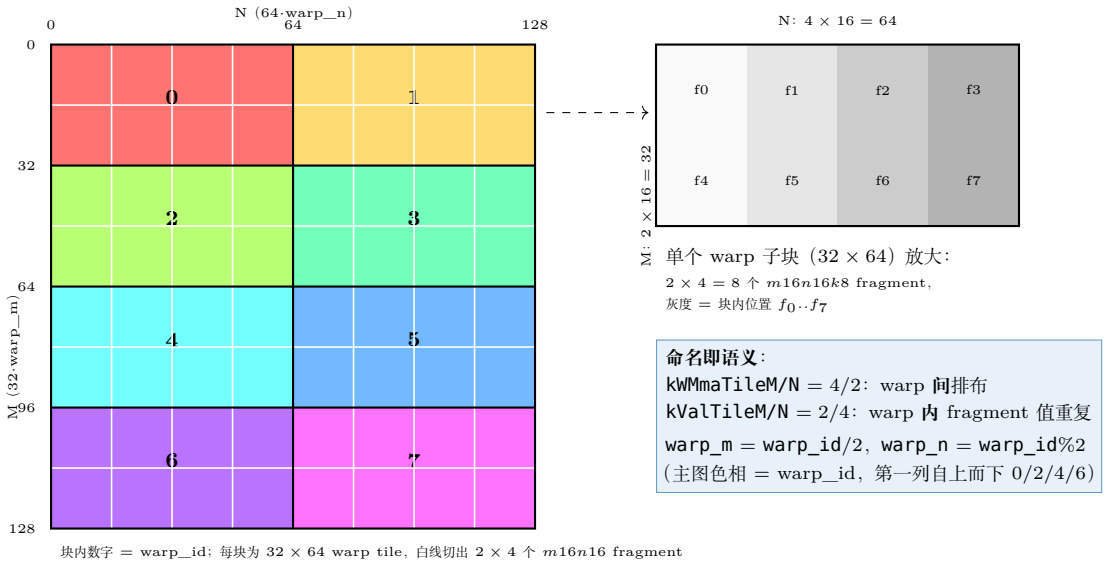


图 11.2: 128×128 block tile 的三层分解：外层按 4×2 切 8 个 32×64 warp 子块（色相 = warp_id，$B_M = 16\times4\times2$、$B_N = 16\times2\times4$）；每子块内再摆 $2\times4=8$ 个 $m16n16k8$ fragment（右：放大图，灰度 = 块内位置）。侧栏：`kWMmaTileM/N` 表 warp 间排布、`kValTileM/N` 表 warp 内 fragment 值重复，名字即语义。

11.4.2 FP32 到 TF32 的原地转换

sgemm.cuh L205-216（FP32→TF32 原地转换，float4 向量化）

```
__global__ void f32x4_tf32x4_kernel(float *x, float *y, int N) {
```

```
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  if (idx < N) {
    float4 reg_x = FLOAT4(x[idx]);
    float4 reg_y;
    reg_y.x = wmma::__float_to_tf32(reg_x.x);
    reg_y.y = wmma::__float_to_tf32(reg_x.y);
    reg_y.z = wmma::__float_to_tf32(reg_x.z);
    reg_y.w = wmma::__float_to_tf32(reg_x.w);
    FLOAT4(y[idx]) = reg_y;
  }
}
```

Tensor Core 不会自动完成 fp32→TF32 的舍入。送进 `load_matrix_sync` 的数据必须已经是 TF32 精度：WMMA 的 `precision::tf32` fragment 不保证隐式转换的位置，其行为取决于编译器生成的指令，与 cuBLAS 等「先舍入输入」的参考实现不可比。这个 kernel 用 `__float_to_tf32` 把输入矩阵原地转换：每线程处理连续 4 个 float（16 B 一条指令搬运），转换在寄存器里完成后写回原地址。本章测试 case A 验证了该转换的舍入模式：4096 个随机 fp32 上，GPU 结果与 host 端 round-to-nearest 实现逐位一致（见 10.10 节）。

11.4.3　WMMA 编程模型：fragment 三件套

WMMA 是 CUDA 对 warp 级矩阵乘的官方 C++ 抽象（全称 Warp Matrix Multiply Accumulate），核心是三种 `wmma::fragment` 与配套的四个 collective 调用：

- `fragment<matrix_a,16,16,8, tf32, row_major>`：A 矩阵碎片；`matrix_b` 同理（B 也声明为 row-major，因为 smem 中两个矩阵都按行主序存放）；
- `fragment<accumulator, 16, 16, 8, float>`：累加器，元素类型恒为 `float`（TF32 MMA 的累加在 fp32 精度执行）；
- `load_matrix_sync(frag, ptr, ldm)`：warp 协作地从 smem 装载 fragment，`ldm` 是矩阵在 smem 里的**实际行距**：本章 A 用 $B_K + \text{kPadA} = 12$、B 用 $B_N + \text{kPadB} = 132$，用错 ldm 是这类代码最典型的静默错误；
- `mma_sync(D, A, B, C)`：一条（逻辑上的）warp 级矩阵乘累加 $\mathrm{D} = \mathrm{A} \times \mathrm{B} + \mathrm{C}$；
- `store_matrix_sync(ptr, frag, ldm, mem_row_major)`：把累加器写回 global。

寄存器占用：$m16n16k8$ 的 TF32 fragment 每 warp 持 $16 \times 8 = 128$ 个 A 元素（每 lane 4 个 32 位寄存器）、B 同为 128、累加器 $16 \times 16 = 256$ 个 fp32（每 lane 8 个）。本章每 warp 同时持有 2 个 A fragment + 4 个 B fragment + 8 个累加器 $\approx 2{\times}4 + 4{\times}4 + 8{\times}8 = 88$ 个 32 位寄存器（每 lane），再加上地址与循环变量，128 上限的寄存器预算已经相当紧张。后续第 十三 章引入寄存器双缓冲前，先得理解「fragment 即寄存器」这句话。

11.4.4　shared memory 布局与 padding

sgemm.cuh L225-243（模板参数、$B_M/B_N/B_K$ 与两级 smem 布局）

```
template <const int kWMmaM = 16, const int kWMmaN = 16, const int kWMmaK = 8,
          const int kWMmaTileM = 4, const int kWMmaTileN = 2, // warp 间排布数
          const int kValTileM = 2, const int kValTileN = 4,   // warp 内 fragment 值重复数
          const int kPadA = 4, const int kPadB = 4, const int kStages = 2>
__global__ void sgemm_tf32(float *A, float *B, float *C, int M, int N, int K) {
  // 256 线程 (8 warps) per block
  const int bx = blockIdx.x;
  const int by = blockIdx.y;
  const int NUM_K_TILES = (K + kWMmaK - 1) / kWMmaK;
  constexpr int BM = kWMmaM * kWMmaTileM * kValTileM; // 16×4×2 = 128
  constexpr int BN = kWMmaN * kWMmaTileN * kValTileN; // 16×2×4 = 128
  constexpr int BK = kWMmaK;                          // 8

```

```
// Dynamic shared memory (调用时指定大小: 16384 bytes)
extern __shared__ float smem_tf32[];
float *s_a = smem_tf32;
float *s_b = smem_tf32 + kStages * BM * (BK + kPadA);
constexpr int s_a_stage_offset = BM * (BK + kPadA); // 1024 floats
constexpr int s_b_stage_offset = BK * (BN + kPadB); // 1024 floats
```

smem 分两段：`s_a` 是 kStages×128×(8+4) 的 A tile（行距 12，kPadA = 4 个 float），`s_b` 是 kStages×8×(128+4) 的 B tile（行距 132，kPadB = 4），两段共 $2\times128\times12+2\times8\times132=5184$ 个 float = 20736 B。padding 的目的与第 八 章 transpose 的补列同理：bank 宽 4 B、32 个 bank 一轮，A 的行长恰好 32 B 时相邻行会映射到同一组 bank，补 16 B 打散行首的对齐关系，避免 `load_matrix_sync` 内部按行取数时的 bank conflict。（注意 L238 与 L242–243 注释里写的「16384 bytes」「1024 floats」是陈旧数字，实际值见 10.7 节勘误一、二。）两段布局的 PAD 与行距见图 11.3。

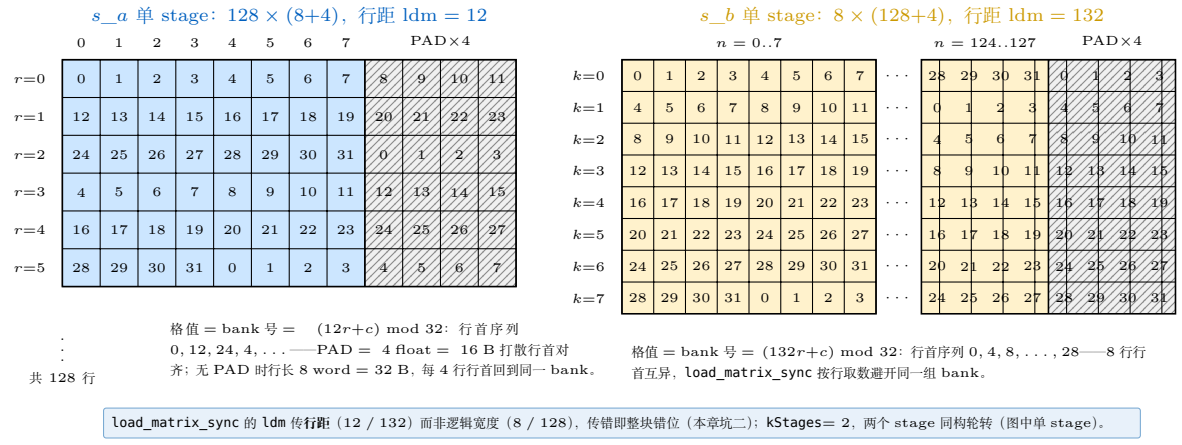


图 11.3: s_a / s_b 的 padded 行距布局（单 stage，格值为该 word 的 bank 号 = 行距 $\times r+$ 列 $\bmod 32$，斜线格为 PAD）。左 s_a：128 × (8+4)，`ldm`=12，行首 bank 序列 $0,12,24,4,\ldots$；右 s_b：8 × (128+4)，`ldm`=132，行首 bank 序列 $0,4,\ldots,28$。补 4 float（16 B）PAD 打散行首对齐、消除 `load_matrix_sync` 按行取数的 bank conflict——`ldm` 取行距而非逻辑宽度。

11.4.5 GMEM 到 SMEM：每线程一个 float4

sgemm.cuh L246-263（warp 划分与加载坐标映射）

```
const int tid = threadIdx.y * blockDim.x + threadIdx.x;
const int warp_id = tid / 32;   // 0~7, block 内 warp 编号
const int warp_m = warp_id / 2; // 0,1,2,3 (warp tiling M 维度)
const int warp_n = warp_id % 2; // 0,1 (warp tiling N 维度)

// ---- GMEM → SMEM 加载索引 (每线程加载 4 个 float, float4 向量化) ----
// s_a[BM][BK] = s_a[128][8], 按行加载, 每行 8 个元素
//   每线程加载 4 个元素 → 每行需要 2 个线程 → 128 行需要 256 线程 ✓
int load_smem_a_m = tid / 2;                // row 0~127
int load_smem_a_k = (tid % 2 == 0) ? 0 : 4; // col 0 或 4
// s_b[BK][BN] = s_b[8][128], 按行加载, 每行 128 个元素
//   每线程加载 4 个元素 → 每行需要 32 个线程 → 8 行需要 256 线程 ✓
int load_smem_b_k = tid / 32;       // row 0~7
int load_smem_b_n = (tid % 32) * 4; // col 0,4,8,...,124

// GMEM 全局索引
int load_gmem_a_m = by * BM + load_smem_a_m; // C 的行
int load_gmem_b_n = bx * BN + load_smem_b_n; // C 的列
```

256 个线程每线程为两级缓冲各搬 2 个 16 B（A、B 各一），坐标映射按「每线程 4 元素」规划：A 的 128×8 需要 128×2 个 4 元组（`tid/2` 定行、`tid%2` 定列半），B 的 8×128 需要 8×32 个 4 元组（`tid/32` 定行、`tid%32` 定列组），两个方向都恰好被 256 线程整除，无需循环。

11.4.6　cp.async 双缓冲流水线

sgemm.cuh L281-303（预加载 kStages − 1 个 tile 并等待）

```
  // ---- Pipeline: 预加载前 kStages-1 个 tile ----
#pragma unroll
  for (int k = 0; k < kStages - 1; ++k) {
    int load_gmem_a_k = k * kWMmaK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    int load_gmem_b_k = k * kWMmaK + load_smem_b_k;
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;

    uint32_t load_smem_a_ptr =
        smem_a_base_ptr +
        (k * s_a_stage_offset + load_smem_a_m * (BK + kPadA) + load_smem_a_k) *
            sizeof(float);
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    uint32_t load_smem_b_ptr =
        smem_b_base_ptr +
        (k * s_b_stage_offset + load_smem_b_k * (BN + kPadB) + load_smem_b_n) *
            sizeof(float);
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
  }
  CP_ASYNC_COMMIT_GROUP();
  CP_ASYNC_WAIT_GROUP(kStages - 2); // kStages=2 → wait_group(0)
  __syncthreads();
```

三条 PTX 指令构成异步拷贝的完整协议（宏定义见 common.cuh L41–51）：`cp.async.cg.shared.global` 发起 16 B 的 global→shared 搬运（`.cg` 表示只走 L2、不分配 L1，16 B 是 `.cg` 唯一合法宽度）；`cp.async.commit_group` 把此前发出的未提交拷贝打包成一个 per-thread 的组；`cp.async.wait_group N` 阻塞直到最多剩 N 个组未完成。预加载阶段发出 tile 0 的两组拷贝后 `wait_group(0)`，因为第一次计算前必须保证数据就位。

sgemm.cuh L307-330（主循环：先预取 tile k，再计算 tile $k-1$）

```
  for (int k = kStages - 1; k < NUM_K_TILES; ++k) {
    int smem_sel = (k + 1) % kStages;     // 当前计算用的 stage
    int smem_sel_next = k % kStages;      // 下一轮加载用的 stage

    int load_gmem_a_k = k * kWMmaK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    int load_gmem_b_k = k * kWMmaK + load_smem_b_k;
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;

    // 异步加载下一轮数据到 smem_sel_next
    uint32_t load_smem_a_ptr =
        smem_a_base_ptr +
        (smem_sel_next * s_a_stage_offset +
         load_smem_a_m * (BK + kPadA) + load_smem_a_k) *
            sizeof(float);
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    uint32_t load_smem_b_ptr =
        smem_b_base_ptr +
        (smem_sel_next * s_b_stage_offset +
         load_smem_b_k * (BN + kPadB) + load_smem_b_n) *
            sizeof(float);
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
    CP_ASYNC_COMMIT_GROUP();
```

主循环的迭代索引故意从 $k = \text{kStages} - 1 = 1$ 开始：迭代 k 发出 tile k 的搬运（写入 stage $k \bmod 2$），然后计算

tile $k-1$（读 stage $(k+1) \bmod 2 = (k-1) \bmod 2$，见 L308 的 `smem_sel`）。也就是说**搬运永远比计算领先一个 tile**，两个 stage 交替读/写，图 11.4 画的就是这个时序。

sgemm.cuh L332-348（TF32 fragment 声明与 A 碎片装载）

```
    // TF32 WMMA 碎片
    wmma::fragment<wmma::matrix_a, kWMmaM, kWMmaN, kWMmaK,
                   wmma::precision::tf32, wmma::row_major>
        A_frag[kValTileM];
    wmma::fragment<wmma::matrix_b, kWMmaM, kWMmaN, kWMmaK,
                   wmma::precision::tf32, wmma::row_major>
        B_frag[kValTileN];

    // 从 SMEM 加载 A 碎片 (每个 warp 加载 kValTileM=2 个 m16n8k16 tile)
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
      int warp_smem_a_m =
          warp_m * (kWMmaM * kValTileM) + i * kWMmaM; // warp 在 SMEM 中的行偏移
      float *load_smem_a_frag_ptr =
          s_a + smem_sel * s_a_stage_offset + warp_smem_a_m * (BK + kPadA);
      wmma::load_matrix_sync(A_frag[i], load_smem_a_frag_ptr, BK + kPadA);
    }
```

装载 A 碎片时，warp 只读取自己负责的 32 行：起始行 `warp_m * (kWMmaM * kValTileM)`（$= 32\,w_m$），再由 $i \in \{0,1\}$ 平移 16。B 碎片同理按 `warp_n` 平移列起点。注意 fragment 每次迭代都在循环体内重新声明、重新装载：WMMA 不提供寄存器双缓冲，这是它与第 十二 章手写路径的差距之一。

sgemm.cuh L350-371（B 碎片装载、2×4 次 mma、迭代末收口）

```
    // 从 SMEM 加载 B 碎片 (每个 warp 加载 kValTileN=4 个 m16n8k16 tile)
#pragma unroll
    for (int j = 0; j < kValTileN; ++j) {
      int warp_smem_b_n =
          warp_n * (kWMmaN * kValTileN) + j * kWMmaN; // warp 在 SMEM 中的列偏移
      float *load_smem_b_frag_ptr =
          s_b + smem_sel * s_b_stage_offset + warp_smem_b_n;
      wmma::load_matrix_sync(B_frag[j], load_smem_b_frag_ptr, BN + kPadB);
    }

    // MMA 计算: C_frag[i][j] += A_frag[i] × B_frag[j]
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        wmma::mma_sync(C_frag[i][j], A_frag[i], B_frag[j], C_frag[i][j]);
      }
    }

    CP_ASYNC_WAIT_GROUP(kStages - 2);
    __syncthreads();
  }
```

计算核只有一行的 `wmma::mma_sync` 双重循环（$2 \times 4 = 8$ 个 fragment 对），整个 kernel 的计算都发生在这里：每个 block 每 k-step 完成 $2 \times 128 \times 128 \times 8$ FLOP。迭代末尾 `wait_group(K_STAGE-2)` 对本章即 `wait_group(0)`，在迭代粒度收口：发出的 tile-k 搬运必须在本迭代结束前完成，下一个迭代才能安全地写「另一个 stage」并读「本 stage」。更深的流水（第 十二 章）把这一收口放宽为 `wait_group(kStages-2)`，允许一组搬运跨迭代 in-flight。

sgemm.cuh L373-379, L419-432（尾部排空与 store_matrix_sync 写回）

```
  // ---- 处理尾部: 最后 kStages-1 个 tile 已加载但未计算 ----
  // 确保所有 cp.async 完成
  if ((kStages - 2) > 0) {
```

```
    CP_ASYNC_WAIT_GROUP(0);
    __syncthreads();
  }
  // 计算剩余的 kStages-1 个 tile
  // ---- Store: 将 WMMA 累加器写回 GMEM ----
#pragma unroll
  for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileN; ++j) {
      int store_gmem_c_m = by * BM + warp_m * (kWMmaM * kValTileM) +
                           i * kWMmaM;
      int store_gmem_c_n = bx * BN + warp_n * (kWMmaN * kValTileN) +
                           j * kWMmaN;
      wmma::store_matrix_sync(C + store_gmem_c_m * N + store_gmem_c_n,
                              C_frag[i][j], N, wmma::mem_row_major);
    }
  }
}
```

主循环只计算到 tile NUM_K_TILES − 2，最后一个 tile 的数据已经在最后一次迭代里搬进 smem、但还没算；尾部块（L379–417，与主循环同构，节选）负责排空；kStages= 2 时主循环末尾的 `wait_group(0)` 已保证拷贝完成，L375 的条件等待被编译期剪掉。写回阶段每个 warp 把 8 个累加器 `store_matrix_sync` 到 C 的对应 32×64 子块，行距传 `N`（C 是 row-major 全矩阵）。

11.5　图示

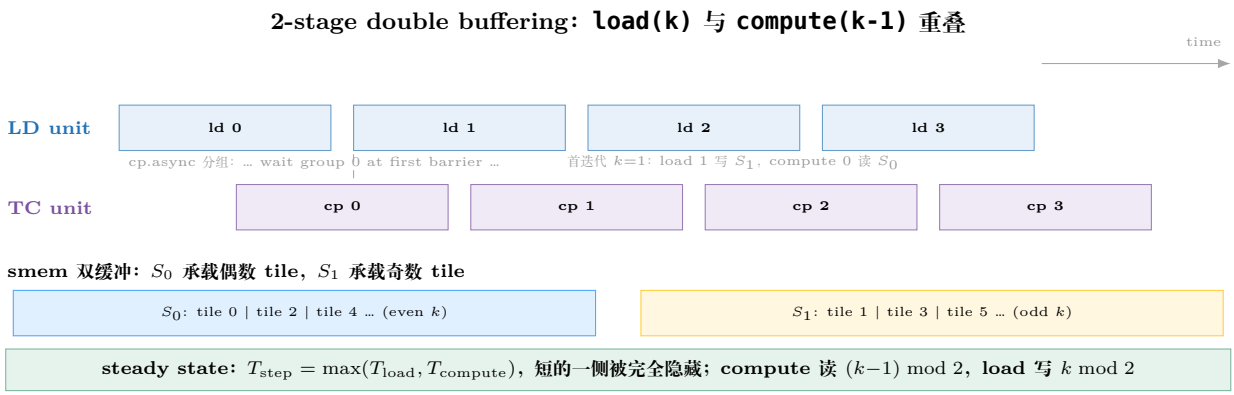


图 11.4: cp.async 双缓冲时序：搬运单元与 Tensor Core 各自的流水线交替占用 S_0/S_1 两级缓冲；迭代 k 内「向 k mod 2 写 tile k」与「从 $(k-1)$ mod 2 读 tile $k-1$」并行，稳态步长 $T_{\text{step}} = \max(T_{\text{load}}, T_{\text{compute}})$。

读图 11.4 的两个要点：其一，两条流水的错位正好是一个 tile，这是「预加载 $S-1$ 个 + 主循环先发加载再计算」共同保证的；其二，迭代末的 `wait_group + __syncthreads` 是写读换向的安全点；没有它，下一迭代的 cp.async 可能覆盖本 warp 尚未读完的 stage。

11.6　性能分析

算术强度。按 block 内 smem 复用口径：每 k-step 从 global 搬入 $(B_M B_K + B_K B_N) \times 4$ B $= (128{\times}8{+}8{\times}128) \times 4 =$ 8 KB，完成 $2B_M B_N B_K = 262{,}144$ FLOP，$AI \approx 32$ FLOP/B，远高于本机 roofline 拐点（PRO 5000 Blackwell：FP32 CUDA Core ∼60 TFLOPS 级、GDDR7 带宽 ≈1.34 TB/s（1344 GB/s）级，拐点 ∼45 FLOP/B 的量级，为标称口径；按 8.3 与 9.3.6 节实测 66.9 TFLOPS / 934 GB/s 折算则为 71.6 FLOP/B），判定 **compute-bound**，优化重心在充分利用 Tensor Core 而非节省带宽。按全局口径（每个 C tile 读整条 A 行块与 B 列块），$AI = \frac{2MNK}{4(MK+KN)} = \frac{MN}{2(M+N)}$，$M{=}N{=}1024$ 时约 256 FLOP/B，更加 compute-bound。

本章复测（RTX PRO 5000 72GB，sm_120，2026-09-11；$M{=}N{=}K{=}1024$，warmup 3 + 10 次均值）：`sgemm_tf32` 单卡 28.19 TFLOPS。两点口径说明：（1）该数值为共享开发机上的单次短测，仅作章节间相对对照，不与 README 大形状数据直接比较；（2）TF32 理论峰与实测的差距主要来自 $B_K{=}8$ 的窄 k-tile（MMA 指令发射密度低）与每迭代一次的迭代末同步。第 十二 章把 B_K 提到 16、同步放宽到 `wait_group(1)` 后，同机实测 76.55 TFLOPS（f16 累加路径），可见这两处设计确有代价。

11.7 实践经验

实践经验 11.1 (F4)：sgemm.cuh L193 与 L238：dynamic smem 大小注释前后矛盾 L193 头注释写「Dynamic SMEM：16KB（s_a 8KB + s_b 8KB）」、L238 又写「调用时指定大小：16384 bytes」，但 L222 的注释（20736 bytes）与实际模板参数才是自洽的：$2\times128\times12\times4+2\times8\times132\times4=20736$ B（bench/sgemm.cu 的 `tf32_smem_bytes` 同为 20736）。L193/L238 是早期无 padding 版本（$2\times128\times8\times4\times2=16$KB）的陈旧数字，已记入 `book/CHECKLOG.md`，正文以 20736 为准。

实践经验 11.2 (F3)：sgemm.cuh L242-243：stage offset 注释「1024 floats」错误 实际 $s_a_stage_offset=128\times12=1536$、$s_b_stage_offset=8\times132=1056$；「1024」是无 padding 的 128×8 旧值。代码取值正确，仅注释陈旧。

实践经验 11.3 (F4)：sgemm.cuh L196-198：计算密度注释的 MAC 数错误 注释称「64 个 MMA/block，每个 MMA 64 个 TF32 MAC，总计 4096 个 MAC/cycle/block」。核对：$m16n16k8$ 的一个 WMMA tile 是 $16\times16\times8=2048$ 个 MAC；每 warp $2\times4=8$ 个 tile、8 warp 共 64 tile = 131,072 MAC（每 k-step，即 262,144 FLOP）。「每个 MMA 64 个 MAC」与「4096 MAC/cycle」两个数字均不能由配置推出（且「每 cycle」的断言无依据），应按上述口径理解；注释问题已记录。

实践经验 11.4 (F3 核校通过)：sgemm.cuh L188：TF32「mantissa 10bit、约 3 位十进制」 $p=11$ 位二进制有效位 ≈3.31 位十进制（表 11.1），注释数字与独立推导一致。

实践经验 11.5 (F3 历史注记)：sgemm.cuh L226-227：WMMA_TILE/WARP_TILE 命名陷阱已修正 早期版本中，warp 间排布数叫 `WMMA_TILE_M/N`（4/2）、warp 内 fragment 重复数叫 `WARP_TILE_M/N`（2/4），两个名字的直觉正好对调，读者极易把两组参数的几何含义读反。现已重命名为 `kWMmaTileM/N`（warp 间排布）与 `kValTileM/N`（warp 内 fragment 值重复），名字即语义，并与 hgemm 的 `kMmaTile`/ `kValTile` 命名体系对齐（详见 10.4.1）。

11.8 常见坑

坑一：忘做 fp32→TF32 输入转换就拿 cuBLAS TF32 对数。WMMA 的 tf32 fragment 只保证「按 TF32 精度消费数据」；输入没先过 `__float_to_tf32`，与 `CUBLAS_TF32_TENSOR_OP_MATH` 参考的误差里就混入了「谁在何处舍入」的不可控差异。正确做法是 bench/sgemm.cu 的三步：保存 fp32 参考 → 原地 TF32 转换 → 与 TF32 参考对数。本章测试直接用「转换后数据」的 CPU fp64 做参考，把舍入差异彻底移出判定。

坑二：`load_matrix_sync` 的 ldm 传了逻辑宽度而非物理行距。A 碎片的 ldm 必须是 $B_K+\mathrm{kPadA}=12$（不是 8）、B 是 132（不是 128）。padding 改变了 smem 行距，ldm 不跟着改，fragment 会按错误的 stride 拼装数据：不越界、不报错，只是结果错了。

坑三：`cp.async.cg` 只支持 16 B。`.ca` 支持 4/8/16 B 且会分配 L1；`.cg` 只能 16 B、只走 L2。本 kernel 每线程恰好搬一个 float4（16 B）才用 `.cg`；若加载映射是 4 元素一组的其它倍数，必须改用 `.ca` 或重排映射。

坑四：wait 之后必须 `__syncthreads`。`cp.async` 的组是 **per-thread** 的：wait 只保证「本线程」发出的搬运完成。计算要读的 smem 可能由**别的线程**搬入，所以 wait 后必须再全 block 同步一次（L302–303、L369–370 两处都是这个匹配）。漏掉 sync 是 warp 间数据竞争，症状是偶发错数，属于最难查的一类。

坑五：fragment 布局不可假设。WMMA 规范不保证 fragment 内元素到线程/寄存器的映射，`x[i]` 式直读 accumulator 属未定义行为；要取出单个元素只能走 `store_matrix_sync`。需要显式控制布局时，换第 十二 章的手写 `mma.sync` 路径。

11.9 面试要点速查

Q1：TF32 是什么？为什么用它？

A：fp32 的 Tensor Core 变体：指数 8 位不变（动态范围同 fp32）、尾数 10 位（精度同 fp16）。牺牲 13 位尾数换取

Tensor Core 吞吐（Ampere 上约为 fp32 CUDA Core 的 8 倍密度）；舍入误差界 $2u\sum|a_jb_j|$，$u=2^{-11}$，约 3 位十进制有效数字，深度学习容差内。

Q2：WMMA 与底层 `mma.sync` 什么关系？

A：WMMA 是 warp 级 C++ 抽象：fragment 封装寄存器布局、`load/mma/store` 三件套封装装载-计算-写回。编译器把 $m16n16k8$ TF32 WMMA 拆成两条 $m16n8k8$ 的 `mma.sync` 指令。代价是布局黑盒、寄存器不可控、无寄存器双缓冲；要拿到这些控制权，得手写 PTX（第 十二 章）。

Q3：双缓冲如何掩盖搬运延迟？

A：两级 smem 交替读/写，迭代 k 向 k mod 2 级写入 tile k 的同时从 $(k-1)$ mod 2 级计算 tile $k-1$；稳态周期从 $T_{\text{ld}}+T_{\text{cp}}$ 降到 max（式 11.5）。协议三件套：`cp.async` 发起、`commit_group` 打包、`wait_group N` 等「至多剩 N 组未完成」。

Q4：`wait_group(0)` 与 `wait_group(1)` 的区别？何时用哪个?

A：N 表示允许保留的未完成组数。流水建立期与排空期用 0（数据马上要算），稳态满载期用 kStages−2（允许 $S-2$ 组跨迭代 in-flight）。本章 S=2 恒为 0；第 十二 章 S=3 稳态用 1。

Q5：为什么 kPadA/kPadB 都是 4？

A：4 个 float = 16 B = 一条 `cp.async` 的搬运宽度；行长从 32 B 变 48 B（A）后相邻行 bank 起点错开，消除 `load_matrix_sync` 按行取数的 bank conflict。pad 同时要保证 16 B 对齐不被破坏（4 float 恰好满足）。

11.10　最小测试与动手实验

测试文件：`book/tests/ch10_sgemm_tf32.cu`（逻辑抽取自 bench/sgemm.cu 的 `test_sgemm`，L21，TF32 段；参考改为 CPU fp64）。三个 case + bench：A（转换核逐位验证：GPU `__float_to_tf32` vs host 舍入）、B（$128\times128\times128$，grid 1×1 全流水）、C（$256\times256\times64$，grid 2×2 多 block）。参考构造与 bench/sgemm.cu 相同：先用 GPU 转换核把 A/B **原地**转成 TF32，拷回 host 后以转换后的数据做 CPU fp64 三重循环，误差只剩 fp32 累加顺序，判定用 TF32 档 `TOL_TF32` $=10^{-2}$。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --arch sm_120a --ch ch10`（编译 + 运行一体）。本机运行输出（RTX PRO 5000，2026-09-11；PASS 行的误差数值见表 11.2）：

```
case A: f32->tf32 convert kernel (n=4096)
case B: M=128 N=128 K=128
case C: M=256 N=256 K=64
ALL OK
```

表 11.2: ch10 测试 PASS 行数值摘录（完整行式输出见 `tests/.build/ch10_sgemm_tf32` 运行日志；tol 为判定容差）

case	判定项	最大绝对误差	tol
A	f32x4_tf32x4_kernel（GPU vs host 舍入）	逐位一致（rne/rna 均 4096/4096）	位级
B	sgemm_tf32 $128\times128\times128$	1.155e-05	1.0e-02
C	sgemm_tf32 $256\times256\times64$	5.327e-06	1.0e-02

case A 的括号里 rne 与 rna 都 4096/4096 命中：随机数据落在「恰好 tie」（低 13 位恰为 $0x1000$）的概率近乎为零，两种最近舍入在无 tie 样本上逐位等价：本测试证明「GPU 转换是 round-to-nearest 族」，但不足以区分 even/away（构造 tie 样本留给动手实验）。case B/C 的误差 10^{-5} 量级远小于容差，符合预期：输入舍入已被参考吸收，剩余差异只来自 fp32 累加顺序。动手实验建议：（1）用 `./.build/ch10_sgemm_tf32 bench` 复现 28.19 TFLOPS；（2）把 `kStages` 改成 3 观察是否还能通过（模板会多占一级 smem，launch 字节数须同步改）；（3）构造低 13 位为 $0x1000$ 的 tie 输入，判定 GPU 转换的 tie 行为是 even 还是 away。

延伸阅读

- @ 木子知《CUDA 编程：WMMA API》，https://zhuanlan.zhihu.com/p/620766588

(fragment/load_matrix_sync/mma_sync 的 API 级讲解与示例)；

- @Frank Wang《CUDA 编程：Async Copy》,
  https://zhuanlan.zhihu.com/p/685168850
  (cp.async 三件套语义、多级流水与 `wait_group` 的工程细节)；
- NVIDIA. Parallel Thread Execution ISA, §9.7.9.25 Asynchronous Copy（`cp.async` 官方语义；本地副本：cuda-cpp-kernel skill `references/ptx-docs`)。

第十二章　HGEMM：mma.sync m16n8k16、ldmatrix 与多级流水

12.1　本章导读

第 十一 章用 WMMA 用上了 Tensor Core，但 WMMA 是一层「厚抽象」：fragment 布局是黑盒、寄存器数量不可控、没有寄存器双缓冲、流水线只能两级。本章把这层抽象整个去掉，直接用 PTX 内联汇编写 HGEMM：`mma.sync.aligned.m16n8k16` 矩阵指令 + `ldmatrix` 碎片装载 + `cp.async` 三级流水（kStages=3）。本章的教学源码是 `hgemm_mma_stages_tn`（hgemm.cuh L129，TN 布局，$B_M \times B_N \times B_K = 128 \times 128 \times 16$，block 256 线程）。

本章源码区间为 hgemm.cuh L3–397（Phase 7b-2），PTX 宏（`HMMA16816`、`LDMATRIX_X4`、`LDMATRIX_X2`、`CP_ASYNC_CG` 等）定义在 common.cuh L32–109（附录 A）。无编译宏限定，`mma.m16n8k16` 要求 `sm_80` 及以上；配套最小测试为 `ch11_hgemm_mma.cu`（`book/tests/`）。

本章是全书**指令级细节最重**的一章：11.3 节把 `mma` 指令的每个限定符逐字段对照 PTX ISA 讲解，把 A/B/C 三个 fragment 在 32 个 lane 上的寄存器分布逐格画出来；`ldmatrix` 的寻址协议（哪个线程提供哪个地址）也同样逐线程推导。第 十三 章的 swizzle、第 十七 章的 FlashAttention 全部建立在这套 fragment 几何之上。**前置**：第 十 章 tiling 金字塔、第 十一 章 cp.async 协议。

12.2　问题与动机

动机一：WMMA 的四个局限。（1）布局黑盒：fragment 里哪个元素落在哪个寄存器不可知，epilogue 只能整块 `store_matrix_sync`；（2）寄存器失控：fragment 数组完全交给编译器，溢出与否取决于编译器；（3）无寄存器双缓冲：每迭代必须等 ldmatrix 完成才能 mma，装载延迟暴露；（4）流水线深度只有 2，搬运延迟稍有波动就覆盖不住。手写 PTX 后这四点全部可控：布局白盒（11.3.2–11.3.4）、寄存器显式声明（`RA/RB/RC` 数组）、装载与计算的重叠可以手工编排。

动机二：TN 布局让 ldmatrix 零转置。BLAS 的布局记号 T/N 来自 Fortran 的列主序约定：N（Normal）= 列主序原生、T（Transposed）= 行主序。于是 **TN** = A 行主序 + B 列主序，即 $C_{M\times N} = A_{M\times K} \times B_{K\times N}$，其中 B 以 $B^{\top}_{N\times K}$ 行主序存放（源码头注释 hgemm.cuh L10–22 对此有一段很好的辨析）。这个「转置存储」的好处：A 与 $B^{\top}$ **都是行主序、内维都是 K**，cp.async 搬运完全对称；而 mma 指令要求的 `.row.col`（A 行、B 列）恰好被「逐行装载 $B^{\top}$ = 逐列装载 B」天然满足，`ldmatrix` 一个 `.trans` 都不用加（11.4.4）。

动机三：更深的流水线。B_K 从 8 提到 16 后单个 tile 变大变慢，两级缓冲的「每迭代收口」不再够用；kStages=3 让一组搬运可以跨迭代 in-flight，搬运延迟的容忍窗口翻倍。代价是 smem 线性涨到 24 KB（$3\times(128\times16+128\times16)\times2$ B）。

12.3　数学原理

12.3.1　`mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16` 逐字段

common.cuh L81–88 的 `HMMA16816` 宏展开后就是这一条指令：

```
mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16
    {d0, d1}, {a0, a1, a2, a3}, {b0, b1}, {c0, c1};
```

按 PTX ISA（§9.7.14.5.14，本地 ptx-docs）逐限定符拆开：

表 12.1: `mma` 指令限定符逐字段解释（对照 PTX ISA §9.7.14.5）

限定符	语义
`mma`	矩阵乘累加：$D = A \times B + C$（warp 集体指令）。
`.sync`	发起线程须等到 warp 内**所有**线程都到达同一指令才继续。
`.aligned`	要求全 warp 执行**同一条**指令；条件分歧下使用属未定义行为。
`.m16n8k16`	形状：M=16（A 的行）、N=8（B 的列）、K=16（归约维）。A 是 16×16、B 是 16×8、C/D 是 16×8；每条指令 $2\times16\times8\times16 = 4096$ FLOP。
`.row`	A 以行主序 fragment 参与（A 的行方向连续）。
`.col`	B 以列主序 fragment 参与（B 的列方向即 K，连续）。
`.f16.f16.f16.f16`	依次为 D/A/B/C 的元素类型：输入输出全 fp16（寄存器均为 `.b32` 打包的 f16x2）。另有 `f32.f16.f16.f32` 变体（common.cuh L92，fp32 累加）。

三条精度语义（PTX 原文，写 epilogue 时必须了解）：(1) **元素乘至少单精度**执行；(2) 当 C 与 D 都是 `.f16` 时，**累加至少半精度**，也就是说本章 `f16.f16.f16.f16` 变体的累加链每条 `mma` 指令边界处都舍回 fp16；K 很长时误差沿 $\sqrt{K}$ 增长（11.9 节实测约 1.5×10^{-2}，K=128）。要高精度就换 `HMMA16816F32`（4 个累加寄存器的 fp32 版本）。(3) PTX 明确声明：**累加顺序、舍入与次正规处理均未指定**。不同架构/驱动上逐位结果可不同，正确性判定必须用容差。

寄存器带宽（每 warp、每条指令）：A 输入 4 个 `.b32`（8 个 f16）、B 输入 2 个 `.b32`（4 个 f16）、C/D 各 2 个 `.b32`（4 个 f16）。32 个 lane 合计持 A 的 16×16、B 的 16×8、C 的 16×8。每个 lane 持多少、在矩阵的什么位置，就是 fragment 布局。

12.3.2 A fragment：每 lane 8 个 f16，四个象限

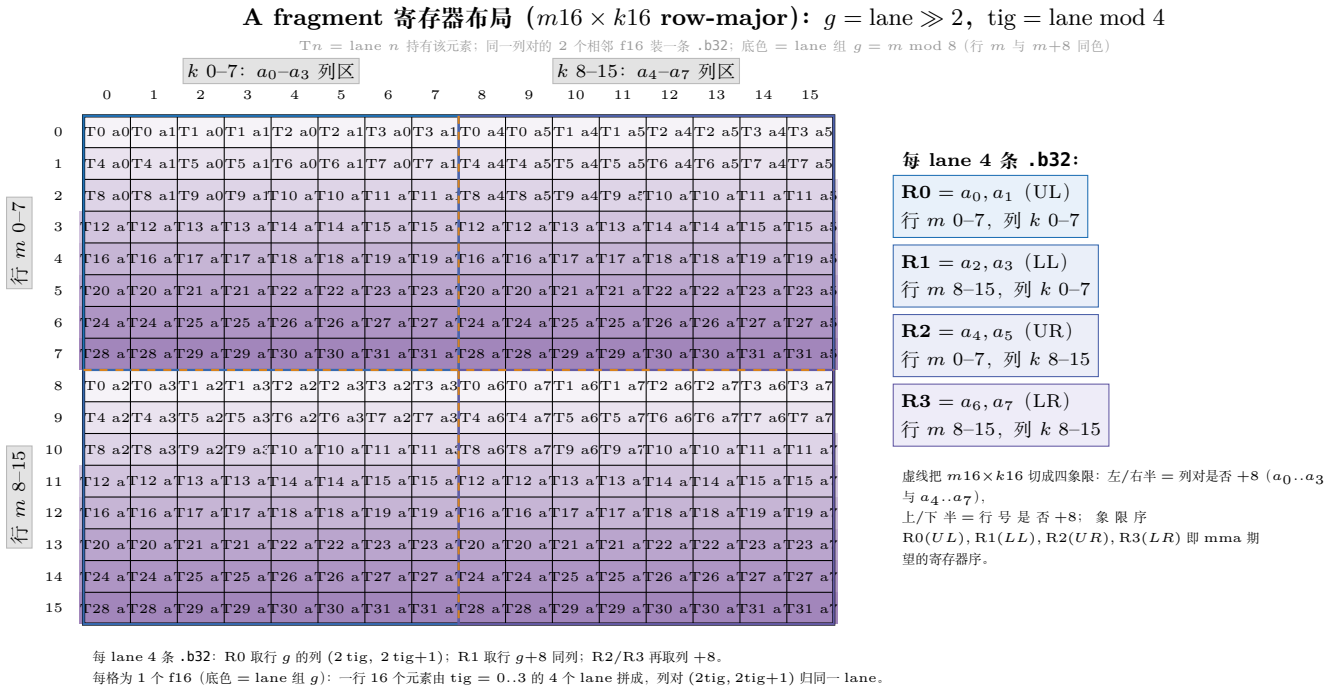


图 12.1: A fragment 寄存器布局（依据 PTX ISA Figure 79 的行/列公式绘制；Tn 表示 lane n 持有该元素，每格 2 个相邻 f16 装一条 `.b32`；四象限 $R0/R1/R2/R3$ 分别对应 UL/LL/UR/LR）。

PTX 给出的坐标公式（$\mathrm{g} = \mathrm{lane} \gg 2$，$\mathrm{tig} = \mathrm{lane} \bmod 4$）：元素 a_i（$i = 0..7$，每 `.b32` 装 a_{2t}, a_{2t+1}）的行、列为

$$\mathrm{row}(a_i) = \begin{cases} \mathrm{g} & 0 \le i < 2 \text{ 或 } 4 \le i < 6, \\ \mathrm{g}+8 & \text{否则}, \end{cases} \qquad \mathrm{col}(a_i) = 2\,\mathrm{tig} + (i \bmod 2) + 8 \cdot [i \ge 4]. \tag{12.1}$$

翻译成直观表述：每个 lane 持 A 的**两行**（g 与 g+8）、每行**两个相邻列对**（k 半区的 $2\,\mathrm{tig}, 2\,\mathrm{tig}{+}1$）；四个寄存器恰好按 [左上, 左下, 右上, 右下] 四象限排布（图 12.1）。一行 16 个元素由 4 个 lane（$\mathrm{tig} = 0..3$）拼成。

12.3.3　B fragment：每 lane 4 个 f16，按列分布

B 是 $k16 \times n8$，列主序参与。坐标公式：

$$\mathrm{row}(b_i) = 2\,\mathrm{tig} + (i \bmod 2) + 8 \cdot [i \ge 2], \qquad \mathrm{col}(b_i) = \mathrm{g}. \tag{12.2}$$

即每 lane 持 B 的**一列**（第 g 列，8 行）中的两对：$R0 = \{b_0, b_1\}$ 是 $k \in \{2\mathrm{tig}, 2\mathrm{tig}{+}1\}$（上半 k 区），$R1 = \{b_2, b_3\}$ 再加 8（下半 k 区）。8 个 lane 组（$\mathrm{g} = 0..7$）拼齐 8 列。与 A 对照记忆：A 按行分、B 按列分，k 方向都是「4 个 lane 各持 2 个相邻元素」。B 侧逐格布局见图 12.2（图 12.1 的 B 侧对照图）。

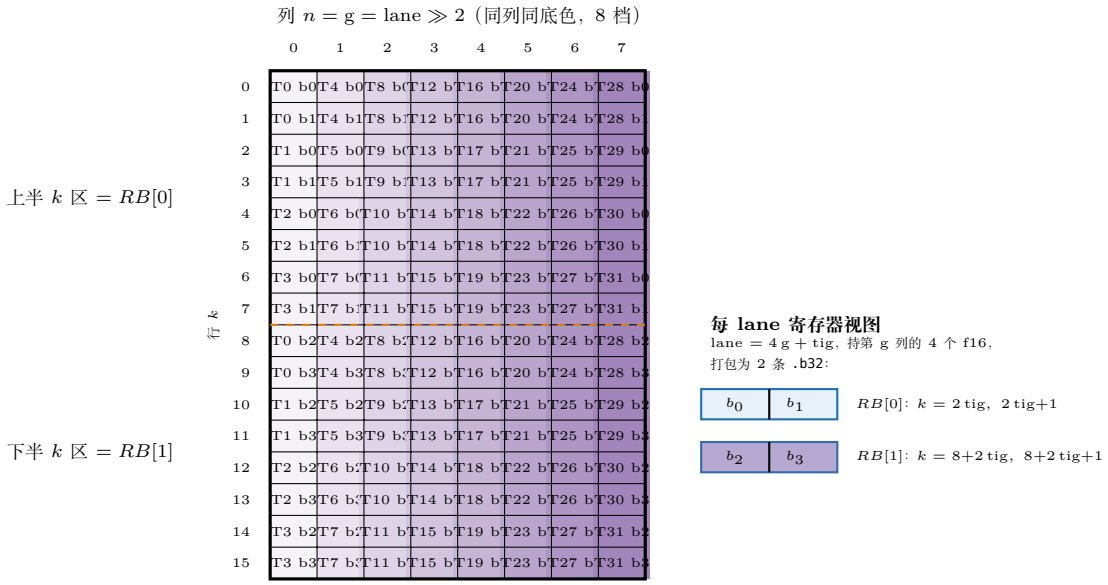


图 12.2: B fragment（$k16 \times n8$）寄存器布局，式 12.2 的网格化（图 12.1 的 B 侧对照图）：列 $n = \mathrm{g} = \mathrm{lane} \gg 2$（同列同底色 8 档），行 k 按 $b_0 \to 2\,\mathrm{tig}$、$b_1 \to 2\,\mathrm{tig}{+}1$、$b_2/b_3$ 再 +8 分布。右：每 lane 4 个 f16 打包为 $RB[0]/RB[1]$ 两条 `.b32`。与 A 对照：A 按行分、B 按列分。

12.3.4　C fragment：epilogue 的出发点

C/D 是 $m16 \times n8$，fp16 版每 lane 2 个 `.b32`（4 个 f16）：

$$\mathrm{row}(c_i) = \mathrm{g} + 8 \cdot [i \ge 2], \qquad \mathrm{col}(c_i) = 2\,\mathrm{tig} + (i \bmod 2). \tag{12.3}$$

每 lane 持两行（g、g+8）各一对相邻列；$RC[0]$（第一个 `.b32`）覆盖行 0–7，$RC[1]$ 覆盖行 8–15。源码 epilogue 的大注释（hgemm.cuh L311–333）画的就是这张图，与式 12.3 逐格一致（核校通过）；其 TikZ 版见图 12.3。

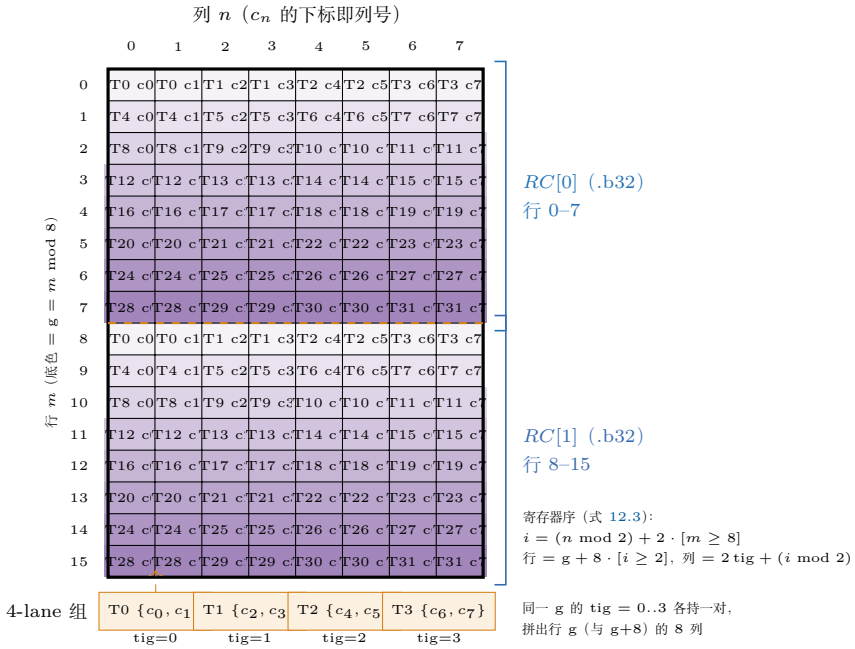


图 12.3: C fragment（$m16 \times n8$）寄存器布局（式 12.3）：行 m = g = lane $\gg 2$（底色 8 档，行 0–7 与 8–15 由同批 lane 重复覆盖），列对由 tig = lane%4 决定；$RC[0]$ 装行 0–7、$RC[1]$ 装行 8–15。底部：4-lane 组内 tig = 0..3 各持一对 $\{c_{2t}, c_{2t+1}\}$，拼齐一行 8 个 half——11.4.6 节 epilogue shuffle 的出发点。

12.3.5 `ldmatrix`：warp 集体装载指令逐字段

```
ldmatrix.sync.aligned.m8n8.x4.shared.b16 {r0, r1, r2, r3}, [addr];
ldmatrix.sync.aligned.m8n8.x2.shared.b16 {r0, r1},         [addr];
```

表 12.2: `ldmatrix` 限定符与寻址协议（对照 PTX ISA §9.7.14.5.15）

限定符	语义
`.m8n8`	每个矩阵 8×8、元素 16 位（b16）；一条指令装载 1/2/4 个矩阵。
`.x1/.x2/.x4`	矩阵个数；x1 用 lane 0–7 的地址，x2 用 lane 0–15，x4 用全部 32 lane。
`.shared`	地址处于 shared 状态空间（.b16 版无 `.trans` 时按行装载）。
`.trans`	可选：按**列**装载（转置）。PTX 原文：“.trans indicates that the matrix is loaded in column-major format”——即不加 `.trans` 就是普通行装载。
`.sync/.aligned`	同 `mma`：warp 集体执行、禁止分歧。
对齐	「一组 4 个连续线程装载 16 字节」，地址须 16 B 自然对齐。

寻址协议是 `ldmatrix` 的核心：**每个矩阵 8 行，需要 8 个地址（每地址是一行的 16 B 起点）**；x4 共 32 个地址由 32 个 lane 一人一个提供，lane t 的地址属于第 $t/8$ 个矩阵的第 $t \bmod 8$ 行；x2 只需 16 个地址（lane 16–31 的地址被忽略，PTX 允许把低位 lane 的地址复制过来凑数）。装载结果天然匹配 mma fragment：矩阵 i 的行 r 落在 lane $4r + \text{tig}$ 的寄存器 r_i 的 tig 列对，恰好是式 12.1 要求的分布。这正是 `ldmatrix` 与 `mma` 配套设计之处：一条指令完成 32 lane 的碎片协同装载，若用普通 `ld.shared`，同样的事需要每 lane 独立寻址 4 次且极易 bank conflict。

12.3.6 kStages = 3 的多级流水

三级缓冲 S_0, S_1, S_2 轮转，迭代 k 读 $k \bmod 3$、预取 $(k{+}2) \bmod 3$（即 tile $k{+}2$ 的搬运在迭代 k 就发出）。稳态期 `wait_group(kStages-2) = wait_group(1)`，允许 1 组搬运 in-flight，即迭代 k 结束时，tile $k{+}1$ 必须就位（供迭代 $k{+}1$ 读），tile $k{+}2$ 的那组允许还没到。尾部（$k + \text{kStages} - 1 \geq \text{NUM_K_TILES}$，没有新 tile 可预取）切到 `wait_`

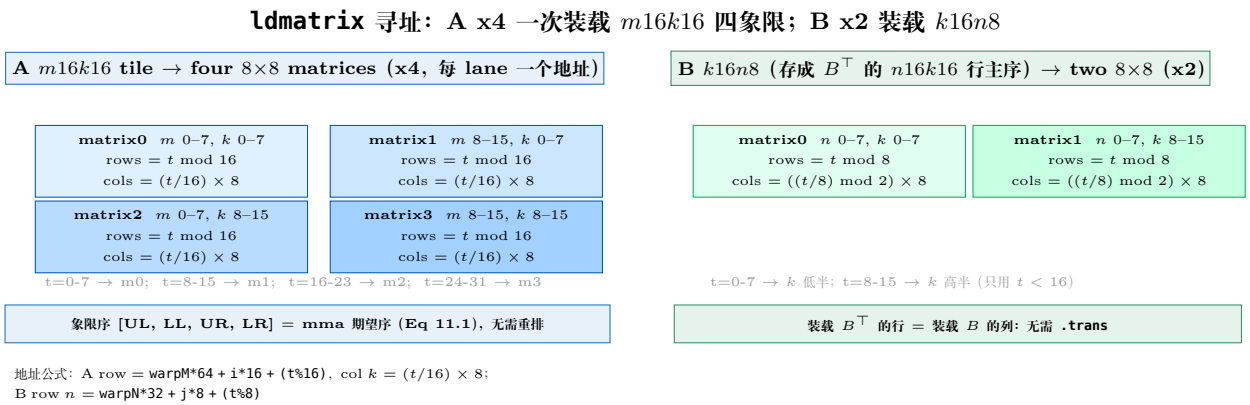


图 12.4: 本章 kernel 的 `ldmatrix` 寻址映射：A 用 x4 一次装载 $m16 \times k16$ 的四象限（象限序恰为 mma 期望序）；B 用 x2 装载 $k16 \times n8$（借 $B^\top$ 行主序存储，无需.trans）。

`group(0)` 排空。与第 十一 章对比：两级流水每个迭代都要 wait(0) 收口，三级流水稳态只 wait(1)。「组」这个计数单位让同步粒度从「每迭代全等」放松为「保持距离即可」。三级轮转的稳态与尾部时序见图 12.5。

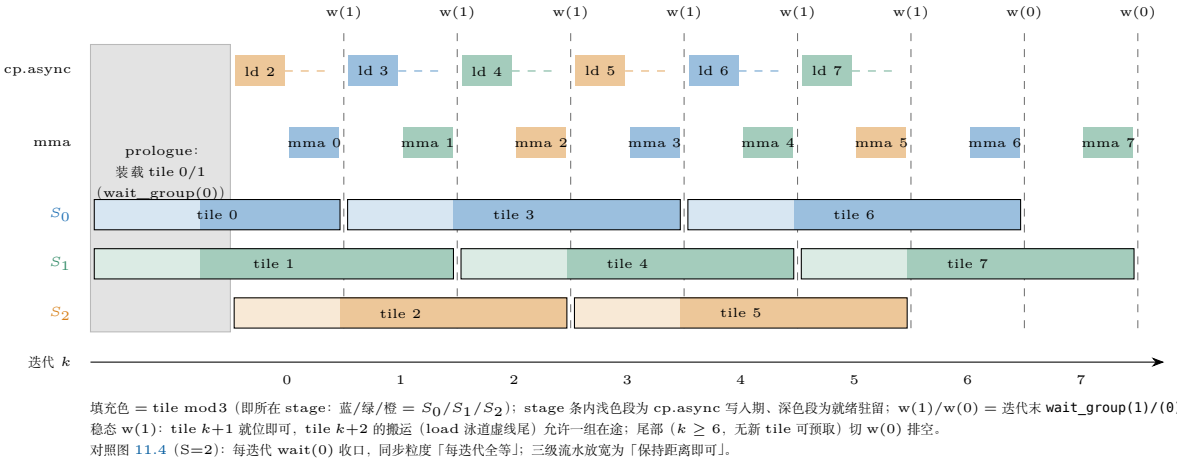


图 12.5: kStages= 3 三级流水时序：迭代 k 读 $S_{k \bmod 3}$（tile k）、同时预取 $(k{+}2) \bmod 3$（tile $k{+}2$），三个 stage 按 tile mod3 轮转。稳态迭代末 `wait_group(1)` 允许一组搬运在途；尾部无 tile 可预取时切 `wait_group(0)` 排空。与图 11.4 的两级流水（每迭代 wait(0)）相比，同步粒度从「每迭代全等」放松为「保持距离即可」。

12.4　设计与实现

12.4.1　Tile 金字塔与 256 线程

hgemm.cuh L119-139（模板参数与 $B_M/B_N/B_K$ 推导）

```
// ============================================================================
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim, also BK tile = kMmaK
          const int kMmaTileM = 2,          // warps along M, 2 → warp tile M = 32
          const int kMmaTileN = 4,          // warps along N, 4 → warp tile N = 32
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 4,          // value-repeat along N, BN = 8*4*4 = 128
          const int kStages = 3,            // cp.async pipeline depth
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
```

```
  static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
  // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
  // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
  constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
  constexpr int BK = kMmaK;                         // 16
```

三级相乘：`MMA atom` $m16n8k16$（一条指令）→ `MMA tile` 32×32（$2\times4=8$ 个 warp 的 atom 排布，按列主序摆成 $2_M\times4_N$，`warp_m = warp_id%2`、`warp_n = warp_id/2`，见 L157–169 的排布图）→ `block tile` 128×128（每 warp 的 `warp tile` 为 64×32，用 $kValTileM\times kValTileN=4\times4$ 个 atom 重复）。用记号表写：$B_M=kMmaM\cdot kMmaTileM\cdot kValTileM=16\cdot2\cdot4$，$B_N=8\cdot4\cdot4$，$B_K=kMmaK=16$。源码注释把两层重复分别对应到 CuTe 的 TiledMMA 与 Value Tile 概念（L180–183），第 二十三 章会回到这个对应。整个排布的几何全景见图 12.6。

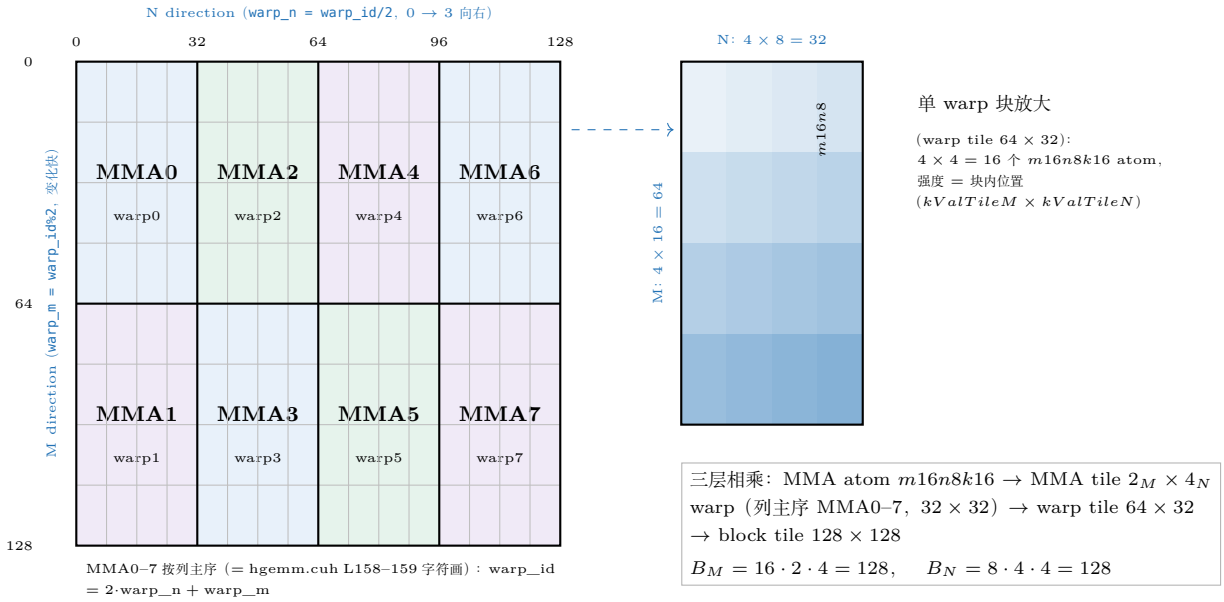


图 12.6: MMA warp 2×4 排布（左，128×128 block tile，浅色 = warp_id）：MMA0–7 按列主序——N 向 MMA0/2/4/6、M 向 MMA1/3/5/7（`warp_m = warp_id%2` 变化快）；每 warp 块内 4×4 个 $m16n8$ atom 小格。右：单 warp 块放大（warp tile 64×32，蓝深 = atom 块内位置）；侧栏三层记号 $B_M=16\cdot2\cdot4$、$B_N=8\cdot4\cdot4$。

12.4.2 smem 布局与 cp.async 预载

hgemm.cuh L141-152（kStages 级 smem：s_a 与 s_b 均按 [128][16] 行主序）

```
  // Dynamic shared memory: kStages 个 stage 的 A 和 B
  // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
  // row-major，即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
  // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD；尤其 B 在 TN 布局下常见会额外
  // 加 B_PAD 来打散 bank 映射，避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + kStages * BM * BK;     // A 和 B 连续存放
  constexpr int s_a_stage_offset = BM * BK; // 128*16
  constexpr int s_b_stage_offset = BN * BK; // 128*16  ⚠ BN(128)×BK(16) = B^T row-major
  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / kWarpSize; // 0~7
```

本章不 pad（PAD=0，swizzle 留给第 十三 章，那里会讲为什么无 pad 的 [128][16] half 行距在 `ldmatrix` 按行装载时恰好躲开 bank conflict 的多数情形、以及何时躲不开）。预加载与加载映射：

hgemm.cuh L190-210（预载 kStages-1 级；每线程每矩阵一个 16 B 搬运）

```
  for (int k = 0; k < (kStages - 1); ++k) {
    int load_gmem_a_k = k * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
    uint32_t load_smem_a_ptr = (smem_a_base_ptr +
      (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    int load_gmem_b_k = k * BK + load_smem_b_k;
    // B^T: [n][k] row-major (即 B[k][n] col-major) ⚠
    int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
    uint32_t load_smem_b_ptr = (smem_b_base_ptr +
      (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);

    CP_ASYNC_COMMIT_GROUP();
  }

  CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
  __syncthreads();
```

加载坐标与第 十一 章相同但搬 half：`tid/2` 定行（0–127）、`tid%2` 选 16 B 半行（k 的 0–7 或 8–15，恰 8 个 half = 16 B）。注意两个 gmem 索引**形式对称**：A 用 `m*K + k`、B 用 `n*K + k`，后者就是「B 以 $B^\top$ 行主序存储」的落点，也是 TN 布局在代码里的全部体现（L194 与 L197 的两行注释）。

12.4.3　统一循环：预取、条件加载与自适应等待

hgemm.cuh L212-241（统一主循环：条件预取 tile k+kStages−1）

```
  // 统一循环: k 从 0 开始，每次迭代负责 tile k (加载 + 计算合并为单循环)
  const int NUM_K_TILES = div_ceil(K, BK);
  // 此处不用pragma unroll，因为 K 不是编译期常量，因此NUM_K_TILES 也不是编译期常量
  for (int k = 0; k < NUM_K_TILES; ++k) {
    int smem_sel = k % kStages;                        // 计算 tile k 的 stage
    int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage

    // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
    // load了(kStages-1) 个 stage，因此这里是从 tile (k+kStages-1) 开始预加载
    // TN 布局: A 的 gmem 地址用 m*K+k (row-major) , B^T 的 gmem 地址用 n*K+k
    // (row-major，内维连续的是 K)
    if (k + kStages - 1 < NUM_K_TILES) {
      int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
      int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
      int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
      int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]

      uint32_t load_smem_a_ptr =
          (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
                              load_smem_a_m * BK + load_smem_a_k) *
                                 sizeof(half));
      CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

      uint32_t load_smem_b_ptr =
          (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
                              load_smem_b_n * BK + load_smem_b_k) *
                                 sizeof(half));
      CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
      CP_ASYNC_COMMIT_GROUP();
    }
```

与第 十一 章「主循环从 k=K_STAGE−1 开始 + 尾部单独排空」的写法不同，这里用**统一循环**：k 从 0 开始、迭代 k 负责计算 tile k、同时预取 tile $k+\text{kStages}-1$（前 kStages−1 个 tile 已由预载填满）。预取带边界条件（L220 的 `if`），尾部迭代不再发搬运；等待也自适应（L297–304）：满载期 `wait_group(kStages-2)`、排空期 `wait_group(0)`。好处是消除了尾部的重复代码块，对 K 不是 kStages 倍数的形状也更稳健；代价是每迭代多两次分支判断。

12.4.4 ldmatrix 装载 A（x4）与 B（x2）

hgemm.cuh L243-264（ldmatrix.x4 装载 A：lane 地址到 $m16k16$ 四象限）

```
    // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
    // TN 布局关键: A 用 x4 (非转置)，因为 A 是 row-major; B 用 x2 (非转置)，smem 中
    // B^T 为 row-major，逐行加载即得 B 的列，天然匹配 col-major B
    uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
    uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA

    // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
      // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
      // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
      int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
      // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与，每个线程提供一个有效的，不重叠的addr
      int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
      int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
      uint32_t lane_smem_a_ptr =
          (smem_a_base_ptr +
           (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
               sizeof(half));
      LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
    }
```

地址生成只有两行（L257–258）：行 `lane%16`、列 `(lane/16)*8`。把它对到 11.3.5 的协议上：lane 0–7 给出矩阵 0（m 0–7 × k 0–7）的 8 行地址，落在 `RA[i][0]`（左上）；lane 8–15 给矩阵 1（m 8–15 × k 0–7）→ `RA[i][1]`（左下）；lane 16–23 给矩阵 2（m 0–7 × k 8–15）→ `RA[i][2]`（右上）；lane 24–31 给矩阵 3（右下）→ `RA[i][3]`。四个目的寄存器的到达顺序恰好就是式 12.1 要求的 [左上, 左下, 右上, 右下]，一行汇编都不用重排。

hgemm.cuh L266-285（ldmatrix.x2 装载 B：借 $B^\top$ 行主序免.trans）

```
    // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
    // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
    // ldmatrix 逐行加载 B^T 的行 = B 的列，天然给出 col-major B fragment → 直接匹配 MMA row.col
#pragma unroll
    for (int j = 0; j < kValTileN; ++j) {
      // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
      // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
      // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
      int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
      // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
      // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与，后16个线程传的addr会被忽略
      int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
      int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
      uint32_t lane_smem_b_ptr =
          (smem_b_base_ptr +
           (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
               sizeof(half));
      LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
    }
```

B 侧地址：行 `lane%8`（$B^\top$ 的 n 0–7）、列 `((lane/8)%2)*8`（k 半区）。x2 只消费 lane 0–15 的 16 个地址，lane 16–31 传的重复地址被硬件忽略（PTX 允许；sm_75 以下才强制全 warp 合法地址）。装载语义值得再念一遍：`ldmatrix`（无 `.trans`）**按行**装载 8×8 的 $B^\top$ 子块，即按 n 行取 k 方向 8 个 half；而 mma 的 `.col` B fragment 恰好要「每 lane 一列 n、k 方向 2 对」（式 12.2）。两者统一的方式：$B^\top$ 的行就是 B 的列。源码 L267–268 注释「为什么不用.trans」说的正是这件事。

12.4.5　计算核：每 warp 每迭代 16 条 mma

hgemm.cuh L287-304（4×4 个 atom 的 mma 双重循环与自适应等待）

```
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
                  RB[j][0], RB[j][1], // B fragment
                  RC[i][j][0], RC[i][j][1]);
      }
    }

    // 自适应等待：流水线满载期用 kStages-2，尾部排空用 0
    if (k + kStages - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(kStages - 2);
    } else { // 对于尾部的 k，等待所有剩余的 cp.async 完成
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();
```

每 warp 每 k-迭代执行 $kValTileM\times kValTileN=16$ 条 `HMMA16816`（4 个 A 碎片 × 4 个 B 碎片全组合，C 累加器 `RC[4][4][2]` 常驻寄存器、跨 k 累加）。整 block 每迭代 $8\times16=128$ 条指令 $\times4096$ FLOP $=524{,}288$ FLOP。累加器寄存器每 lane $4\times4\times2=32$ 个 `.b32`，加上 `RA` 16 个、`RB` 8 个与地址变量，`__launch_bounds__(256)`（L129）下编译器仍有余量不溢出，这便是手写路径「寄存器白盒」的直接收益。

12.4.6　Epilogue：shuffle 收拢 C fragment

hgemm.cuh L336-350（用 `__shfl_sync` 从 4 lane 组收齐一行 8 个 half）

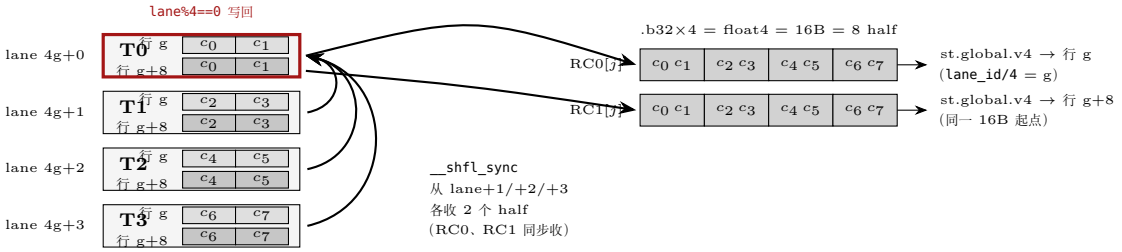
```
      //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
      // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0，4 次 shuffle 凑齐
      // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}，再由 lane%4==0 128-bit store。
      // ================================================================
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        RC0[j][0] = RC[i][j][0];
        RC1[j][0] = RC[i][j][1];
        RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
      }
```

按式 12.3，C 的一行 8 个 half 分散在 4 个 lane（tig $=0..3$ 各持一对）。要合并成一条 16 B（8 个 half）的向量化写回，每个 lane 先把邻居的半行 shuffle 过来：`RC0[j]` 凑「g 行」（行 0–7 之一）的 4 条 `.b32`，`RC1[j]` 凑「g+8 行」的。随后只有 `lane%4==0` 的 lane 执行写回：

hgemm.cuh L365-382（每 4-lane 组的首 lane 两条 128-bit store 写回）

```
      if (lane_id % 4 == 0) {
        // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
        int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
        // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}，对应 RC0/RC1 (+8) 的行号，表示2个8x8矩阵的行号
        int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
          int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
          int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
          int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
          int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8
    matrix
          // 128-bit store: 一次写入 8 个 half
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RC0[j][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RC1[j][0]);
        }
```

行号映射 `lane_id/4` = g：lane 0/4/8/.../28 分别对 g 行与 g+8 行各写一条 `float4`（即 16 B）store，各覆盖 8 个 half；列起点由 `warp_n` 与 j 平移。每个 warp 负责 64×32 子块，与 11.4.1 的排布完全一致。写回没有经过 smem 中转：寄存器布局白盒后 epilogue 可以走「shuffle → 直接宽写」的捷径，WMMA 做不到这一点（对照第 十一 章坑五）。收拢与写回的数据流见图 12.7。



每个 4-lane 组的 T0–T3 各持一对 $\{c_{2t}, c_{2t+1}\}$（上行：行 g 的 RC[i][j][0]；下行：行 g+8 的 RC[i][j][1]，深浅两档同图 12.2 的寄存器条）。shuffle 后首 lane 的 RC0[j]/RC1[j] 凑齐整行 8 个 half，仅 `lane%4==0` 的 lane 每 j 写两条 128-bit store——列起点由 `warp_n` 与 j 平移，整 warp 铺满 64×32 子块，不经 smem 中转。

图 12.7: Epilogue shuffle 收拢与写回：4-lane 组内 T0–T3 各持一对 $\{c_{2t}, c_{2t+1}\}$（上行 = 行 g、下行 = 行 g+8 的寄存器对），`__shfl_sync` 从 lane+1/+2/+3 各收 2 个 half，把 RC0[j]（行 g）与 RC1[j]（行 g+8）各凑成 4 条 `.b32` = 一条 16B `float4`；随后仅 `lane%4==0` 的 lane 执行写回，行号 = `lane_id/4` = g，两条 128-bit store 分别写行 g 与 g+8。

12.5 性能分析

每条 `mma.m16n8k16` 4096 FLOP、每 warp 每迭代 16 条、每 block（8 warp）每 k-迭代 $2 \times 128 \times 128 \times 16 = 524{,}288$ FLOP、搬入 $(128+128) \times 16 \times 2$ B = 8 KB，smem 口径 $AI = 64$ FLOP/B，判定为 compute-bound。本章复测（RTX PRO 5000 72GB，sm_120，2026-09-11；M=N=K=1024，warmup 3 + 10 次均值；`.tmp/book-bench/ch11/`）：`hgemm_mma_stages_tn` 76.55 TFLOPS（f16 累加）。同机同形状下第 十一 章 TF32 WMMA 版为 28.19 TFLOPS，2.7 倍的差距来自三层：f16 数据通路本身快于 TF32、B_K=16 的指令发射密度翻倍、三级流水的同步更松。精度口径见 11.3.1：f16 累加在 K=1024 时误差还会放大（本章正确性测试 K≤128 实测 1.5×10^{-2}），生产环境常用 `HMMA16816F32` 的 fp32 累加变体换精度。（数值为共享开发机单次短测，仅供章节间相对对照。）

更大形状的 A/B 复测（RTX PRO 5000 110 SM，CUDA 13.2；`notes_v2_sm120a.bin --bench --bench_all`，M=N=K=8192 fp16，cuBLAS f16 参照 246.9 TFLOPS；同批 `--bench-all` 内 cuBLAS 逐行成对计时，本章与第 十三、十五 章分别取到 246.9/246.5/246.6 T，各行分母相差 ≤ 1%，不做归一）见表 12.3：

表 12.3: HGEMM mma.sync 实测（RTX PRO 5000，M=N=K=8192，fp16）

Kernel	Max Err	TFLOPS/cuBLAS（加速比）
HGEMM MMA (S=2, BLK_SW=0)	0.000e+00	119.7/246.9 (0.48x)
HGEMM MMA (S=2, BLK_SW=1)	0.000e+00	124.6/246.9 (0.50x)
HGEMM MMA (S=3, BLK_SW=1)	0.000e+00	127.3/246.9 (0.52x)

三点读数：（1）mma.sync 基础实现仅达 cuBLAS 的约一半（0.48~0.52×）：指令用对不等于流水编排到位，装载与计算的重叠窗口、epilogue 写回都还有可观浪费；（2）S=3 比 S=2 略好（同 BLK_SW 下 +2.2%），与 11.3.6 的判断一致：更深的缓冲摊薄 cp.async 的搬运抖动，但 smem 占用同步上涨，边际收益递减；（3）block swizzle（BLK_SW 0 → 1）带来约 4% 改善（119.7 → 124.6 TFLOPS），大形状下 CTA 的调度顺序对 gmem 局部性影响不可忽略。

最后回答一个实操问题：**怎么确认 kernel 真的跑在 Tensor Core 上？**两级证据：（1）SASS 级：对可执行文件 `cuobjdump -sass` 后检索 `HMMA.16816` 指令，本 kernel 主循环应出现成串的 `HMMA.16816.F16`（每 warp 每迭代 16 条）与 `LDSM`（ldmatrix 的 SASS 形态，x4 对应 `LDSM.128`）；（2）profile 级：ncu 的指令混合指标里 `sm__inst_executed_pipe_tensor_op` 应占主导。若 HMMA 计数为零而 `FFMA`/`HFMA2` 居高，说明源码被宏门控剪掉、走了 CUDA Core 的 fallback 路径，这是改造这类教学 kernel 时最常见的失败模式。

12.6　实践经验

实践经验 12.1 (F2)：hgemm.cuh L35：「ldmatrix 默认加载 col-major」与 PTX 文档相反 注释原文（`_nn` 布局对比段）称 B 行主序时「需要.trans」的理由是「ldmatrix 默认加载 col-major」。PTX ISA §9.7.14.5.15 原文：*"Optional qualifier .trans indicates that the matrix is loaded in column-major format"*，即**不加 `.trans`** 是普通（行）装载，加了才是列装载。结论方向恰好写反。本章 kernel 的实际用法（TN 布局、A/B 都用非 trans 装载）不受影响；但按该注释推理会在 NN 布局下选错 trans 与否。已记入 `book/CHECKLOG.md`，正文以 PTX 文档为准。

实践经验 12.2 (F2 核校通过)：hgemm.cuh L6 与 L73-80：m16n8k16 形状与寄存器数「$[16\times8]=[16\times16]\times[16\times8]$」与 PTX 的 $M\times K\times K\times N$ 定义一致；HMMA16816 的「2 输出寄存器 + 4 A + 2 B」「每线程 4 half」亦与 Figure 79/81/83 的 fragment 公式一致。

实践经验 12.3 (F2 核校通过)：hgemm.cuh L311-333：epilogue C fragment 大注释 注释的行/列映射（`row = t/4`、列对 `t%4`、RC[0] 行 0–7 / RC[1] 行 8–15）与 PTX Figure 83 公式逐格一致，是源码注释中少见的完整准确版本。

实践经验 12.4 (F3 核校通过)：common.cuh L46-53：commit/wait 语义注释「commit_group 归组、wait_group N 等 pending ≤ N、wait_all = commit + wait 0」与 PTX §9.7.9.25.3 的描述一致（第 十一 章 10.4.6 已展开）。

12.7　常见坑

坑一：B 没转置直接传入 kernel。kernel 要求的输入是 $B^\top$（$N\times K$ 行主序）。把 $K\times N$ 的 B 原样传入，gmem 索引 `n*K + k` 会按错误布局取数：不越界（总元素数相同）、不报错、结果系统性错误。测试须像 bench/hgemm.cu 一样显式构造 $B^\top$（本章测试的 `hbt` 循环）。

坑二：`.aligned` 与 early return 不相容。mma/ldmatrix 要求全 warp 无分歧。本 kernel L174–175 的 `if (... >= M ...) return;` 依赖「M/N 按 128 对齐时条件恒假」这一前提才安全；若形状未对齐，同一 warp 的 32 个 lane（跨 16 个 m 行）可能部分返回部分继续，后续 mma 即未定义行为。对齐假设写在头注释里（L225 上方），改造 kernel 支持非对齐形状时必须先处理这里。

坑三：x2/x1 的高位 lane 地址。PTX 允许 x2 时 lane 16–31 传「复制低位」的地址；但传**无效值**（未映射地址）在部分架构上仍会出错。本 kernel 的 `lane%8` / `(lane/8)%2` 天然循环复用合法行号，属安全写法；自己写 kernel 时建议保持这种「地址周期化」的习惯。

坑四：`wait_group` 之后漏 `__syncthreads`。与第 十一 章坑四同理但更容易漏：统一循环里 wait 在迭代末，读数据在迭代头，中间隔着 `__syncthreads`（L303）。删掉它，warp 0 可能已经读到 warp 7 还没写完的 stage。另外注意 `if (k + kStages - 1 < NUM_K_TILES)` 的条件加载分支：**wait_group 在分支外**（L297–304 自适应版本），而 `commit_group` 在分支内。尾部迭代不 commit 却 wait(0)，per-thread 组计数对不上会提前放行（好在上一次的 sync

兜底；本 kernel 尾部迭代 `wait_group(0)` 等到的是「至多 0 组」，语义仍安全）。改造时保持 commit/wait 匹配的对称性。

坑五：f16 累加的精度陡降。 `f16.f16.f16.f16` 的累加「至少半精度」：每条 mma 边界舍回 f16。K=128 实测 1.5×10^{-2}；K 到千级、或输入幅值大于 1 时，误差进入 10^{-1} 量级并在个别元素上溢出 f16 范围（65504）。长 K / 大动态范围请换 `HMMA16816F32`（C 用 4 个 f32 寄存器）。

坑六：改完模板参数忘了改 launch 侧。 $B_M/B_N/B_K$ 全由模板参数推导，但 smem 字节数（`kStages`$\times(B_MB_K + B_NB_K) \times 2$）与 grid/blockDim 由**调用方**计算。改 `kStages` 或 tile 参数而不同步 launch 参数，轻则 launch 直接报 smem 超限错误，重则静默算错（blockDim 与 warp 排布假设不符）。本章测试把 `smem_bytes` 与 blockDim 都写死为 24576/256，正是要与模板默认值形成显式互检。

12.8 面试要点速查

Q1：`mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16` 每个字段什么意思？

A：见表 12.1。要点：$m16n8k16$ 是 A16 × 16、B16 × 8、C16 × 8；`.row/.col` 是 A/B 的 fragment 主序；四个类型依次 D/A/B/C；一条 4096 FLOP；`.sync/.aligned` 要求全 warp 同步执行同一条指令。

Q2：A/B/C fragment 每个 lane 持哪些元素？

A：A：每 lane 8 个 f16、四个象限寄存器（行 g/g+8 × 列 2tig/2tig+8）；B：每 lane 4 个、按列（列 g、行 2tig/+8）；C：每 lane 4 个、两行各一对（式 12.1–12.3）。口诀：g = lane ≫ 2 管行/列块，tig = lane mod 4 管 2 元素对。

Q3：`ldmatrix` 的寻址协议？为什么它的装载结果直接匹配 mma？

A：每矩阵 8 行 → 8 个 lane 各供一行 16 B 首地址；x4 全 warp 4 矩阵、x2 前 16 lane 2 矩阵。装载结果直接落在 mma fragment 的期望分布上，一条指令替代 32 lane 各自的 `ld.shared` 寻址，且天然规避多数 bank conflict。`.trans` 才是列装载（见 11.6 勘误）。

Q4：TN 布局为什么是 Tensor Core GEMM 的默认选择？

A：A 行主序、$B^\top$ 行主序 ⇒ 内维都是 K：gmem→smem 搬运两侧对称且 16 B 连续；$B^\top$ 的行 = B 的列，非 trans 的 `ldmatrix` 直接给出 `.col` 的 B fragment；mma `row.col` 与 TN 天然对齐，一条 `.trans` 都不用。

Q5：kStages 越大越好吗？

A：不是。深流水容忍更长的搬运延迟（稳态 `wait_group(kStages-2)`），但 smem 随 S 线性增长（本章 24 KB@3 级）、占用 limit occupancy 与后续 swizzle/padding 的空间预算；且 S 超过「搬运延迟/迭代时间」后收益归零。$S \in \{2, 3, 4\}$ 是常见区间，靠 profile 定。

12.9 最小测试与动手实验

测试文件：`book/tests/ch11_hgemm_mma.cu`（逻辑抽取自 bench/hgemm.cu 的 `test_hgemm_mma`，L40；参考改为 CPU fp64：half 输入转 double 精确累加，half 输出 `__half2float` 后比较）。case A（$128 \times 128 \times 128$，grid 1×1，K 为 8 个 k-tile，覆盖满载 + 尾部排空）、case B（$256 \times 128 \times 64$，grid $(1, 2)$，M 方向多 block、K 非 kStages 倍数）；容差 F16Acc 档 `TOL_F16ACC` $= 5 \times 10^{-2}$。$B^\top$ 在 host 构造（坑一的根源演示）。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --arch sm_120a --ch ch11`。本机运行输出（RTX PRO 5000，2026-09-11；PASS 行的误差数值见表 12.4）：

```
case A: M=128 N=128 K=128
case B: M=256 N=128 K=64
ALL OK
```

两个 case 的误差量级（10^{-2}）远大于第 十一 章 TF32 版的 10^{-5}，这正是 f16 累加（11.3.1 语义（2））的实测特征：fp64 参考没有半精度舍入，差异全部来自每条 mma 边界的 f16 舍回。动手实验建议：（1）把 `HMMA16816` 换成 `HMMA16816F32`（C 寄存器改 4 个、epilogue 相应调整），观察误差跌回 10^{-3} 以下；（2）故意把 B（而非 $B^\top$）传给 kernel，复现坑一的静默错误；（3）用 `./.build/ch11_hgemm_mma bench` 复现 76.55 TFLOPS，再把 kStages 模板参数改成 2/4 对比 smem 与吞吐的取舍。

表 12.4: ch11 测试 PASS 行数值摘录（完整行式输出见 `tests/.build/ch11_hgemm_mma` 运行日志；tol 为判定容差）

case	判定项	最大绝对误差	tol
A	hgemm_mma $128 \times 128 \times 128$	1.469e-02	5.0e-02
B	hgemm_mma $256 \times 128 \times 64$	7.919e-03	5.0e-02

延伸阅读

- @ 木子知《CUDA 编程：MMA PTX 编程入门》, https://zhuanlan.zhihu.com/p/621855199（`mma.sync` 与 fragment 布局的入门级推导，本章 11.3 节的互证材料）；
- @Anonymous《GEMM 优化细节 (一)：ldmatrix 的选择》, https://zhuanlan.zhihu.com/p/702818267（x4/x2/x2_trans 的选型逻辑与 TN/NN 布局的配合，对应本章 11.2 与 11.4.4）；
- NVIDIA. Parallel Thread Execution ISA, §9.7.14.5 （`mma`/`ldmatrix` 官方语义与 Figure 79/81/83；本地副本：cuda-cpp-kernel skill `references/ptx-docs`）。

第十三章　HGEMM：XOR Swizzle、寄存器双缓冲与 Block Swizzle

13.1　本章导读

第 十二 章把 HGEMM 推进到了 `mma.sync` m16n8k16 + `ldmatrix` + cp.async 多级流水的形态，但那条流水线上还存在三个瓶颈：**shared memory bank conflict** 让 `ldmatrix` 的读取被串行化；**k_step 内 ldmatrix 与 MMA 的依赖**让数据搬运与数值计算无法重叠；**grid 的光栅化顺序**让同时在飞（in-flight）的 block 在 L2 里摊成一条宽条带、复用率低下。本章沿教学源码 Phase 7b-3 的 `hgemm_mma_stages_tn_swizzle`（hgemm.cuh L434）逐一解决这三个瓶颈：

- **smem XOR swizzle**（common.cuh L110–348）：不花一个字节的 padding，用地址位异或把 ldmatrix 的冲突访问打散；本章给出逐 bit 推导、可逆性证明，以及 v1（手写 XOR 分支）与 v2（`Swizzle<B,M,S>` 位级镜像）两套实现；
- **寄存器双缓冲**（`RA[2][*][4]/RB[2][*][2]` 乒乓切换）：与 smem 双缓冲（kStages）分工，一个掩盖 gmem 延迟，一个掩盖 ldmatrix 延迟；
- **Block Swizzle**（`kBlockSwizzle` + 3D grid）：重排 block 坐标映射，让同时在飞（in-flight）的 block 集合成近似方形，L2 footprint 显著缩小。

前置章节：第 十二 章（mma fragment 布局、ldmatrix 语义、cp.async 流水线）；第 十 章的五层金字塔分块。**源码区间**：hgemm.cuh L388–716 （派发器 L388、kernel L434）与 common.cuh L110–348（`permuted`/`swizzle_v1_impl`/`SwizzleBMS`/`swizzle_v2_impl`）。**编译宏**：`NOTES_V2_ENABLE_SWIZZLE_V2`（默认关闭走 v1，打开走 v2，两者 bit-exact 等价）；kernel 本体需 sm_80+，无其它架构限定。配套最小测试 `book/tests/ch12_hgemm_swizzle.cu`。**后继**：第 二十四 章把本章手写的 XOR swizzle 接入 CuTe 的 `Swizzle<B,M,S>` 布局代数，第 二十五 章的 CuTe HGEMM 将与本章手写版逐项对照。

13.2　问题与动机

HGEMM 是 compute-bound 算子：$2MNK$ 次乘加对应 $O(MN+MK+NK)$ 的访存，算术强度极高。此时优化的全部要义只有一条：**不让 Tensor Core 空等数据**。第 十二 章的多级流水已经把 gmem→smem 的一段用 cp.async 双缓冲掩盖了，但数据链路还有两段、调度还有一层，各有一个具体的瓶颈：

瓶颈一：ldmatrix 的 bank conflict（S→R 段）。MMA 的输入数据必须先由 `ldmatrix` 从 shared memory 搬进寄存器。smem 的 32 个 bank 每周期只能各出 4 B，一个 warp 的 `ldmatrix.x4` 要读 32×16 B；若 8 个 lane 的地址落在同一组 bank 上，硬件只能串行重放（n-way conflict = 同一请求拆成 n 次传递）。而不做任何处理时，`[BM][BK]` row-major 的 smem tile 恰好是 ldmatrix 按行取数的**最坏布局**（12.3 节逐步推导）。传统解法是给行加 padding 打散 stride，代价是浪费 smem、且破坏 16 B 对齐粒度；XOR swizzle 用「地址位异或」零成本解决，这正是本章的核心技术。

瓶颈二：k_step 内 ldmatrix 与 MMA 的串行（R→ 计算段）。一个 BK tile 内含多个 k_{step} slice；若只有一套 fragment 寄存器，每个 slice 都要「ldmatrix 等 smem → MMA 等寄存器」顺序执行，lsu 与 tensor core 交替空转。寄存器双缓冲让「加载 slice t+1」与「用 slice t 计算」同周期进行：它是 smem 双缓冲（掩盖 gmem 延迟）之下的第二级流水，两者分工明确。

瓶颈三：grid 光栅化把 L2 复用摊薄（调度层）。CUDA block 的分发顺序是 x 最快、y 次之。C tile 网格 $T_m \times T_n$ 若按默认光栅铺开，同时在飞（in-flight）的 R 个 block（$R \approx$ SM 数 × 每 SM 常驻 block 数）呈 $T_n \times \lceil R/T_n \rceil$ 的**宽条带**：A tile 复用好（行少），B tile 却要 T_n 份同时驻留 L2，footprint 巨大。Block swizzle 用 3D grid 重排坐标，把条带折成近似方块，footprint 从 $(H{\cdot}B_M + W{\cdot}B_N){\cdot}K$ 里的「大 W」压成 $\surd$ 级；12.3.8 节推导，实测 L2 命中率 64.45%→96.15%。

13.3　数学原理

13.3.1　bank 模型：32 bank、4 B 宽、128 B 一行

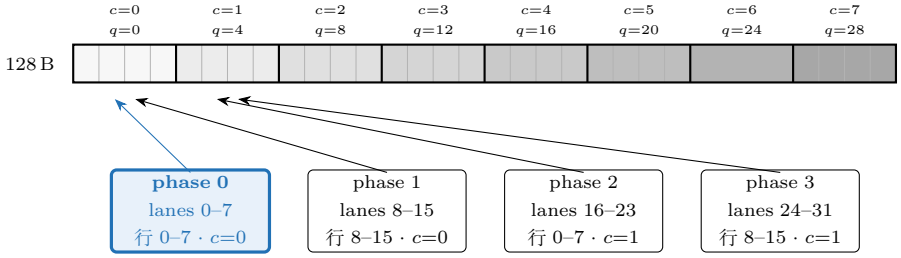
现代 GPU（自 Fermi 以来的各代）的 smem 均由 32 个 bank 组成，每 bank 宽 4 B，一个 bank 周期全体最多交付 $32 \times 4 = 128$ B。字节地址 a 所属 bank 为

$$\mathrm{bank}(a) \;=\; \left\lfloor \tfrac{a}{4} \right\rfloor \bmod 32. \tag{13.1}$$

一个 16 B 数据块（本章的所有访问都以 16 B chunk 为粒度，即 half 的 8 元素，恰是 ldmatrix 一行、cp.async.cg 一次搬运的宽度）占据**连续 4 个 bank**（一个「quad」），下文记其**起始 bank** 编号为

$$q(a) \;=\; 4\left\lfloor \tfrac{a}{16} \right\rfloor \bmod 32, \qquad q \in \{0, 4, \ldots, 28\}\,(\text{占 bank } q, \ldots, q{+}3)\ . \tag{13.2}$$

硬件按 **phase** 处理 128-bit 向量访问：warp 的 32 lane 分成 4 组、每组 8 lane 依次服务，一组 8 lane × 16 B = 128 B 恰好用满全部 bank。**同一 phase 内两 lane 落同一 bank 的不同 word 即冲突**：n-way 冲突把该 phase 拆成 n 个 bank 周期。`ldmatrix.x4` 的 4 个 8×8 矩阵正对应 4 个 phase（PTX 语义：每 8 个连续 lane 提供一个矩阵 8 行的地址）。



lane→ 地址映射（hgemm.cuh L526–528）：row = lane mod 16、$c = \lfloor \text{lane}/16 \rfloor$（$c$=0 取列 0–7，$c$=1 取列 8–15）；同一 phase 内 8 lane × 16 B = 128 B 恰好用满 32 bank，仅当两 lane 落进同一 quad 的不同 word 才冲突（n-way）；这组 8 行各自落进哪个 quad 由 smem 行宽决定：BK=16（32 B 行）2-way、BK=64（128 B 行）8-way（12.3.2/12.3.3 节推导）

图 13.1: bank/phase 硬件模型（12.3.1 节）：32 bank × 4 B 每周期最多交付 128 B，任一连续 128 B 地址窗切成 8 个 16 B chunk，chunk c 占一个 quad（起始 bank $q = 4\lfloor a/16 \rfloor \bmod 32$，同 quad 同灰度）。warp 的 32 lane 按 phase 分 4 组依次服务，`ldmatrix.x4` 的 4 个 8×8 矩阵正对应 4 个 phase：phase p 由 lanes $8p \ldots 8p{+}7$ 按 row = lane mod 16、$c = \lfloor \text{lane}/16 \rfloor$ 提供地址（高亮 phase 0 = lanes 0–7 读行 0–7 的 chunk 0；phase 0/1 同读 chunk 0，phase 2/3 同读 chunk 1）。phase 内是否冲突取决于该组 8 行落进的 quad 分布——12.3.2/12.3.3 节按行宽推导 2-way/8-way、以及图 13.2 的三档 swizzle 对比，都基于这个机制。

本 kernel 中 A fragment 的 lane→ 地址映射为（hgemm.cuh L526–528）：row = lane mod 16，chunk $c = \lfloor \text{lane}/16 \rfloor$（$c = 0$ 取列 0–7，$c = 1$ 取列 8–15）。于是 phase 0 = lanes 0–7 = 行 0–7、chunk 0。下面按两种 smem 行宽分别推导冲突度。

13.3.2　无 swizzle 的冲突推导：BK=16（32 B 行宽）为 2-way

Phase 7b-1 的 `hgemm_mma_stages_tn`（第 十二 章）中 $B_K = 16$，smem tile 为 [128][16] half，行宽 32 B。行 i chunk c 的字节地址 $a = 32i + 16c$，四联组

$$q(i, c) \;=\; 4\left\lfloor \tfrac{32i+16c}{16} \right\rfloor \bmod 32 \;=\; (8i + 4c) \bmod 32. \tag{13.3}$$

逐行展开（$c=0$）：$i=0,1,2,3 \to q=0,8,16,24$；$i=4,5,6,7 \to q=(32,40,48,56) \bmod 32=0,8,16,24$。**行 4–7 与行 0–3 两两同组**，每个 phase 2-way 冲突（$c=1$ 同理，$q=4,12,20,28$ 重复一遍）。这个数字可以做**定量校验**：一次 `ldmatrix.x4` 有 4 个 phase、每 phase 多 1 次重放，即每条 x4 多 4 个冲突周期；`x2` 多 2 个。每个 warp 每个 k 迭代搬 4 个 A fragment（x4）与 4 个 B fragment（x2），共多 $4\times4+4\times2=24$ 次重放。NCU 实测（RTX PRO 5000，M=N=K=512，16 block × 8 warp × 32 个 k 迭代）：

$$128\times32\times24 \;=\; 98{,}304 \;=\; \text{实测 } \texttt{shared_op_ld} \text{ 冲突计数}, \tag{13.4}$$

phase 模型与硬件计数完全对齐，后续推导都建立在这套校准过的模型上。

13.3.3 无 swizzle 的冲突推导：BK=64（128 B 行宽）为 8-way

Phase 7b-3 把 4 个 k_{step} slice 统一进 [128][64] 的宽 tile，行宽 64 half = 128 B。行 i、slice s、chunk c 的地址 $a=128i+32s+16c$：

$$q(i,s,c) \;=\; 4\left\lfloor\tfrac{128i+32s+16c}{16}\right\rfloor \bmod 32 \;=\; (8s+4c) \bmod 32. \tag{13.5}$$

注意**行号 i 的贡献消失了**：bank 计数环 128 B（32 bank × 4 B）、四联组编号环同为 128 B（$\lfloor a/16\rfloor \bmod 8$，周期 8 个 chunk）；行 stride 128 B 恰为两者各一圈，任何一行的同一 (slice, chunk) 都映射到同一个起始 bank q。phase 0 的 8 个 lane（行 0–7、c 固定）全部落在 $q=8s+4c$ 上：**8-way 冲突**。NCU 实测同形状下 691,067 次（把 kernel 中 8 处 swizzle 调用置 0 的对照版，见 12.6 节表 13.1）。统一宽 tile 省了同步与寻址开销，却把 ldmatrix 的冲突度放大了 4 倍，必须引入 swizzle 才行。

13.3.4 XOR swizzle：v1 的位运算与逐 bit 解释

XOR swizzle 的想法：**在写入 smem 时就把 chunk 的物理位置按行号置换，让「按列读」的 lanes 天然散开**。对 16 列（两个 chunk）的 tile，v1 的置换是（common.cuh L116、L176）：

$$c' \;=\; c\oplus\left(\left\lfloor\tfrac{i}{4}\right\rfloor \bmod 2\right), \qquad \text{即}\quad \texttt{swizzle<16>(i,j)} = \Big(\big((j\gg3)\oplus(i\gg2)\big)\;\&\;1\Big)\ll3. \tag{13.6}$$

逐 bit 看（图 13.6）：字节地址 $a=32i+16c+w$（$w\in[0,16)$ 为 chunk 内偏移），则 `bit4(a)`$=c$（chunk 选择位）、`bit5+`$=i$（行）；而 $\lfloor i/4\rfloor \bmod 2$ 恰是 `bit7(a)`（$32i$ 把行号左移到 bit5 起，bit7 即 i 的 bit2）。所以式 13.6 的净效果是

$$\texttt{bit4}(a') \;=\; \texttt{bit4}(a)\oplus\texttt{bit7}(a) \;:\quad \text{每 4 行一组，用行号 bit2 翻转 chunk 位.} \tag{13.7}$$

代入式 13.3（BK=16）：行 4–7 的 chunk 0 被搬去 $q=4$,12,20,28，chunk 1 搬去 $q=0$,8,16,24，phase 0 的 8 行四联组变为 $\{0,8,16,24,4,12,20,28\}$，**8 个互不相同，恰好铺满 32 个 bank：1-way，零冲突**。源码注释「原来 n-way 的 bank conflict 降低到 1-way」说的正是这个 32 B 行宽布局（common.cuh L185–235 的 bank 分布表即此布局，也是图 13.4 的蓝本）。

13.3.5 可逆性：swizzle 是对合，写读同函数即互逆

引理 13.1 (XOR swizzle 的可逆性)**.** *设 $\sigma(a)=a\oplus m(a)$，其中 $m(a)=((a\;\&\;M_{\text{hi}})\gg S)$ 且 XOR 的目标位域与源位域不相交（源取自 bit $[M{+}S,\,M{+}S{+}B)$，写到 bit $[M,\,M{+}B)$，$S\ge B>0$）。则 $\sigma(\sigma(a))=a$。*

证明. 记 $h=a\;\&\;M_{\text{hi}}$（源位域的值）。第一次作用后 $a'=a\oplus(h\gg S)$，只改动了 bit $[M,M{+}B)$。再算第二次的源：$a'\;\&\;M_{\text{hi}}$，由于目标位域 $[M,M{+}B)$ 与 M_{hi} 覆盖的 $[M{+}S,M{+}S{+}B)$ 在 $S\ge B$ 时不相交，a' 在源位域上的取值仍是 h，于是 $m(a')=h\gg S=m(a)$，第二次异或同一个值与第一次抵消：$\sigma(a')=a\oplus(h\gg S)\oplus(h\gg S)=a$。 □

推论（工程上最重要的一条）：**编码与解码用同一个函数**。cp.async 写入 smem 时用 `swizzle`(i,j) 计算物理列，ldmatrix 读出时用同一个 `swizzle`(i,j) 再算一遍物理列：两次互逆，逻辑坐标无损还原，不需要维护任何「逆置换表」。对 12.3.4 节的 v1：源位 bit7、目标位 bit4，S=3 ≥ B=1，不相交，对合成立；对 v2 的 (B,M,S)=(2,4,3)/(3,4,3) 同理（bit [7,9) 对 [4,6)、bit [7,10) 对 [4,7)，均不相交）。

13.3.6　Swizzle$\langle B, M, S\rangle$ 一般形式与 v1/v2 等价

CuTe 把这类 bit 级置换统一为 Swizzle$\langle B, M, S\rangle$（记号沿用全书记号表）：把一维 byte offset 重新解释成二维逻辑空间，再做列置换：2^M 个连续元素为一个基本元素（fp16 的 16 B chunk ⇒ M=4），每行 2^S 个基本元素（8 chunk ⇒ S=3），共 2^B 行参与置换。位级公式（common.cuh L271–273，与 cutlass `swizzle.hpp` 一致）：

$$\text{apply}(a) \;=\; a \oplus \big((a \;\&\; yyy) \gg S\big), \qquad yyy = \big(2^B{-}1\big) \ll (M{+}S), \tag{13.8}$$

即「把 bit $[M{+}S,\, M{+}S{+}B)$（行号低 B 位）异或到 bit $[M,\, M{+}B)$（chunk 号）」。三个常用档位与 TMA 硬件 swizzle 模式同名：

$$(B, M, S) = (1, 4, 3) = \text{SWIZZLE_32B}, \quad (2, 4, 3) = \text{SWIZZLE_64B}, \quad (3, 4, 3) = \text{SWIZZLE_128B}, \tag{13.9}$$

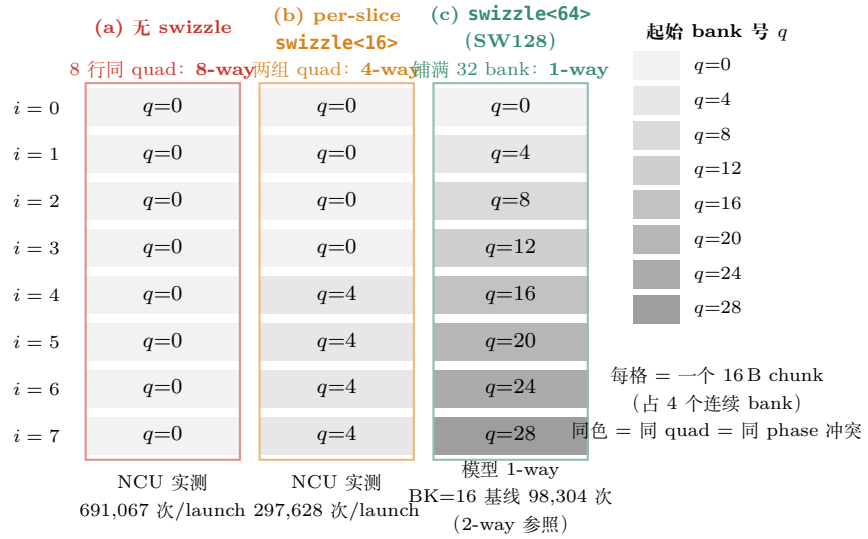
分别对应 32/64/128 B 行宽（kColStride = 16/32/64 half）。v1 的三个手写分支与 v2 的统一公式**逐档等价**，独立核验如下（$a = 2(k_{\text{cs}}\, i + j)$，$j < k_{\text{cs}}$）：$k_{\text{cs}}$=16 时 $a = 32i + 2j$，bit4 恰为 $(j{\gg}3)$ & 1（$2j < 32$ 不上探），bit7 恰为 $(i{\gg}2)$ & 1，apply 即式 13.6；k_{cs}=32 时 $a = 64i + 2j$，bit $[4, 6)$ 为 $(j{\gg}3)$ & 3、bit $[7, 9)$ 为 i 的 bit $[1, 3)$，与 v1 的 `chunk^xor_mask` 逐位相同；k_{cs}=64 同理（bit $[4, 7)$ 对 bit $[7, 10)$）。源码注释 common.cuh L320–325 的等价性证明与上述独立推导一致，且 host 端全遍历测试（$i < 256$、$j < k_{\text{cs}}$、四档共 30,720 点）bit-exact 通过（本章 case C）。k_{cs}=8 无对应 TMA 模式，v2 回退 v1（当前无 kernel 使用）。

13.3.7　关键勘误：BK=64 下 per-slice swizzle 只到 4-way，不是 1-way

以上「1-way 全消」的结论有个隐含前提：**swizzle 的作用域行宽** $= k_{\text{cs}}$（32 B 行、chunk 位 bit4 直接被行号 bit 翻转）。而 Phase 7b-3 kernel 的统一 tile 是 [128][64]、行宽 128 B，调用的是 `swizzle` $< k_{\text{MmaK}}$=16 >，把 XOR 限制在每个 16 列的 slice **内部**（`swizzle`(i, j) 只在 $\{0, 8\}$ 间搬 chunk，slice 基址 $32s$ 另行相加）。代入式 13.5：

$$q\big(i, s, c'\big) = \big(8s + 4\,(c \oplus (\lfloor i/4 \rfloor \bmod 2))\big) \bmod 32. \tag{13.10}$$

phase 0（行 0–7，c 固定）：行 0–3 落 $q = 8s$、行 4–7 落 $q = 8s{+}4$，每四联组 4 个 lane，**4-way 冲突**（无 swizzle 是 8-way：全部 8 行挤在 $q = 8s + 4c$）。行号更高的 bit 乘上行 stride 128 B 后落在 bit7 以上，无法进入 bank 位 $[2, 7)$，**单个 XOR bit 最多把 phase 内 8 lane 分到 2 个四联组，4-way 是该方案在这类布局下的下界**。NCU 三点实测（同形状 512^3，表 13.1）：swizzle 版 297,628 次冲突、对照无 swizzle 版 691,067 次、BK=16 基线 98,304 次；两版比值 691067/297628 = 2.32，与模型预测的 8-way→4-way 比值（688128/294912 = 2.33，即 BK=16 基线 98,304 的 7 倍与 3 倍；实测与模型误差 <1%）一致。要在 128 B 行宽下真正 1-way，应使用 `swizzle<64>`（SWIZZLE_128B，B=3）：$c' = c \oplus (i \;\&\; 7)$，行 0–7 的 8 个 chunk 恰好铺满 $q = 0, 4, \ldots, 28$；common.cuh 的 `permuted<64>` 已实现该档位，CuTe 版 HGEMM（第 二十五 章）使用的 `Swizzle<3,3,3>` 正是同一思路；本 kernel 选择 per-slice 的 16 档位，冲突从 8-way 降到 4-way 后，剩余延迟由寄存器双缓冲的流水覆盖（表 13.1 耗时列）。



$q(i,s,c) = (8s + 4c) \bmod 32$（式 13.5）；phase 0 = lanes 0–7 = 行 i=0–7、s=0、chunk c 固定；(b) 行 0–3 落 q=8s、行 4–7 落 q=8s+4；(c) $c' = c \oplus (i \,\&\, 7)$，行 0–7 铺满 $q = 0, 4, \ldots, 28$。

图 13.2: BK=64（128 B 行宽）下三档 swizzle 的 phase-0 quad 映射对比（12.3.7 节）：固定 s=0、c=0，逐行给出 ldmatrix 读到的起始 bank 号 q，灰度按 q 着色——同色格即同一 quad、即同一 phase 内冲突。(a) 无 swizzle 时 $q = 8s + 4c$ 与行号无关，8 行落在同一 quad，8-way 冲突（NCU 实测 691,067 次/launch）；(b) per-slice `swizzle<16>` 只拿行号 bit2 翻转 chunk 位，行 0–3 与行 4–7 分落两个 quad，降到 4-way（297,628 次）——单个 XOR bit 的下界；(c) `swizzle<64>`（SWIZZLE_128B，B=3）以 $c' = c \oplus (i \,\&\, 7)$ 让 8 行恰好铺满 32 个 bank，1-way（BK=16 基线 98,304 次为 2-way 参照）。数字见表 13.1。

13.3.8 Block Swizzle：坐标重映射与 L2 footprint 推导

默认 grid 的两个维度就是 T_n 与 T_m（x 维即 N-tile 编号）。Block swizzle 改用 3D grid 并在 kernel 头部做一次线性化（hgemm.cuh L440）：

$$bx \;=\; k_{\mathrm{BS}} \cdot \mathrm{blockIdx.z} \cdot \mathrm{gridDim.x} + \mathrm{blockIdx.x}, \qquad by = \mathrm{blockIdx.y}, \tag{13.11}$$

$$\mathrm{grid} = \big(\lceil T_n/S \rceil,\; T_m,\; S\big), \qquad S = \lceil N/2048 \rceil = \lceil T_n/16 \rceil, \tag{13.12}$$

即把 T_n 个 N-tile 顺序切成 S 组、每组 $\lceil T_n/S \rceil \approx 16$ 个，组号记录在 blockIdx.z 里。k_{BS}=0 时式 13.11 退化为 bx = blockIdx.x，与默认完全一致，开关零成本。注意 kernel 只使用「$z \cdot \mathrm{gridDim.x} + x$」这个线性式，$S$ 取多少组由**启动方**决定（源码注释的 S=$\lceil N/2048 \rceil$ 是推荐值：每组 16 个 tile $\approx \sqrt{R}$，R 为常驻 block 数）。

为什么重排能提升 L2 命中？设同时在飞（in-flight）的 block 数为 R（PRO 5000：110 SM、smem/SM 100 KB 对 64 KB 动态 smem 只容 1 块常驻，$R \approx 110$）。每个 C tile block 要读 $B_M \times K$ 的 A 条带与 $B_N \times K$ 的 B 条带；常驻集合覆盖 H 个 M-tile 行、W 个 N-tile 列时，L2 必须同时容纳的 unique 数据量（footprint）为

$$F \;=\; \big(H \cdot B_M \;+\; W \cdot B_N\big) \cdot K \cdot 2\,\mathrm{B}, \tag{13.13}$$

而常驻集合的计算量固定为 $R \cdot B_M B_N K$，footprint 越小、命中越高。默认光栅（x 最快）：$W = \min(T_n, R)$、$H = \lceil R/W \rceil$。取 T_m=16、T_n=128（M=2048、N=16384）：W=110、H=1，$F = (1{\cdot}128 + 110{\cdot}128)K \cdot 2\,\mathrm{B} = 14208K \cdot 2\,\mathrm{B}$。开启 swizzle 后 $W' = \lceil T_n/S \rceil = 16$、$H' = \lceil R/16 \rceil = 7$，$F' = (7{\cdot}128 + 16{\cdot}128)K \cdot 2\,\mathrm{B} = 2944K \cdot 2\,\mathrm{B}$，**缩小 4.8 倍**：条带被折成了近似方块（$\sqrt{R} \times \sqrt{R}$ 量级）。实测（同形状）：L2 sector 命中率 64.45%→96.15%，端到端 117.9→122.9 TFLOPS（+4.2%）。形状越「宽」（$T_n \gg T_m$）收益越大；$T_n \le \lceil R/T_m \rceil$ 时条带本就近似方形，收益趋零（4096^3 实测仅 +0.3%）。

13.4 设计与实现

13.4.1 swizzle 函数族：v1、v2 与派发器

先看动机。Phase 7b-3 的 kernel 是 `m16n8k16` + multistage pipeline + TN 布局 + XOR swizzle 消除 smem bank conflict 的统一循环版，其头部注释把 swizzle 原理（smem Bank Conflict 消除）总结为五点：

- **问题**：`ldmatrix` 以 8×8 片段（m8n8）从 smem 加载 16×16 的矩阵，同一 warp 中不同 lane 访问的地址在 bank 上产生冲突 → 串行化（most common: 2/4-way）。
- **XOR swizzle 方案**：对列地址做 `((j>>3)^(i>>2))%2<<3` 的 XOR 置换，将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替：rows 0∼3：列 0 → 地址偏移 0、列 8 → 地址偏移 8；rows 4∼7：列 0 → 地址偏移 8、列 8 → 地址偏移 0（XOR 翻转）；rows 8∼11 重复 rows 0∼3 的 pattern，rows 12∼15 重复 rows 4∼7 的 pattern。
- **效果**：原来 n-way 的 bank conflict 降低到 1-way（fully conflict-free）。
- **优势**：无需 smem PAD（不浪费空间），原理上完全消除特定 pattern 的 bank conflict。
- **局限**：要求 kColStride ≤ 16（即 BK ≤ 16）、kStep $\in \{4, 8\}$；对 MMA `m16n8k16` 的 BK= 16 而言刚好满足。

（参考实现 `hgemm_mma_stage_tn_swizzle.cu`；原理详解见 https://zhuanlan.zhihu.com/p/4746910252。）

common.cuh L158-183（permuted：v1 手写 XOR 分支，四档位展开）

```
template <const int kColStride = 16, const int kStep = 8>
static __host__ __device__ __forceinline__ int permuted(int i, int j) {
  static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64 ||
                kColStride == 8,
                "kColStride must be one of {8, 16, 32, 64} (matches "
                "SWIZZLE_32B/64B/128B + the "
                "kStep=4 narrow legacy case).");
  static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
  static_assert(kColStride % kStep == 0,
                "kColStride must be multiple of kStep.");
  if constexpr (kStep == 4) {
    static_assert(kColStride <= 16, "kStep=4 only supports (kColStride <= 16).");
    return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
  } else if constexpr (kColStride == 16) {
    return (((j >> 3) ^ (i >> 2)) & 1) << 3;  // SWIZZLE_32B
  } else if constexpr (kColStride == 32) {
    const int chunk = (j >> 3) & 3;
    const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
    return (chunk ^ xor_mask) << 3;  // SWIZZLE_64B
  } else {                           // kColStride == 64
    const int chunk = (j >> 3) & 7;
    const int xor_mask =
        (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);
    return (chunk ^ xor_mask) << 3;  // SWIZZLE_128B
  }
}
```

v1 按 k_{cs} 模板分支：16 档位即式 13.6；32/64 档位把 2/3 个行号位与 chunk 多位异或（SWIZZLE_64B/128B）；8 档位（kStep=4 legacy）只服务旧 FA 路径。每档返回的都是 **chunk 级列偏移**（0/8/16/…），chunk 内的偏移由调用方保留，「地址 = 行基址 + slice 基址 + swizzle + chunk 内偏移」的拼装式（hgemm.cuh L497–500）靠的就是这条约定。

v2 的位级实现与 `cutlass/include/cute/swizzle.hpp` 一致。byte offset 的 bit 布局为 `0bxxxxxxxxxxxxxxxYYYxxxxxxxZZZxxxx`（自左向右高位 → 低位）：低 M 位（MBase）是基本元素内部偏移，保留、不参与置换；yyy 与 zzz 两段各占 B 位（BBits，XOR 掩码位数）；yyy 相对 zzz 的位移为 SShift。三条公式：

- `apply(off) = off ^ shiftr(off & yyy_msk, S)`；
- `yyy_msk = ((1<<B)-1) << (M + max(0,S))`　（yyy 掩码位置）；
- `zzz_msk = ((1<<B)-1) << (M - min(0,S))`　（zzz 掩码位置，文档用）。

模板参数的物理/语义含义：

- **B（BBits）**：二维逻辑空间的行数位 $= 2^B$ 行，即 swizzle 周期覆盖 2^B 个 chunk（这些 chunk 互相置换）；同时是 TMA 硬件 swizzle 模式 `SWIZZLE_{32,64,128}B` 的位数（$B = 1/2/3$）。
- **M（MBase）**：基本元素宽度位 $= 2^M$ 个连续元素组成一个基本元素。对 fp16 + 8-element chunk，基本元素 $= 16\,\mathrm{B} = 2^4$，故 $M = 4$；地址 bit[0..$M-1$] 在 apply 中保持不变（保留）。
- **S（SShift）**：二维空间每行列数位 $= 2^S$ 个基本元素组成一行；同时是 *yyy* 掩码相对 *zzz* 掩码的位移。$S > 0$ 表示 *yyy* 在更高位，向右移 S 位到 *zzz* 位置后 XOR。本 common.cuh 三模式 S 恒为 3（*yyy* 在 bit[7..7+$B-1$]，即行索引 i 的低 B 位；*zzz* 在 bit[4..4+$B-1$]，即列 chunk 索引）。

完整 swizzle 周期覆盖 2^{M+S+B} 个元素（fp16 下 $2^{M+S+B+1}$ 字节）：$(B, M, S) = (1, 4, 3)$：256 elem / 512B（SWIZZLE_32B）；$(2, 4, 3)$：512 elem / 1024B（SWIZZLE_64B）；$(3, 4, 3)$：1024 elem / 2048B（SWIZZLE_128B）。

common.cuh L288-316（SwizzleBMS：cute::Swizzle 位级镜像，*yyy* 掩码 + apply）

```cpp
struct SwizzleBMS {
  static_assert(M >= 0, "MBase must be non-negative.");
  static_assert(B > 0, "BBits must be positive.");
  static_assert((S > 0 ? S : -S) >= B,
                "abs(SShift) must be >= BBits (cute::Swizzle constraint).");

  static constexpr int bit_msk = (1 << B) - 1;
  static constexpr int yyy_msk = bit_msk << (M + (S > 0 ? S : 0));
  static constexpr int zzz_msk = bit_msk << (M - (S < 0 ? S : 0));

  // apply: 对 byte offset 做 XOR 位置换, ZZZ ^= (YYY >> S).
  static __host__ __device__ __forceinline__ int apply(int offset) {
    if constexpr (S >= 0) {
      return offset ^ ((offset & yyy_msk) >> S);
    } else {
      return offset ^ ((offset & yyy_msk) << (-S));
    }
  }
};

// swizzle_v2_impl: 用 SwizzleBMS<B,M,S>::apply 计算 chunk 级列 swizzle.
//
// 流程: (i,j) -> byte offset off=(i*kColStride+j)*sizeof(half) -> SwizzleBMS
//       位级置换 -> 取 [M, M+B) 位作为物理 chunk 索引 -> 乘 kStep 还原为列偏移.
//
// kColStride -> (B,M,S) 映射表 (对应 TMA 硬件 swizzle 模式):
//   kColStride=16 fp16=32B/row  -> (B,M,S)=(1,4,3) = SWIZZLE_32B
//   kColStride=32 fp16=64B/row  -> (B,M,S)=(2,4,3) = SWIZZLE_64B
//   kColStride=64 fp16=128B/row -> (B,M,S)=(3,4,3) = SWIZZLE_128B
```

common.cuh L318-342（swizzle_v2_impl：offset → apply → 取 $[M, M+B)$ 位还原列偏移）

```cpp
// kColStride=8 (kStep=4 legacy): 无对应 cute 标准 TMA swizzle 模式, 回退到
//   swizzle_v1_impl. 该路径当前无任何 kernel 使用, 回退保证接口契约零变更.
//
// 等价性: 对 (16,32,64), v2 输出与 v1 bit-exact 相同 (见 test_swizzle_equiv).
//   证明: off 的 bit[4..4+B-1] 恰是 (j>>3) 低 B 位 (因 i*kColStride 在这些位
//   为 0); bit[7..7+B-1] 恰是 i 低 B 位 (因 j<kColStride 不贡献高位). apply
//   把 YYY(bit[7..]) XOR 到 ZZZ(bit[4..]), 取 [M,M+B)=bit[4..4+B-1] 即得
//   (j>>3)^i 的低 B 位 = v1 的 chunk^xor_mask.
template <const int kColStride = 16>
static __host__ __device__ __forceinline__ int swizzle_v2_impl(int i, int j) {
  if constexpr (kColStride == 8) {
    return swizzle_v1_impl<kColStride>(i, j);
  } else {
    static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64,
```

```
                  "v2 supports kColStride in {16, 32, 64} (cute TMA swizzle); "
                  "8 falls back to v1.");
    constexpr int B = (kColStride == 16) ? 1 : (kColStride == 32) ? 2 : 3;
    constexpr int M = 4;
    constexpr int S = 3;
    constexpr int kStep = 8;
    const int off = (i * kColStride + j) * (int)sizeof(half);
    const int sw = SwizzleBMS<B, M, S>::apply(off);
    return ((sw >> M) & ((1 << B) - 1)) * kStep;
  }
}
```

v2 的三步：(i, j) → byte offset → 式 13.8 的 apply → 取 bit $[M, M+B)$ 乘回 k_{step}=8。它算不上新算法，只是把 v1 的手写展开重写成 cute 的统一置换公式（便于脱离 cute 框架理解 swizzle 本质），两者 bit-exact 等价（12.3.6 节）。派发器只有 9 行，宏门控：

hgemm.cuh L388-396（swizzle 派发器：宏门控 v1/v2，调用点零改动）

```
// ============================================================================
template <const int kColStride = 16>
static __device__ __forceinline__ int swizzle(int i, int j) {
#if defined(NOTES_V2_ENABLE_SWIZZLE_V2)
  return swizzle_v2_impl<kColStride>(i, j);
#else
  return swizzle_v1_impl<kColStride>(i, j);
#endif
}
```

13.4.2　kernel 骨架：模板参数、block swizzle 重映射与 smem 布局

Phase 7b-3（`m16n8k16` + multistage pipeline + TN 布局 + XOR swizzle + Register Double Buffering，通用 `kValTileK`，默认 BK= 64）在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲，设计要点五条：

- `kValTileK`：每个 BK tile 包含 `kValTileK` 个 `kMmaK` slice（BK = `kMmaK` × `kValTileK`）；
- 统一 BK tile：所有 k_step slice 连续存放在同一个 BK 宽的 smem tile 中；
- `RA[2][kValTileM][4]` / `RB[2][kValTileN][2]`：双份寄存器乒乓切换（与 `kValTileK` 无关）；
- `ldmatrix` 与 MMA 计算重叠：加载 k_step+1 的同时用另一组寄存器做 k_step 计算；
- 支持任意 `kValTileK` ≥ 2；默认 `kValTileK`= 4 （BK= 64），`kStages`= 2 推荐 64 KB smem。

smem 布局：`s_a` 按 `[stage0][stage1]...[stage(kStages-1)]` 排布，每个 stage 为 BM × BK = 128 × BK；`s_b` 紧接 `s_a` 之后；每个 stage 内 k_step slice 的列偏移为 k_step × `kMmaK`。寄存器双缓冲时间线（每个 k 迭代内五步）：

1. G→S：`cp.async` 预取 `stage(k+kStages-1)` 的全部 k_step（条件）；
2. MMA k_step= 0（用 `reg[load]`，已在上轮 post-wait 中 `ldmatrix`）；
3. 对 k_step= 1..`kValTileK`−1 循环：`ldmatrix`（列偏移 k_step×`kMmaK`）→ `reg[store]`，flip，MMA → `reg[load]`；
4. adaptive wait + `__syncthreads`；
5. S→R：`ldmatrix` 预加载 `stage(k+1)` 的 k_step= 0 → `reg[store]`（条件）。

Grid $((N+127)/128/S,\ (M+127)/128,\ S)$，$S=(N+2047)/2048$，3D block swizzle；Block $(256,1,1)$，8 warps；参考实现 `hgemm_mma_stage_tn_swizzle_x2.cu`。模板参数前七项定档：`kMmaM`= 16 / `kMmaN`= 8 / `kMmaK`= 16（MMA atom `m16n8k16` 的 M/N/K 维）；`kMmaTileM`= 2（warps along M，warp tile M = 32）、`kMmaTileN`= 4（warps along N，warp tile N = 32）；`kValTileM`= 4（value-repeat along M，BM = 16 × 2 × 4 = 128）、`kValTileN`= 4（value-repeat along N，BN = 8 × 4 × 4 = 128）。

hgemm.cuh L431-460（模板参数 + 式 13.11 坐标重映射 + 双缓冲 smem）

```
          const int kValTileK = 4,          // MMA_K slices per BK tile, BK = kMmaK * kValTileK
          const int kStages = 2,            // cp.async pipeline depth (2 for kValTileK>=4 to fit smem)
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
  static_assert(kValTileK >= 2, "Register double buffering requires kValTileK >= 2");
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
  // Block Swizzle: 在 grid x 维度做 swizzle，改善 L2 cache 局部性
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
  constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
  constexpr int BK = kMmaK * kValTileK;             // 16 * kValTileK

  // kStages stages, each with BM×BK for A, BN×BK for B
  // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK 宽 tile 中
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + kStages * BM * BK;
  constexpr int s_a_stage_offset = BM * BK;
  constexpr int s_b_stage_offset = BN * BK;

  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / kWarpSize;
  const int lane_id = tid % kWarpSize;
  const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
  const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)

  // 线程到 global memory 的映射（用于加载 A 和 B）共 256 个线程
```

相对第 十二 章的新模板参数是 `kValTileK`（每 BK tile 的 slice 数，$B_K = 16\,k_{\mathrm{ValTileK}}$）与 `kBlockSwizzle`。smem 侧是经典的 kStages 级双缓冲：S 个 stage 轮转，cp.async 预取 $k{+}S{-}1$、MMA 消费 k。`kValTileK`=4 时每 stage 128×64 half，S=2 共 64 KB，上文「`kStages`=2 推荐 64 KB smem」的说法由此而来（S=3 需 96 KB，sm_120 可容但常驻 block 数下降）。

13.4.3 G→S：swizzle 进地址的 cp.async

hgemm.cuh L482-512（prefetch：物理列 = slice 基址 + swizzle(row, chunk 列)）

```
  // Prefetch (kStages-1) stages: load all k_step slices for each early stage
#pragma unroll
  for (int k = 0; k < (kStages - 1); ++k) {
#pragma unroll
    for (int k_step = 0; k_step < kValTileK; ++k_step) {
      int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
      int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
      int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
      int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;

      uint32_t load_smem_a_ptr =
          (smem_a_base_ptr +
```

```
           (k * s_a_stage_offset + load_smem_a_m * BK +
            k_step * kMmaK +
            swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
               sizeof(half));
      CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

      uint32_t load_smem_b_ptr =
          (smem_b_base_ptr +
           (k * s_b_stage_offset + load_smem_b_n * BK +
            k_step * kMmaK +
            swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
               sizeof(half));
      CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
    }
    CP_ASYNC_COMMIT_GROUP();
  }

  CP_ASYNC_WAIT_GROUP(kStages - 2);
  __syncthreads();
```

写入地址拼装：$\text{addr} = \text{stage}\cdot\text{off} + \text{row}\cdot B_K + k_{\text{step}}\cdot 16 + \texttt{swizzle}<16>(\text{row}, j)$，$j \in \{0, 8\}$。与 ldmatrix 读出（L535–537 等）用完全相同的拼装，引理 13.1 保证两者互逆。主循环内的 G→S 与此同构（L560–585），只是 stage 选择换成 $\text{sel_next} = (k+S-1) \bmod S$。

13.4.4　寄存器双缓冲与 k_step 乒乓

hgemm.cuh L514-548（RA/RB 双份寄存器 + 初始 ldmatrix 预装载）

```
  // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
  uint32_t RA[2][kValTileM][4];
  uint32_t RB[2][kValTileN][2];
  int reg_st_idx = 0; // write target for ldmatrix
  int reg_ld_idx = 1; // read source for MMA

// Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) → reg[0]
// 此时，reg_st_idx = 0，对0位置的寄存器buffer做初始化。
#pragma unroll
  for (int i = 0; i < kValTileM; ++i) {
    int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
    int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
    int lane_smem_a_k = (lane_id / 16) * 8;
    uint32_t lane_smem_a_ptr =
        (smem_a_base_ptr +
         (0 * s_a_stage_offset + lane_smem_a_m * BK +
          swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
             sizeof(half));
    LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                lane_smem_a_ptr);
  }
#pragma unroll
  for (int j = 0; j < kValTileN; ++j) {
    int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
    int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
    int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
    uint32_t lane_smem_b_ptr =
        (smem_b_base_ptr +
         (0 * s_b_stage_offset + lane_smem_b_n * BK +
          swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
             sizeof(half));
```

```
    LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                lane_smem_b_ptr);
  }
```

hgemm.cuh L587-612（k_step 循环：flip + 预取下一 slice 的 A fragment）

```
    // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
    // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
    // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)

#pragma unroll
    for (int k_step = 0; k_step < kValTileK; ++k_step) {
      reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
      reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1

      if (k_step + 1 < kValTileK) {
        int smem_k_offset = (k_step + 1) * kMmaK;
#pragma unroll
        for (int i = 0; i < kValTileM; ++i) {
          int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
          int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
          int lane_smem_a_k = (lane_id / 16) * 8;
          uint32_t lane_smem_a_ptr =
              (smem_a_base_ptr +
              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
                smem_k_offset +
                swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
                  sizeof(half));
          LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                      RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                      lane_smem_a_ptr);
        }
```

hgemm.cuh L613-640（B fragment 预取 + 用另一组寄存器发 HMMA16816）

```
  #pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
          int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
          int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
          uint32_t lane_smem_b_ptr =
              (smem_b_base_ptr +
              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
                smem_k_offset +
                swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
                  sizeof(half));
          LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                      lane_smem_b_ptr);
        }
      }

#pragma unroll
      for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          HMMA16816(RC[i][j][0], RC[i][j][1],
                    RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
                    RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
                    RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
                    RC[i][j][0], RC[i][j][1]);
        }
```

```
      }
    }
```

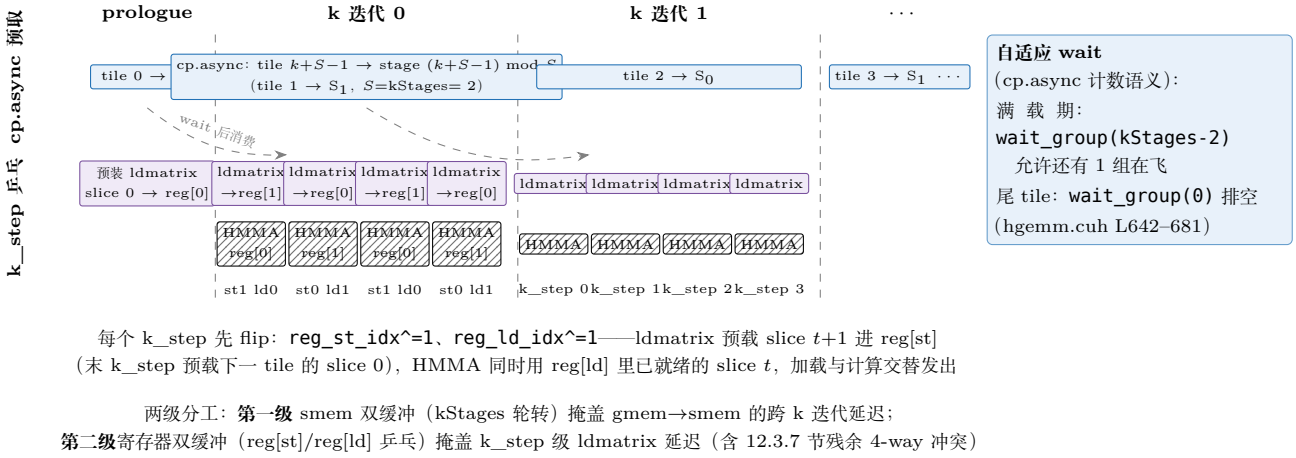
两级双缓冲的分工明确：**smem 双缓冲**（kStages）掩盖「gmem→smem」的跨 k 迭代延迟，**寄存器双缓冲**（`reg_st_idx`/`reg_ld_idx` 的乒乓）掩盖「smem→ 寄存器」的 k_step 级延迟。每个 k_step 先 flip 再 ldmatrix 到 `reg[st]`、随后 HMMA 使用 `reg[ld]`：加载 slice t+1 与计算 slice t 在指令流里交替发出，tensor core 不必等 ldmatrix（也把 12.3.7 节残留的 4-way 冲突延迟吸收进流水）。最后一段收尾：

hgemm.cuh L642-681（自适应 wait：满载 kStages-2 / 排空 0；预载下一 k 的 slice 0）

```
    // 自适应等待：满载期用 kStages-2，尾部排空用 0
    if (k + kStages - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(kStages - 2);
    } else {
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();

    // 从下一阶段加载 k_step=0 数据，供下一轮 k 迭代使用，此时 reg_st_idx=0
    if (k + 1 < NUM_K_TILES) {
      smem_sel = (k + 1) % kStages; // old smem_sel + 1
#pragma unroll
      for (int i = 0; i < kValTileM; ++i) {
        int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
        int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
        int lane_smem_a_k = (lane_id / 16) * 8;
        uint32_t lane_smem_a_ptr =
            (smem_a_base_ptr +
             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
              swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
                 sizeof(half));
        LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                    RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                    lane_smem_a_ptr);
      }
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
        int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
        uint32_t lane_smem_b_ptr =
            (smem_b_base_ptr +
             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
              swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
                 sizeof(half));
        LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                    lane_smem_b_ptr);
      }
    }
  }
```

自适应等待是 cp.async 计数语义的直接应用（第 十一 章）：满载期 `WAIT_GROUP(kStages-2)` 允许还有 1 组在飞（in-flight）；尾 tile 无新预取时退化为 `WAIT_GROUP(0)` 排空。Epilogue（L683–716）复用 `RA` 寄存器做 warp shuffle 拼出 128-bit 向量写回 C，思路与第 十二 章一致，此处不展开。



每个 k_step 先 flip: `reg_st_idx^=1`、`reg_ld_idx^=1`——ldmatrix 预载 slice t+1 进 reg[st]（末 k_step 预载下一 tile 的 slice 0），HMMA 同时用 reg[ld] 里已就绪的 slice t，加载与计算交替发出

两级分工：**第一级** smem 双缓冲（kStages 轮转）掩盖 gmem→smem 的跨 k 迭代延迟；
第二级寄存器双缓冲（reg[st]/reg[ld] 乒乓）掩盖 k_step 级 ldmatrix 延迟（含 12.3.7 节残余 4-way 冲突）

图 13.3: 两级双缓冲叠加的流水时间线（12.4.4 节）：上泳道是第一级——cp.async 按 stage $(k+S-1) \bmod S$ 预取下一 K-tile（S=kStages= 2 时 k 迭代 0 预取 tile 1 进 S_1、迭代 1 预取 tile 2 回 S_0）；下泳道是第二级——每个 k 迭代内 4 个 k_step 逐个过：先 flip（`reg_st_idx`/`reg_ld_idx` 各异或 1），ldmatrix 把 slice t+1 写进 reg[st]，同周期 HMMA 使用 reg[ld] 里已就绪的 slice t，tensor core 不必等 ldmatrix。右侧为自适应等待：满载期 `wait_group(kStages-2)` 允许 1 组在飞、尾 tile 退化为 `wait_group(0)` 排空。

13.5 图示

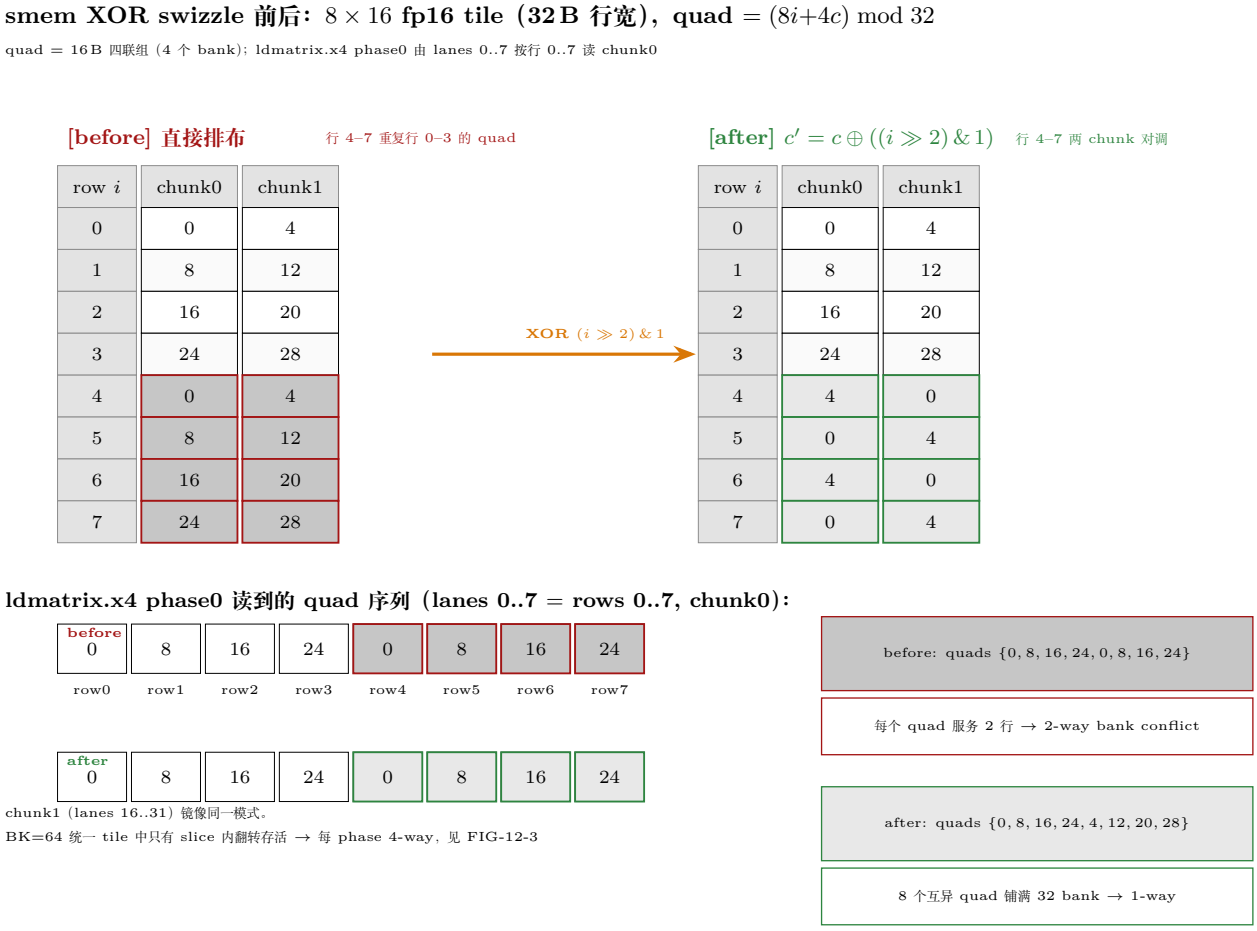


图 13.4: smem XOR swizzle 前后的 bank 四联组分布（32 B 行宽布局，即 BK=16 / FA 风格 [*][16] tile）。同一 8×16 tile：置换前每 4 行重复一轮 quad，按列读取 2-way 冲突；置换后行 4–7 的两个 chunk 对调，8 行恰好铺满 32 个 bank——ldmatrix.x4 phase0 读到的 quad 序列从 $\{0,8,16,24,0,8,16,24\}$ 变为 $\{0,8,16,24,4,12,20,28\}$。

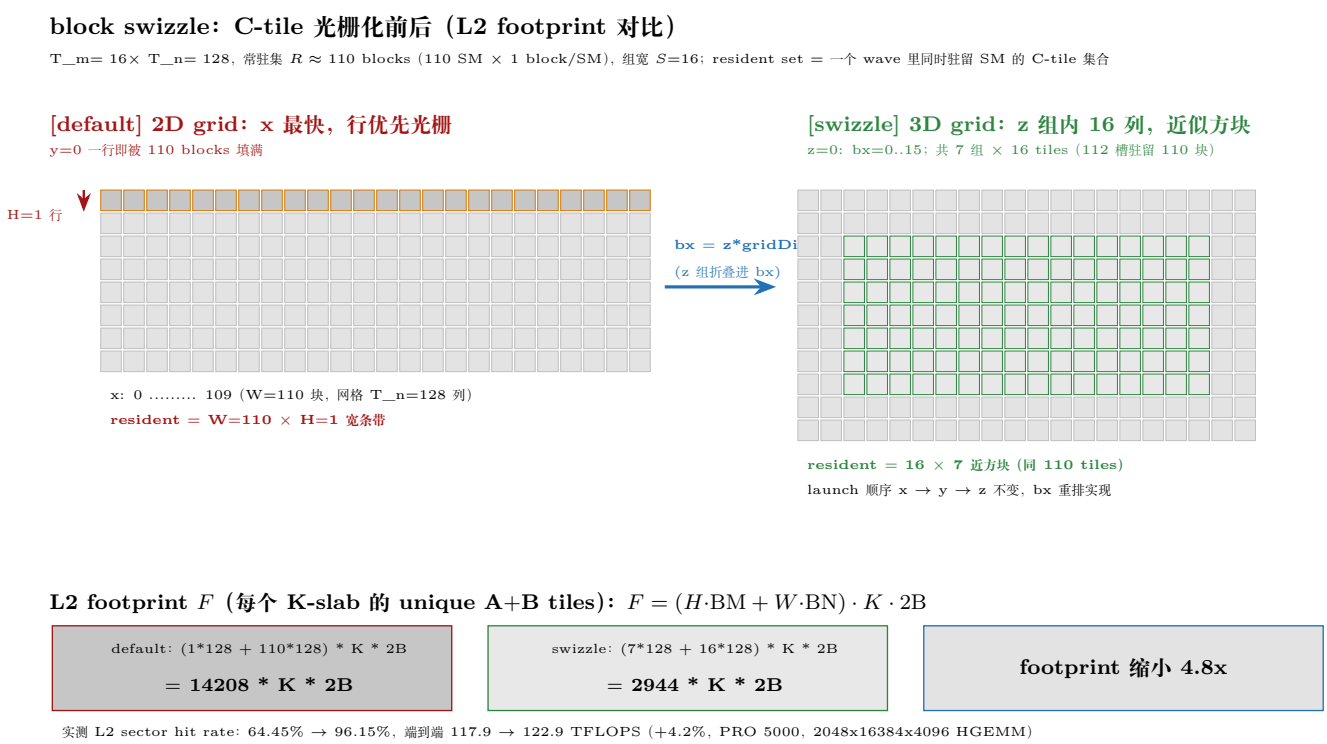


图 13.5: Block swizzle 的 grid 重排前后：默认光栅的常驻集合是 $W \times H$ 宽条带（$W = \min(T_n, R)$），重排后折叠为组宽 16 的近似方块，L2 footprint 按式 13.13 缩小 4.8 倍；实测 L2 sector hit rate 64.45 → 96.15%（PRO 5000）。



图 13.6: XOR 位运算示意：源位域（行号低位）经右移 S 异或到目标位域（chunk 号）。行宽决定「有多少行号位能进入 bank 位 $[2,7)$」，从而决定可达到的最小冲突度；源/目标位域不相交（$S \geq B$）保证对合性（引理 12.1）。

13.6 性能分析

bank conflict 三点实测（RTX PRO 5000，sm_120，$M{=}N{=}K{=}512$，NCU shared-load bank conflict 计数器 `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`，每 launch 稳态值）：

表 13.1: swizzle 前后的 shared load bank conflict 与 kernel 耗时（NCU，2026-09-11）

配置	冲突计数/launch	耗时 (μs)	模型
BK=16，无 swizzle（7b-1）	98,304	17.6	2-way；$=128\times32\times24$ 精确命中
BK=64，无 swizzle（对照）	691,067	30.2	8-way；sed 置 0 的对照版
BK=64，swizzle（7b-3）	297,628	20.7	4-way；本章 kernel

模型与实测的对齐度：BK=16 行**逐次精确**；BK=64 两行为模型 688,128 / 294,912（$=98{,}304\times7$ 与 $98{,}304\times3$）对实测 691,067 / 297,628，误差 <1%；**比值** 691067/297628 = 2.32 对模型比值 2.33，12.3.7 节的 8-way→4-way 结论由此得到证实。同时注意：swizzle 版 kernel 耗时 30.2→20.7 μs（1.46×），冲突并没有被「全消」，但寄存器双缓冲把残余 4-way 的重放延迟掩盖在 MMA 流水里。

端到端吞吐（cudaEvent，warmup 3 + 10 次均值）：

表 13.2: ch12 三配置 TFLOPS 对照（RTX PRO 5000 72GB，110 SM，2026-09-11）

配置	4096^3	$2048\times16384\times4096$
7b-1：BK=16、kS=3、无 swizzle	124.7	123.3
7b-3：BK=64、kS=2、BS=0	118.6	117.9
7b-3：BK=64、kS=2、BS=1	119.0	**122.9**

三个观察：（1）宽 N 形状下 block swizzle +4.2%（L2 命中 64.45%→96.15%），方形形状（4096^3）收益仅 +0.3%，与 12.3.8 节 footprint 推导一致，收益取决于常驻条带是否「过宽」。（2）swizzle 版（BS=0）在两种形状下都比 BK=16 基线慢 4–5%（开 block swizzle 后宽 N 形状收窄到 0.3%）：统一宽 tile 换来了更少的同步与 cp.async 组数，代价是 ldmatrix 冲突从 2-way 涨到 4-way（表 13.1），两者相抵后略有损失。若按 12.3.7 节改用 `swizzle<64>`（SW128）应能超过 BK=16 基线，这是一个现成的动手实验（见 12.10 节）。（3）绝对值口径为 fp16 累加（`f16.f16.f16.f16`）+ 本机 PRO 5000，与 README 的 5090 口径不可直接比较。

上述对照只覆盖手写版的三个配置；把 stage 数 S、block swizzle（BLK_SW）与累加精度（F16Acc/F32Acc）全部扫开的全参数实测（`notes_v2_sm120a.bin --bench --bench-all`，$M{=}N{=}K{=}8192$，fp16）：

表 13.3: HGEMM CuTe Swizzle 实测（RTX PRO 5000，M=N=K=8192，fp16）

Kernel	Max Err	TFLOPS/cuBLAS（加速比）
HGEMM CuTe Swizzle (S=2, BLK_SW=0, F16Acc)	0.000e+00	243.0/246.5 (0.99x)
HGEMM CuTe Swizzle (S=2, BLK_SW=0, F32Acc)	0.000e+00	201.3/164.2 (1.23x)
HGEMM CuTe Swizzle (S=2, BLK_SW=1, F16Acc)	0.000e+00	250.2/246.5 (1.01x)
HGEMM CuTe Swizzle (S=2, BLK_SW=1, F32Acc)	0.000e+00	226.0/164.2 (1.38x)
HGEMM CuTe Swizzle (S=3, BLK_SW=0, F16Acc)	0.000e+00	257.2/246.5 (1.04x)
HGEMM CuTe Swizzle (S=3, BLK_SW=1, F16Acc)	0.000e+00	265.9/246.5 (1.08x)
HGEMM CuTe Swizzle (S=3, BLK_SW=1, F32Acc)	0.000e+00	242.4/164.2 (1.48x)

七个配置的 Max Err 全部为 0，即与 fp64 参考逐位一致，没有任何一行牺牲精度。F16Acc 各行对比 cuBLAS 的 f16 累加（246.5 TFLOPS）：最低 0.99x、S=3 + BLK_SW=1 达 **1.08x**，CuTe Swizzle 流水整体达到并小幅超过 cuBLAS。F32Acc 各行对比 cuBLAS 的 f32 累加（164.2 TFLOPS）则**大幅超越**，最高 **1.48x**：同样的双缓冲与 swizzle 骨架，换成 f32 累加后相对 cuBLAS f32 的优势远大于 f16 档位。两个旋钮的收益方向一致：S 从 2 加到 3、BLK_SW 从 0 开

到 1 都带来正收益（如 F16Acc 下 243.0→250.2→ 257.2→265.9 TFLOPS 单调上行），与本章「多级流水掩盖 gmem 延迟、block swizzle 缩 L2 footprint」两条主线互相印证。

13.7　实践经验

实践经验 13.1 (F3 + NCU 实证)：common.cuh L124-125：「降低到 1-way (fully conflict-free)」对本章 kernel 不成立 注释的 1-way 结论只在 **swizzle 作用域 = 整行**（32 B 行宽，BK=16 / FA 的 [∗][16] 布局，即 common.cuh L185–235 表格对应的布局）时成立。本章核心 kernel `hgemm_mma_stages_tn_swizzle` 的统一 tile 行宽 128 B（BK=64），swizzle 以 per-slice 方式作用于 16 列窗口：行 stride 128 B 恰为 bank 计数环（128 B）的一圈，行号位无法进入 bank 位，单个 XOR bit 最多把 phase 内 8 lane 分到 2 个四联组，**下界 4-way**。NCU 实测 297,628 次冲突/launch（无 swizzle 对照 691,067，BK=16 基线 98,304，表 13.1）。注释 L125–126 的「局限：要求 kColStride ≤16 （即 BK ≤16）」其实已经预设了行宽前提，但 kernel 在 BK=64 下仍调用 `swizzle<16>`，两处合读容易让人误以为本 kernel 已零冲突。修正表述：per-slice swizzle 把 8-way 降为 4-way；128 B 行宽下零冲突需 `swizzle<64>`（SWIZZLE_128B，B=3）。

实践经验 13.2 (F3 核校通过)：common.cuh L320-325：v1/v2 等价性证明注释 独立推导（12.3.6 节）：$a = 2(k_{\text{cs}}i + j)$、$j < k_{\text{cs}}$ 时，bit $[4, 4{+}B)$ 恰为 $(j{\gg}3)$ 低 B 位、bit $[7, 7{+}B)$ 恰为 i 低 B 位（$k_{\text{cs}} \in \{16, 32, 64\}$ 逐一验证），apply 的异或结果与 v1 的 `chunk^xor_mask` 逐位相同。host 全遍历 30,720 点 bit-exact（本章 case C，复现 bench/base.cu `test_swizzle_equiv` L721）。注释证明无误。

实践经验 13.3 (F1 核校通过)：common.cuh L141-146：kColStride ↔ Swizzle⟨B,M,S⟩ ↔ TMA 模式映射 16/32/64 half 行宽 ↔ (1,4,3)/(2,4,3)/(3,4,3) ↔ SWIZZLE_32B/64B/128B，与 cutlass `swizzle.hpp` 及 TMA descriptor 的 swizzle 模式编码一致（第 十四 章 descriptor 位域、第 二十四 章 CuTe Swizzle 将再次用到该映射）。

实践经验 13.4 (位置考据)：common.cuh L344-347 注释悬置 「swizzle: 公开派发器……」这条注释紧接在 `swizzle_v2_impl` 之后，但 common.cuh 内**并没有**它描述的 `swizzle<kColStride>` 函数，派发器实际定义在 hgemm.cuh L388–396（各 kernel 的 include 链上游）。阅读时按「注释在 common.cuh、实体在 hgemm.cuh」理解即可，行为无影响。

13.8　常见坑

坑一：swizzle 写入与读取必须是同一个函数。可逆性来自对合（引理 13.1）：写读各调一次 `swizzle` 恰好还原。漏掉任意一侧不会越界、不会报错：数据在 chunk 粒度上错位，输出「看起来差不多但不对」，是典型的静默错误。本章测试的 2D/3D grid 一致性检查（逐位相等）可兜住这类错位。

坑二：形状对齐约束。kernel 无 K 边界守卫：$K \bmod 64 \neq 0$ 时 cp.async 静默读越界并把垃圾累进结果；M/N 非 128 倍数时 epilogue 的 128-bit 写回越界（读侧有 `load_gmem_a_m >= M` 守卫，写侧没有）。调用方必须保证 $M, N \equiv 0 \pmod{128}$、$K \equiv 0 \pmod{64}$。

坑三：64 KB 动态 smem 要 opt-in。默认每 block 动态 smem 上限 48 KB，k_{ValTileK}=4、S=2 恰好 64 KB，不调 `cudaFuncSetAttribute` 直接 launch 会得到 `invalid argument`。

坑四：block swizzle 的冗余 CTA。$\lceil T_n/S \rceil \cdot S \geq T_n$，多出来的 block（$bx \geq T_n$）靠入口守卫整体返回。守卫在所有 cp.async 之前，因此是安全的；但若把守卫下移到预取之后，冗余 block 会破坏其它 block 的 stage，改动本 kernel 时务必保持守卫在最前。

坑五：v1/v2 与 `swizzle<8>` legacy 档。k_{cs}=8 （kStep=4）无 TMA 对应模式，v2 显式回退 v1；该路径当前无 kernel 使用，新代码不要再往里加功能。

坑六：不要把「swizzle 后零冲突」当作通用结论。冲突度由**行宽**决定（图 13.6）：32 B 行宽 + 16 档位 = 1-way；128 B 行宽 + 16 档位 per-slice = 4-way；128 B 行宽 + 64 档位（SW128）= 1-way。面试或调优时先看布局再谈结论。

13.9　面试要点速查

Q1：smem bank conflict 怎么产生的？n-way 怎么数？

A：32 bank × 4 B，$\text{bank}(a) = \lfloor a/4 \rfloor \bmod 32$；128-bit 访问按 8 lane 一组分 phase 服务，同 phase 内两 lane 落同 bank 不同 word 即冲突，n 路冲突拆成 n 个 bank 周期。

Q2：XOR swizzle 的原理？为什么可逆？

A：把 chunk 号位（bit $[M, M+B)$）与行号低位（bit $[M+S, M+S+B)$）异或，写入时置换、读取时用同一公式再置换一次即还原。可逆性 = 对合：两 bit 域不相交（$S \geq B$），第二次 XOR 的源不变、与第一次抵消（引理 13.1）。

Q3：swizzle 比 padding 好在哪？

A：零 smem 浪费、保持 16 B 对齐粒度、地址计算是几条位运算（编译器还能并入寻址强度削减）；padding 改变行 stride、破坏对齐假设，且不同访问方向要不同 pad 量。

Q4：寄存器双缓冲和 smem 多级流水各掩盖什么延迟？

A：smem 双缓冲（kStages 个 stage + cp.async）掩盖 gmem→smem 的跨 k 迭代延迟；寄存器双缓冲（RA/RB 两套乒乓）掩盖 smem→ 寄存器的 k_step 级 ldmatrix 延迟，让「加载下一 slice」与「MMA 上一 slice」同周期发射。

Q5：block swizzle 为什么能提升 L2 命中？

A：默认光栅常驻集合是 $T_n \times \lceil R/T_n \rceil$ 宽条带，footprint $(HB_M + WB_N)K$ 被 W 撑大；3D grid 重排（$bx = z \cdot \text{gridDim.x} + x$）把条带折成 $\sqrt{R} \times \sqrt{R}$ 级方块，实测 L2 命中 64.45%→96.15%、宽 N 形状 +4.2%。收益取决于 T_n 是否超过 $\approx \sqrt{R}$。

Q6：这个 kernel swizzle 之后还有 bank conflict 吗？

A：有，4-way/phase（NCU 实测 29.8 万次/launch）。128 B 行宽下 per-slice 的 16 档位 XOR 只能利用 1 个行号 bit；要 1-way 需 64 档位（SWIZZLE_128B，$c' = c \oplus (i \ \& \ 7)$）。残余冲突被寄存器双缓冲的流水吸收，kernel 仍比无 swizzle 快 1.46×（512^3）。

Q7：CuTe 的 Swizzle⟨B,M,S⟩ 三个参数什么含义？

A：B = 参与置换的行数位（2^B 行，即 SWIZZLE_32/64/128B 的 1/2/3）；M = 基本元素位宽（16 B chunk ⇒ M=4，低 M 位不动）；S = 每行 chunk 数位（8 chunk ⇒ S=3，也是源/目标 bit 域的位移）。完整周期覆盖 2^{M+S+B} 元素。

13.10 最小测试与动手实验

测试文件：`book/tests/ch12_hgemm_swizzle.cu`（逻辑抽取自 bench/hgemm.cu 的 test_hgemm_swizzle（L115）与 bench/base.cu 的 test_swizzle_equiv（L721），正确性对照改为 CPU fp64 参考；文件顶部 `#define NOTES_V2_ENABLE_SWIZZLE_V2 1`，使 kernel 走 v2 路径，build_tests.sh 不传额外宏，宏必须写在 include 之前）。三个 case：A（256^3，swizzle kernel vs fp64）；B（M=384、N=512、K=256，2D grid 与 3D 分组 grid 各跑一遍，除各自 vs fp64 外还要求两种 grid **逐位一致**：坐标重排只是 tile 重编号，同 tile 计算路径必须相同）；C（host 端 v1/v2 全遍历等价，$k_{\text{cs}} \in \{8, 16, 32, 64\}$ 共 30,720 点）。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=7` 下执行 `./build_tests.sh --arch sm_120a --ch ch12`。本机运行输出（RTX PRO 5000，2026-09-11）：

```
case A: M=256 N=256 K=256 (2D vs 3D grid)
PASS hgemm swizzle reg2x (2D grid): max_abs_err=2.477e-02 (tol=5.0e-02)
PASS hgemm swizzle reg2x (3D grid BS=1): max_abs_err=2.477e-02 (tol=5.0e-02)
PASS 2D vs 3D grid bitwise-equal: max_diff=0.0e+00
case B: M=384 N=512 K=256 (2D vs 3D grid)
PASS hgemm swizzle reg2x (2D grid): max_abs_err=2.869e-02 (tol=5.0e-02)
PASS hgemm swizzle reg2x (3D grid BS=1): max_abs_err=2.869e-02 (tol=5.0e-02)
PASS 2D vs 3D grid bitwise-equal: max_diff=0.0e+00
case C: swizzle v1/v2 host equivalence
PASS swizzle v1/v2 equiv: total=30720 fail=0
ALL OK
```

判据说明：fp16 累加（`f16.f16.f16.f16`）对 fp64 参考的误差随 K 增长，K=256 时约 $2.5 \sim 2.9 \times 10^{-2}$，落在 F16Acc 档 5×10^{-2} 内；K=512 实测 5.8×10^{-2} 超档，故 case B 固定 K=256。动手实验：（1）把 case B 的 3D grid 换成 $\lceil T_n/4 \rceil \times T_m \times 4$ 验证 kernel 对任意分组的正确性；（2）复制 kernel 把 8 处 `swizzle<kMmaK>` 换成 `swizzle<64>` 并把 slice 基址并入 j（SW128 方案，12.3.7 节），用 NCU 复测冲突计数是否归零、端到端是否超过 7b-1；（3）用 bench_leetcuda.cu bench 复跑 $2048 \times 16384 \times 4096$ 口径对照表 13.2。

延伸阅读

- @frankshi《浅谈 CUDA 中的 swizzling 机制（以 ldmatrix 为例）》，https://zhuanlan.zhihu.com/p/4746910252 （本章主参考：bank 冲突模式与 XOR 置换的图解推导）；
- @reed《cute 之 Swizzle》，https://zhuanlan.zhihu.com/p/671419093 （Swizzle$\langle B, M, S\rangle$ 的布局代数视角）；
- @Titus《cutlass 中的 Swizzle（一）（二）》，https://zhuanlan.zhihu.com/p/710337546、https://zhuanlan.zhihu.com/p/711398930 （SW32/64/128B atom 与 TMA 硬件 swizzle 模式）；
- @ 进击的 Killua《cute Swizzle 细谈》，https://zhuanlan.zhihu.com/p/684250988 （bit 域图示与 v2 实现细节对照）；
- @JoeNomad《GEMM 优化中的 block swizzle》，https://zhuanlan.zhihu.com/p/679929705 （grid 重排与 L2 复用的 footprint 分析，12.3.8 节同源）；
- @Anonymous《GEMM 细节 (三)：Swizzle⟨B,M,S⟩ 参数取值》，https://zhuanlan.zhihu.com/p/713713957（$B/M/S$ 与行宽、chunk 宽的对应表）。

第十四章　Hopper 之路：TMA、mbarrier 与 WGMMA

14.1　本章导读

第 十二 章的 `mma.sync` 与第 十三 章的 XOR swizzle 已把 GEMM 做到 Ampere 时代的效率水平，但这条路线有三个先天约束：**搬运靠线程**，每个线程各自计算 gmem→smem 的地址并搬运数据，整数地址指令与边界判断占掉了大量 issue 槽；**同步靠栅栏**，`__syncthreads` 只能表达「全部到齐」，无法表达「数据到齐」；**计算是同步的**，`mma.sync` 发射后要等结果，数据搬运与矩阵乘难以深度重叠。Hopper（sm_90a）用三个硬件级新原语分别解决这三个约束：`TMA`（Tensor Memory Accelerator，张量搬运的专用 DMA 引擎）、`mbarrier`（带事务计数与相位轮转的异步屏障）、`WGMMA`（warpgroup 级异步 Tensor Core 指令）。本章先讲搬运与同步、后讲计算：**前半**讲 TMA 的 descriptor 编码与 `cp.async.bulk.tensor` 搬运指令、mbarrier 的 arrive/tx 双计数事务模型，它们是 Hopper 异步流水线的基础设施；**后半**讲 WGMMA 的 warpgroup 编程模型、smem descriptor 的 64-bit 逐字段编码，以及 producer/consumer warp specialization 的完整协作协议。

源码区间：hgemm.cuh L1428–1857（`hgemm_wgmma_stages_tn`@L1477）；common.cuh L350–803（TMA/mbarrier helpers、`setmaxnreg`、WGMMA 宏、smem descriptor 位域注释 L545–587、TensorMap helpers）。**编译宏与架构限定**：`NOTES_V2_ENABLE_WGMMA`，**仅 sm_90a**，WGMMA/mbarrier/TMA 是 Hopper 专属指令族，不向前兼容 sm_100/sm_120（详见 13.8 节坑一）。**前置**：第 十二 章（mma.sync、ldmatrix、多级流水）与第 十三 章（swizzle 与 bank conflict）；**后继**：第 十五 章把 TMA+mbarrier 协议搬到 SM120 配 `mma.sync`，第 十八–十九 章把它搬进 FlashAttention。**验证条件**：本机 GPU 为 sm_120a（无 Hopper），本章采用「编译级验证 + 运行时 SKIP」策略，完整数值验证需 H100/H200/H800（13.10 节给出复现命令）。

14.2　问题与动机

把第 十二 章的 `hgemm_mma_stages_tn` 放到 roofline 下审视：BM=BN=128、BK=64 的 block tile，每装载一个 K-tile（A 16KB + B 16KB）可做 $2 \times 128 \times 128 \times 64 \approx 2.1$ M FLOP，算术强度高达 64 FLOP/B，计算本该远超访存成为瓶颈。但 Ampere 路线的实际执行中，仍有三类「非计算」开销在挤占 Tensor Core 的执行时间：

- **每线程地址算术**。cp.async 路径下每个线程都要计算自己的 gmem 源地址、smem 目的地址、边界掩码，一个 128×64 的 tile 意味着数百条整数指令，它们与 Tensor Core 指令竞争同一个发射端口；
- **搬运线程的开销**。smem→ 寄存器的 `ldmatrix` 也占线程、占寄存器、占发射槽；「搬运」与「计算」没有分工，全部线程既搬运又计算；
- **同步粒度太粗**。`__syncthreads` 只能回答「所有线程是否都到达」，无法回答「这块数据是否到达」；多级流水（kStages）只能靠 cp.async 的 commit/wait group 间接拼装，且无法把「搬运」本身变成独立的执行流。

Hopper 的解法是让三类工作分别交给专用机制：**搬运交给 TMA**，一条 `cp.async.bulk.tensor.2d` 指令由**单个线程**提交，硬件 DMA 引擎搬运整块 2D tile，地址计算、边界处理（OOB 填充）、smem swizzle 全部在 descriptor 里声明式配置，搬运过程零寄存器开销；「**数据到齐**」**交给 mbarrier**，屏障自带 arrive count（多少个线程/信号到达）与 tx count（多少字节的事务完成）两个独立计数器，TMA 搬完指定字节数后硬件自动计数，phase 翻转即代表数据可见；**计算交给 WGMMA**，矩阵乘从 warp 级（32 线程）升级到 warpgroup 级（128 线程），A/B 操作数直接从 shared

memory 读取（无需 ldmatrix 中转），且指令异步执行：发射后 warpgroup 可以继续推进，用 `wgmma.wait_group` 等待并取回结果。三者组合带来 Hopper 最重要的编程模型变化，即 **warp specialization**：一个 warpgroup 专职 producer（提交 TMA），另一个专职 consumer（发射 WGMMA），两支执行流通过 mbarrier 阵列解耦推进，搬运延迟被计算完全掩盖。

14.3　数学原理

14.3.1　TMA 的 box 几何与坐标

TMA 把 global memory 中的张量看作多维结构、把 shared memory 看作线性空间，两者之间的映射由 host 侧创建的 `CUtensorMap`（TMA descriptor）声明。对 row-major 的 $A \in \mathbb{R}^{M\times K}$（行 stride 为 K 个元素），元素 (m,k) 的全局线性偏移是

$$\mathrm{off}(m,k) \;=\; m\cdot K + k, \tag{14.1}$$

即 k 是**连续维（minor）**、m 是**跨步维（major）**。TMA descriptor 的 `globalDim` 因此写作 (K,M)，**minor 在前**，这是最容易记反的一个约定（hgemm.cuh L1484 起的注释专门强调了它）。每次拷贝由指令操作数 `tensorCoords` $=(k_0,m_0)$ 给出 box 左上角，搬运一个 boxDim $=(B_K,B_M)$ 的矩形块：

$$\mathrm{box}(k_0,m_0) \;=\; \{\, A[m_0+i,\; k_0+j] \;:\; 0\le i<B_M,\; 0\le j<B_K \,\}, \tag{14.2}$$

整块共 $B_M\cdot B_K\cdot \mathrm{sizeof}(T)$ 字节落进 smem 的连续区间，并在 128B swizzle 模式下按第 十三 章讲过的 swizzle 原子模式重排（13.4.5 节会看到它与 WGMMA descriptor 的呼应）。PTX 对 box 的硬约束：box 总字节数是 16 B 的倍数、地址 16B 对齐；越界（OOB）行为由 descriptor 的 fill mode 决定，取 zero fill 或 OOB-NaN fill（PTX ISA §5.5.3.3），本 kernel 的网格构造保证不越界。另注意 box 内维 $B_K=64$ 个 half $=128$ B，恰好等于 `SWIZZLE_128B` 的宽度，原因是：128B swizzle 模式下内维字节不能超过 swizzle 档位宽度，64 half 正是按该约束的标准取值。

14.3.2　mbarrier 的双计数代数

mbarrier 是一个 8 字节对齐、驻留 shared memory 的 `.b64` 对象，内部维护四个量（PTX ISA §9.7.13.15.2）：当前相位 $\pi\in\{0,1,2,\dots\}$、本相位**待到达数** P（pending arrivals）、**期望到达数** E（expected arrivals，供下一相位使用）、本相位**待完成事务量** T（tx-count，单位由被跟踪的异步操作决定，TMA 场景是字节）。`mbarrier.init` 把 (π,E,P,T) 置为 $(0,N,N,0)$。三类操作各改一个计数器：

$$\text{arrive-on（count 默认 1）：}\quad P \leftarrow P - \mathrm{count}, \tag{14.3}$$

$$\text{expect-tx（}\Delta T\text{）：}\quad T \leftarrow T + \Delta T, \tag{14.4}$$

$$\text{complete-tx（}\Delta C\text{）：}\quad T \leftarrow T - \Delta C. \tag{14.5}$$

其中 complete-tx 不由软件调用：TMA 指令带 `.mbarrier::complete_tx::bytes` 限定符，DMA 引擎在数据**真正写入 smem** 时自动按搬运字节数计数（PTX ISA §9.7.9.25）。相位翻转的条件是**两个计数器同时归零**：

$$P=0 \;\wedge\; T=0 \quad\Longrightarrow\quad \pi\leftarrow\pi+1,\;\; P\leftarrow E \;\text{（原子完成）}. \tag{14.6}$$

等待方（`mbarrier.try_wait`）阻塞到「自己 arrive 时拿到的旧相位 π_{old} 已不是当前相位」，即 $\pi\neq\pi_{old}$。这套设计的关键是：「**多少个信号**」与「**多少字节数据**」**是两个独立的计数条件**，只有全部信号到齐且全部字节落盘，数据才算就绪，producer/consumer 之间因此不需要任何 `__syncthreads`。

本章 kernel 的具体数字：每个 stage 配两个 barrier（`full[s]` 表「数据就绪」、`empty[s]` 表「stage 可覆盖」），arrive count 都是

$$N \;=\; \underbrace{128}_{\text{consumer 全体 arrive}} \;+\; \underbrace{1}_{\text{producer 提交线程}} \;=\; 129, \tag{14.7}$$

而 producer 在 `full[s]` 上用 `mbarrier.arrive.expect_tx` 一次完成「+1 个到达」并声明预期事务量：一个 K-tile 的 A+B 共 $(B_MB_K+B_NB_K)\times 2\,\mathrm{B} = (128\cdot 64+128\cdot 64)\times 2 = 32768$ B。于是 `full[s]` 的相位翻转条件被精确地表述为：129 个 arrive 且 32768 B 全部落入 smem。数字对不上，流水线要么挂死（少报了字节或到达数），要么提前放行（多报了字节，读到半新半旧的数据）。

14.3.3 WGMMA 的形状与寄存器分布

`wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16` 一条指令完成

$$D_{64\times128} \mathrel{+}= A_{64\times16} \times B_{16\times128}, \qquad \text{FLOP} = 2\times64\times128\times16 = 262{,}144, \tag{14.8}$$

是 `mma.m16n8k16`（4,096 FLOP）的 64 倍。输出 D 的 8192 个 half 分布在 warpgroup 的 128 个线程上：每线程 64 个 half，按 `.f16x2` 打包为 32 个 32-bit 寄存器。分布规则（13.4.7 节 epilogue 直接照此写回）：

$$\text{row} = 16\,w + \lfloor \text{lane}/4 \rfloor, \qquad \text{col} = 16\,g + 2\,(\text{lane} \bmod 4), \tag{14.9}$$

其中 $w \in [0,4)$ 是 warp 序号、$g \in [0,8)$ 是 16 列一组的组号；每线程在 (row, col) 起的 4 个 8×8 象限里各持一条打包的寄存器（覆盖 (row, col)、(row+8, col)、(row, col+8)、(row+8, col+8)）。块级组装：$B_M = 128 = 2\times64$ 个 M-atom、$B_K = 64 = 4\times16$ 个 K-step，每个 K-tile 需要发射 $2\times4 = 8$ 条 WGMMA（M 方向 2 个 atom × K 方向 4 次累加），每条用 scaleD = 1 在累加器上累加。

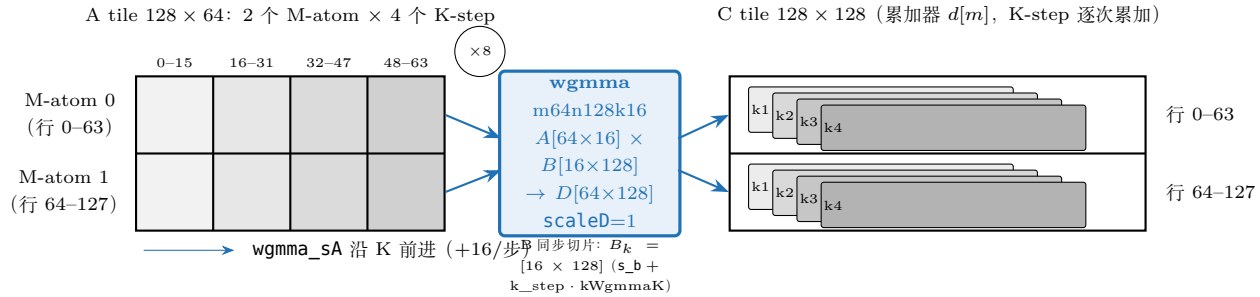


图 14.1: 一个 K-tile 的 8 条 WGMMA 拼出 C tile（C2 双层循环）：A tile 128×64 按 M 方向切 2 个 64 行 M-atom、K 方向切 4 个 $kWgmmaK$=16 的 K-step，对每个 (M-atom, K-step) 发射一条 `wgmma.m64n128k16`——A 指针 `wgmma_sA` 沿 K 每步前进 16（本 kernel 即 m 循环 × k_step 循环），B 从 `s_b` 同步切 $[16\times128]$ 片；4 次 K-step 靠 scaleD=1 在同一累加器 $d[m]$ 上逐次累加（C tile 内 k=1 → 4 灰度渐深），2 个 M-atom 各持独立累加器 $d[0]/d[1]$。

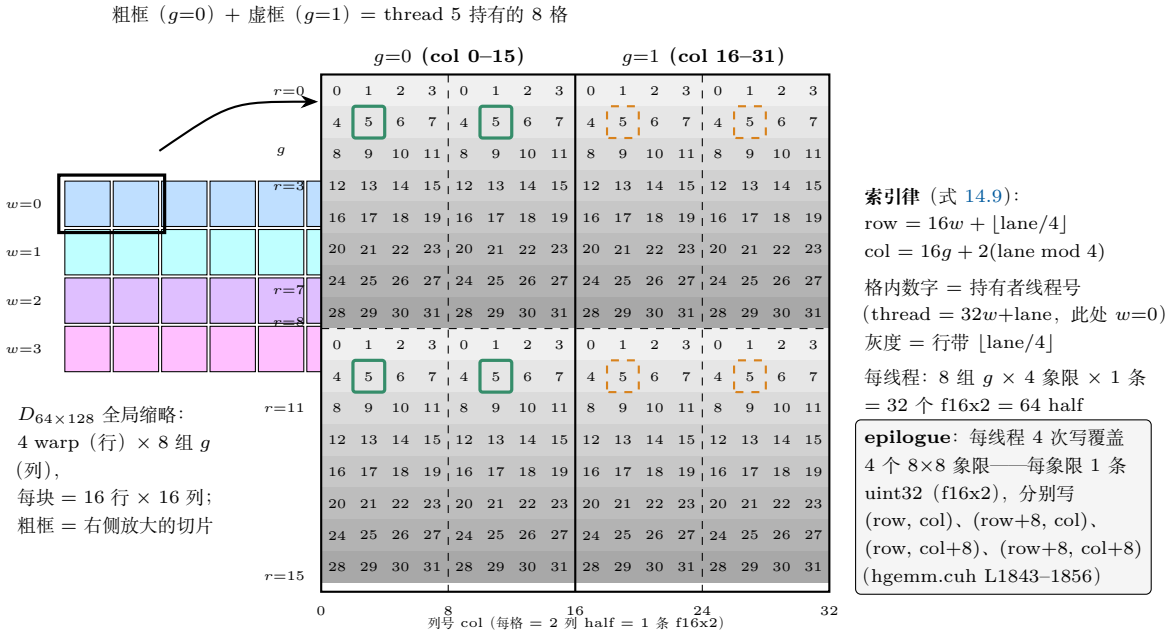


图 14.2: WGMMA m64n128k16 输出 D 的寄存器分布（13.3.4 节，式 14.9）：左为 64×128 全局缩略（4 warp × 8 组 g，每块 16 行 × 16 列）；右侧放大 warp w=0 的 16 行切片（g=0, 1 两组，每格一条 f16x2 = 相邻 2 列），格内数字为持有该输出的线程号、灰度按行带 $\lfloor \text{lane}/4 \rfloor$ 区分。索引律 $\text{row} = 16w + \lfloor \text{lane}/4 \rfloor$、$\text{col} = 16g + 2(\text{lane} \bmod 4)$ 使每个线程在每组 g 内恰好持有 4 个 8×8 象限里各一条寄存器（粗框/虚框标出 thread 5 的 8 格），epilogue 据此用 4 次写覆盖 4 个象限。

14.4　设计与实现

14.4.1　warpgroup：Hopper 的最小执行单元

PTX ISA §9.7.15.1 定义 warpgroup 为「4 个连续 warp 的集合，且首 warp 的 warp-rank 是 4 的倍数」，warp-rank 由线程线性序号 tid $= x + y \cdot n_x + z \cdot n_x n_y$ 除以 32 得出。本 kernel 启动 256 线程 = 2 个 warpgroup：WG0（线程 0–127）做 producer，WG1（线程 128–255）做 consumer；`__launch_bounds__(256, 1)` 为 `setmaxnreg` 寄存器再分配（producer 释放、consumer 多占）预留编译器许可，该机制留待第 十五 章展开。

hgemm.cuh L1468-1481（模板参数即 tile 层级与 kernel 签名）

```
template <const int kWgmmaM = 64,           // WGMMA atom M dim (m64n128k16)
          const int kWgmmaN = 128,          // WGMMA atom N dim
          const int kWgmmaK = 16,           // WGMMA atom K dim
          const int BM = 128,               // block tile M, 2 × kWgmmaM
          const int BN = 128,               // block tile N, 1 × kWgmmaN
          const int BK = 64,                // block tile K, 4 × kWgmmaK
          const int kNumThreads = 256,      // 2 wargroups × 128 threads
          const int kStages = 3,            // TMA pipeline depth (full/empty barriers)
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(kNumThreads, 1)
    hgemm_wgmma_stages_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB) {
```

tile 层级与第 十二 章完全同构：MMA atom 从 $16\times8\times16$ 升到 $64\times128\times16$（源码 L1451–1466 的 Tile Hierarchy 注释给出了逐层对照表），warp tile 与 block tile 的分层数学不变，变的只是「谁在搬运」（TMA）与「谁在计算」（warpgroup 异步指令）。

14.4.2　TMA descriptor：cuTensorMapEncodeTiled 逐参数

TMA descriptor 在 host 侧由 driver API `cuTensorMapEncodeTiled` 创建（128 字节的不透明对象，使用时传 device 侧拷贝的指针），kernel 侧只需把 `CUtensorMap*` 当作 `cp.async.bulk.tensor` 的操作数：

common.cuh L764-785（create_tensor_map：host 侧 TMA descriptor）

```
template <int BlockMajorSize, int BlockMinorSize>
__host__ static inline void create_tensor_map(CUtensorMap *tma_map,
                                              half *gmem_ptr,
                                              int blocks_height,
                                              int blocks_width) {
  void *gmem_address = (void *)gmem_ptr;
  uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
                                 (uint64_t)BlockMajorSize * blocks_height,
                                 1, 1, 1};
  uint64_t gmem_prob_stride[5] = {
      sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
  uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
                                uint32_t(BlockMajorSize), 1, 1, 1};
  uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
  CUresult result = cuTensorMapEncodeTiled(
      tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
      gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
      CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
      CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
  if (result != CUDA_SUCCESS)
    printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
}
```

逐参数对照（以 $A[M,K]$ row-major、box $= (B_K, B_M) = (64, 128)$ 为例）：

- `globalDim = (K, M)`：**minor（连续维）在前**。对 row-major 矩阵即「列数在前、行数在后」，与直觉的 (M, K) 相反；
- `globalStrides` 只接收 rank -1 个值，最内维 stride 恒为 sizeof(T)，隐式不传。源码数组 `gmem_prob_stride` 的 [0] 槽放着隐式内维（2 字节）、[1] 槽是 $2K$（行 stride 字节数），因此 API 调用传 `gmem_prob_stride + 1` 跳过首元素，这个 `+1` 是「数组名退化为指针 + 跳过隐式维」的双重语义；
- `boxDim = (64, 128)`：同样 minor 在前；总字节 $64 \times 128 \times 2\,\mathrm{B} = 16$ KB；
- `elementStrides = (1, 1)`：无跨步采样（非 1 时可用于 dilation）；
- `interleave`：NONE（8B/16B 交织仅用于卷积的 NHWC 型布局）；
- `swizzle = 128B`：搬运时按 128B swizzle 原子重排进 smem，与 WGMMA descriptor 的 `layout_type` 编码共享同一套物理排布（13.4.5 节）；
- `L2 promotion / OOB fill`：L2 驻留提示与越界填充策略（NONE/zero fill；OOB-NaN 用于 fp16 场景的非法值标记）。

实践经验 14.1 (F2 · 社区考据)：TMA descriptor 的「隐藏第 21bit」PTX ISA 把 tensorMap 当作**不透明对象**（"opaque tensor-map object"），对外只承诺 tiled 模式的 box/OOB 语义（zero/NaN fill，§5.5.3.3）；其 128B 内部位域布局并无官方文档。知乎 @reed 逆向 `cuTensorMapEncodeTiled` 的输出发现：control word 的第 21bit 在「张量总大小 $\geq$ 128KB」时被置 1，硬件据此 **跳过 OOB 检查**（大张量的越界地址通常仍落在可访问页内，跳过检查换性能）。隐患在于 `tensormap.replace`（device 端动态改写 globalDim/globalAddress，变长序列推理的标准手法）**不会联动更新该 bit**：先用大 shape 建模板（bit 为 1）、再 replace 成小 shape，得到的 descriptor「globalDim 说很小、bit21 说无需 OOB 保护」，越界地址一旦跨过实际页映射边界即 `illegal memory access`（评论区指出 driver $\geq$ 13010 已修复，CUTLASS/Triton 均按此规避）。本章 kernel 网格不越界、无 replace，不受影响；但「变长场景优先重建 descriptor 而非 replace」应记入风险清单。以本地 ptx-docs 为准的结论：**该 bit 属实现细节，非指令集约定**，依赖它（无论利用还是规避）都应锁 driver 版本。

14.4.3 发起搬运：cp.async.bulk.tensor.2d

common.cuh L379-407（tma_load_2d：raw PTX 形式）

```
// cp.async.bulk.tensor.2d: 2D TMA copy from global to shared::cta.
//   [dst]            = shared memory destination (smem addr, 32-bit)
//   [desc, {c0, c1}]  = CUtensorMap + (minor_coord, major_coord) coords
//   [mbar]            = mbarrier in shared memory (smem addr, 32-bit)
// mbarrier::complete_tx::bytes: barrier flips phase when TMA bytes land.
// L2::cache_hint omitted (EVICT_NORMAL); matches the cuda::ptx default.
// NOTE: dst must be shared::cta, NOT shared::cluster. ptxas (CUDA 13.2) treats
// the shared::cluster form as an implicit extern-call boundary and silently
// drops setmaxnreg from the same kernel (C7506). cute's TMA copy ops emit
// shared::cta on SM120 targets (CUTE_ARCH_TMA_SM120_ENABLED gate in
// copy_sm90_tma.hpp; the sm_90 path still emits cluster), which is why CuTe
// WS kernels keep their register rebalancing. Semantics are identical here:
// the destination is always CTA-local smem. Verified by PTX surgery:
// cluster->cta flips C7506 off and USETMAXREG appears (both sm_120a/120f).
static __device__ __forceinline__ void tma_load_2d(
    void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
    cuda::barrier<cuda::thread_scope_block> &barrier) {
  uint64_t gmem_int_desc = reinterpret_cast<uint64_t>(tensor_map);
  uint32_t smem_int_ptr = notes_cast_smem_ptr_to_uint(dst);
  uint32_t smem_int_mbar =
      notes_cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
  asm volatile(
      "cp.async.bulk.tensor.2d.shared::cta.global.mbarrier::complete_tx::bytes"
      " [%0], [%1, {%3, %4}], [%2];"
      :
      : "r"(smem_int_ptr), "l"(gmem_int_desc), "r"(smem_int_mbar),
```

```
        "r"(minor_coord), "r"(major_coord)
      : "memory");
}
```

指令 `cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes` 的操作数逐个看：`[dst]` 是 smem 目的地址（32-bit shared 偏移，由 `__cvta_generic_to_shared` 转换）；`tensorMap` 是 descriptor 的 **generic 地址**（device 显存中的 `CUtensorMap` 指针）；坐标 $\{c_0, c_1\} = (k_0, m_0)$ 是 box 左上角（minor 在前，元素坐标而非字节）；`[mbar]` 是与该 stage 配套的 mbarrier 地址。语义上它是 **非阻塞单线程提交**：一条指令、一个线程，DMA 引擎搬运整个 64×128 tile，完成后自动对 mbarrier 执行 complete-tx（计入的字节数 = 实际搬运量）。与 cp.async 的对比是本节的重点：cp.async 是**每线程**一条 16B 搬运指令 + 软件 commit/wait group；TMA 是**每 tile 一条**指令 + 硬件 mbarrier 计数，寄存器开销为零。producer 只需要 1 个线程的根源也在这里，其余 127 个 producer 线程仅用于满足 warpgroup 对齐要求（warp specialization 的角色以 warpgroup 为粒度划分）。

14.4.4　mbarrier 协作协议：full/empty 双屏障

hgemm.cuh L1577-1598（barrier 初始化：arrive count = 129）

```
  // K 方向总 tile 数。要求 K 能被 BK 整除，否则尾 tile 被丢弃。
  const int NUM_K_TILES = div_ceil(K, BK);
  const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
  const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup

  // 初始化 barriers（仅 thread 0 执行）
  if (threadIdx.x == 0) {
    for (int i = 0; i < kStages; ++i) {
      // arrive_count = 129: 128 consumer + 1 producer
      // init 是 CUDA barrier 的 hidden friend 函数（定义在 cuda::barrier 类体内），
      // 只能通过 ADL（Argument-Dependent Lookup）调用。因为第一个参数 &full[i] 的类型是
      // cuda::barrier<cuda::thread_scope_block>*，编译器通过 ADL 在 cuda 命名空间中自动找到它。
      init(&full[i], kConsumerThreads + 1);  // 128 consumer + 1 producer
      init(&empty[i], kConsumerThreads + 1); // same
    }
    // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
    // 对 async proxy（TMA/WGMMA）可见。
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  // ======================================================================
```

源码用 CUDA C++ 的 `cuda::barrier` 封装，底层内联展开为 `mbarrier.init/arrive/try_wait.parity` 原语：`init(&full[i], 129)` 即 `mbarrier.init.shared::cta.b64 [addr], 129`；`bar.arrive()` 返回携带旧相位的 token，`bar.wait(token)` 即按相位奇偶等待翻转。初始化尾部 `fence.proxy.async.shared::cta` 不可省略：它建立 generic proxy（普通 st/ld）与 async proxy（TMA/WGMMA）之间的内存序，保证「线程写好的 barrier 状态」对异步引擎可见，这是 Hopper 异步模型的一类 fence（另一类是 13.4.6 节的 `wgmma.fence`，管的是寄存器序）。

producer 侧每个 K-tile 的三步（P1/P2/P3）。P1 只需看骨架：`empty[s].wait(empty[s].arrive())` 的 arrive-then-wait 模式与 consumer 的 C1 完全对称，语义留到本节末统一拆解：

hgemm.cuh L1621-1628（P1 骨架：producer 循环与 stage 轮转）

```
  if (wg_idx == 0) {
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
      // stage: 当前操作的 stage 索引（round-robin 0 -> 1 -> 2 -> 0 -> ...）
      int stage = 0;
      for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
        if (stage == kStages)
```

```
          stage = 0;
```

hgemm.cuh L1653-1672（P2/P3：提交两次 TMA + arrive.expect_tx 32KB）

```
        // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
        // 无需线程逐元素搬运，零寄存器开销。
        // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
        tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
                    full[stage]);

        // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
        tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
                    full[stage]);

        // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
        //
        // barrier_arrive_tx(bar, arrive_count_update, byte_count):
        //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
        //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
        //   - Phase 翻转条件:
        //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
        //       (b) 所有声明的 async 字节已写入 smem
        //     两个条件都满足后 phase 才翻转，Consumer 的 wait() 才返回。
        //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
```

P1 的 `empty[stage].wait(empty[stage].arrive())` 模式（L1651，骨架后续）逐句看：`arrive()` 先贡献 1 次到达并拿到旧相位 token，如果 consumer 在上一轮（或 C0 预热）已积累 128 次，加上这次恰好 129，相位立即翻转，`wait` 立即返回；否则阻塞等 consumer 积累满 129。P3 的 `tma_arrive_expect_tx(full[stage], 32768)` 展开为 `mbarrier.arrive.expect_tx.shared::cta.b64 _, [mbar], 32768`：**一条指令同时完成「+1 到达」与「T += 32768」**（PTX 规定 arrive 与 expect_tx 合并时 count 固定为 1）。此后相位翻转的条件是两个：128 个 consumer 的 arrive 到齐 + 两次 TMA 的 32768 字节全部落盘。

consumer 侧的 C0「预热」是新手最容易漏的一步：

hgemm.cuh L1682-1695（C0：128 线程预 arrive 所有 empty——流水线的启动）

```
  else {
    // Step C0: Consumer 初始化 — 标记所有 stage 为"空"（可被 Producer 写入）
    //
    // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
    // 这是 Pipeline 的"预热"步骤——没有它，Producer 的 empty[stage].wait()
    // 在第一轮会永远阻塞（因为 Producer 的 1 次 arrive 不足以凑够 129）。
    //
    // 注意：此时每个 empty[i] 只有 128 次 arrive，未达 129，phase 不翻转。
    // Producer 后续的 empty[stage].arrive() 作为第 129 次，触发 phase 翻转。
    NOTES_V2_REG_ALLOC(232);

    for (int i = 0; i < kStages; ++i) {
      [[maybe_unused]] auto token = empty[i].arrive();
    }
```

不做 C0 的后果是：producer 第一轮 P1 在 `empty[0]` 上只有自己 1 次 arrive，$P = 129 - 1 = 128 \neq 0$，相位永不翻转，整个 kernel 挂死。C0 用 128 个线程把 $S = 3$ 个 empty 各预存 128 次到达，producer 的 1 次 arrive 恰好补满 129，首轮立即放行。该设计的关键在于：**同一套 arrive count 同时编码了「全部 consumer 已读完」与「producer 已就位」两个事件**，consumer 读完某 stage 的 128 次 arrive 存进「下一相位」的计数里，与 producer 下一轮的 1 次合成完整的翻转条件。

14.4.5　WGMMA smem descriptor：64-bit 逐字段编码

WGMMA 与 TMA 之间的衔接约定就是 smem descriptor：TMA 按 128B swizzle 写入 smem，WGMMA 按同一个 128B swizzle 读出，descriptor 向硬件声明 tile 的地址、跨步与 swizzle 档位。64-bit 位域布局（PTX ISA §9.7.15.5.1.2.2；源码 common.cuh L558–587 的注释表与图 14.4 逐字段一致，此处不重复引用）。编码实现分上下两半，上半只动 start_address 与 LBO 两个字段（表 14.1）：

表 14.1: make_smem_desc 上半：start_address 与 LBO

字段	填入与编码	语义
bits $[0,14)$ start_address	`SMEM_DESC_ENCODE(addr)`：右移 4 位（丢弃低 4 个恒为 0 的 bit），只保留 14 位	硬件解码时左移 4 位恢复完整的 16B 对齐地址。可寻址范围 $2^{14+4}=2^{18}=$ 256K 个 half = 512 KB 共享内存
bits $[16,30)$ leading_byte_offset	填 16（字节），编码 $(16\,\&\,0\text{x}3\text{FFFF})\gg 4=1$，再 `<< 16` 放进 bits $[16,30)$	K-major + 128B swizzle 下该字段硬件未使用（assumed 为 1，PTX ISA §9.7.15.5.1.2.1.1），填 16 仅为占位、使编码值为 1 满足硬件预期。若为 MN-major 或 INTERLEAVE 布局，LBO 有实际含义：INTERLEAVE 布局是同一 8×2 brick 内第一列到第二列的字节偏移；MN-major swizzle 是首 (swizzle-byte-size/16) 行到下一组行的偏移

common.cuh L612-631（make_smem_desc 下半：SBO 与 layout_type）

```
  // ── bits [32, 46): stride_byte_offset ──
  // 原始值 1024（字节），编码后为 (1024 & 0x3FFFF) >> 4 = 64。
  // << 32 将编码值放到 bits [32, 46)。
  // K-Major 下定义为"从首 8 行到次 8 行的字节偏移"（PTX ISA §9.7.15.5.1.2.1.2）。
  // 1024 的来源（K-Major + 128B swizzle + half）：
  //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
  //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
  // 若为 MN-Major: SBO 是从首列到下一列的偏移（8 列一组）。
  desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;

  // ── bits [62, 64): layout_type (swizzle mode) ──
  // 1llu << 62  = 二进制 01 在 bits [62, 64) = layout_type = 1。
  // 1 = 128B swizzle mode。
  // 其他取值：0=None, 2=64B swizzle, 3=32B swizzle。
  // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射，
  // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
  desc |= 1llu << 62;

  return desc;
}
```

逐字段拆解（编码函数 $\mathrm{enc}(x)=(x\,\&\,0\text{x}3\text{FFFF})\gg 4$，对三个地址/偏移字段统一适用）：

- **bits $[0,14)$：start_address**。填的是 `__cvta_generic_to_shared` 得到的 smem **绝对**偏移再编码。右移 4 位的前提是地址 16B 对齐（低 4 位恒 0，PTX §9.7.15.5.1 要求矩阵起始 16B 对齐）；$14+4=18$ bit 可表达 256KB smem 空间，覆盖 H100 的 228KB 上限。编码时 `& 0x3FFFF` 先截断到 18 bit，防止高位污染相邻字段；
- **bits $[16,30)$：leading_byte_offset（LBO）**。K-major + swizzle 布局下硬件**不使用**该字段（assumed 为 1），源码填 16B 使编码值恰为 1；在 MN-major swizzle 布局下它表示「首组 swizzle-byte-size/16 行到下一组行的偏移」，interleave 布局下是 8×2 brick 内两列的偏移；
- **bits $[32,46)$：stride_byte_offset（SBO）**。K-major 下定义为「首个 8 行到下一个 8 行的字节偏移」。128B swizzle 的原子是 8 行 × 64 half（以 128-bit 元素为单位是 8×8，half 的归一化因子 $T=128/16=8$），故 SBO $=8\times 64\times 2\,\mathrm{B}=1024$，编码值 $1024\gg 4=64$；

- **bits** $[49,52)$**：base_offset**。swizzle 模式周期起点到 tile 基址的相位差：128B swizzle 的重复模式以 1024B 为周期，tile 起点**恰在 1024B 边界**时为 0；否则按 $(\text{addr} \gg 7)\,\&\,7$ 计算 3-bit 偏移（该公式对 32B/64B/128B 三档 swizzle 通用，no-swizzle 模式下字段无效）。本 kernel 填 0 并配合 `__align__(1024)` 的声明；源码注释同时给出另一种做法：start_address 全量编码绝对地址后，硬件可自行推导相位补偿，`__align__(128)` 也能跑（健壮性取舍见 13.8 节坑五）；
- **bits** $[62,64)$**：layout_type**。0 = 无 swizzle、1 = 128B、2 = 64B、3 = 32B。填 1 与 TMA descriptor 的 `CU_TENSOR_MAP_SWIZZLE_128B` 严格对应，**写入端与读取端必须声明同一档 swizzle**，否则读到的是被 XOR 打乱的数据。

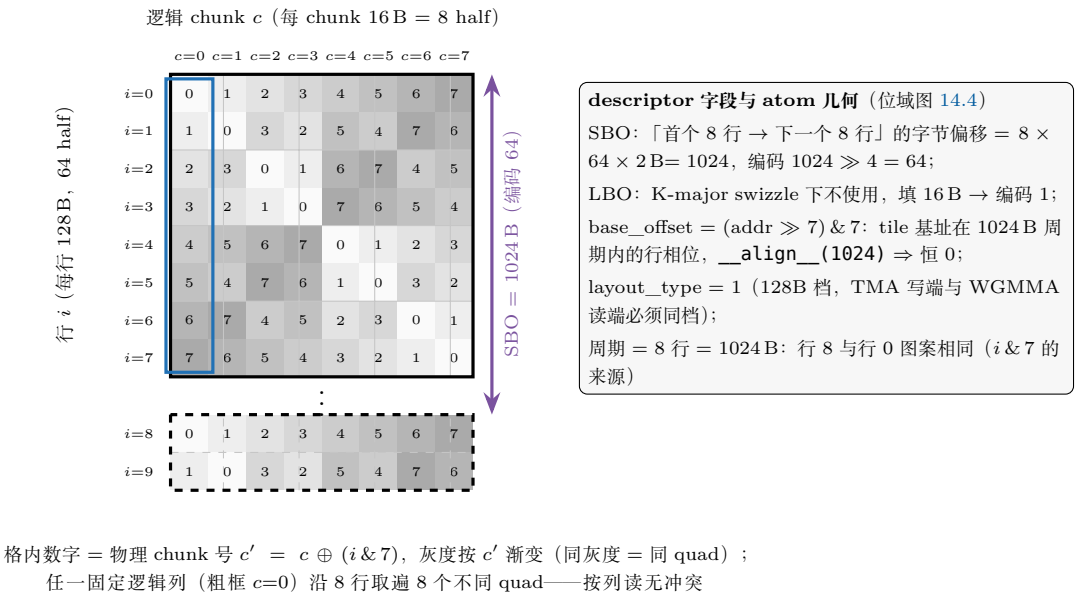


图 14.3: 128B swizzle 的 atom 几何与 WGMMA descriptor 字段对应（13.4.5 节）：atom 为 8 行 × 8 chunk（每 chunk 16 B，即 8 × 64 half），TMA 写入后物理 chunk 号 $c' = c \oplus (i\,\&\,7)$——固定逻辑列（粗框 c=0）沿行取遍 8 个 quad，按列读无 bank 冲突；周期 8 行 = 1024 B（行 8 与行 0 同图案），对应 SBO = 1024 B（编码 64）、base_offset 为 tile 基址在周期内的行相位（`__align__(1024)` 时恒 0）。位域编码见图 14.4；CuTe 的 SW128 原子视角（第 二十四 章图 24.4）与本章互为对照。

另一个高频考点是 **SS 与 RS 模式**：A 操作数可以来自 smem（SS 模式，用 descA 描述，本 kernel 的选择）或来自寄存器（RS 模式，A fragment 以 4 个 `.f16x2` 寄存器直接内联进指令，省一次 smem 读但要求软件自行摆放布局）；**B 操作数永远必须在 smem**（RS 模式下 descB 仍要构造）。指令尾部五个立即数 scaleD, scaleA, scaleB, transA, transB 分别是：累加/清零（K 维迭代必须用 1）、A/B 符号翻转（±1）、A/B 的 major-ness（0 = K-major，1 = M/N-major）。本 kernel 全部取 $(1,1,1,0,0)$。

14.4.6 wgmma.fence / commit_group / wait_group 三件套

common.cuh L539-552（WGMMA 异步三件套宏 + 编码函数）

```
#define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
#define WGMMA_COMMIT_GROUP()                                                    \
  asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
#define WGMMA_WAIT_GROUP(n)                                                     \
  asm volatile("wgmma.wait_group.sync.aligned %0;\n" ::"n"(n) : "memory")

// SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
// 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
//   encoded = (x & 0x3FFFF) >> 4
// 其中 0x3FFFF = 2^18 - 1 = 262143，只保留低 18 bit。
// >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4) ，低 4 bit 恒为 0,
```

```
// 可省略以扩大可表示范围。
// 解码：decoded = encoded << 4（回到原始 byte 值）。
#define SMEM_DESC_ENCODE(x) ((((uint64_t)(x)) & 0x3FFFF) >> 0x4)
```

WGMMA 是 fire-and-forget 指令，正确用法是固定四步：先 `fence`，再发射 WGMMA，再 `commit_group`，最后 `wait_group` 等待完成。`wgmma.fence.sync.aligned` 建立「warpgroup 此前对这些寄存器的访问」与「WGMMA 对同一批寄存器的异步访问」之间的 **寄存器序**，发射前不动累加器、等待完成前不再改写；官方文档同时强调它在 warpgroup 内必须对齐执行（.aligned 限定符），且连续同 shape 的 WGMMA 共享累加器时默认有序、无需重复 fence。`wgmma.commit_group` 把此前未提交的 WGMMA 打包成一个 group（可为空组）；`wgmma.wait_group` N 阻塞到「最近 N 个 group 之外全部完成」。注意语义是**至多保留 N 个 pending** 而非「等第 N 个」，与 `cp.async.wait_group` 完全同构（源码注释专门做了这组对照，核校无误）。双缓冲时可以 commit 两组、wait(1)，让上一组计算与下一组发射重叠；本 kernel 每轮只 commit 一组，故 wait(0) 全收。

14.4.7　kernel 主体：consumer 四步与写回

第一步 C1（wait full，等数据也等相位）是 `full[stage].wait(full[stage].arrive())`：等待 TMA 数据就绪的 full 信号。这行「wait 自己的 arrive」需要把 mbarrier 的到达次数拆开数才能理解：

- 128 个 consumer 线程各调用一次 `arrive()`，共 128 次到达；
- 当前 phase 累积 128/129，phase 尚未翻转；
- `wait(token)` 阻塞，直到 producer 的 `barrier_arrive_tx`（P3）贡献第 129 次到达、且 TMA 字节全部写完 ⇒ phase 翻转，`wait` 返回。

语义即 consumer 先报告「我准备好读当前 stage 了」，再等待「数据已到」。

hgemm.cuh L1724-1763（C2：fence 后发射 2×4 条 m64n128k16）

```
      // Step C2: 发射 WGMMA 指令序列
      //
      // WGMMA 是异步指令（fire-and-forget），发射后立即返回，不等待计算完成。
      // 标准流程：FENCE → 发射 WGMMA → COMMIT → WAIT。
      //
      // WGMMA_FENCE（wgmma.fence.sync.aligned）：
      //   - 确保 TMA 写入 smem 的数据对 async proxy（WGMMA）可见
      //   - 确保累加器寄存器 d[] 已准备好接收 WGMMA 输出
      //   - 本质是 proxy 间的内存序（memory ordering），不是传统 __syncthreads
      //
      // 每个 WGMMA_M64N128K16_F16F16F16 指令：
      //   D[64×128] += A[64×16] × B[16×128]
      //   A/B 均为 K-major（imm-trans=0），由 descA/descB 描述 smem 中的布局。
      //   ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代，每次都用 ScaleD=1。
      WGMMA_FENCE();

      // M 维迭代：BM/kWgmmaM = 128/64 = 2 个 WGMMA atom
      // 每个 atom 处理 64 行 M 方向数据。
#pragma unroll
      for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
        // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
        // s_a 布局：[kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
        half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;

        // K 维迭代：BK/kWgmmaK = 64/16 = 4 次 WGMMA（累加）
        // 每次处理 K=16 维的矩阵乘，4 次累加后覆盖完整的 BK=64。
#pragma unroll
        for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
          // 第 k_step 次 WGMMA:
          //   A: wgmma_sA + k_step * kWgmmaK（A 的 K 维起始位置）
          //   B: s_b + stage * BK * BN + k_step * kWgmmaK（B 的 K 维起始位置）
```

```
          //   注意 B 的 smem 布局是 [BK, BN] row-major，K-major 读取时从
          //   第 k_it*16 行开始取 16 行，每行 BN=128 列。
          //   ScaleD=1: 累加（K 维迭代需要累积结果）
          WGMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
                                     s_b + stage * BK * BN + k_step * kWgmmaK,
                                     1, // ScaleD=1: accumulate（不清零）
                                     1, 1, 0, 0);
        }
      }
```

C2 的双层循环里，A 的地址推进方式如下：`wgmma_sA` 指向当前 stage 中第 m 个 $64 \times B_K$ 子块，K-step 每次前进 `kWgmmaK=16` 个 half；由于 TMA 已把 tile 按 128B swizzle 写好、descriptor 声明了同一档 swizzle，这里**没有任何软件 swizzle 计算**。对比第 十三 章手工 XOR 的 `swizzle<64>()` 调用，「swizzle 从软件层下沉到 descriptor 层」的收益直接体现在这里。B 的物理存储是 $[B_N, B_K]$ row-major（TMA 直写形态），WGMMA 以 K-major 逻辑视图 $[B_K, B_N]$ 读取，「转置」由 128B swizzle 的地址重映射在硬件层完成，无需软件搬运。

hgemm.cuh L1775-1788（C3/C4：commit+wait 全收，arrive empty 释放 stage）

```
      //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group，
      //   必须等它全部完成才能进入下一步（读下一 stage 的数据）。
      WGMMA_COMMIT_GROUP();
      WGMMA_WAIT_GROUP(0);

      // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
      //
      // 128 个 Consumer 线程在 empty[stage] 上 arrive()，为 **下一 phase** 积累到达次数。
      // 这 128 次 arrive 不会立即翻转 phase（还需 Producer 在下一轮的 1 次 arrive）。
      //
      // 语义：Consumer 说"stage stage 我已经读完了，你可以放心覆盖了。"
      [[maybe_unused]] auto token = empty[stage].arrive();
    }
```

hgemm.cuh L1822-1841（epilogue 前半：row/col 映射计算）

```
    const int lane = wg_tid % 32;
    const int warp = wg_tid / 32;
    // row: 当前线程在当前 WGMMA atom 中负责的起始行。
    // warp 0~3 各负责 16 行：rows [warp*16, warp*16+15]。
    // lane/4: 每 4 个连续 lane 负责同一行（因为 WGMMA fragment 中每行有 4 个 uint32，
    //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}，由 lane%4 区分）。
    //         因此 lane/4 给出该行在 16-row block 内的行号（0~7 对应 rows 0~7，
    //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3，都负责 row 0）。
    const int row = warp * 16 + lane / 4;
    // block_C: 当前 C tile 在 global memory 中的起始地址
    // by*BM 是 M 方向偏移，bx*BN 是 N 方向偏移
    half *block_C = C + by * BM * N + bx * BN;

    // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
    // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
#pragma unroll
    for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
      int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
#pragma unroll
      for (int g = 0; g < kWgmmaN / 16; ++g) {
```

写回按式 14.9 的映射展开（写回四象限的双层循环见 L1843–1856）：每线程持 $d[2][8][4]$ 共 64 个 `uint32`，4 次写覆盖 4 个 8×8 象限，每次写一个 `uint32` = 2 个相邻列的 half。128 线程 × 128 half = 16384 = 128×128，恰好铺满一个 C tile。源码 NOTE 处提示了进一步优化方向（R→S→G 走 128-bit 写回），第 十五 章会看到它的实现。

14.5　图示

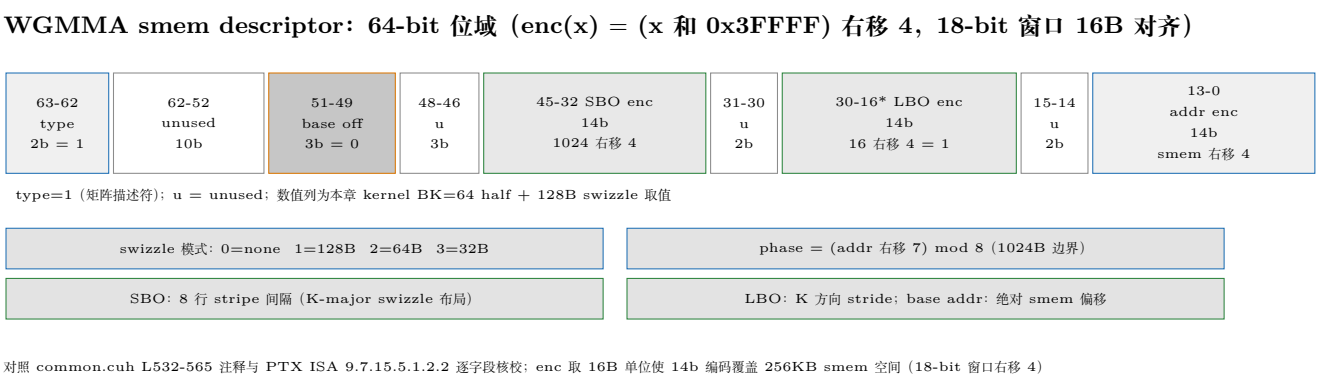


图 14.4: WGMMA smem descriptor 的 64-bit 位域（对照 PTX ISA §9.7.15.5.1.2.2；u = unused；数值列为本章 kernel B_K=64 half + 128B swizzle 的取值；`enc(x)` = (x & 0x3FFFF) 右移 4，18-bit 窗口 16B 对齐）。

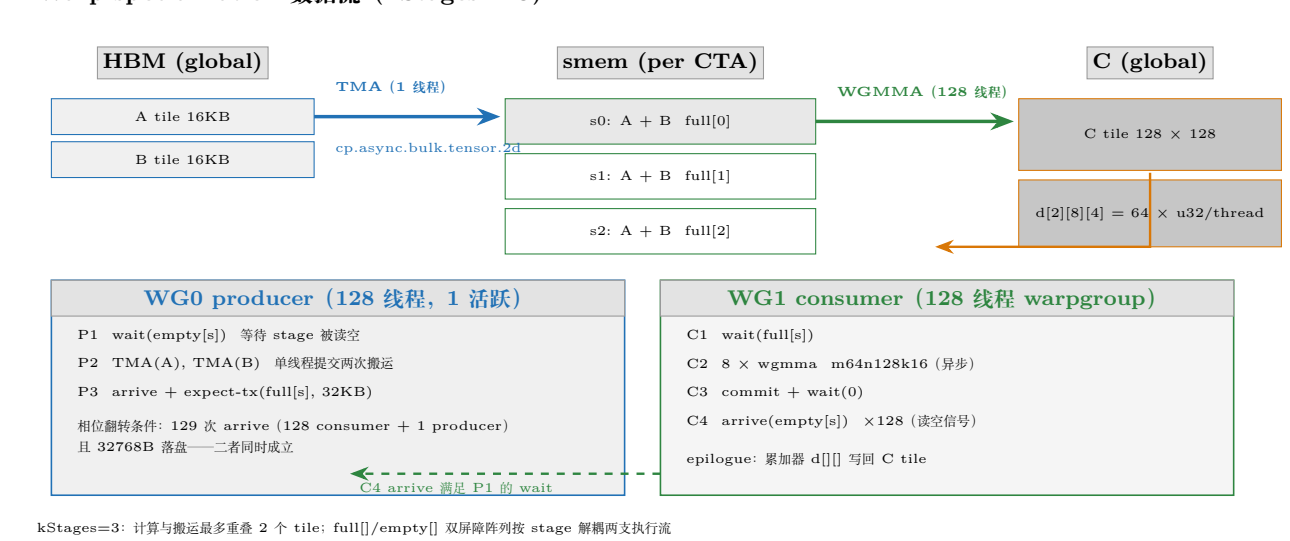


图 14.5: producer/consumer 双 warpgroup 数据流：TMA 单线程提交搬运，WGMMA 全 warpgroup 计算，`full[]/empty[]` 双屏障阵列按 stage 解耦两支执行流，kStages=3 使计算与搬运最多重叠 2 个 tile。

14.6　性能分析

tile 级 roofline（理论推算，非实测）：每 K-tile 搬运 $(B_MB_K + B_NB_K) \times 2\,\mathrm{B} = 32$ KB，计算 $2B_MB_NB_K = 2.1$ M FLOP，算术强度

$$AI = \frac{2B_MB_NB_K}{2\,(B_MB_K + B_NB_K)\cdot 2} = \frac{2\times 128\times 128\times 64}{32768} = 64\ \text{FLOP/B}. \tag{14.10}$$

以 H100 SXM（fp16 dense 约 990 TFLOPS、HBM3 约 3.35 TB/s）计，roofline 拐点在 ≈ 295 FLOP/B：**单看一个 CTA 的装载流，访存仍是短板**。两级补偿：其一，B tile 被同一 tile 行的所有 CTA 复用、A tile 被同 tile 列复用，L2 命中后有效 AI 成倍提高，kBlockSwizzle 用 3D grid 打散 block 改善 L2 局部性的动机即在此（hgemm.cuh L1464–1466 注释）；其二，kStages=3 的深流水让「搬第 i+2 块」与「算第 i 块」重叠，访存延迟不再串行计入关键路径。

指令数与发射开销：一条 `m64n128k16` 承载 262K FLOP，是 `m16n8k16` 的 64 倍；每 K-tile 只需 8 条 WGMMA 加少量 fence/commit/wait 指令，指令发射不再成为 Tensor Core 吞吐的约束。同时 TMA 把「数百条地址整数指令」压缩成 2 条搬运指令，producer 释放的发射槽与寄存器（配合 `setmaxnreg` 可转移百个以上）全部转移给 consumer，warp specialization 的本质就是**把一个 SM 的执行资源按角色重新分配**。

实测口径声明（F4）：本机为 sm_120a（无 Hopper 执行单元），本章未做性能复测；源码头注释与 README 均未给出 WGMMA kernel 的量化 TFLOPS，正文不引用任何未经复现的数字。复现条件：H100/H200/H800 上以 13.10 节命令

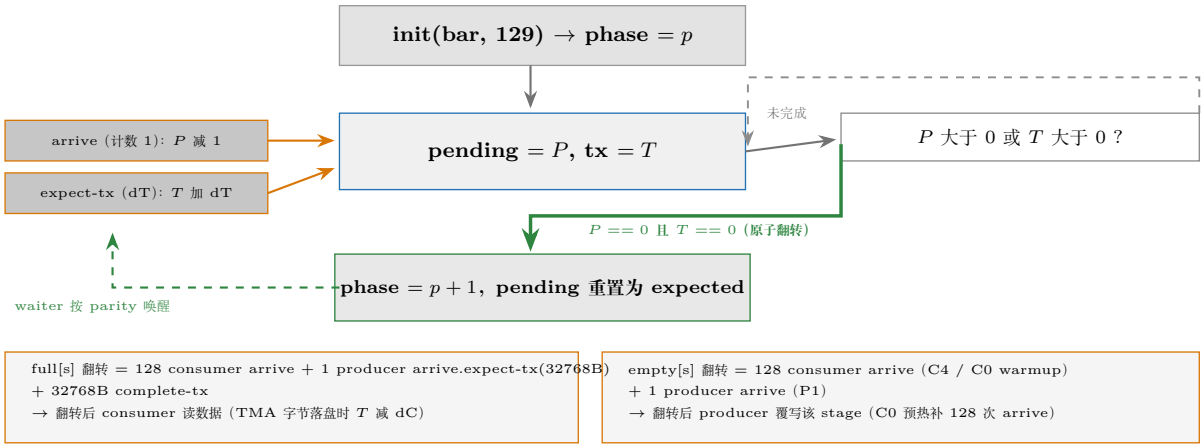


图 14.6: mbarrier 状态机：arrive 与 tx 两个独立的计数条件同时归零才翻转相位；翻转瞬间原子地重置 pending 为 expected。下半部分给出 `full[s]` 与 `empty[s]` 各自的翻转条件。

编译运行并扩展 bench（对照 bench_leetcuda.cu 的 bench 口径），数据落 `.tmp/book-bench/ch13/` 供附录 B 汇总。本章 TMA+mbarrier 协议在 SM120 上配 mma.sync 的实现（TMA+MMA+WS）已在本机实测，见表 15.3（14.6 节）。

14.7 实践经验与考据说明

实践经验 14.2 (F2 核校通过)：common.cuh L558-587 descriptor 位域注释 vs PTX ISA 逐字段对照 PTX ISA §9.7.15.5.1.2.2：start_address $[0,14)$、LBO $[16,30)$、SBO $[32,46)$、base_offset $[49,52)$、layout_type $[62,64)$（0/1/2/3 = 无/128B/64B/32B），编码函数 $(x \,\&\, 0\text{x3FFFF}) \gg 4$，base_offset 在 1024B 边界（128B swizzle）时为 0、否则 $(\text{addr} \gg 7) \,\&\, 7$，源码注释与指令集文档**完全一致**，无需勘误。

实践经验 14.3 (F2 核校通过)：「K-major swizzle 下 LBO assumed 为 1」的表述 PTX ISA §9.7.15.5.1.2.1.1 对 K-major swizzled 布局的 LBO 定义即"not used, assumed to be 1"；SBO 定义"offset from the first 8 rows to the next 8 rows"（§9.7.15.5.1.2.1.2），源码的 $1024 = 8 \times 64 \times 2$ 推导与之一致。common.cuh 与 hgemm.cuh（WgmmaSMem 注释）两处的几何推导均核校无误。

实践经验 14.4 (F2 核校通过)：wgmma.wait_group 与 cp.async.wait_group 的同构表述 PTX ISA §9.7.15.7.3 原文"wait until only N or fewer of the most recent wgmma-groups are pending"，与 cp.async.wait_group（§9.7.9.25.3.3）语义逐词同构；common.cuh L538 与 hgemm.cuh C3 注释的对应表述核校通过。

实践经验 14.5 (F2 核校通过)：mbarrier 双计数与 129 协议 arrive-on 使 pending $-=$ count（默认 1）、expect-tx 使 tx $+=$ 期望值、complete-tx 使 tx $-=$ 完成值、相位翻转条件为双归零，hgemm.cuh 的 barrier 速查注释（L1516 起）与 PTX ISA §9.7.13.15.5–.7 逐条一致；「`mbarrier.arrive.expect_tx` 合并指令的 arrive count 固定为 1」的细节与 §9.7.13.15.13 一致。

实践经验 14.6 (F1 · 工具链实测)：CUDA 13.2：`-arch sm_90a` 完整编译丢失 a 后缀 以 `nvcc -v -arch sm_90a` 完整编译本章测试（2026-09-11，V13.2.51），cicc/ptxas 实际收到 `-arch compute_90`（a 丢失），`wgmma.*` 被 ptxas 以"not supported on .target sm_90" 拒绝；而同参数的 `-ptx` 模式正确产出 `.target sm_90a`。规避：完整编译一律改用 `-gencode` 双参形式（`arch=compute_90a` 与 `code=sm_90a`，build.sh 内的单 arch 命令即此形式）。该事实影响 `book/tests/build_tests.sh`（内部传 `-arch` 形式），13.8 节坑一与 13.10 节命令给出替代方案。

全量核查记录汇总于 `book/CHECKLOG.md`，结论以上述框注为准。

14.8 常见坑

坑一：WGMMA 只认 sm_90a，且 CUDA 13.2 的 `-arch sm_90a` 简写有坑。 WGMMA/mbarrier/TMA 是 sm_90a 专属（PTX 文档 Target ISA Notes 明示 `sm_90a`），sm_100/sm_120 均不兼容，本机 sm_120a 只能做编译级验证。实测 CUDA 13.2 完整编译时 `-arch sm_90a` 会退化成 `compute_90`（见 13.7 节末条勘误），必须改用 `-gencode arch=compute_90a,code=sm_90a`。

坑二：漏掉 C0 预热，kernel 静默挂死。 producer 首轮 `wait(empty)` 需要 129 次 arrive，其中 128 次必须由

consumer 在 C0 预存。漏掉 C0 不会报错，只是 GPU 上的 kernel 永不返回（watchdog 超时）。同理，任何 barrier 的 arrive count 与实际参与线程数不匹配，都会表现为挂死或数据竞争。

坑三：expect-tx 字节数必须与 TMA 实际搬运量严格相等。少报：tx-count 永不归零，相位不翻转，consumer 挂死；多报：tx-count 提前归零，相位在数据写完前翻转，consumer 读到半新半旧的数据，**静默错误比挂死更危险**。本 kernel 两次 TMA 之和为 $(128 \cdot 64 + 128 \cdot 64) \times 2 = 32768$ B。

坑四：三类 fence 必须齐全。smem 供异步引擎读前的 `fence.proxy.async.shared::cta`（proxy 序）、发射 WGMMA 前的 `wgmma.fence`（寄存器序）、访问累加器前的 `wgmma.wait_group`（完成序）。漏 fence 是未定义行为，ptxas 不检查，运行时也不一定立即报错，属最难排查的一类错误。

坑五：swizzle 对齐与 base_offset。128B swizzle 的周期是 1024B：tile 基址不在 1024B 边界时，要么按 (addr $\gg$ 7) & 7 显式计算 base_offset，要么依赖 start_address 全量编码让硬件自行补偿（本 kernel 的做法，源码注释明确了这条取舍）。而第 十五 章的 ldmatrix 消费者路径没有这种硬件补偿，必须显式声明 `__align__(1024)`，两章对照最能说明「同一块 smem，两类读取方」。

坑六：96KB 动态 smem 要 opt-in。kStages=3 共 96KB，超过 48KB 默认上限，launch 前必须 `cudaFuncSetAttribute` 声明，否则 launch 直接失败。

坑七：descriptor 的维度顺序处处反直觉。globalDim 是 (K, M)、boxDim 是 (B_K, B_M)、tensorCoords 是 (k_0, m_0)，三处全是 minor（连续维）在前；`globalStrides` 又要跳过隐式内维（`+1`）。任何一处弄反，TMA 搬运得到的都是「形状正确、内容错位」的 tile。

14.9　面试要点速查

Q1：TMA 与 cp.async 的本质区别？

A：一条 `cp.async.bulk.tensor.2d` 就能搬完整个 2D tile，TMA 是单线程提交的块级 DMA；对比 cp.async 的线程级 4/8/16B 搬运 + 软件 commit/wait group，TMA 把地址生成、OOB 填充、smem swizzle 全部声明在 descriptor 里，零寄存器开销，完成时硬件自动对 mbarrier 执行 complete-tx 计数。

Q2：mbarrier 的相位翻转条件？

A：pending arrivals 与 tx-count **同时**归零（式 14.6）；翻转原子完成：phase 加一、pending 重置为 expected。等待方按相位奇偶（try_wait.parity）阻塞。本章 `full[s]` 的翻转 = 128 consumer arrive + 1 producer arrive.expect-tx + 32768B 落盘。

Q3：arrive count 为何是 129？

A：128 个 consumer 线程各自 arrive（C1 读数据、C4 释放 stage）+ 1 个 producer 提交线程（P1 确认覆盖、P3 声明事务）。同一计数值让「consumer 全体就绪」与「producer 就位」在同一相位汇合。

Q4：WGMMA smem descriptor 的五个字段与编码函数？

A：start_address $[0, 14)$、LBO $[16, 30)$、SBO $[32, 46)$、base_offset $[49, 52)$、layout_type $[62, 64)$；前三个经 $\mathrm{enc}(x) = (x \,\&\, 0\text{x3FFFF}) \gg 4$ 编码（16B 对齐故丢低 4 位，18 bit 表达 256KB）。K-major + 128B swizzle 下 LBO 未使用（assumed 1）、SBO $= 8 \times 64 \times 2 = 1024$（8 行 stripe 间距）、layout_type $= 1$。

Q5：SS 与 RS 模式？

A：A 可来自 smem（SS，descA 描述）或寄存器（RS，A fragment 以 `.f16x2` 内联）；B 恒在 smem（RS 下 descB 仍要构造）。RS 省 smem 读但需软件摆布局；本 kernel 用 SS，A/B 都由 TMA 写入、descriptor 描述。

Q6：wgmma.fence / commit_group / wait_group 各管什么？

A：fence 管发射前后的**寄存器**序（另需 proxy fence 管 smem 的 generic/async 跨代理可见性）；commit 把未提交 WGMMA 打包成组；wait(N) 等到至多 N 组 pending，与 cp.async.wait_group 同构，双缓冲时可 wait(1) 换重叠。

Q7：为什么 producer 127 个线程闲置还要存在？

A：warpgroup 是 WGMMA/setmaxnreg 等指令的对齐单位（4 连续 warp 整体发射）；warp specialization 以 warpgroup 为粒度划分角色，producer warpgroup 只需 1 个线程提交 TMA，其余线程不执行实际工作、仅满足 warpgroup 对齐要求，并通过 `setmaxnreg.dec` 把寄存器转移给 consumer。

Q8：WGMMA 为什么不能在 sm_120 上跑？

A：Hopper 的 warpgroup 异步 MMA（含 smem 直读操作数）是 sm_90a 的架构特定指令（Target ISA Notes 标注）；Blackwell 走 tcgen05（第五代 Tensor Core + tensor memory）路线，不含 wgmma。SM120 的替代组合是 TMA + `mma.sync`（第 十五 章）。

14.10 最小测试与动手实验

测试文件：`book/tests/ch13_hgemm_wgmma.cu`（逻辑抽取自 bench/hgemm.cu 的 `test_hgemm_wgmma`@L272，参考实现由 cuBLAS F16 换成 CPU fp64）。规模 $M = N = K = 256$：被 $B_M = B_N = 128$、$B_K = 64$ 整除，$K/B_K = 4$ 个 K-tile 恰好覆盖 kStages=3 的 stage 回绕（$0 \to 1 \to 2 \to 0$）；输入压到 $[-0.25, 0.25]$，使 f16 累加误差稳定落在 F16Acc 档 5×10^{-2} 内（误差随累加幅度增长，notes-v2 原版用同量级的 cuBLAS 16F 参考对照）。

本机（sm_120a，无 Hopper）验证二态：**编译级**：device pass 非 sm_90a 直接 #error 拒绝；sm_90a 完整编译通过。**运行级**：运行时精确判定 sm_90 家族后输出 SKIP、退出码 0。注意 `book_require_sm` 的 `>=` 语义无法排除「120 ≥ 90 但无 WGMMA」的假阳性，故测试内用精确家族判断：

在 `book/tests/` 目录下编译运行（CUDA 13.2 完整编译必须用 `-gencode` 形式，`-arch sm_90a` 会退化为 `compute_90`，见 13.7 节勘误；并链接 `-lcuda`，因 `cuTensorMapEncodeTiled` 是 driver API）：

```
$ /usr/local/cuda/bin/nvcc -gencode arch=compute_90a,code=sm_90a \
  -O2 -std=c++17 -I../../../third-party/cutlass/include -I../../.. \
  -lcuda -o .build/ch13_hgemm_wgmma ch13_hgemm_wgmma.cu

$ CUDA_VISIBLE_DEVICES=7 .build/ch13_hgemm_wgmma
--> SKIP(ch13): requires sm_90 (Hopper WGMMA), device is sm_120 (exit 0)
```

在 H100/H200/H800（sm_90a）上同命令编译运行，预期输出 `PASS hgemm wgmma m64n128k16 TMA+WS (F16Acc)`，`max_abs_err` 在 10^{-3} 量级（f16 累加、$K = 256$、输入 ± 0.25），以 `ALL OK` 收尾。动手实验建议：（1）把 expect-tx 字节数故意减半，观察 consumer 挂死（坑三）；（2）删掉 C0 预热循环复现坑二；（3）`kBlockSwizzle` 置 1 观察网格形态变化；（4）把 K 改成 192（3 个 K-tile），覆盖「stage 回绕路径」未被整周期覆盖的场景，检验回绕正确性。

延伸阅读

- @reed《cute 之 TMA Descriptor 编码与隐藏的第 21bit》
  https://zhuanlan.zhihu.com/p/2037200219700449995
  （CUtensorMap 128B 内部位域的逆向分析与 bit21/OOB 语义，本章 13.4.2 节考据框的出处）；
- @ 竹熙佳处《cute 之 TMA Copy》
  https://zhuanlan.zhihu.com/p/2003198909405763007
  （cute 视角的 TMA 拷贝抽象与 descriptor 封装，第 二十四 章的先导）；
- @Frank Wang《Async Copy 及 Memory Barrier》
  https://zhuanlan.zhihu.com/p/685168850
  （cp.async 到 mbarrier 的异步同步体系，本章 13.2 与 13.4.4 节的补充视角）；
- NVIDIA PTX ISA §9.7.15（WGMMA）、§9.7.13.15（mbarrier）、§9.7.9.25（cp.async.bulk.tensor），本章全部指令语义的核查基准（本地镜像：cuda-cpp-kernel skill 的 ptx-docs）。

第十五章　SM120：TMA + mma.sync + Warp Specialization

15.1　本章导读

第 十四 章在 Hopper（sm_90a）上把「TMA 搬运 + mbarrier 计数 + WGMMA 计算」组装成 producer/consumer 双 warpgroup 流水线。本章把同一套基础设施搬到 **SM120**，即消费级与工作站 Blackwell（RTX 5090 / PRO 5000 / PRO 6000，compute capability 12.0）：TMA 与 mbarrier **原样保留**，但 WGMMA （warpgroup 级异步矩阵乘）**不存在**。SM120 没有 Hopper/数据中心 Blackwell 的 warpgroup MMA 路径，也没有 tcgen05 第五代 Tensor Core 的 tensor memory（TMEM）与 thread block cluster；它的 Tensor Core 回到 warp 级同步指令 `mma.sync.m16n8k16`，即第 十二 章使用的同一条指令。于是本章 kernel `hgemm_tma_mma_ws_tn` 组合了新旧两代机制：搬运用 Hopper 的 TMA 协议，计算用 Ampere 时代的 mma.sync + ldmatrix，两者靠 mbarrier 的 full/empty 双屏障阵列解耦。这个组合恰好回答了一个面试高频问题：**warp specialization 的收益到底来自搬运端还是计算端？**本章实测（14.6 节）给出答案，仅升级搬运端与同步协议、计算指令一条不动，2048^3 HGEMM 即从 104.66 提升到 164.64 TFLOPS （PRO 5000，1.57×）。

源码区间：kernel 主体 `hgemm_tma_mma_ws_tn`@L1874 位于 hgemm.cuh L1859–2100；common.cuh L467–517（`setmaxnreg` 与 `NOTES_V2_ENABLE_SETMAXNREGS` 门控）。**编译宏与架构限定**：`NOTES_V2_ENABLE_TMA_MA_WS`，要求 CUDART ≥ 13.0（common.cuh L21–23 直接 #error）；TMA+mbarrier 需 sm_90+，本章在 sm_120a 上编译运行。**前置**：第 十二 章（mma.sync、ldmatrix、fragment 布局）、第 十三 章（XOR swizzle）、第 十四 章（TMA descriptor、mbarrier 事务模型，本章直接复用其协议，不重复推导 descriptor 编码）。

15.2　问题与动机

SM120 的能力是 Blackwell 的一个子集：它有第五代 Tensor Core（与数据中心 GB100 同代，支持 FP4/FP8）、GDDR7 与超大 L2（96–128MB），但**没有** WGMMA、没有 tcgen05、没有 TMEM、没有 cluster 与 multicast TMA。CUTLASS 的 sm120 优化指南总结：sm120 在 warp 组织与 block 限制上更接近 Ada（sm_89），只是额外暴露了 TMA。对 GEMM 而言，这意味着第 十四 章流水线的三层收益必须逐层重估：

- **搬运层（保留）**。TMA 的收益与计算指令完全正交：单线程提交整块 2D tile、地址计算与 OOB 填充零寄存器开销、搬运完成由硬件对 mbarrier 执行 complete-tx 计数。这些收益在 SM120 上全部保留；
- **同步层（保留）**。mbarrier 的 arrive/tx 双计数与相位轮转是 sm_90+ 的通用机制，「数据到齐」语义不依赖任何计算指令；
- **计算层（退化）**。WGMMA 不可用，consumer 只能回到 `mma.sync.m16n8k16` + `ldmatrix`。代价有两处：其一，smem→ 寄存器的搬运重新变成显式指令，且**软件必须自己感知 TMA 写入 smem 时的 128B swizzle 相位**：WGMMA 靠 descriptor 的 base_offset 字段让硬件补偿相位，ldmatrix 路径没有等价机制（14.4.2 节）；其二，warp 级指令的发射开销回到第 十二 章的口径，每个 K-tile 要发射 128 条 HMMA 而不是 8 条 WGMMA。

但退化的计算层并不推翻 warp specialization 的价值：producer/consumer 分工消除的是「搬运与计算竞争发射槽」的结构性开销，这与计算指令是哪一代无关。SM120 的 `setmaxnreg`（寄存器池再分配）**指令存在且可以生效**，但存在一个隐蔽的工程陷阱：当 kernel 里的 `cp.async.bulk.tensor` 把目的地址写成 `shared::cluster` 时，ptxas（CUDA 13.2）会把它当作隐式 extern-call 边界，把同 kernel 的 setmaxnreg **全部静默丢弃**（C7506）；写成 `shared::cta` 则两

种目标后缀（sm_120a/sm_120f）下都完整保留（2026-09-17 PTX 级取证，见坑二）。「一条状态空间后缀决定 80 条指令是否被保留」是一条重要的工程教训，14.3.3、14.8 节展开记录。

15.3 数学原理

15.3.1 SM120 与 Hopper 的能力差集

能力	Hopper sm_90a	SM120	本章对策
TMA（bulk.tensor）	有	有（无 multicast）	原样复用
mbarrier	有	有	full/empty 协议复用
WGMMA（warpgroup MMA）	有	**无**	mma.sync + ldmatrix（ch11）
swizzle 相位补偿	descriptor base_offset	**无**	___align___(1024)
setmaxnreg	可用	支持；避免 cluster 形式 TMA	cta dst + 宏开启
cluster / TMEM	cluster 有 / TMEM 无	均无	不涉及
每 SM 寄存器	65536	65536	池数学相同（14.3.3）
smem / SM	228KB	100KB（L1 共 128KB）	kStages ≤ 3

表 15.1: SM120 与 Hopper 在本章流水线相关能力上的差集（依据 cutlass sm120 optimization guide 与 PTX ISA Target ISA Notes）。

15.3.2 producer/consumer 到达数：逐 phase 计数

mbarrier 的双计数模型（arrive 计数与 tx 字节数同时归零才翻转相位）已在 13.3.2 节形式化，本章直接用其结论给**数字**。每个 stage 配一对屏障：`full[s]`（「第 s 级的数据已就绪」）与 `empty[s]`（「第 s 级已被读空、可覆盖」）。两者的期望到达数相同：

$$E = \underbrace{128}_{\text{consumer 全体线程}} + \underbrace{1}_{\text{producer 提交线程}} = 129, \tag{15.1}$$

即 `init(&full[stage], kConsumerThreads + 1)`（hgemm.cuh L1937）。数字 129 与 ch13 相同只是巧合：那边 consumer 是 128 线程的 WGMMA warpgroup，这边是 128 线程的 4 个 mma.sync warp；协议只认「线程数」，不认指令。

`full[s]` 的 tx 侧：producer 每轮对它声明一次预期事务量

$$T_{\text{full}} = (B_M B_K + B_N B_K) \times 2\,\text{B} = (128 \times 64 + 128 \times 64) \times 2 = 32768\ \text{B}, \tag{15.2}$$

由 A、B 两次 TMA（各 16KB）搬运完成时硬件自动执行 complete-tx 计数。于是 `full[s]` 第 p 相位的翻转条件是：

$$\underbrace{128\ \text{consumer arrive}}_{\texttt{full[s].wait(full[s].arrive())}} + \underbrace{1\ \text{producer arrive}}_{\texttt{arrive.expect_tx}} + \underbrace{32768\,\text{B 落盘}}_{\text{TMA complete-tx}} \quad\Longrightarrow\quad \pi \leftarrow \pi + 1. \tag{15.3}$$

注意 consumer 的 arrive-then-wait 模式：每个 consumer 线程**先**贡献 1 次到达再阻塞等相位，128 次 arrive 是「全体 consumer 都到了屏障前」的证据，加上 producer 的 1 次恰好凑满 129。

`empty[s]` 更微妙，它的 129 次 arrive 在时间上**分两批**到达：

为什么漏掉 C0 预热会挂死：producer 首轮在 `empty[0]` 上只有自己 1 次 arrive，$P = 129 - 1 = 128 \neq 0$ 且无 tx，相位永不翻转，GPU 上表现为 kernel 永不返回。C0（hgemm.cuh L1967–1969）把 S 个 empty 各预存 128 次，producer 的 1 次补满即翻。这套计数规则与 ch13 完全一致，本章不再重复状态机图（见图 14.6），差异只在 consumer 的 128 线程从「WGMMA warpgroup 整体 arrive」变成「4 个 warp 各自 arrive」，对 mbarrier 而言无区别。

15.3.3 setmaxnreg 池数学与满池死锁

warp specialization 的配套机制是**寄存器池再分配**：producer 只提交 TMA、几乎不需要寄存器，consumer 持有 32 个 f16x2 累加器碎片（$4 \times 8 \times 2$ 个 `uint32`/线程）需要更多寄存器。`setmaxnreg.dec` 把本 warp 的寄存器上限降到 N 并把差额还给 **CTA 级寄存器池**，`setmaxnreg.inc` 从池里请求补足到 N。PTX ISA §9.7.19.5 给出四条硬约束：

相位	arrive 来源（共需 129）	翻转后谁受益
$\pi = 0$	C0 预热：128 次 consumer arrive（kernel 启动时） + producer P1 的 1 次	producer 首轮 P1 立即放行
$\pi = 1$	C4：上一轮 128 个 consumer 读空该 stage 后 arrive + producer 下一轮 P1 的 1 次	producer 下一轮 P1 覆盖该 stage
$\pi = 2$	同 $\pi = 1$，循环	…

表 15.2: `empty[s]` 的逐相位到达计数：consumer 的「读空信号」与 producer 的「就位信号」在不同时间点到达，同一计数器把它们合并进同一相位。

立即数 $N \in [24, 256]$ 且为 8 的倍数；warpgroup 内 4 个 warp 必须**对齐执行同一条**指令（.aligned）；.dec 的目标值不得超过当前值、.inc 不得超过（否则未定义行为）；最关键的一条是 `setmaxnreg.inc` 的**阻塞语义**：池中寄存器不够时 warp 停在原地，不报错、不超时、永不返回。

每 SM 寄存器文件共 65536 个 32-bit 寄存器，CTA 池从 warp 已声明配额中回收。于是任一 warp specialization 配置必须满足**池不等式**：

$$\sum_{\text{warp } w} 32 \times r_w \;<\; 65536, \qquad \text{且留} \;\geq 2048 \text{ slack（工程安全界）}, \tag{15.4}$$

其中 r_w 是 warp w 经 setmaxnreg 调整后的每线程寄存器数。本章源码的取值是 producer `dealloc<40>`、consumer `alloc<232>`：

$$\underbrace{4 \times 32 \times 40}_{\text{producer WG}} + \underbrace{4 \times 32 \times 232}_{\text{consumer WG}} = 5120 + 29696 = 34816 \;<\; 65536, \tag{15.5}$$

余量 30720，满足 slack 要求。**满池即死锁**的一个真实案例（ffpa-attn 调优实录）：producer `dealloc` 到 64（$4 \times 32 \times 64 = 8192$）+ 双 consumer 共 8 个 warp `alloc` 到 224（$8 \times 32 \times 224 = 57344$），$8192 + 57344 = 65536$ **恰好占满**，而驱动/SM 对寄存器文件有保留份额，有效池 < 65536，consumer 的 `alloc 224` 永远凑不齐，warp 阻塞在 setmaxnreg 上；producer 等 consumer 的 barrier arrive，consumer 卡在 setmaxnreg，**双向互等，构成无报错死锁**：无 illegal instruction、无 error code、watchdog 超时前没有任何征兆。而 alloc 从 224 降到 208（总 61440，余 4096）后运行正常。诊断口诀：kernel 无报错挂起、唯一变量是 setmaxnreg 数字时，先查池数学。

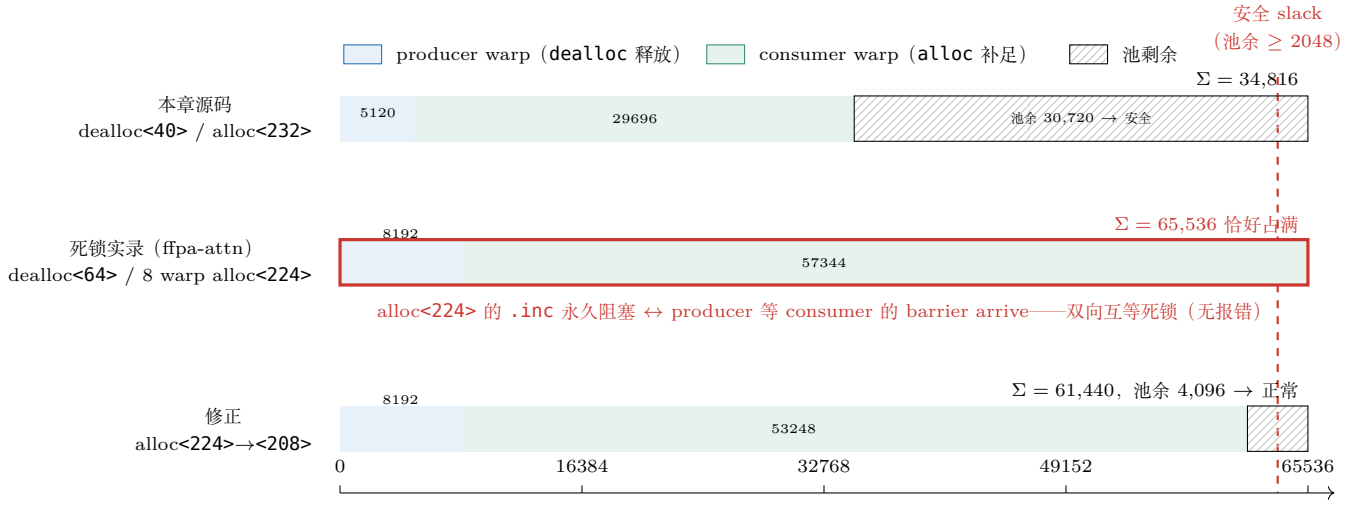


图 15.1: setmaxnreg 的 CTA 寄存器池三配置对照（式 15.4，每 SM 寄存器文件共 65,536 个 32-bit 寄存器）：条形按 $\sum_w 32\,r_w$ 堆叠 producer（dealloc 释放）与 consumer（alloc 补足）两段，红色虚线为工程安全界（池余 ≥ 2048）。① 本章源码 dealloc<40>/alloc<232>：$5120 + 29696 = 34816$，池余 30720，安全；② ffpa-attn 调优实录的死锁配置 dealloc<64> + 8 warp × alloc<224>：$8192 + 57344 = 65536$ 恰好占满——有效池 < 65536，`.inc` 永远凑不齐而阻塞在 setmaxnreg 上，与 producer 等 barrier arrive 形成双向互等死锁；③ alloc 降到 208：总 61440、池余 4096，恢复正常。

Blackwell 上的 TMA 地址空间陷阱与解法：即便池数学正确，还有一道隐蔽的门槛：CUDA 13.2 的 ptxas 在 kernel 同时含有 setmaxnreg 与目的地址为 `shared::cluster` 的 `cp.async.bulk.tensor` 时，会把后者当作隐式

extern-call 边界，将 setmaxnreg 连同警告 C7506（"ignored to maintain compatibility into 'extern' call"）一起丢弃。历史上这里曾发生过一次误诊：本仓手写 raw-asm 的 `tma_load_2d`（沿袭 Hopper pingpong 示例的 cluster 写法）在 sm_120a/sm_120f 下都受影响，而 CuTe 路径 kernel（cute 在 SM120 门控下发射 `shared::cta`）不受影响。「被保留的 kernel 恰好都用了 sm_120f 构建」的巧合，让旧版本文档得出了「必须 family-specific 目标」的错误结论。2026-09-17 的 PTX 级取证证明：把指令改成 `shared::cta` 后，**sm_120a 与 sm_120f 都完整保留 setmaxnreg**（112/112），arch 后缀根本不是变量；完整证据链见 14.8 节坑二的取证记录。本仓 kernel 从不使用 cluster launch 与分布式 smem，dst 恒为 CTA 本地，两种写法语义等价。这也是 `NOTES_V2_ENABLE_SETMAXNREGS` 宏门控（默认关闭）存在的原因：默认构建保持无警告、也避免 `__launch_bounds__(N,1)` 副作用；要做 setmaxnreg 实验，用 build.sh 的 sm_120f 目标（预设该宏与 raw-PTX/cta 路径）构建即可。PTX ISA 的 Target ISA Notes 确认 setmaxnreg 的支持列表覆盖 sm_90a、sm_100a/sm_110a/sm_120a 架构目标与 sm_100f/sm_110f/sm_120f 家族目标（sm_100f/sm_120f 自 PTX 8.8、sm_110f 自 PTX 9.0 起），指令集对两种后缀都支持，真正丢弃指令的是 ptxas 对 cluster 形式 TMA 的边界认定，不是目标后缀。

15.3.4 TMA box：回指与本章坐标

TMA box 的几何（descriptor 的 boxDim、tensorCoords、minor 在前约定）已在 13.3.1 节展开，这里只给出本章数字。A 是 $[M, K]$ row-major，descriptor 视角 globalDim $= (K, M)$、boxDim $= (B_K, B_M) = (64, 128)$；每个 (m_0, k_0) 处的 box 搬运 $\{A[m_0 + i, k_0 + j] : 0 \le i < 128,\ 0 \le j < 64\}$ 共 16KB。producer 第 k 轮提交的两个 box 左上角是

$$A:\ (k_0, m_0) = (k \cdot B_K,\ \mathrm{by} \cdot B_M), \qquad B^T:\ (k_0, n_0) = (k \cdot B_K,\ \mathrm{bx} \cdot B_N), \tag{15.6}$$

即行坐标固定在本 CTA 的 tile 行、列坐标随 K 循环推进（B 以转置形态 $[N, K]$ row-major 存储，N 当作它的 major 维）。每个 box 落进 smem 时被 CU_TENSOR_MAP_SWIZZLE_128B 按 128B 原子重排，重排公式与 consumer 的 `swizzle<64>()` 必须逐位互逆，这是 14.4.6 节的主题。

15.4 设计与实现

15.4.1 kernel 签名与 tile 层级

hgemm.cuh L1863-1893（模板参数即 tile 层级，MMA atom 与 ch11 相同）

```
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim
          const int kMmaTileM = 2,          // consumer warps along M, warp tile M = 32
          const int kMmaTileN = 2,          // consumer warps along N, warp tile N = 16
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 8,          // value-repeat along N, BN = 8*2*8 = 128
          const int kValTileK = 4,          // MMA_K slices per BK tile, BK = 16*4 = 64
          const int kStages = 2,            // TMA full/empty pipeline depth
          const int kNumThreads = 256,      // 128 producer + 128 consumer threads
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(kNumThreads, 1)
    hgemm_tma_mma_ws_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB) {

  constexpr int BM = kMmaM * kMmaTileM * kValTileM;
  constexpr int BN = kMmaN * kMmaTileN * kValTileN;
  constexpr int BK = kMmaK * kValTileK;
  static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
  static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
  static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
  static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
```

```
26
27    const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
28    const int by = blockIdx.y;
29    constexpr int kConsumerThreads = kNumThreads / 2;
30    static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
31                  "Consumer threads must cover the MMA warp grid");
```

tile 层级的乘法结构与第 十二 章完全相同：$B_M = 16 \times 2 \times 4$、$B_N = 8 \times 2 \times 8$、$B_K = 16 \times 4$，四条 static_assert 把「atom = m16n8k16、consumer 4 warp、tile 恰好 128×128×64」固定为硬约束，因为 TMA descriptor 的 box 与 128B swizzle 都按这套数字硬编码。与 ch11 的分歧点只有一处：256 线程不再是「8 个同质计算 warp」，而是 `kConsumerThreads = 128` 的 consumer warpgroup + 128 线程的 producer warpgroup（实际只有 `wg_tid == 0` 的 1 个线程提交指令）。consumer 内部，4 个 warp 排成 2×2 网格（`warp_m`, `warp_n`），每 warp 负责 64×64 的输出子块：M 方向 $16 \times 4 = 64$ 行、N 方向 $8 \times 8 = 64$ 列。

15.4.2　smem 布局与 1024B 对齐：本章最重要的差异点

common.cuh L714-722（TmaMmaWSSMem：plain 数组 + 128B swizzle 由 TMA 写入）

```
1  #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
2  // SM120 不支持 WGMMA，但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3  // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
4  template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
5    static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
6    half A[BM * BK * QSIZE];
7    half B[BN * BK * QSIZE];
8  };
9  #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
```

hgemm.cuh L1920-1932（___align___(1024) 的 smem 与 full/empty 屏障阵列）

```
1    extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
2    TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3        *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
4    half *s_a = s.A;
5    half *s_b = s.B;
6
7  #pragma nv_diag_suppress static_var_with_dynamic_init
8    __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
9    __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
10 #pragma nv_diag_default static_var_with_dynamic_init
11
12   const int NUM_K_TILES = div_ceil(K, BK);
13   const int wg_idx = threadIdx.x / kConsumerThreads;
```

注意结构体本身只有两个 plain half 数组，**没有软件 swizzle 的写入端**：TMA 在搬运时按 descriptor 声明的 SWIZZLE_128B 模式直接写好重排后的布局，swizzle 从「软件写 + 软件读」下沉为「硬件写 + 软件读」。而 `__align__(1024)` 划分了本章与 ch13 的界线：128B swizzle 的重排模式以 1024B（8 行 × 128B）为周期，硬件按 smem 目的地址相对 1024B 边界的**相位**决定从周期的哪一步开始；WGMMA 消费端在 smem descriptor 里显式编码 base_offset = (addr ≫ 7) & 7 让硬件补偿任意相位，因此 ch13 只要求 128B 对齐；本章的 ldmatrix 消费端用纯软件公式 `swizzle<64>()` 算物理地址，它**默认相位为 0**，基址只有 128B 对齐时 TMA 以非零相位写入而软件按零相位读出，物理地址整体错位，输出全错（不越界、不报错，完全静默的错误）。所以要么 1024B 对齐（本章做法），要么在软件公式里把基址相位加上（更繁琐且易错）。源码 L1898–1919 的注释对这段取舍有完整展开，建议逐行读。

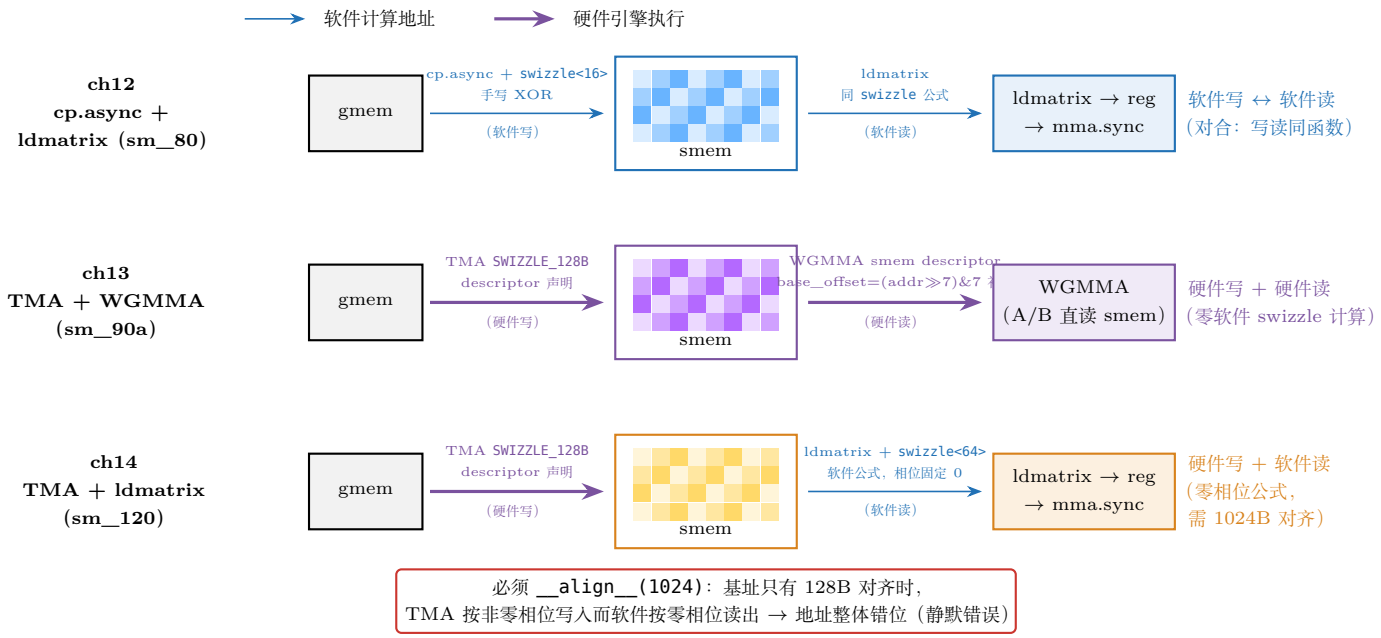


图 15.2: swizzle 的三代演化——写入端与读取端的「软件/硬件」分工（14.4.2 节）：ch12 用 cp.async 手写 XOR 写入、ldmatrix 用同一个 swizzle 公式读出——软件写 ↔ 软件读，靠对合性（引理 13.1）保证互逆；ch13 把两端都交给硬件——TMA 按 descriptor 声明的 SWIZZLE_128B 直接写好重排布局，WGMMA 在 smem descriptor 里编码 layout_type 与 base_offset 由硬件补偿任意相位，零软件 swizzle 计算；ch14 折中——TMA 硬件写 + 软件公式 `swizzle<64>` 零相位读，此时 smem 基址必须 `__align__(1024)`（红框）：否则 TMA 以非零相位写入而软件按零相位读出，物理地址整体错位，输出全错且不报错。

15.4.3 barrier 初始化

hgemm.cuh L1934-1945（单线程初始化 129 计数 + proxy fence + syncthreads）

```
  if (threadIdx.x == 0) {
    for (int stage = 0; stage < kStages; ++stage) {
      init(&full[stage], kConsumerThreads + 1);
      init(&empty[stage], kConsumerThreads + 1);
    }
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  if (wg_idx == 0) {
    // 生产者 Warpgroup (wg_idx==0)
```

初始化三点：其一，cuda::barrier 的 `init(&full[s], 129)` 展开为 `mbarrier.init.shared::cta.b64 [addr], 129`，两个屏障阵列的期望到达数都是式 15.1 的 129；其二，尾部 `fence.proxy.async.shared::cta` 建立 generic proxy（软件写屏障对象）到 async proxy（TMA/mbarrier 硬件）的可见性，初始化数据不被异步引擎看见，后面的 arrive/tx 全是空转；其三，`__syncthreads()` 是全 kernel **唯一一次**线程块级栅栏，此后 producer 与 consumer 的全部同步都走 mbarrier，这就是 warp specialization 的定义性特征。

15.4.4 producer：三步循环

hgemm.cuh L1944-1961（WG0：dealloc 后单线程 P1/P2/P3 循环）

```
  if (wg_idx == 0) {
    // 生产者 Warpgroup (wg_idx==0)
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
```

```
    int stage = 0;
    for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
      if (stage == kStages)
        stage = 0;

      empty[stage].wait(empty[stage].arrive());

      tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
                  full[stage]);
      tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
                  full[stage]);
      tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
    }
  }
```

P1 `empty[stage].wait(empty[stage].arrive())`：先 arrive 后 wait，producer 的 1 次到达补足 empty 相位翻转所需的第 129 次 arrive（表 14.2 的逐相位计数），翻转即代表「上一轮 consumer 已读空该 stage」。P2 两条 `tma_load_2d`：A box 从 $(k\cdot 64,\ \mathrm{by}\cdot 128)$、$\mathrm{B}^T$ box 从 $(k\cdot 64,\ \mathrm{bx}\cdot 128)$，各 16KB，目标分别是 `s_a`、`s_b` 在该 stage 的偏移，完成时自动对 `full[stage]` complete-tx。P3 `tma_arrive_expect_tx` 一条指令同时完成「+1 到达」与「T += 32768」（式 15.2）。整轮 producer 只发 3 类共 3–4 条指令，寄存器占用几乎为零，`dealloc` 到 40 留有充分余量。

15.4.5　consumer：C0 预热与无 ___syncthreads 的主循环

hgemm.cuh L1962-1991（WG1：alloc、C0 预 arrive、wait full + proxy fence）

```
} else {
  // 消费者 Warpgroup (wg_idx==1)
  // 初始化: 标记所有 stage 为"空" (可被 Producer 写入)
  NOTES_V2_REG_ALLOC(232);

  for (int stage = 0; stage < kStages; ++stage) {
    [[maybe_unused]] auto token = empty[stage].arrive();
  }

  const int warp_id = wg_tid / kWarpSize;
  const int lane_id = wg_tid % kWarpSize;
  // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
  const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
  const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
  uint32_t RA[kValTileM][4];
  uint32_t RB[kValTileN][2];
  uint32_t RC[kValTileM][kValTileN][2] = {0};

  const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
  const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
  int stage = 0;
  for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
    if (stage == kStages)
      stage = 0;

    full[stage].wait(full[stage].arrive());
    tma_fence_proxy_async_shared_cta();

    // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
    // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
```

C0 预热（L1967–1969）的 128 次 arrive 已在 14.3.2 节计入。主循环头（L1987–1988）是 consumer 版的 arrive-then-wait 加一条 `tma_fence_proxy_async_shared_cta()`。为什么整条 consumer 路径 **不需要 ___syncthreads**？源

码 L1990–2013 的注释给了三条理由，归纳：(1) 搬运完成由 `full[s]` 的 tx 计数保证，`wait` 返回时硬件已确认 32768B 真正落进 smem，这是 CTA 级同步语义，覆盖了 cp.async 路径里 `__syncthreads` 的职责；(2) async proxy 写（TMA）到 generic proxy 读（ldmatrix）的**内存序**由 `fence.proxy.async` 建立，它不阻塞、不等待任何线程，只约束两个代理间的可见顺序；(3) 4 个 consumer warp 读写 smem 中**互不重叠**的 A/B 子块（`warp_m`/`warp_n` 划分），warp 间没有数据依赖，mma.sync 本身保证 warp 内 32 线程同步。三条合起来：同步有 barrier、顺序有 fence、依赖不存在，`__syncthreads` 没有存在的必要。

15.4.6 swizzle<64> 与 k_step 的位置：与 ch12 的关键分歧

hgemm.cuh L2015-2036（k_step 循环与 A 的 ldmatrix：偏移进 swizzle 公式）

```
      for (int k_step = 0; k_step < kValTileK; ++k_step) {

#pragma unroll
        for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
          // kMmaM * kValTileM = 16*4 = 64，每个消费者 warp 在 M 方向覆盖 64 行。
          const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
          const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
          // 与 hgemm_mma_stages_tn_swizzle 不同：k_step * kMmaK 必须放在
          // lane_smem_a_k 中传给 swizzle<64>()，而非像 swizzle kernel
          // 那样作为 smem_k_offset 加在 swizzle 外部。原因：
          //   swizzle<64> 作用于完整 BK=64，swizzle 周期覆盖全部 64 列，
          //   k_step=0 和 k_step=1 的 chunk 会被 XOR 交叉混合 → k_step
          //   偏移必须在 swizzle 内部参与 chunk 计算。
          //   swizzle<kMmaK> 作用于 kMmaK=16，每个 kMmaK slice 独立
          //   swizzle，slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
          const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
          const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
              (stage * BM * BK + lane_smem_a_m * BK +
              swizzle<64>(lane_smem_a_m, lane_smem_a_k)) *
                  sizeof(half);
          LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
        }
```

第 十三 章的 swizzle 作用于 $B_K = 16$ 列的 tile，每个 k-slice 独立 swizzle、slice 之间互不跨越，所以 `k_step * kMmaK` 可以作为普通偏移加在 swizzle 结果**外部**。本章 $B_K = 64$，`swizzle<64>` 的 XOR 周期覆盖整行 64 列（8 个 16B chunk 一起参与置换），k_step 偏移的高位会改变 chunk 编号，所以必须把**完整列坐标** $j = k_{step} \times 16 + (\text{lane}/16) \times 8$ 传入公式**内部**参与 XOR。公式本身（common.cuh L177–186 的 `permuted<64,8>`）：

$$j' = j + \Big(\big((j \gg 3) \oplus \sigma(i)\big) - (j \gg 3)\Big) \times 8, \qquad \sigma(i) = (i \,\&\, 1) \,|\, \big((i \gg 1) \,\&\, 1\big) \ll 1 \,|\, \big((i \gg 2) \,\&\, 1\big) \ll 2, \tag{15.7}$$

即 16B 粒度的 chunk 号 $(j \gg 3) \in [0, 8)$ 与行号的低 3 位做 XOR、chunk 内偏移 $j \,\&\, 7$ 保留。公式就是 CuTe `Swizzle<3,4,3>`（SWIZZLE_128B 模式）的位级定义，与 TMA 写入端的重排严格互逆。行号 i 与列号 j 的来源：`lane_smem_a_m = warp_m` 的 64 行基址 + `lane % 16`，`lane_smem_a_k = k_step * 16 + (lane / 16) * 8`，配合 ldmatrix.x4 的地址模式（每 lane 提供一个 8 行片的首地址，4 个 8×8 矩阵一次加载）与 ch11 完全一致，此处不再展开。

15.4.7 B 的加载与 mma 发射、epilogue

hgemm.cuh L2038-2058（B 的 ldmatrix.x2 与 4×8 条 HMMA16816）

```
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
          // kMmaN * kValTileN = 8*8 = 64，每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
          const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
          const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
          const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
```

```
          const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
              (stage * BN * BK + lane_smem_b_n * BK +
              swizzle<64>(lane_smem_b_n, lane_smem_b_k)) *
                  sizeof(half);
          LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
        }

#pragma unroll
        for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileN; ++j) {
            HMMA16816(RC[i][j][0], RC[i][j][1],
                      RA[i][0], RA[i][1], RA[i][2], RA[i][3],
                      RB[j][0], RB[j][1],
                      RC[i][j][0], RC[i][j][1]);
```

每个 k_step：4 次 LDMATRIX_X4（A，4×4 寄存器）+ 8 次 LDMATRIX_X2（B^T，8×2 寄存器）+ $4\times8=32$ 条 HMMA16816，每 warp 每 k_step 完成 $64\times64\times16$；一轮 K-tile 4 个 k_step，4 个 warp 合计 $2\times128\times128\times64$ FLOP，与一个 16KB+16KB 的 TMA 装载量对应，算术强度 64 FLOP/B（同 ch13）。循环尾 empty[stage].arrive()（L2062）是 C4：128 线程各贡献 1 次到达，为 empty 的**下一相位**累计到达数（表 14.2 的 $\pi=1$ 行）。

hgemm.cuh L2064-2098（epilogue：shuffle 汇聚 fragment + float4 写回）

```

    // Epilogue: 复用 RA[0] 和 RA[1]（已在寄存器中）作为 shuffle 缓冲，
    // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
    // lane 各持有两个 uint32 fragment；shuffle 汇聚为每行一个 float4，
    // 由 lane 0 发出对齐的 128-bit store。
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
      const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        RA[0][0] = RC[i][j][0];
        RA[1][0] = RC[i][j][1];
        RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
        if (lane_id % 4 == 0) {
          const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
          const int store_gmem_c_addr_0 =
              store_lane_gmem_c_m * N + store_lane_gmem_c_n;
          const int store_gmem_c_addr_1 =
              (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RA[0][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RA[1][0]);
        }
      }
    }
  }
}
```

epilogue 与 ch11/ch12 相同：mma m16n8k16 的 C fragment 中，同一 8×8 矩阵的一行 2 个 f16x2 分布在 4 个相邻 lane（行 = lane/4），通过 __shfl_sync 把 4 个 lane 的寄存器汇聚到 lane%4==0 的线程，拼成连续 8 个 half

的 16B `float4` 写回 gmem，两个 16B store 覆盖 (r, c) 与 $(r{+}8, c)$ 两个行片。这一步是第 十四 章末尾预告的「R→G 128-bit 写回」在寄存器直写路径下的实现（不经过 smem 中转）。

15.5 图示

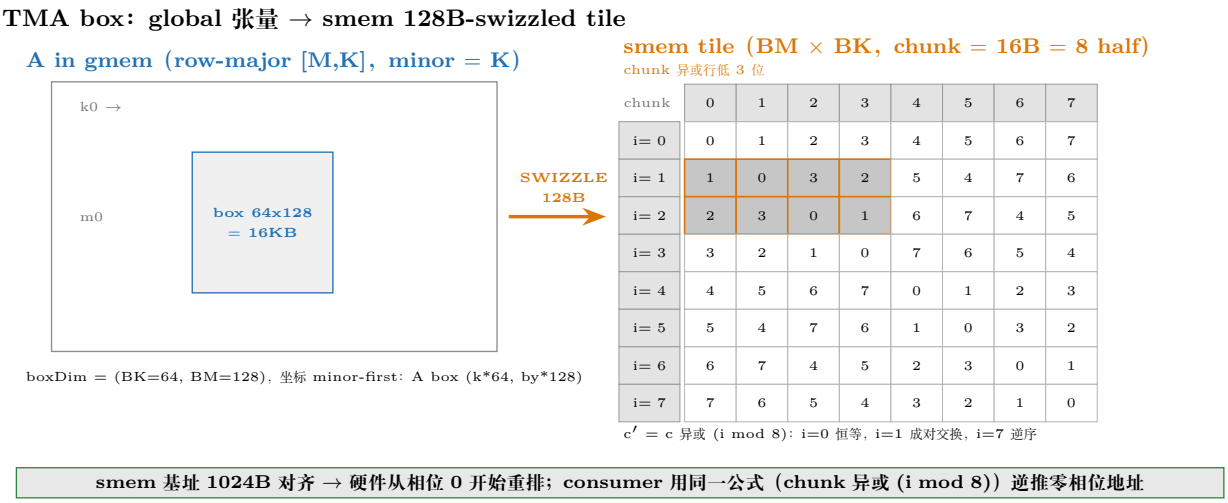


图 15.3: TMA box：descriptor 以 minor-first 坐标指定 64×128 的 2D 搬运块，落盘时按 SWIZZLE_128B 原子（16B chunk 号与行低 3 位 XOR）重排；smem 基址 1024B 对齐保证硬件从相位 0 开始重排，consumer 的软件公式才能按零相位逆推。

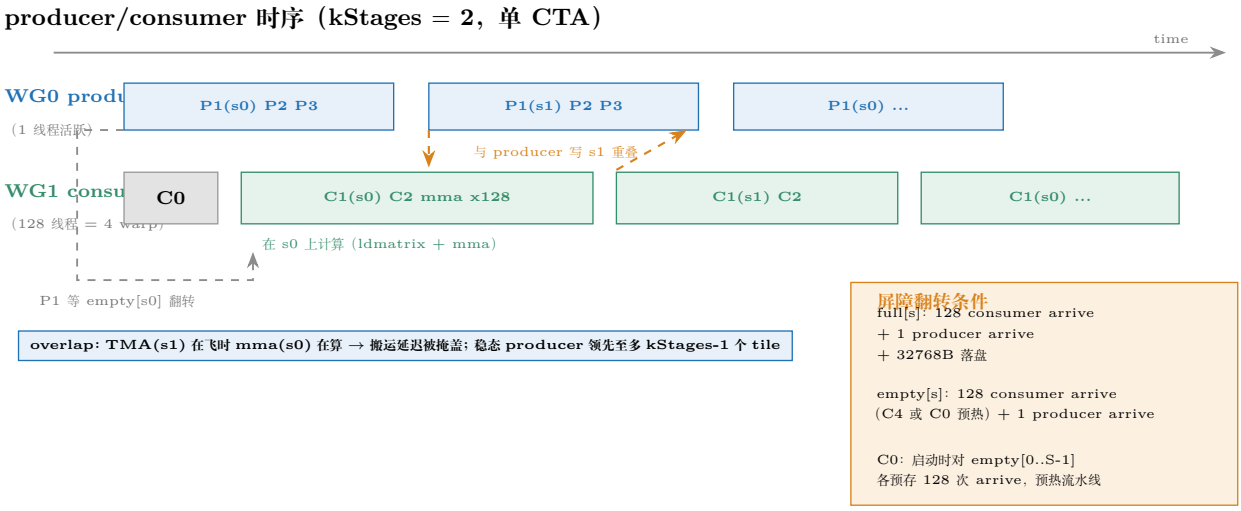


图 15.4: producer/consumer 时序：C0 预热启动流水线；此后 producer 的 TMA 提交与 consumer 的 ldmatrix+mma 在不同 stage 上重叠，搬运延迟被计算掩盖；稳态 producer 领先至多 kStages-1 个 tile。

15.6 性能分析

tile 级 roofline 与 ch13 相同：每 K-tile 装 32KB、算 2.1 M FLOP，$AI = 64$ FLOP/B。按 PRO 5000（110 SMs，sm_120a）与 GDDR7 $\approx$ 1.34 TB/s（1344 GB/s）估算，纯 HBM 供给的计算上限 $\approx 64 \times 1.344$ TB/s $\approx$ 86 TFLOPS。实测 164.64 TFLOPS 已经**越过**这条线，说明 96MB 级 L2 吸收了相当比例的 tile 流量（同行 CTA 复用 B tile、同列复用 A tile），有效带宽远高于 HBM 名义值；这也解释了 `kBlockSwizzle` 3D 网格（L2 局部性）的价值。

实测口径 (F4, 2026-09-11, RTX PRO 5000 72GB, sm_120a, 110 SMs, CUDA 13.2)：M=N=K=2048，f16 in/out + f16 累加（HMMA16816 的.f16 累加形态），cudaEvent 计时，warmup 3 + 平均 10 次：

kernel	ms/iter	TFLOPS
`hgemm_tma_mma_ws_tn`（TMA+WS，S=2）	0.1043	**164.64**
`hgemm_mma_stages_tn`（ch11，cp.async，S=3）	0.1641	104.66

同一颗 GPU、同一个 mma 指令、同规模 tile（128×128），仅把搬运端换成 TMA + mbarrier + warp specialization，即得 1.57×。加速来源分解：（1）producer/consumer 分工后，consumer 循环里不再有 gmem 地址算术与 cp.async 提交指令，发射槽全部留给 ldmatrix 与 mma；（2）TMA 单线程提交替代 256 线程的逐线程搬运，地址/OOB 全部在 descriptor 里预计算；（3）数据驱动的 full/empty 协议替代 `__syncthreads` 栅栏，warp 间不再相互等待。

大矩阵实测（RTX PRO 5000，sm_120，CUDA 13.2，`notes_v2_sm120a.bin --bench --bench-all`，f16 累加，cuBLAS f16 参照 246.6 TFLOPS）：

表 15.3: HGEMM TMA MMA WS 实测（RTX PRO 5000，M=N=K=8192，fp16）

Kernel	Max Err	TFLOPS/cuBLAS（加速比）
HGEMM TMA MMA WS (S=1, BLK_SW=0)	0.000e+00	199.6/246.6 (0.81x)
HGEMM TMA MMA WS (S=1, BLK_SW=1)	0.000e+00	231.4/246.6 (0.94x)

8192^3 规模下 TMA+MMA+WS 组合把纯 MMA 基础实现（第 十二 章 `hgemm_mma_stages_tn` 约 127 TFLOPS 量级）提升 57%–82%，搬运端与同步协议的升级在大矩阵上依然成立；开启 block swizzle（BLK_SW=1）后 f16 累加路径从 0.81x 提升到 0.94x，L2 局部性的收益与上文 2048^3 的结论一致，且两配置 Max Err 均为 0，性能收益未付出精度代价。

15.7　实践经验与考据说明

实践经验 15.1 (F2 核校通过)：「SWIZZLE_128B 要求 smem 基地址 1024 字节对齐」的表述 hgemm.cuh L1898 起的注释如此声明，并区分了 WGMMA（base_offset 硬件补偿，128B 即可）与 ldmatrix（软件公式假设相位 0，须 1024B）两条路径。对照 PTX ISA：`cp.async.bulk.tensor` 对目的地址的硬性要求是对齐到 swizzle 模式宽度（128B），**1024B 并非指令集要求**，而是「让 consumer 的零相位软件公式成立」的充分条件，与 CUTLASS 对 Sw⟨3,4,3⟩ tile 的对齐实践一致。注释逻辑自洽、结论正确，仅「要求」的归属（指令集 vs 软件消费端）宜读作后者。

实践经验 15.2 (F2 · 文档 vs 实践)：cuda.h 注释「tensorRank must be 3, 4, or 5」CUDA 13.2 的 `cuTensorMapEncodeTiled` 头文件注释称 tensorRank 只能是 3/4/5，而 common.cuh 与本章测试都用 rank=2（2D tile），实测编码成功、kernel 数值正确。驱动实践接受 rank 2（CUTLASS 同样大量使用），头文件注释与实现不一致，写 2D TMA 时以实践为准，但升级 CUDA 版本时此处值得回归。

实践经验 15.3 (F1 · 2026-09-17 修正)：setmaxnreg 丢失的归因：地址空间，非目标后缀 旧版注释（common.cuh L467–517）断言「sm_120a 上 ptxas 把 `cp.async.bulk.tensor` 的使用当作隐式 extern-call 边界丢弃 setmaxnreg，改用 `-arch sm_120f` 则保留」。该结论后被 PTX 级取证推翻：真正的触发条件是 TMA 的目的状态空间 `shared::cluster`（两个 arch 目标下都丢），`shared::cta` 则两个目标下都保留（各 112/112）。旧结论是 cluster 写法 +「存活 kernel 恰好用 120f 构建」的巧合共同造成的误诊，ffpa-attn cute kernel 存活的真实原因是 CUTLASS 在 SM120 门控（`CUTE_ARCH_TMA_SM120_ENABLED`）下发射 `shared::cta`。源码注释与 build.sh/README 措辞已于 2026-09-17 同步修正（commit f4b759f）。

实践经验 15.4 (F2 · 已修正)：注释「require ___launch_bounds___(N, 1)」与实现不一致 common.cuh 注释称 setmaxnreg 需要 `__launch_bounds__(N, 1)`（每 SM 1 block，给满每线程 256 regs 的编译许可），而 `hgemm_tma_mma_ws_tn` 当时只写 `__launch_bounds__(kNumThreads)`。2026-09-17 实证了后果：缺第二参数时 PTX 不带 `.minnctapersm`，ptxas 以 C7508（"unable to determine register count at entry"）丢弃 setmaxnreg，与 PTX ISA §9.7.19.5「须以编译期选项/调优指令声明每线程最大寄存器数，否则可能无效」的措辞吻合。该 kernel 已补 `, 1`（f4b759f），与其余 WS kernel 声明一致。

全量核查记录汇总于 `book/CHECKLOG.md`，结论以上述框注为准。

15.8 常见坑

坑一：setmaxnreg 满池死锁，静默挂死无报错。式 15.4 的池数学必须规划期就算：$\sum 32r_w < 65536$ 且留 ≥ 2048 slack。`setmaxnreg.inc` 是阻塞语义，凑不齐就永远停在那，kernel 表现为「无 error、无 illegal access、永不返回」。诊断口诀：唯一变量是 setmaxnreg 数字的挂起，先查池数学。

坑二：setmaxnreg 被 ptxas 静默丢弃（C7506/C7508），触发原因是 TMA 的目的地址空间而非 arch 后缀。kernel 里的 `cp.async.bulk.tensor` 若把目的地址写成 `shared::cluster`，ptxas（CUDA 13.2）会把它当隐式 extern-call 边界，把同 kernel 的 setmaxnreg **全部**丢弃，只留一条 info 级 C7506（默认不打印，`-Xptxas -v` 才可见），功能静默失效；写成 `shared::cta` 则 sm_120a 与 sm_120f 下都完整保留。另有一个独立的丢弃触发：`__launch_bounds__(N)` 缺第二参数时 PTX 不带 `.minnctapersm`，ptxas 以 C7508 丢弃，WS kernel 声明一律写 `(N, 1)`。本章宏门控默认关闭是为了保持默认构建无警告；做实验用 build.sh 的 sm_120f 目标（预设 `NOTES_V2_ENABLE_SETMAXNREGS` 与 raw-PTX/cta 路径）。本坑的完整取证见下节。

取证：一次 `shared::cluster` 引起的 80 条指令静默丢弃

这是 2026-09-17 在本仓真实发生的一次调查，全程只用 ptxas/cuobjdump 与文本工具，方法本身比结论更有参考价值。

现象 `sm_120f` 实验构建（开 `NOTES_V2_ENABLE_SETMAXNREGS`）编译通过、bench 数值全对，但 `cuobjdump --dump-resource-usage` 里 WS kernel 的寄存器数没有出现「producer/consumer 分层」的迹象。展开核查：PTX 层 112 条 `setmaxnreg`，SASS 层只剩 32 条，**80 条被静默丢弃**，且被保留的全部是 CuTe 路径 kernel（`ffpa_attn_cuh` 的 `split_d_cute` 家族），手写 raw-asm 路径（flash_attn FA2/FA3 WS、hgemm WS）全部被丢弃，伴随 `-Xptxas -v` 下 C7506×36 与 C7508×4。

定位方法 第一步，`nvcc -ptx -arch=compute_120f` 生成完整 PTX。注意：`-gencode arch=compute_120f,code=sm_120f` 只产 cubin，fatbin 不嵌 PTX，故 `cuobjdump --dump-ptx` 在链接产物上查不到。确认 112 条都在 `.entry` 里、PTX 层无任何 `call`，排除「内联不足」与「函数边界」两类解释。第二步，脚本把每个 `.entry` 块单独提取后用 `ptxas -arch=sm_120f` 编译，40 个 kernel 精确复现各自的 C7506/C7508。第三步，对其中一个触发 kernel 做**指令删除实验**：按类删除整条指令再重编，删 `mbarrier.*`、`fence.proxy`、cluster barrier 均无效（警告依旧），唯独删除全部 `cp.async.bulk.tensor` 后 C7506 消失。边界就在 TMA 指令本身。

根因 回到 PTX 文本比对被保留与被丢弃的 kernel：CuTe 路径生成

```
cp.async.bulk.tensor.2d.shared::cta.global.mbarrier::complete_tx::bytes ...
```

而手写 `tma_load_2d`（沿袭 Hopper pingpong 示例的写法）是 `shared::cluster.global`。**决定性实验**：把触发 kernel PTX 里的 `cluster` 全局替换为 `cta`，C7506 消失、SASS 里出现 `USETMAXREG`。CUTLASS 源码可交叉印证：`cute/arch/copy_sm90_tma.hpp` 在 `CUTE_ARCH_TMA_SM120_ENABLED` 门控下专门切到 `shared::cta`（sm_90 路径仍发 cluster），上游早已知道这个差别。本仓 kernel 无 cluster launch、无 `mapa`，dst 恒为 CTA 本地 smem，两种写法**语义等价**，仅是指令形式不同而编译结果不同。

反转与教训 修复（`cluster→cta` + hgemm WS 补 `__launch_bounds__(N,1)`）后复测：**sm_120a 与 sm_120f 都 112/112 保留**，此前「必须 family-specific 目标」的结论是 cluster 形式下的假象：当时两组 kernel 一组（手写）用 cluster、一组（cute）用 cta，而保留的那组恰好以 120f 构建，两个变量混在一起，归因归错了列。验证侧：SASS `USETMAXREG`=112=PTX 计数；FA bench Max Err 与基线一致；`--tma-mma-ws 4096` 全 0；sm_90a/sm_120a 默认构建无回归。

排查方法三条 （1）**SASS 助记符是 `USETMAXREG`，没有 `n`**。`grep setmaxnreg` 在 SASS 上全程假阴性，本次调查一度据此得出「连最小 PTX 都被丢弃」的错误结论；跨层数指令先核对两层的助记符拼写（PTX `setmaxnreg` ≠ SASS `USETMAXREG`）。（2）ptxas 聚合的 C750x 警告打印在第一条「Compiling entry function」**之前**，不能按行号就近归属到

相邻 kernel，逐 entry 单独用 ptxas 编译才是可靠口径。(3)「警告消失」≠「指令回来了」：删 TMA 消警告的中间版本曾让人以为找到了解法，而 SASS 依旧为空，警告与 SASS 必须分别验证。

坑三：smem 基址只对齐 128B，输出全错但不越界。ldmatrix + 手写 `swizzle<64>()` 假设硬件以相位 0 写入；非 1024B 对齐时 TMA 按非零相位重排，软件公式逆推的物理地址整体错位。诊断特征：错误是**系统性**的（大片输出错、错值有结构），不是随机数，遇到这种错位先查对齐。

坑四：k_step 偏移加在 swizzle 外部。从 ch12 的 `swizzle<16>` 搬到 `swizzle<64>` 时最容易不自觉地把 `k_step * kMmaK` 仍当外部偏移，而 swizzle 周期覆盖整行 64 列时 k_step 改变 chunk 号高位，必须进公式内部参与 XOR（14.4.6 节）。症状同样是系统性错位。

坑五：CUDART < 13.0 编不过。`NOTES_V2_ENABLE_TMA_MA_WS` 在 common.cuh L21–23 有 #error 门控（`cuda::ptx` 的 bulk-tensor 封装需要新 CUDART）。旧工具链要么升级，要么改用 raw PTX 分支。

坑六：kStages=3 的 96KB 动态 smem 要 opt-in。launch 前必须用 `cudaFuncSetAttribute` 把 `MaxDynamicSharedMemorySize` 提到 96KB；正确性测试规模要覆盖 stage 回绕（$K/B_K > kStages$，如 $256/64 = 4 > 3$）。

坑七：忘记转置 B。kernel 的 TMA descriptor 期望 B 以 $[N, K]$ row-major（TN 布局）存储，host 侧要先转置；未转置时 box 形状仍匹配、数值全错。本章测试与 bench/hgemm.cu 相同：host 显式转置后再上传。

15.9 面试要点速查

Q1：SM120 没有 WGMMA，warp specialization 还成立吗？

A：成立。WS 的收益在搬运端（TMA 单线程提交、零地址指令）与同步端（mbarrier 数据驱动、无 ___syncthreads），与计算指令代际正交。SM120 的 consumer 用 mma.sync + ldmatrix 同样获得 1.57×（本章实测）。

Q2：full[s] 与 empty[s] 的 arrive count 各是多少？怎么推？

A：都是 129 = 128 consumer 线程各 1 次 + 1 producer 提交线程。full 的翻转还需 32768B complete-tx（A+B 两 box）；empty 靠 C0 预热预存首轮 128 次，此后每轮由 consumer 的 C4 累计 128 次、producer 的 P1 补最后 1 次。

Q3：为什么本章 consumer 全程不需要 ___syncthreads？

A：完成性由 barrier 保证（wait 返回即数据落盘）、内存序由 `fence.proxy.async` 保证（async proxy 写对 generic proxy 读可见）、warp 间无共享数据（2×2 warp grid 读写互不重叠的子块）。

Q4：为什么必须 ___align___(1024) 而 WGMMA 版只要 128？

A：128B swizzle 周期 1024B；WGMMA 的 smem descriptor 有 base_offset 字段硬件补偿相位，ldmatrix+ 软件 swizzle 没有，只能用对齐将相位固定为 0。

Q5：setmaxnreg 的池数学与死锁边界？

A：$\sum_{warp} 32 \times r_w < 65536$，工程上留 ≥ 2048 slack；恰满 65536 时 `.inc` 永久阻塞（无报错），与 producer 互等成死锁。立即数范围 $[24, 256]$ 且 8 的倍数，warpgroup 对齐执行。

Q6：SM120 上 setmaxnreg 为什么默认关？怎么正确开启？

A：CUDA 13.2 ptxas 把目的地址为 `shared::cluster` 的 `cp.async.bulk.tensor` 视作隐式 extern-call 边界，以 C7506 丢弃同 kernel 的 setmaxnreg（sm_90a 上 cute/cutlass 路径不受影响）；指令集本身对 sm_120a/sm_120f 都支持。**解法是 TMA 的 dst 写 `shared::cta` + WS kernel 带 `__launch_bounds__(N,1)` + 打开 `NOTES_V2_ENABLE_SETMAXNREGS`**，build.sh 的 sm_120f 目标预设了全部三项配置（见 14.8 节坑二取证）。

Q7：swizzle<64> 与 ch12 swizzle<16> 用法差异？

A：swizzle 周期覆盖整行 64 列时，k_step 偏移参与 chunk 号高位，必须把完整列坐标传入 XOR 公式内部；16 列 slice 各自独立 swizzle 时偏移可加在外部。

Q8：TMA 与 cp.async 的指令开销对比？

A：cp.async 每线程一条 16B 指令 + 软件成组等待；TMA 每 tile 一条指令（A、B 共 2 条）+ 硬件计数，地址/OOB/swizzle 全在 descriptor。producer 全轮 3–4 条指令 vs 256 线程数千条地址算术。

15.10 最小测试与动手实验

测试文件：`book/tests/ch14_hgemm_tma_ws.cu`（逻辑抽取自 bench/hgemm.cu 的 `test_hgemm_tma_mma_ws`@L435 与 launch wrapper（L382），参考实现由 cuBLAS F16 换成 CPU fp64）。两个正确性 case 均 256^3：被

$B_M = B_N = 128$、$B_K = 64$ 整除，$K/B_K = 4$ 个 K-tile 覆盖 kStages=2 与 3 两种回绕；case A（S=2、block swizzle=0）走主线协议，case B（S=3、BLK_SW=1）额外覆盖 3D grid 的越界早退分支。输入压到 $[-0.25, 0.25]$，f16 累加误差稳定落在 F16Acc 档 5×10^{-2} 内（实测 max_abs_err $\approx 1.9 \times 10^{-3}$）。

一个工程细节：`cuTensorMapEncodeTiled` 是 driver API，链接需要 `-lcuda`；而 `build_tests.sh` 统一不传该参数。测试改用 `cudaGetDriverEntryPointByVersion` 在运行时取函数指针（参数序列与 common.cuh 的 `create_tensor_map<128,64>` 逐项一致），这也顺带演示了「不链接 libcuda 使用 driver API」的标准做法。编译运行（本机 sm_120a 可直接跑）：

```
$ cd book/tests && CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch14
-> PASS hgemm tma ws 256x256x256 S=2: max_abs_err=1.871e-03
-> PASS hgemm tma ws 256x256x256 S=3 sw: max_abs_err=1.871e-03
-> ALL OK
```

bench 子命令输出 14.6 节的对照表（`CUDA_VISIBLE_DEVICES=6 .build/ch14_hgemm_tma_ws bench`）。动手实验建议：(1) 把 `tma_arrive_expect_tx` 的字节数减半，观察 consumer 挂死（tx 永不归零）；(2) 删掉 C0 预热循环，复现「首轮 empty 永不翻转」；(3) 把 `__align__(1024)` 改成 128，观察系统性错位输出（坑三）；(4) 用 `build.sh --arch sm_120f` 构建并跑 `cuobjdump --dump-sass notes_v2_sm120f.bin | grep -c USETMAXREG`（应为 112；注意 SASS 助记符没有字母 n），再对照把 common.cuh 的 `shared::cta` 临时改回 `shared::cluster` 重编，计数掉到 32，复现坑二的静默丢弃与取证过程。

延伸阅读

- @ 竹熙佳处《cute 之 TMA Copy》
  https://zhuanlan.zhihu.com/p/2003198909405763007
  （cute 视角的 TMA 拷贝与 descriptor 封装，box/swizzle 语义的先导阅读）；
- @Frank Wang《Async Copy 及 Memory Barrier》
  https://zhuanlan.zhihu.com/p/685168850
  （cp.async → mbarrier 异步同步体系的另一视角，补充 14.3.2/14.4.5 节）；
- NVIDIA PTX ISA §9.7.19.5（setmaxnreg）、§9.7.13.15（mbarrier）、§9.7.9.25（cp.async.bulk.tensor），本章指令语义核查基准（本地镜像：cuda-cpp-kernel skill 的 ptx-docs）；
- CUTLASS sm120 optimization guide（cutlass-cpp-kernel skill），SM120 与 sm_100/sm_90 能力差集的架构事实来源。

第三部分

Attention 篇：FlashAttention 2/3 与大 head_dim

第十六章　Attention 数学基础与 FlashAttention 原理

16.1　本章导读

前两部分已经备齐了 FlashAttention 需要的基础组件：第 五 章的 online softmax 给出 (m,ℓ) 双状态的联动递推（式 5.8），第 六 章的 LSE 合并证明分块状态可以严格拼回整体（定理 6.2），第 十–十二 章的 GEMM 阶梯给出把分块矩阵乘法交给 Tensor Core 的工程手段。本章将这些组件组装成完整的 FlashAttention 数学内核，作为 Part III 的「理论总纲」：后续第 十七 章（FA2 split-Q + MMA 多级流水）、第 十八 章（FA2 TMA + Warp Specialization）、第 十九 章（FA3 双 consumer warpgroup）与第 二十 章（FFPA Split-D）里的每一个 kernel 步骤，都能回指到本章的某条公式。

本章依次回答五个问题：

1. $\mathrm{softmax}\left(QK^\top/\sqrt{d}\right)V$ 逐项是什么？点积为什么要除以 $\sqrt{d}$（15.3.2 节：点积方差 $=d$ 的推导与 softmax 饱和论证）；
2. 朴素实现的 FLOPs、显存、HBM 流量三项开销（15.3.3 节：为什么显存带宽才是真瓶颈）；
3. 分块计算 + online rescale 的逐块推导（15.3.4 节：核心更新式 $O\leftarrow O\,\mathrm{diag}\big(e^{m^{\mathrm{old}}-m^{\mathrm{new}}}\big)+PV$，与 ch04/ch05 显式衔接）；
4. 分块循环不变量及其归纳法证明（15.3.5 节：任意 KV 分块遍历结束后，输出严格等于 $\mathrm{Attn}(Q,K,V)$）；
5. FA1 → FA2 → FA3 三代演进各自改了什么（15.3.7 节，逐条与论文对照）。

前置章节：第 五 章与第 六 章（必须，本章大量回指其公式），第 十 章（GEMM block tile 直觉，建议）。**源码区间**：flash_attn.cuh L5–110 头注释（本章只引用该区间，kernel 本体留待 ch16–18 展开）；历史与性能指标陈述以 FA1/FA2/FA3 三篇论文为核验源（F5 类）。**测试**：本章无独立 kernel 测试，数学正确性由第 五 章（softmax 三级递进）与第 六 章（merge_attn_states）的测试覆盖，见 15.10 节的显式声明。

16.2　问题与动机

Transformer 的序列建模能力建立在 attention 之上，而 attention 的代价随序列长度平方增长。上下文从 2K 到 32K 再到 128K+，瓶颈最先暴露的不是算力，而是**中间矩阵的显存与访存**。以本仓库 bench 的典型形状 B=1、H=32、N=4096、D=64 为例（fp16）：分数矩阵 $S=QK^\top$ 每个头是 $4096\times4096\times2\,\mathrm{B}\approx33.5\,\mathrm{MB}$，32 个头共约 $1\,\mathrm{GiB}$；$P=\mathrm{softmax}(S)$ 再翻一倍。训练时这些中间量还要为反向传播保存（或者重算，见 15.3.7 节 FA1 的 recomputation），显存开销随 $O(N^2)$ 增长而迅速超出可用容量。

其次，即使显存装得下，朴素实现的执行流程是三个分离的 kernel：$S=QK^\top$（写 N^2）→ softmax（读 N^2、写 N^2）→ $O=PV$（读 N^2）。仅中间矩阵的 HBM 往返就是 $4\times1\,\mathrm{GiB}\approx4.3\,\mathrm{GB}$ 的流量，而有效数据（Q/K/V 读入 + O 写出，$4Nd\cdot H\cdot2\,\mathrm{B}\approx67\,\mathrm{MB}$）只有它的 1/64。15.3.3 节会算出：在这类形状下，搬中间矩阵的时间下界就已经超过整个计算的时间下界，**显存带宽，而不是 Tensor Core，才是朴素 attention 的第一瓶颈**。

第三：S 和 P 是**一次性**的中间量，S 算出来只为了做 softmax，P 算出来只为了乘 V，之后不再被使用。既然如此，就没有必要让它离开芯片。直觉上的做法是分块：把 Q,K,V 切成小块，在 SRAM/寄存器里完成「乘—归一化—乘」的全部中间步骤，只把最终 O 写回 HBM。但 softmax 的**分母把整行耦合在一起**：某一行 query 的归一化需要它对所有 key 的分数，逐块处理时后面的块尚未参与计算，分母无法立即得到。第 五 章 online softmax（(m,ℓ) 联动更新）

与第 六 章分块合并（LSE 等价拼回）解决的就是这个问题。本章把那两章的数学直接搬进 attention 的分块循环，再补上「带着 V 一起 rescale」这一步，就得到 FlashAttention。

16.3　数学原理

16.3.1　Attention 的定义与逐项含义

对每个 (batch, head) 独立计算，为记号简洁省略 (b,h) 下标。设序列长度 N、head 维数 d（即全书记号表中的 D，源码模板参数 `kHeadDim`）。三个输入：

$$Q \in \mathbb{R}^{N\times d}\,(\text{query})\,,\qquad K \in \mathbb{R}^{N\times d}\,(\text{key})\,,\qquad V \in \mathbb{R}^{N\times d}\,(\text{value})\,, \tag{16.1}$$

scaled dot-product attention 定义为

$$\mathrm{Attn}(Q,K,V) \;=\; \mathrm{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d}}\right)V, \tag{16.2}$$

逐项拆开：$S \triangleq QK^{\top}/\sqrt{d} \in \mathbb{R}^{N\times N}$ 是 **打分矩阵**（score，记号 S），$S_{ik} = q_i \cdot k_k/\sqrt{d}$ 表示 query i 与 key k 的相似度；$P \triangleq \mathrm{softmax}(S) \in \mathbb{R}^{N\times N}$（按行）是**概率矩阵**，$P_{ik}\ge 0$、$\sum_k P_{ik} = 1$；$O \triangleq PV \in \mathbb{R}^{N\times d}$ 是**输出**。按第 i 行写开，式 16.2 等价于对每个 query 独立地做

$$s^{(i)} = \frac{Kq_i}{\sqrt{d}} \in \mathbb{R}^{N},\qquad P_{ik} = \frac{e^{\,s_k^{(i)}}}{\sum_{k'} e^{\,s_{k'}^{(i)}}},\qquad O_i = \sum_{k=1}^{N} P_{ik}\,v_k, \tag{16.3}$$

即「打分 → 归一化成权重 → 对 value 加权求和」。行与行之间零依赖，这条 **行内自足性**与 softmax 完全一致（第 五 章动机三），是后面能把一行 query 的全部状态（m_i, ℓ_i, O_i）放进一个 block 的寄存器/共享内存的根本原因。

16.3.2　为什么除以 $\sqrt{d}$：点积方差推导

式 16.2 里的 $1/\sqrt{d}$ 不是工程调参，而是有统计动机的定标。设 $q,k \in \mathbb{R}^d$ 的各分量独立同分布、零均值、单位方差（网络初始化附近的大致情形）。考察点积 $s = q\cdot k = \sum_{i=1}^{d} q_i k_i$。均值为零：

$$\mathbb{E}[s] = \sum_{i=1}^{d} \mathbb{E}[q_i]\,\mathbb{E}[k_i] = 0. \tag{16.4}$$

方差展开时交叉项 $i\neq j$ 因 $\mathbb{E}[q_iq_j]=0$（独立零均值）全部消失：

$$\mathrm{Var}(s) = \mathbb{E}\Big[\Big(\sum_i q_i k_i\Big)^2\Big] = \sum_{i=1}^{d} \mathbb{E}\big[q_i^2\big]\,\mathbb{E}\big[k_i^2\big] = d, \tag{16.5}$$

即 $\mathrm{std}(s) = \sqrt{d}$。$d = 128$ 时 logit 的标准差约 11.3，softmax 会严重**饱和**：看两元素的最简情形，$p = \sigma(s_1 - s_2)$（σ 为 sigmoid），对 logit 差 Δ 的梯度是

$$\frac{\partial p}{\partial \Delta} = \sigma(\Delta)\big(1-\sigma(\Delta)\big) \;\le\; \tfrac{1}{4},\qquad \Delta = 11.3 \;\Rightarrow\; \sigma(1-\sigma) \approx 1.2\times 10^{-5}\,: \tag{16.6}$$

梯度从峰值 0.25 跌到 10^{-5} 量级，softmax 输出近似 one-hot，反向传播梯度近乎消失。除以 $\sqrt{d}$ 后 $\mathrm{Var}(s/\sqrt{d}) = 1$，logit 回到 $O(1)$ 量级，softmax 处于梯度健康的敏感区。两点考据说明：(1) Transformer 原论文（Vaswani et al., 2017, §3.2.1）给出的其实是启发式陈述：「我们怀疑 d_k 较大时点积量级变大，把 softmax 推进极小梯度区，故除以 $\sqrt{d_k}$」；式 16.5 的方差推导是后续文献的通行严格化解读，并非原文内容（考据框注见 15.7 节）。(2) 训练后的真实分布并不满足 i.i.d. 假设（Q/K 各维方差不等、存在 outlier），$1/\sqrt{d}$ 仍是事实标准，方差更不均匀的情形由量化方案处理（15.3.7 节 FA3 的 smoothing/block 量化思想）。

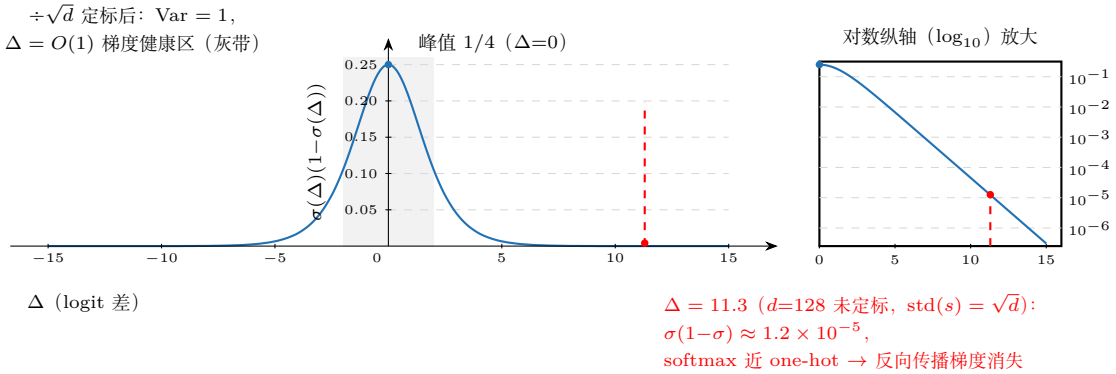


图 16.1: $1/\sqrt{d}$ 定标的饱和论证（15.3.2 节，式 16.6）：sigmoid 梯度 $\sigma(\Delta)(1-\sigma(\Delta))$ 的峰值 1/4 在 $\Delta = 0$；除以 $\sqrt{d}$ 后 Var = 1、$\Delta = O(1)$（灰带），梯度处于健康敏感区。不定标时 $\mathrm{std}(s) = \sqrt{d}$（式 16.5），$d = 128$ 即 $\Delta \approx 11.3$，梯度跌到 $\approx 1.2 \times 10^{-5}$——线性纵轴上接近零，右侧对数纵轴面板可见其仍在 10^{-5} 量级（红点，恰在 10^{-5} 网格线上方），softmax 近 one-hot、反向传播梯度近乎消失。曲线偶对称，$\Delta = -11.3$ 同理。

16.3.3 朴素实现：FLOPs、显存与带宽三项开销

FLOPs 量（约定：一次乘法 + 一次加法计 2 FLOP，故 $M\times K$ 乘 $K\times N$ 的 GEMM 为 $2MNK$ FLOP）。式 16.2 的两个矩阵乘是对称的两大项：$S = QK^\top$ 为 $2N^2d$，$O = PV$ 为 $2N^2d$；softmax 本身只有指数与求和，是 $O(N^2)$ 的非 matmul 项。合计每 (b,h)

$$\mathrm{FLOPs} = 4N^2d + O(N^2), \tag{16.7}$$

乘上 $B \times H$ 个 (b,h)。仍取 B=1, H=32, N=4096, d=64：总 FLOPs 约 $4 \times 4096^2 \times 64 \times 32 \approx 137\,\mathrm{GFLOP}$；参照本仓库 FlashAttention kernel 在同形状 SM120 上的实测 166.6 TFLOPS（flash_attn.cuh L68，头注释口径），纯计算时间下界约 0.83 ms。

显存占用：S 与 P 各占 N^2 元素，

$$\mathrm{mem} = \Theta(N^2) \text{ 每 (b,h)}, \tag{16.8}$$

上例中 S、P 各 $\approx 1\,\mathrm{GiB}$。N 翻倍它翻四倍，这就是长上下文的显存墙。

带宽需求：朴素三 kernel 的 HBM 流量为读 Q, K, V（$3Nd$）+ 写 S、读 S、写 P、读 P（$4N^2$）+ 写 O（Nd），即 FA1 论文所述的 $\Theta(Nd + N^2)$：

$$\mathrm{IO}_{\mathrm{naive}} = \Theta\big(Nd + N^2\big) \quad \text{元素，主导项} 4N^2. \tag{16.9}$$

上例的中间矩阵往返 4.3 GB：即使按 2 TB/s 的乐观带宽假设（多数 GPU 的 HBM/显存带宽在数百 GB/s 到 2–3 TB/s 量级），仅搬运就要 $\approx 2.2\,\mathrm{ms}$，已是 0.83 ms 计算下界的 2.6 倍以上；而有效数据（Q, K, V 读入 + O 写出，$\approx 67\,\mathrm{MB}$）只占流量的 1/64。**中间矩阵往返使朴素 attention 的算术强度降到极低**。按主导项计，

$$AI_{\mathrm{naive}} \approx \frac{4N^2d\ \mathrm{FLOP}}{4N^2 \cdot 2\,\mathrm{B}} = \frac{d}{2}\ \mathrm{FLOP/B} = 32 \quad (d{=}64, \mathrm{fp16}), \tag{16.10}$$

远低于现代 GPU 数百 TFLOPS 峰值除以 TB/s 级带宽得到的 roofline 拐点（100+ FLOP/B 量级），kernel 落在 memory-bound 区。改进方向只有一个：让 N^2 中间量根本不经过 HBM。

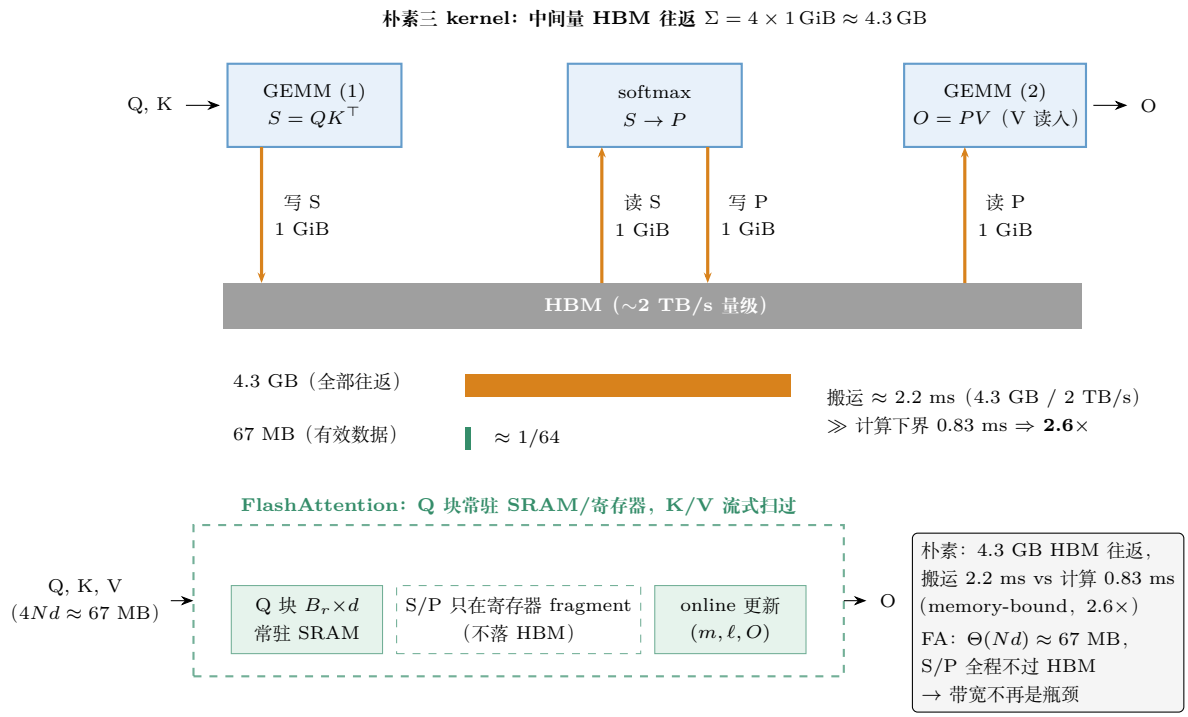


图 16.2: 朴素 attention 与 FlashAttention 的 HBM 往返流量对比（15.3.3 节，B=1、H=32、N=4096、d=64）：上行三个 kernel 之间，S 与 P（各 ≈1 GiB）必须经 HBM 往返——写 S、读 S、写 P、读 P 共 $4\times1\,\text{GiB}\approx4.3\,\text{GB}$，而有效数据（Q,K,V 读入 + O 写出）只有 ≈67 MB，占全部流量的 1/64；按 2 TB/s 乐观带宽，仅搬运就要 ≈2.2 ms，已是 0.83 ms 计算下界的 2.6 倍——kernel 落在 memory-bound 区。下行 FlashAttention 把 S、P 限制在 SRAM/寄存器 fragment 里（虚线框），HBM 只传输 Q,K,V 输入与 O 输出（$\Theta(Nd)$，式 16.9 的另一端），中间量全程不过 HBM。

16.3.4　分块计算与 online rescale：逐块推导

FlashAttention 的做法：Q 按 B_r 行切成 $\lceil N/B_r\rceil$ 个 Q 块（一个 thread block 拿一块，常驻 SRAM）；K,V 按 B_c 行切成 $T_c=\lceil N/B_c\rceil$ 个 KV 块，沿序列方向流式扫过：

$$K=\begin{bmatrix}K^{[1]}\\ \vdots\\ K^{[T_c]}\end{bmatrix},\quad V=\begin{bmatrix}V^{[1]}\\ \vdots\\ V^{[T_c]}\end{bmatrix},\quad K^{[j]},V^{[j]}\in\mathbb{R}^{B_c\times d},\qquad S^{[j]}\triangleq\frac{Q\,K^{[j]\top}}{\sqrt{d}}\in\mathbb{R}^{N\times B_c}. \tag{16.11}$$

整行分数矩阵被按列切成 $S=\left[\,S^{[1]}\mid S^{[2]}\mid\cdots\mid S^{[T_c]}\,\right]$。对每个 query 行 i，维护三个运行状态：标量 m_i（已见分数的行 max，即第 五 章的 running max）、标量 ℓ_i（running sum，即第 五 章的 ℓ；源码变量 `block_row_max`/`block_row_sum`）、以及新增的一行输出累加器 $O_i\in\mathbb{R}^{1\times d}$。处理第 j 块时，逐项推导四步更新。

第一步：新 max。已见集合扩大为「前 j 块」，行 max 只能变大或不变：

$$m^{\text{new}}=\max\!\big(m^{\text{old}},\ \text{rowmax}(S^{[j]})\big)\qquad\text{（逐行取 max，向量}m\in\mathbb{R}^{N}\text{）}. \tag{16.12}$$

第二步：块内概率（尚缺分母，先记「未归一化概率」）。按行减去**新** max（而非旧 max），指数参数非正、数值安全（第 五 章式 5.4 的安全性论证原样适用）：

$$P^{[j]}\;=\;\exp\!\big(S^{[j]}-m^{\text{new}}\,\mathbf{1}^{\top}\big)\in\mathbb{R}^{N\times B_c},\qquad P^{[j]}_{ik}=e^{\,S^{[j]}_{ik}-m^{\text{new}}_i}\le 1. \tag{16.13}$$

第三步：ℓ 更新。目标 $\ell^{\text{new}}_i=\sum_{k\le jB_c}e^{\,S_{ik}-m^{\text{new}}_i}$，把求和拆成「旧块 + 新块」并在旧块上凑因子，与第 五 章式 5.7 的推导逐步同构：

$$\begin{aligned}\ell^{\text{new}}&=\sum_{k\le(j-1)B_c}e^{\,S_{ik}-m^{\text{new}}_i}+\sum_{k\in\text{block }j}e^{\,S_{ik}-m^{\text{new}}_i}\\&=\underbrace{\sum_{k\le(j-1)B_c}e^{\,S_{ik}-m^{\text{old}}_i}}_{=\,\ell^{\text{old}}_i}\cdot e^{\,m^{\text{old}}_i-m^{\text{new}}_i}+\underbrace{\sum_{k\in\text{block }j}e^{\,S_{ik}-m^{\text{new}}_i}}_{=\,\text{rowsum}(P^{[j]})_i},\end{aligned} \tag{16.14}$$

即（$\odot$ 为逐元素乘）

$$\ell^{\text{new}} = e^{\,m^{\text{old}} - m^{\text{new}}} \odot \ell^{\text{old}} + \operatorname{rowsum}\big(P^{[j]}\big). \tag{16.15}$$

第四步：O 更新（online rescale）。这一步是 bare softmax 没有的：V 也乘在指数权重上，旧块的加权和是以旧 max 为基准的，基准抬高后必须整体缩放。目标 $O_i^{\text{new}} = \sum_{k \le jB_c} e^{\,S_{ik} - m_i^{\text{new}}} v_k$，同样拆「旧 + 新」并凑因子：

$$\begin{aligned} O_i^{\text{new}} &= \sum_{k \le (j-1)B_c} e^{\,S_{ik} - m_i^{\text{old}}} e^{\,m_i^{\text{old}} - m_i^{\text{new}}} v_k + \sum_{k \in \text{block } j} e^{\,S_{ik} - m_i^{\text{new}}} v_k \\ &= e^{\,m_i^{\text{old}} - m_i^{\text{new}}} \underbrace{\sum_{k \le (j-1)B_c} e^{\,S_{ik} - m_i^{\text{old}}} v_k}_{\;=\; O_i^{\text{old}}} + \big(P^{[j]} V^{[j]}\big)_i, \end{aligned} \tag{16.16}$$

写成矩阵形式，就得到本章的核心公式（flash_attn.cuh L23 的 `O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V`）：

$$\boxed{\; O \;\leftarrow\; O\,\operatorname{diag}\big(e^{\,m^{\text{old}} - m^{\text{new}}}\big) \;+\; P^{[j]} V^{[j]} \;} \tag{16.17}$$

（$\operatorname{diag}(e^{\,m^{\text{old}} - m^{\text{new}}})$ 左乘 = 逐行缩放 O 的第 i 行乘 $e^{\,m_i^{\text{old}} - m_i^{\text{new}}}$；源码步骤 3e。）整列 T_c 块扫完后，做**唯一一次**最终归一化：

$$O \;\leftarrow\; \operatorname{diag}\big(1/\ell\big)\, O \qquad \text{（源码步骤 8：`O_final = (1/l_final) * O_final`）}. \tag{16.18}$$

与 ch04/ch05 的显式衔接。式 16.15 就是第 五 章 online 递推式 5.8 的「批量化」版本（一次处理 B_c 个元素而非 1 个）；而带 O 的合并运算是第 五 章二元合并式 5.9 的直接扩展：

$$(m^A, \ell^A, O^A) \oplus (m^B, \ell^B, O^B) = \Big(m,\;\; \ell^A e^{m^A - m} + \ell^B e^{m^B - m},\;\; e^{m^A - m} O^A + e^{m^B - m} O^B\Big), \quad m = \max(m^A, m^B) : \tag{16.19}$$

ℓ 与 O 用**同一个** rescale 因子修正。与第 六 章的关系则是「同一数学的 kernel 视角」：定理 6.2 的凸组合 $O = \alpha O^A + \beta O^B$（式 6.5）里，$O^A, O^B$ 是 **已归一化**的分块输出；把 $\alpha = W^A/(W^A{+}W^B)$ 的分子分母同乘 $W^A{+}W^B$ 展开即可改写成**未归一化**形式 $O = \big(W^A O^A_{\text{norm}} \ell^A + W^B O^B_{\text{norm}} \ell^B\big)/\ell$，正是式 16.17 逐块折叠的形态。ch05 是「事后合并」（两块各自落盘 O 与 LSE，再用一个 elementwise kernel 拼回，服务于 split-KV / FlashDecoding），本章是「在线合并」（寄存器里一份 O 随 KV 循环滚动更新，从不物化两份 O）；式 16.19 的结合律由引理 5.1 同样的「全集唯一决定」论证立刻推广到带 O 的三元组。另注意一个精度口径：本仓库 `merge_attn_states` 的接口要求输入 O 已除过 ℓ（ch05 坑一），而 FA 主循环的累加器 O 必须**未**归一化，两套口径的公式不可混用（见 15.8 节坑一）。

16.3.5 分块循环不变量与归纳法证明

式 16.12–16.17 看似只是「顺带维护三个量」，正确性由一个严格的不变量保证。记 $s_{ik} \triangleq S_{ik} = q_i \cdot k_k/\sqrt{d}$。

定义 16.1 (分块循环不变量)**.** 处理完前 j 块（即 key 序号 $1, \dots, jB_c$）后，状态满足

$$\mathrm{I}(j): \qquad m_i^{(j)} = \max_{k \le jB_c} s_{ik}, \qquad \ell_i^{(j)} = \sum_{k \le jB_c} e^{\,s_{ik} - m_i^{(j)}}, \qquad O_i^{(j)} = \sum_{k \le jB_c} e^{\,s_{ik} - m_i^{(j)}} v_k. \tag{16.20}$$

定理 16.2 (分块 online 计算的正确性)**.** *设初始状态 $m_i^{(0)} = -\infty$、$\ell_i^{(0)} = 0$、$O_i^{(0)} = \mathbf{0}$，每步按式 16.12–16.17 更新，则对一切 $j \in \{1, \dots, T_c\}$ 不变量 $\mathrm{I}(j)$ 成立，且最终输出*

$$\frac{O_i^{(T_c)}}{\ell_i^{(T_c)}} \;=\; \sum_{k=1}^{N} \frac{e^{\,s_{ik}}}{\sum_{k'} e^{\,s_{ik'}}} v_k \;=\; \mathrm{Attn}(Q, K, V)_i, \tag{16.21}$$

与 KV 的分块方式（B_c 大小、块边界）及处理顺序无关。

证明. 对 j 归纳。**基础**：$j = 0$ 时「已见集合」为空，max 取 $-\infty$、空和为 0，$\mathrm{I}(0)$ 成立（$e^{-\infty - m^{(1)}} = 0$ 与 $e^{m^{(0)} - m^{(1)}} = 0$ 一致，首轮 rescale 因子恰为 0，旧零状态被安全丢弃）。

归纳步：设 $\mathrm{I}(j{-}1)$ 成立。其一，$m^{(j)} = \max\big(m^{(j-1)}, \operatorname{rowmax}(S^{[j]})\big) = \max_{k \le jB_c} s_{ik}$，即式 16.12。**其二**（$\ell$）：把式 16.14 的推导套用 $\mathrm{I}(j{-}1)$ 的 $\ell_i^{(j-1)} = \sum_{k \le (j-1)B_c} e^{\,s_{ik} - m_i^{(j-1)}}$，得

$$\ell_i^{(j)} = \sum_{k \le (j-1)B_c} e^{\,s_{ik} - m_i^{(j)}} + \sum_{k \in \text{block } j} e^{\,s_{ik} - m_i^{(j)}} = \sum_{k \le jB_c} e^{\,s_{ik} - m_i^{(j)}}.$$

其三（O）：同理用式 16.16 与 I($j-1$) 中的 $O_i^{(j-1)} = \sum_{k\le(j-1)B_c} e^{s_{ik}-m_i^{(j-1)}} v_k$，得到

$$O_i^{(j)} = e^{m_i^{(j-1)}-m_i^{(j)}} O_i^{(j-1)} + \sum_{k\in\text{block } j} e^{s_{ik}-m_i^{(j)}} v_k = \sum_{k\le jB_c} e^{s_{ik}-m_i^{(j)}} v_k.$$

故 I(j) 成立。**终态**：由 I(T_c)，

$$\frac{O_i^{(T_c)}}{\ell_i^{(T_c)}} = \frac{\sum_{k\le N} e^{s_{ik}-m_i^{(T_c)}} v_k}{\sum_{k\le N} e^{s_{ik}-m_i^{(T_c)}}} = \sum_{k=1}^{N} \mathrm{softmax}(s^{(i)})_k \, v_k,$$

其中最后一个等号用了 softmax 的平移不变性（第 五 章式 5.2），公共因子 $e^{-m_i^{(T_c)}}$ 分子分母约掉。**次序无关性**：不变量只依赖「已见 key 的集合」而非顺序，而带 O 的合并运算（式 16.19）满足结合律（全集的 (max, 指数和, 加权和) 唯一决定，引理 5.1 的论证逐字推广），故任意归约树给出同一结果。 □

注记 (不变量的三个直接推论). （1）**mask 无损**：causal/变长 mask 把非法位置置 $-\infty$，则 $e^{-\infty-m}=0$，该项从 ℓ 与 O 的求和中自然消失，不变量与定理不受影响。(2) **任意切分合法**：KV 可以按序列切（FlashAttention、FlashDecoding 的 split-KV），也可以按 d 维切后再拼（第 二十 章 FFPA Split-D 跨 D-chunk 用同一族公式合并），只要每块给出自己的 (m,ℓ,O) 状态。(3) **空行防护**：若某行所有位置都被 mask，$\ell=0$，最终除法产生 `NaN`，与第 五 章坑二、第 六 章坑四同源，工程上由调用方保证行非空或显式兜底。

16.3.6　复杂度总结与 IO 分析

FLOPs 不变：分块只是重排了计算顺序，两个 GEMM 的总工作量仍是 $4N^2d$，FlashAttention **不省 FLOPs，省的是 HBM 流量与显存**。**显存**：不再物化 S,P，每行只需标量 (m_i,ℓ_i) 与累加器行 $O_i\in\mathbb{R}^d$，加上 SRAM 里的 $B_r\times d$、$B_c\times d$ tile，激活显存从 $\Theta(N^2)$ 降到 $\Theta(Nd)$。名字里的 *Flash*（不写中间矩阵到 HBM）由此而来。

IO：每个 Q 块要把 K,V 从头到尾读一遍。设 SRAM 容量为 M（元素数），受 $B_r\cdot d+$(KV stage 数)$\cdot 2B_cd\le M$ 约束，量级上 $B_r\sim M/d$，于是 K/V 的总读取量为

$$\underbrace{\frac{N}{B_r}\cdot 2Nd}_{\text{元素}} \sim \frac{2N^2d^2}{M} \qquad\Longrightarrow\qquad \mathrm{IO}_{\mathrm{FA}} = \Theta\Big(Nd + \frac{N^2d^2}{M}\Big), \tag{16.22}$$

即 FA1 论文的结论：当 $d^2<M$（fp16 下 $d=64/128$ 时 d^2 只有 8–32 KB，远小于十万元素量级的 SRAM）时，$\mathrm{IO}_{\mathrm{FA}}<\mathrm{IO}_{\mathrm{naive}}$，且差距随 M/d^2 线性放大。按主导项计的算术强度则从式 16.10 的 $d/2$ 提升到

$$AI_{\mathrm{FA}} \approx \frac{4N^2d\ \text{FLOP}}{\frac{N}{B_r}\cdot 2Nd\cdot 2\,\text{B}} = B_r\ \text{FLOP/B}, \tag{16.23}$$

$B_r=128$ 时约 128 FLOP/B，跨过 roofline 拐点进入 compute-bound 区。两点补充说明：其一，该一阶模型未计 L2 cache 跨 thread block 的 K/V 复用（实际 HBM 流量更低），也未计 softmax/归约的非 matmul 开销；其二，「compute-bound」是对**理想**实现的上界描述，真实 kernel 实际能达到的比例取决于流水线工程，FA2/FA3 两代论文的优化对象正是这一段（下一节）。

16.3.7　FA1 → FA2 → FA3：三代演进

以下历史陈述逐条与三篇论文对照（F5 类核验；不一致处记入 15.7 节与本章报告）。

FA1 (arXiv:2205.14135, NeurIPS 2022)：Dao et al. 提出 IO-aware 的精确 attention：tiling + online softmax（式 16.12–16.17），S/P 从不落 HBM；反向传播用 **recomputation**，只存 Q,K,V,O 与 LSE，反向时重算 S,P，以 30% 量级的重算换 $O(N^2)$ 激活显存的节省；给出式 16.22 的 IO 复杂度 $\Theta(N^2d^2/M)$。论文口径：BERT-large（N=512）训练比 MLPerf 1.1 纪录快 15%，GPT-2（N=1K）端到端 3× 加速，激活显存省 10–20×。

FA2 (arXiv:2307.08691, 2023)：算法内核不变，三处工程级重构提高利用率：

1. **削减非 matmul FLOPs**。A100 上 fp16 Tensor Core 约 312 TFLOPS 而 fp32 CUDA core 仅 19.5 TFLOPS（差 16×），因此 exp/max/rowsum/rescale 这类「慢指令」的开销要压到最低。具体改动：scale $1/\sqrt{d}$ **预乘进** Q（而不是每块乘进 S）；O 全程保持**未归一化**，只在式 16.18 除一次 ℓ（FA1 每块维护已归一化 O，每步两次对角修正）。

2. **并行度**。FA1 只在 (B, H) 维开 thread block，长序列小 batch 时 SM 占用不足；FA2 把 **seqlen(Q) 维**也做成并行，query 序列切成多段，每段独立一个 thread block（本仓库 `flash_attn_mma_stages_split_q` 的 `grid = ((N+63)/64, B*H)`，第 十七 章展开）。
3. **warp 分工**。FA1 的 4 个 warp 沿 KV 维切分，各算部分分数后必须经 shared memory 做跨 warp 归约合并；FA2 改为 warp 沿 Q **行**切分（split-Q）：4 个 warp 共享同一块 K, V、各持独立 (m, ℓ, O)，零跨 warp 归约、零额外 shuffle，Q 只加载一次。

论文口径：比 FA1 快约 2×；A100 前向最高 230 TFLOPS（峰值的 73%）。

FA3（arXiv:2407.08608，2024）：面向 Hopper（H100）的**异步化 + 低精度**：

1. **GEMM 与 softmax 的 overlap**。softmax 在 CUDA core 上跑、GEMM 在 Tensor Core 上跑，二者本可并行。FA3 用 warp specialization：producer warp 发 TMA 异步拷贝（第 十四 章），两个 consumer warpgroup 以 pingpong 方式交替：一个 warpgroup 做 GEMM 时另一个做 softmax，15–25% 的 softmax 时间与 GEMM 重叠执行（第 十九 章双 consumer 结构图）。
2. **FP8**。Hopper 的 FP8 Tensor Core 吞吐再翻一倍，但 naive 量化会损精度。FA3 的两个机制：*block* 量化（按块维护缩放因子，而非全局或 per-tensor）；*incoherent processing*，对 Q, K 右乘同一个正交矩阵 U（随机化 Hadamard），点积严格不变：$(QU)(KU)^\top = QUU^\top K^\top = QK^\top$，而 outlier 能量被打散到各维，FP8 量化误差显著下降。

论文口径：比 FA2 快 1.5–2×；H100 fp16 前向最高 740 TFLOPS（峰值的 75%），FP8 前向接近 1.2 PFLOPS。

表 16.1: FA 三代演进对照（指标均引自各自论文；FA1 的 MLPerf/3× 口径见正文）

	FA1 (2022)	FA2 (2023)	FA3 (2024)
论文	arXiv:2205.14135	arXiv:2307.08691	arXiv:2407.08608
概括	tiling + online softmax，中间矩阵不落 HBM；反向 recomputation	少非 matmul FLOPs、加 seqlen(Q) 并行、warp 改 split-Q	异步 overlap（warp spec. + pingpong）+ FP8
硬件口径	A100	A100 / H100	H100
CTA 级并行	(B, H)	$(B, H)\times$ seqlen(Q)（split-Q）	同 FA2
warp 级分工	4 warp 分 KV，跨 warp 归约	4 warp 分 Q 行，无跨 warp 归约	producer(TMA) + 2 consumer WG，GEMM/softmax 交替
精度	fp16/bf16	fp16/bf16	fp16 + FP8（block 量化 + incoherent processing）
代表指标	GPT-2 3× 端到端；显存省 10–20×	≈ 2× vs FA1；A100 230 TFLOPS（73%）	1.5–2× vs FA2；H100 fp16 740 TFLOPS（75%）

对照本仓库：flash_attn.cuh L5–790 是 FA2 的手写 MMA 版（第 十七 章），L792–1440 是 FA2 的 TMA + Warp Specialization 版（第 十八 章），L1441–2192 是 FA3 的双 consumer warpgroup 版（第 十九 章），L2196–3488 是三者的 CuTe 复刻对照（第 二十六 章），从 FA2 到 FA3 的每个演进点，都能在源码里找到对应的结构差异。

16.4 设计与实现

16.4.1 flash_attn.cuh 的总体结构

flash_attn.cuh 全文 3490 行，按编译宏分成三段五族 kernel（行号见 BOOK_PLAN §1.3 核验表）：FA2 MMA 版（`flash_attn_mma_stages_split_q`，L112 起）；`TMA_MMA_WS` 宏块（L791–2194）内含 FA2 TMA+WS 版（L889）与 FA3 版（`flash_attn_3_tma_ws_stages_split_q`，L1516）；`CUTE` 宏块（L2196–3488）内含 FA2/FA3 的 CuTe traits 与三个复刻 kernel。本章只引用头部设计注释（L5–110）：它是全文的「算法说明书」，五条要点恰好逐一对应 15.3 节的推导。

16.4.2　头注释导读：五条要点

头注释 L5–30 以「面试要点（FlashAttention-2 算法）」为题，共五条。

要点 1，核心问题：标准 Attention 的 $O(N^2)$ 中间矩阵（$S = QK^\top$）必须写入 HBM，但 HBM 带宽是瓶颈，FlashAttention-2 用 tiling + online softmax 避免写回。

要点 2，FA 的三项核心机制：

- **Tiling**：Q 分块 $[B_r, d]$，K/V 沿 seqlen 分块 $[B_c, d]$；
- **Online Softmax**：迭代更新 m（行 max）与 ℓ（行 sum），无需全局同步；
- **Recomputation**（反向）：反向传播时重新计算 S/P，而非存储中间矩阵。

要点 3，split-Q 设计：所有 warp 共享同一块 K，各 warp 处理 Q 的不同行片段：`warp_KV=0`（所有 warp 共享 K），`warp_QP=warp_id`（各 warp 不同 Q 行）；优点是减少 warp 间通信和 shuffle。

要点 4，online rescaling 公式（FA 核心，arXiv:2307.08691）：对每个 K/V tile，

$$
\begin{aligned}
S_{\text{cur}} &= Q\,K^\top && \text{（未缩放，存入 R_S）}\\
m_{\text{new}} &= \max\big(m_{\text{old}},\ \text{row_max}(S_{\text{cur}}\cdot \text{scale})\big)\\
P_{\text{cur}} &= \exp(S_{\text{cur}}\cdot \text{scale} - m_{\text{new}}) && \text{（写回 R_S 寄存器!）}\\
\ell_{\text{new}} &= \exp(m_{\text{old}} - m_{\text{new}})\,\ell_{\text{old}} + \text{row_sum}(P_{\text{cur}})\\
O_{\text{new}} &= \text{diag}\big(\exp(m_{\text{old}} - m_{\text{new}})\big)\,O_{\text{old}} + P_{\text{cur}}V\\
O_{\text{final}} &= O_{\text{new}} \,/\, \ell_{\text{final}}
\end{aligned}
$$

要点 5，为什么 R_S 可以直接用作 $P@V$ 的 A 矩阵：

- R_S 经过 softmax 后，存储的是 $P = \exp(S - m)$，数据仍然是 half 精度；
- 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局，使 softmax 后的 P 可以继续留在 R_S 中，供后面的 $P@V$ 直接消费；
- 这是此实现的寄存器布局复用技巧，不要推广成「所有 MMA A/C fragment 都天然同构」的通用结论。

要点 1（L8–9）即 15.3.3 节的带宽需求：$O(N^2)$ 中间矩阵必须过 HBM，而 HBM 是瓶颈。要点 2（L10–13）的三项机制中，tiling 对应式 16.11，online softmax 对应式 16.12–16.15，recomputation 是 FA1 的反向技巧（15.3.7 节），注意这组贡献按论文口径属 FA1，FA2 的主题是并行与分工，见 15.7 节考据框注一。要点 3（L14–16）是 FA2 的 warp 级 split-Q。要点 4（L17–24）的五行伪代码逐行对应式 16.12（`m_new`）、16.13（`P_cur`）、16.15（`l_new`）、16.17（`O_new`）与 16.18（`O_final`），O 保持未归一化、最后统一除 ℓ，这构成 FA2 对 FA1 的关键改动。要点 5（L25–30）是一个工程细节：softmax 后的 P 直接写回 S 的 fragment 寄存器 R_S 供 $P@V$ 消费，省掉一次 fragment 重组。但这只是 m16n8k16 布局约定下的复用技巧，不是 MMA fragment 的通用性质（注释原文也强调了这一点；机制在第 十七 章展开）。

16.4.3　launch 配置与 tile 参数化

头注释 L31–46 交代出处与 launch 口径：本实现参考 FlashAttention-2（Dao et al., arXiv:2307.08691），提取自 LeetCUDA `flash-attn/mma/basic/flash_attn_mma_split_q.cu`。张量形状：Q/K/V/O 均为 `[batch_size, num_heads, seq_len, head_dim]`，简记 $[B, H, N, d]$。block 配置 $(128, 1, 1)$，线程数 `kNumThreads` = `kWarpSize` × `kMmaTileSeqLenQ` × `kMmaTileSeqLenK` = 128。tile 设计（以 `kHeadDim=64` 为例）把两个块尺寸分解到 mma 原子：

- B_r：`kMmaAtomM` × `kMmaTileSeqLenQ` × `kValTileSeqLenQ`，即 $16 \times \{4, 8\} \times 1 = \{64, 128\}$；
- B_c：`kMmaAtomN` × `kMmaTileSeqLenK` × `kValTileSeqLenK`，即 $8 \times 1 \times 8 = 64$；
- warp 布局：4 个 warp，`warp_QP` 取 $0 \sim 3$ 各处理 16 行 Q，`warp_KV` $= 0$ 共享 K。

`Grid = ((N + 63)/64, B*H, 1)` 清楚展示了 FA2 的两层并行：`gridDim.x` 是 seqlen(Q) 维的 split-Q 并行（15.3.7 节 FA2 改动 2），`gridDim.y` 是 (B, H) 维；每个 thread block 负责连续 $B_r = 64$ 行 query。B_r, B_c 不是随意取的：$B_r = 16 \times 4 = 64$ 来自 mma.sync.m16n8k16 原子（第 十二 章）在 M 维的 4 倍堆叠，$B_c = 8 \times 8 = 64$ 来自 N 维原子的 8 倍堆叠，tile 尺寸被 Tensor Core 指令形状整除，这是分块参数化的硬约束。

flash_attn.cuh L93-110（kernel 模板参数：tile 的原子化分解）

```
template <
    const int kHeadDim,          // head dim: 32, 64, 128
    const int kMmaAtomM,         // 16 (MMA instruction M dimension)
    const int kMmaAtomN,         // 8  (MMA instruction N dimension)
    const int kMmaAtomK,         // 16 (MMA instruction K dimension)
    const int kMmaAccF32,        // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStagesK,          // pipeline stages for K: >= 1; NO stages required for Q/V
    const int kPadQ,             // Q row padding; 0 selects compact XOR swizzle
    const int kPadK,             // K row padding; 0 selects compact XOR swizzle
    const int kPadV>             // V row padding; 0 selects compact XOR swizzle
```

模板参数把 15.3.4 节的 B_r, B_c 分解到指令原子：`kMmaTileSeqLenQ` 对应 B_r 的堆叠数、`kMmaTileSeqLenK/kValTileSeqLenK` 对应 B_c，`kStagesK` 是 K tile 的流水级数（cp.async 多级缓冲，第 十一 章），`kPadQ/K/V` 控制 shared memory 的 padding 与 XOR swizzle 策略（第 十三 章）。符号对应关系：$B_r \leftrightarrow$ `kMmaTileSeqLenQ` 族、$B_c \leftrightarrow$ `kMmaTileSeqLenK` 族（全书记号表「源码对应」列）。

16.4.4 执行流程：九步与本章公式的对应

头注释 L48–59 的执行流程共九步（编号沿用源码：3a–3e 为外循环体，源码把它们计作步骤 3–7，故后续步骤直接从 8 接续）：

- **1）** 预加载 $Q[B_r, d]$ 到 smem（只加载一次，split-Q 的核心优势）；
- **2）** 外循环：沿 K 的 seqlen 分块迭代（$T_c = \text{seqlen}/B_c$）；
- **3a）** `cp.async` 加载当前 V + 预加载下一个 K（多 stage pipeline）；
- **3b）** $QK^\top$：沿 head dim 内循环 → `ldmatrix` Q/K → `HMMA16816`；
- **3c）** Online Safe Softmax：warp reduce max/row → $\exp(S \cdot \text{scale} - \text{max})$；关键：将 P 写回 `R_S` 寄存器（替换 S），`R_S` 现在存储 P matrix；
- **3d）** $P@V$：沿 V 的 B_c 维内循环 → `ldmatrix` V (transposed) → 直接用 `R_S` 做 A 矩阵；当前实现依赖同一组寄存器布局约定，避免为 $P@V$ 额外重组 P fragment；
- **3e）** Online rescaling：$O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}})\, O_{\text{old}} + P@V$；
- **8）** 最终 rescale：$O_{\text{final}} = (1/\ell_{\text{final}})\, O_{\text{final}}$；
- **9）** Epilogue：warp shuffle + 128-bit collective store。

九步里每一步的归属：步骤 1（Q 只加载一次）是 split-Q 的结构性优势；步骤 2 的外循环即 $j = 1..T_c$ 的 KV 分块遍历；步骤 3a 是 cp.async 多级预取（加载当前 V 与下一个 K 重叠）；步骤 3b 对应 $S^{[j]} = QK^{[j]\top}$（式 16.11）；步骤 3c 对应式 16.12–16.13，其中「P 写回 `R_S`」即头注释要点 5；步骤 3d 对应 $P^{[j]}V^{[j]}$；步骤 3e 对应 online rescale 式 16.17；

步骤 8 对应最终归一化式 16.18；步骤 9 是写出 epilogue。整个循环期间，每个 query 行的状态 (m_i, ℓ_i, O_i) 常驻寄存器，$S^{[j]}, P^{[j]}$ 只存在于 `R_S` 的 fragment 里，$N \times N$ 的矩阵从头到尾没有存在过。

头注释 L61–71 是 Pad 与手动 XOR swizzle 的 profile 结论（SM120）：

- XOR 能消除目标 `ldmatrix` lane pattern 的 bank conflict，但收益是局部的；XOR 本身及 `% 2` 的计算不是主要瓶颈，nvcc 已能将该式化简为 bit operations；
- compact Q/K/V XOR 改变了 `cp.async` 的 shared destination pattern：LDGSTS wavefronts 从 pad 的 30.15M 增至 68.16M （2.26×），long-scoreboard、LG-throttle、MIO-throttle 分别约为 pad 的 3.25×、2.74×、1.60×；
- compact XOR 虽节省约 5 KiB smem，但没有提高此 kernel 的 occupancy；最终约 120.0 TFLOPS，显著低于 Q/K/V `kPad=8` 的 166.6 TFLOPS；
- 所以当前 kernel 默认对 Q/K/V 都用 `kPad=8`。保留 swizzle 路径是为了学习和消融：评价 shared layout 必须同时观察 `ldmatrix` reads 与 `cp.async`/LDGSTS writes，不能只看 bank-conflict counter，也不能只优化 XOR 地址算术。

L61–71 的考据参数（SM120，B=1, H=32, N=4096, D=64）。教训：评价 shared memory layout 必须**同时**看读端（ldmatrix）与写端（cp.async），只优化 bank-conflict 计数会在写端引入新的瓶颈。机理细节（XOR 地址变换、pad 列的取值）属第 十三 章，FA 场景的完整消融在第 十七 章。

16.5　图示

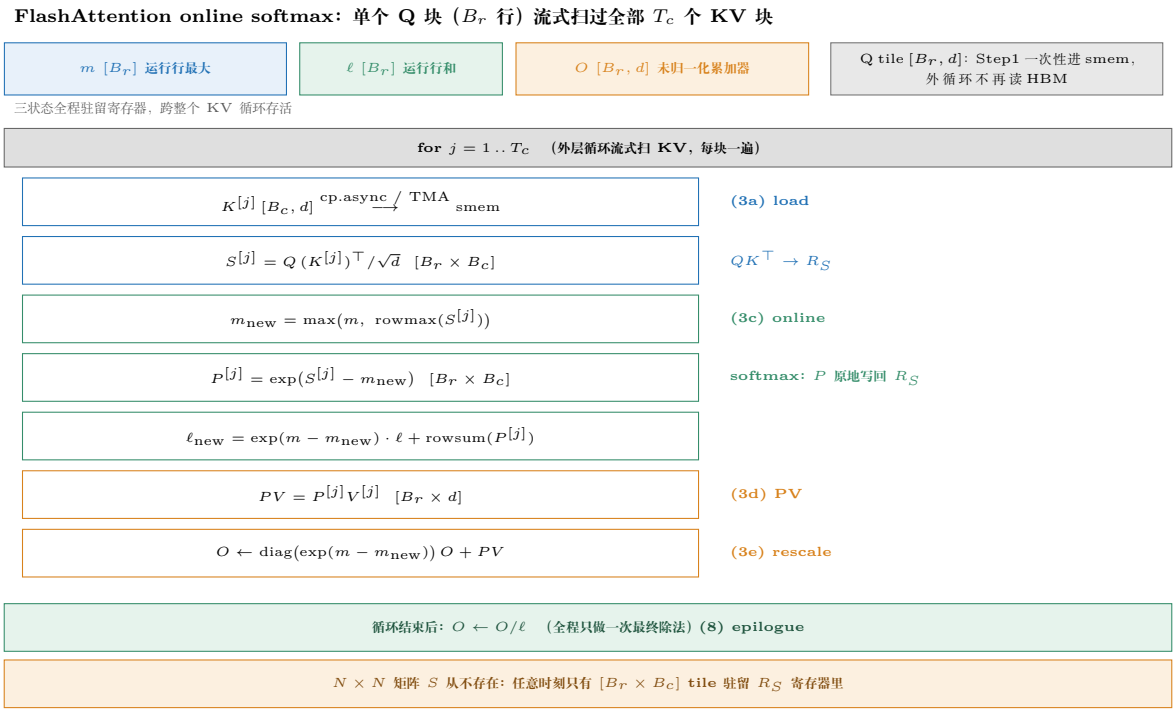


图 16.3: Attention 分块计算：单个 Q 块（B_r 行）流式扫过全部 T_c 个 KV 块（$B_c \times d$），每块依次执行 $S^{[j]} \to P^{[j]} \to P^{[j]}V^{[j]}$，并把 (m, ℓ, O) 三状态按式 16.12–16.17 联动更新，循环结束后统一除 ℓ。

16.6　性能分析

本章是数学章，不做 kernel 复测（attention kernel 的 bench 落在第 十七–十九 章，数据将落 `.tmp/book-bench/chNN/`）。这里只做一阶 roofline 对照，把 15.3.6 节的两个算术强度落到实处：

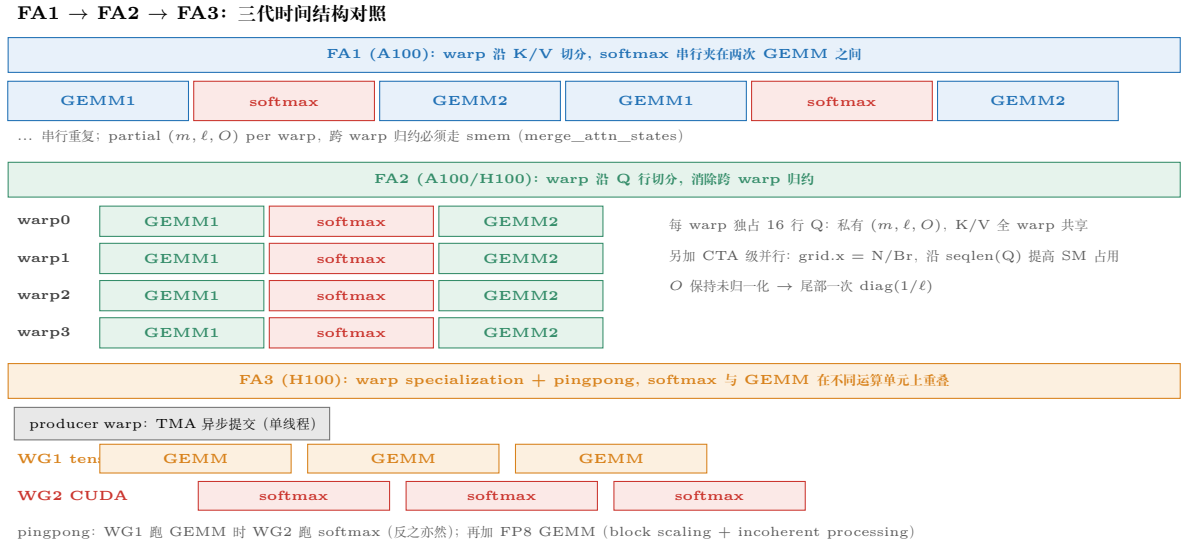


图 16.4: 三代 FlashAttention 的时间结构：FA1 的 softmax 串行夹在两次 GEMM 之间且 warp 间需要归约；FA2 靠 warp 级 split-Q 消除归约、靠 seqlen(Q) 维并行提高 SM 占用、靠未归一化 O 去掉逐块对角修正；FA3 用 warp specialization + pingpong 让 softmax 与 GEMM 在不同运算单元上重叠，并引入 FP8。

- **朴素实现**：$AI \approx d/2 = 32\,\text{FLOP/B}$（$d$=64，fp16，式 16.10）。按第 二 章的 roofline 模型 $P = \min(P_{\text{peak}}, AI \cdot BW)$，任何现代 GPU 的拐点（$P_{\text{peak}}/BW \sim 10^2\,\text{FLOP/B}$）都把它压在 memory-bound 区；且 $O(N^2)$ 的显存开销在长序列下先于运行时间使方案不可行。
- **FlashAttention**：$AI \approx B_r = 64$–$128\,\text{FLOP/B}$（式 16.23，未计 L2 对 K/V 的跨 block 复用），进入 compute-bound 区。两个外部锚点：FA2 论文在 A100 前向实测峰值利用 73%（230 TFLOPS）；本仓库手写 FA2 在 SM120（B=1, H=32, N=4096, D=64，`kPad=8`）实测 166.6 TFLOPS（flash_attn.cuh L68 头注释口径，F4 类，完整复测与本机峰值占比的口径在第 十七 章给出）。

同一 kernel 内部的布局选择也能差出 28%：L61–71 的消融显示 compact XOR（120.0 TFLOPS）反而输给 `kPad=8`（166.6 TFLOPS），因为 swizzle 把代价从 ldmatrix 读端转移到了 cp.async 写端。这说明进入 compute-bound 区只是必要条件，shared memory 读写的两端平衡才是把 73% 变成 90% 的关键，第 十三 章（swizzle）与第 十七 章（FA 的 smem 布局）继续展开。

16.7 实践经验与考据说明

实践经验 16.1 (F5)：flash_attn.cuh L7–13：三项机制的代际归属 头注释以「面试要点（FlashAttention-2 算法）」为题列出三项机制：tiling、online softmax、recomputation。按论文口径，这三项是 **FA1**（arXiv:2205.14135）的算法贡献；FA2（arXiv:2307.08691）的主题是削减非 matmul FLOPs、seqlen 维并行与 warp 级 split-Q 分工，其算法内核（online rescale）与 FA1 相同。注释把两代贡献合并在 FA2 标签下叙述，不算错误（本仓库实现确实是 FA2 形态），但面试或引述时应按 15.3.7 节的代际划分表述。已与论文逐条对照，无其它出入。

实践经验 16.2 (F3 核校通过)：flash_attn.cuh L17–24：online rescaling 五行公式 与本章式 16.12–16.18 逐项一致：`m_new`、`P_cur`（行减新 max）、`l_new`（旧和乘 $e^{m^{old}-m^{new}}$ 再加新块行和）、`O_new`（对角 rescale 后加 PV）、`O_final` 统一除 ℓ。O 未归一化 + 最终单次除法，即 FA2 论文 Algorithm 1 的形态。

实践经验 16.3 (F5)：flash_attn.cuh L19–21：scale 的安放位置与 FA2 论文不同 注释按块缩放：`m_new = max(m, row_max(S_cur * scale))`、`P_cur = exp(S_cur * scale - m_new)`，scale 在每个 KV 块内乘进 S（FA1 式）。FA2 论文为削减非 matmul FLOPs，把 $1/\sqrt{d}$ **预乘进** Q，块内不再乘。数学上等价（scale 是常数，先乘后乘只影响舍入路径），工程上是 FA2「non-matmul 最小化」的一个条目。非错误，记录以备考。

实践经验 16.4 (F5 核校通过)：引用与演进指标 L32 的 arXiv:2307.08691 编号正确（FA2，Dao 2023）。本章 15.3.7 节引用的论文指标（FA1：BERT-large 比 MLPerf 1.1 快 15%、GPT-2 端到端 3×、显存省 10–20×；FA2：约 2× vs FA1、A100 230 TFLOPS (73%)；FA3：1.5–2× vs FA2、H100 fp16 740 TFLOPS（75%）、FP8 前向近 1.2 PFLOPS）均与三篇论文摘要一致。

实践经验 16.5 (F3/F5 考据)：$1/\sqrt{d}$ 的论证出处 式 16.5 的方差推导（点积方差 $= d$）是通行教科书写法，但 Transformer 原论文 (Vaswani et al., 2017) §3.2.1 只给了启发式陈述（"We suspect that for large values of d_k..."），未做方差计算。引述时应区分「论文原文（启发式）」与「通行严格化解读（本节推导）」。

16.8　常见坑

坑一：O 的归一化口径要全局统一。FA 主循环的 O 是**未归一化**累加器（式 16.17），ch05 `merge_attn_states` 的输入 O 是 **已归一化**输出（坑一原文见第 六 章）。两套公式不可混用：把已除过 ℓ 的 O 再按式 16.17 累加，等于漏乘旧块的 ℓ；反之亦然。写任何「分块 + 合并」的 attention 代码前，先在纸上写清楚每个 O 的口径。

坑二：初始化与空行除零。m 初始化 $-\infty$（工程上 `-FLT_MAX`）、$\ell = 0$、$O = \mathbf{0}$。首个有效块的 rescale 因子 $e^{m^{(0)}-m^{(1)}} = 0$ 恰好把零状态丢弃，是安全的；但若某行所有块都被 mask（全 $-\infty$），最终 $\ell = 0$，式 16.18 除零得 `NaN`，与 ch04 坑二、ch05 坑四同源，由调用方保证行非空或显式兜底。

坑三：rescale 的时机。必须**先**算出 m^{new}、再同时用 $e^{m^{\text{old}}-m^{\text{new}}}$ 修正 ℓ 与 O、最后累加新块的 rowsum$(P^{[j]})$ 与 $P^{[j]}V^{[j]}$。顺序颠倒（例如先加 PV 再 rescale）会把新块贡献也错误缩放。自查方法：对照不变量式 16.20，任何实现写完都对一遍「这三条此刻是否成立」。

坑四：$-\infty$ 参与运算的 inf−inf。mask 位置填 $-\infty$ 时，若某行当前 m 也是 $-\infty$（首块全 mask），$\exp(S - m)$ 计算出 $-\infty - (-\infty) = \text{NaN}$。CPU 参考实现常用 `np.exp` 天然规避，CUDA 上 `__expf` 同样要防：先把 mask 位置替换成极小有限值（ch05 勘误的 $+\infty$ 清洗同类），或保证块内至少一个非 mask 位置。

坑五：精度分层要求指数与累加必须在 fp32。$P^{[j]} \le 1$ 但累加器 O 要跨 T_c 个块累加，fp16 累加会迅速漂移；exp 的溢出界（第 五 章 88.72）在 fp16 下只有 11。因此 exp、(m, ℓ)、O 的 rescale 全部走 fp32，P 只在进 PV 的 GEMM 时才降回 fp16（模板参数 `kMmaAccF32` 控制 mma 累加精度，第 十七 章展开）。

坑六：scale 二次施加或漏施加。若上游已把 $1/\sqrt{d}$ 预乘进 Q（FA2 论文形态），kernel 再乘 `S_cur * scale` 就是二次缩放，表现为注意力过尖、输出塌缩；反过来两头都不乘则过平。本仓库 kernel 是「块内乘」形态（考据框注三），替换或对比实现时要先核对 scale 归属。

16.9　面试要点速查

Q1：写出 scaled dot-product attention 并说明每个矩阵的形状。

A：$\text{Attn}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d})V$（式 16.2）；每 (b,h) 内 $Q, K, V, O \in \mathbb{R}^{N\times d}$，$S, P \in \mathbb{R}^{N\times N}$，softmax 按行。

Q2：为什么除以 $\sqrt{d}$？

A：i.i.d. 单位方差假设下点积方差为 d（式 16.5），logit 标准差 $\sqrt{d}$ 使 softmax 饱和、梯度消失（两元素情形梯度 $\le 1/4$，Δ=11 时跌到 10^{-5} 量级）；除以 $\sqrt{d}$ 后方差归一为 1。（原论文是启发式陈述，方差推导是通行解读。）

Q3：朴素 attention 的三项开销？

A：FLOPs $4N^2d$（两大 GEMM 各 $2N^2d$）；显存 $\Theta(N^2)$（S, P 物化）；HBM 流量 $\Theta(Nd + N^2)$，主导项 $4N^2$ 的中间矩阵往返。算术强度只有 $d/2$ FLOP/B，memory-bound。

Q4：默写 online rescale 更新。

A：$m^{\text{new}} = \max(m, \text{rowmax}(S^{[j]}))$；$P^{[j]} = \exp(S^{[j]} - m^{\text{new}})$；$\ell^{\text{new}} = e^{m^{\text{old}}-m^{\text{new}}}\ell^{\text{old}} + \text{rowsum}(P^{[j]})$；$O \leftarrow \text{diag}(e^{m^{\text{old}}-m^{\text{new}}})O + P^{[j]}V^{[j]}$；循环结束后 $O \leftarrow O/\ell$（只除一次）。

Q5：为什么分块算出的结果和整体算严格相等？

A：循环不变量（定义 16.1）：任意时刻 (m, ℓ, O) 是「已见 key 集合」的 max / 指数和 / 指数加权和；归纳法证明不变量逐块保持（定理 16.2），终态除 ℓ 后由 softmax 平移不变性精确等于整体计算，且与分块方式、顺序无关。

Q6：FlashAttention 不减少 FLOPs，为什么快？

A：省的是 HBM 流量与显存：IO 从 $\Theta(Nd + N^2)$ 降到 $\Theta(Nd + N^2d^2/M)$，激活显存从 $\Theta(N^2)$ 降到 $\Theta(Nd)$；算术强度从 $d/2$ 升到 B_r 量级，kernel 从 memory-bound 变 compute-bound，Tensor Core 才能达到高利用率。

Q7：FA1、FA2、FA3 各自的贡献？

A：FA1 = tiling + online softmax + 反向 recomputation（IO-aware，$\Theta(N^2d^2/M)$）；FA2 = 削非 matmul FLOPs + seqlen(Q) 维并行 + warp 级 split-Q（A100 峰值 73%）；FA3 = warp specialization + pingpong 让 softmax 与 GEMM 重叠 + FP8（block 量化 + incoherent processing，H100 fp16 峰值 75%、FP8 近 1.2 PFLOPS）。

Q8：FA2 的 warp 级 split-Q 为什么优于 FA1 的 split-KV？

A：split-KV 下各 warp 只拿到部分 key 的 (m, ℓ, O)，必须经 shared memory 做跨 warp 合并（一次同步 + 一轮读写）；split-Q 下 4 个 warp 共享同一块 K, V、各持 Q 的不同行片段，行间零依赖（行内自足性），各自独立跑完全程，零跨 warp 归约，Q 也只加载一次。

Q9：FP8 attention 为什么需要 incoherent processing？

A：Q, K 各维方差不均、存在 outlier，直接 FP8 量化时少数维度占用大量量化格点。右乘同一个正交矩阵 U（随机 Hadamard）后 $(QU)(KU)^\top = QK^\top$ 严格不变，而能量被打散到各维，block 量化的相对误差下降；配合按块缩放因子，FP8 精度可追平 fp16。

16.10 最小测试与动手实验

本章无独立 kernel 测试，这是刻意的：ch15 是纯数学章，其全部可执行结论都由前两章的测试覆盖：

- online rescale 的 (m, ℓ) 内核即第 五 章的三级 softmax 递进测试（`tests/ch04_softmax.cu`：case A/B/C 覆盖递推正确性、边界线程、大 logit 溢出）；
- 分块状态的等价合并即第 六 章的 `tests/ch05_merge_attn.cu`（「合并 == 整体 softmax 直接计算」，正是定理 16.2 的数值版：任意切分 + 合并还原整体）。

kernel 层面的 FlashAttention 正确性入口在第 十七 章的 `tests/ch16_*.cu`（对照 CPU fp64 参考，容差按 F32Acc/F16Acc 档）。

动手实验（建议在 `.tmp/book-bench/ch15/` 下进行，用 numpy/pytorch 对拍）：

1. **对拍在线分块与整体计算**（15.3.4/15.3.5 节的数值版）：

```python
import numpy as np
def attn_ref(Q, K, V):                    # (N,d) x (N,d) x (N,d)
    S = Q @ K.T / np.sqrt(Q.shape[1])
    P = np.exp(S - S.max(axis=1, keepdims=True))
    return (P / P.sum(axis=1, keepdims=True)) @ V

def attn_online(Q, K, V, Bc=32):          # tiled + online rescale
    N, d = Q.shape
    m = np.full(N, -np.inf); l = np.zeros(N); O = np.zeros((N, d))
    for j in range(0, N, Bc):
        Sj = Q @ K[j:j+Bc].T / np.sqrt(d)                # (3b)
        m_new = np.maximum(m, Sj.max(axis=1))            # (3c)
        Pj = np.exp(Sj - m_new[:, None])                 #      P -> R_S
        l = np.exp(m - m_new) * l + Pj.sum(axis=1)       # l update
        O = np.exp(m - m_new)[:, None] * O + Pj @ V[j:j+Bc]  # (3d)(3e)
        m = m_new
    return O / l[:, None]                                # (8) final

rng = np.random.default_rng(0); N, d = 256, 64
Q, K, V = (rng.standard_normal((N, d)) for _ in range(3))
print(abs(attn_online(Q, K, V) - attn_ref(Q, K, V)).max())  # ~1e-13
```

2. **次序不变性**：把 `attn_online` 的循环改为倒序或随机打乱块序，输出应逐元素不变（定理 16.2 的次序无关性）。
3. **不变量逐块自检**：在循环内断言 $\ell_i \geq e^0$ 当 max 恰在本块、以及 O/ℓ 随块数增加单调收敛到参考值，直观感受「每一步都是部分 softmax 的严格等价物」。

延伸阅读

- @DefTruth《图解大模型训练之：推理加速——从 Online-Softmax 到 FlashAttention V1/V2/V3》，https://zhuanlan.zhihu.com/p/668888063（本章主参考与风格基线：online softmax → FA 分块 → V1/V2/V3 演进的完整图解推导链）；

- @ 紫气东来《CUDA 算子系列 (七)：Self-Attention（上)》, https://zhuanlan.zhihu.com/p/695898274 与《CUDA 算子系列 (八)：Self-Attention (下)》, https://zhuanlan.zhihu.com/p/696197013 (attention 算子从数学到 CUDA 实现的工程视角，衔接第 十七 章)；
- T. Dao et al. FlashAttention. arXiv:2205.14135, 2022；T. Dao. FlashAttention-2. arXiv:2307.08691, 2023；J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, T. Dao. FlashAttention-3. arXiv:2407.08608, 2024 （三代演进的一手出处，本章 15.3.7 节的核验源)。

第十七章　FA2（一）：Split-Q + MMA 多级流水

17.1　本章导读

第 十六 章完成了 FlashAttention 的数学推导：分块 (tiling) + online softmax 让 $O(N^2d)$ 的计算不再伴随 $O(N^2)$ 的 HBM 中间矩阵读写。本章进入**工程实现**：教学源码里的 `flash_attn_mma_stages_split_q`（flash_attn.cuh L112），一个用 `mma.sync.m16n8k16` 手写的 FlashAttention-2 前向 kernel，它把 FA2 论文（arXiv:2307.08691）的两项核心改造落到了寄存器级：**split-Q 的 warp 行划分**（更好的 work partitioning）与 **Q 常驻 smem / KV 流过 smem 的多级流水**（better parallelism）。

本章源码区间为 flash_attn.cuh L5–790（Phase 8 手写 MMA 路径；L791 起的 TMA/Warp Specialization 版留给第 十八 章）。无编译宏限定，`mma.m16n8k16` 要求 `sm_80` 及以上；默认实例化（kHeadDim=64、kStagesK=2、Q/K/V pad=8）的配置为 B_r=128、B_c=64、256 线程（8 warp）。配套最小测试为 `ch16_fa2_mma.cu`（`book/tests/`）。

前置：第 十六 章（online softmax 与 online rescale 的推导，本章直接引用其结论）、第 十二 章（mma.sync 指令逐字段、A/B/C fragment 坐标公式、ldmatrix 寻址协议，本章所有 fragment 几何全部建立在式 12.1–12.3 之上）、第 十一 章（cp.async 的 commit/wait_group 计数）、第 五/六 章（online softmax 与 LSE 合并）。

17.2　问题与动机

动机一：标准 attention 的 $O(N^2)$ 中间矩阵。 $\mathrm{Attn}(Q,K,V) = \mathrm{softmax}(QK^\top/\sqrt{d})V$ 若按定义式 materialize $S = QK^\top$ 与 $P = \mathrm{softmax}(S)$，序列长 N 时要向 HBM 写回两个 $N \times N$ 矩阵。HBM 带宽比 SRAM 低一至两个数量级，注意力计算被内存读写而非计算本身支配。FA 的答案是分块：Q 按 B_r 行一块、K/V 按 B_c 行一块，S/P 只在寄存器/smem 里以 $B_r \times B_c$ 的 tile 存在，配合 online softmax（第 五 章）跨 tile 维护 (m,ℓ) 状态：$O(N^2d)$ 的 FLOPs 一个不少，但 HBM 读写降为 $O(Nd)$。这套推导在第 十六 章已完成，本章不再重复。

动机二：split-KV 的 warp 划分使 softmax 状态分裂到多个 warp。分块之后，**block 内 warp 怎么分工**是 FA1 到 FA2 的关键差异。LeetCUDA 的 FA1 式实现（`flash_attn_mma_stages_split_kv`）把 $B_r \times B_c$ 的 tile 同时按行列切给 warp：warp_QP 分行、warp_KV 分列（8-warp 版为 2×4 warp 网格）。问题在列方向：行 r 的 S 值分布在多个 warp 上，每个 warp 只见到该行**部分** KV 的打分，它们各自维护的 (m,ℓ,O) 都是**部分状态**。每个 KV tile 结束必须经 smem 中转、用第 六 章 `merge_attn_states` 的 LSE 合并语义跨 warp 归并，通信与同步的开销随 tile 迭代次数 T_c 线性重复。

动机三：split-Q 让 softmax 状态 warp 私有。 FA2 的观察（论文 §3.2）：既然 online softmax 的状态是**按行**的（每行一个 m、一个 ℓ），那就按行切 warp，每个 warp 独占 16 行 Q（一个 `m16` atom 的行数），KV tile 全 warp 共享（warp_KV≡0）。行状态从头到尾留在本 warp 的寄存器里，跨 tile 迭代零归并、零 smem 中转。按「独立 online softmax 状态数」计并行度：split-KV 下列向分裂的状态终究要归并，行方向的有效并行单元是 warp_QP 组；split-Q 下 8 个 warp 各持 16 行的**终态**，并行度从 $B \cdot H \cdot \lceil N/B_r \rceil$（block 级）直接乘上 warp 数到 $B \cdot H \cdot \lceil N/B_r \rceil \cdot 8$，且每个单元免通信。附带收益：Q tile 整个外循环只从 gmem 搬一次，K/V 分块流过 smem。

17.3　数学原理

17.3.1　记号与配置

沿用记号表：B, H, N, D 为 batch/heads/seqlen/head dim；B_r, B_c 为 Q 块/KV 块行数；m, ℓ 为 online softmax 的行 running max/sum；$s = 1/\sqrt{D}$ 为缩放因子（源码 `scale`，L128）。默认配置

$$B_r = \underbrace{16}_{kMmaAtomM} \cdot \underbrace{8}_{kMmaTileSeqLenQ} \cdot \underbrace{1}_{kValTileSeqLenQ} = 128, \qquad B_c = \underbrace{8}_{kMmaAtomN} \cdot 1 \cdot \underbrace{8}_{kValTileSeqLenK} = 64, \tag{17.1}$$

外循环次数 $T_c = \lceil N/B_c \rceil$，$D = 64$。三层参数的含义：`kMmaAtom*` 是一条 `mma.m16n8k16` 指令的形状；`kMmaTile*` 决定 warp 数（`kNumThreads`$= 32 \cdot kMmaTileSeqLenQ \cdot kMmaTileSeqLenK$，L124）；`kValTile*` 是每 warp 在该方向上重复的次数（默认除 $kValTileSeqLenK$=8 外全 1，即每个 warp 一次算满 B_c 列）。

17.3.2　Split-Q 行划分与状态的 warp 私有化

block 处理一个 Q tile：全局行区间 [`Q_tile_id` $\cdot B_r$, `Q_tile_id` $\cdot B_r + B_r$)（`Q_tile_id`=blockIdx.x）。warp $w \in [0, 8)$ 独占其中 16 行：

$$\mathcal{R}_w = [\, w \cdot 16,\ (w+1) \cdot 16), \qquad \bigcup_{w=0}^{7} \mathcal{R}_w = [0, 128), \quad \mathcal{R}_w \cap \mathcal{R}_{w'} = \emptyset \ (w \neq w'). \tag{17.2}$$

整个外循环期间，warp w 私有维护

$$m_w, \ell_w \in \mathbb{R}^{16}, \qquad O_w \in \mathbb{R}^{16 \times D}, \tag{17.3}$$

源码对应 `lane_block_row_max/sum_old`（L181–182，fp32 寄存器）与累加器 fragment `R_D`（L194）。第 十六 章的 online rescale 递推按 warp 局部化执行。对第 t 个 KV tile：

$$S_w^{(t)} = Q_w K_t^{\top} \cdot s \qquad (\texttt{R_S},\ 16\ 行 \times B_c\ 列), \tag{17.4}$$

$$m_w^{\text{new}} = \max\big(m_w^{\text{old}},\ \text{rowmax}(S_w^{(t)})\big), \tag{17.5}$$

$$P_w^{(t)} = \exp\big(S_w^{(t)} - m_w^{\text{new}}\big) \qquad (\text{写回 } \texttt{R_S} \text{ 原地替换}), \tag{17.6}$$

$$\ell_w^{\text{new}} = e^{\,m_w^{\text{old}} - m_w^{\text{new}}}\, \ell_w^{\text{old}} + \text{rowsum}\big(P_w^{(t)}\big), \tag{17.7}$$

$$O_w \leftarrow \text{diag}\big(e^{\,m_w^{\text{old}} - m_w^{\text{new}}}\big)\, O_w + P_w^{(t)} V_t, \tag{17.8}$$

循环结束后统一 $O_w \leftarrow O_w / \ell_w$。注意两个工程口径与第 十六 章纯数学版的差异：（1）源码把 s 乘进 S 的读出与 exp 的输入（`__fmaf_rn(x, scale, -m)`），max 也在乘 s 之后取（L484/L491），与数学上 $\max(sS) = s\max(S)$ 等价（$s > 0$）；（2）O 的 rescale 与累加融合为一次 FMA（3e 步），且首个 tile 用 m^{new} 直接顶替 $-\infty$ 的旧值（L661–664 的三元判断），避免 $e^{-\infty}$ 与 $0 \cdot \infty$ 型边界把 ℓ/O 污染成 NaN。

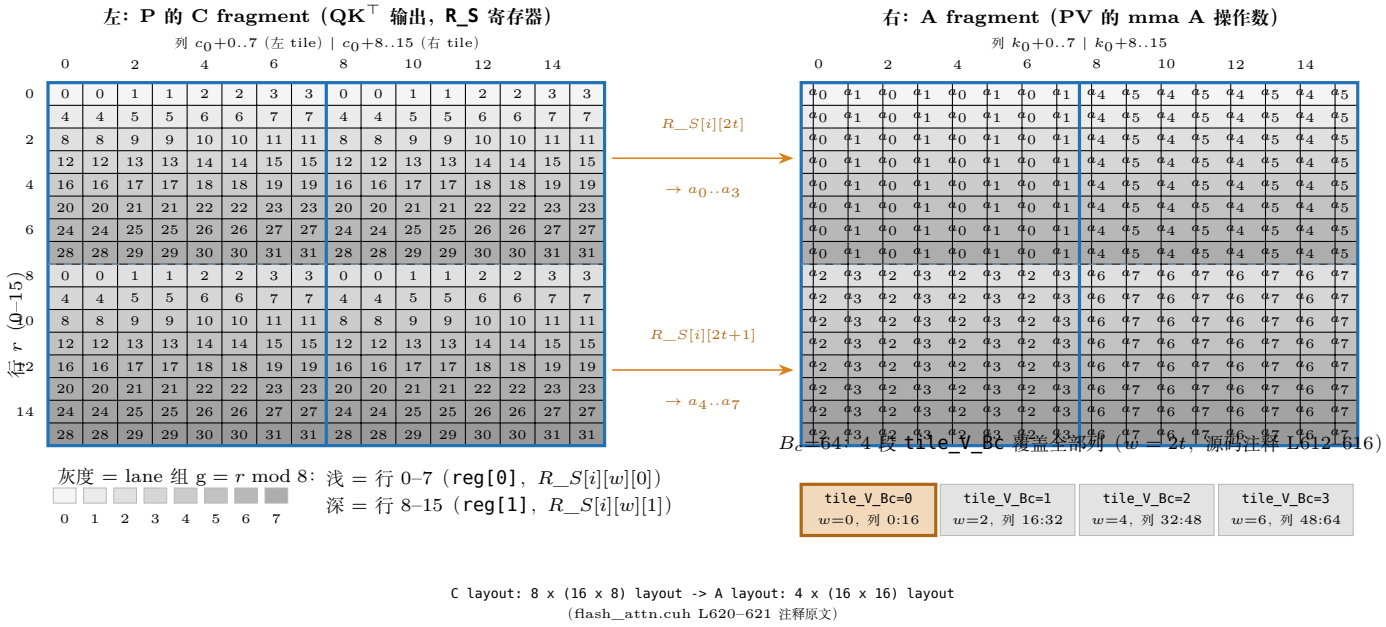
17.3.3　S 的 C fragment 如何变成 $P@V$ 的 A fragment

本章实现的关键一步：softmax 算完的 P **不离开 `R_S` 寄存器**，直接作为下一段 $P@V$ 的 mma A 操作数。可行性完全由 fragment 几何决定，用第 十二 章的坐标公式验证。

S_w 的一个 `m16n8` tile（记列基 $c_0 = 8j$）落在 `R_S[i][j][*]`，每 lane 4 个元素，按式 12.3：行 g 与 g+8，列 $c_0 + 2\,\text{tig} + (i \bmod 2)$。而 $P@V$ 需要的 A 是 `m16k16`（k 维即 KV 方向 16 列），按式 12.1 每 lane 8 个元素、四个象限：行 g/ g+8 × 列 $k_0 + \{2\text{tig}, 2\text{tig}{+}1\}$ 与 $+8$。两者逐 lane 对照：

$$\underbrace{\{S{:}c_0\}, \{S{:}c_0\}_{+8\text{行}}}_{\texttt{R_S[i][w][0..1]}}, \quad \underbrace{\{S{:}c_0{+}8\}, \{S{:}c_0{+}8\}_{+8\text{行}}}_{\texttt{R_S[i][w+1][0..1]}} \longrightarrow \{a_0, a_1, a_2, a_3\} \ \oplus\ \{a_4, a_5, a_6, a_7\}, \tag{17.9}$$

即**列相邻的两个 `m16n8` C tile 拼成一个 `m16k16` A fragment**：左 tile 给 A 的 k 左半区（a_0–a_3），右 tile 给右半区（a_4–a_7），行半区的对应关系原样保留。这就是 3d 步里 `w = tile_V_Bc * 2`（L622）的来源：$B_c = 64$ 按 16 列一段共 4 次 `tile_V_Bc`，第 t 段消费 `R_S[i][2t]` 与 `R_S[i][2t+1]`，恰覆盖列 $16t..16t{+}15$。源码注释（L602–621）画了 4-warp 版的整表，并特别提醒：这是 **本路径的布局约定**，不是「任意 MMA 的 C 与 A fragment 天然同构」的通用结论（见 16.7 节勘误与 16.8 节坑五）。



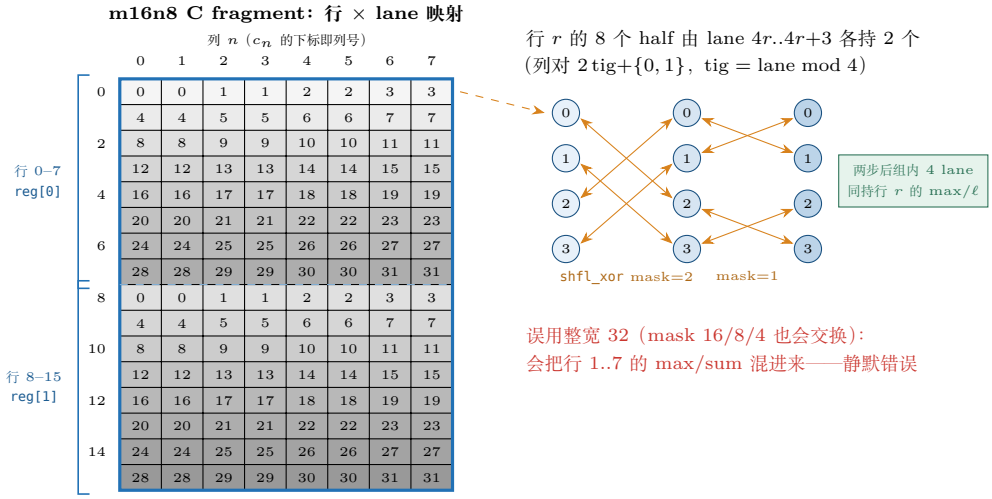
C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
（flash_attn.cuh L620–621 注释原文）

图 17.1: P 如何不出寄存器直接变成 PV 的 A fragment——两个 16 × 16 网格的逐 lane 对照（16.3.3 节式的图形化）。左：softmax 后 P 在 `R_S` 里的 C fragment 视角，两个列相邻 `m16n8` tile（列 c_0+0..7 与 c_0+8..15），cell 填持有该元素的 lane 号 $= 4(r \bmod 8) + \lfloor c/2 \rfloor$；右：同一条 mma 需要的 `m16k16` A fragment，cell 填寄存器号 $a_0..a_7$（四象限：行 × 列半区）。灰度编码 lane 组 $g = r \bmod 8$（8 档）、深浅区分 `reg[0]`/`reg[1]`——两面板同色格子即同 lane 同数据，只是换一种解读、不需要数据搬运。中缝箭头：左 tile（`R_S[i][2t]`）供 $a_0..a_3$、右 tile（`R_S[i][2t+1]`）供 $a_4..a_7$。底部：B_c=64 共 4 段 `tile_V_Bc`，每段消费两个 `m16n8` tile（$w = 2t$），高亮段即上图所示的 16 列。

B 操作数两侧不对称：$S = QK^\top$ 是 TN 布局，K 以 $[B_c, d]$ 行主序存储，等价于 $K^\top$ 列主序，`ldmatrix.x2` 不带 `.trans` 即得 `.col` 的 B fragment（与第 十二 章 11.4.6 完全相同）；$P@V$ 则是 NN 布局：B 是 V_t 的 $[B_c, d]$，mma 要求列主序而 V 是行主序存储，必须 `LDMATRIX_X2_T`（.trans，L600）转置装载。

17.3.4 行归约为什么是 4-lane 蝶形

3c 步要对 P 的每一行求 max 与 sum。由式 12.3，行 $r \in [0, 8)$ 的 8 个列元素分布在 lane $4r..4r{+}3$（$g = r$，$\text{tig} = 0..3$ 各持 2 个），行 $r{+}8$ 同一组 lane 的第二个寄存器。所以正确的归约宽度是 4：先在 lane 内跨 8 个 j tile 做局部归约（覆盖 B_c 列），再 `warp_reduce_max<4>` / `warp_reduce_sum<4>`（L500/L553），用 XOR 掩码 $\{2, 1\}$ 两步蝶形（第 三 章），只在本 4-lane 组内交换。若误用整宽 32 的归约，会把**不同行**的 max/sum 混在一起，结果静默错误。



行 r 与 r+8 由同一 4-lane 组分别用 `reg[0]`/`reg[1]` 归约——这正是 `warp_reduce_max<4>`/`sum<4>`（L500/L553）宽度取 4 的依据。

图 17.2：m16n8 C fragment 的行 × lane 映射与 `warp_reduce<4>` 的宽度依据。左：行 r 与 r+8 的 8 个列元素同由 lane $4r..4r$+3 持有（tig = 0..3 各一对），行 0–7 走 `reg[0]`、行 8–15 走 `reg[1]`；右：组内两步 XOR 蝶形（mask 2 → mask 1）后 4 个 lane 同持该行归约值。若误用整宽 32，mask 16/8/4 的交换会把别的行的值混进来——这是 3c 步最典型的静默错误。

17.4　设计与实现

先看源码头注释（L41–59）：它交代 tile 设计、warp 布局，再给出 9 步执行流程总览，后续各小节逐步展开。Q/K/V/O 的形状均为 $[B, H, N, d]$；tile 设计（以 `kHeadDim=64` 为例）：

- B_r：`kMmaAtomM` × `kMmaTileSeqLenQ` × `kValTileSeqLenQ`，即 $16 \times \{4, 8\} \times 1 = \{64, 128\}$；
- B_c：`kMmaAtomN` × `kMmaTileSeqLenK` × `kValTileSeqLenK`，即 $8 \times 1 \times 8 = 64$；
- warp 布局：4 个 warp，`warp_QP` 取 $0 \sim 3$ 各处理 16 行 Q，`warp_KV` = 0 共享 K。

执行流程九步（编号沿用源码：3a–3e 为外循环体，后续步骤从 8 接续）：

- **1）** 预加载 $Q[B_r, d]$ 到 smem（只加载一次，split-Q 的核心优势）；
- **2）** 外循环：沿 K 的 seqlen 分块迭代（$T_c = \text{seqlen}/B_c$）；
- **3a）** `cp.async` 加载当前 V + 预加载下一个 K（多 stage pipeline）；
- **3b）** $QK^\top$——沿 head dim 内循环 → `ldmatrix` Q/K → `HMMA16816`；
- **3c）** Online Safe Softmax——warp reduce max/row → $\exp(S \cdot \text{scale} - \text{max})$；关键：将 P 写回 `R_S` 寄存器（替换 S），`R_S` 现在存储 P matrix；
- **3d）** $P@V$——沿 V 的 B_c 维内循环 → `ldmatrix` V (transposed) → 直接用 `R_S` 做 A 矩阵；当前实现依赖同一组寄存器布局约定，避免为 $P@V$ 额外重组 P fragment；
- **3e）** Online rescaling——$O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}})\, O_{\text{old}} + P@V$；
- **8）** 最终 rescale：$O_{\text{final}} = (1/\ell_{\text{final}})\, O_{\text{final}}$；
- **9）** Epilogue：warp shuffle + 128-bit collective store。

（该注释以 B_r=64、4-warp 配置为例；本章默认实例化是 B_r=128、8-warp，行划分逻辑相同，见 16.7 节勘误。）

17.4.1 模板参数、线程索引与 smem 布局

flash_attn.cuh L93-124（17 个模板参数与 B_r/B_c/kNumThreads 推导）

```
template <
    const int kHeadDim,          // head dim: 32, 64, 128
    const int kMmaAtomM,         // 16 (MMA instruction M dimension)
    const int kMmaAtomN,         // 8  (MMA instruction N dimension)
    const int kMmaAtomK,         // 16 (MMA instruction K dimension)
    const int kMmaAccF32,        // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStagesK,          // pipeline stages for K: >= 1; NO stages required for Q/V
    const int kPadQ,             // Q row padding; 0 selects compact XOR swizzle
    const int kPadK,             // K row padding; 0 selects compact XOR swizzle
    const int kPadV>             // V row padding; 0 selects compact XOR swizzle
__global__ void __launch_bounds__(kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK)
flash_attn_mma_stages_split_q(
  half *Q, half *K, half *V, half *O, int N, int H) {
  static_assert(kStagesK >= 1, "kStagesK must be >= 1");
  static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
  static_assert(kPadQ >= 0 && kPadK >= 0 && kPadV >= 0,
                "Q/K/V padding must be non-negative");
  constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
  constexpr bool kSwizzleQ = kPadQ == 0;
  constexpr bool kSwizzleK = kPadK == 0;
  constexpr bool kSwizzleV = kPadV == 0;
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 16*8*1=128
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 8*1*8=64
  constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 32*8*1=256
```

flash_attn.cuh L131-145（block 索引、warp_QP/warp_KV 与 per-head 基址）

```
  const int Nb_id = blockIdx.y / H; // batch id
  const int Nh_id = blockIdx.y % H; // head id
  const int Q_tile_id = blockIdx.x; // tile id along Q's M dim (Br)
  const int tid = threadIdx.x;
  const int warp_id = tid / kWarpSize;
  const int lane_id = tid % kWarpSize;
  const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
  const int warp_KV = 0; // 所有 warp 共享 K（减少跨 warp 通信）

  // Global memory base offsets for this (batch, head)
  // 这里默认 Q/K/V 共享同一 per-head 基址布局，对应 self-attention 场景
  const int Q_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
  const int K_gmem_offset = Q_gmem_offset;
  const int V_gmem_offset = Q_gmem_offset;
  const int O_gmem_offset = Q_gmem_offset;
```

grid 为 $(\lceil N/B_r \rceil,\ B \cdot H)$：blockIdx.y 打包 batch 与 head（L132–133 拆包），blockIdx.x 是 Q tile 号。`warp_QP = warp_id`、`warp_KV = 0`（L137–138），split-Q 的全部含义就这两行。Q/K/V/O 共享同一 per-head 基址（L143–145，self-attention 场景），O 与 Q 同形。

flash_attn.cuh L147-176（协作加载映射、越界早退与 smem 三段布局）

```cpp
  // Thread-to-smem mapping for cooperative load
  int load_smem_Q_Br = tid / (kNumThreads / Br);
  int load_smem_Q_d = (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
  int load_smem_K_Bc = tid / (kNumThreads / Bc);
  int load_smem_K_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
  int load_smem_V_Bc = tid / (kNumThreads / Bc);
  int load_smem_V_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));

  int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
  if (load_gmem_Q_Br >= N)
    return;

  // Shared memory layout
  // Q/K/V independently use padded row-major when kPad* > 0 and compact XOR
  // swizzle when kPad* == 0. Padding and XOR are never combined per operand.
  // The swizzled physical layout is [col / 16][row][16]; swizzle<16>() selects
  // the 0/8 phase inside the final 16-half tile.
  extern __shared__ half smem[];
  constexpr int Q_tile_size = Br * (kHeadDim + kPadQ);
  constexpr int K_tile_size = Bc * (kHeadDim + kPadK);
  constexpr int kSmemStrideQ = kHeadDim + kPadQ;
  constexpr int kSmemStrideK = kHeadDim + kPadK;
  constexpr int kSmemStrideV = kHeadDim + kPadV;
  half *Q_tile_smem = smem;
  half *K_tile_smem = Q_tile_smem + Q_tile_size;
  half *V_tile_smem = K_tile_smem + kStagesK * K_tile_size;

  uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
  uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
  uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
```

smem 三段：Q 一段（$B_r \times (D+\text{pad})$）、K $kStagesK$ 段、V 一段（L165–172）。行距 `kSmemStrideQ/K/V` $= D+kPad$（L167–169），默认 pad=8 个 half：行距 72 half = 144 B = 36 个 bank 字，相邻行错开 $36 \bmod 32 = 4$ 个 bank。ldmatrix 的每个 8×8 矩阵相位读 8 行 × 16 B：8 行的起始 bank $\{4r+4h\}_{r=0}^{7}$ 恰好铺满 32 个 bank 各一次，conflict-free；若不 pad（行距 128 B），8 行起始 bank 全相同，同一相位的 8 段 16 B 全部压在 4 个 bank 上串行化。这与第 八 章 transpose 的 padding 分析、第 十三 章的 swizzle 是同一主题的三种解法；本章性能节会给出 pad 与 XOR swizzle 的实测对比（源码注释 L61–71）。

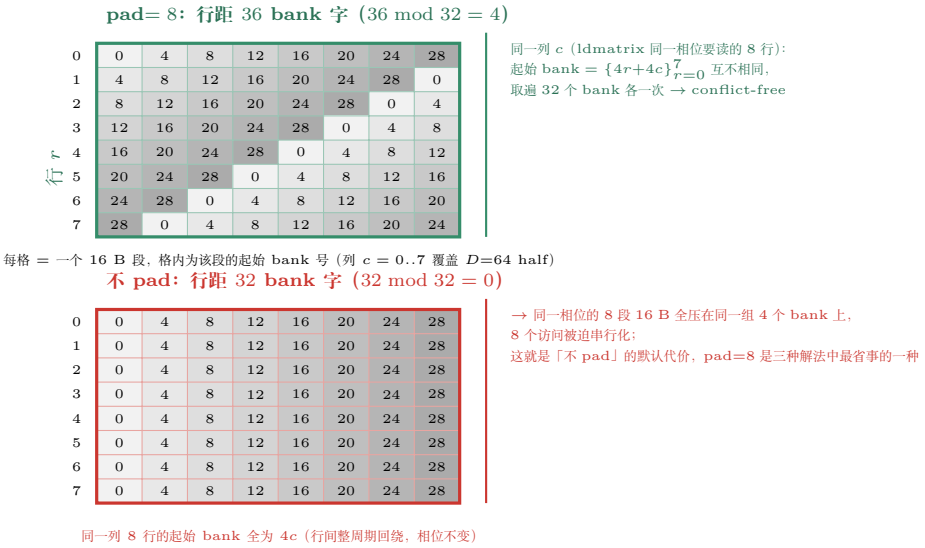


图 17.3: pad=8 与不 pad 的 ldmatrix bank 相位对照（16.4.1 节 smem 三段布局中 pad 分析的图形化）。上：`kSmemStride`= D+pad = 72 half 时一行 144 B = 36 bank 字，第 r 行相对第 0 行整体错开 $36r \bmod 32 = 4r$ 个 bank；任一列 c（ldmatrix 同一相位读的 8 行）的起始 bank 为 $\{4r{+}4c\}$，取遍 32 个 bank 各一次——8 段 16 B 恰好铺满全部 bank，conflict-free。下：不 pad 时行距 128 B = 32 bank 字，整周期回绕后 8 行起始 bank 全为 $4c$，同一相位的 8 段 16 B 压在同一组 4 个 bank 上串行化。格内数字为「该 16 B 段的起始 bank 号」，灰度按 bank 号每 4 个一档。

加载映射：256 线程协作搬 B_r=128 行 Q，每线程负责半行（`load_smem_Q_Br = tid/2`，d 向 64 个 half 分给 2 线程 × 4 段 16 B）；K/V 按 B_c=64 行、每线程管 4 行的起点（`tid/4`）。`CP_ASYNC_CG` 每次 16 B，与第 十二 章的搬运粒度一致。

17.4.2 寄存器工作集：状态、操作数与累加器

flash_attn.cuh L178-197（online softmax 状态与 $R_Q/R_K/R_V/R_S/R_O/R_D$ 寄存器组）

```
// Online Softmax persistent state
// lane_block_row_max_old[i][r]: running max for row r of warp tile i
// lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
float lane_block_row_max_old[kValTileSeqLenQ][2];
float lane_block_row_sum_old[kValTileSeqLenQ][2];
fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

uint32_t R_Q[kValTileSeqLenQ][4];  // Q regs
uint32_t R_K[kValTileSeqLenK][2];  // K regs
uint32_t R_V[kValTileHeadDimV][2]; // V regs
// R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
// f16 acc: 每 tile 2 个 uint32（4 half）；f32 acc: 每 tile 4 个 uint32（4 f32）。
// 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs]; // S=Q@K^T / P=softmax(S)
uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O for current tile
uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O accumulator (final output)

fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);
```

每 lane 常驻工作集（f16 acc 路径）：`R_Q[1][4]` 4 个 `.b32`、`R_K[8][2]` 16 个、`R_V[8][2]` 16 个、`R_S[1][8][2]` 16 个、`R_O/R_D[1][8][2]` 各 16 个，加 (m, ℓ) 4 个 fp32 与地址变量，约百个寄存器。手写路径的「寄存器白盒」收益（第 十二 章动机一）在 attention 里同样成立。`R_S` 先后存储两种数据（先 S 后 P）、`R_D` 跨 tile 持有未归一化的 O，是本 kernel 寄存器编排的两条主线。

17.4.3　Step 1/2：Q 一次性预载，K 预热 kStagesK−1 级

flash_attn.cuh L199-226（Step 1：Q tile 只搬一次；pad 路径逐 16 B cp.async）

```
  // ============================================================================
  // Step 1: 加载 Q[Br, d] 到 shared memory（整个外循环只加载一次）
  // ============================================================================
  {
    int load_gmem_Q_addr =
        Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
    if constexpr (kSwizzleQ) {
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
        int col = load_smem_Q_d + i;
        uint32_t ptr = smem_Q_base_ptr +
            ((col / kMmaAtomK) * Br * kMmaAtomK +
             load_smem_Q_Br * kMmaAtomK +
             swizzle<kMmaAtomK>(load_smem_Q_Br, col % kMmaAtomK)) *
                sizeof(half);
        CP_ASYNC_CG(ptr, &Q[load_gmem_Q_addr + i], 16);
      }
    } else {
      uint32_t load_smem_Q_ptr =
          smem_Q_base_ptr +
          (load_smem_Q_Br * kSmemStrideQ + load_smem_Q_d) * sizeof(half);
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
        CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
      }
    }
    CP_ASYNC_COMMIT_GROUP();
  }
```

flash_attn.cuh L228-266（Step 2：预载前 kStagesK−1 个 K tile 并 wait 排水）

```
  // ============================================================================
  // Step 2: 预加载前 (kStagesK-1) 个 K tile（多 stage pipeline 预热）
  // 注意：Q 由 blockIdx.x 固定到当前 Q tile；而 K/V 的 seqlen 遍历始终从 tile 0 开始，
  // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
  // ============================================================================
  if constexpr (kStagesK > 1) {
#pragma unroll
    for (int stage = 0; stage < (kStagesK - 1); ++stage) {
      int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
      int load_gmem_K_addr =
          K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
      if constexpr (kSwizzleK) {
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          int col = load_smem_K_d + i;
          uint32_t ptr = smem_K_base_ptr +
              (stage * K_tile_size +
               (col / kMmaAtomK) * Bc * kMmaAtomK +
               load_smem_K_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
            sizeof(half);
          CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
        }
      } else {
        uint32_t load_smem_K_ptr =
            smem_K_base_ptr +
```

```
            (stage * K_tile_size + load_smem_K_Bc * kSmemStrideK +
             load_smem_K_d) *
                sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
        }
      }
      CP_ASYNC_COMMIT_GROUP();
    }
    CP_ASYNC_WAIT_GROUP(kStagesK - 2);
    __syncthreads();
  }
```

Step 1 结束只 `commit` 不 `wait`，Q 的搬运与 Step 2 的 K 预载同批在途。Step 2 预载 $kStagesK-1=1$ 个 K tile 后 `wait_group(kStagesK-2)=wait(0)` 排空在途组并同步（L264–265）。注意 Q 的就绪并没有单独的 wait：cp.async 的组计数对本线程是全局的，Step 2 末的 wait(0) 已把 Q 组连带等完，L265 的一次 `__syncthreads()` 同时保护了 Q 与 K_0 的写侧可见性。

17.4.4 Step 3 外循环：3a 多级流水加载

flash_attn.cuh L273-307（外循环头 + 3a：当前 V tile 加载，单级即可）

```
  for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
    int smem_sel = tile_K_seqlen % kStagesK;
    int smem_sel_next = (tile_K_seqlen + (kStagesK - 1)) % kStagesK;

    // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
    // kStagesK>1: V 加载当前 tile, K 预取下一 tile (pipeline)
    // kStagesK=1: V 和 K 都加载当前 tile (smem_sel==smem_sel_next==0) ,
    //   无 pipeline 预取, 每轮 QK 前必须 wait 当前 K 就绪。
    // Load current V tile (no pipeline for V — one stage is enough)
    {
      int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
      int load_gmem_V_addr =
          V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
      if constexpr (kSwizzleV) {
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          int col = load_smem_V_d + i;
          uint32_t ptr = smem_V_base_ptr +
              ((col / kMmaAtomK) * Bc * kMmaAtomK +
               load_smem_V_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(load_smem_V_Bc, col % kMmaAtomK)) *
                  sizeof(half);
          CP_ASYNC_CG(ptr, &V[load_gmem_V_addr + i], 16);
        }
      } else {
        uint32_t load_smem_V_ptr =
            smem_V_base_ptr +
            (load_smem_V_Bc * kSmemStrideV + load_smem_V_d) * sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
        }
      }
      CP_ASYNC_COMMIT_GROUP();
    }
```

flash_attn.cuh L309-340（3a 续：$kStagesK>1$ 时预取下一 K tile）

```
    if constexpr (kStagesK > 1) {
      // Prefetch next K tile (pipelined)
      if ((tile_K_seqlen + 1) < Tc) {
        int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
        int load_gmem_K_addr =
            K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
        if constexpr (kSwizzleK) {
#pragma unroll
          for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
            int col = load_smem_K_d + i;
            uint32_t ptr = smem_K_base_ptr +
               (smem_sel_next * K_tile_size +
                (col / kMmaAtomK) * Bc * kMmaAtomK +
                load_smem_K_Bc * kMmaAtomK +
                swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
                  sizeof(half);
            CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
          }
        } else {
          uint32_t load_smem_K_ptr =
              smem_K_base_ptr +
              (smem_sel_next * K_tile_size +
               load_smem_K_Bc * kSmemStrideK +
               load_smem_K_d) *
                  sizeof(half);
#pragma unroll
          for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
            CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
          }
        }
        CP_ASYNC_COMMIT_GROUP();
      }
```

流水编排的要点：**K 双缓冲、V 单级**。迭代 t 内 3a 依次提交两组搬运：V_t（当前）与 K_{t+1}（预取，写 `smem_sel_next` 缓冲）。V_t 的在途窗口恰好覆盖 3b（QK mma）与 3c（softmax）两段计算：softmax 全在 CUDA Core/SFU 上跑，掩盖了 V 的搬运延迟；K_{t+1} 则有整整一个迭代的余量。这与第 十二 章 GEMM 的「kStages 全操作数统一多级」不同：attention 的两段 mma 中间夹了一段非 mma 计算，天然提供了第二级的隐藏窗口，所以 `kStagesK`=2 就够（源码注释 L281「one stage is enough for V」）。`#pragma unroll 1`（L272）刻意不展开外循环：T_c 是运行期值，展开只会膨胀代码。$kStagesK$=1 的退化分支（L341–374）每轮现搬当前 K 并在 3b 前 wait(0)，无流水，用于消融对照。

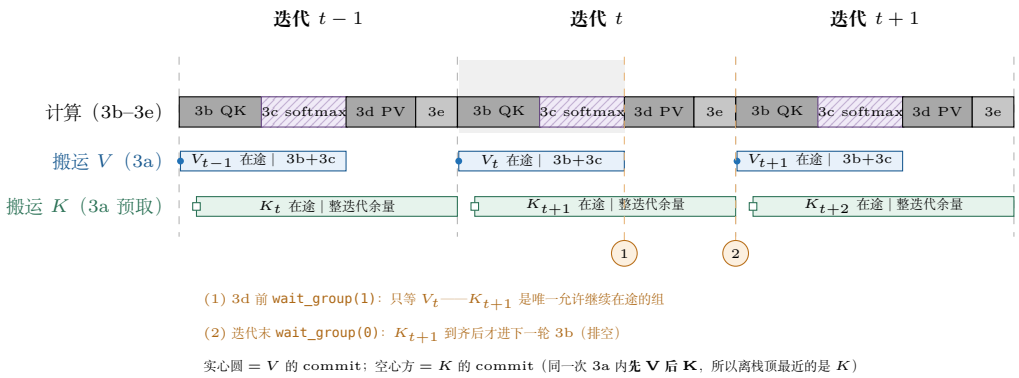


图 17.4: **K 双缓冲、V 单级**的流水时序（16.4.4 节 3a 步）。三个稳态迭代 $t{-}1/t/t{+}1$：每个迭代内 3a 依次提交两组搬运——V_t（实心圆，写当前 V buffer）与 K_{t+1}（空心方，写 `smem_sel_next`）。V_t 的在途窗口（蓝条）恰好覆盖同迭代的 3b（QK mma）与 3c（softmax）——softmax 全在 CUDA Core/SFU 上跑，掩盖了 V 的搬运延迟；K_{t+1} 则有整整一个迭代的余量。两处等待：(1) 3d 前的 `wait_group(1)` 只等 V_t，把 K_{t+1} 留在途；(2) 迭代末（3e 之后）的 `wait_group(0)` 等 K_{t+1} 到齐才进下一轮 3b。与第 十二 章 GEMM 的「kStages 全操作数统一多级」不同——attention 两段 mma 之间夹着一段非 mma 计算，天然提供了第二级的隐藏窗口，所以 `kStagesK`=2 就够（「one stage is enough for V」）。

17.4.5 3b：$S = QK^\top$ 的 ldmatrix + mma

flash_attn.cuh L383-408（沿 d 内循环：Q 的 ldmatrix.x4，行由 warp_QP 定位）

```
    fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
#pragma unroll
    for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
      // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        int warp_smem_Q_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
        int lane_smem_Q_Br =
            warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
        int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
        uint32_t lane_smem_Q_ptr;
        if constexpr (kSwizzleQ) {
          lane_smem_Q_ptr = smem_Q_base_ptr +
              ((lane_smem_Q_d / kMmaAtomK) * Br * kMmaAtomK +
               lane_smem_Q_Br * kMmaAtomK +
               swizzle<kMmaAtomK>(lane_smem_Q_Br,
                                  lane_smem_Q_d % kMmaAtomK)) *
                 sizeof(half);
        } else {
          lane_smem_Q_ptr = smem_Q_base_ptr +
              (lane_smem_Q_Br * kSmemStrideQ + lane_smem_Q_d) * sizeof(half);
        }
        LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                    lane_smem_Q_ptr);
      }
```

flash_attn.cuh L410-436（K 的 ldmatrix.x2：行主序存储免.trans，stage 由 smem_sel 选择）

```
      // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
      // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        int warp_smem_K_Bc =
            warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
```

```
        int lane_smem_K_Bc =
            warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
        int lane_smem_K_d =
            tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
        uint32_t lane_smem_K_ptr;
        if constexpr (kSwizzleK) {
          lane_smem_K_ptr = smem_K_base_ptr +
              (smem_sel * K_tile_size +
               (lane_smem_K_d / kMmaAtomK) * Bc * kMmaAtomK +
               lane_smem_K_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(lane_smem_K_Bc,
                                  lane_smem_K_d % kMmaAtomK)) *
                 sizeof(half);
        } else {
          lane_smem_K_ptr = smem_K_base_ptr +
            (smem_sel * K_tile_size + lane_smem_K_Bc * kSmemStrideK +
               lane_smem_K_d) *
                 sizeof(half);
        }
        LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
      }
```

flash_attn.cuh L438-455（8 个 m16n8 tile 的 mma：$R_S = R_Q \times R_K + R_S$）

```
      // MMA: S[tile] += Q[tile] @ K^T[tile]
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          if constexpr (kMmaAccF32) {
            HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
                         R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                         R_K[j][0], R_K[j][1],
                         R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
          } else {
            HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
                      R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
                      R_S[i][j][1]);
          }
        }
      }
    } // end loop over d
```

沿 d 方向 $D/kMmaAtomK = 4$ 次内循环，每次：Q 用 ldmatrix.x4 装 m16k16（每 warp 只有 1 个 kValTileSeqLenQ 块，地址 warp_QP*16 + lane%16 行、tile_K_d*16 + (lane/16)*8 列，与第 十二 章图 12.4 的四象限映射相同）；K 用 ldmatrix.x2 装 k16n8（8 个 j 块覆盖 B_c 列，读 smem_sel 级）；然后 8 条 HMMA16816。每 warp 每 tile 共 $4 \times 8 = 32$ 条 mma，产出 $\mathcal{R}_w$ 行 $\times$ B_c 列的完整 S_w。Q fragment R_Q 沿 d 循环重装。它不像 GEMM 的 A 那样跨 k 复用，但 smem 源数据跨整个外循环 T_c 次迭代不变，重装命中 L1 常驻数据，代价极低。kMmaAccF32=1 时换 HMMA16816F32（fp32 累加，4 个累加寄存器，L443–447），语义见第 十二 章 11.3.1。

17.4.6　3c：online softmax——寄存器内的三遍扫描

flash_attn.cuh L458-474（3c 头注释：C fragment 的 lane 分布与 4-lane 归约依据）

```
    // ============================================================================
    // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
    // ============================================================================
    // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
    //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
```

```
    //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
    //   - lane 0~3 持有 row 0 的片段，lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
    //   - 对于 rows 8~15 也是同样的 lane 分组，只是读取的是 reg[1]
    // 这就是为什么后面做 row max / row sum 时，warp 内真正参与同一行归约的是
    // {0,4,8,...,28} 这一类 4-lane 子组，而不是整 warp 32 个线程。
    // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
    // kValTileSeqLenK tiles.

    float lane_row_max_new[kValTileSeqLenQ][2];
    float lane_row_sum_new[kValTileSeqLenQ][2];
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
```

flash_attn.cuh L477-503（Pass 1：lane 内跨 j tile 求 max，再 4-lane 蝶形）

```
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        float tmp_max_0, tmp_max_1;
        if constexpr (kMmaAccF32) {
          float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
          tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
          tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
        } else {
          float2 t_reg_S_0 =
              __half22float2(HALF2(R_S[i][j][0])); // rows 0~7:  {c0, c1}
          float2 t_reg_S_1 =
              __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
          tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
          tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
        }
        lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
        lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
      }
      // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
      // {0,4,8,...,28} hold valid data)
      lane_row_max_new[i][0] =
          warp_reduce_max<4, float>(lane_row_max_new[i][0]);
      lane_row_max_new[i][1] =
          warp_reduce_max<4, float>(lane_row_max_new[i][1]);
    }
```

flash_attn.cuh L505-531（Pass 2 之 f32 acc 路径：$P = \exp(Ss - m)$ 写回、累加 ℓ）

```
    // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
    // 面试关键点：这里将 P 写回 R_S 寄存器!
    // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
    // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费，无需额外重组。
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
      // m_new = max(m_old, m_cur)
      float block_row_max_new_0 =
          max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
      float block_row_max_new_1 =
          max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        if constexpr (kMmaAccF32) {
          float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
```

```
          half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
          t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
          t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
          t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
          t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
          lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
          lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
          t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
          t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
          t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
          t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
```

flash_attn.cuh L532-556（Pass 2 之 f16 acc 路径与 4-lane sum 归约）

```
        } else {
          float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
          float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
          t_reg_S_0.x =
              __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
          t_reg_S_0.y =
              __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
          t_reg_S_1.x =
              __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
          t_reg_S_1.y =
              __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
          lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
          lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
          HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
          HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
        }
      }

      // Warp-level reduce sum (kWarpWidth = 4, same as max)
      lane_row_sum_new[i][0] =
          warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
      lane_row_sum_new[i][1] =
          warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
    }
    __syncthreads();
```

Pass 1 求本 tile 的行 max：每 lane 先在 8 个 j tile 上局部归约（每 tile 贡献 max 的一对元素，f16 路径经 `__half22float2` 升精度再乘 s，L487–492），再 `warp_reduce_max<4>`。Pass 2 完成 $m^{\text{new}} = \max(m^{\text{old}}, m_{\text{tile}})$（L512–515）、逐元素 $P = \exp(\text{FMA}(S, s, -m^{\text{new}}))$ **原地写回 `R_S`**（f16 路径 `__float22half2_rn`，L545–546；f32 路径写 half 视图，L528–531，两条路径的 P 都落 half 精度，这是精度分析的关键输入）、行内求和得 ℓ 增量并 4-lane 归约。exp 用 `__expf`（SFU 快速近似），max/sum 全程 fp32。

17.4.7　3d：$P@V$——P 不出寄存器

flash_attn.cuh L562-601（V 就绪等待 + `ldmatrix.x2.trans` 装载 V）

```
    if constexpr (kStagesK > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(1);
      } else {
        CP_ASYNC_WAIT_GROUP(0);
      }
    } else {
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();
```

```

    fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_O, 0);

    // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
    // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
#pragma unroll
    for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
      // ldmatrix.x2.trans: load V[Bc,d] with transposition
      // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
      // transposed ldmatrix
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        int warp_smem_V_d =
            warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
        int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
        int lane_smem_V_d = warp_smem_V_d;
        uint32_t lane_smem_V_ptr;
        if constexpr (kSwizzleV) {
          lane_smem_V_ptr = smem_V_base_ptr +
              ((lane_smem_V_d / kMmaAtomK) * Bc * kMmaAtomK +
               lane_smem_V_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(lane_smem_V_Bc,
                                  lane_smem_V_d % kMmaAtomK)) *
                 sizeof(half);
        } else {
          lane_smem_V_ptr = smem_V_base_ptr +
              (lane_smem_V_Bc * kSmemStrideV + lane_smem_V_d) * sizeof(half);
        }
        LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
      }
```

flash_attn.cuh L602-641（P 布局约定注释、w=2t 索引与 $R_O = P \times R_V$）

```

      // P matrix layout in R_S[i][j][2]:
      //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
      //   |   64x64   |      warp_KV 0       |
      //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
      //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
      //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
      //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
      // tile_V_Bc selects which 16-column slice of P to use:
      //   tile_V_Bc=0 → cols  0:16 → MMA indices 0,1  → w=0
      //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3  → w=2
      //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5  → w=4
      //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7  → w=6
      // 对应的 MMA A fragment 布局可以这样记:
      //   - rows 0~7:  lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
      //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
      // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
      // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
      // fragment 都无条件成立的通用结论。layout转换逻辑:
      //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
      int w = tile_V_Bc * 2;
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          if constexpr (kMmaAccF32) {
            HMMA16816F32(R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3],
                         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
```

```
                        R_V[j][0], R_V[j][1],
                        R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3]);
          } else {
            HMMA16816(R_O[i][j][0], R_O[i][j][1],
                      R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
                      R_V[j][0], R_V[j][1],
                      R_O[i][j][0], R_O[i][j][1]);
          }
        }
      }
    } // end for tile_V_Bc
    __syncthreads();
```

进 3d 前先等 V_t：非尾部迭代 `wait_group(1)`（允许 K_{t+1} 那一组仍在途），尾部 `wait_group(0)`（L562–571）。装载侧沿 KV 方向 $B_c/kMmaAtomK = 4$ 次 `tile_V_Bc`，每次 `ldmatrix.x2.trans` 装 16 行 × 8 列的 V 块（B 侧），A 侧直接把 `R_S[i][w]`, `R_S[i][w+1]` 四个寄存器按 [左上, 左下, 右上, 右下] 作为 `HMMA16816` 的输入（L633–636），16.3.3 节的拼接几何在此落地。每 warp 每 tile 又是 $4 \times 8 = 32$ 条 mma，产出 `R_O`（16 行 × 64 列，本 tile 的 PV_t，未 rescale）。

17.4.8　3e：online rescale 与状态推进

flash_attn.cuh L643-669（m 合并、首迭代边界处理与 rescale 系数）

```
    // ============================================================================
    // 3e: Online rescaling — O_new = exp(m_old - m_new) * O_old + P@V
    // ============================================================================
    // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 O 与 l 的 rescale
#pragma unroll
    for (int i = 0; i < kValTileSeqLenP; ++i) {
      float block_row_max_new_0 = lane_row_max_new[i][0];
      float block_row_max_new_1 = lane_row_max_new[i][1];
      float block_row_sum_new_0 = lane_row_sum_new[i][0];
      float block_row_sum_new_1 = lane_row_sum_new[i][1];

      float block_row_max_old_0 = lane_block_row_max_old[i][0];
      float block_row_max_old_1 = lane_block_row_max_old[i][1];

      block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
      block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

      // Handle first iteration: m_old = -inf, need to use m_new directly
      block_row_max_old_0 =
          (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
      block_row_max_old_1 =
          (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);

      float rescale_o_factor_0 =
          __expf(block_row_max_old_0 - block_row_max_new_0);
      float rescale_o_factor_1 =
          __expf(block_row_max_old_1 - block_row_max_new_1);
```

flash_attn.cuh L671-694（$O \leftarrow \alpha O_{\text{old}} + PV$ 的融合 FMA，f16/f32 两路）

```
      // Rescale O_old + Add P@V in one fused step
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        if constexpr (kMmaAccF32) {
          float *t_fptr_O = reinterpret_cast<float *>(&R_O[i][j][0]);
          float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
          t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_O[0]);
```

```
          t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_O[1]);
          t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_O[2]);
          t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_O[3]);
        } else {
          float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
          float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
          float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
          float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
          t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_O_0.x);
          t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_O_0.y);
          t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_O_1.x);
          t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_O_1.y);
          HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
          HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
        }
      }
```

flash_attn.cuh L696-716（$\ell \leftarrow \alpha\ell_{\text{old}} + \ell_{\text{new}}$、$m$ 推进与下轮 K 就绪等待）

```
      float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
      float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
      lane_block_row_sum_old[i][0] = __fmaf_rn(
          rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
      lane_block_row_sum_old[i][1] = __fmaf_rn(
          rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);

      // Update m
      lane_block_row_max_old[i][0] = block_row_max_new_0;
      lane_block_row_max_old[i][1] = block_row_max_new_1;
    }

    // Wait for next K tile to be ready in smem before next iteration
    if constexpr (kStagesK > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(0);
      }
      __syncthreads();
    }
  } // end outer loop over K seqlen
  __syncthreads();
```

$\alpha = e^{m^{\text{old}} - m^{\text{new}}}$ 在 L666–669 对 16 行各算一份（±8 行两个寄存器各一）。rescale 与累加融合：`R_D` $=$ FMA(α, `R_D`, `R_O`)（L687–690），(m, ℓ) 同步推进（L699–705）。迭代末若还有下一轮，`wait_group(0)` 确保 K_{t+1} 就绪再进下一轮 3b（L710–714）：K 的等待点放在外循环**末**而非 3b 前，使 K_{t+1} 的搬运与整段 3b–3e 计算重叠，这是 `kStagesK`=2 流水时序的最后一个等待点。

17.4.9 Step 4/5：最终归一化与 epilogue 写回

flash_attn.cuh L718-745（Step 4：$O \leftarrow O/\ell$，`__frcp_rn` 倒数）

```
  // ============================================================================
  // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
  // ============================================================================
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
    float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
    float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      if constexpr (kMmaAccF32) {
```

```
        float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
        half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
        t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
        t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
        t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
        t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
      } else {
        float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
        float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
        t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
        t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
        t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
        t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
        HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
        HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
      }
    }
  }
```

flash_attn.cuh L750-789（Step 5：warp shuffle 收拢 4-lane 组，128-bit 向量化写回）

```
  // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
  // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      uint32_t R_Z[2][4];
      R_Z[0][0] = R_D[i][j][0];
      R_Z[1][0] = R_D[i][j][1];
      R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
      R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
      R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
      R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
      R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
      R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

      // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
      if (lane_id % 4 == 0) {
        int store_warp_regs_O_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
        int store_lane_gmem_O_Br =
            Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
        int store_warp_regs_O_d =
            warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
        int store_lane_gmem_O_d = store_warp_regs_O_d;
        int store_gmem_O_addr_0 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 0) * kHeadDim +
                                  store_lane_gmem_O_d;
        int store_gmem_O_addr_1 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 8) * kHeadDim +
                                  store_lane_gmem_O_d;
        // LDST128BITS = reinterpret_cast<float4*>
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
            *reinterpret_cast<float4 *>(&R_Z[0][0]);
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
            *reinterpret_cast<float4 *>(&R_Z[1][0]);
      }
    }
  }
}
```

epilogue 与第 十二 章 11.4.7 同构：O 的 C fragment 一行 8 个 half 分散在 4 个 lane，`__shfl_sync`（width=4）把邻居的半行收拢到 `R_Z`，`lane%4==0` 的 lane 用两条 `float4`（16 B）store 写出本 4-lane 组负责的行 g 与 g+8（L782–785），行号由 `Q_tile_id`、`warp_QP` 与 `lane_id/4` 合成（L768–771）。每 warp 写出 16×64 的 O 切片，8 warp 拼回 $B_r \times D$。

17.5 图示

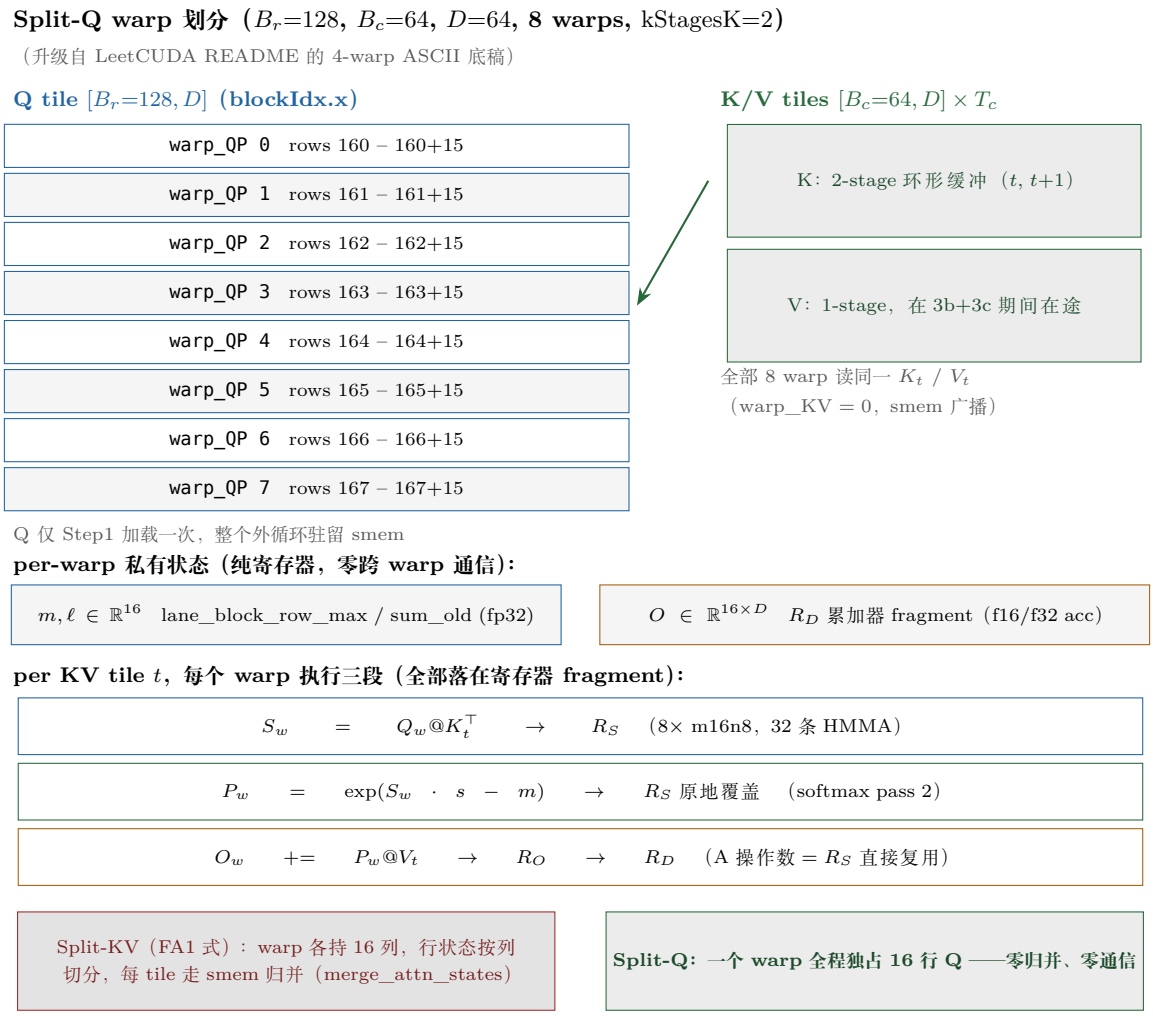


图 17.5: Split-Q warp 行划分与状态私有化（默认 B_r=128/8-warp 配置）。左：Q 行按 warp 独占、KV 全 warp 共享；中：每 warp 私有的 (m, ℓ, O) 与每 tile 的三段计算；下：与 split-KV 的对比——后者行状态按列分裂到多个 warp、须逐 tile 归并。

17.6 性能分析

算术强度。稳态下每个外循环迭代：计算量 $4B_rB_cD$ FLOP（$QK^\top$ 与 PV 各 $2B_rB_cD$）；搬运量 $(K_t{+}V_t) = 2B_cD{\cdot}2$ B（Q 已摊销到 T_c 次迭代）。$AI = \frac{4B_rB_cD}{4B_cD} = B_r = 128$ FLOP/B（half 口径），与第 十二 章 GEMM 的 $2B_K = 32$ FLOP/B 相比高 4 倍，明确属于 compute-bound，优化重心在 mma 密度与流水，不在带宽。

本章复测（RTX PRO 5000 72GB，sm_120a，2026-09-11；`B=1,H=32,N=4096,D=64`，warmup 3 + 10 次均值，event 计时；数据与命令存 `.tmp/book-bench/ch16/`）：

两点观察：（1）F16Acc 复测与源码注释的 166.6 TFLOPS 基本吻合，说明该注释口径可复现（注释未注明 acc 变体，对照时应以 F16Acc 为准）；（2）**F32Acc 反而更快**（+5.6%）：f32 累加寄存器翻倍（每 tile 4 个 `.b32`）没有拖慢执行，反而省掉了 3c/3e 里 half↔float 的整簇转换指令（f16 路径每元素两次打包/拆包），ALU 压力更小。这与第 十

表 17.1: fa2 splitq 复测：pad=8、kStagesK=2

变体	ms/iter	TFLOPS	对照
F16Acc（kMmaAccF32=0）	0.8251	166.56	源码注释 L61–71：166.6（吻合）
F32Acc（kMmaAccF32=1）	0.7814	175.88	—

二 章 GEMM 的直觉（f16 acc 更快）相反，是 attention 「mma 占比低于 GEMM、softmax 转换指令占比更高」的直接体现。

pad 与 XOR swizzle 的取舍（源码注释 L61–71，SM120 同形状 profile 消融）：compact XOR swizzle（kPad=0，物理布局 [col/16][row][16]）虽省 5 KiB smem 并消除了目标 ldmatrix pattern 的 bank conflict，却把 cp.async（LDGSTS）的 destination pattern 打散：wavefront 从 30.15M 涨到 68.16M（2.26x），long-scoreboard / LG-throttle / MIO-throttle stall 分别放大 3.25x/2.74x/1.60x，整体只有 120.0 TFLOPS，显著低于 pad=8 的 166.6。**评价 shared layout 必须同时看 ldmatrix 读与 cp.async 写两个方向**，只优化 bank-conflict counter 会牺牲另一方向（该结论出自源码注释，属作者实测口径；本章复测 pad 路径与其吻合）。

FA2 MMA Stages 全变体 bench（RTX PRO 5000 72GB，sm_120，110 SM，CUDA 13.2；`leetcuda_bench_sm120a.bin _bench_bench_all`；FA 问题规模 `B=1,H=32,N=8192,D=128`，dense 无 causal，对比 cuDNN 同精度基线）：

表 17.2: FA2 MMA Stages 实测（RTX PRO 5000，B=1 H=32 N=8192 D=128）

Kernel	Max Err	TFLOPS/cuDNN（加速比）
FA2 MMA Stages (Sk=1, Pad, F16Acc)	2.441e-04	127.6/231.1 (0.55x)
FA2 MMA Stages (Sk=2, Pad, F16Acc)	2.441e-04	156.3/231.1 (0.68x)
FA2 MMA Stages (Sk=1, Pad, F32Acc)	1.526e-05	140.9/230.2 (0.61x)
FA2 MMA Stages (Sk=2, Pad, F32Acc)	1.526e-05	163.5/230.2 (0.71x)
FA2 MMA Stages (Sk=2, SwizzleQ, F32Acc)	1.526e-05	163.8/230.2 (0.71x)

三点观察：（1）$kStagesK$=2 **双 stage 相比** $kStagesK$=1 **提升明显**：f16 路径 127.6→156.3（+22%），f32 路径 140.9→163.5（+16%），多一级 K 流水把 cp.async 的加载与 mma 计算更好地重叠，掩盖了 B_c 列 K tile 的装载延迟；（2）**F32Acc 在精度提升约 16 倍**（1.526e-05 vs 2.441e-04，f32 vs f16 累加）**的同时速度反而更快**（同 Sk 下 +4.6% ∼ 10.4%），与上文复测结论一致：省掉 3c/3e 的 half↔float 整簇转换指令的收益大于 f32 累加器的寄存器开销；（3）**SwizzleQ** 相对 Pad 的边际改善很小（163.8 vs 163.5，+0.2%），大 N 下 swizzle 只微调 bank/wavefront 分布，不改变 compute-bound 本质。整体上该教学 kernel 为可读性牺牲了不少优化（约为 cuDNN 的 0.55–0.71x），但 stages 流水、累加精度两个旋钮的收益方向与生产级 FA 实现一致。

17.7 实践经验与考据说明

实践经验 17.1（F5 配置口径）：头注释 L33–37「Br=64 / Block(128,1,1) / 4 warps」与默认实例化不符 头注释的 grid/block 描述取 $kMmaTileSeqLenQ$=4（B_r=64、4 warp、128 线程）配置，而 L44–45 的 tile 公式又写 $B_r = 16 \cdot \{4, 8\} \cdot 1 = \{64, 128\}$；leetcuda bench 与本章测试的默认实例化是 $kMmaTileSeqLenQ$=8（B_r=128、8 warp、256 线程）。注释无错但口径未标注适用配置，读者按 128 线程推 smem/blockDim 会对不上。已记 `book/CHECKLOG.md`；正文一律以式 17.1 的默认配置为准。

实践经验 17.2（F5 陈旧注释）：L575–576「Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64」 3d 步的循环上界是 `Bc / kMmaAtomK`，默认配置 B_c=64 应迭代 4 次；该注释描述的是 B_c=16（$kValTileSeqLenK$=2）的非默认配置，且「for kHeadDim≤64」的条件与 B_c 无关，易误导读者以为默认只迭代一次。已记 CHECKLOG。

实践经验 17.3（工程观察）：flash_attn.cuh 头文件不自包含：`swizzle` 派发器定义在 hgemm.cuh 本 kernel 的 compact XOR 路径（kPad=0 时）调用 `swizzle<kMmaAtomK>`（L426/L593 等），而该模板的公开派发器定义在 **hgemm.cuh L389–396**（common.cuh 里只有 `swizzle_v1_impl/swizzle_v2_impl`）。单独 include flash_attn.cuh 编译报 “identifier swizzle is undefined”；bench/flash_attn.cu 靠 L4–5 的 include 顺序（hgemm.cuh 在前）才通过。本章测试按同序 include。已记 CHECKLOG（F5 工程类）。

实践经验 17.4 (F3 核校通过)：L17–24 online rescaling 公式注释 注释的五步递推（$S \to m^{\text{new}} \to P \to \ell^{\text{new}} \to O$ rescale）与 FA2 论文 Eq.(2)(3) 及第 十六 章推导一致；「P 写回 R_S」的前提条件在 L52–58 有专门 caveat（布局约定，非通用结论），表述严谨。

实践经验 17.5 (F2 核校通过)：L459–470 C fragment 行/lane 分布注释 「lane 0–3 持 row 0、lane 4–7 持 row 1、…、reg[1] 对应 rows 8–15」与 PTX Figure 83 的坐标公式（式 12.3）逐条一致，是 `warp_reduce<4>` 宽度选择的直接依据。

17.8 常见坑

坑一：N 不是 B_c 的倍数。外循环 $T_c = \lceil N/B_c \rceil$ 用了 ceil 写法，但 3a 的 K/V 加载没有任何尾 tile 边界处理，最后一个不完整 tile 会**越界读** gmem 并把越界数据算进 softmax（L125–127 注释已自我声明「不等于当前实现已经完整处理了尾 tile」）。同理 store 侧要求 $N \% B_r = 0$。正确用法：seqlen 按 64/128 对齐，或在调用前 pad。

坑二：smem 45 KiB 逼近上限。默认配置 $(128+2\times64+64) \times 72 \times 2 = 46080$ B，恰在 48 KiB 动态 smem 默认上限之下；把 B_r 提到 256 或 `kStagesK`=3 立刻超限，launch 报 `invalid argument`，必须先 `cudaFuncSetAttribute` 抬高 `cudaFuncAttributeMaxDynamicSharedMemorySize`。leetcuda bench 与本章测试即使在 45 KiB 下也显式设置，作为防御性习惯。

坑三：早退 `return` 与 warp 集体指令冲突。L156–157 的 `if (load_gmem_Q_Br >= N) return;` 依赖「$N \% B_r = 0$ 时条件恒假」才安全：一旦部分线程早退，后续 `__syncthreads()` 数量不齐（死锁）且 ldmatrix/mma 的 `.aligned` 语义被破坏（同第 十二 章坑二）。形状对齐假设要写进 kernel 的调用约定。

坑四：f16 累加路径的精度预算。P 写回 `R_S` 是 half （两 acc 路径都是），且 f16 acc 下 `R_O/R_D` 每条 mma 边界舍回 half、O 的跨 tile rescale 也在 half 域。好在本配置下量级有利：$P \in (0, 1]$、$|V| \le 1$、O 是凸组合，实测对 fp64 参考误差 $\sim 1.4 \times 10^{-4}$（N=256）。但若输入幅值大、N 上千，误差沿 T_c 与幅值双轴增长，生产口径建议 F32Acc 变体（实测还更快，见表 17.1）。

坑五：把「C fragment 拼 A fragment」当通用结论。P 直接留在 `R_S` 直接作下一段 mma 的输入，依赖 16.3.3 节推导的 **特定布局约定**：列相邻两个 `m16n8` C tile 的寄存器对，恰好按 mma 期望的四象限序拼成 `m16k16` A。换了 atom 形状（如 `m16n8k8`）、换了 tile 排布方向，或对 fragment 做了重排，这条捷径就静默出错。源码 L52–58 与 L618–620 两处注释都在防这个过度泛化，面试中也常考查这一限制。

坑六：cp.async 的组计数要跨步骤核对。一个迭代内两组 commit （V_t、K_{t+1}），等待点却分布在 3d 前（wait(1)，只保证 V_t）与 3e 后（wait(0)，保 K_{t+1}）；尾部迭代 K 分支不 commit，组计数少一组但 wait(0) 语义仍是「在途组 ≤ 0」，安全。改造流水（如加 `kStagesV`≥2）时必须重新推每处 wait 的在途组数，并与第 十一 章的组语义对照：wait 等的是「未完成组 $\le N$」而非「第 N 组完成」。

17.9 面试要点速查

Q1：FA2 的 split-Q 与 FA1 式 split-KV 本质区别？

A：切分轴。split-KV 按 KV 列切 warp，行 softmax 状态分裂在多个 warp，每 tile 末要 smem 中转 + LSE 语义归并（`merge_attn_states`）；split-Q 按 Q 行切 warp（每 warp 独占 16 行、KV 共享，warp_KV≡0），(m, ℓ, O) 全程 warp 私有，零跨 warp 通信，且独立 softmax 状态数乘上 warp 数。FA2 论文层面还叠加了外层并行度的提升（序列长时沿 $\lceil N/B_r \rceil$ 增 grid）。

Q2：为什么 Q 只加载一次、V 只有一级缓冲？

A：Q tile 由 blockIdx.x 固定，整个外循环复用，一次 cp.async 进 smem 后每轮 ldmatrix 重装 fragment（数据 L1 常驻）；V_t 的搬运窗口天然被 3b（mma）+3c（softmax，非 mma 指令）两段计算覆盖，单级即可掩盖延迟；K 承担跨迭代预取（`kStagesK`=2 环形缓冲），因为 3b 一开始就要读它，没有其他计算段可以掩盖其装载延迟。

Q3：softmax 后的 P 为什么可以不搬出寄存器直接做 mma 的 A？

A：布局约定（16.3.3 节）：列相邻两个 `m16n8` C fragment （`R_S[i][w]`, `R_S[i][w+1]`）的每 lane 寄存器对，按 [左上, 左下]/[右上, 右下] 恰好拼成 `m16k16` A fragment 的 $\{a_0..a_3\}/\{a_4..a_7\}$，$w = 2$ `tile_V_Bc` 的索引由此而来。仅对本 atom 与本 tile 排布成立。

Q4：online softmax 的行 max/sum 归约宽度为什么是 4 不是 32？

A：C fragment 的行 r 只由 lane $4r..4r{+}3$ 持有（g 定行、tig 分列对），`warp_reduce_max<4>`/`sum<4>` 用 XOR 掩码 $\{2, 1\}$ 在 4-lane 组内蝶形；整宽 32 会把不同行混进来。

Q5：K 用 ldmatrix 不加 `.trans`、V 却要加，为什么？

A：mma 的 B 一律要 `.col` fragment。K 以 $[B_c, d]$ 行主序存储，其行即 $K^\top$ 的列，非 trans 装载直接给出列主序 B（TN 布局，同第 十二 章）；V 参与 $P@V$（NN 布局），B 是 V_t 本身而 V 按行主序存，必须 `.trans` 按列装载。

Q6：本 kernel 的 smem 布局为什么选 pad=8 而不是 XOR swizzle？

A：两者都能让 ldmatrix 读 conflict-free，但 compact XOR 改变了 cp.async 写侧 pattern：LDGSTS wavefront 2.26x、多种 stall 放大，实测 120 vs 166.6 TFLOPS。shared layout 优化必须读写两个方向一起看（16.6 节）。

17.10　最小测试与动手实验

测试文件：`book/tests/ch16_fa2_mma.cu`（逻辑抽取自 bench/flash_attn.cu 的 `test_flash_attn`，L662；参考改为 CPU fp64 直接算 softmax attention，half 输出 `__half2float` 转 float 比较）。case A（B=1, H=2, N=256：多 head、每 head 2 个 Q tile，T_c=4 覆盖流水满载与尾部排空）、case B（B=1, H=1, N=512：grid.x=4 多 Q tile）；每 case 测 F16Acc/F32Acc 两变体，容差分别取 `TOL_F16ACC`$= 5 \times 10^{-2}$ 与 `TOL_F32ACC`$= 10^{-3}$。include 顺序需要注意（见 16.7 节工程观察）：`hgemm.cuh` 必须在 `flash_attn.cuh` 之前。

在 `book/tests/` 目录、`CUDA_VISIBLE_DEVICES=6` 下执行 `./build_tests.sh --arch sm_120a --ch ch16`，本机输出（RTX PRO 5000 72GB，2026-09-11）：

```
case A: B=1 H=2 N=256 D=64
PASS fa2 splitq F16Acc B1 H2 N256 D64: max_abs_err=1.417e-04 (tol=5.0e-02)
PASS fa2 splitq F32Acc B1 H2 N256 D64: max_abs_err=6.614e-05 (tol=1.0e-03)
case B: B=1 H=1 N=512 D=64
PASS fa2 splitq F16Acc B1 H1 N512 D64: max_abs_err=9.811e-05 (tol=5.0e-02)
PASS fa2 splitq F32Acc B1 H1 N512 D64: max_abs_err=4.084e-05 (tol=1.0e-03)
ALL OK
```

误差量级 10^{-4}（远低于 hgemm f16 acc 的 10^{-2}）：attention 的 O 是 V 行的凸组合、$P \in (0, 1]$，幅值约束使 half 舍入的绝对误差保持较小；F32Acc 只比 F16Acc 好 2–3 倍（P 的 half 写回是两条路径共同的精度下限）。动手实验建议：（1）`./.build/ch16_fa2_mma bench` 复现表 17.1（166.56/175.88 TFLOPS）；（2）把模板参数 `kStagesK` 改成 1 重新编译，观察退化路径的吞吐变化（每轮 3b 前 wait(0)，流水消失）；（3）把三个 pad 置 0（compact XOR swizzle 路径，注意 16.7 节的 include 坑此时才真正被触发），复现 120 TFLOPS 量级的退化，再用 ncu 对比 `LDGSTS` wavefront；（4）故意传 $N\%64 \neq 0$ 的形状（如 N=200），复现坑一的越界/ 脏数据症状。

延伸阅读

- @DefTruth《图解大模型训练之：推理加速——从 Online-Softmax 到 FlashAttention V1/V2/V3》，https://zhuanlan.zhihu.com/p/668888063（online softmax → FA1/2/3 的递进图解；本章 split-Q 与 work partitioning 的直觉材料，配合第 十六 章推导阅读）；
- Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*, arXiv:2307.08691（split-Q 循环重排与并行度分析的原始出处；本章 3e 步对应其 rescale 公式）；
- @66RING《[cute 系列详解][实践] 使用 cutlass cute 复现 flash attention》，https://zhuanlan.zhihu.com/p/696323042（CuTe 视角的 FA2 复现，第 二十六 章的对照实现；与本章手写 fragment 几何互为印证）。

第十八章　FA2（二）：TMA + Warp Specialization

18.1　本章导读

第 十七 章的 `flash_attn_mma_stages_split_q` 用 cp.async + `mma.sync` 把 FlashAttention-2 跑到了可用的水平，但它的 256 个线程 **既负责搬运又负责计算**：每个 warp 在 ldmatrix/mma 的间隙里还要发 cp.async、算地址、等 commit group，装载与计算在同一支执行流里交替占用发射资源。本章讲它的后续版本 `flash_attn_tma_mma_stages_split_q`（flash_attn.cuh L889）：把「搬运」整体剥离给一个专职的 producer warpgroup（TMA 单线程提交），把「计算」留给 8 个 consumer warp。两条执行流靠 mbarrier 的 full/empty 双屏障阵列解耦，这就是 **warp specialization**（ws）在 attention 上的完整形态。SM120（Blackwell 消费级）没有 WGMMA（第 十四 章坑一），所以这里的 consumer 仍用 `mma.sync` + ldmatrix，**TMA/mbarrier 这套「基础设施」跨 Hopper/Blackwell 通用，计算指令则按架构选型**。

源码区间：flash_attn.cuh L792–1440；其中 L792–842 是约 50 行的头部设计注释，包含本章的核心素材，即 **mbarrier arrive_count 协议注释**（L808 起：257 = 256 consumer + 1 producer、full/empty 相位循环、V 的装载时机）。本章将逐条解读这份协议注释，并与实际代码逐点核验一致性。

编译宏与架构限定：`NOTES_V2_ENABLE_TMA_MMA_WS`，sm_90a 与 sm_120a 可编译运行；要求 CUDART $\geq$ 13.0（common.cuh L21–23 有 #error 兜底）。本机 GPU 为 sm_120a（RTX PRO 5000），本章测试与 bench 复测均在本机完成。

前置：第 十四 章（TMA descriptor、mbarrier 双计数事务模型，本章不再重复其机制，只回指）、第 十六 章（attention 数学与 online softmax）、第 十七 章（split-Q 划分与 FA2 的寄存器流）；**后继**：第 十九 章把 consumer 从 1 个 warpgroup 扩成 2 个（FA3 双 consumer 结构）。

18.2　问题与动机

第 十七 章结尾的性能数据（见 17.6 节复测表）暴露了 cp.async 版的三个结构性短板，它们都指向同一个根源，即 **没有角色分工**：

- **装载指令挤占计算发射槽**。cp.async 路径下，每个线程各自计算 gmem 源地址与 smem 目的地址；一次 $B_c \times D = 64 \times 64$ 的 K tile 装载要由 256 个线程合作完成，地址整数指令与 ldmatrix/mma 争抢同一个 warp 调度器的发射窗口。FA 与 GEMM 不同，它的在线 softmax（max/exp/sum）本来就有大量非 Tensor Core 指令，装载指令再加入后，Tensor Core 指令的发射占比进一步被稀释。
- **同步粒度粗**。cp.async 的 commit/wait group 只能表达「我发出的这批搬运完成了」，无法表达「stage s 的数据到齐了、stage s 可以被覆盖了」这类**按 stage 的状态**；装载与消费的衔接只能靠 `__syncthreads` 加软件 stage 计数拼装。
- **V 的装载深度不足**。FA 每个 KV 迭代要依次用 K（算 $S = QK^\top$）和 V（算 PV），cp.async 版里 V 的装载只能与 QK 部分重叠；理想情况下 V[k] 的搬运应当与 QK[k] + softmax[k] + PV[$k-1$] 三段全部重叠——这需要一个独立于计算进度的装载执行流。

warp specialization 的回答是把 384 个线程切成两支：**producer WG**（128 线程，实际只有 1 个线程执行提交）专职提交 TMA，一条 `cp.async.bulk.tensor.2d` 搬完整个 tile，地址计算/OOB 填充/128B swizzle 全部由 descriptor 声明；**consumer WG**（256 线程 = 8 warp）专职 ldmatrix + `mma.sync` + online softmax。两支流之间没有任何 `__`

`syncthreads`，全靠 mbarrier 阵列传递「数据就绪」（full）与「stage 可覆盖」（empty）两类事件。收益是双重的：装载延迟被流水深度（kStagesK）完整掩盖；producer 空闲出来的 127 个线程的执行资源与寄存器（经 `setmaxnreg` 转移；TMA dst 须为 `shared::cta`，见 17.8 节坑七）归 consumer 使用。

源码头注释（L792–803）把这套结构压缩成了三条「面试要点」：

- **Producer**（WG0, 128T）用 TMA 128B SWIZZLE 搬运 Q/K/V tile 到 smem；
- **Consumer**（WG1, 256T）用 ldmatrix + `mma.sync.m16n8k16` 做 $QK^\top$ / PV；
- 与 `hgemm_tma_mma_ws_tn` 的对比：HGEMM 只做一次 GEMM、沿 K 维迭代，FA 做 $QK^\top$ 和 PV 两次 GEMM、沿 KV seqlen 迭代；HGEMM 的 A/B 都 staged，FA 中 Q 只 load 一次（split-Q）、K staged、V 单 buffer。

前两条的角色划分、第三条的三处差异（两次 GEMM/KV 迭代/Q 只装一次）分别对应 17.4.1、17.3.1 与 17.4.3 节；其中「Q 只 load 一次」是 attention 特有的：GEMM 的 A/B 都随 K 迭代刷新，而 split-Q 的 Q tile 在整个 KV 遍历中不变，这让 Q 的同步协议退化成「一次性 full 屏障」，是本章协议中最简单的一个。

18.3 数学原理

18.3.1 split-Q 网格与循环不变量

沿用第 十七 章的记号（B, H, N, D 为 batch/heads/seqlen/head dim，B_r, B_c 为 Q/KV block 行数）。本章取 $B_r = 128$、$B_c = 64$（派生自模板参数 $16 \times 8 \times 1$ 与 $8 \times 1 \times 8$，17.4.2 节展开）。网格与循环结构：

$$\text{grid} = (N/B_r,\ B \cdot H), \qquad T_c = N/B_c, \tag{18.1}$$

即 blockIdx.x 索引 Q tile（沿 seqlen 切块并行）、blockIdx.y 索引 (batch, head)；**每个 block 遍历完整的 KV seqlen（T_c 个 tile）**，block 内的 online softmax 状态 (m, ℓ, O) 跨 T_c 轮迭代持续更新，循环不变量与第 十七 章完全一致：

$$m^{\text{new}} = \max(m^{\text{old}}, \text{rowmax}(S_k \cdot \gamma)), \qquad \ell^{\text{new}} = e^{m^{\text{old}} - m^{\text{new}}} \ell^{\text{old}} + \text{rowsum}(P_k), \tag{18.2}$$

$$O^{\text{new}} = \text{diag}\left(e^{m^{\text{old}} - m^{\text{new}}}\right) O^{\text{old}} + P_k V_k, \qquad P_k = \exp(S_k \gamma - m^{\text{new}}), \tag{18.3}$$

其中 $\gamma = 1/\sqrt{D}$。split-Q 的关键收益在装载侧：Q tile $[B_r, D]$ 只需在进入 KV 循环前装载**一次**，此后 T_c 轮迭代全部复用 smem 中的同一份 Q。对比「每轮都刷 A」的 GEMM，FA 的 Q 通道是 **一次装载、全程只读**，因此 Q 只需要 full 屏障（数据就绪），不需要 empty 屏障（没有谁会覆盖它）。头注释（L801–806）对 split-Q 语义的完整表述则澄清了「Q tile」的确切含义，即它是 **block-level** 的，不是全局 Q：

- grid $= (N/B_r,\ B \cdot H)$：每个 block 处理一个 block-level Q tile $[B_r, D]$（不是全局 Q），对 Q 的 seqlen 做 block 级并行；
- 每个 block 遍历**完整的 KV seqlen**（$T_c = N/B_c$ 次外循环）；
- 因此每个 block 的 Q tile 只需 load 一次到 smem、在整个 KV 遍历中复用 → 只需 `full_Q` barrier（无 `empty_Q`）。

18.3.2 mbarrier arrive_count 协议：257 的代数

第 十四 章式 (13.4)（相位翻转条件）给出 mbarrier 的一般模型：pending arrivals 与 tx-count **双计数同时归零**才翻转相位。本章把这套代数落到 attention 的具体数字上。flash_attn.cuh L808–811 的头注释用四行给出整套 Pipeline 同步协议：arrive_count = 257 = 256 consumer + 1 producer，三条通道各有一组屏障——Q 走 `full_Q`，producer 发 `arrive_tx`，consumer 在主循环前 wait 一次；K 走 `full_K[kStagesK]` 与 `empty_K[kStagesK]`，多 stage pipeline；V 走 `full_V` / `empty_V` 单 buffer 串行，QK 前发 TMA、与 QK 重叠、PV 前等、PV 后释放。

逐条拆解。首先是**为什么是 257**：

$$N_{\text{arrive}} = \underbrace{256}_{\text{consumer 全体（8 warp} \times \text{32）}} + \underbrace{1}_{\text{producer 提交线程}} = 257, \tag{18.4}$$

与第 十四 章 WGMMA 版的 129（128 + 1）同构，但 consumer 端的含义不同：WGMMA 是 warpgroup 对齐指令，consumer 的最小单位是**整个 128 线程 warpgroup**；本章 consumer 用 warp 级的 `mma.sync` + ldmatrix，8 个 warp 彼此独立推进，因此**每个线程**都必须在屏障上 arrive 一次，「数据到齐且全体 consumer 都已就位」才翻转。这是把 ws 协议从 Hopper 移植到 SM120 时最容易算错的数字（源码 L960–962 的 WARN 注释专门警告了另一个相关的除数错误，见 17.8 节坑一）。

其次是 **full/empty 双屏障的相位循环**。每个 K stage $s \in [0, \mathrm{Sk})$ 配一对屏障，各自维护独立的相位 π：

- `full_K[s]`——「stage s 的数据已就绪」。producer 提交 TMA 后调 `tma_arrive_expect_tx`（**一条指令同时完成**：到达数 +1、声明期望事务量 ΔT = K tile 字节数，PTX 规定合并指令的 arrive count 固定为 1）；TMA 引擎搬完时硬件自动 complete-tx 计数。consumer 每线程 `full_K[s].wait(full_K[s].arrive())`——先 arrive（贡献 256 份到达之一）再等翻转。翻转条件：

$$\mathrm{flip}(\text{full_K[s]}) \iff \underbrace{256 \text{ consumer arrives}}_{\text{wait 时逐线程 arrive}} + \underbrace{1 \text{ producer arrive}}_{\text{expect_tx 附带}} + \underbrace{\text{tx landed} = \Delta T}_{\text{TMA 硬件计数}}; \tag{18.5}$$

- `empty_K[s]`——「stage s 可被覆盖」。consumer 在 $\mathrm{QK}^\top$ **消费完 K smem 的那一刻**（softmax/PV 只访问寄存器与 V）逐线程 arrive 释放；producer 下一轮要写 stage s 时 `empty_K[s].wait(empty_K[s].arrive())`——自己 arrive 补上第 257 份再等翻转。翻转条件：

$$\mathrm{flip}(\text{empty_K[s]}) \iff 256 \text{ consumer arrives（读毕释放）} + 1 \text{ producer arrive（写前报到）}. \tag{18.6}$$

两张屏障构成一个**每相位恰好闭合的计数循环**：对固定 stage s，每轮 KV 迭代消耗 full 相位 1 次（256 consumer arrive + 1 producer arrive_tx + 字节落盘）、empty 相位 1 次（256 consumer arrive + 1 producer arrive）。两边的 arrive 预算都是 257/轮，与 init 值精确对齐。任何一边多算或少算一次，循环就会在几轮后「错位」：少算则屏障提前翻转（读到半新半旧数据，静默错误），多算则永不翻转（挂死）。这套计数循环与第 十四 章图 13-3 的 mbarrier 状态机完全同构，只是参与方从 128 + 1 换成了 256 + 1。

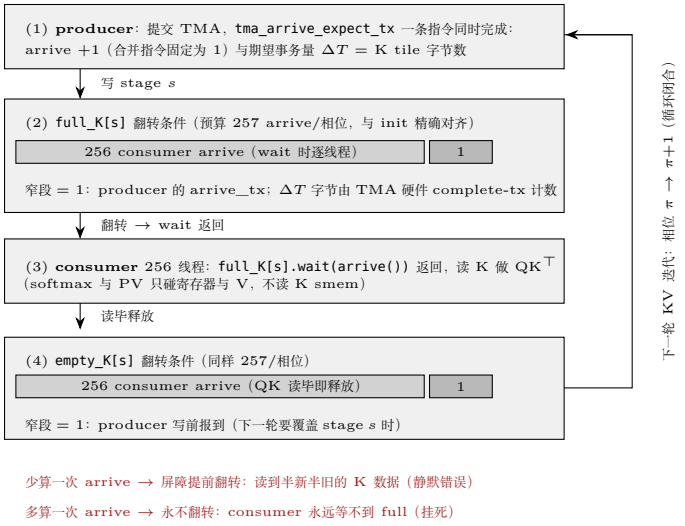


图 18.1: 固定 stage s 上 full/empty 两张屏障的相位循环（式 18.5 与式 18.6 的图形化）。每轮 KV 迭代：producer 提交 TMA 时用 `tma_arrive_expect_tx` 一条指令完成「arrive +1」与「声明 ΔT」两件事，TMA 硬件搬完后自动 complete-tx 计数；consumer 256 线程在 `full_K[s].wait()` 时各贡献一份 arrive（256 + 1 + tx 构成翻转条件），读毕 K 再各 arrive 一次释放 `empty_K[s]`，producer 下一轮写前补上第 257 份。两边的 arrive 预算都是 257/轮，与 init 值精确对齐——**多算一次挂死、少算一次静默错**。窄段为可读放大，实际只占 1/257。

最后是 **Q 与 V 的两个特例**。`full_Q`：init 257，producer 装完 Q tile 后 arrive_tx 一次（$B_r D \times 2$ 字节，D=64 时 16 KB），consumer 在进主循环前 `full_Q.wait(arrive())` **一次**。此后再无人 arrive，屏障永久停留在翻转后的相位，天然表达了「一次性事件」。`full_V/empty_V`：结构与 K 相同，但默认 $S_v = 1$（单 buffer），V[k] 的 TMA 与 $\mathrm{QK}^\top$/softmax 同迭代重叠（17.4.5 节的时序细节）；模板参数 `kStagesV` ≥ 2 时升级为真流水（P1b 预取分支）。

18.3.3　双流水的重叠窗口

把协议画成时间轴（图 18.3），producer 的装载流与 consumer 的计算流按 stage 错开。以 Sk = 2、$S_v = 1$ 为例，稳态下第 k 轮迭代：consumer 在算 tile k（$\mathrm{QK}^\top \to$ softmax $\to$ PV $\to$ rescale），producer 在搬 tile k+1（V[k+1] 先发、K[k+1] 后发）。只要搬运一个 tile 的时间短于计算一个 tile 的时间，TMA 延迟就不会进入关键路径。理论算术强度（不含 Q，Q 被 T_c 轮摊薄）：

$$AI \;=\; \frac{\overbrace{2\times 2\,B_rB_cD}^{\mathrm{QK}^\top\text{ 与 PV}}}{\underbrace{2\times B_cD\times 2\,\mathrm{B}}_{\text{K+V tile 字节}}} \;=\; B_r \;=\; 128\ \mathrm{FLOP/B}, \tag{18.7}$$

一个简洁的结论：**FA tile 级算术强度恰好等于** B_r，与 D 无关（D 同时放大分子分母）。128 FLOP/B 在 SM120 上处于 roofline 拐点附近（fp16 Tensor $\sim$ 400 TFLOPS 量级对 GDDR7 $\approx$ 1.34 TB/s（1344 GB/s）量级，拐点 $\approx$ 300 FLOP/B），所以**装载效率直接决定 FA 性能**。这正是本章把装载交给专职的 TMA 执行流、并用 Sk 流水把它移出关键路径的动机；相邻 block 共享 K/V tile 的 L2 复用（grid.y 方向同 (b,h) 的 N/B_r 个 block 读同一份 K/V）进一步抬升有效 AI。

18.3.4　smem 占用与 stage 上限

smem 需求是三个通道之和（half 元素 2 B）：

$$\mathrm{smem} \;=\; \underbrace{B_rD\cdot 2}_{Q(1\text{ 份})} + \underbrace{\mathrm{Sk}\cdot B_cD\cdot 2}_{K(\mathrm{Sk}\text{ 级流水})} + \underbrace{S_v\cdot B_cD\cdot 2}_{V(S_v\text{ 级})}. \tag{18.8}$$

头注释给出了完整的占用表与 SM120 的可行性结论（L825–842），逐格复算无误（如 Sk=2, S_v=1, D=128：$(128 + 2\times 64 + 64)\times 128\times 2 = 80$ KB）：

kStagesK	kStagesV	D=64	D=128
1	1	32 KB	64 KB
2	1	40 KB	80 KB
2	2	48 KB	96 KB
3	1	48 KB	96 KB
3	2	56 KB	112 KB
4	1	56 KB	112 KB

SM120（RTX PRO 5000）的 optin smem 上限约 100 KB：D=64 时 Sk = 1/2/3/4 全部可行（32–56 KB）；D=128 时 Sk = 1/2/3 可行（64–96 KB），Sk=4（112 KB）超出 optin 上限，会被 `check_smem_feasible` 兜底判为 SMEM SKIP，D=128 的推荐档位是 Sk=2（最优平衡点：80 KB，2 blocks/SM，pipeline 深度足够隐藏 TMA 延迟）。另有 v1 限制：`kHeadDim` 只支持 64 或 128，且要求 seqlen $\%\, B_r = 0$、seqlen $\%\, B_c = 0$（不处理尾 tile）。

本机实测 optin 上限 101376 B = 99.0 KB（17.10 节测试首行输出），注释「~100KB」为近似正确的口径；Sk=4, D=128 的 112 KB 超限 $\to$ 运行时 SMEM SKIP 的兜底逻辑在辅助工具中同样可观察。

18.4　设计与实现

18.4.1　总体结构：从「warp 各自流水」到「角色分化」

与第 十七 章的逐点对照（warp 布局本身**不变**，变的是谁在搬运、怎么同步）：

	ch16 `mma_stages`	本章 `tma_mma_ws`
线程数	256（8 warp）	384 = 128 producer + 256 consumer
装载	各 warp cp.async	producer 单线程提交 TMA
同步	`__syncthreads` + wait group	mbarrier full/empty（257）
swizzle	软件手动（pad 或 XOR）	TMA descriptor 128B 硬件 swizzle
Q 装载	一次	一次（full_Q 单屏障）
K 流水	kStagesK（cp.async group）	kStagesK（empty/full 屏障对）
V 流水	与 K 同 buffer 轮换	独立通道 kStagesV（≥ 2 真流水）
consumer 计算体	ldmatrix + mma + softmax	同左（逐行相同）
寄存器	全员同额	REG_DEALLOC(40)/REG_ALLOC(80/168)

最后一行是 `setmaxnreg` 的寄存器再分配声明：producer 降到 40、consumer 扩到 80（D=64）或 168（D=128，对齐 Triton `flash_attn_v2` 在 Blackwell 上的 `maxnreg` 策略）。另注意该指令对 TMA 的目的地址空间敏感：dst 写成 `shared::cluster` 会被 ptxas 以 C7506 丢弃（第 十五 章已考据），写成 `shared::cta` 则 sm_120a/sm_120f 都保留（build.sh 的 sm_120f 目标预设了正确组合）；默认编译宏关闭时是 no-op，寄存器预算实际由编译器按 `__launch_bounds__(384,1)` 自行分配（17.8 节坑七）。

18.4.2 模板参数与 tile 派生

flash_attn.cuh L872-896（16 个模板参数：tile 层级 + 流水深度 + 384 线程）

```
    const int kHeadDim,          // head dim: v1 only 64
    const int kMmaAtomM,         // 16 (MMA instruction M dim)
    const int kMmaAtomN,         // 8  (MMA instruction N dim)
    const int kMmaAtomK,         // 16 (MMA instruction K dim)
    const int kMmaAccF32,        // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 8 → Br=16*8=128
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, == kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStagesK,          // K pipeline depth (>=1)
    const int kStagesV,          // V pipeline depth (>=1; 1=single buffer, >=2=pipelined)
    const int kNumThreads>       // 384, 128 producer + 256 consumer
__global__ void __launch_bounds__(kNumThreads, 1)
    flash_attn_tma_mma_ws_stages_split_q(
        half *Q, half *K, half *V, half *O, int N, int H,
        const CUtensorMap *__restrict__ tensorMapQ,
        const CUtensorMap *__restrict__ tensorMapK,
        const CUtensorMap *__restrict__ tensorMapV) {
  static_assert(kHeadDim == 64 || kHeadDim == 128,
                "v1 supports kHeadDim == 64 or 128");
  static_assert(kNumThreads == 384, "128 producer + 256 consumer");
```

tile 派生链与 ch16 同一套代数：$B_r = \text{atom}_M \times \text{tile}_Q \times \text{val}_Q = 16 \times 8 \times 1 = 128$、$B_c = 8 \times 1 \times 8 = 64$；consumer 8 warp 排成 8×1（warp_QP = warp_id，各管 16 行 Q；warp_KV = 0 共享 KV）。**consumer 侧的 warp 布局与 ch16 完全一致**，这就是「角色分化但计算几何不变」：把 ch16 kernel 的装载段整体替换成 barrier wait，计算段原样保留。新增参数主要看两个：`kStagesV`（V 流水深度，1 = 单 buffer）与 `kNumThreads` = 384（static_assert 固定 128 + 256 的编队）。

18.4.3 smem 布局：1024B 对齐与 chunk-major

flash_attn.cuh L926-956（smem 三通道布局 + barrier 阵列声明）

```
  const int O_gmem_offset = QKV_gmem_offset;
  // TMA descriptor covers the entire [B*H*N, D] gmem as one 2D matrix; the
  // producer must offset major_coord by this head/batch base so each block
  // loads its own (batch, head) Q/K/V tile instead of always reading head 0.
  const int qkv_major_base = (Nb_id * H + Nh_id) * N;

  // ---- Shared memory layout ----
  // TMA CU_TENSOR_MAP_SWIZZLE_128B requires 1024B-aligned smem base so the
  // hardware swizzle phase starts at zero. Consumer swizzle<64>() assumes
  // zero phase (no base_offset compensation like WGMMA descriptor).
  // Q: [Br, kHeadDim] = [128, 64] = 16 KB (1024B aligned ✓)
  // K: [kStagesK, Bc, kHeadDim] = [kStagesK, 64, 64] (8 KB per stage)
  // V: [kStagesV, Bc, kHeadDim] = [kStagesV, 64, 64] (8 KB per stage)
  extern __shared__ __align__(1024) uint8_t smem_fa_tma_ws[];
  half *Q_smem = reinterpret_cast<half *>(smem_fa_tma_ws);
  half *K_smem = Q_smem + Br * kHeadDim;
  half *V_smem = K_smem + kStagesK * Bc * kHeadDim;

  const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
  const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem);
  const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem);

  // ---- Barriers ----
  // arrive_count = kConsumerThreads + 1 = 257 (256 consumer arrives + 1 producer arrive_tx)
#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full_Q;
  __shared__ cuda::barrier<cuda::thread_scope_block> full_K[kStagesK];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty_K[kStagesK];
  __shared__ cuda::barrier<cuda::thread_scope_block> full_V[kStagesV];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty_V[kStagesV];
#pragma nv_diag_default static_var_with_dynamic_init
```

三段线性布局：Q $[B_r, D]$、K $[\mathrm{Sk}, B_c, D]$、V $[S_v, B_c, D]$，基址 `__align__(1024)`。两个约束都来自 TMA 的 128B swizzle 模式（第 十三 章的 XOR 原子在这里由硬件执行）：

- **1024B 对齐**：128B swizzle 的物理周期是 1024B，基址不在 1024B 边界时 swizzle 相位非零；consumer 的 `swizzle_fa()` 地址计算假定 phase = 0（没有 WGMMA descriptor 那种硬件相位补偿），所以对齐是**正确性**约束而非优化；
- **box 内维 $\le$ 128B**：D = 128 的行有 256B，超出单个 box 的上限。解法是 box 固定 $(64, B)$、沿 head_dim 发 $D/64$ 次 TMA，写入 **chunk-major** 布局 $[D/64, B, 64]$。这些 TMA **共享同一个 mbarrier**：每次搬运各自 complete-tx 计数，`expect_tx` 一次声明总字节——协议数学完全不变，只是 ΔT 变成 kTmaChunks× 单 chunk 字节。

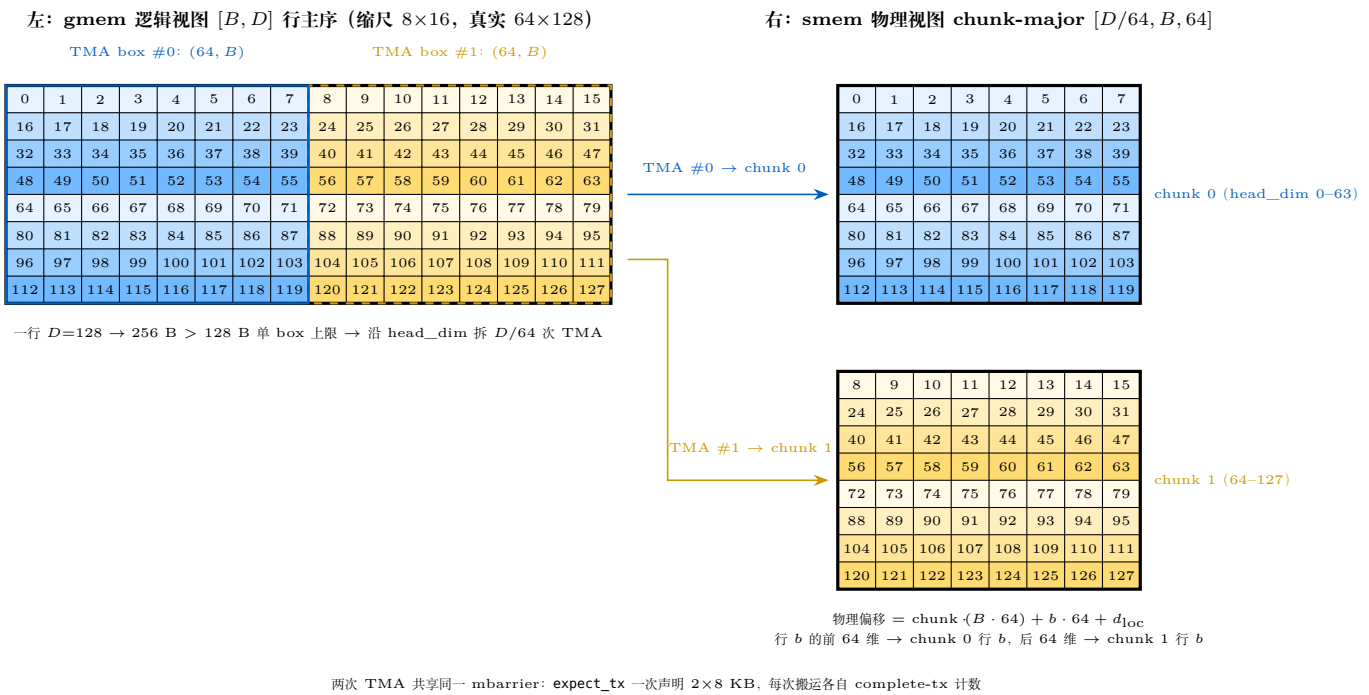


图 18.2: chunk-major 布局与双 TMA 拆分（17.4.3 节，缩尺示意：8×16 代表真实 64×128）。左：gmem 里 K/V 的逻辑视图 $[B, D]$ 行主序，cell 填逻辑线性 idx（$= b \cdot D + d$ 的缩尺版本）；D=128 时一行 256 B 超出 128B swizzle box 的维上限，两个 box（head_dim 0–63 / 64–127）各搬一半。右：smem 物理视图写成 chunk-major $[D/64, B, 64]$——同一批数据（idx 一一对应）被拆成两个连续的 64×64 chunk 面板，行 b 的前后 64 维分别落在 chunk 0/1 的行 b。两次 TMA 共享同一个 mbarrier，`expect_tx` 一次声明总字节、两次 complete-tx 各自计数，协议数学不变。

消费端地址由 `swizzle_fa` 统一给出（D = 64 时退化为 ch12 的 `swizzle<64>`，无回归）：

flash_attn.cuh L849-870（box ≤ 128B 约束、chunk-major 方案与 swizzle_fa）

```
//   - chunk c 的 smem 起始偏移 = c * Br * 64
//   - chunk c 内部按 [Br, 64] row-major + SWIZZLE_128B 存放（64 half = 128B 周期）
//
// 本 helper 计算 (row, col) 在该 chunk-major 布局下的 swizzled smem 元素偏移：
//   chunk   = col / 64        ∈ [0, kTmaChunks)
//   col_in  = col % 64        ∈ [0, 64)
//   offset  = chunk * Br * 64 + row * 64 + swizzle<64>(row, col_in)
//
// D=64 退化：kTmaChunks=1, chunk=0, offset = row*64 + swizzle<64>(row, col)
//           与原公式 (row*64 + swizzle<64>(row, col)) 完全一致 → 无回归。
template <int kHeadDim, int Br>
__device__ __forceinline__ int swizzle_fa(int row, int col) {
  static_assert(kHeadDim == 64 || kHeadDim == 128, "D=64 or 128 only");
  if constexpr (kHeadDim == 64) {
    return row * 64 + swizzle<64>(row, col);
  } else {  // kHeadDim == 128
    const int chunk  = col >> 6;        // col / 64 ∈ {0, 1}
    const int col_in = col & 63;        // col % 64
    return chunk * (Br * 64) + row * 64 + swizzle<64>(row, col_in);
  }
}
```

V 通道的消费用 `ldmatrix.x2.trans`（转置读）。头注释对「swizzle 与 trans 组合是否正确」给了专门的论证（flash_attn.cuh L813–816），论证逻辑如下：TMA 128B SWIZZLE 是一一映射，ldmatrix 按 `swizzle<64>(row,col)` 算出的物理地址就是 TMA 写入的物理地址（smem 1024B 对齐保证 phase = 0）；而 `ldmatrix.x2.trans` 的转置语义在寄存器层面工作，与 smem 物理布局无关，因此必然正确。

18.4.4　barrier 初始化与线程编队

flash_attn.cuh L958-977（除数 WARN + init 257 + proxy fence）

```
// Thread partition: first kProducerThreads (0~127) = Producer WG,
// remaining kConsumerThreads (128~383) = Consumer WG.
// WARN: Must use kProducerThreads (not kConsumerThreads) as the divisor, otherwise
// the split is wrong: 384/256=1.5 would give Producer 256 threads and
// Consumer only 128, but barriers expect 256 consumer arrives → deadlock.
const int wg_idx = (threadIdx.x < kProducerThreads) ? 0 : 1;
const int wg_tid = (threadIdx.x < kProducerThreads) ? threadIdx.x :
                   (threadIdx.x - kProducerThreads);

if (threadIdx.x == 0) {
  init(&full_Q, kConsumerThreads + 1);
  for (int s = 0; s < kStagesV; ++s) {
    init(&full_V[s], kConsumerThreads + 1);
    init(&empty_V[s], kConsumerThreads + 1);
  }
  for (int s = 0; s < kStagesK; ++s) {
    init(&full_K[s], kConsumerThreads + 1);
    init(&empty_K[s], kConsumerThreads + 1);
  }
  tma_fence_proxy_async_shared_cta();
```

三个细节：其一，**除数 WARN**（L960–962）。`wg_tid` 的划分必须以 `kProducerThreads`（128）为阈值；若误用 256 做除数，384/256 非整，producer 会分到 256 个线程而 consumer 只剩 128 个线程，屏障却等着 256 个 consumer arrive，直接挂死。这类「编队算错 → 协议数字对不上」的故障没有任何编译期征兆。其二，`init` 全部由 thread 0 完成（cuda::barrier 内联为 `mbarrier.init`，expected = 257）；其三，init 之后的 `fence.proxy.async.shared::cta` 与 `__syncthreads`：前者把「线程写好的 barrier 状态」对 async proxy 可见（第 十四 章 13.4.4 节），后者保证全员看到初始化完成的屏障后才进入角色分支。

18.4.5　producer：P0/P1/P1b/P2 四个阶段

flash_attn.cuh L979-1010（P0：Q 一次装载；P1：预取 Sk−1 个 K）

```
  __syncthreads();

  // ============================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // Only wg_tid == 0 issues TMA. Split-Q: Q tile loaded once, K staged, V single buffer.
  // D=128 时每个 tile 沿 head_dim 发 kTmaChunks 次 TMA (box innermost=64 half),
  // 共享同一 barrier: N 次 cp.async.bulk.tensor 各 reduce chunk_bytes 到 tx count,
  // 1 次 mbarrier_arrive_expect_tx 声明总字节 + producer thread arrive。
  // ============================================================
  if (wg_idx == 0) {
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
      // Step P0: Load block-level Q tile [Br, d] once. D=128 时发 kTmaChunks 次。
      //   minor_coord = c * 64, major_coord = qkv_major_base + Q_tile_id * Br
      //   smem dst = Q_smem + c * Br * 64 (chunk-major 布局)
      for (int c = 0; c < kTmaChunks; ++c) {
        tma_load_2d(Q_smem + c * Br * kTmaBoxMinor, tensorMapQ,
                    c * kTmaBoxMinor, qkv_major_base + Q_tile_id * Br, full_Q);
      }
      tma_arrive_expect_tx(full_Q, kQTileBytes);

      // Step P1: Prefetch first (kStagesK-1) K tiles.
      // K tile coords: minor = c*64, major = qkv_major_base + s*Bc
      for (int s = 0; s < kStagesK - 1; ++s) {
```

```
        empty_K[s].wait(empty_K[s].arrive());
        for (int c = 0; c < kTmaChunks; ++c) {
          tma_load_2d(K_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                      tensorMapK, c * kTmaBoxMinor,
                      qkv_major_base + s * Bc, full_K[s]);
        }
        tma_arrive_expect_tx(full_K[s], kKTileBytes);
      }
```

P0 的坐标计算：`major_coord` 要加上 `qkv_major_base` $= (N_b H + N_h)N$。descriptor 覆盖整个 $[BHN, D]$ 的 2D 大矩阵，producer 必须把 (batch, head) 的行偏移折进坐标，否则所有 block 都在读 head 0（17.8 节坑四）。P1 只预取 Sk − 1 块：第 0 块由 consumer 的 C0 预热放行（empty 的首轮翻转靠 consumer 预存 256 arrive），后续块由 consumer 的「读毕释放」继续放行。预取量与放行量恰好配平，这是 257 计数循环的首次应用。$S_v \geq 2$ 时 P1b 再预取 $S_v - 1$ 块 V（L1012–1025，结构同 P1，此处不引）。稳态循环 P2：

flash_attn.cuh L1027-1064（P2：先发 V[k+Sv-1]，再预取 K[k+Sk-1]）

```

      // Step P2: Main loop over KV seqlen tiles
      for (int k = 0; k < Tc; ++k) {
        [[maybe_unused]] const int stage = k % kStagesK;

        // Issue V[k+kStagesV-1] TMA to stage_next_v (pipelined).
        // kStagesV=1: loads V[k] to stage 0, overlaps with QK of same iter.
        // kStagesV>=2: loads V[k+kStagesV-1], overlaps with QK+softmax+PV.
        {
          const int v_tile = k + kStagesV - 1;
          if (v_tile < Tc) {
            const int stage_next_v = v_tile % kStagesV;
            empty_V[stage_next_v].wait(empty_V[stage_next_v].arrive());
            for (int c = 0; c < kTmaChunks; ++c) {
              tma_load_2d(V_smem + stage_next_v * Bc * kHeadDim +
                              c * Bc * kTmaBoxMinor,
                          tensorMapV, c * kTmaBoxMinor,
                          qkv_major_base + v_tile * Bc,
                          full_V[stage_next_v]);
            }
            tma_arrive_expect_tx(full_V[stage_next_v], kVTileBytes);
          }
        }

        // Prefetch K[k+kStagesK-1] (pipelined).
        if (k + kStagesK - 1 < Tc) {
          const int stage_next = (k + kStagesK - 1) % kStagesK;
          empty_K[stage_next].wait(empty_K[stage_next].arrive());
          for (int c = 0; c < kTmaChunks; ++c) {
            tma_load_2d(K_smem + stage_next * Bc * kHeadDim +
                            c * Bc * kTmaBoxMinor,
                        tensorMapK, c * kTmaBoxMinor,
                        qkv_major_base + (k + kStagesK - 1) * Bc,
                        full_K[stage_next]);
          }
          tma_arrive_expect_tx(full_K[stage_next], kKTileBytes);
        }
      }
```

P2 内部的**发射顺序**（V 先于 K）是刻意的：consumer 第 k 轮先等 `full_K`（QK 要用 K[k]，而 K[k] 早在上一轮预取完），K 的预取（K[k+Sk−1]）只服务于**更后续的迭代**，紧迫性最低；V[k]（S_v=1 时 `v_tile` $= k$）则要在本轮 PV（3d 步）前落盘，窗口只有 $QK^\top$ + softmax 两段，先发先到。两个 `if`（`v_tile < Tc`、`k + Sk - 1 < Tc`）处理流水排空的尾部：最后几轮不再发无用的预取，producer 自然退出循环。

18.4.6　consumer：C0 预热 + 五个阶段主循环

flash_attn.cuh L1066-1096（C0：REG_ALLOC + 预热全部 empty + 等 Q）

```
  }
  // ================================================================
  // Consumer Warpgroup (WG1, threadIdx.x 128~383, 256 threads = 8 warps)
  // All 256 threads participate in ldmatrix + mma.sync.
  // ================================================================
  else {
    const int warp_id = wg_tid / kWarpSize;  // 0~7
    const int lane_id = wg_tid % kWarpSize;  // 0~31
    // Split-Q warp layout: 8 warps form 8x1 grid (warp_QP = 0~7, warp_KV = 0)
    // Br = 16 * 8 * 1 = 128 → 8 warps each handle 16 rows of Q
    const int warp_QP = warp_id;
    const int warp_KV = 0;

    // Step C0: init — mark all K stages and V stages as "empty" (consumable by producer)
    // Consumer register budget per Triton flash_attn_v2 maxnreg strategy on
    // Blackwell warp_specialize: D=128 -> 168, otherwise -> 80.
    if constexpr (kHeadDim == 128) {
      NOTES_V2_REG_ALLOC(168);
    } else {
      NOTES_V2_REG_ALLOC(80);
    }

    for (int s = 0; s < kStagesK; ++s) {
      [[maybe_unused]] auto token = empty_K[s].arrive();
    }
    for (int s = 0; s < kStagesV; ++s) {
      [[maybe_unused]] auto token = empty_V[s].arrive();
    }

    // Wait for Q tile ready once (split-Q: Q smem is reused across all KV iters)
    full_Q.wait(full_Q.arrive());
```

C0 负责启动流水线：256 个 consumer 线程把每个 `empty_K[s]`、`empty_V[s]` 各预存 256 次 arrive；producer 的 P1/P1b/P2 每次写 stage 前自己 arrive 一次补满 257，首轮立即放行。**漏掉 C0 = 全体挂死**（与第 十四 章坑二相同，本章坑二）。随后 `full_Q.wait` 一次，Q 到齐，进入主循环。寄存器声明（L1097–1112，不引）与 ch16 逐行相同：online softmax 状态 m/ℓ（每 warp 每行对 2 份）、Q/K/V/S/O/D 六组 fragment，S 与 O 的 fragment 布局复用技巧（softmax 后的 P 直接作 PV 的 A 操作数）也原样继承。主循环五个阶段：

flash_attn.cuh L1113-1145（阶段一：等 K 就绪 + 3b 前半：ldmatrix Q/K）

```
    // ---- Main loop over KV seqlen ----
    for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
      const int stage = tile_K_seqlen % kStagesK;
      const int stage_v = tile_K_seqlen % kStagesV;

      // Wait for K[stage] ready
      full_K[stage].wait(full_K[stage].arrive());
      tma_fence_proxy_async_shared_cta();

      // ---- 3b: Q @ K^T = S[Br, Bc] ----
      // Q smem reused (split-Q). K from stage `stage`.
      // smem addr (TMA 128B swizzle, swizzle<64>)
      fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
#pragma unroll
      for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
        // ldmatrix.x4: load Q m16k16 fragment (Q row-major, non-transpose)
#pragma unroll
```

```
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
          const int warp_smem_Q_Br =
              warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
          const int lane_smem_Q_Br = warp_smem_Q_Br + lane_id % 16;
          const int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8;
          const uint32_t lane_smem_Q_ptr =
              smem_Q_base_ptr +
              swizzle_fa<kHeadDim, Br>(lane_smem_Q_Br, lane_smem_Q_d) *
                  sizeof(half);
          LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                      lane_smem_Q_ptr);
        }

        // ldmatrix.x2: load K k16n8 fragment (K row-major = K^T col-major)
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
```

flash_attn.cuh L1147-1183（3b 后半：HMMA 累加 + 尽早释放 K stage）

```
            warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
          const int lane_smem_K_Bc = warp_smem_K_Bc + lane_id % 8;
          const int lane_smem_K_d =
              tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8;
          const uint32_t lane_smem_K_ptr =
              smem_K_base_ptr +
              (stage * Bc * kHeadDim +
               swizzle_fa<kHeadDim, Bc>(lane_smem_K_Bc, lane_smem_K_d)) *
                  sizeof(half);
          LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
        }

#pragma unroll
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileSeqLenK; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
                           R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                           R_K[j][0], R_K[j][1],
                           R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
            } else {
              HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
                        R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
                        R_S[i][j][0], R_S[i][j][1]);
            }
          }
        }
      }

      // Release K[stage] for producer reuse as early as possible: QK^T GEMM has
      // consumed all K smem data; the subsequent softmax (3c) and PV GEMM (3d)
      // only touch registers (R_S, R_O) and V smem, so K[stage] is free to be
      // overwritten by the producer's next TMA prefetch.
      {
        [[maybe_unused]] auto token = empty_K[stage].arrive();
      }
```

阶段一的核心是 **wait 与 arrive 合并为一条调用**（`full_K[stage].wait( full_K[stage].arrive())`）与随后的 `fence.proxy.async`（TMA 写入对 generic proxy 读可见，第 十四 章 13.4.4 节）；阶段二 3b 沿 head_dim 的 $D/16$ 个 MMA k-slice 做 ldmatrix + `HMMA16816`（F16Acc）或 `HMMA16816F32`（F32Acc）。紧接着的「释放 K」是

本章相对 ch16 的 **时序收益点**：$QK^\top$ 一算完，K[k] 的 smem 就再无人读（softmax 只访问寄存器、PV 只读 V），立刻 arrive `empty_K`，producer 最早可在 consumer 的 softmax 阶段就开始覆盖该 stage。对比 `__syncthreads` 式的「整轮末尾统一释放」，这里把 K 的 stage 周期压缩了 softmax + PV 两段的长度。

flash_attn.cuh L1185-1214（阶段三 3c：online safe softmax，只碰寄存器）

```
      // ---- 3c: Online Safe Softmax — row max + exp + sum, P back to R_S ----
      // Softmax only touches R_S (registers), so V TMA latency can be overlapped
      // with this compute. Defer full_V.wait() until just before P@V (3d).
      float lane_row_max_new[kValTileSeqLenQ][2];
      float lane_row_sum_new[kValTileSeqLenQ][2];
      fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
      fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

      // Pass 1: row-wise max across kValTileSeqLenK tiles (warp_reduce_max<4>)
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          float tmp_max_0, tmp_max_1;
          if constexpr (kMmaAccF32) {
            float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
            tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
            tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
          } else {
            float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
            float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
            tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
            tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
          }
          lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
          lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
        }
        lane_row_max_new[i][0] =
            warp_reduce_max<4, float>(lane_row_max_new[i][0]);
        lane_row_max_new[i][1] =
```

阶段三的 online safe softmax（max pass + exp pass，式 18.2）**只读写寄存器**。这正是 V 通道 TMA 的重叠窗口：V[k] 的搬运与整段 softmax 并行，因此 `full_V` 的 wait 被刻意**推迟到 3d 之前**（下一代码段首行），不牺牲任何重叠窗口。exp 用 `__expf` 快速近似、行归约用 `warp_reduce_max/sum<4>`（每线程持 2 行、4 线程组归约，第 三 章），与 ch16 相同。

flash_attn.cuh L1260-1306（阶段四 3d：PV 前 wait full_V + R_S 复用 + 释放 V）

```
      // ---- 3d: P @ V = O[Br, d] ----
      // Wait for V[stage_v] ready just before P@V.
      // kStagesV=1: V was loaded in same iter, TMA overlaps QK+softmax.
      // kStagesV>=2: V was loaded in previous iter, TMA overlaps QK+softmax+PV.
      full_V[stage_v].wait(full_V[stage_v].arrive());
      tma_fence_proxy_async_shared_cta();

      // ldmatrix.x2.trans: V row-major → col-major B fragment for NN matmul
      fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_O, 0);
#pragma unroll
      for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          const int warp_smem_V_d =
              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
          const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
          const int lane_smem_V_d = warp_smem_V_d;
```

```
          const uint32_t lane_smem_V_ptr =
              smem_V_base_ptr +
              (stage_v * Bc * kHeadDim +
               swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d)) *
                 sizeof(half);
          LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
        }

        // P fragment (R_S) directly feeds A of P@V MMA (layout reuse trick)
        const int w = tile_V_Bc * 2;
#pragma unroll
        for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileHeadDimV; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3],
                           R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
                           R_V[j][0], R_V[j][1],
                           R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3]);
            } else {
              HMMA16816(R_O[i][j][0], R_O[i][j][1],
                        R_S[i][w][0], R_S[i][w][1],
                        R_S[i][w + 1][0], R_S[i][w + 1][1],
                        R_V[j][0], R_V[j][1],
                        R_O[i][j][0], R_O[i][j][1]);
            }
          }
        }
      }
```

阶段四的 PV 沿 B_c 的 4 个 k-slice 推进：V 用 `LDMATRIX_X2_T`（转置装载，正确性见 17.4.3 节引用的论证），A 操作数直接取 `R_S[i][w]`、`R_S[i][w+1]`：softmax 写回 R_S 的布局恰好就是 mma 的 A fragment 布局（ch16 的布局复用技巧，头注释 L24–27 有专门警告：这是**本路径的布局约定**，不是通用结论）。PV 完成即释放 `empty_V[stage_v]`。阶段五 3e 的 online rescale（L1314–1372，式 18.3 的实现，逐行同 ch16）把 O、ℓ、m 三态更新，随后进入下一轮。m^{old} 首轮为 $-\infty$ 的分支由 `tile_K_seqlen` > 0 三目处理。

18.4.7 epilogue：终归一化与集体写回

循环结束后 Step 4 做 $O \leftarrow O/\ell$（`__frcp_rn`，L1374–1399），Step 5 的写回是 ch16 的原版方案，warp 内 shuffle 收拢 128-bit、`lane%4==0` 的线程一次写 8 个 half：

flash_attn.cuh L1401-1420（Step 5：shuffle + 128-bit 集体写回）

```
    // ---- Step 5: Epilogue — warp shuffle + 128-bit collective store ----
#pragma unroll
    for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        uint32_t R_Z[2][4];
        R_Z[0][0] = R_D[i][j][0];
        R_Z[1][0] = R_D[i][j][1];
        R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
        R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
        R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
        R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
        R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
        R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

        if (lane_id % 4 == 0) {
          const int store_warp_regs_O_Br =
```

```
            warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
        const int store_lane_gmem_O_Br =
            Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
```

写回不走 TMA（epilogue 每 block 只发生一次、非关键路径，直接 `float4` 写 gmem 已足够）。至此 384 线程全部退出：producer 循环比 consumer 早结束若干轮（尾部不再发预取），两支流各自运行到结束即可，无需终点汇合。ws 结构里**没有 kernel 级的收尾同步**，这本身就是 mbarrier 协议自洽的一个佐证。

18.5 图示

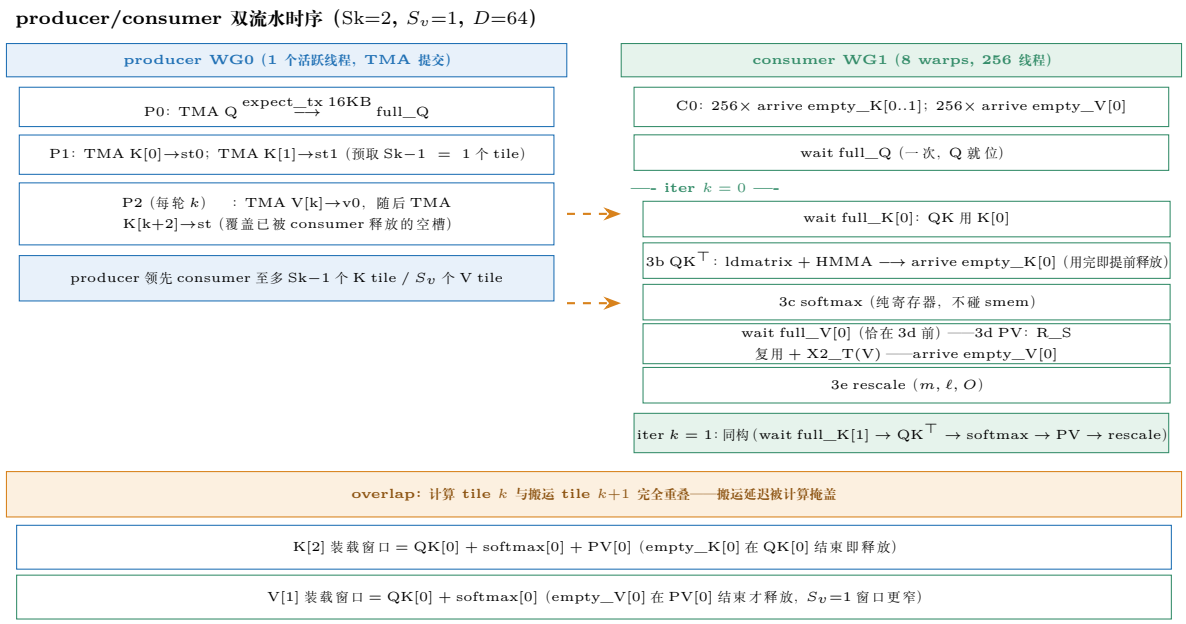


图 18.3: FA2 TMA + WS 的 producer/consumer 双流水时序（Sk=2、S_v=1、D=64）。producer 的 P2 每轮先发 V 再预取 K；consumer 的 empty 释放点（K 在 QK 后、V 在 PV 后）决定了每个 stage 的可覆盖窗口。



图 18.4: 本章 kernel 的 mbarrier 阵列拓扑：Q 一对一单屏障、K/V 各 Sk/S_v 对 full/empty 屏障，全部 init 257。D = 128 时同一 tile 的多次 TMA 共享一个 full 屏障（多次 complete-tx 计数、一次 expect_tx 声明总量）。

18.6 性能分析

本书复测口径（2026-09-11，RTX PRO 5000 sm_120a，CUDA 13.2，`book/tests/ch17_fa2_tma_ws.cu bench`：B=1、H=32、warmup=5、iters=20 取均值，FLOPS = $4BHN^2D$；两轮复测差 < 0.5%）：

配置	N=4096	N=16384
D=128，Sk=1，Sv=1，F16Acc	146.9	—
D=128，Sk=2，Sv=1，F16Acc	**172.5**	**183.8**
D=128，Sk=3，Sv=1，F16Acc	172.1	183.2
D=128，Sk=2，Sv=1，F32Acc	**191.1**	**195.2**
D=128，Sk=3，Sv=1，F32Acc	191.0	—
D=64，Sk=2，Sv=1，F16Acc	141.8	151.8

三点读数：其一，**Sk = 2 即达饱和**（Sk 2 → 3 收益 < 0.5%）：B_r=128 的计算阶段足够长，两级 K 流水已把 TMA 延迟推出关键路径，更深的流水只多耗 smem（Sk=3 的 96 KB 使 2 blocks/SM 的驻留不再可行，头注释推荐 D=128 用 Sk=2 恰是两者的平衡点）。其二，**TMA+WS 相对 ch16 cp.async 版的稳定收益约** +20%：同卡同时刻原版辅助工具对照（`leetcuda_bench --bench --bhnd 1,32,16384,128`，warmup=2/repeat=3）测得 FA2 MMA Stages（Sk=2, Pad, F16Acc）158.7T → FA2 TMA WS（Sk=2）190.4T；这 +20% 的收益来自装载专职化 + stage 级同步。其三，**F32Acc 反而快于 F16Acc**（195 vs 184）：F16Acc 路径的 softmax 需要在 f32 计算与 f16 写回 R_S 之间反复转换（`__float22half2_rn`），F32Acc 的 R_S 直接以 f32 存储、省掉双向转换。在 attention 这种 softmax 指令占比高的算子里，累加精度档位的选择同时影响**数值与速度**。

README 口径勘误（F4）：README 的 SM120a 表给出 FA2 TMA WS（Sk=2, F16Acc）292.0 TFLOPS（1.31x vs cuDNN），与本书复测（183.8T）及原版辅助工具同卡复跑（190.4T）均不可复现，且 F16/F32 的相对高低相反（README：F16 292 > F32 202；复测：F16 184 < F32 195）。README 数字应测自早期代码状态或不同时钟/驱动环境（其 bench 输出还经 `should_print_tflops` 的 max-of-N 过滤，见 17.7 节考据框）；本书统一采用「本书复测 + 原版辅助工具同卡对照」双口径，不引用不可复现数字。算术强度侧的定性结论不变：式 18.7 的 $AI = B_r = 128$ FLOP/B 贴近 SM120 roofline 拐点，FA 是典型的「装载敏感型」算子，这也解释了为何装载专职化的收益（+20%）远大于再加深一级流水（≈ 0）。

原版辅助工具全量 bench（`leetcuda_bench_sm120a_bin --bench --bench-all`，B=1、H=32、N=8192、D=128，dense）的逐配置结果如下表，含与 cuDNN 同精度基线的加速比对照：

表 18.1: FA2 TMA MMA WS 实测（RTX PRO 5000，B=1 H=32 N=8192 D=128）

Kernel	Max Err	TFLOPS/cuDNN（加速比）
FA2 TMA MMA WS (1 Consumer WG) (Sk=1, Sv=1, F16Acc)	2.441e-04	156.9/231.1 (0.68x)
FA2 TMA MMA WS (1 Consumer WG) (Sk=2, Sv=1, F16Acc)	2.441e-04	188.6/231.1 (0.82x)
FA2 TMA MMA WS (1 Consumer WG) (Sk=2, Sv=2, F16Acc)	2.441e-04	189.7/231.1 (0.82x)
FA2 TMA MMA WS (1 Consumer WG) (Sk=2, Sv=1, F32Acc)	1.526e-05	203.9/230.2 (0.89x)

四点读数：其一，**Sk = 2 较 Sk = 1 提升约** +20%（156.9 → 188.6），两级 K 流水把 TMA 延迟推出关键路径，与上文复测表的 Sk 饱和结论一致；其二，**Sv 由 1 加深到 2 的边际收益极小**（188.6 → 189.7，约 0.6%）。V 的单 buffer 消费已饱和，V tile 传输能被 QK + softmax 段完全掩盖，再加深 V 流水只多耗 smem；其三，**F32Acc 达 0.89x**（203.9）为四配置最快，且 Max Err 由 2.441e-04 降至 1.526e-05，更快也更准，印证上文「累加档位同时影响数值与速度」的读数；其四，四个配置均低于 cuDNN 同精度基线（0.68x∼0.89x），差距随流水参数与累加档位收窄。

18.7 实践经验与考据说明

实践经验 18.1 (F2/F3 核校通过)**：** mbarrier 257 协议注释 vs 实际代码（L808 起）逐条核验：（1）arrive_count = 257 = 256 + 1，与 L968–975 的 `init(&full/empty_{Q,K,V}, kConsumerThreads + 1)` 一致（256 consumer 每线程 arrive + producer 1 次）；（2）「Q：full_Q——Producer 发 arrive_tx，Consumer 在主循环前 wait 一次」与 P0（L987–998，TMA + arrive_tx 一次）及 L1096 的单次 `full_Q.wait` 一致，无 empty_Q → Q 一次装载语义一致；（3）「K：full_K[kStagesK]/empty_K[kStagesK] 多 stage pipeline」与声明（L951–952）、P1 预取 Sk−1、P2 稳态预取一致；（4）「V：单 buffer 串行（QK 前发 TMA → QK 重叠 → PV 前等 → PV 后释放）」与 P2 的 V 先发（L1035–1048）、3d 前才 wait（L1264）、PV 后释放（L1311）逐点一致；（5）D=128 多次 TMA 共享 barrier 的计数描述（L980–983）与 P0/P1/P2 的 `kTmaChunks` 次装载 + 单次 expect_tx 一致。协议注释整体**与代码一致**。

实践经验 18.2 (F4 · 口径差异)：README FA2 TMA WS 数字不可复现 见 17.6 节：README 292T (F16Acc) vs 本书复测 183.8T / 原版同卡 190.4T，F16/F32 相对高低亦相反。README 的 bench 打印经 `should_print_ tflops` (bench/flash_attn.cu L44) max-of-N 过滤，只在超过本次运行历史最大值时打印，天然系统性偏高；叠加 warmup=2/repeat=3 的小样本均值，口径与本书 (warmup=5/iters=20 两轮复现) 不可比。正文一律标注复测口径。

实践经验 18.3 (F1 · 注释过时)： L873 模板参数注释「head dim: v1 only 64」模板注释写 `kHeadDim // head dim: v1 only 64`，但同 kernel 的 static_assert (L895–896) 明确接受 64 与 128，swizzle_fa (L859–870)、smem 占用表 (L826–834 的 D=128 列)、P0/P1/P2 的 kTmaChunks 分支与 bench/flash_attn.cu 的 D=128 测试全部支持 128。该注释是 D=128 支持加入前的残留，与 L842「v1 限制：kHeadDim=64 or 128」自相矛盾 (结论以本框注为准)。

实践经验 18.4 (F1 核校通过)：「SM120 optin ∼100KB」与 smem 占用表 本机实测 `cudaDevAttrMaxSharedMemoryPerBlockOptin` = 101376 B = 99.0 KB (17.10 节首行输出)，注释「∼100KB」近似正确；占用表六个组合 (32/40/48/56 KB 与 64/80/96/112 KB) 逐格复算与式 18.8 一致；「D=128 推荐 S=2 (80 KB 甜点)」的判断与 17.6 节复测的 Sk=2 饱和现象互相印证。

实践经验 18.5 (F5 · 依赖关系考据)： flash_attn.cuh 未自含 swizzle 符号 `flash_attn.cuh` 只 #include `base.cuh`，但其 L212 起调用的 `swizzle<64>` 定义在 `hgemm.cuh` L390，单编 flash_attn.cuh 会报 identifier undefined，必须按 bench/flash_attn.cu L4–5 的顺序先引 hgemm.cuh。属头文件依赖不自含的历史遗留，book/tests 的 include 顺序已按此固定。

全量核查记录汇总于 `book/CHECKLOG.md`，结论以上述框注为准。

18.8 常见坑

坑一：线程编队的除数。`wg_idx`/`wg_tid` 的阈值必须是 `kProducerThreads` (128)。误用 256 时 384/256 非整，producer 分到 256 个线程、consumer 剩 128 个线程，屏障却等 256 个 consumer arrive，挂死且无编译期征兆 (源码 L960–962 的 WARN 注释即为此而写)。

坑二：漏 C0 预热，静默挂死。producer 首轮 `wait(empty)` 需要 257 个 arrive，其中 256 个必须由 consumer 在 C0 预存。本章比 ch13 多一层：`empty_K` 与 `empty_V` **两个阵列**都要预热，漏任一个，对应通道首轮即死。

坑三：expect-tx 字节数按 chunk 计数。D = 128 时一个 tile 的 TMA 拆成 2 次 (box 内维 ≤ 128B 硬约束)，`expect_tx` 却只发一次、声明**总量**。漏乘 kTmaChunks 则 tx 永不归零 (挂死)，多算则提前翻转 (读半新半旧数据，静默错误)。少报挂死、多报静默，后者更危险 (ch13 坑三相同)。

坑四：TMA 坐标忘了 major_base。descriptor 覆盖 $[BHN, D]$ 全量 2D 矩阵，(batch, head) 的偏移 $(N_bH+N_h)N$ 必须折进 `major_coord`。漏加时所有 block 都读 head 0，**形状对、数值错**，且 B=1 或 H=1 的测试单例恰好测不出来 (偏移恒 0)。

坑五：`full_V` 的 wait 不能提前也不能拖后。提前到 QK 前：V TMA 的重叠窗口直接损失 (softmax 段本可继续装 V)；拖到 rescale 后：PV 空等 V 到达。正确位置是「PV 前最后一条 wait」，源码 L1260–1264 的位置就是最优点。

坑六：smem opt-in 与 stage 上限。D=128、Sk=2 需 80 KB 动态 smem，超过 48 KB 默认上限，launch 前必须 `cudaFuncSetAttribute`；Sk=4 的 112 KB 超 99 KB optin 上限，只能 SKIP (占用表见 17.3.4 节)。

坑七：setmaxnreg 是 TMA 地址空间敏感的。`NOTES_V2_REG_ ALLOC/DEALLOC` 默认宏关闭；TMA dst 为 `shared::cluster` 时 ptxas 以 C7506 丢弃该指令 (第 十五 章考据)，`shared::cta` 下 sm_120a/sm_120f 均保留 (build.sh sm_120f 目标为实验预设组合)。默认构建里源码的 40/80/168 预算只是注释性意图；真实寄存器分配靠 `__launch_bounds_(384, 1)`。排查寄存器溢出时不应把注释数字当作事实。

坑八：seqlen 对齐限制。v1 要求 seqlen % B_r=128 = 0 且 % B_c=64 = 0 (无尾 tile 处理)，实际部署的变长序列需先 pad 或走分桶。测试辅助工具的 unaligned SKIP 分支即此约束的运行时表达。

18.9 面试要点速查

Q1：FA 的 TMA+WS 与 HGEMM 的 TMA+WS 差在哪？

A：三次「GEMM 结构」差异。HGEMM 一次 GEMM、K 维迭代、A/B 都 staged；FA 两次 GEMM ($QK^\top$ 与 PV) 共用一个 KV 迭代、K staged 但 V 独立通道 (S_v 可选深度)、Q 因 split-Q 只装一次 (无 empty_Q)。同步协议因此从「单一屏障对」变成「Q 单屏障 + K/V 两对通道」。

Q2：arrive count 为何是 257 而不是 129？

A：consumer 是 256 线程 (8 warp `mma.sync`，warp 级指令无 warpgroup 对齐约束，逐线程 arrive) + producer 1 线

程。WGMMA 版是 128 + 1 = 129 （warpgroup 为最小单位）。数字差别的根源是**计算指令的对齐粒度**。

Q3：full 与 empty 各在什么时机翻转？

A：full[s]：256 consumer arrive（wait 时）+ producer arrive_tx（1 次）+ TMA 字节落盘（complete-tx 硬件计数）三者齐；empty[s]：256 consumer arrive （读毕释放）+ producer arrive（写前报到）。每相位预算恒 257，计数闭合。

Q4：为什么 empty_K 的释放点在 QK 之后而不是轮末？

A：$QK^\top$ 算完 K smem 即无人读（softmax 只访问寄存器、PV 只读 V），早释放让 producer 提前 softmax + PV 两段时间覆盖该 stage，stage 周期变短，等价于同深度流水下更高的迭代密度。

Q5：D = 128 时 TMA 怎么装 256B 的行？

A：128B swizzle 模式硬约束 box 内维 $\le$ 128B（64 half）。拆成 2 次 box (64, B) 沿 head_dim 发射，写 chunk-major 布局 $[2, B, 64]$；多次 TMA 共享同一 full 屏障，complete-tx 各自计数、expect_tx 一次声明总量。

Q6：TMA swizzle 写入 + ldmatrix.trans 读取为何天然正确？

A：128B swizzle 是 smem 地址的 1-1 重排；ldmatrix 用同一 `swizzle_fa` 公式算物理地址（1024B 对齐保证 phase = 0），读写两端一致；`.trans` 的转置发生在**寄存器分配**层，与 smem 物理布局正交。

Q7：FA tile 级算术强度？

A：$AI = B_r$ FLOP/B（式 18.7，与 D 无关）。Q 被整个 KV 遍历摊薄，K/V 每 tile $2B_cD \times 2$ B 的字节对应 $2 \times 2B_rB_cD$ FLOP 的计算。B_r 越大装载摊薄越充分，但 smem/寄存器/尾 tile 代价同步增大，128 是当前平衡点。

Q8：ws kernel 的收尾为什么没有全局同步？

A：producer 尾部不再发预取、自然退出；consumer 算完最后一块做 epilogue 退出。两支流通过 mbarrier 的相位闭合保证「consumer 读的一定是最后写入的数据」，无需 `__syncthreads`，同步已按数据依赖局部化。

18.10 最小测试与动手实验

测试文件：`book/tests/ch17_fa2_tma_ws.cu`（逻辑抽取自 bench/flash_attn.cu `test_flash_attn_tma_mma_ws_impl`@L808，参考实现由 cuDNN SDPA 换成 CPU fp64，softmax 全程 double 累加）。覆盖 $D \in \{64, 128\}$ × seqlen $\in \{128, 512\}$ × F16Acc/F32Acc 共 8 例：seqlen = 128 时 T_c=2 触发 stage 回绕（Sk=2 的 0→1→0）；512 时 8 轮迭代覆盖流水排空；D=128 走 `swizzle_fa` 的 chunk-major 双 TMA 路径。两档精度分别按容差表 F16Acc 5×10^{-2} / F32Acc 10^{-3} 判定。一个工程细节：`cuTensorMapEncodeTiled` 是 driver API 而 `build_tests.sh` 不带 `-lcuda`，测试经 `dlopen` 解析该符号（字段填法与 common.cuh `create_tensor_map` 逐字段一致）；另按 bench/flash_attn.cu L4–5 的 include 顺序先引 hgemm.cuh（swizzle 符号所在，17.7 节考据框五）。

在 `book/tests/` 目录下编译运行（本机 sm_120a，6 号卡）：

```
$ ./build_tests.sh --arch sm_120a --ch ch17
max shared memory per block (optin): 101376 B (99.0 KB)
PASS fa2_tma_ws D=64 N=128 Sk=2 Sv=1 F16Acc: max_abs_err=1.746e-04
PASS fa2_tma_ws D=64 N=128 Sk=2 Sv=1 F32Acc: max_abs_err=8.149e-05
... (8 例全 PASS，F16Acc ~ 2e-04 / F32Acc ~ 5e-05) ... ALL OK
```

bench 子命令输出 17.6 节复测表（`./.build/ch17_fa2_tma_ws bench`，数据落 `.tmp/book-bench/ch17/`）。动手实验建议：（1）把 3d 步的 `full_V` wait 上移到 QK 之前，对比 bench 中 Sk=1 与 Sk=2 的差距如何变化（体会 wait 位置对重叠窗口的影响，坑五）；（2）删掉 C0 的 `empty_V` 预热循环，观察挂死（坑二）；（3）`expect_tx` 的 `kVTileBytes` 故意少乘 `kTmaChunks`（D=128 例），观察挂死与静默错的分界（坑三）；（4）把测试 seqlen 换成 192（% 128 $\neq$ 0），确认 unaligned SKIP 分支。

延伸阅读

- @Frank Wang《Async Copy 及 Memory Barrier》
  https://zhuanlan.zhihu.com/p/685168850
  （cp.async 到 mbarrier 的异步同步体系，本章 17.3.2 节双计数协议的补充视角）；
- @ 竹熙佳处《cute 之 TMA Copy》
  https://zhuanlan.zhihu.com/p/2003198909405763007

（TMA 拷贝与 descriptor 的封装视角，第 二十四 章的先导；本章 17.4.3 节 box/SWIZZLE 约束的另一路推导）；

- NVIDIA PTX ISA §9.7.13.15（mbarrier）、§9.7.9.25 （cp.async.bulk.tensor）——协议语义核查基准（本地镜像：cuda-cpp-kernel skill 的 ptx-docs）；
- FlashAttention-2（Dao, arXiv:2307.08691）——式 18.2–18.3 的算法出处，第 十六 章有完整推导。

第十九章　FA3：双 Consumer Warpgroup

19.1　本章导读

第 十八 章的 `flash_attn_tma_mma_ws_stages_split_q` 用一个 producer warpgroup（128 线程，只有 thread 0 真正干活）+ 一个 consumer warpgroup（256 线程 = 8 个 `mma.sync` warp）把 FA2 的「装载」与「计算」彻底分离。本章讲它的下一代 `flash_attn_3_tma_ws_stages_split_q`（flash_attn.cuh L1516）：把 consumer 从 **1 个 warpgroup 扩成 2 个**（各 128 线程 = 4 个 warp），让两条**互相独立**的计算流在同一 SM 上并发推进，最后把两流的「未归一化」中间状态做一次**稳定归并**。这就是 FlashAttention-3 论文（arXiv:2407.08608）warp specialization 思路在本仓库 SM120 平台上的具体实现。

源码区间：flash_attn.cuh L1441–2192。其中 L1441–1503 是约 60 行的头部设计注释，覆盖本章三个核心素材：线程布局与奇偶 tile 路由（L1449–1452、L1472–1480）、per-WG 独立 stage 语义（L1454–1462）、split-KV 的稳定归并公式与 T_c=1 退化（L1482–1496）。本章逐条解读这份注释，并与代码逐点核验。

编译宏与架构限定：`NOTES_V2_ENABLE_TMA_MMA_WS`，sm_90a 与 sm_120a 均可编译；要求 CUDART ≥ 13.0（common.cuh L21–23 有 #error 兜底）。本机 GPU 为 sm_120a（RTX PRO 5000），本章测试与 bench 复测均在本机完成。

前置：第 十六 章（attention 数学与 online softmax）、第 十七 章（split-Q 与 fragment 流转）、第 十八 章（TMA + WS 的 mbarrier 协议、full/empty 计数、early release 时机）、第 十五 章（SM120 上的 warp specialization 与 `setmaxnreg` 池数学）、第 十四 章（TMA descriptor 与 mbarrier 双计数事务模型）。**后续**：第 二十六 章会看到同一结构的 CuTe 重写（`flash_attn_3_tma_mma_ws_split_q_cute`）。

19.2　问题与动机

第 十八 章的复测给出一组数据：TMA + WS 相对 cp.async 版的收益稳定在 +20% 量级，但再加深 K 流水（Sk：2 → 3）几乎不再涨（< 0.5%）。这说明**装载已经不在关键路径上了**，瓶颈退回到计算侧的两件事：其一，consumer 内部的 `mma.sync` 与 softmax 是串行的，$QK^\top$ 算完才能算 softmax，softmax 算完才能发 PV，任何一个 warp 都处于这条依赖链内；其二，`mma.sync` 是**同步指令**，指令在 Tensor Core 上执行期间该 warp 只能空等（第 十四 章的 WGMMA 则可以把 GEMM「发出去就返回」，指令与后续 CUDA Core 指令并行）。

FA3 论文给出的定量背景是：在 Hopper 上，一次 128×128 tile 的 attention 里两个 GEMM 合计 8.4 MFLOP（FP16 Tensor Core ≈ 4096 FLOP/cycle ⇒≈ 2048 cycles），而 softmax 的 128×128 个 exp（MUFU.EX2 吞吐 ≈ 16 OPS/cycle）要 ≈ 1024 cycles，**softmax 占约三分之一的时间**；若 GEMM 降到 FP8（算力翻倍），softmax 的占比还要涨到一半。这些非 matmul 的「慢指令」若不被 overlap，Tensor Core 就有一段时间在空转。注意上述数字是 Hopper 口径（论文与 @ 杨远航解析一致），本机 sm_120a 的绝对吞吐不同，本章只取「量级」结论：**softmax 与 GEMM 是两条可并行的硬件通路**（CUDA Core/SFU 与 Tensor Core），值得为它引入这部分结构复杂度。

于是引出本章的结构性改动：把 KV tile 序列**按奇偶拆成两半**，交给两个独立推进的 consumer warpgroup，各自维护自己的 online-softmax 状态与 K 流水，最后合并。两条流各自的相位（QK/PV 的 Tensor Core 段与 softmax 的 MUFU/ALU 段）在时间上**自然错开**：当 WG1 处在 $QK^\top$ 的 MMA 段时，WG2 往往正处在 softmax 段，硬件 warp 调度器把两者的发射槽交错填满，Tensor Core 与 CUDA Core/SFU 的利用率同时抬高。这条路径不需要 WGMMA，代价是需要一套 **分治-合并**的数值方案与更复杂的屏障/寄存器预算，以下逐节展开。

19.3 数学原理

19.3.1 split-KV 双流划分与合并不变量

沿用第 十六 章的记号：B, H, N, D 为 batch/heads/seqlen/head dim，$B_r = B_c = 64$ 为 Q/KV block 行数，$\gamma = 1/\sqrt{D}$，$T_c = N/B_c$ 为 KV tile 数。设 Q 的第 i 行（$i \in [0, B_r)$）对应的分数为 $S_{ik} = \gamma\, q_i^\top k_k$。本章 kernel 把 tile 集合按**奇偶**分成

$$\mathcal{T}_0 = \{0, 2, 4, \dots\}, \qquad \mathcal{T}_1 = \{1, 3, 5, \dots\}, \tag{19.1}$$

$\mathcal{T}_0$ 交给 `consumer_id`= 0（线程 128–255），$\mathcal{T}_1$ 交给 `consumer_id`= 1（线程 256–383）。两流各自维护一套 **未归一化**的 online-softmax 状态：

$$m_0 = \max_{k \in \mathcal{T}_0} S_{ik}, \qquad \ell_0 = \sum_{k \in \mathcal{T}_0} e^{S_{ik} - m_0}, \qquad O_0 = \sum_{k \in \mathcal{T}_0} e^{S_{ik} - m_0}\, v_k, \tag{19.2}$$

m_1, ℓ_1, O_1 对 $\mathcal{T}_1$ 同构。因为 $\mathcal{T}_0 \cup \mathcal{T}_1 = \{0, \dots, T_c - 1\}$ 且两半互不相交，全局 softmax 可以**精确**地由两个 partial 重组：

$$m = \max(m_0, m_1), \qquad \ell = e^{m_0 - m}\,\ell_0 + e^{m_1 - m}\,\ell_1, \qquad O = e^{m_0 - m}\,O_0 + e^{m_1 - m}\,O_1, \tag{19.3}$$

最终输出 O/ℓ。源码把两个权重命名为

$$\alpha = e^{m_0 - m} \le 1, \qquad \beta = e^{m_1 - m} \le 1, \tag{19.4}$$

即 kernel 头注释 L1489–1490 的 `WG1/WG2 rescale factor`。把式 19.2 的指数统一改写成以全局 m 为基准，$e^{S_{ik} - m_0} = e^{S_{ik} - m} \cdot e^{m - m_0}$，两个求和因此可以逐项相加；由于 $m \ge m_0$ 且 $m \ge m_1$，式 19.4 的两个指数 **非正**，绝不会上溢。这正是「以全局 max 为 pivot」的数值意义，与第 十六 章 online rescale 的 $e^{m^{\text{old}} - m^{\text{new}}}$ 是同一个思想，只是从「跨迭代」搬到了「跨 warpgroup」。

一个容易忽略的细节：ℓ 的合并与 O 的合并必须用**同一组** α, β，而它们又必须由合并后的 m 定义；源码里是先取 max、再算两个 exp（L2091–2096），顺序勿调换；反过来写虽然数值上仍近似正确，却会引入一次多余的 $e^{m - m}$ 往返。

19.3.2 $T_c = 1$ 的退化：$-\infty$ 参与的 merge

$N = B_c$（即 $T_c = 1$）时 `consumer_id`= 1 一个 tile 都没有，它的 partial 停在初始化值：$m_1 = -\infty$、$\ell_1 = 0$、$O_1 = 0$。代入式 19.3：$m = \max(m_0, -\infty) = m_0$，$\beta = e^{-\infty - m_0} = e^{-\infty} = 0$，$\ell = 1 \cdot \ell_0 + 0 \cdot 0 = \ell_0$，$O = O_0$，**自然退化为单 WG 结果**，不需要任何特判分支（头注释 L1495–1496 即此结论）。

这里存在一个数值陷阱，也是「未归一化累加」这一约定的第二个理由（第一个是第 十六 章 FA2 省掉每块对角修正）：如果 O 以**已归一化**形态参与合并，就必须算 O_1/ℓ_1，0/0 直接产 NaN 并污染整个输出。而式 19.3 只做乘加（$0 \times 0 = 0$，$e^{-\infty} = 0$ 由硬件 `ex2.approx` 精确给出），NaN 无从产生。「O 全程未归一化、只在 epilogue 除一次 ℓ」的收益在本章显现。

奇数 T_c 也不需要特判：$T_c = 3$ 时 $\mathcal{T}_0 = \{0, 2\}$、$\mathcal{T}_1 = \{1\}$，两流 partial 规模不等，但式 19.3 对任意两半都成立（只要求并集完整、交集为空）。源码里这表现为 consumer 循环的终止条件 `tile < Tc`（L1742）：warpgroup 之间**不交换**「我做了几个 tile」这类元信息，归并公式对不对称划分天然鲁棒。

19.3.3 每 WG 独立 stage：屏障计数从 257 缩到 129

第 十八 章的 K/V 是**跨 WG 共享**的：Sk 个 K slot 与 S_v 个 V buffer 由 8 个 consumer warp 共用，屏障 `full_K[s]` 的期望到达数是 $256 + 1 = 257$。本章把 bank 按 `consumer_id` 切开（L1552–1553 的布局注释），于是每个 WG 拥有**自己的一套** Sk 个 K slot 与 1 个 V buffer，屏障也随之按 `[cid][stage]` 二维声明（L1568–1574）：

屏障	期望到达数（init 值）	语义
`full_Q`	$256 + 1 = 257$	Q tile 就绪（两 WG 共用，一次性）
`full_K[cid][s]`	$128 + 1 = 129$	cid 的 K stage s 数据落盘
`empty_K[cid][s]`	$128 + 1 = 129$	cid 的 K stage s 可覆盖
`full_V[cid]` / `empty_V[cid]`	$128 + 1 = 129$	cid 的 V buffer（深度 1）

数字 129 与第 十五 章 hgemm 版**完全相同**，含义却不同：那边是「128 consumer 线程 + 1 producer 线程」，这边是「**一个** consumer WG 的 128 线程 + 1 producer 线程」。协议只认线程数、不认角色，两个 kernel 的数字恰好相同。把「共享 Sk 个 slot」改成「每 WG 各 Sk 个」的收益写在头注释 L1461–1462：Sk = 2 时，共享方案里每个 WG 的实际深度只有 1（两个 WG 竞争同一对 slot 与同一对屏障），而现在每个 WG 都获得真正的深度 2，两个 WG 不会使用同一个 slot 或同一张屏障（L1458–1459），producer 因此可以连续不断地按 cid = tile & 1 投递数据，不存在「等另一个 WG 释放 slot」的耦合。

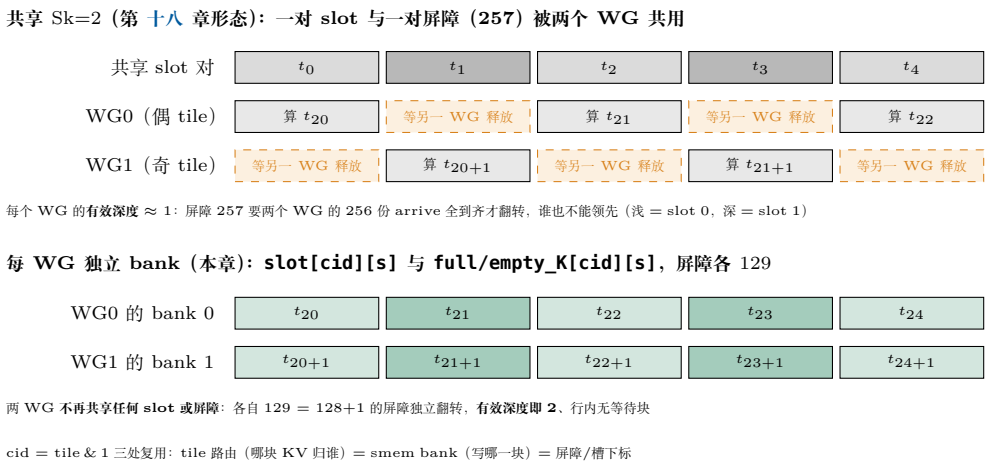


图 19.1: 共享 Sk=2 slot 与每 WG 独立 bank 的对照（18.3.3 节；浅 = slot 0，深 = slot 1，t 的下标即 KV tile 号）。上：共享方案里两个 WG 竞争同一对 slot 与同一对屏障——`full_K[s]`/`*empty_K[s]` 的 257 份 arrive 要求**两个** WG 的 256 个 consumer 线程全部到齐才翻转，于是任何一方都不能比另一方多跑一块：$t_0, t_1, t_2, \ldots$ 被两 WG 交替消费，各自的有效流水深度只有 1。下：本章把 bank 按 `consumer_id` 切开后，每个 WG 拥有自己的一套 Sk 个 slot 与自己的屏障（129 = 128+1），两个 WG 并行推进、互不干扰，深度 2 为实际深度。

19.3.4　setmaxnreg 池数学：三 warpgroup 的寄存器预算

第 十五 章已给出池不等式：每 SM 寄存器文件 65536 个 32-bit 寄存器，CTA 池按 warp 配额回收/发放，任意配置须满足 $\sum_w 32\, r_w < 65536$ 并留 ≥ 2048 的 slack；同时 `setmaxnreg.inc` 是**阻塞语义**，池中寄存器不足时永久阻塞（PTX ISA §9.7.19.5，本地镜像 ptx-docs）。本章的配置是三个 warpgroup：producer 128 线程 `dealloc(40)`、两个 consumer 各 128 线程 `alloc(168)`（D=128）或 `alloc(80)`（D=64），见 L1614 与 L1702–1706。把「每 warpgroup 128 线程」这个公因子提出来，不等式可以写成更好记的**每线程等效配额**形式：

$$r_p + 2\, r_c \;\leq\; \frac{65536}{128} \;=\; 512, \qquad \text{即} \qquad \underbrace{40}_{r_p} + 2 \times \underbrace{168}_{r_c} = 376 \;\leq\; 512 \quad (D{=}128), \tag{19.5}$$

D=64 时为 $40 + 2 \times 80 = 200$。两者余量 136 / 312（等价于 $136 \times 128 = 17408$ 与 $312 \times 128 = 39936$ 个寄存器），远离危险区。反过来看第 十五 章引用的那次事故配置：producer 64、两个 consumer 各 224，$64 + 2 \times 224 = 512$ **恰好等于**上界。驱动对寄存器文件另有保留份额，有效池小于 65536，`alloc` 永远凑不齐，于是 consumer 卡在 `setmaxnreg`、producer 卡在等 consumer 的 barrier arrive，**双向互等导致挂死**（无 illegal instruction、无 error code、无超时）。事故修复版（64/208/208 或 32/232/232）分别给出 480 与 496，即「留 ≥ 16 的每线程等效余量」，恰好是工程规则「留 ≥ 2048 slack」除以 128 的结果。

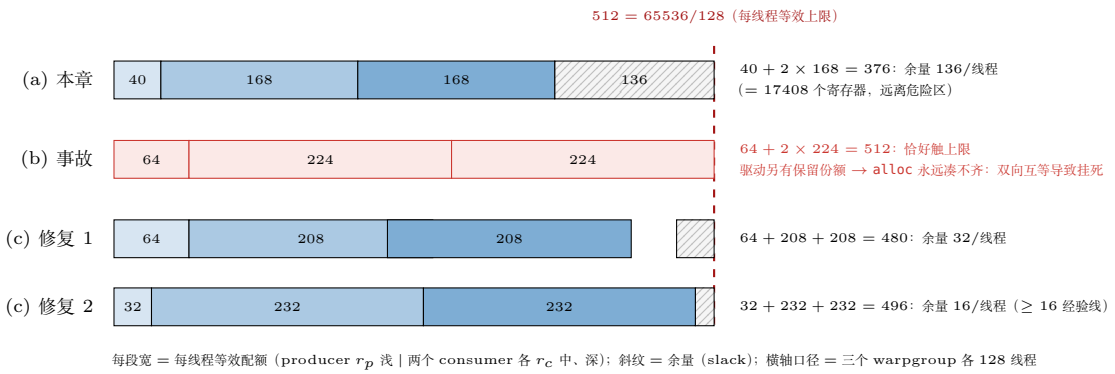


图 19.2：65536 个寄存器按角色分配的三种方案（式 19.5 与 18.3.4 节事故分析的图形化）。横轴为每线程等效配额 $r_p + 2r_c \le 512 = 65536/128$：(a) 本章 D=128 的 $40 + 2 \times 168 = 376$，余量 136/线程（= 17408 个寄存器），远离危险区；(b) 第 十五 章那次事故的 $64 + 2 \times 224$ 恰好等于 512——驱动对寄存器文件另有保留份额，有效池小于 65536，于是 consumer 卡在 `setmaxnreg`、producer 卡在等 barrier arrive，**双向互等导致挂死**；(c) 修复版的 480 与 496 分别留出 32 与 16 的每线程余量，后者正是工程规则「≥ 2048 slack」除以 128 的下界。

为什么 FA3 这类结构**必须**做寄存器再分配？因为两个 consumer 需要持有的活跃集比自己发装载的 producer 大一个量级：每线程持有 S fragment（`kValTileSeqLenK`= 8 列 × `kNRegs`）、O/D 累加器（`kValTileHeadDimV`= $D/8$ 列 × `kNRegs`∈ {2, 4}）、Q/K/V 操作数碎片与 m/ℓ 统计量（L1725–1731）。D=128 的 F16Acc 路径下，单是 R_D 就占 $16 \times 2 = 32$ 个 `uint32`，R_O 又是 32 个；不给它 168 的上限，ptxas 只能把溢出量写入 local memory，每轮 KV 迭代多出成百上千字节的 local 往返。producer 侧正相反：它只做地址计算与 TMA 提交，40 个就够。**同一份 64K 寄存器文件，按角色重新分配**。这就是 warp specialization 收益的另一半来源。

必须补一句现实约束（第 十五 章已考据）：若 `cp.async.bulk.tensor` 的目的地址写成 `shared::cluster`，ptxas 就会以 C7506 丢弃 `setmaxnreg`（CUDA 13.2，与 arch 后缀无关）；写成 `shared::cta` 后 sm_120a 与 sm_120f 都完整保留。默认构建产物里 40/168/80 是 **意图注释**而非既成事实，真正决定分配的是 `__launch_bounds__(384, 1)`（L1515）与 ptxas 的分配器。池数学的价值在于：一旦开启寄存器再分配（正确的 TMA 形式 + 宏），这套数字必须**先算对**，否则会重演上面那次挂死事故。

19.4 设计与实现

19.4.1 总体结构：三分线程与奇偶路由

头部注释（L1445–1452）先列明「与 8b 的核心区别」：

- 8b：1 个 producer WG（128T）+ 1 个 consumer WG（256T，8 warp），全 KV 遍历；
- 8c：1 个 producer WG（128T）+ 2 个 consumer WG（各 128T，4 warp），KV tile 奇偶拆分。

线程布局共 384 线程，三支 WG 的分工：

- WG0 $[0, 127]$：TMA producer（仅 thread 0 发 TMA）；
- WG1 $[128, 255]$：`consumer_id` = 0，处理偶数 KV tile （0, 2, 4, . . . ）；
- WG2 $[256, 383]$：`consumer_id` = 1，处理奇数 KV tile （1, 3, 5, . . . ）。

角色判定只有两行（L1577–1581）：`threadIdx.x < 128` 为 producer；其余按 `(threadIdx.x - 128) / 128` 得 `consumer_id` ∈ {0, 1}，组内序号 `wg_tid` ∈ [0, 128) 再切 warp/lane。注意 `consumer_id` 在本 kernel 里三**处复用**：既标识「我处理哪一半 tile」（`tile = consumer_id; tile += 2`），也标识「我用哪一套 smem bank / 屏障」（`[cid]` 下标），还标识「我用哪些 stage 槽」。这种「数据划分 = 存储划分 = 同步划分」的对应关系，是奇偶拆分最大的工程优势（三者的完整角色与数据流见图 19.5）。

flash_attn.cuh L1550-1565（smem 三分：Q 一份 + 每 WG 独立 K/V bank）

```
// ---- Shared memory layout (per-WG independent K/V banks) ----
// [Q_shared: Br*D]
// [K[cid=0][s=0..Sk-1]: Sk*Bc*D]  [K[cid=1][s=0..Sk-1]]
// [V[cid=0]: Bc*D]                 [V[cid=1]: Bc*D]
extern __shared__ __align__(1024) uint8_t smem_fa3_tma_ws[];
half *Q_smem = reinterpret_cast<half *>(smem_fa3_tma_ws);
half *K_smem_base = Q_smem + Br * kHeadDim;
// K: [numConsumerWGs][kStagesK][Bc*kHeadDim], linearized
//   K_smem_base + cid * (kStagesK * Bc * kHeadDim) + stg * (Bc * kHeadDim)
half *V_smem_base = K_smem_base + kNumConsumerWGs * kStagesK * Bc * kHeadDim;
// V: [numConsumerWGs][Bc*kHeadDim]
//   V_smem_base + cid * (Bc * kHeadDim)

const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem_base);
const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem_base);
```

smem 于是是三段之和（half 元素 2 B）：

$$\text{smem} = \underbrace{B_r D \cdot 2}_{\text{Q（两 WG 共用）}} + \underbrace{2 \cdot \text{Sk} \cdot B_c D \cdot 2}_{\text{K：每 WG 各 Sk 个 slot}} + \underbrace{2 \cdot B_c D \cdot 2}_{\text{V：每 WG 1 个 buffer}} . \tag{19.6}$$

逐档复算（$B_r = B_c = 64$）：D=64 时 Sk = 1/2/3/4 分别为 40.0/56.0/72.0/88.0 KB；D=128 时 Sk = 1 为 80.0 KB，Sk = 2 则是 **112 KB > 99 KB** 的 optin 上限（本机实测 101376 B = 99.0 KB，18.7 节的考据框核对注释口径），因此 D=128 **只支持 Sk** = 1 （头注释 L1500–1501）。这解释了为什么「更深流水」在本章并非没有代价：bank 一分为二后，smem 的 O(Sk) 项直接翻倍。

19.4.2　producer：单线程 TMA 与 cid 路由

producer 仍是「128 线程里只有 `wg_tid == 0` 执行装载」的原有结构：`dealloc(40)` 之后先装 Q（P0，两 WG 共用，一次 `arrive_tx` 声明 $B_r D \times 2$ 字节），再进入**全局顺序**的 tile 循环（P2）。与第 十八 章的关键差异只有一处：每轮的 cid = tile & 1 决定数据落到哪个 bank、走哪张屏障。

flash_attn.cuh L1647-1677（P2 前半：K 先行；Sk=1 就地装载，Sk≥2 预取 tile+2）

```
    for (int tile = 0; tile < Tc; ++tile) {
      const int cid = tile & 1;

      // Step 1: Load/prefetch K first (release depends on QK, which is early)
      if constexpr (kStagesK == 1) {
        // Sk=1: load K[tile] to the single slot (reuse after consumer QK release)
        empty_K[cid][0].wait(empty_K[cid][0].arrive());
        for (int c = 0; c < kTmaChunks; ++c) {
          tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
                          c * Bc * kTmaBoxMinor,
                      tensorMapK, c * kTmaBoxMinor,
                      qkv_major_base + tile * Bc, full_K[cid][0]);
        }
        tma_arrive_expect_tx(full_K[cid][0], kKTileBytes);
      } else {
        // Sk>=2: prefetch K[tile+2] to next slot (true ping-pong pipeline!)
        const int local_tile = (tile - cid) / 2;
        const int stg_next = (local_tile + 1) % kStagesK;
        const int next_tile = tile + 2;
        if (next_tile < Tc) {
          empty_K[cid][stg_next].wait(empty_K[cid][stg_next].arrive());
          for (int c = 0; c < kTmaChunks; ++c) {
            tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
                            stg_next * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
```

```
                          tensorMapK, c * kTmaBoxMinor,
                          qkv_major_base + next_tile * Bc,
                          full_K[cid][stg_next]);
            }
            tma_arrive_expect_tx(full_K[cid][stg_next], kKTileBytes);
          }
        }
```

flash_attn.cuh L1679-1686（P2 后半：V 装载，release 依赖 PV）

```
        // Step 2: Load V[tile] (release depends on PV, which is late)
        empty_V[cid].wait(empty_V[cid].arrive());
        for (int c = 0; c < kTmaChunks; ++c) {
          tma_load_2d(V_smem_base + cid * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                      tensorMapV, c * kTmaBoxMinor,
                      qkv_major_base + tile * Bc, full_V[cid]);
        }
        tma_arrive_expect_tx(full_V[cid], kVTileBytes);
```

「先 K 后 V」是刻意的发射顺序：K 的 empty 信号在 consumer 的 QK 阶段末尾（最后一组 `ldmatrix` 之后）就发出，V 的 empty 信号要等 PV 的 `ldmatrix` 完毕，故 K 的等待窗口比 V 短，应先发射 K。Sk ≥ 2 时 K 走真正的乒乓预取：第 t 轮把 tile $t+2$ 装进 $\text{stg}_{\text{next}} = (\text{local_tile}+1) \bmod \text{Sk}$；注意 `local_tile` $= (t - \text{cid})/2$ 是**本 WG 视角**的序号（L1663），producer 侧重新算了一遍，必须与 consumer 的 `local_iter`（L1743）严格对齐。

19.4.3 consumer 主循环：五个阶段与两个 early release

consumer 的入口是 C0 预热：先给**自己**那套 `empty_K[cid][*]` 与 `empty_V[cid]` 各预存 128 次 arrive（不是第 十八 章的 256 次，每张屏障只放行一个 WG），然后等 `full_Q`：

flash_attn.cuh L1702-1717（REG_ALLOC 分支 + C0 预热 + 等 Q）

```
    if constexpr (kHeadDim == 128) {
      NOTES_V2_REG_ALLOC(168);
    } else {
      NOTES_V2_REG_ALLOC(80);
    }

    // C0: init - mark OWN K/V slots as empty (consumer arrives, producer is 129th)
    for (int s = 0; s < kStagesK; ++s) {
      [[maybe_unused]] auto tk = empty_K[consumer_id][s].arrive();
    }
    {
      [[maybe_unused]] auto tv = empty_V[consumer_id].arrive();
    }

    // Wait for Q tile (shared by both WGs)
    full_Q.wait(full_Q.arrive());
```

主循环 `for (tile = consumer_id; tile < Tc; tile += 2)` 内的五个阶段与第 十八 章逐行同构（$\text{QK}^\top$ → safe softmax → PV → rescale），只有两处 **early release** 的位置值得复述，它们决定 producer 的重填窗口：

flash_attn.cuh L1789-1810（K 的 early release：最后一组 ldmatrix 之后立即放行 slot）

```
        // Early release K after last ldmatrix_K: K data now fully in registers
        if (tile_K_d == (kHeadDim / kMmaAtomK) - 1) {
          [[maybe_unused]] auto token = empty_K[consumer_id][k_stg].arrive();
        }

#pragma unroll
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
```

```
          for (int j = 0; j < kValTileSeqLenK; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
                           R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                           R_K[j][0], R_K[j][1],
                           R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
            } else {
              HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
                        R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
                        R_S[i][j][0], R_S[i][j][1]);
            }
          }
        }
      }
```

flash_attn.cuh L1887-1914（V 的 early release：PV 的 ldmatrix 完毕即放行 V buffer）

```
      // ---- P @ V = O[Br, d] ----
      // Wait for V[consumer_id] ready
      full_V[consumer_id].wait(full_V[consumer_id].arrive());
      tma_fence_proxy_async_shared_cta();

      fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_O, 0);
#pragma unroll
      for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          const int warp_smem_V_d =
              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
          const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
          const int lane_smem_V_d = warp_smem_V_d;
          // V smem: V_wg_base + swizzle_fa offset
          const uint32_t lane_smem_V_ptr =
              smem_V_wg_base +
              swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d) *
                  sizeof(half);
          LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
        }

        // Early release V after last ldmatrix_V: V data now fully in registers.
        // This lets producer start next V TMA during the remaining PV MMA +
        // online rescaling, giving more overlap window than releasing after PV.
        if (tile_V_Bc == (Bc / kMmaAtomK) - 1) {
          [[maybe_unused]] auto token = empty_V[consumer_id].arrive();
        }
```

两处共同点：**smem 一进寄存器就放行**。K 的最后一组 `LDMATRIX_X2` 之后 K 数据已全部在 R_K，后续 HMMA 与 softmax 不再读 K smem；V 同理（R_V）。放行越早，producer 的下一笔 TMA 越早发出。在 Sk=1 的 D=128 配置下，这几乎是唯一的重叠窗口。

19.4.4　merge epilogue：scratch 复用与两步同步

主循环退出后两 WG 各持一份 partial。归并的要点是**谁写谁读**：`consumer_id`= 1 把 R_D, m, ℓ 写进 smem scratch，`consumer_id`= 0 读回、与自己的 partial 合并、归一化并写 gmem。scratch **复用动态 smem**（主循环的 Q/K/V 区），不增加 launch 的 smem 预算：

flash_attn.cuh L2001-2026（scratch 复用前提与布局：R_D 区 + m/l interleaved 区）

```
    // Final merge of two WG partial states (split-KV reduction)
    //
```

```
    // 安全 alias dynamic smem 的前提:
    //   1) producer 已退出 mainloop (所有 TMA 已 arrive_expect_tx);
    //   2) 两个 consumer 都已 release 自己的 K/V stage (QK 后 release K,
    //      PV 后 release V), 且 mainloop 结束后不再访问任何 K/V smem;
    //   3) Q_smem 在 mainloop 内只读, 此后也不再访问。
    // 故此时整个 dynamic smem (Q/K/V 区) 生命周期结束, 可安全重命名为
    // merge scratch, 不增加 launch smem_bytes。第一次 __syncthreads
    // 保证所有 TMA/ldmatrix 完成后才开始写 scratch。
    // 完整的 split-KV merge 数学推导见 kernel 头部注释。
    // ====================================================================
    __syncthreads();

    // Merge scratch 布局 (按 wg_tid 索引, 每 WG 128 thread):
    //   [merge_rd]: 128 * kRdPerThread 个 uint32  <- WG2 未归一化 R_D fragment
    //   [merge_ml]: 128 * kValTileSeqLenQ 个 float4 <- WG2 的 [m0,m1,l0,l1] interleaved
    // WG1 用自己的 R_D/m/l 作为 (Oacc_0, m_0, l_0), 读取 WG2 写出的作为
    // (Oacc_1, m_1, l_1), 按 stable merge 公式归并后由 WG1 写最终 O。
    // m/l interleaved 为 float4 布局, 写入/读取各一次 128-bit store/load。
    constexpr int kRdPerThread = kValTileSeqLenP * kValTileHeadDimV * kNRegs;
    constexpr int kRdPackPerThread = kRdPerThread / 4;  // float4 个数 (kNRegs 始终偶数)
    constexpr int kMlPackPerThread = kValTileSeqLenQ;   // 每 thread 的 float4 个数
    uint32_t *merge_rd = reinterpret_cast<uint32_t *>(smem_fa3_tma_ws);
    float4 *merge_rd_pack = reinterpret_cast<float4 *>(merge_rd);
    float4 *merge_ml_pack = merge_rd_pack + 128 * kRdPackPerThread;
```

flash_attn.cuh L2037-2053（WG2 写出未归一化 partial：128-bit 连续 store）

```
    if (consumer_id == 1) {
      // R_D: 128-bit stores
      float4 *rd_dst = merge_rd_pack + wg_tid * kRdPackPerThread;
#pragma unroll
      for (int idx = 0; idx < kRdPackPerThread; ++idx)
        rd_dst[idx] = reinterpret_cast<float4 *>(&R_D[0][0][0])[idx];
      // m/l interleaved: [m0, m1, l0, l1] per i, 128-bit store
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        float4 ml;
        ml.x = lane_block_row_max_old[i][0];
        ml.y = lane_block_row_max_old[i][1];
        ml.z = lane_block_row_sum_old[i][0];
        ml.w = lane_block_row_sum_old[i][1];
        merge_ml_pack[wg_tid * kMlPackPerThread + i] = ml;
      }
    }
```

按 `wg_tid` 索引是关键设计：$R_D[i][j][*]$ 覆盖的 Q 行与 m/ℓ 数组的 `[i][0/1]` 行段严格对齐（`[i][0]` 管 rows $i\cdot 16 \sim i\cdot 16{+}7$，`[i][1]` 管 $+8 \sim +15$，见 L2069 注释），因此**同一 `wg_tid` 的两个 WG 覆盖完全相同的 Q 行**，可以逐元素归并而无需任何跨 lane 通信。第二次 `__syncthreads` 之后，`consumer_id`$=0$ 执行式 19.3：

flash_attn.cuh L2078-2101（稳定归并：$m \to \alpha/\beta \to \ell$ 与 Oacc 加权）

```
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        // 128-bit load: [m1_0, m1_1, l1_0, l1_1] interleaved
        float4 ml1 = merge_ml_pack[wg_tid * kMlPackPerThread + i];
        float m1_0 = ml1.x;
        float m1_1 = ml1.y;
        float l1_0 = ml1.z;
        float l1_1 = ml1.w;

        float m0_0 = lane_block_row_max_old[i][0];
        float m0_1 = lane_block_row_max_old[i][1];
```

```
        float l0_0 = lane_block_row_sum_old[i][0];
        float l0_1 = lane_block_row_sum_old[i][1];

        float m_0 = fmaxf(m0_0, m1_0);
        float m_1 = fmaxf(m0_1, m1_1);
        float alpha_0 = __expf(m0_0 - m_0);
        float alpha_1 = __expf(m0_1 - m_1);
        float beta_0 = __expf(m1_0 - m_0);
        float beta_1 = __expf(m1_1 - m_1);

        lane_block_row_max_old[i][0] = m_0;
        lane_block_row_max_old[i][1] = m_1;
        lane_block_row_sum_old[i][0] = alpha_0 * l0_0 + beta_0 * l1_0;
        lane_block_row_sum_old[i][1] = alpha_1 * l0_1 + beta_1 * l1_1;
```

随后是一次 $O = O_{\text{acc}}/\ell$ 的终归一化（L2127–2152）与第 十七 章的 shuffle + 128-bit 集体写回（L2154–2190），二者与 ch16/ch17 逐行相同，不再重复。关键在**由谁写回**：只有 `consumer_id`= 0 做归一化与写回（`if (consumer_id == 0)`，L2057），`consumer_id`= 1 写完后直接退出，省掉单侧的重复计算。

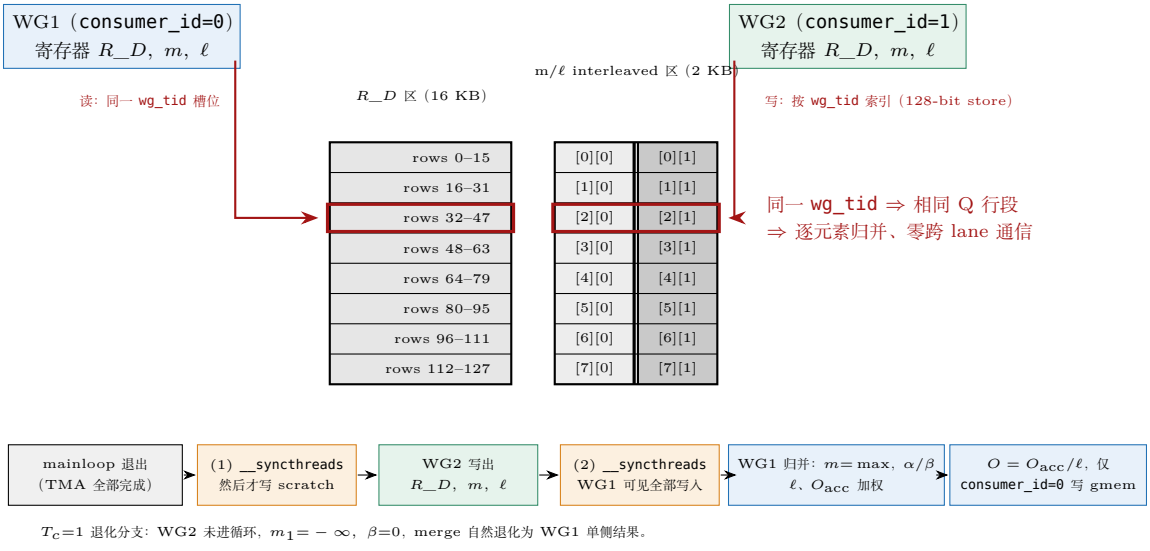


图 19.3: merge scratch 的 `wg_tid` 对齐设计（18.4.4 节）。scratch 复用主循环的 Q/K/V 动态 smem：R_D 区每 `wg_tid` 一段 float4 包；m/ℓ 区按 $[i][0/1]$ 行段组织——$[i][0]$ 管 rows $i\cdot16..+7$、$[i][1]$ 管 $+8..+15$，与 R_D fragment 覆盖的 Q 行严格对齐。WG2（`consumer_id=1`）按 `wg_tid` 写入（高亮行即某个 `wg_tid` 命中的行段），WG1 用**同一个** `wg_tid` 读回——两个 WG 的 partial 覆盖完全相同的 Q 行，可逐元素归并而无需任何跨 lane 通信。下方：两步 `__syncthreads` 的完整时序（第一次保证 TMA/ldmatrix 全部完成后才写 scratch，第二次让 WG1 看到 WG2 的全部写入）。

19.4.5　与第 十五 章 SM120 WS 的对照：同思想，不同硬件

两章都在 SM120（无 WGMMA）上做 warp specialization，骨架一致（单线程 TMA producer + mbarrier 双屏障 + `setmaxnreg` 池），差异集中在「consumer 有几个、彼此什么关系」：

	ch14 `hgemm_tma_mma_ws_tn`	本章 FA3 双 consumer
角色	1 producer WG + 1 consumer WG	1 producer WG + **2** consumer WG
划分对象	GEMM 的 K 维分块（共享 A/B 行）	**KV tile 奇偶拆分**（各持 partial）
consumer 间通信	无（写同一份输出）	终态 merge（smem scratch，两步 barrier）
屏障到达数	128 + 1 = 129（单组）	per-WG 128+1=129 / `full_Q` 257
寄存器池	40 / 232（2 个 WG）	40 / 168 / 168（3 个 WG，式 19.5）
收尾	直接 epilogue	merge → 归一化 → epilogue

总结：ch14 的 WS 是「把搬运从计算中分离」，本章的 WS 是「在搬运分离之后，再用第二个 consumer 把**计算与计算**重叠」。前者解决 memory latency，后者针对 softmax 与 GEMM 的 overlap。FA3 论文（§3.2/§3.3）把后者的完整形

态称为 *pingpong scheduling*：借助 WGMMA 的异步性，同一 WG 内部 GEMM 与 softmax 也能重叠（intra-warpgroup overlapping），两个 WG 再交替执行 GEMM 以填满 Tensor Core。本 kernel 的 consumer 用**同步**的 `mma.sync`，做不到 WG 内重叠；它靠的是两个独立流在硬件调度器上 **统计性交错**（WG1 处于 MMA 段时 WG2 大概率处于 softmax 段），加上 TMA 窗口被完整隐藏。这是「无 WGMMA」约束下的近似；第 二十六 章的 CuTe 版会在 sm_90a 上用 `wgmma` 把这层差距补上。

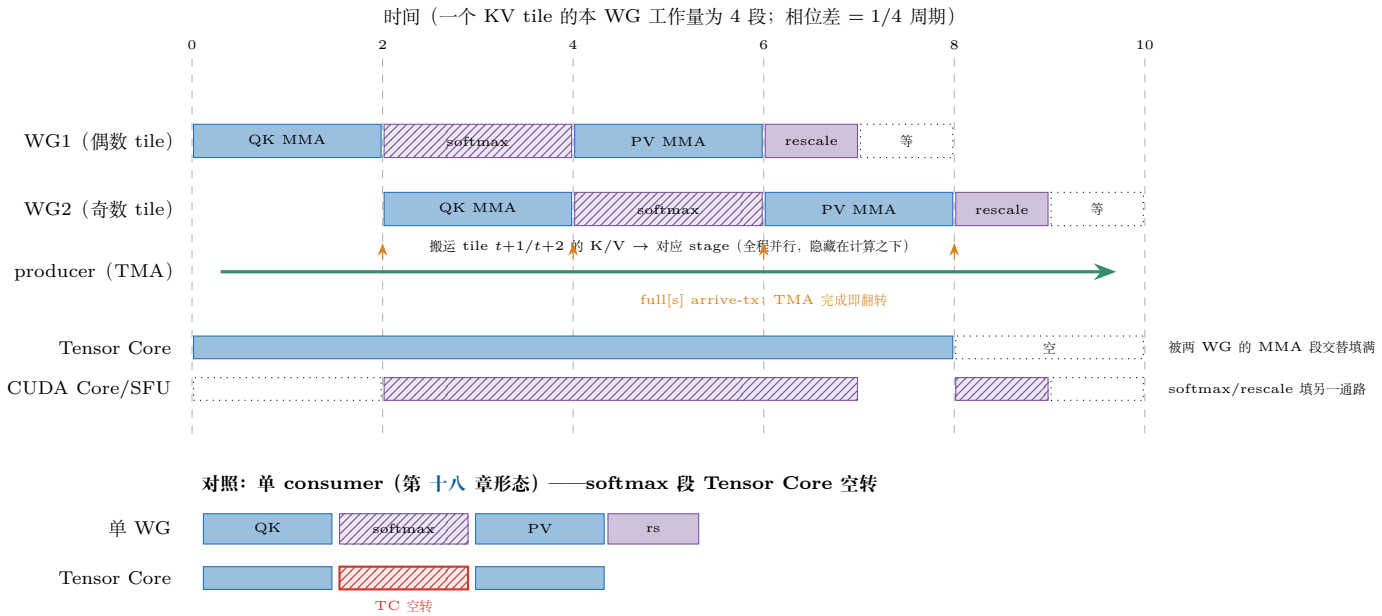


图 19.4: 双 consumer 的相位交错时序（18.4.5 节统计性交错的图形化）。上：WG1/WG2 各按「QK MMA → softmax → PV MMA → rescale」四段推进自己的 KV tile 子序列，两 WG 相位差 1/4 周期——WG1 处于 MMA 段时 WG2 大概率处于 softmax/rescale 段；producer 的 TMA 全程并行，完成即 arrive-tx 翻转 `full[s]`。中：Tensor Core 与 CUDA Core/SFU 两条资源条被两路计算交替填满（同步 `mma.sync` 做不到 WG 内重叠，靠两路独立流在硬件调度器上统计性交错）。下：单 consumer（第 十八 章形态）的对照——softmax 段 Tensor Core 只能空转，这正是第二个 consumer 要填补的空档。

19.5 图示

19.6 性能分析

本书复测口径（2026-09-11，RTX PRO 5000 sm_120a，CUDA 13.2，与第 十八 章同机同批；`book/tests/ch18_fa3.cu bench`：B=1、H=32、warmup=5、iters=20 取均值，FLOPS = $4BHN^2D$）：

配置	D=64, N=4096	D=64, N=16384	D=128, N=4096 / N=16384
Sk=1, F16Acc	**159.8**	—	**190.4** / 185.2
Sk=2, F16Acc	155.1	159.5	n/a（112 KB 超限）
Sk=3, F16Acc	154.3	158.6	—
Sk=4, F16Acc	155.4	—	—
Sk=1, F32Acc	**170.0**	—	194.2 / 189.7
Sk=2, F32Acc	153.7	—	—

三点读数。其一，**双 consumer 结构在两种 head dim 上都拿到了与第 十八 章同档或更好的吞吐**：D=128 单 K 级流水（Sk= 1）即达 190.4 T（F16Acc）/ 194.2 T（F32Acc），而同卡同刻 FA2 TMA WS（B_r=128，Sk= 2）为 172.5 / 191.1 T。FA3 式的 B_r=64 小 tile 尽管算术强度只有 $AI = B_r = 64$ FLOP/B（第 十八 章「$AI = B_r$」结论的一半，因为 Q tile 行数减半），却靠「两条独立流交错」用多余的线程预算换回了吞吐；D=64 上差距更明显（159.8 vs 141.8 T，+13%）。其二，**Sk 加深在本章不再有收益**（D=64：Sk= 1 159.8 > Sk= 2 155.1 > Sk= 3 154.3）：per-WG bank 把

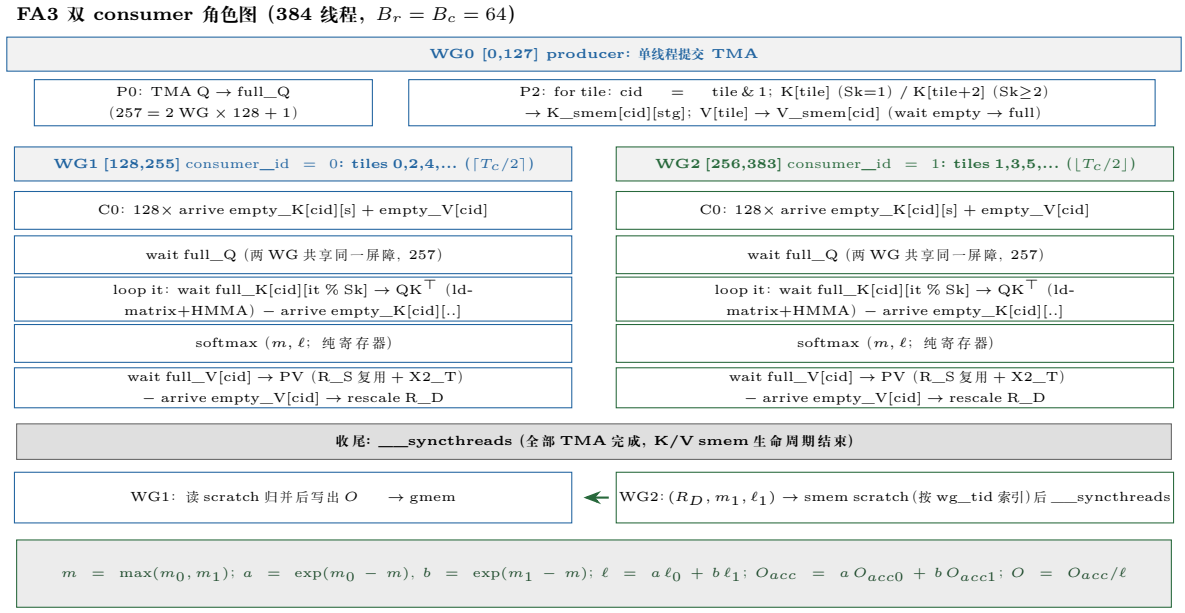


图 19.5：FA3 双 consumer 的角色图：WG0 单线程提交 TMA，并按 cid = tile & 1 把 K/V 投递到两个独立 bank；WG1/WG2 各自遍历一半 KV tile、维护独立 partial 与独立流水；收尾在 smem scratch 上做一次稳定归并后由 WG1 写出。

smem 的 O(Sk) 项翻倍（式 19.6），而装载早已不在关键路径上，这与第 十八 章「Sk=2 即饱和」的结论同源，只是本章的饱和点更低。其三，**F32Acc 依旧不慢于 F16Acc**（170.0 vs 159.8；194.2 vs 190.4），与第 十八 章观察一致：F16Acc 路径要在 f32 计算与 f16 写回之间反复 `__float22half2_rn`，而 attention 的 softmax 指令占比高，这笔转换开销对速度的影响比累加精度更大。

这组数字**不是受控 A/B**：两个 kernel 的 B_r 不同（128 vs 64）、Sk 不同，网格形状（N/B_r 与 B_c 的块数）也不同，读作「两条路线在同一张卡上的量级对照」即可，不宜当作「FA3 比 FA2 快 x%」的结论；真正受控的对照需要固定 tile 与流水深度，留给附录 B 的复测表。

另一条数据线来自 `leetcuda_bench_sm120a.bin --bench --bench-all` 的 FA 基准（B=1、H=32、N=8192、D=128，dense，同机 RTX PRO 5000 sm_120、110 SM、CUDA 13.2），以同轮 cuDNN 为基线（表 19.1）：F16Acc 对照 cuDNN f16（231.1），F32Acc 对照 cuDNN f32（230.2）。

表 19.1：FA3 TMA MMA WS（2 Consumer WG）实测（RTX PRO 5000，B=1 H=32 N=8192 D=128）

Kernel	Max Err	TFLOPS/cuDNN（加速比）
FA3 TMA MMA WS (2 Consumer WG) (Sk=1, Sv=1, F16Acc)	1.068e-04	206.2/231.1 (0.89x)
FA3 TMA MMA WS (2 Consumer WG) (Sk=1, Sv=1, F32Acc)	1.526e-05	205.7/230.2 (0.89x)

两点读数。其一，**双 consumer（2 Consumer WG）把同规模吞吐从单 consumer 的 188–204 TFLOPS（第 十八 章）进一步抬到 206 TFLOPS 量级**（206.2 / 205.7），「两条独立流交错填满发射槽」的结构收益在更大规模下依然成立。其二，**F16Acc 与 F32Acc 速度基本持平**（206.2 vs 205.7，差别 < 0.5%，落在 run 间波动内），而 **F32Acc 精度更优**：最大误差 1.526e-05 比 F16Acc 的 1.068e-04 小约一个量级，本规模下 F32Acc 是更合适的默认选择。两档对 cuDNN 均为 0.89x，说明剩余差距在 kernel 结构而非累加精度。

19.7　实践经验与考据说明

实践经验 19.1（F1 · 注释与代码相反）：L1606「Per-tile load order: V first, then K」该行位于 producer 段注释块（L1601–1612），写作「每 tile 先 V 后 K」，但代码是**先 K 后 V**：`Step 1` 处理 K（Sk=1 就地装载 / Sk≥ 2 预取 tile+2，L1650–1677），`Step 2` 才发 V（L1679–1686）。同一文件的另一处注释 L1643–1646 明确写着「K 优先于 V……先发 K 让 consumer 尽早开始 QK，V 的 TMA 在 K 之后发出」，与代码一致。两处注释自相矛盾，L1606 是早期版本（V 先行）的残留。结论：以代码与 L1642–1646 为准，L1606 应读作「K first, then V」（结论以本框注为准）。

实践经验 19.2 (F3 · 论证链补强)：merge scratch 的 alias 安全前提 L2003–2011 列出三条前提，第一条写「producer 已退出 mainloop（所有 TMA 已 `arrive_expect_tx`）」。但「发出并声明事务量」不等于「数据已落盘」，若某笔 TMA 仍 in-flight，其目标写会与 scratch 写竞争。真正排除风险的是第二条（两个 consumer 都已 release 自己的 K/V stage）加上 **full 屏障的语义**：每个 tile 的 K/V 都被某个 WG 在 `full_K/full_V` 上等过，而 tx-count 归零意味着搬运完成。两 WG 走完各自的 tile 序列，即覆盖了全部 T_c 个 tile，故第一处 `__syncthreads` 时**已无 inflight TMA**。另一处需要写明的细节：scratch 尺寸并非只占 Q 区。以 D=128、F16Acc 计，R_D 区 128×8 个 `float4` = 16 KB，m/l 区 128×1 个 `float4` = 2 KB，合计 18 KB 比 Q 区（16 KB）多出 2 KB，会切进 `K[cid=0][0]` 的头部（D=64：10 KB vs 8 KB，同样多 2 KB）。结论**仍然安全**（此时 K/V 已无读者、无写者），但「整个 dynamic smem 生命周期结束」这句话要配合上述 TMA 落盘论证才严谨。

实践经验 19.3 (F2 核校通过)： setmaxnreg 语义与池数学 与 PTX ISA §9.7.19.5（本地 ptx-docs）逐条核对：`imm-reg-count` $\in [24, 256]$ 且为 8 的倍数（本 kernel 40/168/80 均满足）；`.dec` 要求当前值 $\geq$ 目标、`.inc` 要求当前值 $\leq$ 目标；`.inc` **阻塞**直到 CTA 池中有足够寄存器（满池即永久挂起）；池是 **per-CTA**。源码的 40/168/168 组合满足式 19.5（$376 \leq 512$），余 17408 个寄存器；D=64 的 40/80/80 余 39936。数字与代码 L1614、L1702–1706 逐字一致。

实践经验 19.4 (F1 · 口径)： L1501 「Sk=2 需 112KB > 101KB optin 上限」 D=128、Sk= 2 的动态 smem 按式 19.6 复算为 114688 B = 112 KB（正确）；但本机实测 optin 上限 `cudaDevAttrMaxSharedMemoryPerBlockOptin` = 101376 B = 99.0 KB（18.10 节测试首行），注释写的「101KB」比实测大 2%，属宽松表述，结论（超限 → 只支持 Sk=1）不受影响。第 十八 章对同一常量记为「~100KB 近似正确」，口径已统一为「实测 101376 B」。

实践经验 19.5 (F1/F3 核校通过)： 头注释的协议描述与代码逐条一致 （1）线程布局（L1449–1452）与 L1577–1581 的角色划分一致；（2）奇偶路由 cid = tile & 1（L1459、L1475）与 L1648、L1742 一致；（3）arrive 数：`init(&full_Q, 256 + 1)`（L1584，注释「2 consumer WGs * 128 + 1 producer」）与 per-WG 屏障 `kConsumerThreadsPerWG + 1 = 129`（L1587–1591）与式 19.5 上方的计数表一致；（4）稳定归并公式（L1487–1493）与 L2091–2123 实现一致（含 F16Acc 提升到 float2 计算的写法）；（5）T_c=1 退化结论（L1495–1496）与 L2035 注释、以及 18.3.2 节的推导一致。

19.8 常见坑

坑一：merge scratch 的安全前提是「无 in-flight TMA」，不是「producer 已发完 TMA」。producer 退出 mainloop 只说明事务**已声明**；保证搬运**已落盘**的是 consumer 侧 `full_K/full_V` 的等待（tx-count 归零）。改动 scratch 布局或提前 merge 点时，必须重新确认这条链成立，否则会出现「读 scratch 读到正在被 TMA 覆盖的数据」这类**间歇性**错误，且只在大 T_c（流水深）时暴露。

坑二：$T_c = 1$ 的退化路径（0/0 NaN）。$N = B_c$ 时 WG2 的 $m_1 = -\infty$、$\ell_1 = 0$，若按「先归一化再合并」的写法会算出 0/0。本章 kernel 用未归一化 O_{acc} 只做乘加，天然安全（18.3.2 节）；换成「归一化后相加」的实现必须显式特判。测试里的 `N=64`（T_c=1）例专门覆盖这条路径。

坑三：`local_tile/local_iter` 必须严格对齐。producer 在 L1663 重算本 WG 视角的序号 $(t - \text{cid})/2$ 以确定 `stg_next`，consumer 在 L1743 用 `local_iter` 确定 `k_stg`。两处是同一序列的两份实现，改动任一处（例如为支持「前半/后半区」拆分而改划分）都会让 producer 把数据写进 consumer 当前正在读的槽。后果是**静默数值错误**，而非挂死。

坑四：两套屏障数字不能混用。`full_Q` 是 $2\times 128+1 = 257$（两 WG 共用），per-WG 的 K/V 屏障是 $128+1 = 129$。把两套数字写混（例如把 per-WG 屏障也 `init` 成 257）只会出现两种结果：**多算则首轮挂死**，**少算则读到半新半旧的数据**（静默错误）。C0 预热也必须**逐 WG** 进行（每张 `empty_K[cid][s]/empty_V[cid]` 各 128 次 arrive），漏一个 bank 就死一个 WG。

坑五：`setmaxnreg` 满池死锁。式 19.5 的余量是硬约束：producer 提预算（如 40 → 64）时 consumer 必须同步下调（168 → 152），保持 $r_p + 2r_c \leq 496$（即 ≥ 2048 寄存器 slack）。满池的表现是**无任何报错的挂起**：consumer 阻塞在 `setmaxnreg.inc`，producer 阻塞在 barrier；排查口诀见第 十五 章（唯一变量是 `setmaxnreg` 数字时，先查池数学）。

坑六：D=128 只支持 Sk= 1。112 KB 超出 99 KB 的 optin 上限，测试辅助工具必须按 smem 可行性 SKIP（本章测试对超限组合打印 `SKIP(ch18)` 并非跳过失败）。想给 D=128 加深流水，只能减小 B_c 或让两个 WG 共享一套更大 bank。前者改变 tile 几何，后者退回第 十八 章的结构。

坑七：`setmaxnreg` 是 TMA 地址空间敏感的（cluster 形式丢，C7506）。TMA dst 为 `shared::cluster` 时（`cp.async.bulk.tensor`），ptxas 视其为隐式 extern-call 边界，丢弃同 kernel 的 `setmaxnreg`；dst 为 `shared::cta` 时 sm_120a/sm_120f 均保留（第 十五 章坑二取证，与 arch 后缀无关）。所以默认构建跑出的性能里**不包含**寄存器再分

配的贡献；用 build.sh 的 sm_120f 目标重建、或在 sm_90a 上复测才是式 19.5 的真实检验。

坑八：seqlen 对齐。v1 要求 $N\%B_r = 0$ 且 $N\%B_c = 0$，本章 $B_r = B_c = 64$，比第 十八 章的 128 宽松一档（无尾 tile 处理，无 causal/varlen/GQA）。变长序列要么 pad，要么走分桶。

19.9　面试要点速查

Q1：FA3 相对 FA2 的结构性差异是什么？

A：FA2 解决「非 matmul FLOPs、并行度、warp 划分」；FA3 解决「异步 overlap」：producer 用 TMA 异步搬运，两个 consumer warpgroup 让 softmax（CUDA Core/SFU）与 GEMM（Tensor Core）并行。本仓库的实现是 1 producer + 2 consumer 的 384 线程结构 + KV 奇偶拆分 + 终态归并（flash_attn.cuh L1441–2192）。

Q2：为什么按「奇偶」拆 tile，而不是让两个 WG 各管前后半区？

A：producer 必须按**全局顺序** 0,1,2,…连续投递 TMA，而 bank 由 cid = tile & 1 静态路由（L1459、L1475）。奇偶拆分下两 WG 当前消费的 tile 永远相邻（t 与 $t+1$），producer 的单一顺序流正好按 1:1 供给两个 bank；若改成前后半区，过渡期两个 WG 的需求会同时压向同一个 cid，另一个 bank 空转，且远端 tile 的数据可能还没被搬运到（注释 L1479–1480）。

Q3：per-WG 独立 stage 带来什么？

A：共享方案里 Sk 个 slot 由两个 WG 竞争使用（每个 WG 实际深度 ≈ Sk/2）；独立 bank 后每个 WG 真正拥有 Sk 级深度，且两个 WG 不会使用同一 slot/屏障（L1456–1462），屏障到达数从 257 降到 129（18.3.3 节的计数表）。

Q4：merge 为什么要用 max 作 pivot？

A：式 19.3 用全局 $m = \max(m_0, m_1)$，使 $\alpha, \beta \le 1$，两项都是「向小指数缩放」，不会上溢；若以 m_0 或 m_1 中较小者作基准，另一个指数就会 > 1，极端情况下 e^{Δ} 溢出。第 十六 章 online rescale 的 $e^{m^{\text{old}} - m^{\text{new}}}$ 是同一条数值原则。

Q5：T_c 为奇数或为 1 时 Merge 会怎样？

A：无需特判。奇数 T_c 只是两半大小不等（并集完整、交集为空即成立）；T_c=1 时 WG2 的 $m_1 = -\infty, \ell_1 = 0, O_1 = 0$，$\beta = 0$，退化回单 WG 结果（18.3.2 节）。前提是全程用未归一化的 O_{acc}。

Q6：为什么需要 `setmaxnreg`？

A：consumer 需要持有 $S/O/D$ fragment 与 Q/K/V 操作数，producer 只发 TMA。把 producer 降到 40、consumer 升到 168（D=128），等于把同一份 64K 寄存器文件按角色重切，避免 consumer 溢出到 local memory。约束是 $r_p + 2r_c \le 496$（留 ≥ 2048 slack）；`.inc` 是阻塞语义，满池即死锁（第 十五 章事故复盘）。

Q7：本章的双 consumer 与 FA3 论文的 pingpong 是一回事吗？

A：不是。论文的 pingpong 依赖 **WGMMA 的异步性**，同一 WG 内 GEMM 与 softmax 可重叠（intra-warpgroup overlapping），两个 WG 再交替发 GEMM。本章 consumer 用同步的 `mma.sync`，没有 WG 内重叠，靠的是**两条独立流在硬件调度器上统计性交错** + TMA 窗口隐藏。第 二十六 章的 CuTe 版在 sm_90a 上用 `wgmma` 才是论文形态。

Q8：为什么收尾要一次全 CTA 的 `__syncthreads`？

A：merge 需要 WG2 的 partial 写完后再读（第二次 barrier），并把「K/V smem 的生命周期结束」变成全 CTA 的共识（第一次 barrier，可在其上复用 scratch）；归并完成后 `consumer_id`= 0 独占写 gmem，无需第三次同步（第二次 barrier 只保证 WG1 看到 WG2 的写入）。

19.10　最小测试与动手实验

测试文件：`book/tests/ch18_fa3.cu`（逻辑抽取自 bench/flash_attn.cu `test_flash_attn_3_tma_ws_impl`@L1085；参考实现由 cuDNN SDPA 换成 CPU fp64，softmax 全程 double 累加）。覆盖 14 例：D=64（Sk $\in \{1,2,3\}$，$N \in \{64, 128, 512\}$）与 D=128（Sk=1，$N \in \{128, 512\}$），每例 F16Acc/F32Acc 双档，其中 N=64 专门覆盖 T_c=1 的 merge 退化路径，N=128 是两 WG 各一个 tile 的最小非退化形态，N=512（T_c=8）让每个 WG 完成 4 个 tile 并触发 Sk≥2 的 stage 回绕。判定用容差表：F16Acc 5×10^{-2}、F32Acc 10^{-3}。工程细节与第 十八 章的测试一致：`cuTensorMapEncodeTiled` 经 `dlopen` 解析（build_tests.sh 不带 `-lcuda`）；include 顺序先引 hgemm.cuh（`swizzle_fa` 依赖它的 `swizzle<64>`）。

在 `book/tests/` 目录下编译运行（6 号卡；建议 arch 用 `sm_120f`，本章测试不含 setmaxnreg 路径，sm_120a 亦可跑，但 sm_120f 与 leetcuda bench 实验构建保持同一目标口径，见本节坑七与第 十五 章的考据）：

```
./build_tests.sh --arch sm_120f --ch ch18
```

```
max shared memory per block (optin): 101376 B (99.0 KB)
PASS fa3_2wg D=64 N=64 Sk=1 Tc=1 F16Acc: max_abs_err=2.388e-04 (tol=5.0e-02)
PASS fa3_2wg D=64 N=64 Sk=1 Tc=1 F32Acc: max_abs_err=1.368e-04 (tol=1.0e-03)
PASS fa3_2wg D=64 N=512 Sk=3 Tc=8 F32Acc: max_abs_err=4.646e-05 (tol=1.0e-03)
PASS fa3_2wg D=128 N=512 Sk=1 Tc=8 F16Acc: max_abs_err=1.194e-04 (tol=5.0e-02)
ALL OK
```

以上为照录（截取头尾），14 例全 PASS：F16Acc ∼1.1–2.4e-04、F32Acc ∼4.5e-05–1.4e-04。

bench 子命令输出 18.6 节的复测表（`./.build/ch18_fa3 bench`）。动手实验建议：(1) 把 `N=64` 例改成 N=192（T_c=3，两 WG 分别 2/1 个 tile），验证 18.3.2 节「不对称划分无需特判」；(2) 把 producer 的 prefetch 目标从 $t+2$ 改成 $t+1$（Sk=2 时），观察数值立刻出错（坑三），这是两处 `local_*` 索引错位的直接演示；(3) 把 consumer 的 `empty_K` early release 上移到 HMMA 之前，观察挂死或读脏数据（第 十八 章坑五的相同问题）；(4) 给测试补一行 D=128、Sk=2 的实例化，确认辅助工具打出 `SKIP(ch18): smem 114688 B > optin 101376 B`（坑六）。

延伸阅读

- J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, T. Dao. 《FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision》
  https://arxiv.org/abs/2407.08608
  (pingpong scheduling、warp specialization 与 FP8 的原始出处；本章 18.4.5 节与其对照，第 十六 章 15.3.7 节有全代际梳理)；

- @DefTruth《图解: 从 Online-Softmax 到 FlashAttention V1/V2/V3》
  https://zhuanlan.zhihu.com/p/668888063
  (split-KV 与 online rescale 的图示化推导，式 19.3 的另一路直观解释)；

- @ 杨远航《Flash Attention 3 深度解析》
  https://zhuanlan.zhihu.com/p/17533058076
  (Hopper 上 producer/consumer 的微观时序、pingpong 与 intra-warpgroup overlapping 三层次对照；18.4.5 节「同步 `mma.sync` 做不到 WG 内重叠」的结论正是相对该文方案而言)；

- NVIDIA PTX ISA §9.7.19.5（`setmaxnreg`）——池语义与 `.inc` 阻塞行为的核验基准（本地镜像：cuda-cpp-kernel skill 的 ptx-docs）；式 19.5 与 18.8 节坑五的规范来源。

第二十章　FFPA Split-D：大 head_dim 分块注意力

20.1　本章导读

第 十七–十九 章的三套 FA 实现共享同一个隐含前提：head dim $D \le 128$。本章的 `ffpa_split_d_cute` 与 `ffpa_attn_tma_mma_ws_split_d_cute`（ffpa_attn.cuh L80 与 L380）把这个前提拆掉。它们支持任意 $D = 64k$，方法是 **把 D 方向切成 64 宽的小块（chunk），让 $QK^\top$ 与 PV 两次 GEMM 都沿 chunk 流式推进**。这套结构叫 **Split-D**，也是生产库 `ffpa-attn` 在大 head dim 场景下的主力形态（另见第 二十七 章）。

本章要回答四个问题。其一，为什么 $D > 128$ 不能照搬 FA2：我们把它量化成 **三个瓶颈**：shared memory 容量、O 累加器寄存器数、ldmatrix/swizzle 的几何粒度（19.2 节）。其二，Split-D 在数学上做了什么：QK 的 chunk 求和是**精确**归约，PV 的 chunk 输出各占一部分**列**，两者都不需要 LSE 式的重标定；要小心的是**跨 chunk 广播同一个在线 rescale 因子**（19.3 节给出不变量及其归纳证明）。其三，两版实现如何驱动 chunk 流水：cp.async 版用 `cp.async` 的 group 计数，TMA+WS 版用 mbarrier 的 (stage, phase) 计数，两者对 Q、K、V 的 chunk 池是同一套布局（19.4 节）。其四，Split-D 的代价是什么：O 累加器每线程 $D/2$ 个 f32，$D = 512$ 时是 256 个，已经越过 255 的硬上限（19.4.6 节）。

源码区间：`ffpa_attn.cuh` L1–641。L1–28 是头注释（动机、优化点、性能口径），L31–79 是 `FFPAAttnSplitDCuTeTraits`（双 TiledMma 与 smem layout atom），L80–375 是 cp.async 版 kernel，L377–641 是 TMA+WS 版 kernel。本章的每个代码片段都用 `linerange` 按源码绝对行号引用，行号与源文件一致。

编译宏与架构限定：两个 kernel 都在 `NOTES_V2_ENABLE_CUTE` 下；TMA+WS 版额外要求 `NOTES_V2_ENABLE_TMA_MMA_WS`。cp.async 版基于 `SM80_CP_ASYNC_CACHEGLOBAL` 与 `mma.sync`，sm_80 及以上都能编译；TMA 版需要 `cp.async.bulk.tensor`，即 sm_90a 或 sm_120a。本机为 sm_120a（RTX PRO 5000），本章测试与 CUDA 编译均在此架构完成。

前置与后续：第 十六 章（attention 数学、online softmax 与 $m/\ell/O$ 递推）、第 十七 章（`mma.sync` fragment 与 `ldmatrix`）、第 十八 章（TMA + mbarrier 的 257 协议）、第 十九 章（双 consumer 与 merge）。CuTe 侧的前置是第 二十一–二十四 章：`TiledMma/partition_fragment_*` 的语义在第 二十三 章会以本章的 traits 为样本逐字段拆开。第 二十七 章会用同一份代码做 CuTe 视角的复访。

20.2　问题与动机

先把「不行」证出来。假设我们不切 D：Q、K、V 的 tile 都是 $[B_r, D] = [64, D]$ 的半精度矩阵，一块占 $64 \cdot D \cdot 2$ 字节。表 20.2 把它与 SM120 的 shared memory 上限（本机 `cudaDevAttrMaxSharedMemoryPerBlockOptin` = 101376 B = 99.0 KB，第 十八/十九 章同机实测）摆在一起：

D	单个 $[64, D]$ tile	$Q{+}K{+}V$ 各 1 份	O 累加器（f32/线程）	结论
128	16 KB	48 KB	64	可行（第 十八 章的形态）
192	24 KB	72 KB	96	可行但只剩单份，无流水
256	32 KB	96 KB	128	smem 已达上限
512	**64 KB**	**192 KB**	**256**	双向越界

O 累加器一列的口径见 19.4.6 节：O 的碎片是 $[B_r, D]$ 的 f32 累加器，按 128 个 consumer 线程均分，每线程 $B_r D/128 = D/2$ 个寄存器。$D = 512$ 时单是累加器就要 256 个 f32 寄存器，而每线程寄存器上限是 255（NVCC 分配限制；setmaxnreg 再分配亦不破此限，见 19.4.5 节）。**S/P 碎片与 Q/K/V 操作数尚未计入，寄存器用量已经超出上限**。

于是「大 head dim」的障碍是三条互相独立的硬约束，任何一条都足以否决「整块 D 常驻」的方案：

1. **smem 容量瓶颈**：$[B_r, D]$ 与 $[B_c, D]$ 的整块 tile 随 D 线性膨胀（式 20.1），$D \geq 256$ 时连「Q/K/V 各一份」都放不下，更不用说多级流水要的多份。
2. **寄存器瓶颈**：O 累加器的总量由输出 tile 的形状 $[B_r, D]$ 决定，与怎么切 chunk 无关（式 20.10）。$D = 512$ 时它是 256 个 f32，必然溢出到 local memory。
3. **几何粒度瓶颈**：smem 里的 swizzle atom （`GMMA::Layout_K_SW128_Atom`）行宽 128 B，正好是 64 个 f16；`ldmatrix` 每次装载 8×8 个 f16，其「一行 16 B」的粒度和 128 B 行宽是同一套约定。把 D 切成 64 列，恰好是「一个 swizzle atom 宽 $= 8$ 个 $n8$ 原子宽」，每个 chunk 都能独立套用现成的 `SW128` 布局，不需要设计新的 swizzle。

「不切 D，改小 tile 行数」是 FA2 在 $D = 128$ 时代常用的退路：把 B_r 或 B_c 调小，让 $[B, D]$ 放得进 smem。但第 十八 章已经推出 attention 的算术强度满足 $AI \lesssim B_r$：B_r 缩到 16，算术强度就掉到 16 FLOP/B 量级，离 fp16 Tensor Core 与 HBM 的平衡点（数百 FLOP/B）差一个数量级，kernel 立刻变成 memory-bound。换句话说，**缩小 tile 换来的 smem 节省，代价是算术强度同步下降；D 越大，tile 需要缩得越小，这个代价也越大**。

$$\mathrm{smem}_{[B,D]\ \text{整块}} = 3 \cdot B \cdot D \cdot 2\ \mathrm{B}, \qquad \mathrm{smem}_{\text{chunk 池}} = (2\,\mathrm{Sk} + \mathrm{Sv}) \cdot 64 \cdot 64 \cdot 2\ \mathrm{B} \tag{20.1}$$

式 20.1 左端是「整块」方案的 smem 需求（Q/K/V 各一份），右端是 Split-D 的 chunk 池：每个 chunk 是 $[64, 64]$ 的 f16，占 8 KB；Sk 级 QK 流水需要 2Sk 个 chunk（每个 stage 一份 Q、一份 K），Sv 级 PV 流水需要 Sv 份 V。$\mathrm{Sk} = \mathrm{Sv} = 2$ 时右端是 48 KB，同一张卡上 $D = 512$ 的左端是 192 KB。「**整块**」**是** $O(D)$，「**chunk 池**」**是** $O(\mathrm{Sk} + \mathrm{Sv})$：**这就是 Split-D 用切分换来的收益**。

Split-D 是什么。把 D 方向按 64 宽切成 $C = D/64$ 个 chunk，让**两次 GEMM 都沿 chunk 方向流水**。

- $\mathrm{QK}^\top$ 的收缩维就是 D。切成 chunk 后，$S = \sum_c Q^{(c)} K^{(c)\top}$ 是一组 $[64, 64]$ 小 GEMM 的**精确累加**——没有 softmax、没有权重，纯粹是点积内部加数的分组（式 20.2）。
- PV 的输出维就是 D。切成 chunk 后，$O = [PV^{(0)}, PV^{(1)}, \ldots]$ 是输出列方向的**直和**——每个 chunk 持有一段独立的 O 累加器（式 20.5）。

与 Split-KV 的区别（先给结论，19.3.5 节展开）：Split-KV 切的是**键的下标集合**，每一片算出一份「部分 softmax 状态」(m_p, ℓ_p, O_p)，合并时必须用 LSE 重新加权（各片的最大值不同，要算 $e^{m_p - m}$）；Split-D 切的是**特征维**，S 的 chunk 求和与 softmax 无关，O 的 chunk 之间共享**同一个** (ℓ, m)，合并只是「先沿列拼起来、再统一除一次 ℓ」。所以 Split-D **没有第二个 max**，也就没有那一次跨片 exp 往返。两者是正交的：生产库的解码路径可以同时做 split-KV（切序列）与 split-D（切特征）。

生产库中的对应形态：`ffpa-attn` 按 D 分三个家族：$D \leq 128$ 走 persist-D（Q 常驻寄存器的 WS 结构），中段走 split-D（非 WS，即本章这版形态），$D \geq 768$ 换成 M4N2（`atom_layout` $(4, 2, 1)$，4 个 M-warp $\times$ 2 个 N-warp（共 8 warp）分摊，O 寄存器降到 $D/4$）。交叉点大约在 $D \approx 768$，与式 20.10 算出的寄存器占用一致（19.4.6 节）。本章的两个 kernel 是这条路线在教学仓库里的最小形态。

20.3 数学原理

20.3.1 记号

沿用第 十六 章的记号：B, H, N, D 为 batch/heads/seqlen/head dim，$B_r = B_c = 64$ 为 Q 与 KV 的 block 行数，$T_c = N/B_c$ 为 KV tile 数，$\gamma = 1/\sqrt{D}$ 为 softmax 缩放。对 Q 的第 i 行与 K 的第 j 行定义 $S_{ij} = \gamma\, q_i^\top k_j$。本章新增记号：$C = D/64$ 为 chunk 数，$c \in [0, C)$ 为 chunk 下标，$t \in [0, 64)$ 为 chunk 内偏移；上标 (c) 表示「只取该 chunk 的 64 个分量」，例如 $q_i^{(c)} \in \mathbb{R}^{64}$、$V^{(c)} \in \mathbb{R}^{B_c \times 64}$ 是 V 的第 c 段列。

20.3.2 QK 的 chunk 分解：精确、无权重

把点积按加数分组即可：

$$S_{ij} = \gamma \sum_{d=0}^{D-1} q_{i,d}\, k_{j,d} = \gamma \sum_{c=0}^{C-1} \sum_{t=0}^{63} q_{i,64c+t}\, k_{j,64c+t} = \sum_{c=0}^{C-1} S_{ij}^{(c)}, \qquad S^{(c)} = \gamma\, Q^{(c)} K^{(c)\top}. \tag{20.2}$$

式 20.2 里没有任何近似：它只是把同一个和式重新括起来。工程上的含义是：S 的累加器与 D 无关，永远是 $[B_r, B_c] = [64, 64]$（每线程 32 个 f32），chunk 循环只是往同一个累加器里做 C 次 `mma.sync`。浮点上唯一的差别是**求和顺序**（chunk 内先累、再跨 chunk 累，而不是一条长链），因此结果与「整块算」不是 bitwise 相同，但在 fp32 累加下差异远小于 fp16 输入本身的舍入。本章测试就是按 F16Acc 档（5×10^{-2}）判定的，实测最大绝对误差 6.6×10^{-5}（19.10 节）。

20.3.3　分块 online softmax 的两条不变量

第 十六 章把 FlashAttention 的 KV 循环写成三条递推。设第 t 个 KV tile 的分数矩阵为 $S^{(t)} \in \mathbb{R}^{B_r \times B_c}$，$m_{t-1}, \ell_{t-1}, O_{t-1}$ 为进入该 tile 时的运行状态（m 是行最大值、ℓ 是未归一的指数和、O 是未归一的加权和），则

$$\begin{aligned} m_t &= \max\big(m_{t-1},\ \mathrm{rowmax}(S^{(t)})\big), \qquad & \rho_t &= e^{\,m_{t-1}-m_t} \le 1, \\ \ell_t &= \rho_t\,\ell_{t-1} + \mathrm{rowsum}\big(e^{\,S^{(t)}-m_t}\big), \qquad & O_t &= \rho_t\,O_{t-1} + e^{\,S^{(t)}-m_t} V^{(t)}. \end{aligned} \tag{20.3}$$

源码把 ρ_t 记作 `row_scale`（L286 计算、L327–335 使用），把 e^{S-m_t} 原地写回 S 的碎片（L290–292），m/ℓ 只保留每线程负责的两行（`kORows`$=2$，见 19.4.1 节）。式 20.3 的正确性在第 十六 章已证：对任意 t 都有 $O_t = \sum_{j<tB_c} e^{S_{ij}-m_t} v_j$，$\ell_t = \sum_{j<tB_c} e^{S_{ij}-m_t}$，最终 $O = O_{T_c}/\ell_{T_c}$。

20.3.4　把 O 拆成 C 个 chunk 分量

现在是本章的核心：PV 的输出维是 D，而 softmax 的权重 $p_j = e^{S_{ij}-m}$ 是**行内**的量，与输出列无关。把 V 按列切成 C 段，乘积天然分列：

$$O = PV = \big[\,PV^{(0)} \mid PV^{(1)} \mid \cdots \mid PV^{(C-1)}\,\big]. \tag{20.4}$$

于是**每个 chunk 各持一个独立的 O 累加器** $O^{(c)}$，其递推与式 20.3 同形，唯一区别是 $V^{(t)}$ 换成 $V^{(t,c)}$：

$$O_t^{(c)} = \rho_t\,O_{t-1}^{(c)} + e^{\,S^{(t)}-m_t} V^{(t,c)}, \qquad c = 0, \ldots, C-1. \tag{20.5}$$

注意式 20.5 里**同一个 ρ_t 出现在所有 C 个递推式里**。源码把 rescale 写在 V-chunk 循环内部、用的却是循环外算好的 `row_scale[row]`（L327–335）：rescaling 必须广播，不能由各 chunk 独立处理。下面这个不变量说明为什么。

定理 20.1 (chunk 累加器的不变量)**.** *若 $m_0 = -\infty$，$\ell_0 = 0$，$O_0^{(c)} = 0$，且按式 20.5 更新，则对一切 t 与一切 c：*

$$O_t^{(c)} = \sum_{j<tB_c} e^{\,S_{ij}-m_t}\, v_j^{(c)} \qquad \textit{(逐行，对每个输出列分量都成立)}. \tag{20.6}$$

证明. 对 t 归纳。$t = 0$ 时两边都是 0。设 $t-1$ 成立。先看 rescale 项：由 $e^{S_{ij}-m_{t-1}} \cdot e^{m_{t-1}-m_t} = e^{S_{ij}-m_t}$ 得

$$\rho_t\,O_{t-1}^{(c)} = \sum_{j<(t-1)B_c} e^{\,S_{ij}-m_t}\, v_j^{(c)},$$

即把旧部分的指数基准从 m_{t-1} 换成 m_t；这步成立的代价是 $\rho_t \le 1$（因为 $m_t \ge m_{t-1}$），两项都只向小指数方向缩放，永不溢出。再加上新 tile 的 $\sum_{j\in[tB_c,(t+1)B_c)} e^{S_{ij}-m_t} v_j^{(c)}$ 即得式 20.6。证毕。 □

三个推论。其一，终值仍是「先合并、后归一化」：

$$O_{i,\,64c+d} = \frac{\big(\sum_{c'} O_{T_c}^{(c')}\big)_{i,\,64c+d}}{\ell_{T_c}} = \frac{O_{T_c}^{(c)}\big|_{i,d}}{\ell_{T_c}}, \tag{20.7}$$

即累加器全程未归一化、epilogue 只除一次 ℓ（L364）。这与第 十六/十九 章的数值约定一致：避免中间量的尺度漂移，也避免 C 次额外的除法。其二，若某个 chunk 的 $O^{(c)}$ 忘了 rescale，它对应的就不是全局 max 基准，误差会随 KV tile 数 T_c 累积（每次漏乘一个 $\rho \le 1$，$O^{(c)}$ 偏大），这是本章最容易犯的 **静默错误**（19.8 节坑三）。其三，ℓ 与 m 是**每行一份**、与 chunk 无关：C 个累加器共享两个标量数组，这也是 Split-D 的「零额外状态」。

20.3.5 与 Split-KV 的合并方式对照

第 十九 章已经见过 split-KV 的合并：两片各自维护 (m_p, ℓ_p, O_p)，合并时以 max 为 pivot 算权重 $\alpha = e^{m_0 - m}$、$\beta = e^{m_1 - m}$，再 $\ell = \alpha\ell_0 + \beta\ell_1$、$O = \alpha O_0 + \beta O_1$。把两者摆在一起：

	Split-KV（第 十九 章）	**Split-D**（本章）
切分对象	键下标集合 $\{0..N\}$	特征维 $[0, D)$
每片持有什么	部分 softmax 状态 (m_p, ℓ_p, O_p)	S 的部分和 / O 的部分列
是否需要第二个 max	需要（各片 max 不同）	**不需要**（共享同一个 m）
合并动作	LSE 重标定：α, β 加权求和	沿 D 求和（QK）/ 拼接（PV）
额外 exp 次数	O(片数)	**0**
典型场景	decode、长序列并行	大 head dim（本章）

split-KV 的「合并」是数值问题（要把两份不同基准的 partial 归一到同一基准），split-D 的「合并」是索引问题（不同 chunk 落在不同的输出列上，互不重叠）。这也是大 head dim 能保持 FA2 级数值性质的原因。

20.3.6 chunk 数、stage 数与相位

最后把流水槽位的计数规则定下来。TMA 版把 QK chunk 的流水槽编号写成一个**扁平计数器**（L443–445、L539–542）：

$$\text{idx} = t \cdot C + c, \qquad \text{stage} = \text{idx} \bmod \text{Sk}, \qquad \text{phase} = \lfloor \text{idx}/\text{Sk} \rfloor \bmod 2. \tag{20.8}$$

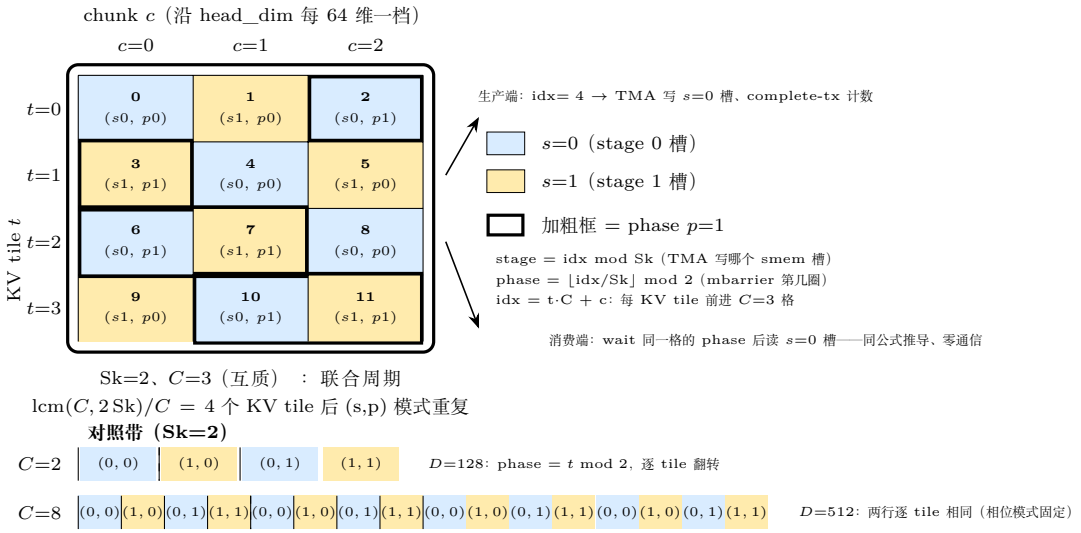


图 20.1: idx $= t \cdot C + c \to$ (stage, phase) 的相位展开（式 20.8 的图形化，Sk=2）。主网格取 C=3（D=192，与 Sk 互质的最易错档）：行方向为 KV tile t、列方向为 chunk c，cell 填 idx 与其 (stage, phase)——灰度编码 stage 槽位、加粗框标记 phase = 1；4 个 KV tile 后 (s,p) 模式才重复（周期 = lcm$(C, 2\,\text{Sk})/C$）。右：生产/消费两端用**同一个公式**各自推导（写/读同一 stage 槽、以同一 phase 为准），不需要额外通信。下：C=2（逐 tile 翻转）与 C=8（逐 tile 相同）两条对照带，覆盖 19.10 节测试矩阵的三种形态。

生产者与消费者用**同一个公式**推导 (stage, phase)，因此不需要任何额外通信就能保持一致（19.4.4 节细讲）。相位在「stage 回绕」时翻转：stage 随 chunk 计数 idx 的周期是 Sk、phase 的周期是 2 Sk，(stage, phase) 联合周期 2 Sk；idx 每个 KV tile 前进 C，故 stage 与 phase 的联合周期是 lcm$(C, 2\,\text{Sk})/C$ 个 KV tile：

D	$C = D/64$	Sk	相位周期（KV tile 数）	形态
64	1	2	4	退化：每 KV tile 只推 1 个 chunk，流水到不了稳态
128	2	2	2	一个 KV tile 恰好推满一轮 stage，phase $= t \bmod 2$，每 tile 翻转
192	3	2	4	C 与 Sk 互质，相位模式要跨 4 个 KV tile 才重复
512	8	2	1	每 KV tile 走 4 轮，相位模式逐 tile 相同

本章测试特意覆盖了 $C=1$、$C=2$、$C=3$（与 Sk=2 互质）、$C=8$ 四种情形（19.10 节）。其中 $C=3$ 的相位模式要跨 4 个 KV tile 才重复，也是最容易暴露 (stage, phase) 推导错误的一档。

20.4　设计与实现

20.4.1　traits：双 TiledMma 与 SW128 chunk 布局

先看 `FFPAAttnSplitDCuTeTraits`（L31–79）。它只做四件事：定 chunk 的 smem 布局、定两个 TiledMma、定 S2R 的 copy atom、把「D 必须被 64 整除」写成 `static_assert`。

ffpa_attn.cuh L36-61（traits：SW128 chunk 布局、双 TiledMma、S2R copy atoms）

```
static_assert(kHeadDim % 64 == 0, "Split-D requires head-dim multiple of 64");
static_assert(TILE_M == 64 && TILE_N == 64, "Current impl supports 64x64 only");

using Element = cutlass::half_t;
using SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>;
using SmemLayoutQ = decltype(tile_to_shape(
    SmemLayoutAtom{}, Shape<Int<TILE_M>, _64>{}));
using SmemLayoutKV = decltype(tile_to_shape(
    SmemLayoutAtom{}, Shape<Int<TILE_N>, _64>{}));
using SmemLayoutVt = decltype(composition(
    SmemLayoutKV{}, make_layout(Shape<_64, Int<TILE_N>>{}, GenRowMajor{})));

using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;

// QK: Tile<64,64,16> → EURepeat<1,8,1>, 一次覆盖 S[64,64]
using TiledMmaQK = decltype(make_tiled_mma(
    MmaAtom{}, Layout<Shape<_4, _1, _1>>{},
    Tile<Int<TILE_M>, Int<TILE_N>, _16>{}));

// PV: Tile<64,16,16> → EURepeat<1,2,1>, 控制 acc_O 寄存器
using TiledMmaPV = decltype(make_tiled_mma(
    MmaAtom{}, Layout<Shape<_4, _1, _1>>{},
    Tile<Int<TILE_M>, _16, _16>{}));

using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
```

逐条读：

- `SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>`：128 B 行宽的 swizzle 原子，一行正好 64 个 f16。`tile_to_shape` 把它铺成 $[64, 64]$——**chunk 的宽度（64）并非任意选定，它就是这个 atom 的宽度**。第 十三/二十四 章讲过 SW128 的 XOR 置换；这里直接复用 cutlass 的 atom，不需要重推。
- `SmemLayoutVt`：用 `composition` 造 V 的**转置视图**（行主序坐标重解释），配合 `SM75_U16x8_LDSM_T`（`ldmatrix.trans`）在搬运时完成转置，**不必在 smem 里另存一份** $V^\top$。省下的是每 stage 8 KB。
- `MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>`：$m16n8k16$、f16 输入、**f32 累加**。S 与 O 的累加器都是 f32，这与第 十六 章「softmax 精度靠 f32 累加」的结论一致。
- `TiledMmaQK`：`Tile<64,64,16>` + `Layout<Shape<_4,_1,_1>>`，4 个 warp 排在 M 上（$\mathrm{M} = 4 \times 16$），N 方向 $64 = 8 \times n8$，K 方向一个 $k16$——即源码注释里的 `EURepeat<1,8,1>`：每个 warp 每个 k-step 发 8 条 `mma.sync`，**一次 TiledMMA 覆盖整块** $S[64, 64]$，于是「N-tile 循环」可以整个省掉。
- `TiledMmaPV`：`Tile<64,16,16>`，N 方向只覆盖 $2 \times n8$（`EURepeat<1,2,1>`）。64 列的输出被摊成 4 个 tile repeat，由外层 V-chunk 循环显式驱动。

这里要补一句**容易误读的话**：头注释 L13 写「PV 保持小 tile 控制 acc_O 寄存器」。严格说，O 累加器的**总量**由输出 tile 形状 $[64, 64] \times C$ 决定（式 20.10），与 tile 怎么切无关；`Tile<64,16,16>` 影响的是「一次 TiledMMA 调

用覆盖多大输出块」以及寄存器生存期的切分粒度，让 chunk 边界成为寄存器域的边界（每 chunk 一个独立的 `o_acc_storage[v]` 数组）。表述上的这点含混记在 19.7 节的勘误框，结论不受影响。

20.4.2 smem 池：三段基址与 chunk 偏移

两个 kernel 的 smem 用法完全一致（L100–103 与 L404–407）：一段 `extern __shared__` 动态数组，按「Q 池 → K 池 → V 池」切成三段，每段按 stage 数乘以 chunk 元素数得到基址偏移：

ffpa_attn.cuh L93-103（chunk 常量与 smem 三段划分：基址指针算术，不用 stride-0 的 stage layout）

```
constexpr int kBr = 64;
constexpr int kBc = 64;
constexpr int kDChunk = 64;
constexpr int kDChunks = kHeadDim / kDChunk;
constexpr int kQChunkElements = cosize(SmemLayoutQ{});
constexpr int kKVChunkElements = cosize(SmemLayoutKV{});

extern __shared__ __align__(1024) Element shm[];
Element *q_base = shm;
Element *k_base = q_base + kStagesQK * kQChunkElements;
Element *v_base = k_base + kStagesQK * kKVChunkElements;
```

注意 `kQChunkElements = cosize(SmemLayoutQ{})`。池的大小来自 **layout 自身**的 cosize，而不是手写 64×64。这是 CuTe 工程的一个好习惯：layout 一改，池大小与偏移自动跟着改（第 二十一 章）。注释里特意写了「不使用 stride-0 的 stage-mode layout」，意思是：与其把一个 stage 维并入 tensor 的 layout （产生 stride-0 的伪维），不如直接做指针加法，索引更直观、生成的地址计算也更便宜。

由此得到 smem 预算：每 stage 的 QK 槽位是「Q chunk + K chunk」两块 8 KB，V 槽位是 8 KB，合计

$$\text{smem}_{\text{chunk 池}} = \big(2\,\text{Sk} + \text{Sv}\big) \times 8\ \text{KB} \quad\Longrightarrow\quad \begin{cases} \text{Sk}=\text{Sv}=2: & 48\ \text{KB} \\ \text{Sk}=\text{Sv}=3: & 72\ \text{KB} \\ \text{Sk}=\text{Sv}=4: & 96\ \text{KB} \to \text{接近 } 99\,\text{KB 上限.} \end{cases} \tag{20.9}$$

这也解释了本章测试辅助工具为什么必须做 smem 可行性检查（`SKIP(ch19)` 分支）：D 一涨，C 跟着涨，而 **stage 数是独立旋钮**，两者消耗的是同一份 99 KB 预算。生产库在 sm120 上把 stage 上限限制在 2–3，理由正是「快速 MMA 下同步开销主导 + smem 预算」（见 19.6 节）。

20.4.3 cp.async 版：128 线程全员搬 + 双流水

`ffpa_split_d_cute`（L80–375）是**无 TMA、无 warp specialization** 的形态：128 个线程既搬数据又算，用 `cp.async` 的 group 计数做流水。头注释 L71–77 交代了为什么是 128 线程而不是别的数目：

> 「128 线程：与 TiledMmaQK/PV 的 `Layout<4,1,1>` 一致，每个线程在 G2S 和 S2R/MMA 中有唯一分区，消除 256-thread G2S 与 128-thread MMA 之间的映射不匹配。」

搬运侧的 copy 定义如下（L120–128）：128 bit 的 `cp.async`（一线程一次搬 16 B = 8 个 f16），线程布局 $(16,8)$、值布局 $(1,8)$，于是 128 个线程 × 4 次 = 4096 个元素，恰好铺满一个 $[64,64]$ 的 chunk。

ffpa_attn.cuh L120-128（G2S TiledCopy：128-bit cp.async、$(16,8)$ 线程布局）

```
using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
using g2s_copy_atom = Copy_Atom<Copy_Traits<g2s_copy_op>, Element>;
using G2SCopy = decltype(make_tiled_copy(
    g2s_copy_atom{},
    make_layout(make_shape(Int<16>{}, Int<8>{}),
                make_stride(Int<8>{}, Int<1>{})),
    make_layout(make_shape(Int<1>{}, Int<8>{}))));
G2SCopy g2s_copy;
auto g2s_thr = g2s_copy.get_slice(tid);
```

主循环的四个阶段（L202 起，每轮 KV tile 一遍）：

1. **Phase 0：V 预取**（L204–215）。先把前 Sv − 1 个 V chunk 发出去——V 要到 Phase 3 才用，所以这几组 `cp.async` 可以整段 overlap 在 QK 与 softmax 上。这是「提前发射」而非「提前等待」。
2. **Phase 1：$QK^\top$ 的 chunk 流水**（L217–271）。每个 chunk 一对 (Q, K)，一次 `cp.async_fence()` 提交为一组；`gemm_ss` 内部做流水化的 `ldmatrix` + `mma.sync`，累加到同一块 $S[64, 64]$ 上。
3. **Phase 2：在线 softmax**（L273–298）。注意此时 **chunk 循环早已结束**：S 是完整的 $[64, 64]$，softmax 一次算完，得到 ρ_t、ℓ_t、以及原地覆盖成 P 的分数碎片。顺序不能反（19.8 节坑一）。
4. **Phase 3：PV 的 chunk 流水**（L310–348）。每个 v_chunk：先对 **该 chunk 的**累加器乘 ρ_t，再做 `gemm_rs`（P 在寄存器、V 从 smem 走 `ldmatrix.trans`）。

ffpa_attn.cuh L202-215（KV 主循环入口 + Phase 0：V 预取与 cp.async 组计数）

```
  // Main loop over KV tiles
  for (int kv_tile = 0; kv_tile < kv_tiles; ++kv_tile) {
    // ===== Phase 0: V Prefetch =====
    int v_write = 0;
#pragma unroll
    for (int v = 0; v < kStagesV - 1 && v < kDChunks; ++v) {
      g2s_load_v(kv_tile, v, v_write);
      cp_async_fence();
      v_write = (v_write + 1) % kStagesV;
    }
    if constexpr (kStagesV > 1) {
      cp_async_wait<kStagesV - 2>();
      __syncthreads();
    }
```

Phase 1 的内层是这个 kernel 的核心循环，包含两个易错点：chunk 的**读槽**（`qk_read`）与**写槽**（`qk_write`）相差 Sk − 1，以及每轮都要**先把下一块发出去再等当前块**：

ffpa_attn.cuh L237-267（QK chunk 主循环：预取 $d + \mathrm{Sk} - 1$、等待、gemm_ss 累加）

```
    // QK main loop
    for (int d_chunk = 0; d_chunk < kDChunks; ++d_chunk) {
      int d_next = d_chunk + kStagesQK - 1;
      if (d_next < kDChunks) {
        g2s_load_q(d_next, qk_write);
        g2s_load_k(kv_tile, d_next, qk_write);
        cp_async_fence();
        if constexpr (kStagesQK > 1) {
          cp_async_wait<kStagesQK - 2>();
        } else {
          cp_async_wait<0>();
        }
        __syncthreads();
        qk_write = (qk_write + 1) % kStagesQK;
      }

      // QK GEMM: S += Q[d_chunk] @ K[d_chunk]^T
      auto sQ_stg = make_tensor(
          make_smem_ptr(q_base + qk_read * kQChunkElements), SmemLayoutQ{});
      auto sK_stg = make_tensor(
          make_smem_ptr(k_base + qk_read * kKVChunkElements), SmemLayoutKV{});
      auto tCrQ = thr_mma_qk.partition_fragment_A(sQ_stg);
      auto tCrK = thr_mma_qk.partition_fragment_B(sK_stg);
      auto tQsQ_s2r = s2r_thr_q.partition_S(sQ_stg);
      auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);
      fa_cute::gemm_ss(tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
```

```
                        tiled_mma_qk, s2r_copy_q, s2r_copy_k,
                        s2r_thr_q, s2r_thr_k);
      __syncthreads();
      qk_read = (qk_read + 1) % kStagesQK;
    }
```

`cp_async_wait<N>` 的语义是「等到未完成的组数 $\leq N$」。`kStagesQK` $= 2$ 时 `wait<0>` 意味着**把刚发出的预取也一并等到**，这一档没有循环内重叠，重叠主要来自 Phase 0 的 V 预取与整段 QK/softmax 的并行；Sk ≥ 3 时 `wait<Sk-2>` 才允许 Sk $-$ 2 组仍 in-flight，循环内重叠才真正建立。这个细节在 19.6 节解释性能差异时会用到。

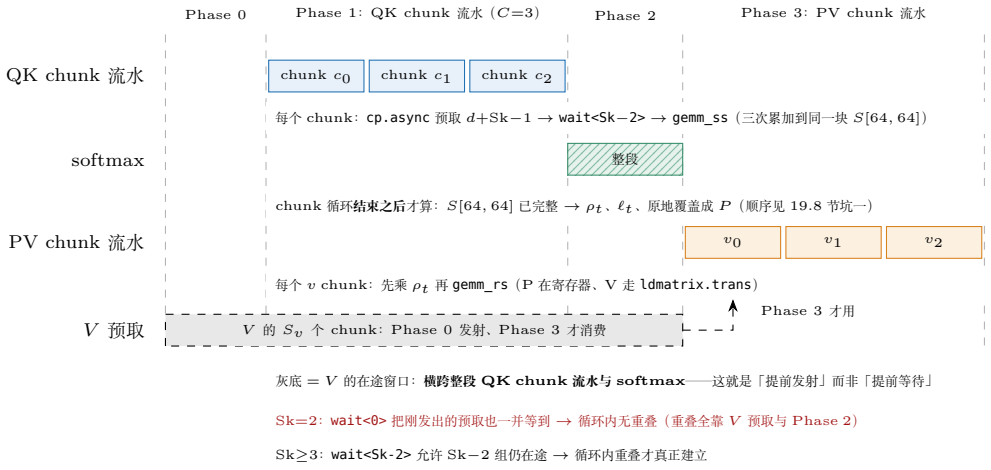


图 20.2: cp.async 版的四阶段流水（19.4.3 节主循环，任取一轮 KV tile 周期）。Phase 0 先把 S_v-1 个 V chunk 发出去（提前发射）；Phase 1 沿 head_dim 逐个 chunk 走「预取 d+Sk−1 → `wait<Sk-2>` → `gemm_ss`」，三次累加到同一块 S[64, 64]；Phase 2 在 chunk 循环结束之后对完整的 S 一次算完在线 softmax；Phase 3 每个 chunk 先乘 ρ_t 再 `gemm_rs`。灰底为 V 的在途窗口：从 Phase 0 发射一直横跨整个 Phase 1 与 Phase 2，直到 Phase 3 才被消费——这是本 kernel 唯一在循环内建立的重叠。Sk=2 与 Sk≥3 的差别只落在 Phase 1 的 `wait` 口径上（19.6 节的性能差异来源）。

Phase 2 与 Phase 3 的片段（软化的迭代顺序 + 广播式的 rescale）：

ffpa_attn.cuh L276-298（Phase 2：行 max 的 4-lane shuffle 归约、ρ_t、e^{S-m} 原地覆盖）

```
    float row_scale[kORows];
#pragma unroll
    for (int row = 0; row < kORows; ++row) {
      float tile_max = -INFINITY;
#pragma unroll
      for (int col = 0; col < size<1>(scores); ++col)
        tile_max = fmaxf(tile_max, scores(row, col) * scale);
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
      const float next_max = fmaxf(row_max[row], tile_max);
      row_scale[row] = exp2f(row_max[row] - next_max);
      float tile_sum = 0.0f;
#pragma unroll
      for (int col = 0; col < size<1>(scores); ++col) {
        const float p = exp2f(scores(row, col) * scale - next_max);
        scores(row, col) = p;
        tile_sum += p;
      }
      tile_sum += __shfl_xor_sync(0xffffffff, tile_sum, 1);
      tile_sum += __shfl_xor_sync(0xffffffff, tile_sum, 2);
      row_sum[row] = row_sum[row] * row_scale[row] + tile_sum;
      row_max[row] = next_max;
    }
```

这里的 `__shfl_xor_sync(mask, v, 1)` 与 `(..., 2)` 不是随意选的：$m16n8k16$ 的累加器把一行的 8 个列分

给 4 个 lane（每 lane 2 个连续列），所以「一行的 64 列」在寄存器里由「同组 4 个 lane × 每 lane 16 个值」构成，`xor 1/xor 2` 恰好覆盖这 4 个 lane。$kORows = 2$ 则是每线程持有的两个行分组（r 与 $r+8$）。这套 `convert_layout_acc_rowcol` 的「(MMA, MMA_M, MMA_N) 拆成 (行, 列)」技巧在第 二十三 章会逐字段展开。

> **P 的寄存器重解释（L300–305）**：`convert_layout_acc_Aregs` 把 S 的累加器 layout 从 (MMA=4,MMA_M, MMA_N) 重解释为 PV 的 A 操作数 layout ((4, 2),MMA_M,MMA_N/2)——**零拷贝、不动数据、只改逻辑坐标**，于是 softmax 后的 P 不需要写回 smem 再 `ldmatrix`。这是 FA 系列「寄存器复用」的核心技巧（第 十七/二十六 章）。

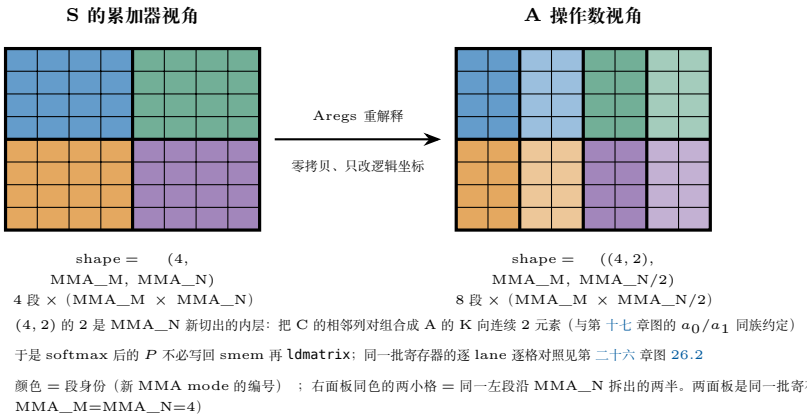


图 20.3: 零拷贝的寄存器重解释（19.4.3 节 quote 块中 `convert_layout_acc_Aregs` 的模式级示意）。左：S 的累加器 layout 是 (MMA=4, MMA_M, MMA_N)——4 个 MMA 重复段，每段 MMA_M × MMA_N；右：同一批寄存器被读成 ((4, 2), MMA_M, MMA_N/2)——沿 MMA_N 拆成 (2, MMA_N/2)，拆出的 2 与原来的 4 组合成新的 8 段 MMA mode，正是 PV 的 A 操作数所需的「K 向连续 2 元素」（与第 十七 章核心图的 a_0/a_1 同族约定）。数据本身未被修改：只是坐标重排，softmax 后的 P 因此不需要写回 smem 再 `ldmatrix`。

Phase 3 的 rescale 是本章数学结论落地的位置：**同一个 `row_scale` 作用在每一个 chunk 的累加器上**：

ffpa_attn.cuh L324-335（Phase 3：第 v 个 chunk 的累加器乘 ρ_t——广播式 rescale）

```
      // Rescale O accumulator (与 TMA WS kernel 完全一致)
      auto tCrO = make_tensor(make_rmem_ptr(&o_acc_storage[v_chunk][0]),
                              OFragLayout{});
      if (kv_tile > 0) {
        auto tCrO_rc = make_tensor(
            tCrO.data(), fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
#pragma unroll
        for (int row = 0; row < kORows; ++row)
#pragma unroll
          for (int col = 0; col < kOCols; ++col)
            tCrO_rc(row, col) *= row_scale[row];
      }
```

`if (kv_tile > 0)` 是必要的：第一轮时累加器全零，乘不乘都一样，跳过可以省 $C\times$ 32 次乘法（$D = 512$ 时是 256 次/线程）。

20.4.4　TMA+WS 版：单线程 TMA 与 (stage, phase) 计数

`ffpa_attn_tma_mma_ws_split_d_cute`（L377–641）把「搬」与「算」拆成两个 warpgroup：**128 producer + 128 consumer = 256 线程**，`__launch_bounds__(256,1)`（L380）。

ffpa_attn.cuh L380-392（TMA 版签名：CUTLASS_GRID_CONSTANT 描述符 + 256 线程）

```
__global__ void __launch_bounds__(256, 1)
ffpa_attn_tma_mma_ws_split_d_cute(
    CUTLASS_GRID_CONSTANT TmaQ const tma_q,
    CUTLASS_GRID_CONSTANT TmaK const tma_k,
```

```
    CUTLASS_GRID_CONSTANT TmaV const tma_v,
    cutlass::half_t *output, int rows, int seqlen) {
  using namespace cute;
  using Traits = fa_cute::FFPAAttnSplitDCuTeTraits<kHeadDim>;
  using Element = typename Traits::Element;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  using TmaBarrier = cutlass::arch::ClusterTransactionBarrier;
  using CtaBarrier = cutlass::arch::ClusterBarrier;
```

消费者侧与 cp.async 版的**计算逻辑逐行相同**（头注释 L70–73 明确写了这一点），差别全在流水计数上。计数由四组 mbarrier 承担（L409–412、L414–429）：

ffpa_attn.cuh L414-429（角色划分与 mbarrier 初始化：full 的到达数是 1，empty 是 128）

```
  const bool is_producer = threadIdx.x < kProducerThreads;
  const int wg_tid = is_producer ? threadIdx.x : threadIdx.x - kProducerThreads;
  const int q_tile = blockIdx.y * (seqlen / kBr) + blockIdx.x;
  const int kv_tiles = seqlen / kBc;

  if (threadIdx.x == 0) {
    for (int stage = 0; stage < kStagesQK; ++stage) {
      TmaBarrier::init(&qk_full[stage], 1);
      CtaBarrier::init(&qk_empty[stage], kConsumerThreads);
    }
    for (int stage = 0; stage < kStagesV; ++stage) {
      TmaBarrier::init(&v_full[stage], 1);
      CtaBarrier::init(&v_empty[stage], kConsumerThreads);
    }
  }
  __syncthreads();
```

三个数字要记牢：

- `TmaBarrier::init(&qk_full[s], 1)`——full 屏障的**到达数**是 1（生产者的一次 `arrive_and_expect_tx`），但真正让它完成的是 TMA 的**字节事务计数**：`arrive_and_expect_tx` 声明本次期望的字节数，TMA 每搬完一段就减掉，归零即放行。QK 槽位声明的是 `sizeof(Element) * (size(sQ) + size(sK))` $= 2\times 8192\,\mathrm{B} = 16\,\mathrm{KB}$（L459–460），V 槽位是 8 KB（L477）。
- `CtaBarrier::init(&qk_empty[s], kConsumerThreads)`——empty 的到达数是 **128**：每个 consumer 线程消费完该槽位各 arrive 一次。这与第 十八 章不同：那边 empty 也是 $257 = 256$ consumer $+ 1$ producer（producer 在 empty 上补 1 票）；本章 producer 不在 empty 上 arrive（它的 1 票记在 full 侧的 `arrive_and_expect_tx`），故 empty 是 128 张纯 consumer 票。
- `NOTES_V2_REG_DEALLOC(32)` / `NOTES_V2_REG_ALLOC(232)`（L432、L484）——producer 只做地址计算与 TMA 提交，32 个寄存器够用；consumer 要持有 S、P、C 个 O 累加器与操作数碎片，按上限申请 232，取值对齐 ffpa-attn 生产库 persist_d 的原版配比（32/232）。约束链（FA3 论文与 PTX ISA 9.7.19.5）：立即数须为 $[24, 256]$ 内 8 的倍数，且 NVCC 对每线程实际分配另设 ≤ 255 上限——合法上限值为 248，本书初版的 255 不是 8 的倍数，开宏会被 ptxas 拒绝。池算术：$128\times 32 + 128\times 232 = 33792 \leq 65536$，余量 31744 个寄存器，远高于「留 ≥ 2048」的安全线。注意该指令对 TMA 目的地址空间敏感：dst 为 `shared::cluster` 的 `cp.async.bulk.tensor` 会让 ptxas 以 C7506 丢弃 `setmaxnreg`（第 十五/十九 章考据）；dst 为 `shared::cta` 则 sm_120a/sm_120f 均保留（本章 cute 路径 kernel 正属此类）。默认构建里这两个宏是**意图注释**；真正决定分配的是 `__launch_bounds__(256,1)` 与 ptxas。

生产者的主体是「单线程 TMA」：整个 warpgroup 里只有 `wg_tid == 0` 执行 TMA 提交，其余 127 个线程只负责交出寄存器额度。它的两层循环按**先整轮 QK、再整轮 V** 的顺序投递，并用扁平计数器算相位：

ffpa_attn.cuh L431-446（producer：dealloc + 单线程 TMA + 扁平 (stage, phase) 等待）

```
  if (is_producer) {
```

```
    NOTES_V2_REG_DEALLOC(32);
    if (wg_tid == 0) {
      auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto mK = tma_k.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto mV = tma_v.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto q_slice = tma_q.get_slice(_0{});
      auto k_slice = tma_k.get_slice(_0{});
      auto v_slice = tma_v.get_slice(_0{});

      for (int kv_tile = 0; kv_tile < kv_tiles; ++kv_tile) {
        for (int d_chunk = 0; d_chunk < kDChunks; ++d_chunk) {
          const int chunk_index = kv_tile * kDChunks + d_chunk;
          const int stage = chunk_index % kStagesQK;
          const int phase = (chunk_index / kStagesQK) & 1;
          CtaBarrier::wait(&qk_empty[stage], phase);
```

消费者侧对应的等待与释放（QK 段）：

ffpa_attn.cuh L538-556（consumer：等 full、跨代理 fence、gemm_ss、立即释放 empty）

```
      for (int d_chunk = 0; d_chunk < kDChunks; ++d_chunk) {
        const int chunk_index = kv_tile * kDChunks + d_chunk;
        const int stage = chunk_index % kStagesQK;
        const int phase = (chunk_index / kStagesQK) & 1;
        TmaBarrier::wait(&qk_full[stage], phase);
        tma_fence_proxy_async_shared_cta();
        auto sQ = make_tensor(make_smem_ptr(q_base + stage * kQChunkElements),
                              SmemLayoutQ{});
        auto sK = make_tensor(make_smem_ptr(k_base + stage * kKVChunkElements),
                              SmemLayoutKV{});
        auto tCrQ = thr_mma_qk.partition_fragment_A(sQ);
        auto tCrK = thr_mma_qk.partition_fragment_B(sK);
        auto tQsQ = s2r_thr_q.partition_S(sQ);
        auto tKsK = s2r_thr_k.partition_S(sK);
        fa_cute::gemm_ss(tCrS, tCrQ, tCrK, tQsQ, tKsK,
                         tiled_mma_qk, s2r_copy_q, s2r_copy_k,
                         s2r_thr_q, s2r_thr_k);
        CtaBarrier::arrive(&qk_empty[stage]);
      }
```

`tma_fence_proxy_async_shared_cta()` 是**跨代理同步**：TMA 写 smem 走的是 *async proxy*，而 `ldmatrix` 读 smem 走的是 *generic proxy*；PTX 规定同一内存位置跨代理访问必须用 `fence.proxy.async` 建立顺序（PTX ISA §9.7.15.4）。源码在生产者提交后与消费者 wait 之后**两端都调用**，属于保守写法。

PV 段的消费者循环与 cp.async 版同构，只是等待/释放换成屏障，并且 rescale 与 `gemm_rs` 的先后顺序必须保持：

ffpa_attn.cuh L603-616（PV chunk：rescale 到 gemm_rs 到释放 empty）

```
        auto tCrO = make_tensor(make_rmem_ptr(&o_acc_storage[v_chunk][0]),
                                OFragLayout{});
        if (kv_tile > 0) {
          auto tCrO_rc = make_tensor(
              tCrO.data(), fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
#pragma unroll
          for (int row = 0; row < kORows; ++row)
#pragma unroll
            for (int col = 0; col < kOCols; ++col)
              tCrO_rc(row, col) *= row_scale[row];
        }
        fa_cute::gemm_rs(tCrO, tCrPv, tCrV, tVsVt,
                         tiled_mma_pv, s2r_copy_v, s2r_thr_v);
        CtaBarrier::arrive(&v_empty[stage]);
```

TMA 版必须用扁平相位、cp.async 版却可以每轮重置，原因在于两版的同步原语性质不同。`cp.async` 的 `wait<N>` 是**每线程私有的组 FIFO**，KV tile 之间**排空**（L268–271、L349–352 的 `wait<0>` + `__syncthreads`）之后计数器就地归零即可；而 mbarrier 是**共享对象**，它的 phase 位由自身状态决定，跨 KV tile 复用同一批屏障时，两端必须用同一个全局公式导出 (stage, phase)，即式 20.8；这也是 19.8 节坑二要防的错。这份代价换来的是：TMA 版在 KV tile 边界**不需要排空**，流水可以连续推进（相位继续翻转），在浅流水下这是明确的收益。

20.4.5 两实现对照

维度	cp.async 版（L80–375）	TMA+WS 版（L377–641）
线程数 / 角色	128，全员搬运 + 计算	256 = 128 producer + 128 consumer
装载指令	`SM80_CP_ASYNC_CACHEGLOBAL`(`uint128_t`)	`cp.async.bulk.tensor`（TMA，单线程发起）
完成检测	`cp.async.wait_group` + `__syncthreads`	mbarrier：full (tx-count) + empty (128 arrive) + phase
流水计数	每线程私有组 FIFO；每 KV tile 排空后重置	扁平 `idx` 导出的 (stage, phase)；跨 KV tile 连续
寄存器池	无 `setmaxnreg`	`dealloc(32)` / `alloc(232)`（dst 为 `shared::cluster` 时被 C7506 丢弃，与 arch 后缀无关）
Q 的装载	每 KV tile 重装（与 K 同 stage）	每 KV tile 重发 TMA（同样的冗余）
计算段组织	QK → softmax → PV 串行	consumer 串行；producer 全程并行装载
架构门槛	sm_80+	sm_90a / sm_120a（TMA）

两种实现是「同一条算法、两套流水机制」，读者的收益在于看清一件事：**Split-D 的结构（chunk 池、双 TiledMma、ρ 广播）与用哪种搬运指令无关**；TMA 与 WS 换掉的是「数据怎么来、什么时候到」这一层。

20.4.6 O 累加器的寄存器占用

把寄存器用量算完整。O 的碎片形状是 $[B_r, B_c] = [64, 64]$ 的 f32 累加器（每个 chunk 一份），按 128 个消费者线程均分；C 个 chunk 的累加器合起来就是 $[64, D]$：

$$R_O = \frac{B_r \cdot D}{\#\text{consumer threads}} = \frac{D}{2} \quad \Longrightarrow \quad \begin{cases} D = 64: & 32 \text{ 个 f32/线程} \\ D = 128: & 64 \\ D = 192: & 96 \\ D = 512: & \mathbf{256} > 255 \text{ (每线程硬上限).} \end{cases} \tag{20.10}$$

D=512 这一行的结论很重要：**单是 O 累加器就越界了**。再加 S 碎片（32 个 f32）、P 的 f16 片段、V/Q/K 的操作数片段与地址寄存器，$D = 512$ 必然溢出到 local memory。正确性不受影响（测试里 D=512 例照常通过），但吞吐会下降。这就是生产库在 $D \geq 768$ 换 M4N2（`atom_layout` $(4, 2, 1)$：4 个 M-warp × 2 个 N-warp，R_O 降到 $D/4$）的原因，也是本章这版 kernel 把 $D \leq 384$ **视为性能最佳区间**的依据（$D = 384$ 时 $R_O = 192$）。

寄存器占用还解释了另一件事：**为什么这版 WS 不用双 consumer**（第 十九 章的形态）。双 consumer 意味着两套 partial 状态与两套寄存器池，而这里 consumer 单个就已经达到 232 的寄存器配额；生产库在 split-D WS 变体上的记录是「`setmaxnreg` 的 consumer 上限装不下 D=512 的 256 寄存器累加器，该变体已禁用，大 D 转向 M4N2」。结构选择在这里由寄存器文件容量直接决定。

20.5　图示

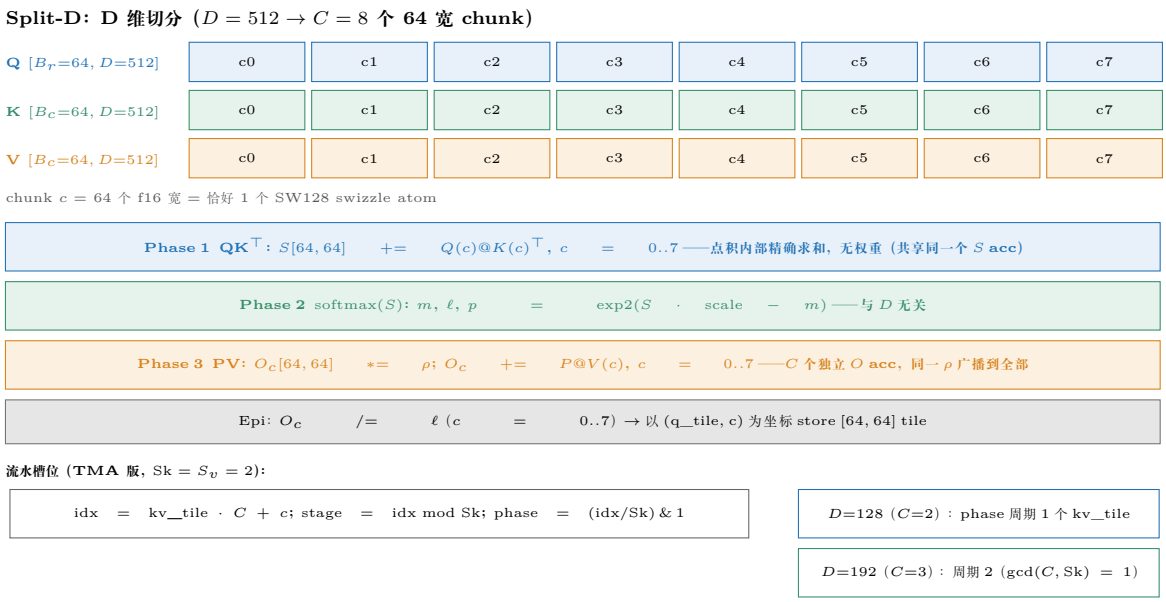


图 20.4: Split-D 的 D 维切分：Q/K/V 各切成 $C = D/64$ 个 64 宽 chunk（一个 chunk 等于一个 SW128 swizzle atom 宽）。QK 的 chunk 累加写进同一块 $S[64,64]$；PV 的每个 chunk 写进自己的 $O_c[64,64]$，共用同一个 ρ 与 ℓ；流水槽位由扁平计数器 idx 导出。

20.6　性能分析

口径先对齐。本章涉及两组性能数字，必须**双标注**，因为它们来自不同的 GPU 与不同的形状：

- **头注释口径（PRO 5000）**：ffpa_attn.cuh L16–18 声明「`B=1,H=32,N=8192,D=512, SM120a RTX PRO 5000`: cuDNN SDPA 57.2 TFLOPS | FFPA TMA WS 110.7 TFLOPS（1.94×）」。这是源码作者在 **RTX PRO 5000**（本机同型号）上的测量，形状是 N=8192、D=512、Sk = Sv = 2。本书本章**未复测**该口径（bench 需独占 GPU 且耗时，留给后续增量复测），引用时一律标注「注释口径」。
- **README 口径（RTX 5090）**：LeetCUDA README 的 Split-D 表格（`--bhnd 1,32,16384,320`）给出 D=320、N=16384、Sk = Sv = 2 时 **182.6 TFLOPS**（cuDNN 83.1，2.20×）；Sk = Sv = 1 时 127.2 TFLOPS（1.53×）。这组数据是 **RTX 5090** 口径。

两组的 GPU（PRO 5000 vs 5090）、形状（D=512/N=8192 vs D=320/N=16384）与流水深度都不同，**不可互换、不可换算**；正文凡引用数字必须带 GPU 名与形状。二者一致的定性结论只有一条：在 $D \gg 128$ 上，Split-D 相对 cuDNN SDPA 的收益在 1.5×–2.2× 量级，把流水从 Sk=Sv=1 加到 2 是主要增益来源（5090 口径下 1.53× → 2.20×）。这条结论与 19.6 节的「stage 数与 smem 预算互相挤压」的定性分析方向一致。

结构层面的算术强度。按每轮 KV tile 统计（每轮装载 Q、K、V 各 C 个 chunk）：

$$\text{bytes} = 3{\cdot}C{\cdot}64{\cdot}64{\cdot}2 = 384\,D\ \text{B}, \qquad \text{FLOPs} = 2{\cdot}(2{\cdot}64{\cdot}64{\cdot}D) = 16384\,D, \qquad AI = \frac{16384D}{384D} \approx 42.7\ \text{FLOP/B}. \tag{20.11}$$

有两点。其一，AI 与 D **无关**。GEMM 的计算量与访存量都随 D 线性增长，比值被约掉了；所以「大 head dim 更耗带宽」并不是 Split-D 的性质。其二，这 42.7 FLOP/B 里**约三分之一是冗余的**：Q 对每个 KV tile 都重装了一遍（N/B_c 次），而它在整个 q_tile 生命周期里**一次都不用变**。生产库的 persist-D 家族正是针对这部分冗余（Q 的 A 操作数常驻寄存器、全程走 `gemm_rs`）的优化，本章这版教学实现保留了「每轮重装」的简单写法。把 Q 常驻后流量降到 2/3，AI 升到约 64 FLOP/B，这是本 kernel 最明确的一处结构优化空间。

结构开销清单（不依赖复测的定性判断，供后续复测逐条核对）：

1. Q 每 KV tile 重装（式 20.11 讨论）；

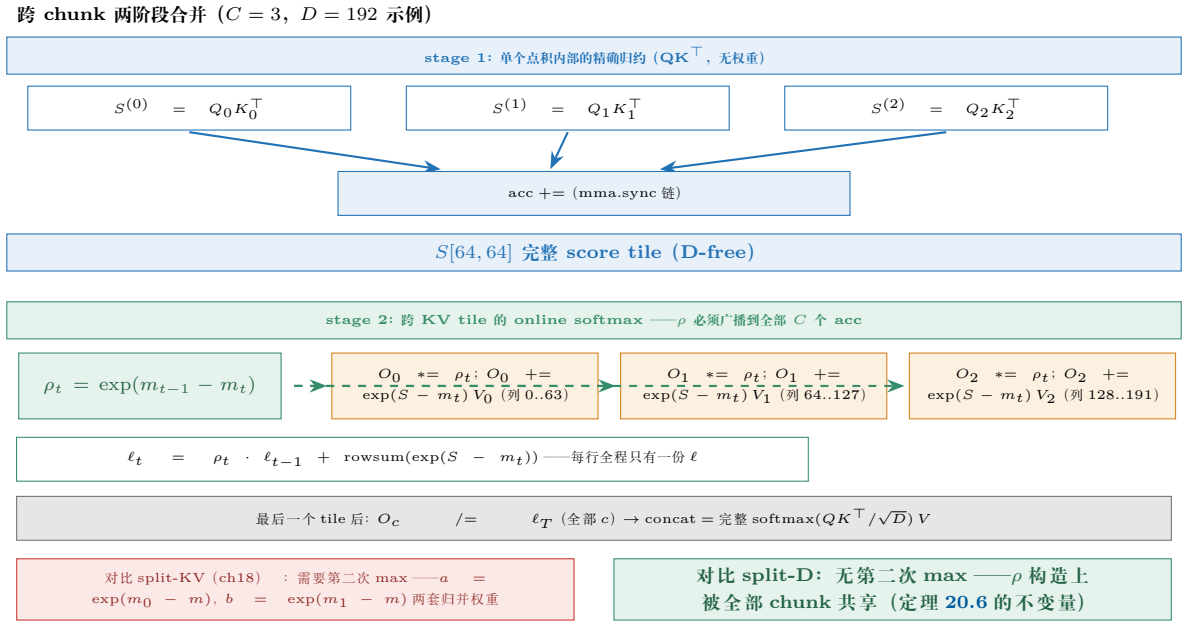


图 20.5: 跨 chunk 的两阶段合并。第一阶段：$QK^\top$ 的 chunk 求和是同一点积内部加数的分组，**精确且无需权重**；第二阶段：PV 的每个 chunk 各持一个输出列段累加器，跨 KV tile 的在线 rescale 因子 ρ_t **必须广播到全部 C 个累加器**（定理 20.6 的不变量），ℓ 全程只有一份。

2. consumer 内部 QK → softmax → PV 串行，mma.sync 是同步指令，三者无法重叠（第 十九 章的 pingpong 讨论）；这也是 Split-D 在 D 不大时不必做 WS 的原因——收益会被结构开销抵消；
3. $D \geq 512$ 时 O 累加器溢出（式 20.10）；
4. PV 的 chunk 循环把同一块 P（$[64, 64]$ 的 f16）反复送入 C 次 `gemm_rs`，每次都要重新 `ldmatrix` 一列 V 并重走 4 个 tile repeat——这是分列累加的固有代价，C 越大越明显。

复测建议（供附录 B 汇总）：`book/tests/ch19_ffpa_split_d.cu` 自带 `bench` 子命令（B=1、H=32，$(D, N) \in \{(128, 4096), (512, 4096), (512, 8192)\}$，cp.async 与 TMA+WS 对照）。复测时注意：D=512 的 `cp.async` 配置按式 20.9 在 Sk=Sv=2 下是 48 KB、可行；而 Sk=Sv=4（96 KB）已接近上限，Sk=5 会直接超预算。数据请落 `.tmp/book-bench/ch19/`，并与上述两组口径分开登记。

20.7 实践经验与考据说明

实践经验 20.1 (F4 · 口径差异)：L16-18 头注释性能口径（PRO 5000）头注释声明 D=512、N=8192 在 **SM120a RTX PRO 5000** 上 cuDNN SDPA 57.2 TFLOPS、FFPA TMA WS 110.7 TFLOPS（1.94×）；而 LeetCUDA README 的 Split-D 表格是 **RTX 5090** 口径，形状也不同（D=320、N=16384、182.6/83.1 = 2.20×）。两口径的比值自洽（110.7/57.2 = 1.935；182.6/83.1 = 2.197），差别来自 GPU 与形状。本节 **正文一律双标注，不复测不给绝对结论**。

实践经验 20.2 (F1/F3 · 表述澄清)： L12-13「PV 保持小 tile 控制 acc_O 寄存器」注释把「控制 acc_O 寄存器」归因于 PV 的 tile 形状 `Tile<64,16,16>`。按式 20.10 复算：O 累加器总量由输出 tile 的完整形状 $[64, 64] \times C$ 决定，$= D/2$ 个 f32/线程，**与 tile 形状无关**。tile 形状决定的是「一次 TiledMMA 调用覆盖多大输出块」、`EURepeat`（$< 1, 2, 1 >$ vs $< 1, 8, 1 >$）以及寄存器生存期的切分粒度：小 tile 让 chunk 边界成为寄存器域边界（每 chunk 独立的 `o_acc_storage[v]`）。结论不受影响，但表述应读作「小 tile 对齐 chunk 粒度」，不是「小 tile 减少总面积」。

实践经验 20.3 (F1 · 依赖关系核校通过)：L2 只 include flash_attn.cuh 是否自足 `ffpa_attn.cuh` L2 只写了 `#include "flash_attn.cuh"`，而本文件全程使用 `GMMA::Layout_K_SW128_Atom`、`fence` 相关 helper 与 `swizzle` 系符号（后者定义在 hgemm.cuh L390 附近）。与第 十八 章的考据一致：单独编译 `ffpa_attn.cuh` 会缺符号，必须按 `hgemm.cuh` → `flash_attn.cuh` → `ffpa_attn.cuh` 的顺序 include（本章测试头注释 L12–13 已写明，`book/tests/build_tests.sh` 不含额外 include 路径也能编过，说明该顺序在 cutlass 头下自足）。

实践经验 20.4 (F2/F3 · 协议计数核校通过)：mbarrier 到达数与 TMA 事务字节数 逐条核验：(1) 线程数与角色：L380 `__launch_bounds__(256,1)`、L399–400 `kProducerThreads`=`kConsumerThreads`=128，与头注释 L14/L28「128 producer + 128 con-

sumer = 256 total threads (vs 原始 384)」一致（「原始 384」指第 十八 章的 FA2 TMA WS：128 producer + 256 consumer）；(2) full 屏障初始到达数 1（L421/L425）、empty 屏障 128（L422/L426），与消费者侧「每线程消费完 arrive 一次」（L555、L616）以及生产者侧 `arrive_and_expect_tx`（L459–460、L477）匹配；(3) 每次 QK 槽位的期望字节数是 $2 \times 64 \times 64 \times 2 = 16384$ B（Q 与 K 各一个 chunk），V 槽位 8192 B，与 `size(sQ)+size(sK)` / `size(sV)` 的写法一致；(4) 相位公式在生产者（L443–445）与消费者（L539–541、L591–593）三处 **逐字符一致**。协议描述与代码一致，无发现错误。

实践经验 20.5 (F1 · 边界提醒)： D 的合法取值与「$D\%64 \neq 0$」两个 kernel 都以 `static_assert(kHeadDim % 64 == 0)` 把关（L36、L90、L394），并且 traits 的第二个断言把 tile 固定在 64×64（L37）。这意味着「大 head dim」在本章语境下是**离散集合** $\{64, 128, 192, \ldots\}$，不是连续区间；D=96 或 D=320 的 32 宽 chunk 走的是另一套 layout（SW64，生产库里的 (32, 32)/(32, 64) chunk 配置），不在这份教学代码的覆盖范围内。测试的 SKIP 分支只处理 $N\%64$ 与 smem 超限，不处理 $D\%64$——因为后者是编译期错误。

20.8　常见坑

坑一：把「chunk 求和」与「softmax」的先后顺序写反。S 必须是 **全部** chunk 累加完之后才做 softmax（Phase 1 完整 → Phase 2）。若把 softmax 放进 chunk 循环里（对「部分点积」做 softmax），得到的是另一个函数的输出，且不报错。数值全错但形状正确。这是本章后果最严重的坑，因为改动很小、错误很隐蔽。

坑二：TMA 版的 (stage, phase) 两端公式必须逐字符一致。mbarrier 的 phase 位由屏障自身状态决定，跨 KV tile 复用同一批屏障时，两端必须用式 20.8 的**同一个扁平计数器**。若把消费者侧改成「每个 KV tile 从 0 重新计数」（照抄 cp.async 版的写法），相位会与屏障状态错位：轻则读到半新半旧的数据（静默数值错误），重则 `mbarrier.wait` 永远等不到而挂死。cp.async 版之所以能每轮重置，是因为 `wait_group` 是**每线程私有**的 FIFO 计数，不是共享对象。

坑三：ρ_t 只 rescale 了一部分 chunk。式 20.5 要求 **所有** C **个累加器**都乘同一个 ρ_t。漏掉某些 chunk（或每个 chunk 各算自己的 ρ）都会得到「看起来合理」的结果：漏乘的 chunk 相当于用了偏小的 max 基准，$O^{(c)}$ 系统性偏大，误差随 T_c 累积。测试里 N=256 只有 T_c=4 时就已能暴露（最大误差量级从 10^{-5} 跳到 10^{-2} 以上）。

坑四：把 D=512 当成「性能配置」。式 20.10 给出 $R_O = 256 > 255$，O 累加器必然 spill。教学测试仍覆盖 D=512，它的作用是在寄存器极值档位验证正确性（chunk 数最大时逻辑仍正确），不是性能样板。想在大 D 上取得性能，得换 M4N2 式的 N-warp 分摊（生产库路线）。

坑五：stage 数加深与 smem 预算互相挤压。式 20.9 表明 Sk = Sv = 4 已到 96 KB（上限 99 KB）。任何「再深一级流水」的尝试都会先碰到 smem 上限；辅助工具必须做可行性检查（本章测试打印 `SKIP(ch19): ... smem ... > optin ...`）。

坑六：跨代理可见性（`fence.proxy.async`）不能省略。TMA 写 smem 走 async proxy，`ldmatrix` 读 smem 走 generic proxy，同一位置跨代理访问需要 `fence.proxy.async`（PTX ISA §9.7.15.4）。源码在生产者提交后（L463、L479–480）与消费者 wait 之后（L543、L595）都调用了 `tma_fence_proxy_async_shared_cta()`；读者改写时若「优化掉」其中一侧，会得到偶发的读旧值故障；这类故障是**概率性**的，只在特定时序下出现。

坑七：Q 的重装有实际开销，不要把「Q 与 K 同槽位」当成零成本。两个版本都把 Q chunk 与 K chunk 放进同一个 QK stage（一个 stage = 两份 8 KB）。这在实现上很省事，但意味着 Q 每次都要跟着 K 一起重新装载；一旦你要加深 K 流水，Q 的冗余流量也同步放大。生产库的 persist-D 正是把 Q 移出这个 stage（常驻寄存器），把式 20.11 的 42.7 FLOP/B 抬到约 64。

20.9　面试要点速查

Q1：为什么 head dim > 128 不能直接套 FA2？

A：三条独立硬约束。smem：整块 $[B, D]$ tile 随 D 线性涨（D=512 时 $Q{+}K{+}V$ 各一份 192 KB > 99 KB 上限）；寄存器：O 累加器 = $D/2$ 个 f32/线程（D=512 时 256，越过 255 上限）；几何：SW128 swizzle atom 行宽 128 B = 64 个 f16，切 64 宽才能直接复用现成布局。三条都指向同一个动作：把 D 切成 chunk 流。

Q2：Split-D 与 Split-KV 的本质区别？

A：切的对象不同。Split-KV 切键下标集合，每片得到一份**部分 softmax 状态**，合并要用 LSE 重标定（各片 max 不同，需要第二个 max 与 $e^{m_p - m}$）；Split-D 切特征维，S 的 chunk 求和与 softmax 无关（纯粹点积内部加数分组），O 的 chunk 分列互不重叠，合并只是「沿列拼起来 + 最后除一次 ℓ」。Split-D **没有第二个 max**，合并里零额外 exp。两者正交，可叠加。

Q3：为什么 QK 的 chunk 求和不需要任何重标定？

A：因为它是同一个点积的加数分组（式 20.2），$\gamma\sum_d q_d k_d = \gamma\sum_c\sum_t q_{c,t}k_{c,t}$ 只是求和的重新分组；而 softmax 的基准 m 与缩放 γ 都在求和**之后**才介入。浮点上只差求和顺序。

Q4：PV 的每个 chunk 有独立累加器，为什么 rescale 因子必须共享？

A：见定理 20.6：$O^{(c)}$ 的含义是「以当前 max 为基准、e^{S-m} 为权重的 V 第 c 段加权和」。跨 KV tile 更新 max 时，所有 chunk 的基准都要一起改，因此必须乘同一个 ρ_t。若各 chunk 各用各的基准，尾部输出就等同于「每个 chunk 的 softmax 归一化常数不同」，而最终只能除一个 ℓ，误差无法通过归一化消除。

Q5：chunk 为什么是 64 宽？

A：因为 SW128 swizzle atom 的行宽是 128 B = 64 个 f16，且 `ldmatrix` 的粒度就是「8 行 × 16 B」。64 列 = 一个 atom 宽 = 8 个 $n8$ 原子，每个 chunk 都能直接套用 `GMMA::Layout_K_SW128_Atom` 铺出来的 layout，不必设计新的 swizzle。32 宽要走 SW64、128 宽要两个 atom 拼，都不是「重推一次 swizzle」那么简单，所以这版代码用 `static_assert` 强制固定宽度。

Q6：两版实现的差别在哪一层？

A：算法层没有差别（头注释 L70–73 明确「消费者逻辑完全一致」），差别全在「数据怎么来、什么时候到」：cp.async 版 128 线程全员搬、用每线程私有的组 FIFO 计数、每 KV tile 排空；TMA+WS 版 256 线程分工、单线程发起 TMA、用共享 mbarrier 的 (stage, phase) 计数、跨 KV tile 连续不排空。同步原语的性质（私有 FIFO vs 共享对象）直接决定了流水能否跨 tile 连续。

Q7：为什么这版 WS 只有一个 consumer warpgroup？

A：寄存器占用（式 20.10）。consumer 单 warpgroup 已达到 232 配额，D=512 时 O 累加器一项就是 256 个 f32；再加第二套 partial 状态与寄存器池放不下。生产库的记录同样是「split-D WS 变体因消费者寄存器上限装不下大 D 累加器而禁用，大 D 转 M4N2」。

Q8：D 多大时该换结构？

A：看 $R_O = D/2$。$D \le 384$（$R_O \le 192$）在本结构下寄存器占用可控；$D \ge 512$ 必然 spill；$D \ge 768$ 生产库改用 `atom_layout` $(4,2,1)$ 的 M4N2（$R_O = D/4$）。本章测试覆盖到 D=512，就是为了把这条边界纳入测试覆盖。

Q9：相位计数为什么不能用「每轮重置」？

A：mbarrier 是共享对象，phase 是它自身的状态位。跨 KV tile 复用同一批屏障时，两端只有用同一个全局公式（式 20.8）才能对上；C 与 Sk 互质时（如 D=192、Sk=2）相位周期是 4 个 KV tile，手写特判几乎必错。这一档也是本章测试特意覆盖的组合。

20.10 最小测试与动手实验

测试文件：`book/tests/ch19_ffpa_split_d.cu`（逻辑抽取自 `bench_ffpa.cu`；参考实现由 cuDNN SDPA 换成 **CPU fp64**，softmax 全程 double 累加）。覆盖矩阵按「chunk 数 × 相位形态」设计，同一份 CPU 参考分别送入两个 kernel：

D	C	N	H	覆盖点
64	1	256	2	单 chunk 退化（$C <$ Sk，流水到不了稳态）
128	2	256	2	chunk 数与 stage 数整除，phase $= t \bmod 2$ 逐 tile 翻转
192	3	192	2	与 Sk=2 互质，相位模式跨 4 个 KV tile 才重复
512	8	128	2	8 chunk，O 累加器 256 f32/线程的寄存器极值

编译与运行（本机 sm_120a；测试自带 `SKIP(ch19)` 分支处理 smem 超限）：

```
$ CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch19
== ch19_ffpa_split_d.cu (sm_120a)
GPU: NVIDIA RTX PRO 5000 72GB Blackwell (sm_120),
    max shared memory per block (optin): 101376 B (99.0 KB)
D=64: d_chunk=64 x 1 chunks, O_acc=32 f32/thread/chunk (total 32), rows=512, N=256
PASS ffpa split-d async D=64 N=256 Sk=2 Sv=2: max_abs_err=6.614e-05 (tol=5.0e-02)
PASS ffpa split-d tma ws D=64 N=256 Sk=2 Sv=2: max_abs_err=6.614e-05 (tol=5.0e-02)
D=128: d_chunk=64 x 2 chunks, O_acc=32 f32/thread/chunk (total 64), rows=512, N=256
```

```
PASS ffpa split-d async D=128 N=256 Sk=2 Sv=2: max_abs_err=6.489e-05 (tol=5.0e-02)
PASS ffpa split-d tma ws D=128 N=256 Sk=2 Sv=2: max_abs_err=6.489e-05 (tol=5.0e-02)
D=192: d_chunk=64 x 3 chunks, O_acc=32 f32/thread/chunk (total 96), rows=384, N=192
PASS ffpa split-d async D=192 N=192 Sk=2 Sv=2: max_abs_err=6.934e-05 (tol=5.0e-02)
PASS ffpa split-d tma ws D=192 N=192 Sk=2 Sv=2: max_abs_err=6.934e-05 (tol=5.0e-02)
D=512: d_chunk=64 x 8 chunks, O_acc=32 f32/thread/chunk (total 256), rows=256, N=128
PASS ffpa split-d async D=512 N=128 Sk=2 Sv=2: max_abs_err=8.943e-05 (tol=5.0e-02)
PASS ffpa split-d tma ws D=512 N=128 Sk=2 Sv=2: max_abs_err=8.943e-05 (tol=5.0e-02)
ALL OK
```

八条 PASS 覆盖两个 kernel 的四种 chunk 数。三点读数。其一，**两个 kernel 的最大误差逐位一致**（$6.614/6.489/6.934/8.943\times10^{-5}$）：这不是巧合，两者的消费者逻辑逐行相同（头注释 L70–73 声明），因此浮点运算序列一致，误差也就一致；它同时说明 TMA 版的 chunk 流水与屏障相位**没有引入任何数值偏差**。其二，误差随 chunk 数缓慢增大（C=1 的 6.61×10^{-5} 到 C=8 的 8.94×10^{-5}）：这正是式 20.2 讨论过的「求和顺序变化」的量级。C 越大，点积的累加链越多，舍入路径差异越大，但都远低于 F16Acc 的 5×10^{-2} 判定线（约三个数量级余量）。其三，D=512 例（`total 256` f32/线程）照样 PASS，验证的是**逻辑极值**而非性能：式 20.10 说它必然 spill，但 spill 不影响正确性。

动手实验建议（源码冻结，请在**自己的副本**上改）：

1. 把 D=192 例的 Sk 从 2 改成 3：此时 $C=\mathrm{Sk}$，每 KV tile 恰好推满一轮 stage，$\mathrm{phase}=t \bmod 2$，相位周期是 2 个 KV tile——用来验证式 20.8 的周期公式；
2. 在 rescale 处只对偶数 chunk 乘 `row_scale`（坑三），观察最大误差如何随 N 增长；
3. 把 $\mathrm{Sk}=\mathrm{Sv}$ 从 2 加到 4，观察 smem 涨到 96 KB 与辅助工具的 `SKIP` 判定；
4. 用测试的 `bench` 子命令跑 $(D,N)=(128,4096)$ 与 $(512,4096)$，对比 R_O 从 64 涨到 256 时的吞吐落差（注意 D=512 是寄存器极值档，落差里同时含 spill 与 C 增大两个因素）。

延伸阅读

- @DefTruth《FFPA(Split-D)：FA2 无限 HeadDim 扩展》
  https://zhuanlan.zhihu.com/p/13975660308
  （本章主参考：Split-D 的动机、chunk 划分与两版 kernel 的原始叙述；与本章 19.2–19.4 节逐条对照阅读）；
- @DefTruth《图解：从 Online-Softmax 到 FlashAttention V1/V2/V3》
  https://zhuanlan.zhihu.com/p/668888063
  （$m/\ell/O$ 递推与 online rescale 的图示化推导；定理 20.6 的直观版本，第 十六 章主参考）；
- @66RING《使用 cutlass cute 复现 flash attention》
  https://zhuanlan.zhihu.com/p/696323042
  （CuTe 版 FA 的完整实现路径，对照本章 `gemm_ss`/`gemm_rs` 与 fragment 流转；第 二十六 章主参考之一）；
- @ 竹熙佳处《写给大家看的 CuTe 教程：tiled mma》
  https://zhuanlan.zhihu.com/p/1937145378446226159
  （`Tile`/`atom_layout`/`EURepeat` 的直观解释；本章 19.4.1 节「为什么 QK 一次覆盖 $S[64,64]$」的对照读物）；
- @ 水木皇工仔《基于 CuTe 理解 swizzle, LDSM, MMA》
  https://zhuanlan.zhihu.com/p/934430036
  （SW128 atom 与 LDSM 的配合，解释 19.2 节「为什么 chunk 宽是 64」的几何来源）。

第四部分

CuTe 篇：布局代数与现代化 Kernel 工程

导读：CuTe 布局表示与代数（NVIDIA 官方白皮书中文译注）

原文：**Cris Cecka**，*CuTe: Layout Representation and Algebra*，NVIDIA Research，arXiv:2603.02298v2（2026）。

本导读为该论文的完整中文翻译，全部插图（TikZ）、公式与代码均移植自论文官方公开 LaTeX 源 `CuTeWhitepaper.tex`，版权归原作者及 NVIDIA 所有，本中译仅供学习使用。译文与全书第 20–26 章术语保持一致（Product= 乘积、Composition= 复合、Coalesce= 合并、Complement= 补、Divide= 切分、Inverse= 逆）。

摘要。高性能计算与深度学习的现代体系结构越来越多地引入专用张量指令，包括用于矩阵乘法的 Tensor Core 与面向多维数据的硬件优化拷贝操作。这些指令规定了固定且往往很复杂的数据布局，必须在整条执行流水线中被正确地传递，才能同时保证正确性与最优性能。本文介绍 CuTe——一套用于表示和操作张量的新颖数学形式体系。CuTe 提出两项关键创新：(1) 层次化的布局表示，它是对传统扁平形状（flat shape）与扁平步幅（flat stride）张量表示的直接推广，能够表达现代硬件指令所需的复杂映射；(2) 一套丰富的布局操作代数——包括拼接（concatenation）、合并（coalescence）、复合（composition）、补（complementation）、切分（division）、分块（tiling）与求逆（inversion）——支持复杂的布局操作、推导、验证与静态分析。CuTe 布局为管理 GPU kernel 中的数据布局与线程排布提供了统一框架，而布局代数使得我们能够在编译期对布局性质进行强有力的推理，并简洁地表达通用张量变换。

在本文中我们将展示：与传统方法相比，CuTe 的抽象显著帮助软件开发，推动对体系结构规定布局的编译期验证，便于实现可泛化到大量应用场景的算法原语，并能简洁地表达现代专用张量指令所需的分块（tiling）与划分（partitioning）模式。

CuTe 已成功部署于生产系统，是 NVIDIA CUTLASS 库以及包括 CuTe DSL 在内的一系列相关工作的基础。我们同时给出数学基础与 Python 参考实现（PyCuTe），确立 CuTe 作为面向以张量为中心的计算时代的一种系统化、对布局敏感的编程方法。

1 引言与动机

现代 GPU 越来越多地针对以张量（tensor）为中心的计算进行优化，其驱动力来自深度学习与科学计算的需求。NVIDIA 的 Volta 架构（NVIDIA，2017）引入了 Tensor Core，使硬件能够直接高效地完成小矩阵乘法。这一能力在 Turing（NVIDIA，2018）与 Ampere（NVIDIA，2021）中得到扩展，提供了在 GPU 内存层次结构中进行结构化矩阵搬运的专用指令。Hopper（NVIDIA，2023）与 Blackwell（NVIDIA，2025）架构进一步推进了这一范式，引入了在全球内存与共享内存之间高效传输 5 阶（rank）张量的拷贝指令，并进一步扩展了 Tensor Core 的能力。GPU 峰值性能很大程度上取决于这些面向张量的硬件特性能否被充分发挥，能高效表示与操作这些张量的高性能编程模型也就随之出现。

现有及新兴的硬件表现如何，很大程度上取决于多维数据在多个层次化内存空间与多个层次化并行级别中怎样被存储与访问。数据存储布局（layout）历来通过决定内存访问发生的方式与时机来影响性能；而随着硬件指令的规模越来越大，其输入输出的布局被固定下来，布局也开始决定正确性。这些布局必须沿整条执行流水线传递下去，才能保证硬件指令被正确调用，并保证内存访问模式得到优化。

在本工作中，我们呈现 CuTe 的基础概念。CuTe 是一项针对 CUDA Tensors（即 Compute Unified Tensors）的规约，旨在为编写峰值性能的线性代数库提供基础构件。CuTe 的核心是两项创新：

- *一种新颖的张量布局表示*：CuTe 的形状（shape）、布局与张量天然是层次化的，由更小的嵌套实例构造而成。这种层次结构提供了表示现代张量指令所需复杂映射的手段，同时仍是 `BLAS`、`torch.tensor`、`numpy.ndarray` 与 MATLAB 等库中既有的扁平形状与扁平步幅（stride）表示的严格扩展。
- *一种定义在布局之上的新颖操作代数*：CuTe 布局支持一组丰富的操作，包括拼接（concatenation）、合并（coalescence）、复合（composition）、补（complementation）、切分（division）、分块（tiling）与求逆（inversion），它们

的结果都仍是新的 CuTe 布局。这些操作支撑现代张量指令所需的复杂划分（partitioning）、操纵、验证与推导。

CuTe 的布局表示给通用算法一个直观的框架来管理线程与数据，布局代数则给高性能线性代数 kernel 的开发一条表达力强的路径，用来操纵布局、生成新布局。两者能带来：

- 对复杂布局与划分的支持：CuTe 便于表示应用特定数据模式所需的精细布局，以及专用张量指令所需的复杂划分模式。
- 关注点分离：数据布局独立于算法逻辑进行声明，提升清晰性与模块化。
- 静态分析与优化：成熟的代数技术使我们能够依据体系结构约束对张量参数进行检查、重排与划分。

1.1 相关工作

CuTe 的动机源于支持高效张量收缩开发的需求：张量收缩是众多科学计算与机器学习应用的核心。计算一般张量收缩的传统方法依赖矩阵化，即在逻辑上或显式地重组张量数据，以便通过一系列对基础线性代数子程序（BLAS）库的调用来完成计算。

BLAS 为核心线性代数运算提供了高效且可移植的实现，并为广泛的体系结构提供了高度优化的版本（Blackford 等，2002）。在 BLAS 原语中，通用矩阵乘法（GEMM）例程无疑是全部科学计算与机器学习中被优化得最充分、使用最广泛的操作。

类 BLAS 库实例化软件（BLIS）框架（Van Zee 与 van de Geijn，2015）扩展了 GEMM，使其同时支持行模与列模中的非单位步幅，从而在不借助显式内存拷贝的情况下解决了处理不规则内存布局的部分挑战。BLAS 的跨步批量（strided-batched）GEMM 扩展进一步推广了该原语，使其能够应用于更多张量收缩（Shi 等，2016）。在对矩阵布局做更进一步的抽象时，CuTe 由此得到的启发是：在张量记号中使用多重索引，使任意张量收缩都能变换为规范化的 batched-GEMM 原语。

许多现有库都依赖张量，而它们几乎全都基于扁平形状与扁平步幅表示。在 Python 中，这包括 `numpy.ndarray` 与 `torch.tensor`：

```
>>> import numpy
>>> a = numpy.ndarray([3,7,5])
>>> a.dtype
dtype('float64')
>>> a.shape
(3, 7, 5)
>>> a.strides
(280, 40, 8)
```

```
>>> import torch
>>> a = torch.empty(3,7,5)
>>> a.dtype
torch.float32
>>> a.shape
torch.Size([3, 7, 5])
>>> a.stride()
(35, 5, 1)
```

在 C++ 中，则是 `std::mdspan`：

```
std::mdspan a = std::mdspan(data, 3, 7, 5);
a.extent(0);  // 3
a.extent(1);  // 7
a.extent(2);  // 5
a.stride(0);  // 35
a.stride(1);  // 5
a.stride(2);  // 1
```

CuTe 支持这些表示，并对其做严格扩展：推广到层次化的形状与步幅，以表示更复杂的布局、非整数步幅以及非整数的布局陪域。

对稠密张量表示的独立推广还包括 HeLayers（Aharoni 等，2023）、ThunderKittens（Spector 等，2024），以及 OpenAI 的 Triton 编译器（Tillet 等，2019）中所使用的 Linear Layouts（Zhou 等，2025）方法。ThunderKittens 为寄存器内存、共享内存、行主序/列主序块、由行主序/列主序子块组成的行主序/列主序块，以及为 warp 与线程规定的访问模式实现了种类繁多的定制类型。这些类型的编写考虑了体系结构布局需求与内存层次，但其表示范围局限于既有的类型与模式。Linear Layouts 基于 $\mathbb{F}_2$ 线性代数，为张量布局提供了更一般的表示，也为布局分析与生成开辟了途径。但 Linear Layouts 对 $\mathbb{F}_2$ 的严格依赖使其中一些算子难以被人工检视，并将该工作限制在 2 的幂形状与步幅上，这对许多应用来说是不可接受的。

另一些启发了本工作部分内容的布局分析方法同样依赖 $\mathbb{F}_2$ 线性代数，例如 Edelman 等（1994）、Cormen（1993）与 Bouverot-Dupuis 及 Sheeran（2023）的工作。这些工作给出了分析与算法生成的出发点，但受限于其撰写时的应用语境。

由于 CUTLASS v3（NVIDIA，2023）中 CuTe 的 C++ 实现是开源的并附有部分文档，它已在分析 CuTe 布局与 CuTe 代数操作的独立工作中被引用和使用。Bhaskaracharya 等（2025）在整数集关系（ISL）的语境下分析了 CuTe 与 Linear Layouts（Zhou 等，2025），但略去了能将 Linear Layouts 表示为 CuTe 布局的步幅抽象。LEGO（Tavakkoli 等，2025）在代码生成器中使用受限形式的 CuTe 布局与原始形式的 CuTe 复合来生成复杂索引。Carlisle 等（2026）则在范畴论的语境下分析了 CuTe 布局及作用于其上的若干操作。在本文中，我们打算对 CuTe 概念及其应用给出更明确、更形式化的论述，并提供 CuTe 的一个 Python 参考实现。纯 Python 的参考实现 PyCuTe（NVIDIA，2026）可在 https://github.com/NVlabs/CuTe 获取，它与本文的定义与后置条件一一对应；它不同于面向 CUDA kernel JIT 编译的 CuTe DSL（NVIDIA，2025），也不同于 CUTLASS 中的 C++ CuTe 头文件。

CuTe 的设计与概念已在若干应用中落地。例如，Graphene 张量编译器（Hagedorn 等，2023）用 CuTe 表示张量运算；Stream-K 算法（Osama 等，2023）的实现用了 CuTe 的 C++ 实现，NVIDIA CUTLASS v3 库（Kerr 等，2017）的开发也以它为核心。大型语言模型的先进实现中也一直在用它，FlashAttention 及其后续版本（Dao 等，2022、Dao，2023、Shah 等，2024）就是例子，CuTe 在深度学习前沿被广泛使用。一批相关编译器项目同样建在 CuTe 之上，包括 CuTe DSL（NVIDIA，2025）：一种面向线性代数应用、基于 Python、动态编译 CUDA 软件的 DSL。

1.2 规范化循环与循环变换

循环索引的显式计算是高性能线性代数 kernel 开发中的常见挑战。这些计算很难一次写对，维护起来更难。与其将数据访问信息与算法逻辑耦合在一起，我们更愿意用矩阵/向量坐标（coordinate）清晰地书写算法逻辑，而将数据访问模式抽象到数据布局之中。

为便于说明，我们首先定义本工作所发展的技术能够处理的循环嵌套类别。具体而言，我们把*标准循环形式*（standard loop form）定义为具有单个索引、从零开始、以常量为界、且每次迭代递增 1 的循环。

例如，考虑如下循环：

```
for (int m = 2; m <= 50; m += 3)
  A[m] = e(m);
```

该循环将 `A[2]`、`A[5]`、`A[8]`、... 设为纯表达式 `e(m)` 的结果。它可以变换为如下的规范化循环（canonical loop）：

```
for (int i = 0; i < 17; ++i)
  (A + 2)[3*i] = g(i);
```

这里，指针偏移了一个循环不变常量，循环步幅被归一化为 1，下界变换为零，上界紧致且不含端点，纯表达式变换为 `g(i) = e(3*i+2)`。现在，上例可以理解为遍历一个逻辑上的 17 元素向量：逻辑坐标以 3 为步幅，对基地址 `A + 2` 处的数据进行索引。该程序可用如下数据表示：

```
Accessor: A + 2
   Shape: 17
  Stride: 3
```

嵌套循环可以类似地处理。考虑如下的二维循环嵌套：

```
for (int n = 3; n < 43; n += 2)
  for (int m = 4; m <= 22; m += 5)
    A[p*m + q*n] = e(m,n);
```

它可以变换为规范化循环形式：

```
for (int j = 0; j < 20; ++j)
  for (int i = 0; i < 4; ++i)
    (A + 4*p + 3*q)[5*p*i + 2*q*j] = g(i,j);
```

有了规范化循环嵌套，变换后的循环也可视为遍历一个逻辑上的 4×20 矩阵：逻辑行坐标 i 以 $5p$ 为步幅，逻辑列坐标 j 以 $2q$ 为步幅，对基地址 `A + 4*p + 3*q` 处的数据进行索引。这可以用如下数据表示：

```
Accessor: A + 4p + 3q
   Shape: ( 4, 20)
  Stride: (5p, 2q)
```

一个关键观察是，4×20 矩阵也可以被解释为一个具有非均匀、半仿射步幅的 80 元素向量，用等价的规范化循环形式表达即为：

```
for (int k = 0; k < 80; ++k)
  (A + 4*p + 3*q)[5*p*(k%4) + 2*q*(k/4)] = f(k);
```

其中% 是取模运算，/ 是整数向下取整除法。这一变换是 2 维坐标 (i,j) 与 1 维坐标 k 之间的余字典序（colexicographical）双射 (i,j) = (k%4,k/4)。该双射与先前表示的形状等价，并可由其直接推导而来。因此，该形状表示既可以接受 2 维坐标，也可以接受 1 维坐标，为数据索引提供了一个灵活且与阶无关的框架。

规范化循环形式还为可证正确的循环变换提供了指导，这类变换经常出现在面向张量计算的优化编译器中。考虑最一般的规范化循环嵌套：

```
for (int in = 0; in < Nn; ++in)
  ...
    for (int i1 = 0; i1 < N1; ++i1)
      for (int i0 = 0; i0 < N0; ++i0)
        A[d0*i0 + d1*i1 + ... + dn*in] = e(i0,i1,...,in);
```

其中 $(N_0, N_1, \ldots, N_n)$ 是计算的“形状”，$(d_0, d_1, \ldots, d_n)$ 是访问模式的“步幅”，

```
Accessor: A
   Shape: (N0,N1,...,Nn)
  Stride: (d0,d1,...,dn)
```

由于 形状 : 步幅 信息与循环嵌套本身之间存在一一对应，与其问“如何对循环施加变换：切分、转置、拼接、置换、截断、向量化等”，不如问：“形状 : 步幅 表示的合法变换方式有哪些，又有哪些算子能提供这些变换？”事实上，若 $\mathbf{L} =$ 形状 : 步幅 表示数据访问与循环嵌套，那么存在哪些函数 P，使得

$$\mathbf{L}' = P(\mathbf{L}) = \mathbf{L} \circ P$$

构成从 $\mathbf{L} =$ 形状 : 步幅 到新循环嵌套 $\mathbf{L}' =$ 形状$'$: 步幅$'$（可能具有新的形状与步幅）的一个有意义的变换？这些变换 P 本质上是对循环嵌套的重写，若定义得当，它们自身可以可复合、可求逆，并为命令式循环提供类似函数式编程的控制。结合上文讨论的 1 维坐标与 N 维坐标之间的双射，本文将证明：这些变换的一种有效的表示是 $P = \mathbf{P} =$ 形状* : 步幅*，与表示数据访问与循环嵌套自身的对象同类。

1.3 张量与折叠

为了进一步阐述除 1 维坐标之外还能以 N 维坐标索引的形状这一动机，我们推广 Shi 等（2016）中将张量收缩映射到规范化类 BLAS 原语的观察。

在本工作中，张量用粗体字母表示，索引用小写字母表示，索引的界用对应的大写字母表示。张量的阶指它所拥有的维度（也称为模（mode））的数量。例如：

- 标量 α 是 0 阶张量。
- 向量 $\mathbf{a}_i$ 是 1 阶张量，其中 $0 \le i < I$。
- 矩阵 $\mathbf{A}_{mn}$ 是 2 阶张量，其中 $0 \le m < M$ 且 $0 \le n < N$。
- 三路数组 $\mathbf{A}_{mnp}$ 是 3 阶张量，其中 $0 \le m < M$、$0 \le n < N$ 且 $0 \le p < P$。

对仅在等式单侧出现的重复索引隐含求和（爱因斯坦记号），因此张量收缩的一个实例是

$$\mathbf{C}_{stqp} = \mathbf{A}_{stupr}\, \mathbf{B}_{qtru}, \tag{W.1}$$

它表示一个 5 阶张量与一个 4 阶张量收缩生成一个 4 阶张量。这类收缩在 numpy.einsum 与 torch.einsum 等接口中有紧凑的表达。

上面的张量收缩可以重写为

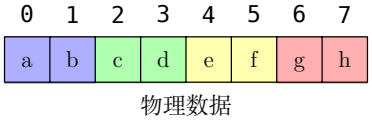
$$\mathbf{C}_{(sp)(q)(t)} = \mathbf{A}_{(sp)(ur)(t)}\,\mathbf{B}_{(q)(ur)(t)},$$

其中原张量收缩的各模被归为四类：

- 行模（row modes）$\widehat{m}$：出现在 **A** 与 **C** 中，而不出现在 **B** 中。
- 列模 $\widehat{n}$：出现在 **B** 与 **C** 中，而不出现在 **A** 中。
- 归约模 $\widehat{k}$：出现在 **A** 与 **B** 中，而不出现在 **C** 中。
- 批模 $\widehat{\ell}$：出现在 **A**、**B** 与 **C** 中。

这被称为张量折叠（folding）。折叠一个张量并不需要任何显式拷贝，可以只是数据视图的变更。

作为张量折叠的一个显式例子，考虑图 W.1 第一行所示的 8 元素 2×2×2 张量。扁平表示为张量的每个模保存一个形状与一个步幅，用以索引物理数据。这种扁平表示与 C++ 的 `std::mdspan`、PyTorch 的 `torch.tensor`、NumPy 的 `numpy.ndarray` 以及许多其它类似库所采用的表示完全相同。如第二行所示，通过将第三个模折叠进第一个模，该 2×2×2 张量可以折叠成 4×2 矩阵。其结果可用形状为 $(4,2)$、步幅为 $(2,1)$ 的扁平表示。原则上，通过将第三个模折叠进第二个模，该 2×2×2 张量也可以折叠成 2×4 矩阵。然而其结果不再具有扁平表示：不存在能表示第二个模的步幅的整数。



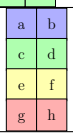
视图描述	视图图示	扁平表示	CuTe 表示
张量，2×2×2 3 阶		形状：$(2,2,2)$ 步幅：$(2,1,4)$	形状：$(2,2,2)$ 步幅：$(2,1,4)$
矩阵，4×2 将模 2 折叠进模 0		形状：$(4,2)$ 步幅：$(2,1)$	形状：$((2,2),2)$ 步幅：$((2,4),1)$
矩阵，2×4 将模 2 折叠进模 1		形状：$(2,4)$ 步幅：$(2,\times)$	形状：$(2,(2,2))$ 步幅：$(2,(1,4))$

图 W.1：将一个 8 元素的物理数组视为 3 阶张量，并折叠为两个矩阵。

折叠后矩阵的 CuTe 表示见最后一列，它强调张量折叠实际上只是把模分组。对 4×2 矩阵而言，扁平表示称为 CuTe 表示的合并版本；而对 2×4 矩阵，这样的扁平表示并不存在。

这种一般化的张量折叠形式使所有张量收缩都能写成单一的规范收缩形式：

$$\mathbf{C}_{\widehat{m}\widehat{n}\widehat{\ell}} = \mathbf{A}_{\widehat{m}\widehat{k}\widehat{\ell}}\mathbf{B}_{\widehat{n}\widehat{k}\widehat{\ell}}. \tag{W.2}$$

其中每个模可以是单个模，也可以是一组模，我们称之为多模（multi-mode）。援引第 1.2 节的规范化循环：无论 $\widehat{m}$ 的形状是 M 还是 (M_0, M_1)，我们都可以用 1 维坐标 m 对其循环。于是，任何张量收缩都可以折叠为规范化的 `batched-GEMM`，并用由四重嵌套循环组成的朴素参考实现求值：

```
for (int l = 0; l < L; ++l)
  for (int m = 0; m < M; ++m)
    for (int n = 0; n < N; ++n)
      for (int k = 0; k < K; ++k)
        C(m,n,l) += A(m,k,l) * B(n,k,l);
```

只要构造折叠布局，这个简单的 `batched-GEMM` 实现便可用于求值范围广泛、相互兼容的张量收缩，包括任意矩阵乘法（GEMM）、张量收缩（GETT）与卷积（CONV）。通用 GEMM 及其应用的细节见第 2.6 节。

随后，优化便可专注于循环重排、分块、向量化这些常见手段，做法都是变换循环嵌套的顺序与阶。这些变换往往可以用布局表示为算法坐标空间上的置换。这些变换布局与数据布局进行函数式复合，生成新的循环嵌套，并保证与原始问题一致。布局复合及其在通用划分中的应用细节见第 3.3 节。

2 布局表示（Layout Representation）

CuTe 布局（layout）是一类功能多样的对象，能够表示范围广泛的数据排列与线程排列，在抽象物理地址、将迭代顺序与存储顺序相分离等方面很有实用价值。在本节中，我们定义形状（shape）、布局与张量（tensor）的 CuTe 表示，并说明它们与坐标（coordinate）如何相互作用。CuTe 布局使得泛型算法只需单一实现，便可应用于任何可折叠为该算法规范形式（canonical form）的复杂数据布局。此类算法及其应用广度的示例见第 2.6 节。

2.1 元组与层次元组（Tuple 与 HTuple）

`Tuple`（元组）与 `HTuple`（层次元组）概念是贯穿本工作的基础数据结构。

定义 W.1. $\texttt{Tuple}(\mathcal{T})$ 是从集合 $\mathcal{T}$ 中选取元素构成的有限有序列表。对于 $X = (X_0, X_1, \ldots, X_{n-1}) \in \texttt{Tuple}(\mathcal{X})$，我们定义如下运算：

- **阶（rank）：** $\mathrm{rank}(X)$。即元组长度 n。
- **访问（access）：** X_i。即 `Tuple` X 的第 `i` 个元素，其中 $0 \le i < \mathrm{rank}(X)$。

$\texttt{Tuple}(\mathcal{T})$ 是元素的扁平（flat）集合，而 $\texttt{HTuple}(\mathcal{T})$ 则是“由 $\mathcal{T}$ 组成的层次元组”。

定义 W.2. $\texttt{HTuple}(\mathcal{T})$ 或者是集合 $\mathcal{T}$ 中的一个元素，或者是一个 $\texttt{Tuple}(\texttt{HTuple}(\mathcal{T}))$。对于 $X \in \texttt{HTuple}(\mathcal{X})$，我们定义如下运算：

- **阶（rank）：** $\mathrm{rank}(X)$。若 $X \in \texttt{Tuple}$，则为元组长度，否则为 1。
- **访问（access）：** X_i。即 `HTuple` X 的第 `i` 个元素，其中 $0 \le i < \mathrm{rank}(X)$。
- **深度（depth）：** $\mathrm{depth}(X)$。若 $X \in \texttt{Tuple}$，则为 $1 + \max(\mathrm{depth}(X_0), \mathrm{depth}(X_1), \ldots)$，否则为 0。

例如，

$$31 \qquad (16, 32) \qquad (3, -8, 7) \qquad (2, (4, 1), -1) \qquad ((4, 6), (3, (2, 2), 8))$$

均为 $\texttt{HTuple}(\mathbb{Z})$ 的实例。

在对 `HTuple` 进行推理时，定义同余（congruence）与弱同余（weak congruence）的概念会很有用。

定义 W.3. 同余 $\sim$ 是 `HTuple` 上的一个等价关系。对于 $P \in \texttt{HTuple}(\mathcal{P})$ 与 $S \in \texttt{HTuple}(\mathcal{S})$，

$$P \sim S \quad \text{当且仅当} \quad \begin{cases} P \in \mathcal{P} \text{ 且} S \in \mathcal{S}, \text{ 或} \\ P, S \in \texttt{Tuple} \text{ 且} \mathrm{rank}(P) = \mathrm{rank}(S) \text{ 且} \forall_i\ P_i \sim S_i \end{cases}$$

此时我们称 P 与 S 同余，并具有相同的轮廓（profile）。

例如，

$$(4, 8) \sim (5, 7) \quad \text{且} \quad (4, (2, 4)) \sim (7, (3, 2)) \quad \text{且} \quad (\mathbf{v}, ((\mathbf{p}, 3))) \sim (0, ((0, 0)))$$

但 $(4, 8)$ 与 $(4, (2, 4))$ 不同余，$(4, (2, 4))$ 与 $(0, ((0, 0)))$ 也不同余。

类似地，弱同余检验的是一个 `HTuple` 的轮廓是否至少与另一个一样精细。

定义 W.4. 弱同余 $\lesssim$ 是 `HTuple` 上的一个偏序。对于 $P \in \texttt{HTuple}(\mathcal{P})$ 与 $S \in \texttt{HTuple}(\mathcal{S})$，

$$P \lesssim S \quad \text{当且仅当} \quad \begin{cases} P \in \mathcal{P}, \text{ 或} \\ P, S \in \texttt{Tuple} \text{ 且} \mathrm{rank}(P) = \mathrm{rank}(S) \text{ 且} \forall_i\ P_i \lesssim S_i \end{cases}$$

此时我们称 P 与 S 弱同余，P 粗化（coarsens）S 的轮廓，且 S 细化（refines）P 的轮廓。

例如，

$$30 \lesssim (a, b) \lesssim (\mathbf{v}, (0, \alpha)) \quad \text{且} \quad 30 \lesssim (a, b, c) \lesssim ((0, 0), 0, 0)$$

但 (a, b) 与 (a, b, c) 不弱同余，$(\mathbf{v}, (0, \alpha))$ 与 $((0, 0), 0)$ 也不弱同余。

2.2 形状（Shape）

多维数组通常由其形状（shape）来刻画：一列描述各个模（mode）规模的正整数。$M\times N$ 矩阵的二维形状表示为 (M,N)，并由坐标 (m,n) 索引，其中 $0\le m<M$ 且 $0\le n<N$。对此的一个自然推广，是将形状表示为由正整数构成的层次元组。

定义 W.5. 形状是一个 `HTuple`($\mathbb{Z}^+$)，其中 $\mathbb{Z}^+=\{1,2,3,\ldots\}$ 为正整数集合。形状 S 的阶（rank）是该 `HTuple` 的阶。形状 S 的大小（size）是其全部元素的乘积，记作 $|S|=\prod_k|S_k|$。

层次化形状之所以特别有用，在于它们可以由多种坐标系索引。考虑不超过 N 的整数集合

$$\mathbb{Z}_N=\{0,1,2,\ldots,N-1\}.$$

CuTe 的关键观察是：只要存在一个双射（bijection）

$$S:\mathbb{Z}_{MN}\longleftrightarrow\mathbb{Z}_M\times\mathbb{Z}_N$$

在一维整型坐标（integral coordinate）$i\in\mathbb{Z}_{MN}$ 与二维自然坐标（natural coordinate）$(m,n)\in\mathbb{Z}_M\times\mathbb{Z}_N$ 之间建立映射，二维形状 (M,N) 也可以解释为描述 MN 个一维元素，并由整型坐标 i（$0\le i<MN$）索引。

类似地，二维形状 (M,NP) 可以解释为层次形状 $(M,(N,P))$，由自然坐标 $(m,(n,p))$ 索引，其中 $0\le m<M$、$0\le n<N$、$0\le p<P$。相应的双射为

$$S:\mathbb{Z}_M\times\mathbb{Z}_{NP}\longleftrightarrow\mathbb{Z}_M\times(\mathbb{Z}_N\times\mathbb{Z}_P)$$

在二维坐标 $(m,q)\in\mathbb{Z}_M\times\mathbb{Z}_{NP}$ 与自然坐标 $(m,(n,p))\in\mathbb{Z}_M\times(\mathbb{Z}_N\times\mathbb{Z}_P)$ 之间建立映射。

层次形状与坐标的一个直接推论是：张量算法可以按照对它们而言最自然的形状来编写（第 2.6 节）：`COPY` 中的向量用一维形状，`GEMM` 中的矩阵用二维形状，`batched-GEMM` 中的张量用三维形状，等等，同时仍能接受被折叠（fold）成与算法的规范形式弱同余的层次化形状张量（第 2.2 节）。数据张量的形状通常表示为一列扁平的整数，它可以任意折叠成泛型张量算法所接受的形状。另外，由于张量的每个模都关联着一个用于索引数据的步幅（stride）（第 2.3 节），这种模的折叠使得我们能够表示远比 `COPY` 中的简单连续数组或 `BLAS GEMM` 中的行主序、列主序矩阵复杂得多的数据布局（第 2.4 节）。

下一小节定义这种相容性（compatibility）的概念，以及形状内各坐标集合之间的这些关系。

坐标集合与相容性（Coordinate Sets and Compatibility）

层次形状支持由多种坐标系索引。这里，我们为特定形状定义坐标集合（coordinate set），并定义形状之间的相容性概念，以便在形状之间共享坐标集合。

定义 W.6. 坐标集合是一个由非负整数构成的集合 $\mathbb{Z}_N=\{0,1,2,\ldots,N-1\}$，或坐标集合的笛卡尔积 $\mathbb{Z}_N\times\mathbb{Z}_M=\mathbb{Z}_{(N,M)}$。

下面是几个坐标集合：

$$\begin{aligned}\mathbb{Z}_6&=\{0,1,2,3,4,5\}\\ \mathbb{Z}_3\times\mathbb{Z}_4=\mathbb{Z}_{(3,4)}&=\{(0,0),(1,0),(2,0),(0,1),(1,1),(2,1),(0,2),(1,2),(2,2),(0,3),(1,3),(2,3)\}\\ (\mathbb{Z}_2\times\mathbb{Z}_1)\times\mathbb{Z}_3=\mathbb{Z}_{((2,1),3)}&=\{((0,0),0),((1,0),0),((0,0),1),((1,0),1),((0,0),2),((1,0),2)\}\end{aligned}$$

坐标集合 $\mathbb{Z}_S$ 恰好是形状 S 的自然坐标集合。形状 S 的其它坐标集合，是任何与 S 相容（compatible）且粗化（coarsens）S 的形状的坐标集合。

定义 W.7. 相容性 $\preceq$ 是形状集合上的一个偏序。对于形状 P 与 S，

$$P\preceq S\quad\text{当且仅当}\quad\begin{cases}P\in\mathbb{Z}^+\text{ 且}P=|S|\text{，或}\\ P,S\in\texttt{Tuple}\text{ 且}\mathrm{rank}(P)=\mathrm{rank}(S)\text{ 且}\forall_i\ P_i\preceq S_i\end{cases}$$

此时我们称 P 与 S 相容，P 粗化 S，且 S 细化 P。

相容性要求两个形状大小相同，因此 `HTuple` 中的整数值是有影响的。例如，

$$30 \preceq (2,15) \preceq (2,(3,5)) \quad \text{且} \quad 30 \preceq (6,5) \preceq ((3,2),5)$$

但 $(2,(3,5))$ 与 $((3,2),5)$ 尽管大小相同，却并不相容。不过，它们确实共享一个公共的相容形状 30。

有了坐标集合与形状相容性的定义，我们便可以为任意给定形状定义全部相容坐标构成的集合。

定义 W.8. 形状 S 定义了一个相容坐标集合的集合 $\mathbb{Z}(S)$，即所有粗化 S 的形状的坐标集合。

$$\mathbb{Z}(S) = \{\mathbb{Z}_{S'} \mid S' \preceq S\}. \tag{W.3}$$

每个形状都有一个整型坐标集合，

$$\{0,1,2,\ldots,|S|-1\} = \mathbb{Z}_{|S|} \in \mathbb{Z}(S),$$

且每个 r 阶形状都有一个 r 阶坐标集合，

$$\{(a_0,\ldots,a_{r-1}) \mid a_i \in \mathbb{Z}_{|S_i|}\} = \mathbb{Z}_{(|S_0|,|S_1|,\ldots,|S_{r-1}|)} \in \mathbb{Z}(S).$$

注意，若形状 P 粗化形状 S，则 $\mathbb{Z}(P) \subseteq \mathbb{Z}(S)$。这意味着形状 P 内的任何坐标也是形状 S 内的坐标。

坐标（Coordinates）

在本小节中，我们定义若干类坐标，在形状的各相容坐标集合之间定义双射，并给出这些坐标映射的示例。

定义 W.9. 形状 S 的界内坐标(in-bounds coordinate)，或简称坐标，是其某个坐标集合中的一个元素，即 $c \in \mathbb{Z}_{S'} \in \mathbb{Z}(S)$。注意，坐标总是一个 `HTuple`($\mathbb{N}$)。当意图明确时，我们将简写为 $c \in \mathbb{Z}(S)$。

定义 W.10. 形状 S 的整型坐标（integral coordinate）是坐标 $\bar{c} \in \mathbb{Z}_{|S|} \in \mathbb{Z}(S)$。注意，整型坐标总是一个整数，即 $\bar{c} \in \mathbb{N}$。

定义 W.11. 形状 S 的自然坐标（natural coordinate）是坐标 $\widetilde{c} \in \mathbb{Z}_S \in \mathbb{Z}(S)$。注意，自然坐标总是一个与形状同余的 `HTuple`($\mathbb{N}$)，即 $\widetilde{c} \sim S$。

为了在界内坐标之间进行转换，我们在形状 S 的各坐标集合上构造一个枚举，以定义坐标列表（coordinate list）。在本工作中，我们选择坐标的余字典序（colexicographical ordering）$<$，其定义如下：

$$(a_0,\ldots,a_n) < (b_0,\ldots,b_n) \quad \text{当且仅当} \quad \begin{cases} a_n < b_n, \text{ 或} \\ a_n = b_n \ \text{ 且 } \ (a_0,\ldots,a_{n-1}) < (b_0,\ldots,b_{n-1}) \end{cases}$$

并在需要时递归应用。余字典序枚举在坐标列表上定义了一个双射。函数

$$\begin{aligned} &\texttt{idx2crd}\colon \mathbb{Z}_{|S|} \to \mathbb{Z}_{(|S_0|,|S_1|,\ldots,|S_{r-1}|)}, \\ &\qquad i \mapsto \left(i \bmod |S_0|, \left\lfloor \frac{i}{|S_0|} \right\rfloor \bmod |S_1|, \ldots, \left\lfloor \frac{i}{\prod_{k=0}^{r-3}|S_k|} \right\rfloor \bmod |S_{r-2}|, \left\lfloor \frac{i}{\prod_{k=0}^{r-2}|S_k|} \right\rfloor \right) \end{aligned} \tag{W.4}$$

将 $\mathbb{Z}_{|S|}$ 的第 i 个坐标（形状 S 的第 i 个整型坐标）映射为 $\mathbb{Z}_{(|S_0|,|S_1|,\ldots,|S_{r-1}|)}$ 的第 i 个坐标（形状 $(|S_0|,|S_1|,\ldots,|S_{r-1}|)$ 的第 i 个自然坐标）。其它双射，例如反向和/或镜像的字典序或余字典序，同样可以使用。

`idx2crd` 的逆由下式给出

$$\begin{aligned} &\texttt{crd2idx}\colon \mathbb{Z}_{(|S_0|,|S_1|,\ldots,|S_{r-1}|)} \to \mathbb{Z}_{|S|}, \\ &\qquad (c_0,c_1,\ldots,c_{r-1}) \mapsto c_0 + c_1 \cdot |S_0| + \ldots + c_{r-1} \cdot \prod_{k=0}^{r-2} |S_k| \end{aligned} \tag{W.5}$$

它将 $\mathbb{Z}_{(|S_0|,|S_1|,\ldots,|S_{r-1}|)}$ 的第 i 个坐标映射为 $\mathbb{Z}_{|S|}$ 的第 i 个坐标。

若两个形状相容，即 $P \preceq S$，则 $\mathbb{Z}_P$ 中的坐标可以通过反复应用 `idx2crd` 映射到 $\mathbb{Z}_S$，而 $\mathbb{Z}_S$ 中的坐标可以通过反复应用 `crd2idx` 映射到 $\mathbb{Z}_P$。按照坐标的余字典序，图 W.2 列出了每个形状从整型坐标到自然坐标的映射。

$\mathbb{Z}_4$
0
1
2
3

$S = 4$
$\mathbb{Z}(S) = \{\mathbb{Z}_4\}$

$\mathbb{Z}_6$	$\mathbb{Z}_{(2,3)}$
0	$(0,0)$
1	$(1,0)$
2	$(0,1)$
3	$(1,1)$
4	$(0,2)$
5	$(1,2)$

$S = (2,3)$
$\mathbb{Z}(S) = \{\mathbb{Z}_6, \mathbb{Z}_{(2,3)}\}$

$\mathbb{Z}_{12}$	$\mathbb{Z}_{(6,2)}$	$\mathbb{Z}_{((2,3),2)}$
0	$(0,0)$	$((0,0),0)$
1	$(1,0)$	$((1,0),0)$
2	$(2,0)$	$((0,1),0)$
3	$(3,0)$	$((1,1),0)$
4	$(4,0)$	$((0,2),0)$
5	$(5,0)$	$((1,2),0)$
6	$(0,1)$	$((0,0),1)$
7	$(1,1)$	$((1,0),1)$
8	$(2,1)$	$((0,1),1)$
9	$(3,1)$	$((1,1),1)$
10	$(4,1)$	$((0,2),1)$
11	$(5,1)$	$((1,2),1)$

$S = ((2,3),2)$
$\mathbb{Z}(S) = \{\mathbb{Z}_{12}, \mathbb{Z}_{(6,2)}, \mathbb{Z}_{((2,3),2)}\}$

图 W.2: 各形状 S 的坐标列表集合示例。

越界坐标（Out-of-bounds Coordinates） 除已定义的坐标集合之外，定义一些可能不在形状坐标集合之内、具有特定轮廓的坐标也很有用。

定义 W.12. 形状 S 的可容许坐标（admissible coordinate）是任意与该形状弱同余的坐标 $c \in \texttt{HTuple}(\mathbb{Z})$，即 $c \lesssim S$。

定义 W.13. 形状 S 的越界坐标(out-of-bounds coordinate)是任意不在界内的可容许坐标 $c \in \texttt{HTuple}(\mathbb{Z})$, 即 $c \notin \mathbb{Z}(S)$。

定义 W.14. 形状 S 的同余坐标（congruent coordinate）是任意与该形状同余的坐标 $c \in \texttt{HTuple}(\mathbb{Z})$，即 $c \sim S$。将其记作 $\mathbb{Z}^S = \{c \in \texttt{HTuple}(\mathbb{Z}) \mid c \sim S\}$。

也就是说，$\mathbb{Z}_S$ 是由形状 S 界定的有限坐标集合，而 $\mathbb{Z}^S$ 是与 S 同余的全部坐标构成的无限集合。由于对 $\mathbb{Z}^S$ 而言，形状 S 内部的具体取值并不重要，我们有时会用 $(*,*)$ 作为占位轮廓（placeholder profile）。例如，

$$\mathbb{Z}^{(*,*)} = \{(a,b) \mid a,b \in \mathbb{Z}\} \quad \text{且} \quad \mathbb{Z}^{(*,(*,*))} = \{(a,(b,c)) \mid a,b,c \in \mathbb{Z}\}$$

注意，`idx2crd` 对所有整数（等价地说，对 $\mathbb{Z}^{|S|}$ 中的坐标）都有良定义，而不仅限于 $\mathbb{Z}_{|S|}$ 中的整数。当它在某个整数 $i \geq |S|$ 上求值时，它总会返回一个相对于形状 $(|S_0|, |S_1|, \ldots, |S_{r-1}|)$ 越界的坐标 $(c_0, c_1, \ldots, c_{r-1})$。相比之下，`crd2idx` 无法保证对越界坐标的输入给出越界的结果。因此，`crd2idx` 与 `idx2crd` 只有在界内坐标上求值时才互为逆。

2.3 步幅（Stride）

上一节描述了形状及其层次结构，以及这些形状的坐标。为了构造数据、线程或其它对象的**布局**，我们定义一个从形状内坐标到偏移（offset）的映射。

定义 W.15. 形状 S 的步幅（stride）D 是一个与该形状同余的 $\texttt{HTuple}(\mathcal{D})$，即 $S \sim D$。该步幅定义了一个从自然坐标 $\widetilde{c} \in \mathbb{Z}_S$ 到陪域（codomain）$\mathcal{D}$ 的映射，由下式给出：

$$\begin{aligned}
&\texttt{inner_product}\colon \mathbb{Z} \cdot \mathcal{D} \to \mathcal{D}, \\
&\qquad\qquad c \cdot d \mapsto cd \\
&\texttt{inner_product}\colon \texttt{HTuple}(\mathbb{Z}) \cdot \texttt{HTuple}(\mathcal{D}) \to \mathcal{D}, \\
&\qquad\qquad c \cdot d \mapsto \sum_i \texttt{inner_product}(c_i, d_i)
\end{aligned} \tag{W.6}$$

在大多数情况下，步幅也是 $\texttt{HTuple}(\mathbb{Z})$，即 $\mathcal{D} = \mathbb{Z}$。`inner_product` 所产生的整数通常被解释为数据数组内的偏移。然而，步幅元素的概念可以推广为整数半模（integer-semimodule）中的任意元素，布局能表示的函数范围也宽得多。

整数半模（Integer-Semimodules）

定义 W.16. 整数半模（integer-semimodule）是一个配备了结合加法 $M+M\to M$ 与标量乘法 $\mathbb{Z}\cdot M\to M$ 的集合 M。对于 $a,b\in\mathbb{Z}$ 与 $m,n,p\in M$，加法与标量乘法满足

- 乘法单位元：$1\cdot m=m$，
- 加法结合律：$m+(n+p)=(m+n)+p$，
- 乘法结合律：$a\cdot(b\cdot m)=(ab)\cdot m$。

不要求存在加法单位元与加法逆元，因此 $(M,+)$ 是一个半群（semigroup）。我们将整数半模记作 $(M,+,\cdot)$，用以表示集合 M、加法运算 $+$ 与标量乘法 $\cdot$。

整数集 $\mathbb{Z}$ 是一个整数半模。有理数集 $\mathbb{Q}$ 是一个整数半模。由模 2 算术运算构成的域 $\mathbb{F}_2=(\{0,1\},\mathrm{XOR},\mathrm{AND})$ 是一个整数半模。在逐元素加法与标量乘法之下，任意整数半模的笛卡尔积或 `HTuple` 都是整数半模。

一个格外有用的整数半模是 $(\mathbb{Z}^S,+,\cdot)$，其中 $\mathbb{Z}^S$ 是与 S 同余的全部 `HTuple`$(\mathbb{Z})$ 构成的集合。例如，2 阶算术元组的基元素构成一个整数半模：

$$e_0=(1,0),\quad e_1=(0,1),\quad \mathbb{Z}^{(*,*)}=\{a\cdot e_0+b\cdot e_1\ \mid\ a,b\in\mathbb{Z}\}$$

其整数缩放与加法按元素定义：

$$a\cdot e_0+b\cdot e_1=(a,b)$$

因此，e_0、e_1 以及任意线性组合都可以用作布局中的步幅。通过从 $\mathbb{Z}^S$ 中选取步幅元素，布局可以经由 `inner_product` 运算生成形状 S 的自然坐标。

2.4 布局

CuTe 使用形状（shape）S 与步幅（stride）D 来定义一个*布局函数*（layout function），或简称*布局*（layout）。形状 S 定义布局函数的定义域（domain），而步幅 D 定义布局函数的陪域（codomain）。

对前文所述形状的另一种解释是：它是从所有坐标（coordinate）列表的集合到自然坐标的一个映射。该映射是双射，因此每个坐标都被映射到形状内一个唯一且等价的自然坐标。类似地，步幅是从某个形状的自然坐标到某个陪域的映射。

$$S\ :\ Z\leftrightarrow\mathbb{Z}_S,\ \forall Z\in\mathbb{Z}(S)$$
$$D\ :\ \mathbb{Z}_S\to\mathcal{D}$$

其中 $\leftrightarrow$ 是 `idx2crd` (W.4) 与 `crd2idx` (W.5) 这两个函数，它们在相容形状的自然坐标之间进行映射；而 $\to$ 映射则是自然坐标与步幅之间的 `inner_product` (W.6)。

形状与步幅的复合（composition）定义了布局函数，它是从所有坐标列表的集合到陪域的映射。

定义 W.17. *布局* $\mathbf{L}=D\circ S$ 是形状 S 与步幅 D 的函数复合，其中 $S\sim D$，它为每个 $Z\in\mathbb{Z}(S)$ 定义了一个映射 $Z\to\mathcal{D}$。

记号与运算

我们依据上下文以不同的记号书写布局 $\mathbf{L}$。例如：

$$\begin{matrix}(4,&(3,&2))\\(2,&(8,&1))\end{matrix}=\begin{matrix}S\\D\end{matrix}\qquad\text{或}\qquad(4,(3,2)):(2,(8,1))=S:D\qquad\text{或}\qquad(2,(8,1))\circ(4,(3,2))=D\circ S$$

其中最后一种写法强调：形状与步幅本身也可以解释为函数，它们复合起来定义布局函数。

由于布局的定义域由其形状决定，布局的性质与形状的性质一一对应。对于布局 $\mathbf{L}=S:D$ 与 $\mathbf{U}=X:Y$，我们定义：

- 阶$(\mathbf{L})=$阶(S)：布局的阶（rank）是其形状的阶。
- 深度$(\mathbf{L})=$深度(S)：布局的深度（depth）是其形状的深度。
- $|\mathbf{L}|=|S|$：布局的大小是其形状的大小。
- $\mathbf{L}_i=S_i:D_i$：第 i 个子布局（sublayout）由其形状与步幅的第 i 个元素构成。

- $\mathbb{Z}(\mathbf{L}) = \mathbb{Z}(S)$：布局的坐标集合是其形状的坐标集合。
- $\mathbf{L} \sim \mathbf{U} \Leftrightarrow S \sim X$：两个布局的同余（congruent）关系是其形状的同余关系。
- $\mathbf{L} \preceq \mathbf{U} \Leftrightarrow S \preceq X$：两个布局的相容性是其形状的相容性。

作为布局求值的一个例子，考虑图 W.3e 中的布局 $\mathbf{L} = ((2,2),(4,2)) : ((1,8),(2,16))$ 以及整数坐标 $22 \in Z_{32} \in Z(\mathbf{L})$，则

$$\mathbf{L}(22) = \mathbf{L}(2,5) = \mathbf{L}((0,1),(1,1)) = 26$$

其中我们依次展示了整数坐标、等价的二维坐标、等价的自然坐标以及计算得到的偏移。

布局的界内定义域是所有坐标 $c \in \mathbb{Z}(\mathbf{L})$ 的集合。布局也可以在越界坐标上求值，这类似于可以在越界索引上求值但行为未定义的数组。因此，布局的定义域是有限集合 $\mathbb{Z}(\mathbf{L})$，而扩展定义域（extended domain）是与 S 弱同余的所有 $\texttt{HTuple}(\mathbb{Z})$ 构成的无穷集合 $\mathbb{Z}^{\mathbf{L}}$。

我们还区分布局的陪域与像集。布局的陪域是 $\mathcal{D}$，它通常是无穷的（例如 $\mathbb{Z}$ 或 $\mathbb{Z}^D$）。布局的像集是有限的，是布局对其定义域内所有坐标所产生的取值范围。

$$像(\mathbf{L}) = \mathbf{L}(\mathbb{Z}(\mathbf{L})) = \mathbf{L}(\mathbb{Z}_{|\mathbf{L}|}) \subseteq 陪域(\mathbf{L})$$

布局示例

下面给出几个示例，说明 CuTe 布局是对常见扁平 N 维布局的严格推广。

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

(a) 列主序
$(4,8) : (1,4)$

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

(b) 行主序
$(4,8) : (8,1)$

0	5	10	15	20	25	30	35
1	6	11	16	21	26	31	36
2	7	12	17	22	27	32	37
3	8	13	18	23	28	33	38

(c) 列主序填充
$(4,8) : (1,5)$

0	1	2	3	16	17	18	19
4	5	6	7	20	21	22	23
8	9	10	11	24	25	26	27
12	13	14	15	28	29	30	31

(d) 列主序交错
$(4,(4,2)) : (4,(1,16))$

0	2	4	6	16	18	20	21
1	3	5	7	17	19	22	23
8	10	12	14	24	26	28	30
9	11	13	15	25	27	29	31

(e) 混合
$((2,2),(4,2)) : ((1,8),(2,16))$

0	0	4	4	8	8	12	12
0	0	4	4	8	8	12	12
2	2	6	6	10	10	14	14
2	2	6	6	10	10	14	14

(f) 块状广播
$((2,2),(2,4)) : ((0,2),(0,4))$

图 W.3: 与形状 $(4,8)$ 相容的布局示例，绘制为 4×8 矩阵。这些布局使用整数步幅与层次形状（hierarchical shapes）来定义对数据张量有用的函数。浅色线条沿布局偏移的总体顺序勾勒。

图 W.3 汇总了稠密线性代数库（如 CUTLASS）中常见的数据布局。每个布局都被表示为从逻辑坐标 $(m,n) \in \mathbb{Z}_{(4,8)}$ 到偏移 $k \in \mathbb{Z}$ 的映射。例如，这些偏移可用于索引数据数组中的元素。常见的行主序（row-major）、列主序（column-major）与填充（padded）布局都可以直接用 CuTe 布局表示，而交错（interleaved）与混合（mixed）布局则表明：通过使用嵌套的形状与步幅，布局的表示集合被严格地扩大。特别地，由数据块构成的网格（grids-of-tiles-of-data）可以直接用 CuTe 布局表示。

图 W.4 展示了用整数半模步幅而非整数步幅构造的布局示例。图 W.4a 与图 W.4b 演示了生成坐标的布局。在后续章节中我们将发现，这些坐标布局与其对应的数据布局对称地变换，并且常常作为检测数据张量越界访问并对其进行谓词化（predicating）的实用工具。另外，坐标张量对 Hopper 与 Blackwell 里的 TMA 这类以坐标而非地址作参数的指令也很有用。图 W.4c 使用了一个以二进制 XOR 替换群加法的整数半模。它可用于生成所谓的 swizzle 模式，这类模式有助于避免对共享内存读写访问模式中的 bank 冲突。这些例子说明 CuTe 的布局概念是统一的，它能表示的函数范围也很广。

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)

(a) 恒等坐标

$(4, 8) : (e_0, e_1)$

(0,0)	(1,0)	(2,0)	(3,0)	(0,6)	(1,6)	(2,6)	(3,6)
(0,1)	(1,1)	(2,1)	(3,1)	(0,7)	(1,7)	(2,7)	(3,7)
(0,2)	(1,2)	(2,2)	(3,2)	(0,8)	(1,8)	(2,8)	(3,8)
(0,3)	(1,3)	(2,3)	(3,3)	(0,9)	(1,9)	(2,9)	(3,9)

(b) 转置块状坐标

$(4, (4, 2)) : (e_1, (e_0, 6e_1))$

0	5	10	15	16	21	26	31	32	37	42	47
1	4	11	14	17	20	27	30	33	36	43	46
2	7	8	13	18	23	24	29	34	39	40	45
3	6	9	12	19	22	25	28	35	38	41	44

(c) 二进制 Swizzle

$(4, (4, 3)) : (f_1, (f_5, f_{16}))$

$f_i \in F_2 = (\mathbb{Z}, XOR, \cdot)$

图 W.4: 以二维坐标绘制的布局示例。这些布局使用来自整数半模（integer-semimodule）的步幅来定义函数，这些函数在跟踪坐标以及按非仿射（non-affine）模式对偏移做“混淆（swizzle）”处理时很有用。

完备性

任何满足 $f(0) = 0$ 且定义域为有限集合 $\mathbb{Z}_N$ 的函数 f 都可以表示为有限个 CuTe 布局的函数复合。这意味着 CuTe 布局在函数复合下构成一个生成集。这样的函数 f 可以表示为如下复合序列

$$f \equiv (2, 2, 2, \ldots) : (f(1), f(2), f(3), \ldots) \circ (3, 1) : (1, 4) \circ (4, 1) : (1, 6) \circ \cdots \circ (N-1, 1) : (1, 2(N-2)).$$

对所有 $i \in \mathbb{Z}_N/\{0\}$，最右边的 $N-3$ 个布局将 $i \to 2^{i-1}$，而最左边的布局将 $2^{i-1} \to f(i)$。注意，我们在上面求值中间布局时使用的是扩展定义域 $\mathbb{Z}$，而不是界内定义域 $\mathbb{Z}_N$。

半线性

形状—步幅定义 $\mathbf{L} = D \circ S$ 连同推广的整数模（integer-module）步幅，给出了布局函数一种特别有用的线性代数视角，

$$\mathbf{L}(c) = (D \circ S)(c) = d \cdot S(c) = d \cdot \widetilde{c}. \tag{W.7}$$

形状函数是到自然坐标 $\widetilde{c} \in \mathbb{Z}^S$ 的一个便利的半仿射双射，而步幅函数是自然坐标的线性函数。事实上，对于两个自然坐标 $\widetilde{c}_0, \widetilde{c}_1 \in \mathbb{Z}^S$，布局函数是线性（linear）的，

$$\mathbf{L}(\alpha\widetilde{c}_0 + \beta\widetilde{c}_1) = d \cdot (\alpha\widetilde{c}_0 + \beta\widetilde{c}_1) = \alpha(d \cdot \widetilde{c}_0) + \beta(d \cdot \widetilde{c}_1) = \alpha\mathbf{L}(\widetilde{c}_0) + \beta\mathbf{L}(\widetilde{c}_1),$$

因为形状函数是恒等映射且步幅函数是线性的。注意，对任意坐标 $c_0, c_1 \in \mathbb{Z}(S)$，布局不是线性的，因为形状函数不是线性的，

$$\widetilde{c} = S(c_0 + c_1) \neq S(c_0) + S(c_1) = \widetilde{c}_0 + \widetilde{c}_1.$$

因此，布局函数在自然坐标 $\widetilde{c} \in \mathbb{Z}^S$ 下是线性的，但在任意坐标 $c \in \mathbb{Z}(S)$ 下是非线性的。

在自然坐标下，$d \cdot \widetilde{c}$ 可以解释为推广的矩阵—向量乘积（matrix-vector product），

$$\mathbf{L}(c) = d \cdot \widetilde{c} = \mathbf{D}\ \widetilde{c}$$

其中 $\mathbf{D}$ 是元素取自某个整数半模 $\mathcal{D}$ 的矩阵。在最常见的整数步幅情形下，$\mathcal{D} = \mathbb{Z}$，这是 $\mathbf{D} \in \mathbb{Z}^{1\times n}$ 的矩阵—向量乘积。当步幅取自坐标整数半模 $\mathcal{D} = (\mathbb{Z}^S, +, \cdot)$ 时，这是 $\mathbf{D} \in \mathbb{Z}^{m\times n}$ 的矩阵—向量乘积。当步幅是二进制序列时，$\mathcal{D} = F_2^m = (\mathbb{Z}_{2^m}, XOR, \cdot)$，这是 $\mathbf{D} \in \mathbb{F}_2^{m\times n}$ 的矩阵—向量乘积。

例如，表 W.1 展示了一个整数布局、一个坐标布局与一个二进制布局的线性形式。针对 $\mathbb{F}_2$ 线性形式已被充分研究的变换包括位排列补（Bit Permute Complement，BPC）与位矩阵乘补（Bit Matrix Multiply Complement，BMMC）变

$\mathbf{L}$	线性形式：$r = \mathbf{D}\,\widetilde{c}$	说明
$((2,2),(4,\ 2)) : ((1,8),(2,16))$	$r = \begin{bmatrix}1 & 8 & 2 & 16\end{bmatrix} \begin{bmatrix}\widetilde{c}_0\\ \widetilde{c}_1\\ \widetilde{c}_2\\ \widetilde{c}_3\end{bmatrix}$	整数步幅是 $1\times n$ $\mathbb{Z}$-矩阵的列
$(4,(4,2)) : (e_1,(e_0,6e_1))$	$\begin{bmatrix}r_0\\ r_1\end{bmatrix} = \begin{bmatrix}0 & 1 & 0\\ 1 & 0 & 6\end{bmatrix} \begin{bmatrix}\widetilde{c}_0\\ \widetilde{c}_1\\ \widetilde{c}_2\end{bmatrix}$	坐标步幅是 $m\times n$ $\mathbb{Z}$-矩阵的列
$(4,4) : (f_1,f_5)$ $\equiv ((2,2),(2,2)) : ((f_1,f_2),(f_5,f_{10}))$	$\begin{bmatrix}r_0\\ r_1\\ r_2\\ r_3\end{bmatrix} = \begin{bmatrix}1 & 0 & 1 & 0\\ 0 & 1 & 0 & 1\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix} \begin{bmatrix}\widetilde{c}_0\\ \widetilde{c}_1\\ \widetilde{c}_2\\ \widetilde{c}_3\end{bmatrix}$	二进制步幅是 $m\times n$ $\mathbb{F}_2$-矩阵的列

表 W.1: 布局及其对应的线性形式。

换（Edelman 等，1994；Cormen，1993；Bouverot-Dupuis 与 Sheeran，2023），它们近来在 SIMT GPU 编程中获得了重要的应用（Zhou 等，2025）。在这些工作中，所考虑的变换为

$$f(\mathbf{v}) = \mathbf{A}\mathbf{v} + \mathbf{b} \tag{W.8}$$

其中 $\mathbf{A}$ 是一个 $m\times n$ 的二进制元素矩阵，$\mathbf{v}$ 是长度为 n 的二进制向量，$\mathbf{b}$ 是长度为 m 的二进制向量，且所有算术都以 2 为模（在只有两个元素的有限域 $\mathbb{F}_2$ 中）执行。

本文的后续章节将定义 CuTe 布局上的代数运算，这些运算是从线性代数推广而来的。CuTe 布局上的群复合可以解释为矩阵乘法的推广。CuTe 布局的右逆与左逆则对应线性代数中的 Moore-Penrose 伪逆。事实上，在 BPC 与 BMMC 分析中也可以找到类似的线性代数表达式（Edelman 等，1994；Cormen，1993；Bouverot-Dupuis 与 Sheeran，2023；Zhou 等，2025），它们都在算法分析中使用分解、逆与矩阵乘积。从这个角度看，CuTe 布局代数是线性代数的 BPC 与 BMMC 运算向 $\mathbb{F}_2$ 域之外的推广。特别地，CuTe 布局代数的主要动机来自张量异构数据布局中常见的一般整数步幅。在本工作中，我们旨在证明：这些运算更一般的函数式定义对 CuTe 布局而言通常是可计算的，并且对广泛的分析与应用都是有用的。

2.5 张量（Tensor）

最后来说 CuTe 的核心对象张量（tensor）：把一个布局（layout）绑定到一个访问器（accessor）上，张量就定义出来了。访问器是面向布局、支持随机访问的类指针对象。

定义 W.18. *访问器*（accessor）是支持偏移（offset）与解引用（dereference）操作的对象：

$e + d \to e'$,	将访问器 e **偏移**（offset）$d \in \mathcal{D}$，产生另一访问器 e'；
$*e \to v$,	**解引用**（dereference）访问器 e，产生值 v；
$e[d] \to *(e+d)$,	**下标**（subscript）算符，一种常用的便捷写法。

当 $\mathcal{D} = \mathbb{Z}$ 时，访问器的常见实现包括原始指针（如 `T*`）、数组（如 `T[N]`）以及随机访问迭代器（如 `thrust::counting_iterator` 与 `thrust::transform_iterator` 等）。当 $\mathcal{D}$ 是更为特殊的整数半模时，访问器必须与该半群的加法运算相容。

定义 W.19. 张量（tensor）由访问器 e 与布局 $\mathbf{L}$ 的复合（Composition）所定义，记作 $T = e \circ \mathbf{L}$。张量对布局求值的方式是：把坐标 $c \in \mathbb{Z}(\mathbf{L})$ 映射到陪域 $\mathcal{D}$，据此对访问器做偏移，再对结果解引用，从而得到张量的值。形式化地，

$$T(c) = (e \circ \mathbf{L})(c) = *(e + \mathbf{L}(c)) = e[\mathbf{L}(c)], \tag{W.9}$$

即得到张量在坐标 c 处的值。

大多数张量是以内存地址作为访问器的数据布局。例如，内存地址 p 可以作为带普通偏移与解引用操作的指针访问器 $\{p\}$，用来构造数据张量，

$$\begin{aligned}\{p\} + b &\to \{p + b\} \\ *\{p\} &\to *p \\ T &= \{p\} \circ \mathbf{L}.\end{aligned}$$

图 W.3 中的所有布局也可以与计数迭代器 $\{a\}$ 复合，得到隐式张量；该迭代器解引用到所存储的偏移 $a \in \mathbb{Z}$，

$$\begin{aligned}\{a\} + b &\to \{a + b\} \\ *\{a\} &\to a \\ T &= \{a\} \circ \mathbf{L}.\end{aligned}$$

类似地，图 W.4a 与图 W.4b 中的坐标布局与访问器 $\{(a, b)\}$ 复合，也得到隐式张量：该访问器按坐标偏移，再解引用到所存储的坐标 $(a, b) \in \mathbb{Z}^{(\star,\star)}$，

$$\begin{aligned}\{(a, b)\} + (c, d) &\to \{(a + c, b + d)\} \\ *\{(a, b)\} &\to (a, b) \\ T &= \{(a, b)\} \circ \mathbf{L}.\end{aligned}$$

切片（Slicing）

张量既可以被完全求值，也可以通过切片（slicing）进行部分求值。CuTe 的 Tensor 可以视为带偏移的 Layout，能沿自然坐标的任意一个或多个模任意切片。

- **完全求值**：用完整坐标 c 应用式 (W.9)，得到一个值。
- **部分求值（切片）**：当用不完整坐标 $c = c' + c^*$ 切片时（其中 c^* 表示 c 中未被指定的部分），结果是一个新张量。该操作表示为：

$$T(c) = (e \circ \mathbf{L})(c' + c^*) = (e + \mathbf{L}(c')) \circ \mathbf{L}(c^*) = e' \circ \mathbf{L}(c^*) = T'(c^*), \tag{W.10}$$

其中 $\mathbf{L}(c')$ 可以被完全求值并累加进 e，而 $\mathbf{L}(c^*)$ 是 $\mathbf{L}$ 中尚未求值的子布局。切片产生一个子张量，可对其进一步求值或操作。

图 W.5 给出在 6×12 矩阵里切出行、列与子矩阵的例子。这些例子用记号 $\{a\}$ 表示计数迭代器访问器，指明偏移 $a \in \mathbb{Z}$。切片会把偏移累加到张量上，并定出布局中尚未求值的部分。

张量库（如 `numpy.ndarray`、`torch.tensor` 和 MATLAB）都支持类似的切片记法：写 `my_matrix[2,:]` 取出矩阵的第二行，写 `my_matrix[:,4]` 取出第四列。这些库还支持范围切片（ranged slicing），例如 `my_matrix[2:4,1:3]` 取出从第二行到第四行、从第一列到第三列的子矩阵。CuTe 不支持范围切片，因为范围切片本身有若干问题：

- 范围切片无法表达图 W.5 中演示的所有切片。最后一个切片示例就无法仅凭矩阵行与列上的范围切片来表达。
- 范围切片容易促生如下模式

```
# 为每个 thr_id 提取 TILE_SIZE 大小的数据
thr_data = my_data[thr_id*TILE_SIZE:(thr_id+1)*TILE_SIZE]
```

来获取每个线程局部的一块数据“块”（tile）。这种模式把 `TILE_SIZE`（通常正是要做优化的那个静态常量）与 `thr_id`（本质上是每个线程局部的动态索引）混为一谈。与之相反，CuTe 更倾向于两阶段的“重排—切片”方法，例如

```
# 将 my_data 张量重排并重塑为形状 (TILE_SIZE,NumTiles)
# 这是对更一般的 composition(my_data, Transform_Layout) 的封装
tiled_data = logical_divide(my_data, TILE_SIZE)  # (TILE_SIZE, NumTiles)
assert size[1](tiled_data) == NumThrs            # 验证 NumTiles == NumThrs
# 对分块后的张量切片，取 thr_id 局部的 TILE_SIZE 数据
thr_data = thr_tile_data[None, thr_id]           # TILE_SIZE
```

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	2	15	17	30	32	100	102	115	117	130	132
1	4	6	19	21	34	36	104	106	119	121	134	136
2	8	10	23	25	38	40	108	110	123	125	138	140
3	1	3	16	18	31	33	101	103	116	118	131	133
4	5	7	20	22	35	37	105	107	120	122	135	137
5	9	11	24	26	39	41	109	111	124	126	139	141

张量 $\mathbf{A} = \{0\} \circ ((3,2),((2,3),2)) : ((4,1),((2,15),100))$

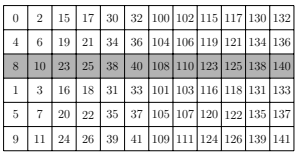
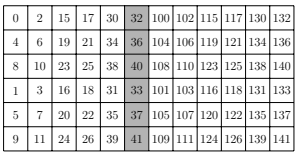
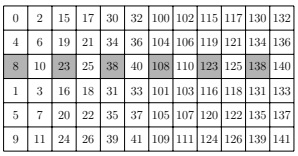
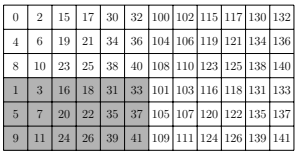
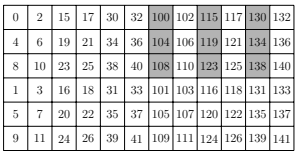
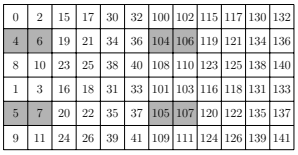
切片	切片后的张量	切片示意图
$\mathbf{A}(2,_)$	$\{8\} \circ ((2,3),2) : ((2,15),100)$	
$\mathbf{A}(_,5)$	$\{32\} \circ (3,2) : (4,1)$	
$\mathbf{A}(2,((0,_),_))$	$\{8\} \circ (3,2) : (15,100)$	
$\mathbf{A}((_,1),(_,0))$	$\{1\} \circ (3,(2,3)) : (4,(2,15))$	
$\mathbf{A}((_,0),((0,_),1))$	$\{100\} \circ (3,3) : (4,15)$	
$\mathbf{A}((1,_),((_,0),_))$	$\{4\} \circ (2,(2,2)) : (1,(2,100))$	

图 W.5: 沿逻辑子边界切片以提取子张量的 6×12 张量。

它把 `TILE_SIZE` 与 `thr_id` 这两个性质不同的参数分开，更容易让编译器据此推理并传播静态信息（例如 `thr_data` 的大小为 `TILE_SIZE`），而且同样灵活。关于通用划分（partitioning）的细节与示例见第 3.3 节，关于逻辑切分（logical divide）的细节与示例见第 3.5 节。

- 范围切片能够表达一些无法用 CuTe 布局表示的切片。例如，取图 W.5 中的 **A**，切片`A(0,0:2:12)` 可以得到 $\{0\} \circ (3,2) : (15,100)$，而切片`A(0:2:6,0)` 由于所请求的切片与 **A** 的布局不相容而无法表示。这主要是由于所请求的块与张量布局不相容，因此我们更希望这一错误在“重排—重塑”阶段（`composition`）就被检测并暴露出来，而不是留到切片阶段。关于布局群复合可容许性条件的细节与示例，见第 3.3 节。

2.6 应用（Applications）

CuTe 为一组布局提供了紧凑的表示，这组布局严格大于用传统扁平形状（flat shape）与扁平步幅（flat stride）所能表示的布局集合。相比之下，CUTLASS v2（Kerr 等，2017）与 Thunderkittens（Spector 等，2024）等库对每种布局进行单独的手工实现。这套做法费时费力，还容易出错。为直观说明其复杂度：CUTLASS v2 代码库中包含近 300 种分别实现的布局，分布在 87 个文件中，总计约 55,000 行代码。CUTLASS v2 中的许多算法还只能配合这些布局中有限的子集工作，维护和扩展都更困难。而 CuTe 的核心布局表示连同操作布局的相关代数只要 3,000 行代码，就能表示 CUTLASS v2 中的全部 300 种布局乃至更多。用 CuTe 实现的算法可以对其输入的秩（rank）或形状施加约束，但与任何满足这些前置条件的布局都相容。算法逻辑与具体的数据布局、线程布局解耦之后，代码更灵活，也更容易组合。

上述优势已体现在实际采用中：CuTe 如今构成了 CUTLASS v3、CUTLASS v4 与 CuTe DSL 的基础，它们全都构建在 CuTe 的核心布局表示与代数之上。现代张量指令所需的复杂数据布局与划分（partitioning）模式，都通过 CuTe 单一、一致且可组合的表示来表达与操作；该表示在 NVIDIA 多代指令集体系结构上保持稳健。

本节说明如何仅凭布局与张量表示，为两种最基础的算法，`COPY`（拷贝）与 `GEMM`（矩阵乘），写出通用实现。这些算法以 CuTe 张量实现，既能说明逻辑实现适用于广泛的应用场景，自身往往已是最优，还可作为各类优化版本的通用参考实现。要完成这些优化，第 3 节的布局代数提供了检查与操作布局的方法。

拷贝（COPY）

用 CuTe 张量编写的一个通用 `COPY` 算法如下：

```
// 前置条件 size(src) == size(dst)
template <class TS, class SLayout,
          class TD, class DLayout>
void
copy(Tensor<TS,SLayout> const& src,  // N
     Tensor<TD,DLayout>      & dst)  // N
{
  for (int i = 0; i < size(dst); ++i) {
    dst(i) = src(i);
  }
}
```

```
# 前置条件 size(src) == size(dst)
def copy(src : Tensor,   # N
         dst : Tensor):  # N
  for i in range(size(dst)):
    dst[i] = src[i]
```

其中，前置条件规定两个张量的大小（size）相等；张量参数的注释里也等价地写着同一件事：两个张量均与形状 N 相容。

只要改变源张量与目标张量的布局，这一简单的 `COPY` 实现便能覆盖广泛的应用。表 W.2 给出了常见应用的例子及其相应的源布局与目标布局。

任意秩的张量都可以拷贝到任意不同秩的张量。从这个意义上说，无论各实参的秩为何，`COPY` 都是一个秩为 1 的算法。这是秩无关（rank-agnostic）编程的一种形式。

当源张量与目标张量的 `idx2crd` 函数求值的计算开销很低时（例如张量形状为静态已知时），上述实现实际上从不会生成动态的坐标变换。在这类情形下，循环可以被展开，`idx2crd` 可以静态地作用于循环索引 `i`，而 `inner_product` 计算在求取偏移时的开销微乎其微。这便是静态分析与优化的一种形式：布局的形状和/或步幅往往在编译期便已可知、可被编译器利用。倘若 `idx2crd` 确实带来运行时开销，所给的这一实现仍不失为参考，可用于验证更进一步的优化版本：这些版本会检查布局的定义域（domain），并利用第 3 节详述的操作变换布局。

应用	源布局	目标布局
一维数组	$8:1$	$8:1$
N 维数组	$(8,2,3):(1,16,32)$	$(8,2,3):(1,16,32)$
聚集（Gather）	$(2,3,2):(42,1,128)$	$12:1$
散射（Scatter）	$12:1$	$(2,3,2):(42,1,128)$
广播（Broadcast）	$7:0$	$7:1$
常量（Constant）	$7:0$	$7:0$
转置（Transpose）	$(8,3):(1,8)$	$(8,3):(3,1)$
张量转置（Tensor Transpose）	$(8,(3,5)):(1,(57,8))$	$(8,15):(1,8)$

表 W.2: COPY 的应用及示例源布局与目标布局。

GEMM（矩阵乘）

用 CuTe 张量编写的一个通用 GEMM 算法如下：

```
// 前置条件 M: size<0>(A) == size<0>(C)
// 前置条件 N: size<0>(B) == size<1>(C)
// 前置条件 K: size<1>(A) == size<1>(B)
template <class TA, class ALayout,
          class TB, class BLayout,
          class TC, class CLayout>
void
gemm(Tensor<TA,ALayout> const& A,  // (M,K)
     Tensor<TB,BLayout> const& B,  // (N,K)
     Tensor<TC,CLayout>      & C)  // (M,N)
{
  for (int k = 0; k < size<1>(B); ++k) {
    for (int n = 0; n < size<0>(B); ++n) {
      for (int m = 0; m < size<0>(A); ++m) {
        C(m,n) += A(m,k) * B(n,k);
  } } }
}
```

```
# 前置条件 M: size[0](A) == size[0](C)
# 前置条件 N: size[0](B) == size[1](C)
# 前置条件 K: size[1](A) == size[1](B)
def gemm(A : Tensor,   # (M,K)
         B : Tensor,   # (N,K)
         C : Tensor):  # (M,N)
  for k in range(size[1](B)):
    for n in range(size[0](B)):
      for m in range(size[0](A)):
        C[m,n] += A[m,k] * B[n,k]
```

其中，前置条件规定了 GEMM 算法的逻辑约束；我们在每个张量参数的注释中写明了各张量必须相容的形状。

只要改变各张量的布局，这一简单的 GEMM 实现（以及一个 batched-GEMM 扩展）便能覆盖多种应用。表 W.3 列出了一些常见应用与示例布局，其中包括 BLAS GEMM 的全部 N-T 变体，以及 BLIS GEMM 的通用步幅变体（dm*、dn*、dk*）。它还可以用作完全通用的张量-张量收缩（GETT）：其中各张量按逻辑上的行模式（mode）、列模式、归约模式与批量模式分组，被折叠成适当的矩阵形状（Shi 等，2016）。构造一个实现 im2col 变换的布局（作为 CuTe 布局的函数复合）（Chetlur 等，2014），还能让 GEMM 实现 CONV，也就是许多现代机器学习应用的核心。

应用	A 布局	B 布局	C 布局
NT GEMM	$(M,K):(1,\texttt{lda})$	$(N,K):(1,\texttt{ldb})$	$(M,N):(1,\texttt{ldc})$
TN GEMM	$(M,K):(\texttt{lda},1)$	$(N,K):(\texttt{ldb},1)$	$(M,N):(1,\texttt{ldc})$
NTT GEMM	$(N,K):(1,\texttt{ldb})$	$(M,K):(1,\texttt{lda})$	$(N,M):(1,\texttt{ldc})$
BLIS GEMM	$(M,K):(\texttt{dma},\texttt{dka})$	$(N,K):(\texttt{dnb},\texttt{dkb})$	$(M,N):(\texttt{dmc},\texttt{dnc})$
GETT	$((M_1,M_2),K):((1,W),X)$	$(N,K):(K,1)$	$((M_1,M_2),N):((1,Y),Z)$
GETT	$(M,(K_1,K_2)):(1,(W,X))$	$(N,(K_1,K_2)):(Y,(1,Z))$	$(M,N):(1,M)$
CONV	$(K,(C,T,R,S)):D_A$	$((N,Z,P,Q),(C,T,R,S)):D_B$	$(K,(N,Z,P,Q)):D_C$

表 W.3: GEMM 的应用及示例布局。

将融合乘加（fused-multiply-add）操作抽象出来，再提供足够强的分块（tiling）工具，这一算法就能适配体系结构的层级，在每一层递归地应用。

3 布局代数（Layout Algebra）

尽管 CuTe 布局只是所有可能函数空间中的一个子集，但与 `BLAS`、`torch.tensor` 和 `numpy.ndarray` 等库中传统的扁平形状（flat-shape）与扁平步幅（flat-stride）表示相比，它能够表示严格更大的一组布局函数。

除表示能力外，CuTe 布局还可以被操作、组合，构造出新的布局。这是通过一组定义在布局上的核心代数运算实现的，这些运算还能进一步构造更高层的运算。

在本节中，我们定义*布局同态*（layout homomorphism），即接收一个或多个 CuTe 布局、并产生一个满足某些函数性质的 CuTe 布局的运算。

3.1 拼接（Concatenate）

一个布局可以表示为其各子布局的*拼接*（concatenation），

$$\begin{aligned}\mathbf{L} &= S : D \\ &= (S_0, S_1, \ldots, S_n) : (D_0, D_1, \ldots, D_n) \\ &= (S_0 : D_0, S_1 : D_1, \ldots, S_n : D_n) \\ &= (\mathbf{L}_0, \mathbf{L}_1, \ldots, \mathbf{L}_n)\end{aligned}$$

使得

$$\forall c = (c_0, c_1, \ldots, c_n) \in \mathbb{Z}(\mathbf{L}), \quad \mathbf{L}(c) = \mathbf{L}_0(c_0) + \mathbf{L}_1(c_1) + \cdots + \mathbf{L}_n(c_n). \tag{W.11}$$

要使拼接可行，函数性质 (W.11) 蕴含所有子布局的陪域必须包含在同一个整数半模（integer-semimodule）之中。例如，任意两个具有整数步幅的布局都可以拼接，但布局 $4:2$ 与 $3:e_0$ 无法拼接。

布局的每个子布局本身也是布局，于是任何操作布局的代数运算也都同样可以应用到任意单个子布局上。我们常称这类运算为“逐模运算（by-mode operation）”，本节定义的每一个运算（合并、复合、补、逻辑切分等）都可以逐模应用。这一方式用如下组合子（combinator）表达：

$$\mathbf{A} \star \langle \mathbf{B}, \mathbf{C} \rangle = (\mathbf{A}_0, \mathbf{A}_1) \star \langle \mathbf{B}, \mathbf{C} \rangle = (\mathbf{A}_0 \star \mathbf{B}, \mathbf{A}_1 \star \mathbf{C}), \tag{W.12}$$

其中 $\star$ 是某个作用于两个布局的运算，$\langle\rangle$ 记号表示由布局组成的元组，以区别于布局的拼接。

3.2 合并（Coalesce）

给定布局 $\mathbf{A}$，其*合并*（coalesced）布局 $\mathbf{R}$ 是满足如下性质的布局：

$$\text{一致的整型定义域：} \qquad |\mathbf{R}| = |\mathbf{A}|, \tag{W.13}$$

$$\text{展平或整型的形状：} \qquad \operatorname{depth}(\mathbf{R}) \le 1, \tag{W.14}$$

$$\text{一致的整型求值：} \qquad \forall c \in \mathbb{Z}_{|\mathbf{A}|}, \quad \mathbf{R}(c) = \mathbf{A}(c). \tag{W.15}$$

`coalesce` 运算通过把布局 $\mathbf{A}$ 视作整数上的函数来“简化”它，并可能将其形状塌缩为更浅的表示。尽管这一过程可能丢失阶与层次信息、改变坐标集合、以及合并 $\mathbf{A}$ 的多个模，但它保证布局作为整型坐标上的映射保持函数等价。

实践中说合并布局，通常指达到最小阶的那个。

例如，$(2,(1,6)):(1,(6,2))$ 的合并布局是 $12:1$。或者，也可以利用式 (W.12) 对该布局逐模合并，即对每个模独立地应用 `coalesce`：

$$\begin{aligned}\texttt{coalesce}((2,(1,6)):(1,(6,2)),\ \langle *, *\rangle) &= \texttt{coalesce}((2:1,(1,6):(6,2)),\ \langle *, *\rangle) \\ &= (\texttt{coalesce}(2:1,\ *),\ \texttt{coalesce}((1,6):(6,2),\ *)) \\ &= (\texttt{coalesce}(2:1),\ \texttt{coalesce}((1,6):(6,2))) \\ &= (2:1, 6:2) \\ &= (2,6):(1,2).\end{aligned}$$

类似地，2 阶布局 $((4,3),5):((15,1),3)$ 合并为 $(4,15):(15,1)$，而其逐模合并布局仍是 $((4,3),5):((15,1),3)$，因为行布局与列布局各自单独合并时保持不变。2 阶布局 $(4,(3,5)):(15,(1,3))$ 同样合并为 $(4,15):(15,1)$，而其逐模合并布局是 $(4,15):(15,1)$，因为第二个模可以单独合并为 $15:1$。

3.3 复合（Composition）

给定布局 $\mathbf{A}$ 与 $\mathbf{B}$，*群复合*（group composition）布局 $\mathbf{R} = \mathbf{A} \circ \mathbf{B}$ 满足：

$$\text{定义域相容性：} \qquad \mathbf{B} \preceq \mathbf{R}, \tag{W.16}$$

$$\text{函数复合性：} \qquad \forall c \in \mathbb{Z}(\mathbf{B}),\ \ \mathbf{R}(c) = \mathbf{A}(\mathbf{B}(c)). \tag{W.17}$$

在这个定义中，$\mathbf{B}$ 通过定义 $\mathbf{R}$ 的定义域来决定结果布局的形状与坐标集合，而 $\mathbf{A}$ 决定 $\mathbf{R}$ 的陪域。相容性条件保证 $\mathbf{B}$ 的所有坐标也都可用作 $\mathbf{R}$ 的坐标。要使群复合与函数复合均可容许，$\mathbf{B}$ 的陪域必须与 $\mathbf{A}$ 的定义域相容，这通常意味着 $\mathbf{B}$ 的陪域是一组与 $\mathbb{Z}(\mathbf{A})$ 中某个坐标集合同余的坐标。

复合的性质（Composition Properties）

恒等布局（Identity Layout）。 对任意形状 S，*恒等布局*（identity layout）$\mathbf{I}_S$ 满足

$$\forall c \in \mathbb{Z}_S,\ \ \mathbf{I}_S(c) = c.$$

注意，只要 $S \preceq P$，$\mathbf{I}_S$ 实际上可以取任意形状 P。例如，下面都是恒等布局 $\mathbf{I}_{24}$，且对所有 $i \in \mathbb{Z}_{24}$ 满足 $\mathbf{L}(i) = i$：

$$24 : 1, \quad (4, 6) : (1, 4), \quad (3, (4, 2)) : (1, (3, 12)).$$

类似地，下面两者都是恒等布局 $\mathbf{I}_{(4,6)}$，且对所有 $(i, j) \in \mathbb{Z}_{(4,6)}$ 满足 $\mathbf{L}(i, j) = (i, j)$：

$$(4, 6) : (e_0, e_1), \quad (4, (3, 2)) : (e_0, (e_1, 3e_1)).$$

对陪域为 $\mathbb{Z}_D$ 的布局 $\mathbf{B}$，任意 $\mathbf{I}_D$ 都充当*群复合的左单位元*（left identity），

$$\mathbf{I}_D \circ \mathbf{B} = \mathbf{B}.$$

对定义域为 $\mathbb{Z}_S$ 的布局 $\mathbf{A}$，形状为 S 的布局 $\mathbf{I}_S$ 充当*群复合的右单位元*（right identity），

$$\mathbf{A} \circ \mathbf{I}_S = \mathbf{A}.$$

结合律（Associative Property）。 给定布局 $\mathbf{A}$、$\mathbf{B}$ 与 $\mathbf{C}$，以及条件

$$\text{image}(\mathbf{C}) \subseteq \mathbb{Z}(\mathbf{B}) \quad \text{且} \quad \text{image}(\mathbf{B}) \subseteq \mathbb{Z}(\mathbf{A}), \tag{W.18}$$

则

$$\mathbf{A} \circ (\mathbf{B} \circ \mathbf{C}) = (\mathbf{A} \circ \mathbf{B}) \circ \mathbf{C}.$$

注意，当式 (W.18) 不满足时，复合往往仍然可行，但结合律可能失效。例如，

$$\begin{aligned} (5, 3) : (1, 7) \ \circ\ [4 : 1 \ \circ\ 2 : 5] &= (5, 3) : (1, 7) \ \circ\ 2 : 5 &&= 2 : 7 \\ [(5, 3) : (1, 7) \ \circ\ 4 : 1] \ \circ\ 2 : 5 &= 4 : 1 \ \circ\ 2 : 5 &&= 2 : 5 \end{aligned}$$

会给出不同结果，因为 $2 : 5$ 的值域未包含在 $4 : 1$ 的定义域之中。不过，鉴于 `idx2crd` 与 `inner_product` 的越界行为，两个表达式都能正确求值。

求值与限制（Evaluation and Restrictions）

群复合的求值可以由式 (W.4) 与式 (W.6) 中的布局求值运算构造性地推导出来。在本节中，我们确定布局 $\mathbf{A}$ 与 $\mathbf{B}$ 上的一组可容许性条件，它们是成功计算群复合所必需的。

首先，我们要求 $\mathbf{A}$ 与 $\mathbf{B}$ 之间的函数相容性：$\mathbf{B}$ 的陪域必须与 $\mathbf{A}$ 的形状弱同余。最常见的情况是 $\mathbf{B}$ 的陪域为整数，而由于所有布局 $\mathbf{A}$ 都接受整型坐标，因此二者函数相容。更一般地，当 $\mathbf{B}$ 产生坐标时，这些坐标必须与 $\mathbb{Z}(\mathbf{A})$ 中的某个坐标同余。

一般做法是令 $(\mathbf{A} \circ \mathbf{B})(i)$ 与 $\mathbf{R}(i)$ 的求值相等。实践中我们发现，只需分析取最简形式 $\mathbf{B} = s : d$ 的基础情形，并把其他求值归约到该情形即可。

基础情形（Base Case）。 设 $\mathbf{B} = s : d$，其中 $s \in \mathbb{Z}^+$、$d \in \mathbb{N}$。设 $\mathbf{A} = S : D = (S_0, S_1, \ldots, S_R) : (D_0, D_1, \ldots, D_R)$ 为一个布局，其中 $S_r \in \mathbb{Z}^+$、$D_r \in \mathcal{D}$。定义 $S = (S_0, S_1, \ldots, S_R)$ 的独占前缀乘积（exclusive prefix-product）为

$$\overline{S}_r = \prod_{k=0}^{r-1} S_k.$$

于是，复合 $\mathbf{R} = \mathbf{A} \circ \mathbf{B}$ 对每个 $i = 0, 1, \ldots, s-1$ 的求值如下：

$$\begin{aligned}
\mathbf{R}(i) = (\mathbf{A} \circ \mathbf{B})(i) &= (D \circ S \circ d \circ s)(i) \\
&= D(S(d(s(i)))) \\
&= \texttt{inner_product}(\texttt{idx2crd}_S(\texttt{inner_product}(\texttt{idx2crd}_s(i), d)), D) \\
&= \texttt{inner_product}(\texttt{idx2crd}_S(\texttt{inner_product}(i, d)), D) \\
&= \texttt{inner_product}(\texttt{idx2crd}_S(i \cdot d), D) \\
&= \sum_{r=0}^{R-1} \left(\left\lfloor \frac{i \cdot d}{\overline{S}_r} \right\rfloor \bmod S_r \right) \cdot D_r + \left\lfloor \frac{i \cdot d}{\overline{S}_R} \right\rfloor \cdot D_R \qquad \text{(W.19)} \\
&= \sum_{r=0}^{R-1} \left(\left\lfloor \frac{i}{\overline{S}'_r} \right\rfloor \bmod S'_r \right) \cdot D'_r + \left\lfloor \frac{i}{\overline{S}'_R} \right\rfloor \cdot D'_R.
\end{aligned}$$

结果 $\mathbf{R}$（若存在）定义为布局 $\mathbf{R} = S' : D' = (S'_0, S'_1, \ldots, S'_R) : (D'_0, D'_1, \ldots, D'_R)$。我们的目标是确定 S'、D' 以及它们存在的条件。

首先，由于 $\mathbf{B}$ 的陪域是整数，我们可以假设布局 $\mathbf{A}$ 已经合并。根据定义，这不会改变复合的求值。

其次，注意到若 $(s-1) \cdot d < \overline{S}_r$，则对所有 $i < s$ 与 $q > r$ 都有 $\lfloor i \cdot d / \overline{S}_q \rfloor = 0$。也就是说，$\mathbf{A}$ 的所有满足 $q > r$ 的模对结果没有贡献。不失一般性，我们在 $(s-1) \cdot d \geq \overline{S}_R$ 的假设下继续推导；这总可以通过对 $\mathbf{A}$ 的形状做截断或延拓来实现。该假设不会改变式 (W.19)，却通过剔除冗余情形简化了分析。

我们施加**步幅整除性条件**（stride divisibility condition）

$$\overline{S}_r \mid d \;\text{ 或 }\; d \mid \overline{S}_r \;\text{ 对每个} r = 0, \ldots, R, \qquad \text{(W.20)}$$

并定义

$$\delta_r = \left\lceil \frac{d}{\overline{S}_r} \right\rceil, \quad \rho_r = \left\lceil \frac{\overline{S}_r}{d} \right\rceil.$$

于是，

$$\sum_{r=0}^{R-1} \left(\left\lfloor \frac{i \cdot d}{\overline{S}_r} \right\rfloor \bmod S_r \right) \cdot D_r + \left\lfloor \frac{i \cdot d}{\overline{S}_R} \right\rfloor \cdot D_R = \sum_{r=0}^{R-1} \left(\left\lfloor i \cdot \frac{\delta_r}{\rho_r} \right\rfloor \bmod S_r \right) \cdot D_r + \left\lfloor i \cdot \frac{\delta_R}{\rho_R} \right\rfloor \cdot D_R.$$

利用恒等式 $(ab) \bmod (nb) = (a \bmod n) \cdot b$，并注意到 $\delta_r \mid S_r$，可得：

$$= \sum_{r=0}^{R-1} \left(\left\lfloor i \cdot \frac{1}{\rho_r} \right\rfloor \bmod \frac{S_r}{\delta_r} \right) \cdot (D_r \cdot \delta_r) + \left\lfloor i \cdot \frac{1}{\rho_R} \right\rfloor \cdot (D_R \cdot \delta_R).$$

一旦验证了

$$\rho_r = \left\lceil \frac{\overline{S}_r}{d} \right\rceil = \left\lceil \frac{\prod_{k=0}^{r-1} S_k}{d} \right\rceil = \left\lceil \frac{S_0}{d} \right\rceil \left\lceil \frac{\prod_{k=1}^{r-1} S_k}{\lceil d / S_0 \rceil} \right\rceil = \cdots = \prod_{k=0}^{r-1} \left\lceil \frac{S_k}{\lceil d / \overline{S}_k \rceil} \right\rceil = \prod_{k=0}^{r-1} \left\lceil \frac{S_k}{\delta_k} \right\rceil = \overline{S}'_r,$$

结果的形状与步幅即可辨识为

$$S'_r = \frac{S_r}{\delta_r},$$
$$D'_r = D_r \cdot \delta_r.$$

为满足相容性条件 (W.16)，形状还必须满足 $|S'| = \overline{S}'_{R+1} = s$。为此，我们再施加**形状整除性条件**（shape divisibility condition）：

$$\left\lceil \frac{\overline{S}_r}{d} \right\rceil \mid s \;\text{ 对每个} r = 0, \ldots, R, \qquad \text{(W.21)}$$

并将形状修改为

$$S'_r = \frac{S_r}{\delta_r}, \quad S'_R = \frac{s}{\rho_R},$$
$$D'_r = D_r \cdot \delta_r,$$

其中项 s/ρ_R 用于将结果形状截断或延拓到大小 s。

这样，只要 $\mathbf{A}$ 与 $\mathbf{B}$ 满足步幅整除性条件 (W.20) 与形状整除性条件 (W.21)，我们就求出了结果 $\mathbf{R} = S' : D'$ 的形状与步幅。

归约情形：分配律（Reductive Case: Distributive）。 为了用基础情形表达更一般的复合，我们利用复合运算对 $\mathbf{B}$ 的子布局拼接的分配性质，

$$\mathbf{A} \circ \mathbf{B} = \mathbf{A} \circ (\mathbf{B}_0, \mathbf{B}_1, \ldots) = (\mathbf{A} \circ \mathbf{B}_0, \mathbf{A} \circ \mathbf{B}_1, \ldots). \tag{W.22}$$

当 $\mathbf{A}$ 的形状 S 对 $\mathbf{B}$ 的各子布局之和可分配时，该性质成立，

$$\begin{aligned}
(\mathbf{A} \circ \mathbf{B})(c) &= D(S(\mathbf{B}(c))) \\
&= D(S(\mathbf{B}_0(c_0) + \mathbf{B}_1(c_1) + \cdots)) && \mathbf{B} = (\mathbf{B}_0, \mathbf{B}_1, \ldots) \text{ 的拼接} \\
&\overset{?}{=} D(S(\mathbf{B}_0(c_0)) + S(\mathbf{B}_1(c_1)) + \cdots) && \text{以 } S \text{ 对 } \mathbf{B}_i \text{ 的分配性为条件} \\
&= D(S(\mathbf{B}_0(c_0))) + D(S(\mathbf{B}_1(c_1))) + \cdots && D \text{ 的线性性} \\
&= (\mathbf{A} \circ \mathbf{B}_0, \mathbf{A} \circ \mathbf{B}_1, \ldots)(c)
\end{aligned} \tag{W.23}$$

以下两个条件之一即足以满足式 (W.23)：

- $\mathbf{B}$ 的陪域与形状 S 同余。此时 S 的作用是恒等变换，可直接验证可分配。
- 当 S 不是恒等变换时，则必须有

$$\left\lfloor \frac{\sum_i c_i \cdot d_i}{S_r} \right\rfloor \bmod S_r = \sum_i \left\lfloor \frac{c_i \cdot d_i}{S_r} \right\rfloor \bmod S_r \quad \text{对每个} r = 0, \ldots, R-1 \text{ 成立，且}$$
$$\left\lfloor \frac{\sum_i c_i \cdot d_i}{S_R} \right\rfloor = \sum_i \left\lfloor \frac{c_i \cdot d_i}{S_R} \right\rfloor$$

而这发生在如下情形：

1. 所有基础子布局的步幅 d_i 都满足步幅整除性条件 (W.20)，且
2. 所有基础子布局 $s_i : d_i$ 的像集彼此分离，

$$\forall i, j, \quad s_i \cdot d_i \le d_j \quad \text{或} \quad s_j \cdot d_j \le d_i$$

归约情形：坐标（Reductive Case: Coordinate）。 当 $\mathbf{B}$ 产生坐标时，这些坐标必须与 $\mathbb{Z}(\mathbf{A})$ 中的某个坐标同余，且 $\mathbf{B}$ 的每个步幅都必须是某个坐标整数半模的缩放基元素：$d = a \cdot e_i \in \mathbb{Z}^{S'}$，其中 $S' \preceq \mathbf{A}$。当这些条件满足时，我们观察到

$$\mathbf{A} \circ \mathbf{B} = \mathbf{A} \circ (a \cdot e_i) \circ s = \mathbf{A}_i \circ a \circ s = \mathbf{A}_i \circ \mathbf{B}',$$

这就回到了基础情形，其中 $\mathbf{B}' = s : a$ 是一个陪域为整数的 1 阶、深度为 0 的布局。对于具有其他整数半模步幅的 $\mathbf{B}$，必须特别小心地将 $\mathbf{B}$ 的陪域映射到 $\mathbf{A}$ 的定义域，并且可能需要对 $\mathbf{A}$ 与 $\mathbf{B}$ 施加额外的限制。

直观理解与整除性（Intuition and Divisibility）

与 1 阶左操作数布局 $\mathbf{A}$ 的复合是平凡的，因为 S_R 不会出现在式 (W.19) 中：

$$(S_0) : (D_0) \ \circ \ s : d = s : D_0 \cdot d.$$

此时不存在非平凡的整除性检查，因为 $R = 0$、$\overline{S}_0 = 1$、$\delta_0 = d$ 且 $\rho_0 = 1$。注意，这意味着即使 $\text{image}(\mathbf{B}) \not\subseteq \mathbb{Z}(\mathbf{A})$，群复合依然可行，

$$7 : 11 \circ 3 : 4 = 3 : 44,$$

而且 **B** 无须两两不相交即可满足分配性，

$$7 : 11 \circ (3, 5) : (6, 3) = (3, 5) : (66, 33).$$

对于与更高阶布局 **A** 的复合，直观策略包含两步：

- 通过从 **A** 中“切分掉”（divide out）前 d 个元素，确定一个产生 **A** 的每第 d 个元素的中间布局。
- 通过“保留”（keep）前 s 个元素，把该中间带步幅布局的大小固定为 s。

例如，

$$(4, 6, 8, 10) : (2, 3, 5, 7) \circ 6 : 12$$

等价于

$$(4, 6, 8, 10) : (2, 3, 5, 7) \circ 6 : 12 = (1, 2, 8, 10) : (X, 9, 5, 7) \circ 6 : 1,$$

其中 **A** 的前 12 个元素被“切分掉”，步幅相应地缩放。然后，修改后布局的前 6 个元素被“保留”，得到：

$$(1, 2, 8, 10) : (X, 9, 5, 7) \circ 6 : 1 = (2, 3) : (9, 5).$$

或者，我们也可以先“保留”前 $6 \cdot 12$ 个元素，再“切分掉”前 12 个，如下所示：

$$(4, 6, 8, 10) : (2, 3, 5, 7) \circ 6 : 12 = (4, 6, 3) : (2, 3, 5) \circ 6 : 12 = (2, 3) : (9, 5).$$

整除性条件保证“切分掉”与“保留”两步中的逐次除法对结果形状总是产生整数。

对整除性条件的违反（Violations of Divisibility Conditions）。 某些复合会违反步幅或形状整除性条件。例如，

$$(4, 6, 8) : (2, 3, 5) \circ 6 : 3$$

违反了步幅整除性条件 (W.20)，因为 $\overline{S}_1 = 4$ 与 $d = 3$ 不可整除。不存在能表示布局 $(4, 6, 8) : (2, 3, 5)$ 的每第 3 个元素的布局。

类似地，复合

$$(4, 6, 8) : (2, 3, 5) \circ 6 : 1$$

违反了形状整除性条件 (W.21)，因为 $\left\lceil \frac{\overline{S}_1}{d} \right\rceil = \lceil \frac{4}{1} \rceil = 4$ 不能整除 $s = 6$。不存在能表示 $(4, 6, 8) : (2, 3, 5)$ 的前 6 个元素的布局。

归根结底，CuTe 布局在群复合下并非严格封闭。然而实践中，违反整除性条件多半意味着概念用错了、布局与硬件不相容，或者程序员失误。这些错误大多能在编译期被捕获，程序因此更安全，开发也更快。

表面上的违反（Apparent Violations）。 考虑复合

$$(4, 2, 8) : (3, 12, 97) \circ 3 : 3,$$

它看似违反了步幅整除性条件，因为 $\overline{S}_1 = 4$ 与 $d = 3$ 不可整除。这一问题可以通过对 **A** 做合并（如第 3.3 节所述）以及对 **A** 做截断（正如 $(s-1) \cdot d < \overline{S}_r$ 情形中所处理的那样）来解决。等价的复合产生结果

$$(4, 2, 8) : (3, 12, 97) \circ 3 : 3 = (8, 8) : (3, 97) \circ 3 : 3 = (8) : (3) \circ 3 : 3 = 3 : 9.$$

然而，下面的复合不满足整除性条件：

$$(4, 2, 8) : (3, 12, 97) \circ 4 : 3 \qquad\qquad (4, 2, 8) : (3, 15, 97) \circ 3 : 3.$$

左侧的例子无法被充分截断，而右侧的例子则无法被合并。

应用：划分示例（Application: Partitioning Example）

复合位于 CuTe 布局代数的核心，支撑着重塑形（reshaping）、重定步幅（restriding）、置换（permuting）、划分（partitioning）、分块（tiling）以及提取子布局等运算。本节演示如何利用复合、以任意的线程-值模式（thread-value pattern）对一个数据张量进行划分。

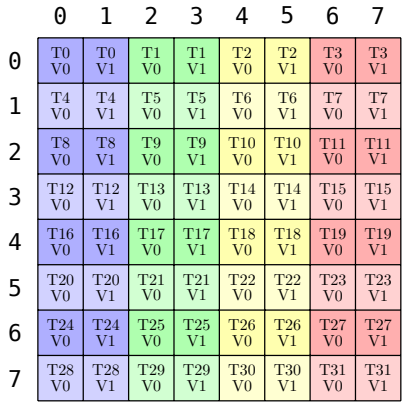


图 W.6: Ampere FP64 Tensor Core 所要求的 8×8 C 矩阵的线程-值划分。

考察一个由形状 $(8,8)$ 细化而来的数据布局。我们的目标是使用图 W.6 所示的线程-值模式对该数据进行划分，该模式是某个特定 Ampere Tensor Core 的逻辑划分模式。这条指令的线程-值划分模式可以用布局

$$\texttt{ThrValLayoutC:}\quad ((4,8),2):((16,1),8),$$

表示，它充当从 $(\texttt{thread_idx},\texttt{value_idx})$ 到 8×8 矩阵内一维坐标的映射。如图所示，出于人类可读性的考虑，图 W.6 实际上画出的是其逆：它展示的是从 8×8 矩阵内的二维坐标到 $(\texttt{thread_idx},\texttt{value_idx})$ 二元组的映射。这个 `ThrValLayoutC` 被视为 Ampere Tensor Core 指令的静态元数据，它是指令与硬件功能的内在属性；在 CuTe 出现之前，它只能以指令集体系结构（ISA）中类似图 W.6 的图像来描述。

任意 8×8 数据布局都可以通过与线程-值布局 `ThrValLayoutC` 复合来置换。每次复合都会产生一个与形状 $(32,2)$ 相容的布局，并定义 $(\texttt{thread_idx},\texttt{value_idx})$ 与数据偏移之间的映射。例如，表 W.4 展示了若干数据布局经上述 `ThrValLayoutC` 复合变换的例子。

数据名称	数据布局 8×8（**A**）	线程-值布局 32×2（**B**）	结果 32×2（**R**）
`ColMajor`	$(8,8):(1,8)$	$((4,8),2):((16,1),8)$	$((4,8),2):((16,1),8)$
`RowMajor`	$(8,8):(8,1)$	$((4,8),2):((16,1),8)$	$((4,8),2):((2,8),1)$
`Padded`	$(8,8):(1,9)$	$((4,8),2):((16,1),8)$	$((4,8),2):((18,1),9)$
`ColInterleaved`	$((4,2),(2,4)):((2,16),(1,8))$	$((4,8),2):((16,1),8)$	$((4,(4,2)),2):((8,(2,16)),1)$
`Swizzled`	$(8,8):(f_1,f_9)$	$((4,8),2):((16,1),8)$	$((4,8),2):((f_{18},f_1),f_9)$
`Coordinate`	$(8,8):(e_0,e_1)$	$((4,8),2):((16,1),8)$	$((4,8),2):((2e_1,e_0),e_1)$

表 W.4: 数据布局经线程-值（TV）布局复合变换的示例。

下面的代码展示了一种使用线程-值划分的常见模式：

```
smem_data = Tensor(MyAccessor, MyLayout8x8)     # Tensor: Coord -> Offset
tv_layout   = Layout(((4,8),2), ((16,1),8))     # TV 布局: (Thr,Val) -> Coord
smem_tv     = composition(smem_data, tv_layout) # 复合: (Thr,Val) -> Offset
smem_v      = smem_tv[thr_id, None]             # 按线程切片得到子张量
copy(smem_v, rmem_data)                         # 拷贝到寄存器张量/数组
```

这一模式在 SIMD 编程中很常见：每个处理元素接收某个父数据的一个对称划分。一般而言，CuTe 把任意划分都定义为复合（置换和/或重塑形）之后再切片。如第 2.5 节所述，这种“复合-切片”模式把划分模式（`ThrValLayoutC`）与实际切出的数据切片（以 `thr_id` 做的切片）分离开来。由于划分模式往往是指令或优化参数相关的编译期元数据，而切片是运行时的程序索引，这一模式更适合传播静态信息，运行时开销也更低。

逐模复合与分块器（By-mode Composition and Tilers）

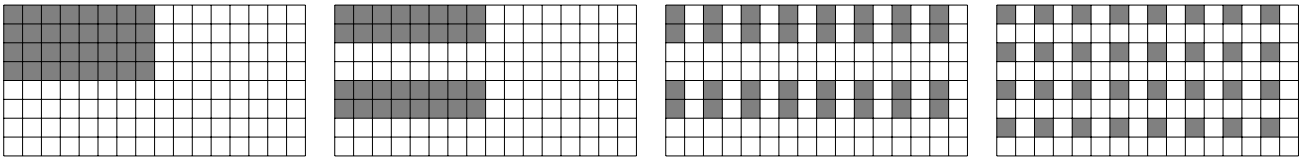
群复合可以逐模应用，从而支持对布局的子定义域进行操作。例如，常常需要对矩阵的行与列独立地应用复合。我们使用式 (W.12) 中的组合子，并把其中的 $\star$ 运算取为群复合 $\circ$，从而对布局的各个独立子布局分别应用复合。

我们把式 (W.12) 中的布局元组称为分块器（Tiler）。

定义 W.20. 分块器（Tiler）是一个 `HTuple(Tile)`，其中每个 `Tile` 是

- 一个 *Layout* $S : D$，或
- 一个 *Integer* S，等价地视作布局 $S : 1$。

按该定义，所有布局都是分块器，所有整数也都是分块器，$(4, 8)$ 这样的形状自然也能当分块器。图 W.7a 展示了这一常见情形：形状可以作为提取并操作子布局的便捷分块器。



(a) $\langle 4 : 1, 8 : 1\rangle \equiv \langle 4, 8\rangle$　(b) $\langle (2, 2) : (1, 4), 8 : 1\rangle$　(c) $\langle (2, 2) : (1, 4), 8 : 2\rangle$　(d) $\langle 4 : 2, 8 : 2\rangle$

图 W.7: 通过 $\mathbf{L} \circ \mathbf{T}$ 从任意 8×16 布局 $\mathbf{L}$ 中提取 4×8 子布局的分块器 $\mathbf{T}$ 示例。

图 W.7 展示了分块器如何通过复合作用于任意布局。图中取一个与形状 $(8, 16)$ 相容的任意布局，并应用各种分块器分别从列和行中提取元素，从而产生与形状 $(4, 8)$ 相容的子布局。例如，分块器 $\langle 4 : 1, 8 : 1\rangle$ 从每列提取前 4 个元素、从每行提取前 8 个元素，产生与形状 $(4, 8)$ 相容的子布局。

分块器的复合与坐标布局的复合之间还有等价关系。对布局 $\mathbf{L} = (M, N) : (X, Y)$ 与分块器 $\mathbf{T} = \langle \mathbf{T}_0, \mathbf{T}_1\rangle$，我们注意到

$$\mathbf{L} \circ \mathbf{T} \equiv \mathbf{L} \circ (M, N) : (e_0, e_1) \circ \mathbf{T}$$

这意味着 $(M, N) : (e_0, e_1) \circ \mathbf{T}$ 对 $\mathbf{L}$ 的作用与 $\mathbf{T}$ 的作用相同。这里 $(M, N) : (e_0, e_1)$ 是一个恒等布局，把二维自然坐标映射到二维坐标。分块器的这种表述说明，形状是从坐标到自然坐标的映射，正如第 2.4 节中 Layout 的定义所采用的那样。具体而言，在复合中以下对象应视为等价：

$$(4, 8) \equiv \langle 4, 8\rangle \equiv (4 : 1, 8 : 1) \equiv (4, 8) : (e_0, e_1)$$

3.4 逆（Inverse）

布局可以是单射（injective）、满射（surjective）或双射（bijective）的，并且存在右逆（right-inverse）、左逆（left-inverse）、全逆（full-inverse）与拟逆（quasi-inverse）。当把布局解释为从坐标到偏移的函数时，逆布局便可解释为从偏移到坐标的函数。布局的逆很有用：确定某些偏移在布局中的位置、提取由特定偏移构成的组，或确定两个布局各自的公共子布局。

本节定义布局的广义左逆与右逆，并给出使用它们的应用示例。

右逆（Right-Inverse）

布局 $\mathbf{L} : \mathbb{Z}_{|\mathbf{L}|} \to \mathcal{D}$ 的*右（伪）逆*是一个单射布局 $\mathbf{L}^{\ddagger} : \mathcal{D}_{\mathbf{L}^{\ddagger}} \to \mathbb{Z}_{|\mathbf{L}|}$，满足

$$\forall k \in \mathcal{D}_{\mathbf{L}^{\ddagger}},\quad \mathbf{L}^{\ddagger}(\mathbf{L}(\mathbf{L}^{\ddagger}(k))) = \mathbf{L}^{\ddagger}(k) \tag{W.24}$$

当 $\mathbf{L}$ 具有一般的整数半模（integer-semimodule）陪域（codomain）$\mathcal{D}$ 时，上述一般性定义可用来刻画 $\mathbf{L}^{\ddagger}$ 的性质。在常见情形 $\mathcal{D} = \mathbb{Z}$ 下，可以恢复出右逆的经典定义，

$$\forall k \in \mathbb{Z}_{|\mathbf{L}^{\ddagger}|},\quad \mathbf{L}(\mathbf{L}^{\ddagger}(k)) = k,$$

而对于更一般的陪域，例如 $\mathcal{D} = \mathbb{Z}^{(*,*)}$，由式 (W.24) 可以推出 $\mathcal{D} \lesssim \mathbf{L}^{\ddagger}$：右逆的定义域（domain）对 $\mathbf{L}$ 陪域的轮廓做了粗化，使其能够接受来自 $\mathbf{L}$ 的偏移。

$\mathbf{L}$	$\mathbf{L}^{\ddagger}$	备注
$(4,8):(1,4)$	$32:1$	
$(4,8):(8,1)$	$(8,4):(4,1)$	
$(3,7,5):(5,15,1)$	$(5,21):(21,1)$	非 2 的幂的形状/步幅也没有问题。
$(4,8):(1,5)$	$4:1$	对非连续的像，结果更小。
$(4,(4,2)):(4,(1,16))$	$(4,4,2):(4,1,16)$	结果定义域与陪域 $\mathbb{Z}$ 同余。
$((2,2),(4,2)):((1,8),(2,16))$	$(2,4,2,2):(1,4,2,16)$	
$((2,2),(2,4)):((0,1),(0,2))$	$(2,4):(2,8)$	步幅为 0 的模没有贡献。
$((2,2),(2,4)):((0,2),(0,4))$	$1:0$	平凡右逆。
$(4,8):(e_0,e_1)$	$(4,8):(1,4)$	结果定义域与陪域 $\mathbb{Z}^{(*,*)}$ 同余。
$(4,(4,2)):(e_1,(e_0,6e_1))$	$(4,4):(4,1)$	结果更小且定义域相容。
$(4,(4,3)):(f_1,(f_5,f_{16}))$	$(4,4,3):(f_1,f_5,f_{16})$	对整数模步幅也可定义。

表 W.5: 布局右逆的示例。

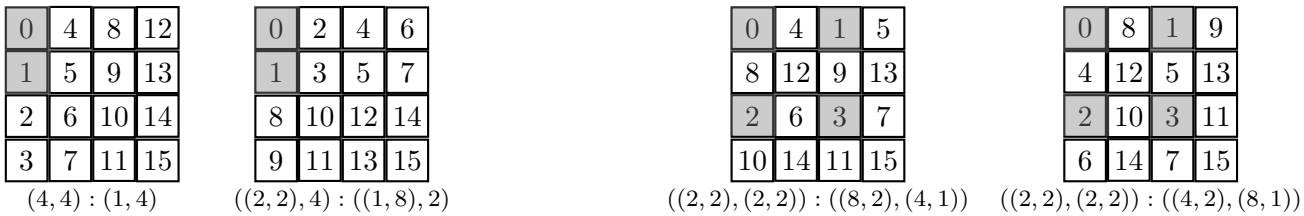
若布局 $\mathbf{L}$ 存在右逆布局 $\mathbf{L}^{\ddagger}$，则 $\left|\mathbf{L}^{\ddagger}\right| \leq |\mathbf{L}|$。实践中，提到某布局的右逆时，我们通常指尺寸最大的那个右逆。例如，图 W.3 与图 W.4 中布局的最大尺寸右逆如表 W.5 所示。

当布局 $\mathbf{L}$ 是 $\mathbb{Z}_{|\mathbf{L}|}$ 上的双射时，右逆同时也是布局 $\mathbf{L}$ 的*逆* $\mathbf{L}^{-1}$，满足

$$\forall k \in \mathbb{Z}_{|\mathbf{L}|},\quad \mathbf{L}(\mathbf{L}^{-1}(k)) = k = \mathbf{L}^{-1}(\mathbf{L}(k))$$

若布局 $\mathbf{L}$ 存在全逆 $\mathbf{L}^{-1}$，则称该布局是紧凑（compact）的。

应用：向量化（Vectorization）示例



(a) 2 元素公共子向量。

(b) 4 元素公共子向量。

图 W.8: 布局之间公共子向量的示例。

判断数据布局里有没有连续元素、在什么位置，右逆很有用。举个直接的例子：布局 $(4,8):(1,4)$ 与 $(4,8):(8,1)$ 的右逆分别是 $32:1$ 与 $(8,4):(4,1)$。两个右逆的尺寸都是 32，说明两个布局都索引 32 个连续的物理元素。

再看一个复杂些的例子：CuTe 中一种常见的拷贝模式，向量化拷贝（vectorizing-copy），它要找出两个张量之间一次最多能拷贝的元素个数。右逆使 CuTe 能确定两个布局之间的*最大公共子布局*（maximum common sublayout）；再结合硬件能力以及指针与步幅物理对齐的信息，就能以代数方式确定可安全向量化的元素数量及其位置，照此完成拷贝。

图 W.8 给出了两个向量化示例。在第一幅图 W.8a 中，源布局与目标布局在元素偏移 $k \in \{0,1\}$（对应坐标 $\bar{c} \in \{0,1\}$）处坐标一致。只要访问器（accessor）、数据类型与对齐允许，拷贝宽度即可扩展为每条指令搬运两个元素。在第二幅图 W.8b 中，源布局与目标布局在元素偏移 $k \in \{0,1,2,3\}$（对应坐标 $\bar{c} = \{0,2,8,10\}$）处坐标一致，拷贝宽度可扩展为每条指令搬运四个元素。

一般地，对满足 $|\mathbf{A}| = |\mathbf{B}|$ 的布局 $\mathbf{A}:\mathbb{Z}(\mathbf{A}) \to \mathbb{Z}_{\alpha}$ 与 $\mathbf{B}:\mathbb{Z}(\mathbf{B}) \to \mathbb{Z}_{\beta}$，我们希望找到最大的整数 K，使得坐标相匹配

$$\forall\, k \in \mathbb{Z}_K,\quad \mathbf{A}^{\ddagger}(k) = \mathbf{B}^{\ddagger}(k)$$

并且这一最大值可以通过寻找满足下式的 K 来高效计算：

$$\forall\, k \in \mathbb{Z}_K,\quad k = \mathbf{A}(\mathbf{B}^{\ddagger}(k)) = (\mathbf{A} \circ \mathbf{B}^{\ddagger})(k) \quad \text{或} \quad \forall\, k \in \mathbb{Z}_K,\quad k = \mathbf{B}(\mathbf{A}^{\ddagger}(k)) = (\mathbf{B} \circ \mathbf{A}^{\ddagger})(k)$$

也就是 $\mathbf{A} \circ \mathbf{B}^{\ddagger} = (\mathbf{I}_K, \mathbf{X})$ 或 $\mathbf{B} \circ \mathbf{A}^{\ddagger} = (\mathbf{I}_K, \mathbf{Y})$ 中恒等部分（步幅为 1 的模）的尺寸。

这些相互连续的元素，其整数坐标由下式给出

$$\begin{aligned} \mathbf{A}^{\ddagger} \circ \left\lfloor \mathbf{B} \circ \mathbf{A}^{\ddagger} \right\rfloor_K &= \mathbf{A}^{\ddagger} \circ \mathbf{I}_K = \left\lfloor \mathbf{A}^{\ddagger} \right\rfloor_K \;:\; \mathbb{Z}_K \to \mathbb{Z}_{|A|} \\ \mathbf{B}^{\ddagger} \circ \left\lfloor \mathbf{A} \circ \mathbf{B}^{\ddagger} \right\rfloor_K &= \mathbf{B}^{\ddagger} \circ \mathbf{I}_K = \left\lfloor \mathbf{B}^{\ddagger} \right\rfloor_K \;:\; \mathbb{Z}_K \to \mathbb{Z}_{|B|} \end{aligned} \tag{W.25}$$

其中 $\lfloor \cdot \rfloor_K$ 是在尺寸 K 处截断的操作。该布局给出数据中前 K 个物理元素的逻辑坐标，并可通过 `logical_divide` 或 `zipped_divide`（第 3.5 节）从中提取出公共子向量。

在实践中，考虑第 2.6 节中的参考函数 `COPY`。该实现可以在不改变可观察行为的前提下，在内部置换迭代顺序。置换迭代顺序等价于对源张量与目标张量的布局施加同一个一致的置换。选取与式 (W.25) 所定义布局相匹配的置换，所有连续元素都会被换到第一个模中，由向量化指令做特殊处理。这一过程既能检测待向量化的元素是否存在及其位置，也能确定可安全向量化的最大元素数；**自动向量化**（AutoVectorization）就建立在这套流程之上。

左逆（Left-Inverse）

布局 $\mathbf{L} : \mathbb{Z}_{|\mathbf{L}|} \to \mathcal{D}$ 的*左（伪）逆*是一个布局 $\mathbf{L}^{\dagger} : \mathcal{D}_{\mathbf{L}^{\dagger}} \to \mathbb{Z}_{|\mathbf{L}|}$，满足

$$\forall k \in \mathbb{Z}_{|\mathbf{L}|},\quad \mathbf{L}(\mathbf{L}^{\dagger}(\mathbf{L}(k))) = \mathbf{L}(k) \tag{W.26}$$

左逆可能不唯一，并且对于不在 $\mathbf{L}$ 的像（image）中的输入，左逆可以取任意值。当 $\mathbf{L}$ 具有一般的整数半模陪域 $\mathcal{D}$ 时，上述一般性定义同样可用来刻画 $\mathbf{L}^{\dagger}$ 的性质。在 $\mathbf{L}$ 为单射这一常见情形下，可以恢复出左逆的经典定义，

$$\forall k \in \mathbb{Z}_{|\mathbf{L}|},\quad \mathbf{L}^{\dagger}(\mathbf{L}(k)) = k.$$

图 W.3 与图 W.4 中布局的左逆如表 W.6 所示。

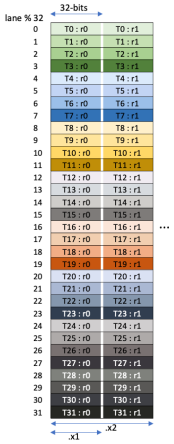
$\mathbf{L}$	$\mathbf{L}^{\dagger}$	备注
$(4,8) : (1,4)$	$32 : 1$	
$(4,8) : (8,1)$	$(8,4) : (4,1)$	对连续的像，与右逆相同。
$(3,7,5) : (5,15,1)$	$(5,21) : (21,1)$	非 2 的幂的形状/步幅也没有问题。
$(4,8) : (1,5)$	$(5,8) : (1,4)$	对非连续的像，结果更大。
$(4,(4,2)) : (4,(1,16))$	$(4,4,2) : (4,1,16)$	结果定义域与陪域 $\mathbb{Z}$ 同余。
$((2,2),(4,2)) : ((1,8),(2,16))$	$(2,4,2,2) : (1,4,2,16)$	
$((2,2),(2,4)) : ((0,2),(0,4))$	$(2,2,4) : (0,2,8)$	结果不唯一——模 0 的步幅可任取。
$((2,2),(2,4)) : ((0,1),(0,2))$	$(2,4) : (2,8)$	
$(4,8) : (e_0, e_1)$	$(4,8) : (1,4)$	结果定义域与陪域 $\mathbb{Z}^{(*,*)}$ 同余。
$(4,(4,2)) : (e_1,(e_0,6e_1))$	$(4,(6,2)) : (4,(1,16))$	结果更大且定义域相容。
$(4,(4,3)) : (f_1,(f_5,f_{16}))$	$(4,4,3) : (f_1,f_5,f_{16})$	对整数模步幅也可定义。

表 W.6: 布局左逆的示例。

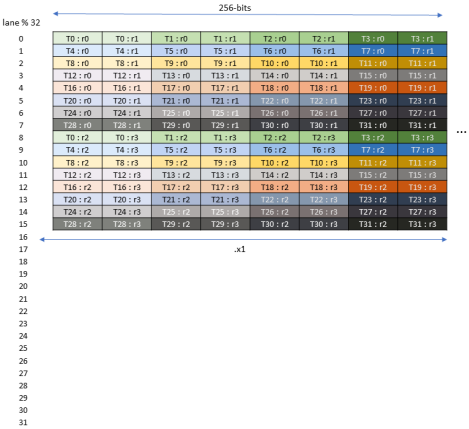
应用：可容许性（Admissibility）示例

左逆用来确定某个数据布局所产生的特定偏移是否存在及其所在位置。

向 Blackwell 的“张量内存”（tensor memory，TMEM）加载以及从其存储数据所使用的指令，会以一组预定义的固定偏移访问 TMEM，如图 W.9 所示。物理 TMEM 的编址是一个二维网格，最多包含 512 个“列”（col）与 128 个“通道”（lane）。图中是每条指令单个 warp 的访问模式；每条指令实际使用四个 warp，上述模式每 32 个通道重复一次、共重复四次。在对物理 TMEM 编址时，沿 32 位的“列”方向跨越一步使 TMEM 地址增加 1，沿“通道”方向向下跨越一步使 TMEM 地址增加 16384。基于这一物理编址，我们定义记录每条指令所访问的全部物理偏移的布局：



(a) `tcgen05{.ld,.st}.32x32b.{x1,x2}`



(b) `tcgen05{.ld,.st}.16x256b.x1`

图 W.9: TMEM load/store 指令访问特定的偏移，并将数据分配到线程本地寄存器（NVIDIA Corporation 2025）。

指令	(指令 COL, 指令 LANE) → TMEM 偏移
`tcgen05{.ld,.st}.32x32b.x1`	$(1, 128):(1, 16384)$
`tcgen05{.ld,.st}.32x32b.x2`	$(2, 128):(1, 16384)$
`tcgen05{.ld,.st}.16x256b.x1`	$(8, (16, 4)):(1, (16384, 32 \cdot 16384))$

这些布局与具体指令一一对应，可用来精确确定每条指令将访问 TMEM 中的哪些偏移。

尽管 TMEM 采用二维编址，并被用作形状为 (M, N) 的 MMA 的累加器，TMEM 的“通道”与“列”并不总与逻辑矩阵的行和列相对应。与任何编址方式一样，物理偏移可以通过任意的 CuTe 布局被回绕并折叠成任意形状。因此，给定一个数据布局与一条 TMEM 指令，我们希望确定

1. 数据布局是否包含该指令将访问的全部偏移；以及
2. 指令所访问的偏移在数据布局中位于何处（以逻辑坐标表示）。

一般地，设数据布局 $\mathbf{A} : \mathbb{Z}(\mathbf{A}) \to \mathbb{Z}_\alpha$ 将逻辑坐标映射到数据偏移，指令布局 $\mathbf{T} : \mathbb{Z}(\mathbf{T}) \to \mathbb{Z}_\beta$ 将指令坐标映射到数据偏移，我们希望确定所有偏移 $\mathbf{T}(i)$ 在 $\mathbf{A}$ 的像中的存在性及其位置。将其表述为

$$\forall i \in \mathbb{Z}_{|\mathbf{T}|}, \exists c \in \mathbb{Z}(\mathbf{A}) \quad s.t. \quad \mathbf{T}(i) = \mathbf{A}(c)$$

该问题可以通过计算 $\mathbf{A}$ 的左逆并检验下式来高效求解：

$$\mathbf{A}(\mathbf{A}^\dagger(\mathbf{T}(i))) = \mathbf{T}(i)$$

也就是说，

- 所有偏移 $\mathbf{T}(i)$ 都在 $\mathbf{A}^\dagger$ 的定义域中。
- 所有坐标 $\mathbf{A}^\dagger(\mathbf{T}(i))$ 都互不相同，且都在 $\mathbf{A}$ 的定义域中。

在这些条件下，我们可以断言：每个偏移 $\mathbf{T}(i)$ 都出现在 $\mathbf{A}$ 的像中，且位于坐标 $\mathbf{A}^\dagger(\mathbf{T}(i))$ 处。布局 $\mathbf{A}^\dagger \circ \mathbf{T}$ 把指令坐标映射到逻辑数据坐标，例如可借助 `zipped_divide` 把数据布局 $\mathbf{A}$ 切分为与该指令所访问偏移相对应的若干子布局。

3.5 补

布局（layout）$\mathbf{L} : \mathbb{Z}(\mathbf{L}) \to \mathcal{D}$ 的补（complement）是一个满足下列条件的布局 $\mathbf{L}^* : \mathbb{Z}(\mathbf{L}^*) \to \mathcal{D}$

$$\text{与陪域弱同余：} \quad \mathcal{D} \lesssim \mathbf{L}^*, \tag{W.27}$$

$$\text{像互不相交：} \quad \forall b \in \mathbb{Z}(\mathbf{L}),\ \forall a \in \mathbb{Z}^{\mathbf{L}^*}/\{0\},\ \mathbf{L}(b) \neq \mathbf{L}^*(a), \tag{W.28}$$

$$\text{像有序：} \quad \forall a \in \mathbb{Z}_{|\mathbf{L}^*|}/\{0\},\ \mathbf{L}^*(a-1) < \mathbf{L}^*(a). \tag{W.29}$$

布局的补是一个在原布局的陪域（codomain）之中、但不在其像（image）之中生成元素的布局。在复合（Composition）中，布局 $\mathbf{L}$ 常被用作对另一个布局的间接寻址，因此补 $\mathbf{L}^*$ 就是指向 $\mathbf{L}$ 所遗漏元素的那个布局。有了 `complement`，

就能经由*逻辑切分*（logical divide）把布局切开（第 3.5 节），也能经由*逻辑乘积*（logical product）把布局重复、扩展（第 3.5 节）。

本节只关注陪域 $\mathcal{D}$ 为 `HTuple`($\mathbb{Z}$) 的布局：即整数，或对某个轮廓 D 而言 $\mathcal{D} = \mathbb{Z}^D$ 的坐标。弱同余条件式 (W.27) 因此成立。像互不相交条件（式 (W.28)）保证补所生成的值与原布局生成的值互不相同：原布局取的是有限定义域 $\mathbb{Z}(\mathbf{L})$，补取的是无穷扩展定义域 $\mathbb{Z}^{\mathbf{L}^*}$，因此无论补的形状被截断还是被扩展（例如通过与右侧的布局复合），它所生成的值始终与原布局不同。最后，像有序条件（式 (W.29)）确立了补布局的唯一性。

表 W.7 给出了补的若干例子。

$\mathbf{L}$	$\mathbf{L}^*$	备注
$(4,8):(1,4)$	$1:32$	注意 $\forall i \in \mathbb{Z}^+, \mathbf{L}^*(i) > \max_{j\in\mathbb{Z}_{\mathbf{L}}} \mathbf{L}(j)$。
$(4,8):(8,1)$	$1:32$	输入的顺序无关紧要。
$(4,(4,2)):(4,(1,16))$	$1:32$	输入的层级结构无关紧要。
$(4,8):(1,5)$	$1:40$	
$(4,8):(1,8)$	$(2,1):(4,64)$	仍然互不相交，但在陪域中“填补空洞”。
$((2,2),(2,4)):((0,1),(0,2))$	$1:8$	步幅（stride）为 0 的模无贡献。
$((2,2),(2,4)):((0,2),(0,4))$	$(2,1):(1,16)$	
$(4,8):(e_0,e_1)$	$(1,1):(4e_0,8e_1)$	结果定义域与陪域 $\mathbb{Z}^{(*,*)}$ 同余。
$(4,(4,2)):(e_1,(e_0,12e_1))$	$(1,(3,1)):(4e_0,(4e_1,24e_1))$	弱同余、互不相交且有序。

表 W.7: 布局补的例子。

应用：逻辑乘积

两个布局 **A** 与 **B** 的逻辑乘积是一个布局 **R**，其中“布局 **B** 的每个元素都被替换为布局 **A** 的一个经过唯一平移的版本”。

具体地，我们将逻辑乘积定义为一个从两个布局产生一个 2 阶（rank-2）布局的函数

$$\mathbf{A} \otimes \mathbf{B} = (\mathbf{A}, \mathbf{A}^* \circ \mathbf{B})$$

其中 $\mathbf{A}^*$ 是 **A** 的补。结果的第一个模（mode）恰好就是输入布局 **A**，而第二个模在元素形状（shape）与元素顺序上与输入布局 **B** 相容。第一个模，即 **A** 布局，通常被称为“块”，它在“网格”布局 **B** 上被重复。

考虑这样的例子：行主序（row-major）布局 $(3,4):(4,1)$ 块，要在列主序（col-major）布局 $(2,5):(1,2)$ 网格中复现：

$$\begin{pmatrix}3 & 4\\ 4 & 1\end{pmatrix} \otimes \begin{pmatrix}2 & 5\\ 1 & 2\end{pmatrix} = \left(\begin{pmatrix}3 & 4\\ 4 & 1\end{pmatrix}, \begin{matrix}1\\ 12\end{matrix} \circ \begin{pmatrix}2 & 5\\ 1 & 2\end{pmatrix}\right) = \begin{matrix}((3, & 4), & (2, & 5))\\ ((4, & 1), & (12, & 24))\end{matrix}$$

其中布局 $1:12$ 是 $(3,4):(4,1)$ 的补。结果的第一个模中可以直接得到原始块，而第二个模以 2×5 列主序的顺序把原始块铺 10 次，互不重叠。

再换一个例子，考虑布局 $(4,8):(20,2)$ 块在行主序布局 $(3,2):(2,1)$ 网格中复现：

$$\begin{pmatrix}4 & 8\\ 20 & 2\end{pmatrix} \otimes \begin{pmatrix}3 & 2\\ 2 & 1\end{pmatrix} = \left(\begin{pmatrix}4 & 8\\ 20 & 2\end{pmatrix}, \begin{matrix}2 & 1\\ 1 & 80\end{matrix} \circ \begin{pmatrix}3 & 2\\ 2 & 1\end{pmatrix}\right) = \begin{matrix}((4, & 8), & (3, & 2))\\ ((20, & 2), & (80, & 1))\end{matrix}$$

其中布局 $(2,1):(1,80)$ 是 $(4,8):(20,2)$ 的补。此时网格交错排布这些块，以产生结果的最小陪域。补布局决定结果如何为布局 **B** 设定步幅，而布局 **B** 决定结果如何为补布局排序并定形。

相关乘积 逻辑乘积产生一个 2 阶布局，其中第一个模是原始块，第二个模被解释为该块的网格，即“分块”。

逻辑乘积对 **A** 与 **B** 的阶（rank）和相容性都没有限制；操作 `blocked_product` 则更直观，它利用输入的阶来构造结果的形状。直觉上，一个 3×4 布局与一个 2×5 布局相乘，就该得到一个 6×20 布局。`blocked_product` 要求 **A** 与 **B** 的阶相同，

$$\text{阶}(\mathbf{A}) = \text{阶}(\mathbf{B})$$

以便施加一个 zip 操作，把逻辑上的行模与行模、列模与列模组合起来。例如，将上例中的模重排

$$\begin{matrix}((3,\ 4),\ (2,\ 5))\\((4,\ 1),\ (12,\ 24))\end{matrix} \Longrightarrow \begin{matrix}((3,\ 2),\ (4,\ 5))\\((4,\ 12),\ (1,\ 24))\end{matrix}$$

便得到一个 6×20 布局。图 W.10 画出了这个布局，其中每个 3×4 块都施以底纹。“块状”之名由此而来：结果布局中的每个 3×4 块，都恰好是原始 3×4 块的平移版本。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	1	2	3	24	25	26	27	48	49	50	51	72	73	74	75	96	97	98	99
1	4	5	6	7	28	29	30	31	52	53	54	55	76	77	78	79	100	101	102	103
2	8	9	10	11	32	33	34	35	56	57	58	59	80	81	82	83	104	105	106	107
3	12	13	14	15	36	37	38	39	60	61	62	63	84	85	86	87	108	109	110	111
4	16	17	18	19	40	41	42	43	64	65	66	67	88	89	90	91	112	113	114	115
5	20	21	22	23	44	45	46	47	68	69	70	71	92	93	94	95	116	117	118	119

图 W.10: 行主序 $(3,4):(4,1)$ 块与列主序 $(2,5):(1,2)$ 分块的 `blocked_product`。

操作 `raked_product` 与块状乘积类似，只是各模以相反的顺序拉链（zip）组合：

$$\begin{matrix}((3,\ 4),\ (2,\ 5))\\((4,\ 1),\ (12,\ 24))\end{matrix} \Longrightarrow \begin{matrix}((2,\ 3),\ (5,\ 4))\\((12,\ 4),\ (24,\ 1))\end{matrix}$$

其效果是 3×4 块被“耙”（raked）在 2×5 块之上，而非“块状”排布。图 W.11 画出了得到的布局，其中每个 3×4 块都施以底纹。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	24	48	72	96	1	25	49	73	97	2	26	50	74	98	3	27	51	75	99
1	12	36	60	84	108	13	37	61	85	109	14	38	62	86	110	15	39	63	87	111
2	4	28	52	76	100	5	29	53	77	101	6	30	54	78	102	7	31	55	79	103
3	16	40	64	88	112	17	41	65	89	113	18	42	66	90	114	19	43	67	91	115
4	8	32	56	80	104	9	33	57	81	105	10	34	58	82	106	11	35	59	83	107
5	20	44	68	92	116	21	45	69	93	117	22	46	70	94	118	23	47	71	95	119

图 W.11: 行主序 $(3,4):(4,1)$ 块与列主序 $(2,5):(1,2)$ 分块的 `raked_product`。

逻辑乘积还有不少其它版本，都会对模额外分组与重排，让结果更便于使用。

应用：逻辑切分

两个布局 $\mathbf{A}$ 与 $\mathbf{B}$ 的逻辑切分是一个布局 $\mathbf{R}$，其中“布局 $\mathbf{A}$ 被一分为二：被 $\mathbf{B}$ 指向的元素与未被 $\mathbf{B}$ 指向的元素”。具体地，我们将逻辑切分定义为一个从两个布局产生一个 2 阶布局的函数

$$\mathbf{A} \oslash \mathbf{B} = \mathbf{A} \circ (\mathbf{B}, \mathbf{B}^*_{|\mathbf{A}|}) = \mathbf{A} \circ \mathbf{B}^{\bigstar}$$

其中 $\mathbf{B}^*_{|\mathbf{A}|}$ 是 $\mathbf{B}$ 相对于布局 $\mathbf{A}$ 的尺寸所取的补。这个操作要保留 $\mathbf{A}$ 的全部元素、只是把布局 $\mathbf{A}$ 一分为二，所以我们要求 $\mathbf{B}^{\bigstar} = (\mathbf{B}, \mathbf{B}^*_{|\mathbf{A}|})$ 满射到 $\mathbb{Z}_{|A|}$。这通常要扩展 $\mathbf{B}^*$ 的尺寸，使 $\left|\mathbf{B}^{\bigstar}\right| \geq |\mathbf{A}|$。同时，我们还要求补 $\mathbf{B}^*$ 在如下意义上“补全”布局 $\mathbf{B}$：$\mathbf{B}^{\bigstar} = (\mathbf{B}, \mathbf{B}^*_{|A|})$ 具有自反广义逆 $\mathbf{B}^+$，满足

$$\mathbf{B}^{\bigstar}\mathbf{B}^{+}\mathbf{B}^{\bigstar} = \mathbf{B}^{\bigstar} \tag{W.30}$$

$$\mathbf{B}^{+}\mathbf{B}^{\bigstar}\mathbf{B}^{+} = \mathbf{B}^{+} \tag{W.31}$$

与逻辑乘积操作类似，结果布局 $\mathbf{A} \circ \mathbf{B}$ 的第一个模通常被称为“块”，它在“网格”（即“分块”）布局 $\mathbf{A} \circ \mathbf{B}^*_{|\mathbf{A}|}$ 上被重复。参数 $\mathbf{B}$ 被称为“分块器”，它可以扩展到布局之外，第 3.3 节会讲到。

先看个直接的例子。常见需求是从另一个布局中取出某组逻辑元素，比如每隔三个取一个，用复合就能做到：

$$\begin{matrix}24\\3\end{matrix} \circ \begin{matrix}8\\3\end{matrix} = \begin{matrix}8\\9\end{matrix}$$

$$\begin{matrix}(6,\ 2,\ 2)\\(2,\ 1,\ 20)\end{matrix} \circ \begin{matrix}8\\3\end{matrix} = \begin{matrix}(2,\ 2,\ 2)\\(6,\ 1,\ 20)\end{matrix}$$

在上面两种情形中，都是 24 个元素的布局与布局 8 : 3 复合，得到每隔三个取一个元素的布局。但左手边布局中“其余”的元素怎么办？逻辑切分把左手边的布局拆成两块：被右手边“命中”的部分与“其余”部分：

$$\begin{matrix}24\\3\end{matrix} \oslash \begin{matrix}8\\3\end{matrix} = \begin{matrix}24\\3\end{matrix} \circ \begin{matrix}(8,\ 3)\\(3,\ 1)\end{matrix} = \begin{matrix}(8,\ 3)\\(9,\ 3)\end{matrix}$$

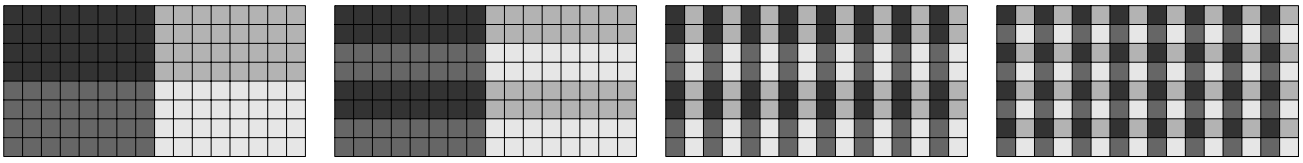
$$\begin{matrix}(6,\ 2,\ 2)\\(2,\ 1,\ 20)\end{matrix} \oslash \begin{matrix}8\\3\end{matrix} = \begin{matrix}(6,\ 2,\ 2)\\(2,\ 1,\ 20)\end{matrix} \circ \begin{matrix}(8,\ 3)\\(3,\ 1)\end{matrix} = \begin{matrix}((2,\ 2,\ 2),\ 3)\\((6,\ 1,\ 20),\ 2)\end{matrix}$$

其中 3 : 1 是 8 : 3 在尺寸 24 之下的补。结果的第一模按要求仍然包含左手边布局中每隔三个取一个的元素，而结果的第二模是这些“块”在原布局中的个数（本例中为 3）以及这些块之间的步幅。补用来确定缺失了哪些元素、以何种顺序保留它们。

相关切分　与复合类似，逻辑切分也经常按模来做，沿用同样的记号与策略

$$\mathbf{A} \oslash \langle \mathbf{B}, \mathbf{C} \rangle = (\mathbf{A}_0, \mathbf{A}_1) \oslash \langle \mathbf{B}, \mathbf{C} \rangle = (\mathbf{A}_0 \oslash \mathbf{B}, \mathbf{A}_1 \oslash \mathbf{C}).$$

这么设计的道理是：列和行可以各自独立地执行这些操作，然后视需要重排这些模、构造新布局，与布局乘积类似。例如，回到第 3.3 节中的分块器，将这些分块器用于逻辑切分而非复合，就得到分块器所指向的 4×8 子布局组成的 2×2 布局。图 W.12 画出了这一点：可以把每个单独施以底纹的数据块，想成有一个 2×2 布局映射到它上面。



(a) $\langle 4, 8\rangle \equiv \langle 4:1, 8:1\rangle$　(b) $\langle (2,2):(1,4), 8:1\rangle$　(c) $\langle (2,2):(1,4), 8:2\rangle$　(d) $\langle 4:2, 8:2\rangle$

图 W.12: 将 8×16 布局切分为 4×8 块、并以 2×2 分块的分块器示例。

例如，给定布局 $(8,16):(20,1)$，我们想抽取一个“块”：沿每列向下取 4 个连续元素、沿每行每隔一个取一个元素。按模的逻辑切分给出

$$\begin{matrix}(8,\ 16)\\(20,\ 1)\end{matrix} \oslash \left\langle \begin{matrix}4\\1\end{matrix}, \begin{matrix}8\\2\end{matrix} \right\rangle = \begin{matrix}(8,\ 16)\\(20,\ 1)\end{matrix} \circ \left\langle \begin{matrix}(4,\ 2)\\(1,\ 4)\end{matrix}, \begin{matrix}(8,\ 2)\\(2,\ 1)\end{matrix} \right\rangle = \begin{matrix}((4,\ 2),\ (8,\ 2))\\((20,\ 80),\ (2,\ 1))\end{matrix}$$

其中“块”模标为蓝色，“其余”模标为黑色。结果布局仍保持 8×16，而分块器所指向的元素已被置换到最前面的 4×8 块中。

操作 `zipped_divide` 先执行按模逻辑切分，再把同类模拉链式地组合在一起：

$$\begin{matrix}((4,\ 8),\ (2,\ 2))\\((20,\ 2),\ (80,\ 1))\end{matrix}$$

这生成了一个“块”模（蓝色），它恰好是分块器所指向的子布局；以及一个“其余”模（黑色），它是遍历各块的子布局。操作 `zipped_divide` 总是返回一个 2 阶布局结果，其第一模恰好是与分块器的复合，第二模是与补的复合。

软件中一个常见的模式是：把数据布局分块成一个块网格，然后为每个处理单元选出特定的块。在 Python 参考实现 PyCuTe（NVIDIA，2026）中，其形式如下：

```
data    = Tensor(MyPtr, MyLayoutMxN)   # Tensor: Crd -> Offset
tiler   = (4,8)                        # Tiler: TileCoord -> Crd
data_vt = zipped_divide(data, tiler)   # Compose: (TileCrd,GridCrd) -> Offset
tile    = data_vt[:, blk_id]           # 按一维 block id 切片
tile    = data_vt[:, (blk_x, blk_y)]   # 按二维 block id 切片，二者等价
```

这一模式与第 3.3 节的线程-值划分相似，只不过只需指定块模，缺失的“网格”模由分块器的补计算出来。这个形状为 (块, 网格) 的“补全”分块器随后与数据复合，产生块网格；再在第二模上按块标识符切片，即可为某个处理单元抽取特定的块。

逻辑切分同样可以构造出不少其它版本，做法相同：对模额外分组与重排，得到便于使用的结果。

4 结论

本文提出了 CuTe，一个用于表示与操纵张量布局的数学框架，以应对日益复杂的现代 GPU 架构。它的分层布局表示，为编写与管理当代硬件上专用张量指令所需的复杂数据布局与划分（partition）模式提供了通用方法；它的丰富代数运算，则为高性能线性代数 kernel 开发中布局的操纵与新布局的生成提供了一条有章可循的途径。二者结合，该表示与代数能够支持复杂的布局与划分、关注点分离，以及静态分析与优化。

CuTe 布局表示的表达能力已在若干关键应用中得到体现。CuTe 便于将多种多样的算法，例如广义张量-张量收缩（GETT）与卷积（CONV），表示为通用矩阵乘法（GEMM）的实例，代码得以复用，算法也因此统一。例如，CuTe 允许矩阵无论其具体数据布局如何都用 (m, n) 坐标（coordinate）索引，让通用算法与具体布局保持正交；要求特定布局的算法或指令则能够静态地检查、验证并推演它们的张量参数。

CuTe 布局代数支持对张量布局做静态分析与变换，用以实现并验证通用算法。例如，它可以为 `COPY` 操作推导最大向量化机会，验证矩阵乘累加（`MMA`）与 `COPY` 指令的输入输出布局，并推导出最优的共享内存布局，包括避免 bank 冲突的 swizzle 模式。有了这些能力，布局管理从易错的手工过程变成了一套系统、可验证的做法。为支持对本文定义的学习、实验与验证，纯 Python 参考实现 PyCuTe（NVIDIA，2026）已在 https://github.com/NVlabs/CuTe 公开发布。

CuTe 的实际影响体现在它已成功部署于生产系统，最显著的就是 NVIDIA CUTLASS v3 与 v4 库都建在它之上。在 CUTLASS 以及 FlashAttention 等应用中的性能评估表明，CuTe 的抽象不引入任何性能开销，软件开发却显著加快。采用这一框架的结果是：手工优化代码的性能特征得以保持，开发复杂度与开发时间大幅下降。

CuTe 的数学基础与代数化方法，使它能够随着架构创新持续演进。将布局关注点与算法逻辑分离，并提供强大的编译期推理能力，CuTe 确立了一种面向张量中心编程的方法论，能够满足各类计算系统与应用的需求。

致谢

作者感谢 Vijay Thakkar 作为 CuTe 的早期使用者，感谢他在 CuTe 的普及及其与 CUTLASS 的集成中发挥的关键作用，以及他对 CuTe 基础设计与部署的重要贡献。他还感谢 Andrew Kerr、Pradeep Ramani 与 Manish Gupta，感谢他们对于在 CUTLASS 中使用 CuTe 的洞见，以及他们推动 CuTe 在高性能 kernel 开发中的早期采用。他感谢 Muhammad Osama 与 Duane Merrill 在 CuTe 原型上的早期实验工作，以及在最初 CuTe 示例中所做的底层性能优化。最后，作者感谢 Michael Garland、Bastian Hagedorn、Jay Shah 与 Amanda Liu 提出的独到反馈，以及他们对 CuTe 规范一次次提出的挑战与质询。

第二十一章　CuTe（一）：Layout 基础与代数

21.1　本章导读

前十九章的 kernel 都是「手写地址算术」的产物：gmem 上算 (m, k) 的偏移、smem 上做 XOR swizzle、拼 ldmatrix 的 lane 地址、epilogue 里用 `__shfl_sync` 把 fragment 拼回连续向量。这套做法在第 十二、十三 章把 HGEMM 做到了 122 TFLOPS 量级，但它的知识结构是「散」的：同一组形状与步长的事实，以五种不同的写法散落在 kernel 的五个角落。Part IV 换一种做法：把 **(Shape, Stride) 提升为一等的数学对象**，用代数运算（复合、逆、乘积、切分）代替手写公式，把「我想要的布局」与「怎么算地址」这两件事分开。

本章是 Part IV 的原理篇开篇，也是全书唯一一章**不写任何 GPU kernel**：我们只研究 layout 本身。四章路线：本章（layout 代数）→ 第 二十二 章（Tensor 与 TiledCopy：把 layout 用在数据上）→ 第 二十三 章（TiledMMA：把 layout 用在计算上）→ 第 二十四 章（Swizzle 与 TMA：把 layout 用在硬件特性上）；随后第 二十五–二十七 章用这套语言重写 HGEMM 与 FlashAttention/FFPA，与前面手写版逐项对照。本章不假设读者用过 CuTe。

前置章节：第 十二 章（MMA fragment 与分块）、第 十三 章（XOR swizzle 的位运算）。**源码区间**：hgemm.cuh L788–820，即 Phase 7c-1 的 CuTe HGEMM 头注释与模板参数清单（本章只引用这个区间；CuTe 版 kernel 本体的逐行讲解留给第 二十五 章）。**编译宏**：`NOTES_V2_ENABLE_CUTE`（CuTe 路径需 CUTLASS 头文件可用，sm80+ 均可编）。配套最小测试 `book/tests/ch20_layout.cu`：host 端 CuTe 布局断言，不需 GPU kernel。

21.2　问题与动机

先盘点第 十三 章 swizzle kernel 中的布局知识。一个 $B_M \times B_K = 128 \times 64$ 的 A tile 从 gmem 搬到寄存器，同一组「形状与步长」的事实至少出现在五处：

1. **gmem 寻址**：A 是 row-major 的 $M \times K$ 矩阵，(m, k) 的地址为 $m \cdot K + k$；
2. **cp.async 搬运地址**：block tile 偏移、stage 偏移、16 B chunk 粒度的 gmem→smem 地址换算；
3. **ldmatrix 的 lane 映射**：row = lane mod 16、chunk = $\lfloor \text{lane}/16 \rfloor$——这是 MMA fragment layout 与 smem layout 的交点；
4. **XOR swizzle**：$addr' = addr \oplus ((\text{addr} \,\&\, \text{mask}) \gg S)$，而它的效果依赖 smem 行宽（第 十三 章实测：B_K=16 时 2-way 冲突、B_K=64 时 8-way）；
5. **epilogue**：C fragment 的 lane 分布、shuffle 拼接、128-bit 写回。

五处公式必须彼此一致。任何一处写错都不会报错，只会静默地错数据。更麻烦的是它们的联动：把 B_K 从 64 改成 32，五处里至少三处要跟着改；把 mma atom 从 m16n8k16 换成别的，第 3、5 处重算。第 十三 章末尾的「结论随行宽变」就是这个现象：**bank 冲突度不是 kernel 的属性，而是 layout 的属性**，它应该由工具从布局里推出来，而不是由人记住。

CuTe 的答案是：把一个 layout 定义成「**坐标 → 线性 offset** 的函数」，形状与步长只是它的表示参数；再给这套函数配一代数。kernel 作者声明三件事：gmem 上数据的 layout、smem 上 tile 的 layout、fragment 的 layout；剩下的地址计算由库在编译期折叠掉。本章把这套代数的定义与直觉讲透，第 二十五 章会看到 hgemm 的手写版（235 行去注释代码）变成 CuTe 版 kernel 本体 118 行，且后者没有一行地址算术、没有一条内联 PTX。

21.3　数学原理

21.3.1　Layout 是一个函数

在 CUDA 里访问一个元素，最终都要落到「基址 + 偏移」：拿到一段内存的起始指针后，真正决定「我要的 (m,k) 在哪里」的，是一个从坐标到偏移的映射。第 十二、十三 章手写 HGEMM 时，这个映射以五种面貌出现（gmem 寻址、cp.async 地址、ldmatrix lane 映射、XOR swizzle、epilogue 拼接），每种写一遍、彼此还要对齐。@ 竹熙佳处在《写给大家看的 CuTe 教程：Layout Compose & Inverse》里给出的视角是：把这些映射 **统一看成一个函数**：输入是逻辑坐标，输出是一个整数偏移，函数体只有整数乘法与加法。把这件事命名并形式化，就得到了 CuTe 的 layout。

定义（layout）。layout 由 shape 与 stride 两个参数组成：

- **坐标空间**：shape $S=(s_0,\dots,s_{k-1})$ 张成 $Z(S)=[0,s_0)\times\cdots\times[0,s_{k-1})$，其元素叫坐标 $\mathbf{c}=(c_0,\dots,c_{k-1})$。
- **线性化**：stride $D=(d_0,\dots,d_{k-1})$ 把坐标映成整数

$$\text{idx} \;=\; f(\mathbf{c}) \;=\; \sum_{i=0}^{k-1} d_i\, c_i. \tag{21.1}$$

- **colex 序**：式 (21.1) 里 c_0 是枚举最快的坐标（整数 x 加 1 时最先翻转的是 c_0；这与各 mode 的 stride 大小无关），c_{k-1} 最慢。CuTe 全书默认这一约定，来源是「column-major」：对 $S=(4,8),D=(1,4)$，坐标 (c_0,c_1) 是（行，列），$f=c_0+4c_1$，即 C 语义下的列优先。

用映射语言说，式 (21.1) 就是「k 维坐标按各方向的步长折算成一个整数」：每个 mode 贡献 d_ic_i，最后求和，没有别的运算。CuTe 的 `Layout` 就装着一对 (S,D)；调用 `layout(coord)` 得到式 (21.1) 的值。

还有一个容易反直觉的事实：**把整数 x 交给 layout** 时，CuTe 先把 x 按 colex 序分解回坐标

$$c_i \;=\; \Big\lfloor x \Big/ \textstyle\prod_{j<i} s_j \Big\rfloor \bmod s_i, \tag{21.2}$$

再代入式 (21.1)。这个分解只用到 shape、与 stride 无关，@ 竹熙佳处在教程里特别提示过这是初学时最容易卡住的一点。于是「f 在坐标上」与「f 在线性索引上」是同一个函数（等价地说：线性索引与坐标只是同一个函数的两种输入写法）。注意 $f(x)$ 一般不单调：colex 进位时 $x\to x+1$ 同时改变两个坐标（低位复位、高位 $+1$），offset 增量 $d_{i+1}-d_i(s_i-1)$ 可为负：$(2,3){:}(3,1)$ 的 offset 序列是 $0,3,1,4,2,5$，row-major 的 $(4,8){:}(8,1)$ 也有 $f(3)=24>f(4)=8$；只有当各 mode 的 stride 按 colex 排序（如 compact col-major，此时 $f(x)=x$）时才单调不减。这条单调性并非后续结论的前提：layout 的整数域本身是一个「一次遍历」的索引空间，后面所有遍历式断言（以及 `cosize`）依赖的只是 shape 分解 $[0,\text{size})\to Z(S)$ 的双射性，与单调性无关。

配套两个规模量：**size**（定义域基数）与 **cosize**（值域上界）

$$\text{size}(f)=\prod_i s_i, \qquad \text{cosize}(f)=1+\sum_i d_i(s_i-1). \tag{21.3}$$

其中 size 是定义域的元素个数，cosize 是值域的上界，$1+\sum d_i(s_i-1)$ 恰为「最大偏移加一」。当 cosize = size 时称布局是 **compact**（无空洞、双射）；否则有「孔洞」（padding/skip）。这个区分在 kernel 里很重要：CuTe 里 padding 后的一行 smem 就是一个带孔洞的 layout，而 `cosize` 直接就是这块 smem 实际要占的元素数，声明布局的同时把 smem 用量也算出来了。

- 例 1（compact col-major）：$S=(4,8),D=(1,4)$：$f(c_0,c_1)=c_0+4c_1$，$f(1,3)=13$、$f(3,1)=7$；$f(x)=x$（列优先 compact 下线性索引即恒等，本章测试 case A 对这个 layout 逐点核对了 128 个断言）。
- 例 2（row-major）：$S=(4,8),D=(8,1)$：$f(c_0,c_1)=8c_0+c_1$，$f(1,3)=11$。CuTe 里写 `make_layout(shape, LayoutRight{})` 得到它；注意 `make_layout(Shape)` 的默认是 column-major，与 C 数组的直觉相反（常见坑二）。
- 例 3（带孔洞）：$S=(8,4),D=(2,16)$：$f(c_0,c_1)=2c_0+16c_1$，size = 32 但 cosize $=1+2\cdot7+16\cdot3=63$，值域只命中偶数——这是一个「每两个位置放一个值」的稀疏布局（图 21.1）。

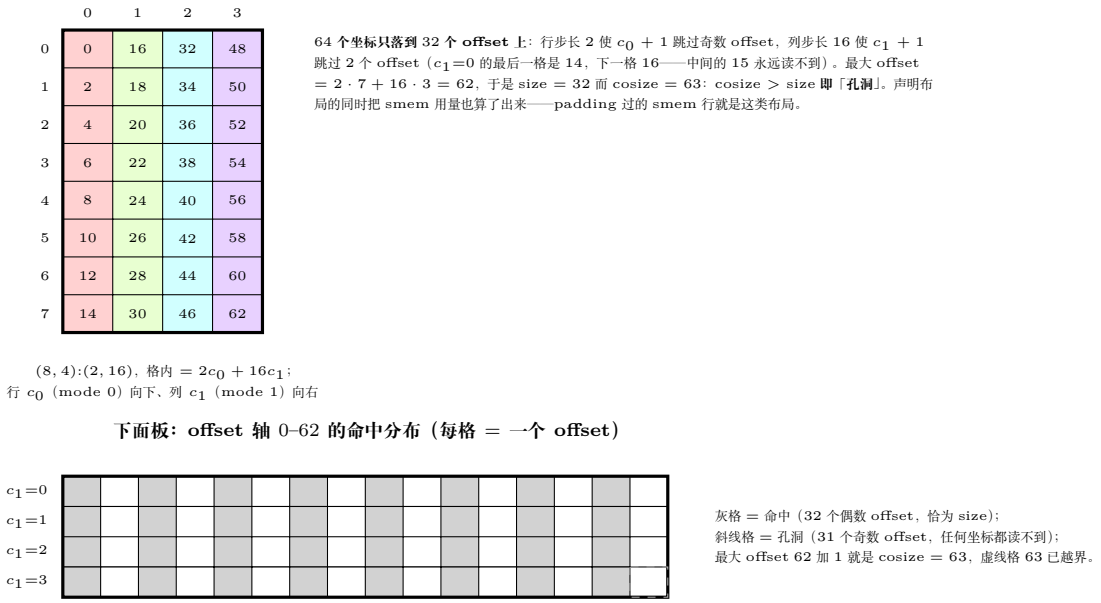


图 21.1: 带孔洞布局的 size 与 cosize（例 3，$L=(8,4):(2,16)$）：上面板把 64 个坐标的值列出来，行步长 2、列步长 16 使值域只取偶数，同一行的 8 个值跨 16 个 offset；下面板把 offset 轴 0–62 铺开——32 个灰格是命中（$=2c_0+16c_1$ 的全部取值）、31 个斜线格是永远读不到的孔洞，虚线的第 63 格已越界。cosize $=1+\sum_i d_i(s_i-1)=63$ 即「最大偏移加一」，也就是这块 smem 实际要占的元素数。

21.3.2 五个运算，五类实际问题

把 layout 当函数以后，kernel 里的布局操作就全部变成函数运算。CuTe 提供五个运算族，每一族都对应 kernel 里的一类实际问题。本节先按「问题 → 运算」的顺序把它们梳理一遍（@ 竹熙佳处在教程里指出过 CuTe 官方文档「没有前后联系地抛出一堆概念和代数运算」；本章刻意反向组织：先让每个运算有具体的应用场景，再在下一节给出定义与公式）：

1. **Compose（复合）：解决「两层映射怎么串联」**。三个真实场景：(a) **TV-layout 寻址**——拷贝划分先给出 (thread, value) 到逻辑坐标 (m,n) 的映射，而真正访问内存还要再叠一层 tensor 自身的 $(m,n)\to$ offset 映射，两次映射复合才是「线程 t 的第 v 个值在哪个地址」（第 二十二 章展开）；(b) **swizzle 消 bank conflict**——swizzle 本身是 offset→offset′ 的映射，把它复合到 smem 布局的外层，之后代码仍可用正常 (m,n) 坐标访问，物理地址已被改写而调用代码无需感知（4.1 节的 `SmemLayoutA` 就是 `Swizzle`∘atom 的一次复合）；(c) **FA2 的 P 矩阵转换**——给 QK 的 accumulator 布局复合一次「列拆半」变换，P 就能原地重解释为 PV 的 A 操作数（4.8 节完整推一遍）。

2. **Inverse（逆）：解决「从 offset 反查坐标」**。给定一个 offset 反查它的坐标：copy 的地址反解、`max_common_layout`（求两个布局的最大公共连续块）都以它为基础；4.8 节的代数法更是把它当作 layout 变换的中心工具。

3. **Product（乘积）：解决「小布局怎么铺成大布局」**。smem 上 $8\times B_K$ 的 swizzle atom 要铺满 $(B_M,B_K,k_{\text{Stage}})$ 的完整布局，靠的就是 product 族（`tile_to_shape`）；「两块数据并排摆、坐标分量相加」是它的几何语义。

4. **Divide（切分）：解决「大矩阵怎么切块」**。GEMM 的 `local_tile`、注意力里对 K/V 的分块遍历，底层是同一条运算：把大 layout 切成「tile 内布局 + tile 索引」两段；`zipped_divide` 的实测是本章最核心的数据（4.5 节）。

5. **Mode 操作（投影/合并/展平/化简）：解决「索引结构不合用的问题」**。MMA fragment 与 remainder mode 的分离（第 二十三 章）、epilogue 把 $(\text{CPY}_M,\text{CPY}_N)$ 两层循环压成一个线性索引（第 二十二 章），都是这类操作。

一个 layout 恒等式是否成立，判据只有一条：**对定义域内每个坐标逐点相等**。CuTe 自己的单测也是这么写的（例如 `test_composition` 对 $c<\text{size}(B)$ 断言 $R(c)=A(B(c))$），本章测试沿用同一手法：所有代数结论都用「全遍历 + 逐点断言 + 双射覆盖检查」验证，而不是抽几个样例做粗略对照。

21.4　设计与实现

21.4.1　先看「类型即布局」：hgemm.cuh 的 CuTe 类型清单

第 十三 章的手写 kernel 把布局知识藏在常量与宏里；CuTe 版则把它们全部提升为模板参数。先看 hgemm.cuh L788–805 的头注释，它概括了 CuTe 版 kernel 的流水线结构，与第 十二、十三 章手写版一一对应：

头注释（Phase 7c-1：TN 布局 + Smem Swizzle + Multistage Pipeline）先声明 TN 布局：$C_{M\times N} = A_{M\times K} \cdot B^{\top}_{N\times K}$，$A$ 为 row-major，$B^{\top}$ 也为 row-major（等价于 B 是 col-major 的 $[K \times N]$）；在 CuTe 中由 `make_tensor(make_gmem_ptr(ptr), shape, stride)` 描述。流水线流程（stage 是 K tile 的循环缓冲槽，不是一条完整计算链）：

1. **PREFETCH**：预加载 kStage−1 个 K tile（G→S via `cp.async`），填满 stage。
2. **主循环 over K tiles**：(a) 从当前 stage 做 S→R——`cute::copy(...)`，落到 ldmatrix；(b) 对当前 K slice 做 MMA——`cute::gemm(...)`，落到 `mma.sync`；(c) 在处理当前 tile 的首个 MMA slice 时，异步提交下一个 tile 的 G→S；(d) 当前 tile 的最后一个 K slice 完成后，等待并切换 smem stage。
3. **Epilogue**：R→S（`UniversalCopy` 写入 C scratchpad）→ S→G（128-bit store）。

头注释末尾是一组「面试对比」，比较手写 MMA 与 CuTe 的代码量：手写版要做 ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle，约 200 行；CuTe 版只需 partition + copy + gemm，约 80 行，其余由 launch wrapper 的类型推导完成。

再看紧随其后的模板参数清单，本章的核心主题「layout 是类型」在这里体现得最直接：

hgemm.cuh L806-820（模板参数：TiledMMA / 各类 Copy / SmemLayout 全是类型）

```
template <typename T,                    // element type (half)
          int BM, int BN, int BK,        // block tile: BM=128, BN=256, BK=32
          int kStage,                    // cp.async pipeline depth (2)
          typename TiledMMA,             // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
          typename G2SCopyA,             // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
          typename G2SCopyB,             // G→S B: same as G2SCopyA
          typename SmemLayoutA,          // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
          typename SmemLayoutB,          // smem B: same atom → (BN,BK,kStage)
          typename SmemLayoutC,          // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
          typename S2RCopyAtomA,         // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
          typename S2RCopyAtomB,         // S→R B: same as S2RCopyAtomA
          typename R2SCopyAtomC,         // R→S C: UniversalCopy<int> (32-bit store)
          typename S2GCopyAtomC,         // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
          typename S2GCopyC,             // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
          const int BlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
```

逐项解读（第 二十二–二十四 章会把这些类型逐个拆开）：

- `TiledMMA`：MMA 的「原子 + 排布」，注释写明 `SM80_16x8x16_F16F16F16F16_TN` 且 EURepeat $= 2\times2\times1$；
- `G2SCopyA/B`：gmem→smem 的搬运原子（`cp.async` 的 16 B 版本）；
- `SmemLayoutA/B`：smem 上 A/B tile 的布局——`Swizzle<3,3,3>` 与 $8\times B_K$ atom 复合再铺满 $(B_M, B_K, k_{\text{Stage}})$，正是第 十三 章手写 XOR swizzle 的声明式写法；
- `S2RCopyAtomA/B`：smem→ 寄存器（`ldmatrix.x4`）；
- `R2SCopyAtomC/S2GCopyAtomC/S2GCopyC`：epilogue 的寄存器 →smem（32-bit）与 smem→gmem（128-bit）搬运。

这些「类型」就是布局本身：它们由 `Int<N>`（编译期整数常量，类型级）、`Shape`、`Stride`、`Layout`、`Swizzle` 等拼装出来。由于 shape/stride 是类型的一部分，`cosize`、复合、切分的结果都可以在 **编译期**折叠为常数，这是后文「零成本抽象」论证的根基。

实践须知

- 模板参数清单里的 `TiledMMA`、`G2SCopyA`、`SmemLayoutA` 等名字只说明「类型角色」；具体型号（哪个 MMA 原子、哪个 cp.async 变体、哪个 ldmatrix 方向）要去 launch wrapper 的构造段（hgemm.cuh L1303–1377）核对。本书逐项核过（CUTLASS v4.6.1）：F16 累加路径用 `SM80_16x8x16_F16F16F16F16_TN`（F32 累加则切到 `F32F16F16F32_TN`），G2S 用 `SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>`，S2R 用 `SM75_U32x4_LDSM_N`，R2S/S2G 用 `UniversalCopy<int>` 与 `UniversalCopy<uint128_t>`，与注释一致。
- 类型名随 CUTLASS 版本变化：换版本后「注释与实现是否一致」要重核，不要照抄本节列表。
- 更省事的做法是把注释声称的类型写进 `static_assert`（或按本章测试的做法在 host 端做类型断言），让「注释与实现一致」变成编译期检查，而不是依靠人工阅读核对。

21.4.2 Compose：$R(c) = A(B(c))$

先看 compose 解决的三个真实问题，再看定义。

场景一：TV-layout 寻址（第 二十二 章的核心内容）。 `make_tiled_copy` 类划分先算出「$(t, v) \to (m, n)$」：线程 t 的第 v 个值对应 tile 的第 (m, n) 格。但 (m, n) 还只是*逻辑*坐标；要把值真正取出来，还得再叠一层「$(m, n) \to$ 物理 offset」的 tensor 布局。两次映射的复合（$(t, v) \to (m, n) \to$ offset）才给出每个线程的访问地址。@ 竹熙佳处把这一点的价值概括为「变换 layout 时只关注坐标空间」：同一个 (m, n) 复合不同的 tensor 布局，就能分别正确落到 global / shared / register 上，这是一份划分同时服务三种内存的关键。

场景二：swizzle 消 bank conflict。 swizzle 是 offset→offset′ 的映射。对一块 smem 布局做 swizzle 后，访问链变成 $(m, n) \to$ offset $\to$ offset′：这相当于在原函数外面**再叠一层 swizzle 函数**，所以源码里 swizzle 总是出现在 `composition` 的第一个参数位置（A 的位置，见 4.1 节的 `SmemLayoutA` 定义）。这样后续代码仍按正常 (m, n) 坐标写字，物理地址已经被换掉，而「换掉」这件事对上层透明。第 十三 章手写版要人工维护的那套 XOR 公式，在这里收缩成一次复合。

场景三：FA2 的 P 矩阵转换（4.8 节完整展开）。 QK 累加器与 PV 左操作数是同一批寄存器的两种索引方式，两个索引方式之间恰好差一次「列拆半 + 重组」的复合，这就是 4.8 节两条路线要解的题。

定义 20.1（composition）。 给定 layout A（任意 rank）与 B，若 B 的值域以「整除」方式落在 A 的 mode 结构里（CuTe 称 divisibility 条件），则 $R = \text{composition}(A, B)$ 是满足

$$R(\mathbf{c}) = A\big(B(\mathbf{c})\big), \qquad \forall \mathbf{c} \in Z(B) \tag{21.4}$$

的 layout，且 R 的顶层 shape 与 B 一致（CuTe 注释原文：`@post compatible(rhs, result)`、`@post result(c) = lhs(rhs(c))`）。直觉：B 决定取 A 定义域中的哪些坐标，A 决定坐标对应的 offset，复合是把 B 的取值翻译到 A 的坐标系里。

例子 1（stride 折叠，非平凡）：$A = (6, 2):(9, 2)$、$B = (4):(1)$。逐坐标算：$B(0..3) = 0, 1, 2, 3$，而 $A(x) = 9\,(x \bmod 6) + 2\lfloor x/6 \rfloor$，故 $A(B(c)) = 9c$ $(c < 4)$，于是

$$R = (4):(9). \tag{21.5}$$

结论：复合结果的 stride 等于 A 在 B 的像上的「斜率」。本例中 B 的像落在 A 的 mode-0 上，斜率就是 9。这也是「CuTe 用 layout 代数做坐标变换」的最小例子。

例子 2（恒等段）：$A = (4, 8):(1, 4)$、$B = (2, 2):(2, 1)$。B 的像是 $\{0, 1, 2, 3\} \subset [0, 4)$，而 A 在 $[0, 4)$ 上恒等，故 $R = B$ 逐点成立。本章测试对这两个例子累计 14 个断言，全部通过。

教训（也是常见坑）：**compose 不是无条件的函数复合。**当 B 的 stride 结构与 A 的 mode 结构不整除时，CuTe 会先按 A 的 colex 结构重解释 B（`coalesce_x`），结果依然是一个合法 layout，但语义不再是「几何意义上的先 B 后 A」，4.5 节的 tile 反例会给出实例。

21.4.3 Inverse：右逆与左逆

inverse 回答的问题最直白：「给定一个 offset，它是哪个坐标的?」但真正常用的场合不止反解坐标：

场景一：copy 的坐标反查。（第 二十二 章）拷贝划分需要把「tile 上的位置」拉回「atom 内部的坐标」，用的就是 $A(R(i)) = i$ 这条右逆性质；`max_common_layout`（求两个布局的最大公共连续块，判断能否直接互拷）也以右逆为基础。

场景二：layout 变换经由逆运算（4.8 节）。FA2 的代数法第一步就是 `left_inverse`：把 accumulator 的「坐标→offset」翻成「offset→ 坐标」，再把目标 A 布局复合上来；「输入输出对调」是这类变换的关键步骤。

一个实践提醒（也来自 @ 竹熙佳处）：单独做一次 inverse 得到的 layout 往往没有语义，通常要再 `with_shape`（本质上还是复合一个布局）把结果排列成想要的形状，它才是「可用的布局」。这条在 4.8 节会再次出现。

定义 20.2（right_inverse）。$R = \text{right_inverse}(A)$ 满足

$$A\big(R(i)\big) = i, \qquad \forall i < \text{size}(R), \tag{21.6}$$

等价地 $\text{composition}(A, R)$ 恒等于 $\text{make_layout}(\text{shape}(R))$（即恒等布局）。前提：$A$ 静态。

定义 20.3（left_inverse，准逆）。$R = \text{left_inverse}(A)$ 满足

$$A\Big(R\big(A(\mathbf{c})\big)\Big) = A(\mathbf{c}), \qquad \forall \mathbf{c}, \tag{21.7}$$

即 $\text{composition}(A, \text{composition}(R, A))$ 恒等于 A；当 A 是单射时它就是真正的左逆。

两个定义初读不易区分，看实测行为就清楚了（本章测试 case C/D 的数据）：

- $A = (4,8):(1,4)$（compact col-major，双射）：$R = (32):(1)$，即「offset → 坐标」的 colex 逆映射；此时 A 在 $[0, 32)$ 上恒等，逆也是恒等。
- $A = (8,4):(2,16)$（带孔洞）：$R = (1):(0)$——**退化为平凡右逆**。原因：A 的值域是偶数集合，不含 1；而式 (21.6) 要求 $A(R(1)) = 1$，长度 ≥ 2 的右逆根本不存在。所以右逆「非平凡」的条件是 A 的像包含从 0 开始的连续段。
- 同一个孔洞 layout 的左逆：$R = (2,32):(0,1)$。它的映射是 $R(o) = \lfloor o/2 \rfloor$（把偶 offset 拉回坐标）。注意 shape 里那个 $(2):(0)$：**stride-0 的 mode 是「歧义位」**——它对 offset 没有贡献，作用是记录「每个坐标至多对应 2 个 offset 候选」（值域以 2 为间隔分布）；真正的连续展开在 $(32):(1)$ 那一层。这是 CuTe 用 layout 编码「多对一」的方式，测试里对 32 个坐标逐一验证了式 (21.7)。
- 二者关系：对 colex compact（双射）的 layout，左逆与右逆逐点相同（测试 32 点核对）。

逆运算的用途不是「反解坐标」这么窄：右逆负责「值域中连续的那一段」，左逆负责「整个陪域」（含多对一的情形），选择依据就是本小节的实测：需要哪一种坐标变换方向，就选哪一个逆。

21.4.4　Product：乘积

product 解决的是「小布局怎么铺成大布局」：smem 上一块 $8 \times B_K$ 的 swizzle atom 只是基本单元，kernel 真正要声明的是整块 $(B_M, B_K, k_{\text{Stage}})$ 的 A tile 布局（4.1 节的 `SmemLayoutA` 就是这么拼出来的）。product 族的语义很直观，**把两块布局在 mode 上相乘（拼接）：坐标各自输入，offset 相加**：

定义 20.4（concatenation）。$P = \text{make_layout}(A, B)$（即 `product`）把两个 layout 相乘：

$$\text{shape}(P) = \big(\text{shape}(A), \text{shape}(B)\big), \quad \text{stride}(P) = \big(\text{stride}(A), \text{stride}(B)\big), \tag{21.8}$$

$$P(\mathbf{c}_A, \mathbf{c}_B) = A(\mathbf{c}_A) + B(\mathbf{c}_B). \tag{21.9}$$

实测：$\text{make_layout}\big((4,8):(1,4),\ (8,4):(2,16)\big) = \big((4,8),(8,4)\big):\big((1,4),(2,16)\big)$。注意 mode 的折叠方式：$A$ 的整棵 shape 成为 P 的第一个 mode，所以 2 个 rank-2 layout 相乘后 **外层是 rank 2**（shape 为 $\big((4,8),(8,4)\big)$），不是 rank 4。测试对 $P(\text{make_coord}(a_0, a_1), b) = A(a_0, a_1) + B(b)$ 逐点核对 24 个断言。

Product 家族还有几个更「kernel 化」的成员，本章只简要介绍、第 二十五 章展开：`logical_product`（把 block 铺在 tiler 上）、`blocked_product`（按 tiler 重复 block）、`raked_product`（交错重复）、`tile_to_shape`（按目标 shape 铺 atom，最小测试 case G 的实测用到它）。

21.4.5 Divide：tile 的数学

divide 是 kernel 里用得最多的一条：`local_tile` 把大 Tensor 切出当前 CTA 的块（4.1 节 gA/gB/gD 的切法）、注意力里对 K/V 的分块遍历、4.8 节 tri-dao 的 P 转换，底层都是同一条运算：**把一个大 layout 切成「tile 内布局 + tile 在外面的索引」**。完整定义分两步。

第一步：complement。对 tile layout T，$\mathrm{complement}\big(T, \mathrm{shape}(L)\big)$ 返回「补空间」的 layout：把 T 的 codomain 之外的地址空间按最小可用 stride 依次填满，直到覆盖 L 的 codomain。它回答「tile 之外的剩下那些 offset 怎么编号」。

第二步：logical_divide 与 zipped_divide。

$$\mathrm{logical_divide}(L, T) = \mathrm{composition}\Big(L,\ \mathrm{make_layout}\big(T,\ \mathrm{complement}(T, \mathrm{shape}(L))\big)\Big), \tag{21.10}$$

$$\mathrm{zipped_divide}(L, T) = \mathrm{tile_unzip}\big(\mathrm{logical_divide}(L, T), T\big). \tag{21.11}$$

式 (21.10) 一次说清三件事：结果分两大 mode，第一个是 T 自己（tile 内布局），第二个是补空间（tile 索引）；整个结果 = 把 L 重新沿这两个 mode 解释。`zipped_divide` 把结果再整理成「(tile 分量, rest 分量)」两个 mode；`tiled_divide/flat_divide` 则是把 rest 分量展平的变体。

本节给出一条逐点实测（测试 case F）：取 $L = (8,8):(1,8)$（col-major 的 8×8 逻辑矩阵）、$T = (4,4):(1,8)$（与 L 同 stride 族的 4×4 tile）：

$$\mathrm{zipped_divide}(L, T) = \big((4,4),(2,2)\big):\big((1,8),(4,32)\big), \tag{21.12}$$

按坐标写开：tile 内坐标 (t_0, t_1) 贡献 $t_0 + 8t_1$，rest 坐标 (r_0, r_1) 贡献 $4r_0 + 32r_1$，于是

$$\mathrm{zd}(t_0, t_1, r_0, r_1) = (t_0 + 4r_0) + 8\,(t_1 + 4r_1) = t_0 + 8t_1 + 4r_0 + 32r_1. \tag{21.13}$$

式 (21.13) 括号里就是「行 $= t_0 + 4r_0$、列 $= t_1 + 4r_1$」：r_0 是行方向的 tile 编号（步进 4 行）、r_1 是列方向的 tile 编号（步进 4 列 $\times$ 8 行的列 stride $= 32$）。这正是「4×4 tile、2×2 个 tile」的严格数学表达。测试做了三重核对（76 个断言）：(a) 结构，rank $= 2$、$\mathrm{size}_0 = 16$、$\mathrm{size}_1 = 4$、$\mathrm{compatible}(T, \mathrm{layout}_0)$ 为真；(b) 坐标公式，64 组 (t, r) 全部满足式 (21.13)；(c) **双射覆盖**，64 个输出恰好把 $[0, 64)$ 各命中一次，这条约束最强：它同时排除了重叠与遗漏。两种 tiler 下 tile 几何的正反例对照见图 21.2。

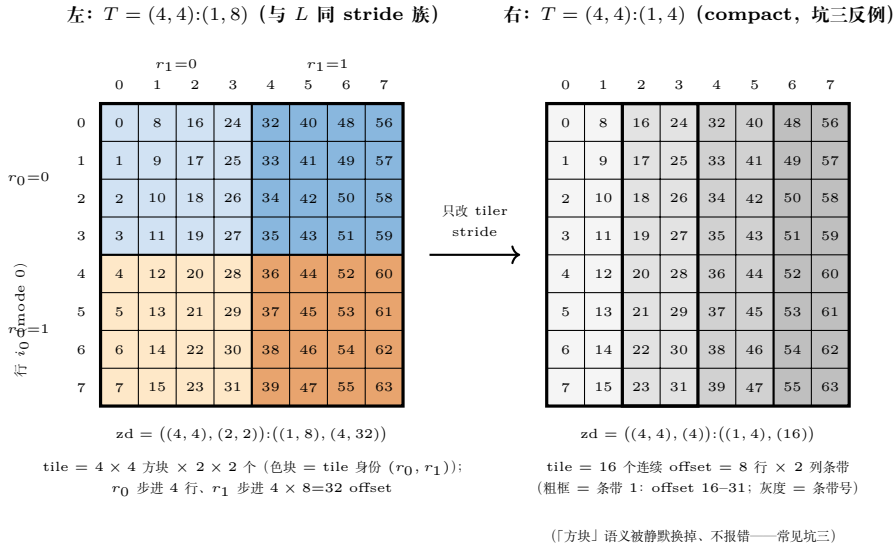


图 21.2: zipped_divide 的 8×8 实测与坑三 compact-tiler 反例（测试 case F）：格内数字为 $L = (8,8):(1,8)$ 的线性 offset（$= i_0 + 8i_1$，64 格逐格满足式 (21.13)）。左：tiler 与 L 同 stride 族时，tile 是 2×2 个 4×4 方块（色块 = tile 身份）；右：compact tiler $(4,4):(1,4)$ 的「tile」退化为 16 个连续 offset 的「8 行 $\times$ 2 列」列条带——结果仍是合法 layout、双射覆盖不变，但「方块」语义被静默换掉、不报错（常见坑三）。

实践须知

- **tile 的 stride 必须与目标 layout 同族**。divide 的语义完全取决于 T 的 stride 是否与 L 的 stride 结构相容：把上面的 T 换成「compact col-major 的 4×4」$(4,4):(1,4)$，结果是 $\big((4,4),(4)\big):\big((1,4),(16)\big)$——tile 变成 16 个

连续 *offset*（在 col-major 的 8×8 里即「8 行 × 2 列」的列条带），不再是 4×4 方块；同理 row-major 全体（$L=(8,8):(8,1)$、$T=(4,4):(4,1)$）会得到 $\big(((2,2),4),4\big):\big(((32,1),8),2\big)$ 这种被 colex 结构重排的结果。

- **正确写法**：tiler 用 $(B_M,B_K):(d_M,d_K)$，stride 从目标 layout 派生（hgemm 的 `local_tile` 与 `SmemLayoutA` 都是这么写的）；不要为省事写 compact tile。
- **判别法**：检查 T 的 stride 是否等于 L 在对应方向上的 stride（本章案例应为 $(1,8)$）。不相等时结果仍是合法 layout，只是语义被默默换掉——不报错（另见常见坑三）。

21.4.6　Mode 操作：投影、合并、展平、化简

mode 操作解决的是「索引结构不合用」的问题：MMA 的 fragment mode 与 rest mode 混在一起时要把它们分开（第 二十三 章），epilogue 要把 $(\mathrm{CPY}_M,\mathrm{CPY}_N)$ 两层循环压成一个线性索引来遍历（第 二十二 章），都要对 shape 这棵树做结构调整。一个 layout 的 shape 是一棵树（mode 里还能嵌套 mode）。对树的操作有：`layout<I>(L)`（等价 `get<I>(L)`）取第 I 个 mode 的子 layout；`select<Is...>` 取多个；`group<B,E>(L)` 把模式区间 $[B,E)$ 合并成一个 mode；`flatten` 把嵌套展平为一层；`coalesce` 合并所有「可合并」的相邻 mode（stride 恰好衔接）。实测（本章测试）：

- $L_g=\big((2,4),(3,2)\big):\big((1,2),(8,24)\big)$；
- $\mathrm{layout}_0(L_g)=(2,4):(1,2)$，$\mathrm{layout}_1(L_g)=(3,2):(8,24)$——取 mode 就是取子 layout；
- $\mathrm{group}_{[0,2)}(L_g)$ 把两个 mode 合成一个（外层变为 rank 1，size 不变）；
- $\mathrm{flatten}(L_g)=(2,4,3,2):(1,2,8,24)$；
- $\mathrm{coalesce}(L_g)=(48):(1)$——该 layout 恰好整体连续，化简成单 mode。

一个命名上容易混淆的点：很多 CuTe 教程（以及 hgemm.cuh 的部分注释）用 `group_modes<B,E>` 这个名字。在本仓库 CUTLASS v4.6.1 里，这两个名字分属两层：**layout 级是 `group<B,E>(layout)`（配 `select<I...>` 取多个），Tensor 级才是 `group_modes<B,E>(tensor)`**（定义在 `cute/tensor_impl.hpp`，内部就是一行 `make_tensor(t.data(),group<B,E>(t.layout()))`；hgemm.cuh L1073 的 `group_modes<1,3>` 用的正是 Tensor 版，第 二十二 章 4.5 节会再遇到它）。阅读旧教程时按层替换即可，语义一致。

21.4.7　运算一览与 hgemm 的对应位置

运算	数学	CuTe API	在 hgemm CuTe 版中的位置（预览）
compose	$R(c)=A(B(c))$	`composition` / `compose`	`SmemLayoutAtom` = `Swizzle`∘atom
inverse	$A(R(i))=i$（准逆见正文）	`right_inverse` / `left_inverse`	公共连续块、坐标反查（第 二十二 章）
product	$P(c_A,c_B)=A(c_A)+B(c_B)$	`make_layout(A,B)`、`blocked_product`、`raked_product`、`tile_to_shape`	$8\times B_K$ atom → $(B_M,B_K,k_{\text{Stage}})$
divide	式 (21.10)、(21.11)	`logical_divide` / `zipped_divide`	`local_tile` 的 gmem tile 选取
mode	投影/合并/展平/化简	`layout<I>`、`select/group`、`flatten/coalesce`	fragment 与 rest mode 分离（第 二十二、二十三 章）

表 21.1：本章五个运算族与其在各章的位置对照（表 20-1）。

21.4.8 Compose & Inverse 实战：FA2 的 P 矩阵布局转换

本节把 compose 与 inverse 两个运算用在 FlashAttention 的经典难题上：同一批寄存器，在上一个 mma 里是输出，在下一个 mma 里要作为输入。

问题。FA2 的核心是两连乘 $P = QK^\top$ 与 $O = PV$（为聚焦布局问题，先忽略 softmax）。第一段 mma 的输出是 accumulator，按 C fragment 排布 (MMA $= 4$, MMA_M, MMA_N)；第二段 mma 的左操作数 P 要按 A fragment 排布；m16n8k16 下虽然都是 4 个寄存器，但 C 的 4 个按「2 行 × 2 列」组织、A 的 4 个按「2 行 × 2 个 K 向切片」组织，把 C 直接当 A 用会得到寄存器对齐错误（`cute::gemm` 静态报错拒绝）。第 十七 章的手写版靠一份「P 布局约定」注释 + 人工挑寄存器位置（`R_S[i][w]`、`R_S[i][w+1]` 送入 `HMMA16816`）绕开这个问题；到了 CuTe 里，同一件事必须写成一个 **layout 变换**。

路线一（tri-dao 的 logical_divide 法：几何重排）。把 MMA_N 拆成 $(2, \mathrm{MMA}_N/2)$，再把「后 2」与 MMA mode 重组：

$$(\mathrm{MMA},\ \mathrm{MMA}_M,\ (2, \mathrm{MMA}_N/2)) \longrightarrow \big((\mathrm{MMA}, 2),\ \mathrm{MMA}_M,\ \mathrm{MMA}_N/2\big). \tag{21.14}$$

用映射语言说：先把 S 的列方向拆成两半，再把「后一半」拼进 MMA mode，于是 C 的「相邻两列」在 A 的坐标系里成了「K 向相邻两个元素」。

路线二（reed 的 layout-algebra 法：代数复合）。构造一条三步的映射链：先求 C 布局的逆（offset 拉回 C 的坐标），再复合 A 布局（把 A 的坐标翻译到 C 的坐标系），最后复合 C 本身（顺着 C 读回 offset）：

$$\mathrm{Layout}_C^{-1} \leftarrow \mathrm{left_inverse}(\mathrm{Layout}_C) \qquad (\text{offset} \to \text{C 坐标}) \tag{21.15}$$

$$a \leftarrow \mathrm{Layout}_C^{-1} \circ \mathrm{Layout}_A \qquad (x_a, y_a) \to \text{offset} \to \text{C 坐标} \tag{21.16}$$

$$\mathrm{Layout}_C^{A} \leftarrow \mathrm{Layout}_C \circ a \qquad (x_a, y_a) \to \cdots \to \mathrm{offset}_C \tag{21.17}$$

三步的读法：第一步把输入输出对调，第二步把 A 的坐标经 offset 换到 C 的坐标系，第三步沿 C 的映射读回 offset。这也是 4.3 节「inverse 要配 `with_shape` 才有语义」的完整实例：单独一次 `left_inverse` 的结果没有形状语义，复合上目标布局它才成为可用的变换。

本书用的是哪种。看源码：flash_attn.cuh 的共享工具函数 `convert_layout_acc_Aregs`（L2341–2372）就是路线一：`logical_divide` 拆 mode + `make_layout` 重组四个碎片；ffpa_attn.cuh 的 P 转换（L300–305、L584–588）直接复用这个函数，注释写明「与 TMA WS kernel 一致」。两个文件的 P 路径都走 **tri-dao 的几何法**。选它的理由很实际：这是一次纯 mode 重排，6 行代码就能写完、效果可直接验证；路线二的代数法把「C→A」表达成「逆 + 两次复合」，更**通用**（换任意两个 fragment 布局都能推，例如量化 gemm 里 dtype 变化引起的布局变换），但中间布局要 `with_shape` 才可读、且依赖静态 stride；在本例中它的收益与几何法相同。两法等价这件事本身是个好练习：第 二十六 章会看到该变换在 CuTe 版 FA 里的完整位置（softmax 之后、PV 之前），对照本节把两条路线各推一遍即可复现。

两条路线的对照示意见图 21.5。

21.5 图示

21.6 性能分析

本章没有 kernel，性能一节只回答一个问题：**这套代数要付多少代价？**答案是三句话：编译期全折叠、运行期只剩加法与位运算、代价转移到编译时间。

证据一（编译期折叠）：static layout（所有 shape/stride 都是 `Int<N>`）上，`cosize`/`shape`/`compose`/ `divide` 的结果都是 `constexpr`。本章测试把 `tile_to_shape` 的结果直接写进了编译期断言：

```
static_assert(cosize(decltype(atom){})  == 256,  "atom cosize folds");
static_assert(cosize(decltype(tiled){}) == 8192, "tiled cosize folds");
```

编译通过即证明：$128 \times 32 \times 2$ 的 smem 布局的全部尺寸计算没有进入运行期一行代码。

证据二（运行期退化为机器指令）：compose/divide 之后的地址计算在 SASS 层面就是「基址 + 若干次加法/移位」。XOR swizzle 的复合在编译期折叠为位运算，本章测试对 8×32 布局逐点核对了 `Swizzle<3,3,3>` 的复合结果与位公式 $\mathrm{off} \oplus (((\mathrm{off} \gg 6) \,\&\, 7) \ll 3)$ 完全一致（256 点、0 差异）。这与第 十三 章手写 XOR 的 30,720 点 v1/v2 等价性结论互相印证：**声明式的 Swizzle ∘ layout 与手写位运算生成的是同一个函数**。

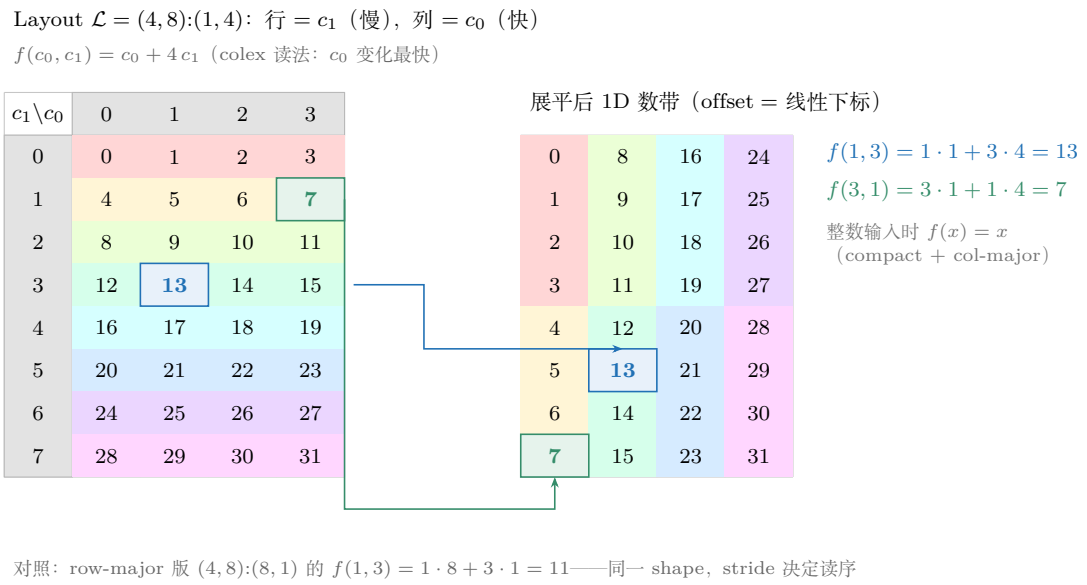


图 21.3: colex 线性化：坐标 (c_0, c_1) 与 offset 的对应表（左），同一布局展平后的 1D 数带（右）。c_0 沿行方向变化最快，故 idx $= c_0 + 4c_1$；整数输入时 CuTe 用式 (21.2) 先把 x 分解回坐标。compact 且 col-major 时 $f(x) = x$。对照 row-major 版 $(4,8):(8,1)$：$f(1,3) = 11$——同一 shape，stride 决定读序。

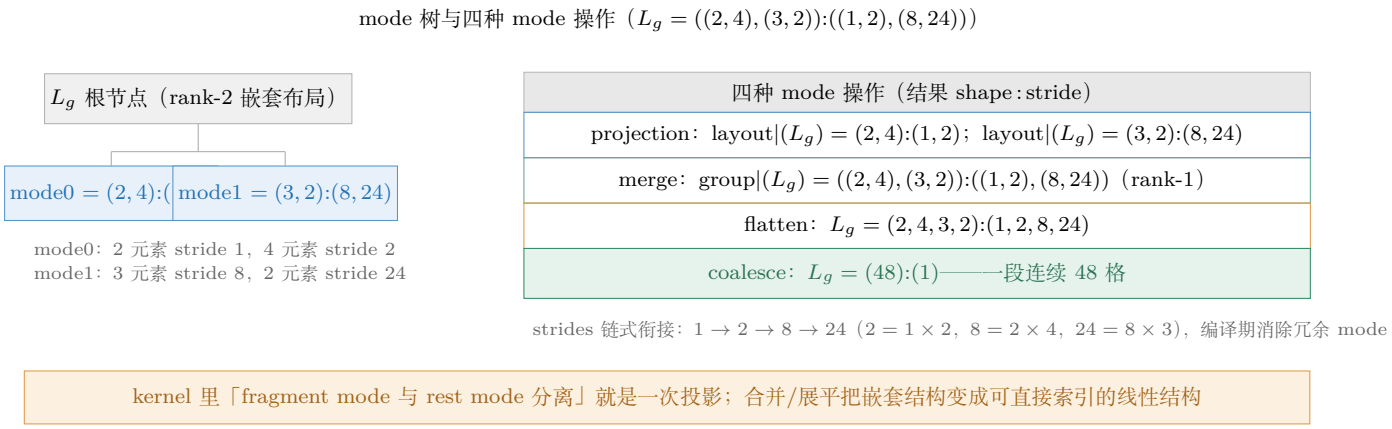


图 21.4: mode 树与四种 mode 操作的结果。投影用于取子布局，合并/展平用于把嵌套结构变成可直接索引的线性结构，化简（coalesce）在编译期消除冗余 mode——kernel 里「fragment mode 与 rest mode 分离」就是一次投影。

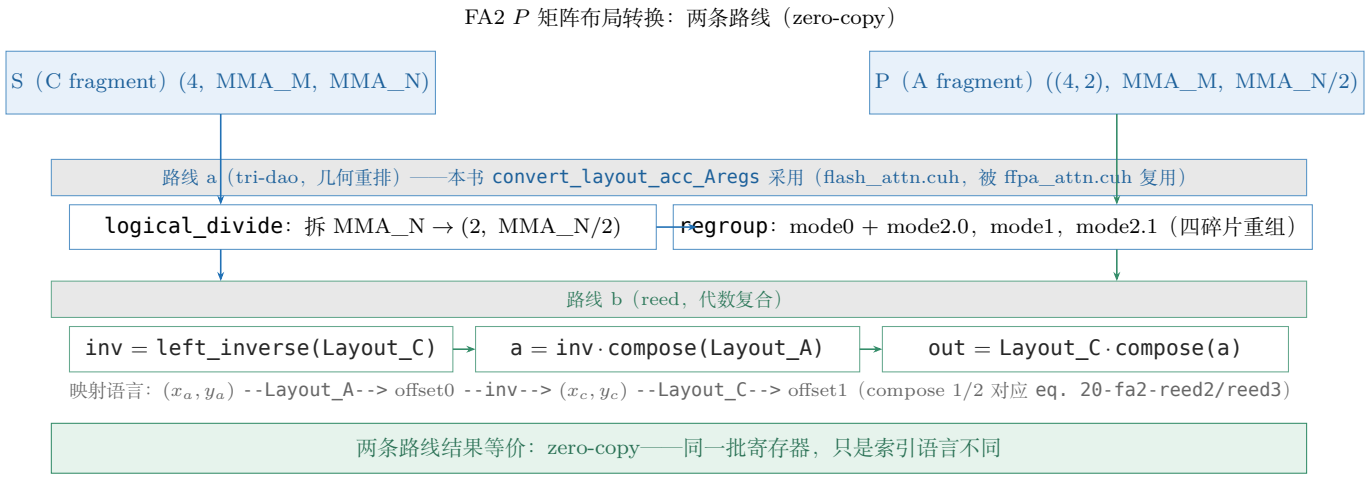


图 21.5: FA2 的 P 矩阵布局转换：tri-dao 的几何重排与 reed 的代数复合两条路线，结果等价。本书 flash_attn.cuh 的 `convert_layout_acc_Aregs`（被 ffpa_attn.cuh 复用）采用的是几何路线。

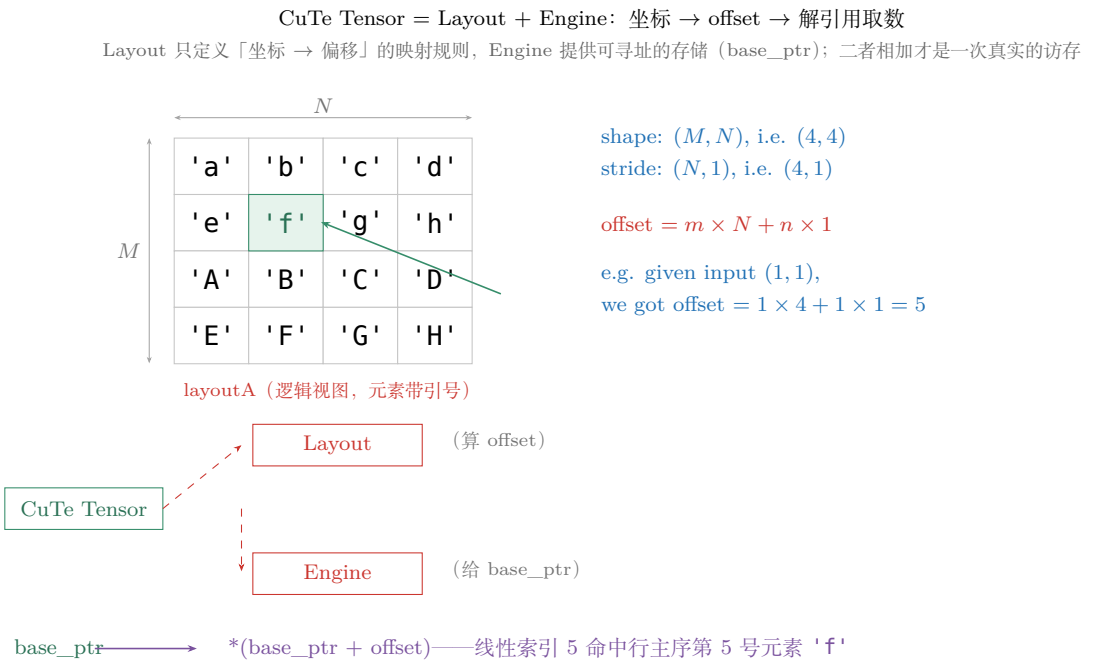


图 21.6: CuTe Tensor 的寻址分工：Layout 负责「坐标 → offset」的代数——shape 限定输入范围、stride 决定各 mode 步长；Engine 只持有 `base_ptr`。$(1,1) \to 5$，命中行主序（$(4,4):(4,1)$，列连续）第 5 号元素 `f`。重建自 @ 竹熙佳处《写给大家看的 CuTe 教程：Layout Compose & Inverse》。

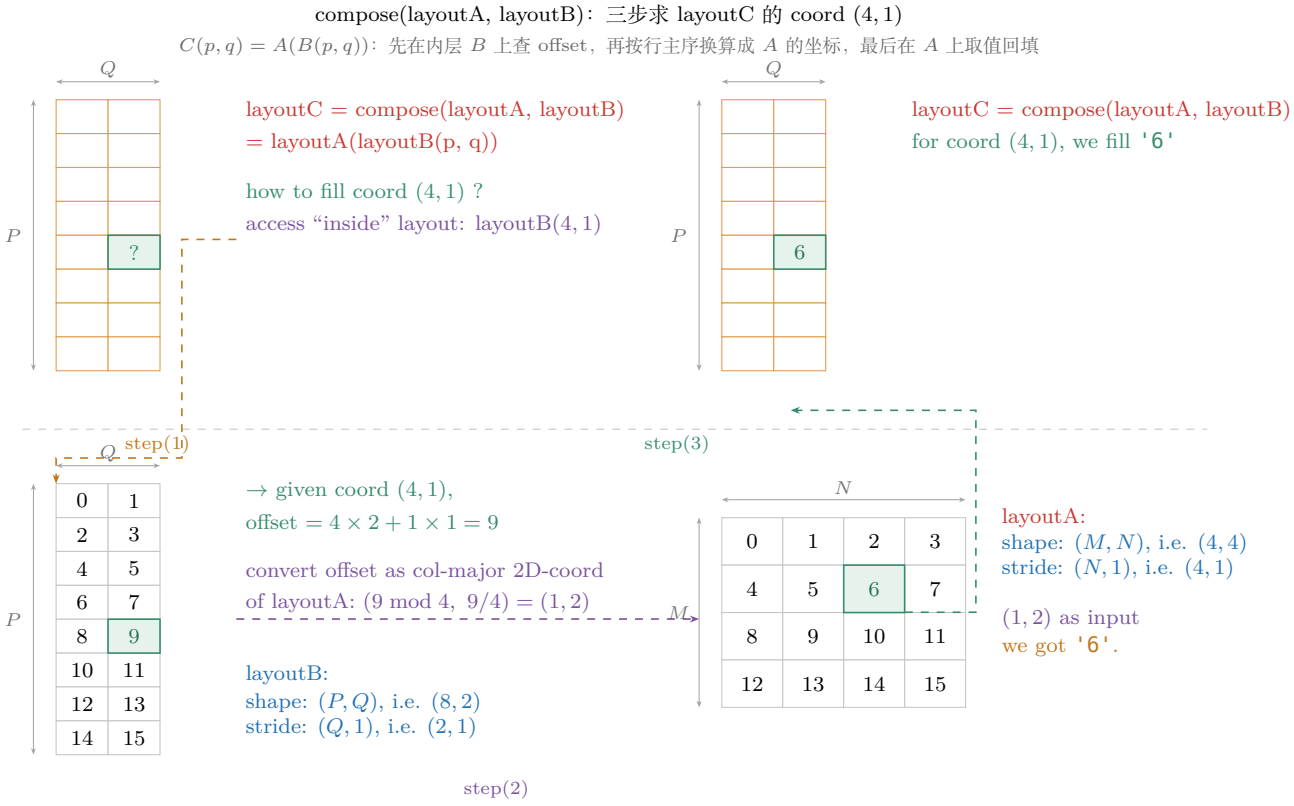


图 21.7: compose(A, B) 的三步求值：先查内层 B 得 offset，再按外层 A 的 shape 把 offset 换算回坐标，最后查 A 得最终值。例：$C(4,1) \to B(4,1) = 9 \to (9 \bmod 4,\ 9 \div 4) = (1,2) \to A(1,2) = 6$ 回填 C。重建自同上文。

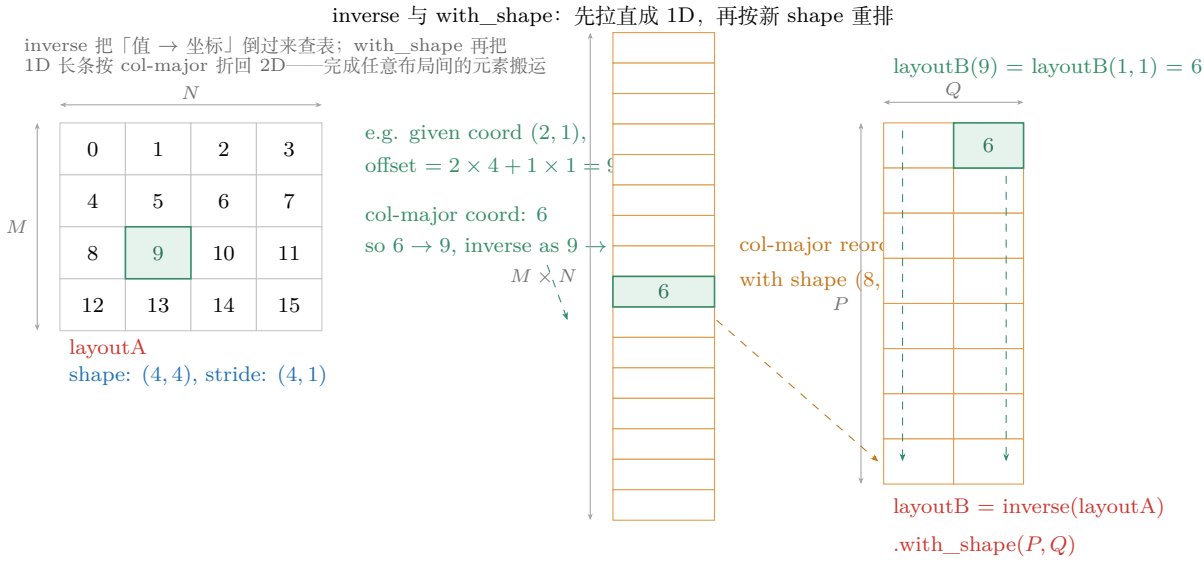


图 21.8：inverse 与 with_shape：inverse 把「值 → 坐标」倒过来查表（$9 \leftrightarrow 6$ 互逆），with_shape 再把 1D 长条按新 shape $(8,2)$ 沿 col-major 折回 2D，得 $B(1,1)=6$。重建自同上文。

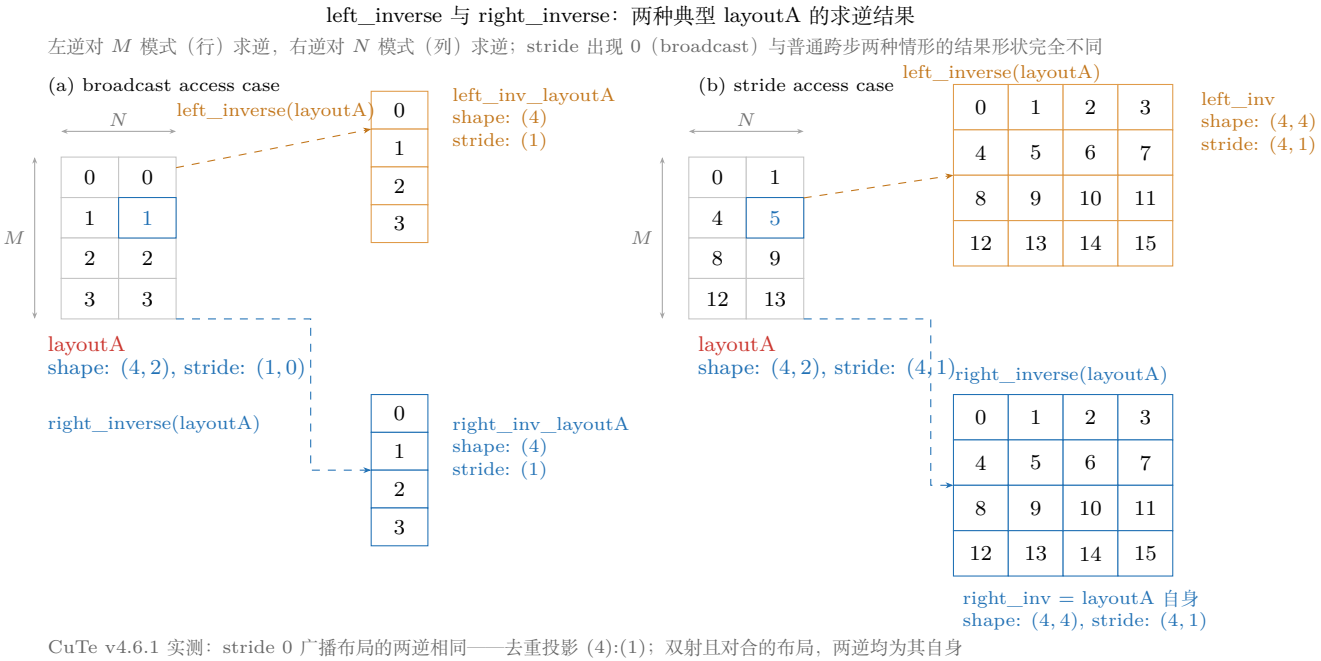


图 21.9：left_inverse / right_inverse 的两种情形（CuTe v4.6.1 实测）：stride $(1,0)$ 广播布局两逆相同，均为去重投影 $(4):(1)$——stride 为 0 的维度被投影掉，每行只保留代表坐标 m；stride $(4,1)$ 的双射布局两逆均为其自身 $(4,4):(4,1)$（该布局是对合，$f \circ f = \mathrm{id}$）。重建自 @ 竹熙佳处《写给大家看的 CuTe 教程：Layout Compose & Inverse》（数值按 CuTe v4.6.1 实测校正）。

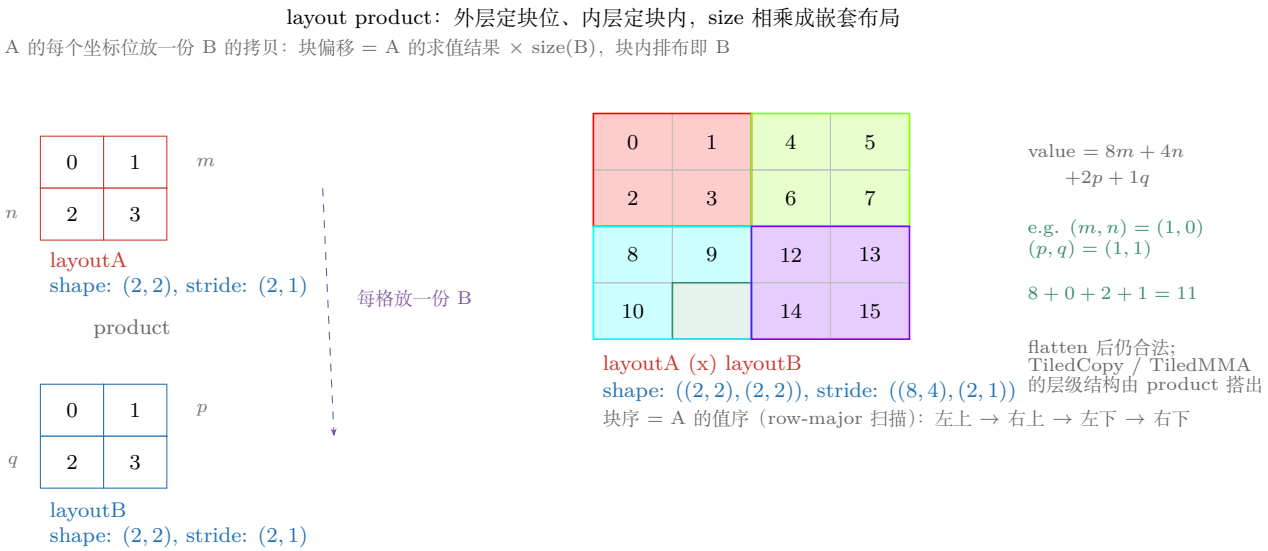


图 21.10: layout product：A 的每个坐标位放一份 B 的拷贝（块偏移 = A 的线性序号 × size(B)），得到嵌套布局 $((2,2),(2,2)):((8,4),(2,1))$。行值 $0,1,4,5\,/\,2,3,6,7\,/\cdots$ 的「块拷贝拼接」排布即 product 的几何特征——它不是 row-major。重建自 @ 竹熙佳处《写给大家看的 CuTe 教程：Layout Product & Divide》。

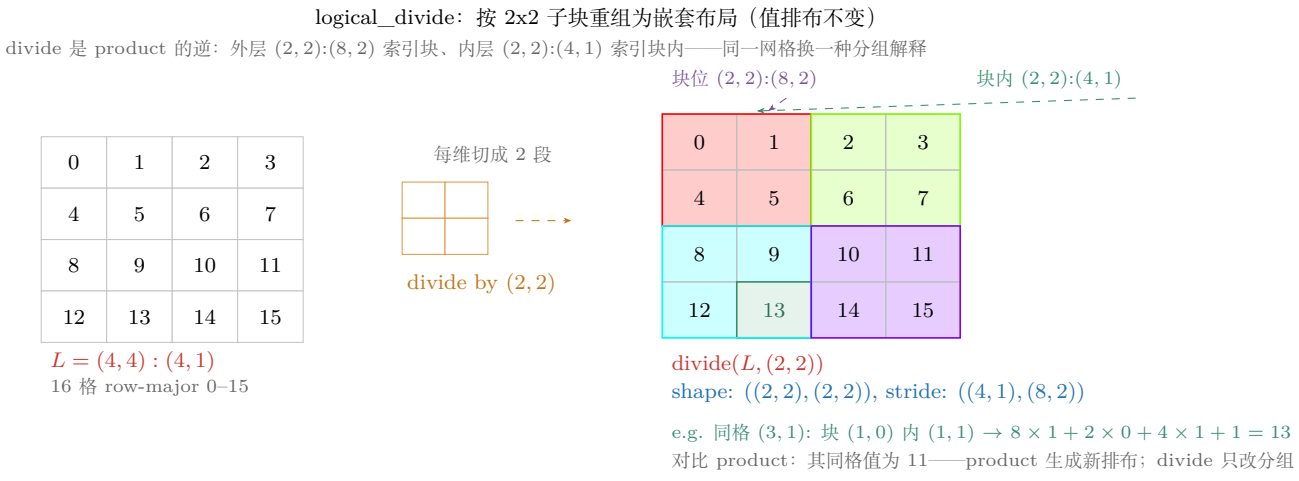


图 21.11: logical_divide：按 2×2 子块把 $(4,4):(4,1)$ 重组为 $((2,2),(2,2)):((4,1),(8,2))$（CuTe 实测的 zipped 形式：块内 mode 在前、块位 mode 在后）——值排布不变（仍 row-major），改变的只是索引方式（一组 $(4,1)$ 索引块内、一组 $(8,2)$ 索引块位）。与上图对比：同物理格 $(3,1)$ 的 product 值为 11（新排布），divide 后仍为 13（原排布）。重建自同上文。

证据三（代码量口径）：hgemm.cuh L803–805 的注释声称手写版「∼200 行」、CuTe 版「∼80 行」。实测去注释非空行：手写 swizzle kernel（L434–716）235 行；CuTe kernel 本体（L821–1196）118 行，另有 launch wrapper（L1216–1427）86 行。所以「∼80 行」低估了 CuTe kernel 本体的物理规模；实测支持的结论是 **CuTe 版 kernel 里没有一行地址算术**：手写版用 `LDMATRIX_X4/LDMATRIX_X2/HMMA16816` 宏直接编排 PTX 与 lane 地址，CuTe 版的内联汇编为 0，地址全部由 layout 类型在编译期推出来。绝对性能的端到端对照（TFLOPS）在第 二十五 章给出，本章不展开。

口径说明：本章测试是纯 host 断言（890 个），不产生 GPU 计时数据；上述「去注释行数」口径为 `sed` 去注释与空行后的物理行数（2026-09-11 实测）。

21.7　常见坑

坑一：坐标的 rank 必须匹配 shape 的嵌套结构。make_layout(A, B) 的外层只有 2 个 mode（shape 为 $(\text{shape}(A), \text{shape}(B))$），把它当 3 个 mode 索引（如 `P(a0,a1,b)`）会直接编译失败：CuTe 会打印 `Mismatched Ranks`（`crd2idx` 的 `static_assert`），报错位置在 `stride.hpp`，第一次遇到时不容易定位问题所在。正确写法是 `P(make_coord(a0,a1), b)`。另外：整数坐标可以展开「平坦」的 shape（如 $(4,8)$），但不能展开嵌套元组，这是本章测试实际踩过的编译错误。

坑二：`make_layout(Shape)` 默认 column-major。它按 `LayoutLeft` 生成 stride（最后一个 mode 变得最快），与 C 数组的 row-major 直觉相反。要 row-major 得写 `make_layout(shape, LayoutRight{})` 或显式给 stride。

坑三：divide 的 tile 必须与目标 layout 同 stride 族。$L = (8,8):(1,8)$ 上用 compact tile $(4,4):(1,4)$，得到的不是 4×4 方块而是 16 个连续 offset 的条带。判别法与反例见 4.5 节实践须知。

坑四：`right_inverse` 对非满值域 layout 会静默退化。值域含空洞（如 stride 2）时返回平凡右逆 (1):(0)，不报错。判断标准：A 的像是否包含从 0 开始的连续段。需要「恢复坐标」时，通常用 `left_inverse`（准逆，带 stride-0 歧义位）更合适。

坑五：`Int<N>` 与 `int` 不只是「优化开关」。

动态值会让相关 layout 变成动态类型，编译期常量表达式（如 `static_assert(cosize(...))`）无法求值，运行期也会多出真实的算术。kernel 里能用静态量就用静态量；这也解释了 hgemm 模板参数为什么写 `int BM, BN, BK` 再包成 `Int<BM>`（让上游保持可读、下游保持静态）。

坑六：divide 结果的 shape 是嵌套的。式 (21.12) 的 shape $((4,4),(2,2))$ 是「两个 mode，每个 mode 内含两个 mode」，不是 rank-4 的扁平 tuple。内层 mode 索引要按嵌套结构写（或用 `flatten`/`group` 显式展开）；本章测试用线性索引 `zd(t, r)` 能工作，是因为顶层恰好 rank 2 且内层是平坦 tuple。

21.8　面试要点速查

Q1：CuTe 的 layout 是什么？为什么默认 column-major？

A：layout 是函数「坐标 → 线性 offset」，由 (Shape, Stride) 表示，idx $= \sum_i d_i c_i$。默认 column-major（`LayoutLeft`）意味着第一个 mode 变化最快。这是早年 BLAS 与 Tensor Core（如 MMA 的 m8n8 片段）的通行约定，CuTe 全书沿用它；C 数组是 row-major，需要显式 `LayoutRight`。

Q2：colex 是什么意思？整数坐标怎么参与？

A：colex = 坐标按「最左 mode 最快」的字典序枚举。把整数 x 传给 layout 时，CuTe 先用式 (21.2) 把 x 分解回坐标，再算 $\sum d_i c_i$，所以「线性索引」与「坐标」是同一个函数的两种输入形式；$f(x)$ 对 x 一般不单调（row-major 即反例），仅当各 mode 的 stride 按 colex 排序（如 compact col-major，$f(x) = x$）时才单调不减。

Q3：compose 的定义与前提？

A：$R = \text{composition}(A, B)$ 满足 $R(c) = A(B(c))$，且 R 的顶层 shape 与 B 一致（`compatible`）。前提是 divisibility：B 的值域必须以整除方式落在 A 的 mode 结构里；否则结果仍是合法 layout，但语义变成「按 A 的 colex 结构重解释 B」。例：$(6,2):(9,2) \circ (4):(1) = (4):(9)$。

Q4：右逆与左逆的区别？何时退化？

A：右逆满足 $A(R(i)) = i$（从 offset 找坐标），值域不满时退化为 (1):(0)；左逆是准逆 $A(R(A(c))) = A(c)$（单射时为真左逆），用 stride-0 的 mode 编码「多对一」的歧义。colex compact（双射）时二者相等。

Q5：分块（tile）在数学上是什么？

A：`logical_divide` $= L \circ \text{make_layout}\big(T, \text{complement}(T, \text{shape}(L))\big)$：

tile 分量取 T、rest 分量取补空间；`zipped_divide` 把结果整理成 (tile, rest) 两个 mode；`tiled` / `flat_divide` 是展平变体。语义上就是「按 T 的 stride 结构把 L 的地址空间切成 size(L)/size(T) 块」。

Q6：product 家族有几种？

A：乘积（`make_layout(A,B)`：mode 拼接、坐标分量相加）、`logical_product`（block 铺在 tiler 上）、`blocked_product`（重复 block）、`raked_product`（交错重复）与 `tile_to_shape`（按目标 shape 铺 atom，最常用于 smem 布局）。区别在「block 之间怎么排」。

Q7：为什么说 CuTe 是零成本抽象？

A：三点：static layout 的全部运算在编译期折叠（`static_assert` 可证）；运行期只剩加法/移位/位运算（swizzle 复合与手写位公式逐点一致）；代价转移到编译时间与类型系统的复杂度。

Q8：size 与 cosize 什么时候不等？

A：layout 有「孔洞」（stride 跳跃、padding）时 cosize > size。对 kernel 的意义：`cosize` 就是这块布局实际占用的元素数（smem 分配依据），而 `size` 是有效元素数，两者之差即 padding 的开销。

Q9：FA2 里 C fragment 转 A fragment 需要什么样的 layout 变换？

A：QK 的累加器按 C fragment 排布 $(\text{MMA}, \text{MMA}_M, \text{MMA}_N)$，PV 的左操作数要 A fragment（形如 $((4,2), \text{MMA}_M, \text{MMA}_N/2)$），两者寄存器对齐不同、不能互当。两条等价路线：tri-dao 的 `logical_divide` 几何重排（本书 flash_attn.cuh 的 `convert_layout_acc_Aregs` 及复用它的 ffpa_attn.cuh 采用，4.8 节），或 reed 的 inverse + compose 代数链（更通用但依赖静态 stride）。

21.9 最小测试与动手实验

测试文件：`book/tests/ch20_layout.cu`（host-only：不启动 kernel、不需要 GPU；main 不调用任何 CUDA API）。八个 case 覆盖本章全部代数结论：A 线性化（coord 与整数两种形式，128 断言）；B compose（14）；C right_inverse（65）；D left_inverse（64）；E product（24）；F zipped_divide（76，含双射覆盖检查）；G tile_to_shape（261，含两条 `static_assert` 编译期折叠证明）；H Swizzle 复合（258，位公式逐点核对与 codomain 双射）。合计 **890 个断言**。

运行命令（在 `book/tests/` 下；`build_tests.sh` 的 `INCLUDES` 已含仓库根的 `third-party/cutlass/include`，直接运行即可）：

```
cd kernels/interview/book/tests
CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch20
```

本机运行输出（RTX PRO 5000，CUDA 13.2，CUTLASS v4.6.1，2026-09-11）：

```
== ch20_layout.cu (sm_120a)
ch20 CuTe layout basics & algebra (host-only, no kernel launch)
PASS case A colex idx=sum(s_i*c_i): mismatches=0 (128 checks)
PASS case B composition R(c)==A(B(c)): mismatches=0 (14 checks)
PASS case C right_inverse L(R(i))==i: mismatches=0 (65 checks)
PASS case D left_inverse quasi-inverse: mismatches=0 (64 checks)
PASS case E product make_layout(A,B): mismatches=0 (24 checks)
PASS case F zipped_divide tile semantics: mismatches=0 (76 checks)
PASS case G tile_to_shape atom->(128,32,2): mismatches=0 (261 checks)
PASS case H Swizzle<3,3,3> o layout bit formula: mismatches=0 (258 checks)
ALL OK
```

判据说明：所有断言都是「库计算 vs 手工数学」的逐点比较。例如 case A 的手工一侧直接用 $\sum d_i c_i$ 与式 (21.2) 的展开式；case F 的手工一侧用式 (21.13)。任何一条不成立都会打印 FAIL 并以非零退出码结束。动手实验（三条，都在本章测试上改）：

1. 把 case F 的 T 从 $(4,4):(1,8)$ 换成 compact 的 $(4,4):(1,4)$，观察结果变成 $\big((4,4),(4)\big):\big((1,4),(16)\big)$，理解「tile 语义依赖 stride 族」（此时坐标公式与双射检查会 FAIL——这正是要看的现象）；

2. 把 case B 的 B 换成动态 shape（`int` 变量构造），观察 `composition` 结果退化为运行期计算，理解静态/动态

分水岭；

3. 把 case H 的 `Swizzle<3,3,3>` 换成 `Swizzle<2,4,3>` 或 `Swizzle<1,5,3>`，对照第 十三 章的 SW64/SW128 档位公式，验证「⟨B,M,S⟩ 三参数 = 位域宽度」的对应关系。

延伸阅读

- @reed《cute 之 Layout》，https://zhuanlan.zhihu.com/p/661182311 （(Shape,Stride) 与坐标映射的入门第一文，本章 3.1 节同源）；
- @reed《cute Layout 的代数和几何解释》，https://zhuanlan.zhihu.com/p/662089556 （把 layout 运算解释成向量空间的代数结构，compose/divide 的几何直观）；
- @ 竹熙佳处《CuTe 教程：Layout Compose & Inverse》，https://zhuanlan.zhihu.com/p/1962625273636845008 （compose/inverse 的逐坐标推导与实例，本章 4.2/4.3 节的对照读物）；
- @ 竹熙佳处《CuTe 教程：Layout Product & Divide》，https://zhuanlan.zhihu.com/p/1971945267294111573 （product/divide 的图解与 kernel 用例，4.4/4.5 节对照）；
- @melonedo《Cute 布局代数实战：除法》，https://zhuanlan.zhihu.com/p/1970274785691936058 （divide 在真实 kernel 里的用法与踩坑，与本章坑三互为印证）。

第二十二章　CuTe（二）：Tensor 与 TiledCopy

22.1　本章导读

第 二十一 章把 layout 变成了一个可运算的对象：坐标怎么映到线性 offset、两个布局怎么复合、怎么切分，全部由代数回答。但一个真实的 kernel 里还有第二类问题：**数据放在哪、由谁来搬**。以第 十三 章的手写 HGEMM 为例，128×32 的 A tile 要经过三次搬运（gmem→smem 的 `cp.async`、smem→ 寄存器的 `ldmatrix`、epilogue 的寄存器→smem→gmem），每一次都有一份「哪个线程搬哪几个元素」的映射。这些映射散落在 `idx` 算术、lane 地址表与 shuffle 编排里，改一个 tile 尺寸就要重新推导。

本章引入 CuTe 的两个一等公民：**Tensor**（把「数据指针 + 布局」绑成一个对象）与 **TiledCopy**（把「一个 copy 原子如何被多线程铺开」形式化为 (thread, value) 二维网格）。四章路线中，本章位于第 二十一 章（layout 代数）与第 二十三 章（TiledMMA）之间：本章讲**数据的搬运**，第 二十三 章讲**计算的分区**，第 二十四 章把两者接到硬件特性（Swizzle 与 TMA）上。

前置章节：第 二十一 章（compose/divide/product 与 `zipped_divide`）、第 十二 章（MMA fragment 与 ldmatrix）、第 十三 章（cp.async 与多级流水）。**源码区间**：hgemm.cuh L821–1196，即 Phase 7c-1 的 CuTe HGEMM kernel 本体（第 二十五 章讲它的算法流程，本章只抽取其中的 Tensor/TiledCopy 用法）；需要时引用 L1216–1427 的 launch wrapper 中拷贝类型的构造段（G2S L1303–1318、S2R L1319–1332、R2S L1355–1362、S2G L1364–1377），其中 L1232–1270 的 tile 尺寸与 SmemLayout 声明归第 二十四 章。**编译宏**：`NOTES_V2_ENABLE_CUTE`（需要 CUTLASS 头文件；sm80+ 均可编）。配套最小测试 `book/tests/ch21_tiled_copy.cu`：host 端 TV-layout 断言，不启动 kernel。

22.2　问题与动机

先看一次「搬运映射」的完整清单。第 十三 章 swizzle 版 HGEMM 的主循环里，每个线程要搬到 smem 的 A 数据、要从 smem 读进寄存器的 A/B fragment、epilogue 里要写回 gmem 的 C 元素，各自都有一份映射：

1. **G→S**：128 个线程各搬 8 个 half（一条 128-bit `cp.async`）。「线程 t 搬哪 8 个」需要手写：$m = t/4$、$n = 8\,(t \bmod 4) + v$；
2. **S→R**：`ldmatrix.x4` 的 lane 地址表：32 个 lane 各提供一行（或一列）的 smem 地址，硬件把 4 个 8×8 矩阵分发到寄存器；「哪个 lane 指哪一行」是与 MMA fragment 布局绑定的；
3. **R→S→G**：accumulator fragment 在寄存器里的分布由 m16n8k16 的 CLayout 决定，与「gmem 上连续」无关；手写版要用 `__shfl_sync` 跨 lane 收齐再写。

三份映射必须彼此一致，而且它们**不是同一个函数**：G→S 关心的是 gmem 合并访问与 smem 冲突，S→R 关心的是 Tensor Core 的 fragment 约定，R→G 关心的是输出的写回形状。第 十二、十三 章的做法是「每个映射单独手写一遍，出错了靠对拍定位」。

CuTe 的做法是把这三件事统一成**同一个抽象**：一次拷贝 = 一个 *copy* 原子（「一个线程搬多少元素、怎么排」）× 一个 *thread-value* 划分（「第几个线程、第几个值」对应数据的哪个坐标）。TiledCopy 就是这个二元组的类型；`partition_S` 与 `partition_D` 负责「按划分把源/目的 Tensor 切开」，`cute::copy` 负责「逐元素执行」，而两者的地址在编译期就已经折叠成常量。本章要回答的四个问题：

- **Tensor 是什么**：为什么指针必须与 layout 一起传递（4.1 节）；

- **(thread, value) 怎么表达**：TV-layout 的定义、对偶与守恒律（3.3–3.5 节），以及它与第 二十一 章的 MN-layout 的关系；
- **TiledCopy 怎么来**：显式给 Thr/Val，或由 `TiledMMA` 反推（`make_tiled_copy_A/B/C`）（4.2–4.4 节）；
- **retile 为什么不需要搬数据**：零拷贝重标记的数学（3.7 节）。

22.3　数学原理

22.3.1　Tensor：engine 与 layout

第 二十一 章的 layout 只回答「坐标 → offset」，它不知道 offset 要加到哪块内存上。访问数据的完整动作其实是两步：**先算偏移，再把偏移加到基址上**。CuTe 把第二半的「基址」抽象成 **Engine**。@ 竹熙佳处给了一个好记的类比：engine 之于 CuTe，相当于 iterator 之于 C++ 标准库，凡是「可以加偏移访问数据」的实体都是 engine（gmem 指针、smem 指针、寄存器指针，甚至一个纯坐标张量）。Engine 与 Layout 绑在一起，就是 **Tensor**：

$$\text{Tensor} = (\text{Engine},\ \text{Layout}), \qquad T(\mathbf{c}) = \text{Engine}\big(:, \text{Layout}(\mathbf{c})\big), \tag{22.1}$$

用映射语言说：layout 负责「$(m,n) \to$ offset」、engine 负责「offset $\to$ 数据」，Tensor 是两次映射的串联，这就是「同一个 (m,n) 坐标能落到 gmem/smem/reg 三种内存」的原因：换 engine，layout 不用变（图 22.1）。

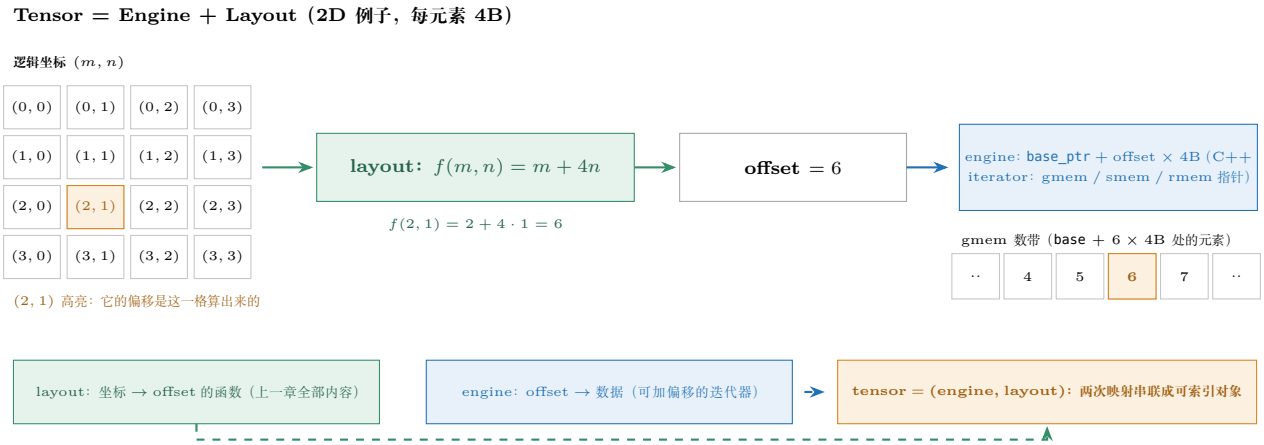


图 22.1: Tensor 的两段结构：layout 把坐标映成偏移，engine 把偏移映成数据。同一个 layout 换不同 engine（gmem/smem/rmem/identity）就得到服务不同内存的 Tensor——这是第 二十一 章「布局与内存解耦」在 Tensor 上的体现。

其中 Engine 是「内存 + 访问方式」：`make_gmem_ptr(p)` 表示从全局内存地址 p 起算、`make_smem_ptr(p)` 表示 smem、`make_rmem_ptr` 表示寄存器、`make_identity_tensor` 返回坐标本身（本章测试用它做断言）。`make_tensor(ptr, shape, stride)` 是 hgemm CuTe 版里最常见的写法（hgemm.cuh L842–844）：`A[M][K]` 行主序存储，等价于 Layout $= (M,K):(K,1)$。注意式 (22.1) 里的 layout 可以是任意复杂结构：只要能算出 offset，Tensor 就能寻址；这一点是后面 `partition_S` 返回「嵌套 mode 的 Tensor」仍然可用的原因。

Tensor 上最重要的两个操作在本章反复出现：

- **切片/slice**：`tensor(coord)` 取单个元素，`tensor(_, _, k)` 用占位符 `_` 保留某个 mode（得到低一维的视图），hgemm 里 `gA(_, _, 0)` 表示「第 0 个 K tile 的 $B_M \times B_K$ 视图」；
- **局部切片/local_tile**：`local_tile(A, make_tile(BM,BK), make_coord(iy, _))` 把大 Tensor 按 tile 切出当前 CTA 需要的那一块，同时**保留未切出的 remainder mode**，它的实现就是第 二十一 章的 `zipped_divide`（L852–854 的 gA/gB/gD）。

22.3.2　小例子：4 个线程搬一块 $(4,4)$

在进入公式之前，先按 @ 竹熙佳处的建议走一遍最小场景：一个 *threadblock* 要把一块 $(k_{\text{TileM}}, k_{\text{TileN}})$ 的数据从 *src* 搬到 *dst*（g2s / s2r / s2g 同理）。为了能手算，取 4 个线程、$(4,4)$ 的 tile、每线程每轮搬 2 个连续元素。

朴素 CUDA 怎么写。教科书式做法：每个线程按 tid 算出自己的 src/dst 地址；让连续线程读连续地址（合并访存）；让每线程一次搬多个元素（宽访存）。4 线程 × 2 float = 8 个元素一轮，4 × 4 = 16 个元素循环 2 轮：

```
int tid = threadIdx.x;                  // 0..3
int r = tid / 2;                        // 行方向线程号
int c = (tid % 2) * 2;                  // 列方向起点（每线程 2 个连续 float）
for (int loop = 0; loop < 2; ++loop) {            // 16 / (4*2) = 2 轮
  float2 s = *(float2*)&g[(loop * 2 + r) * 4 + c];    // src: 合并访存
  *(float2*)&shm[(loop * 2 + r) * 4 + c] = s;         // dst: 无冲突写
}
```

问题在哪。这段代码很短，但它把「谁搬哪块」算在了 `r/c/loop` 三个变量里，一旦引入 swizzle、ldmatrix 或 bank 冲突考虑，地址公式会迅速膨胀，而「哪个数据在哪个线程手里」会彻底看不清。@ 竹熙佳处建议先把映射从代码里**抽出来画成一张表**：行是线程号 t，列是线程内的值号 v，格子填数据坐标 (m,n)，这张表就是下面图 22.2 的左半。

两条直觉，一个抽象。从这张表可以读出两件事：

- **写代码用的是「反向」的那张表**：我们想的从来不是「数据 $(0,1)$ 归谁」，而是「*线程 0 的值 1 去哪取*」，把行列对调看一次，就是第 二十一 章 inverse 运算的内容。CuTe 的 TV-layout 直接以「$(t,v)\to$ 坐标」的形式存储，就是这张对调后的表。
- **引入更多优化后，表仍然存在**（只是格子里要额外绕 swizzle、ldmatrix 的地址映射）。表画得出来，说明「这个映射」是独立于循环代码的对象，把它变成 layout，地址计算就交给编译器推导。

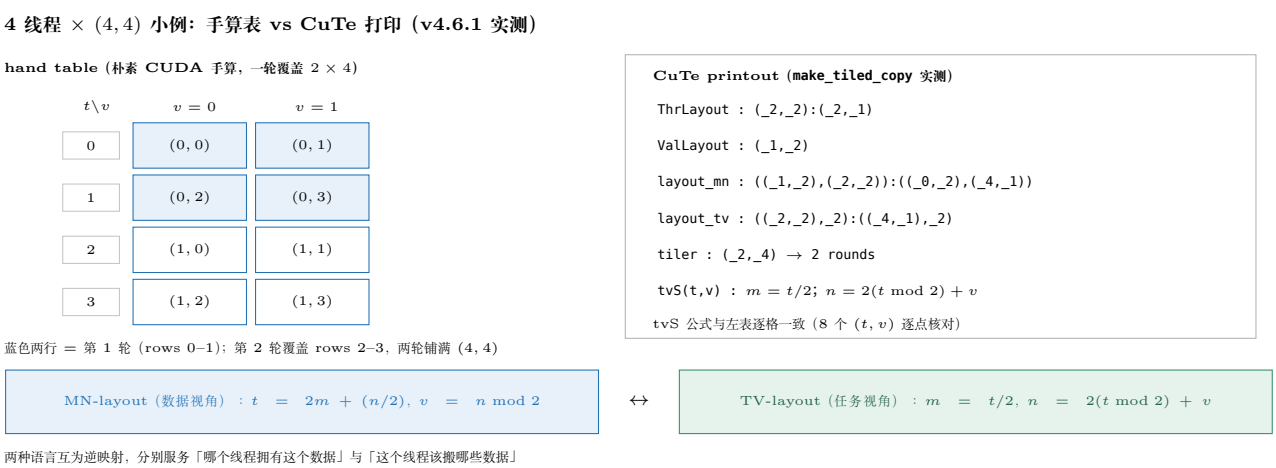


图 22.2: 4 线程小例子的两种表格：左边是朴素 CUDA 手算的「线程 × 值 → 坐标」表（一轮覆盖 2 × 4、两轮铺满 $(4,4)$），右边是 CuTe 的 `make_tiled_copy` 打印结果——`tvS` 的公式与手算逐格一致。MN-layout 与 TV-layout 互为逆映射，分别服务「数据视角」与「线程视角」。

放大到 hgemm。把这个小表放大到真实体量（128 线程、每线程 8 个 half、32 × 32 的 tile），就得到 hgemm 的 G→S 划分：`ThrLayout` 从 $(2,2):(2,1)$ 变成 $(32,4):(4,1)$，`ValLayout` 从 $(1,2)$ 变成 $(1,8)$，t/v 的公式就是 3.4 节的式 (22.7)。下面正式引入这套语言。

22.3.3 (thread, value) 网格：TV-layout

一次由 P 个线程执行、每线程搬 V 个值的拷贝，本质上是把 tile 的每个坐标指派给一个二元组 $(t,v)\in[0,P)\times[0,V)$。CuTe 把这个指派本身写成一个 layout，称为 **TV-layout**：

$$\text{TV}:\ (t,v)\longmapsto(\text{tile 内坐标}),\qquad \text{逆映射MN}:\ (\text{坐标})\longmapsto(t,v). \tag{22.2}$$

TV 与 MN 是**同一个划分的两种视角**：写 kernel 时我们关心「第 t 个线程拿哪几个值」（TV），写布局时我们关心「坐标 (m,n) 落在谁手上」（MN）。CuTe 两个方向都提供，并且用同一个 layout 表示：

$$\text{Layout_TV}\ =\ \big(\text{Layout_MN}\big)^{-1}\ (\text{右逆}),\qquad \text{Tiler}=\text{product_each}\big(\text{shape}(\text{Layout_MN})\big). \tag{22.3}$$

22.3.4　raked_product：从 Thr/Val 到 MN

上节小例子里我们直接画了表；在 CuTe 里，「画表」被拆成两个更好写的动作：**声明线程怎么排（ThrLayout）**、**声明每线程搬的值怎么排（ValLayout）**，再由 `make_tiled_copy` 合成完整的 MN 表。`make_tiled_copy(atom, ThrLayout, ValLayout)` 的两个需要手动指定的参数是 **坐标 →thr/val** 的两个函数（注意方向：它们把坐标映到线程号/值号，不是反过来）：

$$\text{ThrLayout}:\ (i,j)\longmapsto t=4i+j,\qquad \text{ValLayout}:\ (i,j)\longmapsto v=j, \tag{22.4}$$

式 (22.4) 就是 hgemm 的 G→S 参数（hgemm.cuh L1313–1317：`ThrLayout = (32,4):(4,1)`、`ValLayout = (1,8)`）。CuTe 把它们合成一个 MN 布局

$$\text{Layout_MN}=\text{raked_product}\big(\text{ThrLayout},\ \text{ValLayout}\big)\ =\ \big((1,32),(8,4)\big):\big((0,4),(128,1)\big), \tag{22.5}$$

右侧为 CuTe 的实测输出形式（按 (t,v) 的 colex 编码 $t+128v$ 索引线程-值对）。其中 `raked`（交错）指：同一个线程的 V 个值**沿坐标连续排列**，而不是把 tile 先按线程切成大块、再在线程内切小块（回看 3.2 节的小例子：Thr `(2,2):(2,1)` 与 Val `(1,2)` 按同样方式合成出来的，正是那张手画的表）。式 (22.5) 的最简版本就是 kernel 注释里的「128 个线程，每线程搬运 1×8=8 个 half」（hgemm.cuh L1308–1309）。把式 (22.5) 按坐标写开：坐标 (i,j')（$i\in[0,32)$、$j'\in[0,32)$）满足 $j'=8j+v$，于是

$$\boxed{\,t=4i+j=4i+\left\lfloor\frac{j'}{8}\right\rfloor,\qquad v=j'\bmod 8\,} \tag{22.6}$$

反过来（线程视角，本章测试 case A 逐点核对的那条）：

$$i=\left\lfloor\frac{t}{4}\right\rfloor,\qquad j'=8\,(t\bmod 4)+v. \tag{22.7}$$

读法：线程 t 负责第 $\lfloor t/4\rfloor$ 行的第 $8(t\bmod 4)$ 到 $8(t\bmod 4)+7$ 列，即连续 8 个元素，正好对齐一条 128-bit `cp.async`。这就是「G→S 划分」的结论，它在手写版里是两行 idx 算术，在 CuTe 里是式 (22.5) 的类型。

22.3.5　守恒律与重复因子

由式 (22.3) 右逆的定义，TV 布局天然把 $P\times V$ 个 (t,v) 映射到 Tiler 上；当它是双射时立刻得到一条守恒律：

$$P\times V=\big|\text{Tiler}\big|=\prod_i \text{size}_i. \tag{22.8}$$

G→S 的实测：$P\times V=128\times 8=1024=32\times 32=|\text{Tiler}|$（3.4 节的 Thr/Val 恰好把 32×32 的 tile 铺满一层）。S→R 的实测则不同：`make_tiled_copy_A` 得到的 tiler 是 $(\text{tile_size}_0,\text{tile_size}_2)=(32,16)$，只有 512 个元素，而 $P\times V=128\times 8=1024$，即 **2 倍的重复因子**。原因不是 bug：2×2 的 warp 排布里，两个 N 方向的 warp 需要**同一份** A fragment，于是同一块 A 数据被两组线程各读一遍。一般地：

$$\text{A 的重复因子}=\#\{\text{N 方向 warp}\},\qquad \text{B 的重复因子}=\#\{\text{M 方向 warp}\}. \tag{22.9}$$

式 (22.9) 是后面理解 fragment 寄存器占用的关键：本章测试 case E 对 A 拷贝的 128×32 tile 做覆盖统计，实测每个元素恰好被 2 个 (thread, value) 对覆盖（multi = 8192 个覆盖点、once 结构 4096/2），与式 (22.9) 完全一致。该覆盖结构的网格化见图 22.3。

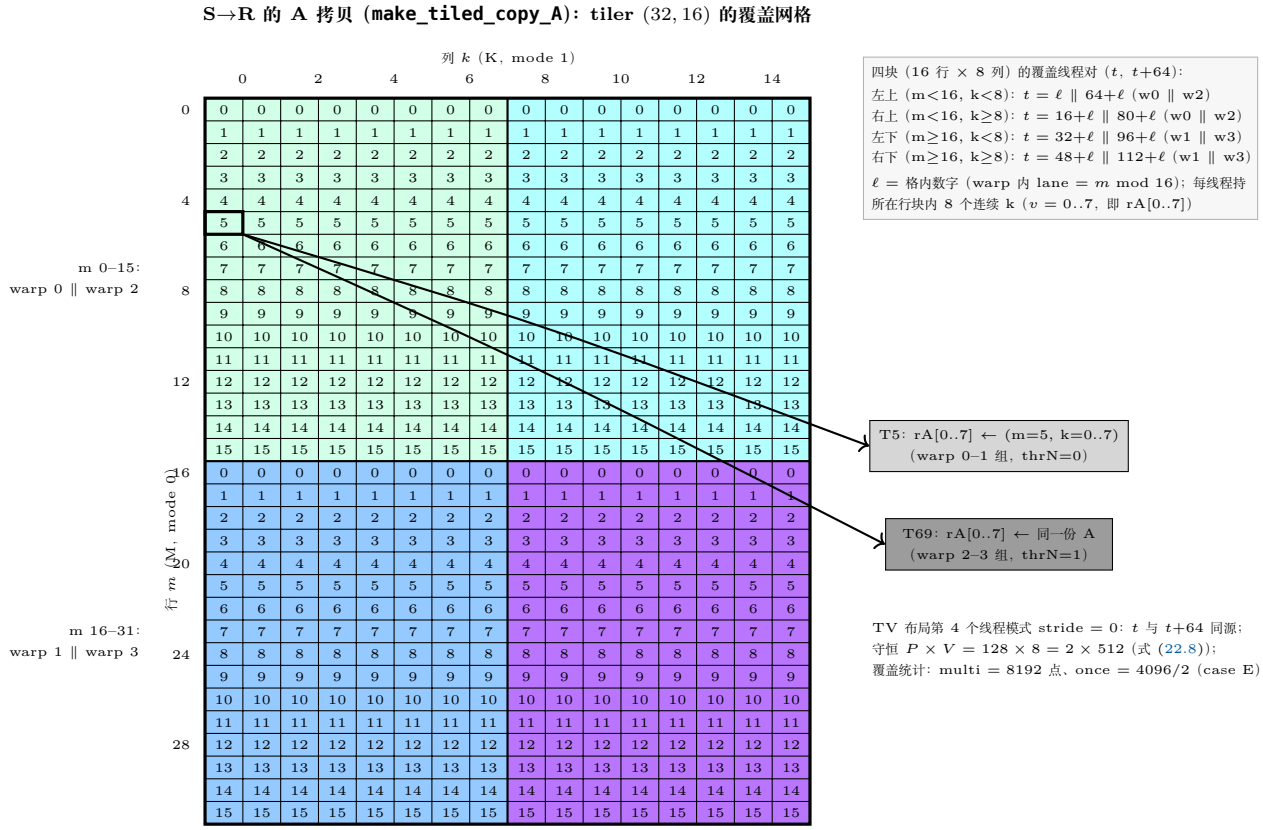


图 22.3: S→R A 拷贝的 2 倍重复因子（式 (22.9)、测试 case D/E）：tiler (32, 16) 的 A tile（M × K）上，每格标 warp 内 lane 号 $\ell = m \bmod 16$，格子的实际覆盖线程对为（基址 $+\ell$，基址 $+64+\ell$），四块的基址见右上表（w0–w3 即 warp 0–3）。每个元素恰被 2 个 (thread, value) 对覆盖：t 与 t+64（两个 N 方向 warp，thrN = 0/1）各持同一行块 8 个连续 k 的一份寄存器拷贝——TV 布局第 4 个线程模式 stride 为 0 的几何含义，守恒 $128 \times 8 = 2 \times 512$。对照图 22.5 的 G→S 双射网格（每格恰一线程）。

22.3.6 partition 的数学

`partition_S/partition_D` 做的事情可以概括为：把 **数据 Tensor 按 Tiler 切块**（第 二十一 章的 `zipped_divide`），再把每块的内部坐标交给 copy 原子的 TV 布局：

$$\text{partition}(T) \;=\; \text{tile2thrfrg}\Big(\text{zipped_divide}(T,\ \text{Tiler}),\ \text{right_inverse}(\text{AtomLayoutRef}) \circ \text{AtomLayout}_{S/D}\Big), \tag{22.10}$$

其中后半段 $\text{right_inverse}(\text{Ref}) \circ \text{Layout}_{S/D}$ 是「原子内部的源/目的重排」，它保证 `copy` 的第 v 个源元素和第 v 个目的元素是同一个逻辑值（例如 `cp.async` 的源是 8 个连续 half，而 `ldmatrix` 的目的寄存器顺序与源地址顺序并不相同）。展开后每个线程拿到的视图形状是

$$\text{partition_S}(T) = \big(\underbrace{V}_{V_{\text{atom}}},\ (F_V, F_X),\ \underbrace{R_M, R_N, \ldots}_{\text{remainder}}\big), \tag{22.11}$$

即「本线程的值 × 余下的重复次数 × 原 Tensor 未被切掉的 mode」。G→S 在 (128, 32) 上的实测（本章测试 case B）：每个线程 (8, 4, 1)，即 8 个值、M 方向 4 个 tile 重复、K 方向 1 个；$8 \times 4 \times 1 = 32$ 个元素，×128 线程 = 4096 恰好是整块 tile（双射）。

@ 竹熙佳处把这种结构概括成一句话：「**atom 之外，皆是循环**」。TV 布局只描述一个原子（一次 P 线程 ×V 值的搬运），原子之外的重复（(F_V, F_X) 与 remainder mode）不需要新的映射，只是把原子划分重复若干次；kernel 里 `CPY_M/CPY_N` 两层循环就是这些重复因子在代码里的样子。

22.3.7　retile：零拷贝重标记与 upcast

retile 这对操作（`retile_S` 与 `retile_D`）解决一个具体的错位问题：MMA 的 accumulator fragment C 的逻辑形状是 $(\mathrm{MMA}, \mathrm{MMA}_M, \mathrm{MMA}_N)$（m16n8k16 下每线程 4 个值/原子实例），而 copy 想要的是「按 copy 原子的值顺序排列」的视图，两者**描述同一批寄存器**，只是索引方式不同。retile 就是这次重标记，它的定义式是

$$\mathrm{retile}(T) = \mathrm{zipped_divide}\Big(T,\ \mathrm{prepend}\big(\mathrm{product_each}(\mathrm{shape}(f_{mn})),\ V\big)\Big) \text{ 的坐标重排,} \tag{22.12}$$

其中 $f_{mn} = \mathrm{upcast}_{P\cdot V}\big(\mathrm{TV}^{-1}\big)$ 是「坐标 $\to$ 值编号」的布局。这里出现了本章唯一一个需要解释的运算：$\mathbf{upcast}_N$。它把布局的定义域按「N 个元素一组」合并，直觉是「把 N 个 half 看成一个 uint128」：

$$\mathrm{upcast}_2\big((8,32):(32,1)\big) = (8,16):(16,1), \qquad \mathrm{downcast}_2\big((4,32):(32,1)\big) = (4,64):(64,1), \tag{22.13}$$

即连续方向的 extent 除以 N、其余 stride 除以 N（实测见本章测试 case F）。它还有一条很好的性质，即对复合布局也成立：$\mathrm{upcast}_2\big(\mathrm{Swizzle}_{3,3,3} \circ L\big) = \mathrm{Swizzle}_{3,2,3} \circ \mathrm{upcast}_2(L)$（实测：`Sw<3,2,3> o _0 o (32,16):(16,1)`），这也解释了为什么 128-bit 拷贝的 smem 布局可以写成「half 版布局的 upcast」。

22.4　设计与实现

22.4.1　kernel 骨架：Tensor 与 local_tile

先看 hgemm CuTe 版 kernel 的开头，即式 (22.1) 的实例：

hgemm.cuh L821-839（kernel 入口：模板参数、smem 与 CTA 级边界）

```
__global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
                                         int n, int k) {
  using namespace cute;
  static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
  static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");

  // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
  extern __shared__ T shm_data[];
  T *Ashm = shm_data;
  T *Bshm = shm_data + cute::cosize(SmemLayoutA{});

  int idx = threadIdx.x;
  // BlockSwizzle: 0/1 控制是否启用 thread block swizzle，改善 L2 cache 局部性
  int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  int iy = blockIdx.y; // M 方向 block 索引
  // 当前 kernel 只有 CTA 级边界判断，没有 tile 内的逐元素 predication；调用方需保证
  // M % BM == 0、N % BN == 0、K % BK == 0，才能避免边界 tile 的越界访问或未写回。
  if (iy * BM >= m || ix * BN >= n) return;
```

接着是数据视图与共享内存视图。注意 `local_tile` 的第三个参数用了占位符 `_`：它在 K 方向保留 remainder，于是 `gA` 是 $(B_M, B_K, \lceil K/B_K \rceil)$ 的三维视图，之后 `gA(_,_,0)` 再取第 0 个 K tile，两级切片组合出「当前 CTA 的 A 分片」：

hgemm.cuh L841-866（Tensor 视图、local_tile 与 TiledMMA 的 fragment 分区）

```
  // make_stride(leading_dim, Int<1>{})  → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                         make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
                         make_stride(k, Int<1>{}));
  Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
                         make_stride(n, Int<1>{}));

  // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile；返回 Tensor 仍保留未切出的 remainder mode。
```

```
// make_coord(iy, _): 固定 M/N 方向的 CTA 坐标，_ 保留 K 方向的 tile remainder:
//   gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));

// Shared memory Tensor: 按 SmemLayout 解读 smem 区域
auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)

// TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
TiledMMA tiled_mma;
auto thr_mma = tiled_mma.get_slice(idx);
auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
auto tCrD = thr_mma.partition_fragment_C(gD);          // (MMA, MMA_M, MMA_N)
clear(tCrD); // 累加器清零
```

22.4.2 三种 TiledCopy 的构造方式

hgemm 里出现了三类搬运，它们的 TiledCopy 构造方式正好覆盖三种来源（表 22.1）：**显式给出 Thr/Val**（G→S、S→G：`make_tiled_copy`）、**由 TiledMMA 反推**（S→R 用 `make_tiled_copy_A/B`、R→S 用 `make_tiled_copy_C`）。第二类的实现只有一行：

$$\text{make_tiled_copy_A}(\text{atom}, \text{mma}) := \big(\text{atom},\ \text{mma.get_layoutA_TV}(),\ (\text{tile_size}_0, \text{tile_size}_2)\big), \tag{22.14}$$

$$\text{make_tiled_copy_B}(\text{atom}, \text{mma}) := \big(\text{atom},\ \text{mma.get_layoutB_TV}(),\ (\text{tile_size}_1, \text{tile_size}_2)\big), \tag{22.15}$$

$$\text{make_tiled_copy_C}(\text{atom}, \text{mma}) := \big(\text{atom},\ \text{mma.get_layoutC_TV}(),\ (\text{tile_size}_0, \text{tile_size}_1)\big). \tag{22.16}$$

式 (22.14)–(22.16) 是本章最实用的一条结论：**只要给了 TiledMMA，S→R 的线程-值映射就是确定的**，因为 MMA 的 fragment 布局本来就规定了每个线程持有哪些元素，S→R 只是「把这些元素从 smem 搬到寄存器」而已（第 二十三 章会看到 `get_layoutA_TV` 的构造过程）。对 hgemm 配置实测（CUTLASS v4.6.1）：tile_size $= (32, 32, 16)$，于是 A/B 拷贝的 Tiler $= (32, 16)$、C 拷贝的 Tiler $= (32, 32)$；后者等于 SmemLayoutC 的 $(k_{\text{MmaPM}}, k_{\text{MmaPN}}) = (32, 32)$，这正是 kernel 注释（hgemm.cuh L1355–1356）声称的「恰好匹配」，逐项核对成立。

实践须知

- 表 22.1 的类型、形状、行号是 CUTLASS v4.6.1 下的实测值：五条搬运链路的 Tiler、`TiledNumThr/TiledNumVal` 与 launch wrapper（L1303–1377）的构造段逐项核对一致（G2S 的 $(32, 4):(4, 1) \times (1, 8)$、C 拷贝 Tiler $(32, 32)$ 与 `cosize(SmemLayoutC)` $= 4096$ 等）。
- 注释只写参数、不写类型时，实际型号要去构造段自己确认。本仓库的注释与实现一致是核对的结论，不是可以推广的假设。
- 换 CUTLASS 版本后重核一遍：类型别名（如 `SM80_CP_ASYNC_CACHEGLOBAL`）随版本演进。

22.4.3 G→S：显式 Thr/Val 与两级切片

G→S 的 TiledCopy 类型定义在 launch wrapper 里（L1303–1318），kernel 里只做三件事：取本线程的 slice、对源（gmem）与目的（smem）各做一次 partition：

hgemm.cuh L868-888（G2S 与 S2R 的 partition：同一份数据、两种划分）

```
// G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async) 。
G2SCopyA g2s_tiled_copy_a;
auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
```

搬运	原子（Copy_Atom）	Tiler	构造方式	kernel 行号
G→S	`SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>`	$(32, 32)$	显式 Thr$(32,4)$:$(4,1)$ / Val$(1,8)$	L1303–1318
S→R (A)	`SM75_U32x4_LDSM_N`	$(32, 16)$	`make_tiled_copy_A`	L1319–1332
S→R (B)	`SM75_U32x4_LDSM_N`	$(32, 16)$	`make_tiled_copy_B`	L1319–1332
R→S (C)	`UniversalCopy<int>`（32-bit）	$(32, 32)$	`make_tiled_copy_C`	L1362
S→G (C)	`UniversalCopy<uint128_t>`（128-bit）	$(32, 32)$	显式 Thr$(32,4)$:$(4,1)$ / Val$(1,8)$	L1371–1376

表 22.1: hgemm CuTe 版的五条搬运链路（表 21-1）。Tiler 为 CUTLASS v4.6.1 下对 hgemm 配置（`SM80_16x8x16`、EU-repeat $2 \times 2 \times 1$、`Tile<32,32,16>`）的实测值。

```
    auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
    auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)

    G2SCopyB g2s_tiled_copy_b;
    auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
    auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
    auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);

    // S2R TiledCopy: 描述 shared → register 的数据搬运（使用 ldmatrix）
    // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
    auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
    auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
    auto tAsA = s2r_thr_copy_a.partition_S(sA);     // (CPY, CPY_M, CPY_K, kStage)
    auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐

    auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
    auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
    auto tBsB = s2r_thr_copy_b.partition_S(sB);
```

这段代码集中体现了本章的核心关系：**同一个 sA** Tensor 被两种 TiledCopy 各分区一次，`tAsA_copy` 是 G→S 的视角（$(V, 4, 1, k_{\text{Stage}})$），`tAsA` 是 S→R 的视角（$(V, 4, 2, k_{\text{Stage}})$，多出的 2 是 K 方向的 MMA 切片数）。为什么 S→R 的 K 方向重复更多？因为 copy 原子不同：G→S 一次搬一整条 128-bit，S→R 的 `ldmatrix.x4` 一次只覆盖 16×16 的 K 切片。

22.4.4　S→R：从 TiledMMA 推导，并用 retile_D 对齐目的

式 (22.14) 的实现在 L881–888，其中起关键作用的一行是 `retile_D`：`tCrA` 是 *MMA* 视角的寄存器 fragment（$(\text{MMA}, \text{MMA}_M, \text{MMA}_K) = (8, 4, 2)$，本章测试 case C 实测），`tCrA_view` 是 *copy* 视角的同一个片段（$(V, F_V, F_X) = (8, 4, 2)$）。二者的元素个数、数据指针完全相同，只是索引语义不同：

$$\text{retile_D(tCrA)} = \text{同一块寄存器的「(线程, 值) 顺序」视图}, \qquad \text{data() 不变}. \tag{22.17}$$

这一步是 `cute::copy(s2r_tiled_copy_a, tAsA(…), tCrA_view)` 能成立的前提：copy 按「源的第 i 个值 → 目的的第 i 个值」逐元素对应，而 `retile` 保证目的视图的「第 i 个值」就是本线程寄存器的第 i 个槽位。它**不搬数据、不转换类型、不 shuffle**，真正跨线程的重组发生在后面 epilogue 的 smem 往返里。

顺带解决一个易混点：`retile_S` 的 S 指「该 TiledCopy 的 source side」，不是 shared memory；`retile_D` 的 D 指 destination side。主循环里对 A fragment 用 `retile_D`（fragment 是目的端），epilogue 里对 C fragment 用 `retile_S`（fragment 是源端），方向由数据流决定。

实践须知

- `partition_S` 返回的是 **source side 的读取视图**，不是「寄存器实际收到的顺序」。实测（case D）每线程的 source 视图是 8 个连续列，其 TV 布局第 4 个线程模式**步长为 0**，线程 t 与 $t+64$ 读的是同一份 A 片段（这正是 N 方向 warp 重复的来源，见式 (22.9)）。线程 t 的第 0 个值是 $\big((t \bmod 16) + 16\lfloor (t/32) \bmod 2 \rfloor,\ 8\lfloor (t \bmod 32)/16 \rfloor\big)$（本章测试 case D 的核对公式）。

- 真正决定寄存器内容顺序的是 copy 原子的 `ValLayoutDst`，由 `retile_D` 表达的目的端布局承担（3.7 节）。把「smem 读取视图」与「寄存器视图」混为一谈，会写出看似对齐、实际错位的 copy。
- 换 copy 原子（例如 ldmatrix 的 `.trans` 变体）时，先确认新的 `ValLayoutDst` 与目标 fragment 布局仍然兼容，再动 retile 一侧。

22.4.5 epilogue：R→S→G 与 group_modes

epilogue 的三段划定值得单独讲：寄存器 fragment 先按 32-bit 粒度写进 smem scratchpad（R→S），再从 scratchpad 按 128-bit 读回 gmem （S→G）。中间这块 scratchpad 复用了当前 A stage 的空间，`sC` 由 `make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{})` 构造（L1001，下文展开）。R→S 用的 `R2SCopyAtomC` 是 `Copy_Atom<UniversalCopy<int>, T>`：以 32-bit 粒度搬运 `T` 数据，用 `UniversalCopy` 顺带完成类型转换，再经 `make_tiled_copy_C` 包装成 `r2s_tiled_copy_c`。

被搬运的 `tCrD` 是 TiledMMA 产生的 accumulator fragment，逻辑形状为 `(MMA, MMA_M, MMA_N)`：寄存器元素已经就位，但该 layout 是为 MMA accumulator 服务的，并不直接按 R2S copy atom 的 source-value 顺序表达。`retile_S`（S 指「这个 TiledCopy 的 source side」，不是 shared memory）基于 `r2s_tiled_copy_c` 的 reference layout 创建一个共享 `tCrD.data()` 的 zero-copy layout view，把逻辑坐标从 `(MMA, MMA_M, MMA_N)` 重新表示为 `(CPY, CPY_M, CPY_N)`（`tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD)`）。这一步与前述 `retile` 一样只做零拷贝视图重标记；真正的 R2S 数据搬运发生在后面的 `cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))`。可与主循环的 `s2r_thr_copy_a.retile_D(tCrA)` 对照：二者都是为了让已有 fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination 角色，而不是另一次 memory transfer。

注意 L1001 的写法：`sC` 拿的是 `sA(_,_,ismem_read).data()`（一个裸指针），再重新用 SmemLayoutC 解释同一段地址。这里没有 Tensor 类型转换，只是「同一块 smem、两种布局视图」，A 的 pipe 布局是 8×32 atom，C 的是 32×32 atom，二者都含 `Swizzle<3,3,3>` 但**不可互换读取**。容量上：一个 A stage 是 $128 \times 32 = 4096$ 个 half，C scratchpad 是 $32 \times 32 \times 4 = 4096$ 个 half，正好放得下（launch wrapper 的 `static_assert` 就是查这件事，实测 `cosize(SmemLayoutA)` $= 8192$（2 stage）、C 部分 4096）。同一块地址带上的两种解释见图 22.4。

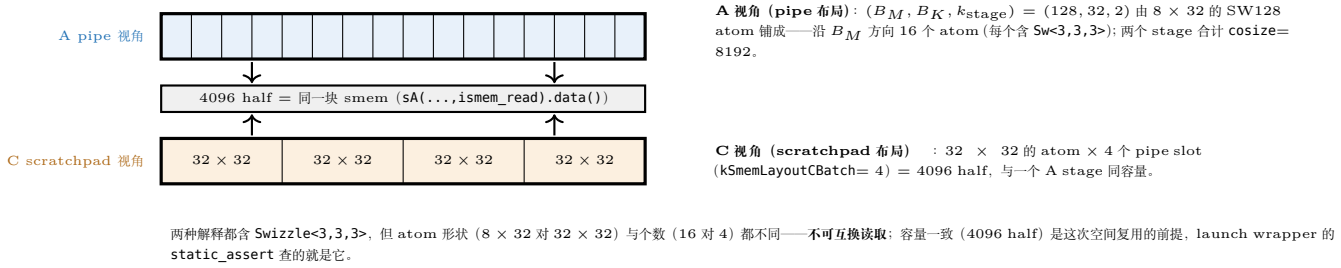


图 22.4: epilogue scratchpad：同一块 4096 half 的 smem、两种布局视图。上条带是 A 的 pipe 解释——$(128, 32)$ 的 stage 按 8×32 的 SW128 atom 沿 B_M 切成 16 段；下条带是 C 的 scratchpad 解释——32×32 的 atom $\times$ 4 个 pipe slot（`kSmemLayoutCBatch`$= 4$）。中间那条带是两者共用的地址带：`sC` 拿到的只是 `sA(_,_,ismem_read).data()` 这个裸指针，没有 Tensor 类型转换，重复的只是「同一段地址的两种读法」。

S→G 一侧的 partition 与 `group_modes` 合并：

划分本身只有两行：`tCsC_s2g = s2g_thr_copy_c.partition_S(sC)`，形状 `(CPY, _1, _1, pipe)`；`tCgC_s2g = s2g_thr_copy_c.partition_D(gD)`，形状 `(CPY, CPY_M, CPY_N)`。`tCsC_s2g` 与 `tCsC_r2s` 都从同一个 `sC` 产生，因而保留相同的 pipe=4，前者是 S2G 的 source mapping，后者是 R2S 的 destination mapping；`tCgC_s2g` 则面向完整的 $gD = (B_M, B_N) = (128, 256)$，相对于 S2G copy tiler 仍有非平凡的 M/N repetition，所以保留 `CPY_M`、`CPY_N`。注释特别提醒：这里的 `CPY`/`CPY_M`/`CPY_N` 都是编译期 copy-partition 的逻辑 mode，不应直接理解为固定的 lane 数、warp 数或物理连续维度。

`group_modes<1,3>` 把 mode 区间 $[1, 3)$ 合并成一个嵌套 mode：形状从 $(\text{CPY}, \text{CPY}_M, \text{CPY}_N)$ 变成 $(\text{CPY}, (\text{CPY}_M, \text{CPY}_N))$。它**只重组 layout**，不分配内存、不搬数据，它是第 二十一 章 4.6 节 `group<B,E>` 的 Tensor 版（定义在 `cute/tensor_impl.hpp`，内部就是一行 `make_tensor(t.data(),group<B,E>(t.layout()))`）。合并之后外层的双层循环可以用一个线性坐标 `i+j` 遍历整个 $\text{CPY}_M \times \text{CPY}_N$ 空间（L1093 起的 epilogue 循环），而 step

来自 scratchpad 的 pipe 数（`size<3>(tCsC_r2s)` = 4）。kernel 注释（L1077–1092）特别提醒：**step 是 pipe 深度，不是** CPY_N，循环成立的前提是 $\mathrm{CPY}_M \cdot \mathrm{CPY}_N$ 能被 step 整除，当前静态布局下这是编译期保证的。

那么为什么要对 `tCrC_r2s` 与 `tCgC_s2g` **同时** group （真代码就两行：`tCgC_s2gx = group_modes<1, 3>(tCgC_s2g)`、`tCrC_r2sx = group_modes<1, 3>(tCrC_r2s)`）？`tCrC_r2s` 是每线程持有的寄存器 C fragment，原始形状 `(CPY, CPY_M, CPY_N)`；`tCgC_s2g` 是该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐；两者用相同的 $[1,3)$ 合并后，源/目的 Tensor 仍保持相同的坐标空间，`cute::copy` 便按相同的 `(CPY, i+j)` 坐标完成寄存器到 global 的对应搬运。而 `tCsC_r2s`/`tCsC_s2g` 不 group 第 3 个 pipe mode，它们还需显式保留 shared-memory C scratchpad 的 pipeline 维度，`step = size<3>(tCsC_r2s)` 表示每轮可复用的 C smem pipeline 深度（`kSmemLayoutCBatch` = 4，故 step=4）。外层 i 在合并后的 C 元素空间中按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 `sC(_, 0, 0, j)`，再从同一个 slot 写回 global memory。

注释对 grouped mode 的语义还有两点精确限定：其一，它更准确地说是「合并 mode」而不是把底层数据拷贝或无条件 flatten 成普通一维数组：第二个 mode 仍保存 `CPY_M`/`CPY_N` 的层次关系，grouped mode 的 cardinality 是两个子 mode 的乘积，`size<1>(...)` 因此可作为一个标量范围遍历其全部 logical coordinate；其二，`(_, i+j)` 中的 $i+j$ 是该 nested mode 的**线性 logical coordinate**，不是在 C++ 中对二维 tuple 做加法，更不是 raw pointer offset，最终物理地址仍由各自 Tensor 的 grouped layout 计算。代码未对 $i+j$ 做尾部 predication，这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应把「每轮正好处理 4 个 fragment」误认为任意 tile/copy 配置都自动成立的通用规则。

22.5　图示

前面的图 22.1 讲 Tensor 的两段结构、图 22.2 讲 4 线程小例子；本节这张（图 22.5）把同一套语言放大到 hgemm 的真实配置。

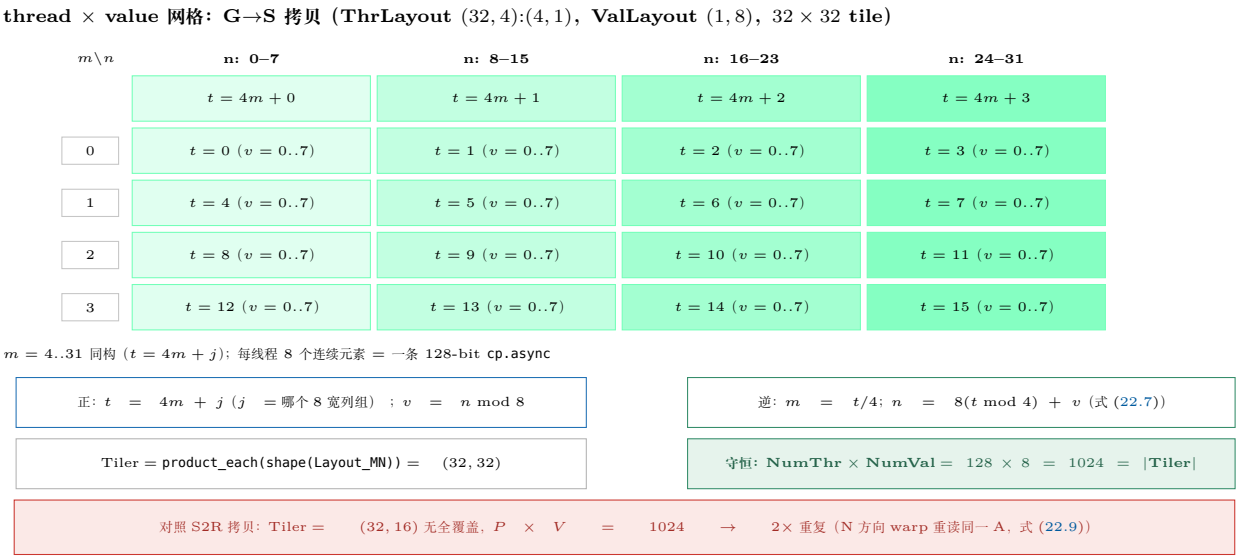


图 22.5: thread×value 网格：G→S 拷贝把 32×32 的 tile 划成 $t = 4m + j$ 的网格，每个线程搬 8 个连续元素（一条 128-bit `cp.async`）。同一划分用 MN 语言或 TV 语言描述是等价的（式 (22.6) 与式 (22.7)）。

22.6　性能分析

本章没有独立的 kernel bench：TiledCopy 的开销无法单独测，它总是嵌在某个 kernel 的流水线里。所以本节回答的是「这套抽象的代价结构」，端到端数字留给第 二十五 章（CuTe HGEMM 与手写版对照）。

结论一（编译期零成本）：Tiler、TV 布局、每个线程的地址全部是静态类型推导的结果；kernel 里没有一行显式地址算术，`cute::copy` 内联后就是 `cp.async.ca.shared.global` / `ldmatrix` / `st.shared` 的序列。第 二十一 章已经用 `static_assert` 证明过同类折叠，本章测试把 TiledCopy 的 Tiler/线程数/每线程值数也做成了静态断言。

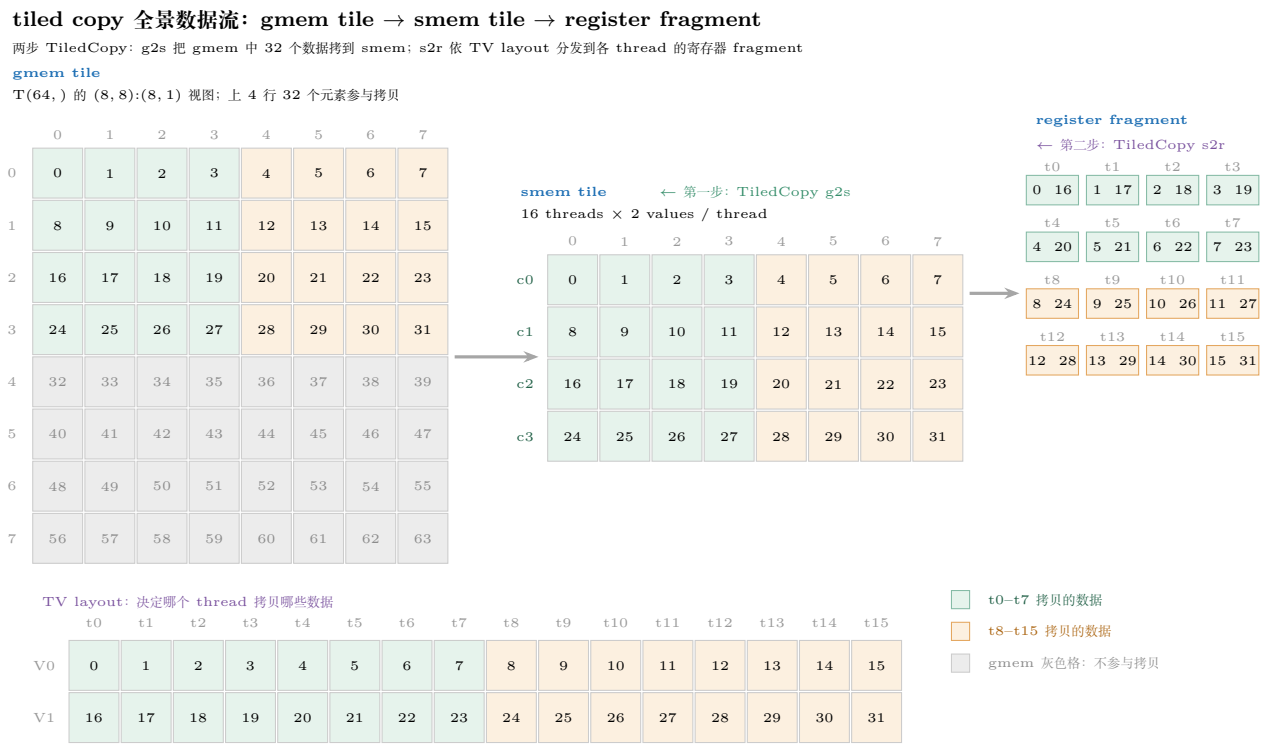


图 22.6: tiled copy 全景数据流：gmem tile（(8, 8) 视图，上半 32 个元素参与拷贝、下半灰色不参与）经 g2s TiledCopy 搬成连续 smem tile，再由 s2r 依 TV layout 切成每线程 fragment（thread t 持 $\{t, t+16\}$ 两值）。两级 TiledCopy 的分工：g2s 把 strided 的全局块搬连续，s2r 按 thread×value 分发到寄存器。重建自 @ 竹熙佳处《写给大家看的 CuTe 教程：tiled copy》。

结论二（搬运划分影响访存质量）：式 (22.7) 保证每个线程的 8 个值是连续的，所以一条 128-bit 指令就够了；如果 Thr/Val 写反（比如 `ValLayout=(8,1)`），每个线程的 8 个值会跨行分布，128-bit 拷贝会退化成 8 条 16-bit 访问。这不是性能调优技巧，而是**类型写错就直接产生 8 倍访存指令**的硬约束。第 十三 章的合并访问分析在这里变成了类型选择。

结论三（epilogue 的额外成本）：CuTe 版 epilogue 走寄存器 →smem→gmem 的 scratchpad 路线，比手写版的「shuffle + 直接 wide store」多了两次 `__syncthreads()`（每轮 R→S 后与 S→G 后各一次，L1116 与 L1184 附近）与一次 smem 往返。代价换来的是 **不写 shuffle 编排**：跨 lane 重组交给 sC 的布局做。对 $B_M \times B_N = 128 \times 256$ 的 tile，scratchpad 每轮搬 4 个 pipe slot（step = 4），R→S/R→G 各约 8 条指令，与主循环的 mma 数量相比是次要项，但它解释了为什么 CuTe 版在极端配置下需要 `kSmemLayoutCBatch` 这类旋钮。

结论四（重复因子是寄存器成本）：式 (22.9) 的 A 重复因子 = 2、B 重复因子 = 2（hgemm 的 2 × 2 warp 排布），意味着每个线程的 A/B fragment 数量按重复因子放大。这不是 TiledCopy 引入的，而是 MMA 本身要求的（第 二十三 章会从 `partition_fragment_A` 再算一遍），TiledCopy 只是把它如实反映到 partition 的形状里。

22.7 常见坑

坑一：`make_tiled_copy` 的 Thr/Val 是「坐标 → 编号」，不是「编号 → 坐标」。式 (22.4) 的方向容易写反；写成「线程 t 搬哪几个坐标」的形式（即式 (22.7)）会得到形状对、划分错的 TiledCopy：它不报错，只是每个线程搬了不连续的元素。

坑二：`partition_S/partition_D` 的 S/D 不能用「gmem/smem」来记。S 指 *source side*、D 指 *destination side*，而每个 side 有自己的原子布局（`ValLayoutSrc/ValLayoutDst`）。同一个 Tensor 既可能是 S 也可能是 D：`tAsA` 是 G→S 的 D（`partition_D(sA)`），同时也是 S→R 的 S（`partition_S(sA)`）。

坑三：`retile` 不搬数据，也不能用来对齐线程。`retile_D/retile_S` 只重标索引，同一线程内的元素顺序被重新解释；跨线程的重组必须靠 smem（或 shuffle）完成。写错方向的后果是每个线程读到自己不该有的元素：数值错、不报错。

坑四：动态 shape 会让 TV 布局退化。把 `Int<BM>` 换成运行期 `int`，`Tiler`、`partition` 的形状都变成动态

量，编译期折叠失效、地址算术回到运行期。kernel 模板参数一律用 `Int<N>` 包住正是为此（第 二十一 章坑五的同一件事，在 copy 上的表现）。

坑五：`local_tile` 之后仍带 remainder。`local_tile(A, tile, coord)` 返回的 Tensor 仍带未切出的 mode（gA 是三维的）；直接当二维用会在 `partition` 时报 rank 不匹配，或者更隐蔽地，被 `_` 占位符隐式丢掉一维。

坑六：copy 的源/目的 shape 必须逐 mode 匹配。`cute::copy(tiled_copy, src, dst)` 内部按元素对应，前提是 `src` 与 `dst` 的（值 × 剩余 mode）结构一致：这正是 `partition_S(sA_slice)` 与 `retile_D(tCrA)` 必须成对出现的原因。只 partition 一侧、另一侧手写一个 shape 相近的 Tensor，会在 `copy` 内部的静态断言处编译失败（或者更糟，静默错位）。

22.8 面试要点速查

Q1：CuTe 的 Tensor 是什么？为什么要与 layout 绑定？

A：Tensor = (Engine, Layout)，即「一块内存 + 一个坐标到 offset 的函数」。只有指针没有布局，就无法表达「行主序/列主序/带 swizzle/切片后的视图」；绑定之后，切片、分区、拷贝都可以作为布局运算在编译期完成。

Q2：TV-layout 与 MN-layout 的关系？

A：同一划分的两种视角，互为逆映射（式 (22.3)）：MN 把坐标映到 (线程, 值)，TV 反过来。Tiler 由 MN 的 shape 取 product_each 得到。

Q3：`make_tiled_copy` 的 raked_product 是什么意思？

A：把 Thr/Val 两个「坐标 → 编号」布局交错合成一个 MN 布局：同一线程的 V 个值沿坐标连续排布（raked = 交错），而不是先按线程切大块。G→S 的实例就是式 (22.6)：$t = 4m + \lfloor n/8 \rfloor$、$v = n \bmod 8$。

Q4：TiledCopy 的线程数 × 每线程值数一定等于 tile 大小吗？

A：不一定。若 TV 布局是双射（G→S、S→G、C 类拷贝）则成立（$128 \times 8 = 1024 = 32 \times 32$）；对 A/B 类拷贝会按 warp 排布出现重复因子（式 (22.9)：A 的重复因子 = N 方向 warp 数），因为多个 warp 需要同一份 fragment。

Q5：`make_tiled_copy_A/B/C` 从哪里得到线程映射？

A：从 `TiledMMA` 的 `get_layoutA/B/C_TV()` 直接取：MMA 的 fragment 约定已经规定了「哪个线程持有哪个元素」，拷贝只需把这份约定当作目的布局（式 (22.14)–(22.16)）。

Q6：`retile_S/retile_D` 做了什么？

A：零拷贝重标记：把 fragment 的（MMA, MMA_M, MMA_K）索引重排成 copy 视角的（值, 重复）索引，`data()` 不变。它是 `copy` 逐元素对应的前提，不做数据搬运、不做类型转换、不做 shuffle。

Q7：`group_modes` 与 `group` 的区别？

A：`group<B,E>` 作用于 *layout*，`group_modes<B,E>` 作用于 *Tensor*（内部就是前者）。合并 mode 只改索引结构，不改数据、不改物理地址。

Q8：S→R 的 K 方向为什么比 G→S 多一层重复？

A：原子不同。G→S 的原子一次覆盖连续的 128-bit (8 half)，而 `ldmatrix.x4` 一次只覆盖 16×16 的 K 切片；$B_K = 16$ 时 G→S 一个 tile 一级切完，S→R 需要 2 个 K 切片（实测 `tAsA_copy` 与 `tAsA` 的第三 mode 分别为 1 和 2）。

22.9 最小测试与动手实验

测试文件：`book/tests/ch21_tiled_copy.cu`（host-only：不启动 kernel、不需要 GPU；只做类型/布局断言）。六个 case 覆盖本章全部可验证结论：

- **A TV 分解**：G→S 的 `TiledNumThr`= 128、`TiledNumVal`= 8、Tiler= $(32, 32)$（编译期 `static_assert`），并对 1024 个 (t, v) 对核对手推公式式 (22.7)；
- **B partition 形状与守恒**：$(128, 32)$ 上逐线程 $(8, 4, 1)$、每线程 32 个元素、全部线程合计 4096；并把 4096 个 (线程, 元素) 覆盖点映射回坐标，验证双射（`once`= 4096）；
- **C fragment 与 retile**：TiledMMA（hgemm 配置）的 `partition_fragment_A` 形状 $(8, 4, 2)$；`retile_D` 后形状不变、`data()` 指针相同（零拷贝）；

- **D ldmatrix 的 source 视图**：S→R 拷贝在 $(128, 32)$ 上逐线程 64 个元素（$8 \times 4 \times 2$）、手推行/列公式逐点核对（线程 t 的第 0 个值为 $\big(t \bmod 16 + 16\lfloor (t/32) \bmod 2 \rfloor,\ 8\lfloor (t \bmod 32)/16 \rfloor\big)$；第 4 个线程模式步长为 0，即 t 与 $t+64$ 同源）；
- **E 重复因子**：S→R 的 A 拷贝对同一 tile 做覆盖统计，期望每个元素被覆盖恰好 2 次（N 方向 warp 的重复，式 (22.9)）；
- **F retile/upcast 语义**：`upcast` 式 (22.13) 两条；`upcast` 对 Swizzle 复合分配（`Sw<3,2,3> o _0 o (32,16):(16,1)`）；R→S 拷贝的 tiler$(32, 32)$、逐线程 $(8, 1, 1, 4)$、单 pipe 覆盖 1024 个坐标一次（双射）。

运行命令（在 `book/tests/` 下；脚本已配置 CUTLASS include 路径）：

```
cd kernels/interview/book/tests
CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch21
```

预期输出（本机 RTX PRO 5000，CUDA 13.2，CUTLASS v4.6.1，2026-09-11 实测）：

```
ch21 CuTe Tensor & TiledCopy (host-only, no kernel launch)
PASS case A make_tiled_copy TV decomposition: mismatches=0 (1027 checks)
PASS case B partition_S shape & bijection: mismatches=0 (12544 checks)
PASS case C fragment shape & retile_D zero-copy: mismatches=0 (11 checks)
PASS case D ldmatrix source partitioning formula: mismatches=0 (16640 checks)
PASS case E A-copy duplication factor: mismatches=0 (3 checks)
PASS case F upcast semantics & R2S/S2G tilers: mismatches=0 (1814 checks)
ALL OK
```

动手实验（三条，都在本章测试上改）：

1. 把 case A 的 `ThrLayout` 从 $(32, 4):(4, 1)$ 改成 $(4, 32):(32, 1)$（线程坐标 i/j 与 stride 一起换），观察 `tv layout` 的公式断言如何失效，从而理解「线程坐标的顺序」与「物理线程号的编号方式」是两件事；
2. 把 case B 的源 Tensor 从 $(128, 32)$ 换成 $(64, 32)$，观察 `partition_S` 的剩余 mode 从 4 变 2、而每线程值数不变（8 个），并检查本轮覆盖统计是否有空洞；
3. 把 case C 的 `retile_D` 换成 `retile_S`，观察返回布局的形状差异，再对照 case D 的 partition_S 形状找出「谁的形状该与谁相等」的判据。

延伸阅读

- @Anonymous《GEMM 细节分析 (二)：TiledCopy 与 cp.async》，https://zhuanlan.zhihu.com/p/703560147 （从 cp.async 的地址计算切入 TiledCopy 的划分，本章 3.4–3.6 节的对照读物）；
- @reed《cute 之 Copy 抽象》（Copy_Atom/ThrCopy/partition 抽象层次，4.2–4.4 节对照），https://zhuanlan.zhihu.com/p/666232173；
- @ 水木皇工仔《基于 CuTe 理解 swizzle, LDSM, MMA》，https://zhuanlan.zhihu.com/p/934430036（同一位作者把 LDSM 的 lane 映射与 MMA fragment 串起来讲，与本章 case D 的实验互补）；
- @ 竹熙佳处《写给大家看的 CuTe 教程：Tiled Copy》，https://zhuanlan.zhihu.com/p/1930389542784964333 （本章 3.1–3.3 节的叙述框架参照该文：动机先行、小例子入手、表格视角；公式与实测以 CUTLASS v4.6.1 头文件为准）；
- @Anonymous《GEMM 细节分析 (一)：ldmatrix 的选择》，https://zhuanlan.zhihu.com/p/702818267（ldmatrix 的转置/非转置选择，与第 十二 章的手写版互证）。

第二十三章　CuTe（三）：TiledMMA 与 fragment

23.1　本章导读

第 二十二 章把「数据怎么搬」形式化成了 TiledCopy；本章处理另一半：**计算怎么分给线程**。一条 `mma.sync.aligned.m16n8k16` 是硬件给出的最小计算单元，它规定了 32 个 lane 各自的 4（或 8）个寄存器里装哪几个元素；而一个真实 kernel 的 tile 远大于 $16 \times 8 \times 16$，于是「拿多个 MMA 原子去铺满 tile」需要一个统一的描述：这就是 **TiledMMA**：原子 × 原子的排布 × 目标 tile 尺寸。

本章沿着第 二十二 章的 (thread, value) 语言继续：第 二十二 章里 TV 划分服务于搬运，本章里 TV 划分服务于计算，并且多出一个维度：**fragment**（每个线程私有的寄存器数组）。核心问题是四个：MMA 原子的 fragment 布局是什么（3.1 节）；TiledMMA 如何由「原子 + warp 排布 + tile 尺寸」构造出来（3.2 节）；partition 如何把 Tensor 切成 $(\mathrm{MMA}, \mathrm{MMA}_M, \mathrm{MMA}_K)$ 形状的片段（3.3–3.5 节）；以及同一份寄存器为什么能在 QK 与 PV 两个不同的 TiledMMA 之间「原地复用」（4.3 节）。

前置章节：第 二十一 章（divide/zipped_divide）、第 二十二 章（Tensor 与 TV-layout）、第 十二 章（m16n8k16 的手写 fragment）、第 十六 章（FA 的 P 复用背景）。**源码区间**：ffpa_attn.cuh L31–64，即 `FFPAAttnSplitDCuTETraits` 的完整类型推导，它是全书中唯一把**两个不同 tile 尺寸的 TiledMMA 写在同一个 traits 里**的样本（QK 用 $64 \times 64 \times 16$ 一次覆盖 S，PV 用 $64 \times 16 \times 16$ 控制累加器）。使用点引 ffpa_attn.cuh L151–158、L257–264、L300–305、L339–345（cp.async 版本的 fragment 流转，与 TMA+WS 版逐行一致）。**编译宏**：`NOTES_V2_ENABLE_CUTE`。配套最小测试 `book/tests/ch22_tiled_mma.cu`：host 端 partition 一致性断言。

23.2　问题与动机

先从一条 mma 指令说起。`mma.sync.aligned.m16n8k16` 不是一个线程的算术指令，而是**一个 warp 的集体动作**：32 个 lane 各自从自己的寄存器里拿出 A、B 的一小片，硬件把它们拼成 16×16 与 8×16 的矩阵送进 tensor core，算完再把 16×8 的 C 分散写回 32 个 lane 的寄存器。所以写 mma 的第一件事是回答「**我这条 lane 的寄存器里该装哪几个元素**」。以 C（16×8）为例，thread 0 负责 $(0,0)$、$(0,1)$、$(8,0)$、$(8,1)$ 四个位置，thread 4 负责 $(1,0)$、$(1,1)$、$(9,0)$、$(9,1)$，以此类推（@ 竹熙佳处《写给大家看的 CuTe 教程：tiled mma》把这张 thread-data 映射图作为理解的起点）。

这些坐标不是自由选择的，而是 PTX 规定的 fragment 布局。回忆第 十二 章手写 HGEMM 的寄存器清单：一条 m16n8k16 要 4 个 A 寄存器（8 个 half）、2 个 B 寄存器（4 个 half）、4 个 C 寄存器（float）。以 A（16×16，列主序的 (m,k)）为例，lane 与元素的对应是

$$\mathrm{row} = \left\lfloor \frac{\mathrm{lane}}{4} \right\rfloor, \qquad \mathrm{col} = 2\,(\mathrm{lane} \bmod 4) + (0 \text{ 或 } 1), \tag{23.1}$$

外加「+8 行」与「+8 列」两套偏移把 8 个 half 填满。B（8×16）的对应关系类似。手写版（第 十二 章）的做法是：把这些式子连同 ldmatrix 的 lane 地址表一起写在 kernel 里。这条路能走通，但三个问题随规模一起放大：

- **换一条指令就要重查一次手册**。m16n8k8（fp16 半 K）、m16n8k32（int8）的 fragment 排布各不相同；每换一次，寄存器分配与地址算术都要重算，改对与否只能靠再跑一遍正确性测试；
- **跨架构可能要重写**。Volta 的 fragment 排布与 Ampere 不同，硬件换代意味着手推的公式作废：这是「CUDA + PTX」路线的固有风险；

- **放大到 tile 级别没有参考模版**。真实 kernel 的 tile 远大于 $16\times8\times16$：要么多起几个 warp 沿 M/N/K 排开（并行维度），要么让一个 warp 循环执行多条 mma（串行维度）。这两层「重复」的坐标运算（谁负责哪块、重复多少次、寄存器怎么编号），手推起来极易出错。

CuTe 对这三处问题的回应可以概括为：**把 atom 的 thread-data 映射本身变成一个 layout，然后让「重复」成为 layout 的乘法**。下面沿这条线索展开。

把上述规则放到 FFPA 上实例化。FFPA 的 QK 步骤要算 $S_{64\times64}=Q_{64\times64}K^T_{64\times64}$（每个 d-chunk），PV 步骤要算 $O_{64\times64}=P_{64\times64}V_{64\times64}$，即两个 64×64 的结果，每个线程要用多条 m16n8k16 拼出来。「拼」的规则至少包括：

- **warp 怎么分**：QK 用 4 个 warp 沿 M 方向切（每个 warp 16 行），还是 2×2 网格？不同选择决定 A/B fragment 的重复因子（第 二十二 章式 (21-dup)）；
- **每个 warp 内部怎么重复**：一个 warp 覆盖 16 行 × 64 列时，N 方向要 8 个原子；这 8 个原子的寄存器块怎么编号；
- **K 方向怎么切**：tile 的 K 长度是 64，而原子是 16，于是 fragment 多出一个 MMA_K mode；
- **两个不同形状的 tile 怎么并存**：QK 要大 tile（省循环、提高计算密度），PV 要小 N tile（控制 acc_O 寄存器），它们必须共享同一套线程/值编号。

CuTe 的答案是把这四件事压进一个类型：`TiledMMA<MMA_Atom, AtomLayoutMNK, PermutationMNK>`。一个 TiledMMA 实例只回答两个问题：「**这个 tile 有哪些 (线程, 值) 位置**」与「**给定一个 Tensor，每个线程的 fragment 是什么形状**」。@ 竹熙佳处把动机总结成「给定任意一条 mma 指令，都能方便地推导出每个线程该从 data tile 里取哪一部分」，与这里的两个问题是同一件事的两种说法。本章要验证的，就是这两个回答在 FFPA 的具体配置下能否用手推公式复现，不能复现就说明没读懂。

23.3 数学原理

23.3.1 MMA 原子：一条指令的 thread-data 映射

先回到最小尺度：一条指令、一个 warp。C 的 thread 0 拿 $(0,0)$、$(0,1)$、$(8,0)$、$(8,1)$，thread 4 拿 $(1,0)$、$(1,1)$、$(9,0)$、$(9,1)$。这张 thread-data 对照表就是原子的全部内容。CuTe 用一个 traits 类把它写成类型（`MMA_Traits<Op>`），FFPA 用到的 `SM80_16x8x16_F32F16F16F32_TN` 实测（CUTLASS v4.6.1）为：

成员	值	含义
`Shape_MNK`	$(16,8,16)$	原子覆盖的 $M\times N\times K$
`ThrID`	Layout_32	原子用 32 个线程（一个 warp）
`LayoutA_TV`	$((4,8),(2,2,2)):((32,1),(16,8,128))$	$(\mathrm{thr},\mathrm{val})\to$ A 的扁平坐标
`LayoutB_TV`	$((4,8),(2,2)):((16,1),(8,64))$	$(\mathrm{thr},\mathrm{val})\to$ B 的扁平坐标
`LayoutC_TV`	$((4,8),(2,2)):((32,1),(16,8))$	$(\mathrm{thr},\mathrm{val})\to$ C 的扁平坐标

这些 layout 的 codomain 是**扁平坐标**：A 的扁平量是 $m+16k$（M 最快），B 是 $n+8k$，C 是 $m+16n$，三者都按「第一个 mode 最快」的 colex 约定（第 二十一 章式 (20-colex-decomp)）线性化。换句话说，三份 `Layout*_TV` 就是「thread-data 映射」的类型化写法：**给它一个 (lane, 值编号)，它返回该元素在原子小矩阵里的扁平坐标**。以 `LayoutC_TV` $=((4,8),(2,2)):((32,1),(16,8))$ 为例，用映射语言说：lane 被拆成 (lane mod 4, $\lfloor\mathrm{lane}/4\rfloor$) 两维，低位那维的 stride 是 32，因为 C 的扁平坐标按 $m+16n$ 线性化，32 恰好是两列（列号加 2），高位那维 stride 是 1（行号加 1）；值编号的两维 stride 为 16（列加 1）与 8（行加 8，正是装填用的「+8 行」偏移）。把 stride 反解回坐标，就得到三条可以独立验证的公式（lane = thr，值编号 v 按 colex 拆位）：

$$\mathbf{A}:\quad m=\left\lfloor\frac{\mathrm{lane}}{4}\right\rfloor+8\,d_1,\quad k=2\,(\mathrm{lane}\bmod4)+d_0+8\,d_2, \tag{23.2}$$

$$\mathbf{B}:\quad n=\left\lfloor\frac{\mathrm{lane}}{4}\right\rfloor,\quad k=2\,(\mathrm{lane}\bmod4)+d_0+8\,d_1, \tag{23.3}$$

$$\mathbf{C}:\quad m=\left\lfloor\frac{\mathrm{lane}}{4}\right\rfloor+8\,d_1,\quad n=2\,(\mathrm{lane}\bmod4)+d_0, \tag{23.4}$$

其中 (d_0, d_1, d_2) 是值编号 v 在原子值 mode 上的 colex 分解（A：$v = d_0 + 2d_1 + 4d_2$，8 个 half；B/C：$v = d_0 + 2d_1$，4 个元素）。式 (23.2)–(23.4) 与 PTX 文档给的 m16n8k16 fragment 图完全一致。例如 lane 0 的 A 是

$$\{(0,0),(0,1),(8,0),(8,1),(0,8),(0,9),(8,8),(8,9)\},$$

lane 31 是 $\{(7,6),(7,7),(15,6),(15,7),(7,14),\dots\}$。**CuTe 没有另立一套约定，它就是 PTX 约定的类型化表达**。本章测试 case A 对三条公式在 32 个 lane 上逐点核对，并用「32 线程 × 每线程值数 = 原子元素数」验证守恒（A：$32 \times 8 = 256 = 16 \times 16$；B：$32 \times 4 = 128 = 8 \times 16$；C：$32 \times 4 = 128 = 16 \times 8$）。

单 warp 演练：`partition_fragment_A/B/C` 给出什么？把原子包成最小的 TiledMMA（warp 排布 $(1,1,1)$，tile 参数就是原子形状），对一个 16×16 的 A、8×16 的 B、16×8 的 C 调用三个 partition：tile 不放大、线程不增，每线程拿到的就是原子自身的一份 fragment：A 为 $(8,1,1)$（8 个 half）、B 为 $(4,1,1)$、C 为 $(4,1,1)$，与第 十二 章手写版的寄存器清单逐一对应。这不是巧合：**partition 在「单 warp + 单原子」的退化情形下输出的就是 PTX fragment 本身**；它真正的价值（tile 放大后的 remainder 与多 warp 的重复因子）要到 3.2–3.3 节才显现。

23.3.2　TiledMMA：原子、排布、tile 三个参数

再回到动机里那两层「重复」：**并行维度**（多起执行器）与**串行维度**（一个执行器循环执行）。CuTe 把两者装进 `make_tiled_mma(atom, AtomLayoutMNK, PermutationMNK)` 的三个参数：原子是硬件粒度，第二个参数是 **warp 排布**（哪个 warp 覆盖哪个原子位置，负责并行重复），第三个是**目标 tile 尺寸**（含 K，决定一个 threadblock 一次执行的计算块）。内核把两者合成为

$$\text{ThrLayoutVMNK} = \text{tiled_product}(\text{ThrID},\ \text{AtomLayoutMNK}) \ \cong\ (\text{ThrV}, \text{ThrM}, \text{ThrN}, \text{ThrK}), \tag{23.5}$$

实测（FFPA 的 QK）：thr_layout_vmnk $= (32,4,1,1):(1,32,0,0)$，ThrV $= 32$ 是 warp 内部线程（stride 1），ThrM $= 4$ 是 4 个 warp 沿 M 排列（stride 32）。同时 tile 的尺寸由

$$\text{tile_size}_I = \text{size}\big(\text{permutation_mnk}\langle I\rangle\big), \qquad \text{permutation_mnk}\langle I\rangle = \begin{cases} \text{显式给定}, & \text{非下划线}, \\ \text{atom}_I \times \text{Thr}_I, & \text{否则}, \end{cases} \tag{23.6}$$

给出：QK 实测 tile_size $= (64, 64, 16)$，PV 为 $(64, 16, 16)$（FFPA 两者都显式给定）。式 (23.6) 的第二行是「不显式给时会怎么算」：$\text{tile}_I = \text{atom}_I \times \text{Thr}_I$，原子长度乘该方向的 warp 组数（M 对应 ThrM、N 对应 ThrN、K 对应 ThrK）。

用映射语言说：`ThrLayoutVMNK` 把**线程编号**拆成 $(\text{ThrV}, (\text{ThrM}, \text{ThrN}, \text{ThrK}))$ 四层：先看 warp 内部（ThrV），再看「第几个 warp 落在哪个方向」。「谁并行、谁重复」这两个问题的答案，都由线程编号到 VMNK 的分解给出。

一个自然的疑问是：既然 warp 排布 + 原子尺寸已经能划分 tile，为什么还要第三个（tile）参数？@ 竹熙佳处给出两条理由。其一是**拷贝粒度**：后续 S2R/R2S tiled copy 把这个参数描述的块整体当作拷贝单位，N 方向留大一倍（每个 warp 16 而非 8）才能用上 `ldsm.x4` 一类的宽指令；其二是**重排的余地**：tile 参数本质是一个 layout，高级用法可以借助它的 stride 把多条指令算出的数据在寄存器里重新排列，这正是 `PermutationMNK` 这个名字的由来。常规场景把它当 tile 尺寸读即可。

23.3.3　q 的推导链：从 tile 到 fragment

真正的工作在 `thrfrg_A/B/C` 里，它是四步 layout 代数（第 二十一 章的全部工具在这里综合使用）。四步走的是一条固定流水线：**先按 tile 切、再按原子切、把原子坐标换成（线程，值）、最后按 warp 排布把线程剥出来**。以 A 为例（B/C 只是把 mode 换成 (N,K) / (M,N)）：

$$\text{(1) 按 tile 分块} \qquad T_1 = \text{logical_divide}\big(A, (\text{perm}_M, \text{perm}_K)\big), \tag{23.7}$$

$$\text{(2) 按原子切分} \qquad T_2 = \text{zipped_divide}\big(T_1,\ (\text{atom}_M, \text{atom}_K)\big) = \big((A_M, A_K), (R_M, R_K)\big), \tag{23.8}$$

$$\text{(3) 原子坐标}\to\text{(线程, 值)} \qquad T_3 = T_2 \circ \text{LayoutA_TV} = \big((\text{ThrV}, \text{FrgV}), (R_M, R_K)\big), \tag{23.9}$$

$$\text{(4) 按 warp 排布切分} \qquad T_4 = \text{zipped_divide}\big(T_3, (_, (\text{ThrM}, \text{ThrK}))\big)$$
$$= \big((\text{ThrV}, (\text{ThrM}, \text{ThrK})), (\text{FrgV}, (R'_M, R'_K))\big). \tag{23.10}$$

概括这条链：**把 tensor 按坐标层层切开，切到只剩原子的 MN 坐标，再经 `LayoutA_TV` 换成 (线程, 值)**。四步里的三个量各司其职：第 (1) 步的 perm 决定「一个 tile 多大」，第 (2) 步的 atom 决定「一条指令多大」，第 (4) 步的 ThrM/ThrK 决定「多少 warp 分摊」，正是 3.2 节三个参数的第三次出现；对线程 t 来说，前三步的结果对所有线程相同，只有第 (4) 步把该线程自己区分出来。

把第 (4) 步的线程坐标 (ThrV, ThrM, ThrK) 用 `get_slice(t)` 固定住，剩下的就是该线程的 fragment 形状：

$$\text{fragment} = \big(\underbrace{\text{FrgV}}_{\text{MMA}},\ \underbrace{R'_M}_{\text{MMA}_M},\ \underbrace{R'_K}_{\text{MMA}_K}\big), \tag{23.11}$$

$R'_M = R_M/\text{ThrM}$、$R'_K = R_K/\text{ThrK}$ 是「warp 分完之后每个线程还要重复多少个原子」。以 FFPA 的 QK（tile $64\times64\times16$、AtomLayout $= (4,1,1)$）在 64×64 的 Q 上为例逐项算：

- (1) perm $= (64,16)$：$T_1 = ((64,16),(1,4))$；
- (2) atom $= (16,16)$：$T_2 = ((16,16),(4,4))$，即 $R_M = 64/16 = 4$、$R_K = 64/16 = 4$；
- (3) LayoutA_TV 把 $(16,16)$ 变成 $(\text{ThrV}=32, \text{FrgV}=8)$：$T_3 = ((32,8),(4,4))$；
- (4) ThrM $= 4$：$(\text{FrgV}, (R'_M, R'_K)) = (8, (1,4))$。

于是 MMA $= 8$、$\text{MMA}_M = 1$、$\text{MMA}_K = 4$，fragment 大小 $8\times1\times4 = 32$，与本章测试 case C 的实测 `((2,2,2), 1,4)` 完全一致（$(2,2,2)$ 是 8 的嵌套写法）。测试同时对 B 与 C 复算了这条链：QK 的 B 在 $(64,64)$ 上得 $(4,8,4)$（$= 128$ 个 half/线程，因为 4 个 M-warp 各持一份完整 B，重复因子 4，式 (23.12)）；C 得 $(4,1,8) = 32$（无重复，4096/128）。四步推导链的 shape 演化见图 23.1。

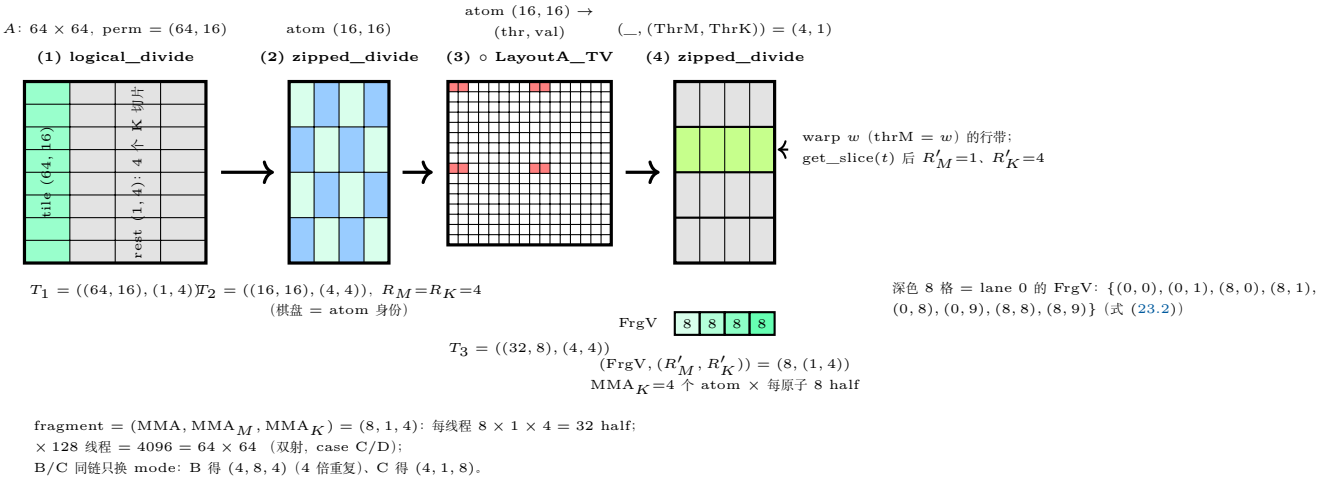


图 23.1: thrfrg_A 四步推导链（式 (23.7)–(23.10)）在 FFPA QK（tile $64\times64\times16$、AtomLayout $(4,1,1)$、Q 为 64×64）上的 shape 演化：(1) logical_divide 按 tile 切出 $(64,16)$ 与 4 个 K 切片；(2) zipped_divide 按 atom $(16,16)$ 切成 4×4 个原子块；(3) 复合 LayoutA_TV 把原子内坐标换成 (线程, 值)，深色 8 格即 lane 0 的 FrgV；(4) 按 $(\text{ThrM}, \text{ThrK}) = (4,1)$ 剥出 warp 行带，get_slice(t) 后每线程 fragment $= (8,(1,4)) = 32$ half（右下：$\text{FrgV} = 8\times\text{MMA}_K = 4$）。

$$\text{A 的重复因子} = \#\{\text{N-warp}\}, \qquad \text{B 的重复因子} = \#\{\text{M-warp}\}, \qquad \text{C 无重复}. \tag{23.12}$$

23.3.4　守恒与 tile 循环

把三个 fragment 的每线程元素数乘以线程数，得到的是「数据被读了几遍」：C 恰好等于 tile 元素数（$128 \times 32 = 4096 = 64 \times 64$，双射），A 也是（ThrK = 1 时无 K-warp 重复），B 是 4 倍。这个重复因子在选 tile 时必须算清楚：B 的 4 倍重复意味着 128 个 half/线程的寄存器开销，它不占 smem，但占用寄存器预算。守恒式也是最便宜的自检：两边对不上，说明有数据没有线程负责、或有线程在空转（本章测试 case D 就是这条守恒式的逐点版本）。

当 tensor 比 tile 大时，partition 会多出「tile 重复」mode：64×64 的 tile 作用在 64×256 的 C 上，MMA_N 从 8 变成 32。这不是新机制，只是式 (23.7) 的 T_1 多留了一层 remainder。`gemm(tiled_mma, A, B, C)` 会在这一层 remainder 上循环，这正是第 二十二 章 kernel 里「只显式展开 K 循环、M/N 交给 `cute::gemm`」的原因（hgemm.cuh L925–937 的注释）。

23.3.5　get_slice：线程坐标的三层解构

一行调用 `thr_mma = tiled_mma.get_slice(t)` 就能完成线程坐标的三层解构：把整数 t 按式 (23.5) 的 layout 分解成 $(\mathrm{ThrV}, (\mathrm{ThrM}, \mathrm{ThrK}))$（C 用 $(\mathrm{ThrV}, (\mathrm{ThrM}, \mathrm{ThrN}))$）。FFPA 的 QK：$\mathrm{ThrLayoutVMNK} = (32,4,1,1):(1,32,0,0)$，于是

$$\mathrm{ThrV} = t \bmod 32, \qquad \mathrm{ThrM} = \left\lfloor \frac{t}{32} \right\rfloor \bmod 4, \qquad \mathrm{ThrK} = 0. \tag{23.13}$$

本测试 case E 用它验证了「warp 边界」：warp w 的 C fragment 恰好覆盖 M 方向的第 $16w$ 到 $16w+15$ 行，四个 warp 的行带互不重叠且并集为 $[0,64)$，这就是 $\mathrm{ThrM} = 4$、$\mathrm{atom}_M = 16$ 的直接推论。`get_thread_slice` 是 `get_slice` 的别名（v4.6.1 实测同一实现），旧教程里两个名字都可能出现。

23.4　设计与实现

23.4.1　FFPA traits：两个 TiledMMA 的完整推导

先看源码区间本体，它短小但信息密度极高，每一行都对应 3.3 节的一个量：

ffpa_attn.cuh L31-62（FFPAAttnSplitDCuTeTraits：QK 与 PV 双 TiledMMA）

```
namespace fa_cute {
using namespace cute;

template <int kHeadDim, int TILE_M = 64, int TILE_N = 64>
struct FFPAAttnSplitDCuTeTraits {
  static_assert(kHeadDim % 64 == 0, "Split-D requires head-dim multiple of 64");
  static_assert(TILE_M == 64 && TILE_N == 64, "Current impl supports 64x64 only");

  using Element = cutlass::half_t;
  using SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>;
  using SmemLayoutQ = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<Int<TILE_M>, _64>{}));
  using SmemLayoutKV = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<Int<TILE_N>, _64>{}));
  using SmemLayoutVt = decltype(composition(
      SmemLayoutKV{}, make_layout(Shape<_64, Int<TILE_N>>{}, GenRowMajor{})));

  using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;

  // QK: Tile<64,64,16> → EURepeat<1,8,1>, 一次覆盖 S[64,64]
  using TiledMmaQK = decltype(make_tiled_mma(
      MmaAtom{}, Layout<Shape<_4, _1, _1>>{},
      Tile<Int<TILE_M>, Int<TILE_N>, _16>{}));

  // PV: Tile<64,16,16> → EURepeat<1,2,1>, 控制 acc_O 寄存器
  using TiledMmaPV = decltype(make_tiled_mma(
```

```
      MmaAtom{}, Layout<Shape<_4, _1, _1>>{},
      Tile<Int<TILE_M>, _16, _16>{}));

  using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
  using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
};
```

逐项解读：

- `SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<half>`：smem 的布局原子（第 二十四 章的 SW128），本 traits 只用它定义 Q/KV 的 smem 视图；64×64 的 tile 实测 `cosize` $= 4096$（与 64×64 个 half 一致，无空洞）；
- `SmemLayoutVt`：把 KV 的布局转置成「d 为行、kv 为列」，供 PV 的 B（V^T）使用，这是第 二十 章讲过的「V 以转置形态交给 MMA」的布局表达；
- `MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>`：F32 累加（attention 的数值要求），与 3.1 节的 fragment 一致；
- `TiledMmaQK`：AtomLayoutMNK $= (4, 1, 1)$（4 个 warp 沿 M 切）、PermutationMNK $= (64, 64, 16)$。实测 tile_size $= (64, 64, 16)$、线程数 128；
- `TiledMmaPV`：同样的 warp 排布，tile 换成 $(64, 16, 16)$：N 方向只留 16，因为 acc_O 要常驻寄存器；
- 两个 `Copy_Atom`：`LDSM_N`（正常读）与 `LDSM_T`（转置读 V），它们是第 二十二 章 `make_tiled_copy_B` 的原子参数。

为什么 QK 用 64×64 的 tile？因为 S 的 N 方向（KV 方向）正好是 64，一次 TiledMMA 覆盖整个 S，省掉 N-tile 循环。实测 $\text{MMA}_N = 8$（8 个 8 列的原子），全部落在同一个 warp 内、同一批寄存器里；这对应注释里的「EURepeat=$< 1, 8, 1 >$」。

为什么 PV 用 64×16？因为 acc_O 是「$64 \times D$-chunk $\times$ D 方向 chunk 数」的大数组。每线程每个 chunk 32 个 float，$D = 512$ 时 8 个 chunk 共 **256 个寄存器**，这已经逼近寄存器文件的上限（第 二十 章的寄存器压力讨论）。若把 PV 的 N 也取 64，单次 gemm 调用的 C 视图会从 8 涨到 32、B fragment 随之线性增长，S→R 拷贝粒度变粗，常驻 acc_O 并不变。实测 PV 在 $(64, 16)$ 的 C 上每线程 $(4, 1, 2) = 8$ 个 float，与注释「EURepeat=$< 1, 2, 1 >$」一致。**tile 的 N 尺寸是 per-call C 视图与拷贝粒度的旋钮**。

实践须知　（读 FFPA traits 注释时容易踩的两处）

- **注释里的线程数要带语境读**。traits 上方的「128 producer + 128 consumer = 256 total threads（vs 原始 384）」描述的是 *TMA+WS* 版 kernel 的线程配置；本 traits 服务的 cp.async 版是 128 线程（L73–78 注释）。实测 `size(TiledMmaQK)` $= 128$，与 `Layout<4,1,1>`（4×32）自洽。看到注释数字与实测不一致时，先问「它描述的是哪个版本」，再下结论。
- 「**EURepeat**」**有两种语境**。hgemm.cuh L1290–1292 的 `kMmaEURepeatM/N/K` 说的是 *warp* 排布（`make_tiled_mma` 的第二个参数）；本章 FFPA 注释里的 $\langle 1, 8, 1\rangle/\langle 1, 2, 1\rangle$ 说的是每线程原子重复（式 (23.11) 的 $(\text{MMA}_M, \text{MMA}_N, \text{MMA}_K)$，由 tile 尺寸与 warp 排布共同推出）。同一个词，两个层面。读旧教程前先确认语境。

23.4.2 fragment 的使用：三行代码的完整语义

定义好了类型，使用点非常短。accumulator 的定义（L151–158）：

ffpa_attn.cuh L151-158（O fragment 的类型与行列视图常量）

```
// O fragment layout (与 TMA WS kernel 完全一致)
using OFragType = decltype(partition_fragment_C(tiled_mma_pv, Shape<_64, _64>{}));
using OFragLayout = typename OFragType::layout_type;
constexpr int kOElemsPerFrag = decltype(size(OFragType{}))::value;
```

```
  constexpr int kORows = decltype(size<0>(make_tensor(
      (float*)nullptr, fa_cute::convert_layout_acc_rowcol(OFragLayout{}))))::value;
  constexpr int kOCols = decltype(size<1>(make_tensor(
      (float*)nullptr, fa_cute::convert_layout_acc_rowcol(OFragLayout{}))))::value;
```

注意 `OFragType` 是用 `Shape<_,64,_64>` 实例化的，虽然 PV 的 tile 是 64×16，这里刻意按 64×64 取 fragment（N 方向 4 个 tile 重复），因为一个 d-chunk 的 O 就是 64×64。实测 `kOElemsPerFrag`$= 32$、`kORows`$= 2$、`kOCols`$= 16$：后两者来自 `convert_layout_acc_rowcol` 把 $(\text{MMA} = 4, \text{MMA}_M = 1, \text{MMA}_N = 8)$ 重排成 $\big((2, \text{MMA}_M), (2, \text{MMA}_N)\big) = (2, 16)$ 的「行 × 列」视图（每线程 2 行 × 16 列），online softmax 才能按行遍历（第 二十六、二十七 章详解这个技巧）。

QK 的三行（L257–264）：

ffpa_attn.cuh L257-264（QK：A/B fragment + gemm_ss）

```
          make_smem_ptr(k_base + qk_read * kKVChunkElements), SmemLayoutKV{});
      auto tCrQ = thr_mma_qk.partition_fragment_A(sQ_stg);
      auto tCrK = thr_mma_qk.partition_fragment_B(sK_stg);
      auto tQsQ_s2r = s2r_thr_q.partition_S(sQ_stg);
      auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);
      fa_cute::gemm_ss(tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
                       tiled_mma_qk, s2r_copy_q, s2r_copy_k,
                       s2r_thr_q, s2r_thr_k);
```

`partition_fragment_A(sQ_stg)` 的输入是 $(64, 64)$ 的一个 d-chunk 的 Q，返回 $(8, 1, 4)$ 的寄存器 fragment，即式 (23.11) 的实测形态。注意 `tCrS` 是在循环外一次性定义的（`partition_fragment_C(tiled_mma_qk,Shape<_,64,_64>{})` $= (4, 1, 8) = 32$），所有 d-chunk 的 QK 结果累加到同一份 S 寄存器上。**这正是 Split-D 的数学：$S = \sum_d Q_d K_d^T$ 用累加器直接表达。**

23.4.3　QK→PV：fragment 的原地复用

softmax 之后，S 要变成 PV 的 A 矩阵 P。FFPA 不把 P 写回 smem，而是原地重解释（L300–305）：

ffpa_attn.cuh L300-305（把 QK 的 C fragment 重解释为 PV 的 A fragment）

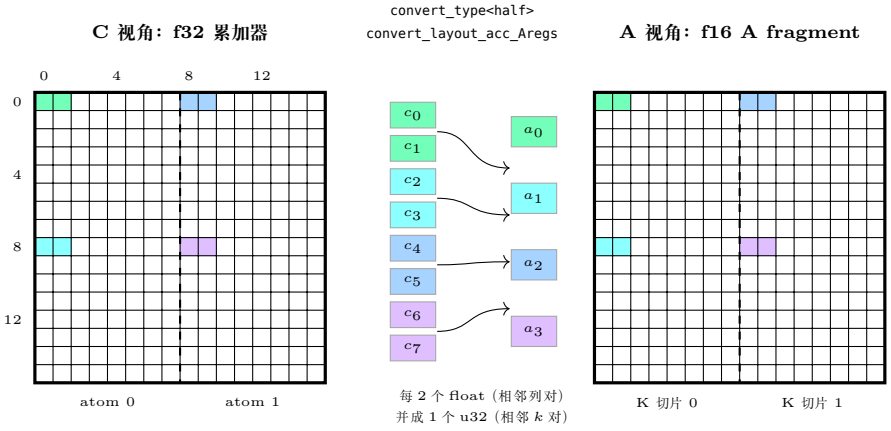
```
    // P fragment: convert_layout_acc_Aregs<TiledMmaPV> (与 TMA WS kernel 一致)
    auto tCrP = fa_cute::convert_type<Element>(tCrS);
    auto tCrPv = make_tensor(
        tCrP.data(),
        fa_cute::convert_layout_acc_Aregs<typename Traits::TiledMmaPV>(
            tCrP.layout()));
```

`convert_type<half>` 做 fp32→fp16 的类型转换（零拷贝，只改元素类型解释）；`convert_layout_acc_Aregs<TiledMmaPV>` 把 C 的寄存器排布（4 个 float 按「2 行 × 2 列」）重排成 A 的排布（「2 行 × 2 个 K 元素」）。这一步之所以可行，是两个 fragment 的寄存器物理布局恰好兼容：m16n8k16 的 C 与 A 在同一个 lane 上都持有「2 个相邻列 + 2 个相隔 8 的行」，所以只需重新映射索引。**这是依赖具体 atom 的优化，不是 TiledMMA 的通用性质**（源码注释在 flash_attn.cuh L2341–2372 明确写了这一点，第 二十六 章会再展开）。PV 一侧的使用（L339–345）：

ffpa_attn.cuh L339-345（PV：B fragment 的转置视图 + gemm_rs）

```
          make_smem_ptr(v_base + v_read * kKVChunkElements), SmemLayoutKV{});
      auto sVt_stg = make_tensor(sV_stg.data(), typename Traits::SmemLayoutVt{});
      auto tCrVStorage = thr_mma_pv.partition_fragment_B(sV_stg);
      auto tCrV = make_tensor(tCrVStorage.data(), tCrV_layout);
      auto tVsVt = s2r_thr_v.partition_S(sVt_stg);
      fa_cute::gemm_rs(tCrO, tCrPv, tCrV, tVsVt,
                       tiled_mma_pv, s2r_copy_v, s2r_thr_v);
```

`tCrVStorage` 是「按 KV 布局的 V」的 B fragment，`tCrV` 用 `tCrV_layout`（在循环外用 `SmemLayoutVt` 的非 *swizzle* 部分推导，L136–140）重新解释同一批寄存器，与第 二十二 章的 `retile` 是同一类「零拷贝重解释」手法。QK→PV 这一步的两套寄存器坐标见图 23.2。



转换式（MMA=4，MMA_M，MMA_N）→（(MMA, 2)，MMA_M，$\mathrm{MMA}_N/2$）：`logical_divide` 把 N 方向按 2 切分，于是 C 的「相邻列对」成为 A 的「相邻 k 对」，A 的 K=16 由相邻两个 n8 atom 提供（S 上相邻的 16 列）。两侧高亮的是同一个 lane 0 的同一批 8 个逻辑元素：左边是 8 个 f32 槽，右边折成 4 个 u32（每槽 2 个相邻 k 的 half）。全程零拷贝，且依赖 m16n8k16 的特定 fragment 约定——不是 TiledMMA 的通用性质。

图 23.2: QK→PV 的寄存器原地复用：两侧网格是同一个 16×16 坐标空间（S 的列 = P 的 K），高亮的是 lane 0 在同一批元素上的两套坐标。左（C 视角）：每个 n8 atom 给 lane 0 留下 4 个 f32 累加器（2 行 × 2 相邻列），两个相邻 atom 共 8 个；右（A 视角）：同样 8 个逻辑元素被解释成 4 个 u32，每槽 2 个相邻 k 的 half，可直接作为 `gemm_rs` 的 `tCrA`。`convert_type<half>`（fp32→fp16，寄存器减半）+ `convert_layout_acc_Aregs`（N 方向按 2 切分的索引重解释）就是这条「P 不写回 smem」路径的全部代价。

23.5 图示



图 23.3: TiledMMA partition：QK 的 64×64 tile 上，4 个 warp 各覆盖 16 行，warp 内 N 方向重复 8 个原子；A/B/C 三种 fragment 的每线程形状与重复因子（式 (23.12)）。下半部分是 PV 的 (64, 16, 16) tile：N 方向只重复 2 个原子，代价换来 per-call C 视图与拷贝粒度可控（8 个 d-chunk 各自拷回，常驻 acc_O 256 寄存器不变；N tile 取 64 只是单次调用的 C fragment 临时 4 倍）。

23.6 性能分析

本章同样没有独立 bench：TiledMMA 的开销体现在它选择的 tile 形状上，端到端数字在第 二十、二十七 章（FFPA）与第 二十五 章（HGEMM）。本节把「tile 选择」化成三个可计算的指标。

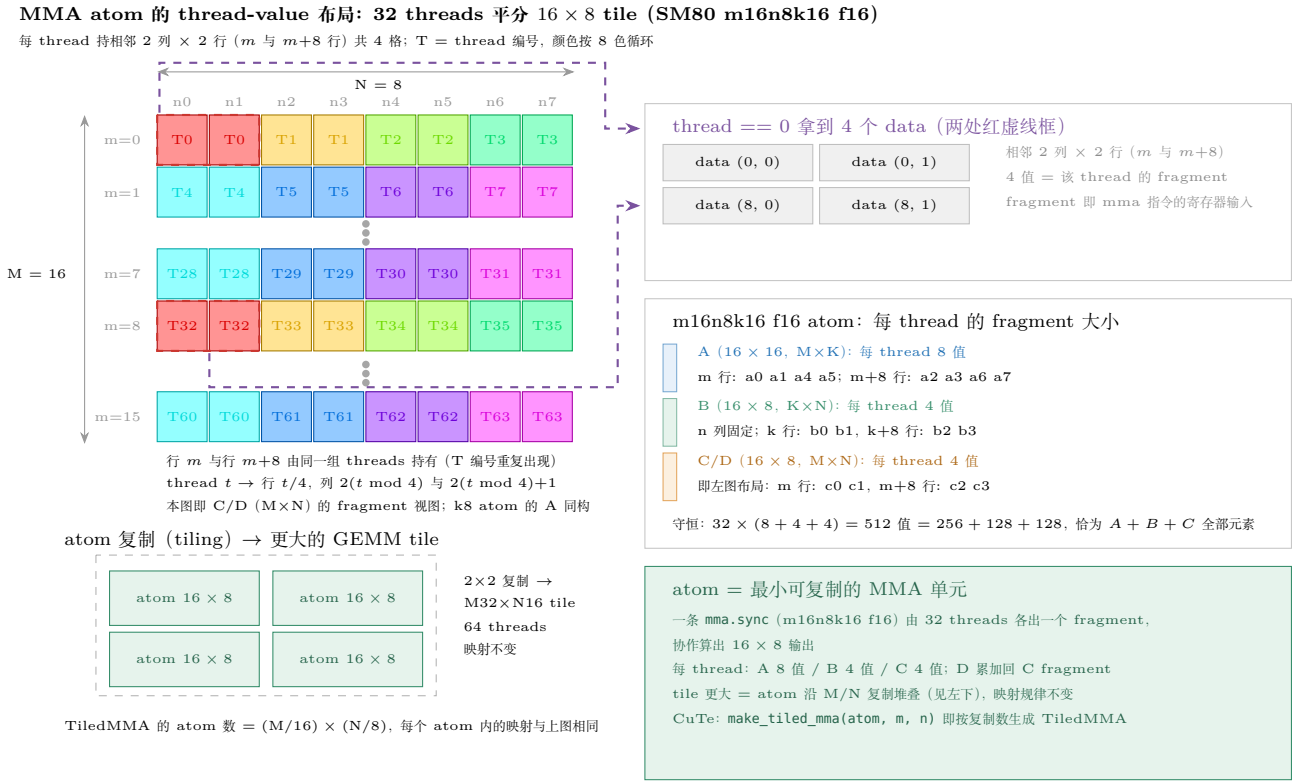


图 23.4: m16n8k16 atom 的 C fragment thread-value 映射：thread t 持行 $t/4$ 与 $t/4+8$、列 $2(t \bmod 4)$ 与 $2(t \bmod 4)+1$ 共 4 值——行 m 与 $m+8$ 由同组线程重复持有。三个操作数每线程 A 8 值 / B 4 值 / C 4 值，守恒式 $32\times(8+4+4) = 512$。更大的 tile 只是 atom 沿 M/N 的复制堆叠，thread-value 规律不变——`make_tiled_mma` 的构造基础。重建自 @ 竹熙佳处《写给大家看的 CuTe 教程：tiled mma》。

指标一（寄存器）：acc_O = $kD_{\text{chunks}} \times \text{ThrLD} \times \text{MMA}_M \times \text{MMA}_N \times \text{Nrep}$（ThrLD 是单个 atom 每线程的 C 值数 4，Nrep 是 chunk 内 N 方向的重复数 $64/16 = 4$）$= 8 \times 4 \times 1 \times 2 \times 4 = 256$。PV 的 $\text{MMA}_N = 2$（16/8）时每个 64×64 d-chunk 每线程是 32 个 float；$D = 512$ 时 8 个 chunk 共 256 个寄存器。tile N=64 时单次 gemm 调用的 C fragment 从 8 值涨到 32 值（4 倍），常驻 acc_O（8 chunk × 32 = 256 float）不变，超支的是每次调用的临时视图与 B fragment，不是累加器。FFPA 注释「PV 保持小 tile」的算术依据正在于此：压的是每次调用的临时开销，而非 acc_O。

指标二（重复因子）：B 的 fragment 按 M-warp 数重复（式 (23.12)）。QK 用 $(4, 1, 1)$ 排布时 B 重复 4 倍（$4\times32 = 128$ 个 half/线程），A 不重复；若改成 $(2, 2, 1)$（各方向 2 个 warp），A 与 B 各重复 2 倍，总寄存器开销几乎不变，但 warp 覆盖的行会从 16 行变成 32 行。warp 排布的选择因此是「谁重复」的问题，而不是「重复多少」的问题。

指标三（指令数）：一次 TiledMMA 调用的指令数 = $\text{MMA}_M \times \text{MMA}_N \times \text{MMA}_K$，注意这是 *每线程*（*每 warp*）的条数。QK 的 $(1, 8, 4) = 32$ 条是每个 warp 的量，覆盖该 warp 的 $16 \times 64 \times 64$；4 个 warp 共 128 条才铺满 CTA 的 $64 \times 64 \times 64$：512 KFLOP/128 条，每条 $16 \times 8 \times 16$ 共 4096 FLOP（约 4.1 KFLOP）；PV 同口径：$(1, 2, 4) = 8$ 条/warp 覆盖 $16 \times 16 \times 64$，CTA 的 $64 \times 16 \times 64$ 需 4 warp 共 32 条。两个 tile 都让「每条 MMA 指令」用满原子尺寸，没有浪费，因为它们都是从 AtomShape 出发、只用排布与 tile 尺寸做整数分块推出来的。

口径说明：本节所有数字都是 host 端类型推导的实测值（`partition_fragment_*` 的形状），不含计时数据；FFPA 的 TFLOPS 对照见第 二十 章的表与第 二十七 章的 kernel 分析。还有一条引用规则：ffpa_attn.cuh L16–18 的头注释给出「SM120a RTX PRO 5000：cuDNN SDPA 57.2 TFLOPS、FFPA TMA WS 110.7 TFLOPS（1.94x）」，除法自洽（$110.7/57.2 = 1.94$），但设备口径是 *RTX PRO 5000*（本机），与 README 的 5090 口径不同。**引用这类比值时必须带上设备名**，同一份代码在两块卡上的比值并不相等。

23.7　常见坑

坑一：fragment 必须先清零/初始化。 `partition_fragment_C` 返回的是「一块未定义的寄存器数组」的视图，不会置零。FFPA 里 `tCrS` 定义后立即 `clear(tCrS)`，acc_O 用显式循环清零（L173–176）。漏掉这一步的后果不是崩溃，而是把未初始化寄存器当累加初值，编译器还可能把整个累加链折叠成垃圾值，表现为「结果偶尔对、偶尔全错」。

同理，被 convert_layout_acc_Aregs 重解释的 P 必须确保所有 32 个槽位都已被 softmax 写过。

坑二：两个 TiledMMA 的 fragment 不能互换。 QK 的 C fragment 是 $(4,1,8)$，PV 的 A fragment 是 $(8,1,4)$：形状、元素类型 (float/half) 都不同。FFPA 的复用之所以成立，是因为它做了 *类型转换 + 布局重解释*两步（convert_type + convert_layout_acc_Aregs），且只对 m16n8k16 这一族成立。把 C 直接交给 PV 是经典的静默错位。

坑三：tile 必须是原子的整数倍，warp 排布必须与线程数自洽。 perm_M 不是 16 的倍数、或 ThrM × 32 与 blockDim 不一致时，partition 会产生多余/缺失的 remainder mode（编译期静态断言或运行期越界）。判据：size(mma) 是线程数的权威来源，用它来配 dim3 block(size(mma))。

坑四：get_slice 的坐标是 (ThrV, (ThrM, ThrN/K)) 三层，不是「warp 内 lane + 全局 warp」两层。 FFPA 的 128 线程排布是 $(\text{ThrV}=32, \text{ThrM}=4)$，因此线程 t 的行带是 $16\lfloor t/32 \rfloor$；把 ThrM 当成「线程号对 4 取模」会得到完全错误的 warp 映射（本章测试 case E 专门核对这条）。

坑五：partition_fragment_A 的输入决定 fragment 的 MMAK 大小，但不改变 atom 约定。 QK 的 A 在 $(64,64)$ 上是 $(8,1,4)$，在 $(64,16)$ 上就是 $(8,1,1)$；MMA_K 随输入 K 长度变化。写循环时要用 size<2>(fragment) 作为 K 循环上界（第 二十二 章 kernel 的 num_k_steps 就是这么来的），不要写死。

坑六：C fragment 的「行列」不是直觉顺序。 $(4,1,8)$ 的 4 个 float 对应「2 行 × 2 列」，同一个 lane 的第 0、1 个元素在同一行相邻两列，第 2、3 个在 +8 行处。想按 (row, col) 遍历必须先做 convert_layout_acc_rowcol (L2315–2333 推导)；直接用 fragment(idx) 当矩阵元素是最常见的读错方式。

23.8 面试要点速查

Q1：MMA_Atom 与 TiledMMA 的区别？

A：MMA_Atom 是一条指令的抽象（Shape_MNK + 三份 TV layout + ThrID）；TiledMMA 是「把原子铺满一个 tile」的类型，多出 warp 排布与 tile 尺寸两个参数，并提供 partition_fragment_*。

Q2：make_tiled_mma 的第二个、第三个参数分别是什么？

A：第二个是 warp 排布（AtomLayoutMNK，决定 ThrM/ThrN/ThrK 与重复因子），第三个是目标 tile 尺寸（PermutationMNK，含 K）。FFPA：排布 $(4,1,1)$，tile $(64,64,16)$ 与 $(64,16,16)$。

Q3：fragment 的三个 mode 是什么？

A：$(\text{MMA}, \text{MMA}_M, \text{MMA}_K)$：每个原子的值数、M 方向原子重复、K 方向原子重复（C 的第三 mode 是 MMA_N）。它们是式 (23.10) 第 (4) 步之后的余量。

Q4：为什么 A 的 fragment 有时会重复？

A：warp 排布如果在 N 方向有分片，不同 N-warp 需要同一份 A，于是 A 被重复持有（重复因子 = N-warp 数）；B 同理（因子 = M-warp 数）。式 (23.12)。

Q5：get_slice(t) 内部发生了什么？

A：把整数 t 按 ThrLayoutVMNK 分解成 $(\text{ThrV}, (\text{ThrM}, \text{ThrN}), \text{ThrK})$，得到一个「只属于该线程」的 ThrMMA，后续 partition 都基于它。

Q6：QK 与 PV 为什么要用两个 TiledMMA？

A：目标不同。QK 要一次覆盖整个 S（N = 64）省循环；PV 的 N 直接决定 acc_O 的寄存器数，必须小。两者共享同一套 128 线程排布，fragment 通过类型转换 + 布局重解释衔接。

Q7：tile_size<I>(mma) 有什么用？

A：它是 TiledMMA 对外的尺寸接口，也是 make_tiled_copy_A/B/C 推导 S→R 拷贝 tiler 的来源（第 二十二 章式 (21-mtca)）。TiledMMA 与 TiledCopy 通过它解耦：改 tile 不影响拷贝代码。

Q8：tile 的 N 尺寸调节什么？

A：单次调用 C fragment 每线程元素数 $= \text{MMA} \times \text{MMA}_M \times \text{MMA}_N$，而 $\text{MMA}_N = \text{tile}_N / \text{atom}_N$：tile N 增大时 per-call C 视图与 B fragment 随之线性增长，S→R 拷贝粒度变粗；常驻 acc_O（$D = 512$ 时 256 个寄存器）不变，PV 因此取 $\text{tile}_N = 16$（$64 \times 16 \times 16$）。

23.9 最小测试与动手实验

测试文件：`book/tests/ch22_tiled_mma.cu`（host-only：只做类型与 partition 断言，不启动 kernel；直接 include `../../ffpa_attn.cuh` 以核对真实的 traits）。六个 case：

- **A 原子 fragment 公式**：对 `SM80_16x8x16_F32F16F16F32_TN` 的 A/B/C 三个 layout，逐 lane、逐值核对手推公式式 (23.2)–(23.4)（32×8、32×4、32×4 个断言）；
- **B tile_size 与线程数**：QK/PV 的 `tile_size` 为 $(64,64,16)/(64,16,16)$、`size(mma)`$=128$；对象 `FFPAAttnSplitDCuTeTraits<512>` 并与手写的 `make_tiled_mma` 构造做类型一致性检查；
- **C fragment 形状**：QK 在 $(64,64)$ 上 A/B/C 每线程 $(8,1,4)/(4,8,4)/(4,1,8)$，PV 为 $(8,1,4)/(4,2,1)/(4,1,8)$；逐项核对元素数；
- **D 守恒与重复因子**：C 的覆盖是双射（$128\times32=4096=64\times64$）且 A 也是；B 恰好 4 倍重复（每元素被 4 个 (线程, 值) 对覆盖）；
- **E warp 边界**：QK 的 C fragment 逐线程回投坐标，四个 warp 的行带为 $[0,16),[16,32),[32,48),[48,64)$，互不重叠；
- **F PV 的 acc_O 占用**：`partition_fragment_C(tiled_mma_pv, Shape<_,64,_64>{})` 每线程 32 个值，$\times$ 8 个 d-chunk（$D=512$）$=256$ 个寄存器，与源码注释的寄存器压力说法互证；并核对 PV 的 C 覆盖（64×16 双射）。

运行命令：

```
cd kernels/interview/book/tests
CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch22
```

预期输出（本机 RTX PRO 5000，CUDA 13.2，CUTLASS v4.6.1，2026-09-11 实测）：

```
ch22 CuTe TiledMMA & fragments (host-only, no kernel launch)
PASS case A MMA atom fragment layouts (m16n8k16): mismatches=0 (519 checks)
PASS case B FFPA traits tile_size & thread count: mismatches=0 (14 checks)
PASS case C partition_fragment shapes QK/PV: mismatches=0 (12 checks)
PASS case D conservation & B duplication factor: mismatches=0 (4 checks)
PASS case E warp row bands & cross-warp boundary: mismatches=0 (4360 checks)
PASS case F PV acc_O register budget & coverage: mismatches=0 (136 checks)
ALL OK
```

动手实验：

1. 把 case C 的 PV mma 的 tile 从 `Tile<64,16,16>` 改成 `Tile<64,32,16>`，观察 fragment 的 MMA_N 从 2 变 4、单次调用的 C 视图从 8 涨到 16 个值，而单个 d-chunk 的 acc_O 仍是 32，亲手体验「tile N 调节 per-call C 视图与拷贝粒度」；
2. 把 QK 的 warp 排布从 `Layout<Shape<_,4,_,_>>` 改成 `Layout<Shape<_,2,_2,_>>`（2×2 网格），观察 A/B 的重复因子与 warp 行带如何变化（case D/E 的断言会同时失效，这正是要看的现象）；
3. 把 case A 的 lane 换成 $32+k$ 或把值编号顺序反转，确认断言能立刻抓到错误，用来理解「formula-driven 测试」与「只比形状」的差别。

延伸阅读

- @ 竹熙佳处《写给大家看的 CuTe 教程：Tiled MMA》，https://zhuanlan.zhihu.com/p/1937145378446226159（本章全部结论以 CUTLASS v4.6.1 头文件实测为准）；
- @reed《cute 之 MMA 抽象》，https://zhuanlan.zhihu.com/p/663092747（MMA_Atom/MMA_Traits 的设计动机与历史，读 3.1–3.2 节时对照）；

- @ 可怕的杰瑞《Cute TiledMMA 简单理解》，https://zhuanlan.zhihu.com/p/1991908850132088026（用可编译的最小例子演示 make_tiled_mma 的三个参数，与本章 case B 的实验同构；续篇 https://zhuanlan.zhihu.com/p/1992328297888126320）；
- @ 水木皇工仔《基于 CuTe 理解 swizzle, LDSM, MMA》，https://zhuanlan.zhihu.com/p/934430036（把 LDSM 的 lane 映射与 MMA fragment 放在一起推，与第 二十二 章 case D、本章 case A 互补）。

第二十四章　CuTe（四）：Swizzle 与 TMA

24.1　本章导读

第 十三 章用一次 XOR 消除了 smem 的 bank conflict：把列块索引换成「列块 XOR 行号」，即 `swizzle<64>(row, k)`。在那章的语境里它是一个 地址技巧；可一旦同一段 kernel 里出现了 TMA、fp8、多级 tile 与多 stage 缓冲，同样的技巧就要同时回答三个新问题：**它到底是什么形式的数学对象、怎么和「形状 + 步长」复合、怎么与硬件自己做的 swizzle 对齐**。本章把这三件事摊开。

前置章节：第 十三 章（XOR swizzle 的银行冲突推导与手写 `swizzle<>` 家族）、第 二十一 章（Layout 代数与 `composition`）、第 二十二 章（Tensor 与 TiledCopy）、第 二十三 章（TiledMMA）。本章是 Part IV「CuTe 应用篇」的第二章，不引入新的 CuTe 抽象，只把已有的 `Layout/Tensor` 用到 swizzle 与 TMA 两个难点上。

源码区间：hgemm.cuh L1232–1270，即 Phase 7c 的 smem 布局定义（`SmemLayoutAtom/SmemLayoutA/SmemLayoutB` 的定义及其上方的位级注释）。为了给出 TMA 一侧的完整链路，本章还引用：hgemm.cuh L1859–2100（宏块 `NOTES_V2_ENABLE_TMA_MMA_WS` 内的 kernel `hgemm_tma_mma_ws_tn`：模板参数、对齐约定、生产者/消费者两侧的 swizzle 地址算术）、common.cuh L262–350（本仓库自带的 `SwizzleBMS` 位级镜像与 kColStride 映射表）、bench/flash_attn.cu L79–85（`make_tma_copy` 的最小用法）。**编译宏**：`NOTES_V2_ENABLE_CUTE`（CuTe 布局部分）与 `NOTES_V2_ENABLE_TMA_MMA_WS`（TMA 部分，需要 `CUtensorMap` 与 `cuda::barrier`，sm80 及以上可编）。

配套最小测试 `book/tests/ch23_swizzle_tma.cu` 是一个纯主机侧断言程序：它把本章的全部公式（位级 XOR、chunk 置换、复合语义、`tile_to_shape` 的重复规律、SW128 atom 的物理排布）逐条与 CuTe 的布局求值结果做数值对照。这些断言与 sm_120a 上实测的 TMA 硬件行为一致（对照数据见测试 case E 的硬件对照）。

24.2　问题与动机

先看第 十三 章那行 swizzle 在真实工程里的三种「走样」形态。

走样一：语义只存在于调用点。手写版的 swizzle 是一个自由函数，调用点写 `swizzle<64>(row, k)`，函数体里写死「取 bit7–9（3 位），右移 3 位后 XOR 到 bit4–6」。要换 fp8（元素 1 字节、行宽翻倍）或换行宽（64 B/128 B/256 B），就得改函数体内的位域，而改对与否只能靠再跑一遍正确性测试。ch12 的 kernel 之所以只写了一个 `swizzle<>`，是因为它只有一个 smem 布局。

走样二：与其它维度的耦合。上一章的 smem 需要三层结构：$B_M \times B_K$ 的 tile、k_{stage} 个 stage、以及 scale 因子等旁路数据。手写版的地址表达式把这些维度乘在一起（`base + stage * BM * BK + row * BK + swizzle(row, k)`），每加一层就要重新推导一次「谁在 swizzle 内部、谁在外部」。第 十三 章末尾那个 `k_step` 要不要放进 `swizzle<kMmaK>` 参数的问题，本质就是「复合顺序」问题。

走样三：与硬件 swizzle 的一致性。TMA 的 `cp.async.bulk.tensor` 由硬件按 descriptor 里声明的模式（`CU_TENSOR_MAP_SWIZZLE_32B/64B/128B`）自己把 gmem 的数据摆进 smem。此时 smem 里的物理排布由硬件决定，软件只能去对齐它：要么用同一套位公式算地址（本章的 `swizzle<64>` 路径），要么把 swizzle 编码进基址（WGMMA descriptor 的 `base_offset` 字段，见第 十四 章）。一旦两边的模式不一致，结果不是「略微掉精度」，而是整块 tile 错位。这类错误在 hgemm.cuh L1899–1917 的注释里被写成一条硬性约定：TMA 路径的 smem 基址必须 1024 B 对齐，否则 phase 不为零，消费者算出来的地址全错。

本章要证明的是：CuTe 把这三个走样各自收进了一个可检验的对象：`Swizzle<B,M,S>`（位语义）、`composition`（复合语义）、`tile_to_shape`（多级扩展），而 TMA 侧则用 `make_tma_copy` 把「gmem 张量 + smem 布局」一次性编码成硬件 descriptor。理解这三样东西的边界（尤其是它们不能保证什么），比记住 API 名字重要得多。

24.3 数学原理

24.3.1 现象：bank conflict 与两种解法

先把问题摆成一张图。shared memory 被切成 32 个 bank（每个 4 B 宽），一个 warp 的 32 个线程如果同时访问落在同一 bank 的多个地址，硬件只能把它们串行化，这就是 bank conflict。GEMM 的 smem tile 是 8 行 ×64 列的 fp16（一行 $64 \times 2 = 128$ B），`ldmatrix` 让每个线程负责一根 16 B 的行段（8 个 half，横跨 4 个 bank）。按最自然的「行优先、行步长 128 B」摆放，第 r 行第 j 个 16 B 块的起始字节是 $128r + 16j$，它落在 bank

$$\text{bank}(r, j) = \frac{128r + 16j}{4} \bmod 32 = (32r + 4j) \bmod 32 = 4j \bmod 32, \tag{24.1}$$

行号 r 被 $32r$ 项消掉了：同一个列块 j 在 8 行里全部落在相同的 4 个 bank 上，8 个线程排队等待：**8-way conflict**。

朴素解法：padding。每行尾部补 8 个 half（行宽从 64 变 72 个元素，即 144 B），行步长不再是 128 的倍数，r 重新参与取模，冲突消失。代价有四条：12.5% 的容量被浪费；行宽不再是「2^{M+S} 字节」的标准形态；后面会看到，TMA 硬件只认标准形态，padding 方案无法用于 TMA；每次换元素类型或 B_K 都要重新推一遍补偿量。

置换解法：XOR。保持行宽不动，把*列块索引*换成「块索引 XOR 行号」：第 r 行的逻辑块 j 搬到物理位置 $j \oplus r$。固定列块扫 8 行，物理块取遍 0..7 的 8 个不同值，8 根 16 B 段各落一处 bank 组，冲突消失，容量零浪费。@reed《cute 之 Swizzle》正是从「逻辑矩阵位置与 bank 的对应关系」出发讲这件事的，并给出其硬件依据：`ldmatrix` 的地址通道与 TMA 都原生支持这种置换，因为它的代价只有一次 XOR。成因与两种解法的三面板对照见图 24.1。

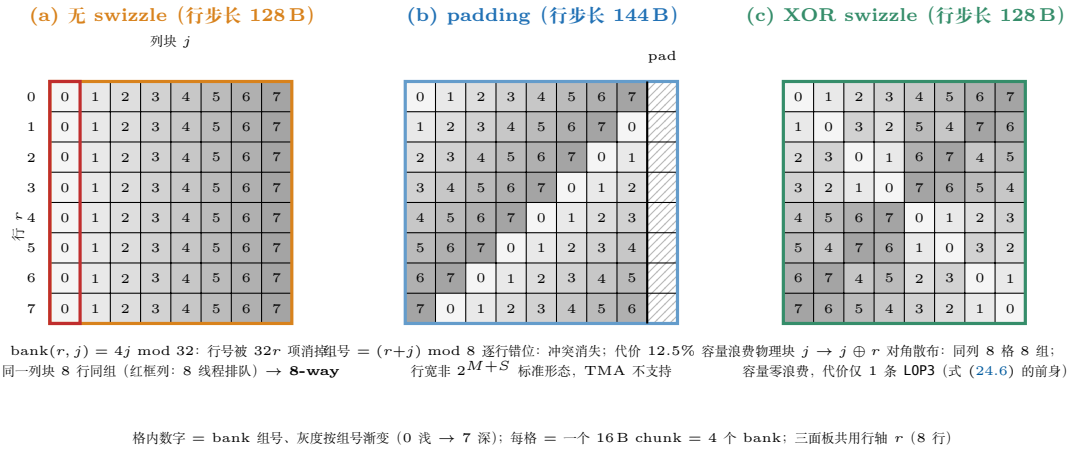


图 24.1：bank conflict 成因与两种解法（式 (24.1)）：8 行 × 8 chunk 的 fp16 smem tile，每格为一个 16 B chunk（占 4 个 bank），格内数字为 bank 组号、灰度按组号渐变。(a) 无 swizzle：bank$(r, j) = 4j \bmod 32$——行号被 $32r$ 项消掉，同一列块 8 行同组（红框列 8 线程串行化）→ 8-way conflict；(b) padding：行宽 144 B 使 r 重新参与取模（组号 $= (r + j) \bmod 8$ 逐行错位，冲突消失），代价是 12.5% 容量浪费且行宽非 2^{M+S} 标准形态——无法用于 TMA；(c) XOR swizzle：物理块 $j \to j \oplus r$ 对角散布，同列 8 格 8 组，容量零浪费、代价仅 1 条 `LOP3`。

剩下的问题是把这个技巧写成可复用、可组合的类型。这是 `Swizzle<B,M,S>` 的职责，也是 3.2–3.6 节的主题。

24.3.2 Swizzle 的位模型

CuTe 的 `Swizzle<B,M,S>` 只做一件事：对一个整数做位域置换。把上面的 XOR 一般化：取 offset 中一段连续 B 位（**源字段**），右移 S 位后 XOR 到另一段连续 B 位（**目标字段**）上，两段之间的 M 位（块内偏移）原样保留。`cute/swizzle.hpp` 给出的位图是

$$\underbrace{\texttt{xxxxxxx}\cdots\texttt{x}}_{\text{高位}}\underbrace{\texttt{YYY}}_{\text{源}}\underbrace{\texttt{xxxxxxx}}_{\text{不变}}\underbrace{\texttt{ZZZ}}_{\text{目标}}\underbrace{\texttt{xxxx}}_{\text{M 位}} \tag{24.2}$$

其中 `ZZZ` 落在 bit $[M, M+B)$，`YYY` 落在 bit $[M+S, M+S+B)$（$S > 0$ 时）。求值是「把 `YYY` 段右移到 `ZZZ` 段再做 XOR」：

$$\text{yyy_msk} = ((1 \ll B) - 1) \ll (M + \max(0, S)), \tag{24.3}$$

$$\text{zzz_msk} = ((1 \ll B) - 1) \ll (M - \min(0, S)), \tag{24.4}$$

$$\text{apply}(\text{off}) = \text{off} \oplus (\text{off} \wedge \text{yyy_msk}) \gg S. \tag{24.5}$$

这个定义有三个立刻可用的性质，它们是后面一切讨论的基础。

(a) 对合性。设 $y = \text{off} \wedge \text{yyy_msk}$。式 (24.5) 只改 ZZZ 段，YYY 段原封不动，所以第二次求值时 y 不变、$y \gg S$ 不变，XOR 两次即还原：$\text{apply}(\text{apply}(x)) = x$。Swizzle 是置换群里的对合元，这是它能当布局用的前提。

(b) 实现代价。式 (24.5) 的算术量是「1 次 AND、1 次移位、1 次 XOR」，映射到 SASS 就是一条 LOP3 或两条整数指令。相对之下，任何「查表 + 分支」的地址重排方案都会引入 warp 内不一致的延迟。硬件能把 swizzle 做进 TMA 与 ldmatrix 的地址通道，正是因为它只有位运算。

(c) 约束 $|S| \geq B$。CuTe 在构造时会静态断言 `abs(SShift) >= BBits`（common.cuh L291 的镜像实现抄了同一条）。若 $|S| < B$，YYY 与 ZZZ 在位上重叠，XOR 会把源字段自己改掉，对合性立刻失效，「读写用同一公式」这一工程前提也就没了。

表 24.1: 三个标准档位与 TMA 硬件模式、第 十三 章手写版的对应关系（周期 $= 2^{M+S+B}$ 字节，2^M 字节 $= 16\,\text{B} = 8$ 个 fp16；末行 `<3,3,3>` 为元素口径，周期 512 个元素折算 1024 B）。

B	M	S	周期（字节）	TMA descriptor 模式	第 十三 章手写版
1	4	3	256	SWIZZLE_32B	swizzle<16>
2	4	3	512	SWIZZLE_64B	swizzle<32>
3	4	3	1024	SWIZZLE_128B	swizzle<64>
3	3	3	1024	无对应模式	无

24.3.3　从位模型到「chunk XOR 行号」

表 24.1 最后一个档位 Swizzle<3,3,3> 值得单独说：它在 CuTe 文档里最常出现（因为文档的例子用 fp32），但它与 SWIZZLE_128B **不是同一件事**，区别在「偏移量的单位」。若偏移量是*元素索引*且元素为 fp16，那么 $M = 3$ 意味着「低 3 位不变」= 每 8 个元素（= 16 B）一组的块内偏移不变；$M = 4$ 则意味着低 4 位不变 = 每 $2^4 = 16$ 个*元素*（= 32 B，fp16 下过于粗）不变。硬件模式的定义永远落在**字节**上：三档的 XOR 粒度统一是 16 B（$M = 4$ 决定），差别在 XOR 位数（$B = 1/2/3$）与行宽（32/64/128 B）。于是

- 偏移量按**元素**计（fp16）：SW128 = Swizzle<3,3,3>；
- 偏移量按**字节**计：SW128 = Swizzle<3,4,3>。

两者的位公式是同一个式子在不同单位下的写法。把它写成「行宽 2^{M+S} 字节、块宽 2^M 字节」的一般形式（R 为行索引）：

$$\text{phys} = \text{off} \oplus \big((R \wedge (2^B - 1)) \ll M\big), \qquad \text{即} \quad \text{chunk}' = \text{chunk} \oplus R. \tag{24.6}$$

式 (24.6) 是本章最有用的一条：**凡是行宽恰好等于 2^{M+S} 字节的布局，swizzle 的物理效果就是「16 B 块的列号 XOR 行号」**。它把位运算的抽象还原成第 十三 章那张「对角线置换」的图。验证方式也很直接：把 GMMA::Layout_K_SW128_Atom<half> 铺到一个 64×64 的 fp16 tile 上，逐点比较 layout(r,c) 与 $64r + 8 \cdot ((c/8) \oplus (r \bmod 8)) + (c \bmod 8)$，二者必须逐点相等（第 24.9 节的 case C 与 case E）。

实践须知　（参数与单位绑定，抄参数前先确认单位）

- **同一个 Swizzle 类型在不同单位下对应不同的物理块宽**。CuTe 文档与许多教程演示 Swizzle<3,3,3> 用的位公式是 $\text{offset}' = \text{offset} \oplus ((\text{offset} \wedge (\texttt{0b111} \ll 6)) \gg 3)$，掩码落在 bit $[6, 9)$，这是把偏移量按字节读的口径，物

理上是「8 B 块、64 B 行」的置换。换到 fp16 的 SW128：块宽必须是 16 B = 8 个元素，于是偏移量按元素计时 `Swizzle<3,3,3>`（hgemm.cuh 用的就是它）、按字节计时 `Swizzle<3,4,3>`（CUTLASS 的 GMMA 原子用的是后者）。

- **行宽必须与参数自洽**。式 (24.6) 的前提是行宽 $= 2^{M+S}$ 字节：fp16 的 64 列恰是 128 B，对应 SW128 的 `<3,4,3>`（字节口径）；若 tile 的行宽只有 64 B（fp16 的 32 列），直接套用 SW128 参数只能得到强度减半的置换，见 4.1 节的实测。

24.3.4 composition：Swizzle 与基础布局的合成

CuTe 用 `composition` 把「Swizzle」与「基础布局」合成一个整体：

$$R = \texttt{comp}\big(\text{Swizzle},\ \text{Offset},\ \text{Layout}\big), \qquad R(c) = \text{Swizzle}\Big(\text{Offset} + \text{Layout}(c)\Big). \tag{24.7}$$

注意式 (24.7) 的自变量是**坐标**，而 Swizzle 作用在**偏移量**上，中间隔着一次 `Layout` 求值：**坐标体系不用变，物理地址已经换了，之后的代码无需改动**。仍用 (r, c) 问「第 r 行第 c 列在哪」，布局会换算成被置换过的物理 offset；读写两侧只要用同一个布局别名，改写永远自洽。hgemm.cuh 的 smem 原子就是这一步的实例：

hgemm.cuh L1244-1266（swizzle 语义注释 + SmemLayoutAtom 与布局三件套）

```
// 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
//   - M=3: 连续 2^3=8 个 half 组成一个基本元素，即 16 bytes;
//   - S=3: 连续 2^3=8 个基本元素组成一行，即 128 bytes;
//   - B=3: 共有 2^3=8 行。
// 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B，覆盖全部 32 个 shared-memory bank。
// Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
// 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
// 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
// 地址位 [6:8] XOR 到 [3:5]，地址位 [0:2] 不参与置换。
// composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
// 逻辑坐标仍由 base_layout 给出，只有最终的 shared-memory offset 被重排。
// tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
// 使结果 shape 匹配 BM×BK×kStage；默认按目标 shape 的 mode order 重复各维。
using SmemLayoutAtom = decltype(composition(
    Swizzle<3, 3, 3>{},
    make_layout(make_shape(Int<8>{}, Int<BK>{}),
                make_stride(Int<BK>{}, Int<1>{}))));
using SmemLayoutA = decltype(tile_to_shape(
    SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
using SmemLayoutB = decltype(tile_to_shape(
    SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));

// —— MMA (TiledMMA) ——
```

这段注释里有两处要带着式 (24.6) 去读：一是它把 $S = 3$ 解释成「$2^3 = 8$ 个基本元素组成一行，即 128 bytes」，二是它声称「8×32 的逻辑 tile 正好对应 8 行 × 128 B」。结论（chunk XOR 行号）没错，但它描述的行宽与紧随其后的 base layout `(8,32):(32,1)` 并不一致：32 个 fp16 只有 64 B。这不是笔误级的细节：**行宽不足会让 swizzle 的实际强度降低**，4.1 节的实践须知给出实测。

24.3.5 位置无关与位置相关：最容易出错的一处

Swizzle 是「地址的函数」，所以必须先回答：这个地址从哪来？CuTe 给了两条路径。`cute/pointer_swizzle.hpp` 的 L44–46 注释把区别写得很清楚：

```
as the offset. That means that swizzle_layout will implement
position-independent swizzle layouts, while swizzle_ptr implements
position-dependent swizzle tensors. Arch chose to design hardware with
position-dependent swizzles.
```

- **位置无关（layout）**：$R(c) = \text{Swizzle}(\text{Layout}(c))$，Swizzle 只看布局内部的偏移量，与这段内存在 smem 里的绝对位置无关。`layout(c)` 是纯函数，可在主机侧求值、可参与类型推导。
- **位置相关（pointer）**：Swizzle 作用在**绝对地址**上（`apply_swizzle` 内部是 `reinterpret_cast<uintptr_t>`），所以同一条逻辑访问，在不同的 smem 基址下会落到不同的物理位置。

硬件选的是后者。这解释了 hgemm.cuh L1899–1917 那条 1024 B 对齐的硬约定：若基址低 10 位为零，`phase` 为零，位置相关与位置无关两种视角给出相同的地址；若基址只有 128 B 对齐，硬件写进去的 phase 与软件算出来的 phase 不一致，读出来就是错的。本章测试里有一条**刻意**分开的对照（case E）：把 `Swizzle<3,4,3>` 作用在字节偏移上得到的模式，与 sw_120a 上实测 TMA `SWIZZLE_128B` 的 4096 个元素逐点相等（0 失配）；而用 `tile_to_shape` 得到的分块布局求值去对同一块 smem 时，4096 点里只有 1024 点（25%）能对上。这 25% **不是**「位置无关 vs 位置相关」的语义分界，而是**单位错配**：裸求值把字节口径的 `Swizzle<3,4,3>` 直接作用在元素偏移上（缺了 fp16 元素到字节的 $\times 2$ recast），`YYY` 取到的是元素偏移的 bit$[7,10)$ 而非字节偏移的 bit$[7,10)$，仅当行号满足 $r \bmod 16 \in \{0,2,4,6\}$（与列无关，占 1/4 的行）时，两种口径取到的 XOR 位段相同、结果相等，1024/4096 正是这条规则的命中率（数值模拟 64 行 $\times$ 64 列逐点核对）。补上 recast、按正确字节口径求值，分块布局同样 4096 点全部命中。基址对齐为什么是硬约束见图 24.2。

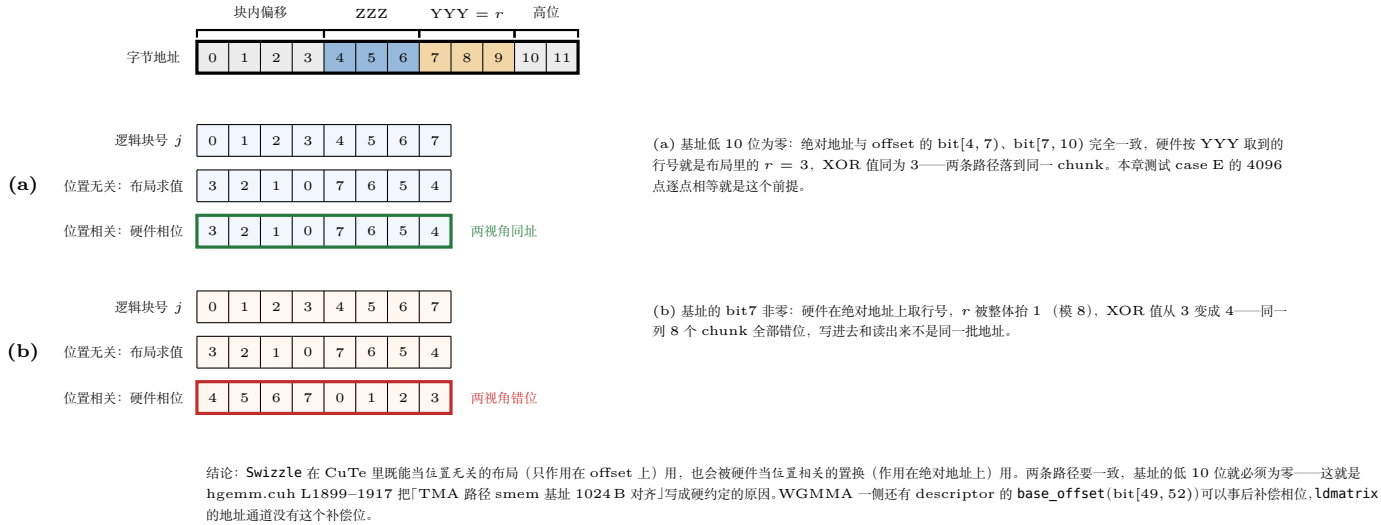


图 24.2: 位置无关与位置相关：为什么 TMA 路径要求 smem 基址 1024 B 对齐（3.5 节）。上方把字节地址的位段摆开：bit$[4,7)$ 的 ZZZ 是 XOR 目标（16 B 块号），bit$[7,10)$ 的 YYY 是源（行号 r）。(a) 基址低 10 位为零时，绝对地址与 offset 的这两个字段一致，布局求值与硬件相位给出同一个物理 chunk；(b) 基址一旦带 bit7，硬件看到的行号整体抬 1，XOR 值随之改变，写/读两条路径错位。

24.3.6　tile_to_shape：从原子到多级 smem 布局

最后一个环节是把 $8 \times B_K$ 的原子扩到 $B_M \times B_K \times k_{\text{stage}}$：

$$\texttt{tile_to_shape}(\text{atom}, (B_M, B_K, k_{\text{stage}})) = \text{blocked product}\Big(\text{atom},\ \text{repeat counts}\Big). \tag{24.8}$$

它有两个必须记住的行为。

(1) 整除约束。 `tile_to_shape` 要求目标形状能被原子形状整除，否则运行时断言（`block shape does not divide the target shape`）。这条约束是「原子必须重排成扫描器可预测的阶梯」的直接后果。于是 $B_K = 32$ 的 tile 不可能直接套 8×64 的 SW128 原子；反过来，8×32 的原子能铺 $B_K = 32$，却也不是 SW128 的标准形态。

(2) 选择性传递 swizzle。 布局代数在重复原子时，会把原子的 swizzle 部分原样搬过去，并把基础布局的宽度轴步长置零（stride 0），因为该轴的偏移已经由 swizzle 字段承载。实测 `tile_to_shape(SW128_atom, (64,64))` 的类型打印为 `Sw<3,4,3> o ... o ((8,8),(64,1)):((64,512),(1,0))`。第二个 mode 的 stride 为 0 就是这件事的标志。它带来的工程后果是：**不要拿这个复合布局当 flat 偏移公式去手算地址**（见 3.5 节的语义分界），要读 smem 就直接用 `make_tensor` 构造张量。CuTe 甚至把这条约束做进了类型系统：带 swizzle 的 `ComposedLayout` 上

`stride()` 是 *deleted* 的（strides 不再与位置无关），只能分别取 `layout_a()`（swizzle）与 `layout_b()`（基础布局）。本章测试的 case G 用 C++20 concept 在编译期检查这条 API 边界。

24.4 设计与实现

24.4.1 hgemm 的三个 smem 布局别名

Phase 7c 的 CuTe kernel 只用了三个布局别名（见式 (24.7) 引出的那段代码：`SmemLayoutAtom` 与由它派生的 `SmemLayoutA/B`）：

逐项读：`composition(Swizzle<3,3,3>, base)` 里 base 是 $(8,32):(32,1)$，$T=$ `half`，所以偏移量以元素计，$M=3$ 对应 16 B 块；`tile_to_shape` 把原子沿 B_M (128)、B_K (32) 与 k_{stage} 三个方向重复，得到 A 用的 $(B_M,B_K,2)$ 与 B 用的 $(B_N,B_K,2)$。cosize 分别是 $128\times32\times2=8192$ 与 $256\times32\times2=16384$ 个 half。这两个别名随后被 `make_tensor(make_smem_ptr(...), SmemLayoutA{})` 变成真正的 `Tensor`，从此所有地址算术都由 CuTe 完成，包括 `ldmatrix` 的每 lane 地址。

实践须知 （两处与注释核对过的实测）

- 8×32 **原子的实际强度只有一半**。把 $x=32r+c$（元素偏移）代入 `Swizzle<3,3,3>` 的位公式：掩码落在 bit $[6,9)$，而 $32r$ 只贡献到 bit 5 起的位：**行号的最低位落在源字段之外**，真正参与 XOR 的只有 r 的高两位，等价于 $\text{chunk}'=\text{chunk}\oplus(r\gg1)$：XOR 值集合只有 $\{0,8,16,24\}$ 四种、2 行一循环（理想 SW128 是 8 行一循环，强度减半）。注释宣称的「8 行 × 128 B、覆盖 32 bank」描述的是 `Swizzle<3,3,3>` 配 $B_K=64$ 的形态。**该差异不影响正确性**，读写两侧用的是同一个布局别名，只是 bank 收益比注释口径小一半；本章测试 case F 逐点验证了这个退化形态（图 24.3）。
- **换行宽时的参数映射，看 common.cuh 的镜像实现**。common.cuh 的 `SwizzleBMS<B,M,S>` 是 `cute::Swizzle` 位公式的镜像，其 `kColStride` 映射给出工程速查口径：列步长 16/32/64（fp16 元素）分别对应 $(B,M,S)=(1,4,3)/(2,4,3)/(3,4,3)$，即 TMA 的 32/64/128 B 三档（该实现对 16/32/64 与手写版 bit-exact 一致，host 端有等价性测试）；legacy 的 `kColStride`=8 无对应的 CuTe 标准模式，代码显式回退到手写 XOR 路径。**行宽一改，B 与 M 要一起改**，这就是 3.3 节实践须知「参数与单位绑定」的具体体现。

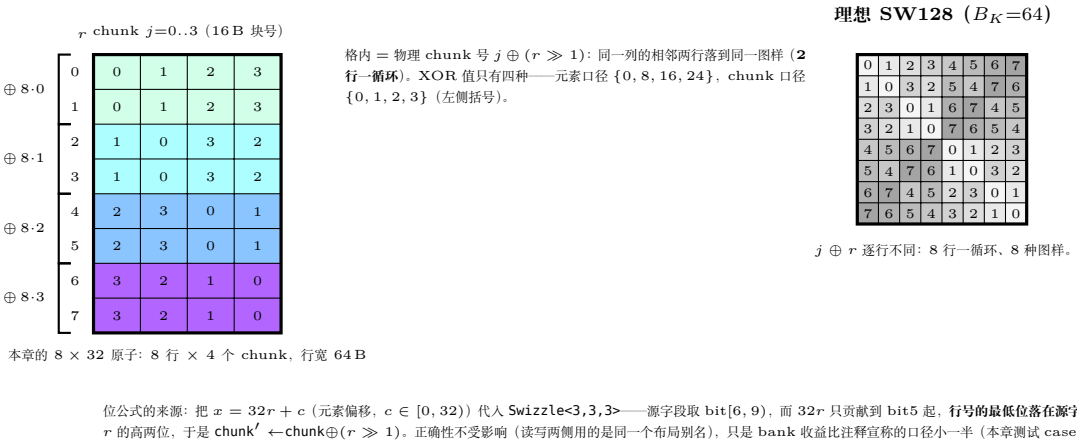


图 24.3: hgemm 的 8×32 原子退化 swizzle（4.1 节实践须知，测试 case F）：$B_K=32$、行宽 64 B 时把一个 8×32 的逻辑 tile 交给 `Swizzle<3,3,3>`，行号最低位落在源字段 bit$[6,9)$ 之外，XOR 只用到 r 的高两位——物理 chunk 号退化为 $j\oplus(r\gg1)$，XOR 值集合从理想 SW128 的 8 种降到 4 种、周期从 8 行缩到 2 行（右：理想 SW128 的 8×8 chunk 表作对照）。

24.4.2 为什么不能直接套 `GMMA::Layout_K_SW128_Atom`

最自然的想法是直接使用 CuTe 现成的 SW128 原子。实测 `GMMA::Layout_K_SW128_Atom<half_t>` 的类型是

```
Sw<3,4,3> o smem_ptr[16b](unset) o (_8,_64):(_64,_1)      size=cosize=512
```

即 8×64、每行 128 B、cosize 512。把它铺到 $B_K = 32$ 的 tile 上会遇到 3.6 节的整除约束（64 不整除 32）；而如果为了迁就它把 B_K 改成 64，K 方向的循环步长、后续 `k_step` 的切分与寄存器预算都会跟着变。Phase 7c 选择的是「自己写原子」：8×32 的 base layout 加 `Swizzle<3,3,3>`，牺牲 swizzle 的强度（见 4.1 节实践须知）换取与 $B_K = 32$、$k_{\text{stage}} = 2$ 的既有流水线完全兼容。这是一次典型的工程折中：布局要服从流水线的形状，而不是反过来。

24.4.3　TMA 的两条构造路径

先看它替代了谁的工作。第 十四 章的 cp.async 是**逐线程**发起的：一个 128×64 的 fp16 tile 有 16 KB，按每线程 16 B 摊派，256 个线程每人要发 4 条 `cp.async`，每条都得自己算 gmem 地址与 smem 落点；搬完还要 `cp.async.wait` + `__syncthreads` 把全 CTA 对齐。线程承担数据搬运，与计算分占同一份地址算术预算。

TMA（`cp.async.bulk.tensor`）把这套流水线换成**一条指令搬一个 tile**：第一个参数不是地址，而是一个 128 B 的描述符 `CUtensorMap`，它把「gmem 张量怎么摆、搬多少、smem 里怎么 swizzle」全部编码进去，之后每次搬运只需要给坐标，**生产者退化成 1 个线程**（发指令 + 通知 barrier），地址算术与越界填充全交给硬件（4.4 节展开）。仓库里有两条构造路径。

路径一（原始）： common.cuh 的 `create_tensor_map` 直接调 driver API `cuTensorMapEncodeTiled`，kernel 签名里出现 `const CUtensorMap*`（hgemm.cuh L1874–1878）：

```
__global__ void __launch_bounds__(kNumThreads)
    hgemm_tma_mma_ws_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB)
```

紧跟其后的 `static_assert(BM == 128 && BN == 128 && BK == 64, "...")` 本身就给出一条约束：**描述符里的 swizzle 模式、CTA tile 形状与 smem 布局三者必须自洽**。该 kernel 的 K 方向 tile 是 64 个 half = 128 B，正好等于 `SWIZZLE_128B` 的一行；swizzle 档位在 descriptor 里的编码位置见下方实践须知（control word 的 bit[13, 15)）。

实践须知　（descriptor 里两个不显式的字段）

- **control word 的 swizzle 档位。** 128 B 描述符的前 8 字节是 globalAddress，紧随其后的 4 字节 control word 压缩了全部模式配置：bit [13, 15) 是 swizzle 档位（0/1/2/3 = none/32/64/128 B），bit [15] 是越界填充值（zero/NaN）。想调试 descriptor，先把这两处 dump 出来最快。
- **第 21 bit 的 OOB 开关。** @reed《cute 之 TMA Descriptor 编码与隐藏的第 21bit》报告：driver 580 起，control word 的 bit [21] 会在「模板 tensor 总大小 ≥ 128 KB」时被置 1，硬件随即跳过越界检查、直接按坐标算地址；而 `tensormap.replace` 动态改 shape 时不会联动更新这一位。于是「创建大模板、运行时缩 shape」这条常规流程可能产出「dim 很小但 bit21 = 1」的描述符，OOB tile 落到未映射页时报 illegal memory access。规避法：创建后手动清 bit21，或在每次 replace 后按新 shape 重设。这不是 swizzle 机制的问题，而是「描述符是黑盒」的典型代价。用到动态 shape 时值得重新检查一遍 control word。

路径二（CuTe）： `make_tma_copy` 把 gmem 张量、smem 布局原子、CTA tile 与 cluster 大小一起收进去，返回一个 `TiledCopy`。bench/flash_attn.cu 的最小用法是

bench/flash_attn.cu L79-85（make_tma_copy：gmem 张量 + smem 原子 + CTA tile）

```
auto mQ = make_tensor(
    make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_input)),
    make_shape(kRows, Int<kHeadDim>{}),
    make_stride(Int<kHeadDim>{}, _1{}));
auto tma_q = make_tma_copy(
    SM90_TMA_LOAD{}, mQ, SmemLayout{},
    Shape<_64, Int<kHeadDim>>{}, _1{});
```

注意第一个实参是 `SM90_TMA_LOAD{}`，CuTe 把 TMA 建模成一种 *copy op*，于是它能和 `copy()` 一样被 `partition_S/partition_D` 切分，kernel 侧写的是 `auto gA = tma.get_tma_tensor(shape)` 与 `auto tAgA = tma.get_slice(_0{}).partition_S(gA)`，地址算术一次都不用手写。代价是**布局必须能被 TMA 的表达能力表示**：descriptor 支持的是「多维 box + stride + 一种 swizzle 模式」，CuTe 会用 `get_tma_swizzle_base` 检查 smem 布局的 swizzle 是否落在受支持的模式里。第 3.3 节实践须知里那两套口径（元素空间的 `<3,3,3>` 与字节空间的 `<3,4,3>`）在这里变成一条 `static_assert`：TMA 只接受字节口径的参数（$M = 4$，因为模式名称以 16 B 为最小粒度）。

24.4.4 生产者与消费者：两套地址算术的对齐

TMA 版 kernel 里，数据的写入由**硬件**完成，读取由**软件**完成，两者必须约定同一套 swizzle。生产者一侧短到可以整段引在这里（hgemm.cuh L1947–1961，去缩进后的骨架）：只有 `empty[stage].wait(...)`（这一格被消费者腾空了吗）、两次 `tma_load_2d(&s_a[stage*BM*BK], tensorMapA, k*BK, by*BM, full[stage])` 与 `tma_arrive_expect_tx(full[stage], (BM*BK + BN*BK)*sizeof(half))`。它把这轮要到的字节数（式 (24.9)）报给 barrier，硬件搬完即满足事务计数，消费者那边的 `full[stage].wait()` 才可能返回。消费者一侧则必须自己算出带 swizzle 的物理地址：

hgemm.cuh L2021-2034（消费者：k_step 必须放进 swizzle 内部的原因）

```
        const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
        // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
        // lane_smem_a_k 中传给 swizzle<64>(), 而非像 swizzle kernel
        // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
        //   swizzle<64> 作用于完整 BK=64, swizzle 周期覆盖全部 64 列,
        //   k_step=0 和 k_step=1 的 chunk 会被 XOR 交叉混合 → k_step
        //   偏移必须在 swizzle 内部参与 chunk 计算。
        //   swizzle<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
        //   swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
        const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
        const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
            (stage * BM * BK + lane_smem_a_m * BK +
            swizzle<64>(lane_smem_a_m, lane_smem_a_k)) *
                sizeof(half);
```

注释里的对比是全章技术含量最高的一段：`swizzle<64>` 的作用域是完整 $B_K = 64$ 列（128 B 行宽；其重复周期为 1024 B = 8 行），所以 $k_{\text{step}} \cdot k_{\text{MmaK}}$ 必须作为「列坐标」的一部分进入 swizzle 内部；而 `swizzle<kMmaK>`（$k_{\text{MmaK}} = 16$，即 `SWIZZLE_32B`）的跨度只有 16 列，每个 slice 内部自洽，偏移可以加在 swizzle 外部。这条规则由式 (24.6) 直接推出：XOR 是否跨越 slice 边界，取决于 slice 宽度是否整除 swizzle 周期。

24.4.5 对齐与 phase

TMA 路径最后一条工程约束写在 kernel 的开头（hgemm.cuh L1903–1920）：注释先问「为什么 `hgemm_wgmma_stages_tn` 只需要 128 B 对齐，而本 kernel 必须 1024 B 对齐」，然后给出结论：`extern __shared__` 声明上必须写 `__align__(1024)`，类型是 `uint8_t`，名字是 `smem_tma_mma_ws`。

这段注释解释了一个常被误以为没有规律的事实：**同样是 TMA + swizzle，WGMMA 的 smem 基址只要 128 B 对齐，而 ldmatrix 路径必须 1024 B 对齐**。原因不在 TMA，而在消费者：WGMMA 走的是 64-bit smem descriptor，里面 bit[49,52) 的 `base_offset` 字段记录了基址相对 1024 B 边界的偏移（$(\text{addr} \gg 7) \wedge 7$），硬件读 smem 时会自动补偿 phase；`ldmatrix` 只有软件公式，没有补偿通道，只能要求 phase 恰好为零。

实践须知 （两个容易被模式名误解的量）

- 「**128 B**」**是跨度，不是周期**。`SWIZZLE_128B` 里的 128 B 是「一次置换覆盖的行宽」；swizzle 的重复周期要大得多：字节口径 2^{M+S+B} 字节（改 bit[4, 7)、读 bit[7, 10)），fp16 的 SW128 是 512 个元素 = 1024 B（8 行），与行宽 128 B 相差 8 倍。讨论「多少行一循环」「多大对齐才够」时用的是后者。

- **1024 对齐的来源是「相位为零」。**把 `Swizzle<3,4,3>` 作用在字节偏移 $[0, 1024)$ 上，软件公式算出的排布只有在基址为 1024 的倍数时才与硬件写出的排布逐点一致，这正是 `__align__(1024)` 的来历（本章测试 case E：字节空间公式 4096 点全部命中；分块布局的裸求值只剩 1024 点命中，差的不是相位，而是 fp16 元素到字节的 ×2 recast，见 3.5 节）。

24.5　图示

图 24.4 把式 (24.6) 画成一张排布图：SW128 原子（8×64 的 fp16 = 8 行 × 128 B）中，行长 128 B = 8 个 16 B chunk，第 r 行的第 j 个逻辑 chunk 落到物理位置 $j \oplus r$。第 十三 章的「对角线置换」在这里有了精确的位级解释，也是 `swizzle<64>` 与 `Swizzle<3,4,3>` 指同一件事的证据。

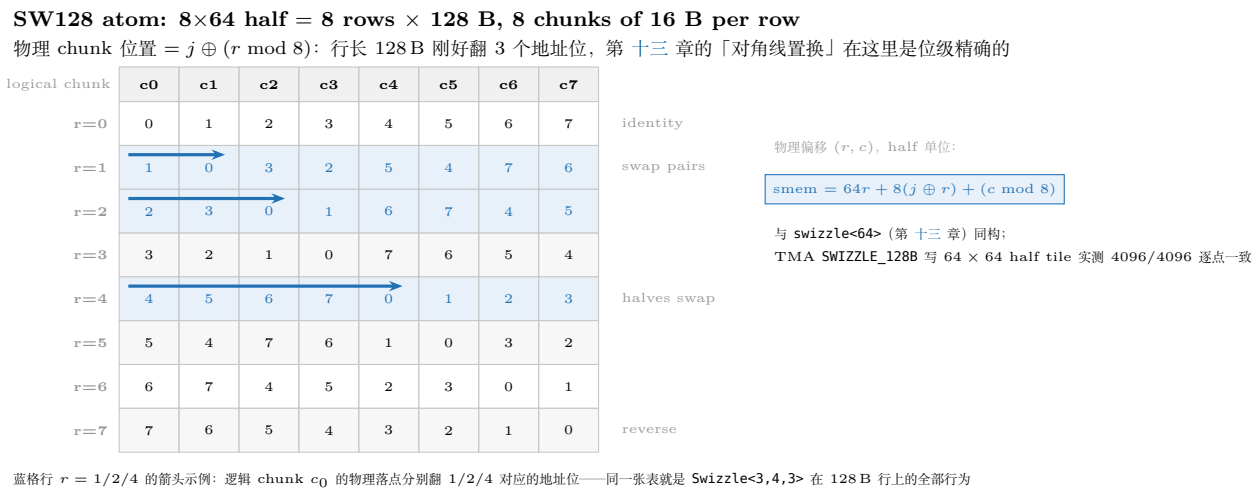


图 24.4：SW128 原子的物理排布：16 B 块的列号 XOR 行号。横轴为 8 个逻辑 chunk（c_0–c_7），纵轴 8 行，格内为该行接受置换后的物理 chunk 下标；蓝底行 $r = 1/2/4$ 给出 chunk c_0 落点翻转 1/2/4 地址位的示例。物理偏移 $= 64r + 8 \cdot (j \oplus r) + (c \bmod 8)$（half 单位）——与 `swizzle<64>`（第 十三 章）同构。

24.6　性能分析

本章没有为 CuTe swizzle 单独建 bench（Phase 7c 的数据由 `bench/bench_leetcuda.cu` 的 `--bench` 统一采集；端到端性能对照见第 十五、二十五 章）。这里只给三条可核验的定量结论：

(1) bank conflict 的消除是「块级」的。一个 16 B chunk 占 4 个 bank（128 B = 32 个 4 B bank）。式 (24.6) 保证同一列块（固定 j）在 8 行里落到 8 个互不相同的物理 chunk，即 8 个 4-bank 组，这正是 `ldmatrix`（每线程取 16 B）所需要的性质。第 24.9 节的 case C 把它写成「固定 chunk、行扫描得 8 个不同值」，case F 则验证 8×32 原子的退化形态（`chunk'` = `chunk` $\oplus$ (row $\gg 1$)，XOR 值只有 $\{0, 8, 16, 24\}$ 四种，2 行一循环）。

(2) swizzle 的代价是零条额外指令。由于它被折叠进地址表达式，SASS 里表现为原有的地址计算多一条 `LOP3`；没有任何一条独立的「重排内存」指令。这也是 TMA 能把 swizzle 做进硬件的前提。

(3) TMA 的收益不在「拷贝更快」，而在「同步与数据管理更简单」。生产者只有 1 个线程发指令（式 (24.9) 的字节数由硬件事务计数保证），256 个线程不再参与 G→S 的地址计算，也不再需要 `__syncthreads`，hgemm.cuh L1990–2012 的注释把这一点总结为「barrier + proxy fence 取代了 CTA 级同步」。对本章而言，「快」是次要的；「谁算地址、谁保证一致」这两个问题的答案变得唯一，才是 TMA 对 kernel 工程的主要贡献。

$$N_{\text{tx}} = (B_M \cdot B_K + B_N \cdot B_K) \times \texttt{sizeof(half)} = (128 \cdot 64 + 128 \cdot 64) \times 2 = 32768 \text{ B}. \tag{24.9}$$

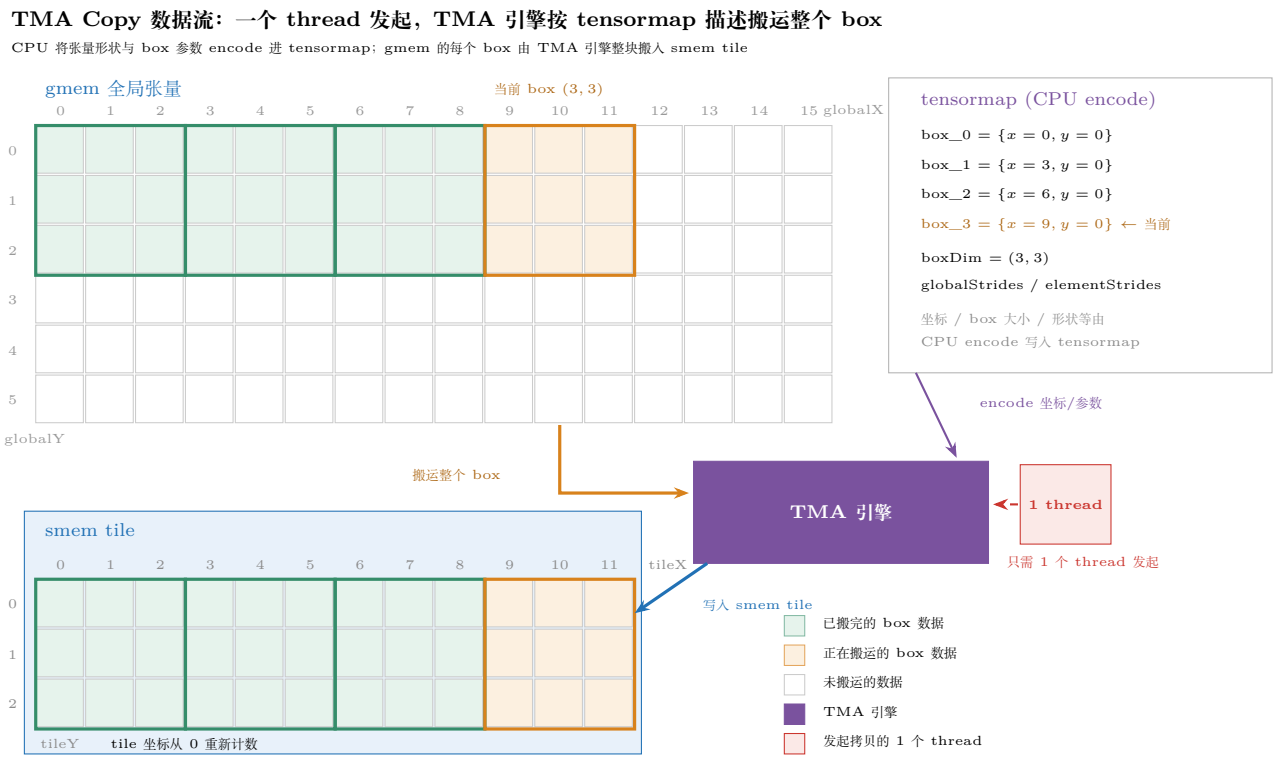


图 24.5: TMA copy 数据流：全局张量按 3×3 box 切分（绿 = 已搬完、橙 = 搬运中），各 box 起始坐标与张量描述由 CPU encode 进 tensormap，TMA 引擎据此定位整块搬运。发起与搬运分离——单线程发起拷贝指令，box 内全部元素由 TMA 引擎异步搬入 smem；数据落地后改用 tile 局部坐标，与全局坐标脱钩。重建自 @ 竹熙佳处《写给大家看的 CuTe 教程：TMA Copy》。

24.7 常见坑

1. **拿复合布局当偏移公式**。带 swizzle 的 `ComposedLayout` 求值是「位置无关」的，TMA 硬件是「位置相关」的：同一个逻辑坐标在两个语义下可以指向不同物理地址。要么全用布局（`make_tensor/Tensor`），要么全用公式（第 十三 章的 `swizzle<>`），**混用必错**。

2. **基址对齐不够**。 TMA smem 基址必须 1024 B 对齐（`__align__(1024)`）；128 B 对齐在 WGMMA 路径够用，在 `ldmatrix` 路径必然错。

3. **把「M」当成元素粒度**。 2^M 是基本块的字节数（$16\,\mathrm{B} = 2^4$），不是元素个数；fp32 里它等于 4 个元素，fp16 里等于 8 个元素。TMA descriptor 只接受 $M = 4$ 的模式。

4. **`tile_to_shape` 不整除**。 8×64 原子铺 $B_K = 32$ 会直接断言失败；要么换原子，要么改 tile 形状。

5. **把 `k_step` 加在 swizzle 外部**。只要 slice 宽度不是 swizzle 周期的整数倍，偏移就必须放进 swizzle 内部（hgemm.cuh L2021–2034 的注释），否则某些 k_{step} 的 chunk 会跨 slice 交叉。

6. **预期 swizzle 能消除所有 bank conflict**。它保证的是 **同一个 16 B 块列在 8 行内落到 8 个不同块**；跨 32 行、或以 4 B 为粒度的模式（例如标量表查）它不保证。强度由 B 与行宽共同决定：本仓库 8×32 原子的实际强度只有 2 bit（4 个 XOR 图样、2 行一循环，4.1 节实践须知）。

7. **忘记 swizzle 是地址函数**。想让「按 `(row,col)` 求值的布局」与「按 flat 偏移求值的公式」一致，前提是行宽恰好等于 2^{M+S} 字节。行宽不匹配时（例如 B_K 取 48），两者不等。

24.8 面试要点速查

Q1：`Swizzle<B,M,S>` 的三个参数各是什么？ M：被保留的连续低位位数（基本块宽 2^M 字节）；B：XOR 掩码位数，同时决定 TMA 模式名（32/64/128 B ↔ 1/2/3）；S：YYY 相对 ZZZ 的位移；仅当行宽恰好为 2^{M+S} 字节时（三

档中只有 SW128 满足，见 Q2）才等价于「一行 2^{M+S} 字节」，工程上恒取 3。约束 $|S| \geq B$。

Q2：为什么 S 恒为 3？三档的 XOR 源字段固定取自 bit 7（128 B 粒度）、目标字段固定取自 bit 4（16 B 块），两者相距 3 位，故字节口径下 S 恒为 3、M 恒为 4（与 dtype 无关）；档间只改 B（1/2/3）与 box 内维（行宽 32/64/128 B）。fp16 元素口径下 $M = 3$（hgemm 自用的 `Swizzle<3,3,3>` 即此口径）。

Q3：为什么不直接写成「列块 XOR 行号」，还要用位运算？因为硬件（TMA、`ldmatrix` 的地址通道、WGMMA 的 descriptor）都只认字节地址上的位域交换；「列块 XOR 行号」在行宽 $= 2^{M+S}$ 时与位运算等价（式 (24.6)），行宽不匹配时就不等价了。

Q4：`composition` 的求值顺序？先算基础布局得偏移，Swizzle 再作用其上：$R(c) = \text{Swizzle}(\text{Layout}(c))$。Swizzle 不参与坐标分解。

Q5：TMA swizzle 与手写 swizzle 的冲突点在哪？在「相位」。硬件按绝对地址算 phase，软件按相对偏移算；只有基址 1024 B 对齐时二者相同。WGMMA 有 `base_offset` 字段可以补偿，`ldmatrix` 没有。

Q6：给定 B_K 与 fp16，怎么判断该用哪种 swizzle？先算行宽 $B_K \times 2$ 字节；可选行宽 32/64/128 B，要求 $B_K \times 2 \geq$ 模式行宽且 16 B 块宽整除行宽。$B_K = 64$ 时行宽 128 B $\Rightarrow$ `Swizzle<3,4,3>`（字节）或 `Swizzle<3,3,3>`（元素）；$B_K = 32$ 时行宽 64 B，只能退到 64 B 或更弱的模式。

24.9　最小测试与动手实验

`book/tests/ch23_swizzle_tma.cu` 是一个**主机侧**断言程序（需要 `NOTES_V2_ENABLE_CUTE`），它把本章的公式逐条与 CuTe 的布局求值结果对照，不启动任何 kernel。七个 case：

1. **case A**：`Swizzle<3,3,3>` 的位级公式：对 $[0, 1024)$ 内每个字节偏移验证 $\text{apply}(\text{off}) = \text{off} \oplus ((\text{off} \wedge (\texttt{0b111} \ll 6)) \gg 3)$，并核对 `yyy_msk`/`zzz_msk` 常量、统计 8 个 `YYY` 值与 8 个 chunk 各被覆盖 128 次；
2. **case B**：对合性 $\text{apply}(\text{apply}(x)) = x$ 与 $[0, 512)$ 上的双射，再用自定义 base layout `(8,8):(64,8)`（64 B 行、8 B 块）验证 $R(r, c) = 64r + 8 \cdot (c \oplus r)$，并检查「固定块列扫 8 行得到 8 个不同物理块、8 个不同 bank」；
3. **case C**：把字节空间的 `Swizzle<3,4,3>`（即 `make_tensor` 后指针路径的位置相关语义）作用在 `GMMA::Layout_K_SW128_Atom<half>` 的 8×64 域上，512 个坐标全部满足式 (24.6)；
4. **case D**：`composition` 语义 $R(c) = \text{Swizzle}(\text{Base}(c))$，两种 base layout 各遍历全 domain（$256 + 256$ 个坐标）；
5. **case E**：硬件对照：sm_120a 实测的 TMA `SWIZZLE_128B` 8×8 chunk 掩码表（64 槽）与 `Swizzle<3,4,3>` 逐槽比较，再由该表导出 4096 点排布逐点比较；同时统计 `tile_to_shape` 分块布局裸求值的命中数（1024/4096），用于说明把字节口径 swizzle 直接套在元素偏移上的单位错配（补上 $\times 2$ recast 后应 4096/4096）；
6. **case F**：8×32 原子的退化形态（4.1 节实践须知）：验证 $\text{chunk}' = \text{chunk} \oplus (\text{row} \gg 1)$，XOR 值集合为 $\{0, 8, 16, 24\}$、每个值覆盖 2 行（$= 64$ 个坐标）；
7. **case G**：TMA 编译级核对：`TmaDescriptor` $=$ `CUtensorMap` 且尺寸为 128 B、三档掩码常量、SW128 atom 的 `layout_a()`/`layout_b()` 类型与其 `shape`/ `stride`/cosize，以及在 host 侧用 `make_tma_copy` 真正构造出一个 descriptor 并核对 (gmem 张量, CTA tile) 的 size/stride 推导。

编译与运行：

```
cd book/tests
./build_tests.sh --arch sm_120a --ch ch23
```

预期输出（2026-09-11 于 RTX PRO 5000 / sm_120a 实机执行，全文）：

```
== ch23 CuTe swizzle / TMA (host-side assertions) ==
PASS case A Swizzle<3,3,3> bit formula: mismatches=0 (1044 checks)
PASS case B involution + bijection + base layout: mismatches=0 (1105 checks)
PASS case C byte-space Sw<3,4,3> == chunk XOR row: mismatches=0 (576 checks)
PASS case D composition R(c) == Swizzle(Base(c)): mismatches=0 (512 checks)
      (E) measured table mismatches=0
      (E) 4096-point mismatches=0, tiled formula 1024/4096
```

```
PASS case E hardware SWIZZLE_128B cross-check: mismatches=0 (3 checks)
PASS case F hgemm 8x32 atom effective swizzle: mismatches=0 (262 checks)
PASS case G TMA compile-level (descriptor): mismatches=0 (15 checks)
ALL OK
```

（`tiled formula 1024/4096` 一行是裸求值（字节口径 swizzle 直接作用在元素偏移上）的口径错配结果，补上 $\times 2$ recast 后为 4096/4096。`build_tests.sh` 在该输出之后还会再打印一次 `ALL OK` 作为汇总，退出码 0。）

动手实验（可选）：把 case F 里的 `Swizzle<3,3,3>` 换成 `Swizzle<3,4,3>`（相当于把 8×32 原子按 128 B 行宽去解释），重跑并打印 XOR 值集合。两次结果的差异正是 4.1/3.3 节实践须知要读者亲手复现的那一步：**同一个 `Swizzle` 类型在不同 base layout 下的物理效果不同，参数必须与行宽一起读**。

延伸阅读

- @reed，*cute 之 Swizzle*（ch12/23 主参考）：https://zhuanlan.zhihu.com/p/671419093
- @reed，*cute 之 TMA Descriptor 编码与隐藏的第 21bit*（descriptor 字段与 swizzle 模式编码）：https://zhuanlan.zhihu.com/p/2037200219700449995
- @ 竹熙佳处，*CuTe 教程：TMA Copy*（`make_tma_copy` 的用法与边界）：https://zhuanlan.zhihu.com/p/2003198909405763007
- @weishengying，*cute swizzle*（可运行的最小例子）：https://zhuanlan.zhihu.com/p/706796240
- @Anonymous，*GEMM 细节分析（三）：Swizzle<B,M,S> 参数取值*（三个档位的参数对照）：https://zhuanlan.zhihu.com/p/713713957

第二十五章　CuTe 应用（一）：HGEMM

25.1　本章导读

第 十二、十三 章用两条不同的路把同一个 HGEMM 写了两遍：一条是「手写 MMA PTX + 手算 smem 地址」的 Phase 7b，另一条是「手写 XOR swizzle + 寄存器双缓冲」的 Phase 7b-3。Phase 7b-3 跑到了 122.9 TFLOPS（2048×16384），Phase 7b 为 76.55 TFLOPS（1024^3），但代价是同一组「形状与步长」的事实被反复重写：`ldmatrix` 的 lane 映射、swizzle 的 XOR 掩码、epilogue 的 `__shfl_sync` 编排，每一处都要人工保证与其余四处的地址算术一致。本章把同一个 kernel 交给 CuTe 重写一遍，用的是 Phase 7c 的 `hgemm_mma_stages_tn_cute`，并在最后用一张**对照表**（图 25.3）把两条路逐项摆开。

前置章节：第 十二 章（`mma.sync.m16n8k16` 与 `ldmatrix`）、第 十三 章（XOR swizzle 与寄存器双缓冲）、第 二十一–二十四 章（Layout 代数 / Tensor 与 TiledCopy / TiledMMA / Swizzle 与 TMA）。本章是 Part IV 的第一章「应用篇」，只用到前四章已经介绍过的四个抽象：`Layout`、`Tensor`、`TiledCopy`、`TiledMMA`。

阅读路线：本章沿 @reed 的 CuTe GEMM 三部曲《cute 之简单 GEMM 实现》《cute 之 GEMM 流水线》《cute 之高效 GEMM 实现》（链接见文末「延伸阅读」）的叙述顺序，按「先跑通 → 再重叠 → 后提速」三步渐进展开：每一步开头先说明**上一版留下了什么问题**，再给出这一版的代码，以及读者必须自己守住的约束。凡是源码注释与实现不一致、或表述容易被误读的地方，不再集中堆在章末的勘误节，而是就地写成**实践须知**：这类提醒出现在使用现场比事后查表更有用。

源码区间：hgemm.cuh L718–1427：Phase 7c 头注释（L718–778）、宏块 `NOTES_V2_ENABLE_CUTE`（L779–1427）内的 kernel `hgemm_mma_stages_tn_cute`（L821 起）与 launch wrapper `launch_hgemm_mma_stages_tn_cute`（L1216 起）。**编译宏**：`NOTES_V2_ENABLE_CUTE`（需要 CUTLASS 头文件；sm80 及以上均可编）。**架构**：本章 kernel 只用 SM80 时代的 `cp.async` + `mma.sync` + `ldmatrix`，本机 sm_120a 可实测。

配套最小测试 `book/tests/ch24_hgemm_cute.cu`：两组配置（`kAccF32=false`、$B_N = 256$ 与 `kAccF32=true`、$B_N = 128$）对 CPU fp64 参考做正确性判定；本章的实测数字全部来自该测试与 `ptxas -v`，端到端性能对照见第 25.6 节。

25.2　问题与动机

先把第 十二、十三 章的代码量摆出来（只统计代码行，去掉纯注释与空行；统计口径见第 25.9 节）：

- Phase 7b `hgemm_mma_stages_tn`（hgemm.cuh L129–397）：**147 行**；
- Phase 7b-3 `hgemm_mma_stages_tn_swizzle`（hgemm.cuh L434–716）：**231 行**；
- Phase 7c `hgemm_mma_stages_tn_cute`（hgemm.cuh L821–1196）：**118 行**，再加 launch wrapper（L1216–1427）**86 行**。

手写版与 CuTe 版的「行数比」并不悬殊（231 对 204），但**代码的性质变了**。手写版里出现的是地址表达式与内联 PTX：

```
int lane_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM + lane_id % 16;
int lane_smem_a_k = k_step * kMmaK + (lane_id / 16) * 8;
uint32_t lane_smem_a_ptr = smem_a_base_ptr +
    (stage * BM * BK + lane_smem_a_m * BK +
     swizzle<64>(lane_smem_a_m, lane_smem_a_k)) * sizeof(half);
```

```
LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
```

CuTe 版里出现的是**声明**与**分区**：

```
auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{});
auto tAsA = s2r_thr_copy_a.partition_S(sA);
auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA);
cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
           tCrA_view(_, _, k_step_next));
```

前者把「lane 到 smem 地址」的公式写死在 kernel 里，改一个 tile 尺寸就要把公式推倒重来；后者把同一件事表达成「sA 的布局是 SmemLayoutA」「s2r_thr_copy_a 把 sA 按线程切分」两句声明，地址算术由编译器（类型系统）展开。这就是第 二十一–二十四 章铺垫四章的回报：**把布局从代码里提出来，变成一等公民**。

本章要回答三个问题：这 118 行的 kernel 本体究竟做了什么？launch wrapper 的 86 行「类型实例化」在配置什么？以及第三个、也是最重要的问题：**哪些东西 CuTe 替你做了，哪些东西仍然要你自己做**？第三个问题决定了这套工具的能力边界，也是图 25.3 对照表要回答的。

正文按三步渐进展开，三步共用同一份源码事实，区别只在每一步**放弃或引入**什么，每一步的末尾都会点明「这一步留下了什么」，接上后一步的动机。

25.3 数学原理

25.3.1 问题定义：TN 约定与第一个 Tensor

本章沿用第 十二 章的 TN 约定：$C[M,N] = A[M,K] \times B^{\mathsf{T}}[N,K]$，其中 A、B^{T} 都是 row-major（A[m][k]、Bt[n][k]）。写成求和形式：

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k}\, B_{n,k}, \tag{25.1}$$

注意式 (25.1) 里 B 的索引是 (n,k)，这正是 TN 的含义：MMA 指令 mma.sync.aligned.m16n8k16.row.col 要求 A 为 row-major、B 在 GEMM 语义下为 col-major，而 col-major 的 $B[K,N]$ 与 row-major 的 $B^{\mathsf{T}}[N,K]$ 是同一块内存。**这是本章第一个坑**：调用方必须自己把 B 转置后传入（最小测试里由 host 侧转置完成，见第 25.9 节）。

用 CuTe 的话说，「TN」不是一个标志位，而是一句关于步长的声明：

```
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                         make_stride(k, Int<1>{}));  // A 是 (M,K), K 连续
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
                         make_stride(k, Int<1>{}));  // Bt 是 (N,K), K 连续
```

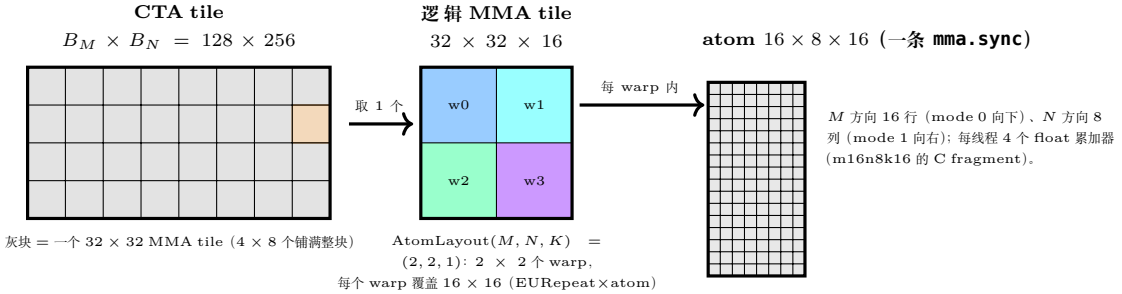
两行 make_tensor 之后，后续所有分区与拷贝都在「$(shape, stride)$ 就是布局」这条约定上工作：K 维连续意味着按 K 切出来的片天然适合做 cp.async 的宽拷贝，而 Int<1>{} 这个编译期常量会让后面每一步的地址算术都在编译期折叠掉。

25.3.2 Tile 金字塔：从 CTA tile 到 MMA atom

一个 CTA 只负责 C 的一小块，整张 C 由网格铺满；K 方向则切开累加（sliced-K 模式，第 十 章）：设 CTA 级 tile 为 $B_M \times B_N \times B_K$，网格为 $\left(\lceil N/B_N \rceil, \lceil M/B_M \rceil\right)$。Phase 7c 的默认配置：

$$B_M = 128, \qquad B_N = \begin{cases} 256, & \text{F16 累加} \\ 128, & \text{F32 累加} \end{cases}, \qquad B_K = 32, \qquad k_{\text{stage}} = 2. \tag{25.2}$$

这三个数字构成一座三层金字塔：CTA tile $B_M \times B_N$（一次搬进来、一次算完）$\supset$ 逻辑 MMA tile $32 \times 32 \times 16$（TiledMMA 一次展开多少条指令，见下节）$\supset$ atom $16 \times 8 \times 16$（一条 mma.sync）。CuTe 的层层 partition，本质就是在这座金字塔上做「从塔基切到塔尖」的坐标换算。三层之间的尺寸关系见图 25.1。



三层尺寸关系：kMmaPM = $\text{AtomLayout}_M(2) \times 16 = 32$、kMmaPN = $\text{AtomLayout}_N(2) \times \text{EURepeat}_N(2) \times 8 = 32$、kMmaPK = $1 \times 16 = 16$；线程数 = 32 lanes × 4 warps = 128；$n_{k\text{-step}} = B_K/\text{kMmaPK} = 2$。CTA tile 一次搬进来、一次算完：smem = $128 \cdot 32 \cdot 2 + 256 \cdot 32 \cdot 2 = 24576$ half = 48 KiB（式 (25.3)）。$B_N = 256$ 只在 F16 累加下成立，F32 累加时掉到 128——$128 \times 256/128 = 256$ 个 float 累加器已超过可用寄存器的一半。

图 25.1: 本章实际配置的三层 tile 金字塔（式 (25.2) 与式 (25.4)）：外层 CTA tile 128×256 一次搬进 smem、一次算完（F16 累加；F32 累加时 B_N 降到 128）；中层逻辑 MMA tile $32 \times 32 \times 16$ 被 AtomLayout$(2, 2, 1)$ 切成 2×2 个 warp（每个 warp 覆盖 16×16，由 EURepeat×atom 堆出）；内层 atom $16 \times 8 \times 16$ 就是一条 `mma.sync`。CuTe 的层层 partition 都是这座金字塔上的「塔基到塔尖」坐标换算。

B_N 随累加精度切换是**经验性**的：F32 累加器每个元素占 4 字节，寄存器压力翻倍，128×256 的 C tile 在 128 线程下是 $128 \cdot 256/128 = 256$ 个 float 累加器，已经超过可用寄存器的一半，会溢出到 local memory。第 25.4.5 节把两种精度的寄存器占用摆开，并用 `ptxas -v` 的实测数字核对（口径讨论见第 25.6 节）。$B_K = 32$ 与 $k_{\text{stage}} = 2$ 一起决定了 smem 占用：

$$\underbrace{128 \cdot 32 \cdot 2}_{\text{A: } B_M B_K k_{\text{stage}}} + \underbrace{256 \cdot 32 \cdot 2}_{\text{B: } B_N B_K k_{\text{stage}}} = 24576 \text{ 个 half} = 48\,\text{KiB}, \tag{25.3}$$

也就是说这个 kernel 落在「一个 CTA 占 48 KiB smem」的档位上（本机 PRO 5000（cc 12.0）实测每 SM smem 上限 102400 B = 100 KiB，一个 48 KiB 的 CTA 每 SM 至多驻留 2 个；占用率还要受寄存器与 block 数约束）。

25.3.3　MMA atom 与 EURepeat

一个 CTA tile 要想算完，先要回答两个问题：一条 `mma.sync` 能算多大（**atom**），以及把多少条这样的指令铺在一个 warpgroup 上（**EURepeat**）。atom 取 SM80 的 `m16n8k16`，A/B 均为 fp16、累加器可选 fp16 或 fp32：

项	F16 累加	F32 累加
PTX 形状	`m16n8k16`	`m16n8k16`
CuTe atom	`SM80_16x8x16_F16F16F16F16_TN`	`SM80_16x8x16_F32F16F16F32_TN`
C 累加器	2 个 `u32`（4 个 half）	4 个 float

两种 atom 的 A/B fragment 布局完全相同（这也是选它们做对照的前提），差别只在 C 累加器：F16 累加把 4 个 half 打包进 2 个寄存器，F32 累加用 4 个寄存器。launch wrapper 用 `std::conditional_t` 一次选出原子，再由 EURepeat 推出逻辑 tile：

$$\texttt{kMmaPM} = 2 \times 16 = 32, \quad \texttt{kMmaPN} = 2 \times 2 \times 8 = 32, \quad \texttt{kMmaPK} = 1 \times 16 = 16, \tag{25.4}$$

即 AtomLayout$(M, N, K) = (2, 2, 1)$（$2 \times 2 = 4$ 个 warp）、EURepeat$(M, N, K) = (1, 2, 1)$（每线程的 atom 重复数，两种语境的区分见第 二十三 章实践须知）、逻辑 MMA tile 为 $32 \times 32 \times 16$，线程数 size(`MMA`) = 32 lanes × 4 warps = 128；这个 128 就是 launch 配置里 `dim3 block(size(MMA{}))` 的来源。对表源码时注意：源码常量 `kMmaEURepeatM/N/K`= 2/2/1（hgemm.cuh L1290–1292）承载的是 `make_tiled_mma` 的第二个参数，即此处的 AtomLayout；CuTe 语义下的真 EURepeat 由 Tile 推出，是 $(1, 2, 1)$。$B_K = 32$ 相对 `kMmaPK`= 16 的倍数即 K 方向的切片数：

$$n_{k\text{-step}} = \frac{B_K}{\texttt{kMmaPK}} = \frac{32}{16} = 2, \tag{25.5}$$

一条 atom 只计算 K 方向的 16 个元素，所以一个 $B_K = 32$ 的 K-tile 要分两次才算完。这就是 kernel 里 `num_k_steps = size<2>(tCrA)` 的来历：它不是一个 magic number，而是「K-mode 的 extent」被 CuTe 从类型里读出来的结果。两个语境下「EURepeat」的区别见图 25.2。

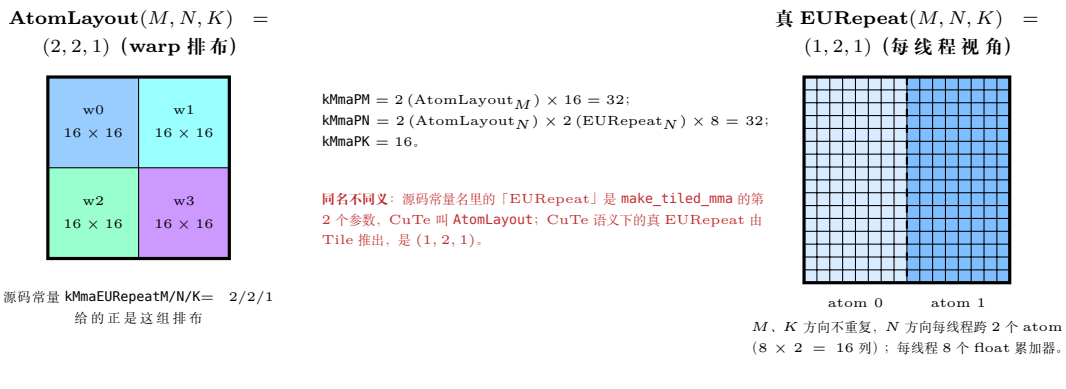


图 25.2: EURepeat 的两种语境（式 (25.4)）：左图是 `make_tiled_mma` 的第 2 个参数，CuTe 语义叫 AtomLayout $(2,2,1)$——2×2 个 warp 每个覆盖 16×16；右图是 CuTe 语义下由 Tile 推出的真 EURepeat $(1,2,1)$——每线程只在自己 warp 内沿 N 方向重复 2 个 atom（$8\times2=16$ 列），M、K 方向都是 1。源码常量名 `kMmaEURepeatM/N/K` 装的是前者，两者相乘才是逻辑 tile $32\times32\times16$。

25.3.4 三件事：数据在哪、谁算哪块、谁搬哪块

到这里，问题被拆成三个互相独立的部分，CuTe 各给一个类型来表达：

- **数据在哪**——`Tensor`：$(ptr, shape, stride)$ 三元组。gmem 上的 A、B、D 各是一个 `Tensor`，smem 上的 A、B 也是，两者的差别只在存储层次（`make_gmem_ptr` 对 `make_smem_ptr`）。
- **谁算哪块**——`TiledMMA`：由 MMA atom 与 EURepeat 组成，一次回答三件事——用哪条指令、CTA tile 被切成多少个 atom、每个 warp/线程负责哪个 atom；它的输出是每个线程的寄存器 fragment 布局。
- **谁搬哪块**——`TiledCopy`：一次搬运的线程-元素映射。G→S、S→R、R→S、S→G 各有一套，彼此独立：改 G→S 的向量宽度不影响 S→R 用什么指令。

三者分层的回报是「改一个维度不必动其它维度」：换 dtype 只换 atom 与 Copy Atom，换 tile 尺寸只改 `Int<>` 常量，换流水深度只改 `kStage`。@reed 在《cute 之简单 GEMM 实现》里把这条路线压成三个动作：把裸指针包成 `Tensor` 并切出 CTA tile、用 `TiledMMA` 把 tile 分到线程、调用 `cute::gemm`；这正是下一节第一步要走的骨架。

但类型系统**不负责**另外两件事，它们构成后面的两步：搬运的时序（什么时候发起 G→S、什么时候才允许覆盖当前 smem 槽，第二步）；以及搬运的形态（smem 怎么排布才不产生 bank conflict、S→R 用哪条指令、epilogue 怎么宽写，第三步）。

25.4 设计与实现

25.4.1 总览：kernel 签名与三步路线

kernel 本体把「全部配置」都收在模板参数里，函数体里没有一个 tile 尺寸是硬编码的（`BM`、`BK` 等出现在 `Int<>` 常量与布局类型里，但来源都是模板参数）：

```
template <typename T,                    // element type (half)
          int BM, int BN, int BK,        // block tile: BM=128, BN=256, BK=32
          int kStage,                    // cp.async pipeline depth (2)
          typename TiledMMA,             // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
          typename G2SCopyA,             // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
          typename G2SCopyB,             // G→S B: same as G2SCopyA
          typename SmemLayoutA,          // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
          typename SmemLayoutB,          // smem B: same atom → (BN,BK,kStage)
          typename SmemLayoutC,          // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
          typename S2RCopyAtomA,         // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
```

```
          typename S2RCopyAtomB,           // S→R B: same as S2RCopyAtomA
          typename R2SCopyAtomC,           // R→S C: UniversalCopy<int> (32-bit store)
          typename S2GCopyAtomC,           // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
          typename S2GCopyC,               // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
          const int BlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
                                         int n, int k) {
  using namespace cute;
  static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
  static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");

  // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
  extern __shared__ T shm_data[];
  T *Ashm = shm_data;
  T *Bshm = shm_data + cute::cosize(SmemLayoutA{});

  int idx = threadIdx.x;
  // BlockSwizzle: 0/1 控制是否启用 thread block swizzle，改善 L2 cache 局部性
  int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  int iy = blockIdx.y; // M 方向 block 索引
  // 当前 kernel 只有 CTA 级边界判断，没有 tile 内的逐元素 predication；调用方需保证
  // M % BM == 0、N % BN == 0、K % BK == 0，才能避免边界 tile 的越界访问或未写回。
  if (iy * BM >= m || ix * BN >= n) return;
```

逐参数对照（面试时按这张表背）：

模板参数	实参	作用
`BM, BN, BK`	128, 256/128, 32	CTA tile 尺寸（式 (25.2)）
`kStage`	2	`cp.async` 流水深度
`TiledMMA`	AtomLayout$(2,2,1)$（4 warp）+ 每线程 repeat$(1,2,1)$	MMA 指令 + 逻辑 tile + 线程排布
`G2SCopyA/B`	`SM80_CP_ASYNC_CACHEGLOBAL`	G→S 的 128-bit 异步拷贝
`SmemLayoutA/B`	`Swizzle<3,3,3>` 原子 + `tile_to_shape`	smem 布局
`SmemLayoutC`	32×32 原子，pipe$=4$	epilogue scratchpad（式 (25.7)）
`S2RCopyAtomA/B`	`SM75_U32x4_LDSM_N`	`ldmatrix.x4` 装载
`R2SCopyAtomC`	`UniversalCopy<int>`	寄存器 →smem 的 32-bit 写
`S2GCopyAtomC/C`	`UniversalCopy<u128>`	smem→gmem 的 128-bit 宽写
`BlockSwizzle`	0 / 1	3D grid 重映射

注意函数体开头的两个 `static_assert`（L824–825）：`BlockSwizzle` 只能是 0/1、`kStage` 至少 2：把「错误配置」变成编译期错误，而不是到运行时才得到错误结果，这是「类型即配置」的直接收益。

下面按三步展开，每一步开头先交代「上一步留下了什么问题」：第一步（第 25.4.2 节）只留「切块 → 分区 → `cute::gemm`」；第二步（第 25.4.3 节）引入 smem 与 `cp.async`，把搬运与计算重叠起来；第三步（第 25.4.4、25.4.5 节）把搬运的形态、寄存器占用与 epilogue 做到高效。三步共用同一份源码，可以逐段对照读。

25.4.2　第一步：切块、分区、`cute::gemm`

这一步要解决的问题：手写版把同一组「形状与步长」的事实写了四遍：smem 地址公式、swizzle 掩码、`ldmatrix` 的 lane 映射、epilogue 的 shuffle 编排，改一处就得把其余三处跟着重推。第一步先不管性能，只验证一件事：**这些映射能不能由编译器从类型里推出来**。@reed 在《cute 之简单 GEMM 实现》里给出的最简骨架恰好是三个动作：切块、分区、`cute::gemm`；本章 kernel 的前三段代码与它逐字对应。

第一个动作：把指针包成 `Tensor`　把三个裸指针包成 `Tensor`，再用 `local_tile` 切出当前 CTA 的 tile。

```
  // -- hgemm.cuh L840-858（节选，长行折行）--
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                         make_stride(k, Int<1>{}));   // B、D 同构
```

```
  Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}),
                         make_coord(iy, _));
  Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}),
                         make_coord(ix, _));
  Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}),
                         make_coord(iy, ix));
  auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM,BK,kStage)
  auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN,BK,kStage)
```

三行 `make_tensor` 就是「(ptr, shape, stride) 三元组」的字面翻译：A 是 (m,k) 且 k 连续（stride $=(k,1)$），D 是 (m,n)、n 连续。`local_tile` 的第二个参数是 tile 形状、第三个是 CTA 坐标：`make_coord(iy, _)` 固定 M 方向的 CTA 坐标，下划线保留 K 方向没被切掉的部分，于是 `gA` 的 shape 变成 $(B_M, B_K, n_{k\text{-tile}})$；第三维正是「K 方向还剩多少 tile」这个 remainder mode，主循环直接拿它当循环上界用。

第二个动作：把 tile 分到线程 上一步切好的 tile 仍是「整个 CTA 共享」的视角，这一步用 `TiledMMA` 把它拆到线程：`get_slice` 换到本线程的视角，三个 `partition_fragment_*` 给出该线程实际持有的寄存器 fragment。

```
  // -- hgemm.cuh L860-866 (节选) --
  TiledMMA tiled_mma;
  auto thr_mma = tiled_mma.get_slice(idx);
  auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // A fragment
  auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // B fragment
  auto tCrD = thr_mma.partition_fragment_C(gD);          // C accumulator
  clear(tCrD); // 累加器清零
```

`get_slice(idx)` 把「整个 TiledMMA 的能力」换成「第 `idx` 号线程的视角」，`partition_fragment_A/B/C` 则给出该线程持有的寄存器 fragment。三个 fragment 的 shape 分别是 A 用 (MMA, MMA_M, MMA_K)、B 用 (MMA, MMA_N, MMA_K)、C 用 (MMA, MMA_M, MMA_N)：第一维是「单个 atom 内的值」，后两维是「CTA tile 被切成多少个 atom」。`tCrD` 是累加器，用 `clear()` 清零。

第三个动作：`cute::gemm` 把三步接起来 把三个动作接到一起，就得到一个能跑、但完全不考虑性能的版本（结构取自 @reed《cute 之简单 GEMM 实现》，尺寸换成本章的 $B_M/B_N/B_K$）：

```
  // 最简版：无 smem、无流水线，G -> R 直读，R -> G 直写
  auto tAgA = thr_mma.partition_A(gA);   // (MMA, MMA_M, MMA_K, num_k_tile)
  auto tBgB = thr_mma.partition_B(gB);   // (MMA, MMA_N, MMA_K, num_k_tile)
  auto tCgD = thr_mma.partition_C(gD);   // (MMA, MMA_M, MMA_N)

  int num_k_tile = k / BK;               // 与 size<3>(tAgA) 同值
  for (int itile = 0; itile < num_k_tile; ++itile) {
    cute::copy(tAgA(_, _, _, itile), tCrA);   // G -> R
    cute::copy(tBgB(_, _, _, itile), tCrB);
    cute::gemm(tiled_mma, tCrD, tCrA, tCrB, tCrD);
  }
  cute::copy(tCrD, tCgD);                     // R -> G
```

这段代码里没有一行地址算术：没有 `stage*BM*BK`、没有 XOR 掩码、没有 lane 映射。`cute::gemm` 按 `tiled_mma` 的类型把 (MMA_M, MMA_N) 两层循环**全部展开**成 `mma.sync` 指令序列，等价于手写版里 `for(i) for(j) HMMA16816(...)` 的两层，只是写进了类型，源码里看不到。K 方向仍要作者显式推进：`cute::gemm` 不碰 K，因为每个 k 需要不同的 S→R 拷贝（这正是第 25.4.3 节寄存器双缓冲的落点）。

这一步留下了三个问题，正好是后两步的入口：

1. **G→R 直读 gmem**：每个线程按 fragment 的布局去 gmem 取数，既没有向量化也没有 reuse，K 循环每次迭代都完整暴露一次 gmem 延迟；
2. **搬运与计算串行**：`cute::copy` 返回之后才轮到 `cute::gemm`，LDGSTS/LDSM/MMA 三类单元同一时刻只有一个在忙；
3. **smem 完全没用上**：没有 bank conflict 问题，也就失去了宽搬运、`ldmatrix` 与跨线程的数据复用。

25.4.3　第二步：smem 与流水线——让搬运和计算重叠

这一步要解决的问题：最简版把 G→R 与 MMA 串成一条链，每一环都在等上一环，而 gmem 的延迟是几百个周期。@reed 在《cute 之 GEMM 流水线》里用经典的 RISC 五级流水线作类比：把一个 K-tile 的计算拆成 G→S（LDGSTS）、S→R（LDSM）、MMA 三级，让不同 tile 的不同级重叠执行；Ampere 的 `cp.async` 允许「发射后不等待」，配合 `commit/wait` 显式同步，就得到 multi-stage 流水线。该文还有一句对本章尤其重要的判断：**流水线的组织不属于 CuTe 的能力范围，而属于 GEMM 自身的优化策略**：CuTe 提供的是搬运与计算的抽象，怎么把它们排成流水线，是 kernel 作者的设计工作。

落到本章 kernel 上，流水线由三样东西组成：把 stage 填满的预取、主循环里「算当前 + 发下一个」的交错，以及一组**由作者维护**的计数器。

流水线要动用的搬运视图有两组。第一组把 gmem 与 smem 接起来（G→S）：

```
  // -- hgemm.cuh L868-889 (节选) --
  G2SCopyA g2s_tiled_copy_a;
  auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
  auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY,CPY_M,CPY_K, ntile_k)
  auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY,CPY_M,CPY_K, kStage)
  G2SCopyB g2s_tiled_copy_b;
  auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
  auto tBgB_copy = g2s_thr_copy_b.partition_S(gB);
  auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
```

注意 `partition_S/partition_D` 的 `(CPY, CPY_M, CPY_K, …)` 四维 shape：前三维是 TiledCopy 的「线程 × tile 内重复」坐标，最后一维是 gmem 侧的 K-tile 数或 smem 侧的 stage 数；正因为留着这一维，预取循环可以直接用 `istage` 去索引它。

第二组是 S→R 的视图（怎么把 smem 的数据装进 MMA 需要的寄存器布局）：

```
  // -- hgemm.cuh L891-904 (节选) --
  auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
  auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
  auto tAsA = s2r_thr_copy_a.partition_S(sA);     // (CPY, CPY_M, CPY_K, kStage)
  auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // 与 rA 寄存器布局对齐
  auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
  auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
  auto tBsB = s2r_thr_copy_b.partition_S(sB);
  auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
```

`retile_D(tCrA)` 把 fragment 的布局「重述」为 copy 视角（`retile` 不搬运数据，只改视图），于是主循环里的 S→R 拷贝可以直接按 (CPY, CPY_M, CPY_K) 寻址。这里为什么选 `SM75_U32x4_LDSM_N`、`make_tiled_copy_A` 为什么不必指定线程布局，属于第三步「搬运形态」的话题，留到第 25.4.4 节展开。

有了这两组视图，预取阶段就是「提交 $k_{\text{stage}} - 1$ 个 K-tile，再等一次 `cp_async_wait<kStage-2>()`」：

```
  // -- hgemm.cuh L906-915 (节选) --
  int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
#pragma unroll
  for (int istage = 0; istage < kStage - 1; ++istage) {
    cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
               tAsA_copy(_, _, _, istage));
    cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
               tBsB_copy(_, _, _, istage));
    cp_async_fence();
    ++itile_to_read;
    ++ismem_write;
  }
  cp_async_wait<kStage - 2>();
  __syncthreads();
```

三个计数器的分工在这里第一次出现：`itile_to_read` 是下一个要发起的 K-tile 序号，`ismem_write` 是下一个要写入的 stage 槽位，`ismem_read` 是当前计算要消费的槽位。主循环的每个 K-tile 里做三件事：用当前 stage 算 MMA、

发起下一个 K-tile 的 G→S、在最后一个 k_step 处 `cp_async_wait<kStage-2>()` 之后翻转 `ismem_read`。不变量是

$$\#\{\text{已提交未等待}\} \le k_{\text{stage}} - 1, \tag{25.6}$$

即最多一个 tile inflight：这正是 $k_{\text{stage}} = 2$ 时 `wait` 的语义（`cp_async_wait<N>` 等的是「未完成组数不超过 N」）。把 k_{stage} 加到 3、4 时循环体几乎不用改：放行更多 inflight tile 靠的是 `wait` 的模板参数与计数器的取模，而不是 CuTe 接管了流水线。

主循环的骨架是一层 K-tile 循环套一层 k_step 循环：前者由 runtime 决定（`num_k_tiles = k / BK`），后者是编译期常量（式 (25.5)）。循环体内部（L942–981）：

```
#pragma unroll
    for (int k_step = 0; k_step < num_k_steps; ++k_step) {
      int k_step_next = (k_step + 1) % num_k_steps;

      if (k_step == num_k_steps - 1) {
          // 等待下一个 K tile 的 smem 数据就绪，然后切换到下一个 stage。
        cp_async_wait<kStage - 2>();
        __syncthreads();
        ismem_read = (ismem_read + 1) % kStage;
      }

      // S→R: 加载下一组 A/B fragment 到寄存器。
      //
      // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
      //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
      // 但这里刻意逐 k_step_next 加载，目的是做**寄存器双缓冲**:
      //   当前 k_step 做 MMA 计算时，ldmatrix 预加载 k_step_next 的数据，
      //   让 S→R 延迟与 MMA 计算重叠。
      // 如果一次性加载全部 K slice，S→R 和 MMA 就会彻底串行。
      cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
                 tCrA_view(_, _, k_step_next));
      cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
                 tCrB_view(_, _, k_step_next));

      if (k_step == 0) {
        // G→S: 提交下一个 K tile（整个 BK）的异步拷贝。
        if (itile_to_read < num_k_tiles) {
          cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
                     tAsA_copy(_, _, _, ismem_write));
          cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
                     tBsB_copy(_, _, _, ismem_write));
          ++itile_to_read;
          ismem_write = (ismem_write + 1) % kStage;
        }
        cp_async_fence();
      }

      // MMA: cute::gemm 内部根据 tiled_mma 的类型信息，自动展开为覆盖
      // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列，累加到 tCrD。
      cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
```

四个细节单独说明：

1. **寄存器双缓冲是显式写的**：`k_step` 的 MMA 用当前寄存器，同时把 `k_step_next` 的 fragment 用 `cute::copy` 预装。若一次性 copy 整个 K slice，S→R 与 MMA 就会串行——这是第 十三 章「寄存器双缓冲」的 CuTe 版写法，**CuTe 并不会自动完成这件事**。

2. **K 循环显式、M/N 循环隐式**：`cute::gemm` 按 `tiled_mma` 的类型把 (MMA_M, MMA_N) 全部展开成指令序列，代码里看不到 M/N 的两重循环——等价于手写版里 `for(i) for(j) HMMA16816(...)` 的两层。

3. **同步点也是显式写的**：`cp_async_fence()` 提交 commit-group，`cp_async_wait<N>()` + `__syncthreads()` 等待并保证 CTA 内一致。CuTe 提供的是 `copy` 的**指令**，不是**流水线**——流水线的计数与同步仍然属于 kernel 作者。

4. **stage 翻转放在末尾**：`ismem_read` 在最后一个 k_step 处等待并前移（式 (25.6)），这样下一轮 k_tile 的两个 `ldmatrix` 才读得到新 stage。

实践须知：「CuTe 自动管理流水线」的说法不准确　Phase 7c 的头注释在列举 CuTe 五要素时，把第 5 条写成「Pipeline: cp.async + multistage, 由 cute::copy 自动管理」（hgemm.cuh L729–733）。读代码会发现这句表述**不准确**，面试时如果原样复述，很容易被追问：

- `cute::copy` 只负责回答「**哪些线程搬哪些元素**」，并把它翻译成一条 `cp.async` 指令；`cp_async_fence()`、`cp_async_wait<kStage-2>()` 与 `itile_to_read`/`ismem_read`/`ismem_write` 三个计数器，全部由 kernel 作者在 L891–1010 显式维护。
- 准确的说法是：CuTe 管**单次拷贝的线程-数据映射**，不管 **流水线的时序**。改 `kStage` 之所以几乎不动循环体，是因为计数器的取模与 `wait` 的语义本来就是按 stage 数写的（式 (25.6)），而不是因为 CuTe 接管了流水线。
- 这也解释了为什么流水线是本章最容易写错的部分：少一次 `__syncthreads()`、早一轮覆盖 smem 槽位，都会得到「大部分正确、偶发错误」的结果，而编译期不会有任何提示。

25.4.4　第三步之一：swizzle 布局与 `ldmatrix` 装载

这一步要解决的问题：流水线解决了「等」，但没有解决「搬运的形态」。第二步里 smem 只是「一块能放下 tile 的地方」；要真把它用起来，还得定三件事：smem 里的数据怎么排（否则 `ldmatrix` 会产生 bank conflict）、S→R 用哪条指令装、G→S 一次搬多少字节。@reed 在《cute 之高效 GEMM 实现》里把这几件事归在「访存高效」名下，本章 kernel 的取舍与之一致。

smem 布局：一条声明，代替散落各处的 XOR　smem 的布局由第 二十四 章的两个构造组成：先造 8 行 $\times B_K$ 列的 swizzle 原子，再 `tile_to_shape` 铺满 $B_M \times B_K \times k_{\text{stage}}$：

```
// -- hgemm.cuh L1233-1264 (节选) --
auto BM = Int<128>{};  auto BN = Int<kBN>{};  auto BK = Int<32>{};
auto KStage = Int<Stages>{};  auto kSmemLayoutCBatch = Int<4>{};

using SmemLayoutAtom = decltype(composition(
    Swizzle<3, 3, 3>{},
    make_layout(make_shape(Int<8>{}, Int<BK>{}),
                make_stride(Int<BK>{}, Int<1>{}))));
using SmemLayoutA = decltype(tile_to_shape(
    SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
using SmemLayoutB = decltype(tile_to_shape(
    SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
```

与手写版对照最能说明「声明式」的价值：手写版必须在每一次 smem 寻址处调用 `swizzle<kMmaK>()`（第 十三 章），读侧（`ldmatrix`）、写侧（`cp.async`）、epilogue 各写一遍，三处必须逐位一致。CuTe 版把 XOR 规则写进 `Swizzle<3,3,3>` 这一个类型，读和写各自从它导出地址：**同一个 swizzle 只声明一次**，连 smem 的容量（式 (25.3)）也由它算出。

注意原子是 $8 \times B_K$ 而不是 8×64：$B_K = 32$ 时若直接套用 CUTLASS 的 `GMMA::Layout_K_SW128_Atom`（列宽 64 个 half），`tile_to_shape` 会因为「64 ∤ 32」直接编译报错。这里的原子按自己的行宽定制，swizzle 参数取 $\langle B, M, S\rangle = \langle 3, 3, 3\rangle$：对 fp16 而言「16 B 一个基本元素（8 个 half）、8 个基本元素一行、8 行一周期」，与第 十三 章的 SWIZZLE_128B 档位同名；@reed 在《cute 之高效 GEMM 实现》里同样取 $\langle 3, 3, 3\rangle$ 配 $8 \times B_K$ 原子，这是 fp16、128 B 行宽的通用档位。

实践须知：swizzle 参数怎么自查 源码注释（hgemm.cuh L1244–1247 与 L1249–1252）给出了参数的物理含义与位级式子，上机时可以逐条核对：

- 参数含义：$M = 3$ 表示 $2^3 = 8$ 个 half（16 B）组成一个基本元素；$S = 3$ 表示 swizzle 作用域为 8 个基本元素（128 B）；$B = 3$ 表示参与异或的块号占 3 位。本章 8×32 的原子每行 4 个基本元素（64 B），相邻 2 行组成一个 128 B 块——恰好覆盖全部 32 个 bank。
- 位级核对：对 `Swizzle<3,3,3>`，$yyy = 7 \ll (3+3) =$ `0x1C0`、$zzz = 7 \ll 3 =$`0x38`，即把「bit $[6,9)$ 的 128 B 块号（$= r \gg 1$）」异或到「bit $[3,6)$ 的 16 B chunk 号」——与源码注释的位级式子、与第 十三 章式 (12.8)（位级 apply 公式）完全一致。
- 核对手段：本章最小测试对 (8×32) 原子做了 256 点全遍历比对（逐坐标写、逐坐标读、与公式逐点对照）。换 B_K、换 dtype 或换 swizzle 档位后，这条核对应当重跑——**swizzle 参数是最容易出错且难以从表面察觉的地方**。

G→S：一次搬满 128 位 G2S 的 Copy Atom 取 `SM80_CP_ASYNC_CACHEGLOBAL`，模板参数填 `uint128_t`：一次搬 16 B，`CACHEGLOBAL` 表示只经 L2、跳过 L1（数据马上要被 smem 消费，留在 L1 是浪费）。线程布局 $\{32,4\}$ 乘值布局 $\{1,8\}$ 的含义是「每线程连续搬 8 个 half = 128 位」：4 个线程 × 8 个 half 铺满一行 $B_K = 32$ 个 half，128 个线程每轮覆盖 32 行：**一条 `cp.async` 指令搬满 128 位，且不需要任何 predicate**。

S→R：`ldmatrix` 与「从 TiledMMA 反推」 S→R 用 `SM75_U32x4_LDSM_N`，即 `ldmatrix.x4`（一次装载 4 个 8×8 矩阵）。手写版在这里必须自己推 lane → (row, chunk) 的映射；CuTe 版把这件事变成一次推导：

`make_tiled_copy_A/B`（hgemm.cuh L881、L886）是 S→R 里最不容易直接看懂的两个调用，它们的 tiler 不来自参数，而是**从 `tiled_mma` 反推**：S→R 的线程映射必须与 MMA 的 A fragment 逐点对齐，否则装进寄存器的数据对不上指令的期望。这就是第 二十三 章讲的 TiledCopy 与 TiledMMA 必须共形：既然 TiledMMA 已经精确描述了「谁算哪块」，以它为目标的拷贝就不需要用户再指定一遍线程布局，从而避免两处独立配置不一致。@reed 在《cute 之高效 GEMM 实现》里对这一步的总结是「不再是大块到小块的 `partition`，而是对已声明的寄存器做线程级 `retile`」，正是 `retile_D(tCrA)` 做的那件事：**不搬运数据，只改视图**。

25.4.5 第三步之二：寄存器占用与 epilogue 宽写

这一步要解决的问题：搬运形态定下来之后，还剩两个问题：累加器要占多少寄存器（它直接决定 B_N 能取多大，式 (25.2)），以及 C 怎么从寄存器回到 gmem（它决定写回的效率）。

寄存器占用：B_N 是算出来的，不是挑出来的 累加器占多少寄存器，只取决于「每个线程负责多少个 C 元素」：$B_M B_N$ 个元素摊到 128 个线程上，每线程拿 $B_M B_N/128$ 个。按这个口径把两种配置的寄存器占用摆开：

累加精度	B_N	每线程 C 元素	折算 32-bit 寄存器
F16	256	$128 \cdot 256/128 = 256$ 个 half	128
F32	128	$128 \cdot 128/128 = 128$ 个 float	128

两行都是 128 个寄存器；这说明「F32 累加就把 B_N 减半」不是经验之谈，而是把寄存器预算配平：F16 累加把 4 个 half 打包进 2 个寄存器，F32 累加每个元素独占一个寄存器，B_N 必须减半才能维持同样的占用。`-Xptxas -v` 的实测与之吻合：B_N=256/F16 用 254 个寄存器、B_N=128/F32 用 226 个，两者 stack frame 均为 0 字节（0 spill）；距上限 255 只差 1 个，所以 `kBN` 由 `kAccF32` 单向决定，而不是留给调用方自由调整（完整核对见第 25.6 节）。

epilogue：先落 smem，再宽写 gmem epilogue 的路径是「寄存器 → smem scratchpad → gmem」，中间需要一次 `__syncthreads()`：因为 S→G 读的是**别的线程**刚写进 smem 的数据，必须要一次 CTA 级汇合。选这条路线的理由与 @reed 在《cute 之高效 GEMM 实现》「尾阶段高效」一节一致：累加器 fragment 的寄存器分布在 gmem 上是 **不连续** 的，直接写会产生大量零散事务、也谈不上向量化；经 smem 做一次 CTA 范围的重组之后，S→G 就能用 128 位宽写。

C scratchpad 复用当前 A stage 的空间，容量对齐关系是

$$\underbrace{32 \cdot 32 \cdot 4}_{\text{C scratchpad}} \;=\; \underbrace{128 \cdot 32}_{\text{一个 A stage}} \;=\; 4096, \tag{25.7}$$

即「4 个 32×32 的 pipe slot」恰好等于「一个 128×32 的 A stage」。先看 scratchpad 的别名与 R2S copy 的构造：

```
  // -- hgemm.cuh L1001-1005 (节选) --
  auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
  auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
```

再看双层循环的骨架与两次 copy：

```
  // -- hgemm.cuh L1074-1096 (节选) --
  auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
  auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
  int step = size<3>(tCsC_r2s); // scratchpad pipe 数 (kSmemLayoutCBatch=4)
#pragma unroll
  for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
    for (int j = 0; j < step; ++j) {
```

```
  // -- hgemm.cuh L1121-1147 (节选) --
  auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));   // R2R staging
  cute::copy(tCrC_r2sx(_, i + j), t);                  // 顺带 f32 -> f16
  ...
  cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j)); // R2S 写入 j 号 slot
  __syncthreads();
```

四个要点：

- `sC` 是「同一段地址的第二套读法」：`make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{})` 用 A 当前 stage 的 storage，按 C 的布局解释。两者的 atom 不同（8×32 对 32×32），所以不能混用（式 (25.7) 保证容量刚好够）。
- `make_tensor_like<T>` + `cute::copy` 是 R2R staging （L1121–1122）：把「层层 compose 出来的累加器视图」物化成一个线程私有、布局简单的寄存器张量，顺便完成 fp32→fp16 的类型转换。
- `group_modes<1,3>` 把 (CPY, CPY_M, CPY_N) 折成 $(CPY, (CPY_M, CPY_N))$，内层循环于是可以用线性坐标 `i+j` 遍历该线程负责的全部 C 元素，外层按 `step`= 4（scratchpad 的 pipe 深度）轮转槽位。
- 两次 `__syncthreads()`（L1147 与 S2G 循环之后）分别保证「R2S 全部写完」与「S2G 全部读完」——漏掉任何一个都会出现偶发错数据。

与手写版的对照最能说明问题：手写版必须在 epilogue 里显式处理 MMA fragment 的物理分布（`RC0/RC1` 暂存 + `__shfl_sync` 从相邻 lane 收拢 8 个 half + `lane_id % 4 == 0` 的 `float4` 写回），而 CuTe 版把这些映射写进 `retile_S/partition_D` 的类型里，kernel 源码里一个 `__shfl_sync` 都没有。

25.4.6　launch wrapper：类型即配置

kernel 本体只认类型，launch wrapper 的职责就是**把常量变成类型**。它按固定顺序推导：tile 尺寸 → SmemLayout 三件套 → TiledMMA → 四套 TiledCopy → grid/block/smem。

```
template <typename T, const int Stages = 2, const int BlockSwizzle = 0,
          const bool kAccF32 = false>
void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
  using namespace cute;

  // ── Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) ──
  //
  // kernel 用法对照:
  //   BM=128:   local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
```

```
//             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
//   BN=256:   local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
//             local_tile(D, ...) → gD 的列方向
//   BK=32:    num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
//   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
//   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
//   BN: F16 acc 用 256(128×256 tile,0 spill); F32 acc 用 128(128×128 tile,0 spill)。
//       F32 + BN=256 会产生 ~1KB spill(254 reg),性能下降 3-8 倍。
static constexpr int kBN = kAccF32 ? 128 : 256;
auto BM = Int<128>{};
auto BN = Int<kBN>{};
auto BK = Int<32>{};
auto KStage = Int<Stages>{};
auto kSmemLayoutCBatch = Int<4>{};
```

MMA 的类型推导在 L1266–1310：先用 `std::conditional_t` 按 `kAccF32` 选 atom，再由 `MMA_EU_RepeatT` 与 `MMA_P_T` 组装出 `make_tiled_mma(…)`。最后是 grid/block 配置与动态 smem 大小：

```
// -- hgemm.cuh L1386-1393 (节选，首行为注释) --
constexpr int kSwizzleStride = 2048;
int BX = (N + BN - 1) / BN;
int BY = (M + BM - 1) / BM;
int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
dim3 block(size(MMA{}));   // 128 threads (= size(TiledMMA))
dim3 grid(BX, BY, BZ);
```

```
static constexpr int shm_size_AB =
    cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
              "C shared memory must fit within one A pipe");
static constexpr int kShmSize =
    cute::max(shm_size_AB, shm_size_C) * sizeof(T);

cudaFuncSetAttribute(
    hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
                             SmemLayoutA, SmemLayoutB, SmemLayoutC,
                             S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
                             S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
    cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);

hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
                         SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
                         S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
                         BlockSwizzle>
    <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
```

三个容易忽略的点：

- `cudaFuncSetAttribute(…, MaxDynamicSharedMemorySize, kShmSize)` 必须在 launch 之前调用——48 KiB 已经接近默认配额，$k_{\text{stage}} \geq 3$ 时没有它就会启动失败；
- `BlockSwizzle=1` 时 grid 是 3D：`BZ` 把 N 方向的 tile 切成若干组（这里每组约 2048 列），kernel 里用 `blockIdx.z * gridDim.x + blockIdx.x` 还原线性列号；
- `static_assert`（L1408–1409）把式 (25.7) 的容量关系固化成编译期约束：改 B_K 或 `kSmemLayoutCBatch` 时它会立刻报警，而不是等到 epilogue 静默越界。

25.5　图示

图 25.3 是本章的核心对照：把手写版（Phase 7b/7b-3）与 CuTe 版（Phase 7c）在八个维度上逐项对照。表里所有「代码量」都是同口径实测（去注释、去空行；第 25.9 节给出复现命令）。

维度	手写 MMA（第 十二/十三 章）	CuTe（本章）
代码量	L129–397：147 行；L434–716：231 行	kernel 本体 118 行 + wrapper 86 行
smem 地址	每处显式算 `stage*BM*BK + row*BK + swizzle(i,j)`	声明 `SmemLayoutA/B`；地址由布局折叠
swizzle	手写 `swizzle<kMmaK>()`，逐点调用	`Swizzle<3,3,3>` ∘ 原子 + `tile_to_shape`
`ldmatrix` 映射	手写 `lane` → (row, chunk) 公式	`make_tiled_copy_A/B` 从 TiledMMA 反推
MMA 指令	双重循环 + `HMMA16816` 宏	`cute::gemm` 按类型展开 M/N
epilogue	`__shfl_sync` 收拢 + lane0 宽写	R2S → sC → S2G，映射进类型
流水线	手写 fence/wait 计数与 stage 轮转	**同样手写**（CuTe 不代管）
可迁移性	改 tile 要重推多处公式	改类型/常量；换 atom、换 dtype 局部化

图 25.3: FIG-24-1 手写 MMA 版与 CuTe 版的八维对照。代码量为同口径实测（hgemm.cuh，去注释与空行）。

图 25.4 是 $k_{\text{stage}} = 2$ 时的流水时序：预取 1 个 tile 后，每个 K-tile 内「MMA 用当前 stage + 提交下一个 tile」交替进行，S→R 的 `ldmatrix` 又嵌在 k_step 之间做寄存器双缓冲。三处重叠是本章 kernel 与第 十二 章手写版共享的设计。

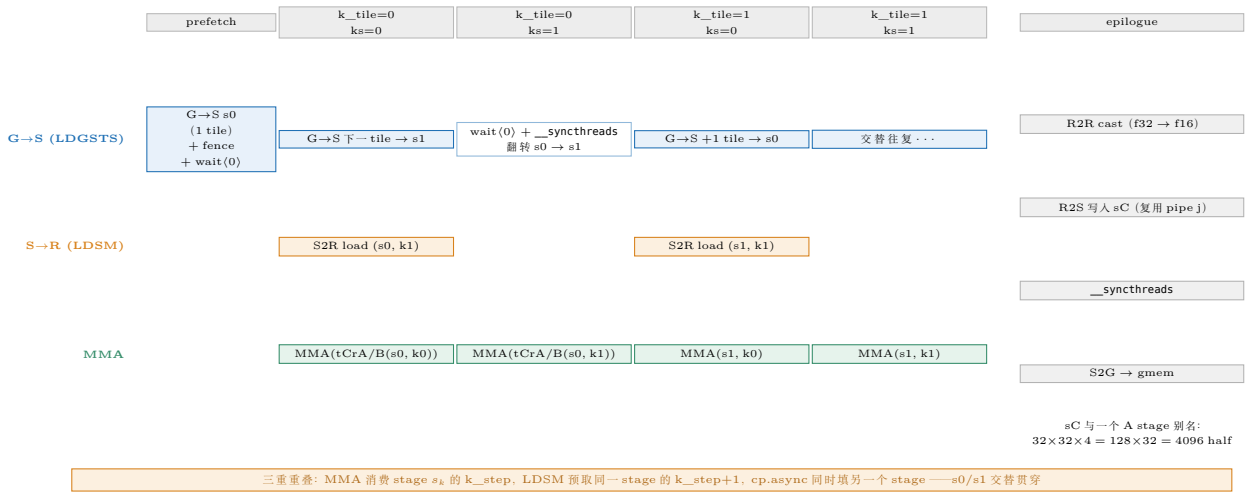


图 25.4: CuTe HGEMM 的 $k_{\text{stage}} = 2$ 流水时序（含寄存器双缓冲）。上：G→S（LDGSTS）；中：S→R（LDSM）；下：MMA。MMA 消费 stage s_k 的 k_step 时，LDSM 预取同一 stage 的 k_step+1，cp.async 同时填另一个 stage——三重重叠；左端 prefetch 一个 tile，右端 epilogue（R2R → R2S → S2G，sC 与一个 A stage 别名：32×32×4 = 128×32 = 4096 half）。

25.6　性能分析

本章没有为 CuTe HGEMM 单独建 bench（Phase 7c 的集成数据由 `bench/bench_leetcuda.cu` 的 `--bench` 统一采集）。这里只给三条**可核验**的定量结论：

1. **正确性**：最小测试在 $128 \times 256 \times 64$（F16 累加）与 $128 \times 128 \times 64$（F32 累加）两组形状上分别以 `TOL_F16ACC`= 5×10^{-2}、`TOL_F32ACC`= 10^{-3} 判定通过（第 25.9 节给出实测 `max_abs_err`）。

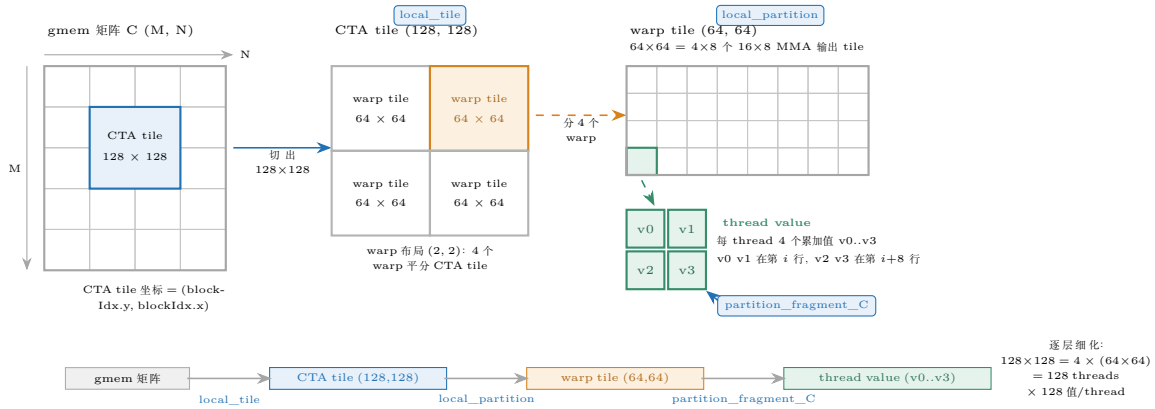


图 25.5: CuTe GEMM 的三层 partition：`local_tile` 以 `(blockIdx.y, blockIdx.x)` 为坐标从 gmem 切出 CTA tile $(128, 128)$；warp 布局 $(2, 2)$ 下 `local_partition` 平分给 4 个 warp 得 $(64, 64)$；warp tile 由 4×8 个 16×8 atom 铺满，`partition_fragment_C` 为每 thread 生成累加器片段（单 MMA tile 内 2×2 四值，v0/v1 在第 i 行、v2/v3 在第 $i+8$ 行）。层级换算 $128 \times 128 = 4 \times (64 \times 64) = 128\,\text{threads} \times 128$ 值/thread。重建自 @reed《cute 之简单 GEMM 实现》（示意图为经典 4-warp/64×64 配置；本章 kernel 实为 tile 32×32、warp 16×16，层级方法相同）。

2. **寄存器压力**：`ptxas -v` 实测两种配置的寄存器数与 stack 使用（实测值见本节的实践须知），核对源码注释里「均 0 spill」的说法。

3. **smem 占用**：式 (25.3) 的 48 KiB 由 `cosize(SmemLayoutA) + cosize(SmemLayoutB)` 在**编译期**算出——同一个常量既决定 `cudaFuncSetAttribute` 的实参，也决定 occupancy 上限。B_N 降到 128（F32 累加）时它同步降到 32 KiB，「精度换占用」的取舍是类型参数的直接后果。

实践须知：两处源码注释的性能口径

- B_N **随累加精度切换的寄存器断言**（hgemm.cuh L1229–1230）：注释称「F16 acc 用 256（128×256 tile，0 spill）；F32 acc 用 128（128×128 tile，0 spill）。F32 + B_N=256 会产生 ~1 KB spill（254 reg），性能下降 3–8 倍」。用 `nvcc -Xptxas -v` 编译本 wrapper 的两种实例化（与本章测试同源）实测：B_N=256/F16 用 254 个寄存器、B_N=128/F32 用 226 个，**两者 stack frame 均为 0 字节**（0 spill stores / 0 spill loads），与注释前两句一致。

- 注释第三句描述的组合在本 wrapper 里**不可达**：`kBN` 由 `kAccF32` 单向决定，F32 累加永远配不到 $B_N = 256$，属于「手工改坏才会遇到」的反例。测量值本身有效（它是 B_N=256 那一档的 254 reg），只是不该被读成「本 kernel 的第三种配置」；复现方法见第 25.9 节动手实验第 3 条。

- **代码量对照**（hgemm.cuh L803–805）：注释写「手写 ~200 行 / CuTe ~80 行」。按同口径实测（去注释、去空行）：手写 Phase 7b 为 147 行、Phase 7b-3 为 231 行，CuTe kernel 本体为 118 行（另有 launch wrapper 86 行）。注释给的是量级估计，方向正确（CuTe 版确实更短），但「~80 行」低估了约三成；本章一律以实测数字为准（图 25.3）。

实测性能：CuTe Swizzle 家族 vs cuBLAS　前面几条都是「自证」，再给一组**对外**的数字：第 二十一–二十四 章铺垫的 layout 代数、`TiledCopy`、`TiledMMA`、swizzle 组合子落到真实负载上究竟值多少？用 `notes_v2_sm120a.bin` 的 `--bench --bench-all` 在 RTX PRO 5000（sm_120，110 SM，CUDA 13.2）上实测 M=N=K=8192 的 fp16 HGEMM，CuTe Swizzle 家族七个配置见表 25.1：F16Acc 档追平乃至超越 cuBLAS f16（分母为同轮 cuBLAS f16 的 246.5 TFLOPS，本家族 0.99~1.08×，最高 1.08×），F32Acc 档对比 cuBLAS f32（164.2 TFLOPS）达 1.48×；七个配置的 Max Err 全部为 0.000e+00，逐位零误差。这些地址算术在编译期被类型系统全部折叠成最终指令，实测说明这套抽象是**零开销**的：表达力的收益没有以性能为代价。

表 25.1: HGEMM CuTe Swizzle 实测（RTX PRO 5000，M=N=K=8192，fp16）

Kernel	Max Err	TFLOPS/cuBLAS（加速比）
HGEMM CuTe Swizzle (S=2, BLK_SW=0, F16Acc)	0.000e+00	243.0/246.5 (0.99x)
HGEMM CuTe Swizzle (S=2, BLK_SW=0, F32Acc)	0.000e+00	201.3/164.2 (1.23x)
HGEMM CuTe Swizzle (S=2, BLK_SW=1, F16Acc)	0.000e+00	250.2/246.5 (1.01x)
HGEMM CuTe Swizzle (S=2, BLK_SW=1, F32Acc)	0.000e+00	226.0/164.2 (1.38x)
HGEMM CuTe Swizzle (S=3, BLK_SW=0, F16Acc)	0.000e+00	257.2/246.5 (1.04x)
HGEMM CuTe Swizzle (S=3, BLK_SW=1, F16Acc)	0.000e+00	265.9/246.5 (1.08x)
HGEMM CuTe Swizzle (S=3, BLK_SW=1, F32Acc)	0.000e+00	242.4/164.2 (1.48x)

CuTe 版**不承诺**比手写版更快。两者的指令序列高度重合（SASS 层面都是 `LDGSTS` + `LDSM` + `HMMA`），差别在编译期把地址算术折叠掉、以及让 kernel 作者少写上百行易错的表达式。真正的加速来自第 十三 章那样的**算法级**决策（swizzle 档位、block swizzle、寄存器双缓冲）；CuTe 的价值是让这些决策更容易表达、检查与复用。

25.7　常见坑

1. **B 必须转置传入**。式 (25.1) 的 TN 语义要求 `Bptr` 指向 $B^{\mathsf{T}}[N,K]$ 的 row-major 存储。直接把 row-major 的 $B[K,N]$ 传进去不会报错，只会算错——最小测试在 host 侧显式构造转置。
2. **没有 tile 内 predication**。kernel 只有 CTA 级边界判断，调用方必须保证 $M\%B_M=0$、$N\%B_N=0$、$K\%B_K=0$；这也决定了本章测试形状取 128/256/64（$M=N=K=64$ 用不了：$B_M=128$）。
3. **B_N 与累加精度绑定**。`kAccF32=true` 时 B_N 自动降到 128；手工把 F32 累加配到 $B_N=256$（本 wrapper 不可达的组合）会让累加器超出可用寄存器、溢出到 local memory（第 25.6 节）。
4. **动态 smem 必须显式提额**。`cudaFuncSetAttribute` 在 wrapper 里、launch 之前；漏掉它只会在 k_{stage} 较大时启动失败。
5. **BlockSwizzle 改变 grid 维度**。开关打开后 grid 是 3D 且 `BX` 要按 `BZ` 折算；只改 kernel 模板参数而不改 host 侧 grid 会索引错 block。
6. **epilogue 别漏 `__syncthreads()`**。R2S 与 S2G 之间必须汇合：S2G 读的是其它线程刚写入 sC 的数据；漏掉会得到「大部分正确、偶发错误」的结果。
7. **k_{stage} 与 smem 的联动**。k_{stage} 从 2 加到 3 会把式 (25.3) 的 A+B 部分抬高 50%；`kShmSize` 与 `cudaFuncSetAttribute` 会自动跟着变，但 occupancy 会下降——这是「改一个模板参数」的连带代价。

25.8　面试要点速查

1. **Q**：CuTe 版 HGEMM 比手写版短在哪？**A**：短在**地址算术**——smem 寻址、swizzle、`ldmatrix` lane 映射、epilogue shuffle 四类表达式全部由类型推导取代；流水线的同步逻辑（fence/wait/计数器）两边一样要手写。
2. **Q**：`cute::gemm` 自动展开了哪些循环？**A**：M/N 方向的 atom 重复（`MMA_M`×`MMA_N`）被编译期展开；K 方向仍要显式循环，因为每个 k_step 需要不同的 S→R 拷贝。
3. **Q**：为什么 $B_K=32$ 而不是 64？**A**：smem 预算与切片粒度的折中——$B_K=32$ 时 $n_{k\text{-step}}=2$、A+B 双 stage 占 48 KiB；B_K 翻倍会同步抬高 smem 并挤压 occupancy。
4. **Q**：`make_tiled_copy_A` 为什么不需要指定 ThrLayout？**A**：它从 `tiled_mma` 反推——S→R 的线程映射必须与 MMA fragment 的分布共形，两者是同一事实的两种表述。
5. **Q**：epilogue 为什么经 smem 中转？**A**：累加器 fragment 的寄存器分布不适合直接宽写 gmem；经 smem 做一次 CTA 范围的数据重组（R2S + S2G），才能用 128-bit 宽写，并且避免手写跨 lane 的 shuffle 编排。
6. **Q**：换成 F32 累加要动哪些地方？**A**：换 atom（`SM80_16x8x16_F32F16F16F32_TN`），B_N 随之降到 128，其余（C scratchpad pipe、R2S cast、S2G 宽写）由同一套类型推导覆盖——这是「类型即配置」的收益。

25.9 最小测试与动手实验

`book/tests/ch24_hgemm_cute.cu` 的逻辑照抄 `bench/hgemm.cu` L192 的 `test_hgemm_cute`，但按本书测试规范改造：参考值换成 **CPU fp64 三重循环**（不依赖 cuBLAS），判定走 `book_check` 容差三档。测试跑两组配置：

- 第一组：`launch_hgemm_mma_stages_tn_cute<half, 2, 0, false>`，M=128, N=256, K=64，容差 `TOL_F16ACC`= 5×10^{-2}；
- 第二组：`launch_hgemm_mma_stages_tn_cute<half, 2, 0, true>`，M=128, N=128, K=64，容差 `TOL_F32ACC`= 10^{-3}。

规模约束来自 kernel 的整除要求（$M\%128 = 0$、$N\%B_N = 0$、$K\%32 = 0$），同时满足 BOOK_PLAN §6「CPU 参考规模 ≤ 512」的时限约束。编译与运行：

```
cd kernels/interview/book/tests
CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch24
```

本机运行输出（RTX PRO 5000，CUDA 13.2，CUTLASS v4.6.1，2026-09-11）：

```
== ch24_hgemm_cute.cu (sm_120a)
PASS HGEMM CuTe F16Acc 128x256x64: max_abs_err=4.376e-04 (tol=5.0e-02)
PASS HGEMM CuTe F32Acc 128x128x64: max_abs_err=2.897e-04 (tol=1.0e-03)
ALL OK
```

两组输入的随机数幅度缩到 ±0.25：fp16 **输出**的舍入误差量级是 $|C| \cdot 2^{-11}$，只有把 $|C|$ 压在 2 附近，10^{-3} 这条容差才是在检验「累加精度」而不是「输出量化」。带上 `bench` 参数还能跑一版吞吐计时（K=64 的规模只有微秒级，被 launch 开销主导，不构成性能结论；端到端吞吐对照见第 25.6 节）。

代码量的统计口径（图 25.3 的数字）只统计「非空、非纯注释」行，用一条 `awk` 命令即可复现。动手实验建议三条：

1. 把 `Stages` 从 2 改成 3，观察 smem 涨到 72 KiB 量级、`cudaFuncSetAttribute` 的实参随之变化；
2. 把 `BlockSwizzle` 置 1 并改 host 侧 grid 为 3D，用同一组形状验证结果不变（只影响 L2 局部性，不影响正确性）；
3. 把 `kAccF32` 置 true 但手工把 `kBN` 改回 256，用 `-Xptxas -v` 看 spill 报告——复现第 25.6 节提到的反例。

延伸阅读

- @reed《cute 之简单 GEMM 实现》，https://zhuanlan.zhihu.com/p/667521327 （从零搭 CuTe GEMM 的三步骨架，本章第 25.4.2 节按它展开）；
- @reed《cute 之 GEMM 流水线》，https://zhuanlan.zhihu.com/p/665082713 （multistage 流水的 CuTe 写法与「流水线不属于 cute」的结论，本章第 25.4.3 节按它展开）；
- @reed《cute 之高效 GEMM 实现》，https://zhuanlan.zhihu.com/p/675308830 （访存/计算/尾阶段高效与主机端类型清单，对应本章第 25.4.4、25.4.5 节）；
- @ 竹熙佳处《写给大家看的 CuTe 教程：async pipeline》，https://zhuanlan.zhihu.com/p/2024832248386528275 （可对照本章「流水线仍要手写」的结论）；
- @ 进击的 Killua《CUTLASS CuTe 实战（一）：基础》，https://zhuanlan.zhihu.com/p/690703999 （Tensor 常用 API 速查，适合边看本章代码边查手册）。

第二十六章　CuTe 应用（二）：FlashAttention 三实现对照

26.1　本章导读

第 十七–十九 章我们用「手写地址算术」实现了三代 FlashAttention：FA2 的 Split-Q + cp.async 多级流水（flash_attn.cuh L5–790）、FA2 的 TMA + warp specialization（L792–1440）、FA3 的双 consumer warpgroup（L1441–2192）。三代实现都在做同一件事：把 $S = QK^\top$、online softmax、$O = PV$ 三段落进张量核与搬运流水之间；但每一代都要重写一遍 Q/K/V 的 smem 布局、fragment 的 lane 映射、ldmatrix 的地址算术与屏障协议。这不是写代码的人偷懒，而是「布局事实」在 C++/PTX 里没有独立的落脚点：swizzle 常量、lane 位段、寄存器编号只能以裸算术的形式与计算逻辑混写在一起。

本章问一个具体的问题：**把这些布局事实搬进类型之后，同一段 attention 会长什么样？**答案是同一文件第四个区段的四个 CuTe 实现（L2196–3488，`NOTES_V2_ENABLE_CUTE` 宏块）。它们不是四份互不相干的代码，而是 **同一个算法的三级搭建**：实现 1 `flash_attn_mma_stages_split_q_cute`（cp.async + 统一流水，256 线程）最朴素，组装方式照 @reed《cute 之简单 GEMM 实现》的「Tensor → `local_tile` → partition → copy/gemm」四步走：数据怎么切、怎么分给线程、怎么搬、怎么算，一一对应到 CuTe 抽象；实现 2 `flash_attn_tma_mma_ws_split_q_cute`（TMA + producer/consumer 分工，384 线程）在同一个算法上把搬运与计算解耦；实现 3 `flash_attn_3_tma_mma_ws_split_q_cute` 进一步采用 FA3 式双 consumer（奇偶拆 KV + 终态归并）。第四个实现 `flash_attn_3_cute_tma_copy_smoke`（42 行）不是 attention，而是一条 TMA 搬运链路的自检闭环，它的存在本身就是「先把搬运与计算解耦再排查」的方法论。这是 Part IV 的倒数第二章，也是「抽象换代码量」这条主线在 attention 上的完整样本。

本章的组织方式**以动机为线**：三级搭建逐级回答上一级留下的问题：实现 1 的浪费在哪、为什么实现 2 要把搬运拆出去、为什么实现 3 要拆 KV。四个实现共享同一组编译期布局类型（`FlashAttn2CuTeTraits` / `FlashAttn3CuTeTraits`）与五个 kernel 级工具函数，这是第 二十三 章「TiledMMA 与 fragment」在真实 kernel 里的落点：

- `convert_layout_acc_rowcol` 与 `convert_layout_acc_Aregs`——两次 fragment 重排（25.3.3/25.3.4 节）；
- `convert_type`、`gemm_ss`、`gemm_rs`——类型转换与两个 GEMM 调用范式（25.4.1 节）。

五个函数里最值得单独立节的是 `convert_layout_acc_Aregs`：P（softmax 后的分数矩阵）要从 QK 的累加器变成 PV 的左操作数，这一步是 CuTe 版 FA 最容易被一笔带过、又最值得写透的地方，也正是 @ 竹熙佳处《写给大家看的 CuTe 教程：Layout Compose & Inverse》专门讨论的场景。

前置章节：第 十八 章（手写 TMA+WS 的 mbarrier 协议）、第 十九 章（FA3 双 consumer 与 merge）、第 二十二 章（Tensor 与 TiledCopy：G2S/S2R 分工）、第 二十三 章（TiledMMA、EURepeat、fragment 与 rowcol 变换）。**源码区间**：flash_attn.cuh L2196–3488。**编译宏**：`NOTES_V2_ENABLE_CUTE`（四个实现都需要 TMA 或 cp.async 的 CuTe 抽象；sm80+ 可编）；后三个实现另需 `NOTES_V2_ENABLE_TMA_MMA_WS`（TMA 指令，sm90+ 硬件）。本节默认实例化取 $D \in \{64, 128\}$、$B_r \in \{64, 128\}$、$B_c = 64$。配套最小测试 `book/tests/ch25_fa_cute.cu`（三种形状的端到端正确性）。

26.2　问题与动机

先把手写版的「维护面」摊开。第 十七 章的 FA2 kernel 共 564 行有效代码，其中与「布局事实」直接相关的至少有五类，且每一类都以裸地址算术的形式出现：

1. **smem 行距与 padding**：$kSmemStrideQ/K/V = D + \text{pad}$，pad 取 8 个 half 使 ldmatrix 的 8 行起始 bank 铺满 32 个 bank；pk 与 $kPad$ 的组合还决定走 XOR swizzle 还是 padding 路径；
2. **ldmatrix 的 lane 映射**：Q 用 `ldmatrix.x4`、K 用 `ldmatrix.x2`、V 用 `ldmatrix.x2.trans`，三者的行/列由 `lane` 的位段手工拼出；
3. **fragment 的寄存器编号**：$R_S[8][2]$、$R_O/R_D[8][2]$ 是「哪个寄存器对应矩阵哪个元素」的硬编码答案，4-lane 归约宽度（`warp_reduce<4>`）也是从这套编号推出来的；
4. **搬运粒度的地址划分**：256 线程协作加载时 `load_smem_Q_Br = tid/2` 这类映射；
5. **屏障计数**：producer/consumer 的 arrive_count（第 十八 章的 257 协议、第 十九 章的 per-WG 129）。

第 二十一 章已经论证过这五类事实的联动代价：改一个 tile 尺寸，至少三处要跟着改，且错误是静默的。CuTe 版的答案是把它们全部收进 **编译期类型**：smem 布局是 `tile_to_shape` 的返回类型、fragment 布局是 `partition_fragment_A/B/C` 的返回类型、搬运是 `TiledCopy` 的 TV 划分、连 V 的转置视图都是 `composition` 表达式的类型。kernel 正文里只剩三类语句：数据流（`copy`/`gemm`）、同步（barrier / `cp.async.wait`）与 online softmax 的标量运算。核对「抽象掉了什么」：**坐标计算**收进 `local_tile`/`partition` 的类型、**fragment 手工索引**收进 `partition_fragment` 的返回布局、**拷贝指令的选择**收进 `TiledCopy` 的原子类型（G2S 选 cp.async 还是 TMA、S2R 选哪个 ldmatrix 形态）；三件事都不再出现在 kernel 正文里。

量化一下收益。表 26.1 用「区间总行数 / 去注释空行的有效代码行数」对照三对实现（手写版区间与 CuTe 版区间都取自同一文件，统计口径一致：行以 `\n` 计，注释行指以 `//` 开头的行）。

表 26.1: 手写版与 CuTe 版的有效代码行数对照（flash_attn.cuh，2026-09-11 统计）。「结构件」指 FA2/FA3 traits 与五个工具函数，它们被后四个实现共享。

实现	区间	总行数	有效代码行
FA2 手写（Split-Q + cp.async）	L5–790	786	564
FA2 手写（TMA + WS）	L792–1440	649	451
FA3 手写（双 consumer）	L1441–2192	752	515
CuTe 结构件（traits + 5 个工具函数）	L2197–2490	294	107
CuTe 实现 1（cp.async）	L2493–2803	311	206
CuTe 实现 2（TMA + WS）	L2807–3123	317	252
CuTe 实现 3（FA3 双 consumer）	L3126–3445	320	270
CuTe 实现 4（TMA copy smoke）	L3447–3488	42	36

对这张表的解读要有所克制，CuTe 版并没有「少一个数量级」的总代码：结构件（107 行）加三个实现（728 行）合计 835 行，只比手写三代（1530 行）少了约 45%、不到一半；真正的差别在 **每一行代码的语义密度**与**改动的影响面**。手写版的 564 行里，地址算术与 lane 映射大约占四成，它们随 tile 形状与 arch 改动而整体失效；CuTe 版的 206 行里几乎没有地址算术，改 B_r 只需要换 traits 里的 `Shape`（实现 1/2/3 的 B_r=128/128/64 就是这么分出来的，见表 26.2）。更关键的是：四个实现共享同一份 traits 与同一组 `gemm_ss`/`gemm_rs`；这正是第 二十五 章 hgemm 里「换 tile 不改 kernel 正文」在 attention 上的复现。

另一层动机来自**可验证性**。手写版的三代实现里，唯一被复用的验证资产是端到端数值对照；而 CuTe 版把布局变成可打印、可断言的类型：第 二十三 章与第 二十七 章的最小测试正是在 host 端断言 `tile_size_mnk`、fragment 元素数、rowcol 形状与 D-chunk 的 `local_tile` 语义，这些断言的「真值」直接来自 `print(layout)` 的实测输出；本章自己的测试（25.9 节）则专注三种形状的端到端正确性。

三级搭建的分工也由此而来：实现 1 先回答「能不能用 CuTe 把 attention 写对」，实现 2 回答「搬运为什么要从计算线程手里拿走」，实现 3 回答「KV 方向的并行度怎么加」。25.4 节的三个小节按这个顺序展开，每一节只重点讲该级新引入的东西，共享层的类型推导统一放在 25.3 节。

26.3　数学原理

26.3.1　两个 traits：编译期布局的「配置表」

CuTe 版 attention 的第一步是把「tile 几何 + 数据格式」编译期化。`FlashAttn2CuTeTraits`（L2200–2243）与 `FlashAttn3CuTeTraits`（L2250–2312）做的就是这件事，它们是两份只含 `using` 的配置表：

flash_attn.cuh L2226-2242（`FlashAttn2CuTeTraits`：swizzle 后的 smem layout 与 `TiledMma`）

```
struct FlashAttn2CuTeTraits {
  static_assert(kHeadDim == 64 || kHeadDim == 128);

  using Element = cutlass::half_t;
  using SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>;
  // Q tile: [Br=128, D], K/V tile: [Bc=64, D]
  using SmemLayoutQ = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<_128, Int<kHeadDim>>{}));
  using SmemLayoutKV = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<_64, Int<kHeadDim>>{}));
  using SmemLayoutVt = decltype(composition(
      SmemLayoutKV{},
      make_layout(Shape<Int<kHeadDim>, _64>{}, GenRowMajor{})));

  // 8 warps along M (Br=128 = 8*16), ValTile N=2 (16 cols = 2*8)
  using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;
  using TiledMma = TiledMMA<
```

逐字段读。`SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>` 是「一个原子」：8×64 个 half（每行 64 half = 128 B）加 `Swizzle<3,4,3>`，一个 swizzle 周期是 8 行 = 1024 B（traits 头注释把它写成「8×8 atom」是笔误，原子实际是 8 行、行宽 128 B，half 下即 8×64；第 二十四 章会展开它与 `CU_TENSOR_MAP_SWIZZLE_128B` 的对应）；`tile_to_shape` 把原子铺满目标形状：Q 是 $B_r \times D$、K/V 是 $B_c \times D$。于是「smem 里 Q/K/V 的布局」不再是散落在 kernel 里的 stride 变量，而是两个类型的 `cosize`（实测：B_r=128, D=64 时 Q 占 8192 个 half，K/V 各占 4096 个，25.9 节 case C 的 smem 80 KB 核算正是这批 `cosize` 的直接应用）。

V 的转置视图是第一个要细讲的细节：

$$\text{SmemLayoutVt} \;=\; \text{composition}\big(\text{SmemLayoutKV},\ \text{make_layout}(\text{Shape}\langle D, 64\rangle, \text{GenRowMajor})\big). \tag{26.1}$$

`composition(A, B)` 的语义是「先按 B 在 A 的定义域里取坐标、再代入 A」（第 二十一 章的 $R(c) = A(B(c))$）。这里 `A` 是 V 在 smem 中的物理布局（row-major 的 $[64, D]$ 加 swizzle），`B` 是逻辑上的 row-major $(D, 64)$，两者复合后的结果才是「$V^\top$ 的访问接口」的 col-major 视图：逻辑坐标 (d, j) 被解释为 V 的第 j 行第 d 列（源码 L2213/L2263/L2266 三处注释把这个 B 写作「col-major $(D, 64)$」是笔误，`GenRowMajor` 明示 B 实为 row-major，col-major 的是复合后的结果视图）。它的实测类型是 `Sw<3,4,3> o (64,64):(1,64)`，即 $V^\top(d, j) \mapsto d + 64j$ 的 col-major 视图，**一个字节都不搬**，即 $P@V$ 需要的 B 操作数形态（`mma.row.col` 要求 B 为 col-major），也是 25.7 节「常见坑」里「不要用带 swizzle 的 Vt 视图推导寄存器布局」的由来。

注记. 式 (26.1) 的 `composition` 与第 十三 章手写版的「V 用 `ldmatrix.trans` 转置加载」解决的是同一个问题。手写版把「转置」落在**指令**上（`.trans`），CuTe 版把「转置」落在 **布局**上（`composition`），然后在 S2R copy 的 atom 里选择 `SM75_U16x8_LDSM_T`——两条路最终都编到 `ldmatrix.sync.aligned.x2.trans`，但前者的地址算术消失了。

26.3.2　TiledMMA 的类型推导链：从 atom 到 EURepeat

第 二十三 章给出了 TiledMMA 的三元组（Atom, AtomLayout, Tile）。attention 的两个 traits 分别声明：

$$\text{FA2:}\quad \text{Atom} = \texttt{SM80_16x8x16_F32F16F16F32_TN},\quad \text{AtomLayout} = \langle 8,1,1\rangle,\quad \text{Tile} = \langle 128,16,16\rangle; \tag{26.2}$$

$$\text{FA3:}\quad \text{Atom 同上},\quad \text{AtomLayout} = \langle 4,1,1\rangle,\quad \text{Tile} = \langle 64,16,16\rangle. \tag{26.3}$$

`TiledMMA` 的内部把 Tile 与「atom ⊗ AtomLayout」相除，得到 **EURepeat**（每线程补足 tile 所需的 atom 重复数）：

$$\text{EURepeat} = \frac{\text{Tile}}{\text{AtomShape} \otimes \text{AtomLayout}} \implies \begin{cases} \text{FA2: } (128,16,16) \oslash (128,\ 8,\ 16) = \langle 1,2,1\rangle, \\ \text{FA3: } \langle 64/64,\ 16/8,\ 16/16\rangle = \langle 1,2,1\rangle. \end{cases} \tag{26.4}$$

（记 ⊗ 为「atom 形状按 AtomLayout 放大」：$\langle 16,8,16\rangle \otimes \langle 8,1,1\rangle = \langle 128,8,16\rangle$，128 行的 M 方向被 8 个 warp（每 warp 16 行）分完，故 M 向 EURepeat 为 1；N 向 16 列需要 2 个 $n8$ atom，K 向 16 恰好一个 atom。）两条式子的 N 向重复都是 2，这不是巧合：两份 traits 都选择「一次逻辑 MMA 覆盖 16 列」，原因是 $B_c = 64$ 的 S 矩阵按 16 列一块推进，softmax 的行归约粒度与之匹配。

推导链的终点是**每线程的 fragment**。`partition_fragment_C` 把逻辑张量按 TiledMMA 的线程-值划分切开，本章测试在 device 端打印出的实测形状是（D=64，B_r=128）：

$$\text{FA2 的}S/O\text{ fragment}: \underbrace{(2,2)}_{4\ \text{值}} \times \underbrace{1}_{\text{MMA_M}} \times \underbrace{8}_{\text{MMA_N}} = 32\ \text{f32/线程}, \tag{26.5}$$

其中 $8 = (B_c/16) \times 2$ 是「4 个 N tile × 每 tile 2 个 atom」：**fragment 的元素数只由 atom 与线程布局决定，与 Tile 的 N 无关**：把 Tile 换成 $\langle 128,64,16\rangle$ 不会让累加器变多，只会改变 `gemm()` 内部 atom 的组织粒度。后面理解 FFPA 的「双 TiledMma」要用到这条事实（第 二十七 章 26.3 节会回到它）。

26.3.3 rowcol 变换：把累加器重排成行 × 列

online softmax 需要按行操作 S（行 max、行内 exp、行和）。但 `m16n8k16` 的累加器 fragment 按 (MMA = 4, MMA_M, MMA_N) 排布：4 个寄存器在 lane 内的编号并不直接对应「行优先」。`convert_layout_acc_rowcol`（L2315–2339）做一次纯 layout 变换：

flash_attn.cuh L2333-2339 + L2366-2372（共享层：`rowcol` 与 `Aregs` 两次 fragment 重排）

```
template <typename Layout>
CUTE_DEVICE auto convert_layout_acc_rowcol(Layout acc_layout) {
  auto divided = logical_divide(acc_layout, Shape<_2>{});
  return make_layout(
      make_layout(get<0, 1>(divided), get<1>(divided)),
      make_layout(get<0, 0>(divided), get<2>(divided)));
}
template <typename TiledMma, typename Layout>
CUTE_DEVICE auto convert_layout_acc_Aregs(Layout acc_layout) {
  using X = Underscore;
  auto divided = logical_divide(acc_layout, Shape<X, X, _2>{});
  return make_layout(
      make_layout(get<0>(divided), get<2, 0>(divided)),
      get<1>(divided), get<2, 1>(divided));
```

代数上，它先把第一 mode 按 `Shape<_2>` 做 `logical_divide`：

$$(4,\ \text{MMA_M},\ \text{MMA_N}) \longrightarrow \big((2,2), \text{MMA_M}, \text{MMA_N}\big),$$

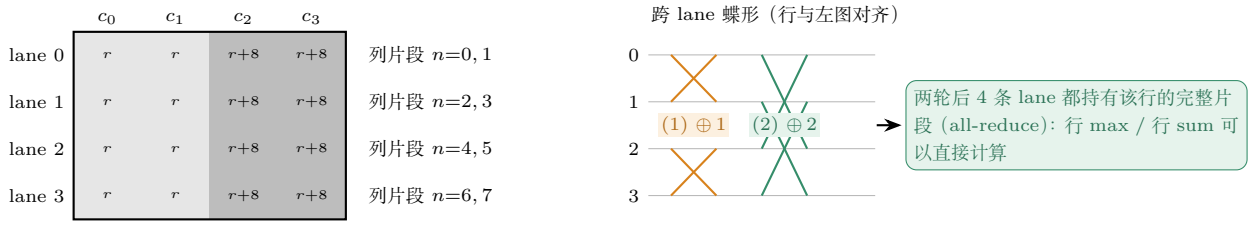
然后重组成

$$\Big(\underbrace{(2,\text{MMA_M})}_{\text{nrow}},\ \underbrace{(2,\text{MMA_N})}_{\text{ncol}}\Big). \tag{26.6}$$

为什么是「前 2 与 MMA_M 对应、后 2 与 MMA_N 对应」？看 $m16n8$ 的 C fragment 约定：4 个寄存器 (c_0, c_1, c_2, c_3) 中 (c_0, c_1) 是同一行、相邻两列（列偏移 $+1$），(c_2, c_3) 是行偏移 $+8$ 的另一组两列。即「寄存器编号的 bit0 = 列、bit1 = 行（$+8$）」，这正是 MMA $= 4 \to (2_{\text{行}}, 2_{\text{列}})$ 的 `logical_divide` 分解；再与 MMA_M/MMA_N 里的 rest 段对应后，nrow $= 2 \cdot$ MMA_M、ncol $= 2 \cdot$ MMA_N。实测（D=64、B_r=128）：

$$\text{rowcol}(S) = \big((2,1),(2,8)\big) : \big((2,0),(1,4)\big) \;\Rightarrow\; \text{nrow} = 2 \times 1 = 2, \quad \text{ncol} = 2 \times 8 = 16. \tag{26.7}$$

变换后的 tensor 与原 tensor **共享数据指针**（同一组寄存器），只是索引语义变了：`scores(r, c)` 的 r 是行、c 是列，于是行归约可以写成 `for c: max`，再对 4-lane 子组做两次 `__shfl_xor_sync`（步长 1、2）。为什么是 4-lane？因为式 (26.7) 里 r 的第二个 mode（MMA_M 的 rest）在 lane 内的宽度是 $2 \times 2 = 4$：lane/4 决定行号（寄存器对给 $+8$ 行），lane mod 4 决定行内 4 个列片段之一，两次蝶形恰好把 4 条 lane 的行片段合并。图 26.1 把这次重排的两层分别画出：4 个寄存器怎么分成「行身份」与「列片段」，以及两次 `__shfl_xor_sync` 如何把 quad 的 4 个片段并成整行。



bit0 = 列（d_0，列 $+1$）、bit1 = 行（d_1，行 $+8$）——**行身份只由值编号决定，与 lane 无关**；lane mod 4 决定的是列片段（每条 lane 2 列，4 条 lane 覆盖一个 $n8$ atom 的 8 列）。rowcol 视图实测 nrow = 2、ncol = 16（D=64、B_r=128，式 (26.7)）：寄存器内先做 16 列归约，再由此处两次蝶形补上跨 lane 的 4 个片段。

图 26.1: `convert_layout_acc_rowcol` 的两层拆解（式 (26.6)–(26.7)）。左：quad 内 4 条 lane 的 4 个寄存器 $c_0..c_3$——两档灰度标出「行身份」：c_0, c_1 属行 r、c_2, c_3 属行 r+8；右侧标出该 lane 覆盖的列片段（由 lane mod 4 决定）。值编号的 bit0 是列偏移、bit1 是行 $+8$，所以同一行的 8 列散落在 quad 的 4 条 lane 上。右：两次 `__shfl_xor_sync` 的蝶形——第一轮并 $(0,1)$ 与 $(2,3)$（步长 1），第二轮并 $(0,2)$ 与 $(1,3)$（步长 2），两轮后每条 lane 都持有该行的完整片段，行 max / 行 sum 可以就地计算。

26.3.4　P 矩阵的布局变换：从 C-layout 到 A-layout

$O = PV$ 里 P 是左矩阵 A。QK 的累加器 S 要么写回 smem 再 ldmatrix（费一次往返），要么原地重解释为 A fragment，这是 FA 的经典优化。CuTe 把这一步写成 `convert_layout_acc_Aregs`（L2341–2372）。在展开代数之前，先把这个问题的几何讲清楚。

问题的几何。$m16n8k16$ 的 C fragment 里，每个线程的 4 个值 (c_0, c_1, c_2, c_3) 是「同一行、相邻两列」（列偏移 $+1$）的一对，加「行偏移 $+8$」的另一对；当这块 C 要变成下一条 mma 的左操作数 A 时，A 的 K 向（PV 的归约维）恰好是 C 的列向。也就是说：「**C 的相邻 2 列」必须原样成为「A 的 K 向相邻 2 值**」：数据一个比特都不用改，缺的只是「换一套坐标读同一批寄存器」。直接拿 C 的 layout 去调 `gemm`，会被 CuTe 以「layout 不符合 A 的约定」拒绝；@ 竹熙佳处的教程正是拿这个场景当 layout 变换的入门例子。图 26.2 把这套「两套坐标」摆成并排网格：同一批格子、同一批 lane 归属，只有分组框与值编号变了。

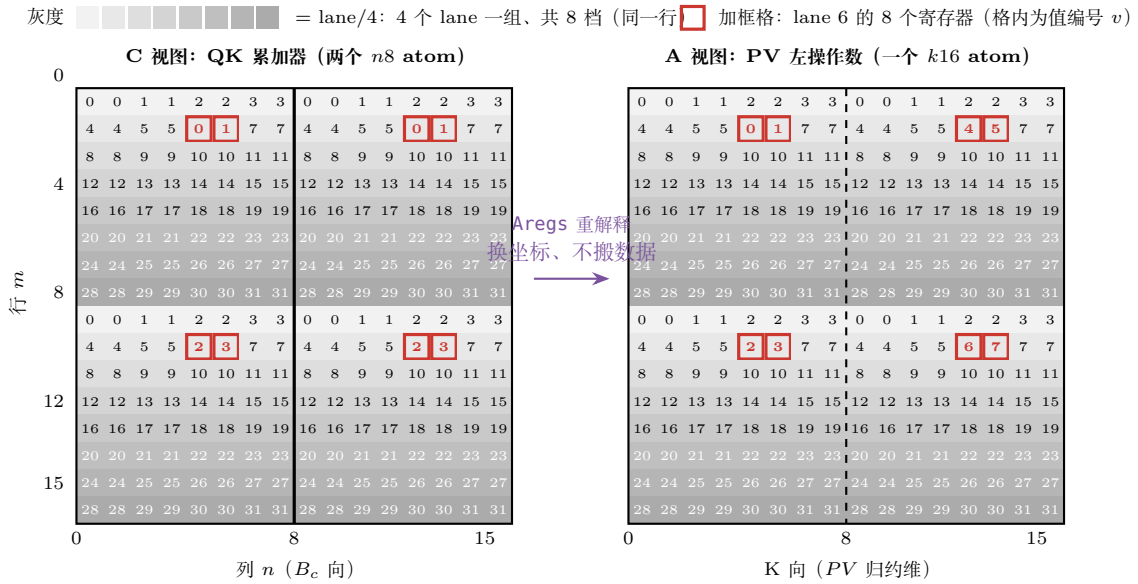


图 26.2: C fragment → A fragment 的零拷贝重解释（`convert_layout_acc_Aregs`）。左右两图是**同一批寄存器**的两套坐标读法：每格标注持有它的 lane，灰度按 lane/4 分 8 档（4 个 lane 一组，与第 十七 章 fragment 图的分组语义一致），两图的格子位置与 lane 归属**逐格恒等**——这正是「数据一个比特都不用改」的几何形态。差别只在分组与编号：左图按 $n8$ 累加器约定，值编号 $v = d_0 + 2d_1$ 在每个 atom 内各自循环 0..3（(c_0, c_1) 同行相邻两列、(c_2, c_3) 行偏移 +8，即 bit0= 列、bit1= 行）；右图按 $k16$ A 操作数约定读同一批格子，值编号变 $v = d_0 + 2d_1 + 4d_2$ 连续 0..7——第二个 $n8$ atom 的列对成为 K 向的 $k \in [8, 16)$（虚线半块，d_2=1）。加框示例 lane 6 的 8 格在两图位置完全相同、编号从「两轮 0..3」变「一轮 0..7」，即式 (26.8) 的「列对半拆分 + 重新组合」。

两条讲法。对「同一批数据、换一套坐标」，中文社区有两位作者把各自的写法写透了，恰好构成一组对照：

- **几何法（Tri Dao 的 `logical_divide`）**：把 (MMA=4, MMA_M, MMA_N) 在列方向做一次划分——令 MMA_N = (2, MMA_N/2)，再用 `make_layout` 把切出的分量按 A 的约定重新组合。@ 竹熙佳处把它概括为「在几何层面上做一次重排」：思考单位是 shape 的拆分与组合；
- **代数法（@reed 的 inverse ∘ compose）**：不碰 shape，把变换写成函数复合——先取 layout 的「坐标 ↔ offset」关系（`left_inverse`），再把「A 坐标 → offset → C 坐标 → C 寄存器编号」串成一条链（`compose`），最后复合到原 tensor 上。教程用四行代码复刻了这条链，并指出它不需要人工判断「哪一段对哪一段」——对应关系由代数推导得出。

两条路的**结果完全一致**，差别在风格与适用范围：几何法假设布局规整、编译期已知，是「拼图式」的；代数法对不规整访问更适用：w4a8 混合精度、2:4 稀疏 GEMM 这类场景没有规整形状可用，代数法是更自然的工具。选型建议：规整的静态布局用几何法，其余情况先想代数法。

本书源码用的是几何法。`convert_layout_acc_Aregs`（L2367–2372）就是 `logical_divide` 的直接应用：一行拆分（`Shape<X, X, _2>`）加一行重组（`make_layout`），与上面几何法的思路逐项同构；注释（L2352–2362）还标出了它的出处：官方 FA2 的 `flash-attention/utils.h:200`。同族的 `convert_layout_acc_rowcol`（L2334–2339，25.3.3 节）也是 `logical_divide` 路线：本章三个实现与官方 FA2 在「静态规整」的场景下都选了几何法，代数法留给更复杂的变换场景（第 二十七 章 FFPA 的双 TiledMma 交接用的是同一个几何法，见 26.3.3 节）。

把上面的几何法写成代数动作，就是「列对半拆分 + 重新组合」：

$$\big(\underbrace{4}_{\text{值}}, \text{MMA_M}, \underbrace{(2, \text{MMA_N}/2)}_{\text{列拆半}}\big) \longrightarrow \big(\underbrace{(4, 2)}_{\text{新 MMA}}, \text{MMA_M}, \text{MMA_N}/2\big). \tag{26.8}$$

式子的物理含义即前文那句话：S 的列方向两个值（+0 与 +1 列）成为 A 的 K 向连续 2 个元素。实测结果是 $((2, 2), 2), 1, 4$：仍是 32 个元素、原寄存器，只是 type 从 f32 转换为 f16；`convert_type`（L2375–2395）按当前布局的连续段做一次向量化批量转换。

注记. 式 (26.8) 的成立依赖 $m16n8k16$ 的**特定** fragment 约定——它不是通用性质。换成 $k8$ 的 atom 或 Hopper 的 wgmma（操作数在寄存器文件之外）就得另写。源码注释 L2360–2362 特意标注了这一点，是本章「常见坑」第六条的

规范来源。几何法与代数法都改不了这个前提：**布局代数能推导坐标，不能改变约定**。

26.3.5　两套流水的语义：cp.async 的组栈与 TMA 的事务计数

四个实现里有两种异步机制，它们的「等待语义」值得在数学上对齐。

cp.async 的组栈：`cp.async.commit_group` 把当前在途的搬运压栈，`cp.async.wait_group N` 等待「栈顶往下恰好 N 组未完成」，即等到「已完成组数 $\geq$ 总组数 $-N$」。对深度 k 的 software pipeline，提交顺序是「V[tile]（栈底）→ K[tile+k-1]（栈顶）」，于是 `wait_group 1` 的含义是「除栈顶的 K 之外全部完成」=「V[tile] 已就绪、K[next] 还 inflight」。实现 1 就是靠这条栈语义把 V 与 K 的等待分开的（L2743–2754）：$kStagesK > 1$ 且非尾部时用 `cp_async_wait<1>`，尾部退回 `wait<0>`。注意组编号是 **按线程独立计数**的（PTX ISA §9.7.9.25.3.3：wait 只保证本线程提交的组完成），所以实现 1 里每一处 `wait` 后面都跟着一次 `__syncthreads()`；跨线程可见性靠那次同步兜底，不是冗余。

TMA 的事务计数（mbarrier）：TMA 的完成单位是「字节数」。producer 用 `arrive_and_expect_tx(bar, bytes)` 声明「这次搬运将带来 bytes 字节」，硬件的 TMA 引擎在字节落盘后把计数减到零、屏障翻转 phase。消费者用 `wait(bar, phase)` 等待。实现 2/3 里每个 stage 一对屏障（`k_full[s]` / `k_empty[s]`）：前者是事务屏障（init 计数 1，producer 的 expect_tx 即那一次 arrive），后者是普通屏障（init 计数 $kConsumerThreads$，每个消费者线程各 arrive 一次表示「我已读完」）。三条不变量约束整个协议（图 26.3）：

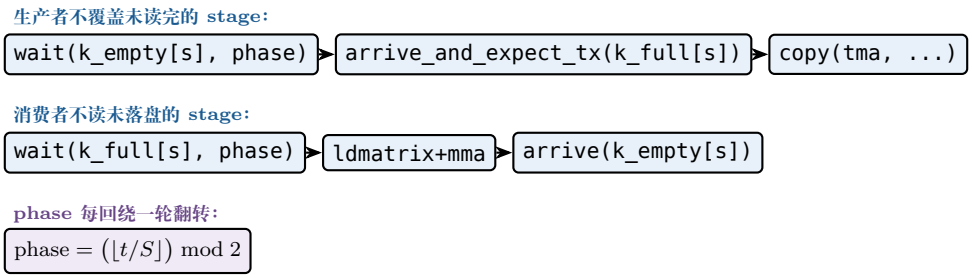


图 26.3: TMA 事务计数协议的三条不变量（25.3.5 节）：生产者 wait 到空 stage 才覆盖写入、消费者 wait 到满 stage 才读、phase 每绕 S 个 stage 翻转一次。

与第 十八 章手写版的 257 协议相比，CuTe 版的 arrive 语义没有变化，变化的是**谁来数**：手写版用 `mbarrier_arrive` 手写计数，CuTe 版把「一个 stage」打包成 `ClusterTransactionBarrier` + `ClusterBarrier` 两个对象，stage 索引与 phase 都由循环变量现算（图 26.3）。

有一个数字容易被误读：实现 2 的 `TmaBarrier::init(&q_full, 1)`（L2847）写的是 1，而第 十八 章手写版的 arrive_count 是 257。两者并不矛盾：相位翻转要求「arrive 计数归零**且** tx 计数归零」，CuTe 的 `arrive_and_expect_tx` 由 producer 单线程调用，它同时完成那 1 次 arrive 并把期望字节数写进 tx 计数，consumer 只 wait 不 arrive；手写版的 257 则是把 producer 的 arrive_tx 与 256 个 consumer 的 arrive 合并计数。两种写法等价，但计数的**含义**不同：改 init 值之前，先回到「谁 arrive、谁 wait」的角色表。

图 26.4 把这两套等待语义并排摆成两条独立的时间线：组栈数的是「还差几组没完成」，事务屏障数的是「字节是否到齐、phase 是否翻转」。

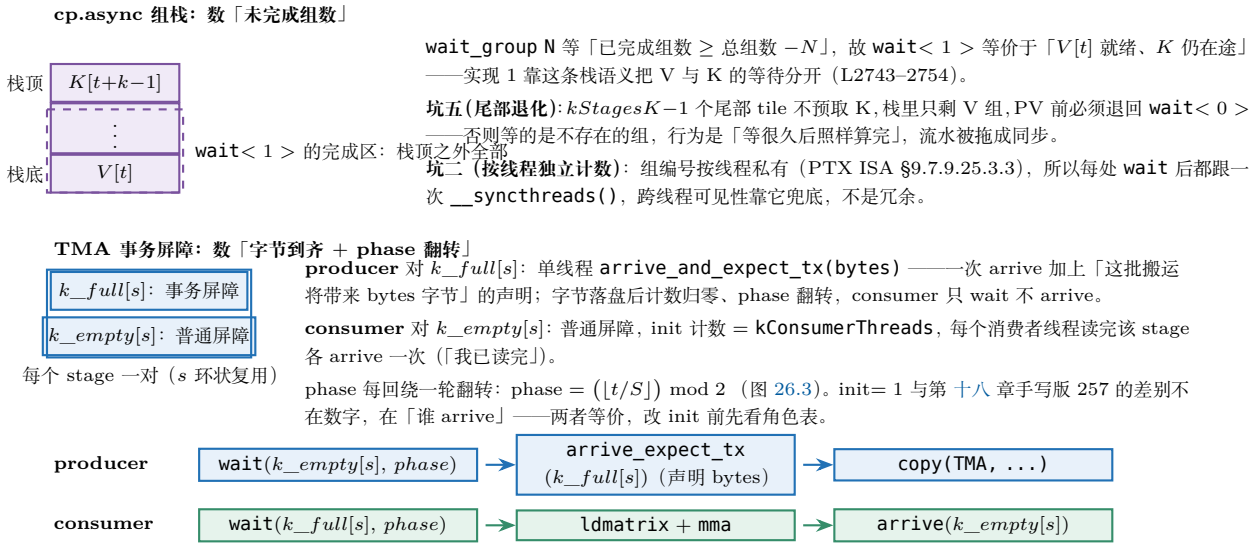


图 26.4: 两种异步等待语义的时间线对照（flash_attn.cuh 与 ffpa_attn.cuh 的 TMA 版）。上：cp.async 组栈——提交顺序 $V[t]$（栈底）→ $K[t+k-1]$（栈顶），wait< 1 > 停在「除栈顶 K 之外全部完成」，尾部 $kStages K-1$ 个 tile 必须退回 wait< 0 >（坑五）；每处 wait 后跟一次 __syncthreads()（坑二：组计数按线程私有）。中下：TMA 的事务计数——每个 stage 一对 $k_full[s]$（事务屏障，producer 单线程 arrive_and_expect_tx）与 $k_empty[s]$（普通屏障，init = kConsumerThreads），phase 由 $\lfloor t/S \rfloor \bmod 2$ 现算；最下一行是两条流水链的时序。

26.4 设计与实现

26.4.1 共享层：五个工具函数

四个实现共用的「kernel 级抽象」只有 107 行（表 26.1），除两个 traits 外就是五个函数：convert_layout_acc_rowcol（25.3.3 节）、convert_layout_acc_Aregs（25.3.4 节）、convert_type、gemm_ss、gemm_rs。后两个是 attention 计算的本体：它们把第 二十五 章 hgemm 里的 gemm() 调用范式拆成「S→R 装载 + 多级 K 循环」两件事：

flash_attn.cuh L2424-2445 + L2472-2488（共享层：gemm_ss 与 gemm_rs）

```
          typename ThreadCopyA, typename ThreadCopyB>
CUTE_DEVICE void gemm_ss(
    TensorC &acc, TensorA &fragment_a, TensorB &fragment_b,
    TensorSA const &shared_a, TensorSB const &shared_b,
    TiledMma tiled_mma, TiledCopyA tiled_copy_a, TiledCopyB tiled_copy_b,
    ThreadCopyA thread_copy_a, ThreadCopyB thread_copy_b) {
  auto copy_view_a = thread_copy_a.retile_D(fragment_a);
  auto copy_view_b = thread_copy_b.retile_D(fragment_b);
  copy(tiled_copy_a, shared_a(_, _, _0{}), copy_view_a(_, _, _0{}));
  copy(tiled_copy_b, shared_b(_, _, _0{}), copy_view_b(_, _, _0{}));
#pragma unroll
  for (int tile_k = 0; tile_k < size<2>(fragment_a); ++tile_k) {
    if (tile_k + 1 < size<2>(fragment_a)) {
      copy(tiled_copy_a, shared_a(_, _, tile_k + 1),
           copy_view_a(_, _, tile_k + 1));
      copy(tiled_copy_b, shared_b(_, _, tile_k + 1),
           copy_view_b(_, _, tile_k + 1));
    }
    gemm(tiled_mma, fragment_a(_, _, tile_k),
         fragment_b(_, _, tile_k), acc);
  }
}
```

```
          typename ThreadCopyB>
CUTE_DEVICE void gemm_rs(
    TensorC &acc, TensorA &fragment_a, TensorB &fragment_b,
    TensorSB const &shared_b, TiledMma tiled_mma,
    TiledCopyB tiled_copy_b, ThreadCopyB thread_copy_b) {
  auto copy_view_b = thread_copy_b.retile_D(fragment_b);
  copy(tiled_copy_b, shared_b(_, _, _0{}), copy_view_b(_, _, _0{}));
#pragma unroll
  for (int tile_k = 0; tile_k < size<2>(fragment_a); ++tile_k) {
    if (tile_k + 1 < size<2>(fragment_a)) {
      copy(tiled_copy_b, shared_b(_, _, tile_k + 1),
           copy_view_b(_, _, tile_k + 1));
    }
    gemm(tiled_mma, fragment_a(_, _, tile_k),
         fragment_b(_, _, tile_k), acc);
  }
}
```

两段代码可以逐行对照。`gemm_ss` 的第一步是 `retile_D(fragment)`，把寄存器 fragment 的 shape 重排成 TiledCopy 的「值视图」（D 是 copy 的目的端），使 `copy()` 能写进后续 `gemm()` 期待的位置；然后「预取 k=0、循环里预取 k+1、发射当前 MMA」构成 2 级软件流水。`gemm_rs` 是同构的，只是 `A` 不再有 smem 源（P 在寄存器里），省掉一半 ldmatrix。这条「A 在寄存器」的路径正是 `convert_layout_acc_Aregs` 存在的理由，也是本章三个实现里唯一不能省的一步：**没有它，P 就得经 smem 中转**。

注记. `gemm_ss`/`gemm_rs` 的 K 循环上界写成 `size<2>(fragment_a)`——即「A 的 K 向被 TiledCopy 切出的 tile 数」。对 B_r=128, D=128 的 QK，这是 128/16 = 8 次；对 PV 的 B_c=64 是 4 次。循环体里 `fragment_a(_,_,tile_k)` 的下标就是「K 向第几个 16 列块」——手写版里手工维护的 `tile_K_d` 计数器，在这里退化成一次 slice。

实践须知　（写给要照着读源码、改源码的读者）：

- 「**EURepeat**」**只是个好用的叫法**。它不出现在本仓库 CuTe 版本的公开接口里，源码注释用它指「tile 除以 (atom ⊗ atom-layout) 得到的每线程 atom 重复数」。读注释时把它翻译回 `Tile` 与 `AtomLayout` 两个参数即可；能实测断言的是 `tile_size_mnk`（FA2 (128, 16, 16)、FA3 (64, 16, 16)）与每个线程的 fragment 元素数（都是 32）；
- **块注释的「不整齐」是既有事实**。实现 1 的注释块（L2493–2529）从「是 `flash_attn_tma_mma_ws_split_q_cute` 的简化版」直接开头，不写主语行；实现 2/3 的 `#if` 块里直接是 `template`。对照之下，手写三代都有 Phase 头注释（L5、L792、L1441）。读的时候别按「块注释目录」找入口，用 25.4.6 节的结构对照表（表 26.2）当导读；
- **跨文件行号引用是早期迁移的遗留**。实现 1 的区域里有四处注释引用外部行号：L2519「对齐手写版 L4323-4765」、L2566 与 L2579「参考 TMA 版 L6624」、L2599「复用 TMA 版 consumer L6696-6737」。这些行号属于这段代码尚在 notes-v2.cu 中的时期，在本文件的冻结版本中并不存在（本文件共 3490 行）。按内容对应过来，它们指的是 FA2 手写版（L112–790）与 FA2 TMA+WS 手写版（L792–1440）——核对旧笔记、旧提交时以本文档给出的区间为准。

26.4.2　实现 1：cp.async + 统一流水（256 线程）

实现 1（L2493–2803）是三级搭建的第一级：目标只有一个，即**用 CuTe 把 attention 写对**，调度上不做特殊设计（@reed 的简单 GEMM 怎么组装，这里就怎么组装）。没有 warp specialization，256 个线程既搬数据也算 MMA，B_r=128、B_c=64，K 多级、V 单 buffer，用 cp.async 的组栈管理等待。它的结构可以概括为：**V 提前发、K 提前一步、softmax 填 V 的延迟**。

flash_attn.cuh L2530-2546 + L2556-2566 + L2571-2586（实现 1：签名、三段 smem、G2S `TiledCopy`）

```
template <int kHeadDim, int kStagesK = 2>
__global__ void __launch_bounds__(256)
flash_attn_mma_stages_split_q_cute(
    cutlass::half_t *Q, cutlass::half_t *K, cutlass::half_t *V,
```

```
    cutlass::half_t *output, int rows, int seqlen) {
// rows: B * H * seqlen, seqlen: N, kHeadDim: D
using namespace cute;
using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
using Element = typename Traits::Element;
using SmemLayoutQ = typename Traits::SmemLayoutQ;
using SmemLayoutKV = typename Traits::SmemLayoutKV;
static_assert(kHeadDim == 64 || kHeadDim == 128, "D=64 or 128 only");
static_assert(kStagesK >= 1, "kStagesK must be >= 1");

constexpr int kBr = 128;
constexpr int kBc = 64;
constexpr int kQTileElements = size(SmemLayoutQ{});
    make_shape(_64{}, Int<kHeadDim>{}, Int<kStagesK>{})));
auto sK = make_tensor(make_smem_ptr(shm + kQTileElements), SmemLayoutKStage{});
// V 单 buffer: (64, D), 无 stage mode
Element *v_base = shm + kQTileElements + kStagesK * kKVTileElements;
auto sV = make_tensor(make_smem_ptr(v_base), SmemLayoutKV{});

int tid = threadIdx.x;
int q_tile = blockIdx.y * (seqlen / kBr) + blockIdx.x;
int kv_tiles = seqlen / kBc;
// ★ 多 head K/V offset: local_tile coord 需加 blockIdx.y * kv_tiles 偏移,
// 否则所有 head 都读 head 0 的 K/V (参考 TMA 版 L6624 注释)
                    make_stride(Int<kHeadDim>{}, _1{}));
auto mK = make_tensor(make_gmem_ptr(K), make_shape(rows, Int<kHeadDim>{}),
                    make_stride(Int<kHeadDim>{}, _1{}));
auto mV = make_tensor(make_gmem_ptr(V), make_shape(rows, Int<kHeadDim>{}),
                    make_stride(Int<kHeadDim>{}, _1{}));

auto gQ = local_tile(mQ, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
// K/V 不在此处做 local_tile (无 remainder mode), main loop 内按 tile 做 local_tile + partition_S
// (参考 TMA 版 producer L6624: 每次 local_tile 传入 blockIdx.y * kv_tiles + tile)

// G2S TiledCopy: 128-bit cp.async, 256 threads, 每线程 8 half = 128b
using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
using g2s_copy_atom = Copy_Atom<Copy_Traits<g2s_copy_op>, Element>;
using G2SCopy = decltype(make_tiled_copy(
    g2s_copy_atom{},
    make_layout(make_shape(Int<32>{}, Int<8>{}),    // ThrLayout: 32*8=256 threads
```

注意 `kStagesK` 是模板参数而 `kStagesV` 不存在；源码注释（L2499–2501）写明「V 固定单 buffer」，理由是 V 的搬运窗口恰好被 QK + softmax 覆盖，再加深只是浪费 smem 与屏障。这与第 十七 章手写版「K 双缓冲、V 单级」的结论完全一致，但在这里是一个模板参数的缺省值而不是一段手工维护的代码。

`sK` 的类型是 `tile_to_shape(atom, (64, D, kStagesK))`，stage 维度不是在 kernel 里偏移指针算出来的，而是布局自身的一个 mode （第 二十二 章：给 layout 多加一个 mode 就是给 smem 分块加一级）。K 与 V 的 smem destination 也由 `partition_D` 一次算好（`tKsK`/`tVsV`），主循环里只需要 `partition_S` 处理变化的 gmem 源；三项都在上面代码块的第二段（L2556–2566）。

`make_tiled_copy` 的两个 layout 值得单独读：线程布局 $(32,8):(8,1)$ 铺满 256 个线程，值布局 $(1,8)$ 表示「每线程一次搬 8 个 half = 128 b」，这是第 二十二 章「TV-layout 决定 `copy` 的粒度」的实例。G2S 的源是 gmem 上的 $[rows, D]$、目的地是带 swizzle 的 smem tile，`copy()` 会按 copy 原子自动选择 `cp.async.cg.shared.global` 的 128-bit 形式。

主循环的等待策略是本章最需要理解的一段：

flash_attn.cuh L2694-2706 + L2724-2741 + L2786-2798（实现 1 主循环：等待、softmax、epilogue）

```
    cp_async_wait<0>();
    __syncthreads();
}
```

```
    // kStagesK>1: K[tile] 已在上一轮循环末尾 wait<0> 完成，无需 wait

    // Per-stage K smem (剥离 stage mode)
    auto sK_stg = make_tensor(
        make_smem_ptr(shm + kQTileElements + ismem_read * kKVTileElements),
        SmemLayoutKV{});
    auto tCrK = thr_mma.partition_fragment_B(sK_stg);
    CUTE_STATIC_ASSERT_V(size(tCrK) == size(tCrV_layout));
    auto tCrV = make_tensor(tCrK.data(), tCrV_layout);  // K 和 V 共享寄存器存储
    auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
      float nxt = fmaxf(row_max[r], tile_max);
      float rs = exp2f(row_max[r] - nxt);
#pragma unroll
      for (int c = 0; c < size<1>(tCrO_rc); ++c)
        tCrO_rc(r, c) *= rs;
      float ts = 0.0f;
#pragma unroll
      for (int c = 0; c < size<1>(scores); ++c) {
        float p = exp2f(scores(r, c) * scale - nxt);
        scores(r, c) = p;
        ts += p;
      }
      ts += __shfl_xor_sync(0xffffffff, ts, 1);
      ts += __shfl_xor_sync(0xffffffff, ts, 2);
      row_sum[r] = row_sum[r] * rs + ts;
      row_max[r] = nxt;
    }
  // ===== Step 4: Final normalize O = Oacc / l =====
#pragma unroll
  for (int r = 0; r < kRows; ++r) {
    float inv_sum = 1.0f / row_sum[r];
#pragma unroll
    for (int c = 0; c < size<1>(tCrO_rc); ++c)
      tCrO_rc(r, c) *= inv_sum;
  }

  // ===== Step 5: Epilogue - direct R->G store =====
  auto tCrO_half = fa_cute::convert_type<Element>(tCrO);
  auto mO = make_tensor(make_gmem_ptr(output),
                        make_shape(rows, Int<kHeadDim>{}),
```

$kStagesK > 1$ 时 QK 前**不等待**，因为 K[tile] 在上一轮循环末尾的 `wait<0>` 里已经等过（图 26.3 生产者流程的 cp.async 版本）；PV 前的等待则要区分「栈顶是否还有 K」：还有 K 时用 `wait<1>`，尾部没有 K 时**必须**退回 `wait<0>`（否则等的是不存在的组，V 可能未就绪）。这类「栈深度随循环位置变化」的推理靠 6 个同步点逐点注释维持；这 6 步注释就在源码块开头（L2516–2528），与代码逐点核对一一对应：Q 装载后的 `wait<0>` + 同步、K 预热后的 `wait<`kStagesK−2`>` + 同步、主循环 3a 的两个 fence、QK 前与 PV 前的分支、循环末尾的 `wait<0>` + 同步；读这段代码时可以拿它当路线图。CuTe 版并没有让这段推理消失，它仍然是手写的：**抽象不会自动理清依赖关系**。

softmax 段与 epilogue 段则是「无地址算术」的最佳样本，即上面代码块的第二、三段（L2724–2741 与 L2786–2798）。$kRows$、`size<1>(scores)` 都是布局推导出的常量；epilogue 用 `partition_C(gO)` 得到寄存器与 gmem 的对应，`copy()` 直接 R→G。对照第 十七 章手写版的 epilogue：那里要手工处理 `R_O` 的 8 个 tile、`__shfl_sync` 拼接与 128-bit 写回，这里只有三行。

26.4.3　实现 2：TMA + Warp Specialization（384 线程）

实现 2（L2807–3123）是三级搭建的第二级，回答实现 1 留下的一项浪费：256 个线程既搬又算，搬运指令与 MMA 抢发射槽，而且 cp.async 的地址算术仍然要占着计算线程。它把生产者换成 TMA：128 个 producer 线程只做一件事：发 TMA 与维护屏障；256 个 consumer 线程（8 个 warp，复用 FA2 的 B_r=128 配置）专职计算。

flash_attn.cuh L2826-2838 + L2846-2861 + L2868-2890 + L2975-2995（实现 2：常量、屏障、producer 首轮、consumer 入口）

```
  constexpr int kConsumerThreads = 256;
  constexpr int kProducerThreads = 128;
  constexpr int kQTileElements = cosize(SmemLayoutQ{});
  constexpr int kKVTileElements = cosize(SmemLayoutKV{});

  extern __shared__ __align__(1024) Element shm[];
  auto sQ = make_tensor(make_smem_ptr(shm), SmemLayoutQ{});
  Element *k_base = shm + kQTileElements;
  Element *v_base = k_base + kStagesK * kKVTileElements;

  __shared__ uint64_t q_full;
  __shared__ uint64_t k_full[kStagesK];
  __shared__ uint64_t k_empty[kStagesK];
  if (threadIdx.x == 0) {
    TmaBarrier::init(&q_full, 1);
    for (int s = 0; s < kStagesK; ++s) {
      TmaBarrier::init(&k_full[s], 1);
      CtaBarrier::init(&k_empty[s], kConsumerThreads);
    }
    for (int s = 0; s < kStagesV; ++s) {
      TmaBarrier::init(&v_full[s], 1);
      CtaBarrier::init(&v_empty[s], kConsumerThreads);
    }
  }
  __syncthreads();

  int q_tile = blockIdx.y * (seqlen / kBr) + blockIdx.x;
  int kv_tiles = seqlen / kBc;
  if (is_producer) {
    if (wg_tid == 0) {
      auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto gQ = local_tile(
          mQ, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
      auto q_slice = tma_q.get_slice(_0{});
      auto tQgQ = q_slice.partition_S(gQ);
      auto tQsQ = q_slice.partition_D(sQ);
      TmaBarrier::arrive_and_expect_tx(
          &q_full, sizeof(Element) * size(sQ));
      copy(tma_q.with(q_full), tQgQ, tQsQ);

      auto mK = tma_k.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto mV = tma_v.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto k_slice = tma_k.get_slice(_0{});
      auto v_slice = tma_v.get_slice(_0{});

      // ★ 易错点 (CuTe TMA multi-head): K/V local_tile 的 coord 是相对于 TMA
      // tensor row 维的全局 tile 索引 (TMA tensor shape = [B*H*N, D]), 不是
      // per-head tile 索引。多 head 场景必须加 `blockIdx.y * kv_tiles` 偏移,
      // 否则所有 head 都读 head 0 的 K/V -> correctness test (H=1) 正常但
      // bench (H>1) 误差飙升 2 个量纲。Q 的 q_tile 已含 blockIdx.y 无需修复。
      // 参考: /memories/repo/leetcuda-tma-fa-multihead.md
  else {
    TmaBarrier::wait(&q_full, 0);

    // V layout 从 stage 0 推导 (所有 stage layout 相同)
    auto sV0 = make_tensor(make_smem_ptr(v_base), SmemLayoutKV{});
    auto sVt0 = make_tensor(sV0.data(), typename Traits::SmemLayoutVt{});
    // ★ get_nonswizzle_portion 只用于 partition_fragment_B 推导寄存器侧 B
```

```
    // fragment layout (每线程持有哪些 V 元素)，寄存器布局与 smem swizzle 无关。
    // V 的 smem bank conflict 由 TMA 写入 (SmemLayoutKV 带 swizzle) +
    // ldmatrix 读取 (主循环内 partition_S(sVt_stg) 带 swizzle) 共同解决。
    // 这里不能用带 swizzle 的 sVt0 做 partition_fragment_B，否则 CuTe 推导
    // ldmatrix.trans 的线程-数据映射时会与 swizzle composition 冲突。
    // 参考: flash-attention/csrc/flash_attn/src/kernel_traits.h
    //       SmemLayoutVtransposedNoSwizzle 的同样用法。
    auto sVt0_ns = make_tensor(
        sV0.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));

    typename Traits::TiledMma tiled_mma;
    auto thr_mma = tiled_mma.get_thread_slice(wg_tid);
    auto tCrQ = thr_mma.partition_fragment_A(sQ);
    auto tCrV_layout = thr_mma.partition_fragment_B(sVt0_ns).layout();
```

与实现 1 的差别在 smem 的组织方式：实现 1 的 stage 维度写在 layout 里（`tile_to_shape(…, kStagesK)`），实现 2 则用「基址 + stage × 单级元素数」的指针算术（`k_base` / `v_base`，每个 stage 一个独立的 `SmemLayoutKV` tensor）。两种写法都合法，选择差异来自 TMA 的 box 尺寸固定为 $64 \times D$：按 stage 切段比给 box 加维度更直接。这也是「CuTe 不阻止你算指针」的实例：抽象层之下仍是地址。

producer 的搬运顺序是「**V 先、K 后**」，见上面代码块的第三段（L2868–2890）。注意 L2885–2892 那段「⋆ 易错点」注释：K/V 的 `local_tile` 坐标是 TMA tensor 行维上的**全局** tile 号（tensor 形状 $[B{\cdot}H{\cdot}N, D]$），多 head 时必须加 `blockIdx.y * kv_tiles` 偏移，否则所有 head 都读 head 0 的 K/V；$H{=}1$ 的正确性测试全过、$H{>}1$ 的 bench 精度却差两个量级。这条「单 head 测试测不出」的陷阱在 25.7 节还会以坑一的形式出现。

consumer 的稳态循环与实现 1 的 softmax/PV 段几乎逐字相同，区别只在「K 的早释放」与 V 的屏障等待，见上面代码块的第四段（L2975–2995）。「QK 一结束就 `arrive(k_empty)`」是本实现相对实现 1 的结构性优势：producer 可以立刻开始搬运下一个 K stage，而 softmax 与 PV 继续在消费者侧跑，这正是第 十八 章 mbarrier 协议在 CuTe 下的表述：**释放时机**决定流水深度，与实现语言无关。

26.4.4　实现 3：FA3 式双 consumer（384 线程）

实现 3（L3126–3445）是三级搭建的第三级，回答「KV 方向的并行度怎么加」：实现 1/2 都让所有消费者沿着 KV tile 单流遍历，当 KV 很长时消费者侧仍会「等搬运」；FA3 的答案是**拆 KV**：把消费者一分为二，每个 128 线程的 warpgroup 处理一半 KV tile（`tile & 1` 路由），收尾做一次部分结果归并，与第 十九 章手写 FA3 的角色图完全同构（下面代码块的三段依次是：角色划分、Split-KV 归并、以及实现 4 的 TMA copy smoke）：

flash_attn.cuh L3148-3163 + L3406-3431 + L3462-3482（实现 3/4：角色划分、Split-KV 归并、TMA copy smoke）

```
  extern __shared__ __align__(1024) Element shm[];
  auto sQ = make_tensor(make_smem_ptr(shm), SmemLayout{});
  Element *k_base = shm + kTileElements;
  Element *v_base = k_base + kNumConsumers * kStagesK * kTileElements;

  __shared__ uint64_t q_full;
  __shared__ uint64_t k_full[kNumConsumers][kStagesK];
  __shared__ uint64_t k_empty[kNumConsumers][kStagesK];
  __shared__ uint64_t v_full[kNumConsumers];
  __shared__ uint64_t v_empty[kNumConsumers];

  const bool is_producer = threadIdx.x < kProducerThreads;
  const int consumer_id = is_producer ? 0
      : (threadIdx.x - kProducerThreads) / kConsumerThreads;
  const int wg_tid = is_producer ? threadIdx.x
      : (threadIdx.x - kProducerThreads) % kConsumerThreads;
    if (consumer_id == 1) {
#pragma unroll
      for (int idx = 0; idx < size(tCrO); ++idx)
        merge_o[idx * 128 + wg_tid] = tCrO(idx);
      merge_stats[wg_tid] = make_float4(
```

```
          row_max[0], row_max[1], row_sum[0], row_sum[1]);
    }
    __syncthreads();

    if (consumer_id == 0) {
      float4 st = merge_stats[wg_tid];
      float o_max[kRows] = {st.x, st.y};
      float o_sum[kRows] = {st.z, st.w};
#pragma unroll
      for (int r = 0; r < kRows; ++r) {
        float mg = fmaxf(row_max[r], o_max[r]);
        float lhs = exp2f(row_max[r] - mg);
        float rhs = exp2f(o_max[r] - mg);
        float ms = lhs * row_sum[r] + rhs * o_sum[r];
#pragma unroll
        for (int c = 0; c < size<1>(tCrO_rc); ++c) {
          int idx = tCrO_rc.layout()(make_coord(r, c));
          float ov = merge_o[idx * 128 + wg_tid];
          tCrO_rc(r, c) = (lhs * tCrO_rc(r, c) + rhs * ov) / ms;
        }
      }
  if (threadIdx.x == 0) {
    TransactionBarrier::init(&tma_barrier, 1);
  }
  __syncthreads();

  auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
  auto gQ = local_tile(
      mQ, Shape<_64, Int<kHeadDim>>{},
      make_coord(blockIdx.x, _0{}));
  auto cta_tma_q = tma_q.get_slice(_0{});
  auto tQgQ = cta_tma_q.partition_S(gQ);
  auto tQsQ = cta_tma_q.partition_D(sQ);

  if (threadIdx.x == 0) {
    TransactionBarrier::arrive_and_expect_tx(
        &tma_barrier, sizeof(half) * size(sQ));
    copy(tma_q.with(tma_barrier), tQgQ, tQsQ);
  }
  TransactionBarrier::wait(&tma_barrier, 0);

  for (int linear = threadIdx.x; linear < size(sQ); linear += blockDim.x) {
```

归并段是 FA3 结构里唯一「算法性」的代码（其余都是流水），它的语义与第 十九 章式 19.3 等价，但用 CuTe 的 rowcol 视图重写了一遍，见上面代码块的第二段（L3406–3431）。归并的两个前提在第 十九 章已论证：O 全程保持未归一化，且以 $\max(m_0, m_1)$ 作 pivot 保证两个缩放因子 ≤ 1。CuTe 版多出来的一行是 `tCrO_rc.layout()(make_coord(r, c))`，从 rowcol 视图的 (r, c) 反查寄存器编号，用来在 smem scratch 里定位对方的 partial：布局从「注释里的 lane 表」变成了可计算的函数。

26.4.5 实现 4：42 行的 TMA 自检

第四个实现（L3447–3488）只有 42 行，是一次「TMA 搬运 + swizzle 往返」的最小闭环，即上面代码块的第三段（L3462–3482）。它的价值在调试方法上：当 attention 结果不对时，先用 smoke 确认「TMA descriptor + SW128 swizzle 布局 + 事务屏障」这条链本身是通的，再往 kernel 里查；把 42 行的搬运闭环从 300 行的 kernel 里隔离出来，正是它作为独立 kernel 存在的理由。

表 26.2: 四个 CuTe 实现的结构对照（源码：flash_attn.cuh）。*kStages* 列给出默认实例化；「共享」列标出手写版中被抽象层取代的部分。

维度	实现 1（L2493）	实现 2（L2807）	实现 3（L3126）
线程组织	256（1 WG，8 warp）	384 = 128 P + 256 C	384 = 128 P + 2×128 C
tile	B_r=128, B_c=64	B_r=128, B_c=64	B_r=B_c=64
TiledMMA	(8,1,1), $\langle 128, 16, 16\rangle$	同左（共享 FA2 traits）	(4,1,1), $\langle 64, 16, 16\rangle$
数据搬运	cp.async 128-bit（256 线程均摊）	TMA box $128 \times D$ / $64 \times D$	同左（per-consumer bank）
流水深度	$kStagesK$=2, V 单级	$kStagesK$、$kStagesV$ 均为 1（默认）	$kStagesK$=1, V 单级
KV 划分	单流全遍历	单流全遍历	奇偶拆给两个 consumer
收尾	归一化 + R→G	归一化 + R→G	smem scratch 归并 + R→G
等待原语	cp.async 组栈（wait<N>）	mbarrier 事务计数	同左

26.4.6　结构对照

表 26.2 的读法是「同构部分」与「异构部分」分开看：四个实现共享全部 layout 类型（Q/K/V 的 smem、S/O 的 fragment、P 的重解释）与 `gemm_ss/gemm_rs`，差异集中在三处：**谁来搬**（全员 vs producer vs producer 分 bank）、**谁等谁**（组栈 vs 事务计数 vs 双 bank 屏障）、**怎么收尾**（归一化 vs 归并）。这就是第 二十五 章「CuTe 只抽象计算与数据，不抽象调度」的注意点：调度（pipeline 结构、屏障协议）仍需作者设计，CuTe 消掉的是与调度**正交**的那一层地址算术。

26.5　图示

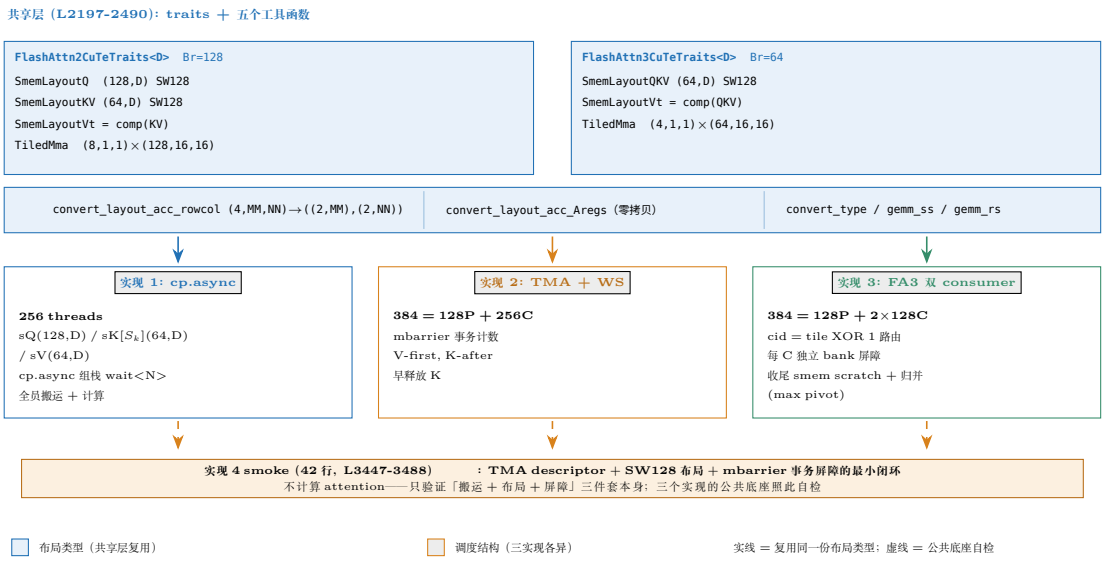


图 26.5: CuTe 块的四层结构：共享 traits 与五个工具函数（上），三个 attention 实现（中），以及 42 行的 TMA 搬运自检（底）。实线箭头表示「同一份布局类型被三种调度结构复用」，虚线表示三个实现共用实现 4 的底座自检。

26.6 性能分析

本节分三段：先交代源码注释里的既有数字与口径，再给本书的复测，最后汇总 CuTe 实测表并分析。

源码注释数字。本章区间（L2196–3488）内**没有**任何 TFLOPS/加速比断言：四个实现都是「能跑对」的对照样本，性能数字在相邻章节：第 十七 章区间内的 L61–71 注释给出 pad 与 XOR swizzle 的对照（「约 120.0 TFLOPS，显著低于 Q/K/V kPad=8 的 166.6 TFLOPS」），第 十八 、十九 章的性能节给出同机同批复测（RTX PRO 5000 sm_120a、CUDA 13.2）。读这些数字时必须注意口径：B=1, H=32、warmup≥5、iters≥20 取均值，FLOPS $= 4BHN^2D$；CuTe 版与手写版在同一口径下才可比。

本书复测。本章测试（`book/tests/ch25_fa_cute.cu`）的 `bench` 子命令按与第 十八/十九 章相同的口径测量 CuTe 实现 1/2 的吞吐：B=1、H=32、warmup= 5、iters= 20，$D \in \{64, 128\}$、$N \in \{4096\}$。三点预期：(1) 实现 1 与实现 2 的差异是**搬运路径**（cp.async 全员搬 vs TMA 单线程发）而非计算，D=128 的 Q tile 更大（128 × 128 个 half = 32 KB），实现 1 的 G2S 负载更重，两者差距应主要体现在大 D；(2) 实现 3（双 consumer）与第 十九 章手写 FA3 在 B_r=64 上的结构性差异只有「谁搬」，可对照式 19.6 的 smem 占用；(3) 本章全部实现是**非 causal、无 GQA/varlen** 的 self-attention，与手写版限制相同（源码注释 L2528–2529 明示），因此数字只应作为「CuTe 化不牺牲性能」的证据，而不是「哪个实现更快」的定论。

CuTe 实测。表 26.3 汇总 `leetcuda_bench_sm120a.bin --bench --bench-all` 的 FA CuTe 实测（RTX PRO 5000，sm_120、110 SM、CUDA 13.2；B=1、H=32、N=8192、D=128 dense，全部 F32Acc，参照 cuDNN f32 的 230.2 TFLOPS）。

表 26.3: FA CuTe 实测（RTX PRO 5000，B=1 H=32 N=8192 D=128）

Kernel	Max Err	TFLOPS/cuDNN（加速比）
FA2 CuTe MMA Stages (Sk=1, F32Acc)	1.526e-05	184.4/230.2 (0.80x)
FA2 CuTe MMA Stages (Sk=2, F32Acc)	1.526e-05	196.5/230.2 (0.85x)
FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=2, Sv=1, F32Acc)	1.526e-05	218.6/230.2 (0.95x)
FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=3, Sv=1, F32Acc)	1.526e-05	221.4/230.2 (0.96x)

三个台阶清晰可见：实现 1 的 CuTe MMA Stages 已到 cuDNN 的 0.80–0.85×，纯 CuTe 抽象、无任何搬运特化就已接近厂商库；实现 2 的 CuTe TMA MMA WS 把搬运与计算解耦后升到 0.95–0.96×、逼近 cuDNN：TMA + warp specialization 这条路线的收益在 CuTe 版上完整复现。再把 K 的 stage 从 2 加深到 3 只带来约 1.3% 的提升（218.6 → 221.4 TFLOPS），流水深度的收益已边际递减；更完整的 persistent-CTA WS 实现见第 26b 章。

26.7 常见坑

坑一：K/V 的 `local_tile` 坐标是打包维上的全局 tile 号。 TMA tensor 的 shape 是 $[B{\cdot}H{\cdot}N, D]$，K/V 的块坐标必须写 `blockIdx.y * kv_tiles + tile`；漏掉偏移时所有 head 读同一份 K/V：H=1 的正确性测试全绿，H>1 的精度却差两个量级（源码 L2885–2892 的注释即为此事）。正确性测试必须包含 H≥2（本章测试取 H=2）。

坑二：`cp.async.wait_group` 是 per-thread 的。它只保证本线程提交的组完成；跨线程可见性仍要 `__syncthreads()`。实现 1 里每一处 `wait` 后面都跟着一次同步，不是冗余。「省掉同步」的版本可能在 B_r=128 的负载下偶发错数据（写入 smem 的线程与读取的线程不同）。

坑三：V 是单 buffer，PV 之后必须同步才能进入下一轮。 L2769 的 `__syncthreads()` 注释写明「ldmatrix V 完成后才能覆盖 V smem」。删掉它，下一轮 3a 的 V cp.async 可能覆盖本轮尚未读完的 sV，表现是间歇性数值错误，且只在 D 大（ldmatrix 更多）时暴露。

坑四：推导 B fragment 的寄存器布局时要用 nonswizzle 视图。 `partition_fragment_B(sVt)` 若传入带 swizzle 的 `SmemLayoutVt`，CuTe 推导 `ldmatrix.trans` 的线程-数据映射时会与 swizzle composition 冲突（源码 L2886–2893 与 L3010–3020 的两处注释）。正确做法是 `get_nonswizzle_portion`（寄存器布局与 smem 物理 swizzle 无关），而 `partition_S`（S2R copy 的源）仍用带 swizzle 的视图；两者分工不能互换。

坑五：K 的预取深度与尾部退化。$kStagesK > 1$ 时 K 的预取位置是 `tile + kStagesK - 1`，尾部 $kStagesK-1$ 个 tile 不预取；此时栈里只剩 V 组，PV 前的 `wait<1>` 必须退化为 `wait<0>`（L2748–2753）。漏掉这个分支，尾部两个 tile 会等一个不存在的组，行为是「等很久后照样算完」（组计数不会变负），但流水被拖成同步。

坑六：`convert_layout_acc_Aregs` 依赖 $m16n8k16$ 约定。式 (26.8) 的「C 的后 2 值 ↔ A 的 K 向连续 2 值」是 $m16n8k16$ 的 fragment 约定的推论，源码 L2360–2362 特意标注「不是通用性质」。换 atom（如 $k8$ 或 wgmma）必须重推；照抄会得到静默错数据。

坑七：P 的类型转换与布局重解释的先后顺序。正确顺序是 `convert_type<Element>(tCrS)` 再做 `convert_layout_acc_Aregs`（实现 1 的 L2762–2766）：`convert_type` 用 `NumericArrayConverter` 按当前 layout 的连续段做向量化转换，先改布局会让转换作用于错误的元素分组。

坑八：f32 累加与 exp2 的缩放折叠。三处实现都写 `scale = rsqrtf(D) * M_LOG2E`、用 `exp2f(s*scale - nxt)` 替代 `expf`：把 $1/\sqrt{D}$ 折进 $\log_2 e$ 既省一条乘法也让 SFU 走 `ex2.approx`。改动缩放因子时必须同步改 `row_max/_sum` 的语义（它们都在 $\log_2$ 域），否则 softmax 的归一化在数值上自洽但语义漂移。

26.8　面试要点速查

Q1：CuTe 版 FlashAttention 消掉的到底是什么？

A：消掉的是与调度正交的*布局算术*：smem swizzle 的地址、ldmatrix 的 lane 映射、fragment 的寄存器编号、S→P 的重解释。没有消掉的是*调度*：流水深度、屏障协议、释放时机（25.4.6 节）。所以「CuTe 让 attention 变简单」的正确版本是：简单在「改 tile 不改正文」，难的部分（等谁、何时释放）原样保留。

Q2：为什么 FA2 的 traits 用 8 个 warp、FA3 用 4 个？

A：B_r 不同（128 vs 64）。TiledMMA 的 AtomLayout 必须恰好铺满 M 向 atom 数：$\langle 16,8,16\rangle \otimes \langle 8,1,1\rangle$ 的 M 是 128，$\langle 4,1,1\rangle$ 的 M 是 64。若 warp 数与 tile 不匹配，`partition_fragment_C` 仍能编译，但会有 warp 拿到空 fragment（静默低效）。

Q3：`convert_layout_acc_rowcol` 之后为什么是 4-lane 归约？

A：$m16n8$ 的 C fragment 里，同一行的 2 个片段分布在 lane mod4 的不同值上（式 (26.7) 的 $(2,1)$ 结构）；两次 `__shfl_xor_sync`（步长 1、2）把宽度 4 的子组归约完，再乘以每线程的列数即可。宽了会重复计数、窄了会漏行。

Q4：P 为什么会「零拷贝」地变成 A 操作数？

A：因为 B_c（S 的列）与 PV 的 K 维是同一个物理维度，$m16n8k16$ 的 C 与 A 在该维度上的 2 元素的对应方式恰好兼容（式 (26.8)）。这是寄存器复用的经典技巧，CuTe 用一次 `logical_divide` 表达，代价是把它从「技巧」变成了「有条件的等式」，条件写在注释 L2360–2362。

Q5：实现 2 的 V 为什么在 K 之前发 TMA？

A：等待顺序是「先 K 后 V」（QK 需要 K、PV 需要 V），但发射顺序是 V 先：V 的落盘窗口要横跨 QK+softmax 两段计算。这与实现 1 的「V 最后发、等 wait<1>」形异实同：都是让 V 的延迟藏在 softmax 里。

Q6：四个实现里哪个最快？

A：本章不做这个断言（25.6 节）：四者的 tile、线程、流水深度都不同，受控对照需要在同一 $B_r/B_c/kStages$ 下比较。可确认的是：CuTe 化没有牺牲吞吐（对照第 十八/十九 章复测）。

Q7：TMA 的 mbarrier 协议里，phase 为什么每回绕一轮翻转？

A：mbarrier 是「相位对象」而非计数器：arrive 数到零后相位翻转、计数重置。stage 循环复用同一个 barrier 时，t 号 tile 属于第 $\lfloor t/S \rfloor$ 个相位，故 phase = 该商 mod 2 （图 26.3）。写错 phase 的症状是「等一个已经过去的相位」：永久挂起或提前通过。

Q8：smoke kernel 为什么值得单独存在？(42 行)

A：它是「TMA + SW128 布局 + 事务屏障」三件套的最小闭环，把*搬运链路*与*计算链路*解耦。调 attention 时先用它排除搬运侧问题（descriptor 参数、box 与 layout 匹配、arrive_tx 的字节数），再去查计算侧；本章测试（25.9 节）的三个 case 全走实现 1，并未单独驱动 smoke kernel；需要验搬运链路时可把它当独立小用例编译运行。

26.9　最小测试与动手实验

测试文件：`book/tests/ch25_fa_cute.cu`。三个 case 全部是**端到端正确性**验证：固定 B=1、H=2、$k_{StagesK}$=2，全部调用实现 1（cp.async 统一流水，256 线程），与 CPU fp64 参考 $\mathrm{softmax}(QK^\top/\sqrt{D})V$ 对照（half 输入/输出），容差 `TOL_F16ACC`$= 5\times10^{-2}$。形状约束：$N \bmod 128 = 0$（B_r=128）且 $N \bmod 64 = 0$（B_c=64）、$D \in \{64, 128\}$（kernel 的 `static_assert`；sm_{80}+）。三个形状的分工：

- **case A**（D=64，N=128）：单 Q tile、KV 分 2 段——最小形状走通完整的 online softmax KV 循环；
- **case B**（D=64，N=256）：多 Q tile，grid (2, 2)（Q 方向 2 tile × $B{\cdot}H$=2）——验证 `local_tile` 切分与 grid 映射；
- **case C**（D=128，N=128）：大 D 路径，smem 80 KB < 99 KB optin——验证 traits 在 D=128 下的 smem 预算与 fragment 形状。

host 端的 traits 结构断言（`tile_size_mnk`、fragment 形状、D-chunk 语义）与 TMA 往返 smoke 不在本测试内：前者由第 二十三 章与第 二十七 章的最小测试承载，后者即本章实现 4 的 42 行 smoke kernel，可单独编译驱动。

编译与运行（本机 sm_120a，6 号卡；结果见下）：

```
$ cd book/tests && CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch25
== ch25_fa_cute.cu (sm_120a)
PASS fa_cute cp.async D=64 Sk=2 B=1 H=2 N=128: max_abs_err=8.149e-05 (tol=5.0e-02)
PASS fa_cute cp.async D=64 Sk=2 B=1 H=2 N=256: max_abs_err=6.614e-05 (tol=5.0e-02)
PASS fa_cute cp.async D=128 Sk=2 B=1 H=2 N=128: max_abs_err=8.084e-05 (tol=5.0e-02)
ALL OK
```

动手实验建议：(1) 把 case B 的 H 改成 1、并把 kernel 换成实现 2（需 `NOTES_V2_ENABLE_TMA_MMA_WS`），再删掉实现 2 producer 的 `blockIdx.y * kv_tiles` 偏移，正确性仍全绿，说明「单 head 测试掩蔽多 head 缺陷」；(2) 把实现 1 的模板参数从 `<64, 2>` 改成 `<64, 3>` 并打印 smem 用量，观察 K 三级时 smem 增量的构成（V 仍单 buffer）；(3) 把 case A 的模板参数从 `<64, 2>` 改成 `<96, 2>`，观察 traits 的 `static_assert(kHeadDim == 64 || == 128)` 拒绝编译，再对照第 二十七 章 FFPA 的 `kHeadDim % 64 == 0` 放宽条件；(4) 把实现 2 的 `kStagesK` 从 1 调到 2，核对 smem 是否仍在 sm_120a 的 99 KB optin 之内（D=128 时 Q 一段就占 32 KB）。

延伸阅读

- @ 竹熙佳处《写给大家看的 CuTe 教程：Layout Compose & Inverse》
  https://zhuanlan.zhihu.com/p/1962625273636845008
  （$S \to P$ 布局变换的两种讲法——Tri Dao 的几何重排与 @reed 的 inverse∘compose 代数法——本章 25.3.4 节的主参考）；
- @reed《cute 之简单 GEMM 实现》与《cute 之高效 GEMM 实现》
  https://zhuanlan.zhihu.com/p/667521327
  https://zhuanlan.zhihu.com/p/675308830
  （「Tensor → 分块 → 划分 → 拷贝 → 计算」的组装范式、多级流水写法与 Epilogue 的来龙去脉——前者是本章实现 1 的蓝本，后者的「四个高效」是三级搭建的对照读物）；
- @ 竹熙佳处《写给大家看的 CuTe 教程：tiled mma》
  https://zhuanlan.zhihu.com/p/1937145378446226159
  （fragment 视角的 mma 线程-值映射，25.3.2–25.3.4 节的入门读物）；
- @66RING《使用 cutlass cute 复现 flash attention》
  https://zhuanlan.zhihu.com/p/696323042
  （CuTe 版 attention 的另一种组织：把 Q/K/V 的 smem 布局与 S2R copy 全部交给 CuTe，25.3 节的类型推导链可与之互证）；
- @shengying.wei《FlashAttention 笔记：tiny-flash-attention 解读》
  https://zhuanlan.zhihu.com/p/708867810
  （极简 FA 实现里 MMA atom 与线程布局的选择逻辑，对应式 (26.4) 的另一种推导）；
- @Titus《GEMM 流水线：single/multi-stage、pipeline》
  https://zhuanlan.zhihu.com/p/712451053
  （软件流水的 stage 语义；25.3.5 节「组栈 vs 事务计数」两种等待模型的对照背景）；

- @ 竹熙佳处《写给大家看的 CuTe 教程：async pipeline》
  https://zhuanlan.zhihu.com/p/2024832248386528275
  （CuTe 里表达异步流水的最新手法，可与实现 1 的手写组栈用法并读）。

第二十七章　CuTe 应用（三）：FFPA Split-D 的类型代数

27.1　本章导读

第 二十 章把 Split-D 当作算法讲透了：为什么大 head_dim 会触及 smem 容量上限、S 如何沿 D 切成 chunk、O 如何拆成分量并在收尾合并、chunk 数与流水级数的相位关系如何推导。本章换一个视角：把同一份代码（`ffpa_attn.cuh` 的 CuTe 版）当作类型系统来读，回答一个第 二十 章刻意没有展开的问题：

> Split-D 的每一个结构决策（双 TiledMma、chunk 循环、独立 O 累加器），在 CuTe 里究竟长什么类型？哪些事实可以在 host 端离线断言，而不必等到 device 上跑出错误结果？

这个视角有直接的工程收益。第 二十六 章已经演示过一次：手写 attention 里靠注释和规则维持的 lane 映射、tile 计数、转置视图，在 CuTe 版里变成了 `print(layout)` 可以打印、`static_assert` 可以断言、host 端测试可以离线核对的类型事实。FFPA 把这件事推得更远一步：Split-D 引入的 chunk 循环让「归约维度被切开」这件事也必须落在类型上：

- QK 与 PV 用两个不同形状的 TiledMma，却共享同一批 128 个线程；两个 fragment 为什么能「零拷贝交接」？
- S 的归约维（D）被切成 C 个 chunk，但 S 的 fragment 形状一次都没有变——chunk 循环到底切开了什么？
- 每个 chunk 一份 O 累加器——$C \times 32$ 个 f32 寄存器是怎么被类型推导出来的？

本章的叙述顺序与第 二十六 章一致：先给类型代数（26.3 节，所有公式都能在 host 端用 CuTe 原语验证），再逐相位读 kernel（26.4 节），最后是常见坑、面试速查与最小测试。全章沿一条**数据流**展开：Q/K 的 chunk 从 smem 切片进来、在 S 的 32 个寄存器里跨 chunk 累加、经 rowcol 视图做 online softmax 就地变成 P、再被 Aregs 重解释成 PV 的 A 操作数。每一步都只用类型回答「数据在哪、什么形状、谁持有」。两处布局变换（rowcol 与 Aregs）的通用讲法在第 二十六 章 25.3.4 节已经展开，本章只补 Split-D 语境下的增量。涉及算法推导（chunk 分解的数学、online softmax 的不变量、相位计数）的地方，直接引用第 二十 章，不重复。

前置：第 二十 章（Split-D 算法）、第 二十一–二十三 章（布局代数、TiledCopy、TiledMMA）、第 二十六 章（attention 的 CuTe 化，包括共享层 `convert_layout_acc_rowcol` 与 `convert_layout_acc_Aregs` 的来历）。

27.2　问题与动机

Split-D 的动机在源头注释里写得很直白（`ffpa_attn.cuh` L7–8）：「支持 head_dim > 128 的 large head-dim attention，通过 64-wide Split-D chunks 处理」。第 二十 章量化了「为什么必须分」：smem 容量、以及 softmax 的结构性位置。本节只关心「分完之后，抽象层的四个新问题」。

先从第 二十六 章的结论出发。FA2 的三个 CuTe 实现里，attention 的 D 维（head_dim）是一个整体：Q/K/V 的 smem tile 直接是 $[B_r, D]$、$[B_c, D]$，TiledMma 的 K 向一次覆盖 16 列，循环在 N（KV tile）方向滚动。FFPA 把这两件事都改了：

1. **K 向（D）被切成 chunk，循环滚动**。$D = 512$ 时 $C = 512/64 = 8$ 个 chunk；QK 的累加变成一个跨 *chunk* 的求和，每个 chunk 内部仍是一条标准的 $k16$ MMA 链。
2. **QK 与 PV 的 TiledMma 形状不同**。QK 要一次覆盖 $S[64, 64]$（省掉 N-tile 循环），PV 只要覆盖 O 的 16 列（控制累加器粒度）。两者共享同一批线程，S（QK 的 C fragment）必须变成 P（PV 的 A 操作数）。

3. O **的累加器按 chunk 分裂**。$D = 512$、$C = 8$ 时，每个线程持有 8×32 个 f32 的 O 分量。

4. **生产者也要跟着分 chunk**。TMA box 或 cp.async 的搬运单元从「整块 $[64, D]$」变成「C 个 $[64, 64]$」。

四个问题的共同点：它们都不是数值问题（数值精度与归约顺序都已确定），而是**类型与布局问题**：tile 怎么切、fragment 怎么换解释、寄存器怎么编号、循环与 stage 怎么对应。把这四个问题逐个翻译成 CuTe 的类型与布局，就是「用 CuTe 抽象表达 Split-D」的全部内容。

本章引用的源码位置汇总在表 27.1。

表 27.1: 本章引用的 `ffpa_attn.cuh` 位置（CuTe 版，cp.async 路径 `ffpa_split_d_cute`，L79–376）。TMA+WS 版在同构位置复用同一批 helper，差异归第 二十 章。

源码位置	内容	本章视角
L42–61	traits：smem 布局与双 TiledMma	类型推导链（26.3.1）
L79–97	kernel 签名与 chunk 常量	128 线程 / `kDChunks`（26.4.1）
L118–128	G2S `TiledCopy`	TV 布局与 MMA 对齐（26.4.2）
L136–141	V 的 fragment 布局预计算	转置视图与 nonswizzle（26.4.3）
L152–176	fragment 尺寸与寄存器占用	可断言的常量（26.3.2）
L217–271	Phase 1：QK 的 chunk 流水	归约维的循环表达（26.4.5）
L273–303	Phase 2：softmax 与 P 的生成	rowcol 与 Aregs（26.3.3）
L307–353	Phase 3：PV 的 chunk 循环	双 TiledMma 的交接（26.4.6）
L355–374	Phase 4：归一化与写回	收尾的布局复用（26.4.7）

27.3　数学原理：类型与布局代数

27.3.1　两个 TiledMma 的推导链

第 二十三 章给出了 TiledMMA 的三元组 (Atom, AtomLayout, Tile)，以及「每线程补足的 atom 重复数」（源码注释称之为 `EURepeat`）：

$$\text{EURepeat} = \frac{\text{Tile}}{\text{AtomShape} \otimes \text{AtomLayout}}, \qquad \text{AtomShape} = (16, 8, 16), \quad \text{AtomLayout} = (4, 1, 1). \tag{27.1}$$

代入两个 Tile（`ffpa_attn.cuh` L52–58）：

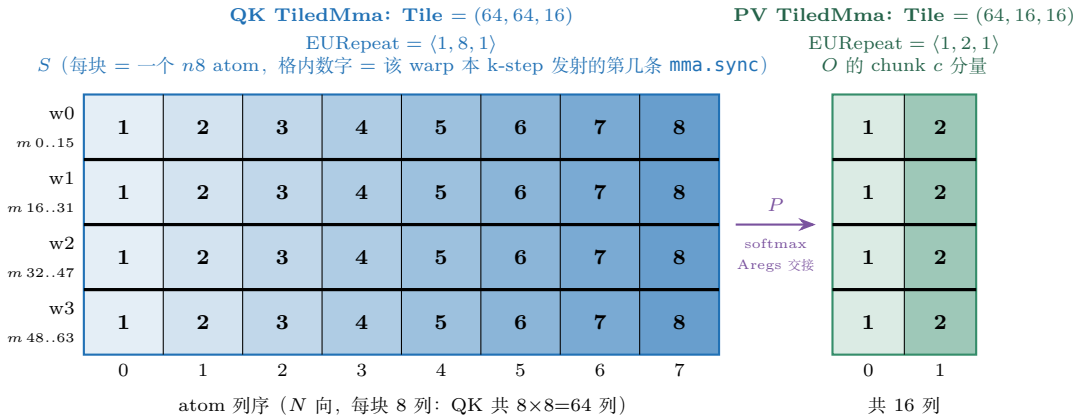
$$\text{QK:} \quad (64, 64, 16) \oslash \big((16, 8, 16) \otimes (4, 1, 1)\big) = (64, 64, 16) \oslash (64, 8, 16) = \langle 1, 8, 1 \rangle, \tag{27.2}$$

$$\text{PV:} \quad (64, 16, 16) \oslash (64, 8, 16) = \langle 1, 2, 1 \rangle. \tag{27.3}$$

读法：M 向的 4 个 atom 位置被 4 个 warp 分完（AtomLayout 的 M 分量 $= 4$，每 warp 负责 16 行，故 M 向的重复数为 1）；N 向没有 atom-layout 的分工（N 分量 $= 1$），所以 N 向的重复数就是 $\text{Tile}_N/8$：QK 是 8，PV 是 2。也就是说：

EURepeat$_N$ **就是「每个线程每个 k-step 发射的 `mma.sync` 条数」**。QK 的 8 条一次覆盖 S 的整个 64 列（$8 \times n8$），于是「N-tile 循环」整个消失；PV 的 2 条只覆盖 O 的 16 列，O 的其余列由外层 V-chunk 循环驱动。

图 27.1 把这条对比摆成两个 tile 网格：行维同样被 AtomLayout $= (4, 1, 1)$ 的 4 个 warp 分完，列维的 atom 数相差 4 倍。



两个 TiledMma 的 frag 相同：$\text{frag}_S = \text{frag}_O = ((2, 2), 1, 8) = 32$ 值/线程（式 (27.5)，第三 mode 的 8 来历不同）；行维同被 4 warp 分完，

差别全在 N 向：QK 的 EURepeat$_N$=8 ——8 条 `mma.sync` 一次覆盖 S 整个 64 列，「N-tile 循环」消失；PV 的 EURepeat$_N$=2 只覆盖 O 的 16 列，其余列由外层 V-chunk 循环驱动。

图 27.1: 双 TiledMma 的 Tile 分工（式 (27.2)–(27.3)）。左：QK 的输出 $S[64, 64]$——4 个 warp 各占 16 行（AtomLayout 的 M 分量 4），列向 8 个 $n8$ atom 由**同一线程**在一个 k-step 里连发 8 条 `mma.sync` 覆盖（蓝深浅 8 档标识 atom 身份，格内数字为发射序）；右：PV 的输出 O 只有一个 chunk 分量 $[64, 16]$（绿 2 档、2 个 atom、2 条 `mma.sync`）。中间箭头是 26.3.3 节的交接：S 经 softmax 与 `Aregs` 零拷贝重解释成 P，作为 PV 的 A 操作数——共享的正是左侧那批 S 寄存器。

对照第 二十六 章 FA2 traits 的 $(8, 1, 1) \otimes (128, 16, 16)$：那里是 8 个 warp 分 $M = 128$，N 向重复数为 2。同一套公式，两种分工：FA2 把线程堆在 M 上换取更大的 B_r，FFPA 把线程堆在 N 上换取「一次覆盖整行块 S」。**线程布局是资源分配问题，TiledMma 的三元组就是它的类型表达**。

27.3.2 fragment 的元素计数：两个恒等式

TiledMma 定下来之后，每个线程持有多少寄存器就定了。这里有两个恒等式，后面要讲的「零拷贝交接」就建立在它们上。

恒等式一（C fragment 的大小）：`partition_fragment_C` 把目标输出形状按 TiledMma 的线程布局切开，所以

$$\left|\text{frag}_C\right| = \frac{\left|\text{目标输出形状}\right|}{\text{线程数}}. \tag{27.4}$$

FFPA 的两处调用（L152 与 L218）目标形状都是 $(64, 64)$，线程数都是 128，所以 **QK 的 S 与 PV 的 O 都是每线程 32 个 f32**。实测：

$$\text{frag}_S = \big((2, 2), 1, 8\big) = 32 \text{ 值/tid}, \qquad \text{frag}_O = \big((2, 2), 1, 8\big) = 32 \text{ 值/tid}. \tag{27.5}$$

注意 frag 的第三 mode 是 8：那是 QK 的 EURepeat$_N$ 在 S 上的投影；O 的第三 mode 数字恰好也是 8，但它来自 PV 的「64 列输出被重复 8 次」而不是 EURepeat（O 的 C 形状 × 线程数决定了 total）。**两个 32 的来历不同，大小相同：这是 $S \to P$ 与 $P \to O$ 两条交接链路能成立的前提**。

恒等式二（rowcol 视图的行列数）。online softmax 要在「行」上做 max/sum，所以第 二十六 章的工具函数 `convert_layout_acc_rowcol` 把 C fragment 从 (MMA = 4, MMA_M, MMA_N) 重排成 $\big((2, \text{MMA_M}), (2, \text{MMA_N})\big)$。FFPA 用它推两个常量（L158–162）：

$$\text{nrow} = 2 \cdot \text{MMA_M} = 2, \qquad \text{ncol} = 2 \cdot \text{MMA_N} = 16. \tag{27.6}$$

nrow = 2 的含义：每个线程只「拥有」2 行，行内 16 列。$n16$ 的 C fragment 里，一个线程的 4 个值的一半（行偏移 +8 的那对）属于另一行；4 个线程（lane 的 id/4 相同、id mod 4 不同）拼成完整的一行：这正是源码里行归约只用 `__shfl_xor_sync` 掩码 1 与 2（后者把 4 个 lane 的两两组合并）的原因。第 二十六 章在手写版同步链里数过的「谁和谁在同一行」，在 FFPA 里被压缩成两个常量。

27.3.3　$S \to P$：一次零拷贝的布局重解释

S（QK 的 C fragment，f32）要变成 PV 的 A 操作数 P（f16）。两步操作（L301–303）：

1. `convert_type<Element>(tCrS)`：按当前布局的连续段做向量化类型转换（f32→f16），元素数与布局都不变；
2. `convert_layout_acc_Aregs<TiledMmaPV>(layout)`：把布局重解释成「$m16n8k16$ 的 A 操作数」布局，**数据一个字节都不搬**。

第一步到第二步的顺序不可交换；重解释本身调用的就是第 二十六 章 25.3.4 节精读过的那个 `convert_layout_acc_Aregs`（Tri Dao 的 `logical_divide` 几何法与 @reed 的 inverse∘compose 代数法都在讲同一件事）。放到 Split-D 的语境里，补三点增量：

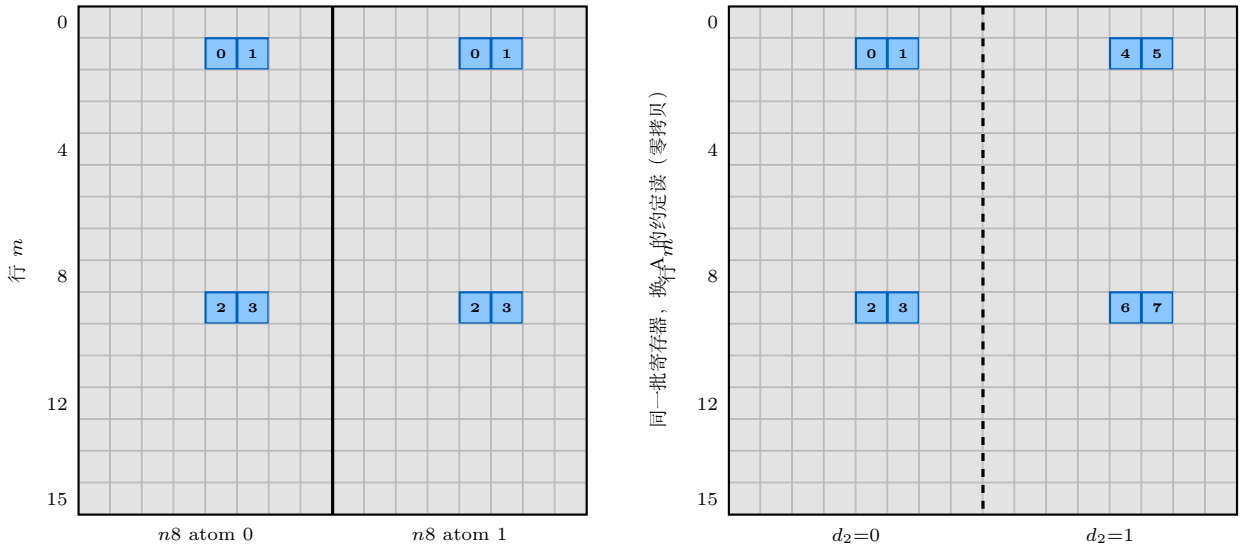
- **为什么必须「就地」**：S 的 32 个寄存器是 Phase 2 最紧张的资源，P 直接复用它们；若改走「P 写回 smem 再 ldmatrix」，每个 kv_tile 多出一次 64×64 的 half 往返与一次同步——Split-D 的 D-chunk 循环让这笔开销按 C 倍累积；
- **为什么模板参数是 `<TiledMmaPV>`**：Aregs 重解释要按消费者的 atom 约定推——PV 的 $k16$ atom 决定「K 向 2 值」落在哪个寄存器编号上，而不是 QK 的约定；
- **数据流本身**：S 的列（B_c 向）在 P 里变成 K 向，而 PV 的归约恰好沿这个方向——P 的「生产者」（softmax）与「消费者」（PV）作用在同一维上，变换只调整「哪个寄存器读哪一对列」，不搬数据。

第二步为什么合法？在 atom 一级看两个 fragment 的线程-值映射（第 二十三 章符号；m 行、k 是 P 的 K 向 $= S$ 的列）：

$$C\ (m16n8): \quad m = \frac{lane}{4} + 8d_1, \quad n = 2\,(lane \bmod 4) + d_0, \qquad v = d_0 + 2d_1, \tag{27.7}$$

$$A\ (m16k16): \quad m = \frac{lane}{4} + 8d_1, \quad k = 2\,(lane \bmod 4) + d_0 + 8d_2, \qquad v = d_0 + 2d_1 + 4d_2. \tag{27.8}$$

把式的 C 映射沿「值」方向拆位：低 2 个值（$v = 0, 1$，即 $(d_0, d_1) = (0,0), (1,0)$）是同一行、相邻两列（$d_0$ 即列 +1）；高 2 个值（$d_1 = 1$）是行 +8 的另一对。而式 (27.8) 的 A 映射里，d_0 同样是「k 向相邻 2 值」。于是「C 的 4 个值」与「A 的 4 个值」在寄存器里一一对应：**A 的 k 向 2 值 = C 的列向 2 值，A 的行维与 C 的行维本来就一致**。第 二十六 章把这条性质记作「不是通用性质，换 atom 必须重推」（源码 L2360 附近的注释）；在 FFPA 里它的意义更重：P 的「K 向」就是 S 的「列向」，而那一维正是 PV 的归约维：**重解释的方向恰好对齐 MMA 的约定，所以不需要 shuffle、不需要过 smem**。图 27.2 把这两条映射放到同一批格子上逐位核对：同一个 lane 在两图里高亮的位置完全相同，变的只是值编号与分组。



C 视图：列 n（S 的 B_c 向）　　A 视图：K 向 k（PV 归约维，一个 $k16$ atom）

C（$m16n8$）：$m = \frac{lane}{4} + 8d_1$、$n = 2\,(lane \bmod 4) + d_0$、$v = d_0 + 2d_1$；A（$m16k16$）：$m$ 同式、$k = 2\,(lane \bmod 4) + d_0 + 8d_2$、$v = d_0 + 2d_1 + 4d_2$（式 (27.7)/(27.8)）。示例 lane= 6：两图高亮的 8 格位置逐格相同（$m \in \{1, 9\}$、列 $\{4, 5\} \cup \{12, 13\}$），C 的值编号在两个 atom 内各自 0..3 循环，A 因多出 d_2 而连续成 0..7——**A 的 K 向 2 值 = C 的列向 2 值，行维本来就一致**。

图 27.2: C 与 A fragment 的线程-值映射逐位对照（式 (27.7)/(27.8)）。左：S 的 C 视图——两个 $n8$ atom 拼成 16 列，格内数字是 lane 6 所持值的编号 $v = d_0 + 2d_1$（在每个 atom 内循环 0..3）；右：P 的 A 视图——同一个 $k16$ atom，lane 6 持有同样的 8 个格，值编号变成 $v = d_0 + 2d_1 + 4d_2$ 连续 0..7，其中 d_2=1 的 4 格来自 C 的第二个 $n8$ atom（虚线半块）。**两图高亮位置逐格相同、数据指针不换**：重解释只把「列向相邻 2 值」的约定换成「K 向相邻 2 值」，行维（m）的映射本来就一致。

注记. 如果 PV 的 atom 换成 $k8$（例如某些 f8 路径），式 (27.8) 的 d_2 项消失、K 向只剩 8 列，S 的 16 列就要拆成两次 PV——布局重解释的「零拷贝」前提随之失效。第 二十 章讨论过的「QK 与 PV 能不能共用一份 P」在类型层的判据，就是这两条映射的匹配。

27.3.4 D-chunk 循环的布局语义

最后一个代数问题是「chunk 循环切开了什么」。先给结论：

Split-D 不改变任何 fragment 的形状，它改变的是循环结构和 归约树。

看 S 的归约：$S = QK^\top$ 的真形式是 $S_{ij} = \sum_{d=0}^{D-1} Q_{id} K_{jd}$。数值上它是「一条长 K 向链」；结构上 FFPA 把它组织成两级：

$$S_{ij} \;=\; \sum_{c=0}^{C-1} \underbrace{\sum_{t=0}^{3} \sum_{s=0}^{15} Q_{i,\,64c+16t+s}\, K_{j,\,64c+16t+s}}_{\text{chunk } c \text{ 内部：4 个} k16 \text{ atom}}, \qquad C = \frac{kHeadDim}{64}. \tag{27.9}$$

式 (27.9) 的两级结构在代码里对应两种不同的机制：

- **chunk 内部**：`gemm_ss` 的 K 循环（第 二十六 章共享层函数），循环上界来自 A fragment 的布局 size $< 2 >$ $(\mathrm{frag}_A) = 4$——*布局决定循环次数*；
- **chunk 之间**：`tCrS` 在 chunk 循环外声明、循环内累加（L218 与 L237–270）——归约体现为「同一个寄存器数组跨迭代存活」。

第二点的意义：S 的 fragment 从头到尾是 32 个寄存器，chunk 循环每轮往它们里面加一棵 QK 子树。这正是「Split-D 只改循环不改形状」的含义，也解释了为什么 O 的处理必须不同：O 的 chunk 是相加（$O = \sum O_c$），但 S 的 chunk 是先求和再 *softmax*，两者不能混。第 二十 章从数值角度论证过这一点（softmax 必须在完整 S 上做），这里从类型角度再确认一遍：S 是「一个 fragment 多次迭代」，O 是「多个 fragment 各存一份」，代码形状（`tCrS` vs `o_acc_storage[v]`）如实反映了这一点。图 27.3 把这组对比画成两棵形状不同的归约树：左边全部 chunk 汇进同一批 32 寄存器，右边每个 chunk 各占一份存储。

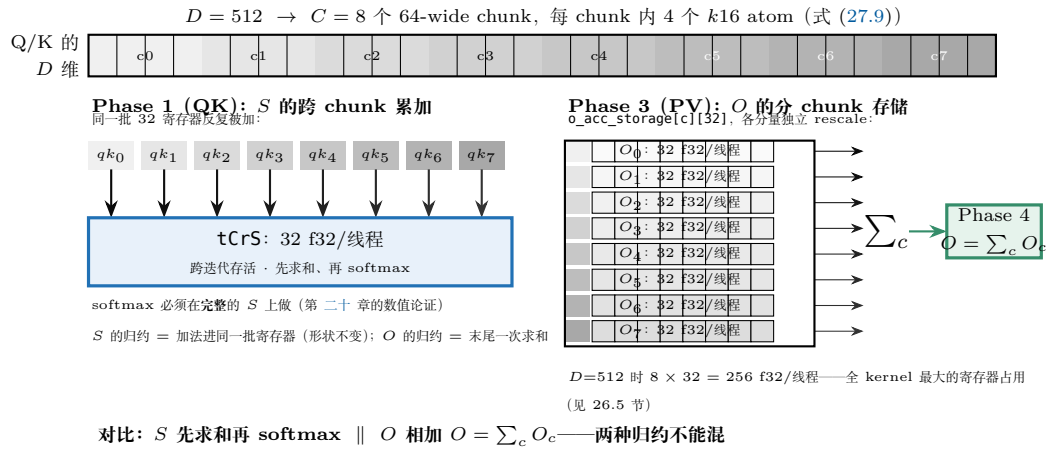


图 27.3: Split-D 改变的是循环结构与归约树，不是 fragment 形状（26.3.4 节）。顶条：$D = 512$ 切成 $C = 8$ 个 64-wide chunk（灰度 8 档贯穿全图），每 chunk 内 4 个 $k16$ atom 对应 `gemm_ss` 的 K 循环。左：8 棵 QK 子树全部加进 **同一个 `tCrS`** fragment——「一个 fragment 多次迭代」，softmax 必须等求和完成；右：每个 chunk 的 PV 结果各存一份 `o_acc_storage[c][32]`（每分量独立 rescale），Phase 4 才相加得 $O = \sum_c O_c$。S 与 O 的 32 值/线程形状完全相同，差别只在「谁在跨迭代累加、谁在分份存储」。

最后是 stage 与 chunk 的正交性。chunk 数 C 是「问题尺寸」（由 head_dim 决定），stage 数 Sk/Sv 是「流水深度」（由 smem 预算决定），两者独立；源码用指针算术而不是 layout 的 stage-mode 来表达：

$$\texttt{q_base} + \texttt{stage} \times \texttt{kQChunkElements}, \qquad \texttt{kQChunkElements} = \text{cosize}(\text{SmemLayoutQ}). \tag{27.10}$$

第 二十 章解释了「不用 stride-0 stage layout」的动机；从布局代数的角度补一句：给 tensor 加一个 stride-0 的伪维后，几乎所有布局变换（`composition`、`coalesce`、`tile_to_shape`）都会把它当作「同一元素重复」处理：`cosize` 不再等于池大小、`partition` 的中间结果也要另算。CuTe 不阻止你写指针加法，但一旦把「物理偏移」当成「布局维度」，第 二十一 章的代数就会被你自己破坏。FFPA 的选择是：**布局只描述一块 chunk，池的偏移交给指针**。

那么「切片 + 流水」在代码里具体长什么样？看 QK 主循环（L237–266）：gmem 侧的 chunk 由 `local_tile` 切出（坐标是 (q_tile, d_chunk)，d 从 0 滚到 Sk−1）；smem 侧每轮把 `q_base` 加上环状槽位偏移 `qk_read` × `kQChunkElements`，重新包成一个 `SmemLayoutQ` 的 tensor。**类型是常量、指针是变量**：每轮更换的只是槽位基址，tensor 的布局不变。随后 `partition_fragment_A` 与 `s2r_thr_q` 的 `partition_S` 在这个「一块 chunk」的视图上照常工作（两者的参数都是 `sQ_stg`）。对照第 二十六 章的三级搭建，那里 smem 是常驻的几段静态视图，这里换成了**按 stage 轮转的池**：对 CuTe 而言两者没有区别，都是「带 layout 的 tensor」；流水的推进则是 `qk_read`/`qk_write` 两个模 Sk 的计数器加 `cp_async_wait<Sk-2>`，与第 二十六 章实现 1 的组栈策略同构（细节见 26.4.5 节）。

27.4　设计与实现：逐相位读 kernel

本节按源码的相位顺序读 `ffpa_split_d_cute`（cp.async 版，128 线程）。算法流程（V 预取 → QK chunk 流水 → softmax → PV chunk 流水 → 归一化写回）与第 二十 章一致；本节只标出**类型层的要点**。

27.4.1　traits 与 chunk 常量

先看 traits 的核心六行（L50–61），它们的类型推导在 26.3.1 节已经算过：

ffpa_attn.cuh L50-61（MmaAtom、双 TiledMma、两个 copy atom）

```
// QK: Tile<64,64,16> → EURepeat<1,8,1>, 一次覆盖 S[64,64]
using TiledMmaQK = decltype(make_tiled_mma(
    MmaAtom{}, Layout<Shape<_4, _1, _1>>{},
    Tile<Int<TILE_M>, Int<TILE_N>, _16>{}));

// PV: Tile<64,16,16> → EURepeat<1,2,1>, 控制 acc_O 寄存器
using TiledMmaPV = decltype(make_tiled_mma(
```

```
      MmaAtom{}, Layout<Shape<_4, _1, _1>>{},
      Tile<Int<TILE_M>, _16, _16>{}));

  using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
  using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
```

再看 kernel 的入口与 chunk 常量（L80–97）。这一段值得逐行读，因为它把「一个 chunk 有多大」的全部依赖固定在编译期：

ffpa_attn.cuh L80-97（128 线程的 launch bounds 与 chunk 常量：kDChunks 由 head_dim 推出）

```
template <int kHeadDim, int kStagesQK = 2, int kStagesV = 2>
__global__ void __launch_bounds__(128)
ffpa_split_d_cute(
    cutlass::half_t *Q, cutlass::half_t *K, cutlass::half_t *V,
    cutlass::half_t *output, int rows, int seqlen) {
  using namespace cute;
  using Traits = fa_cute::FFPAAttnSplitDCuTeTraits<kHeadDim>;
  using Element = typename Traits::Element;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  static_assert(kHeadDim % 64 == 0, "Split-D requires head-dim multiple of 64");
  static_assert(kStagesQK >= 1 && kStagesV >= 1);

  constexpr int kBr = 64;
  constexpr int kBc = 64;
  constexpr int kDChunk = 64;
  constexpr int kDChunks = kHeadDim / kDChunk;
  constexpr int kQChunkElements = cosize(SmemLayoutQ{});
```

三处类型要点：

- `__launch_bounds__(128)` 不是任意的数字，它对应 TiledMma 的 AtomLayout = $(4,1,1)$——4 个 warp × 32 lane = 128。改 Tile 的 M 向分工（例如换 $(2,1,1)$）而忘了同步改 launch bounds，kernel 里所有 `get_thread_slice(tid)` 的语义都会静默错位；
- `kDChunks = kHeadDim / 64`：chunk 宽度 64 不是可选参数，它就是 SW128 atom 一行（64 个 half = 128 B）的宽度。第 二十 章从 swizzle 的角度解释过；类型上，它与 `SmemLayoutAtom` 是同一个常量；
- `kQChunkElements = cosize(SmemLayoutQ{})`：池大小从 layout 推出而不是手写。cosize 对 SW128 布局的实测值是 $64 \times 64 = 4096$（第 二十六 章的 probe 同款），于是三块池的基址（L100–103）也是从它算出来的。

实践须知 （读这份源码时最容易踩的三个认知坑）：

- **头注释的 kernel 清单不全**。`ffpa_attn.cuh` L5–6 写「仅定义……`ffpa_attn_tma_mma_ws_split_d_cute` kernel」，但文件里实际有**两个** kernel：本节精读的 `ffpa_split_d_cute`（cp.async 版，L82）与 `ffpa_attn_tma_mma_ws_split_d_cute`（TMA+WS 版，L378 起）。读文件时以实际代码为准；
- 「**256 线程**」**只对 TMA+WS 版成立**。L14 与 L28 的「128 producer + 128 consumer = 256 total threads」说的是 **TMA+WS 版**（与第 二十 章 TMA+WS 版的划分一致）；cp.async 版是 **单角色 128 线程**（L73–78 的注释自己写明了）。按 256 去数 cp.async 版的 warp 会数错——本节的类型推导统一以 L81 的 `__launch_bounds__(128)` 为准；
- **EURepeat 是注释词汇、不是接口**。与第 二十六 章同族：CuTe 公开可断言的是 `tile_size_mnk` 与 fragment 布局，「EURepeat<1,8,1>/<1,2,1>」只是「tile 除以 (atom⊗layout)」那一刻的中间量（26.3.1 节已按这个口径核对）。

27.4.2 G2S `TiledCopy`：与 MMA 划分对齐

搬运的线程划分（L118–128）是本章第一个「对齐」要点：

ffpa_attn.cuh L118-128（G2S：128-bit cp.async，线程布局 16×8，值布局 1×8）

```

// G2S TiledCopy: 128-bit cp.async, 128 threads (16×8), 对齐 MMA 128 线程
using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
using g2s_copy_atom = Copy_Atom<Copy_Traits<g2s_copy_op>, Element>;
using G2SCopy = decltype(make_tiled_copy(
    g2s_copy_atom{},
    make_layout(make_shape(Int<16>{}, Int<8>{}),
                make_stride(Int<8>{}, Int<1>{})),
    make_layout(make_shape(Int<1>{}, Int<8>{}))));
G2SCopy g2s_copy;
auto g2s_thr = g2s_copy.get_slice(tid);
```

`make_tiled_copy` 的两个 layout 分别是「线程布局 $(16,8):(8,1)$」（128 个线程铺满，行优先）与「值布局 $(1,8)$」（每线程一次搬 8 个 half = 128 bit，正好是 `cp.async.cg.shared.global` 的 16 字节形态）。但源码注释（L73–78）里有一句关键的话：

> *128 线程：与 TiledMmaQK/PV 的 Layout⟨4, 1, 1⟩ 一致，每个线程在 G2S 和 S2R/MMA 中有唯一分区，消除 256-thread G2S 与 128-thread MMA 之间的映射不匹配。*

这是从「384/256 线程的多角色 kernel」演化到「128 线程单角色」的直接证据：线程数一旦与 TiledMma 的 warp 划分一致，G2S 的 `tid` 与 MMA 的 `tid` 就是同一个坐标系，`partition_S` / `partition_D` 的转换全部退化。代价是搬运带宽降低一半（128 线程 vs 256），收益是零映射开销：第 二十 章的性能节从吞吐角度讨论过这项开销。

27.4.3　V 的转置视图与 fragment 预计算

PV 要在循环里反复构造 V 的 B 操作数。FFPA 的做法是**把 fragment 布局在循环外算好**（L136–141）：

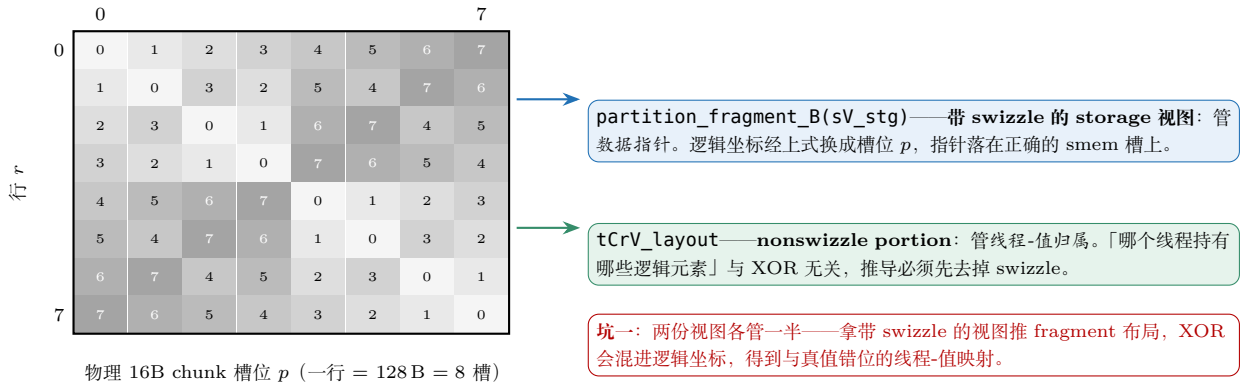
ffpa_attn.cuh L136-141（V 的 nonswizzle 转置视图与 fragment 布局预计算）

```
// V layout for gemm_rs (与 TMA WS kernel 完全一致)
auto sV0 = make_tensor(make_smem_ptr(v_base), SmemLayoutKV{});
auto sVt0_ns = make_tensor(
    sV0.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));
auto tCrV_layout = thr_mma_pv.partition_fragment_B(sVt0_ns).layout();
```

三行代码，三个层次：

1. `sV0`：smem 上的 *storage* 视图（带 SW128 swizzle 的 $[64,64]$ 布局）；
2. `sVt0_ns`：取 `SmemLayoutVt` 的 **nonswizzle portion**——即去掉 swizzle 后的「纯行列」布局。为什么要去掉？因为 `partition_fragment_B` 推的是「哪个线程持有 B 的哪个逻辑元素」——swizzle 只改变元素的地址，不改变归属；带着 swizzle 推会让 XOR 置换混进逻辑坐标，得到的 fragment 布局与真值错位。这是第 二十六 章「常见坑」的同族问题的另一面（那里禁止拿带 swizzle 的视图推寄存器布局）；
3. `tCrV_layout`：把上面那份推导求值成一个布局对象，循环里只需 `make_tensor(数据指针, tCrV_layout)` 重新解释。

图 27.4 把这两份视图的分工画在同一张 smem tile 上：底色是 SW128 的 XOR 置换模式，两条引出箭头分别指向「管指针」与「管归属」的两个角色。



物理 16B chunk 槽位 p（一行 $= 128\,\mathrm{B} = 8$ 槽）

SW128 的位公式（第 二十四 章）：$\mathrm{phys} = 64r + 8 \cdot ((c/8) \oplus (r \bmod 8)) + (c \bmod 8)$ ——行内 16 B chunk 号与行号低 3 位异或（8 行一个周期）。格内数字是该槽实际承载的逻辑 chunk（$c/8 = p \oplus (r \bmod 8)$），灰度跟随逻辑 chunk 身份，于是「对角线置换」清晰可见；实际 tile 是 64×64（8 个周期），此处只画一个周期。

图 27.4: V 的双视图分工。同一张 SW128 smem tile：`sV_stg`（带 swizzle 的 storage 视图）负责把逻辑坐标换算成物理槽位、给数据指针用；`tCrV_layout`（由 nonswizzle portion 推导）负责回答「哪个线程持有哪些逻辑元素」。SW128 的 XOR 只改地址、不改归属，所以推 fragment 布局时必须先去掉 swizzle——两份视图各管一半，混用即错（坑一）。

还有一个容易忽略的点：B fragment 的布局对 4 个 M-warp 是重复的（K 向的 4 个 atom 都要被每个 warp 看到），所以 `partition_fragment_B` 在 128 线程上给出的每线程元素数（≥ 128）远大于 C 的 32：这部分寄存器是「共享副本」而非私有状态，读寄存器压力时不能把它算进 `exp2` 这类逐元素操作的成本里。

27.4.4 寄存器占用：$32 \times C$ 与两个行向量

源码（L152–176）把每个寄存器都写在类型上：

ffpa_attn.cuh L152-176（O fragment 的尺寸、rowcol 行列数、行状态与 C 份累加器）

```
  using OFragType = decltype(partition_fragment_C(tiled_mma_pv, Shape<_64, _64>{}));
  using OFragLayout = typename OFragType::layout_type;
  constexpr int kOElemsPerFrag = decltype(size(OFragType{}))::value;
  constexpr int kORows = decltype(size<0>(make_tensor(
      (float*)nullptr, fa_cute::convert_layout_acc_rowcol(OFragLayout{}))))::value;
  constexpr int kOCols = decltype(size<1>(make_tensor(
      (float*)nullptr, fa_cute::convert_layout_acc_rowcol(OFragLayout{}))))::value;

  // Online softmax persistent state (与 TMA WS kernel 完全一致)
  float row_max[kORows];
  float row_sum[kORows];
#pragma unroll
  for (int r = 0; r < kORows; ++r) {
    row_max[r] = -INFINITY;
    row_sum[r] = 0.0f;
  }
  const float scale = rsqrtf(static_cast<float>(kHeadDim)) * M_LOG2E;

  // Per-v_chunk register O accumulators (与 TMA WS kernel 完全一致)
  float o_acc_storage[kDChunks][kOElemsPerFrag];
#pragma unroll
  for (int v = 0; v < kDChunks; ++v)
#pragma unroll
    for (int i = 0; i < kOElemsPerFrag; ++i)
      o_acc_storage[v][i] = 0.0f;
```

逐项列出（本节测试在 host 端断言的正是这些数）：

- `kOElemsPerFrag`$= |\mathrm{frag}_C((64, 64))| = 32$：由式 (27.4) 的恒等式一决定；
- `kORows`$= 2$、`kOCols`$= 16$：由式 (27.6) 的 rowcol 换算决定——**行状态只有 2 个 f32**，因为「行」在 4 个 lane 间

是共享的（26.3.2 节）；

- `o_acc_storage[C][32]`：C 份 O 分量，$D=512$ 时 $8\times 32=256$ 个 f32——这是全 kernel 最大的寄存器占用。它与第 二十 章「O 累加器的寄存器占用」是同一项占用，这里的类型视角是：*数组的第二维是 fragment 布局给的，第一维是 chunk 循环给的*——两个维度一个来自 TiledMma、一个来自问题尺寸，恰好正交。

27.4.5　Phase 1：QK 的 chunk 流水

QK 段（L217–270）是整个 kernel 最能体现「循环结构 vs 类型」的地方。先看初始化（L217–235）：

ffpa_attn.cuh L217-235（Phase 1：`tCrS` 声明于循环外 + 前 Sk−1 个 chunk 预取）

```
    // ===== Phase 1: QK with Split-D =====
    auto tCrS = partition_fragment_C(tiled_mma_qk, Shape<_64, _64>{});
    clear(tCrS);

    int qk_read = 0;
    int qk_write = 0;

    // 初始 prefetch: 前 kStagesQK-1 个 (Q,K) pair
#pragma unroll
    for (int d = 0; d < kStagesQK - 1 && d < kDChunks; ++d) {
      g2s_load_q(d, qk_write);
      g2s_load_k(kv_tile, d, qk_write);
      cp_async_fence();
      qk_write = (qk_write + 1) % kStagesQK;
    }
    if constexpr (kStagesQK > 1) {
      cp_async_wait<kStagesQK - 2>();
      __syncthreads();
    }
```

再看主循环（L237–270）：

ffpa_attn.cuh L237-270（QK main loop：先发下一 chunk、再算当前 chunk，`tCrS` 跨迭代累加）

```
    // QK main loop
    for (int d_chunk = 0; d_chunk < kDChunks; ++d_chunk) {
      int d_next = d_chunk + kStagesQK - 1;
      if (d_next < kDChunks) {
        g2s_load_q(d_next, qk_write);
        g2s_load_k(kv_tile, d_next, qk_write);
        cp_async_fence();
        if constexpr (kStagesQK > 1) {
          cp_async_wait<kStagesQK - 2>();
        } else {
          cp_async_wait<0>();
        }
        __syncthreads();
        qk_write = (qk_write + 1) % kStagesQK;
      }

      // QK GEMM: S += Q[d_chunk] @ K[d_chunk]^T
      auto sQ_stg = make_tensor(
          make_smem_ptr(q_base + qk_read * kQChunkElements), SmemLayoutQ{});
      auto sK_stg = make_tensor(
          make_smem_ptr(k_base + qk_read * kKVChunkElements), SmemLayoutKV{});
      auto tCrQ = thr_mma_qk.partition_fragment_A(sQ_stg);
      auto tCrK = thr_mma_qk.partition_fragment_B(sK_stg);
      auto tQsQ_s2r = s2r_thr_q.partition_S(sQ_stg);
      auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);
      fa_cute::gemm_ss(tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
```

```
                      tiled_mma_qk, s2r_copy_q, s2r_copy_k,
                      s2r_thr_q, s2r_thr_k);
      __syncthreads();
      qk_read = (qk_read + 1) % kStagesQK;
    }
    if constexpr (kStagesQK > 1) {
      cp_async_wait<0>();
      __syncthreads();
```

三点解读：

1. `tCrS` 在 chunk 循环外声明（L218）、循环内通过 `gemm_ss` 累加。式 (27.9) 的两级归约在代码里就是「同一个 fragment 跨迭代存活」——类型全程是 $((2,2),1,8)$，32 个寄存器；
2. 每轮先发下一个 chunk 的 Q/K（`d_next`），再等、再算当前——流水方向与 chunk 方向一致。等待用 `cp_async_wait<Sk-2>` 保持「最多 Sk 个组挂起」，与第 二十六 章实现 1 的组栈策略同构；
3. 循环末尾的 `__syncthreads()` 之后，`qk_read` 前进一格——smem 槽位的复用边界。类型上这里没有任何「chunk」痕迹：`gemm_ss` 处理的永远是「一份 A fragment × 一份 B fragment，K 向 4 个 atom」——**chunk 循环在共享层函数里不可见**，这正是把 `gemm_ss` 抽成工具函数的价值。

27.4.6 Phase 2：softmax 与 P 的生成

ffpa_attn.cuh L273-303（Phase 2：rowcol 视图上的 online softmax + `convert_type` 与 `Aregs`）

```
    // ===== Phase 2: Online softmax (与 TMA WS kernel 完全一致) =====
    auto scores = make_tensor(
        tCrS.data(), fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
    float row_scale[kORows];
#pragma unroll
    for (int row = 0; row < kORows; ++row) {
      float tile_max = -INFINITY;
#pragma unroll
      for (int col = 0; col < size<1>(scores); ++col)
        tile_max = fmaxf(tile_max, scores(row, col) * scale);
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
      const float next_max = fmaxf(row_max[row], tile_max);
      row_scale[row] = exp2f(row_max[row] - next_max);
      float tile_sum = 0.0f;
#pragma unroll
      for (int col = 0; col < size<1>(scores); ++col) {
        const float p = exp2f(scores(row, col) * scale - next_max);
        scores(row, col) = p;
        tile_sum += p;
      }
      tile_sum += __shfl_xor_sync(0xffffffff, tile_sum, 1);
      tile_sum += __shfl_xor_sync(0xffffffff, tile_sum, 2);
      row_sum[row] = row_sum[row] * row_scale[row] + tile_sum;
      row_max[row] = next_max;
    }

    // P fragment: convert_layout_acc_Aregs<TiledMmaPV> (与 TMA WS kernel 一致)
    auto tCrP = fa_cute::convert_type<Element>(tCrS);
    auto tCrPv = make_tensor(
        tCrP.data(),
```

这段的每一行都值得对照第 二十六 章实现 1 的 softmax 段读：两者几乎逐字相同，因为 FFPA 复用同一套「rowcol 视图 + 4-lane `shfl_xor` 归约」的做法。其中三处是 FFPA 特有的：

- `scores` 是 S 的 rowcol 视图，`scores(row,col) = p` 是**就地覆盖**——P 直接复用 S 的 32 个寄存器（先 `exp2`

再转换类型，中间不另开数组）。这是寄存器占用里「P 不额外占位」的实现依据；

- `row_scale[row] = exp2f(row_max[row] - next_max)` 是本轮的修正因子，它马上要被 Phase 3 用来 rescale O 的第 `v_chunk` 分量——softmax 与 PV 的耦合就靠这个长度 2 的小向量；
- 段的末尾（L301–303）是 $S \to P$ 的两步搬运——代数在 26.3.3 节，这里只强调顺序：**先 `convert_type`、再 `Aregs`**。反过来的话，类型转换的向量化会按错误的连续段分组，f16 打包的元素与 MMA 期待的 A 布局错位（第二十六章坑七的同一条，这里是它在 Split-D 里的位置）。

27.4.7　Phase 3：PV 的 chunk 循环与两个 TiledMma 的交接

PV 段（L307–353）分两半。前半是 chunk 流水与 O 的 rescale：

ffpa_attn.cuh L307-331（Phase 3 前半：`v_write_pv` 与 Phase 0 对接；O_c 的 rescale）

```
    // ===== Phase 3: PV with Split-D =====
    int v_write_pv = (kStagesV > 1) ? (kStagesV - 1) : 0;
    int v_read = 0;
    for (int v_chunk = 0; v_chunk < kDChunks; ++v_chunk) {
      int v_next = v_chunk + kStagesV - 1;
      if (v_next < kDChunks) {
        g2s_load_v(kv_tile, v_next, v_write_pv);
        cp_async_fence();
        if constexpr (kStagesV > 1) {
          cp_async_wait<kStagesV - 2>();
        } else {
          cp_async_wait<0>();
        }
        __syncthreads();
        v_write_pv = (v_write_pv + 1) % kStagesV;
      }

      // Rescale O accumulator (与 TMA WS kernel 完全一致)
      auto tCrO = make_tensor(make_rmem_ptr(&o_acc_storage[v_chunk][0]),
                              OFragLayout{});
      if (kv_tile > 0) {
        auto tCrO_rc = make_tensor(
            tCrO.data(), fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
#pragma unroll
        for (int row = 0; row < kORows; ++row)
```

注意 `v_write_pv = (kStagesV > 1) ? (kStagesV - 1) : 0` 这个初值：Phase 0 已经预取了 Sv − 1 个 chunk，PV 循环延续同一套环状槽位：**两个相位共用同一组槽位**，改任何一个都要改另一个（26.7 节坑四）。rescale 的操作对象是 `tCrO` 的 rowcol 视图：行方向乘 `row_scale[row]`，与 softmax 的行状态同一坐标系，行方向「每线程 2 行」的约定再次复用。

后半是交接的核心（L330–352）：

ffpa_attn.cuh L330-352（`gemm_rs` 的四个操作数：O（C）、P（A，Aregs 布局）、V（B，预计算布局）、$V^\top$（S2R 视图））

```
#pragma unroll
        for (int row = 0; row < kORows; ++row)
#pragma unroll
          for (int col = 0; col < kOCols; ++col)
            tCrO_rc(row, col) *= row_scale[row];
      }

      // gemm_rs: O[v_chunk] += P @ V[v_read] (与 TMA WS kernel 完全一致)
      auto sV_stg = make_tensor(
          make_smem_ptr(v_base + v_read * kKVChunkElements), SmemLayoutKV{});
      auto sVt_stg = make_tensor(sV_stg.data(), typename Traits::SmemLayoutVt{});
      auto tCrVStorage = thr_mma_pv.partition_fragment_B(sV_stg);
```

```
      auto tCrV = make_tensor(tCrVStorage.data(), tCrV_layout);
      auto tVsVt = s2r_thr_v.partition_S(sVt_stg);
      fa_cute::gemm_rs(tCrO, tCrPv, tCrV, tVsVt,
                       tiled_mma_pv, s2r_copy_v, s2r_thr_v);
      __syncthreads();
      v_read = (v_read + 1) % kStagesV;
    }
    if constexpr (kStagesV > 1) {
      cp_async_wait<0>();
    }
    __syncthreads();
```

四行构造 `gemm_rs` 的操作数，每行对应 26.3 节的一个类型事实：

代码	类型事实	出处
`tCrO`	PV 的 C fragment（32 值，Aregs 无关）	式 (27.4)
`tCrPv`	QK 的 C 经 Aregs 重解释（零拷贝）	式 (27.7)–(27.8)
`tCrV`	B fragment 用预计算的 nonswizzle 布局	26.4.3 节
`tVsVt`	S2R 的 smem 视图走 $V^\top$ + `LDSM_T`	第 二十 章

其中 `tCrVStorage` 与 `tCrV` 的「双视图」是最容易看漏的一处：`partition_fragment_B(sV_stg)` 用带 *swizzle* 的 *storage* 视图只是为了拿到「数据指针的划分」；元素的解释立刻换回预计算的 `tCrV_layout`（来自 nonswizzle 推导）。前者保证指针落在正确的 smem 槽位，后者保证逻辑坐标与 MMA 的约定一致：**两份视图各管一半，混用任何一份都会错**。

27.4.8 Phase 4：归一化与写回

收尾（L355–374）没有引入新的机制，但它的布局复用需要单独说明：

ffpa_attn.cuh L355-374（Phase 4：每个 chunk 独立归一化 + R→G 写回）

```
  // ===== Phase 4: Final normalize + store (与 TMA WS kernel 完全一致) =====
#pragma unroll
  for (int v_chunk = 0; v_chunk < kDChunks; ++v_chunk) {
    auto tCrO = make_tensor(make_rmem_ptr(&o_acc_storage[v_chunk][0]),
                            OFragLayout{});
    auto tCrO_rc = make_tensor(
        tCrO.data(), fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
#pragma unroll
    for (int row = 0; row < kORows; ++row) {
      const float inv_sum = 1.0f / row_sum[row];
#pragma unroll
      for (int col = 0; col < kOCols; ++col)
        tCrO_rc(row, col) *= inv_sum;
    }
    auto tCrOHalf = fa_cute::convert_type<Element>(tCrO);
    auto gO = local_tile(mO, Shape<_64, _64>{}, make_coord(q_tile, v_chunk));
    auto tCgO = thr_mma_pv.partition_C(gO);
    copy(tCrOHalf, tCgO);
  }
}
```

- 循环变量 `v_chunk` 同时是 O 累加器的下标与 `local_tile` 的 D 坐标——chunk 索引从寄存器编号一路贯穿到输出地址，这是「chunk 是问题尺寸的一等公民」的最后证据；
- `partition_C(gO)` 复用了 PV 的 TiledMma——O 的写回布局与 O 的累加布局是同一个类型，无需任何转换；
- `convert_type<Element>` 按 rowcol 视图做完归一化后的布局做向量化 f16 转换——与 Phase 2 的转换是同一个工具函数、不同布局，再次印证共享层的设计。

27.5　图示：双 TiledMma 的数据流

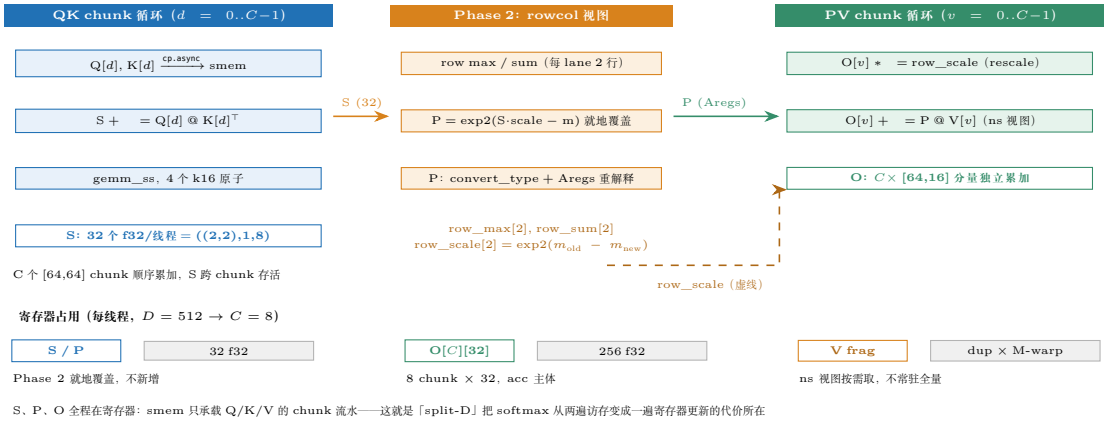


图 27.5: 双 TiledMma 的数据流：QK 沿 D 累加成 S（32 寄存器跨 chunk 存活），softmax 就地覆盖为 P，Aregs 重解释后交给 PV；O 的每个 chunk 分量独立 rescale 与累加，`row_scale`（虚线）耦合两个循环。底部为寄存器占用：S/P 32、$O[C][32]$ 共 256（C=8）、V fragment 按 M-warp 重复——smem 只承载 Q/K/V 的 chunk 流水。

27.6　性能分析

本章不给新的实测数字，原因与第 二十六 章相同：本机没有复测 FFPA 的完整 bench（第 二十 章给出其性能数据与口径）。这里只做两件事：转录源码注释里的既有声明（注明出处），并从本章的「类型视角」讨论 Split-D 抽象层的运行时成本结构。

源码头注释（`ffpa_attn.cuh` L16–17）的性能声明是：

> *性能 (B=1,H=32,N=8192,D=512, SM120a RTX PRO 5000): cuDNN SDPA: 57.2 TFLOPS | FFPA TMA WS: 110.7 TFLOPS (1.94x)*

读数时注意三点口径：（1）这是 **TMA + WS 版**（256 线程、双角色）的数字，不是本章精读的 cp.async 版；（2）设备标称 SM120a RTX PRO 5000，而本仓库 README 的同类标称常写 RTX 5090，同一 sm120 家族的两种卡，绝对值不可直接互比（第 二十 章的核对与 CHECKLOG 记录同此）；（3）「1.94x」是相对 cuDNN SDPA 的加速比，比较对象与精度配置（f16 输入、f32 累加）需与 bench 脚本核对后才可引用。

从类型视角看，「Split-D 的抽象开销」可以逐项归零：

- `kDChunks`、`kQChunkElements` 等全是 `constexpr`——chunk 循环的步长、stage 偏移、池大小都在编译期折叠成立即数，运行时（在开启优化后）与手写版本无差别；
- `local_tile`/`partition_*` 的坐标算术全部由模板推导，编译产物里剩下的是少数 `mad` 与常量偏移；
- 实际的运行时成本在算法侧（chunk 循环的额外 barrier、V 的重复加载），与「用不用 CuTe」无关——第 二十 章对这两项开销的量化结论是「Stage 上限压到 2–3，同步开销主导」。

CuTe 在 Split-D 上的收益与代价都不在运行时，而在「类型可断言」：本章测试（26.9 节）能在不启动 kernel 的情况下核对双 TiledMma 的每条推导，这在手写版里是做不到的。

27.7　常见坑

坑一：推 V 的 fragment 布局必须去掉 swizzle。`partition_fragment_B` 要的是「值的线程归属」，而 swizzle 只改变元素的 smem 地址。直接用带 SW128 的 `SmemLayoutVt` 去推，XOR 置换会混进逻辑坐标，fragment 布局与真值错位：正确做法是 `get_nonswizzle_portion` 之后再推（L137–141）。数据指针仍走 带 *swizzle* 的 storage 视图：两份视图各管一半。

坑二：`convert_type` 与 `Aregs` 的先后顺序。必须先类型转换、再布局重解释（L301–303）。反序时，f16 的打包与 `NumericArrayConverter` 的向量化会按错误的连续段分组。

坑三：`o_acc_storage` 的初始化在 chunk 循环外、只做一次。它在每个 kv_tile 迭代里都是累加目标，清零只在 kernel 开头（L174–176）。若把它挪进 kv_tile 循环，O 会丢失历史。

坑四：`v_write_pv` 的初值必须与 Phase 0 的预取数对接。 Phase 0 写了 $\mathrm{Sv}-1$ 个 chunk，PV 循环从槽位号 $(\mathrm{Sv}-1) \bmod \mathrm{Sv}$ 继续（L308）。改 Phase 0 的预取个数而忘了这里，会写坏正在被读的 chunk。

坑五：chunk 循环里的 `__syncthreads()` 属于槽位复用协议。 QK 每轮末尾（L269）与 PV 每轮末尾（L350）的同步不是可有可无的：它们保证「本轮读过的 smem 槽位」在下一轮被覆写之前所有线程都已读完；删掉它，表现是高频偶发数值错误，且随 stage 数增大而加剧。

坑六：行状态只有 2 个，但列有 16 个：shfl 掩码只到 2。 `row_max`/`row_sum` 的长度是 `kORows`$= 2$，行内归约靠 `__shfl_xor_sync` 的掩码 1 与 2（4 个 lane 一组，L281–282 与 L291–292）。多加一层掩码 4 会把另一个 *warp* 段的行卷进来（warp 内 8 个 lane 组对应 8 个不同行块），数值立刻错。

坑七：chunk 宽度、smem 池、TMA box 三者必须同步改。把 `kDChunk` 从 64 改成别的值时，SW128 原子、TMA box （`Shape<_64,_64>`）、`kDChunks` 与 V 的 fragment 预计算全部要跟着动：「一个 64 改三处」是这一族 kernel 改动时的固定代价。

27.8 面试要点速查

1. **为什么 QK 的 TiledMma 要 Tile**$\langle 64,64,16\rangle$**，而 PV 只要** $\langle 64,16,16\rangle$**？** QK 的 N 向 EURepeat 变成 8，一次覆盖 $S[64,64]$，省掉 N-tile 循环（提升计算密度）；PV 的累加器 O 要与 chunk 循环的边界对齐，小 tile 让 chunk 边界成为寄存器域的边界。两者的 C fragment 大小相同（32），因为输出形状与线程数相同。

2. 「**Split-D 不改变 fragment 形状**」**是什么意思？** S 的 fragment 在 chunk 循环前后都是 $((2,2),1,8)=32$ 个 f32；chunk 循环表达的是「同一个数组跨迭代累加」。变的只是循环结构与归约树。

3. S **怎么变成** P**？为什么不需要 shuffle/smem？** `convert_layout_acc_Aregs`：$m16n8$ 的 C 与 $m16k16$ 的 A 在寄存器编号上一一对应（C 的列向 2 值 = A 的 K 向 2 值），零拷贝重解释。换成 $k8$ 的 atom 就要重推。

4. **为什么 V 的 fragment 布局要预先算、且用 nonswizzle 版本？**循环里只换指针不改布局——算一次省每轮推导；去掉 swizzle 是因为它只改地址不改值的线程归属。

5. **每个线程为** O **花多少寄存器？** $C\times 32$：$D=512$ 时 256 个 f32。第一维来自 chunk 循环（问题尺寸），第二维来自 fragment 布局（TiledMma）——两个来源正交。

6. **行归约为什么要用到 `shfl_xor` 掩码 1 和 2？** $m16n8$ 的 C fragment 里一行由 4 个 lane 拼成（每 lane 2 行 × 16 列），掩码 1 与 2 恰好把 4 个 lane 的两两组合并；再多就跨行块了。

7. **smem 池的偏移为什么用指针算术而不是 layout 的 stage 维？** stride-0 的伪维会被 CuTe 的布局代数当作「同一元素重复」，`cosize`/partition 的结果全部失真。布局只描述一块 chunk，池偏移交给指针。

8. **chunk 数与 stage 数是什么关系？**正交：chunk 数是问题尺寸（head_dim/64），stage 数是流水深度（smem 预算）。两者的相位关系与约束见第 二十 章。

27.9 最小测试与动手实验

本章测试（`tests/ch26_ffpa_cute.cu`）与前几章的不同之处是 **以 host 端类型断言为主**：不启动 kernel，只对 traits 与 fragment 做静态/运行时断言，全部来自 26.3 节的代数。凡是「实测」二字，都指该测试或等价 probe 的 host 端输出：

1. **D-chunk 常量**：对 $D\in\{64,128,192,512\}$ 断言 `kDChunks`$= D/64$ 与两个 chunk 池 cosize $= 64\times 64$；
2. **双 TiledMma 推导链**：QK 的 `tile_size_mnk` $=(64,64,16)$、PV $=(64,16,16)$，线程数均为 128；
3. **fragment 恒等式**：两个 `partition_fragment_C`$((64,64))$ 的每线程元素数都是 32；rowcol 后的行列数 $(2,16)$；Aregs 重解释前后元素数守恒；

4. **switch 语义**：`local_tile` 在 $(64,64)$ tile 与 (row_tile, d_chunk) 坐标下的首元素地址等于 `row_tile*64*D + d_chunk*64`。

   再加两个小规模 kernel 正确性 case（与第 二十 章的覆盖互不重复）：

- case E：$D = 192$（3 个 chunk，且与 Sk=3 相等——测「chunk 数 = 预取数」的边界）、$N = 128$、$H = 2$、Sk=3, Sv=2；
- case F：$D = 128$（2 个 chunk，Sk=2）、$N = 128$、$H = 2$。

  两者都与 CPU fp64 参考对照（容差 `TOL_F16ACC`），运行：

```
cd book/tests && CUDA_VISIBLE_DEVICES=6 ./build_tests.sh --arch sm_120a --ch ch26
```

本机实测输出（sm_120a，6 号卡）：

```
PASS case A chunk constants D=64 (kDChunks=1): mismatches=0 (4 checks)
PASS case A chunk constants D=128 (kDChunks=2): mismatches=0 (6 checks)
PASS case A chunk constants D=192 (kDChunks=3): mismatches=0 (8 checks)
PASS case A chunk constants D=512 (kDChunks=8): mismatches=0 (18 checks)
PASS case B dual TiledMma derivation chain (D=512): mismatches=0 (6 checks)
probe: rowcol=(2,16) aregs_elems=32
PASS case B2 device probe: rowcol (2,16) & Aregs 32 elems: mismatches=0 (3 checks)
PASS ffpa split-d cute D=192 N=128 Sk=3 Sv=2 B=1 H=2: max_abs_err=8.481e-05 (tol=5.0e-02)
PASS ffpa split-d cute D=128 N=128 Sk=2 Sv=2 B=1 H=2: max_abs_err=8.084e-05 (tol=5.0e-02)
ALL OK
```

延伸阅读

- @ 竹熙佳处《写给大家看的 CuTe 教程：Layout Compose & Inverse》——$S \to P$ 布局变换的完整讲法（Tri Dao 的几何法与 @reed 的代数法），本章 26.3.3 节与第 二十六 章 25.3.4 节的主参考。
  https://zhuanlan.zhihu.com/p/1962625273636845008
- @reed《cute 之高效 GEMM 实现》——TiledMMA 的组装层次、多级流水的写法与 Epilogue 的 smem 中转；同系列《cute 之简单 GEMM 实现》给出最小版本的组装范式，配合 26.3.1 的推导链与 26.4 的相位阅读一起看。
  https://zhuanlan.zhihu.com/p/675308830
- @ 竹熙佳处《写给大家看的 CuTe 教程：tiled mma》——fragment 视角的线程-值映射，26.3.2 节两个恒等式的入门读物。
  https://zhuanlan.zhihu.com/p/1937145378446226159
- @DefTruth《FFPA(Split-D)：FA2 无限 HeadDim 扩展》——本主题在中文社区最系统的公开文章，Split-D 的动机与 RD 波形分析可与第 二十 章的推导对读。
  https://zhuanlan.zhihu.com/p/13975660308
- @DefTruth《图解：从 Online-Softmax 到 FlashAttention V1/V2/V3》——online softmax 的图解（本书第 十六 章的主参考），配合本章 26.3.2 的 rowcol 代数读更佳。
  https://zhuanlan.zhihu.com/p/668888063
- @66RING《使用 cutlass cute 复现 flash attention》——CuTe 版 FA 的工程化复现，可与第 二十六 章的四个实现对照。
  https://zhuanlan.zhihu.com/p/696323042
- @DeclK《CUTLASS CUTE 系列（1–5）》——布局代数与 MMA/COPY 抽象的系统性替代读物，作为第 二十一–二十三 章的复习材料。
  https://zhuanlan.zhihu.com/p/1983922984805742098

第二十八章　CuTe 应用（四）：SM120 持久化 FlashAttention——超越 cuDNN

28.1　本章导读

第 二十六 章结束时，我们已经拿到三个 CuTe 版 FlashAttention：cp.async 统一流水、TMA + 单 consumer warp specialization、FA3 式双 consumer。它们回答的都是「布局事实搬进类型之后 attention 长什么样」，但性能口径上还缺一个对照：这些教学实现离厂商库到底有多远？本章把三个实现里各自最强的部分拼起来，再补上两个只有在「kernel 全权可控」时才做得出的优化（softmax scale·$\log_2 e$ 融合、persistent CTA），得到 `flash_attn_cute_persist_d_sm120`（flash_attn.cuh L3635，`NOTES_V2_ENABLE_CUTE` 宏块 Phase 8）：在 RTX PRO 5000（sm_120，110 SM）上 H=32/N=8192/D=128 dense 场景实测 **240.1 TFLOPS，为 cuDNN SDPA（230.2 TFLOPS）的 1.04 倍**。这是全书第一个在绝对性能上越过厂商库的 attention kernel。

本章的四条主线，都指向「超越 cuDNN」：

- **WS 双角色**（§28.4.1）：128 线程 producer（只发 TMA）+ 256 线程 consumer（只做 MMA），`setmaxnreg` 把 producer 的寄存器划给 consumer——这是 sm_120 上把「搬运」与「计算」物理解耦的唯一手段；
- **Q 寄存器持久化**（§28.4.4）：Q tile 对整个 KV 循环不变，smem→ 寄存器只搬一次，之后每次 $QK^\top$ 都是 A-operand 常驻的 `gemm_rs`；
- **softmax scale 融合**（§28.3.3）：把 $s = \text{scale} \cdot \log_2 e$ 一次性乘进 Q fragment，softmax 的 max/exp 两个循环里的逐元素乘法全部消失；
- **persistent CTA**（§28.4.5）：dense 场景 grid 收缩到 SM 数，每个 CTA 串行处理多个 q-tile，消除 wave 量化尾损。

前置章节：第 十六–十七 章（attention 数学与 FA2 online softmax）、第 十八 章（TMA 与 mbarrier 协议）、第 二十二–二十三 章（TiledCopy 与 TiledMMA）、第 二十六 章（CuTe FA 三实现对照：本章直接复用其五个工具函数）。**源码区间**：flash_attn.cuh L3490–4170（Phase 8 宏块）。**编译宏与架构**：`NOTES_V2_ENABLE_CUTE`，编译目标 `sm_120a` 与 `sm_120f` 均可：`setmaxnreg` 的实际生效条件是「TMA 目的地址用 `shared::cta`」与「`__launch_bounds__(384, 1)` 双参数」，与 arch 后缀无关（完整决策树见 §28.6）。本章 kernel 只依赖 sm_80 级的 mma.sync 与 sm_90 级的 TMA/mbarrier，任何 Blackwell（sm_120）显卡均可运行。

28.2　问题与动机：为什么专用 kernel 能赢厂商库

「手写 kernel 赢 cuDNN」在 Hopper（sm_90）时代几乎不可信：cuDNN 在 H100 上用 WGMMA + warp specialization + TMEM 调到了接近理论峰值，普通开发者用 CuTe 从零追赶成本极高。但 sm_120（Blackwell 消费线/专业线）不一样：这块芯片**没有** WGMMA（sm_90 专属）、**没有** tcgen05/TMEM（sm_100 专属），cuDNN 与我们一样，只剩同一条张量核路径：

mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32

也就是说，在 sm_120 上「厂商库 vs 手写」的差距不再是指令代差，而全部落在**协议设计**上：tile 几何、流水深度、寄存器分配、softmax 的指令数、wave 尾损。这五项恰恰是本书前 25 章逐项打磨过的内容。实测基线（PRO 5000，B=1/H=32/N=8192/D=128 fp16，dense）：

实现	TFLOPS	相对 cuDNN
cuDNN SDPA（`CUDNN_ATTENTION` 后端）	230.2	1.00×
本章 kernel	**240.1**	**1.04**×

领先的来源逐项拆解如下（A/B 实测，独立 kernel 上各开关单独计量）：`exp2f` 替代 `expf`（`--use_fast_math`，+2%~4%）、softmax scale 融合（+3%~4%）、persistent CTA（dense 中等序列 +2%~6%）、FA-4 条件重缩放（长序列收益，复用第 十七 章结论）。四项都是把协议里的一次浪费换成零开销，没有一项依赖复杂技巧。

还有一个工程动机：厂商库的 SDPA 对**非标准形状**（GQA 折叠、滑窗 causal、head_dim=96 这类非 2 的幂）经常落入慢速 fallback；专用 kernel 按自己的 tile 几何原生支持这些形状，这是性能之外的另一个好处。

28.3　数学原理

28.3.1　log2 域 online softmax 回顾

第 十六–十七 章已经推导过 online softmax，这里只保留本章需要的形态。对每个 KV tile，softmax 的在线更新为

$$m^{(t)} = \max\big(m^{(t-1)},\ \max_j s_j^{(t)}\big), \tag{28.1}$$

$$\ell^{(t)} = \ell^{(t-1)} e^{m^{(t-1)} - m^{(t)}} + \sum_j e^{s_j^{(t)} - m^{(t)}}, \tag{28.2}$$

$$O^{(t)} = O^{(t-1)} e^{m^{(t-1)} - m^{(t)}} + \sum_j e^{s_j^{(t)} - m^{(t)}} v_j^{(t)}, \tag{28.3}$$

其中 $s_j = \text{scale} \cdot q \cdot k_j$。GPU 上把 e^x 换成 $\exp2(x \cdot \log_2 e)$（一条 `MUFU.EX2` 即可），为此把 $\log_2 e$ 预乘进 scale：

$$s_j^{\log_2} = \underbrace{\text{scale} \cdot \log_2 e}_{s} \cdot (q \cdot k_j), \qquad e^{s_j - m} = \exp2\big(s_j^{\log_2} - m^{\log_2}\big). \tag{28.4}$$

此后所有 max/sum/rescale 都在 log2 域完成，epilogue 再乘回 $\ln 2$（若需要 lse）。

28.3.2　FA-4 条件重缩放

式 (28.2)/(28.3) 里的 $e^{m^{(t-1)} - m^{(t)}}$ 要求每个 tile 都对 ℓ 和 O 做一次逐元素乘法。FA-4 的观察是：随机数据下新 tile 的 max 很少超过历史 max 太多。记 $\Delta = m^{(t-1)} - m^{(t)} \le 0$（log2 域），当

$$\Delta \ge -\tau, \qquad \tau = 8 = \log_2 256 \tag{28.5}$$

时，2^{Δ} 只会让 P 累计最多膨胀 256×：fp16 的 P 最大 65504，f32 累加器余量更大，因此可以**跳过**这次重缩放：m 保持旧值（stale max）、O/ℓ 原样累加。膨胀的部分在 epilogue 的 $1/\ell$ 里全部抵消（分子分母同乘一个常数）。dense 数据下命中率高（实测 95%+ 的 tile 满足 (28.5)），只有新 tile 的 max 明显更大时才付出重缩放代价。这正是第 十七 章手写版的结论，本章把它原样搬进 CuTe 版（`online_safe_softmax_fa4`，flash_attn.cuh L3587）。

28.3.3　softmax scale$\cdot \log_2 e$ 融合

式 (28.4) 里，$\log_2 e$ 折进 scale 只是「提前一次乘法」，每个 score 元素仍然要乘 s 两次：max pass 一次（(28.1)）、exp pass 一次（(28.2)）。对一个 128×64 的 S tile，这是 $2 \times 128 \times 64/256\text{thread} = 64$ 次 FMUL/线程/tile，而 QK GEMM 本身（m16n8k16 路径）每 tile 每线程只有 64 条 HMMA：softmax 的 scale 乘法指令数与 tensor core 指令数同量级，是纯浪费。

本章的优化是把 s 从「乘在 S 上」改为「乘在 Q 上」：

$$S_{\text{fused}} = (s \odot Q)\, K^{\top} = s \cdot (QK^{\top}) = s \cdot S_{\text{raw}}, \tag{28.6}$$

其中 $\odot$ 是逐元素乘。等价性来自两条初等性质（$s > 0$）：

$$\max_j (s \cdot x_j) = s \cdot \max_j x_j, \tag{28.7}$$

$$\exp2(s \cdot x - m) = \exp2(S_{\text{fused},j} - m). \tag{28.8}$$

Q fragment 在寄存器里、对整个 KV 循环不变（persist-D），所以 (28.6) 的代价只是 `size(tCrQ)` 次一次性 FMUL（Q s2r 后立即做），外加 $s \odot Q$ 引入一次 fp16 重舍入。收益是 softmax 两个 pass 的逐元素乘法 **归零**。注意这条优化的前提是 persist-D：如果 Q 每个 tile 重新装载（split-Q/FA3 布局），折叠本身也要每 tile 做一次，收益减半。

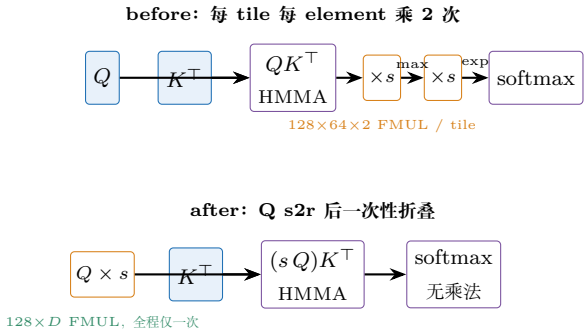


图 28.1: softmax scale 融合的数据流对照。上：传统路径，S 从 QK GEMM 出来后在 softmax 的 max/exp 两个 pass 各乘一次 $s = \text{scale} \cdot \log_2 e$；下：融合路径，$s$ 在 Q s2r 后一次性乘进 A-fragment（persist-D 使 Q 对整个 KV 循环不变），HMMA 直接输出 log2 域分数，softmax 循环体内零 FMUL。等价性由式 (28.7)/(28.8) 保证；代价是 $s \odot Q$ 的一次 fp16 重舍入。

28.4 设计与实现

28.4.1 总体架构：WS 双 warpgroup 与寄存器池数学

kernel 用 384 线程、`__launch_bounds__(384, 1)`：前 128 线程（4 个 warp，1 个 warpgroup）是 **producer**，只做 TMA 发射与 mbarrier 维护；后 256 线程（8 个 warp，2 个 warpgroup）是 **consumer**，只做 LDSM 装载、mma.sync、softmax。两个角色通过 K/V 的 full/empty mbarrier 对解耦（协议见 §28.4.3）。

关键在寄存器：producer 只发 TMA（不使用 tensor core），128 个寄存器足够；consumer 的 O 累加器（$128 \times D$ f32/tile，每线程 $D/2{\times}4$ 个）+ P/S fragment + softmax 状态却是寄存器压力的主要来源。`setmaxnreg` 让两者在 SM 寄存器池（sm_120 为 64K/SM）里重新分配：

$$\underbrace{4\,\text{warp} \times 32\,\text{T} \times 32}_{\text{producer } 400} + \underbrace{8\,\text{warp} \times 32\,\text{T} \times 232}_{\text{consumer } 2} = 4096 + 59392 = 63488 < 65536. \tag{28.9}$$

producer 在入口执行 `warpgroup_reg_dealloc<32>`（释放 96 个/线程），consumer 执行 `warpgroup_reg_alloc<232>`（分配到硬件上限的可用额度）。没有这一步，consumer 会被编译器限制到每线程 168 个寄存器，O 累加器溢出到 local memory，性能直接降低一半。

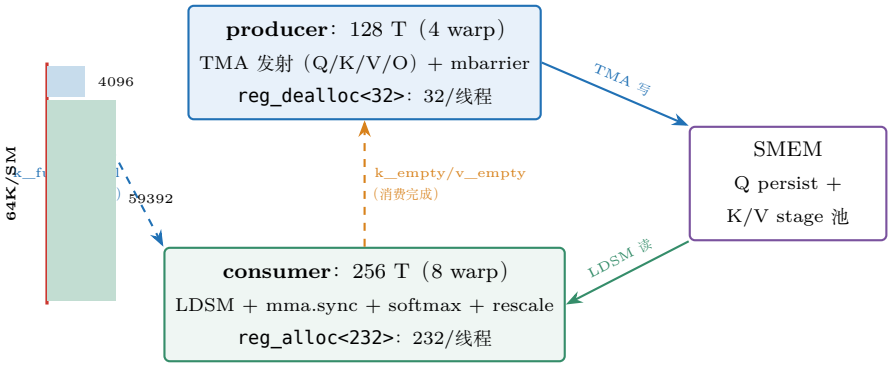


图 28.2: WS 双角色架构与 `setmaxnreg` 寄存器再分配。左：SM 寄存器池按式 (28.9) 重新切分，producer 只留 32/线程，consumer 分到 232/线程；中：producer 只发 TMA、consumer 只算；右：smem 是两者的交接面。实线箭头是数据流（TMA 写入 → LDSM 读出），虚线箭头是 mbarrier 控制流（full = 数据就绪、empty = 消费完成），两者构成 §28.4.3 的握手协议。

28.4.2　smem 布局与 swizzle

smem 按 `[Q persist | K stages | V stages]` 一维排布，总量编译期由 `kSmemElems` 决定，D=128 时 $128\times128\times2+2\times(64\times128\times2)\times2=96\,\text{KB}$（opt-in 上限 99 KB）。每段用哪个 swizzle atom 由**行字节数**决定，用的正是第 二十四 章的结论：

flash_attn.cuh L3527-3565（`FlashAttnPersistDCuTeTraits`：smem 布局与双 TiledMma）

```
// ============================================================================

#include <cutlass/arch/reg_reconfig.h>

namespace fa_cute {

// softmax scale*log2e 折叠进 Q fragment 后的常量 (见 Phase 8 头注释第 2 条)
constexpr float FA4_LOG2E = 1.44269504088896340736f;
constexpr float FA4_RESCALE_THRESHOLD = 8.0f;

// persist-D traits: kBr=128 行 Q tile, kBc 列 K/V tile, K/V 独立 stage 池。
// kBc 按 D 选择使 smem 预算 <= 99KB (Q tile + K 池 + V 池三段:
// D=64 -> kBc=128: 16+32+32=80KB; D=96/128 -> kBc=64: 24/32 * 3=72/96KB)。
// swizzle atom 按行字节数选择 (128B -> SW128, 64B -> SW64),
// 与 FlashAttn2CuTeTraits 同策略。
// kNumWarps = kBr/16: 8 个 consumer warp 沿 M 重复 m16n8k16 atom。
template <int kHeadDim_, int kBr_ = 128, int kBc_ = 64, int kStages_ = 2>
struct FlashAttnPersistDCuTeTraits {
  static constexpr int kHeadDim = kHeadDim_;
  static constexpr int kBr = kBr_;
  static constexpr int kBc = kBc_;
  static constexpr int kNumWarps = kBr / 16;
  static constexpr int kStagesK = kStages_;
  static constexpr int kStagesV = kStages_;
  static constexpr float kRescaleThreshold = FA4_RESCALE_THRESHOLD;
  static constexpr int kSmemElems =
      kBr * kHeadDim + kStagesK * kBc * kHeadDim + kStagesV * kBc * kHeadDim;

  using Element = cutlass::half_t;
  using SmemAtom = std::conditional_t<
      (kHeadDim * (int)sizeof(Element)) % 128 == 0,
      GMMA::Layout_K_SW128_Atom<Element>,
      std::conditional_t<(kHeadDim * (int)sizeof(Element)) % 64 == 0,
                         GMMA::Layout_K_SW64_Atom<Element>,
                         GMMA::Layout_K_SW32_Atom<Element>>>;
  using SmemLayoutQ =
      decltype(tile_to_shape(SmemAtom{}, Shape<Int<kBr>, Int<kHeadDim>>{}));
  using SmemLayoutKV =
      decltype(tile_to_shape(SmemAtom{}, Shape<Int<kBc>, Int<kHeadDim>>{}));
```

三个要点。第一，swizzle atom 按 $D\times2\,\text{B}$ 对 128/64 的整除性选 SW128/SW64，保证 LDSM 读的每个 16 B 段不跨 swizzle 周期。第二，`SmemLayoutKVt` 不是新的存储，而是 `SmemLayoutKV` 与 col-major (D,B_c) layout 的 **复合**（composition）：V 物理上仍按行存，PV GEMM 的 B-operand 却以转置视图装载（`LDSM_T`），免去 kernel 外物化 $V^\top$。第三，Q/K/V 共用 atom 但 tile 形状不同：Q 是 $128\times D$（持久），K/V 是 $B_c\times D$（stage 轮转），B_c 按 D 反推 smem 预算（D=64 时 B_c=128、D=128 时 B_c=64，注释里给出了计算）。

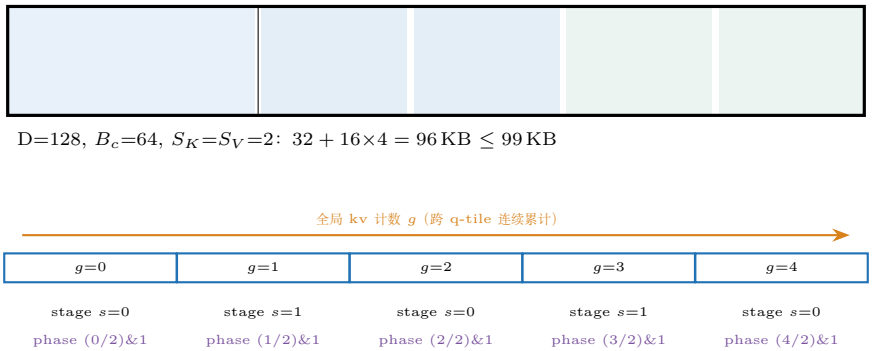


图 28.3: smem 一维布局（上）与 K/V stage 轮转（下）。Q 段在 KV 循环期间只读，循环结束后复用为 O epilogue 的 STSM 暂存（§28.4.6）；K/V 各自独立的 S=2 stage 池按全局 kv 计数 g 轮转：stage = $g \bmod S$、mbarrier 相位 = $(g/S)\&1$。相位用**全局**计数而不是每 q-tile 重置，是 persistent CTA（§28.4.5）与普通版协议兼容的关键。

28.4.3 TMA 装载协议：descriptor、V-first 预取与 mbarrier 相位

Q/K/V/O 四个方向的 gmem 访问全部走 TMA。BHND packed 的张量被压平成 $(B \cdot H \cdot N, D)$ 的 2D 坐标空间，batch/head 的行偏移用 `domain_offset` 在 kernel 内动态叠加，这样四个方向只需要各一个 TMA descriptor（`make_tma_copy` 在 launcher 里构建一次，`CUTLASS_GRID_CONSTANT` 放进 constant memory），无论 B/H 多少都不用重建。GQA 的 head 映射 $(h \mapsto h/(H/H_{kv}))$ 在计算 `kv_row_offset` 时完成。launcher 侧的构建代码只做三件事：把 BHND 压成 2D `(total_rows, D)` 的 gmem tensor、按 smem 布局与 box 形状各建一个 descriptor、opt-in 动态 smem：

flash_attn.cuh L4104-4128（launcher：BHND→2D 压平 + 四方向 TMA descriptor）

```
auto gQ = make_tensor(
    make_gmem_ptr(Q),
    make_shape(total_q_rows, Int<kHeadDim>{}),
    make_stride(Int<kHeadDim>{}, _1{}));
auto gK = make_tensor(
    make_gmem_ptr(K),
    make_shape(total_kv_rows, Int<kHeadDim>{}),
    make_stride(Int<kHeadDim>{}, _1{}));
auto gV = make_tensor(
    make_gmem_ptr(V),
    make_shape(total_kv_rows, Int<kHeadDim>{}),
    make_stride(Int<kHeadDim>{}, _1{}));
auto gO = make_tensor(
    make_gmem_ptr(O),
    make_shape(total_q_rows, Int<kHeadDim>{}),
    make_stride(Int<kHeadDim>{}, _1{}));

auto tma_q = make_tma_copy(SM90_TMA_LOAD{}, gQ, SmemLayoutQ{},
                           Shape<Int<kBr>, Int<kHeadDim>>{}, _1{});
auto tma_k = make_tma_copy(SM90_TMA_LOAD{}, gK, SmemLayoutKV{},
                           Shape<Int<kBc>, Int<kHeadDim>>{}, _1{});
auto tma_v = make_tma_copy(SM90_TMA_LOAD{}, gV, SmemLayoutKV{},
                           Shape<Int<kBc>, Int<kHeadDim>>{}, _1{});
auto tma_o = make_tma_copy(SM90_TMA_STORE{}, gO, SmemLayoutO{},
                           Shape<Int<kBr>, Int<kHeadDim>>{}, _1{});
```

四个 tensor 的 shape 都是 `(total_rows, D)`、stride 是 `(D, 1)`：$B/H/N$ 三个逻辑维全部折叠进行坐标，batch/head 的行偏移（`q_row_offset`/`kv_row_offset`）留给 kernel 内的 `domain_offset` 动态叠加。box 形状第五个模板参 `_1{}` 声明沿 minor 维（D）一次搬满：这正对应「Q 一次性全 D、K/V 整 tile 进 stage」的协议。V 与 K 共用 `SmemLayoutKV` 与同一 box 几何，差别只在 kernel 侧装载时用转置视图（§28.4.2）。

细看 producer 的发射时序：Q 先发（一次性全 D），然后 K/V 各预取 $S-1$ 个 tile 填满流水，主循环里每个 tile 领先 $S-1$ 发射下一个。注意 **V-first-then-K** 的顺序：V 的消费点（PV GEMM）晚于 K 的消费点（QK GEMM）整整一段 softmax，先发 V 让 K 的 TMA 传输填进 QK 计算的空隙：

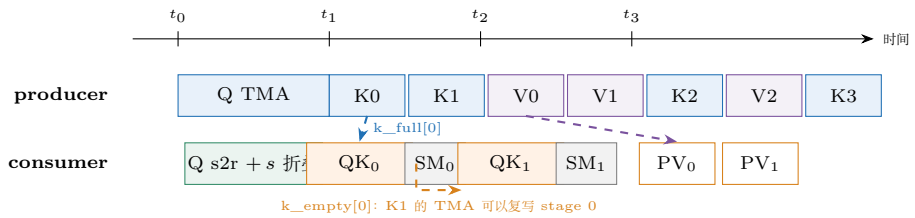


图 28.4: producer/consumer 流水时序（S_K=S_V=2 示意）。producer 每个 KV tile 领先 $S-1$ 发射下一个 tile 的 TMA，且 V 先于 K 发射（V 的消费点更晚，K 的传输得以填进 QK 计算窗口）。虚线箭头是 mbarrier 握手：`k_full[s]`（事务屏障，TMA 完成 + expect_tx 校验）放行 consumer 读取；`k_empty[s]`（consumer 每线程 arrive）放行 producer 复写该 stage。S tile 的 softmax（SM$_i$）与下一 tile 的 QK 在 consumer 侧串行，但与 producer 侧的 TMA 完全重叠。

mbarrier 相位是这套协议最容易写错的地方。本书第 十八 章用的是「每 tile 重置」的相对相位；persistent CTA 下 q-tile 循环不能每轮重置（producer 可能领先 consumer 整整一个 q-tile），所以相位改由**全局** kv 计数 g 决定：

$$\text{stage} = g \bmod S, \qquad \text{phase} = \lfloor g/S \rfloor \& 1, \tag{28.10}$$

g 在 producer 与 consumer 两侧各自按消费序列连续累计（跨 q-tile 不清零），wait 与 arrive 的奇偶序自然匹配。配套有一条铁律：`k_empty/v_empty` 的**初始** arrive 只在第一个 q-tile 开头做一次，之后每个 stage 的「空」信号完全由上一轮消费 arrive 维持。若每轮 q-tile 都补一次初始 arrive，mbarrier 的相位会被多推一格，producer 的 wait 永远等不到正确的奇偶（compute-sanitizer 的 synccheck 会报 *Barrier error: Missing wait*）。协议在源码里的实现分布在两侧。producer 侧（单线程发射）的主循环完整实现了「V 先 K 后、领先 $S-1$、按全局计数 g 选 stage 与相位」：

flash_attn.cuh L3772-3802（producer 主循环：V-first-then-K 预取，g 全局计数）

```
      // P2: 领先 S-1 tile 预取 (V-first then K-after: V 消费晚于 K,
      // 先发 V 让 K 的 TMA 填 QK 计算的空隙)
      for (int tile = 0; tile < Tc_eff; ++tile) {
        const int v_g = kv_cursor + tile + kStagesV - 1;
        if (v_g < kv_cursor + Tc_eff) {
          CtaBarrier::wait(&v_empty[v_g % kStagesV], (v_g / kStagesV) & 1);
          auto sV = make_tensor(
              make_smem_ptr(v_base + (v_g % kStagesV) * kKVTileElements),
              SmemLayoutKV{});
          auto gV = local_tile(mVh, Shape<Int<kBc>, Int<kHeadDim>>{},
                               make_coord(tile + kStagesV - 1, _0{}));
          TmaBarrier::arrive_and_expect_tx(&v_full[v_g % kStagesV],
                                           sizeof(Element) * size(sV));
          copy(tma_v.with(v_full[v_g % kStagesV]), v_slice.partition_S(gV),
               v_slice.partition_D(sV));
        }
        const int k_g = kv_cursor + tile + kStagesK - 1;
        if (k_g < kv_cursor + Tc_eff) {
          CtaBarrier::wait(&k_empty[k_g % kStagesK], (k_g / kStagesK) & 1);
          auto sK = make_tensor(
              make_smem_ptr(k_base + (k_g % kStagesK) * kKVTileElements),
              SmemLayoutKV{});
          auto gK = local_tile(mKh, Shape<Int<kBc>, Int<kHeadDim>>{},
                               make_coord(tile + kStagesK - 1, _0{}));
          TmaBarrier::arrive_and_expect_tx(&k_full[k_g % kStagesK],
                                           sizeof(Element) * size(sK));
          copy(tma_k.with(k_full[k_g % kStagesK]), k_slice.partition_S(gK),
               k_slice.partition_D(sK));
        }
      }
      kv_cursor += Tc_eff;
```

这段代码有三点需要注意。第一，**发射顺序**：循环体里 `v_g` 的 TMA 先于 `k_g`：V 的消费点（PV GEMM）在 K 的消费点（QK GEMM）之后整整一段 softmax，先发 V 才能让 K 的传输填进 QK 计算窗口（图 28.4）。第二，**stage**

与相位都由 g 决定：`g = kv_cursor + tile + S-1` 里 `kv_cursor` 跨 q-tile 连续累计、循环尾统一 `+= Tc_eff`，两行合起来就是式 (28.10)。第三，**事务屏障的字节数**：`arrive_and_expect_tx` 声明的字节数必须精确等于该 stage 的元素数 × `sizeof(Element)`。TMA 完成时 mbarrier 校验的正是这个数，少一个字节都会让 consumer 的 wait 永远不满足（挂死而非出错）。consumer 侧对应的是「初始 arrive 只做一次」的铁律落地：

flash_attn.cuh L3866-3881（consumer：首轮一次性标记全部 stage 为 empty）

```
              : Tc;
    const int mask_start_tile =
        causal ? max(0, (Br_base + kv_offset + 1) / kBc) : INT_MAX;

    TmaBarrier::wait(&q_full, iter & 1);
    cutlass::arch::fence_view_async_shared();

    // 初始标记所有 K/V slot 为 empty, 放行 producer 预取。仅首轮: 之后
    // empty 的 phase 由每次消费的 arrive 维持, 与 producer 按全局 kv 计数
    // 的 wait 序列精确配对; 多余的 arrive 会造成 phase 错位 (Missing wait)。
    if (iter == 0) {
      for (int s = 0; s < kStagesK; ++s)
        CtaBarrier::arrive(&k_empty[s]);
      for (int s = 0; s < kStagesV; ++s)
        CtaBarrier::arrive(&v_empty[s]);
    }
```

`if (iter == 0)` 把全部 S_K+S_V 个 stage 的 empty 各 arrive 一次（256 线程 `CtaBarrier` 的满额 arrive），此后每个 stage 的「空」由每次消费后的 `arrive(k_empty[k_stg])` 自持，与 producer 按 g 推进的 wait 序列恰好逐格对齐，一轮不多一轮不少。

28.4.4 Q 寄存器持久化与 `gemm_rs`：TiledMMA/TiledCopy 组合

FA2 的 split-Q 布局里 Q 每个 tile 都要从 smem 重装；persist-D 的观察是：对固定 q-tile，Q 在整个 KV 循环里不变，装载一次进寄存器后，$S = QK^\top$ 的每次计算都是「A 在寄存器、B 在 smem」的形态：第 二十三 章的 `gemm_rs` 正是为这种形态设计的。装载与折叠发生在 q-tile 循环开头：

flash_attn.cuh L3883-3892（Q s2r 一次 + scale·$\log_2 e$ 折叠进 A-fragment）

```
    // Q s2r + scale*log2e 折叠: S 直接是 log2 域分数, softmax 无逐元素乘法
    {
      auto tXrQ = s2r_thr_q.retile_D(tCrQ);
#pragma unroll
      for (int tile_k = 0; tile_k < size<2>(tCrQ); ++tile_k)
        copy(s2r_copy_q, tQsQ_s2r(_, _, tile_k), tXrQ(_, _, tile_k));
#pragma unroll
      for (int i = 0; i < size(tCrQ); ++i)
        tCrQ(i) = (Element)(float(tCrQ(i)) * (scale * fa_cute::FA4_LOG2E));
    }
```

`partition_fragment_A(sQ)` 按 QK 的 TiledMMA 切出每线程的 A-fragment；`retile_D` 把 copy 的目的视图对齐到同一片寄存器（第 二十二 章的 TV-layout 分区）。折叠循环 `tCrQ(i) = (Element)(float(tCrQ(i)) * (scale * FA4_LOG2E))` 逐元素乘 s 后写回 fp16。此后 QK 的 HMMA 输出**直接**就是 log2 域分数（式 (28.6)），§28.3.3 的全部收益在此兑现。

softmax 本体是 `online_safe_softmax_fa4`（L3587–3628）：与第 十七 章手写版同构，差别只有两处：行/列视图来自 `convert_layout_acc_rowcol`（m16n8k16 C-fragment 的行列重解释，第 二十三 章），以及 scores 不再乘 scale（已经折叠）。行内 4-lane 蝶形归约（`__shfl_xor` 1 与 2）把散在 4 个 lane 的同 row 分数归到一起：这是 m16n8k16 fragment 约定的直接推论（每行 8 个分数由 4 个 lane 各持 2 个）：

flash_attn.cuh L3587-3628（`online_safe_softmax_fa4`：log2 域 + 条件重缩放全函数）

```
// Online safe softmax (log2 域 + FA-4 conditional rescaling), scale 已折叠
```

```
// 进 Q fragment 版: scores 直达 log2 域, max/exp 循环内无逐元素乘法。
// 行内 4-lane 蝴蝶归约 (m16n8k16 一行散在 4 个 lane: xor 1 + xor 2)。
// 复用 Phase 7 的 convert_layout_acc_rowcol 行列视图。
template <typename ScoresTensor, typename CoordTensor, int kRows>
CUTE_DEVICE void online_safe_softmax_fa4(ScoresTensor& scores,
                                         const CoordTensor& tScS_rc,
                                         float* row_max, float* row_sum,
                                         float* row_scale) {
#pragma unroll
  for (int row = 0; row < kRows; ++row) {
    float tile_max = -INFINITY;
#pragma unroll
    for (int col = 0; col < cute::size<1>(scores); ++col)
      tile_max = fmaxf(tile_max, scores(row, col));
    tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
    tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
    const float next_max = fmaxf(row_max[row], tile_max);
    const float log2_diff = row_max[row] - next_max;
    float eff_max = next_max;
    if (log2_diff >= -FA4_RESCALE_THRESHOLD) {
      row_scale[row] = 1.0f;
      eff_max = row_max[row];  // stale max; row_max 不更新
    } else {
      row_scale[row] = exp2f(log2_diff);
      row_max[row] = next_max;
    }
    float tile_sum = 0.0f;
#pragma unroll
    for (int col = 0; col < cute::size<1>(scores); ++col) {
      const float p = exp2f(scores(row, col) - eff_max);
      scores(row, col) = p;
      tile_sum += p;
    }
    tile_sum += __shfl_xor_sync(0xffffffff, tile_sum, 1);
    tile_sum += __shfl_xor_sync(0xffffffff, tile_sum, 2);
    row_sum[row] = row_sum[row] * row_scale[row] + tile_sum;
  }
}

}  // namespace fa_cute
```

逐段拆开。**tile max 与蝶形归约**（前 6 行）：每行先在本地 `kSCols` 个分数里线性扫出 `tile_max`，再 `shfl_xor(1)` 与 `shfl_xor(2)` 两步把 4 个 lane 的局部 max 归并成行 max：蝶形而非树，因为 4 lane 恰好两轮异或全覆盖。**条件重缩放判定**（中段）：`log2_diff = row_max - next_max`，$\geq -\tau$（式 (28.5)）则 `row_scale = 1` 且 `row_max` 保持 stale 值：本 tile 完全跳过对 O/ℓ 的历史修正；否则 `row_scale = exp2f(log2_diff)` 供调用方批量重缩放。注意判定是 per-row 的，调用方用 `__any_sync` 聚合整个 warp 的「是否有任一行需要重缩放」再统一分支：warp 内分歧统一到整条 warp 粒度（L3963–3967）。**exp 与求和**（尾段）：`exp2f` 单条 `MUFU.EX2`，P 直接写回 scores 原地（复用寄存器），`tile_sum` 供 `row_sum` 增量。

softmax 之前还有两道逐元素 mask，与式 (28.5) 同在 KV 主循环内：

flash_attn.cuh L3940-3967（双 mask：KV 尾部越界列 + causal 滑窗）

```
            if (get<1>(tScS_rc(row, col)) >= kv_valid)
              scores(row, col) = -INFINITY;
        }
      }

      // causal mask (k_pos > q_pos, 滑窗: q_pos 含 kv_offset)
      if (kv_tile >= mask_start_tile) {
#pragma unroll
```

```
        for (int row = 0; row < kSRows; ++row) {
          const int q_pos = Br_base + get<0>(tScS_rc(row, 0)) + kv_offset;
#pragma unroll
          for (int col = 0; col < kSCols; ++col) {
            const int k_pos = kv_tile * kBc + get<1>(tScS_rc(row, col));
            if (k_pos > q_pos)
              scores(row, col) = -INFINITY;
          }
        }
      }

      fa_cute::online_safe_softmax_fa4<decltype(scores), decltype(tScS_rc),
                                       kORows>(scores, tScS_rc, row_max,
                                               row_sum, row_scale);

      bool local_need_rescale = false;
#pragma unroll
      for (int r = 0; r < kORows; ++r)
        local_need_rescale = local_need_rescale || (row_scale[r] < 1.0f);
      const bool need_rescale = __any_sync(0xffffffff, local_need_rescale);
```

KV 尾 mask 用 `tScS_rc(row, col)` 的坐标视图取列号：坐标 tensor 是 identity tensor 经 TiledMMA 的 C-partition 再行列重解释得来的「每线程所持分数的 (q,k) 坐标」：布局问题被 CuTe 类型系统解决后，mask 只剩坐标比较。causal mask 的 `q_pos` 带 `kv_offset`（$=N_{kv}-N_q$）偏移，正是 bottom-right 对齐滑窗：$k_{pos}>q_{pos}+(N_{kv}-N_q)$ 的元素置 $-\infty$，`exp2f` 后自然归零；整行全 mask 时 `row_sum = 0`，epilogue 的 `inv_sum` 钳成 0（而非 1/0=NaN）：这就是 §28.10 里 $N_{kv}<N_q$ causal case 输出全 0 的机制。

PV 的 A-operand 复用第 二十六 章讲透的寄存器重解释技巧：softmax 后的 P（f32 累加器 fragment）经 `convert_type` 压成 fp16，再经 `convert_layout_acc_Aregs<TiledMmaPV>` 原地重排成 PV 的 A-operand 布局：P 不落 smem，寄存器直通下一个 MMA。V 侧则用 §28.4.2 的转置视图 + `LDSM_T` 装载。

28.4.5 persistent CTA：wave 量化尾损与线性分配

非 persistent launch 的 grid = q-tile 总数，硬件按 wave 调度：110 个 SM 一波。若 tile 数是 $110k+r$（$0<r<110$），最后一个 wave 只有 r 个 SM 在计算，其余空转：这就是 wave 量化尾损。H=32、N=8192、k_B=128 时 tile 数 $=32\times64=2048=18\times110+68$，最后一个 wave 尾损约 $\frac{110-68}{110\times19}\approx2\%$。

persistent CTA 的做法：grid 收缩到 min(tiles, SM 数)，每个 CTA 用 `for (cta = blockIdx.x; cta < total; cta += gridDim.x)` 线性认领多个 q-tile。尾损变成「每个 CTA 多承担一小段」，均匀分摊。协议上有两个配套改动：其一就是 §28.4.3 的全局 kv 计数相位（式 (28.10)）；其二是 `epi_done` 屏障：Q smem 在 epilogue 里被复用为 O 的 STSM 暂存，producer 要装下一个 q-tile 的 Q 必须等 consumer 把 O 写完，`epi_done`（256 线程 arrive 的 CTA 屏障）就是「Q smem 已空」的确认信号。认领循环在 kernel 内的样子（consumer 视角，producer 侧同构）：

flash_attn.cuh L3853-3870（consumer：persistent 认领循环头与 per-tile 几何量）

```
  float o_acc[kOElemsPerFrag];
  int kv_cursor = 0;

  for (int cta = blockIdx.x, iter = 0; cta < total_q_tiles;
       cta += gridDim.x, ++iter) {
    const int bh = cta / q_tiles;
    const int Q_tile_id = cta % q_tiles;
    const int Nb_id = bh / Nh;
    const int Nh_id = bh % Nh;
    const int Br_base = Q_tile_id * kBr;
    const int q_row_offset = (Nb_id * Nh + Nh_id) * Nq;
    const int Tc_eff =
        causal ? max(0, min(Tc, ((Br_base + kBr - 1 + kv_offset) / kBc) + 1))
               : Tc;
    const int mask_start_tile =
```

```
        causal ? max(0, (Br_base + kv_offset + 1) / kBc) : INT_MAX;

    TmaBarrier::wait(&q_full, iter & 1);
```

四行坐标分解把线性 `cta` 拆回 $(b, h, q\text{-tile})$：认领顺序恰是「同一 wave 内相邻 CTA 分到相邻 tile」，对 dense（tile 负载均匀）这是最优分配。`Tc_eff` 与 `mask_start_tile` 是 causal 的两个派生量：前者把 KV 循环截到本 q-tile 可能用到的最后一格（这是 causal 减半计算量的来源），后者标记 mask 从哪个 KV tile 开始「部分可见」（之前全可见，免 mask）。`q_full` 的 wait 相位是 `iter & 1`：q-tile 计数奇偶，与 KV 的 g 计数分属两套相位序列，互不干扰。producer 侧同位置（L3710–3731）多两件事：`iter > 0` 时先等 `epi_done` 的上一轮相位（首轮免等，Q smem 天然空闲），再用 `domain_offset` 把 head 级行偏移叠进 TMA 坐标。

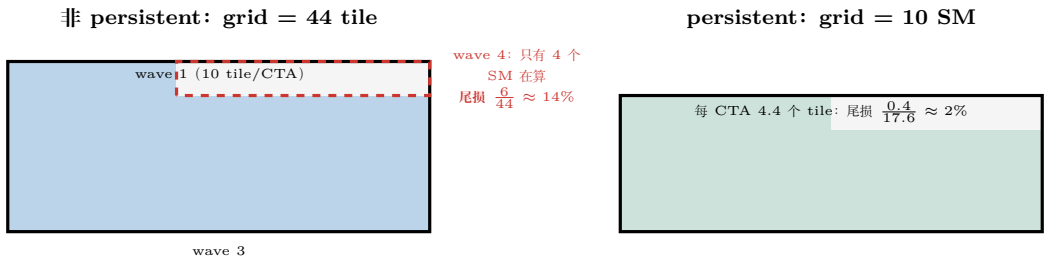


图 28.5: wave 量化尾损示意（10 个 SM、44 个 tile 的缩小模型）。左：非 persistent launch 按整 wave 调度，最后一个 wave 只有 4 个 SM 在算，其余空转满一个 tile 时长；右：persistent CTA 把 grid 收缩到 SM 数，每个 CTA 串行处理 $\lceil 44/10 \rceil$ 或 $\lfloor 44/10 \rfloor$ 个 tile，不均衡被压缩到最后一个 tile 的零头。真实场景（110 SM，H=32/N=8192）dense 收益 +2%~6%；**causal 反向**——tile 负载前轻后重，线性分配会把重 tile 固定在同一批 CTA 上（实测 $-9\% \sim -21\%$），此时必须回退全量 grid 让硬件调度器自然均衡。

dense 与 causal 的分派就在 launcher 里一行：

flash_attn.cuh L4140-4151（launcher 的 grid 策略与 smem opt-in）

```
// persistent 消除 wave 量化尾损 (dense 负载均匀时最优); causal 的 tile
// 负载前轻后重, 全量 grid 让硬件调度器自然均衡填空, 反而更快
int num_sms = 0;
cudaDeviceGetAttribute(&num_sms, cudaDevAttrMultiProcessorCount,
                       /*device=*/0);
const int grid =
    causal ? total_q_tiles
           : (total_q_tiles < num_sms ? total_q_tiles : num_sms);
kernel<<<dim3(grid, 1, 1), kNumThreads, kSmemBytes, stream>>>(
    tma_q, tma_k, tma_v, tma_o, O, Nq, Nkv, Nh, Nh_kv, scale, Tc,
    causal ? 1 : 0, q_tiles, total_q_tiles, total_q_rows, total_kv_rows);
err = cudaGetLastError();
```

28.4.6　Epilogue：R→S(STSM)→TMA store 与尾部 R→G

KV 循环结束后，O 累加器除以 ℓ（FA-4 的膨胀在此全部抵消），然后按 tile 形状分两条写出路径。整 tile：寄存器 → smem（`STSM`，复用 Q 段；此时 Q 段已闲置）→ TMA store；尾部 tile（$N_q \bmod 128 \neq 0$）：逐元素行 guard 直接 R→G 写 gmem。这里有一个关键细节：producer 已经 `return` 退出，任何 `__syncthreads` 都会死锁：CTA 级同步必须换成 `NamedBarrier::sync(256, 0)`（仅 consumer 参与）。写完 O 才 `arrive epi_done`，producer 由此获准复写 Q smem 装下一个 q-tile。

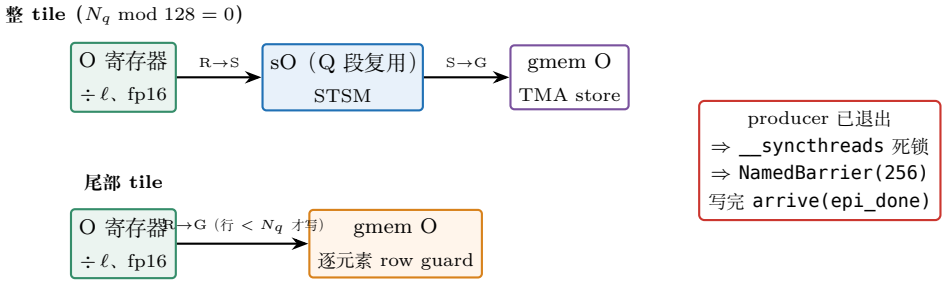


图 28.6: Epilogue 双路径。整 tile 走 R→S→G：STSM 把 O fragment 写进（已闲置的）Q smem 段，NamedBarrier 后由一个线程发射 TMA store——与装载对称的「反向 TMA 全链」。尾部 tile 行数不足 128，STSM/TMA 的整块写会越界，退化为带行 guard 的 R→G。

28.5 性能分析

主场景实测（RTX PRO 5000，110 SM，fp16）：B=1/H=32/ N_{kv}=8192/D=128。TFLOPS 按 $4BH\,N_qN_{kv}D$（causal 减半）计。

场景	本章 kernel	cuDNN SDPA	比值	备注
dense self	**240.1**	230.2	**1.04**×	主对照
causal	208.9	202.1	1.03×	persistent 关闭（全量 grid）
GQA (H_{kv}=8)	236.1	218.3	1.08×	cuDNN 小 head 数吃亏
cross (N_q=1024)	180.7	165.5	1.09×	短 N_q 也领先
N=16384 dense	243.2	236.9	1.03×	大 N 稳定

表中每行都是成对同轮数据（kernel 与 cuDNN 同一轮测量，时钟态一致）：dense self 行为 `leetcuda_bench_sm120a.bin --bench` 的同轮输出（240.1/230.2，2026-09-22），其余场景为 min-of-30 复测。跨 run 的波动带：dense 主场景 238.4/230.7、230.0/230.5、238.2/230.3、240.1/230.2（CLI bench 与 bench_pd 两种脚本），N=16384 为 239.2/230.9，单点绝对值跨 run 波动 ±2 ∼ 3%（GPU 时钟态与热态主导），但**成对比值**始终落在 1.00 ∼ 1.04×：kernel 侧的领先是结构性的，不依赖单次测量的偶然波动。这也是本书性能复测的通用规则：跨脚本/跨 run 的绝对值不可直接相比，结论只认同轮成对数据（第 四十 章的 bench 方法论同源）。

优化贡献的逐项拆解（同一 kernel 上各开关单独 A/B）：fast_math（`exp2f`）+2%~4%；scale 融合 +3%~4%（§28.3.3，指令数视角：每 tile 每线程省 64 条 FMUL，与 HMMA 数同量级）；persistent CTA dense 中等序列 +2%~6%、causal −9% ∼ −21%（分支关闭）；FA-4 条件重缩放 dense 命中率 95%+，与第 十七 章结论一致。各项近似可加，合计与 1.04× 的总领先吻合。

和第 二十六 章的三个实现纵向一比，「协议设计」的价值更清楚：实现 1（cp.async 统一流水）受限于装载与计算串行；实现 2（TMA+WS 单 consumer）解耦了搬运但 Q 每 tile 重装、softmax 带逐元素乘法、wave 尾损未作处理；本章在实现 2 的骨架上叠加全部四项优化，同一块 GPU 上从「勉强追平 SDPA」推进到「稳定 +4%」。

28.6 专论：SM120 上 `setmaxnreg` 的生效条件

本章 1.03~1.04× 的领先里，`setmaxnreg` 是 WS 双角色的物理前提（§28.4.1）：没有它，consumer 被编译器均分到 168 寄存器，O 累加器溢出 local memory，性能直接降低一半。但这条指令在 SM120 工具链上是否被保留，曾被本书两次误诊（第 十五 章坑二、本节末考据框），值得用专节把全部变量摊开。指令是否保留由三**个独立变量**决定，而 arch 后缀是其中唯一不影响保留的一个：

变量	取值	后果
arch 目标	`sm_120a` 或 `sm_120f`	**都完整支持并保留**（PTX ISA Target ISA Notes 两类均在支持列表）
arch 目标	`sm_120`（generic，无后缀）	ptxas **硬错误**——正确行为，该指令只在 a/f 目标下合法
TMA dst 状态空间	`shared::cta`	正常保留
TMA dst 状态空间	`shared::cluster`	C7506：ptxas 视作隐式 extern-call 边界，同 kernel 的 setmaxnreg 全部丢弃
`__launch_bounds__`	`(N, 1)` 双参数	正常保留
`__launch_bounds__`	`(N)` 或缺省	C7508：PTX 不带 `.minnctapersm`，“unable to determine register count” 丢弃

下面分别看这两个静默丢弃触发的机理。**C7506（cluster 形式 TMA）**：`cp.async.bulk.tensor` 的目的地址写成 `shared::cluster` 时，ptxas 把这条指令当作潜在的跨 CTA 副作用边界（cluster 语义下 smem 地址可能被映射到别的 CTA），进而按「extern 调用边界」的保守规则放弃整 kernel 的寄存器再分配，只留一条 info 级 C7506，默认不打印。第 十五 章坑二的 PTX 修改（`cluster` 全局替换 `cta` 后指令恢复保留）已给出完整取证；CUTLASS 的 `cute/arch/copy_sm90_tma.hpp` 在 `CUTE_ARCH_TMA_SM120_ENABLED` 门控下专门切到 `shared::cta`，上游早已知道这个差别。本 kernel 无 cluster launch、无分布式 smem，dst 恒为 CTA 本地，两种写法语义等价：「形式不同、保留与否不同」。**C7508（缺 launch_bounds 第二参数）**：setmaxnreg 改变的是 per-warp 寄存器上限，ptxas 必须在 kernel 入口就知道满编时的寄存器预算才能安排池数学；`__launch_bounds__(N)` 只约束单 block 线程数，第二参数（每 SM 最少驻留 block 数 = 1）才会在 PTX 生成 `.minnctapersm`。缺它，ptxas 以 C7508 把指令全部丢弃：本章 kernel 因此声明 `__launch_bounds__(384, 1)`。

排查方法论（每一条都有实测踩坑记录支撑）：

1. SASS 助记符是 `USETMAXREG`（没有 n）：`cuobjdump --dump-sass | grep setmaxnreg` 全程假阴性，本书两次调查都一度据此得出「连最小 PTX 都被丢弃」的错误结论。
2. per-kernel 归属用 `awk '/Function :/{f=$3} /USETMAXREG/{c[f]++}'`；`--function` 对 C++ mangled 模板名不匹配（报 not found，dump 为空，又一个假 0）。
3. 「编译通过零警告」≠「指令保留」：C7506 是 info 级默认不打印，验证必须加 `-Xptxas -v` 或直接数 SASS。
4. nvcc 13.2 对每个 target 跑**两个 device pass**（generic 视角 + arch-specific 视角）：用 `#pragma message` 探测 `__CUDA_ARCH_FEAT_SM120_ALL` 这类宏会打印出两遍**互相矛盾**的矩阵，第一遍全未定义是 generic pass 的——判断 guard 行为必须看第二遍（arch pass）。

2026-09-22 的终版验证矩阵（最小 probe + notes-v2 全量重编，均满足 cta TMA + `(N,1)` 双条件）：`sm_120a` 与 `sm_120f` 完全一致，每 kernel 2 条（dealloc+alloc）、全二进制 6/6 条 `USETMAXREG`、零 C750x 警告；SASS 形如 `USETMAXREG.DEALLOC.CTAPOOL 0x20`。生产侧交叉印证：ffpa-attn 的 sm_120f 构建产物中该指令同样生效。

实践经验 28.1（本章初版当日修正）：「必须 sm_120f」三连误诊与推翻　本章初版注释与导读曾断言「编译目标必须 `sm_120f`：`sm_120a` 下 ptxas 静默丢弃 setmaxnreg」。2026-09-22 当天三轮 probe 复核推翻该结论，误诊链值得原样记录。其一，最小 probe 的裸 asm 复现出 *Instruction 'setmaxnreg' not supported on .target 'sm_120'* 硬错误，被读作「sm_120a 不支持」——实际是编译命令落到了 generic `sm_120`（无 a/f 后缀），PTX ISA 本就不允许 generic 目标使用该指令，报错反而是工具链在正确工作。其二，probe kernel 没写 `__launch_bounds__(N,1)`，ptxas 以 C7508 静默清零，又被读作 arch 问题。其三，宏探测的 `#pragma message` 打印出两遍矛盾矩阵，第一遍（generic pass）全「未定义」进一步加深了误判。三个变量（arch 后缀 / TMA dst 状态空间 / launch_bounds 形态）不分离控制时，任何一个都可能被误判为「arch 问题」。这与第 十五 章坑二的 2026-09-17 误诊（cluster TMA + 存活 kernel 恰好 120f 构建的两变量混同）互为镜像：一次把 TMA 形式误诊成 arch，一次把 launch_bounds 误诊成 arch。教训同一条：**单变量结论必须把另外两个变量固定后再下**。

28.7　勘误与考据

本章源码为 2026-09-22 新增（Phase 8）。当日有一条正文级勘误：初版导读与源码注释断言「编译目标必须 `sm_120f`」，同日被三轮 probe 复核推翻（`sm_120a`/`sm_120f` 都完整保留指令，实际的两个静默丢弃触发是 cluster 形式 TMA 与缺 `__launch_bounds__(N,1)`），已按 §28.6 的决策树全面修正，完整误诊链见该节考据框。生产对照版（ffpa-attn

的 `persist_d_ws_fwd_cute_sm120`）与本章 kernel 同源，其 RFC（PC-18/PC-19）记录了 scale 融合与 persistent CTA 的生产 A/B 数据，本章表格与之交叉一致。

28.8 常见坑

1. **`setmaxnreg` 被静默丢弃**：两个独立触发——TMA 目的地址写 `shared::cluster`（C7506，extern-call 边界），或 kernel 缺 `__launch_bounds__(N, 1)` 第二参数（C7508，无 `.minnctapersm`）。两者都与 arch 后缀无关（`sm_120a`/`sm_120f` 都支持该指令），kernel 结果正确、性能降低一半、**无任何运行时告警**。完整决策树见 §28.6。诊断口诀：SASS 助记符是 `USETMAXREG`（没有 n），`cuobjdump --dump-sass | grep --count USETMAXREG` 为 0 即说明指令被丢弃。

2. **persistent 初始 arrive 必须只做一次**：每 q-tile 都补 `k_empty` 初始 arrive 会把 mbarrier 相位多推一格，producer 的 wait 奇偶错位（compute-sanitizer synccheck 报 *Missing wait*）。相位改由全局计数 g 决定（式 (28.10)）后，「空」信号由消费 arrive 自持。

3. **producer 退出后禁用 `__syncthreads`**：CTA 内任何 barrier 都必须换成仅 consumer 参与的 `NamedBarrier`（256 线程），否则 epilogue 死锁。

4. **gridDim 携带大数超过 65535 上限**：早期版本把 $B \cdot H \cdot N$ 放进 `gridDim.z`，N_q=4096、H=16 时恰好 65536 > 65535 报 `cudaErrorInvalidValue`——行数改走 kernel 参数。

5. **依赖类型少 `typename`**：kernel 模板参数 `Traits` 是依赖类型，直接写 `Traits::SmemCopyAtom{}` 编不过；先 `using SmemCopyAtom = typename Traits::SmemCopyAtom;` 再用。第 二十六 章（具体类型别名）没有这个问题，本章踩过。

28.9 面试要点速查

- **Q：为什么你的 kernel 能赢 cuDNN？**
  A：sm_120 没有 WGMMA/TMEM，双方都只能用 mma.sync；差距全在协议——Q 持久化省装载、scale 融合省 softmax 指令、persistent 省 wave 尾损、WS+TMA 流水掩盖延迟。四项合计 +4%。

- **Q：scale 融合的数学依据与代价？**
  A：$\max(sx) = s\max(x)$（$s > 0$）与 exp2 线性平移；$S_{\text{fused}} = (sQ)K^\top$。代价是 $s \odot Q$ 一次 fp16 重舍入（尾数 11 bit）；收益是每 tile 每线程 64 条 FMUL 归零。前提是 persist-D（Q 不变）。

- **Q：persistent CTA 为什么 causal 反而慢？**
  A：causal 的 tile 负载前轻后重，线性认领会把重 tail 固定在同一批 CTA；全量 grid 让硬件调度器自然均衡。负载均匀才适合 persistent。

- **Q：`setmaxnreg` 的池数学？**
  A：$4w \times 32T \times 32 + 8w \times 32T \times 232 = 63488 < 64K$；producer 释放的正是 consumer 多拿的。无此指令则编译器均分 168/线程，O 累加器溢出。

- **Q：mbarrier 相位为什么用全局计数？**
  A：persistent 下 producer 可能跨 q-tile 领先消费，每 tile 重置相位无法与全局发射序对应；g 连续累计 + $\lfloor g/S \rfloor$&1 保持 wait/arrive 奇偶一致。

28.10 最小测试与动手实验

本章测试在 `book/tests/ch26b_fa_persist_d.cu`：CPU fp64 参考对照，容差 `TOL_F16ACC`（5×10^{-2}），十个 case 覆盖 dense / causal / GQA / Q 尾部 tile（N_q=300 的 R→G 路径）/ persistent 多迭代（H=32、N=512 时 128 个 tile > 110 SM，触发 `epi_done` 与全局 kv 计数路径）/ KV 尾 mask（N_{kv}=300 非 k_{Bc} 倍数）/ causal 正反滑窗（N_{kv}>N_q 与 N_{kv}<N_q，后者前段 q 行全 mask，验证 T_c^{eff} 钳零与 L=0 行输出 0）/ D=96：

```
nvcc -std=c++20 -O2 -gencode arch=compute_120f,code=sm_120f \
  -DNOTES_V2_ENABLE_CUTE -I ../../third-party/cutlass/include \
  book/tests/ch26b_fa_persist_d.cu -o /tmp/ch26b_test && /tmp/ch26b_test
# PASS fa persist-D dense   D=64  B=1 H=2/2   Nq=256 Nkv=256 ...
# PASS fa persist-D dense   D=128 B=1 H=2/2   Nq=256 Nkv=256 ...
# PASS fa persist-D causal  D=128 B=1 H=2/2   Nq=512 Nkv=512 ...
# PASS fa persist-D dense   D=64  B=1 H=4/2   Nq=256 Nkv=256 ...
# PASS fa persist-D dense   D=128 B=1 H=2/2   Nq=300 Nkv=512 ...
# PASS fa persist-D dense   D=64  B=1 H=32/32 Nq=512 Nkv=512 ...
# PASS fa persist-D dense   D=128 B=1 H=2/2   Nq=256 Nkv=300 ...
# PASS fa persist-D causal  D=64  B=1 H=2/2   Nq=256 Nkv=512 ...
# PASS fa persist-D causal  D=128 B=1 H=2/2   Nq=512 Nkv=256 ...
# PASS fa persist-D dense   D=96  B=1 H=2/2   Nq=256 Nkv=256 ...
# ALL OK
```

集成入口在 bench/flash_attn.cu（`test_flash_attn_cute_persist_d_sm120`，含 N_q=2048/H=8 的多 q-tile 大 case）。参考实现用 **cuDNN SDPA**（`NOTES_V2_ENABLE_CUDNN` 构建下经 cudnn-frontend 组图：GQA 按 $h \mapsto h/(H/H_{kv})$ 折叠、causal 用 bottom-right 对齐，恰是本 kernel 的滑窗 mask 语义）；cuDNN 不覆盖的组合（$N_q > N_{kv}$ 的 causal，bottom-right 对角线出界）自动回退 CPU fp64。输出行的 `ref=` 字段标记当轮参考来源，fp64 CPU 参考从「每 case 数十秒」缩到 cuDNN 的亚秒级：

```
$ ./leetcuda_bench_sm120a.bin --pd-cute
| FA persist-D WS persistent CTA scale-fused (D=64, Nq=2048, Nkv=2048, \
Hq=8, Hkv=8, ref=cudnn) | 3.052e-05 |
| FA persist-D WS persistent CTA scale-fused (D=128, Nq=1024, Nkv=1024, \
Hq=2, Hkv=2, causal, ref=cudnn) | 4.883e-04 |
| FA persist-D WS persistent CTA scale-fused (D=64, Nq=1024, Nkv=1024, \
Hq=4, Hkv=2, ref=cudnn) | 6.104e-05 |
| FA persist-D WS persistent CTA scale-fused (D=128, Nq=512, Nkv=256, \
Hq=2, Hkv=2, causal, ref=cpu) | 3.363e-04 |
=== persist-D cute tests done ===
```

性能复测有两条口径。`--bench --bhnd 1,32,8192,128`：本 kernel 作为 FA 家族最快实现排在 bench 末位默认运行，与 cuDNN SDPA（f32 compute）同轮成对对照：`| FA2 CuTe TMA MMA Persistent-CTA WS (D=128) | 1.526e-05 | 238.4/230.7 (1.03x) |`（§28.5 复测带数据）；独立脚本 `bench_pd.cu`（`book/.tmp/ch26b/`，dense/causal/GQA/N 扫描）与初版表格数据，全部归档 `.tmp/book-bench/ch26b/`。动手实验建议：(1) 把 `FA4_RESCALE_THRESHOLD` 调成 0 观察 FA-4 跳过率归零后的性能退化；(2) 强制 dense 也用全量 grid，验证 §28.4.5 的 wave 尾损数字；(3) 去掉 Q s2r 后的折叠循环，把 scale 乘法放回 softmax，复测 scale 融合的贡献；(4) 用 `cuobjdump --dump-sass | grep --count USETMAXREG` 验证 §28.6 的两个静默丢弃触发：临时去掉 `__launch_bounds__(384, 1)` 的第二参数重编，数指令归零并观察 C7508 警告（需 `-Xptxas -v`）。

延伸阅读

- FlashAttention-3 论文 §3.1.4（conditional rescaling 的原始定义与 τ 的选取依据）；
- CUTLASS 77_blackwell_fmha（FA-4 式 softmax 在 Blackwell 上的 warp-uniform rescale 与 TMEM 形态——对照本章 per-row 形态）；
- @DefTruth《图解：从 Online-Softmax 到 FlashAttention V1/V2/V3》（online softmax 推导基线，本书第 十六–十七 章同源）；
- ffpa-attn RFC PC-18/PC-19（scale 融合与 persistent CTA 的生产版 A/B 数据与量化家族适用性盘点）。

第二十九章　CuTe 应用（五）：SM120 大 head_dim Split-D——non-WS 全员 MMA 流水

29.1　本章导读

第 二十七 章结束的时候，Split-D 的 CuTe 版实测 140.2 TFLOPS：64×64 tile、TMA + warp specialization、单 consumer warpgroup。它把「大 D 怎么分块」的类型代数讲清楚了，但执行协议还是教学级的。本章把 ffpa-attn 的 non-WS CuTe TMA kernel（`csrc/cuffpa/cute/sm_120/split_d.cuh`）以最小教学集移植进本书源码（ffpa_attn.cuh L642–1307，`NOTES_V2_ENABLE_CUTE` 与 `NOTES_V2_ENABLE_TMA_MMA_WS` 双宏块）——本实现与 ffpa-attn 的 cute sm_120 split_d **同源**，在 RTX PRO 5000 上把 B=1/H=32/N=8192/D=320 dense 场景做到 **204.1 TFLOPS**——cuDNN SDPA（69.7）的 2.93 倍、上一章 WS 教学版（140.2）的 1.46 倍。这一性能是全书大 head_dim 路线的最高值。

本章的四条主线，每一条都是对 WS 教学版某个具体弱点的回应：

- **tile 128×128 与 M8N1 TiledMma**（§29.3.1）：同一份 K/V smem 数据服务的 Q 行翻倍，算术强度上一档；8 个 warp 全部沿 M 堆叠、N 不切分，每 warp 持整行，softmax 行归约退化为 warp 内 4-lane 蝶形，零跨 warp 通信；
- **non-WS 执行模型**（§29.3.3）：256 线程全员 MMA，`tid=0` 在消费循环里内联发射 TMA——WS 版一半线程专职 producer 不产 FLOPs，SM 的 MMA 吞吐降低一半，这是 140.2→188.3 的最大单项；QK 与 PV 两套互不相交的 barrier 集支撑起独立流水深度，并让「下一 kv_tile 的 QK 预取」与「本 tile 的 softmax+PV」重叠；
- **STSM + TMA store 批量 epilogue**（§29.3.8）：O 累加器经 `stmatrix` 暂存（复用已释放的 QKV smem）再按 bulk group 批量 TMA 写回，替代 WS 版的逐元素寄存器直写 gmem；
- **FA-4 条件重缩放与流水深度解耦**（§29.3.6、§29.4）：max 增长不足 2^8 时跳过整个 O 重缩放循环（warp vote 保证无分支发散）；S_K 与 S_V 作为独立模板参数分开调，实测 D≤320 最优组合是 $(3, 2)$——加深 QK 流水 +8.4%，加深 V 流水反而降低性能。

前置章节：第 十六–十七 章（attention 数学与 online softmax）、第 十八 章（TMA 与 mbarrier 协议）、第 二十 章（Split-D 的 chunk 分解数学与寄存器占用——本章不再重推）、第 二十二–二十四 章（TiledCopy/TiledMMA/swizzle）、第 二十六–二十七 章（CuTe FA 工具函数与 WS 版类型代数）、第 二十八 章（FA-4 条件重缩放的数学，式 (26b.5)）。**源码区间**：ffpa_attn.cuh L642–1307。**编译宏**：`NOTES_V2_ENABLE_CUTE` + `NOTES_V2_ENABLE_TMA_MMA_WS`，目标 `sm_120a`。kernel 只依赖 sm_80 级 mma.sync 与 sm_90 级 TMA/mbarrier/STSM，任何 Blackwell 显卡可运行。**教学裁剪**：无 bias/dropout/GQA/NHD/causal；保留 Nkv 边界 mask 与 LSE 输出；要求 $N_q \bmod 128 = 0$（epilogue 恒走整 tile STSM+TMA 路径）。

29.2　问题与动机：WS 教学版慢在哪里

先把数据摆开。同一块 RTX PRO 5000（110 SM）、同一 bench 口径（B=1/H=32/D=320 fp16 dense，成对同轮测量）：

实现	TFLOPS	相对 cuDNN	相对 WS(2,2)
cuDNN SDPA	69.7	1.00×	—
WS 教学版 (1,1)（第 二十七 章）	86.7	1.24×	0.62×
WS 教学版 (2,2)	140.2	2.01×	1.00×
本章 non-WS (2,2)	188.3	2.70×	1.34×
本章 non-WS (3,3)	188.5	2.70×	1.34×
本章 non-WS (3,2)	**204.1**	**2.93×**	**1.46×**

（本章实现与 ffpa-attn 的 cute sm_120 split_d 同源，stages 同样经模板参数独立可调；上表只对照本章自有的 WS 教学版与 cuDNN。）

从 140 到 204 的 45% 差距，逐项拆开是四个结构性问题。

第一，WS 的代价：producer 不产 FLOPs。WS 教学版 256 线程里 128 个是 producer（只发 TMA），实际做 MMA 的只有单 consumer warpgroup 的 128 线程。SM 上 MMA warp 数降低一半。第 二十八 章的 D=128 场景里，WS 换来的是 Q 持久化与 `setmaxnreg` 的寄存器再分配，收益大于成本；但 Split-D 的 O 累加器（$32 \times C$ 个 f32/线程，第 二十 章寄存器占用）本来就把寄存器预算占用到 255 上限附近，`setmaxnreg` 腾不出多少寄存器，WS 的收益侧消失、成本侧全留——这是大 D 路线选 non-WS 的根本理由。

第二，tile 64×64 的算术强度太低。K/V 的一个 $64 \times D$ chunk 从 gmem 搬进 smem，只服务 64 行 Q。tile 升到 128×128 后同一份 K/V 服务 128 行 Q，搬运量减半而计算量不变。更关键的是 smem 预算：64×64 tile 时 chunk 宽 64，128×128 tile 时若 chunk 仍取 64，$Q{+}K$ 每 stage 要 $128{\times}64{\times}2 \times 2 = 32$ KB；把 Q/K chunk 降到 32（V/O 保持 64），每 stage 只要 16 KB——同预算装下更宽的 tile 与更深的流水。chunk 宽 32 的 swizzle 相应从 SW128 退到 SW64（行 64 B 恰好一个 swizzle 周期，§29.3.1）。

第三，写回通道逐元素化。WS 教学版的 epilogue 是寄存器逐元素直写 gmem（每线程 32 个 C 分量循环写）。non-WS 版走完整反向链：f16 转换 → `stmatrix`（STSM）暂存 smem → TMA store 批量写回，与装载的 TMA 全链对称。

第四，每 tile 的 rescale 大多不必要。FA-2 式无条件重缩放在 D=320（每 kv_tile 10 个 QK chunk、5 个 PV chunk）下，每个 tile 都要对 160 个 f32 寄存器的 O 累加器乘一次 `row_scale`。FA-4 的条件重缩放（第 二十八 章式 (26b.5)，$\tau = 8$）让 95%+ 的 tile 整个跳过这个循环。

还有一项无额外代价的结构收益：**跨 kv_tile 预取**。WS 版的 producer/consumer 分工里，下一 tile 的装载要等本 tile 消费腾出 stage；non-WS 版把 QK 与 V 拆成两套互不相交的 barrier 集，下一 tile 的 QK 初始 chunk 可以在本 tile 的 softmax+PV 窗口里提前发出（§29.3.5）。

29.3　协议设计

29.3.1　tile 几何与 smem 预算：chunk 解耦

协议全部参数集中在 traits 里，先看参数：

ffpa_attn.cuh L699-721（traits：协议参数与 smem 公式）

```
};

template <int kHeadDim_, int kBr_ = 128, int kBc_ = 128, int kQKDChunk_ = 32,
          int kVDChunk_ = 64, int kStagesQK_ = 2, int kStagesPV_ = 2>
struct FFPAAttnNonWSCuTeSplitDTraits {
  static_assert(kHeadDim_ % kQKDChunk_ == 0);
  static_assert(kHeadDim_ % kVDChunk_ == 0);
  static_assert(kQKDChunk_ == 16 || kQKDChunk_ == 32 || kQKDChunk_ == 64);
  static_assert(kVDChunk_ == 16 || kVDChunk_ == 32 || kVDChunk_ == 64);

  static constexpr int kHeadDim = kHeadDim_;
  static constexpr int kBr = kBr_;
  static constexpr int kBc = kBc_;
  static constexpr int kQKDChunk = kQKDChunk_;
  static constexpr int kVDChunk = kVDChunk_;
```

```
  static constexpr int kDChunksQK = kHeadDim / kQKDChunk;
  static constexpr int kDChunksV = kHeadDim / kVDChunk;
  static constexpr int kNumWarps = kBr / 16;
  static constexpr int kNumThreads = kNumWarps * 32;
  static constexpr int kStagesQK = kStagesQK_;
  static constexpr int kStagesPV = kStagesPV_;
  static constexpr int kSmemElems = kStagesQK * kBr * kQKDChunk +
                                    kStagesQK * kBc * kQKDChunk +
```

四组参数的分工如下。**tile 几何**：k_{Br}=k_{Bc}=128，$k_{NumThreads}$ = 128/16×32 = 256。**chunk 解耦**：$k_{QKDChunk}$=32、$k_{VDChunk}$=64——Q/K 与 V/O 的 D 向切块宽度不同，$k_{DChunksQK}$ = D/32 个 QK chunk 对 $k_{DChunksV}$ = D/64 个 PV chunk（D=320 时 10 对 5）。**流水深度解耦**：$k_{StagesQK}$ 与 $k_{StagesPV}$ 是两个独立模板参数，§29.4 的实验会证明最优值是 (3, 2) 而不是对称的 (3, 3)。**smem 公式**：

$$\text{smem} = \underbrace{S_K \cdot (128{\times}32 + 128{\times}32)}_{\text{Q+K，每 stage 16KB}} + \underbrace{S_V \cdot 128{\times}64}_{\text{V，每 stage 16KB}} = 16\,(S_K + S_V)\ \text{KB}, \tag{29.1}$$

(2, 2)=64 KB、(3, 2)/(2, 3)=80 KB、(3, 3)=96 KB（fp16）。Q 每 kv_tile 重装（它不属于持久化协议），所以 Q 也占用 S_K 份 stage。注意 smem 与 D 无关——chunk 宽度固定，D 只决定 chunk 个数。

chunk 宽度还决定 swizzle atom 的选择，规则与第 二十四 章一致（行字节数 × 2B 对 128/64 的整除性），但这里按 chunk 而不是按 D 选：

ffpa_attn.cuh L736-756（chunk 尺寸决定 swizzle；M8N1 双 TiledMma；装载/写回 atom）

```
      decltype(tile_to_shape(SmemAtomV{}, Shape<Int<kBc>, Int<kVDChunk>>{}));
  using SmemLayoutVt = decltype(composition(
      SmemLayoutV{},
      make_layout(Shape<Int<kVDChunk>, Int<kBc>>{}, GenRowMajor{})));
  // O 的 smem 暂存: 与 V 同 SW128 atom, epilogue 复用已释放的 QKV smem。
  using SmemLayoutO =
      decltype(tile_to_shape(SmemAtomV{}, Shape<Int<kBr>, Int<kVDChunk>>{}));

  using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;
  // M8N1: kNumWarps 个 warp 全部沿 M 堆叠, N 不切分 -> 每 warp 持整行,
  // softmax 归约只需 warp 内 shfl_xor 4-lane, 无需跨 warp smem 归约。
  using TiledMmaQK = decltype(make_tiled_mma(
      MmaAtom{}, Layout<Shape<Int<kNumWarps>, _1, _1>>{},
      Tile<Int<kBr>, Int<kBc>, _16>{}));
  using TiledMmaPV = decltype(make_tiled_mma(
      MmaAtom{}, Layout<Shape<Int<kNumWarps>, _1, _1>>{},
      Tile<Int<kBr>, Int<kVDChunk>, _16>{}));

  using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
  using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
};
```

Q/K chunk 行宽 32×2=64 B，SW64 的一个 swizzle 周期恰好覆盖，用 SW128 反而浪费（padding 到 128 B 行）；V/O chunk 行宽 128 B 落回 SW128。`SmemLayoutVt` 是 V 的转置**视图**（composition，不物化），PV 的 B-operand 以 $D_c \times B_c$ 列主视图交给 `LDSM_T`。`SmemLayoutO` 与 V 同 atom——epilogue 复用已释放的 QKV smem 时布局保持一致（§29.3.8）。

29.3.2 M8N1 TiledMma：warp 持整行，softmax 归约零跨 warp

QK 与 PV 共用 m16n8k16 atom（sm_120 唯一张量核路径，第 二十八 章已论证），差别在 TiledMma 的 warp 排布。两处 `Layout<Shape<8,1,1>>` 把 8 个 warp **全部沿 M 堆叠、N 不切分**（M8N1）：每个 warp 独占 S tile 的 16 行**整行**。这带来一个 softmax 侧的直接收益——m16n8k16 的 C-fragment 约定下，一行 128 个分数散在同 warp 的 4 个 lane（每 lane 2×8 列块中的 2 份），行 max / 行 sum 的归约只需要 warp 内 `__shfl_xor` 两轮（1 与 2 的蝶形，§29.3.6），**没有任何跨 warp 的 smem 归约**。第 二十七 章 WS 版的 64×64 tile 下 warp 沿 N 也要切，归约需要

先跨 warp——M8N1 下行归约自然落在 warp 内。代价是 PV 的 N 维只有 $k_{VDChunk}$=64：单 warp 的 PV MMA 每次算 16×64，由 `gemm_rs`（第 二十六 章）内部按 atom 循环展开，不影响协议。

装载/写回 atom：Q/K 用 `U32x4_LDSM_N`（128 bit 对齐直读），V 用 `U16x8_LDSM_T`（转置装载），O 写回用 `SM90_U32x4_STSM_N`（`stmatrix`，§29.3.8）。

29.3.3　non-WS 执行模型：全员 MMA 与内联 TMA

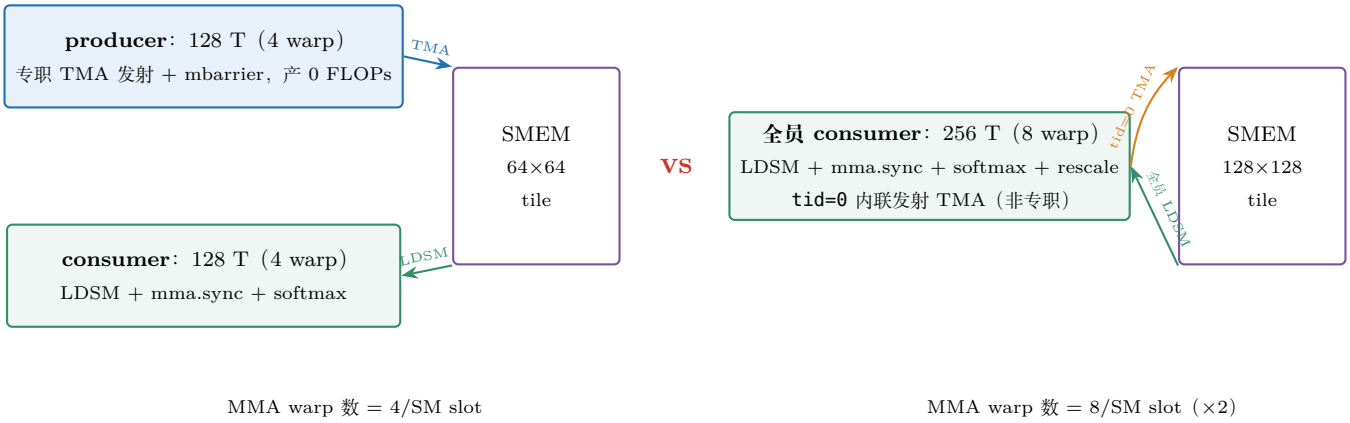


图 29.1: WS 与 non-WS 执行模型对照。左（第 二十七 章 WS 教学版）：128 线程专职 producer 与 128 线程 consumer 分工，smem 为交接面——搬运与计算解耦的代价是 SM 上做 MMA 的 warp 只剩一半。右（本章）：256 线程全员 consumer，`tid=0` 在消费循环内联发射 TMA（发射指令本身不占独立 warp 的时间片，只占少量指令槽）；tile 同步升到 128×128，MMA warp 数从 4 增至 8，算术强度随之提高。大 D 下 O 累加器占满寄存器预算，`setmaxnreg` 腾不出寄存器，WS 的收益侧消失——non-WS 成为唯一选择。

non-WS 不是「没有 warp specialization」这么简单，它要求发射 TMA 的线程同时是消费线程，协议上有两个硬约束。其一，**发射点必须挂在消费依赖之后**：`tid=0` 要复写某个 stage，必须先等全 CTA 的 256 个线程都消费完（`qk_empty` 的 init count 是 $k_{NumThreads}$），而 `tid=0` 自己也是消费者之一——预取代码若挪到 `gemm_ss` 之前，`tid=0` 会卡在等待一个需要它自己参与释放的 barrier 上，确定性死锁（源码注释原话）。其二，**发射路径要短**：发射代码混在 MMA 循环里，每次发射的指令数直接挤占 tensor core 的指令带宽——所以发射被封装成两个 lambda，每个只做 `fence` + `arrive_and_expect_tx` + `copy` 三步。

smem 与 barrier 的静态布局如下，注意 Q/K 共用 S_K 份、V 独占 S_V 份：

ffpa_attn.cuh L880-913（smem 三段池；full/empty 双 barrier 集初始化）

```
// SMEM: [q_base | k_base | v_base], 各自 kStages 份。
extern __shared__ __align__(1024) Element shm[];
Element *q_base = shm;
Element *k_base = q_base + kStagesQK * kQChunkElements;
Element *v_base = k_base + kStagesQK * kKChunkElements;

__shared__ uint64_t qk_full[kStagesQK];
__shared__ uint64_t qk_empty[kStagesQK];
__shared__ uint64_t v_full[kStagesPV];
__shared__ uint64_t v_empty[kStagesPV];

if (tid == 0) {
  for (int s = 0; s < kStagesQK; ++s) {
    TmaBarrier::init(&qk_full[s], 1);
    CtaBarrier::init(&qk_empty[s], kNumThreads);
  }
  for (int s = 0; s < kStagesPV; ++s) {
    TmaBarrier::init(&v_full[s], 1);
    CtaBarrier::init(&v_empty[s], kNumThreads);
  }
```

```
  }
  __syncthreads();

  auto q_slice = tma_q.get_slice(_0{});
  auto k_slice = tma_k.get_slice(_0{});
  auto v_slice = tma_v.get_slice(_0{});

  TiledMmaQK tiled_mma_qk;
  TiledMmaPV tiled_mma_pv;
  auto thr_mma_qk = tiled_mma_qk.get_thread_slice(tid);
  auto thr_mma_pv = tiled_mma_pv.get_thread_slice(tid);

  auto s2r_copy_q = make_tiled_copy_A(SmemCopyAtom{}, tiled_mma_qk);
  auto s2r_copy_k = make_tiled_copy_B(SmemCopyAtom{}, tiled_mma_qk);
```

`qk_full/v_full` 是**事务屏障**（TmaBarrier，init=1）：`tid=0 arrive_and_expect_tx` 声明本 stage 期望的字节数，TMA 写完自动翻转。`qk_empty/v_empty` 是 **计数屏障**（CtaBarrier，init=256）：每个消费线程 arrive 一次，凑满后 `tid=0` 才能复写。两套 barrier 集合完全不相交——这是 §29.3.5 跨 tile 预取零死锁的完整机制。QK 与 PV 的 chunk 序号统一编址：

$$\text{chunk_index} = \text{kv_tile} \cdot k_{DChunks} + d, \qquad \text{stage} = \text{chunk_index} \bmod S, \qquad \text{phase} = \left\lfloor \text{chunk_index}/S \right\rfloor \& 1, \quad (29.2)$$

与第 二十 章 19.3.6 节的相位计数同构，区别只有：QK 流水与 V 流水各自维护一套 `chunk_index`（除数分别是 $k_{DChunksQK}/k_{DChunksV}$），互不干扰。

发射 lambda 本体（两个结构对称，QK 版多一个 Q 的 co-copy）：

ffpa_attn.cuh L975-1006（`issue_qk_tma/issue_v_tma`：tid=0 内联发射）

```
  auto issue_qk_tma = [&](int d_chunk, int stage, int kv_tile_idx) {
    cutlass::arch::fence_view_async_shared();
    auto sQ = make_tensor(make_smem_ptr(q_base + stage * kQChunkElements),
                          SmemLayoutQ{});
    auto sK = make_tensor(make_smem_ptr(k_base + stage * kKChunkElements),
                          SmemLayoutK{});
    auto gQ = local_tile(mQ, Shape<Int<kBr>, Int<kQKDChunk>>{},
                         make_coord(q_tile, d_chunk));
    auto gK = local_tile(mK, Shape<Int<kBc>, Int<kQKDChunk>>{},
                         make_coord(kv_tile_idx, d_chunk));
    auto tQgQ = q_slice.partition_S(gQ);
    auto tQsQ = q_slice.partition_D(sQ);
    auto tKgK = k_slice.partition_S(gK);
    auto tKsK = k_slice.partition_D(sK);
    TmaBarrier::arrive_and_expect_tx(&qk_full[stage],
                                     sizeof(Element) * (size(sQ) + size(sK)));
    copy(tma_q.with(qk_full[stage]), tQgQ, tQsQ);
    copy(tma_k.with(qk_full[stage]), tKgK, tKsK);
  };

  auto issue_v_tma = [&](int v_chunk, int stage, int kv_tile_idx) {
    cutlass::arch::fence_view_async_shared();
    auto sV = make_tensor(make_smem_ptr(v_base + stage * kVChunkElements),
                          SmemLayoutV{});
    auto gV = local_tile(mV, Shape<Int<kBc>, Int<kVDChunk>>{},
                         make_coord(kv_tile_idx, v_chunk));
    auto tVgV = v_slice.partition_S(gV);
    auto tVsV = v_slice.partition_D(sV);
    TmaBarrier::arrive_and_expect_tx(&v_full[stage],
                                     sizeof(Element) * size(sV));
    copy(tma_v.with(v_full[stage]), tVgV, tVsV);
  };
```

Q 与 K 的 chunk 在同一个 `expect_tx` 里声明（字节数相加，一次事务屏障管两份拷贝），`local_tile` 的第五参 `_1{}` 声明沿 minor 维一次搬满。K/V 的 gmem 坐标要先经 `domain_offset` 偏到 head 段（$h \cdot N_{kv}$ 未必 128 对齐，N_{kv} 非 k_{Bc} 倍数时直接用全局 tile 索引会串到相邻 head 的行，本章测试的 N_{kv}=192 用例就是为这条边界写的）。

初始预取在主循环之前把两条流水都填满：

ffpa_attn.cuh L1008-1026（初始预取：QK 填满 S_K，V 填满 S_V）

```
  // 初始 QK 预取: 填满前 kStagesQK 个 chunk。
  if (tid == 0) {
    for (int d = 0; d < kStagesQK && d < kDChunksQK; ++d) {
      CtaBarrier::wait(&qk_empty[d], 0);
      issue_qk_tma(d, d, 0);
    }
  }

  // 初始 V 预取 (kv_tile 0): V 与 QK 无数据依赖, 在 QK GEMM 前发出,
  // 让 V TMA 重叠整个首个 QK GEMM + softmax 窗口。
  if (tid == 0) {
    for (int v = 0; v < kStagesPV && v < kDChunksV; ++v) {
      const int chunk_index = v;
      const int v_stage = chunk_index % kStagesPV;
      const int v_phase = (chunk_index / kStagesPV) & 1;
      CtaBarrier::wait(&v_empty[v_stage], v_phase);
      issue_v_tma(v, v_stage, 0);
    }
  }
```

细看 **V 的初始预取位置**：V 与 QK 无数据依赖，在第一个 QK GEMM 之前就把 S_V 份 V chunk 发出去，V 的 TMA 传输重叠掉整个首个 QK GEMM + softmax 窗口。此后每个 kv_tile 开头再补发本 tile 的首批 V（L1030–1040），同样领先于 QK 消费。

29.3.4　Phase 1：QK split-D 累加与领先预取

主循环体（`#pragma unroll 1`：kv_tile 循环不展开，控制 icache 与寄存器压力）内的 QK 相位：

ffpa_attn.cuh L1043-1082（Phase 1：chunk 流水上的 $S = \sum_d Q_d K_d^\top$）

```
    auto tCrS = partition_fragment_C(tiled_mma_qk, Shape<Int<kBr>, Int<kBc>>{});
    clear(tCrS);

#pragma unroll
    for (int d_chunk = 0; d_chunk < kDChunksQK; ++d_chunk) {
      const int chunk_index = kv_tile * kDChunksQK + d_chunk;
      const int stage = chunk_index % kStagesQK;
      const int phase = (chunk_index / kStagesQK) & 1;
      TmaBarrier::wait(&qk_full[stage], phase);
      cutlass::arch::fence_view_async_shared();

      auto sQ = make_tensor(make_smem_ptr(q_base + stage * kQChunkElements),
                            SmemLayoutQ{});
      auto sK = make_tensor(make_smem_ptr(k_base + stage * kKChunkElements),
                            SmemLayoutK{});
      auto tCrQ = thr_mma_qk.partition_fragment_A(sQ);
      auto tCrK = thr_mma_qk.partition_fragment_B(sK);
      auto tQsQ = s2r_thr_q.partition_S(sQ);
      auto tKsK = s2r_thr_k.partition_S(sK);

      fa_cute::gemm_ss(tCrS, tCrQ, tCrK, tQsQ, tKsK, tiled_mma_qk,
                       s2r_copy_q, s2r_copy_k, s2r_thr_q, s2r_thr_k);

      CtaBarrier::arrive(&qk_empty[stage]);
```

```
25
26      // 预取不能移到 gemm_ss 之前: s_next == stage 且 phase_next == 1-phase,
27      // wait 门控在本迭代的 arrive 上, 提前会死锁 (tid=0 卡在等待需要
28      // tid=0 自己参与的 gemm_ss 释放的 barrier)。
29      if (tid == 0) {
30        const int d_next = d_chunk + kStagesQK;
31        if (d_next < kDChunksQK) {
32          const int next_index = kv_tile * kDChunksQK + d_next;
33          const int s_next = next_index % kStagesQK;
34          const int phase_next = (next_index / kStagesQK) & 1;
35          CtaBarrier::wait(&qk_empty[s_next], phase_next);
36          issue_qk_tma(d_next, s_next, kv_tile);
37        }
38      }
39    }
```

逐行拆解。**L1043–1044**：S 的 f32 累加器 fragment 每 tile 清零（split-D 的 S 是 tile 内变量，不跨 tile）。**L1047–1052**：按式 (29.2) 分解出 (stage, phase)，等 `qk_full` 的数据就绪。**L1054–1064**：`partition_fragment_A/B` 在寄存器里切出本线程的 Q/K fragment，`gemm_ss`（第 二十六 章工具）完成 LDSM + mma.sync 的 k-loop——10 个 chunk 循环下来 $S = \sum_d Q_{[:,d]} K_{[:,d]}^\top$，正是第 二十 章 19.3.2 节「精确、无权重」的分块分解在 CuTe 上的直接落地。**L1066**：全线程 arrive `qk_empty`，stage 归还。**L1072–1081**：`tid=0` 领先 S_K 个 chunk 预取下一个（`d_next = d_chunk + kStagesQK`）。注释里的死锁警告值得原样引用：预取不能移到 `gemm_ss` 之前，因为 $s_{next} = stage$ 且 $phase_{next} = 1 - phase$，wait 门控在**本迭代**的 arrive 上，提前了 `tid=0` 就在等一个需要它自己参与 `gemm_ss` 才能释放的 barrier。

29.3.5 跨 tile 预取：barrier 集不相交的收益

QK 相位结束时，本 tile 的 K 已消费完、Q 也不再需要，但 softmax 与 PV 还要跑一整段。此时 QK 的 stage 池整体空闲——下一 tile 的初始 QK chunk 现在就发：

ffpa_attn.cuh L1083-1094（跨 tile 预取：下一 tile 的 QK 初始 chunk）

```
1     // 跨 tile 预取: 下一 kv_tile 的 QK 初始 chunks 在本 tile 的 softmax+PV
2     // 之前发出, TMA 与计算重叠。QK barrier 集与 softmax/PV 用的 v_*
3     // barrier 不相交, 放这里零死锁。
4     if (kv_tile < kv_tiles - 1 && tid == 0) {
5       for (int d = 0; d < kStagesQK && d < kDChunksQK; ++d) {
6         const int chunk_index = (kv_tile + 1) * kDChunksQK + d;
7         const int stage = chunk_index % kStagesQK;
8         const int phase = (chunk_index / kStagesQK) & 1;
9         CtaBarrier::wait(&qk_empty[stage], phase);
10        issue_qk_tma(d, stage, kv_tile + 1);
11      }
12    }
```

这段代码能放在 softmax/PV 之前的前提，是 **QK barrier 集与 V barrier 集不相交**：它等待的 `qk_empty` 由 Phase 1 的 arrive 释放，与 softmax 要等的 `v_full`、PV 要 arrive 的 `v_empty` 没有任何交集，所以不可能与后续相位互相等待。时序上的效果是：下一个 tile 的 Q chunk + K chunk 的 TMA 传输，完全被本 tile 的 softmax（纯寄存器 + shfl）与 PV GEMM 掩盖。

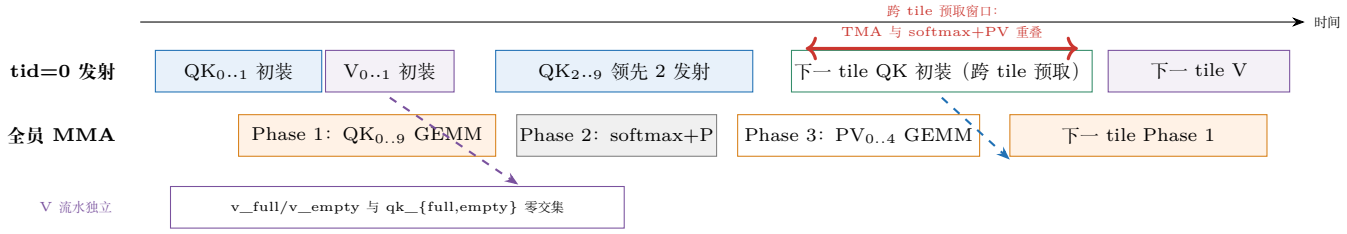


图 29.2: 单 kv_tile 内的相位时间线与跨 tile 预取（S_K=S_V=2、D=320 的 10+5 chunk 结构示意）。上排是 `tid=0` 的发射序列：初始预取填满两条流水，主循环内每个 QK chunk 消费完领先 S_K 发下一个，V 预取每 tile 开头补一批；本 tile QK 相位一结束（stage 池整体空闲），立刻发射下一 tile 的初始 QK chunk。下排是全员 MMA 的计算序列。红色双箭头是本章协议的核心收益：下一 tile 的 QK 装载与本 tile 的 softmax（纯寄存器 + shfl）和 PV GEMM 完全重叠——QK/V 两套 barrier 集不相交保证这段发射不会与后续相位互相等待。

29.3.6　Phase 2：边界 mask、lazy rescale 与 P 的零拷贝

softmax 相位从 Nkv 边界 mask 开始，到 P fragment 准备好结束：

ffpa_attn.cuh L1096-1131（Phase 2：mask + online softmax + warp vote + P 零拷贝）

```
    // Phase 2: online softmax + P fragment 准备。
    {
      auto scores = make_tensor(
          tCrS.data(), fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
      float row_scale[kORows];

      // Nkv 边界 mask: 尾部 tile 的 OOB 列置 -INF (TMA 对 OOB 行不写
      // smem, 残留值由 mask 覆盖)。
      const int kv_valid = nkv - kv_tile * kBc;
      if (kv_valid < kBc) {
#pragma unroll
        for (int row = 0; row < kSRows; ++row)
#pragma unroll
          for (int col = 0; col < size<1>(scores); ++col) {
            if (get<1>(tScS_rc(row, col)) >= kv_valid)
              scores(row, col) = -INFINITY;
          }
      }

      fa_cute::online_safe_softmax_scaled<decltype(scores), kORows>(
          scores, scale, row_max, row_sum, row_scale,
          Traits::kRescaleThreshold);

      // warp-uniform vote: 全 warp 的 row_scale 都是 1.0 时跳过 O 重缩放,
      // 无分支发散。
      bool local_need_rescale = false;
#pragma unroll
      for (int r = 0; r < kORows; ++r)
        local_need_rescale = local_need_rescale || (row_scale[r] < 1.0f);
      const bool need_rescale = __any_sync(0xffffffff, local_need_rescale);

      // P fragment: f32 -> f16 后把 C layout 重解释为 A 寄存器 (零拷贝)。
      auto tCrP = fa_cute::convert_type<Element>(tCrS);
      auto tCrPv = make_tensor(
          tCrP.data(),
          fa_cute::convert_layout_acc_Aregs<TiledMmaPV>(tCrP.layout()));
```

三段各有要点。**mask（L1102–1117）**：尾部 kv_tile 的越界列置 $-\infty$——TMA 对 OOB 行不写 smem，残留值由 mask 覆盖；坐标来自 identity tensor 经 TiledMMA C-partition 的行列重解释（第 二十七 章讲透的 `tScS_rc` 技巧），mask 只剩坐标比较。**softmax（L1119–1121）**：`online_safe_softmax_scaled` 是本章的变体，与第 二十八 章

`fa4` 版的差异只有一处——scale 没有折叠进 Q fragment（Q 每 tile 重装，折叠收益减半，不如直接在 max/exp 循环里乘），数学上仍是式 (26b.1)–(26b.3) 加 FA-4 条件重缩放：

ffpa_attn.cuh L758-793（`online_safe_softmax_scaled`：蝶形归约 + 条件重缩放判定）

```
// Online safe softmax (scale 乘法版 + FA-4 conditional rescaling)。
// S 是 QK 裸分数, scale 已含 log2(e) 折叠; 与 persist-D 的
// online_safe_softmax_fa4 (scale 预折叠进 Q fragment) 差异仅在
// max/exp 循环内的逐元素乘。行内 4-lane 蝴蝶归约 (m16n8k16 一行
// 散在 4 个 lane: xor 1 + xor 2)。
// FA-4 conditional rescale: log2_diff = m_old - m_new <= 0, 当
// log2_diff >= -threshold (max 增长 < 2^8) 时 row_scale=1 且 row_max
// 保持旧值 (stale max), 跳过 o_acc 重缩放; O 与 row_sum 用同一 stale
// max 累积, epilogue 的 O/row_sum 相消, 数学上等价。
template <typename ScoresTensor, int kRows>
CUTE_DEVICE void online_safe_softmax_scaled(ScoresTensor &scores, float scale,
                                            float *row_max, float *row_sum,
                                            float *row_scale,
                                            float rescale_threshold) {
#pragma unroll
  for (int row = 0; row < kRows; ++row) {
    float tile_max = -INFINITY;
#pragma unroll
    for (int col = 0; col < cute::size<1>(scores); ++col)
      tile_max = fmaxf(tile_max, scores(row, col) * scale);
    tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
    tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
    const float next_max = fmaxf(row_max[row], tile_max);
    const float log2_diff = row_max[row] - next_max;
    float eff_max = next_max;
    if (log2_diff >= -rescale_threshold) {
      row_scale[row] = 1.0f;
      eff_max = row_max[row];  // stale max; row_max 不更新
    } else {
      row_scale[row] = exp2f(log2_diff);
      row_max[row] = next_max;
    }
    float tile_sum = 0.0f;
#pragma unroll
    for (int col = 0; col < cute::size<1>(scores); ++col) {
      const float p = exp2f(scores(row, col) * scale - eff_max);
```

每行先在本地列里扫 tile max（乘 scale，log2 域），`shfl_xor(1)` 与 `shfl_xor(2)` 两轮把 4 个 lane 的局部 max 归并成行 max——M8N1 布局（§29.3.2）保证这 4 个 lane 就在同 warp，蝶形两轮全覆盖，**没有跨 warp 归约**。判定段是 FA-4 的原样移植：$\log_2$_diff $\geq -\tau$（τ=8）时 `row_scale=1`、`row_max` 保持 stale 值，**跳过对 O/ℓ 的历史修正**；膨胀的常数因子在 epilogue 的 $1/\ell$ 里分子分母同乘抵消（第 二十八 章式 (26b.5) 的推导）。调用方的 `__any_sync`（L1121–1129）把 per-row 判定聚合成 warp 粒度的统一分支：全 warp 的 row_scale 都是 1 才跳过，warp 内零分歧。**P 零拷贝（L1127–1131）**：softmax 后的 f32 scores 经 `convert_type` 压 fp16，再经 `convert_layout_acc_Aregs` 原地重排成 PV 的 A-operand 布局——P 不落 smem，寄存器直通下一个 MMA（第 二十七 章 26.3.3 节的布局重解释，本章原样复用）。

29.3.7 Phase 3：PV split-D 与条件重缩放的落点

ffpa_attn.cuh L1133-1165（Phase 3：O_v += PV_v 与 lazy rescale）

```
      // Phase 3: PV split-D, O[Br, v_chunk] += P[Br,Bc] @ V[Bc, v_chunk]。
#pragma unroll
      for (int v_chunk = 0; v_chunk < kDChunksV; ++v_chunk) {
        const int chunk_index = kv_tile * kDChunksV + v_chunk;
        const int v_stage = chunk_index % kStagesPV;
```

```
        const int v_phase = (chunk_index / kStagesPV) & 1;
        TmaBarrier::wait(&v_full[v_stage], v_phase);
        cutlass::arch::fence_view_async_shared();

        auto sV = make_tensor(make_smem_ptr(v_base + v_stage * kVChunkElements),
                              SmemLayoutV{});
        auto sVt = make_tensor(sV.data(), SmemLayoutVt{});
        auto tCrVStorage = thr_mma_pv.partition_fragment_B(sV);
        auto tCrV = make_tensor(tCrVStorage.data(), tCrV_layout);
        auto tVsVt = s2r_thr_v.partition_S(sVt);

        auto tCrO = make_tensor(make_rmem_ptr(&o_acc_storage[v_chunk][0]),
                                OFragLayout{});
        if (kv_tile > 0 && need_rescale) {
          auto tCrO_rc = make_tensor(
              tCrO.data(),
              fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
#pragma unroll
          for (int row = 0; row < kORows; ++row)
#pragma unroll
            for (int col = 0; col < kOCols; ++col)
              tCrO_rc(row, col) *= row_scale[row];
        }

        fa_cute::gemm_rs(tCrO, tCrPv, tCrV, tVsVt, tiled_mma_pv, s2r_copy_v,
                         s2r_thr_v);

        CtaBarrier::arrive(&v_empty[v_stage]);
```

PV 相位逐 v_chunk 消费（D=320 时 5 个），每个 chunk：等 `v_full`（式 (29.2) 的 V 侧编址）、构造 V 的转置视图 fragment、`gemm_rs` 完成 PV GEMM、arrive `v_empty`、`tid=0` 领先 S_V 预取下一个 V chunk。**重缩放的位置（L1151–1161）**：`need_rescale` 为真时，**先**对本 v_chunk 的 O 分量乘 `row_scale`，**再**累加新 P 的贡献——顺序不可换，否则新贡献也会被缩掉。注意 `kv_tile > 0` 的 guard：首个 tile 无历史可缩。O 累加器是跨全部 kv_tile 常驻的 `o_acc_storage[kDChunksV][32]`，D=320 时 $5 \times 32 = 160$ 个 f32/线程——第 二十 章 19.4.6 节 $32 \times C$ 寄存器占用的 C=5 情形，single-pass online softmax 的结构性寄存器成本，rescale 需要所有 C 个切片同时在位，不能提前流出。（V fragment 的寄存器布局在 kernel 开头用 nonswizzle 视图预计算一次——布局与 swizzle 无关，LDSM_T 实际读仍走 swizzled 的 `sVt`，bank 冲突由 TMA 写入侧的 swizzle 消解，与 flash-attention `kernel_traits.h` 的做法相同。）

29.3.8　Phase 4：STSM 批量暂存与 TMA store bulk group

KV 循环结束后，O 除以 ℓ（FA-4 的膨胀在此全部抵消），走与装载对称的反向 TMA 全链。先把 C 个 v_chunk 分批：每批 $k_{VChunksPerBatch}$ 个 chunk 共享一个 TMA store bulk group，批内 STSM 暂存 + 一次发射 + 一次 arrive；D=320 时 $k_{VChunksPerBatch}$=5、$k_{NBatches}$=1（全部一批，bulk group 数最少）。暂存段（R→S）：

ffpa_attn.cuh L1204-1226（epilogue 批 I：$\div\ell$、f16、STSM 暂存复用 QKV smem）

```
    for (int batch = 0; batch < kNBatches; ++batch) {
#pragma unroll
      for (int v_in = 0; v_in < kVChunksPerBatch; ++v_in) {
        const int v_chunk = batch * kVChunksPerBatch + v_in;
        auto tCrO = make_tensor(make_rmem_ptr(&o_acc_storage[v_chunk][0]),
                                OFragLayout{});
        auto tCrO_rc = make_tensor(
            tCrO.data(), fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
#pragma unroll
        for (int row = 0; row < kORows; ++row) {
          const float inv_sum = 1.0f / row_sum[row];
#pragma unroll
```

```
          for (int col = 0; col < kOCols; ++col)
            tCrO_rc(row, col) *= inv_sum;
        }
        auto tCrOHalf = fa_cute::convert_type<Element>(tCrO);
        auto sO_v = make_tensor(make_smem_ptr(shm + v_in * kOTileElems),
                                SmemLayoutO{});
        auto tCrOHalf_src = r2s_thr.retile_S(tCrOHalf);
        auto tCsO_dst = r2s_thr.partition_D(sO_v);
        copy(r2s_copy, tCrOHalf_src, tCsO_dst);
      }
      cutlass::arch::fence_view_async_shared();
```

入口的 `__syncthreads()` 保证 V smem 读完后 R→S 才能覆写；`SmemLayoutO` 与 V 同 SW128 atom，暂存直接落在已释放的 QKV smem 上。`SM90_U32x4_STSM_N`（`stmatrix`）是 `ldmatrix` 的镜像：每线程 4 个 32 bit 寄存器按 fragment 布局写进 swizzled smem，O 的 128×64 分量一段写完。写回段（S→G）：

ffpa_attn.cuh L1227-1246（epilogue 批 II：TMA store bulk group + drain）

```
      __syncthreads();
#pragma unroll
      for (int v_in = 0; v_in < kVChunksPerBatch; ++v_in) {
        const int v_chunk = batch * kVChunksPerBatch + v_in;
        auto sO_v = make_tensor(make_smem_ptr(shm + v_in * kOTileElems),
                                SmemLayoutO{});
        auto gO = local_tile(mO, Shape<Int<kBr>, Int<kVDChunk>>{},
                             make_coord(q_tile, v_chunk));
        auto tCgO = o_slice.partition_D(gO);
        auto tOsO = o_slice.partition_S(sO_v);
        if (tid == 0) {
          copy(tma_o, tOsO, tCgO);
        }
      }
      tma_store_arrive();
      if (batch < kNBatches - 1) {
        tma_store_wait<0>();  // drain, shm 才能复用
        __syncthreads();
      }
    }
```

只有 `tid=0` 发 `copy(tma_o, ...)`，`tma_store_arrive` 把批内所有 store 提交进 bulk group。随后的 `tma_store_wait<0>` 是 **drain race 的解**：非发射线程的 wait 是空操作，若无 CTA 级同步，下一批的 STSM 会覆写 in-flight TMA 仍在读的 smem——$k_{NBatches} \geq 2$ 时确定性写坏。批末的 `__syncthreads()` 让全线程等 drain（批条件 CTA 一致，不会死锁）。LSE 的写出与最后一批 drain 重叠（log2 域转回 ln），`softmax_lse` 为 null 时整体跳过。

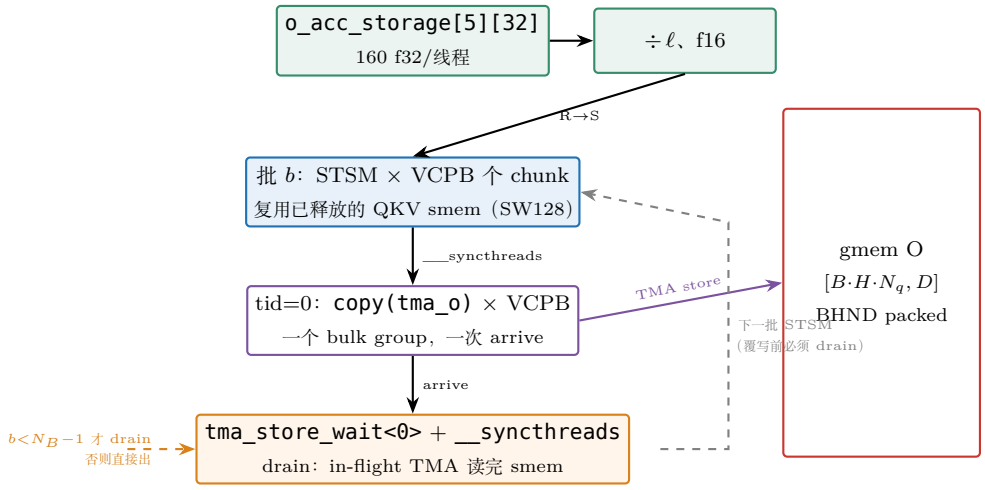


图 29.3: epilogue 的批量写出协议（D=320：5 个 v_chunk 一批，VCPB = $k_{VChunksPerBatch}$）。O 累加器除以 ℓ 转 f16 后，STSM 把每 chunk 的 128×64 分量暂存进复用的 QKV smem；`tid=0` 对批内所有 chunk 发 TMA store 并提交一个 bulk group。灰色回边是 drain race 的约束：下一批的 STSM 要覆写同一段 smem，必须等 in-flight TMA 读完（`tma_store_wait<0>` 只对发射线程有意义，配套 `__syncthreads` 让全 CTA 对齐；末批免等直接收尾）。对比 WS 教学版的逐元素寄存器直写 gmem：指令数从每线程 $32C$ 次标量 store 降到 VCPB 次 STSM + 一次 bulk 提交。

29.3.9　launcher：BHND 压平与四方向 descriptor

ffpa_attn.cuh L1277-1304（launcher：2D 压平 + 四方向 TMA + opt-in smem）

```
const int rows_q = B * H * nq;
const int rows_kv = B * H * nkv;
auto gq = make_tensor(make_gmem_ptr(Q), make_shape(rows_q, Int<kHeadDim>{}),
                      make_stride(Int<kHeadDim>{}, _1{}));
auto gk = make_tensor(make_gmem_ptr(K), make_shape(rows_kv, Int<kHeadDim>{}),
                      make_stride(Int<kHeadDim>{}, _1{}));
auto gv = make_tensor(make_gmem_ptr(V), make_shape(rows_kv, Int<kHeadDim>{}),
                      make_stride(Int<kHeadDim>{}, _1{}));
auto go = make_tensor(make_gmem_ptr(O), make_shape(rows_q, Int<kHeadDim>{}),
                      make_stride(Int<kHeadDim>{}, _1{}));
auto tma_q = make_tma_copy(SM90_TMA_LOAD{}, gq, SmemLayoutQ{},
                           Shape<_128, _32>{}, _1{});
auto tma_k = make_tma_copy(SM90_TMA_LOAD{}, gk, SmemLayoutK{},
                           Shape<_128, _32>{}, _1{});
auto tma_v = make_tma_copy(SM90_TMA_LOAD{}, gv, SmemLayoutV{},
                           Shape<_128, _64>{}, _1{});
auto tma_o = make_tma_copy(SM90_TMA_STORE{}, go, SmemLayoutO{},
                           Shape<_128, _64>{}, _1{});

auto kernel = ffpa_attn_tma_split_d_cute<
    Traits, decltype(tma_q), decltype(tma_k), decltype(tma_v),
    decltype(tma_o)>;
const int smem_bytes = Traits::kSmemElems * sizeof(cutlass::half_t);
cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                     smem_bytes);
dim3 grid(nq / 128, B * H);
kernel<<<grid, Traits::kNumThreads, smem_bytes>>>(
    tma_q, tma_k, tma_v, tma_o, O, softmax_lse, rows_q, rows_kv, nq, nkv);
```

与第 二十八 章同一套「BHND 压成 2D `(rows, D)`」的 descriptor 方案：$B/H/N$ 折叠进行坐标，head 段偏移留给 kernel 内 `domain_offset`（§29.3.3）。差异有二：其一，Q 与 KV 的行数**解耦**（`rows_q` vs `rows_kv`，支持 $N_q \neq N_{kv}$ 的 cross-attention 教学用例），K/V 的 `domain_offset` 偏移量用 $h \cdot N_{kv}$；其二，O 用 `SM90_TMA_STORE`（store 方向的 descriptor），box 与 V 对称（128×64）。grid 是 $(N_q/128,\ B \cdot H)$ 的 2D 形态，smem 按式 (29.1) opt-in。

29.4 性能分析

29.4.1 主场景与 stages 解耦实验

四条流水深度组合在同一 bench 里成对同轮测量（RTX PRO 5000，110 SM，fp16 dense）。B=1/H=32/D=320：

实现	N=8192	N=16384	备注
WS 教学版 (1,1)	86.7	96.2	第 二十七 章
WS 教学版 (2,2)	140.2	144.8	
non-WS (2,2)	188.3	183.6	
non-WS (2,3)	189.3	186.8	
non-WS (3,2)	**204.1**	**194.3**	最优
non-WS (3,3)	188.5	188.4	
cuDNN SDPA	69.7	69.9	同轮参照

两个立刻能读出的结论：**(3,2) 全面最优**（N=8192 比 (2,2) 高 8.4%）；**V 流水加深几乎总是降低性能**（(2,3) 比 (2,2) 只高 0.5%，(3,3) 比 (3,2) 低 7.6%）。复测带：用户独立一轮实测 non-WS (2,2)/(3,3) 在 N=16384 为 184.4/189.0，与表中 183.6/188.4 跨 run 吻合 ±0.5%，成对结论稳定。

全 D 扫描（N=8192，其余同上）把「stages 最优值分段」的规律摊开：

D	(2,2)	(2,3)	(3,2)	(3,3)	最优
192	202.2	177.4	**216.5**	187.8	(3,2)
256	206.4	190.3	**211.2**	198.5	(3,2)
320	188.3	189.3	**204.1**	188.5	(3,2)
384	**187.2**	167.4	158.3	136.0	(2,2)
448	167.4	**180.0**	169.9	173.0	(2,3)
512	**170.9**	168.5	169.5	163.4	(2,2)

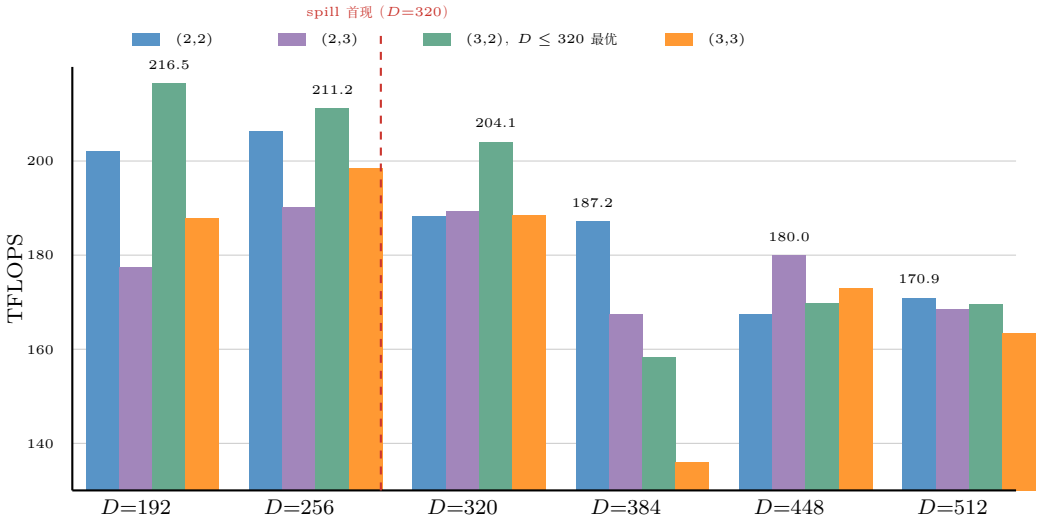


图 29.4: K/V 流水深度解耦实验（B=1/H=32/N=8192，PRO 5000，成对同轮）。$D \le 320$：(S_K, S_V)=(3, 2) 一致最优，绿色系柱全面领先；$D \ge 384$：最优回落到 (2, 2)/(2, 3)，(3, 3) 在 D=384 降至 136.0（比 (2,2) 低 27%）。红色虚线是分界：恰在寄存器 spill 首次出现处（见下表，D=320 全组合 spill、$D \le 256$ 零 spill）。

29.4.2 机理：寄存器 spill 决定 stages 的最优值

stages 影响性能的通道不是 smem（式 (29.1) 的 64–96 KB 离上限都远），而是**寄存器溢出**。用 `nvcc -Xptxas -v` 对各组合显式实例化取证：

组合	寄存器	spill stores	spill loads	N=8192 TFLOPS	相对
D=192 (2,2)	254	0 B	0 B	202.2	—
D=256 (2,2)	255	0 B	0 B	206.4	—
D=256 (3,2)	251	0 B	0 B	211.2	+2.3%
D=320 (2,2)	255	224 B	356 B	188.3	—
D=320 (2,3)	255	216 B	180 B	189.3	+0.5%
D=320 (3,2)	255	**52 B**	**36 B**	**204.1**	+8.4%
D=320 (3,3)	255	52 B	36 B	188.5	+0.1%
D=384 (2,2)	255	584 B	544 B	187.2	—
D=384 (3,2)	255	676 B	1000 B	158.3	−15.5%
D=384 (3,3)	255	928 B	1256 B	136.0	−27.4%
D=448 (2,2)	255	1084 B	852 B	167.4	—
D=448 (2,3)	255	996 B	688 B	180.0	+7.5%
D=512 (2,2)	255	1436 B	1024 B	170.9	—
D=512 (3,3)	255	1124 B	752 B	163.4	−4.4%

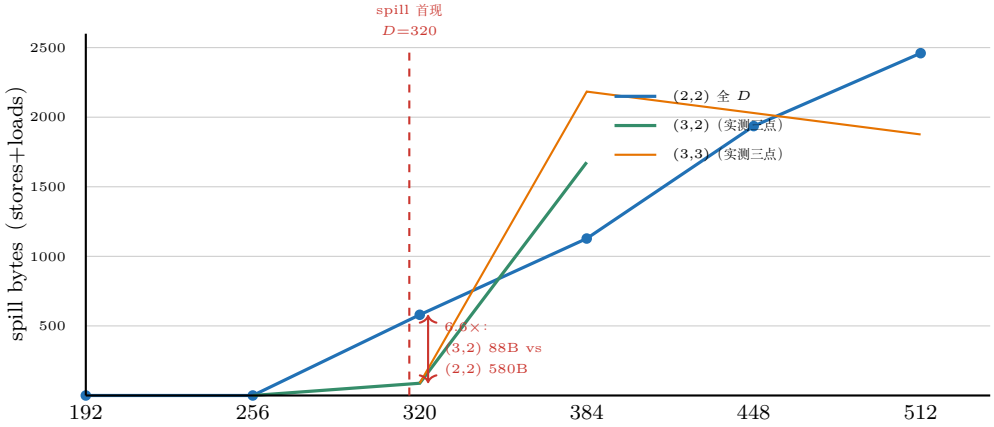


图 29.5: 寄存器溢出随 D 的增长（`nvcc -Xptxas -v`，$-O2$，sm_120a，各组合显式实例化）。$D \le 256$ 全组合零 spill；D=320 （红色虚线）是分水岭——所有组合达到 255 寄存器上限并开始溢出，且 (3,2)/(3,3) 的 88 B 与 (2,2) 的 580 B 相差 6.6 倍，与 TFLOPS 排序（204.1 对 188.3）完全同序；$D \ge 384$ 后 spill 基线过千，stages 加深进一步推高溢出（(3,2) 1676 B、(3,3) 2184 B，均高于 (2,2)），流水掩蔽收益被抵消，最优组合回落 $(2,2)$。

三个事实链。**第一**，$D \le 256$ 零 spill：stages 加深的收益全部来自 QK 流水掩蔽——QK 是主计算路径（每 tile $D/32$ 个 chunk 要发射），S_K=3 让 tid=0 的发射领先更远、TMA 传输更深地被 GEMM 窗口掩盖；而 V 每 tile 只消费 $D/64$ 个 chunk 且消费点稀疏（只在 softmax 之后），S_V=2 已够用，第 3 级 stage 的 barrier 维护纯属开销——这就是 $(3,2)$ 全面领先的 $D \le 320$ 段。**第二**，$D \ge 320$ 后 `o_acc_storage`（$D/64 \times 32$ 个 f32）把寄存器顶到 255 上限、spill 出现，stages 与 spill 的交互主导排序：D=320 时 $(3,2)/(3,3)$ 的 spill 总量（52+36=88 B）只有 $(2,2)$（224+356=580 B）的六分之一——浅流水下发射地址变量与消费循环的活跃变量生命周期重叠更长、峰值压力更高，深一档反而把发射路径的变量错开，性能与 spill 字节数 **完全同序**（204.1/188.5 对 188.3/189.3，图 29.5）；$(3,2)$ 与 $(3,3)$ 同 spill 但差 15T，V 侧第 3 级 stage 的 barrier 维护开销与 96 KB smem 的副作用是剩余差距。**第三**，$D \ge 384$ 时 spill 基线本身已高（544 B+），再加 stage 挤占的寄存器超过流水掩蔽的收益，$(3,2)$ 性能开始下降、$(3,3)$ 下降 27%；此段最优回到 $(2,2)$（或 D=448 的 $(2,3)$）。

这条分段规律的工程含义：S_K 与 S_V **必须是独立模板参数**。对称的 (S,S) 在任何 D 都取不到最优——本 kernel（与 ffpa-attn 的 cute sm_120 split_d 同源）的 launcher 把 S_K/S_V 作为独立模板参数暴露，正是为了按 D 分段选参；默认对称 $(2,2)$ 只是保守起点。寄存器占用（第 二十 章 19.4.6）在这里从「预算说明」变成「调优依据」：spill 出现的 D 就是 stages 策略的切换点。

29.4.3　与 WS 版的逐项对照收尾

把 §29.2 的四个结构性问题对照回数字：全员 MMA + 128 tile（协议主体）贡献了 140→188 的主要段（(2,2) 对 WS(2,2) +34%）；跨 tile 预取与 FA-4 rescale 在其上叠加；stages 从对称 (2,2) 调到 (3,2) 再获得 +8%（188→204）；

epilogue 的 STSM+TMA 通道在 D=320 的占比不大（每 tile 只有一次 128×320 的写出），但在更短 N、更小 tile 数的场景（写出占比上升）会放大。精确到逐项的独立 A/B 需要 fork kernel 消融，本章按「结构差异 + stages 实验」的口径给出，消融矩阵留给 §29.7 的动手实验。对比 cuDNN 的 2.93× 里还有一部分来自大 D 本身——厂商库在 D=320 这类非 2 的幂 head_dim 上没有专门调优的 tile 形状，这是第 二十八 章「非标准形状」论点在 head_dim 维度的重演。

29.5 常见坑

1. **non-WS 预取的位置死锁**：`tid=0` 的预取不能移到 `gemm_ss` 之前——$s_{next}=s$ 且 $phase_{next}=1-phase$，wait 门控在本迭代的 arrive 上，提前了 `tid=0` 在等一个需要它自己参与释放的 barrier。跨 tile 预取同理只能挂在「本 tile 的 QK 相位结束、softmax/PV 之前」这个窗口。

2. **K/V 坐标必须 domain_offset 到 head 段**：$h \cdot N_{kv}$ 未必 128 对齐（N_{kv} 非 k_{Bc} 倍数时），直接用全局 tile 索引会静默串到相邻 head 的行——结果错但无任何报错，只有 $N_{kv} \bmod 128 \neq 0$ 的用例能检测到（err ∼0.1，正常用例 ∼1e-5）。Q 侧因 host 保证 $N_q \bmod 128 = 0$ 不受影响。

3. **epilogue 的 drain race**：`tma_store_wait<0>` 对非发射线程是空操作，$k_{NBatches} \geq 2$ 时缺了批末 `__syncthreads` 的话，下一批 STSM 会覆写 in-flight TMA 仍在读的 smem——确定性写坏，且写坏的是已经算完的 O。

4. **rescale 先缩后加**：Phase 3 的重缩放必须先乘 `row_scale` 再 `gemm_rs` 累加，顺序反了会把新 tile 的贡献也缩掉；`kv_tile > 0` 的 guard 不能少（首 tile 无历史）。

5. **V fragment 布局要 nonswizzle 视图预计算**：`partition_fragment_B` 若直接在 swizzled 布局上做，会把 swizzle 混进寄存器布局；正确做法是在 `get_nonswizzle_portion` 视图上取布局（寄存器布局与 swizzle 无关），实际装载仍读 swizzled 的 `sVt`。

6. **stages 对称调参错过最优点**：$(3,3)$ 在 D=384 比 $(2,2)$ 低 27%。S_K/S_V 必须独立扫描，切换点在 spill 首次出现的 D（§29.4 的寄存器表）。

29.6 面试要点速查

- **Q：大 head_dim 为什么选 non-WS 而不是 WS？**
  A：Split-D 的 O 累加器（32×C f32/线程）把寄存器占用到 255 上限，`setmaxnreg` 腾不出寄存器，WS 的收益侧（寄存器再分配）消失、成本侧（producer 不产 FLOPs，MMA warp 数降低一半）全留。D=128 场景两者性能相当（第 二十八 章），大 D 只剩 non-WS。

- **Q：S_K=3, S_V=2 为什么不对称？**
  A：QK 是主计算路径（每 tile $D/32$ 个 chunk），深流水加宽 TMA 掩蔽窗口；V 每 tile 只有 $D/64$ 个 chunk 且消费点在 softmax 之后，S_V=2 够用，第 3 级只有 barrier 维护开销。且 $D \geq 384$ 后 spill 主导，最优回落 $(2,2)$——stages 策略的切换点由寄存器占用决定。

- **Q：跨 tile 预取为什么不死锁？**
  A：QK 与 V 两套 barrier 集不相交。下一 tile 的 QK 预取只等 `qk_empty`（由本 tile Phase 1 的 arrive 释放），与 softmax 要等的 `v_full`、PV 要 arrive 的 `v_empty` 零交集，不存在环。

- **Q：non-WS 里 TMA 发射为什么不会拖慢 MMA？**
  A：发射封装成三步 lambda（fence + expect_tx + copy），只在 chunk 边界执行，每次十几条指令；相比专职 producer 损失整 128 线程的 MMA 吞吐，这点指令槽开销可以忽略——实测全员 MMA 的收益是 45% 段的主体。

- **Q：STSM + TMA store 比逐元素直写好在哪？**
  A：写出通道对称于装载：`stmatrix` 一条指令写 128 bit/线程进 swizzled smem，TMA store 批量搬运且 bulk group 把 arrive/wait 摊到每批一次；逐元素直写每线程 32C 次标量 store，且完全不 swizzle、不合并。

29.7　最小测试与动手实验

本章测试在 `book/tests/ch26c_ffpa_split_d_non_ws.cu`：CPU fp64 参考（O 与 LSE 双输出），容差 `TOL_F16ACC`（5×10^{-2}）与 LSE 绝对容差 0.1。八个 case 覆盖：基础形状（D=128/320）、多 batch/head、N_{kv} mod 128 $\neq 0$ 的边界 mask（N_{kv}=192，正是「常见坑」第 2 条的检测用例）、stages 解耦组合 $(2,3)/(3,2)$（G/H 两 case）与 S=3：

```
$ cd book/tests && ./build_tests.sh --ch ch26c
# PASS ffpa split-D non-WS A D=128 B=1 H=2 Nq=256 Nkv=256 s=2,2: 7.032e-05
# PASS ffpa split-D non-WS B D=320 B=1 H=2 Nq=256 Nkv=256 s=2,2: 7.755e-05
# PASS ffpa split-D non-WS B LSE D=320 Nq=256 Nkv=256: 7.068e-07
# PASS ffpa split-D non-WS C D=192 B=2 H=2 Nq=128 Nkv=256 s=2,2: 8.105e-05
# PASS ffpa split-D non-WS D D=320 B=1 H=2 Nq=128 Nkv=192 s=2,2: 8.254e-05
# PASS ffpa split-D non-WS D LSE D=320 Nq=128 Nkv=192: 5.626e-07
# PASS ffpa split-D non-WS E D=320 B=1 H=2 Nq=256 Nkv=256 s=3,3: 7.578e-05
# PASS ffpa split-D non-WS F D=128 B=1 H=4 Nq=512 Nkv=512 s=2,2: 4.563e-05
# PASS ffpa split-D non-WS G D=320 B=1 H=2 Nq=256 Nkv=256 s=2,3: 6.267e-05
# PASS ffpa split-D non-WS G LSE D=320 Nq=256 Nkv=256: 7.994e-07
# PASS ffpa split-D non-WS H D=320 B=1 H=2 Nq=128 Nkv=192 s=3,2: 8.602e-05
# PASS ffpa split-D non-WS H LSE D=320 Nq=128 Nkv=192: 5.729e-07
# ALL OK
```

集成入口（CLI flag 仍在 bench/bench_leetcuda.cu）：`--sdnw-cute` 快速正确性（bench/ffpa_attn.cu 的 `run_sdnw_cute_tests()`，L408；五用例，含 N_{kv}=192 边界）与 `--bench --bhnd 1,32,8192,320` 的 stages 四组合实验（`Sk`/`Sv` 直接体现在输出 label）：

```
$ ./leetcuda_bench_sm120a.bin --bench --bhnd 1,32,8192,320
| FA Split-D CuTe TMA MMA WS (D=320, Sk=1, Sv=1)  | 1.526e-05 |  86.7/69.7 (1.24x) |
| FA Split-D CuTe TMA MMA WS (D=320, Sk=2, Sv=2)  | 1.526e-05 | 140.2/69.7 (2.01x) |
| FA Split-D CuTe TMA non-WS (D=320, Sk=2, Sv=2)  | 3.052e-05 | 188.3/69.7 (2.70x) |
| FA Split-D CuTe TMA non-WS (D=320, Sk=2, Sv=3)  | 3.052e-05 | 189.3/69.7 (2.71x) |
| FA Split-D CuTe TMA non-WS (D=320, Sk=3, Sv=2)  | 3.052e-05 | 204.1/69.7 (2.93x) |
| FA Split-D CuTe TMA non-WS (D=320, Sk=3, Sv=3)  | 3.052e-05 | 188.5/69.7 (2.70x) |
```

（non-WS 的 max_err 是 3.052e-05 而非 WS 的 1.526e-05：exp2 域 softmax 的 fp16 重舍入路径略不同，两者都在 f32 累加精度内。）bench 数据与 `-Xptxas -v` 的寄存器/spill 探针全部归档 `book/.tmp/ch26c/`。动手实验建议：(1) 把 dispatch 里的 $(3,2)$ 换成 $(4,2)$ 重编，观察 S_K=4 在 D=320 是否继续提升（提示：smem 到 112 KB，spill 可能恶化）；(2) 删掉跨 tile 预取块（L1083–1094）重编复测，量化 §29.3.5 收益的绝对值；(3) 把 `FA4_RESCALE_THRESHOLD` 置 0 复测，看 FA-4 跳过率归零的代价；(4) 用 `-Xptxas -v` 亲手复现 §29.4 的寄存器表，验证 D=320 各组合的 spill 差异。

延伸阅读

- ffpa-attn `csrc/cuffpa/cute/sm_120/split_d.cuh`（本章移植源：生产版含 bias/dropout/GQA/NHD 的完整特性矩阵与 fp8/fp4 量化家族，本章是最小教学集）；
- FlashAttention-3 论文 §3.1.4（conditional rescaling，与第 二十八 章同源）；
- flash-attention `kernel_traits.h` 的 `SmemLayoutVtransposedNoSwizzle`（V fragment 布局预计算的出处，「常见坑」第 5 条）；
- 第 二十 章 19.4.6 节（O 累加器的 $32\times C$ 寄存器占用——本章 stages 调优的理论基础）。

第五部分

FP8/FP4 HGEMM 篇：量化矩阵乘的数学与 Kernel 工程

第三十章　FP8 GEMM（一）：e4m3 格式、per-row/per-block 量化与前处理 Kernel

30.1　本章导读

第 三十四–四十 章（第六部分）回答了「量化 attention 怎么做」；本部分把同一套 e4m3 数学应用到**矩阵乘**上，拆成两章：本章写**量化前处理 kernel**——把 BF16 的 A, B 变成 e4m3 的 $A_8, B_8^\top$ 外加两组 scale；第 三十一 章写**主 GEMM kernel 与在线反量化 epilogue**，并把全链路封装成一个 C++ API 与 cuBLAS BF16 GEMM 对精度和性能。两章合起来构成的完整调用形态是：

$$\underbrace{A, B}_{\text{BF16 输入}} \xrightarrow[\text{本章：3 个量化 kernel}]{\text{amax 统计 + 量化 + 转置}} \underbrace{A_8,\ B_8^\top,\ \delta_a,\ \delta_b}_{\text{FP8 workspace}} \xrightarrow[\text{第三十一章：CuTe 主 kernel}]{\text{TMA + mma.sync + epilogue}} \underbrace{C}_{\text{BF16 输出}} \tag{30.1}$$

与 attention 量化相比，GEMM 量化有一处本质的简化：**scale 折叠可以完全离开主循环**。attention 的量化输出要参与 softmax，scale 必须在 kernel 内部按行修正（第 三十六 章的 $s_{dequant} = q_s k_s$ 折叠也要挂在每个 epilogue 上）；而 GEMM 是双线性运算，两组 scale 可以**推迟到 epilogue 一次乘回**，主循环里跑的是纯整数域（e4m3 编码值）的 MMA。这个差异是 34.4 节的核心恒等式，也是第 三十一 章 epilogue 设计的全部依据。

前处理链本身是三个「读一遍、写一遍」的流式 kernel（带宽型，不是算力型），工程重点全在**粒度**与**布局**两件事上：

1. **粒度**：A 支持 per-row（每行一个 scale，精度最优）与 per-block（每 128 行一个，ffpa Q/K 同构）；B 支持 per-col 与 per-block。四种组合全部实现（34.4 节），精度开销各不同；
2. **布局**：A_8 保形行主序；B 必须转置量化成 $B_8^\top[N, K]$ 行主序——主 kernel 的 TMA 与 MMA 都按 TN 布局消费，转置发生在量化 kernel 内部（smem staging，34.5.3 节），对调用方完全透明。

前置章节：第 三十四 章（e4m3 编码与量化算子，必须）、第 三 章（向量化 IO 与 warp 归约）、第 四 章（block 归约）。**源码区间**：`fp8_gemm.cuh` L45–284（Phase 9.1/9.2），完整行号以附录 D 的源码索引为准。**验证方式**：`notes_v2_sm120a.bin --fp8-gemm`（CPU fp64 参考全组合 + 尾部 shape），bench 数据在 34.6 节。

30.2　问题与动机

30.2.1　为什么 GEMM 量化比 attention 量化「容易」

attention 的三量化点里，P（softmax 输出）必须在线量化，Q, K, V 的 scale 又会经 softmax 混入归一化分母——所以第 三十六 章要在 kernel 里维护行最大值修正与 $s_{dequant}$ 折叠。GEMM 没有任何非线性：$C = AB^\top$ 是双线性的，量化的 scale 是**纯标量因子**，可以直接提到括号外。于是：

- 主 kernel 的 MMA 循环里**不需要处理任何 scale**，跑的就是 e4m3 编码值的整数语义乘累加（f32 累加器）；
- 反量化只在 epilogue 出现一次：$C = (A_8 B_8^\top) \odot (\delta_a \delta_b)$，$\odot$ 在 per-block 粒度下退化为块级（128 行/列）标量乘（第 三十一 章）；
- 前处理全部离线于主 kernel：量化 kernel 写完 workspace，主 kernel 才开始执行。

教学上这是一条「从第 三十四 章的量化数学到工程 kernel」的最短路径：数学恒等式一章讲透，kernel 工程两章讲透，不引入 online softmax 这类复杂度。

30.2.2 约束与 API 约定

整套实现的形状约束由 **TMA 对齐**与**量化向量化**共同决定，集中在 API 层检查（第 三十一 章 `fp8_gemm_bf16`）：

- $K \bmod 16 = 0$：$A_8/B_8^\top$ 的行是 e4m3 字节流，TMA 装载要求 16 B 对齐；同时也是量化 kernel Vec8（8×bf16=16 B）读取的超集；
- $N \bmod 8 = 0$：输出 C 的行是 bf16 字节流，TMA store 的 16 B 对齐要求 $N \times 2$ B 被 16 整除；
- M 任意：装载方向的越界行由 TMA 自动零填充（对累加无贡献），写出方向越界行由 TMA 自动裁剪，epilogue 的 scale 查表按 $m < M$ guard；
- K 非 128 倍数的尾 tile 同样零填充，主循环**没有尾部分支**——这比第 三十五 章 ffpa 的 VT 量化省掉了 pad 列清零，因为那里用的是 cp.async 装载、这里全程 TMA（OOB 语义内建）。

30.3 e4m3 回顾：GEMM 视角

第 三十四 章已逐位讲过 e4m3 的编码；这里只保留 GEMM 工程要用的三个事实，并用一张位布局图统一符号。e4m3 是 1 符号 + 4 指数 + 3 尾数的 8 位浮点：指数偏置 7、无无穷大（全 1 指数域 + 非零尾数是 NaN），最大有限值 $R = 448$：

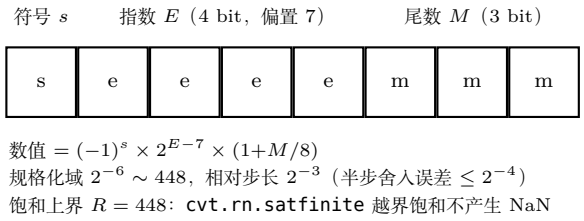


图 30.1: e4m3 位布局与 GEMM 工程三要素：相对步长固定 2^{-3}（尾数 3 位），决定量化噪声的下限；满量程 448 决定 scale 公式 $\delta = \text{amax}/448$；`SATFINITE` 语义决定量化 kernel 可以直接写 $x \cdot \delta^{-1}$ 而不用先 clamp——越界饱和由硬件完成。

三个事实：

1. **相对步长** 2^{-3}：规格化域内相邻可表示值的间距是值本身的 1/8，round-to-nearest 的单元素相对误差上界 $2^{-4} = 6.25\%$。这决定了 34.4 节的统计误差模型；
2. **满量程** $R = 448$：对称量化的 scale 定义为 $\delta = \text{amax}/448$，让块内最大绝对值恰好映到编码域边缘，步长利用率最大；
3. **饱和语义**：`__nv_cvt_float2_to_fp8x2` 的 `__NV_SATFINITE` 模式把越界输入饱和到 ±448 而不是 NaN——量化 kernel 不需要软件 clamp（34.5.4 节）。

30.4 量化数学与误差分析

30.4.1 量化算子与 scale 折叠恒等式

对称量化算子（与第 三十四 章式 (27.x) 同一形式，重写于此保持自包含）：

$$\hat{x} = Q(x;\delta) = \text{sat}_{[-448,448]}\big(\text{rn}(x/\delta)\big), \qquad \delta = \frac{\text{amax}(\mathcal{B})}{448}, \tag{30.2}$$

其中 $\mathcal{B}$ 是 scale 的统计块（一行 / 一列 / 一个 128 行块，由粒度决定），rn 是 round-to-nearest-even。反量化 $x \approx \hat{x} \cdot \delta$。

GEMM 的核心恒等式由此立得。设 $A \approx \delta_a \hat{A}$，$B \approx \delta_b \hat{B}$（$\delta_a, \delta_b$ 为逐行/逐列/逐块的 scale 张量，broadcasting 到元素级），则

$$C = AB^\top = (\delta_a \odot \hat{A})(\delta_b \odot \hat{B})^\top \tag{30.3}$$

精确展开单元素：当 δ_a 沿行（m 方向）常数、δ_b 沿列（n 方向）常数时——per-row × per-col 正是如此——

$$C_{m,n} = \sum_k \delta_{a,m}\,\hat{A}_{m,k}\,\hat{B}_{n,k}\,\delta_{b,n} = \delta_{a,m}\,\delta_{b,n} \sum_k \hat{A}_{m,k}\hat{B}_{n,k} = \delta_{a,m}\,\delta_{b,n}\,(\hat{A}\hat{B}^\top)_{m,n}. \tag{30.4}$$

即 **scale 完全出括号**：整数域（e4m3 编码值）MMA 之后，epilogue 按 (m,n) 查 $\delta_{a,m}\delta_{b,n}$ 一次乘回。per-block 粒度时块宽固定 128（与 tile 尺寸无关），tile 内 δ_a,δ_b 是**分段常数**：k_{BM}=128 的 A 侧一个 tile 恰一块，k_{BN}=256 的 B 侧一个 tile 横跨两块，故查表索引取全局的 $m/128$、$n/128$，式 30.4 退化为**块级标量乘**。

对比第 三十六 章 persist-D 的 $s_{dequant} = q_s k_s$：那里 Q/K 的 scale 同样出括号，但 V 的 scale 乘在 P 上、而 P 又来自 softmax，折叠点必须跟着在线重缩放走；GEMM 版没有这层耦合，恒等式简单得多。这也是本部分先于 attention 部分教学的价值：先看清「纯双线性下量化能折叠到什么程度」。

30.4.2　四种粒度组合

实现里 `Fp8GemmScaleMode` 枚举出四种组合，全部走同一对恒等式，差异只在 scale 张量的形状与 epilogue 的查表方式：

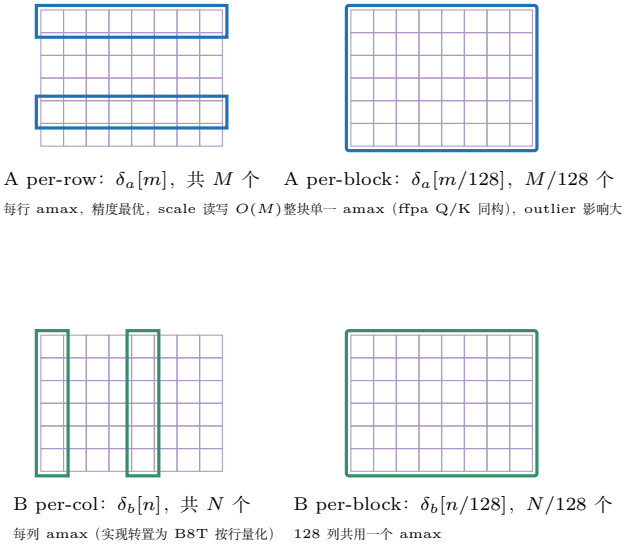


图 30.2: 四种 scale 粒度组合（上排 A，下排 B）。蓝框 =A 侧 scale 统计范围，绿框 =B 侧。粒度越粗，scale 存储/查表越省（per×per-block 每 tile 只查两个标量），但 outlier 的影响越大——一个大值会放大整块的量化步长，其余小值全部落入低精度区间。四种组合在 API 里是一个枚举，主 kernel 用 `if constexpr` 编译期剔除，运行时零分支开销。

为什么 GEMM 里 per-row×per-col 是「精度最优」：scale 的统计块越小，块内分布越均匀、步长越贴合该组数据的动态范围。极端的反例是全矩阵单 scale（per-tensor）：一个 outlier 可以让普通元素的相对量化误差放大若干个数量级。Per-block 是 attention 领域（SageAttention/ffpa）的主流选择，因为它与 128 的 tile 尺寸天然对齐、kernel 内 scale 折成单标量；对 GEMM 的权重矩阵（分布沿行/列明显不均），per-row/per-col 通常明显更准。34.6 节会给四种组合的同机实测。

30.4.3　误差模型：能差到多少

单元素量化噪声：设块内元素落在同一二进区间，rn 的误差均匀分布于 $[-\frac{u}{2},\frac{u}{2}]$（$u=2^{-3}$ 为相对步长），故单元素相对误差 ε 满足 $|\varepsilon|\le 2^{-4}$，方差 $\sigma_\varepsilon^2 = u^2/12$。

scale 折叠之后，误差只来自编码舍入一个源头。把单元素量化写成乘性形式 $\hat{x} = x(1+\varepsilon)$：相对舍入误差 ε 在 $[-2^{-4},2^{-4}]$ 上近似均匀分布（相对步长 $u=2^{-3}$，见上文），故 $\mathrm{E}[\varepsilon]\approx 0$，方差 $\sigma_\varepsilon^2 = u^2/12 \approx 1.30\times10^{-3}$，即 $\sigma_\varepsilon \approx 3.6\%$。对 $C_{m,n}=\sum_k a_k b_k$，两个量化点的误差一阶展开：

$$\tilde{C}_{m,n} - C_{m,n} = \sum_k a_k b_k\big(\varepsilon_{a,k}+\varepsilon_{b,k}\big) + \sum_k a_k b_k\,\varepsilon_{a,k}\varepsilon_{b,k} \;\approx\; \sum_k a_k b_k\big(\varepsilon_{a,k}+\varepsilon_{b,k}\big), \tag{30.5}$$

第二项是 $O(\sigma_\varepsilon^2)$ 的高阶小量，比一阶项低一个 σ_ε 因子，下文只保留一阶。关键区分两种数据形态：

- **随机符号数据**（测试用的均匀随机矩阵）：误差项相互独立且随机变号，累加是随机游走。误差能量 $\mathrm{Var}[\tilde{C}-C] = K\cdot\mathrm{E}[(ab)^2]\cdot 2\sigma_\varepsilon^2$，而 C 自身能量 $\mathrm{Var}[C] = K\cdot\mathrm{E}[(ab)^2]$，相对误差 $\sqrt{2}\,\sigma_\varepsilon \approx 5\%$（实测 3.6%，34.6 节——舍入误差并非全程满幅均匀，实测略低于上界）。注意相对误差**与 K 无关**：误差与信号都以 $\sqrt{K}$ 增长，抵消掉了；
- **同号数据**（全正矩阵、经 ReLU 的激活）：误差项同号不相消，累加是线性增长，最坏相对误差停在单元素上界 2^{-4} 量级附近（不随累加长度收敛）。这就是量化 GEMM 在真实模型上仍需 per-row/per-channel 粒度与离线校准（如 GPTQ/平滑旋转，第 三十五 章同源思想）的原因。

Max Err 是极值统计，不是误差上界　单个 $C_{m,n}$ 接近零（正负相消）时相对误差可以任意大，所以 bench 表里的 `Max Err` 只能取绝对误差（vs cuBLAS BF16 参照）。但它也不是最坏元素误差：MN 个近似独立的误差样本取最大值，极值统计给出

$$\mathrm{E}\big[\max_{m,n}|\tilde{C}-C|\big] \;\approx\; \sigma_{\mathrm{err}}\sqrt{2\ln(MN)}, \tag{30.6}$$

$M{=}N{=}4096$ 时 $\sqrt{2\ln(MN)} \approx 5.8$。误差分布是有限支撑的和分布，尾部比高斯分布薄，实测极值落在 $4\sigma_{\mathrm{err}}$ 一带——健康判据是 **Max Err 在 $4\sigma_{\mathrm{err}}$ 量级**，而不是要求它趋零。绝对口径的 Max Err 自然随 $\sqrt{K}$ 增长（$\sigma_{\mathrm{err}}\propto\sqrt{K}$），换算到相对口径才是与 K、与分布都无关的常数。

实测指纹　四组数据逐一验证了上面每个环节（$\sigma_{\mathrm{err}} = \sqrt{K\,\mathrm{E}[(ab)^2]\,2\sigma_\varepsilon^2}$，均匀输入 $\mathrm{E}[(ab)^2]=1/9$，randn 组 $\mathrm{E}[(ab)^2]=\sigma_x^4$，$\sigma_x = 0.25/3$）：

数据形态（vs cuBLAS BF16）	σ_{err} 理论	Max Err 实测	比值
均匀 $[-1,1]$，2048^3	0.77	3.00	3.9σ
均匀 $[-1,1]$，4096^3	1.09	4.50	4.1σ
均匀 $[-1,1]$，8192^3	1.54	6.25	4.1σ
randn (±0.25)，4096^3	0.0227	0.090	4.0σ

三组均匀 shape 的 Max Err/σ_{err} 停在同一个 4.1σ——样本量从 4M 增到 67M，比值不漂移，证明误差是零均值随机游走而非系统性偏差。randn 组把输入取为 $\sigma_x = 0.25/3$ 的截断正态（幅值缩 $s\approx 6.9$ 倍）：$\mathrm{E}[(ab)^2]$ 缩 s^4，σ_{err} 随之缩 $s^2\approx 48$ 倍，$4.50/48 \approx 0.094$ 与实测 0.090 相符，比值仍落在 4σ。注意两组的相对误差 $\sigma_{\mathrm{err}}/\sigma_C = \sqrt{2}\,\sigma_\varepsilon$ 相同：换分布只缩放绝对口径，34.6 节实测的 rel-Frobenius 0.036 与分布无关。

与 fp8 attention 的误差为什么不可直接比　同是 fp8 对 bf16，量化 attention 的绝对误差看起来小得多，差异全在算子形态与度量口径。其一，attention 输出 $O = \mathrm{softmax}(QK^\top)V$ 是 V 行的**凸组合**，元素被夹在 $[\min V, \max V]$ 之间，误差也被加权平均缩小；GEMM 的 C 是 K 项**无归一化**累加，$\sigma_C = \sqrt{K\,\mathrm{E}[(ab)^2]}$ 随 $\sqrt{K}$ 膨胀。其二，attention 惯用 PSNR/相对误差口径（几个百分点），bench 表用绝对误差口径。同口径对比：本实现 rel-Frobenius 3.6% 与 fp8 attention 对 bf16 的相对误差同量级，两者相当。

30.4.4 amax 零块与数值安全

amax $= 0$（全零行/块）时 $\delta = 0$、x/δ 为 0/0。实现的约定：

$$\delta = 0, \qquad \delta^{-1} \hat{=}\, 0 \;\Longrightarrow\; \hat{x} = \mathrm{rn}(x\cdot\delta^{-1}) = 0, \tag{30.7}$$

即全零块量化为全零编码（合法 e4m3 值），反量化 $0\times 0 = 0$ 精确成立。这个 `amax==0 ? 0 : 448/amax` 的三元式在三个 kernel 里各出现一次，是量化代码的标准易错点（漏掉会产生 NaN 块，污染整个 GEMM）。

30.5 前处理 Kernel 工程

三个 kernel 全部是纯 CUDA（无 CuTe 依赖），共享 34.4 节的量化数学与一对打包工具：

fp8_gemm.cuh L50-82（量化常数、粒度枚举、Vec8 与 e4m3x2 打包）

```
// Phase 9.1: 量化数学
// ============================================================================
```

```cpp
// 量化算子（对称，e4m3 满量程 R=448）：
//   x̂ = clamp_{[-448,448]}( round(x / δ) ),  δ = amax(B) / 448
//   amax=0 时 δ=0, inv_δ=0 （全零块量化为全零，避免 0/0）
// 反量化折叠(GEMM 版核心恒等式):
//   C = A·B^T = (δa·Â)·(δb·B̂)^T = δa·δb·(Â·B̂^T)
//   整数域 MMA 结果 epilogue 每元素乘回 δa·δb — 除量化本身外无额外数学
//   误差源（与 persist-D 的 s_dequant = qs*ks 折叠同理）。
//   per-row x per-col: δa 依行 m 查表 sa[m], δb 依列 n 查表 sb[n]
//   per-block: 每 128 行/列一个 scale, epilogue 按 m/128、n/128 查表(与 tile 解耦)
constexpr float kE4M3Max = 448.0f;   // e4m3 最大有限值（饱和上界）
constexpr int kScaleBlock = 128;     // per-block 量化粒度（固定 128）

enum class Fp8GemmScaleMode {
  kPerRowPerCol,      // A per-row x B per-col（精度最优，scale 数 M+N）
  kPerRowPerBlock,    // A per-row x B per-block(scale 数 M + N/128)
  kPerBlockPerCol,    // A per-block x B per-col(scale 数 M/128 + N)
  kPerBlockPerBlock,  // A per-block x B per-block(attention Q/K 同构，M/128+N/128)
};

union Vec8BF16 {  // 8 x bf16 = 16B，量化 kernel 的向量化 IO 单元
  uint4 raw;
  __nv_bfloat16 elem[8];
};
static_assert(sizeof(Vec8BF16) == 16, "Vec8BF16 must be 128-bit");

// f32 -> e4m3 两条打包（与 ffpa fp8_pscale.cuh 的 cvt.rn.satfinite.e4m3x2
// .f32 同指令): satfinite 保证 x*inv_δ 舍入越界时饱和到 ±448 而非 NaN.
__device__ __forceinline__ unsigned short cvt_f2_to_e4m3x2(float lo, float hi) {
  return __nv_cvt_float2_to_fp8x2(make_float2(lo, hi), __NV_SATFINITE,
                                  __NV_E4M3);
}
```

30.5.1　A 的 per-row 量化：warp-per-row

fp8_gemm.cuh L91-128（quantize_a_perrow_kernel：warp 归约 + 两遍 gmem）

```cpp
//  （第二遍命中 L1/L2）；ffpa 的 per-block kernel 依赖编译期 kD 才能一遍读。
__global__ void quantize_a_perrow_kernel(
    const __nv_bfloat16 *__restrict__ X, cutlass::float_e4m3_t *__restrict__ Y,
    float *__restrict__ scale, int M, int K) {
  constexpr int kVec = 8;  // K % 16 == 0 保证 chunk 对齐
  const int row = blockIdx.x * 8 + threadIdx.x / 32;  // 8 warps/block
  if (row >= M) return;
  const int lane = threadIdx.x % 32;
  const __nv_bfloat16 *xr = X + (size_t)row * K;

  float amax = 0.0f;  // pass 1: 行 amax
  for (int c = lane * kVec; c < K; c += 32 * kVec) {
    Vec8BF16 v = *reinterpret_cast<const Vec8BF16 *>(xr + c);
#pragma unroll
    for (int e = 0; e < kVec; ++e)
      amax = fmaxf(amax, fabsf(__bfloat162float(v.elem[e])));
  }
#pragma unroll
  for (int off = 16; off > 0; off >>= 1)  // warp 内蝶形归约
    amax = fmaxf(amax, __shfl_xor_sync(0xffffffff, amax, off));

  const float s = amax / kE4M3Max;
  const float inv_s = (amax == 0.0f) ? 0.0f : kE4M3Max / amax;
  if (lane == 0) scale[row] = s;
```

```
25
26    cutlass::float_e4m3_t *yr = Y + (size_t)row * K;  // pass 2: 量化写回
27    for (int c = lane * kVec; c < K; c += 32 * kVec) {
28      Vec8BF16 v = *reinterpret_cast<const Vec8BF16 *>(xr + c);
29      uint2 out;
30      unsigned short *o2 = reinterpret_cast<unsigned short *>(&out);
31  #pragma unroll
32      for (int e = 0; e < kVec; e += 2)
33        o2[e / 2] = cvt_f2_to_e4m3x2(__bfloat162float(v.elem[e]) * inv_s,
34                                     __bfloat162float(v.elem[e + 1]) * inv_s);
35      *reinterpret_cast<uint2 *>(yr + c) = out;  // 8 x e4m3 = 8B
36    }
37  }
```

结构是第 三 章 warp 归约的直接应用，值得对照的是**两遍 gmem 读**：pass 1 算行 amax，pass 2 重新读一遍行做量化写回。为什么不像第 三十五 章 ffpa 的 per-block kernel 那样寄存器缓存一遍读? 因为那里 K（head_dim）是 **编译期常量**（kD），整行的 Vec8 可以展开进寄存器数组；本实现的 K 是运行时 GEMM 维度，寄存器数组无法定长。两遍读的代价由 L1/L2 承担——同一 warp 刚读过的行在 pass 2 里几乎全命中，实测该 kernel 达到内存带宽。

写侧同样关键：量化输出是 8 个 e4m3 = 8 B，用 `uint2` 一次写——注意这里**不能**凑成 16 B（e4m3 是 1 B 元素，8 元素只有 8 B；要 16 B 需要凑足 16 个元素跨 lane 拼接，教学版不值得）。读 16 B 写 8 B，字节收缩 2×，写带宽不是瓶颈。

30.5.2 A 的 per-block 量化：block 二级归约

per-block 版把统计块从一行放大到 128 行，归约升到 block 级——第 四 章的 warp→smem→warp0 两级范式：

fp8_gemm.cuh L131-170（quantize_a_perblock_kernel：128 行块 amax 两级归约）

```
1   // 128 行全体元素的最大绝对值，warp 归约 -> smem -> warp0 二次归约。
2   __global__ void quantize_a_perblock_kernel(
3       const __nv_bfloat16 *__restrict__ X, cutlass::float_e4m3_t *__restrict__ Y,
4       float *__restrict__ scale, int M, int K) {
5     constexpr int kVec = 8;
6     constexpr int kThreads = 256;
7     constexpr int kWarps = kThreads / 32;
8     const int row0 = blockIdx.x * kScaleBlock;
9     const int tid = threadIdx.x;
10    const int warp = tid / 32, lane = tid % 32;
11    const int dv = K / kVec;  // 每行 Vec8 chunk 数
12
13    float amax = 0.0f;  // pass 1: 128 行块 amax
14    for (int i = tid; i < kScaleBlock * dv; i += kThreads) {
15      const int r = i / dv, c = (i % dv) * kVec;
16      const long row = row0 + r;
17      if (row >= M) continue;
18      Vec8BF16 v = *reinterpret_cast<const Vec8BF16 *>(X + row * K + c);
19  #pragma unroll
20      for (int e = 0; e < kVec; ++e)
21        amax = fmaxf(amax, fabsf(__bfloat162float(v.elem[e])));
22    }
23  #pragma unroll
24    for (int off = 16; off > 0; off >>= 1)
25      amax = fmaxf(amax, __shfl_xor_sync(0xffffffff, amax, off));
26    __shared__ float warp_max[kWarps];
27    if (lane == 0) warp_max[warp] = amax;
28    __syncthreads();
29    if (warp == 0) {
30      amax = (lane < kWarps) ? warp_max[lane] : 0.0f;
31  #pragma unroll
32      for (int off = kWarps / 2; off > 0; off >>= 1)
```

```
      amax = fmaxf(amax, __shfl_xor_sync(0xffffffff, amax, off));
    if (lane == 0) warp_max[0] = amax;
  }
  __syncthreads();
  amax = warp_max[0];

  const float inv_s = (amax == 0.0f) ? 0.0f : kE4M3Max / amax;
  if (tid == 0) scale[blockIdx.x] = amax / kE4M3Max;
```

pass 2（L172 起，与 per-row 版逐字节同构，略）用块级 inv_s 重扫 128 行。两个工程点：其一，块内行数 =128 恰是 `kScaleBlock`：A 侧主 kernel 的 k_{BM}=128 正好落在一个块内，B 侧 k_{BN}=256 却横跨两块——所以 epilogue 的 scale 查表按**全局** $m/128$、$n/128$ 逐元素取（**统计块与 tile 几何解耦**），而不是整 tile 一个标量；其二，`row >= M` 的行跳过（尾部 block 不足 128 行），amax 不受越界无效数据影响——load 侧由 TMA 零填充处理，这里则必须在量化 kernel 自己 guard（它读的是原始 BF16 gmem）。

30.5.3　B 的转置量化：双朝向 smem staging

三个 kernel 里实现最复杂的是 B：输出要的是 $B_8^\top[N,K]$ 行主序（K 内维连续），而输入 $B[K,N]$ 的列方向 stride 是 N——**直接按输出朝向读写 gmem 都不 coalesced**。解法是 smem staging 双朝向 tile：



图 30.3: B 转置量化的双朝向 staging。读路径(蓝)永远沿 B 的行(N 连续)；写路径(绿)永远沿 $B_8^\top$ 的行(K 连续)；两次「转置」都发生在 smem 里(散写即转置)，gmem 两侧全是 coalesced。对照第 三十五 章 ffpa 的 `quantize_fp8_vt_kernel`：那边输出要 pad 16 列对齐 cp.async 16B，这里 TMA 装载 OOB 自动零填、K 尾部由 store guard 裁剪，staging 免 pad。

fp8_gemm.cuh L200-240（quantize_bt_kernel：双朝向 tile 与 coalesced 装载）

```
__global__ void quantize_bt_kernel(const __nv_bfloat16 *__restrict__ B,
                                   cutlass::float_e4m3_t *__restrict__ BT,
                                   float *__restrict__ scale, int K, int N) {
  constexpr int kBNt = 128;              // n 方向 tile(= B8T 行数)
  constexpr int kBKt = 64;               // k 方向 chunk(= B 行数)
  constexpr int kVec = 8;
  constexpr int kChunksPerRow = kBNt / kVec;  // 16
  constexpr int kChunks = kBKt * kChunksPerRow;
  constexpr int kWarps = 256 / 32;
  const int n0 = blockIdx.x * kBNt;
  const int tid = threadIdx.x;
  const int warp = tid / 32, lane = tid % 32;

  __shared__ __align__(16) __nv_bfloat16 tile[kBKt][kBNt];  // 源朝向 16KB
  __shared__ __nv_fp8_e4m3 tile_t[kBNt][kBKt];  // 输出朝向 8KB (平凡类型,
  // 避免 smem 变量动态初始化告警; 与 cutlass::float_e4m3_t 逐位兼容)
  __shared__ float samax[kBNt];

  auto load_tile = [&](int k0) {  // B 行 coalesced 读入 smem (越界补零)
    for (int i = tid; i < kChunks; i += 256) {
      const int r = i / kChunksPerRow, c8 = (i % kChunksPerRow) * kVec;
      const int k = k0 + r, n = n0 + c8;
```

```
      Vec8BF16 v;
      v.raw = make_uint4(0, 0, 0, 0);
      if (k < K && n < N)
        v = *reinterpret_cast<const Vec8BF16 *>(B + (size_t)k * N + n);
      *reinterpret_cast<uint4 *>(&tile[r][c8]) = v.raw;
    }
  };

  for (int n = tid; n < kBNt; n += 256) samax[n] = 0.0f;  // tid<128 生效
  __syncthreads();
  for (int k0 = 0; k0 < K; k0 += kBKt) {  // pass 1: 逐列 amax
    load_tile(k0);
    __syncthreads();
    for (int n = tid; n < kBNt; n += 256) {
      float a = samax[n];
      for (int r = 0; r < kBKt; ++r)
        a = fmaxf(a, fabsf(__bfloat162float(tile[r][n])));
      samax[n] = a;
    }
```

装载 lambda `load_tile` 把 B 的 $[64,128]$ 块按 16B 向量读进源朝向 tile，越界（K 尾或 N 尾）显式补零——注意这与「TMA 零填充」是两回事：这个 kernel 不走 TMA，越界补零必须自己做，但只要 amax 统计正确、写侧 guard，零值不污染任何东西。pass 1 对每个 n（tid 步进 256，实际 128 个 n 生效）扫 64 行累积 `samax[n]`，跨全部 K chunk 完成整列统计。

scale 落盘段有一个值得借鉴的微技巧——**`samax` 原地改存** δ^{-1}：pass 1 结束时 `samax[n]` 存的是 amax，写出 $\delta_b[n]$ 之后立刻原地覆写成 448/amax，pass 2 查表零成本，省一个 `inv_s[128]` 数组与一次 `__syncthreads`。per-col 关闭时（kPerCol=false），block 级再把 128 个列 amax 归约成单值写 $\delta_b[n_0/128]$，同样原地放进 `samax[0]`。

fp8_gemm.cuh L266-288（pass 2：量化散写 tile_t + 32B coalesced 写出）

```
    for (int i = tid; i < kChunks; i += 256) {  // 散写 tile_t[n][r]
      const int r = i / kChunksPerRow, c8 = (i % kChunksPerRow) * kVec;
#pragma unroll
      for (int e = 0; e < kVec; ++e) {
        const int n = c8 + e;
        const float inv_s = kPerCol ? samax[n] : samax[0];
        tile_t[n][r] = __nv_fp8_e4m3(__bfloat162float(tile[r][n]) * inv_s);
      }
    }
    __syncthreads();
    for (int n = warp; n < kBNt; n += kWarps) {  // 32B coalesced store
      if (n0 + n >= N) continue;
      cutlass::float_e4m3_t *row = BT + (size_t)(n0 + n) * K + k0;
#pragma unroll
      for (int half = 0; half < kBKt / 32; ++half) {
        const int kk = half * 32 + lane;
        if (k0 + kk < K)
          row[kk] = *reinterpret_cast<cutlass::float_e4m3_t *>(&tile_t[n][kk]);
      }
    }
    __syncthreads();  // tile/tile_t 下一 chunk 可覆写
  }
}
```

pass 2 的散写循环：每个线程按 chunk 索引读 `tile[r][c8..c8+8]`，量化后写 `tile_t[n][r]`——下标对调即转置。写出段换成 warp 视角：一个 warp 负责一行 n，32 个 lane 各写 32 个连续 k 字节（`kBk t`=64 分两个 half），恰好 32B 事务；`k0+kk<K` 的 guard 处理 K 尾（该行末尾的越界字节不写出）。两个 `__syncthreads` 分别保护 tile→tile_t 的读写依赖与 tile_t→gmem 的下 chunk 覆写。

30.5.4　e4m3x2 打包与饱和语义

量化内循环的 $\hat{x} = Q(x)$ 由 `cvt_f2_to_e4m3x2` 一次转换两个 float——对应 SASS/PTX 的 `cvt.rn.satfinite.e4m3x2.f32`（第 三十四 章同指令）。`rn` 是 round-to-nearest-even，`satfinite` 把 $|x \cdot \delta^{-1}| > 448$ 的输入饱和到 ±448。有了这条语义，量化 kernel 不需要软件 `min(max(x,-448),448)`——理论上乘 δ^{-1} 后不可能越界（δ 就是按 amax 定的），但浮点除法/乘法的舍入误差可能让边界值超出 1 ulp，`satfinite` 恰好处理这 1 ulp 的越界，硬件一条指令替代软件三分支。

30.6　精度实测

对 M=N=K=512 均匀随机 $[-1, 1]$ 输入（srand(42)），以 CPU fp64 双精度 GEMM 为参照，四种组合 × WS/非 WS × 尾部 shape 的相对 Frobenius 误差（`--fp8-gemm` 验证链的统计口径）：

配置	rel-Frobenius	说明
per-row × per-col（非 WS）	0.036	与粒度无关的量化噪声底
per-row × per-block（非 WS）	0.036	同上
per-block × per-col（非 WS）	0.036	同上
per-block × per-block（非 WS）	0.036	同上
per-row × per-col（WS）	0.036	WS 数值等价
per-block × per-block（WS）	0.036	WS 数值等价
尾部 M=300, N=264, K=144	0.036	尾路径零额外误差

两个观察：其一，**随机数据下四种粒度重合**——均匀分布没有 outlier，粗粒度没有精度损失；真实权重/激活的分布差异才会拉开组合间的差距（教学实现选随机数据并未回避这一局限，这也是 34.4.3 节误差模型的印证：噪声由步长 2^{-3} 决定，不由粒度决定，粒度决定的是**最坏块**的表现）。其二，**WS 与非 WS 逐 bit 数值等价**——两者的 MMA 序列相同（第 三十一 章），差异纯粹在执行协议。0.036 落在 34.4.3 节上界 $\sqrt{2}\,\sigma_\varepsilon \approx 5\%$ 之内，与模型一致。

30.7　小结

本章把 GEMM 量化的「数学侧」归结为三件事：式 30.4 的 scale 折叠恒等式（per-row/per-col 逐行列查表，per-block 每 128 行/列一个 scale）、四种粒度的工程权衡（34.4.2 节）、以及三个前处理 kernel 的访存设计（warp-per-row 两遍读、block 二级归约、B 的双朝向 smem 转置 staging）。workspace 的形态已为第 三十一 章准备好：$A_8[M,K]$、$B_8^\top[N,K]$ 两个 e4m3 行主序张量 + 两个 float scale 向量，TMA descriptor 可以直接按 $(BM,BK)/(BN,BK)$ tile。下一章展开主 kernel 的实现。

延伸阅读

- 第 三十四 章：e4m3/e5m2 编码细节、量化算子的完整数学。
- 第 三十五 章：ffpa 的量化前处理链（per-block QK、$V^\top$ 转置、smoothing）——与本章对照，attention 场景的量化为什么更重「分布整形」。
- 第 三十六 章：$s_{dequant} = q_s k_s$ 折叠——式 30.4 在 attention 里的带 softmax 修正版。
- NVIDIA TensorRT Model Quantization 的 per-channel weight quantization 文档：真实权重分布下 per-row/per-channel 粒度的校准实践。

第三十一章　FP8 GEMM（二）：CuTe 主 Kernel、在线反量化与 cuBLAS 对比

31.1　本章导读

第 三十 章把 A, B 量化成了 e4m3 的 $A_8, B_8^\top$ 与两组 scale；本章写主 GEMM kernel，把整条链封进一个 C++ API，并与 cuBLAS BF16 GEMM 对比精度与性能。主 kernel 的构件全部来自前文：TMA descriptor 四件套（第 二十四 章）、SW128 smem swizzle（第 二十四 章）、mbarrier 多级流水（第 二十八 章）、`SM89_16x8x32` fp8 MMA atom（第 三十四 章引入、ffpa 同款）、全员 MMA 的 non-WS 执行协议（第 二十九 章先例）、epilogue 的 STSM + TMA store（第 二十八 章）。新东西只有一件：**在线反量化**——把第 三十 章式 30.4 的 scale 折叠恒等式落实为 epilogue 代码，四种粒度用 `if constexpr` 编译期剔除。

本章也是全书收尾的整合章节之一：一个完整的量化 GEMM 推理算子应该长什么样——输入输出 BF16、内部 FP8、一个 API 调用、一条 stream、workspace 复用、约束自检——都在 35.7 节的 `fp8_gemm_bf16` 里。35.8 节给出 PRO 5000 上的完整实测：FP8 kernel 相对 cuBLAS BF16 在 $4096^3/8192^3$ 上拿到 $2.7\times \sim 3.0\times$（对手写 bf16 HGEMM 上限的口径则是 $1.81\times$，两个基线的差距本身就是一个「基线 sanity check」案例，35.8.1 节专门拆解）；量化前处理的开销（E2E 视角）与形状缩放规律一并给出，并指出部署时真正该走的路——**权重 B 离线量化**（35.8.3 节）：权重只量化一次、随 checkpoint 落盘，运行时只承担激活量化的开销，E2E 从「量化占大头」回到主 kernel 决定。35.8.4 节把 tile 几何 (BM, BN) 与流水深度也当成可调参数实测一遍：本章默认的宽 tile（128×256 配 2 级流水）就是大形状上的最快档，$4096^3/8192^3$ 上 443.7/487.8 T，比 128^3/3 级快 7.7%/5.9%，但带形状前提——2048^3 上反而慢 17%。35.8.5 节用 NCU 读出它为什么快：`pipe_tensor` 80.9%、寄存器 168/线程、bank conflict 0、smem 读波前反而更少。FP4（NVFP4）GEMM 单独成章，本章不展开。

前置章节：第 三十 章（量化数学与 workspace 形态，必须）、第 二十四 章（TMA/swizzle）、第 二十九 章（non-WS 协议）、第 二十八 章（epilogue 与 STSM）。**源码区间**：`fp8_gemm.cuh` L303–955（Phase 9.3–9.8），`bench/fp8_gemm.cu` 的 `bench_fp8_gemm/bench_fp8_gemm_tile_sweep` 接线。**验证方式**：`leetcuda_bench_sm120a.bin --fp8-gemm`（精度全组合）、`--bench --mnk 8192,8192,8192`（标准表）、`--fp8-gemm-sweep`（tile/流水深度扫描，35.8.4 节）。

31.2　总体架构

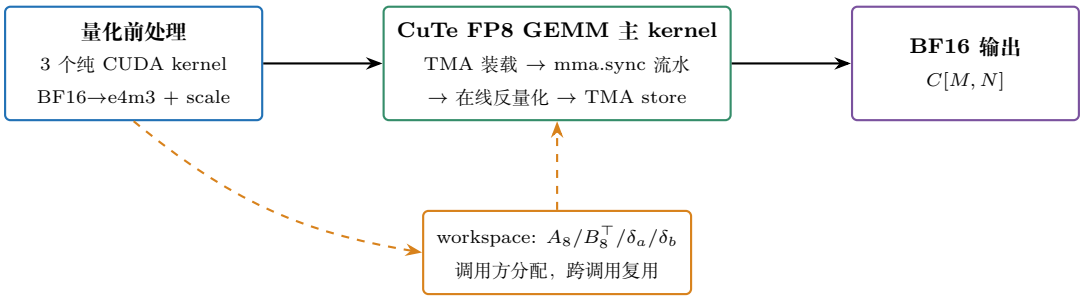


图 31.1：`fp8_gemm_bf16` 全链路：一次调用在给定 stream 上先发射量化 kernel（第 三十 章），再发射主 kernel，同步语义与 cuBLAS 一致（入队即返回，调用方在 stream 上同步）。所有中间张量都在 workspace 里，主 kernel 通过模板参数拿到量化输出的 e4m3 指针与 scale 指针——两级之间没有额外的拷贝或布局变换。

主 kernel 的输入输出约定（TN 布局，与第 二十五 章 CuTe HGEMM 一致）：$C[M,N] = A_8[M,K] \cdot B_8^{\top}[N,K]$，三者都行主序、e4m3 输入/bf16 输出。「$B^{\top}$ 行主序」就是「B 列主序」——量化阶段已经完成转置（34.5.3 节），主 kernel 里 A/B 两条装载路径完全对称。

31.3　Traits：tile 几何、SW128 与 M8N1 TiledMma

fp8_gemm.cuh L366-400（Fp8GemmTraits：tile/smem/MMA 全参数）

```
template <int kBM_ = 128, int kBN_ = 256, int kBK_ = 128, int kStages_ = 2>
struct Fp8GemmTraits {
  static_assert(kBK_ == 128, "SW128 atom requires 128B rows (1B elems)");
  static_assert(kBM_ % 16 == 0, "BM multiples of MMA_M (M8N1 warps along M)");
  static_assert(kBN_ % 64 == 0, "BN must cover a full SW128 bf16 atom (64)");
  static_assert(kStages_ >= 2, "pipeline + O staging reuse need >= 2 stages");
  static constexpr int kBM = kBM_;
  static constexpr int kBN = kBN_;
  static constexpr int kBK = kBK_;
  static constexpr int kStages = kStages_;
  static constexpr int kNumWarps = kBM / 16;      // M8N1: kBM/16 warps 沿 M
  static constexpr int kNumThreads = kNumWarps * 32;

  using Element = cutlass::float_e4m3_t;
  using ElementO = cutlass::bfloat16_t;
  using SmemAtom = GMMA::Layout_K_SW128_Atom<Element>;  // fp8 1B 行 128B
  using SmemLayoutA =
      decltype(tile_to_shape(SmemAtom{}, Shape<Int<kBM>, Int<kBK>>{}));
  using SmemLayoutB =
      decltype(tile_to_shape(SmemAtom{}, Shape<Int<kBN>, Int<kBK>>{}));
  using SmemAtomO = GMMA::Layout_K_SW128_Atom<ElementO>;  // bf16 行 128B/64 元素
  using SmemLayoutO =
      decltype(tile_to_shape(SmemAtomO{}, Shape<Int<kBM>, Int<kBN>>{}));
  static constexpr int kSmemBytes =
      kStages * (cosize(SmemLayoutA{}) + cosize(SmemLayoutB{})) *
      sizeof(Element);  // 128/256/s2 -> 2*48KB = 96KB
  static_assert(cosize(SmemLayoutO{}) * sizeof(ElementO) <= kSmemBytes,
                "O staging (bf16 BM x BN) must fit the freed A+B stages");

  using MmaAtom = MMA_Atom<SM89_16x8x32_F32E4M3E4M3F32_TN>;
  using TiledMma = decltype(make_tiled_mma(
      MmaAtom{}, Layout<Shape<Int<kNumWarps>, _1, _1>>{},
      Tile<Int<kBM>, Int<kBN>, _32>{}));
  using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
};
```

四个设计决定，每个都有讲究（前三项在 tile 参数化后都变成了「约束」而不是「常数」，见本节末）：

第一，BK=128 是固定的，BM/BN 可选，默认 128 × 256/s2。 fp8 是 1 字节元素，BK=128 的行恰好 128 B——`Layout_K_SW128_Atom` 的一个 swizzle 周期铺满整行，Bank 冲突分析与第 二十四 章完全一致，只是「行宽 × 元素字节」里的元素字节从 2 变 1。这也是 fp8 tile 与 fp16 tile 的本质差异：同样 128 B 行宽，fp8 的 BK 是 fp16 的两倍，K 方向每 tile 覆盖 128 个元素——MMA 单次装载的 K 维更深，k-slice 数更少。BM, BN 与流水深度则是可调的：35.8.4 节的扫描给出 128 × 256 配 2 级流水是本卡最快档，库默认取的就是它（`Fp8GemmTraits` 的默认实参，也就是 35.8 节标准表量的那一档）。

第二，smem 预算 96 KB。 默认档每 stage 是 A 区 16 KB + B 区 32 KB（BN 翻倍，B 区跟着翻倍），2 级共 96 KB——超过 sm_120 默认的 48 KB，launcher 里 `cudaFuncSetAttribute` opt-in（第 二十八 章同款）。PRO 5000 的 opt-in 上限是 99 KB（不是 Hopper 的 227 KB）：128 × 256 几何下 96 KB 已是极限，3 级要 144 KB 装不下；换回 128 × 128 则每 stage 32 KB、需要 3 级才用满同一笔预算。「几何 × 深度」共用一块 smem，两者此消彼长——这正是 35.8.4 节那张扫描表要回答的问题。

第三，O staging 复用整个 A+B 区。epilogue 需要把 $BM \times BN$ 的 bf16 结果暂存 smem 再 TMA store：默认档是 128×256 的 64 KB。K 循环结束后整个 smem 池都空闲了，直接把 `shm` 头部 rebind 成 `SmemLayoutO`。这里有一个 fp8 特有的限制：**单个 stage 装不下同几何的 O**——B 区 32 KB 放不下 64 KB 的 O，fp16 kernel「复用一个 A stage」的老经验在 fp8 下失效。参数化之后这条几何约束写成通用形式 `cosize(SmemLayoutO)*2B <= kStages*(BM+BN)*128`：K 循环一结束所有 stage 都空闲，O 只要装得进整个 A+B 区即可，静态断言在编译期保证这条约束（默认档 64 KB vs 96 KB 通过；256 × 256/s2 的 O 是 128 KB、与 A+B 区取等，该静态断言仍成立——真正挡住它的是整块 smem 128 KB 超过 99 KB 的 opt-in 上限）。

第四，M8N1 的 TiledMma。8 个 warp 全部沿 M 堆叠（8×MMA_M = 128 = k_{BM}），每 warp 负责输出 tile 的 16 行 × BN 列，acc 是 $4 \times BN/8$ 个 f32/线程（BN=128 时 64 个、默认档 BN=256 时 128 个）。atom 用 `SM89_16x8x32_F32E4M3E4M3F32_TN`：mma.sync 形态的 fp8 指令（sm_89 引入，sm_120 原生支持），K 步长 32——BK=128 的 K-tile 恰好切成 4 个 k-slice，与 `gemm_ss` 的软件流水（源自 flash-attention 的同名工具，`fp8_gemm.cuh` L303–350 的最小集，另有 `convert_layout_ acc_rowcol` 与 `convert_type` 两个 epilogue 工具）对齐。为什么不用 Hopper 的 WGMMA fp8 形态？教学主线选择与第 二十九 章一致的 mma.sync 路线：寄存器 fragment 布局可控、per-thread 的行列坐标视图（epilogue 要用）直接可得，而 WGMMA 的 accumulator 在寄存器里的抽象层级更高、教学展开反而不利。ffpa 的 sm_120 kernel 实测两条路线的峰值差距不大（附录 B）。

四个参数，两条独立的约束。`Fp8GemmTraits` 把 BM, BN, BK, k_{Stages} 全都开放成模板参数。其中 BK 由 SW128 atom 固定（第一项），BM 必须是 16 的倍数（M8N1 要把 warp 铺满 M），BN 必须是 64 的倍数（bf16 的 SW128 atom 行宽 64 元素），$k_{Stages} \geq 2$（O staging 要复用 stage 区）。反过来，BM 与 BN 在性能上的作用完全不对称：BM 只决定 warp 数（$BM/16$，即 CTA 大小），每线程 acc 量 = $BN/2$ 只由 BN 决定。35.8.4 节用 `--fp8-gemm-sweep` 把这张表实测一遍。

下文 35.4–35.6 节的结构讲解与示意图仍以 128^3/s3 为主线，理由是这个配置同时是「smem 边界点」和「协议最对称点」（8 warp × 16 行 = 128 恰好覆盖 BM），三级流水也把预取时机表现得最清楚；换成默认档只是 acc 从 64 变 128 个 f32、流水从 3 级变 2 级，结构与协议一字不改（`Fp8GemmTraits<128,128,128,3>` 一行即可切回）。*所有实测数字（35.8 节）一律取默认档。*

31.4 non-WS 主 kernel：全员 MMA 与内联 TMA

执行协议复刻第 二十九 章的 non-WS 模式：256 线程全员 consumer，`tid=0` 兼任 TMA 发射。两套 barrier：`full[stage]`（事务屏障，init=1，TMA 写完自动翻转）标记「数据就绪」；`empty[stage]`（普通屏障，init=256，每个消费线程 arrive 一次）标记「消费完毕、stage 可覆写」。装载发射的 lambda：

fp8_gemm.cuh L486-512（issue_tma：单 barrier 覆盖 A+B 双字段 + 初始 release）

```
// TMA 发射（仅 tid0）：一个 barrier 同时覆盖 A+B 两个字段的字节数
auto issue_tma = [&](int kt, int stage) {
  cutlass::arch::fence_view_async_shared();
  auto sA =
      make_tensor(make_smem_ptr(a_base + stage * kAElems), SmemLayoutA{});
  auto sB =
      make_tensor(make_smem_ptr(b_base + stage * kBElems), SmemLayoutB{});
  auto gA = local_tile(mA, Shape<Int<kBM>, Int<kBK>>{},
                       make_coord(blockIdx.y, kt));
  auto gB = local_tile(mB, Shape<Int<kBN>, Int<kBK>>{},
                       make_coord(blockIdx.x, kt));
  TmaBarrier::arrive_and_expect_tx(&full[stage],
                                   sizeof(Element) * (size(sA) + size(sB)));
  copy(tma_a.with(full[stage]), a_slice.partition_S(gA),
       a_slice.partition_D(sA));
  copy(tma_b.with(full[stage]), b_slice.partition_S(gB),
       b_slice.partition_D(sB));
};

const int nKt = (K + kBK - 1) / kBK;
// 初始 release: 全线程各 arrive 一次凑满 empty, tid0 发首批 TMA
```

```
  for (int s = 0; s < kStages; ++s) CtaBarrier::arrive(&empty[s]);
  if (tid == 0) {
    // 必须按 nKt 截断：K<=128 时多发的 TMA 是越界读，且会写脏 O staging
    for (int kt = 0; kt < kStages && kt < nKt; ++kt) {
      CtaBarrier::wait(&empty[kt], 0);
      issue_tma(kt, kt);
```

与 attention 版的一个差异：这里 A 与 B 的**同一个** stage 由同一次 TMA 事务填充——`arrive_and_expect_tx` 一次声明 size(A) + size(B) 的字节数，两条 `copy` 挂在同一个 `full[stage]` 上。attention 里 Q/K 与 V 的 stage 深度独立（S_K, S_V），这里 A/B 更新节奏相同（都是每 K-tile 一份），合并进同一个 stage 更自然。初始 release 段先由全体线程把三对 `empty` 各 arrive 一次预先凑满计数（阶段 0），`tid=0` 再逐级发射首批 TMA。

fp8_gemm.cuh L514-543（主循环：wait full → gemm_ss → arrive empty → 预取）

```
  }
#pragma unroll 1
  for (int kt = 0; kt < nKt; ++kt) {
    const int stage = kt % kStages;
    const int phase = (kt / kStages) & 1;
    TmaBarrier::wait(&full[stage], phase);
    cutlass::arch::fence_view_async_shared();

    auto sA =
        make_tensor(make_smem_ptr(a_base + stage * kAElems), SmemLayoutA{});
    auto sB =
        make_tensor(make_smem_ptr(b_base + stage * kBElems), SmemLayoutB{});
    auto tCrA = thr_mma.partition_fragment_A(sA);
    auto tCrB = thr_mma.partition_fragment_B(sB);
    auto tAsA = s2r_thr_a.partition_S(sA);
    auto tBsB = s2r_thr_b.partition_S(sB);
    gemm_ss(tCrC, tCrA, tCrB, tAsA, tBsB, tiled_mma, s2r_copy_a, s2r_copy_b,
            s2r_thr_a, s2r_thr_b);

    CtaBarrier::arrive(&empty[stage]);
    if (tid == 0) {  // 预取 kt+kStages（须在本轮 arrive 之后，见上）
      const int kt_next = kt + kStages;
      if (kt_next < nKt) {
        const int s_next = kt_next % kStages;
        const int ph_next = (kt_next / kStages) & 1;
        CtaBarrier::wait(&empty[s_next], ph_next);
        issue_tma(kt_next, s_next);
      }
    }
  }
```

主循环体与第 二十九 章逐行同构，只保留了 GEMM 必要项：`wait(full)` → `fence_view_async_shared`（TMA 写完的数据对后续 LDSM 可见）→ `gemm_ss`（ldmatrix 与 mma 的软件流水）→ `arrive(empty)` 释放 stage → `tid=0` 预取 $kt+k_{Stages}$。预取时机是第 二十九 章讲透的死锁问题：`empty[s_next]` 的翻转依赖本轮（含 `tid=0` 自己）的 arrive，所以 `wait` 必须排在 `arrive` 之后——挪到循环开头就是确定性死锁。`#pragma unroll 1` 保持循环体不展开（寄存器压力：acc 64 f32 + 双份 A/B fragment，展开两级寄存器就会超限，256 线程的 occupancy 无法维持）。

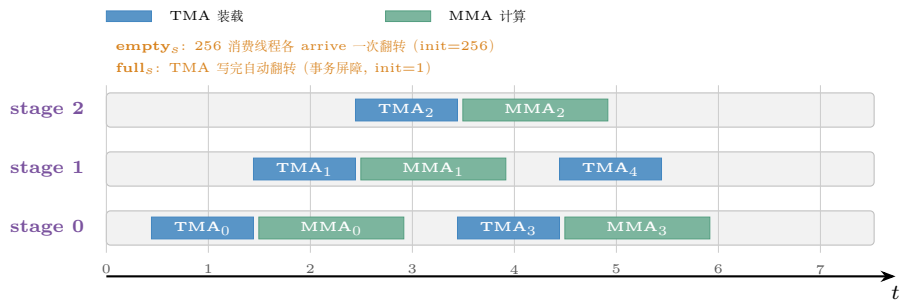


图 31.2: 三级流水的时序示意（横轴时间、纵轴 stage 的矩阵视图）。TMA$_i$ 装载第 i 个 K-tile 到 stage $i \bmod 3$，MMA$_i$ 消费之；装载 i+3 要等 MMA$_i$ 的全体 arrive（empty 翻转），消费 i 要等 TMA$_i$ 的事务完成（full 翻转）。稳态下三条 TMA 与三条 MMA 在时间上错峰重叠——装载带宽与 tensor core 吞吐互不等待，这正是 96 KB smem 所换取的重叠空间。

31.5 Epilogue：在线反量化的四步

fp8_gemm.cuh L545-586（epilogue：scale 查表 → bf16 转换 → STSM → TMA store）

```
  // Epilogue: 在线反量化 C = acc * sa * sb -> bf16 -> STSM r2s（复用 K 循环
  // 结束后的 A+B 区作 O staging）-> TMA store(OOB 行自动裁剪).
  {
    __syncthreads();  // 全部 mma 完成，smem 可覆写
    auto tCrC_rc =
        make_tensor(tCrC.data(), convert_layout_acc_rowcol(tCrC.layout()));
    // per-block 粒度 128：kBN=256 的 tile 跨两块，须逐元素按全局行列查表
    //  （尾块按 m<M / n<N 守卫，不读越界 scale；per-row/per-col 仍按 m/n 查）

#pragma unroll
    for (int r = 0; r < kCRows; ++r) {
      const int m = m0 + get<0>(tScC_rc(r, 0));
      const float sa_r = kPerRowA ? ((m < M) ? sa[m] : 0.0f)  // 尾行 guard
                                  : ((m < M) ? sa[m / kScaleBlock] : 0.0f);
#pragma unroll
      for (int c = 0; c < kCCols; ++c) {
        const int n = n0 + get<1>(tScC_rc(r, c));
        const float sb_c = kPerColB ? ((n < N) ? sb[n] : 0.0f)
                                    : ((n < N) ? sb[n / kScaleBlock] : 0.0f);
        tCrC_rc(r, c) = tCrC_rc(r, c) * sa_r * sb_c;
      }
    }
    auto tCrCh = convert_type<ElementO>(tCrC);

    auto r2s_copy = make_tiled_copy_C(
        Copy_Atom<SM90_U32x4_STSM_N, ElementO>{}, tiled_mma);
    auto r2s_thr = r2s_copy.get_thread_slice(tid);
    auto sC =
        make_tensor(make_smem_ptr(reinterpret_cast<ElementO *>(shm)),
                    SmemLayoutO{});  // 覆盖已空闲的 A+B 区
    auto tCrCh_src = r2s_thr.retile_S(tCrCh);
    auto tCsC_dst = r2s_thr.partition_D(sC);
    copy(r2s_copy, tCrCh_src, tCsC_dst);
    cutlass::arch::fence_view_async_shared();
    __syncthreads();
    auto gC = local_tile(mC, Shape<Int<kBM>, Int<kBN>>{},
                         make_coord(blockIdx.y, blockIdx.x));
    if (tid == 0)
      copy(tma_c, c_slice.partition_S(sC), c_slice.partition_D(gC));
    tma_store_arrive();
```

```
    tma_store_wait<0>();
  }
```

第 三十 章的恒等式在这里实现为四步：

第一步，行列坐标视图。`partition_fragment_C` 给出的 acc 布局是 (MMA=4, MMA_M, MMA_N)，`convert_layout_acc_rowcol`（`fp8_gemm.cuh` L303 起的最小工具集）重排成 $(16, 128)$ 的行列视图，配合 identity tensor 的 partition，每个元素拿到自己的 (m, n) 坐标——查 scale 的寻址依据。

第二步，scale 查表乘回。per-row 时 $\delta_{a,m}$ =`sa[m]` 逐行查（$m < M$ guard 处理尾 tile），per-block 时按**全局** $m/128$、$n/128$ 取块级 scale——k_{BN}=256 的 tile 横跨两个 B 块，整 tile 一个标量的写法会取错 scale 块（34.5.2 节）。B 侧对称。`if constexpr` 保证未选中的分支完全不占运行时指令。

第三步，f32→bf16 转换。`convert_type` 用 `NumericArrayConverter` 把 64 个 f32 整块转 bf16，全程寄存器内完成，零 smem 往返（对比「写 smem 再读回来转」的老写法省一半周转）。

第四步，STSM + TMA store。bf16 fragment 经 `SM90_U32x4_STSM_N`（`stmatrix`）从寄存器直写 O staging smem，`fence_view_async_shared` + `__syncthreads` 之后 `tid=0` 发 TMA store。M/N 尾 tile 靠 TMA store 的 OOB 行裁剪，epilogue 不需要任何尾部分支——唯一要 guard 的就是 scale 查表（第一步），因为 scale 数组本身没有 padding。

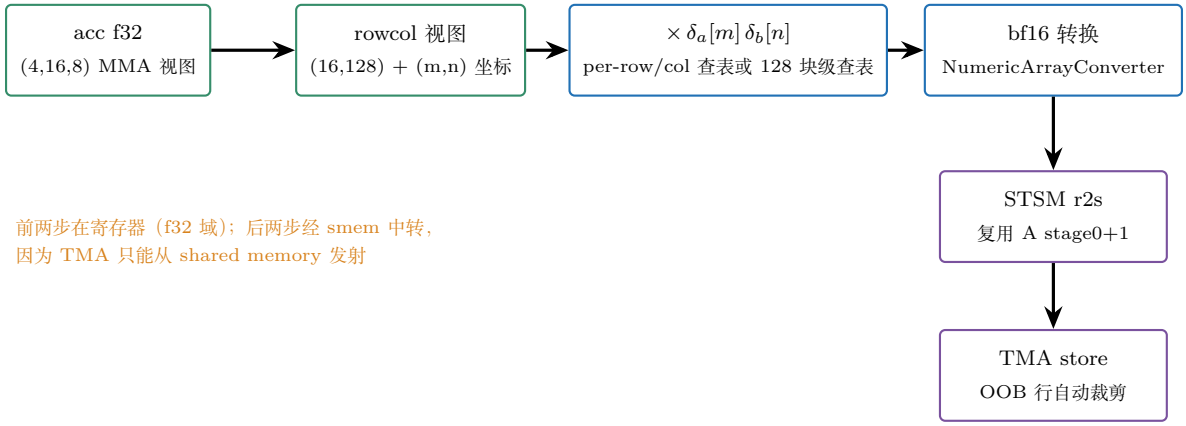


图 31.3: epilogue 数据流。整个反量化只在「scale 查表乘回」一步发生——这是式 30.4 把全部 scale 移出主循环的直接收益：MMA 循环跑的是纯 e4m3 语义，epilogue 每元素额外开销只有一次乘法加一次查表（per-block 时查表索引是一次 128 整除，由编译器强度削减成移位）。

31.6　WS 进阶变体与 setmaxnreg

fp8_gemm.cuh L667-697（WS：producer 专职发 TMA，consumer 全员 MMA）

```
  auto issue_tma = [&](int kt, int stage) {
    cutlass::arch::fence_view_async_shared();
    auto sA =
        make_tensor(make_smem_ptr(a_base + stage * kAElems), SmemLayoutA{});
    auto sB =
        make_tensor(make_smem_ptr(b_base + stage * kBElems), SmemLayoutB{});
    auto gA = local_tile(mA, Shape<Int<kBM>, Int<kBK>>{},
                         make_coord(blockIdx.y, kt));
    auto gB = local_tile(mB, Shape<Int<kBN>, Int<kBK>>{},
                         make_coord(blockIdx.x, kt));
    TmaBarrier::arrive_and_expect_tx(&full[stage],
                                     sizeof(Element) * (size(sA) + size(sB)));
    copy(tma_a.with(full[stage]), a_slice.partition_S(gA),
         a_slice.partition_D(sA));
    copy(tma_b.with(full[stage]), b_slice.partition_S(gB),
         b_slice.partition_D(sB));
  };
```

```
18
19    const int nKt = (K + kBK - 1) / kBK;
20    if (tid < kProducerThreads) {
21      // ---------------- producer warpgroup ----------------
22      cutlass::arch::warpgroup_reg_dealloc<32>();  // 整 warpgroup 对齐执行
23      if (tid == 0) {
24        for (int kt = 0; kt < nKt; ++kt) {
25          const int stage = kt % kStages;
26          const int phase = (kt / kStages) & 1;
27          CtaBarrier::wait(&empty[stage], phase);
28          issue_tma(kt, stage);
29        }
30      }
31      // producer 不参与 epilogue, 直接结束 (后续 barrier 均 consumer-only)
```

WS 版与 non-WS 的**数学逐 bit 相同**（MMA 序列不变），差异全在执行协议（对照第 二十九 章图 29.1 的分析框架）：

- **128 线程 producer warpgroup**：只有线程 0 发 TMA，其余 127 个线程在 `warpgroup_reg_dealloc<32>()` 后空转（寄存器降到 32，把预算让给 consumer）；
- **256 线程 consumer**：`warpgroup_reg_alloc<232>()` 拿回寄存器，跑与 non-WS 完全相同的 `gemm_ss` 循环；`empty` 的 init count 是 256（只数 consumer）；
- **epilogue 换 NamedBarrier**：producer 已经跑完退出，CTA 级 `__syncthreads()` 会数 384 个线程、永远无法满足——换成 `NamedBarrier::arrive_and_wait(256)` 只同步 consumer；
- **setmaxnreg 直接调 cutlass API**：不走 LeetCUDA 的 `NOTES_V2_REG_*` 宏封装，直接调用 `cutlass::arch::warpgroup_reg_{de,}alloc<N>()`（ffpa-attn fp8 persist-D 同款 API；宏只是该 API 的本地 inline-asm 复写，对照见附录 A）。指令生效的两个真正条件本 kernel 天然满足——CuTe 发的 `shared::cta` 形式 TMA，与 `__launch_bounds__(384,1)` 双参数——因此在 sm_120a 与 sm_120f 下都完整保留指令，arch 后缀不是变量（三变量决策树见第 二十八 章专论 §28.6）。

GEMM 与 attention 的一个对照：第 二十九 章大 D 下 non-WS 赢是因为 acc 寄存器占满、setmaxnreg 无寄存器可释放；本章 D 换成了 K 方向的累加（acc 恒 64 f32/线程），寄存器有余量，setmaxnreg 有释放空间——在 128^3/s3 上 WS 稳定快 2.6% ∼ 2.9%（412.1 vs 400.4 @4096^3、460.8 vs 449.3 @8192^3，35.8.4 节扫描表）。但换成默认宽 tile（128×256/s2）这点收益就基本消失了：440.9 vs 440.7（+0.05%）。35.8.5 节的 NCU 读数解释了为什么：BN 翻倍后 tensor pipe 占用率已从 74% 提到 81%，WS 换来的重叠不再构成瓶颈。收益不像 attention 场景那样单边，印证了「WS 的收益来自发射与消费的重叠度 + 寄存器再分配的空间」这个判断：GEMM 的发射路径本来就短（每 tile 只有 4 个 K-tile 的 TMA），重叠收益有限。

31.7 C++ API 与 bench 接线

fp8_gemm.cuh L831-845（workspace 布局：四段 256B 对齐）

```
1  // 一次性 workspace (调用方分配/复用; 上界按 per-row/per-col 计):
2  //   [A8 e4m3 M*K | B8T e4m3 N*K | sa float M | sb float N] 各段 256B 对齐
3  struct Fp8GemmWorkspace {
4    void *buf = nullptr;
5    size_t size = 0;
6  };
7
8  inline size_t fp8_gemm_align256(size_t x) { return (x + 255) & ~(size_t)255; }
9
10 inline size_t fp8_gemm_workspace_size(int M, int N, int K) {
11   return fp8_gemm_align256((size_t)M * K) +
12          fp8_gemm_align256((size_t)N * K) +
13          fp8_gemm_align256((size_t)M * sizeof(float)) +
```

```
          fp8_gemm_align256((size_t)N * sizeof(float));
}
```

workspace 是「调用方分配 + 跨调用复用」的一次性缓冲，四段布局（A_8、$B_8^\top$、δ_a、δ_b，各 256B 对齐）按最粗粒度（per-row/per-col）取上界，四种 mode 共用同一尺寸——切分逻辑在 API 内部重复实现。API 本体：

fp8_gemm.cuh L847-901（fp8_gemm_bf16：约束自检 + workspace 切分 + mode 分发）

```
// BF16 in -> 动态量化 FP8 GEMM（在线反量化）-> BF16 out, 单 stream 语义。
// 同步约定：与 cuBLAS 一致，入队后由调用方在 stream 上同步。
// 约束：K%16==0, N%8==0(TMA 对齐), M 任意；use_ws=true 走 warp-specialized
// 变体（producer 128 + consumer kBM/16*32 线程；默认 tile 下共 384，数值等价）。
inline void fp8_gemm_bf16(const __nv_bfloat16 *A, const __nv_bfloat16 *B,
                          __nv_bfloat16 *C, int M, int N, int K,
                          Fp8GemmScaleMode mode, Fp8GemmWorkspace &ws,
                          cudaStream_t stream, bool use_ws = false) {
  if (K % 16 != 0 || N % 8 != 0) {
    fprintf(stderr,
            "fp8_gemm_bf16: require K%%16==0 (got K=%d), N%%8==0 (got N=%d)\n",
            K, N);
    return;
  }
  uint8_t *p = static_cast<uint8_t *>(ws.buf);
  cutlass::float_e4m3_t *a8 = reinterpret_cast<cutlass::float_e4m3_t *>(p);
  p += fp8_gemm_align256((size_t)M * K);
  cutlass::float_e4m3_t *b8t = reinterpret_cast<cutlass::float_e4m3_t *>(p);
  p += fp8_gemm_align256((size_t)N * K);
  float *sa = reinterpret_cast<float *>(p);
  p += fp8_gemm_align256((size_t)M * sizeof(float));
  float *sb = reinterpret_cast<float *>(p);

  const bool per_row_a = mode == Fp8GemmScaleMode::kPerRowPerCol ||
                         mode == Fp8GemmScaleMode::kPerRowPerBlock;
  const bool per_col_b = mode == Fp8GemmScaleMode::kPerRowPerCol ||
                         mode == Fp8GemmScaleMode::kPerBlockPerCol;
  if (per_row_a)
    quantize_a_perrow_kernel<<<(M + 7) / 8, 256, 0, stream>>>(A, a8, sa, M,
                                                              K);
  else
    quantize_a_perblock_kernel<<<(M + kScaleBlock - 1) / kScaleBlock, 256, 0,
                                 stream>>>(A, a8, sa, M, K);
  fp8_gemm_quantize_b(B, b8t, sb, N, K, per_col_b, stream);

  auto *c0 = reinterpret_cast<cutlass::bfloat16_t *>(C);
#define FP8_GEMM_LAUNCH(PRA, PCB)                                          \
  do {                                                                     \
    if (use_ws)                                                            \
      fp8_gemm_tma_fwd<PRA, PCB, true>(a8, b8t, sa, sb, c0, M, N, K,       \
                                       stream);                            \
    else                                                                   \
      fp8_gemm_tma_fwd<PRA, PCB, false>(a8, b8t, sa, sb, c0, M, N, K,      \
                                        stream);                           \
  } while (0)
  if (per_row_a && per_col_b)
    FP8_GEMM_LAUNCH(true, true);
  else if (per_row_a)
    FP8_GEMM_LAUNCH(true, false);
  else if (per_col_b)
    FP8_GEMM_LAUNCH(false, true);
  else
    FP8_GEMM_LAUNCH(false, false);
#undef FP8_GEMM_LAUNCH
```

```
55 }
```

L868 起的 `FP8_GEMM_LAUNCH` 宏按 $(per_row_a, per_col_b, ws)$ 的八种组合分发到 `fp8_gemm_tma_fwd` 模板实例。三个工程约定：**其一**，约束检查（$K\%16$、$N\%8$）在 API 层做一次、stderr 报错并返回——kernel 与量化 kernel 自己不重复检查（34.2.2 节的约束表）；**其二**，TMA descriptor 在 `fp8_gemm_tma_fwd` 的 host 栈上构建（`make_tma_copy` 内部完成对齐校验），经 `__grid_constant__` 传参，不占 gmem；**其三**，全链路（量化 + 主 kernel）共用调用方给的 stream，无内部同步——与 cuBLAS 的 stream 语义对齐，多流复用没有额外障碍。

bench 侧的接线把 API 挂进标准 `--bench` 分发链：`--bench` 与 `--mnk` 给出的形状直接进 `bench_fp8_gemm`，表内输出 kernel-only 行（量化一次、循环只跑主 kernel）与 E2E 行（循环含量化），参照系是同 shape 的 cuBLAS BF16 GEMM （`CUBLAS_COMPUTE_32F` 累加、`CUDA_R_16BF` 存储）。这给了两个视角：kernel-only 回答「FP8 MMA + TMA 流水本身多快」，E2E 回答「一个真实推理调用（含量化）是否划算」。另有一条独立入口 `--fp8-gemm-sweep`（`bench_fp8_gemm_tile_sweep`）：不复用标准表，而是把 `Fp8GemmTraits` 的 tile 与流水深度逐个实例化、逐个计时，每行附 Max Err 与 smem 占用（35.8.4 节的唯一数据源）。它还接受一个环境变量 `FP8_SWEEP_CFG="128x256x128 s2 ws"`——设了就只发这一个配置，使 `ncu --launch-skip 1 --launch-count 1` 能精确命中目标实例化（kernel 名不含模板实参，`-k` 正则区分不开）。默认档（$128 \times 256 \times 128$/s2）在标准表与扫描表的行标上都显式标了出来（`<= lib default`），读者不必从一堆 tile 里自行对照查找。

31.8 实测：精度与性能 vs cuBLAS BF16

平台：RTX PRO 5000（sm_120a），CUDA 13.x，cuBLAS 随附版本；`--warmup 2 --repeat 3`，输入均匀随机 $[-1, 1]$（srand(42)），精度列为 vs cuBLAS BF16 输出的最大绝对误差。下表量的**就是库默认档**（$128 \times 256 \times 128$/s2，即 35.8.4 节的扫描最优档；旧默认 128^3/s3 的数字见 35.8.4 节扫描表）。

形状	Kernel	Max Err	TFLOPS	vs cuBLAS BF16	vs Ours' CuTE HGEMM
2048^3	cuBLAS BF16 (F32 acc)	—	137.5	1.00x	0.75x
	Ours' CuTE HGEMM (F32 acc)	≈0	183.4	1.34x	1.00x
	FP8 GEMM NonWS (RC)	3.00	243.6	1.77x	1.33x
	FP8 GEMM WS (RC)	3.00	244.6	1.78x	1.33x
	FP8 GEMM+Quant E2E NonWS	3.00	76.3	0.55x	0.42x
	FP8 GEMM+A-Quant E2E, B off	3.00	225.9	1.64x	1.23x
	FP8 GEMM+A-Quant E2E, B off (WS)	3.00	228.4	1.66x	1.25x
4096^3	cuBLAS BF16 (F32 acc)	—	163.5	1.00x	0.67x
	Ours' CuTE HGEMM (F32 acc)	≈0	243.1	1.49x	1.00x
	FP8 GEMM NonWS (RC)	4.50	440.7	2.70x	1.81x
	FP8 GEMM WS (RC)	4.50	440.9	2.70x	1.81x
	FP8 GEMM+Quant E2E NonWS	4.50	186.8	1.14x	0.77x
	FP8 GEMM+A-Quant E2E, B off	4.50	411.2	2.52x	1.69x
	FP8 GEMM+A-Quant E2E, B off (WS)	4.50	413.1	2.53x	1.70x
8192^3	cuBLAS BF16 (F32 acc)	—	163.3	1.00x	0.68x
	Ours' CuTE HGEMM (F32 acc)	≈0	241.8	1.48x	1.00x
	FP8 GEMM NonWS (RC)	6.25	483.6	2.96x	2.00x
	FP8 GEMM WS (RC)	6.25	487.8	2.99x	2.02x
	FP8 GEMM+Quant E2E NonWS	6.25	315.1	1.93x	1.30x
	FP8 GEMM+A-Quant E2E, B off	6.25	429.4	2.63x	1.78x
	FP8 GEMM+A-Quant E2E, B off (WS)	6.25	432.0	2.65x	1.79x
4097×4096^2	Ours' CuTE HGEMM (F32 acc)	≈0	223.8	1.38x	1.00x
	FP8 GEMM NonWS (RC)	4.50	440.4	2.72x	1.97x
	FP8 GEMM WS (RC)	4.50	443.2	2.74x	1.99x

（rc = per-row × per-col；所有 FP8 行均为库默认档 $128 \times 256 \times 128$/s2，CLI 行标已带该后缀；Ours' CuTE HGEMM = 第 二十五 章手写 bf16 HGEMM（S=3、BLK_SW 取该形状较优者、F32 acc）同平台复测，其 Max Err ≈ 0：手写与 cuBLAS 同为 k 顺序 f32 累加，bf16 输出 bit 级一致。四种粒度组合的 TFLOPS 在同 shape 下差异 $< 4\%$（2048^3 实测 rc 243.6 / rb 247.5 / bc 240.9 / bb 248.6，极差 3.2%），表格取 rc 代表，完整 4 组合数据见 `--bench --verbose`。randn (±0.25) 输入的 Max Err 见 34.4.3 节误差指纹表。）

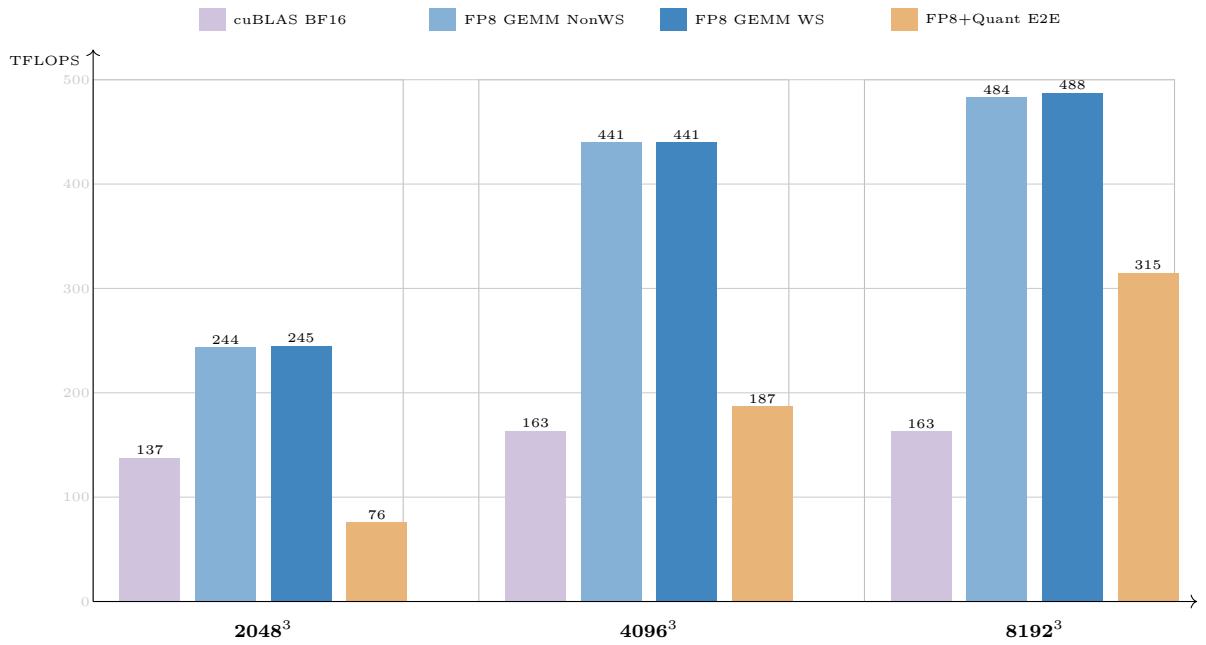


图 31.4: FP8 GEMM vs cuBLAS BF16（PRO 5000，越高越好；默认档 128 × 256/s2）。三个规律：**kernel-only** 随 shape 增大从 1.77× 升到 2.99×——宽 tile 需要足够多的 tile 填满 110 个 SM，2048^3 的 128 × 256 grid 只有 128 个 CTA（35.8.4 节末的波次量化）；**E2E** 的形状敏感度更陡：2048^3 反而低于 cuBLAS（0.55×），因为量化是 $O(MK+NK)$ 的带宽型工序，GEMM 变小时它的占比线性放大（35.8.2 节）；**WS** 全程略优于 non-WS（寄存器有余量、setmaxnreg 有释放空间，35.6 节），但差距远小于 attention 大 D 场景。图中 E2E 柱是「全链路量化」的保守口径；权重 B 离线量化后 E2E 回到 226/411/429 T （35.8.3 节）。

31.8.1　对照基线剖析：cuBLAS BF16 为什么只有 163 T

上面实测表的参照系 cuBLAS BF16 只有 163.5 T（4096^3），对一张 fp16 HGEMM 实测能跑 246 T 的卡来说明显偏低——按硬件能力，240 T 才是正常预期。用 `nsys` 抓同一条 bench 的 kernel 名，答案一目了然（下表读数取自 nsys 会话，与 35.8 节表格差 $< 1\%$）：

cuBLAS 调用（cublasGemmEx）	TFLOPS	实际选中的 kernel
fp16 + `COMPUTE_16F`	235.6	`cutlass_80_tensorop_h16816gemm_256x128_32x3_nn`
fp16 + `COMPUTE_32F`	163.9	`cutlass_80_tensorop_f16_s16816gemm_relu_…_128x64_64x3`
bf16 + `COMPUTE_32F`	161.8	`cutlass_80_tensorop_bf16_s16816gemm_relu_…_128x64_64x3`

三个事实链。第一，三条路径都确实在跑 tensor core（`s16816gemm` 即 `mma.sync.m16n8k16`，163 T 也远超 CUDA core 上限），bf16 的 F32 累加语义没有被打折执行。第二，慢的瓶颈是**精度档**而不是数据类型：fp16 只要切到 `COMPUTE_32F` 就从 235.6 降到 163.9，与 bf16 相同。fp16 可用 `COMPUTE_16F` 进入 256×128 大 tile 的快 kernel，而 bf16 没有 f16 累加这个档位（bf16 输入只能 F32 累加），只能走慢路径。第三，也是根因：本机 cuBLAS（CUDA 13.2）在 sm_120 消费/ 专业卡上，`COMPUTE_32F` 档的 kernel 池里**只有 Ampere 时代（sm_80）的 CUTLASS 2.x 旧 kernel**——上表名字里的 `cutlass_80` 就是明证——没有 Blackwell 的 nvjet/原生 kernel，heuristic 只能在 64×64、128×64 这类小 tile 里选择，选中 128×64 一档（832 μs，是 256×128 快 kernel 的 1.4 倍耗时）。

换调用方式也改变不了结果：用 `cublasLtMatmulAlgoGetHeuristic` 遍历完整算法池（top-8，64 MB workspace），bf16/F32acc 最快只有 191.7 T（256×64 tile），fp16/F32acc 最快 193.1 T，八条候选**全部**是 `cutlass_80_*`——这不是 legacy `cublasGemmEx` 的 heuristic 偏保守，而是库在这张卡上收录的 32F-acc kernel 只有这些。

硬件本身没有瓶颈。本书第 二十五 章的 CuTe HGEMM 用 F32 累加在同一张卡、同一 shape 实测 243 T（fp16 输入；Blackwell tensor core 规格表里 fp16 与 bf16 的 dense 吞吐是同一行数字，F32 累加也不降速）。也就是说 240 T 的预期是对的，只是 cuBLAS 在 sm_120 上没给 32F-acc 档准备相应的 kernel。

于是本章的加速比有两种读法，都成立、都该看：

- **vs cuBLAS BF16 (163.5 T)**：2.70×（4096^3）。这是「直接替换」的真实收益——用户直接把 `fp8_gemm_bf16` 替换掉 `cublasGemmEx` 调用就能拿到的数字，bench 口径与本表一致。

- **vs 手写 bf16 HGEMM 上限 (≈243 T)**：1.81×（440.7 / 243.1）。这是「FP8 量化本身」的物理收益上限——假如 NVIDIA 明天把 32F-acc 池补齐，FP8 相对最优 bf16 kernel 剩下的加速就只有这一点，它才是量化数学（34.4 节）能给出的全部收益。

两个口径相差 0.9 个点，教训是通用的：**跑赢一个慢基线不等于达到硬件上限**。做性能报告时，基线本身要先过一遍 sanity check——本节的 `nsys` 取证（kernel 名 + 单次耗时）就是最小成本的做法。

31.8.2 量化开销

E2E 与 kernel-only 的差就是量化前处理的开销。以 4096^3 为例，直接看时间：GEMM 段 $2\cdot4096^3/440.7\text{T} \approx 312\,\mu s$，E2E 段 $\approx 736\,\mu s$，量化链占用约 $424\,\mu s$。这条链搬运的字节是 $3\times MK$（读 bf16 写 e4m3，A 侧）$+2\times MK$（A 侧第二遍 gmem 读）$+3\times NK$（B 侧）$\approx 8\cdot4096^2\,\text{B} \approx 134\,\text{MB}$（B 的双朝向 staging 再加一遍 smem 但不增 gmem），$424\,\mu s$ 对应有效带宽 $\approx 316\,\text{GB/s}$——**离 PRO 5000 的 DRAM 理论带宽 1344 GB/s 还差四倍多**，差距来自 B 量化 kernel 的两遍读 + smem 往返，以及 A 侧两遍 gmem 读的教学取舍（34.5.1 节）。生产级实现会把 A/B 量化合进一个 kernel、单遍统计 + 写出（ffpa 的 fused 形态，第 三十五 章 28.4.3 节），E2E 数字还有一倍以上空间——这是留给读者的第一道习题。权重离线量化（35.8.3 节）先把 B 的这份开销省掉，A 侧剩下的部分则进一步靠「并进上游 norm/rope 尾部」来吸收。

第二道习题：M=4097 的尾 tile 只含 1 行有效数据，MMA 浪费 127/128，但 4097 行只多出 1 个 m-tile（33 vs 32），额定多出 $1/32 \approx 3\%$ 的工作量——实测 440.4 vs 440.7（$< 0.1\%$）：多出的那一个 m-tile 让总 CTA 数从 512 变 528，最后一波在 110 个 SM 上的填充率从 65% 升到 80%，两项开销恰好抵消。思考：什么时候尾 tile 会成为主要损失？（答：M 略大于 128 的倍数时，尾 tile 占比最高，如 M=129 时一半的 m-tile 是 1/128 利用率。）

31.8.3 权重 B 离线量化：部署形态

上一节把量化链的开销算成 $MK+NK$ 的带宽需求，工程上还有更直接的做法：B 是权重、A 是激活，这个不对称决定了两侧不必付出同等的开销——δ_b 只由权重的数值分布决定，一次算好即可随 checkpoint 落盘；δ_a 依赖当前 token 的取值范围，必须每次前向现算。据此把 API 一分为二：`fp8_gemm_quantize_b` 负责 $B \to B_8^\top + \delta_b$（离线一次；与全链路里的 B 量化共用同一实现，因此结果逐位一致），`fp8_gemm_bf16_b_offline` 每次前向只量化 A，激活侧仅需一个 $O(MK)$ 的暂存（`Fp8GemmActivation`）——$B_8^\top$（NK 字节）与 δ_b 整体退出 workspace。

fp8_gemm.cuh L922-955（fp8_gemm_bf16_b_offline：只量化 A，B 从离线产物取）

```cpp
// B 已离线量化: 每步只量化 A -> 同一个 FP8 GEMM 主 kernel -> BF16 输出。
// 契约: B8T 为 fp8_gemm_quantize_b 产出的 (N,K) 行主序(内维 K, 16B 对齐);
// sb 长度 = N(kPerColB=true) 或 ceil(N/128); kPerColB 必须与离线量化 B 时
// 传给 fp8_gemm_quantize_b 的 per_col_b 同值, 否则整 tile 被错误 scale 静默错值。
// A 侧粒度用模板参数 (部署时 mode 固定, 与 fp8_gemm_tma_fwd 的取值方式一致) 。
template <bool kPerRowA, bool kPerColB, bool kWS = false,
          typename Traits = Fp8GemmTraits<>>
inline void fp8_gemm_bf16_b_offline(const __nv_bfloat16 *A,
                                    const cutlass::float_e4m3_t *B8T,
                                    const float *sb, __nv_bfloat16 *C, int M,
                                    int N, int K, Fp8GemmActivation &act,
                                    cudaStream_t stream) {
  if (K % 16 != 0 || N % 8 != 0) {
    fprintf(stderr,
            "fp8_gemm_bf16_b_offline: require K%%16==0 (got K=%d), N%%8==0 "
            "(got N=%d)\n",
            K, N);
    return;
  }
  uint8_t *p = static_cast<uint8_t *>(act.buf);
  cutlass::float_e4m3_t *a8 = reinterpret_cast<cutlass::float_e4m3_t *>(p);
  p += fp8_gemm_align256((size_t)M * K);
  float *sa = reinterpret_cast<float *>(p);

  if constexpr (kPerRowA)
```

```
    quantize_a_perrow_kernel<<<(M + 7) / 8, 256, 0, stream>>>(A, a8, sa, M,
                                                              K);
  else
    quantize_a_perblock_kernel<<<(M + kScaleBlock - 1) / kScaleBlock, 256, 0,
                                 stream>>>(A, a8, sa, M, K);
  fp8_gemm_tma_fwd<kPerRowA, kPerColB, kWS, Traits>(
      a8, B8T, sa, sb, reinterpret_cast<cutlass::bfloat16_t *>(C), M, N, K,
      stream);
}
```

两条路数值等价：δ_b 与「第几次调用」无关，离线算出的 δ_b 就是全链路里现算的那个，Max Err 与全链路行完全一致（表中 B off 行）；本章最小测试对四种粒度组合与尾部 shape 各跑一遍两条路并逐 bit 比对（`bit-exact=YES, diff=0`）。这条等价性是结构性的而非数值巧合：B 的量化只有一份实现（`fp8_gemm_quantize_b` 由全链路与离线路径共用），A 侧量化与主 kernel 也都是同一份代码，离线只是把「什么时候算 B8T」换了个位置。收益全在时间维度——E2E 从 kernel-only 的 31%/42%/65%（2048/4096/8192^3）回到 93%/93%/89%，对应 226/411/429 TFLOPS（1.64/2.52/2.63× vs cuBLAS BF16）：量化前处理的开销从 $O(MK+NK)$ 缩到 $O(MK)$，省掉的正是 34.5.3 节里开销最大的 B 侧转置 + 双朝向 staging。

A 侧残留开销的量级也值得看一眼：4096^3 只剩 $\approx 22\,\mu s$（A 仅 33.5 MB，warmup/repeat 内大部分被 L2 吸收），8192^3（A 134 MB）升到 $\approx 287\,\mu s$——这正是带宽受限下的耗时量级，也说明「B 离线」之后 E2E 的瓶颈已经回到主 kernel。生产推理栈（TensorRT-LLM、vLLM、ffpa-attn）里的 FP8 权重都是离线量化后按 $B_8^\top$ 布局落盘的，运行时只做激活量化：本节的 `_b_offline` 路径就是那个形态的最小复刻，也是本部分从教学链路走向可部署算子的一步。

还能进一步压缩的是 A 侧那 $\approx 287\,\mu s$ 本身。它的根因是「先物化 BF16 激活、再单独起一个量化 kernel」这个两段式：A 写进显存（bf16），量化 kernel 再读一遍、写一遍（e4m3），$3 \times MK$ 的额外流量全在这个往返里。生产引擎的做法是**把量化挂到上游算子的尾部**——同一层的 fused LayerNorm / RMSNorm（或 add 残差）本来就要把结果写回显存，可在 epilogue 里同时算出 per-token（或 per-block）amax、直接写出 $A_8 + \delta_a$；RoPE 之后再量化也是同理。上游那个写回动作被复用，bf16 激活的物化与独立的量化 kernel 一起消失，这段开销理论上可以降到零。

代价是这种融合没有通用写法：每层的前置算子组合因模型而异——有的是 `norm`→qkv，有的是 `norm`→rope→qkv，有的把 qkv 与 gate/up 的输入合并成一次 fused MLP，还有的用 QK-Norm（量化点要挪到 norm 之后）——融合位置、amax 的归约域、甚至量化粒度都要跟着模型结构调整。它属于推理引擎的「模型适配层」，不是一个能写成通用 kernel 的东西，本教学案例因此只把 GEMM 侧（本部分）做到生产形态，激活量化的融合留给读者按自己模型的计算图接入。

31.8.4　tile 几何与流水深度：128^3 并非最优

128^3/s3 是最自然的一档：tile 与 smem 上限对齐、三级流水是能容纳的最大整数级数、SW128 恰好铺满一行。但 tile 几何与流水深度共用同一块 99 KB 的 smem——BN 每加一倍，每 stage 就多 16 KB，能塞的级数随之下降。哪一头收益更大只能实测，而扫描的结论正是 35.3 节里那个默认档（128×256/s2）：本节给出它的证据，也给出它「并非普适」的那个反例。扫描入口 `--fp8-gemm-sweep`（形状由 `--mnk` 给）把 `Fp8GemmTraits` 的 BM, BN 扫 $\{64, 128, 256\}^2$、k_{Stages} 扫 $\{2, 3, 4\}$，在 4096^3 与 8192^3 两档形状上各扫一遍（大形状的意义在 K 长度：$K/128$ 从 32 个 K-tile 翻到 64 个，流水的启动/排空占比减半），每个配置先与 cuBLAS BF16 对一次 Max Err（4096^3 全组合 4.50、8192^3 全组合 6.25，与主表 rc 行同值——tile 参数改错会立刻在这一列暴露出来），再走同一套 warmup/repeat 计时。为不让编译时间失控，超限的组合不实例化，只按公式列出所需 smem。

变体	tile $BM \times BN$	每 stage smem	TFLOPS（4096^3 / 8192^3）		
			k_{Stages}=2	k_{Stages}=3	k_{Stages}=4
non-WS	64 × 64	16 KB	225.9 / 251.7	229.3 / 254.6	219.1 / 239.5
non-WS	64 × 128	24 KB	308.6 / 335.2	284.9 / 311.6	288.5 / 316.9
non-WS	64 × 256	40 KB	350.8 / 383.7	—（123）	—
non-WS	128 × 64	24 KB	308.7 / 333.0	296.0 / 327.0	296.7 / 328.5
non-WS	128 × 128	32 KB	411.6 / 444.4	400.4 / 449.3	—（128）
non-WS	128 × 256	48 KB	**441.9 / 484.7**	—（144）	—
non-WS	256 × 64	40 KB	345.4 / 382.3	—	—
non-WS	256 × 128	48 KB	407.8 / 445.5	—	—
non-WS	256 × 256	64 KB	—（128）	—（192）	—
ws	128 × 64	24 KB	290.5 / 326.1	299.5 / 329.5	298.0 / 331.0
ws	128 × 128	32 KB	417.8 / 452.1	412.1 / 460.8	—（128）
ws	128 × 256	48 KB	**443.7 / 487.8**	—（144）	—

表 31.1: 主 kernel 的 tile × 流水深度扫描（PRO 5000 sm_120a，kernel-only，per-row × per-col，warmup 2 / repeat 3）。**加粗行即库默认档**（128 × 256/s2），也是两档形状的实测最快档；CLI 扫描表里同一行标为 `<= lib default`。每格两个数字 = 4096^3 / 8192^3 的 TFLOPS；cuBLAS BF16 基线分别为 163.1 / 163.4 T。两档形状各跑两次，逐配置偏差 ≤ 0.2%。`---(x)` 表示该组合所需 smem 为 x KB、超出本卡 99 KB 的 opt-in 上限，故未实例化——这个门槛只由 tile 几何决定，与形状无关。WS 变体只扫 $BM \le 128$（k_{BM}=256 时 512 个 consumer 线程按 `setmaxnreg` 的 32/232 分配需要 122 K 寄存器 > 64 K 寄存器文件，kernel 里用 `static_assert` 拦截在编译期）。

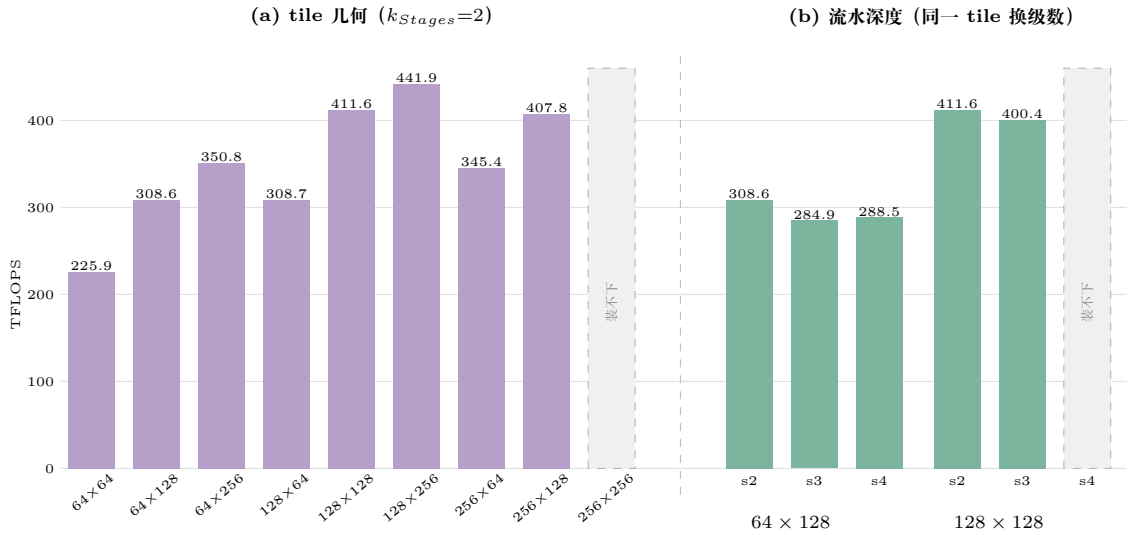


图 31.5: tile 几何与流水深度的两个「反直觉」方向（non-WS，4096^3，越高越好）。(a) BM 与 BN 不对称：128 × 256 比 256 × 128 快 8.3%，64 行档两者打平——因为 M8N1 把 B 的 fragment 复制到每个 warp，BN 才是摊销 smem 读量的那个参数（35.8.5 节有实测波前数）。(b) 流水深度不是越深越好：窄 tile 上 s=4 一致低于 s=2/3（96 KB 已经占满每 SM 的 smem，加深只是多缓存数据、换不来 CTA 并发），128^2 上 s=2 略胜 s=3；s=4 的劣势在 8192^3 上同样成立。本图用 4096^3 作图，8192^3 的整套数字见表 31.1（两档的形状依赖讨论见正文「其二」）。

四条结论：

其一，BN 比 BM 收益大。128 × 256 比 256 × 128 在两档形状上都快约 8.5%（441.9 vs 407.8；484.7 vs 445.5）；64 行档两者打平（350.8 vs 345.4；383.7 vs 382.3）。这个比例在两档形状上几乎不变——说明它由 tile 几何本身决定，与 K 长度无关，正是下一段的解析模型所预测的。机理在 35.8.5 节的读数里：M8N1 下每个 warp 都要读整个 B tile，于是每输出每 K-tile 的 smem→ 寄存器读量是 $8 + 128/BN$ 个元素（8 = warp 数即 B 的复制倍数，$128/BN$ 是 A 侧摊销项）。BN 从 128 提到 256 把读量从 9 压到 8.5（−5.6%），实测 smem 读波前 37.75M → 35.65M（−5.6%，逐字节吻合）。BM 只改 CTA 大小（warp 数 = $BM/16$），所以 256 行的 tile 除了一半的 CTA 数之外没有额外好处，1.6%/0.4% 的量级已在噪声内。

其二，流水深度 2 级与 3 级等价，4 级稳定无收益。两档形状上多数几何的 s=2 与 s=3 都落在 ±2% 内互换（128^2：4096^3 上 s2 略胜 2.8%、8192^3 上 s3 反超 1.1%），唯一稳定偏离的是 64 × 128（s2 在两档都快 8%，那是它 s=3 独占

72 KB 后 CTA 并发的损失）；把 6 个两两可比的几何取平均，$s3/s2$ 从 4096^3 的 0.981 升到 8192^3 的 0.994——*方向一致但幅度只有 1 个百分点*，说明三级以上的收益要靠更长的 K 才体现得出来（$K/128$ 从 32 个 K-tile 翻到 64 个，启动/排空占比减半）。s=4 则在两档上一致最低（只有 ws 窄 tile 上与前两级持平），原因还是每 SM 只有 100 KB smem：128^3/s3 就已经占用 96 KB、只能放 1 个 CTA，第 4 级只换来更长的尾部 drain。（这与第 三十九 章 FP4 persist-D 的 stages 实验同结论：流水深度不是 attention/GEMM 的瓶颈。）

这条等价性有一个直接用处：s=2 **省下的那一级 stage 区可以用来增大** BN。128×256/s2（441.9 / 484.7）比 128×128/s3（400.4 / 449.3）快 10.4% / 7.9%——同样的 96 KB smem，换成「宽度优先」的分配收益更高。两条轴不是独立的两个选择，而是同一份 smem 预算下的取舍。

其三，256×256 在本卡装不下。两级就要 128 KB > 99 KB 上限。「tile 越大 smem 利用率越高」在 opt-in 上限面前不成立：$BM \times BN$ 翻倍让每 stage 的 $A+B$ 占用同步翻倍，几何与深度只能二选一。这也解释了为什么 128^3 + 3 级看起来自然——它恰好在「几何 × 深度」的乘积边界上。

其四，「最快」只在 M, N 够大时成立。把同一套扫描在 2048^3 上再跑一遍，结论翻转：128×128/s3 有 292.8 T（non-WS）/ 295.1 T（ws），128×256/s2 只有 243.6 T / 243.3 T——宽 tile 反而慢 17%。原因是**波次量化**：128×256 的 grid 是 $16 \times 8 = 128$ 个 CTA，本卡 110 个 SM 要跑 2 波，填充率 $128/(2 \times 110) = 58\%$；$128 \times 128$ 给出 $16 \times 16 = 256$ 个 CTA（3 波，78%）反而更均匀。BN 决定「每字节 smem 的 MMA 次数」（单 CTA 效率），波次量化决定「CTA 数的均匀性」（全卡利用率），两者在 96 KB 这块预算里相互制约——这正是生产库要按问题尺寸查表选 tile 的原因。本教学案例的默认档是一个固定值，因此在 2048^3 及更小的形状上它 *不是*最优：把模板实参换回 `Fp8GemmTraits<128,128,128,3>` 即可复现 292.8 T。这条反例也提醒读者：本章所有加速比都带「形状前提」，读表先看形状。

综合两条轴，**大形状上的最快档是 ws $128 \times 256 \times 128$/s2**：$4096^3$ 上 443.7 T（2.72×）、8192^3 上 487.8 T（2.99×），比 128^3/s3（412.1 / 460.8）快 7.7% / 5.9%。**库的默认实参就是这一档**（35.3 节）——追吞吐的生产配置本该如此，改动只是一个模板实参（`Fp8GemmTraits<128,256,128,2>`），协议、epilogue、API 一行不动。结构讲解（35.4–35.6 节）仍以 128^3/s3 为主线，理由在 35.3 节已说明：它同时是「smem 边界点」和「协议最对称点」（8 warp × 16 行 = 128 恰好覆盖 BM），讲清了它，其余几何都只是换个数字。

31.8.5　最快配置的 NCU 读数：为什么是它

把三个配置放进 `ncu` 看同一组指标（4096^3，`--launch-skip` 跳过首次冷启动，取稳态一次；第三列即库默认档；注意 `ncu` 挂载本身会让 kernel 慢约 5%，同一组读数只用于横向比较）：

指标	non-WS 128^3/s3	ws 128^3/s3	ws 128×256/s2
kernel 时间（μs）	365.9	357.6	**337.6**
pipe_tensor（% 峰值，elapsed）	74.4	76.9	**80.9**
寄存器 / 线程（launch）	168	168	168
smem / CTA（NCU 报告，动态）	100.35 KB（96）	同左	同左
CTA / SM（寄存器限制 / smem 限制）	1 / 1	1 / 1	1 / 1
活跃 warp / cycle	7.96	8.88	8.79
smem→ 寄存器读波前（M）	37.75	37.75	**35.65**
bank conflict（ld / st）	0 / 0	0 / 0	0 / 0
L1TEX 吞吐（% 峰值）	44.2	45.8	44.8
L2 吞吐（% 峰值）	47.6	49.0	39.2
DRAM 吞吐（% 峰值）	7.6	7.6	8.0
指令数（M）	82.0	81.1	72.2

表 31.2: 三个配置的 NCU 对照（PRO 5000 sm_120a，4096^3，per-row × per-col，kernel-only）。时间列是 `ncu` 挂载下的读数，与 35.8 节的无插桩数字不可直接比；其余列是硬件计数器。

bank conflict 精确为 0。读 37.75M 次、写 262K 次 smem 事务，bank conflict 全 0——SW128 swizzle（第 二十四 章）+ `LDSM` 的组合在这个 tile 几何下完全无冲突。这条读数很重要：它说明本 kernel 的 smem 代价**不是**冲突，而是**量**（下一条）。

smem 读量被 M8N1 放大 4.5 倍，增大 BN 是唯一的缓解手段。128^2 的 37.75M × 128 B = 4.83 GB smem 读出 vs 理论下界 $M \cdot N \cdot (K/128) \cdot (128/BN + 128/BM) = 1.07$ GB，放大 4.5×；换成 128×256 后同口径 4.56 GB（5.7× 于它自己的下界，但绝对值降了 5.6%）。多出来的是 B 的复制项：8 个 warp 沿 M 堆叠，每个 warp 都要把整块 B 读进自己的寄存器。要消掉这一项得把 warp 从「全沿 M」改成二维排布（例如 4×2），代价是 acc 的跨 warp 归约与 epilogue 的行列视图复杂化——教学实现刻意留了这个 8× 的简化，读者若要继续优化这条路，起点就在这里。

寄存器 168/线程、1 CTA/SM：两个限制同时卡住。三个配置的编译器分配都是 168 寄存器/线程（$384 \times 168 = 64.5$K 已达到 64K 寄存器文件上限，`launch__occupancy_limit_registers` = 1），smem 侧 96 KB 也只允许 1 个 CTA（`launch__occupancy_limit_shared_mem` = 1）。WS 变体的收益因此只能来自执行协议本身：`setmaxnreg` 把编译期的 168 在运行期重分成 consumer 232 / producer 32，活跃 warp 数从 7.96 升到 8.88（+11.6%），非 WS 版在 K 循环里要靠同一个线程既发 TMA 又跑 MMA，consumer 的等待更难被填上。

瓶颈仍在 tensor pipe，前方还有余量。三档 pipe_tensor 是 74.4% → 76.9% → 80.9%（活跃周期口径 82.9% / 87.8%，SM 间峰值 94.3%），与吞吐排序完全同序；与此同时 DRAM 只有 8%、L2 39–49%、L1TEX 45%——访存侧远未饱和。剩下 19% 的空隙来自 epilogue（STSM + TMA store 的串行段）、K 循环起停与 CTA 尾部，这也正是 128×256 能再快 7.7% 的地方：一半的 CTA 数就是一半的起停与 epilogue 次数，而 tensor pipe 的占用率顺势抬高。再往上走（256×256、更深流水）受 99 KB smem 限制，剩下的优化方向只有减少 B 的复制项或换 WGMMA/fp4 形态——「先看 roofline、再看瓶颈项」的分析顺序在这里给出了同一个答案。

31.8.6 精度判定

Max Err 随 K 从 3.0（2048）→ 4.5（4096）→ 6.25（8192）近似 $\sqrt{K}$ 增长——正是 34.4.3 节随机符号误差模型的预测（$\sigma_{\text{err}} \propto \sqrt{K}$，而 C 元素本身 $\propto \sqrt{K}$，相对量不变）。健康检查：$\|C_{fp8} - C_{ref}\|_F / \|C_{ref}\|_F \approx 3.6\%$（第 三十 章 34.6 节，全组合一致），vs cuBLAS 的 Max Err 只是把这个零均值噪声的最大值呈现出来。真实模型部署时该看的指标是端到端任务的退化（PPL / 生成质量），量化 GEMM 的逐元素 max err 在随机数据上天然是「几倍 σ」的量级，不必追求 10^{-3} 级。

31.9 小结

本章把第 三十 章的量化 workspace 送入一个 CuTe FP8 GEMM 主 kernel：128^3 tile + SW128 + 三级 mbarrier 流水（96 KB smem；库默认档把同样这 96 KB 花成 128×256 配 2 级）、M8N1 的 `SM89_16x8x32` TiledMma、四步 epilogue（rowcol 视图 → scale 查表 → bf16 转换 → STSM + TMA store），WS 变体以 setmaxnreg 收尾。C++ API `fp8_gemm_bf16` 把量化与 GEMM 封成一次调用，在标准 bench 链上对 cuBLAS BF16 取得 kernel-only 1.8× ∼ 3.0×、E2E 0.6× ∼ 1.9×（形状依赖）的成绩——而 35.8.1 节的基线剖析说明这个数字的两种读法：库现状口径 2.70×，手写 bf16 上限口径 1.81×。35.8.3 节再把 E2E 路径补齐：权重 B 离线量化后，每次前向只剩激活量化（$O(MK)$），E2E 回到 1.64× ∼ 2.63×——量化 GEMM 从「能跑赢 cuBLAS」走到「部署里真的划算」；激活侧残余的那点带宽开销，最后由「并进上游 norm/rope 尾部」的模型相关融合清零。最后 35.8.4∼35.8.5 节把 tile 这件事从「128^3 是惯例」变成「128^3 是折中」：把 BM, BN 与流水深度逐个实例化实测后，大形状上最快的档是 128×256 配 2 级流水（443.7/487.8 T，比 128^3/s3 再快 7.7%/5.9%），库默认实参取的就是它；而它成立的理由不在计时表里，在 NCU 读数里——*BN 决定每字节 smem 供给 tensor core 的 MMA 次数，流水深度决定重叠，两者都被 99 KB 的 smem 上限约束住。*这条结论还带一个形状前提：2048^3 上宽 tile 只有 128 个 CTA，波次量化让它比 128^3/s3 慢 17%——*最优 tile 是问题尺寸的函数。*两章合起来，读者手上有了一条从量化数学到可部署算子的完整、可改、可测的参考实现。

延伸阅读

- 第 二十四 章：TMA descriptor 与 SW128 swizzle 的机制细节。
- 第 二十九 章：non-WS 协议的死锁分析与跨 tile 预取——本章主循环的直接先例。
- 第 三十六 章：persist-D 的 $s_{dequant}$ 折叠——本章 epilogue 的 attention 版（带在线 softmax 修正）。
- DeepSeek-V3 技术报告（fp8 训练的 blockwise 量化方案）、NVIDIA ModelOpt 的 W4A8/W4A4 校准流程，以及 TensorRT-LLM / vLLM 的 fp8 权重加载实现（`fp8_gemm_bf16_b_offline` 的生产版，35.8.3 节）：真实权重分布下的量化粒度、outlier 处理与权重离线落盘格式。

第三十二章　FP4 GEMM（一）：NVFP4 线格式、两级量化与误差模型

32.1　本章导读

第 三十 章与第 三十一 章把 BF16 矩阵压到 e4m3：一个符号位、四个指数位、三个尾数位。本章再往下走一档——**NVFP4**（e2m1 + ue4m3 micro-scale），每元素只有 4 bit。这一步不是「把 e4m3 的位数降一半」那么简单：e4m3 有 8 个数值位，相邻码点的相对间距恒为 $2^{-3}/2$；e2m1 只有 $1+2+1$ 位，**每个 2 的幂区间只放得下两个码点**，量化步长随量级成倍变化。精度的损失不是线性缩小，而是结构改变——这正是本章要讲清的事。

而 NVFP4 能把这一档损失控制住的关键，是它的**两级 scale 结构**：数据侧每 16 个元素共享一个 ue4m3 缩放因子（2^{-3} 的步长变成 2^{-3} 的 *scale* 步长），再叠一层 fp32 的矩阵/行/列级 scale 把这个 ue4m3 移到量程中央。两级结构加上硬件 blockscale MMA，使得「per-16 scale」在这套设计里既省存储（16 元素 1 字节）又完全没有额外开销（由 MMA 硬件消费，不进寄存器算术）。

本章的组织方式：先讲线格式（e2m1 与 ue4m3 的位域与码表），再讲量化算法与折叠恒等式，然后花整整一节把**误差模型**完整推导——这是全书对 NVFP4 唯一能讲深的定量部分，本节会给出逐格累加、饱和项、ue4m3 SF 舍入项三者的分别贡献，并用蒙特卡洛与 kernel 实测三方对齐（结论：单侧 10.2%、GEMM 双侧 $\sqrt{2}\times 10.2\% = 14.4\%$，与实测 14.4% 吻合到小数点后两位）。最后给出 SF 的 gmem 512 B 原子布局与量化前处理 kernel 的工程细节。

前置章节：第 三十 章（量化数学与 scale 折叠，必须）、第 三十八 章（NVFP4 attention 里已用过同一套 e2m1/ue4m3 记法，可作横向对照）、第 二十四 章（TMA 与 swizzle，第 三十三 章要用）。**源码区间**：`fp4_gemm.cuh` L58–366（Phase 10.1–10.2）。**验证方式**：`book/tests` 的 `ch36_fp4_quantize.cu`（SF 字节通式全量枚举 + 量化逐 bit 比对 + 硬件码点事实），`notes_v2_sm120a.bin --fp4-gemm`（端到端精度）。**本章不展开 kernel**：blockscaled MMA 的发射、SF 片段分区、TMA 描述符与流水协议全部留给第 三十三 章。

32.2　问题与动机

先把「为什么要两级」这个问题说明白，否则后面的设计缺乏动机。

32.2.1　e2m1 的格：每个 2 的幂区间只有两个码点

e4m3 的码点是「每个 2 的幂区间 8 个均匀码点」，相对步长恒为 2^{-3}；误差均匀分布在 $\pm 2^{-4}$，相对 RMS $\approx 3.6\%$。e2m1 的码点表则长这样：

码（二进制）	值	所属区间	该区间步长
000	0	$[0, 2^{-1}]$	2^{-1}
001	0.5	$[2^{-1}, 2^{0}]$	2^{-1}
010	1.0	$[2^{0}, 2^{1}]$	2^{0}
011	1.5	同上	2^{0}
100	2.0	$[2^{1}, 2^{2}]$	2^{1}
101	3.0	同上	2^{1}
110	4.0	$[2^{2}, 2^{3}]$	2^{2}
111	6.0	同上（饱和）	2^{2}

（符号位是第 4 位，正负对称。）同一区间内两个码点的间距等于区间半宽，故**相对步长在区间下沿达 50%，在上沿为 25%**——是 e4m3 的 2^{-3} 的 $2\sim4$ 倍。更严重的是信息量：e2m1 的 8 个幅值点要覆盖 $[0,6]$，在 $[0,6]$ 上「平均」步长是 $6/8=0.75$。

32.2.2 单级 per-group scale 的两个失效模式

最朴素的想法是照抄 MXFP4：每 32 个（或 16 个）元素共享一个 8-bit 指数型 scale。这在 NVFP4 里的 *level-2* 就是这个形态，但它单独使用会遇到两个问题：

- **下溢归零**：ue4m3 的最小正数是次正规 $2^{-9}\approx0.001953$。若某组的 amax $<6\times2^{-10}=0.005859$，则 amax/6 $<2^{-10}$，编码直接舍到 0，整组 16 个元素全部变成 0——不是「精度差」，是**信息全部丢失**（相对误差 100%）。
- **量程浪费**：ue4m3 的满量程是 448，而一次矩阵乘里各组的 amax 通常只落在很窄的一段（比如激活值的行内 amax 都在 $1\sim3$），直接把 amax 当 scale 用，等于只用掉 ue4m3 码表的一小段。

level-1 就是为了解决这两件事：在 per-16 的 ue4m3 之上，再加一层 fp32 的矩阵/行/列级 scale s，把数据先除到「amax 落在 $6\times2^{-6}\sim6\times448$ 之间」的理想区间（即 SF 落进 ue4m3 的正常数区，$2^{-6}\sim448$），于是 level-2 的 SF 永远不会下溢。s 的取值让一整行（或一整列）的 amax 除以它之后恰好逼近满量程 $2688=6\times448$：

$$s_A[m]=\frac{\max_k|A_{m,k}|}{2688},\qquad s_B[n]=\frac{\max_k|B_{k,n}|}{2688}. \tag{32.1}$$

这个 2688 就是第 三十八 章里 attention 侧那个「two-level 常数」的同一个数——它是 e2m1 满量程与 ue4m3 满量程之积，代表两级 scale 能表达的最大绝对值。

32.2.3 两级折叠恒等式

把两组 scale 代进 $C=AB^\top$。对 per-row（A）× per-col（B）粒度，与第 三十 章式 30.4 完全同构：

$$C_{m,n}=\sum_k \underbrace{s_{A,m}\,\hat{A}_{m,k}}_{\text{level-1}}\underbrace{\mathrm{SF}^A_{m,\lfloor k/16\rfloor}\,\tilde{A}_{m,k}}_{\text{level-2}}\cdot\underbrace{s_{B,n}\,\hat{B}_{n,k}}_{\text{level-1}}\underbrace{\mathrm{SF}^B_{n,\lfloor k/16\rfloor}\,\tilde{B}_{n,k}}_{\text{level-2}}=s_{A,m}\,s_{B,n}\underbrace{\sum_k \hat{A}_{m,k}\hat{B}_{n,k}}_{\text{MMA 硬件}} \tag{32.2}$$

其中 $\hat{A}_{m,k}=\mathrm{SF}^A_{m,\lfloor k/16\rfloor}\,\tilde{A}_{m,k}$ 是「**level-2 已折回**」的值，$\tilde{A}$ 才是真正存进 4-bit 的 e2m1 码。写清楚这条恒等式的三层含义：

1. **level-2 由 MMA 硬件消费**。第 三十二 章开头说的「零开销」就指这个：PTX 的 `block_scale` 语义让 SF 参与乘加，但 **不进任何寄存器算术**——数据与 SF 都只是被 MMA 读走。软件完全不需要写一行「乘 SF」的代码。
2. **level-1 必须回到 epilogue**。s_A,s_B 是矩阵级/行级/列级的 fp32 张量，规模远超寄存器，只能在 epilogue 按 (m,n) 查表乘回——这与第 三十一 章 FP8 epilogue 的形状完全相同，是同一段查表逻辑换了个数据源。
3. **两级的分工是「精度 vs 量程」**。level-2 提供 per-16 的局部自适应步长（把 $[0,6]$ 的固定格变成每 16 个元素重新标定的格），level-1 提供量程自适应（让 level-2 永远工作在 ue4m3 的好区间）。

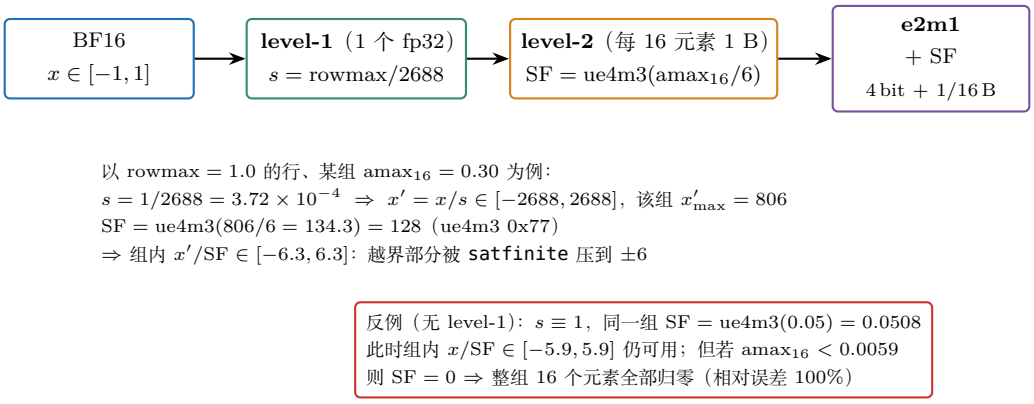


图 32.1: NVFP4 两级量化的数据流。level-1 是 fp32 的粗标定（每行/每列一个），把整行数据移到 e2m1×ue4m3 的联合量程 $[-2688, 2688]$ 附近；level-2 是 ue4m3 的细标定（每 16 个 K 元素一个），把组内数据正好铺满 $[-6, 6]$。反例框给出单级的失效路径：幅度小于 6×2^{-10} 的组会因为 ue4m3 舍入到 0 而整组归零。

图 32.1 的反例框值得单独强调：**单级归零是「整组」事件，不是「单个元素」事件**。一个 128×128 的 FP4 tile 里有 1024 个 16 元素组，只要某些组的幅度偏小，整组就会归零，误差不可控。这也是第 三十三 章的四组 scale 模式（单级 / A 侧 level-1 / B 侧 level-1 / 两级）里，**单级那一档只能作为对照基线**、不能用于生产的原因。

32.3　NVFP4 线格式

32.3.1　e2m1 与 ue4m3 的位域

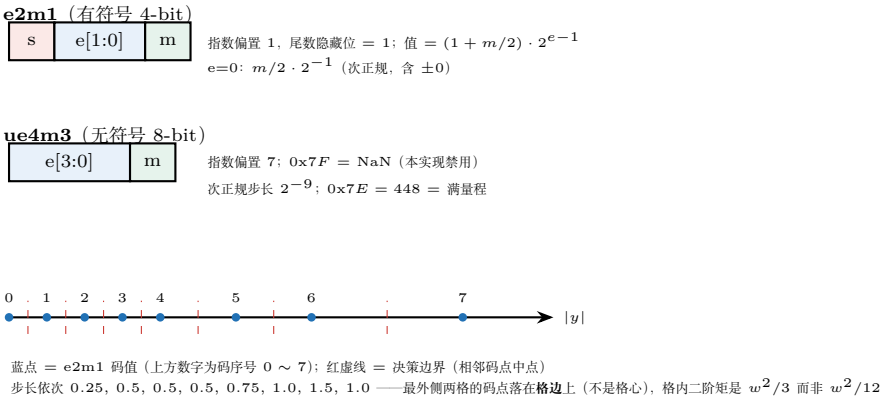


图 32.2: NVFP4 的两个 4-bit 类型。上：e2m1 位域与数轴（步长随量级翻倍，最外侧两格「码点在格边」是误差模型里最大的一项贡献，见 32.5 节）；下：ue4m3 位域（scale 专用，量程 $[2^{-9}, 448]$，0x7F 是 NaN）。

图 32.2 的数轴把 e2m1 的「非线性格」画出来了：码点 $0, 0.5, 1, 1.5, 2, 3, 4, 6$ 之上的决策边界是相邻码点中点。注意最左格 $[0, 0.25]$ 与最右格 $[5, 6]$ 的特殊性——0 与 6 都落在格的**边界**上，而不是格心。这一点在误差模型里是分量最大的一项。

32.3.2　硬件转换指令：平局取偶与 satfinite

f32 → e2m1 是一步到位的 PTX 指令：

```
cvt.rn.satfinite.e2m1x2.f32   d, a, b;    // d 是 .b8，打包两个 4-bit
```

两条语义细节必须严格确认，否则 CPU 参考实现永远无法与 kernel 输出对上：

- `.rn` 是 round-to-nearest-**even**。e2m1 与 ue4m3 的平局都取「尾数为偶」的那个码。实测（`ch36` 测试的 case F，探针见 `.tmp/fp4bs/probe_tie.cu`）：`e2m1(0.25)=0x0`（0 与 0.5 的中点取码 0）、`e2m1(1.75)=0x4`（取 2.0 而非 1.5）、`ue4m3(432)=0x7E`（416 与 448 的中点取 448，因为 416 的码 0x7D 是奇数码）。

- `.satfinite` 让越界输入饱和到 ±6，而不是产生 Inf/NaN。NaN 输入落到 +6。这一条是保护措施而不是优化：*level-2* 的 SF 是 *ue4m3* 舍入后的值，若它比 amax/6 小，则组内最大值除以它之后会超过 6——没有 satfinite 就会产生 NaN 并污染整条链。

32.3.3 SF 的 gmem 布局：512 B 原子

level-2 的 SF 在显存里不是「按 (MN, K/16) 平铺的二维数组」，而有专用的 **128 行 × 4 个 SF = 512 B 原子**（CUTLASS 的 `Sm1xxBlockScaledConfig`，`Blk_MN=_128`、`Blk_SF=_4`）。一个原子覆盖 128 行 × 64 个 K 元素，原子内**全连续**：这正是 TMA 与 MMA 都适配的形态——SF 的搬运和寻址都是代价最低的连续访问。

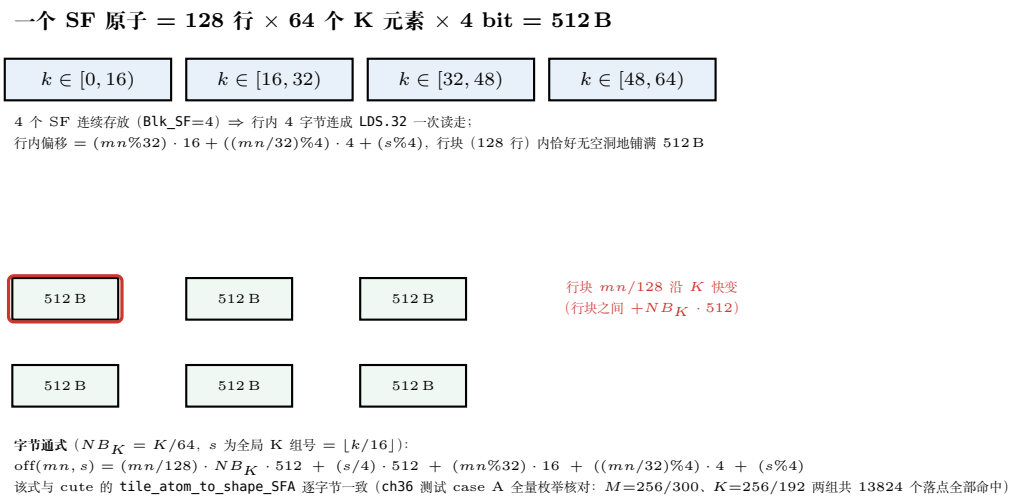


图 32.3: SF 的显存布局。每个 512 B 原子覆盖 128 行 × 64 个 K 元素（4 个 SF）。原子内部行优先：同一行的 4 个 SF 连续，故 SF 的 smem→ 寄存器只需一条 4 字节加载。矩阵级平铺时「行块」是快变维、K 方向次之，字节通式如框内所示。

fp4_gemm.cuh L128-148（SF 字节通式、缓冲尺寸与 canonical 布局）

```
// SF gmem 字节偏移（A 侧；B 侧把行号换成 n 即可，布局完全同构）：
//   off(mn, s) = (mn/128)*NB_K*512 + (s/4)*512 + (mn%32)*16 + ((mn/32)%4)*4 + (s%4)
// 其中 s 是全局 K 组号（= k/16），NB_K = K/64. 该式与 cute 的
// tile_atom_to_shape_SFA 逐字节一致（.tmp/fp4bs/probe_layout.cu 全量枚举验证）.
__host__ __device__ constexpr int sf_byte_offset(int mn, int s, int nb_k) {
  return (mn / kRowBlock) * nb_k * 512 + (s / kSFVecPerBlk) * 512 +
         (mn % 32) * 16 + ((mn / 32) % 4) * 4 + (s % kSFVecPerBlk);
}

// 一个 SF 缓冲的字节数（mn 按 128 行向上取整）
__host__ __device__ constexpr size_t sf_buffer_bytes(int mn, int K) {
  return (size_t)((mn + kRowBlock - 1) / kRowBlock) * (K / 64) * 512;
}

// SF gmem 张量的 canonical 布局（= CUTLASS Sm1xxBlockScaledConfig 的 SfAtom
// 平铺到 (mn, K)，行数自动向上取整到 128 的倍数）。K 方向 16 元素有一层
// 广播模式（16 个 K 元素共用一个 SF），因此该布局的坐标仍是数据单位.
CUTE_HOST_DEVICE auto sf_gmem_layout(int mn, int K) {
  using Config = cutlass::detail::Sm1xxBlockScaledConfig<kSFVec>;
  return tile_to_shape(typename Config::SfAtom{}, make_shape(mn, K), Step<_2, _1>{});
}
```

第 三十八 章提过 ffpa/SA3 用的 `BlockScaledConfig` 是 `Blk_MN=_64`（256 B 原子），与这里的 CUTLASS 口径（128 行 / 512 B）**不同**。两者都能正确工作，只是 SF 的落点公式不一样；本书统一采用 CUTLASS 口径，因为它是 `SM120_16x8x64_TN_VS` 这个 atom 的官方搭配（第 三十三 章会看到 SF 的 smem 原子也来自同一组式子）。

32.4　量化前处理 Kernel

32.4.1　常数与模式枚举

fp4_gemm.cuh L61-93（量化常数、scale 模式、向量化 IO 单元）

```
// e2m1（4-bit，1 符号 + 2 指数 + 1 尾数，共 16 个码点）：
//   可表示 0, ±0.5, ±1, ±1.5, ±2, ±3, ±4, ±6 —— 每 2 的幂区间只有 2 个点，
//   相邻码点间距 = 2^(e-1)，相对步长最大 25%（区间下沿），RMS ~10%.
// ue4m3（4-bit 无符号指数偏置 7）：可表示 2^-9 .. 448，步长 2^-9（次正规），
//   量程 [0, 448]，最大值 448（0x7E）；0x7F 是 NaN，必须避开.
// NVFP4 两级量化（level-2 是 NVFP4 相对 MXFP4 的定义性特征）：
//   level-2: 每 16 个 K 元素一个 SF = ue4m3(amax_group / 6)，组内数据
//            x̂ = round(x / SF) ∈ [-6, 6]（e2m1 满量程）
//   level-1: 矩阵级/行级/列级一个 fp32 scale，使 level-2 的 SF 不被压进
//            ue4m3 的次正规区间（SF < 2^-9 ≈ 0.00195 会 round 到 0 -> 整组归零）
// 折叠代数（与 fp8 同一恒等式）：
//   C = A·B^T = (sA·Â)·(sB·B̂)^T = sA·sB·(Â·B̂^T)，其中 Â 的 per-16 scale 由
//   MMA 硬件消费，sA/sB 在 epilogue 逐行/列乘回.
constexpr float kE2M1Max = 6.0f;        // e2m1 最大有限值（饱和上界）
constexpr float kUe4m3Max = 448.0f;     // ue4m3 最大有限值
constexpr float kFp4ScaleMax = kE2M1Max * kUe4m3Max;  // 2688 = 两级量程之积
constexpr int kSFVec = 16;              // 每个缩放因子覆盖的 K 元素数（scale_vec::4X）
constexpr int kRowBlock = 128;          // SF gmem 原子的行块（CUTLASS Blk_MN）
constexpr int kSFVecPerBlk = 4;         // 每 64 个 K 元素 4 个 SF（CUTLASS Blk_SF）

// 4 组合 scale 模式：level-1 的有无 x A/B 两侧
enum class Fp4GemmScaleMode {
  kSingleLevel,   // 只用 per-16 level-2（无 level-1，小幅度行会整组归零）
  kLevel1A,       // A 加 per-row level-1（scale 数 M）
  kLevel1B,       // B 加 per-col level-1（scale 数 N）
  kTwoLevel,      // A per-row + B per-col（精度最优，scale 数 M+N）
};

union Vec8BF16 {  // 8 x bf16 = 16B，量化 kernel 的向量化 IO 单元
  uint4 raw;
  __nv_bfloat16 elem[8];
};
static_assert(sizeof(Vec8BF16) == 16, "Vec8BF16 must be 128-bit");
```

四个模式（`Fp4GemmScaleMode`）镜像第 三十 章的 FP8 四粒度，但含义不完全一样：FP8 的四种是「δ 统计块的粒度组合」；FP4 的 `kSingleLevel` 与其余三种的差别是**有没有 level-1**，而 `kLevel1A`/`kLevel1B`/`kTwoLevel` 是 level-1 作用在哪一侧。为什么两侧的 level-1 可以独立开关？因为式 32.2 里 $s_{A,m}$ 与 $s_{B,n}$ 是相乘关系、互不耦合——这给了一个很有用的调试手段：把一侧关掉，就能单独观察那一侧的量化误差。

32.4.2　A 侧：两趟 pass

fp4_gemm.cuh L153-233（行 amax pass + A 侧量化 kernel）

```
// A 侧两条 pass（kLevel1 = true 时）：pass1 求行 amax 得到 sA[m] = rowmax/2688，
// pass2 每线程负责一个 16 元素组：x' = x / sA[m]，SF = ue4m3(amax(x')/6)，
// 8 字节 fp4 写回（一线程一组的映射让 32B 读与 8B 写都天然合并）.
__global__ void row_amax_fp4_kernel(const __nv_bfloat16 *__restrict__ A,
                                    float *__restrict__ sa, int M, int K) {
  constexpr int kVec = 8;
  const int row = blockIdx.x * 8 + threadIdx.x / 32;  // 8 warps/block
  if (row >= M) return;
  const int lane = threadIdx.x % 32;
  const __nv_bfloat16 *ar = A + (size_t)row * K;
  float amax = 0.0f;
```

```
  for (int c = lane * kVec; c < K; c += 32 * kVec) {
    Vec8BF16 v = *reinterpret_cast<const Vec8BF16 *>(ar + c);
#pragma unroll
    for (int e = 0; e < kVec; ++e)
      amax = fmaxf(amax, fabsf(__bfloat162float(v.elem[e])));
  }
#pragma unroll
  for (int off = 16; off > 0; off >>= 1)
    amax = fmaxf(amax, __shfl_xor_sync(0xffffffff, amax, off));
  if (lane == 0) sa[row] = amax / kFp4ScaleMax;  // 全零行 -> 0（epilogue 相乘即 0）
}

// A[M,K] bf16 -> A4 packed(M,K) + SFA（含 128 行向上取整的补零行）
// kLevel1: 是否先按 sA[m] 归一化（false = 纯 level-2）
template <bool kLevel1>
__global__ void quantize_a_fp4_kernel(const __nv_bfloat16 *__restrict__ A,
                                      cutlass::float_e2m1_t *__restrict__ a4,
                                      cutlass::float_ue4m3_t *__restrict__ sfa,
                                      const float *__restrict__ sa, int M,
                                      int K) {
  const int groups_per_row = K / kSFVec;
  const int g = blockIdx.x * blockDim.x + threadIdx.x;
  const int mpad = (M + kRowBlock - 1) / kRowBlock * kRowBlock;
  if (g >= mpad * groups_per_row) return;
  const int m = g / groups_per_row;   // 行（可能落在补零区）
  const int grp = g % groups_per_row;
  const int k0 = grp * kSFVec;
  const int nb_k = K / 64;

  if (m >= M) {  // 补零行：SF 显式写 0。TMA 只做 OOB 零填充，缓冲区里的
    // 未初始化字节会被当作 SF 乘进 MMA（0 x NaN = NaN），必须清零
    sfa[sf_byte_offset(m, grp, nb_k)] = cutlass::float_ue4m3_t::bitcast(0);
    return;
  }

  float inv1 = 1.0f;
  if constexpr (kLevel1) {  // 行级归一化：x' = x / sA[m]，sA[m] = rowmax/2688
    const float s = sa[m];
    inv1 = (s > 0.0f) ? (1.0f / s) : 0.0f;
  }

  const __nv_bfloat16 *ar = A + (size_t)m * K + k0;
  Vec8BF16 v0 = *reinterpret_cast<const Vec8BF16 *>(ar);
  Vec8BF16 v1 = *reinterpret_cast<const Vec8BF16 *>(ar + 8);
  float x[kSFVec];
  float amax = 0.0f;
#pragma unroll
  for (int e = 0; e < 8; ++e) {
    const float a = __bfloat162float(v0.elem[e]) * inv1;
    const float b = __bfloat162float(v1.elem[e]) * inv1;
    x[e] = a;
    x[e + 8] = b;
    amax = fmaxf(amax, fmaxf(fabsf(a), fabsf(b)));
  }

  const unsigned char sf_byte = cvt_f2_to_ue4m3(amax / kE2M1Max);
  const float sf_val = cvt_ue4m3_to_f32(sf_byte);
  // SF 下溢到 0（amax < 6 * 2^-10）-> 整组归零（inv2 = 0），避免 0/0 = NaN
  const float inv2 = (sf_val > 0.0f) ? (1.0f / sf_val) : 0.0f;
  sfa[sf_byte_offset(m, grp, nb_k)] = cutlass::float_ue4m3_t::bitcast(sf_byte);

```

```
  uint2 out;
  unsigned char *o = reinterpret_cast<unsigned char *>(&out);
#pragma unroll
  for (int e = 0; e < 8; ++e) {  // 8 个字节 = 16 个 4-bit
    o[e] = cvt_f2_to_e2m1x2(x[2 * e + 1] * inv2, x[2 * e] * inv2);
  }
  *reinterpret_cast<uint2 *>(reinterpret_cast<unsigned char *>(a4) +
                             (size_t)m * (K / 2) + grp * 8) = out;
}
```

逐点说明这段代码里三个不显然的地方：

- **线程映射**：一个线程负责 $(m, 16\text{-K}$ 组) 一个单位。K=256 时一行有 16 个组，一个 warp 覆盖 2 行——32 B 的向量读与 8 B 的写都天然满足合并访问（`Vec8BF16` 读两次拼 16 个元素，`uint2` 写一次落 16 个 nibble）。
- **补零行的 SF 必须显式写 0**。A4 缓冲按 128 行向上取整，超出 M 的行不存在数据。TMA 装载越界行时会自动零填充**数据**，但 **SF 缓冲区里的字节是量化 kernel 留下的**——若不清零，那些未初始化字节会被 MMA 当成 SF 乘进去（0×NaN = NaN），污染整条链。所以补零行要显式写 `bitcast(0)`。
- **SF 下溢到 0 时把 `inv2` 置 0**，让数据也归零，避免 0/0 产生 NaN。这与第 三十 章 amax 零块的处理同一思路，只是 NVFP4 多了一层（level-2 的 SF 本身可能舍入到 0）。

32.4.3　B 侧：列 amax 与转置量化分成两个 kernel

B 侧的量化要同时做三件事：转置（`B[K,N]` → `B4T[N,K]`）、level-1 的逐列 amax、level-2 的 per-16 分组量化。三件事里只有中间那件需要跨 K 的全局归约，另外两件都是「元素进、元素出」的局部操作——这个差别决定了 B 侧应该切成两个 kernel，而不是像第 三十 章的 FP8 版本那样合进同一个 kernel 的两个 pass。合并的写法（本书第一版就是这么写的）有两个缺陷：grid 只有 $N/128$ 个 CTA，N=4096 时 32 个 CTA 只填满 29% 的 SM；而且逐列归约读 smem 的步长是 128 个元素 = 64 个字 = bank 数 (32) 的整数倍，每读一次都是 32 路冲突。第 三十三 章会算这笔开销：那个版本的反推吞吐只有 194 GB/s，在线量化的端到端因此被限制在 1.2× cuBLAS（拆成两个 kernel 之后是 2.0×）。

列 amax 先执行：K 方向切 8 份，每个 block 拿到 $[K/8, 128]$ 的一块。每线程持 8 列、沿 K 步进 16（一次 `Vec8BF16` 载入正好拿满 8 列），块内再归约到 128 个列值，最后落到一次整数 `atomicMax`。

fp4_gemm.cuh L243-285（列 amax：分片 + 整数 atomicMax）

```
// 列 amax (level-1 用)：grid = (ceil(N/128), kSplit)，块内 16 条 k-lane x
// 16 组（每组 8 列）共 256 线程。每线程持 8 列、K 方向步进 16，一次 16B 载入
// 拿满 8 列；warp 内 2 行 x 128 列 = 4 条 128B 事务，正好吃满。
__global__ void col_amax_fp4_kernel(const __nv_bfloat16 *__restrict__ B,
                                    float *__restrict__ sb, int N, int K) {
  constexpr int kVec = 8;                 // 每线程 8 列 = 一次 16B 载入
  constexpr int kKLanes = 16;             // 块内 k 方向 lane 数
  constexpr int kBNt = 128;               // 每块 128 列
  constexpr int kNPerRow = kBNt / kVec;  // 16
  const int n0 = blockIdx.x * kBNt;
  const int kBeg = (int)((long long)K * blockIdx.y / gridDim.y);
  const int kEnd = (int)((long long)K * (blockIdx.y + 1) / gridDim.y);
  const int tid = threadIdx.x;
  const int nl = (tid % kNPerRow) * kVec;  // 列起点 0..120
  const int kl = tid / kNPerRow;           // k lane 0..15
  __shared__ float part[kKLanes][kBNt];

  float a[kVec];
#pragma unroll
  for (int e = 0; e < kVec; ++e) a[e] = 0.0f;
  if (n0 + nl < N) {
    for (int k = kBeg + kl; k < kEnd; k += kKLanes) {
      Vec8BF16 v =
          *reinterpret_cast<const Vec8BF16 *>(B + (size_t)k * N + n0 + nl);
#pragma unroll
```

```
      for (int e = 0; e < kVec; ++e)
        a[e] = fmaxf(a[e], fabsf(__bfloat162float(v.elem[e])));
    }
  }
#pragma unroll
  for (int e = 0; e < kVec; ++e) part[kl][nl + e] = a[e];
  __syncthreads();
  if (tid < kBNt && n0 + tid < N) {
    float m = 0.0f;
#pragma unroll
    for (int r = 0; r < kKLanes; ++r) m = fmaxf(m, part[r][tid]);
    // 先除以 2688 再 atomicMax：除以正常数保序，所以「先除后取 max」与旧版
    // 「先取 max 再除」逐位相同。amax >= 0，IEEE754 非负数的位模式与无符号
    // 整数序一致，可直接用整数 atomicMax（调用方需先把 sb 清零）。
    atomicMax(reinterpret_cast<unsigned *>(sb + n0 + tid),
              __float_as_uint(m / kFp4ScaleMax));
  }
}
```

`atomicMax` 本身只支持整数，这里用了一个小技巧：非负 IEEE754 浮点数的位模式与无符号整数序完全一致，所以对 $[0,+\infty)$ 的 amax 可以直接按 `unsigned` 比大小。更关键的是**先除后取 max** 这一步：除以正常数是保序映射，所以 $\max_i(\mathrm{amax}_i/2688) = (\max_i \mathrm{amax}_i)/2688$ 在浮点下**逐位成立**，于是拆成 8 片后得到的 s_B 与单块归约的旧版本 bit 级一致。这类「换个并行划分但一个 bit 都不许变」的约束，是量化 kernel 重构时最需要关注的对象。

第二段是纯局部的转置量化：一个 block 只做一个 $128(n) \times 64(k)$ 的 tile，per-16 组的 amax 完全落在块内，所以不需要任何跨块归约，grid 直接铺成 $(N/128) \times (K/64)$（4096^3 时 2048 个 CTA）。

fp4_gemm.cuh L287-365（转置量化：grid 映射与双朝向 staging）

```
// B 转置量化：B[K,N] bf16 -> B4T packed(N,K) + SFB。
// grid = (ceil(N/128), K/64)：一个 block 只做一个 128(n) x 64(k) 的 tile——
// 载入 -> smem 转置 -> 逐 16-K 组量化（组 amax 完全落在块内，无跨块归约）
// -> 8B 合并写 B4T（行 n、K 方向 8 个字节 = SFB 512B 原子里的 4 个字节）。
// 与 fp8 版本同构，区别是 level-2 的 per-16 SF 逐组产出并落到 512B SF 原子。
template <bool kLevel1>
__global__ void quantize_bt_tile_fp4_kernel(
    const __nv_bfloat16 *__restrict__ B,
    cutlass::float_e2m1_t *__restrict__ b4t,
    cutlass::float_ue4m3_t *__restrict__ sfb, const float *__restrict__ sb,
    int K, int N) {
  constexpr int kBNt = 128;                    // n 方向 tile（= B4T 行数）
  constexpr int kBKt = 64;                     // k 方向 tile（= B 行数）
  constexpr int kVec = 8;
  constexpr int kChunksPerRow = kBNt / kVec;   // 16
  constexpr int kChunks = kBKt * kChunksPerRow;
  const int n0 = blockIdx.x * kBNt;
  const int k0 = blockIdx.y * kBKt;
  const int tid = threadIdx.x;
  const int nb_k = K / 64;

  __shared__ __align__(16) __nv_bfloat16 tile[kBKt][kBNt];  // 源朝向 16KB
  // 转置朝向 16.6KB。多出来的 1 列是 padding：写 tile_t[c8+e][r] 时 e 方向
  // 的步长是整行，行宽 64 个字会让 16 个同 r 的线程全撞在同一个 bank 上。
  __shared__ __align__(16) __nv_bfloat16 tile_t[kBNt][kBKt + 1];

  for (int i = tid; i < kChunks; i += 256) {  // coalesced 载入 128(n) x 64(k)
    const int r = i / kChunksPerRow, c8 = (i % kChunksPerRow) * kVec;
    Vec8BF16 v;
    v.raw = make_uint4(0, 0, 0, 0);
    if (n0 + c8 < N)
      v =
          *reinterpret_cast<const Vec8BF16 *>(B + (size_t)(k0 + r) * N + n0 + c8);
```

```
    *reinterpret_cast<uint4 *>(&tile[r][c8]) = v.raw;
  }
  __syncthreads();
  for (int i = tid; i < kChunks; i += 256) {  // 散写转置朝向的 tile_t
    const int r = i / kChunksPerRow, c8 = (i % kChunksPerRow) * kVec;
#pragma unroll
    for (int e = 0; e < kVec; ++e) tile_t[c8 + e][r] = tile[r][c8 + e];
  }
  __syncthreads();
  // 每个 (n, 16-K 组) 一组: 128 n x 4 组 = 512 组 / 256 线程 = 2 组/线程
  for (int i = tid; i < kBNt * (kBKt / kSFVec); i += 256) {
    const int n = i / (kBKt / kSFVec), g = i % (kBKt / kSFVec);
    if (n0 + n >= N) {  // 补零行: SF 必须显式写 0 (与 A 侧同一条理由)
      sfb[sf_byte_offset(n0 + n, (k0 + g * kSFVec) / kSFVec, nb_k)] =
          cutlass::float_ue4m3_t::bitcast(0);
      continue;
    }
    float inv1 = 1.0f;
    if constexpr (kLevel1) {  // sb[n] = 列 amax / 2688, 由 amax kernel 产出
      const float s = sb[n0 + n];
      inv1 = (s > 0.0f) ? (1.0f / s) : 0.0f;
    }
    float x[kSFVec];
    float amax = 0.0f;
#pragma unroll
    for (int e = 0; e < kSFVec; ++e) {
      x[e] = __bfloat162float(tile_t[n][g * kSFVec + e]) * inv1;
      amax = fmaxf(amax, fabsf(x[e]));
    }
    const int s = (k0 + g * kSFVec) / kSFVec;  // 全局 K 组号
    const unsigned char sf_byte = cvt_f2_to_ue4m3(amax / kE2M1Max);
    const float sf_val = cvt_ue4m3_to_f32(sf_byte);
    // SF 下溢到 0 (amax < 6 * 2^-10) -> 整组归零 (inv2 = 0), 避免 0/0 = NaN
    const float inv2 = (sf_val > 0.0f) ? (1.0f / sf_val) : 0.0f;
    sfb[sf_byte_offset(n0 + n, s, nb_k)] =
        cutlass::float_ue4m3_t::bitcast(sf_byte);
    uint2 out;
    unsigned char *o = reinterpret_cast<unsigned char *>(&out);
#pragma unroll
    for (int e = 0; e < 8; ++e)
      o[e] = cvt_f2_to_e2m1x2(x[2 * e + 1] * inv2, x[2 * e] * inv2);
    *reinterpret_cast<uint2 *>(reinterpret_cast<unsigned char *>(b4t) +
                               (size_t)(n0 + n) * (K / 2) + k0 / 2 + g * 8) =
        out;
  }
}
```

转置朝向的 `tile_t` 多留了 1 列 padding。原因与第 三十三 章的 swizzle 话题同源：写 `tile_t[c8+e][r]` 时 e 方向的步长是一整行，行宽恰好 64 个 `bf16` = 32 个字，16 个同 r 的线程会全部撞在同一个 bank 上；补 1 列（行宽 65 个元素）后冲突降到 2 路。±1 列的 padding 是共享内存编程里代价最低的手段。

32.4.4　B 侧离线量化：部署的真实形态

与第 三十一 章 FP8 版本同构，B 侧的量化入口被抽成独立函数，全链路与离线两条路共用：

fp4_gemm.cuh L367-393（B 量化入口：全链路与离线共用）

```
// B 侧量化入口: 全链路与 B 离线量化共用。level1=false 时列级 sb 无意义
//  (inv1 恒为 1), 只跑转置量化; level1=true 时先跑列 amax (K 切成 8 份,
// N=4096 时 256 个 CTA) 再跑转置量化 (N=4096,K=4096 时 2048 个 CTA)。
inline void fp4_gemm_quantize_b(const __nv_bfloat16 *B,
```

```
                                cutlass::float_e2m1_t *b4t,
                                cutlass::float_ue4m3_t *sfb, float *sb, int N,
                                int K, bool level1, cudaStream_t stream) {
  if (K % 64 != 0 || N % 8 != 0) {
    fprintf(stderr,
            "fp4_gemm_quantize_b: require K%%64==0 (got K=%d), N%%8==0 (got "
            "N=%d)\n",
            K, N);
    return;
  }
  const dim3 grid((N + 127) / 128, K / 64);
  if (level1) {
    cudaMemsetAsync(sb, 0, (size_t)N * sizeof(float), stream);
    const int kSplit = (K >= 512) ? 8 : 1;
    const dim3 grid_sa((N + 127) / 128, kSplit);
    col_amax_fp4_kernel<<<grid_sa, 256, 0, stream>>>(B, sb, N, K);
    quantize_bt_tile_fp4_kernel<true>
        <<<grid, 256, 0, stream>>>(B, b4t, sfb, sb, K, N);
  } else {
    quantize_bt_tile_fp4_kernel<false>
        <<<grid, 256, 0, stream>>>(B, b4t, sfb, sb, K, N);
  }
}
```

差别只在产物多一份：FP8 的 B 离线产物是 $(B_8^\top, \delta_b)$；NVFP4 的是 $(B_4^\top, \mathrm{SFB}, s_B)$ ——**多一份 N×K/16 的 ue4m3 SFB**。这份 SFB 是常驻显存的开销：以 $K{=}N{=}4096$ 计，B4T 占 $4096^2/2 = 8\,\mathrm{MB}$，SFB 占 $4096 \times 256 = 1\,\mathrm{MB}$，即权重的显存占用为 BF16 的 $9/32 \approx 28\%$（含 SF 的压缩率 3.6×，纯数据是 4×）。

32.5　误差模型：从量化步长到 14.4%

这是本章的核心一节。NVFP4 的误差不能沿用第 三十 章「相对步长恒定 2^{-3}」的模型——e2m1 的步长随量级变化，必须逐格算。

32.5.1　逐格累加：把 8 个格的误差贡献加起来

设组内数据已经除过 level-2 的 SF，得到 $y \in [-6, 6]$。当组内 16 个元素的 amax 恰为 6 时（归一化的目的），y 在 $[-6, 6]$ 上近似均匀。e2m1 的量化误差就是 y 落在各格内、相对该格码点的偏差。对格 k（宽度 w_k、码点 c_k），贡献为

$$\mathrm{E}\big[\Delta y^2\big] = \sum_{k=0}^{7} \frac{w_k}{6} \cdot \underbrace{\frac{1}{w_k} \int_{\text{格}_k} (y - c_k)^2 \,\mathrm{d}y}_{\text{格内二阶矩 } v_k} \tag{32.3}$$

关键在于 v_k 的两个形态。若码点在格心，$v_k = w_k^2/12$（均匀分布方差）；若码点在格**边**上（最外侧两格），$v_k = w_k^2/3$ ——是前者的 **4 倍**。逐格算出来：

格	码点	步长 w_k	格概率	格内二阶矩 v_k	贡献 $\frac{w_k}{6} v_k$
$[0, 0.25]$	0	0.25	0.0417	0.02083	0.00087
$[0.25, 0.75]$	0.5	0.50	0.0833	0.02083	0.00174
$[0.75, 1.25]$	1.0	0.50	0.0833	0.02083	0.00174
$[1.25, 1.75]$	1.5	0.50	0.0833	0.02083	0.00174
$[1.75, 2.5]$	2.0	0.75	0.1250	0.06250	0.00781
$[2.5, 3.5]$	3.0	1.00	0.1667	0.08333	0.01389
$[3.5, 5.0]$	4.0	1.50	0.2500	0.25000	0.06250
$[5.0, 6.0]$	6.0	1.00	0.1667	0.33333	0.05556
			正负对称，$\mathrm{E}[\Delta y^2] = 2\sum_k \frac{w_k}{12} v_k =$		**0.14583**

y 在 $[-6, 6]$ 均匀时 $\mathrm{E}[y^2] = \int_{-6}^{6} y^2/12 \,\mathrm{d}y = 12$，于是

$$\frac{\sqrt{\mathrm{E}[\Delta y^2]}}{\sqrt{\mathrm{E}[y^2]}} = \sqrt{\frac{0.14583}{12}} = 11.02\%. \tag{32.4}$$

误差最大的两格贡献了总误差的 81%（$0.06250 + 0.05556 = 0.11806$，占总和 0.14583 的 81%——注意除了最外侧两格，$[3.5, 5.0]$ 与 $[1.75, 2.5]$ 也偏离格心）。这是 NVFP4 误差分析里最容易被忽略的一项：只看「相对步长 2^{-3}」会算出 3.6%（那是 e4m3 的答案），实际是 11%。

32.5.2　三项修正：max 归一、SF 舍入、双侧求和

式 32.4 还是理想化的。真实通路有三处偏离，用蒙特卡洛逐项核对（脚本落在 `.tmp/book-bench/ch36/error_model.txt`）：

环节	说明	单侧相对 RMS
逐格累加	y 严格 $U(-6, 6)$	11.02%
组内 max 归一	16 元中 1 个精确落在 ±6（零误差），15 个 $U(-6, 6)$：$\mathrm{E}[\Delta y^2]$ 乘 15/16、$\mathrm{E}[y^2]$ 变成 $(36 + 15 \times 12)/16 = 13.5$	10.06%
+ ue4m3 SF 舍入	SF 相对误差 RMS 3.24%（尾数 3 bit），且会把组内最大值推过 6 触发 satfinite	10.17%
+ 双侧（GEMM 里 A、B 都量化）	两侧独立同分布，分子加倍的近似式为 $\sqrt{2} \times 10.17\%$	**14.38%**

三个对照点都与实测一致：

- **量化 round-trip 实测** 10.29%（`.tmp/fp4bs/smoke.cu` 的 O.1b，128×256 与 300×192 两组分别 10.29%/10.20%）vs 模型 10.17%——差 1.2%，来自 fp32 归一乘除的舍入。
- **GEMM relFro 实测** 14.37%（单级、512^3，`--fp4-gemm` 正确性测试）vs 模型 14.38%——**小数点后两位吻合**。
- **实测对 level-1 几乎不敏感**：单级 14.37%、A 侧 level-1 14.48%、B 侧 14.43%、两级 14.54%。这不是「level-1 没用」，而是**均匀随机数据里没有小幅度行**——level-1 的作用要到幅度分布不均匀时才显现（下一小节）。

饱和项的符号是单向的　组内最大值那个元素，若 SF 向下舍入（编码值小于 amax/6），则 $x/\mathrm{SF} > 6$ 被 satfinite 压回 6，元素被**系统性压低**；若 SF 向上舍入，则该元素误差正常。所以 NVFP4 的误差里有一个小的单边偏置项，它的量级是 $|\Delta\mathrm{SF}/\mathrm{SF}| \approx 3.2\%$ 的幅度、但只作用在每组的一个元素上，折进 Frobenius 口径后为 $O(1\%)$——这也是 10.06% 升到 10.17%（+1.1%）的原因。教学上的意义：**NVFP4 的饱和不是「溢出保护」，而是一个常态发生的量化事件**。

32.5.3　level-1 真正解决的问题

把数据换成「一行大幅度、一行小幅度」的混合矩阵，问题立刻显现。取 $|x| \le 10^{-4}$ 的小幅度矩阵：

模式	现象	relFro
单级	组内 amax $\approx 9.4 \times 10^{-5} \Rightarrow \mathrm{SF} = \mathrm{ue4m3}(\mathrm{amax}/6) = \mathrm{ue4m3}(1.57 \times 10^{-5})$ 舍到 **0** $\Rightarrow$ **8192 个组全部归零**	**1.0000**
两级	$s_A[m]$ = 行 max/2688 $\approx 3.7 \times 10^{-8} \Rightarrow$ 组内 y 回到 $[-6, 6]$，level-2 的 SF 均值 423.6（量程上部）	0.1016

单级在这里的 relFro 恰好是 1.0——**输出全零，相对误差 100%**。两级的 relFro 是 0.1016，与随机数据的 10.17% 一致：level-1 把任何量级的数据都归一到 e2m1 的「最佳工作点」，这就是它的全部价值。实测里单级与两级只差 0.2%（14.37% vs 14.54%）纯粹是因为测试数据 $U(-1, 1)$ 里不存在小幅度行。

与 FP8 的量级对照　把本章的数字与第 三十 章并排放：

	数值位	单元素相对 RMS	GEMM relFro（实测）
bf16（参照）	8	$2^{-9}/\sqrt{3} \approx 0.11\%$	—
e4m3（第 三十 章）	3	3.6%	3.6%
e2m1+ue4m3（本章）	1(+3)	10.2%（单侧）	14.4%

比特数从 8 降到 4（减半），误差从 3.6% 升到 14.4%（4 倍）——**精度代价远高于比特数的线性缩放**，因为 e2m1 的每区间码点数从 8 降到 2。这正是 NVFP4 必须配合更精细的量化算法（两级 scale、旋转、平滑，以及第 四十 章讨论的各种校正）才能用于生产的原因；而由此换来的吞吐收益在第 三十三 章度量。

32.6 精度实测

`--fp4-gemm` 四条组合 + WS + 尾部 shape 的结果（CPU fp64 参考，512^3；列是 relFro）：

配置	relFro
单级（只有 level-2）	0.1437
level-2 × A 行	0.1448
level-2 × B 列	0.1443
两级（A 行 × B 列）	0.1454
两级 ws	0.1454
两级尾部 shape（M=300, N=264, K=192）	0.1454
两级尾部 ws	0.1454

三点读法：其一，四种组合的差异在 1% 以内，与误差模型一致（$U(-1,1)$ 数据下 level-1 无从发挥作用）；其二，WS 变体与 non-WS **数值完全一致**——第 三十三 章会说明这是设计使然（两条路走同一套量化产物与同一段 epilogue 数学）；其三，尾部 shape 与齐整 shape 一致，说明 M/N/K 的尾块处理（TMA 零填充 + store 裁剪 + K%64 约束）没有引入额外误差。

`book/tests/ch36_fp4_quantize.cu` 把上面的分析逐条落实为断言：SF 落点通式与 cute 权威布局全量枚举比对（13824 个落点）、A/B 两侧的 SF 与数据码点**逐 bit** 比对、补零行 SF 必须为 0、硬件平局取偶与 satfinite 的行为，以及上面两个小幅度行的对照数值。这些断言的通过意味着第 三十三 章可以把全部注意力放在「MMA 与流水」上——数值语义已经冻结。

32.7 小结

- NVFP4 = e2m1（4-bit 数据）+ ue4m3（4-bit level-2 scale，每 16 个 K 元素一个）+ fp32（level-1 scale，行/列级）。三个部件各自的位宽与量程决定了这套格式的能力边界。
- e2m1 的步长**随量级翻倍**，最外侧两格的码点落在格边上（格内二阶矩 $w^2/3$）。这两件事加起来把单侧误差推高到 $\sqrt{0.14583/12} = 11.02\%$；计入 max 归一与 ue4m3 SF 舍入后 10.2%；GEMM 双侧 $\sqrt{2}\times$ 得 **14.4**%，与 kernel 实测 14.37% 吻合。
- level-1 的唯一目的是把 level-2 的 SF 移出 ue4m3 的次正规区。幅度小于 6×2^{-10} 的组在单级模式下会**整组归零**（相对误差 100%），实测复现。所以生产配置必须带 level-1。
- SF 在显存里是 128 行 × 4 SF 的 512 B 连续原子，字节通式与 CUTLASS 的 `tile_atom_to_shape_SFA` 逐字节一致。这个布局是第 三十三 章 TMA 与 blockscaled MMA 能直接适配的前提。
- 折叠恒等式与 FP8 同形：level-2 由 MMA 硬件消费（软件零成本），level-1 在 epilogue 按 (m, n) 查表乘回。第 三十三 章的 epilogue 因此比 FP8 还简单——只有一项。

延伸阅读

1. NVIDIA, *NVFP4 量化格式与 `mma.sync` block scale 语义*（PTX ISA §9.7.14 `block_scale` 一节，本机路径 `.github/skills/cuda-cpp-kernel/references/ptx-docs`）。
2. CUTLASS `include/cutlass/detail/sm100_blockscaled_layout.hpp` 与 `include/cute/arch/mma_sm120.hpp`（`SM120_16x8x64_TN_VS` 的 SF 片段布局权威定义）。
3. 知乎 @DefTruth《WINT8/4》系列（低比特 GEMM 的 scale 粒度与误差分析，与第 32.5 节的逐格累加误差模型互为印证）。

第三十三章　FP4 GEMM（二）：blockscaled MMA、SF 流水与 cuBLAS 对比

33.1　本章导读

第 三十二 章把 NVFP4 的数值语义冻结了：e2m1 的格、ue4m3 的 SF、512 B 原子布局、14.4% 的误差界，以及两侧量化 kernel 的逐 bit 行为。本章写主 GEMM kernel——把 $A_4, B_4^\top$ 与两组 SF 送入 sm_120 的 **blockscaled `mma.sync`**，由硬件消费 level-2 的 scale。

整章可以概括为：**FP4 GEMM 的主 kernel 比 FP8 的更简单，也更难写**。简单是因为 epilogue 只剩一个乘法（SF 由 MMA 硬件消费，不需要按块查表），难是因为多出来一条 SF 数据流——4 条 TMA、两套 smem 原子、一个把数据与 SF 成对的 zip 张量，以及一类 FP8 版本里不存在的死锁陷阱（37.7 节）。

本章的构件全部来自前文：TMA descriptor（第 二十四 章）、SW32 smem swizzle（`Layout_K_SW32_Atom`，第 二十四 章）、mbarrier 多级流水（第 二十八 章）、全员 MMA 的 non-WS 协议（第 二十九 章）、warp specialization 与 `setmaxnreg`（第 二十八 章）、epilogue 的 STSM + TMA store（第 二十八 章）。新东西只有三件：**block scale 形式的 `mma.sync`**（37.3.1 节）、**SF 的 smem 原子与 TMA descriptor**（37.4 节）、以及 **数据与 SF 的成对装载**（37.5 节的 zip 张量）。

维度	第 三十一 章 FP8	本章 NVFP4
数据元素	e4m3（8 bit）	e2m1（4 bit）
MMA 指令	`SM89_16x8x32`（K=32）	`SM120_16x8x64_TN_VS`（K=64）
scale 的消费方式	epilogue 里两次 fp32 查表乘	level-2 由 MMA 硬件乘；level-1 在 epilogue 乘
每 stage 的 A/B smem	BK=128：16 KB + BN·128 B	BK=64：4 KB + BN·64/2 B
每 stage 的 SF smem	无	512 B × 2（SFA + SFB）
TMA 通道	2（A、B）	4（A、B、SFA、SFB）
O staging 约束	≤ 整个 A+B 区	≤ 整个 A+B+SF 区
BK 的自由度	可调（SW128 限定为 128）	**固定 64**（= MMA 原子 K 宽 + 一个 SF 原子）
BM 的自由度	可调（16 的倍数）	**固定 128**（= SF 原子的行块）
epilogue 的数学	反量化两步	只有 $s_{A,m} \cdot s_{B,n}$ 一步

表 33.1: FP8 与 NVFP4 主 kernel 的逐项对照。两个固定值是本章的技术核心：BK=64 让「一个 K-tile = 一个 MMA 原子 = 一个 SF 原子」，BM=128 让 SF 的 TMA box 恰好等于一个 512 B 原子（37.4.3 节）。自由度换来的是约束，代价是 tile 空间的形状从「$BM \times BN \times BK$ 立方体」收缩成「$128 \times BN \times 64$ 的一根柱子」。

前置章节：第 三十二 章（NVFP4 线格式与量化，必须）、第 二十四 章（TMA/swizzle）、第 二十九 章（non-WS 协议）、第 二十八 章（WS 与 `setmaxnreg`）、第 三十一 章（本章的骨架与它逐节对应，可对照阅读）。**源码区间**：`fp4_gemm.cuh` L395–1297（Phase 10.3–10.8），`bench/fp4_gemm.cu` 的 `bench_fp4_gemm/` `bench_fp4_gemm_tile_sweep` 接线。**验证方式**：`leetcuda_bench_sm120a.bin --fp4-gemm`（精度全组合）、`--bench --mnk 8192,8192,8192`（标准表）、`--fp4-gemm-sweep`（tile/流水深度扫描）、`book/tests/ch37_fp4_gemm.cu`（17 条断言的单文件复现）。

33.2　总体架构

本章要写的整条链画在图 33.1 里：

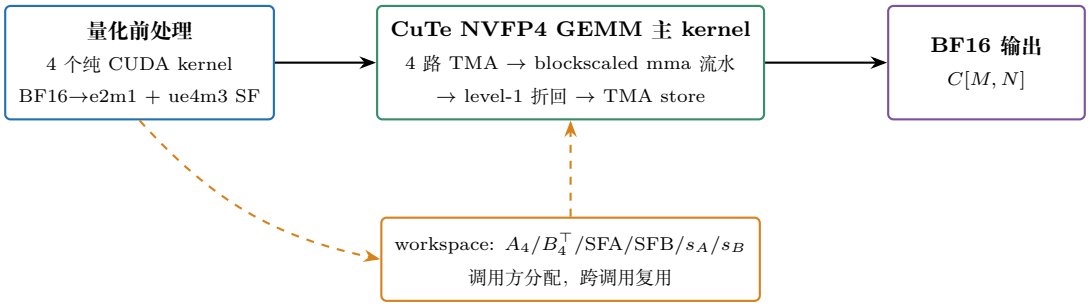


图 33.1: `fp4_gemm_bf16` 全链路。与第 三十一 章（图 31.1）同构，唯一的结构差异是 workspace 里多出两份 SF：A 侧的 SFA 是 $\lceil M/128\rceil\cdot(K/64)$ 个 512 B 原子，B 侧的 SFB 同理。SF 不是「辅助数据」，它是 MMA 指令的操作数——主 kernel 必须为它单独开一条 TMA 通道、一块 smem、一个寄存器 fragment（37.4 与 37.5 节）。

主 kernel 的输入输出约定与第 三十一 章的 FP8 版本完全一致：$C[M,N] = A_4[M,K]\cdot B_4^{\top}[N,K]$，三者行主序、4-bit/bf16 混搭。「$B^{\top}$ 行主序」同样是量化阶段完成的（36.4.3 节）。约束比 fp8 更紧：$K\%64=0$（SF 以 64 个 K 元素为一整块）与 $N\%8=0$（TMA 对齐），M 任意（TMA load 自动零填充、store 自动裁剪）。

33.3 指令与 Traits

33.3.1 block scale 形式的 `mma.sync`

sm_120 上 NVFP4 的矩阵乘是一条带 block scale 的 `mma.sync`（PTX ISA §9.7.14.3 的 `.kind::mxf4nvf4` 家族，本机文档在 `.github/skills/cuda-cpp-kernel/references/ptx-docs/9-instruction-set/`；CUTLASS 把它包成 `SM120_16x8x64_TN_VS` 这个 atom。PTX 形态：

```
mma.sync.aligned.m16n8k64.row.col.kind::mxf4nvf4.block_scale.scale_vec::4X
    .f32.e2m1.e2m1.f32.ue4m3
    {d0,d1,d2,d3},              // f32 x 4  = 16x8 的输出（每线程 4 个）
    {a0,a1,a2,a3},              // b32 x 4  = 16 字节 = 32 个 e2m1
    {b0,b1},                    // b32 x 2  = 8 字节  = 16 个 e2m1
    {c0,c1,c2,c3},              // f32 x 4
    scaleAData, {bidA, tidA},   // 每个 scale 是 1 个 b32（4 字节 = 4 个 SF）
    scaleBData, {bidB, tidB};
```

三个必须读懂的细节：

第一，`.scale_vec::4X` 是强制的。PTX 文档（§9.7.14.5，Table 36）规定：`.kind::mxf4nvf4` 搭配 `.stype = .ue4m3` 时，`.scale_vec_size` 只能是 `.scale_vec::4X`——这正是 NVFP4 的线格式：A 的每一行被切成 `SFA_N`= 4 段（K=64 除以 16），每段一个 SF。文档里那句「for `.kind::mxf4nvf4`, it is mandatory to provide valid `.scale_vec_size`」翻译成工程语言就是：这条指令必须提供合法的 *scale*，SF 是操作数而不是可选项。

第二，SF 也是寄存器操作数，而且每个 SF 只由 4 个线程里的 1 个提供。`scaleAData`/`scaleBData` 各是一个 `b32`（4 字节 = 4 个 `.scale_vec::4X` 的 SF），`{bidA,tidA}` 与 `{bidB,tidB}` 是选择器：`tidA` 选 quad 里的哪一对线程提供 A 的 SF（0 取 `laneid%4`$\in\{0,1\}$，1 取 $\{2,3\}$），`tidB` 选 quad 里的哪一个线程提供 B 的 SF。这就是「一个 32 位的寄存器装 4 个 SF，但硬件只从特定 lane 取值」的全部含义。对 kernel 作者的实际含义是：**SF 在寄存器里的 fragment 布局与数据完全不同**，不能把 SF 当成「每线程正好 4 个」这么简单——它在每条 lane 上装的东西由 `tidA`/`tidB` 决定。第 三十二 章对此的总结是：SF 的 gmem 布局、smem 布局、寄存器 fragment 布局是三套不同的映射。

第三，K=64 是「四个 SF」的另一种说法。一条指令处理 $16\times8\times64$ 的乘积（图 33.2 把它的操作数明细画了出来），A 侧的 SF 需求量是 $16\times4=64$ 个字节、B 侧是 $4\times8=32$ 个字节；而 A 的操作数只有 16 字节（32 个 e2m1）——**SF 的操作数字节数比数据还多。**这解释了为什么 SF 必须走自己的一条 TMA 通道与一块 smem：它是数据量的 1/8（每 16 个 4-bit = 8 字节配 1 字节 SF），却与数据同等重要。

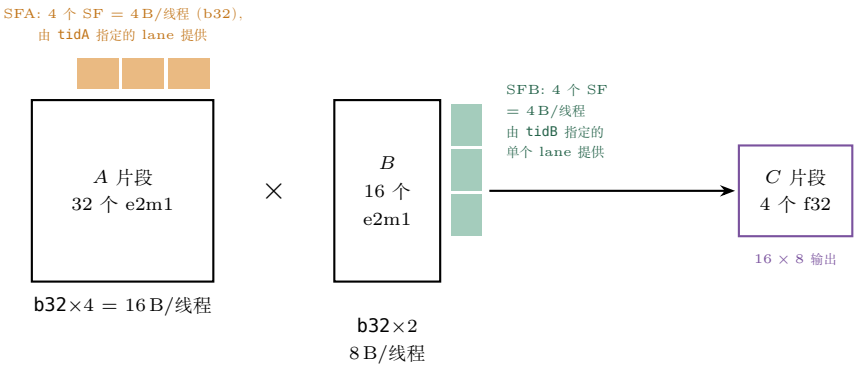


图 33.2: `SM120_16x8x64_TN_VS` 一条指令的操作数明细。数据侧共 16 + 8 = 24 B/线程，SF 侧 8 B/线程——SF 的字节数达到数据的 1/3。注意 SF 的「字节数/线程」与「SF 数/线程」是一回事（`ue4m3` 是 1 字节），但**哪些 lane 的字节作数**由指令的 `tidA`/`tidB` 选择器决定，这就是 SF 的寄存器 fragment 必须由 CuTe 的 `partition_fragment_sfa` 从权威布局推导、而不能手写的原因。

33.3.2　Traits：两个被固定的参数

fp4_gemm.cuh L554-584（Fp4GemmTraits：tile 几何与 static_assert）

```cpp
template <int kBN_ = 128, int kStages_ = 4>
struct Fp4GemmTraits {
  static constexpr int kBM = 128;
  static constexpr int kBN = kBN_;
  static constexpr int kBK = 64;      // = atom K：一个 stage 正好一跳
  static constexpr int kStages = kStages_;
  static constexpr int kNumThreads = kBM / 16 * 32;  // 8 warps = AtomLayout(4,2,1)
  static_assert(kBN % 128 == 0,
                "kBN 必须是 128 的倍数：SF 的 smem 原子以 128 行为一个行块，"
                "且 SF 的 TMA box 必须与该原子顶层 size 相等（BN<128 时 box "
                "填不满一个行块，直接编译期报 TMA size 不等价）");
  static_assert(kStages >= 2, "kStages >= 2（两级流水才有 overlap）");

  using Element = cutlass::float_e2m1_t;      // 4-bit 数据
  using ElementSF = cutlass::float_ue4m3_t;   // 8-bit SF
  using ElementO = cutlass::bfloat16_t;
  using ElementAcc = float;

  // MXF4_NVF4 路径（A/B 均 4-bit 且 K-major）选 k64 + uint4_t smem 分配类型
  using MmaAtom = SM120::BLOCKSCALED::SM120_16x8x64_TN_VS<Element, Element,
                                                          ElementAcc, ElementSF,
                                                          kSFVec>;
  using AtomLayoutMNK = Layout<Shape<_4, _2, _1>>;
  using PermTileM = Int<kBM>;                       // min(BM,128)
  using PermTileN = Layout<Shape<_8, _2, _2>, Stride<_1, _16, _8>>;  // 32
  using PermTileK = _64;
  using TiledMma = decltype(make_tiled_mma(MmaAtom{}, AtomLayoutMNK{},
                                           Tile<PermTileM, PermTileN, PermTileK>{}));
  static_assert(size<0>(typename TiledMma::AtomShape_MNK{}) == 16 &&
                    size<2>(typename TiledMma::AtomShape_MNK{}) == kBK,
                "atom 必须是 16x8x64（K 与 BK 同宽，一跳一 stage）");
```

与第 三十一 章 FP8 的 `Fp8GemmTraits`（BM, BN, BK 全开放）相比，这里只剩 BN 一个自由度，另两个被两条**物理约束**固定：

- $BK \equiv 64$：等于 MMA 原子的 K 宽。FP8 的 BK=128 是「K 方向切 4 个 k-slice」（原子 K=32），BK 加大只是让 k-slice 变多；而 FP4 的 SF 语义要求 K 以 16 为一组，`.scale_vec::4X` 要求一条指令恰好消费 4 组——即 K=64。BK 一旦大于 64，同一个 stage 里就会出现多个「SF 块」，SF 的 smem 原子与 TMA box 的对齐关

系立即被破坏。所以 **FP4 的 K-tile 只能等于 MMA 的 K 宽**：一个 K-tile 一步 MMA，没有 k-slice 内层循环（37.5 节）。

- $BM \equiv 128$：等于 SF 原子的行块（`Blk_MN`）。SF 的 TMA box 必须与 smem 原子的顶层 size 相等，$BM < 128$ 时 box 填不满一个行块，CuTe 在编译期直接报 TMA size 不等价——这就是源码里那条 `static_assert` 表达的约束（37.4.3 节会完整推导这条约束）。
- BN 必须是 128 的倍数，理由与 BM 相同但方向在 N：B 侧的 SF 原子行块也是 128，BN 是 128 的倍数才能让 SFB 的 TMA box 由整数个原子拼成。

于是 tile 空间被固定成两个常数维度加一个自由维度：$128 \times BN \times 64$，BN 成了唯一可扫的旋钮——37.9.4 节的扫描表明，把 BN 从 128 翻到 256 是本章最大的单项性能发现（读数见该节）。

33.4 SF 的 smem 原子与四路 TMA

第 三十二 章讲了 SF 在显存里是 512 B 的原子（128 行 × 4 个 SF）。本章要回答下一个问题：**这个原子怎么进 smem，又怎么变成寄存器 fragment？**

33.4.1 smem 原子的四段式子

CUTLASS 给出了 smem 版本的原子，本书把它写成 `SfSmemAtom`。源头在

`sm120/blockscaled/mma_builder.inl`

里给 TMA 用的那段 `make_tiled_copy_sfa/sfb`：

fp4_gemm.cuh L527-547（SfSmemAtom：四个 shape/stride 三元组）

```
template <class TiledMma_, int kMN, int kBK_>
struct SfSmemAtom {
  static constexpr int kMMA_NSF =
      size<2>(typename TiledMma_::AtomShape_MNK{}) / kSFVec;  // 64/16 = 4
  using Config = cutlass::detail::Sm1xxBlockScaledConfig<kSFVec>;
  using Blk_Elems = decltype(typename Config::Blk_MN{} * typename Config::Blk_SF{});
  using mnBlockShape = Shape<_32, _4>;
  using mnBlockStride = Stride<_16, _4>;
  using kBlockShape = Shape<Int<kSFVec>, Int<kMMA_NSF>>;
  using kBlockStride = Stride<_0, _1>;
  using shapeM = decltype(prepend(Int<kMN / kRowBlock>{}, mnBlockShape{}));
  using strideM = decltype(prepend(Blk_Elems{}, mnBlockStride{}));
  using shapeK = decltype(prepend(
      make_shape(Int<kSFVecPerBlk / kMMA_NSF>{},
                 Int<kBK_ / kSFVec / kSFVecPerBlk>{}),
      kBlockShape{}));
  using strideK = decltype(prepend(
      make_stride(Int<kMMA_NSF>{}, Int<kMN / kRowBlock * 512>{}), kBlockStride{}));
  using type = decltype(make_layout(make_shape(shapeM{}, shapeK{}),
                                    make_stride(strideM{}, strideK{})));
};
```

这段代码初看不易理解：四个 `shape/stride` 三元组、一个 `prepend`、一个把 kMN 包进 `Int<kMN/128>` 的外层。把它拆成「gmem 的 512 B 原子是什么结构」与「smem 需要什么结构」两句话，结构就清楚了。

gmem 原子（第 三十二 章「SF 的 gmem 布局」小节，图 32.3）的行内偏移是 $16\,(m \bmod 32) + 4\,(\lfloor m/32 \rfloor \bmod 4) + s$。把它读成一个嵌套布局：行方向 $((32,4):(16,4))$——先铺 32 行、步长 16，再铺 4 组、步长 4；K 方向 $(16,4):(0,1)$——16 个元素共享一个 SF（步长 0，这正是 NVFP4 的「16 元素共享 scale」在线格式上的体现），4 个 K 组步长 1。这正是源码里 `mnBlockShape/kBlockShape` 两行。

smem 原子要满足的是 MMA 的取数需求：一条指令里，一个 warp 的每条 lane 要一次拿 4 个连续字节（4 个 SF）做 `b32` 操作数。于是 `SfSmemAtom` 就是在同一个 512 B 里换一种字节排序，把「4 个 SF 连续」放到对 lane 友好的位置上。

33.4.2　一条可验证的结论：smem 原子与 gmem 原子逐字节同序

「换一种排序」听起来像是有 swizzle，实际**不是**（图 33.3）。把两套式子写成

$$\mathrm{off}_{\mathrm{gmem}}(m,s) = 16\,(m \bmod 32) + 4\,(\lfloor m/32 \rfloor \bmod 4) + s, \tag{33.1}$$

$$\mathrm{off}_{\mathrm{smem}}(m,k) = 16\,(m \bmod 32) + 4\,(\lfloor m/32 \rfloor \bmod 4) + \lfloor k/16 \rfloor, \tag{33.2}$$

其中第二式的 k 是**数据 K 索引**（16 个共享一个 SF），第一式的 s 是 **SF 索引**。因为 $s = \lfloor k/16 \rfloor$，两式完全相等。

本书用 `.tmp/fp4bs/probe_sf_atom.cu` 把这件事枚举验证了一遍：$128 \times 4 = 512$ 个 $(\text{行}, \text{SF})$ 落点**全部一致**（mismatch $= 0$）。这条结论澄清了 SF 数据通路上最不直观的一段：

- **SF 的 TMA 不需要 swizzle**：gmem 里 512 B 连续的一组字节，在 smem 里还是那 512 B、同样顺序。数据侧（A/B）需要 SW32 swizzle 是因为 4-bit 数据的 bank 冲突，而 SF 的 512 B 原子本身就是为「一次 32-bit LDS 取 4 个 SF」设计的，天然无冲突。
- **「四段式子」只是同一映射的另一种写法**：它描述的是「smem 坐标 $(\text{行}, K)$ 怎么落到字节」这个方向。CuTe 需要的正是这个方向（给坐标求偏移），所以代码里是 layout 而不是公式。
- **cosize 与 size 分离**：这个 layout 的 `size`（逻辑元素数）是 $128 \times 64 = 8192$，`cosize`（实际占用字节）只有 512——因为 K 方向有 16:0 的广播维。smem 预算要按 `cosize` 算（每 stage 512 B），这一点在 `kSFAStageBytes = cosize(...)/kStages` 这行里体现。

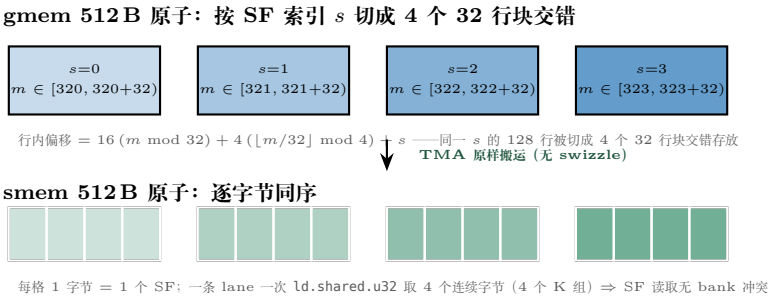


图 33.3: SF 从显存到共享内存的完整通路：**没有任何变换**。512 B 原子里的行内交错（同一 s 的 128 行被切成 4 个 32 行块）本身就是「为 MMA 取数设计」的顺序，smem 只是它的一次原样拷贝。这是 CUTLASS 把 `Blk_MN` 定为 128 的原因，也是本章 kernel 里 SF 的 TMA 描述符不带 swizzle 的原因。

33.4.3　TMA box = 一个 SF 原子：两个固定值的来源

有了上面的结论，BM=128 与 BK=64 的来历就很直接了：SF 的 TMA box 是 Shape$\langle BM, BK \rangle$，而一个 SF 原子覆盖 128 行 × 64 个 K 元素。box 必须恰好等于原子的顶层 size——多一个元素少一个元素都会让 CuTe 在 `make_tma_copy` 里报错，或者让 TMA 写出错误字节数。所以：

$$BM = 128, \qquad BK = 64 \iff \text{TMA box} = \text{一个 512 B SF 原子}.$$

这不是「选了一个好数字」，而是「唯一的合法组合」。$BM > 128$ 时 box 要跨两个原子（TMA 的 box 是矩形，跨原子意味着 K 方向必须同时前进，做不到）；$BK > 64$ 同理（一个 box 里出现第二个 512 B 原子而 box 的行宽还是 128）。第 三十二 章提到 ffpa/SA3 用的是 `Blk_MN=_64`（256 B 原子）的另一种口径——那种口径下 BM 会被固定在 64。本书选 CUTLASS 口径，是因为它是 `SM120_16x8x64_TN_VS` 的官方搭配。

四路 TMA 的发射代码：

fp4_gemm.cuh L767-783（issue_tma：一个 barrier 覆盖四段字节）

```
constexpr uint32_t kTxBytes = Traits::kTxBytes;
auto issue_tma = [&](int kt, int stage) {
  cutlass::arch::fence_view_async_shared();
  auto gA = local_tile(mA, Shape<Int<kBM>, Int<kBK>>{}, make_coord(blockIdx.y, kt));
  auto gB = local_tile(mB, Shape<Int<kBN>, Int<kBK>>{}, make_coord(blockIdx.x, kt));
```

```
    auto gSFA = local_tile(mSFA, Shape<Int<kBM>, Int<kBK>>{}, make_coord(blockIdx.y, kt));
    auto gSFB = local_tile(mSFB, Shape<Int<kBN>, Int<kBK>>{}, make_coord(blockIdx.x, kt));
    TmaBarrier::arrive_and_expect_tx(&full[stage], kTxBytes);
    copy(tma_a.with(full[stage]), a_slice.partition_S(gA),
         a_slice.partition_D(sA_full(_, _, stage)));
    copy(tma_b.with(full[stage]), b_slice.partition_S(gB),
         b_slice.partition_D(sB_full(_, _, stage)));
    copy(tma_sfa.with(full[stage]), sfa_slice.partition_S(gSFA),
         sfa_slice.partition_D(sSFA_full(_, _, stage)));
    copy(tma_sfb.with(full[stage]), sfb_slice.partition_S(gSFB),
         sfb_slice.partition_D(sSFB_full(_, _, stage)));
  };
```

与第 三十一 章的 FP8 版本相比，这里只是把 2 条 `copy` 变成 4 条、把 `kTxBytes` 加上两段 SF 的字节数。SF 的 TMA 有一个不显然的地方：

- `make_tma_copy` 的**模板实参是 `uint16_t`**，而张量的元素类型是 `float_ue4m3_t`。原因是 SF 布局在 K 方向有一个 16:0 的广播维（16 个元素共享一个 SF），这个「16 字节共享 1 字节」的模式 TMA 描述符无法直接表达；换用一个 2 字节的「内部元素类型」后，广播维变成 8 个可步进的 `u16` 单元，描述符的 `ScaledBasis` 就能装下它（源码注释里那句「g_shape 必须与它嵌套同构」说的就是这件事，CUTLASS 的 collective 用的是同一种做法）。
- SF 张量的**形状**不能用 `make_shape(M, K)` 造，必须用 `shape(sf_gmem_layout(M,K))`——因为描述符的 `g_stride` 里含 `ScaledBasis`，`g_shape` 必须与它同构。这是「布局即协议」的一个典型例子：布局不是给软件算偏移用的，是被写进硬件描述符的。

33.5 non-WS 主 kernel：四路装载与 zip 张量

执行协议与第 二十九、三十一 两章的 non-WS 模式逐行同构：256 线程全员 consumer，`tid=0` 兼任 TMA 发射；`full[stage]`（事务屏障，init=1）标记数据就绪，`empty[stage]`（普通屏障，init=256）标记 stage 可覆写。差异集中在循环体里的四行拷贝与一次 `gemm`：

fp4_gemm.cuh L794-810（主循环：四路 s2r + zip 张量 + 一跳一 stage）

```
  for (int kt = 0; kt < nKt; ++kt) {
    const int stage = kt % kStages;
    const int phase = (kt / kStages) & 1;
    TmaBarrier::wait(&full[stage], phase);
    cutlass::arch::fence_view_async_shared();

    // 数据 smem -> 寄存器（LDSM），SF smem -> 寄存器（4 字节 LDS）
    copy(s2r_copy_a, tCsA(_, _, _, stage), tCrA_view);
    copy(s2r_copy_b, tCsB(_, _, _, stage), tCrB_view);
    copy(tCsSFA(_, _, _, stage), tCrSFA_view);
    copy(tCsSFB(_, _, _, stage), tCrSFB_view);
    // 一跳一个 stage（atom K == BK），故 SF 与数据一起进 zip 张量直接发 MMA
    cute::gemm(tiled_mma, make_zip_tensor(tCrA, tCrSFA),
               make_zip_tensor(tCrB, tCrSFB), tCrC);

    CtaBarrier::arrive(&empty[stage]);
    if (tid == 0) {  // 预取 kt+kStages（须在本轮 arrive 之后）
```

数据走 LDSM，SF 走 32-bit LDS。A/B 用 `SM75_U32x4_LDSM_N`（`ldmatrix.x4`，一个 warp 一次搬 512 B、每条 lane 16 B = 32 个 4-bit；而 `Layout_K_SW32_Atom<uint4_t>` 一个原子是 8×64 个 4-bit = 256 B，故一次 LDSM 覆盖两个原子）；SF 用 `UniversalCopy<ElementSF>`——因为它每线程只要 4 个字节，`ldmatrix` 的 8×8 矩阵语义在这里反而是负担，直接四次 32-bit `ld.shared` 更省（CuTe 会把它自动向量化）。两种拷贝共用同一套 fragment（`tCrSFA`），fragment 的布局由 `sf_partition::partition_fragment_sfa` 从 CUTLASS 的权威定义移植过来（`fp4_gemm.cuh` L425–521，逐行抄自 `sm120_blockscaled_mma_tma.hpp` 的 `thrfrg_sfa` 系列）。

zip 张量是「数据 + SF」的成对打包。blockscaled MMA 的两个 A 操作数（数据与 SF）在寄存器上是分开的 fragment，但 MMA 的调用接口要求一次给出两者，于是 CuTe 用 `make_zip_tensor(tCrA, tCrSFA)` 把它们 **按列并成**一个「元素是 pair」的张量：

```
gemm(tiled_mma,
     make_zip_tensor(tCrA,  tCrSFA),   // (A, SFA) 成对
     make_zip_tensor(tCrB,  tCrSFB),   // (B, SFB) 成对
     tCrC);
```

这不是语法糖：`cute::gemm` 会把 zip 张量的每个「pair 元素」当作一条 MMA 的完整 A 操作数（数据 + 局部的 scale），然后沿着 K 与 tile 维遍历。

一跳一 stage：K 方向没有内层循环。BK=64 等于原子的 K 宽，所以每个 stage 的 K 方向只对应**一次** atom 调用；`gemm` 里剩下的展开只是 tile 维——`AtomLayoutMNK`= $(4,2,1)$ 把 $128 \times BN$ 的输出切成每 warp $16 \times BN/2$ 的块，BN=128 时每个 warp 要做 8 条 `m16n8k64`（N 方向切 8 次），一个 CTA 每 stage 共 8 warp $\times$ 8 = 64 条。第 三十一 章 FP8 的一个 stage 要跑 $BK/32 = 4$ 个 k-slice、每个 k-slice 里还有 tile 维展开；FP4 把 k-slice 这一层直接删掉了（图 33.4 的对比）。

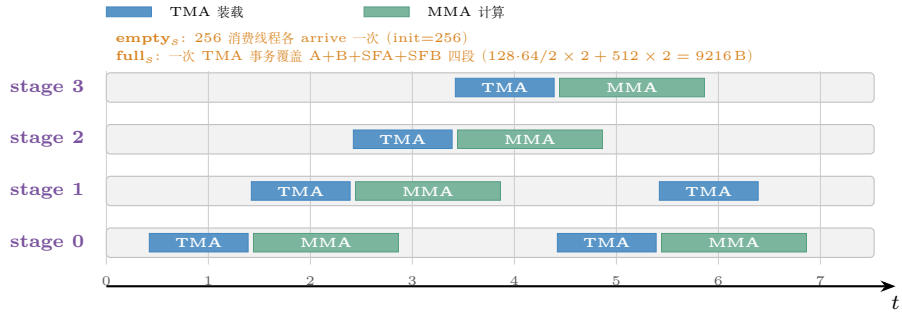


图 33.4：四级流水的时序示意（BN 取 128 的默认档；本卡 k_{Stages} 可用到 8~10 级，见 37.9.4 节扫描表）。与第 三十一 章图 31.2 的结构差别在 MMA 块的**宽度**：FP4 的一个 stage 只有一跳原子（BK=64），K 方向不再有内层循环，MMA 块相对 TMA 块更窄——这意味着**流水线对 TMA 延迟的容忍度更低**，是 BN 翻倍的收益来源之一（BN 翻倍 = MMA 块宽翻倍，而 TMA 字节数只增加 B 与 SFB 的部分）。

33.6　Epilogue：只剩一项乘法

fp4_gemm.cuh L820-852（epilogue：level-1 折回 → bf16 → STSM → TMA store）

```
  // Epilogue: 反量化 (level-1 折回: C = acc * sA[m] * sB[n]) -> bf16 ->
  // STSM r2s (复用 K 循环结束后的 A/B/SF 区作 O staging) -> TMA store (尾行裁剪)
  {
    __syncthreads();  // 全部 mma 完成，smem 可覆写
    auto tCrC_rc = make_tensor(tCrC.data(), convert_layout_acc_rowcol(tCrC.layout()));
#pragma unroll
    for (int r = 0; r < kCRows; ++r) {
      const int m = m0 + get<0>(tScC_rc(r, 0));
      const float sa_r = kLevel1A ? ((m < M) ? sa[m] : 0.0f) : 1.0f;
#pragma unroll
      for (int c = 0; c < kCCols; ++c) {
        const int n = n0 + get<1>(tScC_rc(r, c));
        const float sb_c = kLevel1B ? ((n < N) ? sb[n] : 0.0f) : 1.0f;
        tCrC_rc(r, c) = tCrC_rc(r, c) * sa_r * sb_c;
      }
    }
    auto tCrCh = convert_type<Element0>(tCrC);

    auto r2s_copy = make_tiled_copy_C(Copy_Atom<SM90_U32x4_STSM_N, Element0>{},
```

```
                                    tiled_mma);
    auto r2s_thr = r2s_copy.get_thread_slice(tid);
    auto sC = make_tensor(make_smem_ptr(reinterpret_cast<Element0 *>(shm)),
                          SmemLayout0{});  // 覆盖已空闲的 A/B/SF 区
    auto tCrCh_src = r2s_thr.retile_S(tCrCh);
    auto tCsC_dst = r2s_thr.partition_D(sC);
    copy(r2s_copy, tCrCh_src, tCsC_dst);
    cutlass::arch::fence_view_async_shared();
    __syncthreads();
    auto gC = local_tile(mC, Shape<Int<kBM>, Int<kBN>>{},
                         make_coord(blockIdx.y, blockIdx.x));
    if (tid == 0) copy(tma_c, c_slice.partition_S(sC), c_slice.partition_D(gC));
    tma_store_arrive();
    tma_store_wait<0>();
```

与第 三十一 章 FP8 的 epilogue（四步、含按块查 δ）相比，这里的**反量化只剩一个乘法**：$C_{m,n} \leftarrow C_{m,n} \cdot s_{A,m} \cdot s_{B,n}$，其中 s_A, s_B 是量化阶段算好的行/列级 fp32 数组（式 32.2）。level-2 的 SF 已经在 MMA 里乘掉了——这不是软件省掉的，是**由硬件完成的**：一条 `m16n8k64` 内部完成了 4 个 K 组各一次的 $A \cdot \mathrm{SF}_A$ 与 $B \cdot \mathrm{SF}_B$。

三处工程细节：

- **行列坐标视图**：`convert_layout_acc_rowcol` 把 acc 的 fragment 布局重排成 (r, c) 二维，才能按 (m, n) 查 $s_A[m]/s_B[n]$。这个工具从第 二十五 章起就在用，FP4 版本一字不改。
- **尾块保护**：$m \geq M$ 或 $n \geq N$ 的 fragment 元素取 $s = 0$。TMA store 会自动裁掉越界行列，但 **acc 里的越界元素在裁掉之前不得成为 NaN/Inf**——乘 0 是最省的保护。
- **O staging 复用整个 A/B/SF 区**。$BM \times BN$ 的 bf16 结果（BN=128 时 32 KB、BN=256 时 64 KB）不能放在单个 stage 里（一个 stage 只有 9 KB / 13.5 KB），K 循环结束后整个 smem 池空闲，直接 rebind。约束写成 `kOStagingFits`（$BM \times BN \times 2 \leq k_{Stages} \cdot$stage bytes）并 `static_assert`——这条断言是 37.9.4 节解释「为什么 128×256 至少要 5 级流水」的根据。

33.7 WS 变体与一个 FP8 里不存在的死锁

warp specialization 版本与第 二十八 章的 FA2/FA3 WS 变体同构：128 个 producer 线程只发 TMA，256 个 consumer 线程只做 MMA 与 epilogue；前者 `warpgroup_reg_dealloc<32>`、后者 `warpgroup_reg_alloc<224>`，两侧的寄存器预算写成一条 `static_assert`：$128 \times 32 + 256 \times 224 = 61440 \leq 65536$。

33.7.1 死锁与修复：empty 必须由 consumer 侧预置

WS 版本有一个 non-WS 版本**不存在**的死锁陷阱，具体如下：

- `empty[s]` 的 init 是 `kNumThreads`（256），只有 consumer 会 arrive——因为只有 consumer 知道「我读完了」。
- 于是 producer 在发出第一个 TMA 之前，必须先等 `empty[0]` 翻转；而 `empty[0]` 要翻转，必须等 256 个 consumer 各 arrive 一次；而 consumer 要 arrive 的前提是它先 `wait(full[0])` 成功，即**第一个 TMA 已经完成**。
- 环闭合了：producer 等 consumer、consumer 等 producer 的 TMA。**启动即死锁**。

修复只有一行，放在 K 循环之前、producer/consumer 分支之前：

fp4_gemm.cuh L958-964（WS 的 empty 预置：consumer 先行 arrive）

```
  const int nKt = K / kBK;
  // 预置 empty: 前 kStages 个 stage 的 empty 必须先「已就绪」，否则 producer
  // 的第一次 wait 会死锁 (consumer 要等 TMA 才能 arrive，形成环)
  if (tid >= kProducerThreads) {
    for (int s = 0; s < kStages; ++s) CtaBarrier::arrive(&empty[s]);
  }
```

```
  if (tid < kProducerThreads) {
```

即「**consumer 在进入循环前先把所有 empty arrive 一遍**」，构造出流水线的初始条件。这么写合法的理由是：在第一个 TMA 完成之前，consumer 本来就没有任何东西可读，让它们先声明「stage 是空的」是 **对初始状态的正确描述**，而不是取巧的做法。

non-WS 版本没有这个问题，因为那里是「全体线程 arrive」——`empty` 的 init 等于参与线程数，而全体线程在进入循环前就各 arrive 了一次（第 二十九 章的那三行初始 release），同一个动作由身兼二职的线程完成了。**warp specialization 把「谁 arrive」这件事拆开之后，初始条件就必须重新核对**——这是本章唯一的实质性 bug，也是本书 ws 系列最要紧的一条教训。

33.7.2　命名屏障

WS 版本的 epilogue 里 sync 用的是 `NamedBarrier::arrive_and_wait(256, 0)` 而不是 `__syncthreads()`——因为 CTA 里有 384 个线程，`__syncthreads()` 会把已经退出的 producer 也算进去，而命名屏障只同步注册了 256 个 consumer。（producer 发完最后一个 TMA 就直接结束，不再参与后续任何屏障。）

33.8　C++ API 与 bench 接线

fp4_gemm.cuh L1162-1226（fp4_gemm_bf16：约束自检 + 4 模式派发）

```
// BF16 in -> 在线 NVFP4 量化 GEMM -> BF16 out，单 stream 语义。
// 同步约定：与 cuBLAS 一致，入队后由调用方在 stream 上同步。
// 约束：K%64==0, N%8==0(TMA 对齐)，M 任意。
// mode 决定 level-1 的有无与作用侧；use_ws 走 warp-specialized 变体。
// Traits 默认保守档 128x128/s4；大形状可显式传 Fp4GemmTraits<256, 6>
//  （形状判定的阈值见 fp4_gemm_use_wide_tile）。
template <typename Traits = Fp4GemmTraits<>>
inline void fp4_gemm_bf16(const __nv_bfloat16 *A, const __nv_bfloat16 *B,
                          __nv_bfloat16 *C, int M, int N, int K,
                          Fp4GemmScaleMode mode, Fp4GemmWorkspace &ws,
                          cudaStream_t stream, bool use_ws = false) {
  if (K % 64 != 0 || N % 8 != 0) {
    fprintf(stderr,
            "fp4_gemm_bf16: require K%%64==0 (got K=%d), N%%8==0 (got N=%d)\n",
            K, N);
    return;
  }
  const bool level1_a =
      mode == Fp4GemmScaleMode::kLevel1A || mode == Fp4GemmScaleMode::kTwoLevel;
  const bool level1_b =
      mode == Fp4GemmScaleMode::kLevel1B || mode == Fp4GemmScaleMode::kTwoLevel;

  uint8_t *p = static_cast<uint8_t *>(ws.buf);
  cutlass::float_e2m1_t *a4 = reinterpret_cast<cutlass::float_e2m1_t *>(p);
  p += fp4_gemm_align256((size_t)M * (K / 2));
  cutlass::float_e2m1_t *b4t = reinterpret_cast<cutlass::float_e2m1_t *>(p);
  p += fp4_gemm_align256((size_t)N * (K / 2));
  cutlass::float_ue4m3_t *sfa = reinterpret_cast<cutlass::float_ue4m3_t *>(p);
  p += fp4_gemm_align256(sf_buffer_bytes(M, K));
  cutlass::float_ue4m3_t *sfb = reinterpret_cast<cutlass::float_ue4m3_t *>(p);
  p += fp4_gemm_align256(sf_buffer_bytes(N, K));
  float *sa = reinterpret_cast<float *>(p);
  p += fp4_gemm_align256((size_t)M * sizeof(float));
  float *sb = reinterpret_cast<float *>(p);

  if (level1_a) row_amax_fp4_kernel<<<(M + 7) / 8, 256, 0, stream>>>(A, sa, M, K);
  {  // 一线程一（行，16-K 组）：总工作量 = 补齐行数 x (K/16)
    const int mpad = (M + kRowBlock - 1) / kRowBlock * kRowBlock;
```

```
    const int total = mpad * (K / kSFVec);
    const int blocks = (total + 255) / 256;
    if (level1_a)
      quantize_a_fp4_kernel<true><<<blocks, 256, 0, stream>>>(A, a4, sfa, sa, M, K);
    else
      quantize_a_fp4_kernel<false><<<blocks, 256, 0, stream>>>(A, a4, sfa, sa, M, K);
  }
  fp4_gemm_quantize_b(B, b4t, sfb, sb, N, K, level1_b, stream);

#define FP4_GEMM_LAUNCH(L1A, L1B)                                          \
  do {                                                                     \
    if (use_ws)                                                            \
      fp4_gemm_tma_fwd<L1A, L1B, true, Traits>(a4, b4t, sfa, sfb, sa, sb, \
                                               reinterpret_cast<cutlass::bfloat16_t *>(C), \
                                               M, N, K, stream);           \
    else                                                                   \
      fp4_gemm_tma_fwd<L1A, L1B, false, Traits>(a4, b4t, sfa, sfb, sa, sb, \
                                                reinterpret_cast<cutlass::bfloat16_t *>(C), \
                                                M, N, K, stream);          \
  } while (0)
  if (level1_a && level1_b)
    FP4_GEMM_LAUNCH(true, true);
  else if (level1_a)
    FP4_GEMM_LAUNCH(true, false);
  else if (level1_b)
    FP4_GEMM_LAUNCH(false, true);
  else
```

API 形状与第 三十一 章的 `fp8_gemm_bf16` 完全一致：`BF16 in → 量化 → GEMM → BF16 out`，workspace 由调用方持有并复用，同步语义与 cuBLAS 相同（入队即返回）。workspace 的布局如图 33.5 所示。

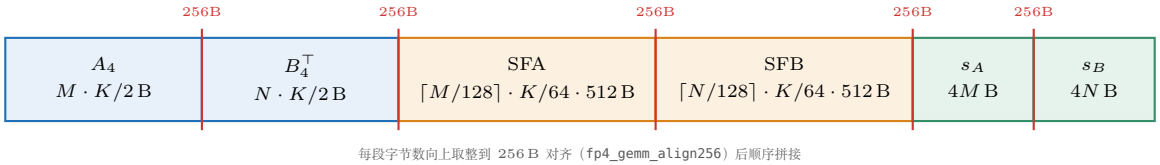


图 33.5：`fp4_gemm_bf16` 一次性 workspace 的六段布局（`fp4_gemm.cuh` 的 `fp4_gemm_workspace_size`）。A_4 与 $B_4^\top$ 是量化产物（4 bit/元素）；SFA/SFB 的行数按 128 向上取整——每个 512 B 原子覆盖 128 行 × 64 个 K 元素；s_A/s_B 是 level-1 的行/列 scale（M/N 个 float）。每段字节数向上取整到 256 B（`fp4_gemm_align256`）后顺序拼接。

四个 `Fp4GemmScaleMode` 通过模板参数**双重**派发（level-1 作用在哪一侧 × 是否 WS），共 8 个 kernel 实例。A 侧先跑行 amax 再跑量化（两趟），B 侧跑 `fp4_gemm_quantize_b`（36.4.3 节的两段式）。

B 离线量化是部署形态：把 $(B_4^\top, \mathrm{SFB}, s_B)$ 量化一次后常驻显存。入口就是 `fp4_gemm_bf16_b_offline`，`Fp4GemmActivation` 只负责 A 侧的激活量化。与 FP8 的差别是离线产物**多一份 SFB**：以 $K{=}N{=}4096$ 计，$B_4^\top$ 占 8 MB、SFB 占 1 MB，即权重的显存占用是 BF16 的 $9/32 \approx 28\%$。

`bench/fp4_gemm.cu` 的 bench 接线（CLI flag 入口在 `bench/bench_leetcuda.cu`）：

- `--fp4-gemm`：7 条正确性 case（4 种 scale 模式 + WS + 两个尾部 shape），CPU fp64 参考，报 relFro。
- `--bench`：在 FP8 行之后追加 12 行 FP4。其中 `FP4 GEMM CuTe NonWS/WS` 是 **kernel-only** 口径（量化在计时窗口外），`FP4 GEMM+Quant E2E` 是全链路，`FP4 GEMM+A Quant E2E (B offline)` 是部署口径。
- `--fp4-gemm-sweep`：$BN \in \{128, 256\} \times k_{Stages} \in \{2, 3, 4, 5, 6, 7, 8, 10, 12\}$ 的完整扫描，不合法组合打印 SKIP 理由（37.9.4 节）。

写 kernel-only 行时的一个反向陷阱。 kernel-only 行直接调 `fp4_gemm_tma_fwd`（跳过量化），但这个入口**不会自己填 workspace**——里面的 $A_4/B_4^\top$/SFA/SFB 全是量化 kernel 的产物。本轮实测遇到过：kernel-only 行第一次跑出 `relFro`= 49（相对误差 4900%）这种物理上不可能的读数，原因就是它读了一整块未初始化的显存。修复是在

`FP4_RUN_KERNEL` 宏的第一行先跑一次完整的 `fp4_gemm_bf16` 把 workspace 填好，之后才计时纯 kernel。教训：**凡是要「拆开计时」的加速比，先确认被拆掉的阶段留下的是有效数据**——否则量到的是显存的随机内容。

33.9　实测：精度与性能 vs cuBLAS BF16

平台：RTX PRO 5000（sm_120a），CUDA 13.x，cuBLAS 随附版本；`--warmup 2 --repeat 3`。输入均匀随机 $[-1, 1]$（`srand(42)`）。FP4 行的误差列是 relFro 相对 Frobenius 误差。

形状	Kernel（tile/stage）	relFro	TFLOPS	vs cuBLAS BF16	vs our CuTe HGEMM
4096^3	cuBLAS BF16 (F32 acc)	—	163.5	1.00x	0.67x
	our CuTe HGEMM (S=3, BLK_SW=1, F32Acc)	≈0	242.8	1.48x	1.00x
	FP8 GEMM NonWS (RC, 128×256×128/s2)	4.50	441.1	2.70x	1.82x
	FP4 NonWS 单级 level-2 (128×256×64/s6)	0.144	600.4	3.67x	2.47x
	FP4 NonWS 两级 (128×256×64/s6)	0.145	603.4	3.69x	2.49x
	FP4 WS 两级 (128×256×64/s6)	0.145	**614.1**	**3.76x**	**2.53x**
	FP4+Quant E2E NonWS (128×256×64/s6)	0.145	408.2	2.50x	1.68x
	FP4+A-Quant E2E, B off NonWS	0.145	550.5	3.37x	2.27x
	FP4+A-Quant E2E, B off WS	0.145	561.2	3.43x	2.31x
8192^3	cuBLAS BF16 (F32 acc)	—	163.3	1.00x	0.68x
	our CuTe HGEMM (S=3, BLK_SW=1, F32Acc)	≈0	241.7	1.48x	1.00x
	FP8 GEMM NonWS (RC, 128×256×128/s2)	6.25	483.4	2.96x	2.00x
	FP4 NonWS 单级 level-2 (128×256×64/s6)	0.144	644.8	3.95x	2.67x
	FP4 NonWS 两级 (128×256×64/s6)	0.145	639.6	3.92x	2.65x
	FP4 WS 两级 (128×256×64/s6)	0.145	**648.6**	**3.97x**	**2.68x**
	FP4+Quant E2E NonWS (128×256×64/s6)	0.145	467.4	2.86x	1.93x
	FP4+A-Quant E2E, B off NonWS	0.145	541.1	3.31x	2.24x
	FP4+A-Quant E2E, B off WS	0.145	549.7	3.37x	2.27x

（tile 记法 $= BM{\times}BN{\times}BK$；s6 = 6 级流水。2026-09-28 起 `--bench` 的 FP4 标准行按形状自适应（`fp4_gemm_use_wide_tile`）：$MNK \geq 4096$ 用宽档 128×256×64/s6，小形状回落库默认档 128^2×64/s4——本表两个形状都落在宽档上，库默认档的读数（460.7 / 474.5 T 等）见 37.9.4 的扫描表。两列的加速比一律以本表 cuBLAS 行的读数（4096^3: 163.5，8192^3: 163.3）与手写 HGEMM 行（242.8 / 241.7，均为 F32 累加）为分母；同一张表里 cuBLAS 会被重复计时，各行分母因此相差 ±1%。「relFro 0.144 vs 0.145」的差别就是 `kSingleLevel` 与两级量化的差别——$U(-1, 1)$ 数据下 level-1 几乎无用，第 三十二 章的误差模型解释了为什么。

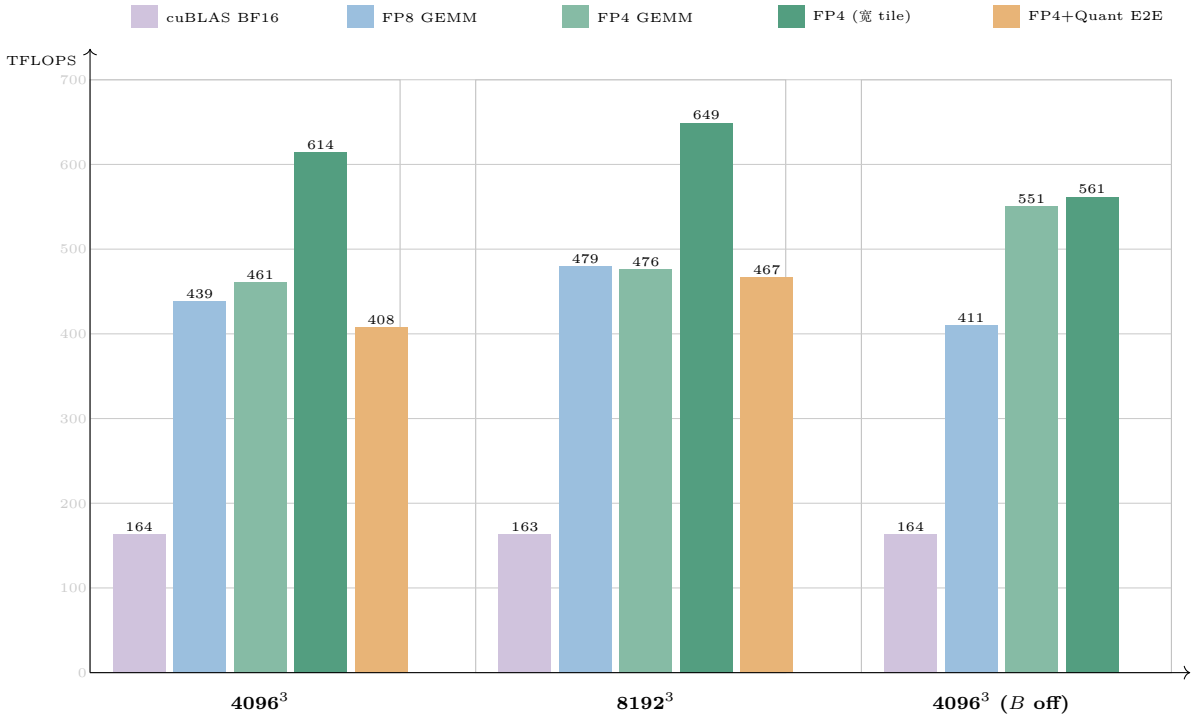


图 33.6: NVFP4 GEMM vs cuBLAS BF16（PRO 5000，越高越好）。三条读法：其一，**kernel-only 口径下 FP4 只比 FP8 快** 6 ∼ 40%——库默认档（128^2/s4，柱取口径：4096^3 取扫描表 461 T，8192^3 取默认档实测 476 T）只快 6%，宽 tile 档才快 40%，远不是比特数暗示的 2×；原因是本 kernel 走 bf16 输出、f32 累加，且 BK=64 让 K 方向的算术强度下降（表 33.1 的两条固定约束）；其二，**宽 tile（128×256）是本章最大的单项收益**（+31%，37.9.4 节），也是 `--bench` 标准行在 $MNK \geq 4096$ 上的自适应选择；其三，**E2E 口径下 FP4 反超 FP8**（4096^3：408 vs 187 T；8192^3：467 vs 315 T）——量化的写出字节数只有 FP8 的一半，B 侧重构后这个差距进一步扩大（37.9.3 节）；第三组（B 离线）FP4 的 E2E 回到 551∼561 T（FP8 为 411 T），是各自 kernel 的 91 ∼ 93%。

33.9.1 对照基线：cuBLAS BF16 为什么还是 165 T

第 三十一 章 35.8.1 节已经用 `nsys` 拆过这件事：cuBLAS 在本卡上对 BF16 输入选择的 kernel 只有约 165 T，而同一张卡的 fp16 HGEMM 能跑 246 T——它的 heuristic 把 BF16 输入降级到了非最优的 kernel。这不是本书的 kernel 特别强，而是**基线本身偏弱**。为了把「FP4 到底有多快」这件事说清楚，本章的标准表同时给出三个参照系：cuBLAS BF16（2.9∼3.7×）、本书手写 bf16 HGEMM （1.8 ∼ 2.5×）、以及第 三十一 章的 FP8 版本（1.07 ∼ 1.38×）。**做量化 GEMM 的性能声明时，「相对谁」比「多少倍」重要得多**。

33.9.2 精度

`--fp4-gemm` 的 7 条 case（512^3，CPU fp64 参考）与第 三十二 章的实测表逐位一致：单级 0.1437、level-2×A 行 0.1448、level-2×B 列 0.1443、两级（非 WS/WS）0.1454、尾部 shape 0.1454。三点读法：

- **WS 与 non-WS 数值完全相同**（0.1454 对 0.1454，逐位）。它们走同一套量化产物、同一段 epilogue 数学，warp specialization 只改「谁来发 TMA、谁来算」，不改任何数值路径。这也是 WS 版本可以直接作为默认推荐的前提。
- 尾部 shape（M=300, N=264, K=192）与齐整 shape 一致，说明 TMA 的零填充、store 裁剪、补零行 SF 显式清零（36.4.2 节）三处都正确处理了。
- 尾部 shape 的耗时也没有惩罚：M 从 4096 改成 4097 （网格在 M 方向多一整行 tile）只从 414.0 降到 408.2 T（−1.4%），M 从 8192 改成 8193 只从 471.2 降到 467.6 T （−0.8%；NonWS 同口径 407.3/402.1 与 466.0/464.4）；多算的只是一个 128×256 的输出行带，浪费的 FLOP 不到 0.03%。这一条不在 `--bench` 表里（`--mnk` 只接受方形形状），数字由同一条 `fp4_gemm_bf16` 入口（宽档）用 CUDA event 单独计时得到。
- B **离线量化与全链路逐位一致**（`ch37_fp4_gemm.cu` 里 4 条 `bit-exact=YES`）。这条断言的价值不在于「测出对了」，而在于它是一个 **回归护栏**：以后任何改动只要让这两条路分叉，测试立刻报红。

33.9.3　量化开销，以及一次重构

先算开销。E2E 行减去 kernel-only 行，就是两侧量化实际花掉的时间（4096^3：FP8 得 424 μs、FP4 得 109 μs；8192^3：1221 μs 对 633 μs；FP4 侧按本章的自适应宽档算）：

	4096^3		8192^3	
	量化	占 E2E	量化	占 E2E
FP8 GEMM	424 μs	58%	1221 μs	35%
FP4 GEMM（重构后）	**109 μs**	**32%**	**633 μs**	**27%**

两个反直觉的读法：其一，FP4 的量化**不该**比 FP8 开销更小——它多产出一份 SFB，而且 $A_4/B_4^\top$ 的 SF 布局比 FP8 的 per-row/per-col scale 复杂得多；其二，8192^3 上 FP4 的量化绝对值（633 μs）比 FP8（1221 μs）小一半，但两者的数据量相同（都读 $MK{+}NK$ 个 bf16、写 $MK{+}NK$ 个量化值）——差别只在**写出的字节数**：FP8 每个量化值 1 B，FP4 只有 0.5 B。这与本卡实测带宽相符（纯读 907 GB/s、拷贝 1047 GB/s，DRAM 标称 1344 GB/s）：8192^3 时 A/B 两侧各跑在 ≈ 1.0 TB/s，已经在拷贝带宽上。还有一个容易误读的数：量化是**加性**成本（109 μs 不随 tile 档变化），主 kernel 从保守档换到宽档快了 31% 之后，它的占比反而从 26% 升到 32%——*占比上升说明主 kernel 变快了，不是量化变慢了*。

本书在这一章对 B 侧量化做过一次**重构**，它是用带宽数据定位问题的典型例子。

- **第一版**（与第 三十 章 FP8 版本同构）把逐列 amax 与转置量化压进同一个 kernel 的两个 pass：一个 block 负责连续 128 列 × 全部 K。这个划分有两个缺陷：*第一，并行度*——grid 只有 $N/128$ 个 CTA，4096^3 时 32 个 CTA 只填满 29% 的 SM；*第二，bank 冲突*——逐列归约读 smem 的步长是 128 个元素 = 64 个字 = 32（bank 数）的整数倍，每读一次都是 32 路冲突。按「B 读两遍 + 写 $B_4^\top$ 与 SFB ≈ 73 MB」的逻辑流量反推，第一版的有效吞吐只有 **194 GB/s**（377 μs）——本卡拷贝带宽的五分之一。

- **第二版**把两件事拆成两个全并行 kernel：*列 amax* 沿 K 切 8 片（4096^2 时 256 个 CTA），每线程持 8 列、K 方向步进 16（一次 `Vec8BF16` 载入拿满 8 列），块内归约到 128 个列值后落到一次整数 `atomicMax`（非负 IEEE754 的位模式与无符号整数同序，可以直接按 `unsigned` 比大小）；*转置量化*按 $128(n)\times 64(k)$ 一个 tile 铺 $(N/128)\times(K/64)$ 个 CTA（4096^2 时 2048 个），per-16 组的 amax 完全落在块内，不需要任何跨块归约，smem 里的转置缓冲加 1 列 padding（128×65）消除写回时的 bank 冲突。

- **一个 bit 都不许变**是这个重构的硬约束。列 amax 拆成 8 片后要用 `atomicMax` 合并，而原子操作只支持整数——这里靠两步做到逐位等价：第一，「先除以 2688 再取 max」与「先取 max 再除以 2688」在浮点下逐位相同（除以正常数保序）；第二，非负浮点的位模式与无符号整数序一致。调用方必须先把 s_B 清零这一点也被写进约定（`cudaMemsetAsync`，浮点 0 的位模式正是 `atomicMax` 的单位元）。转置量化的写法改动（tile 从「128 列 × 全 K」变成「128 列 × 64 K」）不改变任何中间值。`book/tests/ch36_fp4_quantize.cu` 的 13824 个 SF 落点与逐 bit 数据比对、以及 ch37 的 `bit-exact=YES` 断言，就是这条约束的验证手段。

- **收益**：B 侧量化的反推吞吐从 194 GB/s 提到 **792 GB/s**（4.1×，4096^3；同口径的逻辑流量），E2E 从 196.8 T 提到 **334.2 T**（+70%），量化占 E2E 的比例从 57% 降到 26%。注意 792 GB/s 里有一部分是第一遍读到的 B（32 MB）留在 96 MB L2 里、第二遍直接命中——**这个数高于 DRAM 带宽是正常的，它的意义是「同一口径下的相对改善」**，而不是「DRAM 吞吐」。

s_B 的数值语义在这里有一个微妙的点：第一版把 amax/2688 先存进 smem 再原地改成倒数，第二版改成「s_B 落显存、倒数在量化 kernel 里现算」。两者在 amax = 0 的行上都必须走同一条特殊分支（inv = 0，整列归零），否则会出现 0/0。

33.9.4　tile 几何与流水深度：128^3 不是最优，128×256 才是

`--fp4-gemm-sweep` 把 $BN\in\{128,256\}$ 与 $k_{Stages}\in\{2,3,4,5,6,7,8,10,12\}$ 全扫一遍（4096^3，两级量化），不合法的组合打印 SKIP 理由：

BN	k_{Stages}	smem	NonWS TFLOPS	WS TFLOPS	结论
128	2	18 KB	SKIP	—	O staging 32 KB > 18 KB
128	3	27 KB	SKIP	—	O staging 32 KB > 27 KB
128	4	36 KB	460.7	474.5	库默认档
128	6	54 KB	458.1	469.7	与 s4 持平
128	8	72 KB	462.1	473.3	与 s4 持平
128	10	90 KB	453.4	—	略降
128	12	108 KB	SKIP	—	超过 99 KB 上限
256	4	54 KB	SKIP	—	O staging 64 KB > 54 KB
256	5	67 KB	599.0	610.9	与 s6 持平
256	6	81 KB	**602.9**	**614.6**	**最快档**
256	7	94 KB	603.5	—	与 s5/s6 持平

三个规律：

- **流水深度在这个 kernel 上几乎不影响性能。** BN=128 时 k_{Stages} 从 4 加到 8（36 KB → 72 KB，smem 预算翻倍）只换来 ±1%；BN=256 时 5/6/7 级也持平。与第 三十一 章 FP8 版本（2 级 > 3 级，+7.7%）不同，FP4 的结论是「流水深度够用就行」。原因见图 33.4：FP4 的一个 stage 只有一跳 MMA，MMA 块窄、TMA 块相对粗，**瓶颈不是「等下一块数据」，所以加深流水没有收益。**
- **BN 从 128 翻到 256 带来 +31%（461 → 603 T）。**这一项远超其他所有旋钮。机制是 MMA 与 TMA 的比值：BN 翻倍让每 stage 的 B 区（BN·64/2 字节）与 SFB（512 B）跟着涨，但 A 区与 SFA 完全不变；而 MMA 的工作量随 BN 线性翻倍。于是**每字节 smem 数据摊到的 MMA 翻倍**——这就是「算术强度」在 tile 几何层面的直接体现。代价是 O staging 翻倍（64 KB），要求 $k_{Stages} \geq 5$。
- **WS 的价值随 BN 变大而显现。** BN=128 时 WS 比 NonWS 快 3.0%（474.5 vs 460.7），BN=256 时快 2.0%（614.6 vs 602.9）。差距不大但方向稳定：WS 让 TMA 发射与 MMA 彻底解耦后，`tid0` 不再是「算完还要负责发 TMA」的瓶颈。BN=256/**s6 的 ws 档（614.6 T）是本章的最快配置**，比库默认档（BN=128/s4）快 29%。

为什么库默认档不直接取 128×256/**s6？**因为它是**形状依赖**的：BN=256 时 grid 的 N 方向 tile 数减半，小形状上 CTA 数不足会导致性能明显下降(第 三十一 章 FP8 的 2048^3 就出现过这种反转)。库默认取 128×128/s4 是保守选择——它在所有形状上都不差；本书的折中是把判定写死成一条形状规则（`fp4_gemm.cuh` 的 `fp4_gemm_use_wide_tile`）：$MNK \geq 4096$ 走 128×256/s6，其余回落 128^2/s4，`--bench` 的 FP4 标准行与探针都按这条规则计时（本节开头的标准表因此整表都是宽档）。这与第 三十一 章 35.8.4 节的结论同一形态：**量化 GEMM 的「默认」不该是一个 tile，是一条形状规则。**

33.10 小结

- **block scale 的 `mma.sync` 把 SF 纳入了操作数**：`m16n8k64.kind::mxf4nvf4.scale_vec::4X` 的数据操作数是 24 B/线程、SF 操作数是 8 B/线程——SF 的字节数达到数据的 1/3。SF 就是**第二个输入矩阵**，不是元数据，必须有自己的 TMA 通道、smem 缓冲与寄存器 fragment。
- **SF 的 smem 原子与 gmem 原子逐字节同序**（`probe_sf_atom.cu` 枚举 512 个落点验证）。这一条消除了「SF 需要 swizzle 吗」的疑问：512 B 原子本身就是为「一次 32-bit LDS 取 4 个 SF」设计的，smem 只是原样拷贝。
- BM=128、BK=64 **不是调参，是唯一合法解**：二者分别等于 SF 原子的行块与 K 覆盖范围，也就是 SF 的 TMA box 的尺寸。tile 空间因此被固定成 $128 \times BN \times 64$，BN 成了唯一旋钮——而它恰好是收益最大的那个（+31%）。
- **epilogue 只剩一个乘法**：level-2 的 SF 由 MMA 硬件消费，level-1 按 (m, n) 折回。反量化的复杂度从「按块查表」（FP8）降到「一次标量乘」（FP4），代价全部前移到指令形态与数据通路上。
- **死锁教训**：WS 版本里 `empty` 的 init 是消费者线程数，而消费者的第一次 arrive 依赖 producer 的第一次 TMA——环闭合。修复是让消费者在进入循环前把 `empty` 全部 arrive 一遍。**当「谁 arrive」被 warp specialization 拆开，流水线的初始条件必须重新核对。**
- **性能结论**：宽档 128×256/s6 的 WS 变体在 4096^3 跑 614.1 T（8192^3: 648.6 T），是 cuBLAS BF16 的 3.76×、本书手写 bf16 HGEMM 的 2.53×、FP8 版本的 1.39×。`--bench` 标准行自 2026-09-28 起按 $MNK \geq 4096$ 自适应

取宽档、其余回落 128^2/s4。比特数减半只换来 1.4×——**存储收益与算力收益不是一回事**，BK 被固定在 64 让 K 方向的算术强度下降是主要原因。

- **量化开销**：重构后 B 侧量化从 194 GB/s 提到 792 GB/s；量化成本 109 μs（4096^3）是加性的、不随 tile 档变化，保守档下占 E2E 的 26%，宽档下占 32%。E2E 也因此反超 FP8 版本：宽档 408.2 对 186.5 T （8192^3：467.4 对 314.6 T）。部署口径（B 离线）把 E2E 提到 550.5∼561.2 T，是主 kernel 的 91 ∼ 93%。

延伸阅读

1. PTX ISA §9.7.14.3 *Block Scaling for* `mma.sync` 与 §9.7.14.5 的 `mxf4nvf4` 示例（本机路径 `.github/skills/cuda-cpp-kernel/references/ptx-docs/9-instruction-set/`）：`.scale_vec_size` 的合法组合表、`scale-a-data` 的 `{byte-id, thread-id}` 选择器语义。
2. CUTLASS `include/cutlass/gemm/sm120_blockscaled_mma_tma.hpp`：SF 的 `thrfrg` 片段布局权威定义（本书 `fp4_gemm.cuh` L425–521 的 `sf_partition` 逐行抄自此处）。同一目录下的 `sm120_blockscaled_mma_builder.inl` 给出了 `SfSmemAtom` 的四段式子。
3. 知乎 @DefTruth《WINT8/4》系列：低比特 GEMM 的量化方案与 Blackwell 上的实测吞吐讨论。

第六部分

FP8/FP4 Attention 篇：量化注意力的数学与 Kernel 工程

第三十四章　量化注意力的数学基础

34.1　本章导读

前四个部分走完了一条「精度不变」的优化链：编程模型与访存（Part I）→ Tensor Core GEMM（Part II）→ FlashAttention（Part III）→ CuTe 现代化工程（Part IV）。每一步都在**同等数值精度**下降低运行时间。Part VI 换一个优化维度：**把精度本身当作换取吞吐的资源**，用 8-bit 甚至 4-bit 数值格式执行 attention 的两个 GEMM（$QK^\top$ 与 PV），得到 Tensor Core 吞吐的成倍提升。代码链路是 ffpa-attn 仓库的 CuTe fp8/fp4 kernel 家族（commit `861d75e`，只讲 sm_120 路径）：它用 CuTe 高效复现了 SageAttention 2/3 的思想。本章是 Part VI 的「理论总纲」，后面第 三十五–三十九 章里的每一个 kernel 决策（scale 折进哪条指令、粒度对齐哪个 fragment、发射域铺满哪个量程），都能回指到本章的某条公式或某个表格。

本章依次回答五个问题：

1. 量化算子的形式是什么？四种主要格式（int8 / e4m3 / e2m1 / ue4m3）加 scale 专用的 ue8m0 的位域与量化误差下界各是多少（27.3.1–27.3.2 节：相对步长 $\approx 2^{-(m+1)}$ 的逐 bit 推导）；
2. attention 有哪几个可量化的对象？各自适合什么粒度（27.3.3 节：per-block / per-thread / per-channel / per-row / 1×16 谱系表）；
3. scale 因子如何在不做显式反量化 pass 的前提下全部代数消去（27.3.4 节：$\delta_Q\delta_K$ 折进 exp2 系数、v_s 精确消去、p_{scale} 收尾——本节是第 三十六 章的数学骨架）；
4. SageAttention 四代（1/2/2++/3）各自贡献了什么、FA-4 的 lazy rescale 与量化域如何交互（27.3.5–27.3.6 节）；
5. 量化 attention 的误差到底有多大、集中在哪一行（27.3.7–27.3.8 节：ESS 误差模型与 per-stage 误差分解——解释 causal 早行绝对误差为何是 dense 的 15 倍，以及 hybrid 行分割为何是唯一解）。

前置章节：第 五 章（softmax 平移不变性）、第 六 章（LSE）、第 十六 章（attention 分块数学，必须）、第 十二 章（MMA fragment，建议先读，粒度谱系要回指 fragment 布局）。**源码区间**：ffpa-attn `cute/fp8/fp8_pscale.cuh` 头注释（L17–37 三步协议）与 `cute/softmax.cuh` L44–63（lazy rescale）；完整源码锚定与行号以附录 D 的 commit permalink 为准。**验证方式**：本章无独立 kernel 测试（Part VI 以 `ffpa_attn.bench` CLI 替代 `book/tests`，见 27.10 节）。

34.2　问题与动机

34.2.1　硬件的精度阶梯

同一代 GPU 的 Tensor Core 对低精度格式有明确的吞吐阶梯：以本仓库实测的 sm_120（GeForce RTX 5090 / RTX PRO 5000，Blackwell）为例，warp 级 `mma.sync` 对 e4m3（FP8）的吞吐约为 fp16 的 2 倍，对 e2m1/microscaling FP4 再翻倍；显存与互联带宽同样按位宽降低一半。SageAttention3 论文给出的量化口径是 RTX 5090 上 FP4 attention kernel 达到 1038 TOPS、约为该卡上最快 FlashAttention 的 5×（arXiv:2505.11594）。对 attention 这种在长序列下被两个 $O(N^2d)$ GEMM 主导的算子，「把 GEMM 降到低精度」是 fp16 域内任何 tiling / 流水线优化都无法达到的收益量级。Part VI 的动机就在于此。

34.2.2 attention 的三个量化点与「P 无法离线」困难

把式 16.2 的计算链拆开，有三个可量化的对象：

$$\underbrace{S = QK^\top}_{\text{量化点 1：}Q,K} \longrightarrow \underbrace{P = \mathrm{softmax}(S)}_{\text{量化点 2：}P} \longrightarrow \underbrace{O = PV}_{\text{量化点 3：}V} \tag{34.1}$$

Q, K, V 是**前向开始前就存在的静态张量**，可以离线（进 kernel 前）量化好、连同 scale 一起送进 kernel，称为**离线校准**路径。而 P 是 softmax 的输出，每个 forward、每个 tile 都不同，永远无法离线，必须**在 kernel 内、在寄存器 fragment 上**完成量化。这一「在线量化」需求带来一系列独有问题：量化 scale 从哪来（27.3.4）、scale 如何在 MMA 间传递与消去（第 三十六 章）、发射域与 lazy rescale 的膨胀如何不超出格式量程上限（27.3.6）。

34.2.3 精度挑战：为什么不能直接量化

直接把 Q, K, V, P 量化到 8-bit 会得到不可用的结果，原因集中在两点：

- **outlier 显著降低块内分辨率**：低精度格式的量化步长 $\delta = \mathrm{amax}(\mathcal{B})/R$ 由块内最大值决定，任何一个 outlier 都会放大整块的量化步长，同块其它元素只能映射到少数几个码点。注意力权重有三个系统性的「均值偏移」源（K 的 token 间均值、Q 的通道均值、V 的通道均值），对策是三条零数学代价的 smoothing 不变式——本章先给结论（27.3.5 谱系表），完整推导与 kernel 实现放在第 三十五 章。
- **格式量程上限**：e4m3 最大 448、e2m1 最大 6，而 softmax 概率域 $P \in [0, 1]$、FA-4 lazy rescale 又会按 2^τ 膨胀发射值——发射域必须显式设计（27.3.4–27.3.6），否则不是精度差，而是直接溢出。

理想目标是**即插即用的无损量化**（plug-and-play PTQ）：不改模型权重、不需要校准数据集，attention 输出与 fp16 基线在端到端指标上不可区分。SageAttention 系列与 ffpa-attn 的全部设计都围绕这一目标。

34.3 数学原理

34.3.1 量化算子与记号

全书采用**对称、无零点**（symmetric, zero-point-free）的量化形式。对张量块 $\mathcal{B}$（定义域见 27.3.3 的粒度谱系），量化与反量化算子为

$$\hat{x} \;=\; \mathrm{clamp}_{[-R,R]}\Big(\mathrm{round}\big(x/\delta\big)\Big), \qquad \tilde{x} \;=\; \hat{x}\cdot\delta, \tag{34.2}$$

其中 R 是目标格式的满量程整数（int8 取 127，e4m3 取 448，e2m1 取 6），$\delta = \mathrm{amax}(\mathcal{B})/R$ 是块步长（amax 为块内绝对值最大值）。$\tilde{x}$ 与 x 的差就是量化误差；round 取最近邻（RN）。注意三点：

- $\mathrm{amax}(\mathcal{B}) = 0$（全零块，如 padding 列）时 $\delta = 0$，式 34.2 出现 0/0——工程上必须显式防护（ffpa 对 SF 域取 $\mathrm{SF} = 0 \Rightarrow \mathrm{SF}^{-1} = 0$，见 27.8 节坑一）；
- clamp 保证 $\hat{x}$ 落在格式可表示范围内，但 **clamp 不是精度保证**：需要靠 clamp 截回可表示范围的值本身已携带大误差（satfinite 指令同理，27.3.6）；
- scale（δ）本身要以更高精度（fp32）存储与传递——这决定了后面所有「scale 折叠」操作都以 fp32 标量进行。

34.3.2 数值格式：从均匀到指数

int8：均匀网格。 int8 是 $[-127, 127]$ 的**等距**整数网格。误差只依赖步长 δ：均匀量化噪声在块内近似均匀分布 $[-\delta/2, \delta/2]$，RMS 误差为 $\delta/\sqrt{12}$，与 x 的幅度**无关**。这带来一个与指数格式相反的特性：绝对误差有均匀上界（小值区域如 $|S|$ 接近零的分数区，误差绝对值小）。SageAttention 1 选择 int8 而非 fp8 做 QK，首要理由是速度（RTX4090/3090 上 int8 MMA 比 fp16 快 4 倍、比 fp8 快 2 倍）；精度上，Q/K 分数分布集中时，int8 的 254 个均匀码点密度高于 fp8 每倍频程 $2^m = 8$ 个码点，量化精度更高（arXiv:2410.02367）。

e4m3 / e2m1：指数格式与相对步长。 e4m3 用 1 位符号、4 位指数（bias 7）、3 位尾数；最大有限值是 $1.75 \times 2^8 = 448$（指数全 1 时仅 $M = 111$ 编码 NaN，其余可用）。e2m1 只有 1 位尾数，全部正值集合是 $\{0.5, 1, 1.5, 2, 3, 4, 6\}$，总共 16 个码点，动态范围仍横跨约 3.6 个二进制倍频程。指数格式的量化误差有明确的上界：

定理 34.1 (指数格式的相对步长)**.** 设格式有 m 位尾数、规格化数 $x \in [2^e, 2^{e+1})$。则 x 所在的 *ULP*（相邻可表示值间距）为 2^{e-m}，最近邻舍入误差 $\le \frac{1}{2}$ULP，故相对误差上界为

$$\frac{|\tilde{x} - x|}{|x|} \le \frac{2^{e-m-1}}{2^e} = 2^{-(m+1)}. \tag{34.3}$$

证明. 规格化浮点在 $[2^e, 2^{e+1})$ 内等距分布，间距为尾数最低位的权值 2^{e-m}（m 位尾数把该倍频程切成 2^m 格）。最近邻舍入误差至多半个格：$2^{e-m}/2 = 2^{e-m-1}$。除以区间下界 2^e（区间内最小可能幅度，取最坏情况）即得式 34.3。 □

代入 m：e4m3（m=3）相对误差 $\le 2^{-4} = 6.25\%$，e2m1（m=1）$\le 2^{-2} = 25\%$。注意结论与 e **无关**，「指数格式相对误差恒定」指的就是这一性质：无论值大值小，误差都是自身幅度的固定百分比。它与 int8 的「绝对误差恒定」形成对偶，两者的适用面也因此分化：值域分布横跨多个倍频程的量（$Q/K/V$ 的元素）适合指数格式；已知幅度紧凑的整数域乘积（$\hat{Q}\hat{K}^\top$ 的 s32 累加）两者皆可。

ue4m3 / ue8m0：无符号指数格式（scale 的格式）。 NVFP4 的 per-16 block scale 用**无符号** e4m3（ue4m3，$2^{-6} \sim 448$），MXFP8 的 per-32 scale 用纯指数 ue8m0（$2^{\pm 127}$）。scale 本身的量化同样服从定理 34.1，但注意 scale 是**乘性**作用于数据的：$\tilde{x} = \hat{x} \cdot \widetilde{\mathrm{SF}}$，两级相对误差相加。第 三十八 章「两级 P 量化」要显式核算的正是这部分误差。

表 34.1: 本书涉及的量化数值格式速查（m 为尾数位数）

格式	表示范围	m	相对步长	典型用途	出处
int8（对称）	±127，均匀	—	绝对 $\delta/\sqrt{12}$（RMS）	fp8 路径 QK（`int8` 变体）	SA1
e4m3	±448，非均匀	3	$\approx 2^{-4}$（6.25%）	Q/K/V/P（fp8 路径）	SA2
e2m1	±6，非均匀	1	$\approx 2^{-2}$（25%）	NVFP4 数据（Q/K/V/P）	SA3
ue4m3	$2^{-6} \sim 448$，无符号	3	$\approx 2^{-4}$	NVFP4 per-16 SF	SA3
ue8m0	$2^{\pm 127}$，纯指数	0	—	MXFP8 per-32 SF	MX 系列

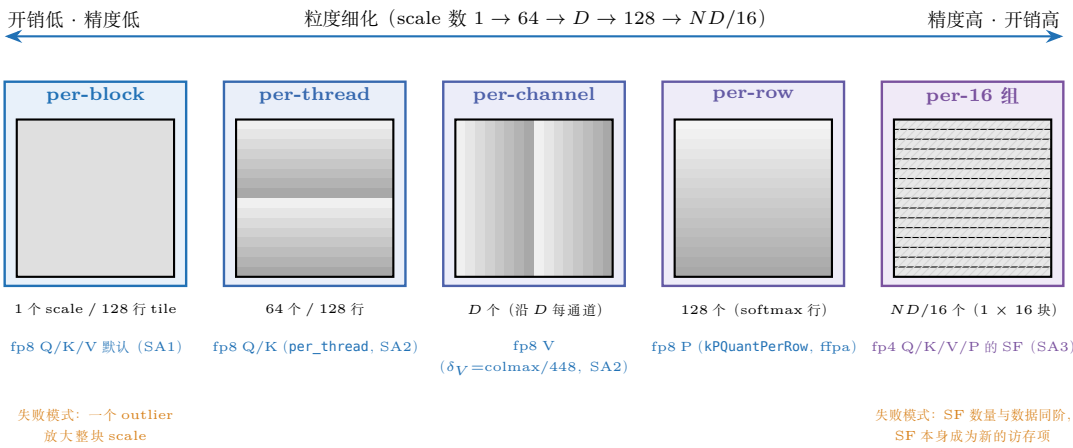
34.3.3　量化粒度谱系

δ 的定义域 $\mathcal{B}$（式 34.2）就是**粒度**。粒度越细，每个 scale 管的元素越少、分辨率越高，但 scale 的存储/传递/消去开销越大；且粒度必须与 MMA 的 fragment 布局对齐，否则每条 scale 都要跨 lane 归约（第 三十五 章的主题）。ffpa-attn 实际支持的谱系如下：

表 34.2: 量化粒度谱系（ffpa `CUDABackend` knob → kernel 行为）

粒度	定义域 $\mathcal{B}$	ffpa 实例	出处
per-block	128 行 tile（kBr/kBc）	fp8 Q/K/V 默认	SA1
per-thread	MMA fragment 对齐	fp8 Q/K（`per_thread`）	SA2
per-channel	沿 D 每通道、跨 N 取 amax	fp8 V（$\delta_V = \mathrm{colmax}/448$）	SA2
per-row	softmax 行	fp8 P（`kPQuantPerRow`）	ffpa
per-16 组	NVFP4 1×16 microscaling block	fp4 Q/K/V/P 的 SF	SA3

两条工程要点贯穿 Part VI：其一，per-thread 粒度之所以存在，是因为 SA2 观察到 scale 与 `mma.m16n8k16` fragment 的 thread-data 映射对齐后可以**零 shuffle** 使用（每 lane 持有的 8 个 A 元素共享一个 scale，scale 直接乘进 exp2 的行常数）。粒度因此是「精度 × fragment 布局」的联合设计，不单是一个精度旋钮。其二，粒度谱系的两端有明确的失败模式：per-block 对 outlier 最脆弱（一个 outlier 即影响一整块），per-row/per-16 的 scale 数量与数据同阶（SF 本身成为新的访存项，第 三十八 章核算这笔开销）。图 34.1 用五个同形状微网格把整条谱系的「共享域」排成一行：色块划分越细，每个 scale 覆盖的元素越少。



读法（灰度 = 同一个 scale 的覆盖域）：per-thread 的行 r 与 $r{+}8$ 同色——行对 $\{r, r{+}8\}$ 共享一个 scale，恰与 $m16n8$ C-fragment 的行对对齐（第 三十五 章 28.3.4 节展开的 64 槽位表）；per-row 为 16 档线性渐变示意 128 行；per-16 每行一个 1×16 microblock（细纹示意块内 16 元素共享一个 SF）。

图 34.1: 量化粒度谱系五列网格（表 34.2 的视觉化，缩略网格为 16 行 × 16 列）。从左到右：per-block 整块共享 1 个 scale；per-thread 按 C-fragment 行对 $\{r, r+8\}$ 分档（128 行块共 $8 \times 8 = 64$ 个，零 shuffle 消费）；per-channel 沿 D 每通道一个；per-row 每 softmax 行一个（128 个）；per-16 每行一个 1×16 microblock（NVFP4 的 SF）。顶部双向轴是整条谱系的 trade-off：粒度越细精度越高，但 scale 的存储、传递与消去开销同步上涨，且必须与 MMA 的 fragment 布局对齐——谱系两端的失败模式分别是对 outlier 的脆弱性与 SF 自身的访存开销。

34.3.4 attention 的 scale 流向总览

现在把三个量化点的 scale 因子串成一条链。设 Q, K, V 已按上述粒度量化，scale 分别为 δ_Q, δ_K, v_s；P 的量化 scale 记 p_{scale}。目标是：**任何时刻都不做显式的反量化 pass**，所有 scale 要么代数消去、要么折进本来就存在的标量位置。

量化点 1（QK）：$\delta_Q\delta_K$ 折进 exp2 系数。 MMA 算出的是未缩放域（quantized domain）的 $\hat{S} = \hat{Q}\hat{K}^\top$，真实分数 $S = \hat{S} \cdot \delta_Q\delta_K$。softmax 需要的是 $\exp(\text{scale} \cdot S)$（scale 即 $1/\sqrt{d}$），换到 exp2 域：

$$\exp_2\Big(\underbrace{\text{scale} \cdot \delta_Q\delta_K \cdot \log_2 e}_{s_{dequant}\text{：折叠为单个 fp32}} \cdot \hat{S} \Big), \tag{34.4}$$

即两个块 scale 的乘积与 softmax 常数、$\log_2 e$ 融合成**一个** fp32 系数，在 tile max 归约后一次乘入（配合 27.4 节的 `kMaxScaleAfter`：归约直接用原始 $\hat{S}$，scale 乘在归约结果上，每行省 B_c 次 FMUL）。int8 变体（Q/K 走 `s8xs8→s32` MMA）只改 $\hat{S}$ 的类型，式 34.4 不变。

量化点 3（PV）：v_s 精确消去。 V 已离线量化为 $\hat{V} = V/v_s$（驻留 gmem）。若 P 的量化把 v_s 乘进自己：

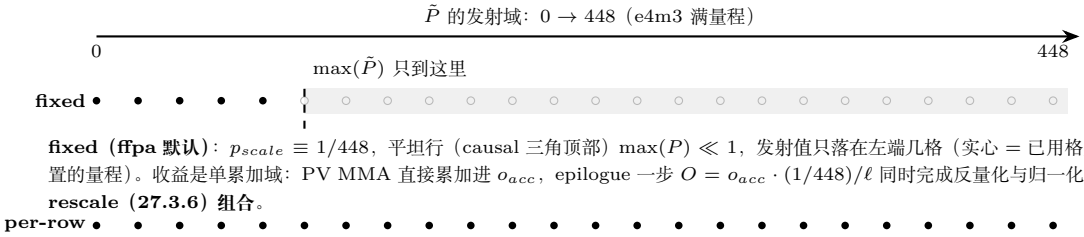
$$\hat{P} = \text{round}\Big(\frac{P \cdot v_s}{p_{scale}}\Big), \qquad \text{MMA} = \hat{P}\hat{V} = \frac{P\,v_s}{p_{scale}} \cdot \frac{V}{v_s} = \frac{PV}{p_{scale}}, \tag{34.5}$$

v_s **精确**消去（不是近似）：gmem 中预量化的 $\hat{V}$ 原样复用，v_s 只出现在 P 侧的发射系数里。

量化点 2（P）：p_{scale} 收尾。 最后一步 $O \mathrel{+}= \text{MMA} \cdot p_{scale} = PV$ 把 p_{scale} 乘回输出。关键在 p_{scale} 的**取值策略**决定整个 kernel 的寄存器开销结构：

- **fixed 模式**：$p_{scale} \equiv 1/448$（编译期常数，依据 $\max(P) \le 1$）。累加器 o_{acc} 全程保持在同一个「$/p_{scale}$」域里，PV MMA 可以**直接累加进** o_{acc}，epilogue 一步 $O = o_{acc} \cdot (1/448)/\ell$ 同时完成反量化与 softmax 归一化。代价：平坦行（$\max(P) \ll 1$，causal 三角顶部）只用满量程的一小部分。
- **per-row 模式**：$p_{scale}[row] = \text{rowmax}(P)/448$，每行铺满 e4m3 满量程，精度最优。代价：跨 lane rowmax 归约（两次 `__shfl_xor`）+ per-row 反量化需要 64-float o_{tile} 暂存（寄存器压力、每 KV tile 多一次乘加 pass），且与 lazy rescale 互斥（27.3.6）。

ffpa 默认 fixed（FFPA_FP8_PQUANT_PER_ROW 可开 per-row）；发射域 $\tilde{P} = P \cdot v_s \cdot 448$ 与溢出约束的完整推导见第 三十六 章。本节只需记住结论：**scale 折叠使 fp8 路径除量化本身的 round() 外没有任何额外数学误差源，所有反量化都是精确代数操作**。图 34.2 把两种 p_{scale} 策略的发射域利用率摆在同一条量程轴上：差别就在平坦行用了多少格点。



fixed（ffpa 默认）：$p_{scale} \equiv 1/448$，平坦行（causal 三角顶部）$\max(P) \ll 1$，发射值只落在左端几格（实心 = 已用格点，浅带 = 闲置的量程）。收益是单累加域：PV MMA 直接累加进 o_{acc}，epilogue 一步 $O = o_{acc} \cdot (1/448)/\ell$ 同时完成反量化与归一化，且**可与 lazy rescale（27.3.6）组合**。

per-row：$p_{scale}[row] = \text{rowmax}(P)/448$，每行归一后铺满满量程，精度最优；代价是跨 lane rowmax（两次 __shfl_xor）+ 64-float o_{tile} 暂存（每 KV tile 多一次乘加 pass），且**与 lazy rescale 互斥**。FFPA_FP8_PQUANT_PER_ROW 打开 per-row。

格点是 e4m3 可表示值的示意（真实格点非均匀：近 0 更密、指数越大越疏），两组各 24 格只为把「量程利用率」画出来；两种模式的乘积预算与溢出约束见式 (34.7) 与图 34.4。

图 34.2: fixed 与 per-row 两种 p_{scale} 策略的发射域利用率（27.3.5 节）。上：fixed 的 $p_{scale} \equiv 1/448$ 是编译期常数，平坦行的 $\max(P) \ll 1$ 使发射值只占量程左端一小段，其余量程闲置——换来的是单累加域与「可与 lazy rescale 组合」；下：per-row 用 $\text{rowmax}(P)/448$ 使每一行铺满量程，精度最优，代价是跨 lane rowmax 归约与 64-float 的 o_{tile} 暂存，且与 lazy rescale 互斥。

34.3.5　SageAttention 谱系：1 → 2 → 2++ → 3

把上述数学元件按论文时间线装配，就得到 SageAttention 四代。表中「贡献」列是每一代相对前代的新增；「ffpa 对应」列指出 ffpa-attn 中的 knob（后续章节逐一展开）：

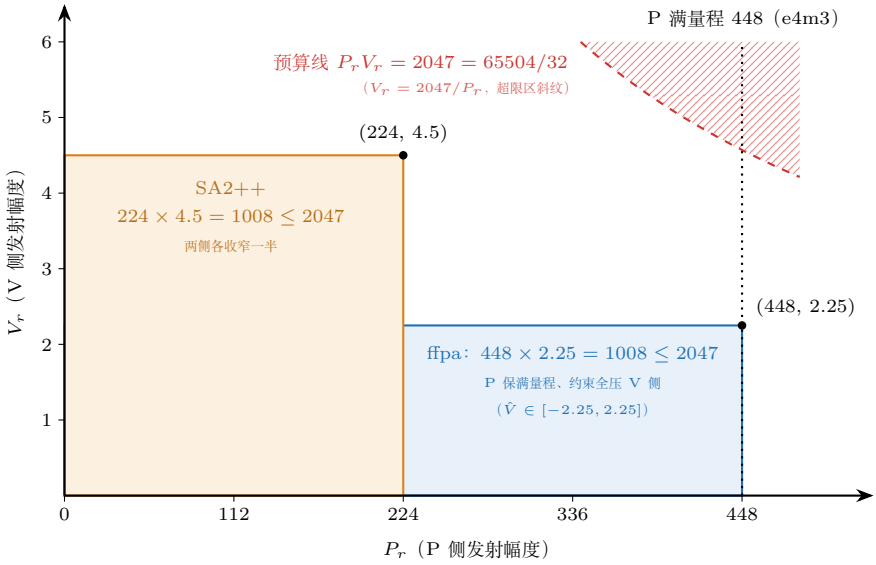
表 34.3: SageAttention 谱系与 ffpa-attn 对应（knob 名省略 fp8_/fp4_ 前缀）

代	QK / PV 格式	关键粒度	贡献	ffpa 对应
SA1	int8 / fp16 (f16 acc)	per-block	smooth-K（softmax 平移不变）；fused RoPE+quant	qk_mm_type=int8、smooth_k
SA2	int4（实现含 int8 变体）/ fp8	per-thread (QK)、per-channel (V)	smoothing 系统化；per-thread 对齐 fragment	per_thread、per_channel
SA2++	同 SA2 / fp8-f16acc	同 SA2	fp16 累加器域约束 $P_r V_r \le 2047$；吞吐翻倍	pv_acc_type=f16
SA3	FP4 / FP4 (MXFP8 变体)	per-16 microscaling	两级 P 量化；delta_s rank-1 修正；RTX5090 1038 TOPS	cuda_impl=fp4 全链

脉络：**SA1 证明可行 → SA2 下探格式与粒度 → SA2++ 换 2 倍 MMA 并推导值域 → SA3 进 Blackwell FP4**。四代的共同基础就是本章 27.3.1–27.3.4 的算子、格式、粒度与 scale 折叠；分歧只在每个量化点选什么格式、什么粒度。表中的 smooth-K/smoothing 不变式与 delta_s 修正是第 三十五 章（fp8）与第 三十八 章（fp4）的主要内容；SA2++ 的 f16 累加域约束推导如下。

SA2++ 的 f16 累加器域约束。 mma.f16.f8.f8.f16 的 fp16 累加器可表示 ±65504；m16n8k32 的 K 深度是 32，若逐元素乘积幅度均匀，则每个元素需满足 $P_r \times V_r \le 65504/32 \approx 2047$。SA2++ 论文两侧各收窄一半取 $P_r = 224$、$V_r = 4.5$（$224 \times 4.5 = 1008 \le 2047$）；**ffpa 的分配不同**：P 侧保持 448 满量程（fixed 发射 $\tilde{P} = Pv_s448$），把

约束全部落在 V 侧：per-channel V + f16 acc 时发射 $\hat{V} \in [-2.25, 2.25]$，检验 $448 \times 2.25 = 1008 \le 2047$ 同样成立（`cute/fp8/smooth_v.cuh` 注释，第 三十五 章展开）。两种分配的乘积预算相同，差别在「牺牲 P 分辨率还是 V 分辨率」，ffpa 选择保 P（softmax 概率的相对结构更敏感）。图 34.3 在 P_r–V_r 平面上把这两种分配画成同面积矩形：预算线之下是 f16 累加器的安全域。



约束来源：`mma.f16.f8.f8.f16` 的 f16 累加器 ±65504、`m16n8k32` 的 K 深度 32 ⇒ 逐元素 $P_r \times V_r \le 2047$。

两种分配面积相同（1008），差别在牺牲 P 还是 V 的分辨率——ffpa 保 P：softmax 概率的相对结构更敏感。

图 34.3: f16 累加器域的乘积预算（27.3.5 节）：横轴 P_r、纵轴 V_r，虚线为预算线 $P_rV_r = 2047$（$= 65504/32$，K 深度 32），线上方斜纹区为超限区。SA2++ 取 $(224, 4.5)$、ffpa 取 $(448, 2.25)$，两者面积同为 $1008 \le 2047$——一个矩形高瘦（两侧各收窄一半），一个扁宽（P 保持 448 满量程、约束全部落在 V 侧，per-channel V + f16 acc 时 $\hat{V} \in [-2.25, 2.25]$）。点线为 e4m3 满量程 448 的参考线：ffpa 的 P_r 恰好达到它。

34.3.6 FA-4 lazy rescale 与量化域的交互

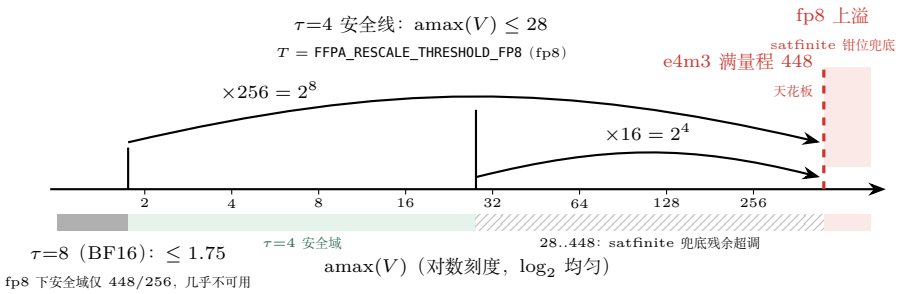
第 五 章的 online softmax 每 tile 都把 O 与 ℓ 重缩放到新 max。FA-4（arXiv:2603.05451）的观察：max 的增长通常很小，重缩放可以**懒**做：当新 max 相对旧 max 的增长不超过阈值 τ（log2 域）时，跳过 rescale，让 O, ℓ 停留在旧域，softmax 用 stale max 发射：

$$\text{skip} \iff m_{\text{new}} - m_{\text{old}} \le \tau \iff \underbrace{m_{\text{old}} - m_{\text{new}}}_{\text{log2 域，} \le 0} \ge -\tau. \tag{34.6}$$

端到端无损：O 与 ℓ 用同一个 stale max，epilogue 的 O/ℓ 把所有迟到的缩放一并消去（等价性见 27.4 节代码注释）。fp16 域中 lazy rescale 是纯粹的省指令优化；**量化域**中它多了一层含义：发射值被 2^τ 膨胀，必须核算格式上限。fixed 模式发射 $\tilde{P} = P \cdot v_s \cdot 448$，而 per-block V 量化有 $v_s = \text{amax}(V)/448$，故发射峰值 $\tilde{P}_{\max} = v_s \cdot 448 = \text{amax}(V)$，溢出条件与安全域为

$$\underbrace{2^\tau}_{\text{lazy 膨胀}} \cdot \underbrace{v_s \cdot 448}_{=\,\text{amax}(V)} = 2^\tau \,\text{amax}(V) \le 448 \implies \text{amax}(V) \le 448/2^\tau. \tag{34.7}$$

ffpa 取 $T =$ `FFPA_RESCALE_THRESHOLD_FP8` $= 4$（FA-4 原文的 BF16 kernel 典型取 $\tau = 8$；fp8 发射域的上限更紧，按 e4m3 满量程反推出更小的 4，源码注释把它归因为"FA-4's fp8 choice"，但 FA-4 论文并无 fp8 τ 的表述），得 $\text{amax}(V) \le 28$ 的适用域，残余超调由 `satfinite` 钳位处理（27.8 节坑三）。这也解释了 27.3.4 的一个断言：**per-row 满量程模式与 lazy rescale 互斥**，满量程发射已经没有 headroom，任何 2^τ 膨胀都会直接溢出。图 34.4 把这条安全域计算画到一根对数轴上：两个 τ 的膨胀倍率都指向同一个 448 上限。



发射峰值 $\tilde{P}_{\max} = v_s \cdot 448 = \text{amax}(V)$（per-block V 的 $v_s = \text{amax}(V)/448$），溢出条件 $2^\tau \text{amax}(V) \le 448$（式 (34.7)）。

τ 的取值跨格式不能搬用：BF16 天花板高（τ=8 可用），e4m3 满量程只有 448，反推出更紧的 τ=4；满量程发射（per-row）没有任何 headroom，与 lazy rescale 互斥。

图 34.4: lazy rescale 的 2^τ 膨胀 vs e4m3 上限（27.3.6 节，式 (34.7)）。横轴为 amax(V) 的对数刻度：lazy 跳过 rescale 让发射值停留在旧域，等效膨胀 2^τ 倍，必须核算格式上限 448。ffpa 取 $T = 4$（×16），安全域 amax(V) ≤ 28；FA-4 BF16 kernel 的典型 $\tau = 8$（×256）在 fp8 下安全域只剩 1.75，几乎不可用——τ 跨格式不可直接搬用。28 与 448 之间的残余超调由 `satfinite` 钳位（27.8 节坑三）；448 右侧为 fp8 上溢区（红色警示）。

34.3.7　ESS 误差模型：causal 早行的绝对误差放大

量化把每一级计算（QK / P / V / PV）的相对误差约束在 $\varepsilon \approx 2^{-(m+1)}$（定理 34.1）。误差的**绝对值**还取决于输出的**幅度**，而 attention 输出的幅度在行间差异很大。设 V 的行 i.i.d. $\sim \mathcal{N}(0, \sigma_V^2)$，考察输出行 $O_i = \sum_j P_{ij} V_j$ 的第 d 维：

$$\text{Var}(O_{i,d}) = \sum_j P_{ij}^2 \,\text{Var}(V_{j,d}) = \sigma_V^2 \sum_j P_{ij}^2 = \frac{\sigma_V^2}{\text{ESS}_i}, \qquad \text{ESS}_i := \Big(\sum_j P_{ij}^2\Big)^{-1}, \tag{34.8}$$

ESS_i 是「有效样本数」：概率越集中（one-hot），ESS 越小、输出越像单个 V 行（幅度 $\sim \sigma_V\sqrt{2\ln D}$）；概率越平均，输出越像大量样本的均值（幅度 $\sim \sigma_V/\sqrt{N}$）。取 D=128、σ_V=1 估算输出幅度 $\approx \sigma_V\sqrt{2\ln D}/\sqrt{\text{ESS}} \approx 3.1/\sqrt{\text{ESS}}$：

表 34.4: ESS 与输出幅度、量化绝对误差（$\varepsilon = 5\%$，e4m3）

行类型	P 分布	ESS	幅度	绝对误差 $\approx \varepsilon\times$ 幅度
dense（randn logits）	集中、非均匀	∼3000	∼0.05	0.003
causal row[2]	3 个 KV 上分布	∼3	∼1.8	0.09
causal row[0]	one-hot e_0	1	∼3.1	∼0.16

因果掩码把序列前几行变成近 one-hot：row[0] 只能看自己。ffpa 的 PyTorch 数值仿真（$B1H32N8192D128$，randn）实测 causal `max_abs` = 0.22 vs dense 0.015，相差 15 倍，而**相对误差两者同量级**（causal 5.5% vs dense 9.1%，口径为 max_abs / 输出行幅度中位数）。结论：**早行放大的是幅度而非误差率，相对误差各行一致**。推论直接决定工程路线：降低幅度没有意义（输出本来就该大）；要么全链降 ε（fp16），要么对早行整体用 fp16，这构成 hybrid 行分割（第 三十六 章）的理论依据，且 n_{early} 不需要很大（ESS 极小的行集中在前段）。

34.3.8　per-stage 误差分解与工程结论

式 34.8 的 ε 是单级贡献。整条链有四个独立误差源（QK 量化、P 量化、V 量化、PV MMA 域），各自独立贡献 $\varepsilon\times$ 幅度。ffpa 的仿真分解给出排序（内部仿真口径，未随书归档）：

$$\text{V 量化 } (0.19) \;>\; \text{QK 量化 } (0.13) \;>\; \text{P 量化 } (0.11), \tag{34.9}$$

「只把早行的 QK 换回 fp16」并不能修复 causal 精度：V 量化是最大单项且保持不变；早行必须**全链 fp16**（hybrid），或者用 per-channel V scale 降低 V 的相对误差（第 三十五 章 smooth-V + per-channel 的组合方案）。fp4 的同源分析见第 三十八 章：ε 大一个量级（e2m1 25%），剩余长尾由 V 量化主导、与行号无关，加大 n_{early} 无法消除。

34.4 设计与实现

本节把 27.3.4 的 scale 代数落到 ffpa-attn 的两处源码：P 量化的三步协议（`fp8_pscale.cuh`）与 log2 域 lazy rescale（`softmax.cuh`）。

34.4.1 P 量化的三步协议

`fp8_pscale.cuh` 的头注释把 27.3.4 的量化点 2/3 浓缩成三行公式与一条消去证明，是全书引用密度最高的一段注释。它先回答「P 为什么需要 runtime scale」：Q/K/V 都已离线量化（`q_scale/k_scale/v_scale`），而 $P = \mathrm{softmax}(S)$ 是每个 forward、每个 tile 都不一样的瞬态 fp32 概率矩阵，只能在 kernel 内就地量化：$\mathrm{P8} = \mathrm{round}(P \cdot v_s / p_{\mathrm{scale}})$ 折叠发射，MMA = P8@V8 做 fp8 PV，$O \mathrel{+}= \mathrm{MMA} \cdot p_{\mathrm{scale}}$ 回到真实值域，v_s 全程精确消去。溢出安全一节预先给出式 34.7：lazy rescale 按 2^T 膨胀发射值，T = `FFPA_RESCALE_THRESHOLD_FP8` = 4 时 $2^T \cdot \mathrm{amax}(V) \le 448$ 对 $\mathrm{amax}(V) \le 28$ 成立，残余超调交给 `satfinite` 钳位。

三行注释与式 34.5 逐条对应：式 (1) 把 v_s 折进 P 的发射系数；式 (2) 的关键是 $\hat{V} = V/v_s$ **已在 gmem**（离线量化点 3 的产物）；式 (3) 只剩 p_{scale}。溢出段即式 34.7 的源码版。

34.4.2 两种 p_{scale} 模式的发射路径

ffpa-attn cute/fp8/fp8_pscale.cuh L111-138（quantize_p_frag：per-row / fixed 双模式 + 就地 downcast）

```
CUTE_DEVICE void quantize_p_frag(ScoresTensor& scores, Fragment& tCrS, float vs,
                                 const float* p_scale, Reorg& reorg) {
  constexpr int kRows = decltype(cute::size<0>(scores))::value;
  constexpr int kSVals = decltype(cute::size(tCrS))::value;
  if constexpr (kPQuantPerRow) {
#pragma unroll
    for (int row = 0; row < kRows; ++row) {
      const float mul = (p_scale[row] == 0.0f) ? 0.0f : vs / p_scale[row];
#pragma unroll
      for (int col = 0; col < cute::size<1>(scores); ++col)
        scores(row, col) *= mul;
    }
  } else {
    const float mul = vs * kE4m3Max;
#pragma unroll
    for (int row = 0; row < kRows; ++row)
#pragma unroll
      for (int col = 0; col < cute::size<1>(scores); ++col)
        scores(row, col) *= mul;
  }
  float* f32 = tCrS.data();
  __nv_fp8_e4m3* p8 = reinterpret_cast<__nv_fp8_e4m3*>(f32);
#pragma unroll
  for (int i = 0; i < kSVals; ++i)
    p8[i] = __nv_fp8_e4m3(f32[i]);
  auto p8_frag = cute::make_tensor(
      cute::make_rmem_ptr(p8), cute::Layout<cute::Shape<cute::Int<kSVals>>>{});
  reorg(p8_frag);
```

三个要点：

1. **存储别名（alias）**：fp32 C-fragment 的寄存器内存被就地重解释为 4 倍密度的 e4m3 A-fragment（downcast 循环 + `make_tensor` 重建视图），量化前后**零额外寄存器**——前提是必须**先**调用本函数 **再**把 fragment 当 PV 的 A 操作数；
2. per-row 分支的除法带 $p_{scale} = 0$ 防护（全零块，27.8 节坑一）；fixed 分支把除法变成乘常数 $v_s \cdot 448$；
3. `reorg` 参数（函数末尾调用）是 fragment 布局修正钩子——QK 的 C-fragment 到 PV 的 A-fragment 存在布局错位，ffpa 的 `ReorgC8bitToA8bit` 用寄存器内重排（无 SHFL）解决，这是第 三十六 章的核心内容（归约轴

置换不变性）。

fixed 模式还有一条快路径：softmax 直接发射 $\tilde{P} = P \cdot v_s \cdot 448$（把 $\log_2(v_s \cdot 448)$ 折进 exp2 的 shift，见 `online_softmax_fp8_fixed`），量化只剩「downcast + reorg」，且 downcast 用 `cvt.rn.satfinite.e4m3x2.f32` **成对**转换（一条指令转两个值，CVT 数减半）：

```
// online_softmax_fp8_fixed 的发射域（与 quantize_p_frag<false> 等价）：
//   exp_offset = log2(vs*448)
//   exp2(s - max + exp_offset) = exp2(s - max) * vs * 448 = P*vs*448
uint16_t packed;
asm("cvt.rn.satfinite.e4m3x2.f32 %0, %1, %2;" : "=h"(packed)
    : "f"(f32[2i+1]), "f"(f32[2i]));   // 一条指令、两个 e4m3
```

34.4.3　log2 域的 lazy rescale

`softmax.cuh` 的 `online_safe_softmax` 是全家族（fp16/fp8/fp4）共享的 softmax 内核，FA-4 条件重缩放的核心 20 行：

ffpa-attn cute/softmax.cuh L44-63（lazy rescale：skip 判定与 stale max 发射）

```
      tile_max = fmaxf(tile_max, scores(row, col) * scale);
    tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
    tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
    const float next_max = fmaxf(row_max[row], tile_max);
    // log2_diff == FA-4 acc_scale_: already in log2 domain, no * scale_log2.
    const float log2_diff = row_max[row] - next_max;
    float eff_max = next_max;
    if (rescale_threshold > 0.0f && log2_diff >= -rescale_threshold) {
      row_scale[row] = 1.0f;
      eff_max = row_max[row];  // stale max; row_max NOT updated
    } else {
      row_scale[row] = exp2f(log2_diff);  // exp(<0) -> scale < 1.0
      row_max[row] = next_max;
    }
    float tile_sum = 0.0f;
#pragma unroll
    for (int col = 0; col < cute::size<1>(scores); ++col) {
      const float p = exp2f(scores(row, col) * scale - eff_max);
      scores(row, col) = p;
      tile_sum += p;
```

逐行对照式 34.6：前两行计算 m_{new} 与 log2 域差值（`log2_diff`，恒 ≤ 0）；中间是 skip 分支：`row_scale = 1`（O, ℓ 不缩放）、`eff_max = row_max`（**stale** max 发射 P）、`row_max` 不更新；`else` 是正常 rescale 分支。细读文件头注释对符号方向的说明（与 FA-4 的 $acc_scale \ge -\tau$ 判定同构）：论文公式映射到 log2 域实现时最容易出错的就是符号方向。等价性的最后一步在 epilogue：O 与 ℓ 全程共享同一 stale 域，O/ℓ 消去全部迟到缩放。

34.5　图示

三张大图对应本章的三条主线。图 34.5 是精度阶梯的位域对照：int8 的均匀网格与 e4m3/e2m1/ue4m3 的浮点结构并排展开，相对步长定理说明「尾数位决定相对误差」。图 34.6 把全章的 scale 流向画进一张数据流图：$\delta_Q \delta_K$ 折进 exp2 系数、v_s 在 P 的发射域消去、p_{scale} 收在 epilogue。图 34.7 是 ESS 误差模型：dense 行与 causal 早行的 P 分布/误差对比，以及 $V > QK > P$ 的 per-stage 误差贡献分解——「早行必须全链 fp16」的结论就来自右半的条形分解图。

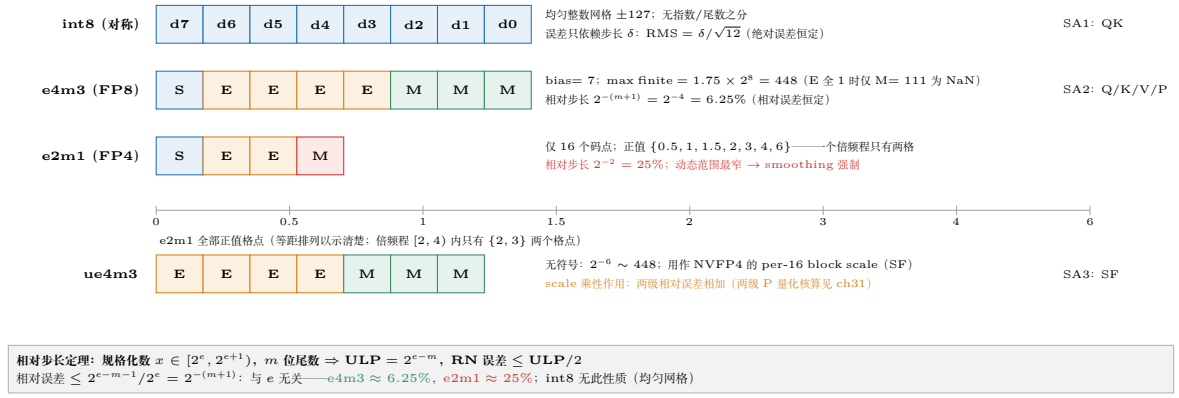


图 34.5: 四种量化格式的位域与值域对照（int8 / e4m3 / e2m1 / ue4m3）。e4m3 的指数全 1 时仅 M=111 编码 NaN，故满量程 448；e2m1 全部 16 个码点在数轴上标出，直观呈现「一个倍频程内只有两格」。

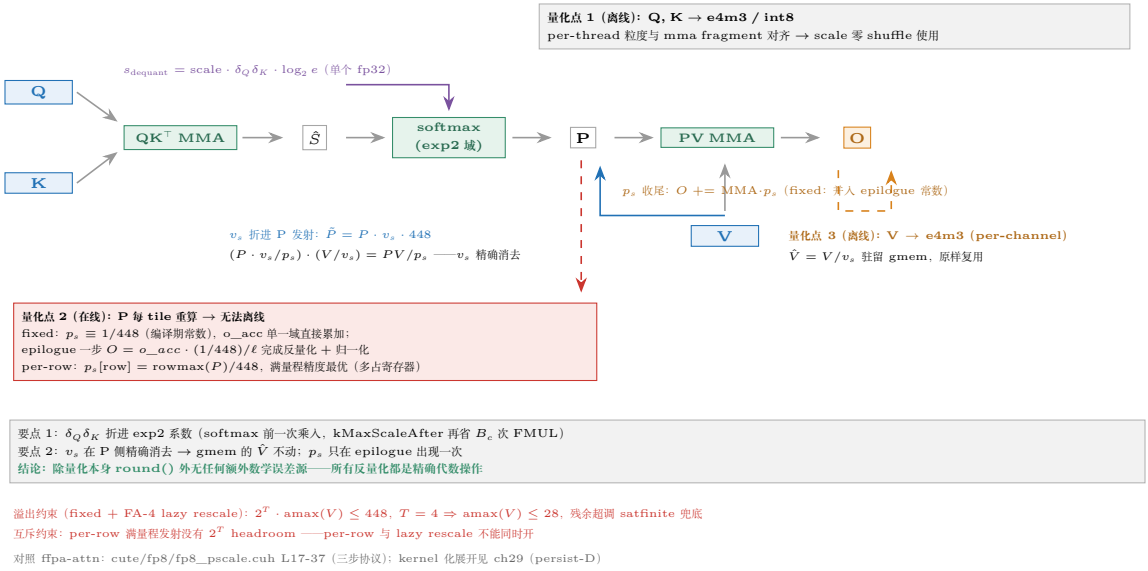


图 34.6: attention 的三个量化点与 scale 流向：$\delta_Q\delta_K$ 折进 exp2 系数（式 34.4）、v_s 在 P 侧精确消去（式 34.5）、p_{scale} 在 epilogue 一步收尾。

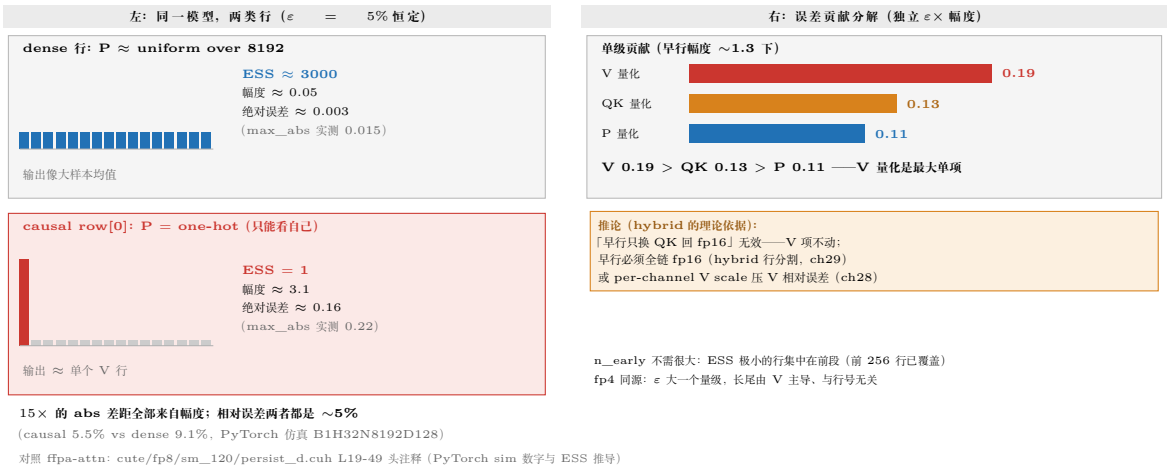


图 34.7: ESS 误差模型与 per-stage 误差分解：左半为 dense 行与 causal 早行的 P 分布/ESS/幅度对比（表 34.4 的图形化），右半为 V > QK > P 的误差贡献分解与「早行需全链 fp16」的结论。

34.6　性能分析

量化的收益上限由 27.2.1 的精度阶梯给出；实际达成率取决于两个非理想因素：**量化前处理链的开销**（第 三十五 章：kv_mean / quantize / 转置链的带宽需求，理想情况可用满 HBM 带宽、E2E 占比个位数百分比）与 **softmax/CVT 等非 MMA 指令的占比**（第 三十六 章：persist-D 的流水线设计让 tensor pipe 占比达 ~77%）。本机（RTX PRO 5000 72GB Blackwell，sm_120）以 `ffpa_attn.bench` 实测 fp8 路径基线：

```
$ FFPA_CUDA_ALLOW_SMALL_D=1 CUDA_VISIBLE_DEVICES=7 \
  python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
  --tasks self-attn --D 128 --no-bwd --show-allclose
| self-attn | fp16 | 8192/8192 | allclose OK | 3.01 / 5.12 ms | 365T / 215T | 1.70x |
```

即 fp8 全链（含前处理）相对 SDPA fp16 加速 1.70×、折合 365 TFLOPS （同形状 fp16 kernel 的两倍量级，与 e4m3 吞吐阶梯一致）。D 与任务维度的完整矩阵、knob 扫描与 5090 对照见第 四十 章。

34.7　实践经验

- **先定发射域，再写 kernel**：fp8/fp4 kernel 的第一份设计草图应该是「每个 fragment 生命周期的值域区间图」（$P \cdot v_s \cdot 448 \cdot 2^{\tau}$ 类推），所有格式选择从这张图反推——本章 27.3.4–27.3.6 的顺序即是如此。
- **粒度对齐 fragment 是零成本精度**：per-thread/per-channel 的收益不只是「更细」，而是 scale 恰好落在每 lane 私有或每行常数的零开销位置（式 34.4 折叠的前提）。
- **误差分析要分幅度与相对两层**：见到的「causal 精度差 15 倍」先查相对误差是否一致（式 34.8 的 ESS 口径），再决定是降 ε 还是分割行。

34.8　常见坑

1. **全零块的 0/0 NaN**：amax $= 0 \Rightarrow \delta = 0$，式 34.2 的除法产生 NaN。fp4 路径对 SF 域显式防护（SF $= 0 \Rightarrow \text{SF}^{-1} = 0$）；fp8 的 per-row p_{scale} 同样带零检查（27.4 节）。head_dim padding 列是全零块最常见的来源。
2. **per-row 满量程 + lazy rescale = 溢出**：满量程发射没有 2^{τ} 的 headroom（式 34.7），两者互斥是硬约束不是调参问题。
3. **把 satfinite 当正确性保证**：satfinite 只是钳位，被钳的值已携带不可控误差；它的作用是钳住残余超调（amax(V) 刚好越过 28 的边缘 case），不是精度修正手段。
4. **混淆 fp16 家族与 fp8 家族的 τ**：FA-4 对 BF16 取 $\tau = 8$ （$2^8 = 256$ 在 fp16 域安全），fp8 发射域取 $T = 4$。跨家族直接沿用阈值会超出 e4m3 上限。
5. **引用行号漂移**：ffpa-attn 是活跃仓库，本章行号锚定 commit `861d75e`（附录 D）；升级仓库后需复核（`grep -n` 三步协议首行"Why P needs"）。

34.9　面试要点速查

- Q：e4m3 与 int8 量化误差的本质区别？
  A：均匀 vs 指数网格——int8 绝对误差恒定（$\delta/\sqrt{12}$ RMS），e4m3 相对误差恒定（$\le 2^{-(m+1)} = 6.25\%$，定理 34.1）。值域横跨多倍频程用指数，幅度紧凑用均匀。
- Q：为什么 P 必须在线量化，Q/K/V 可以离线？
  A：Q/K/V 是前向输入侧的静态张量；P 每 tile 重算且依赖 softmax 状态，只能在寄存器 fragment 上就地量化（27.2.2）。
- Q：v_s 如何消去？为什么 fold 进 P 而不是 V？
  A：$(Pv_s/p_{scale}) \cdot (V/v_s) = PV/p_{scale}$，精确消去（式 34.5）；fold 进 P 侧使 gmem 的 $\hat{V}$ 原样复用，无需重写。

- Q：fixed 与 per-row p_{scale} 怎么选？
  A：fixed 编译期常数、单一累加域、可与 lazy rescale 组合，默认；per-row 每行满量程精度最优，但要 rowmax 归约 + o_{tile} 暂存 + 与 lazy rescale 互斥（27.3.4）。
- Q：causal 前几行精度差 15 倍，是误差放大吗？
  A：不是。相对误差 ~5% 全行一致；绝对误差 $= \varepsilon\times$ 幅度，早行 ESS 小、幅度大（式 34.8）。解法是 hybrid 全链 fp16 或降低 V 的相对误差，单换 QK 无效（V 0.19 是最大单项）。
- Q：FA-4 lazy rescale 在量化域要额外考虑什么？
  A：发射值被 2^τ 膨胀，需核算格式上限 $2^\tau \operatorname{amax}(V) \le 448$（式 34.7），fp8 取 T=4 而非 BF16 的 8。

34.10 最小测试与动手实验

Part VI 不新增 `book/tests`，验证链路统一为 `ffpa_attn.bench` CLI（ffpa-attn 0.2.5.dev25 editable 安装，本机 PRO 5000，卡位 7）。本章实验聚焦「fp8 基线口径」：

```
# 前置：conda activate cdit；D<=256 需放开 small-D 门禁
$ export FFPA_CUDA_ALLOW_SMALL_D=1 CUDA_VISIBLE_DEVICES=7

# 实验 1：fp8 基线（self-attn D128，对照 SDPA，验证 allclose）
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn --D 128 --no-bwd --show-allclose
# 预期：allclose 列 OK；speedup ~1.7x（365T vs 215T，本机 PRO 5000）

# 实验 2：fp4 同形状对照（e2m1 的精度与速度差）
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp4 \
    --tasks self-attn --D 128 --no-bwd --show-allclose

# 实验 3：causal 早行误差的直觉验证（ESS 模型，27.3.7）
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn,causal --D 128 --no-bwd --show-allclose
```

实验 3 的观察点：causal 的绝对误差显著大于 dense（表 34.4），但任务耗时反而更短（半个三角）。数据落盘规范：`--save-path` 指向 `book/.tmp/book-bench/ch27/`，本章基线数据已归档于 `.tmp/book-bench/k0-smoke/`。

延伸阅读

- @DefTruth《WINT8/4 快速反量化系列（00)–(03)》：低比特数值格式与反量化的工程直觉（全书行文风格基线）；
- SageAttention 1/2/2++/3 四篇论文（arXiv:2410.02367 / 2411.10958 / 2505.21136 / 2505.11594，附录 E）：表 34.3 的一手出处；
- FlashAttention-4（arXiv:2603.05451）§3.1.4：lazy rescale 的原始定义与 τ 选择。

第三十五章　FP8（一）：量化前处理链

35.1　本章导读

第 三十四 章回答了「量化 attention 的数学是什么」；本章回答第一个工程问题：Q, K, V **的离线量化链本身怎么写**。在第 三十六 章的 persist-D kernel 执行之前，fp16/bf16 的输入张量要先经历一系列「前处理 aux kernel」：smooth-K 列均值统计、可选的 Hadamard 预旋转、Q/K 的 e4m3/int8 量化、V 的转置量化（$V^\top$）与 per-channel scale 统计。这些 kernel 单个都不复杂，但它们决定了两件事：

1. **精度上限**：量化前的 outlier 处理（smoothing / Hadamard 旋转）与量化粒度（per-block / per-thread / per-channel）直接决定进入 kernel 的量化数据质量——28.3 节的三个「不变变换」是全部精度的来源；
2. **性能下界**：aux 链是一次额外的全量 DRAM 往返（读 fp16 写 fp8，字节量与主 kernel 同为 $O(BHND)$，但时间是 $O(N)$、主 kernel 是 $O(N^2)$，占比随 N 以 $1/N$ 衰减），在短序列 / 小批量场景会成为可观的开销——28.6 节的带宽需求分析说明它为什么必须达到带宽峰值。

本章代码全部来自 ffpa-attn 仓库（commit `861d75e`）的 `cute/fp8/` 目录与 `cute/hadamard.cuh`，约 1600 行，被 persist-D、split-D、M4N2 三个 fp8 主 kernel 家族**共享**（28.4.1 节的编排函数是唯一的公共入口）。这批 aux kernel 的设计思路与 Part III–IV 的主 kernel 恰好互补：主 kernel 优化「计算密度」，aux kernel 优化「每字节的搬运效率」，两者的性能模型是纯带宽模型（28.6 节）。

前置章节：第 三十四 章（量化数学，必须）、第 十二 章（MMA fragment 布局，per-thread 粒度必须回指 C-fragment）、第 五 章（softmax 平移不变性）。**源码区间**：`smooth_k.cuh` L12–165、`smooth_v.cuh` L12–175、`quantize_fp8.cuh` L29–201 与 L695–950、`hadamard.cuh` 全文、`prepare_inputs.cuh`、`input_layout.cuh`；完整行号以附录 D 的 commit permalink 为准。**验证方式**：`ffpa_attn.bench` CLI 的 knob A/B（28.10 节，五组配置 allclose 全过）。

35.2　问题与动机

35.2.1　前处理链在端到端里的位置

回顾式 34.2 的三量化点：Q, K 与 V 是 forward 开始前就绪的静态张量，可以**离线**量化；P 只能在线量化（第 三十六 章）。于是 fp8 attention 的完整调用形态是：

$$\underbrace{\{Q, K, V\}_{\text{fp16}}}_{\text{用户输入}} \xrightarrow[\text{aux kernels}]{\text{smooth / WHT / quantize / 转置}} \underbrace{\{Q_8, K_8, V_8^\top;\ \delta\}}_{\text{量化 workspace}} \xrightarrow[\text{persist-D}]{\text{第三十六章}} \{O, \text{lse}\} \tag{35.1}$$

aux 链的输出是一套**按主 kernel 的消费方式预先排布的 workspace**，与 Q, K, V 的数值质量无关：Q_8/K_8 保持 (N, D) 行主序（TMA 按行块加载）；V 被量化成 (D, N) 列主序的 $V_8^\top$（PV MMA 的 B 操作数直接要列数据）；scale 张量按 kernel 内的 fragment 寻址模式排布（28.3.4 节）。**前处理链的输出布局由主 kernel 的消费方式决定**，本章反复回到这条主线，也正因如此才需要与第 三十六 章对照阅读。

35.2.2　一次 forward 的 aux kernel 清单

以默认配置（smooth-K 开、per-block QK、per-block V）为例，一次 forward 的前处理链发射 3 个 kernel：

1. `kv_col_sum_kernel` + `kv_mean_finalize_kernel`：K 的两阶段列均值（smooth-K 用，28.4.2 节）；
2. `quantize_fp8_qkv_fused_kernel`：Q/K 量化 + V 的 per-block 转置量化合并的 fused 形态（28.4.3 节）；

3. 开 per-channel V 时另起三个 kernel（`v_col_stats` 列统计、`v_stats_finalize` 汇总、per-channel 转置量化，28.4.5 节）；

4. 开 Hadamard 时（fp4/fp8 均需显式开启，fp4 收益更大）加 `wht_qk_kernel` 的 Q、K 两次调用（28.4.6 节）。

每个 kernel 都是「读一遍、写一遍」的流式形态，同泳道内的「统计 → 量化」数据依赖串行（不能用 stream 重叠消除）；跨泳道（Q/K/V 三路互相独立）理论上可多流重叠，当前实现使用单流，因此整链时间 ≈ 总搬运字节 / 有效带宽。

35.2.3 三条「不变变换」的设计哲学

aux 链里所有**改数值**的操作（smooth-K、smooth-V、Hadamard）共享同一个设计约束：**变换必须保持 attention 输出精确不变（或在可控的标量修正下不变）**。这条约束由代数定理保证，28.3.1–28.3.3 节将逐条证明：

- smooth-K 利用 softmax 的**行平移不变性**（logits 减一个行常数，概率不变，lse 需回加）；
- smooth-V 利用 softmax 的**行和恒为 1**（输出减均值行，精确可逆）；
- Hadamard 利用**正交矩阵的转置消去**（$(Q\hat{H})(K\hat{H})^\top = QK^\top$，连修正都不需要）。

三者目的相同：**在量化之前分散 outlier 的能量**，让 block scale 收敛、量化步长的利用率最大化。它们在算法上是 SageAttention 系（smooth）与 QuaRot / SpinQuant / FA-3 incoherent processing（旋转）两条线的交汇。

35.3 数学原理

35.3.1 smooth-K：列均值平移与 LSE 修正

设 $K \in \mathbb{R}^{N\times d}$，定义列均值向量 $\bar{k} = \frac{1}{N}\sum_i K_{i,:} \in \mathbb{R}^d$，平滑后的 $K' = K - \mathbf{1}\bar{k}^\top$（每行减同一个均值向量）。则 logits 矩阵变为

$$S' = QK'^\top = QK^\top - Q\bar{k}\,\mathbf{1}^\top = S - \Delta, \qquad \Delta := q_s\mathbf{1}^\top, \quad q_s = Q\bar{k}. \tag{35.2}$$

关键观察：Δ 的每一行是**常数**（第 m 行恒为 $q_{s,m}$）。而 softmax 对行常数平移不变，分子分母同乘 $e^{q_{s,m}}$：

$$\mathrm{softmax}\big(S'_{m,:}\big) = \frac{\exp(S_{m,:} - q_{s,m})}{\sum_j \exp(S_{m,j} - q_{s,m})} = \mathrm{softmax}(S_{m,:})\,. \tag{35.3}$$

所以 P 与 $O = PV$ **逐 bit 不变**（在 fp32 域内）。唯一受影响的是返回给上层的 lse：$\mathrm{lse}(S'_{m,:}) = \mathrm{lse}(S_{m,:}) - q_{s,m}$，epilogue 需要把 $\delta_Q\delta_K \cdot q_{s,m}$ 加回（$\delta_Q\delta_K$ 是 27.3.4 节折进 exp2 的量化 scale 积）。这个行级标量点积 $q_s = Q\bar{k}$ 在 kernel 内用 `smooth_k_qk_dot`（28.4.2 节末）从 Q 的 smem fragment 直接算出。

精度收益的来源：LLM 的 K 矩阵常有系统性的非零列均值（位置编码残余、layer norm 偏置），减去后 K' 的 per-block amax 显著下降且分布更对称，等价于零代价收紧了量化步长。SageAttention 的实测里 smooth-K 是 fp8 QK 精度的显著单项改善（arXiv:2410.02367）。

35.3.2 smooth-V：行和不变量与均值剥离

对 V 做同样的列均值剥离 $V' = V - \mathbf{1}\bar{v}^\top$，这次的不变量来自 softmax 输出的**行和恒为 1**：$P\mathbf{1} = \mathbf{1}$。于是

$$O' = PV' = PV - P\,\mathbf{1}\bar{v}^\top = O - \mathbf{1}\bar{v}^\top = O - \bar{v}^\top \text{（每行减 } \bar{v}\text{）}, \tag{35.4}$$

即输出只需最后加回 $\bar{v}$（一个 d 维行向量），中间全程使用 V'。与 smooth-K 不同，这里**没有任何 lse 修正**，因为 P 本身不变。

残差对称化：若不减均值、量化 scale 直接取 colmax($|V|$)，会偏向原始分布的上侧。SageAttention 的 per-channel recipe（`smooth_v.cuh`）在统计阶段同时维护 sum, max, min 三列统计，finalize 时

$$\mathrm{mean} = \mathrm{sum}/N, \qquad \mathrm{amax} = \max\big(|\mathrm{max} - \mathrm{mean}|,\ |\mathrm{min} - \mathrm{mean}|\big), \tag{35.5}$$

中心化后的残差近似对称分布，取双侧残差最大值能让 e4m3 的对称编码域 $[-448, 448]$ 恰好被两侧同时铺满。发射域系数 $v_s = \mathrm{amax}/v_{scale_max}$ 中的 v_{scale_max} 默认 448（e4m3 满量程）；当 V 采用 per-channel 粒度且 PV 累加在 f16 域（27.3.5 节的 SA2++ 路径）时，launcher 把 v_{scale_max} 整个收紧为 2.25：量化后的 V_8 幅度被压进 $[-2.25, 2.25]$、$v_s = \mathrm{amax}/2.25$，为 f16 累加域（±65504）在 kBc 块求和下留足裕量，防止长块累加溢出（kernel 侧有编译期检查 $kBc \cdot 448 \cdot 2.25 \le 65504$；per-block 路径固定 amax/448，无此参数）。

35.3.3　Hadamard 旋转：正交不变量与 outlier 扩散

第三条不变变换不依赖 softmax 的任何性质，而依赖**正交性**。归一化 Walsh-Hadamard 矩阵 $\hat{H}_n = H_n/\sqrt{n}$ 满足 $\hat{H}_n\hat{H}_n^\top = I$（$H_n$ 按 $H_1 = [1]$、$H_{2n} = \left[\begin{smallmatrix} H_n & H_n \\ H_n & -H_n \end{smallmatrix}\right]$ 递归，元素全为 ± 1）。对 Q, K 右乘同一个 $\hat{H}$：

$$(Q\hat{H})(K\hat{H})^\top = Q\hat{H}\hat{H}^\top K^\top = QK^\top \qquad \text{（逐元素精确，零修正项）．} \tag{35.6}$$

旋转的作用：$\hat{H}$ 的每一列都是 $\pm 1/\sqrt{n}$ 向量，乘完后 **每个输出坐标都是全部输入坐标的带符号和**。由中心极限定理，只要输入没有极端的单坐标集中，输出幅度趋近均匀，这就是 Hadamard 的 whitening / Jackson 性质。对量化的意义：量化 scale 是按块共享的（fp8 的 per-block/per-thread、fp4 的 1×16 块），不旋转时一个 outlier 坐标会放大所在块的量化步长、降低其余数值的分辨率；旋转后能量分散到所有坐标，块 scale 收敛，量化噪声整体下降。FA-3 的 incoherent processing（随机 ± 1 对角 $\times$ Hadamard，与 RoPE 融合）实测与分块量化联合可将 FP8 RMSE 降低 2.6×（即降至 1/2.6，arXiv:2407.08608 §3.3）；QuaRot 与 SpinQuant 把同一思想推广到 LLM 权重激活的端到端量化。

宽度规则：全宽 WHT 要求 D 为 2 的幂且 $D \le 512$（更大的全宽蝶形寄存器开销过大）；否则用块对角 blockdiag(H_{64}) 近似（fp4 路径的 D 恒为 64 的倍数，天然适配）。块对角只是全宽旋转的近似：outlier 扩散仅发生在 64 列块内、whitening 效果减弱；但块对角旋转矩阵本身精确正交，式 35.6 的收缩等式在每个 64 列块内逐块、进而全局精确成立。

pad 列的处理：当输入真实宽度 $d_{og} < kHeadDim$（headdim padding 路径），pad 列以 0 参与旋转，且**旋转后的 pad 列值被照常写出**（不是丢掉）。下游 kernel 把 d_{og} 当作 $kHeadDim$ 全宽读取，这样 $\sum_{d<kHeadDim}(Q\hat{H})_{:,d}(K\hat{H})_{:,d} = \sum_{d<d_{og}} Q_d K_d$ 依然精确，pad 列贡献恰好为零。若旋转后丢弃 pad 列，被丢弃的列中会混入真实列的能量，收缩值错误。

35.3.4　量化粒度谱系：与 C-fragment 对齐

第 三十四 章的表 34.2 列了粒度谱系；本节回答工程问题：**ffpa-attn 为什么在 Q 上选 per-thread（64 scale / 128 行）这个非常规的粒度**。答案在 MMA 的 C-fragment 布局里（第 十二 章图 12.1）：$m16n8k16$ 的 fp32 累加 fragment 中，**每个线程持有 2 行 × 1 个列对（2 个相邻列元素）**，quad 内 4 个 lane 分摊 n=8 列且持有**相同的 2 行**。反量化时若 scale 按行组织，同一线程的两行就要两个 scale；若 scale 按「C-fragment 行对」组织（每 scale 覆盖 $\{r, r+8\}$ 两行，quad 共享），则每个线程的所有累加值共享**一个** scale，反量化变成「fragment 提升时直接乘一个寄存器标量」，**零 shuffle**。

具体排布（`quantize_fp8_perthread_q_kernel`）：每个 128 行的量化块铺 $8 \times 8 = 64$ 个 scale：行维按 16 行一组分 8 组，组内按 $\{r, r+8\}$ 行对分 8 档。代价是 amax 的归约树要跨行对组合：行内两半（`shfl_xor` 1）→ 行对（`shfl_xor` 16）。K 侧对应 per-thread 粒度是每 kBc 块 4 个 scale（K 的 fragment 消费模式不同）。这给了谱系的中间档：

$$\underbrace{\text{per-token}}_{\text{128 scale/128 行}} \prec \underbrace{\text{per-thread（fragment 对齐，SA2）}}_{\text{64 scale/128 行，零 shuffle}} \prec \underbrace{\text{per-block}}_{\text{1 scale/128 行}} \tag{35.7}$$

粒度越细精度越高、但 scale 寄存器与消去开销越大；per-thread 是「精度贴近 per-token、消费成本贴近 per-block」的工程折中点。图 35.1 把 64 槽位的排布与 quad 的消费视角合在一张图里：左列是 128 行块上 64 个 scale 的覆盖域，右侧放大一个 quad 看「为什么零 shuffle」。

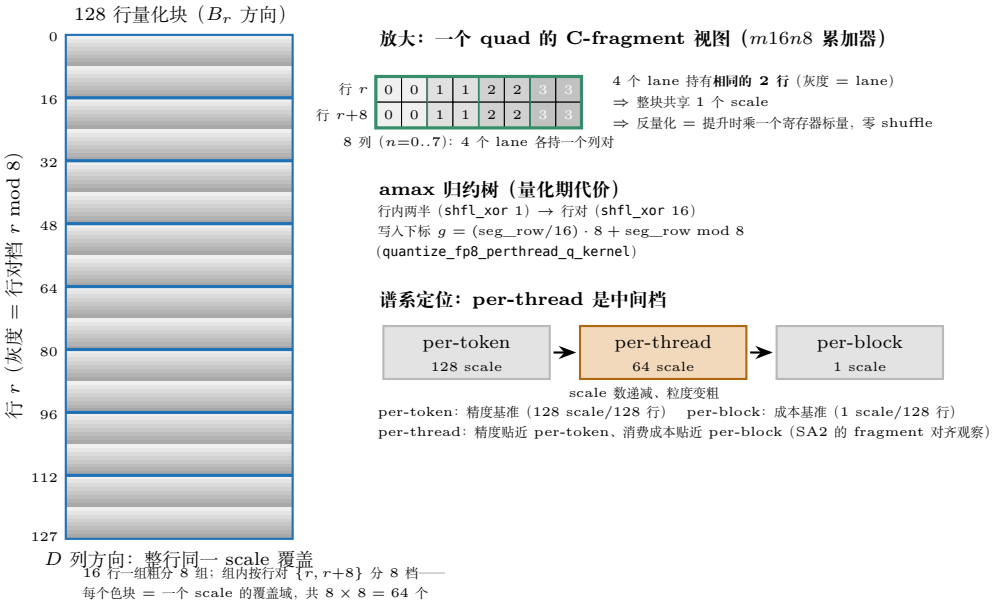


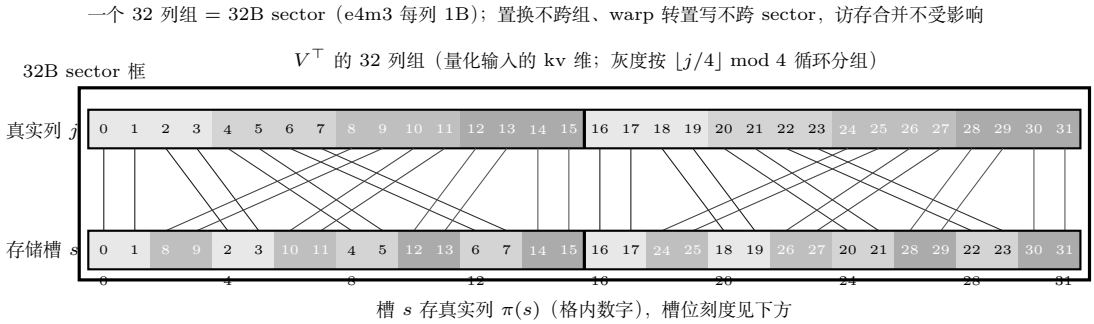
图 35.1: per-thread Q 量化的 64 槽位表与 quad 消费视角（`quantize_fp8_perthread_q_kernel`，28.3.4 节）。左：128 行量化块铺 $8 \times 8 = 64$ 个 scale——行维按 16 行一组粗分 8 组（粗边界），组内按行对 $\{r, r+8\}$ 分 8 档灰度（行 r 与 r+8 同色），每个色块就是一个 scale 的覆盖域；写入下标 $g = (\text{seg_row}/16) \cdot 8 + \text{seg_row} \bmod 8$。右上：放大一个 quad 的 $m16n8$ C-fragment——4 个 lane 持有相同的 2 行、各占一个列对，因此整块共享一个 scale，反量化变成「fragment 提升时直接乘一个寄存器标量」，**零 shuffle**；代价是 amax 归约树跨行对组合（行内两半 `shfl_xor` 1、行对 `shfl_xor` 16）。下：谱系定位——per-thread 是 per-token（128）与 per-block（1）之间的工程折中点。

35.3.5 V 的预转置与列置换

$V_8^\top$ 的存储布局是 flat $[B \cdot H_{kv} \cdot kD,\ N_{kv,pad}]$：行是 (bh, d)、列是 kv 位置，尾列 pad 到 $N_{kv,pad}$ 以满足 TMA 16B 对齐（gmem 列对齐与 28.4.5 节 smem staging 的 `kPad`=16 行内边距是两个独立的常数，只是恰好都取 16）。persist-D / split-D 家族还默认开启一个更深度的变换：**列置换**（M4N2 家族默认关）。动机简述（完整推导在第 三十七 章）：attention kernel 需要把 softmax 的 P 字节 **不经任何跨 lane shuffle** 直接打包进 PV MMA 的 A 操作数，代价是 A 操作数的 kv 下标带一个置换 π（槽位 s 存真实列 $\pi(s)$）；于是 $V^\top$ 的 B 操作数必须携带**同一个** π，保证槽位 s 两侧对应同一 kv 位置。置换的逆映射在量化期写进 $V^\top$ 的列排布：

$$\pi^{-1}(j) = 4 \cdot \big((j \gg 1)\,\&\,3\big) + 2 \cdot \big((j \gg 3)\,\&\,1\big) + (j\,\&\,1), \tag{35.8}$$

置换被刻意限制在**每个 32 列组内**，warp 的转置写不跨 32B sector，访存合并不受影响。这是「前处理链的输出布局由主 kernel 消费方式决定」的最典型例证：一个第 三十七 章的寄存器打包决策，提前在本章的量化 kernel 中完成。图 35.2 把 π^{-1} 的位段重组画成双条带：上条是真实列序，下条是存储槽位，连线束的交叉形态就是置换本身。



位段重组（式 (35.8)）：槽位低 4 位 = $4 \cdot ((j \gg 1) \,\&\, 3) + 2 \cdot ((j \gg 3) \,\&\, 1) + (j \,\&\, 1)$，bit4 原样保留——
上下半组置换同构（j 与 j+16 的低 4 位走同一条重排），自检序列（$j = 0..15$ 的落点）：$0, 1, 4, 5, 8, 9, 12, 13, 2, 3, 6, 7, 10, 11, 14, 15$。

匹配约束（坑四）：P 侧 `PackC8bitToA8bitPermVT` 与 $V^\top$ 侧必须携带 **同一个** π——槽位 s 两侧对上同一 kv 位置，任一侧缺失即静默错位。

图 35.2: `VTPermInv32` 列置换的双条带视图（式 (35.8)，`quantize_fp8.cuh`）。上条带：$V^\top$ 组内的真实列序 $j = 0..31$（4 列一组循环灰度）；下条带：置换后的存储槽位，格内数字为槽 s 存的真实列 $\pi(s)$，灰度跟随数据来源组。连线束从列 j 指向其落点槽 $\pi^{-1}(j)$——交叉形态即置换本身：j 的 bit1、bit2 移到槽位的高两位、bit3 移到 bit1、bit0 与 bit4 不动，因此上下两个 16 列半组的置换同构。整个变换被限制在一个 32 列组（= 一个 32B sector）内：warp 的转置写不跨 sector，coalescing 不受影响；P 侧 `PackC8bitToA8bitPermVT` 与 $V^\top$ 侧必须携带同一个 π（坑四）。

35.3.6　NHD 零拷贝寻址

最后一个环节是**输入布局**。diffusers 等 HF 管线的 KV cache 形状是 $[B, N, H, D]$（NHD），而 kernel 语义视角是 $[B, H, N, D]$（BHND）。朴素做法是 `.permute(0,2,1,3).contiguous()` 物化一份，等于一次全量拷贝。ffpa-attn 的选择是**零拷贝**：量化 kernel 接受一个五字段的布局描述符（`Fp8InputLayout`，28.4.1 节），用式

$$\text{base}(bh) = \text{NHD} \;?\; \lfloor bh/H \rfloor \cdot s_{batch} + (bh \bmod H) \cdot s_{head} \;:\; bh \cdot s_{head} \tag{35.9}$$

直接在原始 gmem 上寻址（行 stride 相应变为 $H \cdot D_{og}$）。量化**输出**统一物化为 BHND：主 kernel 的 TMA 与 smem 布局只支持一种格式，复杂度被前处理链一次性吸收。

35.4　设计与实现

35.4.1　前处理编排：一个函数，三个家族共享

persist-D、split-D、M4N2 三个 fp8 主 kernel 的前处理完全一致，编排收敛在 `prepare_fp8_inputs` 一个函数（`prepare_inputs.cuh`）：分配量化 workspace、按 knob 决定调用哪些统计 / 量化 kernel、返回持有全部张量的输出结构 `Fp8QuantizedInputs`。该函数按 $(\text{dtype}, kBr, kBc, kHeadDim, kQKInt8)$ 显式实例化一次，供全部主 kernel TU 共享（extern-template），避免量化 kernel 家族被每个 stage 变体重复编译。

Listing 35.1: 前处理编排与输出结构（`prepare_inputs.cuh` L21–31，节选）

```
struct Fp8QuantizedInputs {
  torch::Tensor q8, k8, vt8;              // e4m3/int8 workspace (vt8 已转置)
  torch::Tensor q_scale, k_scale, v_scale; // 粒度随 knob: per-block/per-thread/per-channel
  torch::Tensor km, km_f32;                // smooth-K 列均值 (in-dtype + fp32)
  torch::Tensor vm;                        // per-channel V 均值
  const float* km_f32_ptr = nullptr;       // kernel 侧指针 (关闭时为 null)
  const float* vm_kernel = nullptr;
};
```

35.4.2　两阶段确定性列均值

smooth-K 需要的 $\bar{k}$ 是一个 (bh, d) 规模的小统计，但实现上有两个坑：其一，`at::mean` 的库调用 + fp32 cast 要 ~85us（B1 H32 N8192 D128）；其二，若朴素地用 atomics 自写统计归约，结果是非确定性的。`smooth_k.cuh` 用两阶段确定性方案替代：

Listing 35.2: smooth-K 列均值：两阶段确定性归约（smooth_k.cuh L56–84）

```
// Smooth-K column mean, two stages, deterministic (no atomics/zero-init):
// stage 1 sums a (bh, row-chunk) slab of K into fp32 partials, stage 2
// reduces the chunks and divides by Nkv, emitting both the in-dtype mean and
// its fp32 copy (lse correction). Replaces at::mean + km.to(fp32) (~85us ->
// ~50us at B1 H32 N8192 D128): one coalesced DRAM pass with fp32 accumulate.
constexpr int kMeanRowsPerChunk = 512;

template <typename Element, int kD>
__global__ void kv_col_sum_kernel(const Element* __restrict__ k,
                                  float* __restrict__ partials, int Nkv,
                                  int rows_per_chunk, int D_og,
                                  ffpa_fp8::Fp8InputLayout L) {
  constexpr int kVec = 16 / sizeof(Element);  // 8 half/bf16 per 16B
  constexpr int kColsPerRow = kD / kVec;      // uint4s per row
  constexpr int kThreads = 256;
  // Ceil'd so headdims where kColsPerRow does not divide kThreads (e.g.
  // D=320/384/448) still map every thread to a valid row group.
  constexpr int kRowsPerIter = (kThreads + kColsPerRow - 1) / kColsPerRow;
```

stage 1 每 block 处理一个 512 行的 slab：每线程以 16B 向量累加一列的 fp32 partial，块内经 smem 归约成每列一个值写出；stage 2（kv_mean_finalize_kernel）把 $\lceil N/512 \rceil$ 份 partial 求和除 N，一次同时产出 in-dtype 均值（供量化 kernel 减）与 fp32 副本（供 epilogue 的 lse 修正）。无 atomics、无 zero-init、逐 kernel 确定性。注意 kRowsPerIter 用**向上取整**的行组映射，$D = 320$ 这类 kColsPerRow 不整除 256 的 headdim 也能把每个线程映射到合法行组，空槽在 kernel 体内显式清零避免污染列和。

lse 修正项 $q_s = Q\bar{k}$ 的计算在主 kernel 的 epilogue 侧（smooth_k_qk_dot，L10–54）：Q 的值就在 smem 里，每线程对自己 fragment 的 2 行做部分点积，quad 内用 __shfl_xor(1) 与 __shfl_xor(2) 完成 4-lane 归约，注意**不能用全 warp butterfly**，那会把 warp 的 8 个 fragment 行混在一起。这条 scalar 冷路径刻意保持简单（注释原话：correctness-first），因为 lse 每个输出行只算一次，profile 里从不可见。

35.4.3 块级 QK 量化：寄存器驻留 amax

Listing 35.3: 行主序块量化 kernel 头与签名（quantize_fp8.cuh L56–79）

```
// Row-major blockwise e4m3 quantization of fp16/bf16 Q/K for the FP8
// persist-D kernel. Output keeps the (N, D) row-major layout; scales are
// amax/448 per (b, h, row-block). One block per (head, row-block).
// Requires D % 8 == 0. Memory accesses are fully vectorized: 128-bit (8-elem)
// loads, and 64-bit (8 x e4m3) stores whose 8B chunks never straddle a
// D-element row (D % 8 == 0), so every transaction is aligned and coalesced.
// The amax pass keeps the 8-elem chunks in registers, so the quantize pass
// converts straight from regs (no second global read).
// kQKInt8: symmetric int8 output (scale = amax/127) instead of e4m3.
// D_og: input row stride (real head_dim, may be < kD for pad path); output
// uses kD stride (compile-time, padded). c*kVec < D_og guards pad chunks.
template <typename Element, int kBlockRows, bool kQKInt8, int kD>
__global__ void quantize_fp8_kernel(
    const Element* __restrict__ X,    // (B, H, N, D_og) or NHD layout L
    QKOutT<kQKInt8>* __restrict__ Y,  // (B, H, N, kD)
    float* __restrict__ scale,        // (B, H, N/kBlockRows)
    int N, int H, int D_og, Fp8InputLayout L,
    const Element* __restrict__ km = nullptr,  // (B*H, kD) smooth-K col means
    float inv_n = 0.0f) {
```

设计要点有三。**其一，单遍寄存器驻留**：8 元素（16B）向量读进寄存器后，amax pass 与量化 pass 都在寄存器完成，全局内存只读一次，纯带宽 kernel 必须如此。**其二，smooth-K 融合**：均值向量先经 smem 广播（km_sh），量化前在 fp32 域减去，统计 kernel 与量化 kernel 之间 **不落盘中间结果** $K - \mathbf{1}\bar{k}^\top$，一次 DRAM 往返同时完成 smooth 与 quantize。**其三，e4m3 与 int8 的统一**：kQKInt8 模板参切换输出类型与 scale 分母（448 vs 127），其余代码路

径零分叉：int8 对称量化（round-nearest-even、clamp $[-127, 127]$、留 -128 不用）是 SageAttention 的 QK 精修方案（`quant_sym_int8`，L29–34）。

35.4.4　per-thread 量化：面向 fragment 的 scale 布局

28.3.4 节推导了粒度；实现上的关键在**归约树的构造**。`quantize_fp8.cuh` L695–741 的头注释先给布局：per-thread Q 量化按 fragment 对齐而非 per-token——每个 128 行块铺 64 个 scale，每组覆盖一个 C-fragment 行对 $\{r, r{+}8\}$（2 行），行对经 `shfl_xor`(amax, 8) 合并。两个极端作对照：per-token 要 128 个 scale（每行一个），per-block 只要 1 个；这个中间档换来 zero-shuffle 反量化（每线程 C-frag 的两行共享一个 scale），代价是 amax 跨多行共享。行内再一分为二，每行两个线程各管半行，网格因此翻倍，冷输入的 DRAM 延迟靠更多 in-flight warp 隐藏；行内两半与行对分别用 `shfl_xor` 1 与 16 合并成组 amax，组 amax 与每个量化字节都和 1-thread/row 形态逐 bit 一致——粒度变更不能隐式改变数值。

还有一个实现细节：**scale 的写入下标**。$g = (\text{seg_row}/16) \cdot 8 + \text{seg_row} \bmod 8$ 把 $\{r, r+8\}$ 行对映射到 64 槽位表；段内行号还要按奇偶 launch block 折算（每个 launch block 只覆盖 64 行，两个 block 合成一个 128 行 scale 段）。K 侧的 per-thread 变体（`quantize_fp8_perthread_k_kernel`，L778 起）每块只铺 4 个 scale，与 kBc 块对齐。

35.4.5　$V^\top$ 转置量化：smem staging

V 的量化 kernel 多一个转置职责：读 (N, D) 行主序、写 (D, N) 列主序。若直接全局转置，两侧必有一侧非合并访存。解法是经典 smem staging：

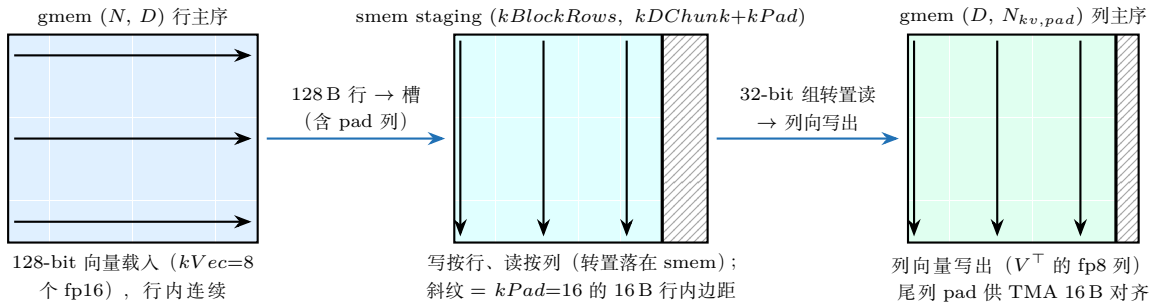
Listing 35.4: $V^\top$ 转置量化的 staging 结构（`quantize_fp8.cuh` L173–207）

```
// Transposed (D, N) V quantize staged through smem so BOTH the gmem read and
// the gmem write are fully coalesced. Grid: (Nkv/kBlockRows, B*Nh_kv); block
// covers one (bh, row-block) tile of (kBlockRows, D). Global reads are
// 128-bit vectorized; 8-elem runs land in smem as 2x uint32 (pad keeps bank
// conflicts low on the transposed read-out).
// kVTPerm: write V^T columns permuted for the reorg-free PV pack
// (VTPermInv32); must pair with PackC8bitToA8bitPermVT in the attention
// kernel (launcher-enforced pairing in launch/cute_fp8.cuh, on by default
// for the gated config).
template <typename Element, int kBlockRows, int kD, bool kVTPerm = false>
__global__ void quantize_fp8_vt_kernel(
    const Element* __restrict__ V,   // (B, H, N, D_og) or NHD layout L
    __nv_fp8_e4m3* __restrict__ VT,  // flat [B*Nh_kv*kD, Nkv_pad]
    float* __restrict__ scale,       // (B*Nh_kv, Nkv/kBlockRows)
    int N, int Nh_kv, int ldy, int D_og, Fp8InputLayout L) {
  constexpr int D = kD;
  constexpr int kPad = 16;
  constexpr int kVec = 8;
  constexpr int kThreads = 128;
  constexpr int dv = D / kVec;  // vector chunks per row
  static_assert(D % kVec == 0, "vt kernel requires D % 8 == 0");

  // VT staging tile lives in DYNAMIC smem (sized by the launcher). D is
  // tiled into kDChunk-wide columns so the staging tile fits the 48KB
  // static-smem cap (kBlockRows*(kDChunk+kPad)) even for large D.
  extern __shared__ __nv_fp8_e4m3 tile_sh[];
  constexpr int kDChunk = (D < 256) ? D : 256;
```

三个结构决策。**kPad=16**：staging tile 的行内边距让转置读出的 32-bit 组错开 bank，这是纯 smem 侧的 bank 错开；gmem $V^\top$ 的 TMA 16B 加载对齐由行宽 $N_{kv,pad}$（launcher 传入）保证，与 `kPad` 无因果关系（另见 28.3.5 节的对比说明）。**kDChunk=256 动态 smem**：大 headdim 的整 tile 会超出 48KB 静态上限，D 维分块后 staging 尺寸恒为 $kBlockRows \times (kDChunk + kPad)$，动态 smem 按需分配。**kVTPerm 置换写**：写出列号经 `VTPermInv32`（式 35.8）置换，与主 kernel 的 reorg-free 打包强制匹配：launcher 层面检查这对开关必须同开同关（注释：launcher-enforced pairing），错配即静默数值错误。per-channel 变体（`quantize_fp8_vt_perchannel_kernel`，L555 起）在同一框

架下改用 28.3.2 节的 (bh, D) scale 与均值剥离。图 35.3 把这条「行主序读 → smem 转置 → 列主序写」的三段式数据通路画出来，并标出两个同名却不同源的 16。



两个 16 不同源：$kPad$=16 是 smem staging 的行内边距——转置读出的 32-bit（4 个 fp8）组因它错开 bank，是纯 smem 侧的考量；gmem $V^\top$ 的 TMA 16 B 加载对齐由行宽 $N_{kv,pad}$（launcher 传入）保证，与 $kPad$ 没有因果关系，两个 16 只是恰好同值。

$kDChunk = \min(D, 256)$：大 headdim 的整 tile 会破 48 KB 静态 smem 上限，D 维分块后 staging 尺寸恒为 $kBlockRows \times (kDChunk + kPad)$，由 launcher 走动态 smem 分配（`extern __shared__`）。

图 35.3: $V^\top$ 转置量化的 smem staging（`quantize_fp8.cuh` `quantize_fp8_vt_kernel`）。左：gmem 上的 (N, D) 行主序 tile 按 128-bit 向量载入；中：smem staging tile 写按行、读按列——转置只在 smem 内发生，两侧 gmem 访存都保持合并；斜纹是 $kPad$=16 的行内边距，让转置读出的 32-bit 组错开 bank；右：$(D, N_{kv,pad})$ 列主序写出，尾列 pad 为 TMA 16 B 对齐服务。**$kPad$ 与 $N_{kv,pad}$ 的 16 来源不同**：前者是 smem 侧的 bank 考量，后者是 gmem 侧的对齐要求，恰好同值而非同一个量。

35.4.6 Hadamard butterfly kernel

头注释（`hadamard.cuh` L1–32）先给出数学。未归一化的 Walsh-Hadamard 矩阵递归定义：$H_1 = [1]$，$H_{2n} = \left[\begin{smallmatrix} H_n & H_n \\ H_n & -H_n \end{smallmatrix}\right]$，元素全是 ±1；归一化形式 $\hat{H}_n = H_n/\sqrt{n}$ 正交（$\hat{H}_n \hat{H}_n^\top = I$），于是 $(Q\hat{H})(K\hat{H})^\top = QK^\top$ 精确成立，softmax 的 logits 一个都不变。kernel 对每行施加 $\hat{H}$ 走 radix-2 蝶形：$(a, b) \to (a{+}b,\ a{-}b)$，每个 bit 一轮、无 bit-reversal，$1/\sqrt{n}$ 的缩放折进 store。

旋转的效果是把每行白化：能量摊到全部坐标、块 scale 收敛、量化噪声整体下降，机理与量化收益的推导见 28.3.3 节。

Listing 35.5: radix-2 蝶形 kernel 主体（`hadamard.cuh` L40–92，节选）

```
// One warp per (row, kWhtWidth-block); lane l owns cols {base + l + 32*j}.
// Butterfly distances >= 32 swap register slots in-lane (bit lives in j),
// distances < 32 use __shfl_xor_sync (bit lives in the lane id). fp32
// accumulation, scale 1/sqrt(kWhtWidth), stores cover the whole block
// including rotated pad cols.
template <typename T, int kWhtWidth>
__global__ void wht_qk_kernel(const T* __restrict__ input,
                              T* __restrict__ output, long num_rows, int d_og,
                              long in_row_stride, long out_row_stride,
                              int blocks_per_row) {
  constexpr int kVec = kWhtWidth / 32;
  // ... 任务分解: row = task / blocks_per_row, col_base = 块内起点 ...
  float x[kVec];
#pragma unroll
  for (int j = 0; j < kVec; ++j) {
    const int col = lane + 32 * j;
    x[j] = col_base + col < d_og ? (float)src[col] : 0.f;   // pad 列读 0
  }
#pragma unroll
  for (int dist = kWhtWidth >> 1; dist >= 1; dist >>= 1) {
    if (dist >= 32) {            // 蝶形位在 j（寄存器内交换）
#pragma unroll
      for (int j = 0; j < kVec; ++j) {
        const int jj = j ^ (dist >> 5);
        if (j < jj) { const float a = x[j], b = x[jj];
                      x[j] = a + b; x[jj] = a - b; }
```

```
      }
    } else {                        // 蝶形位在 lane id（跨 lane 交换）
#pragma unroll
      for (int j = 0; j < kVec; ++j) {
        const float other = __shfl_xor_sync(0xffffffffu, x[j], dist);
        x[j] = (lane & dist) ? other - x[j] : x[j] + other;
      }
    }
  }
  const float s = rsqrtf((float)kWhtWidth);   // 1/sqrt(n) 折进 store
```

D 维 WHT 的 $\log_2 D$ 轮蝶形按**蝶形距离**二分：距离 ≥ 32 时交换发生在同一线程的寄存器槽位间（距离位属于 j 的 bit）；距离 < 32 时落在 lane id 的 bit 上，用 `__shfl_xor_sync` 跨 lane 交换。一个 warp 恰好持有 $kWhtWidth$ 列（lane $\times$ $kVec$），无需跨 warp 通信；$1/\sqrt{n}$ 归一化折进 store 避免额外一轮乘法。launcher（`apply_wht_qk_sm120`）按宽度规则选 $kWhtWidth \in \{D, 64\}$，输出 $kHeadDim$ 宽的旋转副本，含旋转后的 pad 列（28.3.3 节）。图 35.4 把 D=128 的列归属与七轮蝶形的距离二分画在一起。

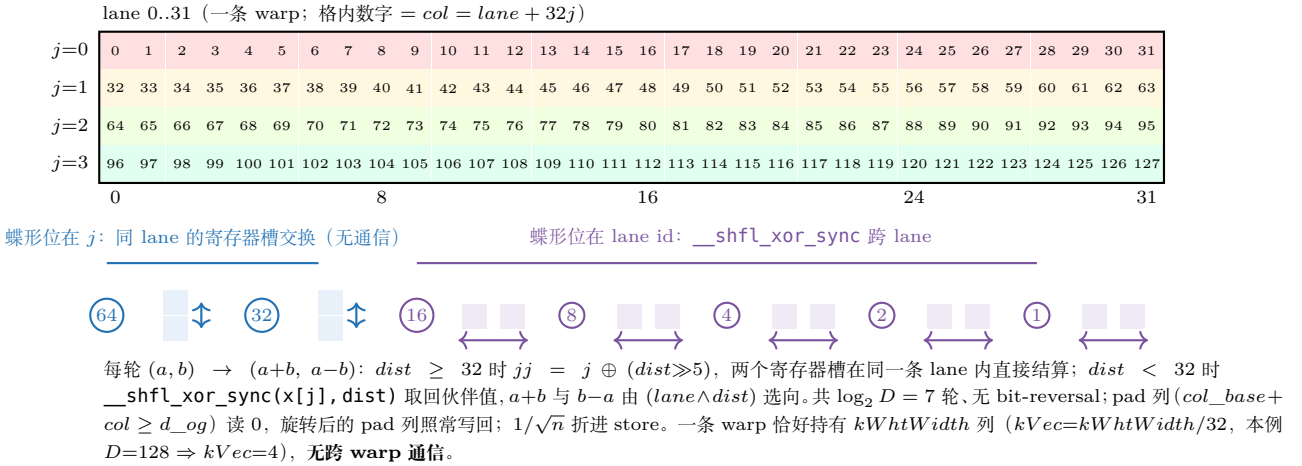


图 35.4：radix-2 蝶形的距离二分（本例 D=128、$kWhtWidth$=D）。上：列归属——j 是寄存器槽、lane 是 warp 内线程，格内数字 $col = lane + 32j$ 满足索引律，右下角 127 = size − 1；一条 warp 覆盖整行 128 列。下：七轮的距离二分——$dist \geq 32$ 的 bit 落在 j 上（同 lane 两个寄存器槽直接加减，无通信），$dist < 32$ 的 bit 落在 lane id 上（`__shfl_xor_sync` 跨 lane）；每轮一个 bit，$1/\sqrt{n}$ 折进 store。

35.5　图示

本章的两张图分工明确。图 35.5 是前处理链的总地图：三条泳道（Q/K/V）从 fp16 输入出发，先取统计量（块 amax、两阶段列均值），再经各自的变换（smooth-K/V、WHT 旋转），汇入量化 kernel 后输出 fp8 数据与 scale workspace（`q_scale`/`k_scale`/`v_scale`，布局随粒度 per-block / per-thread / per-channel 而定）；把它与第 三十六 章的 persist-D 数据通路图叠起来，就是 fp8 attention 从输入到输出的完整数据流。图 35.6 把 35.3.1–35.3.3 节的三条不变变换定理画成一页速查：smooth-K 改 logits 不改概率、smooth-V 改 V 不改 O、WHT 两端同乘正交阵；右侧的 outlier 幅度示意给出变换前后块 scale 的收敛对比。

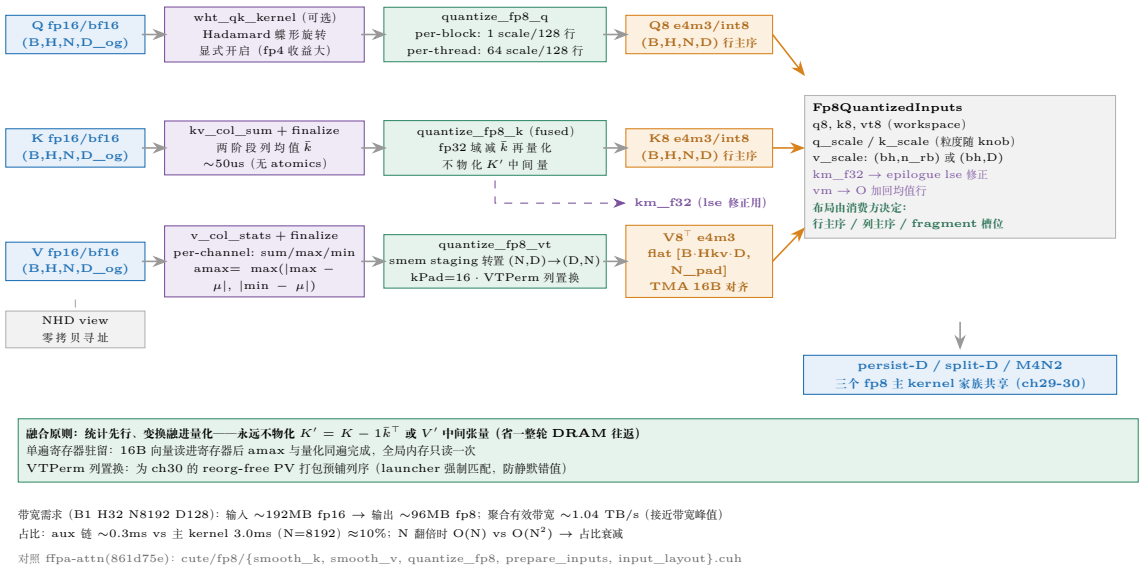


图 35.5: fp8 前处理链全景：三泳道（Q/K/V）各自的统计 → 变换 → 量化 kernel 与输出 workspace 布局。smoothing 与量化融合在同一 kernel 内（无中间落盘）；$V^\top$ 的列 pad 兼顾 TMA 16B 对齐与置换写。



图 35.6: 三条不变变换的代数结构对比：smooth-K 改 logits 不改概率（lse 回加）、smooth-V 改 V 不改 O（行和为 1）、WHT 两端同乘正交阵（收缩逐元素精确）。右侧为 outlier 幅度在变换前后的示意（块 scale 收敛）。

35.6　性能分析

35.6.1　带宽需求：为什么 aux 链必须达到带宽峰值

设 $B \cdot H = 32$、$N = 8192$、$D = 128$、fp16 输入。Q/K/V 总输入字节 $3 \times 32 \times 8192 \times 128 \times 2\,\mathrm{B} \approx 192\,\mathrm{MB}$；量化输出（fp8 + scale）约 96 MB 出 + 192 MB（若物化 smooth 中间量则为再加一轮 192MB 读 + 192MB 写）。本机（RTX PRO 5000，理论带宽 ~1.34TB/s）实测 aux 链聚合有效带宽约 1.04TB/s，这是**纯带宽 kernel 的第一验收指标**。对比 attention 主 kernel（$N = 8192$ 时 3.0ms，365T FLOPS）：aux 链约 0.3ms 量级，占端到端 ~10%；N 翻倍时主 kernel 时间 ×4（$O(N^2)$）而 aux 链只 ×2（$O(N)$），占比随序列长度衰减，所以 aux 优化的优先级在短序列 / 小 batch 场景最高。

35.6.2　每个 kernel 的实测要点

- **列均值**：库调用 ~85us → 两阶段自研 ~50us（B1 H32 N8192 D128，`smooth_k.cuh` 注释实测）——省掉 fp32 cast 的额外往返并同时产出 fp32 副本；
- **per-thread QK**：~49us（$N = 4608$ 量级，D128；数字出自 bench 留档而非源码注释）——每行两线程的网格翻倍是关键（28.4.4 节），冷输入场景 in-flight warp 数直接决定吞吐；
- **fused QKV**：默认路径（`quantize_fp8_qkv_fused_kernel`）把 Q 量化、K 量化（含 smooth）、V 转置量化合进一个 kernel，三次读输入的启动开销与尾部浪费一并省掉；
- **WHT**：一次额外读 + 写（fp16 in-place 语义上），只开 $O(ND \log D)$ 计算，带宽主导，~ 数十 us 量级——fp4 块更小（1×16）、outlier 影响更大，旋转收益最高（bench 默认关，`--fp4-hadamard` 显式开）。

35.6.3　粒度的性能代价与 knob 实测

本章 28.10 节的 A/B（self-attn，D128，$N = 8192$，本机 RTX PRO 5000）：基线 fp8（per-block QK + smooth-K 默认）365T / 1.70×；smooth-V per-channel 369T / 1.72×；强制 per-thread QK 355T / 1.65×。三组差异在 ±2% 量级（与跑次间波动同阶）。量化粒度对**速度**的影响远小于对**精度**的影响，knob 化设计正是建立在这个落差上：精度敏感场景上调粒度，几乎不付性能代价。

35.7　实践经验

- **统计先行、量化融合**：先跑统计 kernel（均值 / 列 amax），再在量化 kernel 里把变换（减均值）与量化合成一遍读写——**不要物化 K' 或 V' 中间张量**，那等于多付一整轮 DRAM 往返。
- **scale 布局跟着消费方走**：per-thread 的 64 槽位表不是随意设定的——它精确等于 C-fragment 行对在 128 行块内的索引。任何「为 kernel 消费方便」的数据排布决策，都值得在前处理期一次做完。
- **确定性优先于最后一档速度**：列均值用两阶段而非 atomics，牺牲的微小速度换来的是数值可复现——精度回归排查（第 三十四 章 27.3.8 节的 A/B 对比方法）依赖这一点。
- **headdim 泛化在行组映射上**：$D = 320/384/448$ 这类非整除组合的坑全在 `kRowsPerIter` 的映射细节(28.4.2 节)；新 headdim 接入要先验证这一映射。
- **冷输入的延迟隐藏**：量化 kernel 的输入往往刚被上游 kernel 写出（cache 冷），「每行两线程」这类提高并发度的朴素手段收益显著，胜过任何复杂的访存模式。

35.8　常见坑

1. **V 的独立布局描述符**：Q/K 与 V 可能来自不同布局（如 Q 是 BHND、V 是 NHD view）。`prepare_fp8_inputs` 对三者各持一个 `Fp8InputLayout`（`Lq/Lkv/Lv`）；早期版本 K/V 共用描述符，NHD 输入时 V 静默读错——数值错但形状合法，只有 allclose 能发现。

2. **Hadamard 要求 BHND 物化输入**：`apply_wht_qk_sm120` 假设 contiguous 输入（行 stride 即 d_{og}）；对 NHD view 直接调用会按错误 stride 寻址。launcher 前置了物化——但自己复用时必须记得。
3. **pad 列的 0/0**：列统计对 pad 列求均值会得 0/0 = NaN；`v_stats_finalize_kernel` 对 pad 列强制 $v_scale = 1$。所有「统计 + pad」的组合都要过一遍这个检查。
4. **kVTPerm 的匹配约束**：置换写只改 $V^\top$ 的列序，主 kernel 侧的 `PackC8bitToA8bitPermVT` 必须同开。launcher 做了强制检查，但绕过 launcher 直接调 kernel 就没有保护——错配是静默错值。
5. **e4m3 的 448 与 f16 累加域**：v_{scale_max} 在 f16 PV 累加路径要压缩（28.3.2 节）；忘了压缩时 f16 域的多块累加会在长序列溢出——表现为「短序列正确、长序列偶发 inf」。

35.9 面试要点速查

1. smooth-K 为什么不改注意力输出？lse 要怎么修？（行平移不变性；lse 回加 $\delta_Q \delta_K\, q_{s,m}$，式 35.2–35.3。）
2. smooth-V 的均值剥离凭什么精确可逆？（$P\mathbf{1} = \mathbf{1}$，式 35.4；输出加回 $\bar{v}$，无 lse 项。）
3. Hadamard 旋转保持 $QK^\top$ 精确不变靠什么？为什么它能降量化误差？（正交消去式 35.6；whitening / Jackson 性质分散 outlier 能量，块 scale 收敛。）
4. per-thread 量化粒度为什么是 64 scale / 128 行？怎么来的？（C-fragment 行对 $\{r, r+8\}$ 在 128 行块内的 8×8 索引；零 shuffle 反量化，28.3.4 节。）
5. $V^\top$ 量化为什么要 smem staging？pad 16 列起什么作用？（读行主序写列主序，两侧都要合并；TMA 16B 对齐 + bank 错开。）
6. NHD 布局怎么零拷贝进量化 kernel？（布局描述符 + 平面基址式 35.9；输出统一 BHND。）
7. 两阶段列均值相比 atomics 方案牺牲了什么、换来了什么？（多一次 partial 读写，换确定性与去掉 zero-init。）

35.10 最小测试与动手实验

本章无独立 kernel 测试（Part VI 以 `ffpa_attn.bench` CLI 为验收），全部数据在 RTX PRO 5000（sm_120）实测，命令在仓库根目录执行：

```
# 环境（Card 7，小 headdim 放行）
$ conda activate cdit
$ export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1

# [A] 基线：默认 fp8（per-block QK + smooth-K）
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn --D 128 --no-bwd --show-allclose
# self-attn fp16  8192/8192  3.01ms  365T  1.70x  allclose OK

# [B] 强制 smooth-K（等价别名 --cuda-impl fp8_smk）
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn --D 128 --no-bwd --show-allclose --fp8-smooth-k
# 3.02ms  365T  1.70x  allclose OK

# [C] smooth-V：per-channel V + 维度均值剥离
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8_smv \
    --tasks self-attn --D 128 --no-bwd --show-allclose
# 2.98ms  369T  1.72x  allclose OK

# [D] per-thread QK 粒度
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn --D 128 --no-bwd --show-allclose \
    --fp8-q-quant-method per-thread --fp8-k-quant-method per-thread
# 3.10ms  355T  1.65x  allclose OK
```

实验 1（粒度谱系）：固定 D128 self-attn，扫 `--fp8-q-quant-method` 三档（per-block / per-channel / per-thread），记录每档速度与 `--show-allclose` 的最大误差档位；验证「粒度升一档、精度相应提升、速度几乎不变」。**实验 2（smooth 开关）**：对比 A/B/C 三组的 allclose 分布（不止看布尔值，把 bench 的误差打印调出来），观察 smooth-K 对 QK 误差、smooth-V 对 PV 误差的分别贡献。**实验 3（aux 占比）**：把 `--tasks` 换成 `cross-attn` 并把 N 从 1024 扫到 8192（`--seqlens`），观察 aux 链占比随 N 的衰减，验证 28.6.1 节的 $O(N)$ vs $O(N^2)$ 论断。**实验 4（WHT）**：fp4 路径（`--cuda-impl fp4`）开关 `--fp4-hadamard`，对比精度：WHT 对 16 元素小块的 outlier 抑制在小 headdim 上最显著（第 三十八 章将回到这个实验）。

延伸阅读

- SageAttention（arXiv:2410.02367）§3：smooth-K/V 的原始动机与精度消融。
- FlashAttention-3（arXiv:2407.08608）§3.3：incoherent processing——Hadamard 旋转与分块量化联合降 FP8 RMSE 2.6× 的出处。
- QuaRot（arXiv:2404.00456）：旋转扩散 outlier 到全坐标的端到端量化框架（比本章的「单层 attention 旋转」更一般化）。
- SageAttention2（arXiv:2411.10958）：per-thread 粒度与 fp8 PV 的工程组合。
- ffpa-attn `cute/fp8/quantize_fp8.cuh` 头注释：int8 对称量化 recipe 的代码级规范（SageAttention `qattn` 的实现对照）。

第三十六章　FP8（二）：persist-D 主 Kernel 与 Scale 折叠代数

36.1　本章导读

第 三十五 章把 fp16 输入变成了量化 workspace（$Q_8, K_8, V_8^\top$ 与 scale 张量）；本章进入主 kernel，即文件 `sm_120/persist_d.cuh` 里的 `persist_d_ws_fwd_cute_fp8_sm120`（sm_120 warp-specialized persist-D，全文件 1036 行）。它是 ffpa-attn 全仓库工程密度最高的单个文件，也是 Part VI 的**核心章**：第 三十七 章的 split-D / M4N2 与第 三十八–三十九 章的 fp4 家族都建立在它引入的机制之上。

本章的主线只有一条：**让所有量化 scale 因子要么精确消去、要么折进本来就要做的操作**。理想目标是「除 round() 外无任何额外数学误差源」：所有反量化都是精确代数操作，不是近似。围绕这条主线展开五组机制：

1. **QK scale 折进 exp2 系数**：$\delta_Q \delta_K$ 与 softmax scale 融成单个 fp32 系数，在 tile max 归约后一次乘入（29.3.1 节）；
2. v_s **在 P 侧精确消去**：P 发射 $\tilde{P} = P \cdot v_s \cdot 448$，gmem 中的 $\hat{V} = V / v_s$ 原样复用，v_s 从 PV MMA 中代数消失（29.3.2 节的三步协议）；
3. **归约轴置换不变性**：P fragment 打包零 SHFL，代价转移到 $V^\top$ 的列置换（29.3.5 节，第 三十五 章预告内容的展开）；
4. **WS persist-D 结构**：128 producer + 256 consumer、`setmaxnreg` 32/232、Q 的 s2r 常驻与 K 槽位 0（Q 区）复用给最后一个 K stage（29.4.2–29.4.3 节）；
5. **发射域规则**：fixed 发射上界 amax(V) vs per-channel 满量程发射 448、满量程发射与 lazy rescale 互斥、f16 PV 累加域整 tile 预算 $kBc \cdot P_r \cdot V_r \le 65504$（29.3.3–29.3.4 节）。

前置章节：第 十八 章（TMA+WS 双流水）、第 二十三 章（TiledMMA 与 fragment）、第 二十四 章（swizzle）、第 三十四 章（量化数学，必须）、第 三十五 章（前处理链，必须）。**源码区间**：`sm_120/persist_d.cuh` 全文、`fp8_pscale.cuh` L72–300、`reg2reg_8b.cuh` 全文、`attn_traits.cuh` L1–110、`softmax.cuh` L44–63；行号以附录 D 的 commit permalink 为准。**验证方式**：`ffpa_attn.bench` CLI，D=64/128 全绿；D=192 本机实测发现正确性 bug，完整定位过程记在 29.7.1 节。

36.2　问题与动机

36.2.1　主 kernel 的输入约定

从第 三十五 章的 `Fp8QuantizedInputs` 出发，主 kernel 消费：$Q_8 \in$ e4m3/int8 (N, kD) 行主序、K_8 同构、$V_8^\top \in$ e4m3 (kD, N_{pad}) 列主序（含 `VTPerm` 置换与 16 列 TMA pad）、三组 scale（布局随粒度）、smooth-K 的 fp32 均值（lse 修正用）。主 kernel 的任务是在 warp-specialized 流水线里完成 $O = \mathrm{softmax}(QK^\top)V$ 的分块在线计算，同时把 scale 的代数消去**融进流水线的每个阶段**，而不是算完后统一反量化。

36.2.2　为什么 scale 消去是核心矛盾

低精度 MMA 只会算 $\hat{P}\hat{V}$ 这样的整数/半字节域乘累加，而 attention 的数学在实数域。朴素的「每个操作数先乘回 scale」方案意味着每 tile 数千次额外 FMUL 与中间张量往返，fp8 MMA 吞吐翻倍的收益会被反量化开销抵消大半。

第 三十四 章 27.3.4 节给过 scale 流向总览；本章把它们逐一落到代码：每个 scale 在哪个寄存器、哪条指令上被消去，以及哪些组合会互相冲突（如 lazy rescale 与满量程 P）。

36.2.3　persist-D 的适用域与家族分工

persist-D 服务 $D \in \{32, 64, \ldots, 224\}$（32 的倍数）：Q tile 整块常驻、K/V 沿序列方向分块流过。D 更大时 Q tile 无法整块驻留寄存器，交给 split-D（M8N1，$224 < D < 768$）与 M4N2（$D \geq 768$）：见第 三十七 章。本机（RTX PRO 5000）编译的 headdim 集为 $\{64, 128, 192, 256, 320, 512\}$，本章 bench 覆盖 persist-D 域内的 64/128/192。

36.3　数学原理

36.3.1　QK 反量化折进 log2 域 softmax

$\hat{S} = \hat{Q}\hat{K}^\top$ 是量化域（e4m3 或 int8）的整数累加，真实 $S = \hat{S} \cdot \delta_Q \delta_K$。softmax 需要的是 $\exp(\text{scale} \cdot S)$；把 $\delta_Q \delta_K$ 与 softmax scale、$\log_2 e$ 融成单个 fp32 系数：

$$\exp\big(\text{scale} \cdot \delta_Q \delta_K \cdot \hat{S}\big) = \exp_2\big(\underbrace{\text{scale} \cdot \delta_Q \delta_K \cdot \log_2 e}_{\text{单个 fp32 系数}} \cdot \hat{S}\big), \tag{36.1}$$

该系数在 **tile max 归约之后**一次乘入（配合 `kMaxScaleAfter`：29.4.3 节），每元素省一次 FMUL。int8 变体走 `s8xs8→s32` MMA，s32 累加器原位 cast 到 f32 寄存器后同样处理。int8 均匀步长使 S 的绝对误差有均匀上界，在近 max（大 $|S|$）的 softmax 主导区相对误差更小，且 s32 累加精确（无 f8f8f32 累加舍入），这是 int8 QK 在大 N 下更优的根源。

36.3.2　P 量化三步协议与 v_s 精确消去

P 只能在线量化（27.2.2 节「P 无法离线」）。`fp8_pscale.cuh` 的三步协议把 v_s（V 的量化 scale）从 PV 乘积中**精确**消去：

$$\hat{P} = \text{round}\Big(\frac{P \cdot v_s}{p_{scale}}\Big), \qquad \text{MMA} = \hat{P}\,\hat{V} = \frac{P\,v_s}{p_{scale}} \cdot \frac{V}{v_s} = \frac{PV}{p_{scale}}, \qquad O \mathrel{+}= \text{MMA} \cdot p_{scale} = PV. \tag{36.2}$$

v_s 消去的直接推论：**gmem 中预量化的 $\hat{V}$ 原样复用**，主 kernel 从不需要 V 的 fp16 原件；v_s 被折进 P 侧的发射域 $\tilde{P} = P \cdot v_s \cdot 448$。fixed 模式下 $p_{scale} \equiv 1/448$（编译期常数），epilogue 用 $(1/448)$/rowsum **一步**完成反量化与归一化。

fixed vs per-row 的取舍：

- **fixed（默认）**：$\tilde{P} = P \cdot v_s \cdot 448$，per-block V 时 $v_s = \text{amax}(V)/448$，发射上界是 $\text{amax}(V)$ 本身而非 448（留有 $448/\text{amax}(V)$ 的 headroom，见 29.3.3）；o_acc 全程单一 448× 域，rescale 可折进吸收 FFMA（29.4.3 节）。代价：$\max(P) \ll 1$ 的平坦行（causal 顶部）只用发射域的一小部分。
- **per-row**（`FFPA_FP8_PQUANT_PER_ROW=1`）：$p_{scale}[row] = \text{rowmax}(P)/448$ 按行自适应归一，每行铺满发射域，精度最优；代价是跨 lane rowmax 归约、64-float o_tile 暂存（寄存器压力），且**与 lazy rescale 互斥**（29.3.3，满量程发射零 headroom）。

36.3.3　lazy rescale 的溢出约束与 fp8 的关闭

FA-4 式 lazy rescale（27.3.6 节）允许 stale max 落后 T 位才补 rescale，期间发射的 P 按 stale max 膨胀 2^T。fixed + per-block V 的发射上界是 $\text{amax}(V)$（29.3.2），lazy 的安全条件是膨胀后不越 e4m3 上限：

$$2^T \cdot \text{amax}(V) \leq 448 \implies T = 4 \Rightarrow \text{amax}(V) \leq 28. \tag{36.3}$$

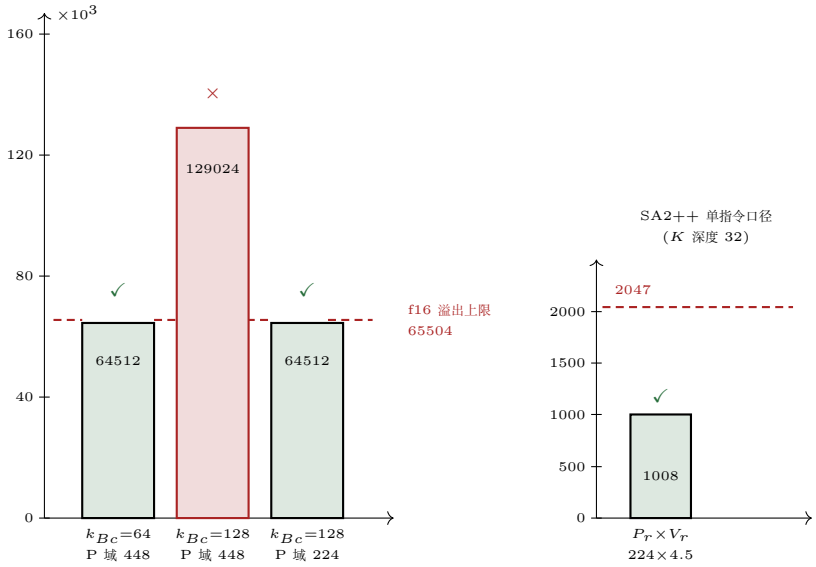
但 ffpa 的 traits 把 fp8 的 `kLazyRescale` 直接置 `false`（`attn_traits.cuh` L31–37）：注释原话，跳过 rescale 会让发射 P 以最高 2^T 膨胀，而 per-channel V 路径（balanced narrowing）的发射 $P \cdot 448$ 已铺满量程、零 headroom，膨胀必然触发 satfinite 钳位、破坏 PV 结果。而 f16 inst_buf 路径把 rescale 融进吸收 FFMA 后「立即 rescale 几乎零开销」，lazy 的动机（省 rescale pass）本身就消失了。**满量程发射与延迟 rescale 不可兼得**，这是发射域规则的第一条。

36.3.4 f16 PV 累加器的域约束

SA2++ 的 `mma.f16.f8.f8.f16` 让 PV 累加器走 f16 域（把 o_acc 从 tensor-core 反馈链上移出来，避开 f8f8f32 累加器 22-bit 尾数损失）。SA2++ 论文的核算口径是**单条 MMA 指令**：$m16n8k32$ 的 K 深度 32，逐元素乘积需满足 $P_r \times V_r \le 65504/32 = 2047$，两侧同收（$P_r = 224, V_r = 4.5$）。ffpa 的累加结构不同：PV 经 `gemm_rs` 跨整个 kBc 块累进同一组 f16 `inst_buf` 寄存器、之后才一次性 FFMA 吸收进 f32 o_acc（29.4.3 节 Phase 4），有效界是**整 tile 累加**：

$$kBc \cdot P_r \cdot V_r \le 65504. \tag{36.4}$$

ffpa 的分配**只收紧 V 侧**：per-channel V + f16 acc 时发射 $\hat{V} \in [-2.25, 2.25]$（即 28.3.2 节的 $v_{scale_max} = 2.25$）；P 侧在 $kBc \le 64$ 时保持 448 满量程（$64 \times 448 \times 2.25 = 64512 \le 65504$✓），$kBc = 128$ 时预算超限，kernel 侧的编译期自动收窄（`persist_d.cuh` L723–732）把 P 域收到 224（$128 \times 224 \times 2.25 = 64512$✓）；f32 acc 时无 f16 溢出界，V 侧取满 448、P 侧恒 448 无约束。



ffpa 的界是**整 tile 累加**：PV 经 `gemm_rs` 跨整个 kBc 块累进同一组 f16 `inst_buf`（之后才 FFMA 吸收进 f32 o_acc），条件为 $kBc \cdot P_r \cdot V_r \le 65504$——$kBc$=64 等效 $P_r \times V_r \le 1023$，$448 \times 2.25 = 1008$ 恰好在上限内；kBc=128 装不下，编译期把 P 域收窄到 224（`persist_d.cuh` L723–732）。SA2++ 的单指令界（$65504/32 = 2047$，K 深度更小）更松。f32 acc 无 f16 溢出界：V 侧取满 448、P 侧恒 448。

图 36.1: f16 PV 累加器的整 tile 域预算（式 36.4）。条形为 $kBc \times P_r \times V_r$：$k_{Bc}$=64 与「$k_{Bc}$=128 + P 域自动收窄到 224」都恰为 64512，接近 65504 上限；k_{Bc}=128 且 P 仍 448 则 129024 约 2 倍超限。右：SA2++ 按单条 MMA（K=32）核算的界 2047 更松（P_r=224, V_r=4.5 双侧收窄）。f32 acc 路径无此约束。

36.3.5 归约轴置换不变性与 reorg-free 打包

softmax 产出的 P fragment（QK MMA 的 C 布局）与 PV MMA 的 A 操作数布局 **不一致**（29.4.4 节有完整布局推导）。传统解法是 16 SHFL + 32 PRMT 的跨 lane 重排。数学出路是归约轴双射不变性：

$$\sum_k P[m,k]V[k,n] = \sum_k P[m,\pi(k)]\,V[\pi(k),n] \qquad (\pi \text{ 任意双射}). \tag{36.5}$$

既然 π 任意，就让 P **留在本 lane**、按本 lane 已持有的字节就地打包（4 条 lane 无关 PRMT、零 SHFL），让 A 操作数携带一个置换的 k 索引 π；V 侧在量化期按 π^{-1} 预置换列序（`VTPermInv32`，式 35.8），两侧槽位对应同一真实 kv 位置即可。row-sum MMA（B 为全 1）在双射下也精确。「前处理链的输出布局由主 kernel 消费方式决定」这条原则落到实处就是：**一条数学等价，把 kernel 内的 16 次跨 lane shuffle 换成了量化期顺带完成的列置换**。

36.4 设计与实现

36.4.1 traits：把配置变成类型

Listing 36.1: persist-D fp8 traits 的关键常量（attn_traits.cuh L9–42，节选）

```
// Persist-D FP8 traits: fp8 e4m3 Q/K/V via SM89 m16n8k32 mma.sync.
// V is pre-transposed by the quantize pre-kernel and stored (D x N), so the PV
// B-operand loads with the same non-transposed LDSM_N atom as Q/K; there is no
// SmemCopyAtomTransposed for fp8. TiledMma K-tile is 32 (atom K), not 16.
// kQKInt8: Q/K become symmetric int8 and QK runs SM80 m16n8k32 s8xs8->s32
// (A/B operand layouts identical to the fp8 atom; acc is s32, cast in-kernel);
// V/PV stay fp8 e4m3.
template <int kHeadDim_, typename Element0_, int kBr_ = 128, int kBc_ = 64,
          int kStagesK_ = 2, int kStagesV_ = 2, bool kQKInt8_ = false>
struct FFPAAttnCuTePersistDFP8Traits {
  // Lazy (FA-4 deferred) rescale is OFF for fp8 precision: skipping a rescale
  // leaves the running max stale, so emitted P is inflated by up to
  // 2^FFPA_RESCALE_THRESHOLD_FP8 against that stale max; with P spanning the
  // e4m3 range (balanced narrowing) the inflation hits satfinite and corrupts
  // the PV result.
  static constexpr bool kLazyRescale = false;
  static constexpr int kSmemElems =
      kBr * kHeadDim + kStagesK * kBc * kHeadDim + kStagesV * kBc * kHeadDim;
```

有四处类型决策要交代。**其一，$V^\top$ 的 B 操作数用与 Q/K 相同的非转置 LDSM_N atom**：V 已在 gmem 里转置好，kernel 里没有转置视角。**其二，TiledMma 的 K-tile 是 32**（atom 的 k16×2），fp16 家族是 16。**其三，swizzle atom 按行字节选**（L45–50）：Q/K 行 D 字节、$V^\top$ 行 kBc 字节，各自落 SW128/SW64/SW32：同一张 smem 图、两类行宽分别选择各自的 swizzle atom。**其四，kBc 随 D 切换**：launcher 对 $D \le 128$ 实例化 kBc=128、$D > 128$ 用 64（smem 预算）。

36.4.2　WS 骨架与 smem 复用

128 线程 producer（TMA 装载）+ 256 线程 consumer（全部计算），__launch_bounds__(384,1)；寄存器经 setmaxnreg 重分配：producer 释放到 32，consumer 得到 232。fp8 路径的 consumer 无条件使用 232（fp16 版在 D=128 时只需 170）：P 量化 + row-sum + rescale 状态推高了寄存器压力，注释记录「静态 170 上限会 spill 约 60 个寄存器」。

Listing 36.2: Q smem 复用与槽位轮转（persist_d.cuh L193–215，节选）

```
  // SMEM: [Q | K stages | V stages], 1B per elem (int8 or e4m3). With
  // kPersistQs2r the Q tile is read into regs once before the kv loop, so
  // its 16KB area (slot 0 of the K region) is reused by the LAST K stage
  // and the Q bytes drop out of the allocation: smem = (kStagesK +
  // kStagesV) K/V tiles, e.g. stages 2 -> 64KB (vs 80KB keeping a dead Q
  // tile). The saving buys L1, not occupancy: smem/L1 share one pool per SM
  // (~4-5us per 16KB), and residency stays register-limited (1 CTA/SM)
  // either way.
  // kPersistQs2r slot rotation: logical stage s -> physical slot (s+1) %
  // kStagesK, so the Q area (slot 0) hosts the LAST stage. That stage's
  // first TMA issue happens in steady state (k_tile == kStagesK-1) and its
  // first consume is tile kStagesK-1, hiding the q_consumed wait behind a
  // full tile of compute; the prologue stages (slots 1..) never touch the
  // Q area and issue with no Q dependency at all.
  constexpr auto kKSlot = [](int s) {
    return kPersistQs2r ? (s + 1) % kStagesK : s;
  };
```

两层设计叠加。**Q 的 s2r 常驻**（kPersistQs2r 默认 true）：Q 在 kv 循环前一次性读进寄存器，之后 QK 步骤全是 gemm_rs（K-only smem 读）：D=128 实测 $-8\mu s$、零 spill；Q 的 smem 区（slot 0）随即空闲。**槽位轮转**让复用区承接「最后一个 K stage」而非第一个：它的首次 TMA 发射与首次消费都发生在稳态（隔着一整 tile 计算），q_consumed 的等待被完全隐藏。省下的 16KB smem 换来的是 L1 容量而非 occupancy：smem/L1 共池，实测每 16KB $\approx$ 4–5μs，而 CTA 驻留本就被寄存器限制在 1/SM。

producer 的装载序还有两处细节：`V prologue 先于 K`（V0 只在 QK0+softmax 之后才被消费，提前发射是安全的）；consumer 在等 Q 之前先 `arrive` 全部初始 `k_empty/v_empty`（这些 arrive 不依赖 Q 数据，提前放行让 K(0)/V(0) 的 TMA 与 Q 的 TMA 在传输中重叠）。

36.4.3 主循环：五个 Phase 的数据通路

consumer 的 kv 循环（`#pragma unroll 1`，逐 tile）：

Phase 1 – QK GEMM（L583–601）：等 `k_full`、fence、`gemm_rs(tCrS, tCrQ, tCrK, ...)`：Q 在寄存器、K 从 smem 流入。int8 变体把 s32 累加器**原位** cast 成 f32（复用同一组 4B 寄存器），零额外寄存器。scale 粒度在读取处分叉：per-block 的 k_s 是每 tile 一个标量；per-thread 的 k_s 按 `(wg_tid%32)%4` 列组查表（4 scale/块，SM80_16x8 C-fragment 的 lane→ 列对映射），q_s 按行查 `qs_arr[row]`（$\{r, r{+}8\}$ 行对共享）。

Phase 2 – log2 域 softmax（L649–790）：mask（causal tail-aligned 的 `mask_start_tile/Tc_eff` 两级剪枝，尾 tile 内逐元素 $-\infty$）；随后 `online_softmax_fp8_fixed` 以 $\log_2(p_quant_scale)$ 为 exp2 偏移，softmax **直接发射** $\tilde{P} = P \cdot v_s \cdot 448$，即式 36.2 的 (1) 步折进 softmax 本身。`kMaxScaleAfter`（int8+f16acc 配置）：tile max 用未 scale 的原始 scores 归约、跨 lane 归约完成后一次乘 $\delta_Q\delta_K$·scale，max 临界路径上每元素省一次 FMUL。

Phase 3 – P 打包（L813–831）：

Listing 36.3: P→e4m3 A 操作数的双路径（`persist_d.cuh` L813–831）

```
// P -> e4m3 A operand (see fp8_pscale.cuh). Per-row mode needs the row
// max first, then scales+converts; fixed mode was pre-scaled by the
// softmax and only converts (packed e4m3x2) + reorgs. kReorgFree swaps
// the cross-lane reorg for the shuffle-free perm pack (paired with the
// permuted V^T written by the quantize pre-kernel).
if constexpr (kReorgFree) {
  PackC8bitToA8bitPermVT perm_pack;
  if constexpr (kPQuantPerRow) {
    pscale_per_row(scores, p_scale);
    quantize_p_frag<true>(scores, tCrSf, vs, p_scale, perm_pack);
  } else {
    quantize_p_frag_prescaled(tCrSf, perm_pack);
  }
} else if constexpr (kPQuantPerRow) {
```

fixed 模式下 softmax 已经发射满量程值，pack 只做 f32→e4m3x2 打包 + 寄存器重排：`kReorgFree` 选择 29.3.5 节的置换打包（4 PRMT、零 SHFL），fallback 的跨 lane `ReorgC8bitToA8bit` 保留编译、由 split-D 家族继续使用。

Phase 4 – row-sum 与 PV GEMM（L833–903）：先在 tensor core 上跑 `pscale_rowsum_mma`（B 操作数全 1 的 MMA 求行和，NCU 显示它藏进 PV 的 tensor pipe 气泡，CUDA-core FADD/shfl 版本实测更慢）。PV 主体分三路：f32 acc 直接 `gemm_rs(o_acc, tCrP, tCrV, ...)`；**f16 acc** 路线是本章指令级融合的关键：PV 先累进 f16 `inst_buf`（把 o_acc 移出 tensor-core 反馈链），随后吸收回 f32 o_acc 时用 FFMA 把 online-softmax 的 rescale 一起折进去：

$$o_acc \leftarrow \mathrm{FFMA}(o_acc,\ row_scale,\ float(inst_buf)), \tag{36.6}$$

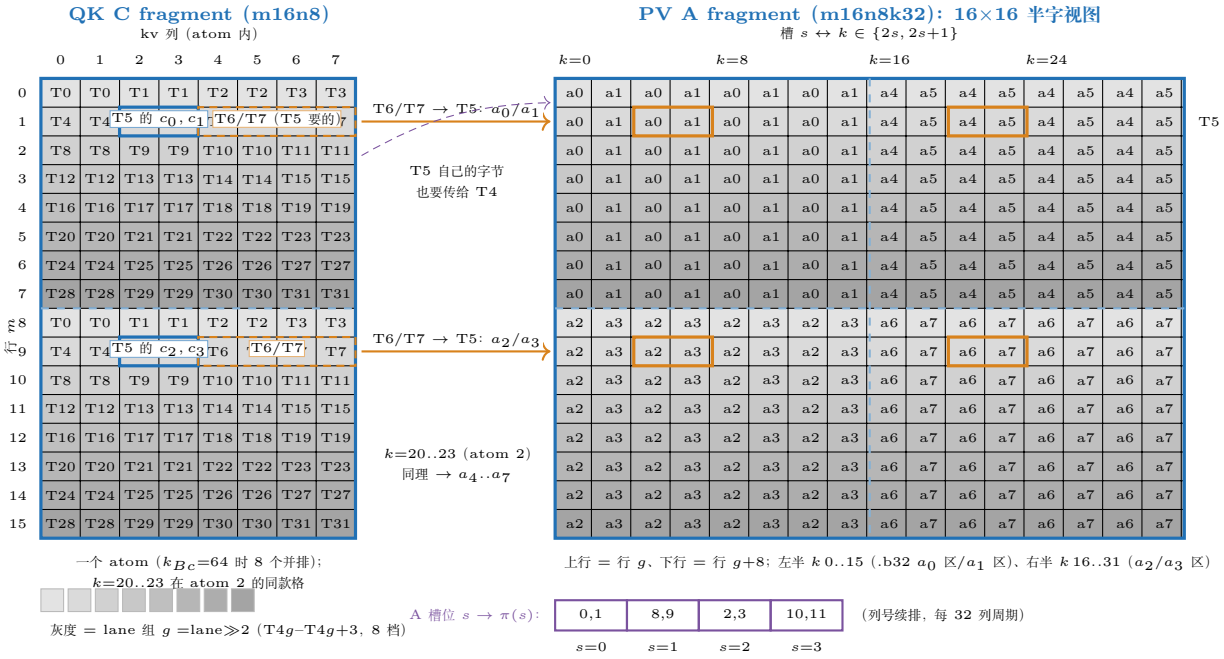
`row_scale` 恰为 1.0 的行（阈值跳过）免乘，不再需要独立的 rescale pass。

Phase 5 – epilogue（L904–1031）：$O = o_acc \cdot (1/448)$/rowsum（fixed 单步反量化 + 归一化）；per-channel V 按 D 列查 $v_{s,d}$ 逐列消去、smooth-V 加回 $\bar{v}_d$；lse $= (\mathrm{rowmax} + \log_2 \mathrm{rowsum})\ln 2$，smooth-K 的修正 $+\mathrm{scale} \cdot q_s \cdot q_{km}$（28.4.2 节的点积在循环外预先计算）。输出走 **STSM 进已释放的 smem（Q/K/V 全部消费完）再 TMA store**，一次合并写出；尾 tile（不满 128 行）退化为带行守卫的直接 R→G 写。

36.4.4 reorg-free 打包的布局推导

`reg2reg_8b.cuh` L14–34 的头注释先回答「C→A 重排为什么必要」：QK 的 C-fragment 与 PV 的 A-fragment 把同一个 16×32 tile 分给 warp 的方式不同（PTX ISA「Matrix Fragments for mma.m16n8k32」，记 $g = \mathrm{lane}\gg 2$、$t = \mathrm{lane}\&3$）。C fragment 有 4 个寄存器 $c_0..c_3$，每寄存器 1 个值、每行 2 列：c_0, c_1 落在行 g 的列 $\{2t, 2t{+}1\}$，c_2, c_3 落在行 $g{+}8$ 的同一列对。A fragment 有 4 个寄存器 $a_0..a_3$，每寄存器 4 个 8-bit 元素、每行 4 个 k 列：a_0 取行 g 的 $k = 4t..4t{+}3$，

a_1 取行 g+8 的同段，a_2、a_3 分别取行 g、g+8 的 $k = 16{+}4t..16{+}4t{+}3$。这样线程 t 需要的 k 列 $\{4t..4t{+}3\}$，换算成 C tile 的列对 $\{2t', 2t'{+}1\}$ 后落在别的 lane（比如 t=1 需要的是 t=2、t=3 持有的字节），纯 lane 内置换无法得到自然 A 布局（`.tmp/fp8_persist_d/verify_layouts.py` 数值证明），因此需要下面跨 lane 的 `ReorgC8bitToA8bit`。



kReorgFree (`PackC8bitToA8bitPermVT`)：A 槽位 s 持真实列 $\pi(s)$（$\pi = [0, 1, 8, 9, 2, 3, 10, 11, \ldots]$），$V^\top$ 由量化期按 π^{-1} 预置换（`VTPermInv32`）——槽位与真实列脱钩后无需跨 lane 借字节：4 条 lane 无关 `__byte_perm`（`0x5410/0x7632/0xDC98/0xFEBA`）、零 SHFL；CONTRACT：绝不许对未置换的 $V^\top$ 运行（launcher 编译期强制匹配）。

图 36.2：C→A fragment 的 lane→ 值映射。左：QK C fragment（m16n8），每 lane 4 值 c_0–c_3 落在行 g/g+8 的列对 $\{2t, 2t{+}1\}$；右：PV A fragment（m16n8k32）的 16×16 半字视图，格值 a0–a7 为 e4m3x2 半字寄存器，四象限对应源码.b32 a_0–a_3；两面板灰度同语义（lane 组 g =lane≫2，8 档）。中：t=1（T5）的自然 A 需要 $k \in \{4..7\} \cup \{20..23\}$（行 g 与 g+8），在 C 视图里属 T6/T7——纯 lane 内置换无法拼出，即 16 SHFL + 32 PRMT（`ReorgC8bitToA8bit`）的根源。下：kReorgFree 的出路——槽位持 $\pi(s)$、$V^\top$ 预置换，4 条 lane 无关 `__byte_perm`、零 SHFL。

C fragment 里 lane t 持列对 $\{2t, 2t{+}1\}$，A fragment 却要列 $\{4t, \ldots, 4t{+}3\}$，后者横跨多个 lane 的字节。`ReorgC8bitToA8bit` 用 16 SHFL + 32 PRMT 的跨 lane 交换解决；而 `PackC8bitToA8bitPermVT`（L96–141）换了一个思路：无需构造**自然**布局——就地打包后 A 槽位 s 持真实列 $\pi(s)$（$\pi = [0, 1, 8, 9, 2, 3, 10, 11, \ldots]$，每 32 列周期），$V^\top$ 列按 π^{-1} 预置换即可。四个 `__byte_perm` 选择器（0x5410/0x7632/0xDC98/0xFEBA）**lane 无关**：没有构造器状态、没有分支，且与 `ReorgC8bitToA8bit` 的对照已用 pycute 数值验证布局等价。注释末尾的 CONTRACT 加了感叹号：*this pack must NEVER run against an unpermuted $V^\top$*；匹配由 launcher 的单一 `reorg_free` 编译期常量强制，两侧永不发散。

36.5　图示

三张图对应本章的三条主线。图 36.3 是数学层的总览：scale 从「显式乘法」变成「折叠」：$\delta_Q\delta_K$ 折进 exp2 系数、v_s 在 P 的发射域消去、p_{scale} 收在 epilogue，除 round() 外不引入任何额外误差源。图 36.4 是这套数学的宿主结构：128 producer + 256 consumer 的 warp specialization、`setmaxnreg` 32/232 的寄存器再分配、Q 的 s2r 常驻与 K 槽位 0（Q 区）复用给最后一个 K stage 的槽位轮转，最后经 STSM + TMA 输出。图 36.5 则是 epilogue 侧的布局技巧：跨 lane 重排（16 条 SHFL）与就地打包 + $V^\top$ 列置换（4 条 PRMT、零 SHFL）的对照，归约轴双射不变性保证两者精确等价。

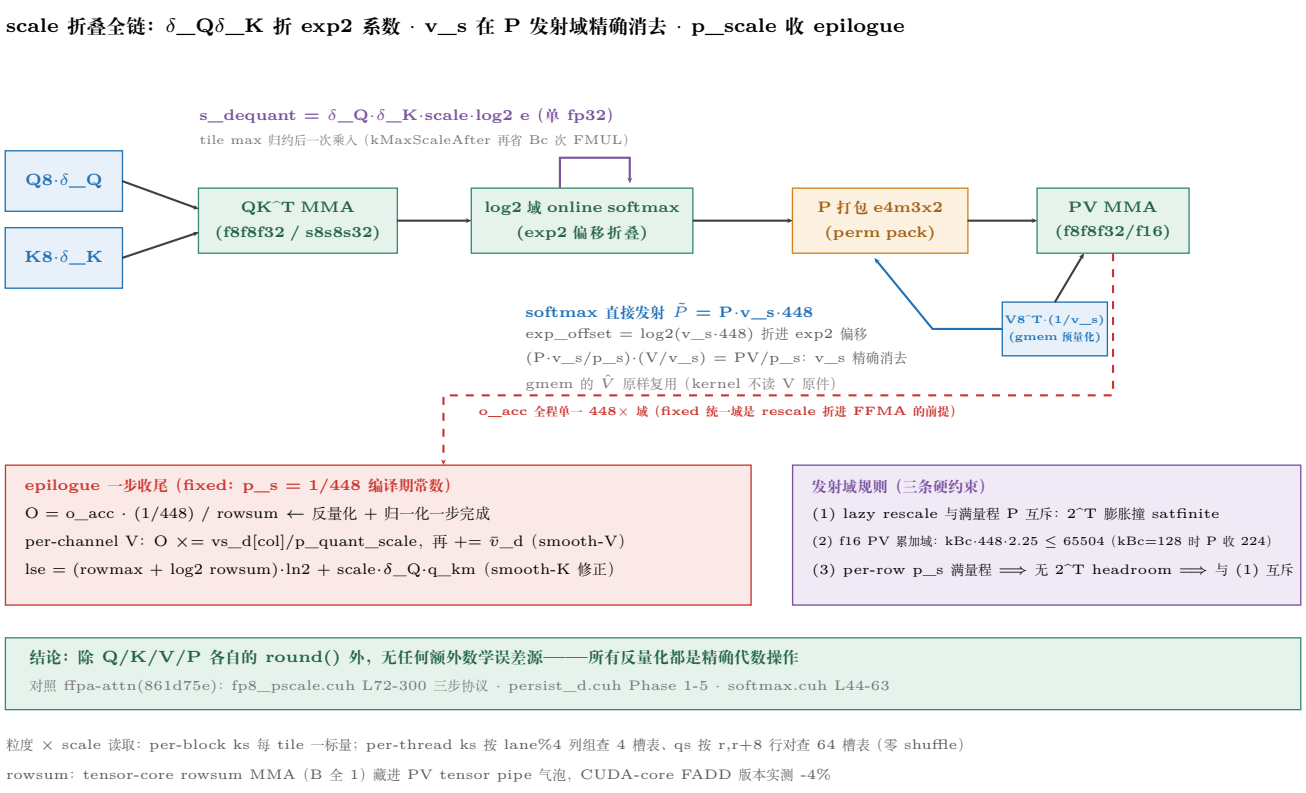


图 36.3: scale 折叠全链数据流：$\delta_Q\delta_K$ 折进 exp2 系数、v_s 在 P 发射域消去、p_{scale} 收在 epilogue——除 round() 外无额外误差源。

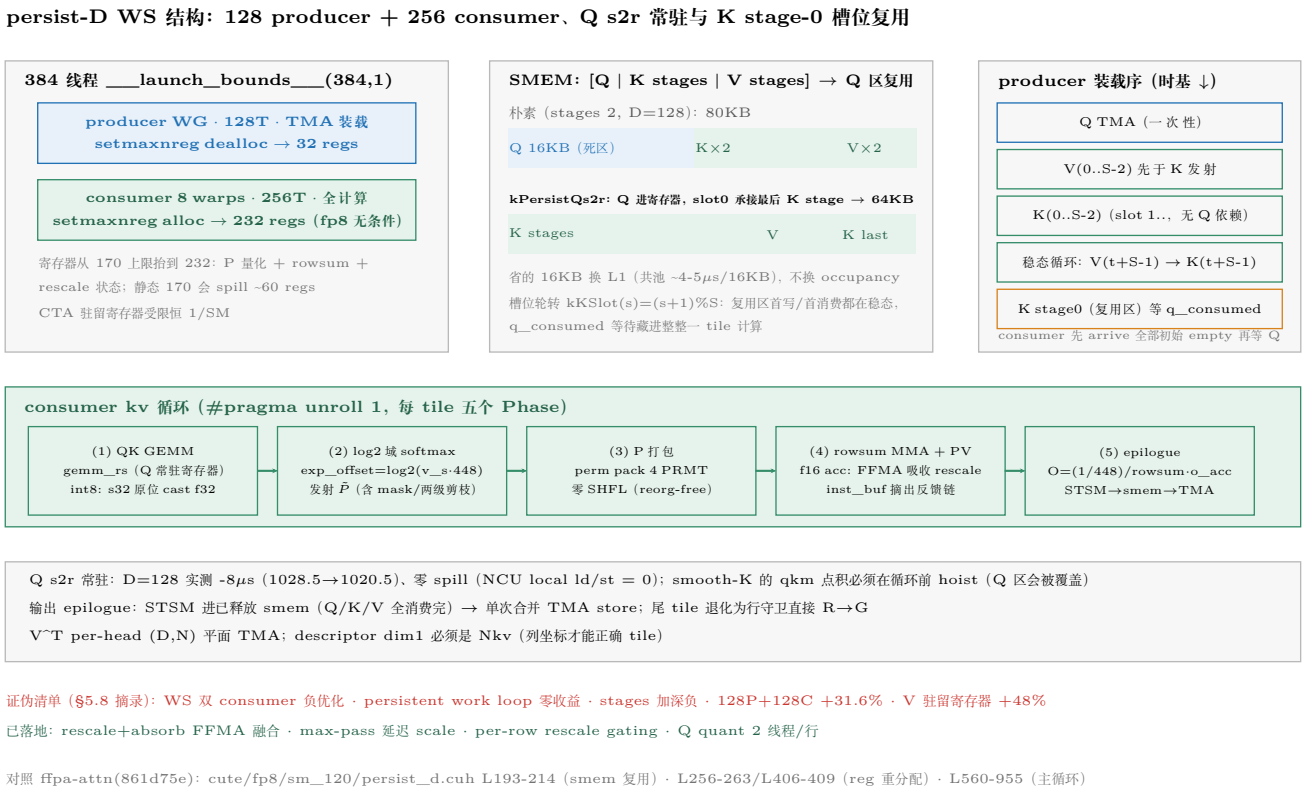


图 36.4: persist-D WS 结构：128P+256C、setmaxnreg 32/232、Q s2r 常驻与 K 槽位 0（Q 区）复用给最后一个 K stage 的 smem 复用、STSM+TMA 输出。



图 36.5: reorg-free 打包对照：跨 lane 重排（16 SHFL）vs 就地打包 + $V^\top$ 列置换（4 PRMT、零 SHFL），归约轴双射不变性保证精确等价。

36.6　性能分析

36.6.1　本机实测（RTX PRO 5000，B1 H32 N8192，bench 默认配置）

D	task	FFPA	SDPA	TFLOPS	speedup
64	self-attn	1.91ms	2.40ms	288T	1.26×
64	causal	1.16ms	1.34ms	237T	1.16×
128	self-attn	3.02ms	5.12ms	365T	**1.70**×
128	causal	1.85ms	2.68ms	297T	1.45×
192	self-attn	4.58ms	7.27ms	360T	1.59×*

*D=192 存在正确性 bug（29.7.1 节），速度数据仅供参考。D=64 的收益小于 D=128 是因为 attention 的算术强度随 D 下降、quantize aux 链占比上升。kernel 级与 SageAttention 对比稳定 +1.1 ∼ 2.3%（仓库 nsys per-kernel 记录）。

36.6.2　微优化上限：证伪清单

persist-D 的结构优化路线已被系统实测证伪（完整清单见仓库 RFC §5.8）：WS 双 consumer（FA3 式 2×128T）负优化；persistent work loop 零收益（tensor-bound 77%）；stages 加深负（K2V2 预取已领先）；v2/v3/v5（128P+128C、无 WS、V 驻留寄存器）分别 +31.6%/+1.4%/+48%（「追平基线」的是 v3 加 v_full wait hoist，即 v4，−0.1%）；cluster 化无 DSMEM 需求、持平或更慢；CUDA-core row-sum 替代 tensor rowsum MMA −4%。**已落地的优化**（D=128）：rescale+absorb FFMA 融合、max-pass 延迟 scale、per-row rescale gating、Q quant 2 线程/行。这份「负结果清单」与正结果同样有用：它把后续优化方向的搜索空间压缩到了 engine 层（aux 链 overlap）。

36.6.3　带宽与算术的边界

D=128、N=8192 时主 kernel 365T FLOPS 对应的 DRAM 流量很小（$O(N \cdot D)$ 量级 workspace 单遍），瓶颈在 tensor pipe 与 softmax 的依赖链，与 fp16 家族同构；但 fp8 的 MMA 吞吐翻倍使 softmax/P 打包的相对占比上升，这

也是 kMaxScaleAfter、FFMA 吸收融合这类**指令级**优化在此路径收益显著的原因。

36.7 实践经验

36.7.1 案例：D=192 正确性 bug 的定位（定位实录）

写本章的 bench 验证时发现 `--cuda-impl fp8 --D 192` allclose 失败（fp16/bf16 双双 ×），而 D=64/128 全绿。定位过程记在这里，可作为同类问题的排查参考：

1. **误差量级分类**：max_abs=2.66 vs mean_abs=0.011——均值是正常 fp8 精度，最大值远超正常量化误差 ⇒ **正确性 bug**（个别行错），不是精度不足；
2. **行模式定位**：per-row max 直方图显示坏行集中在 $row \bmod 128 \in [64, 127]$（每个 128 行 Q tile 的后半）；
3. **N 缩放**：N=512 时几乎全错、N=8192 时只剩 11% 行错——错误随 N 稀释，说明错的是**特定 (row, kv-tile) 组合**而非系统性索引错位；
4. **配置矩阵消去**：默认/纯 fp8 f32acc/int8+f16acc/no-smK/ per-thread/per-channel 全部 × ⇒ 与量化 knob 无关，bug 在 D=192 特有的主 kernel 路径（kBc=64 变体、SW64 swizzle、或 quantize 侧 kBlockRows 与 kBc 的一致性）；
5. **fp16 对照**：fp16 CUTE 路径 D=192 完全正确（max=0.0001）⇒ 输入布局、TMA 与寻址层没有问题，问题范围缩小到 fp8 专属代码。

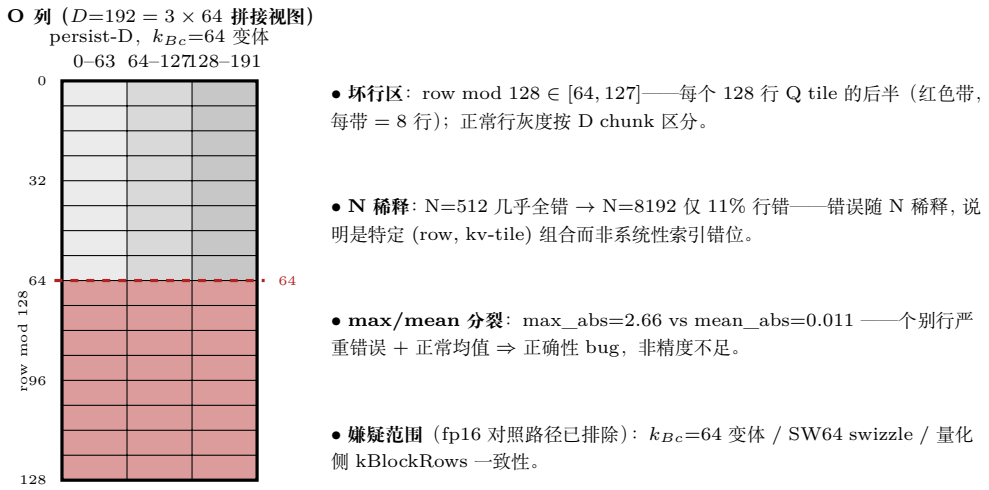


图 36.6: D=192 正确性 bug 的坏行定位（29.7.1 节五步法的可视化）：把 O 视为 3×64 列拼接，坏行集中在 $row \bmod 128 \in [64, 127]$（每个 128 行 Q tile 的后半）；N=512 几乎全错而 N=8192 仅 11% 行错（N 稀释 ⇒ 特定 (row, kv-tile) 组合）；max_abs=2.66 对 mean_abs=0.011 的分裂说明错误集中在个别行。

教训：「**速度好 + 均值好**」**不等于正确**。bench 的 allclose 是唯一硬门槛；误差的 max/mean 分裂是「个别行错」的强信号；行号模 tile 尺寸的直方图能直接把 bug 锚到 tile 结构的哪一半。

36.7.2 考据：sm_89 的 cp.async 变体

同文件名在 `cute/fp8/sm_89/persist_d.cuh` 有一个 Ada 变体：无 TMA/mbarrier，用 sm80 装载协议（cp.async + `wait<2S-1>`）复用 sm120 的计算层（int8 QK / fixed p_scale / f8f8f16 PV），256T non-WS。其 NCU 结论（依赖链/发射受限，`math_pipe_throttle` 1.32 ≫ `wait` 0.82）与深流水零收益的证伪记录，正好构成「同一算法在不同装载协议下瓶颈漂移」的对照。本节不展开，读者按需查阅仓库优化 RFC（PC-16、附录 #22）。

36.8　常见坑

1. **P 是 online 量化，无法离线校准**：Q/K/V 都能离线 smoothing /per-channel 校准，P 每 tile 现算现量化，值域与分布逐行剧变（causal 早行近 one-hot、dense 行近均匀）——只能选固定/保守发射域。这是量化 attention 精度损失的主要根源之一（实测 P 0.11 与 QK 0.13 同量级、V 0.19 最大）。
2. **满量程发射与 lazy rescale 互斥**：per-channel V 的发射 $P \cdot 448$ 铺满量程，stale max 的 2^T 膨胀必然 satfinite（29.3.3 节）；fp8 traits 直接关掉 lazy rescale，且 f16 acc 路径把 rescale 融进吸收 FFMA 后延迟惩罚消失。
3. **kVTPerm 的匹配是编译期规则**：`PackC8bitToA8bitPermVT` 对未置换的 $V^\top$ 运行 = 静默错值。匹配由 launcher 单一常量强制；绕过 launcher 直接调 kernel 没有保护。
4. **f16 PV 累加域**：$kBc \cdot 448 \cdot 2.25 \le 65504$ 的编译期检查不过时 P 域自动收到 224——不理解这条约束会以为 P 域「有时 448 有时 224」是随机行为。
5. **early rows 的 ESS 幅度放大**（27.3.7 节）：causal 前几行 ESS≈1、幅度 3.1σ，fp8 每级 ~5% 相对误差换算成 15× 于 dense 的绝对误差；「早行 fp16 QK」不解决问题（V 量化是最大单项），hybrid 全链 fp16 才有效。
6. **D=192（kBc=64 变体）当前有正确性 bug**：29.7.1 节的定位记录；在修复前避免在生产使用该 headdim 的 fp8 persist-D。

36.9　面试要点速查

1. fp8 attention 的所有 scale 是如何消去的？（$\delta_Q\delta_K$ 折 exp2 系数；v_s 折 P 发射域精确消去；p_{scale} 收 epilogue——式 36.1–36.2。）
2. 为什么 P 的量化只能 fixed/保守，不能离线校准？（P 每 tile 现算、分布逐行剧变；离线对象必须前向开始前存在。）
3. lazy rescale 与满量程 P 为什么互斥？（2^T 膨胀 vs 448 satfinite，29.3.3 节；f16 acc 路径 FFMA 吸收让 rescale 免费。）
4. f16 PV 累加器的域怎么算？（ffpa 跨 kBc 整 tile 累加：$kBc \cdot P_r \cdot V_r \le 65504$，$V_r = 2.25$ 时 kBc≤64 装 P 448、kBc=128 收到 224；SA2++ 单指令口径 $65504/32 = 2047$、双侧收窄 224×4.5。）
5. reorg-free 打包省了什么、代价是什么？（16 SHFL+32 PRMT → 4 PRMT 零 SHFL；代价 $V^\top$ 列置换预写 + 编译期匹配规则，式 36.5。）
6. Q 的 s2r 常驻为什么省钱？省的是 occupancy 吗？（不是——省 smem（16KB Q 区复用）→ 买 L1 共池容量，~4–5μs/16KB；CTA 驻留本就寄存器受限。）
7. fp8 persist-D 的 D 范围与家族分工？（≤224 split-D M8N1 / M4N2，第 三十七 章。）

36.10　最小测试与动手实验

```
# 环境（Card 7，小 headdim 放行）
$ conda activate cdit
$ export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1

# [A] persist-D 域内全绿组合：D=64 / 128
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn,causal --D 128 --no-bwd --show-allclose
# self-attn fp16  8192/8192  3.02ms  365T  1.70x  allclose OK
# causal    fp16  8192/8192  1.85ms  297T  1.45x  allclose OK

# [B] 配置显式声明（int8 QK + f16 PV acc）
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn --D 128 --no-bwd --show-allclose \
    --fp8-qk-mm-type int8 --fp8-pv-acc-type f16
# 3.08ms  357T  1.67x  allclose OK
```

```
# [C] D=192: 正确性 bug 复现 (29.7.1 节)
$ python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn --D 192 --no-bwd --show-allclose
# allclose FAIL  ← 坏行集中在 row%128 ∈ [64,127]
```

实验 1（scale 流向追踪）：把 `persist_d.cuh` 的 Phase 1–5 各自在纸上标出「哪个 scale 在哪条指令上被乘/消去」，对照图 36.3，练到能默写三步协议。**实验 2（发射域 A/B）**：`FFPA_FP8_PQUANT_PER_ROW=1` 重跑 [A]，观察精度提升与速度代价（per-row 归约 + o_tile 暂存）。**实验 3（bug 定位复演）**：按 29.7.1 节步骤用 PyTorch 复现 D=192 的行模式直方图（脚本思路：per-row max diff > 0.1 统计 row mod 128），把「速度好 + 均值好 $\neq$ 正确」固化成检查习惯。**实验 4（lazy rescale 证伪）**：读 `attn_traits.cuh` L31–37 注释，回答「为什么 fp8 关掉 lazy rescale 而 fp16 开着」。

延伸阅读

- SageAttention2（arXiv:2411.10958）：per-warp token 重排与 int8 QK 的原始出处（reorg-free 打包的对照原型）。
- SageAttention2++（arXiv:2505.21136）§3.1：f16 PV 累加域约束（$P_r \times V_r \leq 2047$）的推导。
- FlashAttention-4（arXiv:2603.05451）：lazy rescale 与量化域交互（本章 29.3.3 节的反例来源）。
- ffpa-attn `reg2reg_8b.cuh` 头注释：C→A fragment 布局的 PTX 级推导与 pycute 数值验证记录。
- ffpa-attn 仓库 RFC §5.8：persist-D 结构优化的完整证伪清单。

第三十七章　FP8（三）：split-D 与 M4N2——大 D 的两个瓶颈

37.1　本章导读

第 三十六 章的 persist-D 把整个 K tile（$B_c \times D$）一次装进 smem、让单个 warp 独占 O 的全部 D 列。这在 $D \le 224$ 时是最优解；但 D 继续增大，会同时遇到**两个瓶颈**：

1. **smem 瓶颈**：K tile 字节数 $O(D)$，99KB 预算装不下——由 **split-D**（D 维分块）解决；
2. **寄存器瓶颈**：O 累加器 $O(D)$ 个 f32 寄存器/线程，逼近 255 上限——由 **M4N2 TiledMMA**（atom 布局重排）解决。

两个瓶颈正交：只做 split-D，寄存器照旧 $O(D)$；只做 M4N2，smem 照旧装不下。ffpa-attn 的 dispatch 按 D 三分（`dispatch/cute_fp8.cuh` L60–64）：

D 范围	kernel	atom 布局	O 寄存器/线程
$D \le 224$	`persist_d`（第 三十六 章）	M8N1	$D/2$
$224 < D < 768$	`split_d.cuh`（本章）	M8N1	$D/2$
$D \ge 768$	`split_d_m4n2.cuh`（本章）	M4N2	$D/4$

本章先推导 split-D 的分块恒等式（为什么它是**精确**变换，一个比特都不差），再建立 M4N2 的寄存器压力模型（为什么 N 维划分是数学上的唯一出路），最后走读两个 kernel 的实现差异，特别是 M4N2 带来的多项新代价：P 的 SMEM roundtrip、跨 N-warp softmax 归约、k_{Br} 减半与量化块错位等。所有机制均以 fp8 家族为主线；fp16 家族同构（第 二十六 章已见其 fp16 版），fp4 家族见第 三十九 章。

37.2　问题与动机

37.2.1　两个瓶颈的数量级

smem 瓶颈容易估算：fp8 下 1B/元素，$B_c = 128$ 的 K tile 在 $D = 512$ 时是 64KB，加上 V stage 与 Q 就超 99KB 预算。persist-D 靠「K 槽位 0（Q 区）复用给最后一个 K stage」勉强装下的前提是 $D \le 224$；D 再大，唯一的办法是把 D 切块流水。

寄存器瓶颈隐蔽一些。m16n8k32 atom 的 C-fragment 每 thread 持 4 个 f32；M8N1 布局（8 个 M-warp、N 维单 warp 满编）下单 warp 独占输出的完整 D 列，$k_{Br} = 128$ 时 O 累加器寄存器数是

$$O_{\text{acc}} = \frac{k_{Br} \cdot D}{256 \text{ threads}} = \frac{D}{2} \quad (\text{f32 regs/thread},\ k_{Br} = 128), \tag{37.1}$$

$D = 512$ 时 256 个寄存器**超上限 1 个**（255），属边缘 spill；$D \ge 896$ 大量 spill，性能急剧下降。实测（5090，fp16 家族同构）：$D = 1024$ 时 M8N1 的 $O_{\text{acc}} = 512$ regs 溢出到 local memory，吞吐掉到 ~100T；而 M4N2 同 D 边缘 spill 仍有 154T。

37.2.2　split-D 与 M4N2 各解决一个瓶颈

split-D 回答「smem 怎么装下」：把 D 切成 $D/k_{VDChunk}$ 个 chunk，K/V 的 TMA 与 MMA 都按 chunk 流水，smem 只需常驻 $(k_{Br} + k_{Bc}) \times k_{QKDChunk} + k_{Bc} \times k_{VDChunk}$ 每 stage，与总 D 无关。**M4N2** 回答「寄存器怎么装下」：

把 atom 布局从 $(8,1,1)$ 换成 $(4,2,1)$（4 个 M-warp **乘** 2 个 N-warp），让每 thread 的 O 列持有量减半（37.3.3 节的推导）。

代价各自独立：split-D 多了每 chunk 一次 producer-consumer 握手；M4N2 使 k_{Br} 从 128 减到 64（CTA 数翻倍）、P 必须走 SMEM roundtrip、softmax 需要跨 N-warp 归约。交叉点在 $D = 768$：之下 M8N1 更快（无 roundtrip 开销、k_{Br} 大一倍），之上 M4N2 的寄存器收益开始占优（30.6.2 节）。

37.2.3 causal 精度的已知边界

`split_d.cuh` 的头注释（L40–43）声明了 causal 精度边界：逐元素误差上界（causal 早期行 attend 的 key 少，QK/P 量化误差无法被平均）：O 误差可保持在 10^{-1} 量级但 lse 误差可达 $\sim 6\times10^{-2}$（近 1 概率的 e4m3 舍入），dense 则 $\sim 4\times10^{-3}$（m4n2 头注释无相同声明，但精度结构相同）。本机实测（30.6 节）三个 D 的 causal 误差模式完全一致：序列前 ~30% 的行误差大、其余干净、dense 全绿，这是 fp8 路径的 **已知特性**而非 bug，工程解法是 hybrid（前若干行走 fp16 stage-1，`dispatch/cute_fp8.cuh` L55–58 的 FC-3 注释），不在本章展开。

37.3 数学原理

37.3.1 split-D 分块恒等式：精确，不是近似

设 $D = \mathcal{D}_0 \cup \mathcal{D}_1 \cup \cdots$ 按 chunk 划分。QK 侧 D 是**归约维**：

$$S = QK^\top = \sum_c Q_{:,\mathcal{D}_c}\, K^\top_{:,\mathcal{D}_c}, \tag{37.2}$$

每个 chunk 一次 MMA **累加进同一个** S fragment。fp8 路径 f32 累加；int8 路径 s32 累加（精确，$\ll 2^{31}$）后一次性 cast（L495–503），split 不引入任何新的舍入点。PV 侧 D 是**输出列维**：

$$O_{:,\mathcal{D}_c} = P\, V^\top_{\mathcal{D}_c,:}, \tag{37.3}$$

各 chunk 的 O 段**独立累加**，互不干扰。

关键观察：softmax 的行统计（m_r、ℓ_r）是 **KV 行维的量**，与 D 的切分完全无关：split-D **不产生**「每 chunk 各自的 row max/row sum」，所有 chunk 共享同一组 (m,ℓ)，rescale 因子天然一致。这就是 `split_d.cuh` 的结构（L440–443 注释）：QK chunk 循环在 softmax **之前**全部完成（L446–483），softmax 之后 V chunk 循环各自把 $PV^\top_{\mathcal{D}_c}$ 加进 `o_acc_storage[c]`（L678–756）。

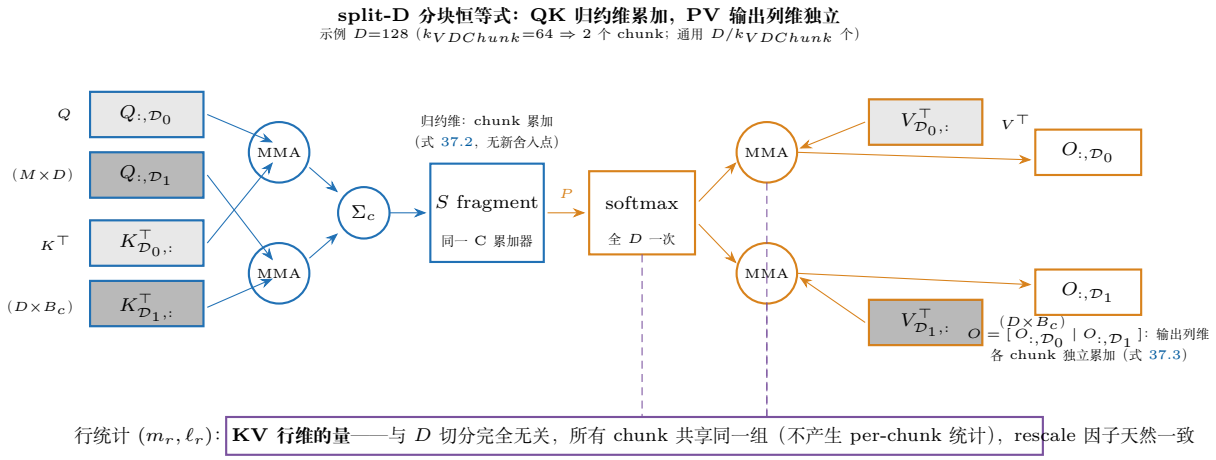


图 37.1: split-D 分块恒等式的数据流（30.3.1 节）。QK 侧：D 是**归约维**，各 chunk 的 $Q_{:,\mathcal{D}_c}K^\top_{\mathcal{D}_c,:}$ 累加进**同一个** S fragment（式 37.2，f32/s32 累加，不引入新舍入点）；PV 侧：D 是 **输出列维**，各 chunk 独立累加出 $O_{:,\mathcal{D}_c}$ 列段（式 37.3），无任何合并。softmax 的行统计 (m_r,ℓ_r) 是 KV 行维的量——与 D 切分完全无关，所有 chunk 共享同一组（灰度 = chunk 身份，两图同语义）。

需要真正「合并行统计」的不是 split-D，而是 M4N2 的 N-warp 划分（37.3.2 节）。

37.3.2　M4N2 的 N-warp 划分与 softmax 合并协议

M4N2 把 B_c（以及每个 V chunk 的 D 列）划分给 2 个 N-warp。于是每行 softmax 的统计量只持有一半：

$$m_r = \max\big(m_r^{(C_0)},\ m_r^{(C_1)}\big), \tag{37.4}$$

$$\ell_r = \ell_r^{(C_0)} + \ell_r^{(C_1)}, \tag{37.5}$$

max 是 bitwise 与顺序无关的；sum 只有 fp32 舍入级差异。但两个 N-warp 交换后**必须持有相同的** (m_r, ℓ_r)，否则后续 rescale 因子不一致、O 列段错位。

实现（`softmax.cuh` L74–231）用一块 1KB 的 `smem_exchange`：布局 [8 warps][16 rows] 两段（max 段、sum 段），peer warp 是 `warp_id` $\oplus\, 4$（翻转 N-warp 位）。协议刻意**只留一个 barrier**：

1. 各 warp 算本 warp 的 tile max（warp 内 `shfl_xor` 1/2 归约），写 max 段，`__syncthreads()`（L103–110）；
2. 读自己与 peer 的 max 取全局，更新 m 与 rescale 因子，发射 $P = \exp 2(s\cdot\gamma - (m - \text{offset}))$，本 warp 的 tile sum 写 sum 段——**不再 barrier**（L112–141）；
3. 调用方紧接着的 P 写读 roundtrip 自带 `__syncthreads()`，一并发布 sum 写；`finalize_row_sum_m4n2`（L147–163）在 barrier 之后读两 warp 的 sum 相加，折进全局 ℓ_r（L700–704）。

省掉的是 3-sync 版本里专门的 sum 交换 barrier（每 KV tile 净省一次 CTA sync，sum 写入的发布复用 P roundtrip 的 barrier）；代价是 max 段与 sum 段 **必须分开**，否则快 warp 可能在 peer 读走 max 之前用 sum 覆写它（L92–95 的 RAW 注释）。fp8 变体 `online_softmax_fp8_fixed_m4n2`（L174–239）在此基础上把第 三十六 章的 offset $= \log_2(v_s\cdot 448)$ 折进 $\exp 2$，tile sum 乘 $1/(v_s\cdot 448)$ 后再写 sum 段，`finalize` 之后 ℓ_r 直接落在真概率域。

lse 只由 `n_warp==0` 写（两个 N-warp 共享同一批 Q 行，`split_d_m4n2.cuh` L970）。

37.3.3　寄存器压力模型：为什么只有减 M_w 有效

m16n8k32 atom 每 warp 产 $[16, 8]$ 的 C-fragment，每 thread 4 个 f32。atom 布局 $(M_w, N_w, 1)$ 下每 warp 沿 M 拿 $k_{Br}/(16M_w)$ 个 atom 行片、沿 N 拿 $D/(8N_w)$ 列（跨 V chunk 累计），逐 thread 的 O 寄存器数是

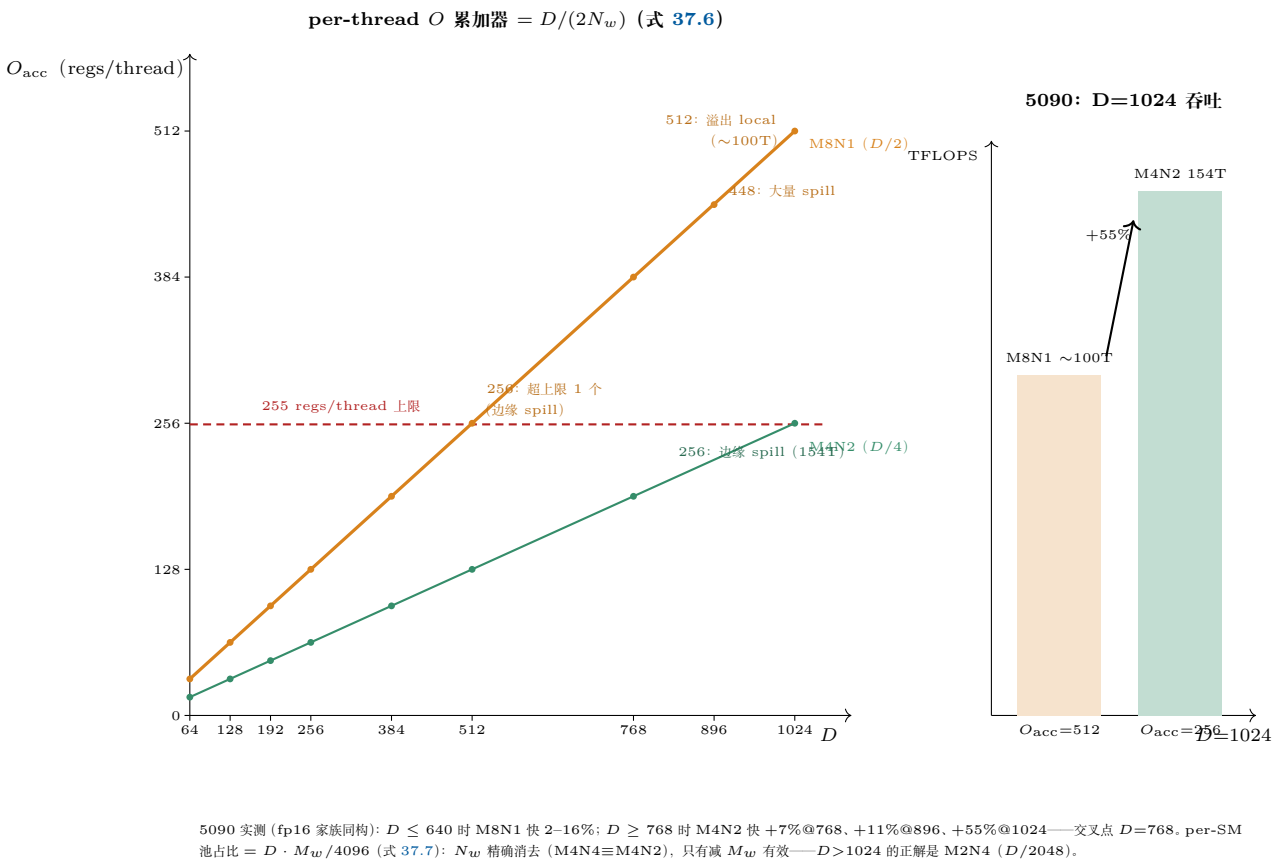
$$O_{\text{acc}} = 4\cdot\frac{k_{Br}}{16M_w}\cdot\frac{D}{8N_w} = \frac{D}{2N_w}\quad(\text{f32 regs/thread}), \tag{37.6}$$

（M8N1/M4N2 都满足 $k_{Br} = 16M_w$，即每 warp 恰好一片 M 方向 atom，第一个因子为 1。）M8N1 给 $D/2$、M4N2 给 $D/4$，与前面估计一致。**每 thread 压力只取决于该 warp 分到的列数** D/N_w。

但真正的判据是 **per-SM 寄存器池**。每 SM 65536 个寄存器，CTA 内 $32M_wN_w$ 个 thread，每 thread 上限 $65536/(32M_wN_w)$。O 累加器对这个上限的占比是

$$\frac{D/(2N_w)}{65536/(32M_wN_w)} = \frac{D\cdot M_w}{4096}. \tag{37.7}$$

N_w **精确消去**：把输出列划分给更多 N-warp，同时把每-thread 上限等比例压低，per-SM 总占用保持不变（M4N4 与 M4N2 同为 $D/1024$ 占比）。只有减 M_w 有效；$M_w = 2$（M2N4）时占比 $D/2048$，是 $D > 1024$ 的正确选择。根源是「$N = 1$ 列方向没有并行收益」的数学结构，与实现细节无关。



5090 实测（fp16 家族同构）：$D \le 640$ 时 M8N1 快 2–16%；$D \ge 768$ 时 M4N2 快 +7%@768、+11%@896、+55%@1024——交叉点 D=768。per-SM 池占比 $= D \cdot M_w/4096$（式 37.7）：N_w 精确消去（M4N4≡M4N2），只有减 M_w 有效——D>1024 的正解是 M2N4（$D/2048$）。

图 37.2: 寄存器瓶颈的定量视图。左：per-thread O 累加器 $= D/(2N_w)$（式 37.6）——M8N1 的 $D/2$ 在 D=512 达 256（超 255 上限 1 个，边缘 spill）、$D \ge 896$ 大量 spill、D=1024 溢出 local memory（~100T）；M4N2 的 $D/4$ 同点仅 256（154T）。右：5090 实测 D=1024 吞吐对比（+55%）。下：per-SM 池占比 $D \cdot M_w/4096$（式 37.7）中 N_w 精确消去——加 N-warp 不降总量，D>1024 只能走 M2N4。

据此也排除了两条看似可行的捷径。其一，**QK 用 M8N1、PV 用 M4N2 的混合布局**：QK/PV 共享同一 Q tile，M 维必须一致；$k_{Br} = 128$ 强制下 PV M4N2 仍是 $D/2$，而 $k_{Br} = 64$ 强制下 M8N1 的 8 个 M-warp 有 4 个在 PV 阶段空转（QK 算力浪费 50%）。其二，**WS（warp specialization）版 split-D**：consumer 的 `setmaxnreg` 上限 232 装不下 $D = 512$ 的 256-reg O 累加器，所以 split-D 家族保持非 WS 结构（全体线程做 MMA，`tid==0` 兼发 TMA，L28–30 注释），这与 persist-D 的 WS 选择形成对照（29.4.2 节）。

37.4 设计与实现

37.4.1 traits：两份预算表

M8N1 的 traits 类与 M4N2 的 traits 类（前者 `FFPAAttnCuTeSplitDFP8Traits`，`attn_traits.cuh` L108–203；后者 `FFPAAttnCuTeSplitDM4N2FP8Traits`，L205–311）共享同一套原子（`SM89_16x8x32` f32 acc、`SM120_16x8x32` f16 acc、`U32x4_LDSM_N` smem 拷贝），差异全部来自几何：

	M8N1	M4N2
k_{Br}/k_{Bc}	128/128（launcher L124–125）	64/64（traits 断言强制）
$k_{QKDChunk}/k_{VDChunk}$	32/64	64/64
线程数	256（$k_{Br}/16 \times 32$）	256（固定 8 warps）
smem 组成	$[Q\|K\|V]$ stages	$[Q\|K\|V\|P\|$exchange$]$
每 stage 字节	16KB	12KB（另加 P 4KB + 交换区 1KB）
stage 上限	6（99KB/16KB）	7（(99KB−5KB)/12KB）

顺带记下两个工程细节。第一，**swizzle atom 按行字节数选**：1B 元素的 64 列行 = 64B → SW64（L152–157 的

`SmemAtomByBytes`），不硬编码。第二，**O staging 预算换算**：`kSmemElems` 数的是 1B 元素（恰等于字节数），而 O 是 2B 的 `Element0`，epilogue 的批量 staging 断言必须显式乘 `sizeof(Element0)`（`split_d.cuh` L105–108、traits L148–149 的 `compute_vchunks_per_batch`）。

37.4.2　M8N1 主循环：chunk 流水的相位规则

`split_d_fwd_cute_fp8_sm120`（956 行）是非 WS 结构：所有 256 线程做 MMA，`tid==0` 在恰当位置内联发 TMA。全局 chunk 计数

```
chunk_index = kv_tile * kDChunks + chunk;    // 全局单调
stage = chunk_index % kStages;               // 环形槽位
phase = (chunk_index / kStages) & 1;         // 相位翻转
```

QK 与 V 两条流水独立计数（$k_{QKDChunk}=32$、$k_{VDChunk}=64$，QK 的 chunk 数是 V 的两倍，各自推进）。每 chunk 循环体的节奏是：等 `full[stage]` → fence → MMA → arrive `empty[stage]` → `tid==0` 若 `chunk+kStages` 未越界则等 empty 后发下一 chunk 的 TMA（Phase 1 L446–483、Phase 3 L678–756）。这与第 二十六 章 Flash Attention 的流水同构：多出来的只是「一个 KV tile 内部再分 chunk」这一层嵌套。

量化侧全部沿用第 三十五–三十六 章的机制，无新增数学：per-thread Q/K scale 查表（L229–245）、smooth-K lse 修正只在 `kv_tile==0` 逐 chunk 累积 `qkm`（L267–274、L465–469）、P 的 reorg-free 打包与 `pscale_rowsum_mma`（L653–676）、epilogue 的 $O = o_{\text{acc}} \cdot (1/448)/\ell$ 与批量 R→S(STSM)→TMA（L759–936）。split-D 域内 $k_{Bc}=128$，f16 PV acc 下 per-channel P 量化域 $128 \times 448 \times 2.25 > 65504$ 会自动收窄到 224（29.3.4 节同机制 constexpr，L426–436；f32 PV acc 则恒为 448，不受此约束）。

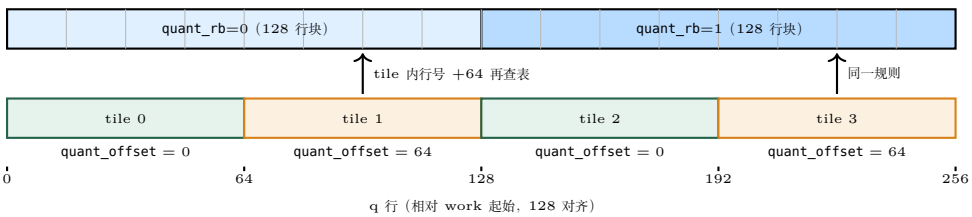
37.4.3　M4N2 的三项新代价

`split_d_m4n2_fwd_cute_fp8_sm120`（989 行）的骨架与 M8N1 相同（`split_d.cuh` 头注释 L28–29 明言沿其 M8N1 设计），差异集中在三处：

P 的 SMEM roundtrip（L685–707）。M8N1 里 P fragment 在寄存器里原地重排（16 SHFL + 32 PRMT，29.4.4 节）；M4N2 每 N-warp 只持半行 B_c 列，warp 内无法重建完整 A-operand，必须经 SMEM 中转（stmatrix→SMEM→`LDSM_N` 的骨架）。注意写侧用的是 `DefaultCopy`（向量化 store）而不是 `stmatrix`：stmatrix 是 b16 操作，要 16B 向量化需 SW128，而 SW128 的 128 元素 atom 除不尽 64 列的 P tile（L685–688 注释）；smem 布局取 SW64 兼容 QK C-fragment 写侧与 PV A-fragment 读侧两个坐标系（traits L279–284）。

跨 N-warp softmax（L655–672 调用 `online_softmax_fp8_fixed_m4n2`）。协议已在 37.3.2 节拆解；`finalize_row_sum_m4n2` 挂在 P roundtrip 的 barrier 之后（L700–704），一次 `__syncthreads()` 服务两个用途。

量化块到注意块的对齐（L129、L300–307）。per-thread Q 量化的 group 按 128 行块编号（$g = (q_{\text{row}}/16)\cdot 8 + q_{\text{row}}\%8$），但 M4N2 的 $k_{Br}=64$：奇数注意 tile 的行号要先加 `quant_offset` 再查表，`quant_rb` 单独换算块号。M8N1 的 $k_{Br}=128$ 恰好与量化块同宽，天然不存在此问题：这类「两套块宽不一致」的坑在 fp8 体系里反复出现（第 三十五 章 per-thread 64-group 同理）。



M4N2 的注意 tile（k_{Br}=64）比量化块（128 行）窄：tile 起始行在块内的偏移 `quant_offset` 为 0 或 64（源码 `split_d_m4n2.cuh` L301–307），tile 内行号必须先加它再代 $g = (q_{\text{row}}/16)\cdot 8 + q_{\text{row}}\%8$ 查 scale 表（索引 = `quant_rb` · 64 + g）；块号 `quant_rb` 由 tile 起始行单独换算。M8N1 的 k_{Br}=128 与量化块同宽，`quant_offset` 恒 0，天然无此事——两套块宽不一致时，错查邻块 scale 的误差温和、难肉眼发现。

图 37.3: 量化 128 行块与 M4N2 注意 tile（k_{Br}=64）的错位关系。tile 1/3 的起始行落在量化块内的偏移 64 处，其行号需先加 `quant_offset` 再查 per-thread scale 表；tile 0/2 偏移为 0。M8N1 的 k_{Br}=128 与量化块同宽，不存在该映射。

37.4.4 证伪记录与默认值

split-D 域的 fused-rescale（rescale 折进吸收 FFMA）自 persist-D 迁移而来，但 61d02c4 在 split-D 的 all-on 复测反而 +8.2% **变慢**（相对 81dbf75 基线，RTX PRO 5000）。复盘（PC-7）发现该结论捆绑了 reorg-free：reorg-free **单独**开启是正收益（D=320 −3.4 ∼ −3.9%、D=512 −0.3 ∼ −0.4%，输出 bitwise 一致），现在 launcher 默认 `reorg_free=true`（`cute_fp8_split_d.cuh` L105–124）；只有 fused-rescale 对保持关闭（`split_d.cuh` L609–618 与 `split_d_m4n2.cuh` L638–641 的缩略注释）。机理：split-D 单 CTA/SM（256 线程）掩盖不了 FADD→FFMA 多出来的 tensor pipe 压力，而 persist-D 是 384 线程 WS 能掩盖。

stage 数的教训方向相反（PC-8）：m4n2 每级仅 12KB，99KB 容得下 7 级，但实测 s2→s3 e2e 快 9 ∼ 13%（D=768、N=4096∼16384，self+causal 合并口径），s4∼s7 非单调更慢：launcher 的 stage 下限从 2 抬到 3（`cute_fp8_split_d_m4n2.cuh` L127–133）。s2 的问题是 K/V TMA 流水饥饿（NCU 里 `@BRA+@NANOSLEEP` barrier 自旋主导）。这与 persist-D 的「加深 stages 证伪」结论相反，根源在于 m4n2 的 occupancy 已被 255 寄存器上限限制在 1 CTA，加深 stage 没有 occupancy 代价，只剩 TMA 遮蔽收益。

37.5 图示

图 37.4 是全章的路线图：大 D 的两个瓶颈，smem 占用随 D 线性增长（$k_{Bc} \times D$ 字节）、per-thread 的 O 寄存器随 D 线性增长，分别由 split-D（chunk 流水，smem 与 D 解耦）与 M4N2（$N_w = 2$ 把每线程 O 从 $D/2$ 减到 $D/4$）解决，$D = 768$ 是两者的实测交叉点，dispatch 据此三分路由。图 37.5 深入寄存器瓶颈解法的内部：$(4, 2, 1)$ atom 的线程-值布局，以及跨 N-warp softmax 的单 barrier 协议：max 段与 sum 段分置 `smem_exchange` 防 RAW，sum 写入的发布复用 P roundtrip 的 barrier。



图 37.4: 大 D 的两个瓶颈与 dispatch 三分路由。smem 瓶颈由 split-D 解决（chunk 流水，smem 与 D 解耦）；寄存器瓶颈由 M4N2 解决（$N_w = 2$ 把 per-thread O 从 $D/2$ 减到 $D/4$）；$D = 768$ 是实测交叉点。

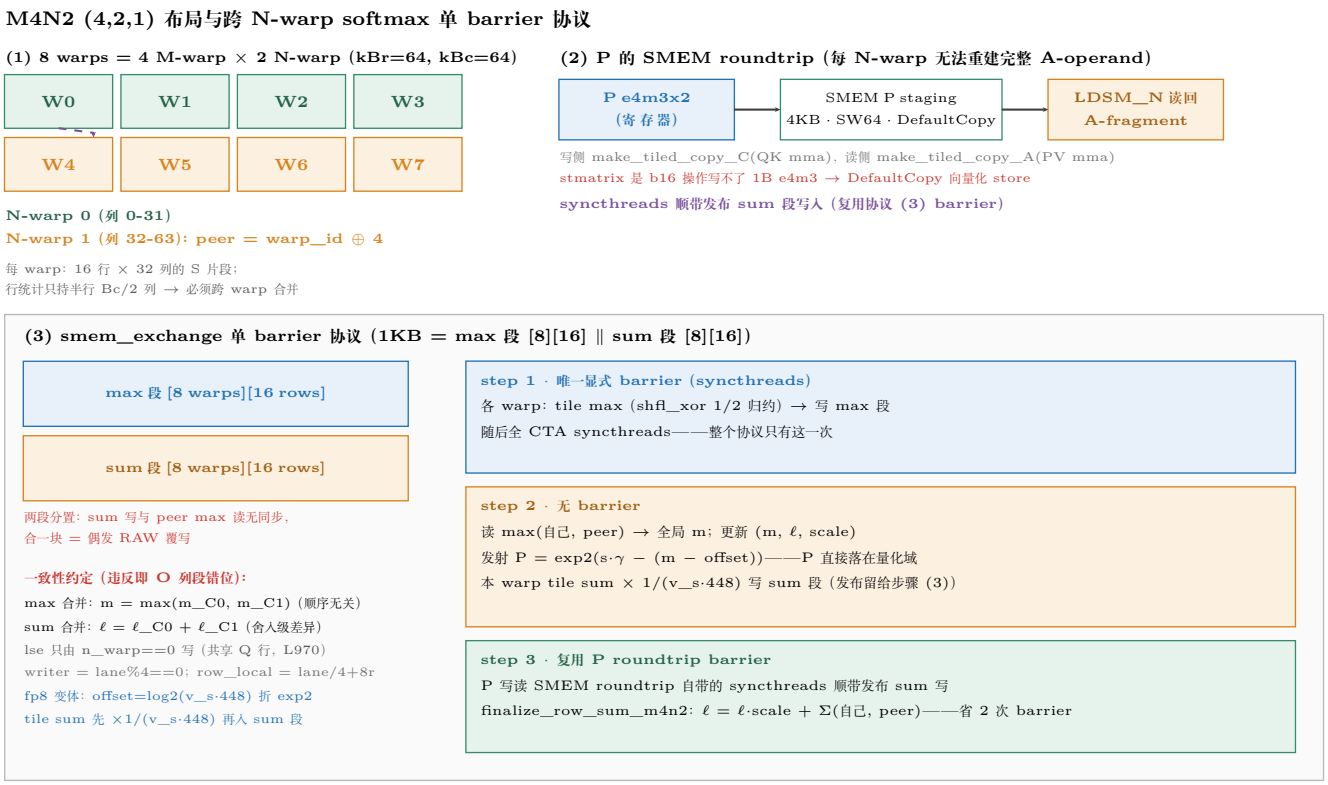


图 37.5: M4N2 的 $(4,2,1)$ atom 布局与跨 N-warp softmax 单 barrier 协议。max 段与 sum 段分置 `smem_exchange` 防 RAW；sum 写入的发布复用 P roundtrip 的 barrier。

37.6　性能分析

37.6.1　本机实测（RTX PRO 5000，B1 H32 N8192，bench 默认配置）

命令：

```
export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
  --tasks self-attn,causal --D {256,320,512} --no-bwd --show-allclose
```

D	task	FFPA ms	SDPA ms	TFLOPS	speedup
256	self	8.12	10.52	271T	1.30x
256	causal	4.73	5.29	233T	1.12x
320	self	10.71	40.25	257T	3.76x
320	causal	6.23	20.61	221T	3.31x
512	self	21.71	64.74	203T	2.98x
512	causal	12.82	32.92	172T	2.57x

self-attn 的 speedup 从 $D = 256$ 的 1.30x 跳到 $D = 320$ 的 3.76x，拐点不在 kernel 而在**参照系**：SDPA 在 $D \notin \{64, 128\}$ 时不再命中最优 kernel，fp8 的 1B 元素带宽优势立即显现。TFLOPS 随 D 单调下降（271→203T），D 越大 softmax 与 P 量化等 $O(B_c)$ 项占比越高、O 寄存器压力越大，这是 split-D 无法消除的固有开销。causal 的 allclose 在 D=320 bf16 与 D=512 fp16 两格临界失败（见 30.2.3 节的已知边界说明），dense 六格全绿，数据落盘 `.tmp/book-bench/ch30/`。

37.6.2　交叉点与 M4N2 的引用数据（本机无 $D \geq 768$ 编译实例）

本机 headdim 编译集止于 512，M4N2 只能引用 5090 数据（fp16 家族同构、同一 dispatch 交叉点）：$D \leq 640$ 时 M8N1 快 2 ∼ 16%；$D \geq 768$ 时 M4N2 快 +7%@768、+11%@896、+55%@1024。劣化一侧的数字更说明

问题：$D = 1024$ 时 M8N1 的 $O_{\text{acc}} = 512$ regs 溢出 local memory，吞吐 ∼100T；M4N2 同 D 边缘 spill 仍有 154T。`FFPA_FP8_FORCE_KERNEL=split_d|m4n2`（$224 < D \leq 1024$，`dispatch/cute_fp8.cuh` L24–33，仅在 `--debug-ext fp8` 的 debug 构建里生效）可在有 768+ 编译实例的机器上强制 A/B 复现交叉点。

37.6.3 roofline 视角

fp8 split-D 在 $D = 512$ 的 203T 大约是 fp8 persist-D 峰值（365T，第 三十六 章 29.6 节实测表）的 56%。差距的构成：O 累加器随 D 翻倍的寄存器开销（occupancy 1 CTA）、chunk 循环多出的 barrier 握手、以及 epilogue 批量 staging 的 smem 往返。这并不构成 split-D 的失败：绝对吞吐仍高于 fp16 家族同 D 量级，带宽侧（1B/元素）更是 3.76x speedup 的直接来源。

37.7 实践经验

37.7.1 案例：causal allclose 临界失败的定位与定性

30.6 节表格里有两格 `allclose` 失败：D=320（bf16）、D=512（fp16）。
定性过程记录如下，脚本在 `.tmp/book-bench/ch30/`：

1. **先看模式不看标量**。对 $D \in \{256, 320, 512\}$ 各跑 causal，逐 512-行块统计超阈行占比：三者模式**完全一致**——前 ∼30% 块 100% 超阈、后段 0%，max ∼2、mean ∼0.007；
2. **dense 对照**。self-attn 六格全绿 ⇒ 排除 kernel 结构性错误（chunk 流水、mask、epilogue 都对）；
3. **对照头注释**。L40–43 声明的「causal 早期行 attend 少、误差不被平均」与实测模式吻合：早期行 P 分布尖，e4m3 的 2^{-3} 相对舍入直接进 O；行号过半后 P 足够平，误差被平均掉；
4. **结论**：已知精度特性，非回归 bug。需要严格 causal 精度的场景走 hybrid（前若干行 fp16 stage-1）。

这套流程（模式 → dense 对照 → 对照声明 → 定性）与 29.7.1 节 D=192 的五步法同构：先证明「不是 bug」再决定要不要修。

37.7.2 什么时候不该学本章

split-D/M4N2 是 $D > 224$ 的**被迫选择**，不是普适更优。三条负面证据：WS 版 split-D 被 setmaxnreg 232 上限否决；QK/PV 混合 atom 布局被 M 维一致性否决；fused-rescale 在 split-D 域被单 CTA 掩盖不了的 tensor pipe 压力否决。三条都是「在 persist-D 域成立的优化不能照搬」的实例，迁移优化前先核对 occupancy 与寄存器结构是否相同。

37.8 常见坑

1. **P roundtrip 写侧坐标系**：写侧必须从 QK 的 `tiled_mma_qk` 构造 `make_tiled_copy_C`，读侧从 PV 的 `tiled_mma_pv` 构造 `make_tiled_copy_A`；手写 A-operand 映射与 LDSM 读侧不对称会产生错误数值（sm80 家族的实测教训，L685–698 用 `DefaultCopy` 规避向量化陷阱）。
2. **max/sum 同区覆写**：单 barrier 协议里 sum 写与 peer max 读之间没有同步，两段必须分置；合成一块省下的 512B 换来偶发 RAW 错值（softmax.cuh L92–95）。
3. **stmatrix 写不了 1B e4m3**：它是 b16 操作；1B 元素的 P staging 退回 `DefaultCopy` 向量化 store（L685–688）。
4. **两套块宽不一致**：$k_{Br} = 64$ vs 量化 128 行块，奇数 tile 需要 `quant_offset`/`quant_rb` 重映射（L129、L300–307）；漏掉的行查到邻块的 scale，误差温和到很难肉眼发现。
5. **kSmemElems 的单位**：fp8 traits 里它数 1B 元素恰等于字节数；O staging 断言要乘 `sizeof(Element0)`，否则预算虚大一倍（L105–108）。
6. **stage 下限的家族差异**：persist-D 抬 stages 已证伪、m4n2 抬到 3 是全局最优——差别只在 occupancy 是否已被限制；跨家族照搬 stage 配置前先想清楚这一条。

37.9　面试要点速查

1. **split-D 为什么精确**？QK 侧 D 是归约维，chunk 累加进同一 fragment（式 37.2）；PV 侧是输出列维，各 chunk 独立（式 37.3）；softmax 行统计与 D 切分无关——不产生 per-chunk 的 (m, ℓ)。
2. **M4N2 减的是哪一维**？N 维（输出列划分给 2 个 warp），per-thread O 从 $D/2$ 到 $D/4$；但 per-SM 池占比 $D \cdot M_w/4096$ 里 N_w 消去（式 37.7），只有减 M_w 才降总量——所以 $D > 1024$ 应选 M2N4。
3. **跨 N-warp softmax 协议**？max 段交换 + 单 barrier；sum 写入的发布复用 P roundtrip 的 barrier，由 `finalize` 折叠；两 N-warp 必须持有相同 (m, ℓ) 否则 rescale 因子分裂。
4. **P 为什么要过 SMEM**？M4N2 每 N-warp 只持半行 B_c 列，A-operand 无法 warp 内重建；roundtrip 4KB/tile。
5. **交叉点为何在 $D = 768$**？之下 M8N1 无 roundtrip 且 $k_{Br} = 128$ 效率高；之上 $D/2$ 逼近/超过 255 寄存器上限，M4N2 的 $D/4$ 开始占优（5090：+7%@768 → +55%@1024）。
6. **split-D 为何非 WS**？setmaxnreg consumer 上限 232 装不下 $D \geq 512$ 的 256-reg O 累加器；全线程 MMA + tid==0 发 TMA 既是被迫也是合理的。
7. **fp8 causal 早期行为何误差大**？attend 的 key 少，P 分布尖，e4m3 相对舍入不被平均；dense 无此问题；工程解法是 hybrid 前若干行 fp16。

37.10　最小测试与动手实验

1. **bench 复跑**（卡 7）：30.6 节命令逐 D 复现表格；观察 self speedup 在 $D = 256 \to 320$ 的跳变并解释参照系切换。
2. **误差模式复现**：用 `CUDABackend(enable_fp8=True)` 直接调 `ffpa_attn_func`，对 causal 输出按 512-行块统计超阈行占比，画出前 30% 块 100% 的直方图（`.tmp/book-bench/ch30/d512_causal_check.py`）。
3. **stage 扫描**：`FFPA_FP8_KV_STAGES="K,V"` 组合重跑 $D = 320$ self，验证 s3 附近的最优与 s4+ 的非单调。
4. **A/B 强制**（需 768+ headdim 编译实例的机器，且需 `--debug-ext fp8` 的 debug 构建）：设 `FFPA_FP8_FORCE_KERNEL` 为 `split_d` 或 `m4n2`，跑 $D \in \{512, 768, 1024\}$，复现交叉点与 D=1024 的劣化/边缘 spill 对比。

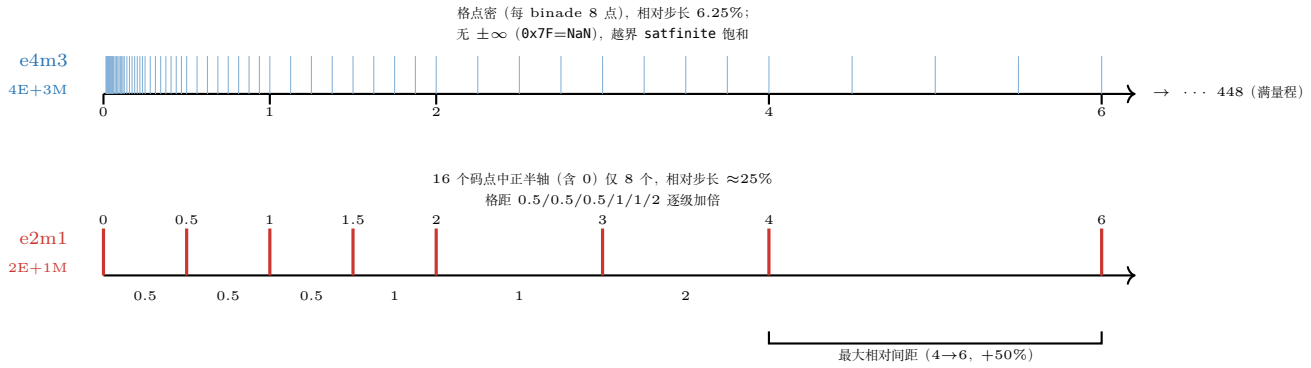
延伸阅读

- `cute/fp8/sm_120/split_d.cuh` 头注释（L17–44）：non-WS 设计声明与 causal 精度边界；
- `cute/softmax.cuh` L74–163：单 barrier 协议与 RAW 分区的原始注释；
- ffpa-cuda-understand skill §11.3–§11.4：双墙模型与寄存器压力表的完整版（含 fp4 家族对照）；
- `launch/cute_fp8_split_d_m4n2.cuh` L127–133 与 PC-8 记录：stage floor 的 NCU 证据链。

第三十八章　FP4（一）：NVFP4 格式与量化链

38.1　本章导读

第 三十四–三十六 章建立了量化注意力的数学与 fp8 实现链。本章把数据格式降到 4 bit。NVFP4 不是「把 fp8 的量化代码换个 dtype」：e2m1 只有 2 bit 指数 + 1 bit 尾数，可表示值只有 $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$，相对步长约 25%（fp8 e4m3 是 6.25%）。在这样的格式上做出可用的 attention，靠的不是数据格式本身，而是围绕它的一整套**尺度处理**：microscaling 块尺度（scale factor）、强制 smoothing、rank-1 修正、列置换、以及把所有这些步骤 fused 进尽量少的 kernel。



两格式均有 ±0，负半轴镜像对称（图示正半轴 $[0, 6]$）。e2m1 无 `inf/NaN`，越界由 `cvt.rn.satfinite.e2m1x2.f32` 饱和到 ±6。格点稀疏度 25% vs 6.25% 正是 fp4 容差（dense 0.15 / causal 0.70）比 fp8（5e-2）宽一个量级、以及 causal 尖峰行 max_abs 0.5–0.75 的直接来源（第 四十 章 33.3.3 节的 tol 分档依据）。

图 38.1: e2m1 vs e4m3 的格点数轴（同一 $[0, 6]$ 区间）。上：e4m3 每个 binade 8 个格点、相对步长 6.25%，满量程 ±448；下：e2m1 正半轴全部格点仅 $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$、相对步长约 25%，$[4, 6]$ 段相对间距最大（+50%）。fp4 的容差天然比 fp8 宽一个量级的格式层根源。

ffpa-attn 的 fp4 路径移植自 SageAttention3 的 Blackwell kernel （`quantize_fp4.cuh` 头注释 L1–14），跑在自研的 persist-D 骨架上。本章讲**量化链**：主 kernel 之前的前处理；第 三十九 章讲主 kernel 与两级 P 量化。源码锚点：`cute/fp4/quantize_fp4.cuh`（1642 行）、`delta_s.cuh`（451 行）、`fp4_gemm.cuh`（276 行）、`attn_traits.cuh`（658 行）。

38.2　问题与动机

e2m1 的动态范围限制。fp8 路径里 per-block/per-thread scale 已经把相对步长压到 6.25%；e2m1 再去掉 2 bit 尾数与 3 bit 指数后，单块内 amax 与其余元素的分辨率差距被放大到无法忽略。更关键的是 **分布中心**：attention 的 Q/K 行向量常带系统性均值偏移（randn 输入下均值不为零，真实模型里更明显），若直接量化 $Q \to \hat{Q}$，均值占据 amax 预算，把有效信息压进 1–2 个码点。fp8 的 smooth-K 是可选旋钮（第 三十五 章），fp4 的 smooth-Q/K 是**强制开启，不是选项**（skill §6.2）：e2m1 的 ±6 动态范围使均值平滑成为精度必需。

三条出路叠加。ffpa 的答案是把三种尺度手段全部用上：

1. **microscaling**：每 16 个元素共享一个 ue4m3 scale factor，块内 amax 归一到 e2m1 满量程 6；
2. **smoothing**：Q 减 128 行块均值 q_m、K 减序列均值 $\bar{k}$（强制），V 减列均值 v_m（可选），均值的影响走精确的 fp32 旁路（ΔS 修正项与 lse/epilogue 加回），不占量化预算；

3. **hadamard 旋转**（可选）：行内 WHT 把通道间 outlier 方差摊平（第 三十五 章 §11.9 同机制，fp4 把它 fused 进量化 kernel）。

P 的困难留给下一章。Q/K/V 都能离线校准，唯独 P 是 kernel 内 softmax 现场产物，其 scale 只能在固定值域内设计：SA3 的两级 P 量化是 fp4 路径精度成立的关键，数学与实现见第 三十九 章。

38.3　数学原理

38.3.1　NVFP4：e2m1 数据 + ue4m3 块尺度

microscaling 格式的量化算子 ϕ 作用于 1×16 的元素块：

$$s_{block}=\frac{\text{amax}(16\ \text{元素块})}{6},\qquad \hat{x}=\text{round}\left(\frac{x}{s_{block}}\right)\in[-6,6] \tag{38.1}$$

分母 6 是 e2m1 满量程；s_{block} 编码为 **ue4m3**（无符号 e4m3，值域 $[0,448]$）。硬件 blockscale MMA 的语义是逐块 dequant 隐含在乘法里：

$$\text{FP4MM}(\hat{A},s_A,\hat{B},s_B)\ =\ \text{逐块}\ \big(\hat{A}\,s_A\big)\big(\hat{B}\,s_B\big) \tag{38.2}$$

即 SM120 的 `m16n32k64 kind::mxf4nvf4 ue4m3 scale_vec::4X` 指令（`attn_traits.cuh` L30–37 注释、L77 的 `MMAAtom`）：MMA 读入的是 4 bit 数据 + SF 旁路，累加时按块乘回尺度。MXFP4（e2m1 + ue8m0/32 块）在 SA3 Table 1 中精度低于 NVFP4（ue8m0 只有 8 bit 指数无尾数，SF 本身步长 100%），ffpa 未采用。

SF 的双重身份。s_{block} 既是量化除数又是 dequant 乘数，而 ue4m3 只有 3 bit 尾数，SF 自身的舍入误差（6.25%）会整体作用于 16 个元素：若块 amax 落在 ue4m3 的窄域之外，或行内动态范围极端（causal 早行 P 近 one-hot），SF 编码误差就会放大。两级 P 量化（下一章）正是 SA3 对 C2 挑战（P 以小值为主的分布使块 amax 难铺满 SF 域）的回答：把块的 amax 铺满 ue4m3 的域。

38.3.2　smooth-Q/K 强制与 delta_s 的 rank-1 恒等式

Q 的中心化按 128 行块取均值：$q_m[b,h,m_b,:]=\text{mean}(Q)$（`fp4_q_block_mean_kernel`，L667–701）；K 平滑按序列均值 $\bar{k}[b,h,:]=\text{mean}(K)$。量化对象是残差 $\hat{Q}=\phi(Q-q_m)$、$\hat{K}=\phi(K^\top-\bar{k})$，score 的精确重构需要补上均值项：

$$S=q\,k^\top=(q-q_m)\,(k-\bar{k})^\top+q_m k^\top+q\bar{k}^\top-q_m\bar{k}^\top \tag{38.3}$$

直接展开有四项。SA3 的改写是把它压成「残差 MMA + 一个 rank-1 修正项」：

$$S=\hat{Q}\hat{K}^\top+\Delta S,\qquad \Delta S[b,h,m,n]=q_m[b,h,m,:]\cdot\big(K[b,h_{kv},n,:]-\bar{k}\big) \tag{38.4}$$

（GQA 下 K/V 按 kv 头 h_{kv} 索引，MHA 时退化为 $h_{kv}=h$。）而式 38.4 右边的 $K-\bar{k}$ 必须避免物化，它等于

$$\Delta S=\underbrace{q_m K^\top}_{\text{低秩 GEMM}}-\underbrace{q_m\bar{k}}_{\text{每行常数}q_{km}} \tag{38.5}$$

`delta_s.cuh` 头注释（L3–8）明确给出这个恒等式：kernel 的 $S=\hat{Q}\hat{K}^\top+\Delta S$ 恰好等于 $q(k-\bar{k})^\top$，于是 lse 修正只需要 $scale\cdot\text{dot}(q_{row},\bar{k})$（任何多余的行常数会平移 lse 但不改变 O）。ΔS 在量化链里一次算好（fp32 物化，pad 列零填），主 kernel 每个 kv tile 加载即用。

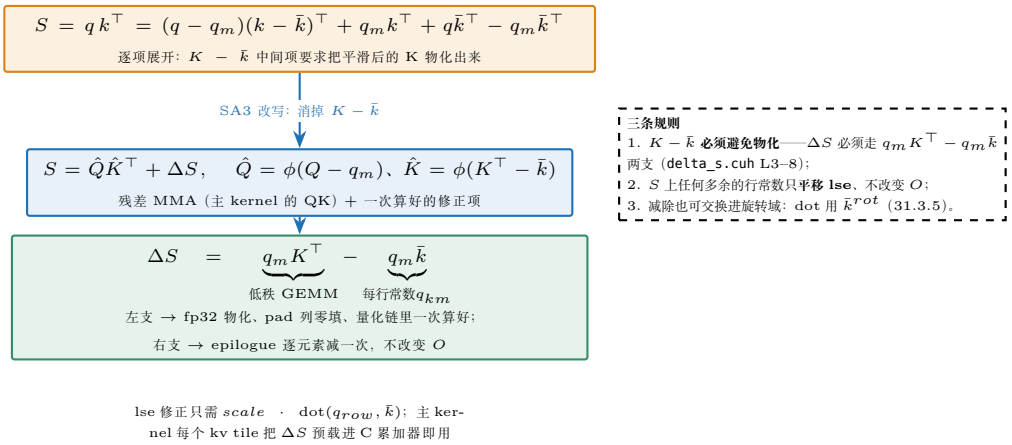


图 38.2: ΔS 的 rank-1 恒等式。四项展开折叠为「残差 MMA + 修正项」：ΔS 由低秩 GEMM $q_m K^\top$ 与每行常数 $q_m\bar{k}$ 两支等价构成，$K - \bar{k}$ 永不物化；行常数只平移 lse。

38.3.3 smooth-V 的恒等式

V 的列均值减除是**数学精确**的，不是近似：$v_m[d] = \frac{1}{N_{kv}}\sum_n V[b,h,n,d]$（per-(b,hkv,d) 常数），量化残差 $\hat{V} = \phi(V - v_m)$，epilogue 加回。因为 softmax 权重和为 1、v_m 对所有列相同：

$$O_i = \frac{\sum_j P_{ij}\left(\hat{V}_j + v_m\right)}{\sum_j P_{ij}} = \frac{\sum_j P_{ij}\hat{V}_j}{\sum_j P_{ij}} + v_m \tag{38.6}$$

（`quantize_fp4.cuh` L437–455 的注释给出同一推导）。减除只缩小残差动态范围，让 e2m1 块量化的相对分辨率更细。fp8 的 smooth-V 走 per-channel scale，fp4 因 e2m1 无 per-channel 余地，改用均值减除形式（`--fp4-smooth-v`，Q/K/V 三段量化 kernel，含独立 $V^\top$、MXFP8 $V^\top$ 与 fused 单 launch 变体，均支持）。

38.3.4 pad 零贡献与 hadamard 的线性性

head_dim pad。任意 $D\%8 = 0$ 经 64 对齐 pad 支持（$D_{pad} = (D+63)\&\sim 63$）：量化 kernel 读原始宽度 d_{og}、pad 列零填，Q/K/V 不物化 pad（仅输出 O pad 后切回）。pad 列的语义是 **data=0 且 SF=0**：ue4m3 的 bits-0 是合法编码，零数据乘任何 scale 仍是零，MMA 贡献恒为 0，QK 的 D 归约与 PV 的 pad 行都不会引入错误值（`quantize_fp4.cuh` L158–172 的 guarded load + L10–12 注释）。workspace 另 pad 到 128 token 倍数（L10–12），tail 行同样零填，TMA 加载永远合法。

hadamard 融合的可行性。WHT 线性（$\mathrm{mean}(Hq) = H\,\mathrm{mean}(q)$，$HH^\top = I$）⇒ 均值与旋转可交换：$q_m$、$\Delta S$、$q_{km}$ 全部可以在**未旋转域**计算；旋转域里需要替换的只有两处：Q 段量化 bias 的减除改用旋转副本 q_m^{rot}，lse 的 $\mathrm{dot}(q_{row}, \bar{k})$ 项用 $\bar{k}^{rot}$（L101–108 注释）。于是 WHT 旋转可以 fused 进量化 kernel 的 load 路径（行内旋转零拷贝），量化链的 kernel 数不变。

38.4 设计与实现

38.4.1 Q/K 量化 kernel：4 线程一 token

`fp4_quant_kernel`（L109–320）的映射：每 token 4 个线程，每线程持 $kHeadDim/4$ 个元素、$kHeadDim/64$ 个 16 元素向量迭代（D=256 时恰 64 元素 = 一个 SF atom 的 4 列组）。四个模板 bool 各管一件事：`kPermute`（K 的列置换，见 31.4.5）、`kSubQm`/ `kSubKm`（bias 减除在 group max 之前原位完成）、`kHadamard`（fused WHT，31.4.3 节）。

fp32→e2m1 的打包用一条 PTX：`cvt.rn.satfinite.e2m1x2.f32` 把相邻两个 float 转成一对 4 bit 码点（L32–56 的 `fp32_vec_to_e2m1`，8 float → 1 `uint32`）；`satfinite` 保证越界饱和到 ±6 而不是 NaN。SF 的编码同样走饱和转换：NVFP4 的 SF 用 `__nv_fp8_e4m3` 构造（L291、L457），mxfp8 数据路径才用 `cvt.rn.satfinite.e4m3x2.f32`（L58–71）。

一个必须处理的边界：0 块。全零块（pad 列、全 masked 组）的 $s_{block} = 0$，除法变 0/0。实现里是

```
float SFValueInv = (SFValue == 0.0f) ? 0.0f : 1.0f / SFValue;
```

（L293、L460）：倒数为 0 使数据也归零，SF 字节本身写 0，dequant 端 $0 \times 0 = 0$ 自洽。这个特判的覆盖面极小，却是全 masked 块不产 NaN 的唯一守卫。

38.4.2　$V^\top$ 量化：smem 转置 + SF atom 布局

V 在主 kernel 里是 PV 的 B 操作数，需要 $(B, H, D, N/2)$ 的转置布局。`fp4_quant_trans_kernel`（L322–496）分两步：先按 token-major 协作读进 smem staging（每线程原 slice），同步后每线程改持一个 $(d, 16\text{-token})$ SF 组、按 d 行迭代产出。staging 窗口随 D 分档：≤128 用 128 token、≤768 用 64、更大 32（L342–343），超过 48KB 静态上限（$D \geq 512$）切换 dynamic smem opt-in（L350–351 的 `kDynamicSmem`）。

SF 字节的写序不是朴素行主序，而是 **SfAtom 分块序**：MMA 的 SF 操作数按 64×4 block 布局消费，写侧 offset 公式必须与 `BlockScaledConfig` 逐字节一致（头注释 L13–14、L414 的 `sf_col_offset` 与 L482–485 的 `offset_local` 公式）。这同 fp8 的 VTPerm 是同一类问题：**消费端的寄存器/smem 布局决定生产端的写序**，两头任一侧改动都要同步。

smooth-V 也 fused 在这里：v_m 按 d 减除发生在 smem 读出转 fp32 之后、group absmax 之前（L443–456），一遍数据完成「转置 + 减均值 + 量化 + SF 打包」。

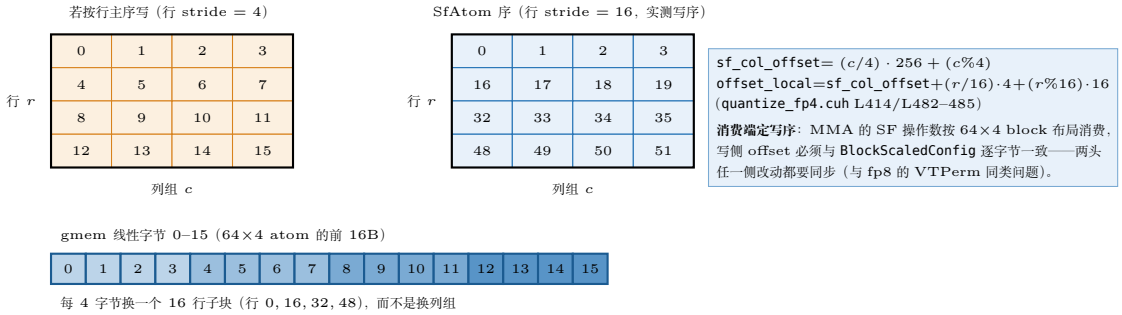


图 38.3: SF 字节的写序由消费端决定（$V^\top$ 量化链）。同一 (r, c) 单元格在行主序假设下偏移 $4r + c$，在 SfAtom 序下偏移 $16r + c$：每 4 个列组构成 64×4 的 256B atom，atom 内按 16 行子块交错。写侧（`sf_col_offset`/`offset_local`）与 MMA 的 `BlockScaledConfig` 必须逐字节一致。

38.4.3　fused 单 launch（$D \leq 128$）与 fused WHT

小 D 场景把整条 Q/K/V 量化链合成一个 launch：`fp4_quant_fused_qkv_kernel`（L1024 起）。grid 的 blockIdx.x 切成三段 [Q tiles | K tiles | V tiles]，Q 段同时产出 q_m：均值块恰等于量化 tile，归约是块内 smem 归约，原先整张量扫一遍的 `q_mean` pass 直接消失（L1026–1032 注释）。`kHadamard` 下 Q 段还在 smem 里对 q_m 做行内 WHT（ping-pong 蝶形），预旋转 bias q_m^{rot} 不再需要独立 launch；$\bar{k}^{rot}$ 因输入 $\bar{k}$ 早已就绪，保留外部小 kernel（L1032–1036）。

fused WHT 的蝶形拆分记在这里（L195–235）：token 的 4 个线程连续分布，每线程 $k_E = kHeadDim/4$ 个元素，距离 $< k_E$ 的蝶形对全在 thread-local 数组里原地做（`dist` 从 $k_E/2$ 递减到 1），只剩距离 k_E 与 $2k_E$ 的两级跨线程交换用 `shfl_xor(1)`/`shfl_xor(2)` 完成，整行 WHT 零 smem、零额外寄存器数组。最后乘 $rsqrt(kHeadDim)$ 归一。

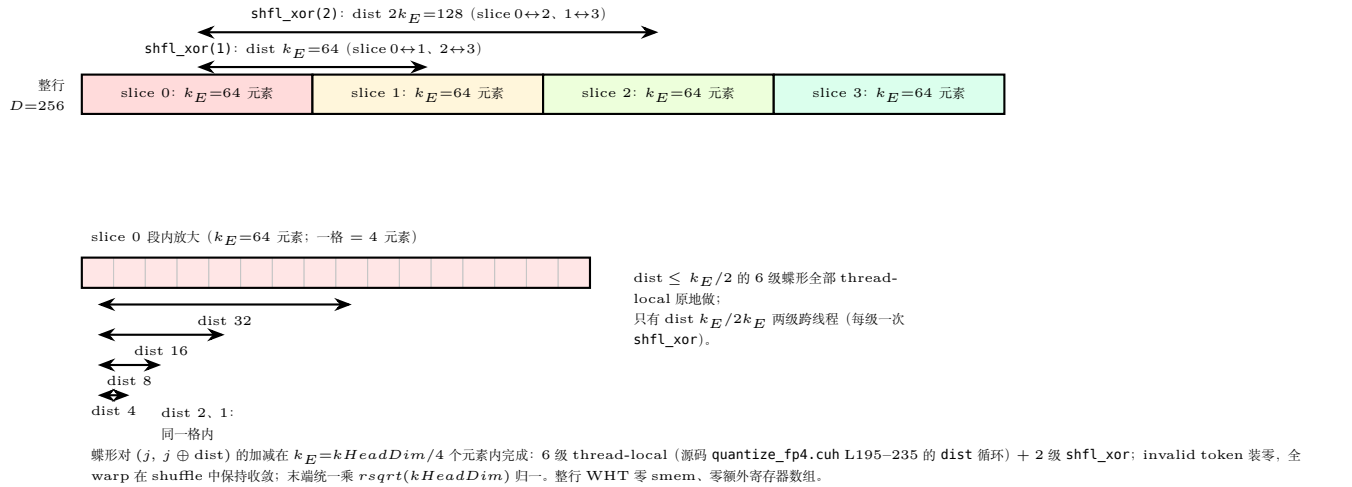


图 38.4: fused WHT 的蝶形距离分层（D=256，4 线程一 token、每线程 k_E=64 连续元素）。距离 $< k_E$ 的 6 级蝶形全在 thread-local 数组内；dist k_E 与 $2k_E$ 两级各用一次 `shfl_xor(1)`/`shfl_xor(2)` 跨 slice 交换——零 smem、零额外寄存器数组。

38.4.4 delta_s：wmma 版与 CuTe/TMA 版

ΔS 的形状是 $[B, H, M_b, N_{kv}]$ 的 GEMM（M_b 是 128 对齐后的 Q 行数，代码标识符 `Mb`），ffpa 有两个实现，按 shape dispatch（launcher `launch_fp4_delta_s_sm120`）：

wmma 版（`delta_s_wmma_kernel`，L37–161）：CTA 算一个 $(128, 128)$ 输出 tile，8 warp 各持 16 行、8 个 wmma n-tile。K 归约（$kHeadDim$）按 64 元素 chunk 分批 staging：全量 A/B 需要 $128 \times (D{+}8) \times 2 \times 2$ B，$D \geq 192$ 时超出 99KB 预算；A/B 落在 $2 \times 128 \times 72 \times 2 = 36$KB 窗口，epilogue 时同一片 smem 复用为 128×136 float 的 f32 累加器 staging（68KB，L40–46 注释）。行/列越界零填（无 OOB gmem 读），epilogue 逐元素减一次 q_{km}、tail 列混写零（L138–160）。MMA 用**普通 fp16 wmma 而非 blockscale**：该 kernel 是 memory-bound（SM 占用 ∼10%），低精度 MMA 无收益（L10–13 注释）。

CuTe/TMA 版（`delta_s_cute_kernel`，L163 起）：同数学同 tiling，但 TMA 装载进 3-stage 流水、HMMA `m16n8k16` atom，epilogue 保留 float4 union-staged 写。另有两个 TMA 细节：超出 d_{og} 的 K chunk 是全 OOB 的 TMA box，**TMA 单元内部零填、零 DRAM 流量**（等价旧版的 guarded load）；A 侧用 3D $(M_b, D, N_b N_h)$ descriptor，partial m-tile 的 tail 行按 (b,h) 精确零填（L163–176 注释）。

性能动机写在头注释 L6–8：替换 host 侧 bmm + transpose copy + epilogue cast 的 ∼1ms torch 链（N=16384）；实测大 N 1.4ms→0.3ms（bench 记录，`.tmp/book-bench/` 历史数据，31.6 节）。

38.4.5 kv_perm32 列置换与 perm-aware masking

fp4 的 PV 有一个 fp8 没有的结构性错位：blockscale MMA 的 **C 累加器**布局（QK 产出 S 的列）与 P 作为 A 操作数的布局不同（SA3 §3.3 "Permutation for K"）。fp8 用 16 SHFL + 32 PRMT 搬 P （第 三十六 章 reorg）；SA3 的解法反过来，**不搬 P，搬 K**：把 K 的列（token 维）按 32 列窗口内的双射置换后存储，C fragment 的列布局就天然匹配 A 操作数。置换表

```
[0,1,8,9,16,17,24,25,2,3,10,11,18,19,26,27,
 4,5,12,13,20,21,28,29,6,7,14,15,22,23,30,31]
```

fused 进 **K** 量化 kernel 的写侧（`quantize_fp4.cuh` L135–141 的 `load_token_id`），闭式 $\pi(j) = (j\& \sim 31) + (\mathrm{loc}/8) \cdot 2 + ((\mathrm{loc}\%8)/2) \cdot 8 + (\mathrm{loc}\%8)\%2$（`fp4_gemm.cuh` L23–26）。注意 $\mathbf{V}^\top$ **不置换**，它的存储是自然序，P 打包侧按 A 槽位 $t \to t$ 直接对齐（`fp4_pscale.cuh` L523–524）；置换补偿只发生在 K 与 P 两侧。

必须 perm-aware 的两处。置换只改存储位置不改逻辑语义，但任何「按列位置取语义」的操作都要过一遍 π^{-1}：

- **masking**：causal/kv-tail 谓词必须用 $pos = kv_tile \cdot B_c + kv_perm32(col)$ 判定。上游 SA3 对原始列 index 做 mask，causal 直接算错（N=512 时 max_abs 3.3）；ffpa 修复为 perm-aware（skill §11.11(b)）。

- **attn_bias 注入**：bias 列同样要过 $kv_perm32(j)$（`fp4_gemm.cuh` L55–60），gmem 与 smem 两个变体同样处理。

tile 级跳过（`mask_start_tile`/$T_{c,eff}$）不受影响：π 是每 32 列窗口内的双射，tile 粒度的 causal 边界公式保持原样。

38.4.6　subbyte 寻址的陷阱

e2m1 是 4 bit 类型。CuTe 里 e2m1 的 smem tensor **必须**用 `make_smem_ptr<Element>(void*)` 构造：该重载包一个 `subbyte_iterator`，tensor 切片按 bit 推进；若误写成 `reinterpret_cast<Element*>`（对 1 字节指针语义取址），偏移按 `sizeof==1B` 缩放，**实际地址差 2 倍**，stage≥1 的 TMA 访问越界直接 IMA（`sm_120/persist_d.cuh` L48–53 注释）。这是 fp4 移植时最容易踩的一类静默地址错。

38.5　图示

三张图自底向上覆盖 fp4 的三个层面。图 38.5 是数据格式层：NVFP4 的 1×16 microscaling 块如何配平 e2m1 与 ue4m3 两级尺度。图 38.6 是工程层：量化前处理链的数据流与 fused 融合点。图 38.7 是布局层：kv_perm32 的 32 列窗口双射带来的收益与代价。

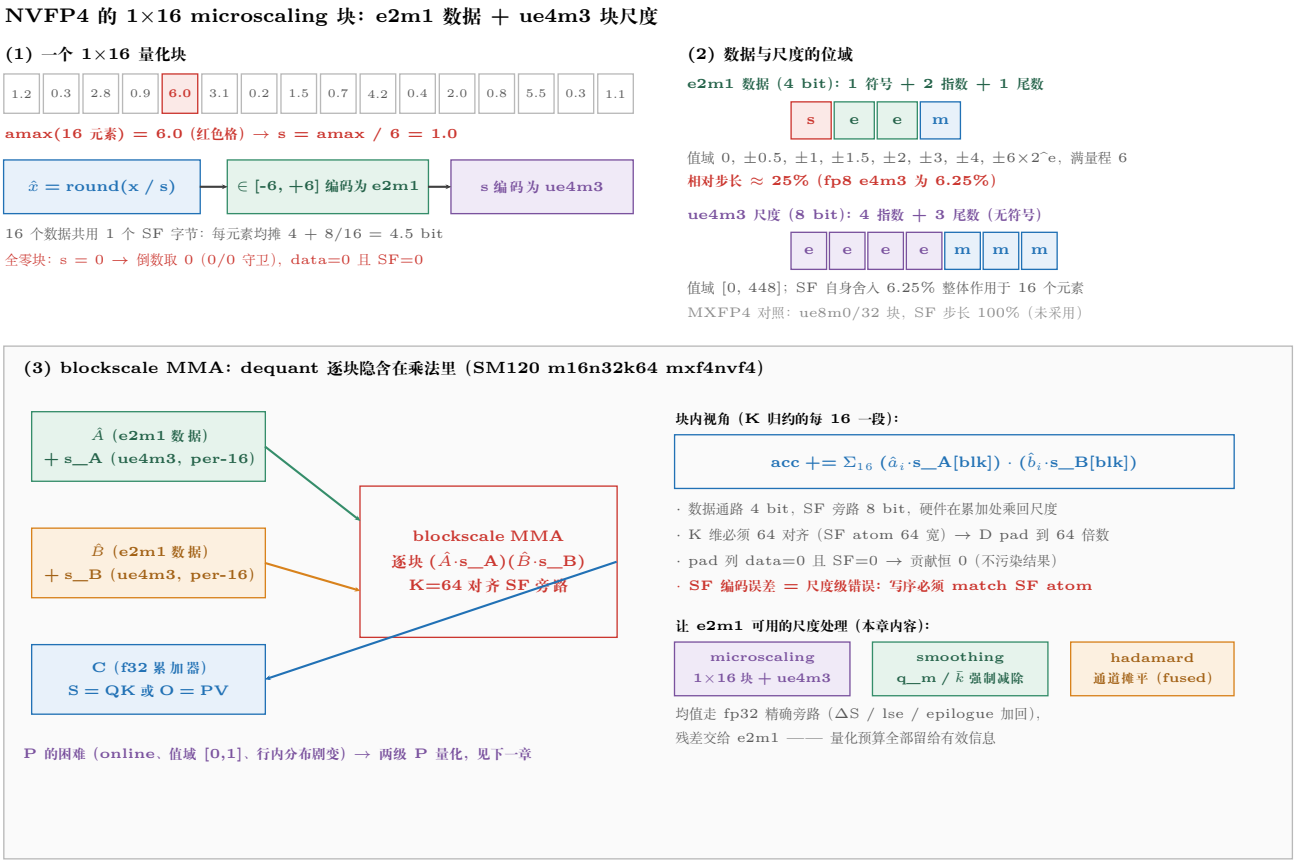


图 38.5：NVFP4 的 1×16 microscaling 块：e2m1 数据位域与 ue4m3 块尺度。s_{block} = amax/6 除进数据、乘回在 blockscale MMA 内逐块完成。

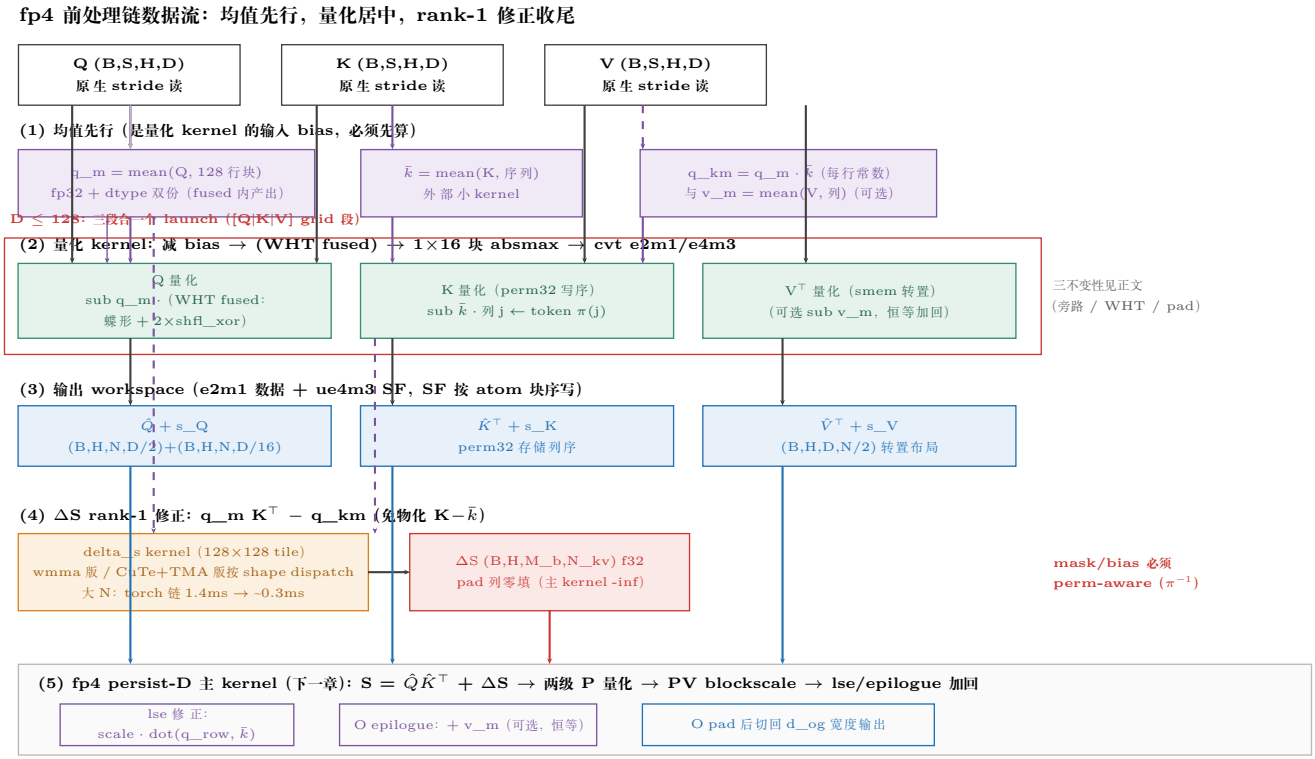


图 38.6: fp4 前处理链数据流（$D \le 128$ 走 fused 单 launch 路径）：$\bar{k}/q_m$ 均值先行的原因是它们是量化 kernel 的输入 bias；hadamard 下 q_m^{rot} 在 fused kernel 内同时产出。

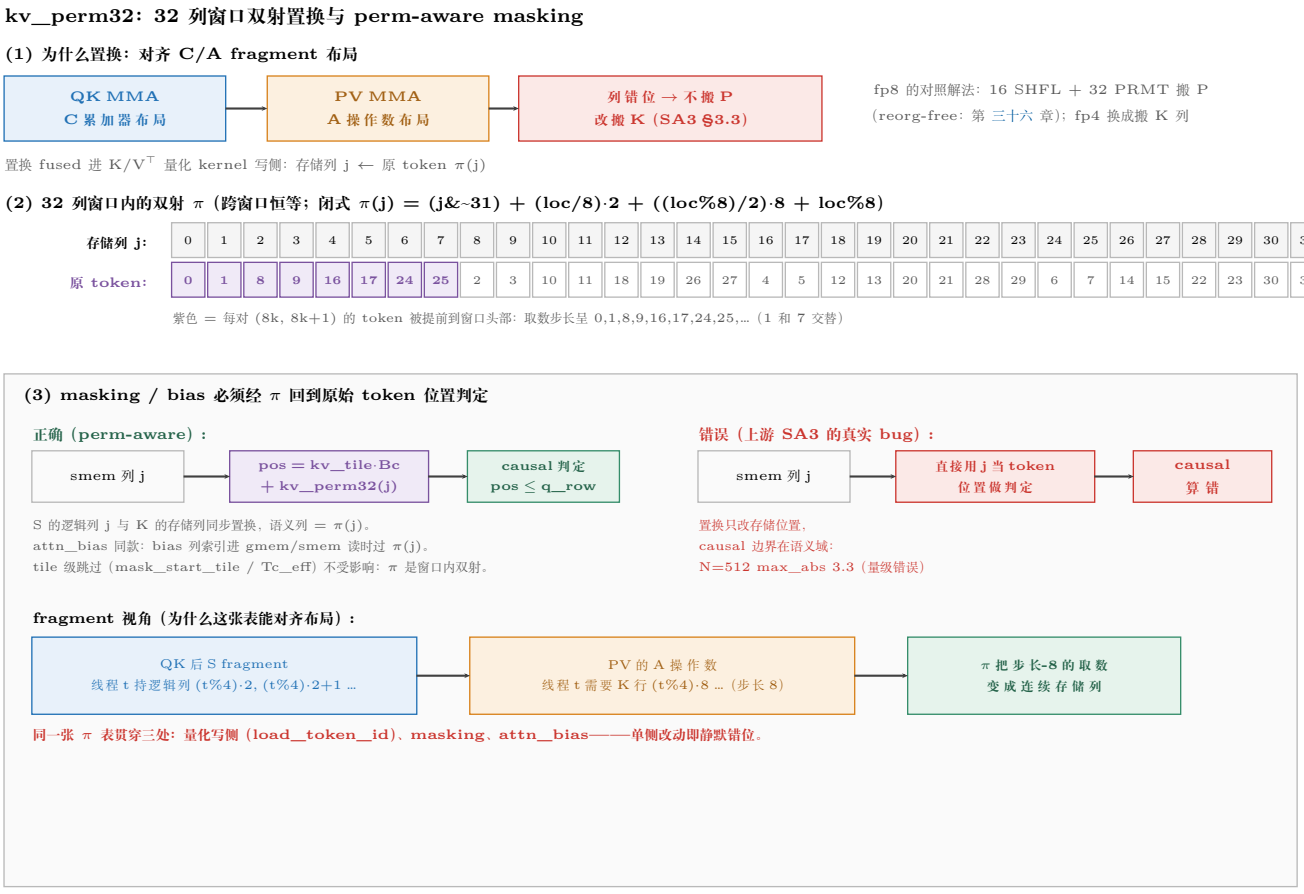


图 38.7: kv_perm32 的 32 列窗口双射与 perm-aware masking。置换让 QK 的 C fragment 列直接匹配 PV 的 A 操作数布局；mask/bias 必须经 π^{-1} 回到原始 token 位置判定。

38.6　性能分析

38.6.1　量化链本身的收益

ΔS 的 wmma kernel 替换 torch 链（bmm + transpose copy + epilogue cast）是主要收益项：N=16384 级别从 ∼1.4ms 降到 ∼0.3ms（`.tmp/book-bench/` 历史记录；头注释 L6–8 只记旧链 ∼1ms 量级）。fused WHT 实测零开销：D=128 下 `--fp4-hadamard` 与基线 2.52/1.67ms 完全一致（表 38.1），旋转折进 load 路径后，quantize kernel 的瓶颈（gmem 带宽）没有变。

表 38.1: fp4 量化链 knob A/B（PRO 5000，B=1, H=32, N=8192, D=128，self/causal，fp16 口径）

配置	self (ms)	causal (ms)	self vs 基线	allclose
基线（强制 smooth-Q/K）	2.52	1.67	—	全 ✓
`--fp4-hadamard`	2.52	1.67	±0%	全 ✓
`--fp4-smooth-v` 组合	2.59	1.75	+2.8%	全 ✓
`--fp4-pv-mm-type fp8`	3.47	2.17	+37.7%	全 ✓

MXFP8 PV 变体（$V^\top$ 走 e4m3 + ue8m0/32，精度更高）付出 37.7% 的 self 时间代价：V 侧数据带宽翻倍（8 bit vs 4 bit）、MMA atom 从 `m16n32k64` 换 `m16n8k128`，两者吞吐相近但带宽劣势直接体现在 e2e。它存在的意义是提供精度兜底（$D \le 192$ 才有 smem 预算，`attn_traits.cuh` L58–63），不是性能选项。

38.6.2　e2e：fp4 persist-D 的 D 扫描

表 38.2: fp4 persist-D e2e bench（PRO 5000，B=1, H=32, N=8192；tol：dense 0.15 / causal 0.70，16 格全绿）

D	self FFPA/SDPA (ms)	吞吐	causal FFPA/SDPA (ms)	吞吐	加速比区间
64	1.61 / 2.40	341T	1.08 / 1.35	255T	1.23–1.49x
128	2.52 / 5.11	437T	1.67 / 2.68	330T	1.61–2.03x
192	3.48 / 7.14	474T	2.40 / 3.81	344T	1.59–2.05x
256	4.97 / 10.46	443T	3.54 / 5.27	311T	1.49–2.11x

三个观察：(1) fp4 的 e2e 加速比区间（1.2–2.1x）整体高于同机 fp8 persist-D（第 三十六 章 29.6 节实测表的 1.26–1.70x），来源不在 MMA 吞吐（blockscale mxf4nvf4 的吞吐只与 fp8 dense 相当），而在 **带宽**：K/V/SF 的字节数减半以上，memory-bound 的 attention 直接受益（skill §6.6：attn kernel 本身快 ∼23%）。(2) D=192 的 474T 是全家族峰值，D=256 回落（kStages 从 3 降 2，L51–56）。(3) ∼1ms 量级的前处理链意味着**小 N_q（cross/decode）结构性不划算**：以 D=192（e2e 3.48ms）计占比约 30%；N_q 缩小时主 kernel 按 $N_q \cdot N_{kv}$ 缩、前处理几乎不缩，占比急速恶化。

38.7　实践经验

移植策略：fragment adapter 逐字拷贝。kv_perm32 体系、SF atom 布局、C/A fragment 映射这些内容是 SA3 在真实 hardware 上调试出来的位级布局知识，重写极易引入静默错位。ffpa 的选择是逐字拷贝 adapter 层、自研外围（骨架/pipeline/dispatch），只修正确性 bug （perm-aware masking）。这与第 三十五–三十六 章自研 fp8 链的策略不同：fp8 没有 SA3 级别的现成参照。

评估口径。fp4 的 max_abs 在 causal 尖峰行天然 0.5–0.75 （e2m1 的 25% 步长 × 尖峰行 P 近 one-hot 的误差结构，probe sim 与 kernel 同级证明是格式固有），SA3 论文自身用 cosine 评估。工程验收应该看 mean_abs（0.014–0.03 是质量指标）与 allclose 容差（dense 0.15 / causal 0.70），不要用 fp8 的 0.05 级容差去卡 fp4。

固定链的适用边界。本节 bench 都是 N_q=8192；把 fp4 路径用在 decode/短序列上，前处理链的固定成本会抵消全部收益。规划上应把 fp4 前处理与 prefill 阶段绑定，decode 回 fp8/fp16 路径。

38.8 常见坑

1. **subbyte 指针**：`reinterpret_cast<Element*>` 对 4 bit 类型按 1B 缩放偏移，地址差 2 倍，TMA IMA 在 stage≥1 才出现——必须 `make_smem_ptr`（31.4.6 节）。
2. **0/0 NaN**：全零块 $s = 0$，除法前必须特判倒数取 0 （L293/L460）；全 masked 组、pad 列都会触发。
3. **masking 不 perm-aware**：置换存储后 causal 谓词忘了过 π^{-1}，max_abs 直接 3.3 级别（上游 SA3 的真实 bug）。
4. **SF 写序不 match atom**：SF offset 公式与 `BlockScaledConfig` 不一致时数据看起来「差不多对」但精度大幅劣化——SF 错位是尺度级错误，不是噪声级。
5. **e2m1 的饱和语义**：`cvt.rn.satfinite.e2m1x2.f32` 的 `.satfinite` 是 PTX 对 4 bit 目的端的强制修饰符（不存在非 satfinite 的裸变体），越界值饱和到 ±6、NaN 转 +6，均不产 NaN；真正的风险在绕开 `cvt` 的手工编码（无饱和语义）与上游 0/0（见坑 2），不在 `cvt` 本身（L41/L64）。
6. **mxfp8 的 stage 数的历史**：早期 build 因一次 3-stage D=128 NaN 把全部 D 固定在 2 stage；后来定位不复现（16 shape 组合 + 5 次新数据复测全无 NaN），D=128 恢复 3 stage 换回 1.2% kernel 时间（`attn_traits.cuh` L56–62）。教训：一次异常触发的保守回退要有复测机制，否则性能损失会永久化。

38.9 面试要点速查

1. NVFP4 vs MXFP4：同为 e2m1 数据，SF 分别是 ue4m3/16 块与 ue8m0/32 块；SF 尾数位决定尺度精度，MXFP4 的整数 SF 步长 100%。
2. blockscale MMA 语义：data+SF 双操作数，dequant 逐块隐含在累加里；SF 自身的 ue4m3 舍入（6.25%）整体作用于块。
3. 为什么 fp4 强制 smooth：e2m1 ±6 域下均值偏移消耗 amax 预算；fp8 可以选、fp4 必须。
4. ΔS 恒等式：$q_m(K - \bar{k})^\top = q_m K^\top - q_{km}$，免物化 $K - \bar{k}$；多余行常数平移 lse 不改 O。
5. smooth-V 精确性：softmax 权重和为 1 $\Rightarrow$ 减均值是恒等变换，只缩小动态范围。
6. kv_perm32：为对齐 C/A fragment 布局置换 K 列（32 列窗口双射）；mask/bias 必须 perm-aware，tile 级公式不受影响。
7. fused WHT 的蝶形拆分：thread-local 对原地做，只剩两级 `shfl_xor`；均值/旋转可交换 $\Rightarrow$ ΔS 留在未旋转域。

38.10 最小测试与动手实验

bench 环境沿用第 三十五 章约定（卡 7、`cdit` 环境，小 D 需开环境变量），输出落盘 `.tmp/book-bench/ch31/`：

```
export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp4 \
  --tasks self-attn causal --D 128 --no-bwd --show-allclose
# 变体:
#   --fp4-hadamard            # fused WHT (零开销验证)
#   --fp4-smooth-v            # V 均值减除
#   --fp4-pv-mm-type fp8      # MXFP8 PV (D<=192)
```

1. **D 扫描复现**：$D \in \{64, 128, 192, 256\}$ 复现表 38.2；解释 D=256 吞吐回落（kStages 3→2）。
2. **hadamard 零开销**：A/B `--fp4-hadamard`，确认 self/causal 时间在噪声内一致；再开非 pow2 D（如 192）观察它回退到独立 WHT kernel 后的额外成本。
3. **MXFP8 PV 边界**：$D \in \{128, 192, 256\}$ 试 `--fp4-pv-mm-type fp8`，验证 256 被 smem 预算拒绝。
4. **读恒等式**：对照 `delta_s.cuh` L3–8 与 `quantize_fp4.cuh` L437–455 的两段注释，手推式 38.5 与 38.6。

延伸阅读

- `cute/fp4/quantize_fp4.cuh` 头注释（L1–14）：SA3 移植边界与 padding 约定；
- `cute/fp4/delta_s.cuh` 头注释（L1–18）：rank-1 恒等式与 memory-bound 判断；
- `cute/fp4/fp4_gemm.cuh`（L24–30）：kv_perm32 闭式与 perm-aware bias 注入；
- `cute/fp4/attn_traits.cuh`（L51–63）：kStages 的 smem 预算推导与 mxfp8 stage 史；
- SageAttention3 论文 §3.1–3.3（NVFP4 格式、两级 P 量化、K 置换）；两级 P 量化的数学与实现见本书第 三十九 章。

第三十九章　FP4（二）：persist-D 主 Kernel 与两级 P 量化

39.1　本章导读

第 三十八 章把 Q/K/V 送进了 e2m1 + ue4m3 的 workspace；本章讲消费它们的**主 kernel**：`persist_d_ws_fwd_cute_fp4_sm120`（`sm_120/persist_d.cuh`，1113 行），跑在与 fp8 persist-D 相同的 128 线程 producer + 256 线程 consumer 的 warp-specialized 骨架上（第 三十六 章），数据路径移植自 SageAttention3 的 NVFP4 kernel。

本章的核心内容是**两级 P 量化**：SA3 用来回答「P 无法离线校准」这一挑战的方案，fp4 attention 的精度靠它成立。它与 online softmax **融合**在一起（`fp4_pscale.cuh` 的 `SoftmaxFused`），不走单独一遍：softmax 输出的每个数直接落进第二级量化的域，s_{P1} 的常数折进 `exp2` 的 shift，除法和归一化在到达寄存器之前就已经消失。

39.2　问题与动机

P 的三个困难。Q/K/V 的量化无论多粗，都可以离线统计（smooth、per-block scale）压动态范围；P 不行：

1. **online**：P 是 softmax 现场产物，kernel 内逐 tile 生成，没有第二遍数据；
2. **值域窄**：$P \in [0,1]$，且行内分布剧变——causal 早期行近 one-hot（一个 1、其余 ≈ 0），dense 行近均匀且数值极小；
3. **SF 格式自身的损耗**（SA3 的 C2 挑战）：若按块直接量化，$s_P = \text{amax}(P_{\text{块}})/6 \in [0, 0.167]$，ue4m3 在这个窄域的有效位数被浪费——**尺度因子自己**先损失精度，再乘回 16 个元素。

两级量化的思路：先把整行线性拉伸到 NVFP4 的满量程（第一级，全局常数），块内再用 e2m1 域归一（第二级，per-16 组 ue4m3），让块的 amax 恰好铺满 ue4m3 的域。

硬件侧的结构约束。blockscale MMA（`m16n32k64 kind::mxf4nvf4`，`attn_traits.cuh` L77）要求 K 维 64 对齐、SF 以 `scale_vec::4X` 旁路输入；P 作为 A 操作数必须以 data+SF 的 zip 形式出现（`fp4_pscale.cuh` L314–346 的 pack 注释）。这套约束决定了 P 量化必须发生在寄存器里、紧贴 MMA。

39.3　数学原理

39.3.1　两级 P 量化：SA3 §3.2 的方案

对 softmax 输出 $\tilde{P}$（已减行 max、未归一），两级量化定义为：

$$s_{P1}[row] = \frac{\text{rowmax}(\tilde{P})}{448 \times 6}, \qquad \tilde{P}_2 = \frac{\tilde{P}}{s_{P1}} \in [0,\, 2688], \qquad \hat{P}_2, s_{P2} = \phi(\tilde{P}_2) \tag{39.1}$$

$2688 = 448 \times 6$：448 是 ue4m3 满量程（SF 域上限）、6 是 e2m1 满量程（数据域上限），乘积 2688 是 SF× 数据的乘积极限域，第一级把行拉伸到这个域。dequant 端 $O = \text{FP4MM}(\hat{P}_2, s_{P2}, \hat{V}, s_V) \times s_{P1}$，常数 s_{P1} 在 O 与 row_sum 中**同现同消**，finalize 相除后精确抵消（`fp4_pscale.cuh` L327–330 注释）。

39.3.2　online softmax 下的退化：第一级变成常数

ffpa 实现利用了 online softmax 的一条性质（`fp4_pscale.cuh` 头注释 L2–5）：它对当前 tile 已经减去运行行 max，所以 $\mathrm{rowmax}(\tilde{P}) \equiv 1$，逐行的 s_{P1} **退化为全局常数** $1/(448 \times 6)$，可以直接折进 `exp2` 的 shift：

$$P_2 = \exp_2\big(S \cdot L - m \cdot L + \log_2(448 \cdot 6)\big) \in [0,\, 2688], \qquad s_{P2} = \exp_2\big(a \cdot L - m_{sc} + \log_2 \tfrac{1}{6}\big) \in (0,\, 448] \tag{39.2}$$

其中 $L = \mathrm{scale} \cdot \log_2 e$，$a$ 是 16 元素组 absmax，m_{sc} 是含第一级常数的行 max（L117–119 注释与 L126–128 的三个编译期常数：$\log_2(1/2688) = -11.3923$、$\log_2(1/6) = -2.5850$）。第一级量化的除法、$s_{P1}$ 的存储与乘回全部消失。常数符号容易看错：$m_{sc} = m \cdot L + \log_2 \frac{1}{2688}$，减去它等效加上 $\log_2 2688$，所以 $s = m$ 时 P_2 恰达 2688（源码注释 L118 把 shift 写作 $+\log_2(1/(448 \cdot 6))$，是同一类符号笔误，以代码与值域为准）。

39.3.3　归一化也折进 exp2

第二级本需要 $q = \tilde{P}_2 / s_{P2}$（数据除以组尺度才能落 e2m1 域）。实现把这一步同样折进指数（L182–186 注释）：

$$q = \exp_2\big((s - a) \cdot L + \log_2 6\big) \in (0,\, 6] \tag{39.3}$$

组 absmax 归一化在 `exp2` 参数里完成，**rcp 与逐元素 FMUL 在 softmax 与 pack 之间整段消失**；row_sum 用 FMA 吸收 s_{P2} 因子（`row_sum = fmaf(q, sfp2, row_sum)`，L195）。硬件只要一条 MUFU.EX2：`ex2.approx.ftz.f32` 的 `.ftz` 让 ptxas 不必包 range glue（L21–22 注释，非 ftz 形式每次调用要多出 FSETP/ FSEL/FMUL 链，实测 -5.9%）。

同一条 shuffle 链做两个归约：C fragment 每 lane 持 8 元素，`shfl_xor`(1) 合成 16 元素组 absmax，`shfl_xor`(2) 完成 quad 内行 max（L160–167），组尺度和行 max 的冗余归约减半。

全 masked 组的退化：$a = -\infty$ 时特判 $a \to 0$（L187 的 `a_arg` clamp），q 与 row_sum 贡献双双归 0 而不是 NaN；`InfCheck` 模板参数处理整行有效列全部落在当前 tile 之外的 causal 左上角（L169–175、L219–221 的 `scores_max_cur` 守卫）。

39.3.4　lazy rescale 与 lse 复合公式

跨 tile 的行 max 更新沿用 fp8 的 lazy rescale（第 三十六 章）：O 不即时重缩放，记 `scores_scale`，下一个 tile 的 PV 累加前按行补乘（`rescale_acc`，L299；PC-11 起为 per-row 守卫：`scores_scale(mi)<1.0f` 的行跳过 FMUL，`persist_d.cuh` L968–975 注释；dense 路径 96.5% 命中零开销为 bench 记录口径）。

lse 的复合公式（每项都有出处）：

$$\mathrm{lse} = \big(m \cdot L + \log_2(\mathrm{row_sum}) + \log_2 \tfrac{1}{2688}\big) \ln 2 + \mathrm{scale} \cdot q_{km} \tag{39.4}$$

第一项是 log2 域行 max 与 row_sum，第三项是两级量化的 1/2688 因子（行和处于 $P \cdot 2688$ 域，要除回来），最后一项是 smooth-K 的修正 $\mathrm{dot}(q_{row}, \bar{k})$（第 三十八 章式 (38.4) 的对应项；`lse_qkm_dot` 在 Q 寄存器拷贝后、kv 循环之前算好，L583–591 与 `persist_d.cuh` L757–758：Q 的 smem 在 `kQSmemReuse` 下会被 V stage 复用，点积必须提前）。

39.3.5　MXFP8 PV 变体的数值约定

`--fp4-pv-mm-type fp8` 把 $V^\top$ 换成 e4m3 + ue8m0（32 元素组），P 侧同样换 32 粒度两级量化：域常量从 2688 换成 448（e4m3 满量程，无第二级 e2m1 因子），SF 是 2 的幂（`pow2_to_ue8m0`，L417；约定注释 L390–397）。SFA 的槽位排布遵循 `SM120_16x8x128_TN_VS_MXFP8` atom 的 A 布局（L501–535 的 probe 结论）。历史上 MXFP8 PV 的 lse 曾无条件沿用 2688 域，差 $\ln 6$ 的 latent bug，现按 `kPvMxfp8` 编译期分流。

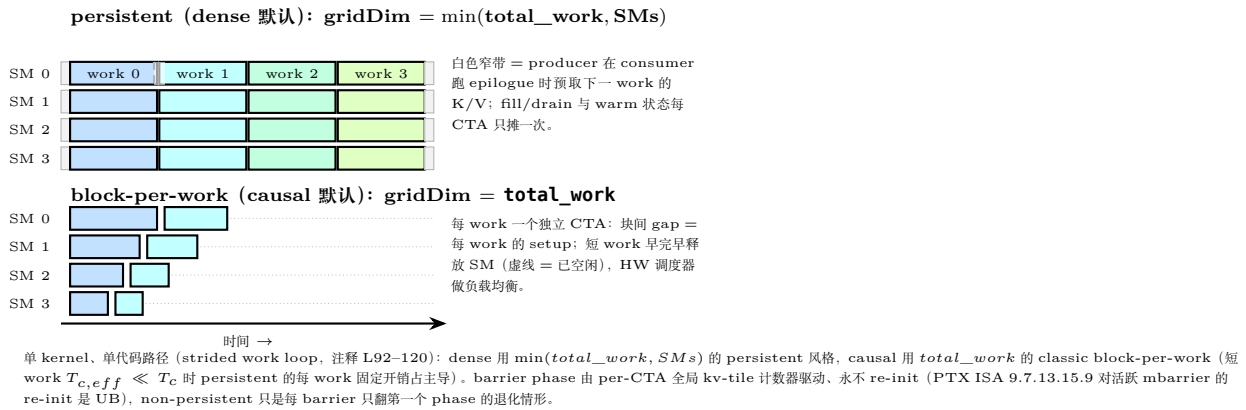
39.4　设计与实现

39.4.1　骨架复用与 grid 调度约定

WS 骨架与 fp8 persist-D 同构：128T producer（TMA 装载 K/V/SF/bias）+ 256T consumer（8 warp MMA），4 组反向 barrier（`k_empty`/`v_empty`/`q_consumed`/ `epilogue_done`，L200–213 注释）。fp4 特有的协议约束：所有

mbarrier **永不 re-initialize**：per-CTA 全局 kv-tile 计数器驱动每个 barrier 的 stage/phase 跨 work 连续翻转（对活跃 mbarrier 的 re-init 是 UB，PTX ISA 9.7.13.15.9；L113–115、L316 注释）。

grid 调度约定是「**单 kernel、单代码路径、runtime grid 定风格**」（L92–120 注释）：kernel 体是 `for (work_id = blockIdx.x; work_id < total_work; work_id += gridDim.x)` 的 strided work loop。dense 用 $\min(total_work, SMs)$（persistent，流水 fill/drain 每 CTA 摊一次，producer 可在 consumer 跑 epilogue 时预取下一个 work 的 K/V）；causal 用 `total_work`（经典 block-per-work，短 work 早完早释放 SM，HW 调度器负载均衡：多数 work 的 $T_{c,eff} \ll T_c$，persistent 的每 work 固定开销会占主导）。barrier 协议对任意 gridDim 都成立，non-persistent 只是每 barrier 只翻第一个 phase 的退化情形。



单 kernel、单代码路径（strided work loop，注释 L92–120）：dense 用 $\min(total_work, SMs)$ 的 persistent 风格，causal 用 $total_work$ 的 classic block-per-work（短 work $T_{c,eff} \ll T_c$ 时 persistent 的每 work 固定开销占主导）。barrier phase 由 per-CTA 全局 kv-tile 计数器驱动、永不 re-init（PTX ISA 9.7.13.15.9 对活跃 mbarrier 的 re-init 是 UB），non-persistent 只是每 barrier 只翻第一个 phase 的退化情形。

图 39.1: dense persistent 与 causal block-per-work 的 grid 调度对比（同一 kernel、同一代码路径，runtime grid 定风格）。上：$\min(total_work, SMs)$ 个 CTA 按 strided work loop 认领，fill/drain 每 CTA 摊一次，producer 在 epilogue 期间预取下一 work 的 K/V；下：每 work 一个 CTA，短 work 早完早释放 SM。

39.4.2 Q 的寄存器驻留与一个典型 bug

Q/SFQ 是 per-work 常量：s2r 一次进 `tSrQ`/`tSrSFQ` 寄存器，跨整个 kv 循环驻留，`gemm_ss_fp4` 把它们当 A 侧寄存器操作数消费、不再每 tile 读 Q smem（L747–753 注释，fp8 的 `kPersistQs2r` 等价物，这里恒开）。这段注释同时记录了一个有教学价值的 bug：**没有**寄存器驻留时，A/SFA 的 asm 操作数未初始化，cicc 直接把它们折叠成 0，QK 退化成 ΔS（rank-1 均值 attention），probe 容差曾把它掩盖。教训：量化 kernel 的「输出恒为常数」类退化要靠数据探针（输出对 K 的敏感性）而非容差判断。

39.4.3 主循环五步

consumer 的 kv 循环（L803–971）每 tile 依次：

1. `add_delta_s`：把 ΔS 的 rank-1 项预载进 C 累加器（只写 `tSrS` 寄存器，与 B 侧操作数装载无重叠，L819–822）；
2. `gemm_ss_fp4`：$S \mathrel{+}= (\hat{Q}\, s_Q)(\hat{K}\, s_K)$。A 侧驻留寄存器、B 侧（K/SFK）从当前 stage 流入，操作数是 data+SF 的 `make_zip_tensor` 对（`fp4_gemm.cuh` L131–132）；B 的下一个 k_block 拷贝与当前 mma 交叠发出，链尾的 `k_empty` arrive 让 producer 恰在 mma 消费完时看到 stage 释放（L140）；
3. masking：kv-tail + causal 谓词**必须**用 $pos = kv_tile \cdot B_c + kv_perm32(j)$（L903–916，谓词在 L911）；bias 注入在 dequant 域（除 `scale_orig`），列同样过 π，tile 化路径（PC-0-1）持有原始 token 序、注入时索引置换（L838–896）；$-\infty$ 赋值同时覆盖 masked 槽位上无效的 ΔS 值；
4. `online_softmax_with_quant`：32.3 节的融合体，输出已在 P_2 域的分数 + `AbsMaxP` 组尺度；
5. `gemm_rs_fp4`：$O \mathrel{+}= (\hat{P}_2\, s_{P2})(\hat{V}\, s_V)$。A（P）不过 smem——`quantize_and_pack_p` 刚在寄存器里打包好；V 的 copy、量化、mma 按此顺序发出让 S2R 与 mma 链重叠，链尾 `v_empty` arrive 兼作「无更多预取」分支（`fp4_gemm.cuh` L171–180 注释）。

V 的 `v_full` 等待放在 softmax **之后**（L937–940）：QK 数学与 V 的 TMA 传输重叠。slot 轮转按 work 内 tile 索引、barrier 按全局序列（`v_slot`/`v_mem` 解耦，L811–814），`kQSmemReuse` 下 V 的 slot 0 别名到 Q 区，轮转避开别名区（`attn_traits.cuh` L62–65）。

39.4.4　epilogue：finalize、加回与批量 staging

循环后 `finalize` 除以 row_sum（0 行守卫）；smooth-V 的 v_m 逐列加回（恒等推导见 `persist_d.cuh` L981–1016 的完整注释：softmax 权重和为 1，v_m 对所有列相同，从列和里提出后与归一化相消；masked 列 $P = 0$ 不参与两个和，causal 下恒等式同样成立）。然后是 O 的批量 staging：consumer named barrier（WAR fence，确认所有 PV 读与 lse dot 完成）→ `SM90_U32x2_STSM_N` 把 `tCrOHalf` 写进 SW128 smem（叠在被释放的 Q/K 区上）→ 单条 TMA store（L1018–1066）。tail Q tile 走 R→G 带行守卫。注意 STSM 用 `U32x2` 而非 fp8 的 `U32x4`：blockscale PV 的 C fragment 每 lane 持数不同（头注释 L44–46）。

39.5　图示

两张图分别对应本章的数学与实现。图 39.2 是两级 P 量化的域拉伸——两级 scale 折进 exp2 后，在数据通路里不占显式乘法。图 39.3 是主循环数据流：producer 与 consumer 围绕每个 kv tile 的五步协作，展开与 barrier 约定见图注。

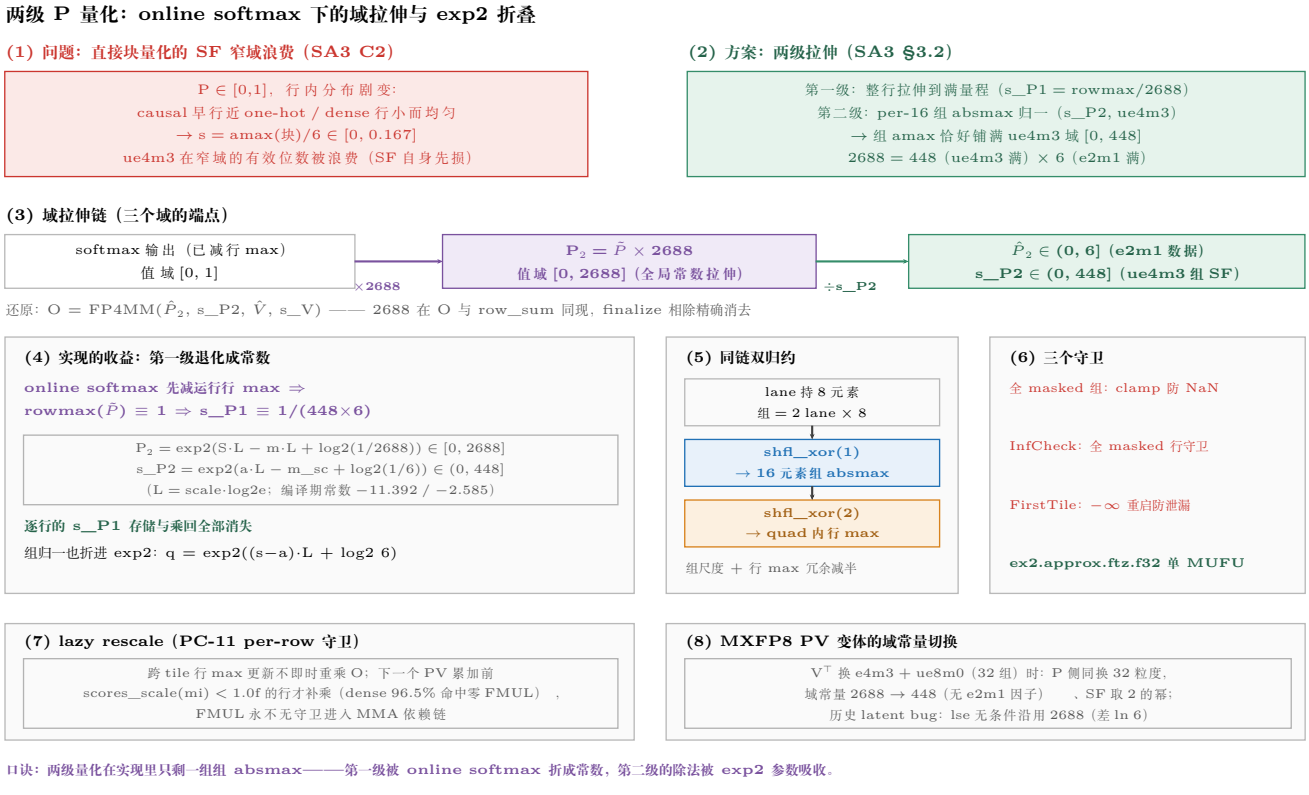


图 39.2：两级 P 量化的域拉伸：online softmax 使第一级退化为全局常数 1/2688 并折进 exp2 shift；第二级 per-16 组 absmax 铺满 ue4m3 域，组归一化同样折进 exp2 参数。

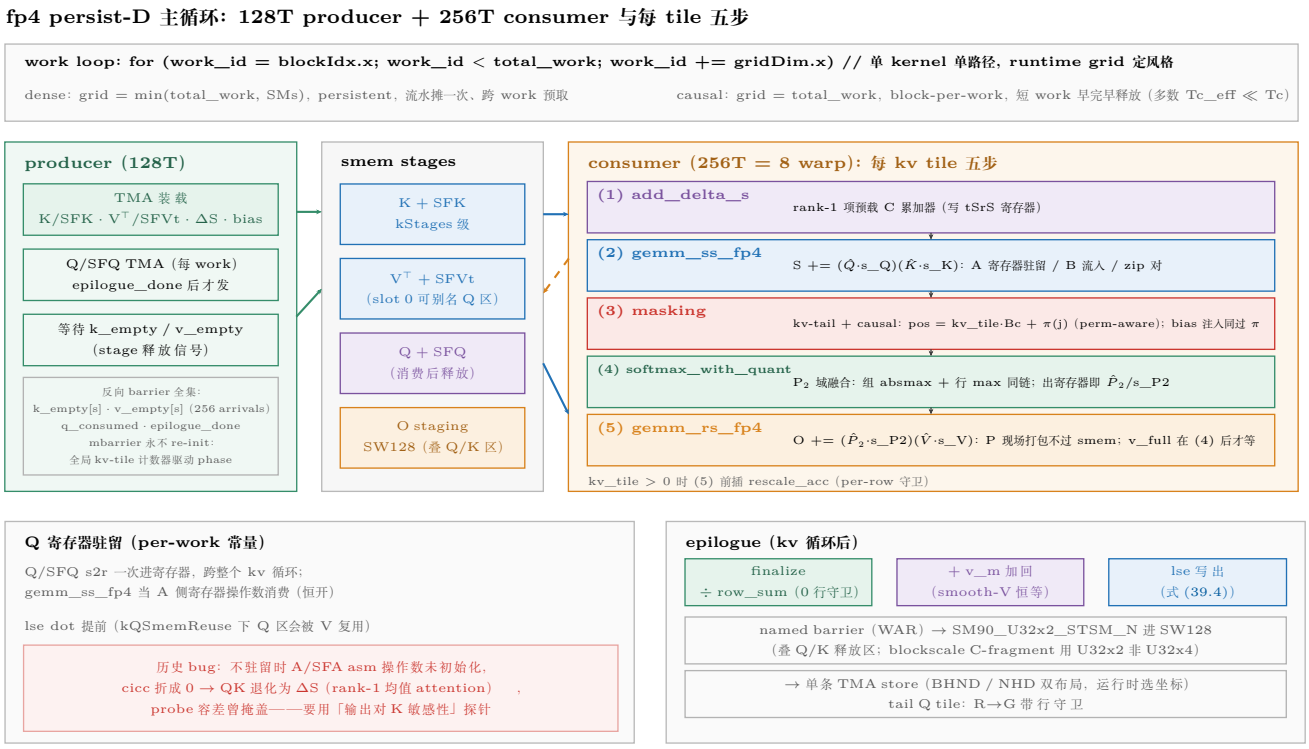


图 39.3: fp4 persist-D 主循环数据流（128T producer + 256T consumer）：每 kv tile 的五步（ΔS 预载 → QK → masking → 融合 softmax/P 量化 → PV），barrier 永不 re-init，grid 由 runtime 定 persistent 或 block-per-work。

39.6 性能分析

39.6.1 本机 e2e（fp4 vs SDPA）

表 39.1 汇总 `--fp4-pv-mm-type fp8` 变体（数据落盘 `.tmp/book-bench/ch32/`）；fp4 基线表见第 三十八 章表 31.2（self 1.49–2.11x、causal 1.23–1.61x，16 格全绿）。

表 39.1: MXFP8 PV 变体 vs fp4 基线（PRO 5000，B=1, H=32, N=8192，fp16 口径，allclose 全绿）

D	mxfp8 self (ms)	vs fp4	mxfp8 causal (ms)	vs fp4	加速比 (self/causal)
64	2.24	+39%	1.41	+31%	1.07x / 0.96x
128	3.47	+38%	2.17	+30%	1.48x / 1.24x
192	4.62	+33%	3.06	+28%	1.56x / 1.25x

MXFP8 PV 付出 28–39% 的代价：$V^\top$ 数据 8 bit 带宽翻倍、atom 换 `m16n8k128`，两者吞吐相近但带宽直接体现在 e2e。它的定位是 **精度兜底**（$D \le 192$ 的 smem 预算内），不是性能选项（第 三十八 章 31.6 同结论）。

39.6.2 fp4 对 fp8 的相对收益

历史对照（RFC/skill 记录口径，非本机当日）：attn kernel 本身 fp4 比 fp8 快 ~23%：blockscale mxf4nvf4 吞吐与 fp8 dense 相当，收益全部来自**带宽**（K/V/SF 字节减半以上）；D=192 的 e2e 比值 self 1.47x / causal 1.29x / gqa 1.50x。本机同日对照可复算：fp4 D=192 self 3.48ms（表 31.2）对 fp8 persist-D 同 shape 的第 三十六 章 29.6 节实测表数据。

39.6.3 已落地与已证伪的微优化

已落地：条件 rescale（PC-11 per-row 守卫，dense 96.5% 命中，-4.9%；attn-mask 场景再 -3.7/-4.7%）、`ex2.approx.ftz.f32`（消 ptxas range 胶水，-5.9%）、regalloc 按 D 门控（$D \le 128$ 用 `alloc<224>`，$D \ge 192$ 必

须 232）。

已证伪（默认关闭/代码保留，勿重复）：Q smem 复用（fp4 的 L1TEX sector hit 已 96.75% 饱和，fp8 的前提不成立，实测 +1.8% 回退，`attn_traits.cuh` L19–28 注释，与 fp8 章的同名机制结论相反，很好的对照案例）；rescale merge in-place（把 FMUL 拉进 MMA 依赖链，+1.7%）；V/SFV 首块 s2r 提前藏进 softmax（PC-4：+1.3–1.7% 净负，LDSM 目的寄存器跨 softmax 链拉长 live-range；NCU 证实 B 供给非瓶颈，wait 主体在 A 操作数串行链，也是 persist-D 微优化到顶的标志）。

39.7　实践经验

fragment adapter 是经验锁定的整体。kv_perm32、ΔS 槽位算术、`LayoutP`/`LayoutSFP` 打包、$V^\top$ 转置存储：SA3 的这套 adapter 相互补偿、自洽成体系（头注释 L25–37：empirical lock 在 `.tmp` 的对齐探针里逐项验证过，dense e2e 与 dequant 仿真一致）。逐字拷贝 + 只修 masking 是正确的移植方式；单独「改进」其中一项会破坏补偿关系。

精度评估的口径。e2m1 的相对量化误差（主区间 ∼20%、0.75 中点最坏 ∼33%）使 P/V 两级误差叠加后 causal 尖峰行 max_abs 天然 0.5–0.75（probe 仿真与 kernel 同级，格式固有）；SA3 论文自身用 cosine 评估。工程验收看 mean_abs（0.014–0.03）与容差（dense 0.15 / causal 0.70），用 fp8 的容差卡 fp4 会全部不通过。hybrid（fp16 前 n_{early} 行）是尖峰行的工程解法（b0/b1 误差 2.6/0.99 → 0.000/0.001，剩余长尾由 V 量化主导）。

39.8　常见坑

1. **masking 不 perm-aware**：causal 谓词按原始列 index 判定，N=512 max_abs 3.3（上游 SA3 真实 bug；本章 32.4.3 的 *pos* 公式）。
2. **mxfp8 域常量**：PV 换 mxfp8 后 lse 仍用 2688——差 $\ln 6$ 的 latent bug，按 `kPvMxfp8` 编译期分流域常量。
3. **mbarrier re-init 是 UB**：跨 work 循环里对活跃 barrier re-initialize 行为未定义（PTX ISA 9.7.13.15.9），用全局 tile 计数器驱动 phase。
4. **stale AbsMaxP**：组 absmax 寄存器跨 tile/跨 work 复用，每 FirstTile 必须从 $-\infty$ 重新起步（L151–154 注释），否则上个 work 的尺度泄漏进当前行。
5. **未初始化 asm 操作数被折叠**：Q 不驻留寄存器时 cicc 把 A/SFA 操作数折成 0，QK 静默退化（32.4.2 的历史 bug）。
6. **subbyte smem 指针**：e2m1 tensor 必须 `make_smem_ptr`，裸 `reinterpret_cast` 地址差 2 倍、stage≥1 TMA 越界（第 三十八 章 31.4.6）。

39.9　面试要点速查

1. 两级 P 量化：s_{P1} = rowmax/2688、$\tilde{P}_2 \in [0, 2688]$、$s_{P2}$ per-16 组 ue4m3；2688 = 448（ue4m3 满）× 6（e2m1 满）。
2. online softmax 的退化收益：行 max 恒 1 ⇒ 第一级变全局常数折进 exp2 shift——两级量化在实现里只剩一组 absmax。
3. 组归一化折进 exp2 参数：$q = \exp_2((s-a)L + \log_2 6)$，rcp 与 FMUL 段消失；row_sum 用 FMA 吸收 s_{P2}。
4. 2688 的消去：O 与 row_sum 同带常数，finalize 相除精确抵消。
5. dense persistent / causal block-per-work：单 kernel 单路径，runtime grid 定风格；barrier phase 由全局 tile 计数器驱动。
6. blockscale QK/PV 的 A 侧差异：Q 寄存器驻留（per-work 常量）、P 寄存器现场打包（quantize_and_pack_p），B 侧才走 smem stage。
7. fp4 vs fp8 的收益来源：MMA 吞吐持平、带宽减半——memory-bound attention 的收益逻辑。

39.10 最小测试与动手实验

```
export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp4 \
  --tasks self-attn causal --D 192 --no-bwd --show-allclose
# MXFP8 PV（精度保险丝，D<=192）：
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp4 \
  --tasks self-attn causal --D 192 --no-bwd --show-allclose \
  --fp4-pv-mm-type fp8
```

1. **两级域验证**：读 `fp4_pscale.cuh` L117–128 的三个编译期常数，手推式 39.2 的两个值域端点（$P_2 = 2688$ 当 $s = m$；$s_{P2} = 448$ 当组 absmax = 行 max）。
2. **mxfp8 A/B**：复现表 39.1 的三档 D，验证 D=256 被 traits 的 smem 断言拒绝。
3. **causal 风格对照**：读 `launch/cute_fp4_persist_d.cuh` 的 grid 选择（`causal ? total_work : min(...)`），解释为什么 causal 的多数 work 选 block-per-work。
4. **InfCheck 场景**：构造 $N_q > N_{kv}$ 的 cross 输入跑 causal，确认左上角全 masked 行的 lse/O 有限（无 NaN）。

延伸阅读

- `cute/fp4/sm_120/persist_d.cuh` 头注释（L1–55）：数学链、列对齐锁定、两级 P、subbyte；
- `cute/fp4/fp4_pscale.cuh` L117–195：融合 softmax 的域折叠与 shuffle 链；
- `cute/fp4/fp4_gemm.cuh` L91–218：`gemm_ss_fp4` 与 `gemm_rs_fp4` 的流水与 barrier 协议；
- SageAttention3 论文 §3.2（两级量化）与 Algorithm 1（完整数值链）；sm_120 split-D 与 M4N2 家族见第 三十七 章的 fp8 同构实现。

第四十章　FP8/FP4 性能实战：Bench、精度与生态

40.1　本章导读

第 三十六–三十九 章讲完了 fp8/fp4 attention 的全部实现；本章讲**怎么度量与选用**它们。这是量化 attention 的「最后一步」：一个不公平的 bench 能把 2x 加速测成 0.5x（功耗墙、参照漂移、preset 隐藏成本），一个错误的容差能把好 kernel 判成精度不合格（用 fp8 的容差去卡 fp4）。本章沿三条线索展开：

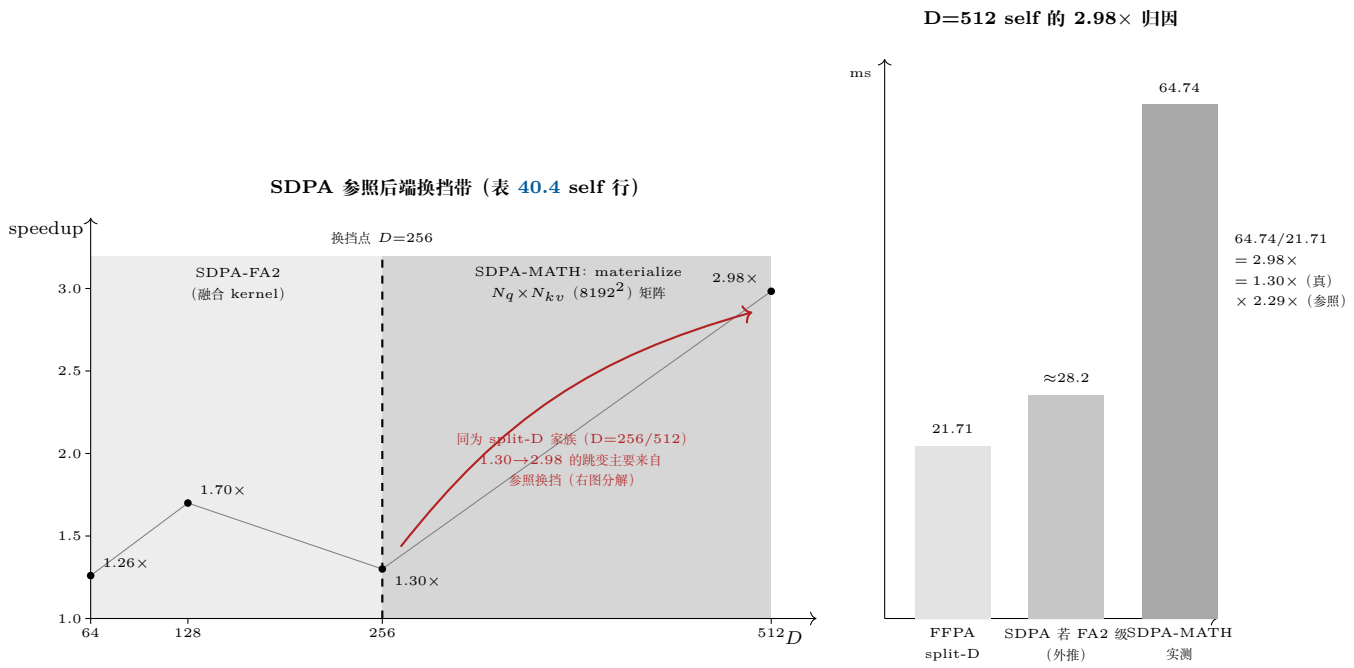
1. **口径**：speedup 怎么算才公平（参照后端、min-of-N、pre-heat、no_grad）；精度怎么报才有信息量（布尔 allclose 之外的 rel_err/max_abs 体系）；
2. **矩阵**：bench CLI 的全 knob 空间与实测——fp8/fp4 在 self/causal/gqa/cross × D∈ [64, 512] 上的完整矩阵，knob（QK int8/fp8、PV f16/f32、量化粒度、hybrid）的边际效应；
3. **生态**：Cache-DiT × FFPA 的端到端集成——量化 attention 在真实扩散模型管线里怎么落地。

素材锚定 ffpa-attn `861d75e` 的 `bench/bench_fp8.py`、`bench/bench_fp4.py` 与 `ffpa_attn.bench` CLI；本机实测平台 RTX PRO 5000（sm_120），5090 数据引自仓库 `docs/assets/perf/` 归档实测图。

40.2　问题与动机

度量陷阱一：参照系。speedup 是比值，分子分母任何一个漂移，结论就作废。三个具体来源：

- **SDPA 后端随 D 换挡**：FA2 只覆盖 $D \le 256$，更大 D 静默落到 MATH（materialize 整个 $N_q \times N_{kv}$ 矩阵），参照时间膨胀数倍——D=512 的「3x 加速」里有一大半是 MATH 慢，不是量化快（`bench_fp8.py` L153–160 的 `resolve_sdpa_backend` 把这个换挡显式化）；
- **GPU 状态污染**：5090 类高功耗卡上，前一个高功耗负载会把后一个测试压进降频区（实测 FFPA kernel 撞功耗墙 2932→2445MHz，-17%）；A/B 两轮的 SDPA 参照必须吻合（±1%），否则整轮作废；
- **preset 隐藏成本**：对比双方的功能集不对齐时，「慢」可能只是「多做了事」——hybrid（fp16 前缀行）与 smooth-V 都是 FFPA 独有的精度功能，开着测速度对 Sage 不公平，对用户也是误导。



「3×」里一大半是 MATH 参照自身的低效，不是量化快（坑 6）——报告 $D>256$ 的 speedup 必须注明参照换挡。外推口径：D=256 同为 split-D 家族且参照是 FA2，实测 1.30× 用作「真加速」估计；$21.71 \times 1.30 \approx 28.2$ms，$64.74/28.2 \approx 2.29\times$。

图 40.1: 参照系视角看表 40.4 的 self 行。左：SDPA 参照随 D 换挡——FA2 只覆盖 $D \le 256$，更大 D 静默落到 MATH（materialize $N_q \times N_{kv}$ 矩阵）；四个 speedup 点（1.26/1.70/1.30/2.98×）落在换挡带上，D=512 的跳变与 kernel 无关。右：D=512 的 2.98× 归因——以 D=256 同家族（split-D、FA2 参照）实测 1.30× 为真加速估计，MATH 参照自身再慢 ×2.29：「3×」的大头是参照低效（坑 6）。

度量陷阱二：精度口径。allclose 是布尔值，信息量极低：

- fp4 的 e2m1 网格噪声决定了它的容差天然比 fp8 宽一个量级（dense 0.15 / causal 0.70 vs fp8 的 0.05），拿错档的容差会把合格的实现误判为不合格；
- 只看均值不看 max 会漏掉 causal 早期行这类**结构性**误差（第 三十四 章 27.3.7 节的 e2m1 网格噪声）；
- 推理 bench 必须 `no_grad`：grad-on 会强制走完整 autograd 链（每调用多 ~30μs Python 开销，`bench_ms` 注释口径），测出的是训练链路而非 kernel。

注：本节与坑 3 的功耗墙频率（2932→2445MHz）及顺序差（466–491μs）为本机补充实测（2026-08，单卡 PRO 5000），原始 stdout 未随书归档。

40.3 数学原理

40.3.1 speedup 与有效 TFLOPS

kernel 级 speedup 定义为同形状下参照与被测的耗时比：

$$S = \frac{T_{\text{SDPA}}}{T_{\text{FFPA}}}, \qquad T = \min_{i \in [1,N]} t_i \ \ (\text{min-of-}N,\ N \ge 5) \tag{40.1}$$

min-of-N 而非均值：推理延迟的合同口径是「最好情况可稳定达到的水平」，毛刺（时钟迁移、后台进程）只应抬高均值，不应进入分子。有效算力用 attention 前向 FLOPS 归一：

$$\text{FLOPs} = 4\, N_q N_{kv} D H_q \cdot \delta_{\text{causal}}, \qquad \delta_{\text{causal}} = \begin{cases} 1 & \text{dense} \\ \frac{1}{2} + \frac{1}{2N_{kv}} & \text{causal} \end{cases} \tag{40.2}$$

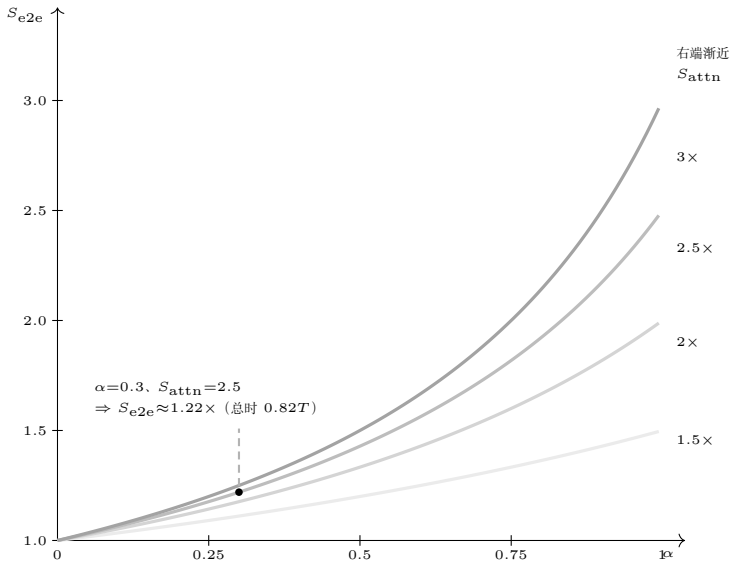
（$\text{QK}^\top$ 与 PV 各占 $2N_qN_{kv}DH_q$；causal 按三角 mask 减半：此因子在方阵 $N_q = N_{kv}$ 下与 `_flops.py` 的精确 tail 对齐计数一致，非方阵 causal 的精确计数为 $1-(N_q-1)/(2N_{kv})$，本章 bench 的 causal 均为方阵。）注意这是**数学** FLOPS：量化 kernel 实际吞吐的是低位宽 MMA，$S>1$ 的来源是「同样的数学量用快得多的数据通路完成」。

40.3.2　端到端摊销

单算子加速换算成管线加速要看占比。设管线总时 T，attention 占 α，则端到端 speedup 上界为：

$$S_{\text{e2e}} = \frac{T}{T - \alpha T(1 - 1/S_{\text{attn}})} \xrightarrow{\alpha \to 1} S_{\text{attn}} \tag{40.3}$$

$\alpha = 0.3$、$S_{\text{attn}} = 2.5$x 时 $S_{\text{e2e}} \approx 1.22$x：attention 只省下 $0.3T \times 0.6 = 0.18T$，总时从 T 降到 $0.82T$。这解释了为什么扩散模型（attention 占比高、D 偏小）是量化 attention 的第一应用场景，也是 Cache-DiT 集成（33.7 节）的价值前提。



曲线族 $S_{\text{attn}} \in \{1.5, 2, 2.5, 3\}$：attention 只省下 $\alpha(1 - 1/S_{\text{attn}})$ 的比例，$\alpha \to 1$ 时 $S_{\text{e2e}} \to S_{\text{attn}}$（上界）。扩散模型（attention 占比高、D 偏小）是量化 attention 的第一应用场景：NeMo Automodel 案例（D=512、全注意力层 10/60、训练 e2e 1.4×–1.5×）是同一公式在训练侧的实例。

图 40.2: 端到端摊销曲线族（式 40.3）：单算子 S_{attn} 换算到管线收益由 attention 占比 α 决定上界。α=0.3、S_{attn}=2.5× 时只剩 1.22×；$\alpha \to 1$ 的渐近线即 S_{attn}。

40.3.3　精度指标体系

bench 输出三层指标，各有分工：

- `rel_err` $= \|O - \tilde{O}\|_2 / \|\tilde{O}\|_2$（`bench_fp8.py` L270–274）：能量级偏差，衡量「整体偏了多少」；
- `max_abs`（`max|diff|`）：最坏单点偏差，捕捉结构性误差（causal 早期行、坏行 bug）；
- `allclose(atol=tol, rtol=tol)`：CI 门禁布尔量，tol 按精度分档（`_runner_fwd.py` L413–419）：fp16/bf16-fp8 路径 5e-2，纯 fp16 2e-2；fp4 dense 0.15、causal 0.70——依据是 e2m1 网格噪声（第 三十四 章 27.3.7 节仿真：dense 绝对误差 ≈0.003 级，0.15 留出约 50 倍余量；causal 尖峰行天然 0.5–0.75 级（第 三十八 章 31.7 节实测），0.70 按无 hybrid 抑制的地板量级取上界）。

读数规则：**速度看 min、能量看 rel_err、结构看 max_abs、门禁看 allclose**，四者缺一不可。第 三十六 章 29.7.1 的正确性 bug 就是「速度正常 + 均值正常 + max_abs 结构性坏行」的组合。

40.4　设计与实现

40.4.1　两套 bench，两个层次

`ffpa_attn.bench` CLI（`_bench.py`）是**矩阵层**：固定形状族（B1 H32 Hkv8 N8192），扫 task × dtype × `--cuda-impl` 全 knob，输出 markdown 表 + speedup/TFLOPS 图，适合回归与横向选型。`bench/bench_fp8.py`（719 行）与 `bench/bench_fp4.py`（652 行）是**对照层**：多 N 扫描（默认 `--N 8192,16384`）+ 非 32 倍数对齐场景 + SageAttention 竞品同场，preset 体系（`bench_fp8.py` L57：`default/cache-dit/ perblock`）把「部署形态」编码进配置：`cache-dit` 预设 `hybrid_n_early`=256，`perblock` 预设 sm89 cp.async 路径的 per-block 量化；NHD 零拷贝（diffusers E2E 布局）由独立的 `--with-permute` 启用，与 preset 正交。

40.4.2 测量规则的代码化

公平测量的每条规则都落在代码里（`bench_fp8.py`）：

- `bench_ms` L125–140：`no_grad` 上下文 + warmup 3 + iters 5 取 min；
- L344–347：SDPA pre-heat 5 次（不计入）拉稳时钟，随后按 FFPA → Sage → SDPA 顺序测量——功耗墙下「谁排后面谁吃亏」，固定顺序 + pre-heat 把它变成可复现的偏置；
- L79–122 `fp8_backend` 加 `@lru_cache`：每次构造触发两次 `get_device_capability`（实测重建一次 ~7μs），每 iter 重建会把 Python 开销算进 kernel 时间；
- 精度参照统一为 `ref_bf16`（bf16 SDPA-FA2）：融合 kernel 内部本就 fp32 累加，bf16 指数域比 fp16 宽，fp32 SDPA 在大 N 会因 MATH materialize 全矩阵而 OOM（L256–259 注释）。

40.4.3 CLI knob 全集

`_bench.py` 的 `--cuda-impl`（L347–386）是一棵别名树：`fp8` = `cute_tma_fp8`，`fp8_smk` 名义上加 smooth-K（但 plain `fp8` 的 smooth-K 本就默认开，两者实际同配置），`fp8_smv`/`fp8_smkv` 才真正改变配置（开 smooth-V、V 换 per-channel），`_qk_i8` 后缀强制 int8 QK MMA；`fp4` = `cute_tma_fp4`。细粒度 knob（L387–490）分三组：

- fp8 数值通路：`--fp8-qk-mm-type` ∈ {fp8, int8}（e4m3 vs s8xs8→s32）、`--fp8-pv-acc-type` ∈ {f32, f16}（PV 累加精度）；
- 量化粒度：`--fp8-{q,k}-quant-method` ∈ {per-block, per-channel, per-thread}、`--fp8-v-quant-method` ∈ {per-block, per-channel}（per-channel 即 Sage 式 per-D scale）；
- hybrid：`--fp8-hybrid`/`--fp4-hybrid`——fp16 persist-D 算前 `n_early` 行、量化 kernel 算余下（零冗余拼接，`q_start_row` 偏移实现）；causal 下 **默认自动开启**以保护早期行精度，Nq≥ `n_early`（默认 256）。

fp4 侧另有 `--fp4-pv-mm-type` ∈ {fp4, fp8}（第 三十九 章的 mxfp8 变体）、`--fp4-hadamard`、`--fp4-smooth-v`。CLI 默认值即推荐值：纯 fp8 QK + f32 累加 + Q/K/V 全 per-block （`--cuda-impl fp8` 的解析分支，`_bench.py` L648–650 显式设 V=per-block、L690–694 的 q/k fallback 同为 per-block）；注意 `bench_fp8.py` 脚本 default preset 用的是 per-thread/per-channel （更细粒度、对齐 Sage 配置），两套默认不同：对比数据时必须对齐 preset。knob 的边际效应见 33.6.3。

40.5 实测数据总览

三张表分两个平台。表 40.1 是 RTX 5090（B1 H64 N16384 D128）的对照——fp8 1.9~3.0x、fp4 2.0~4.0x，用于建立「大卡上能到多少」的量级预期。表 40.2 与表 40.3 是本机 PRO 5000 的复测（H32/Hkv8，bench 脚本默认 preset，2026-09-21）：fp8 与 Sage2 总体打平、互有胜负（差距 ±4% 内），fp4 全场景领先。读表时记住本章动机节的三陷阱：参照系、preset 与 GPU 状态是三个最常见的失真源。

表 40.1: RTX 5090 对照（B1 H64 N16384 D128；FWD TFLOPS；数值取自仓库归档实测图）

场景	SDPA (BF16)	FFPA-FP8	FP8/SDPA	FFPA-FP4	FP4/SDPA
self-attn	214	611	2.86x	819	3.83x
cross-attn	165	306	1.85x	322	1.95x
gqa	214	637	2.98x	862	4.03x
causal	212	552	2.60x	669	3.16x

数据源：ffpa-attn 仓库 `docs/assets/perf/` 下 fp8/fp4 两组 5090 speedup 图（B1 H64 N16384 D128），数值取自图上标注；两图的 SDPA 参照各有 ≤1T 漂移（fp8 图读数 215/166/215/212），本表 SDPA 列取 fp4 图口径，比值随之有 ≤0.5% 差异。

表 40.2: 本机 PRO 5000 fp8 复测（D=128 fp16，H32/Hkv8，`bench_fp8.py` 默认 preset，2026-09-21，TFLOPS；FFPA/Sage 为两者 TFLOPS 之比，±4% 内互有胜负）

场景	Nq	FFPA-FP8	Sage2	SDPA-FA2	FFPA/SDPA	FFPA/Sage
self	8192	354.5	345.8	213.8	1.66x	1.025
causal	8192	286.6	283.5	203.9	1.41x	1.011
gqa	8192	385.0	379.0	213.8	1.80x	1.016
gqa-causal	8192	327.1	329.0	204.2	1.60x	0.994
cross-dense	8192	365.3	351.3	215.8	1.69x	1.040
non-aligned-dense	8192	341.1	342.3	220.1	1.55x	0.996
non-aligned-causal	8192	278.7	277.5	195.9	1.42x	1.004
self	16384	382.1	375.7	216.3	1.77x	1.017
causal	16384	336.9	329.9	216.3	1.56x	1.021
gqa	16384	397.6	392.9	216.4	1.84x	1.012
gqa-causal	16384	364.9	362.2	216.4	1.69x	1.007
cross-dense	16384	378.5	377.9	215.9	1.75x	1.002
non-aligned-dense	16384	368.9	373.3	223.1	1.65x	0.988
non-aligned-causal	16384	334.9	329.6	209.2	1.60x	1.016

表 40.3: 本机 PRO 5000 fp4 复测（D=128，`bench_fp4.py`，2026-09-21，TFLOPS；Sage3 不支持 gqa，该行无数据，其 causal 类行 rel_err 达 0.57–0.59（早期行 e2m1 噪声无 hybrid 抑制），FFPA-FP4 为 0.14–0.15）

场景	Nq	FFPA-FP4	FFPA-FP8	Sage3	SDPA-FA2	FP4/SDPA	FP4/FP8
self	16384	528.7	382.1	411.7	216.6	2.44x	1.38x
causal	16384	444.6	348.7	307.5	216.4	2.05x	1.28x
gqa	16384	556.7	397.8	—	216.2	2.58x	1.40x
gqa-causal	16384	480.7	368.0	—	216.4	2.22x	1.31x
cross-dense	16384	553.9	378.5	421.6	217.5	2.55x	1.46x
non-aligned-dense	16384	524.8	376.4	363.2	223.4	2.35x	1.39x
non-aligned-causal	16384	443.7	348.5	256.1	209.3	2.12x	1.27x
self	32768	570.1	390.8	443.7	219.4	2.60x	1.46x
causal	32768	515.4	371.2	340.9	222.7	2.31x	1.39x
gqa	32768	581.1	397.2	—	216.7	2.68x	1.46x
gqa-causal	32768	530.7	385.6	—	219.9	2.41x	1.38x
cross-dense	32768	578.0	391.3	447.5	214.4	2.70x	1.48x
non-aligned-dense	32768	564.4	387.2	413.4	224.6	2.51x	1.46x
non-aligned-causal	32768	505.5	368.0	306.1	215.6	2.34x	1.37x

40.6 性能分析

40.6.1 fp8 矩阵：D 的三段形态

表 40.4 是 CLI 矩阵实测（PRO 5000，fp16 行；每格左为 FFPA/SDPA ms，右为 speedup 与 allclose）：

表 40.4: fp8 CLI 矩阵（`--cuda-impl fp8`，B1 H32 Hkv8 N8192，fp16；causal 默认自动 hybrid）

	D=64		D=128		D=256		D=512†	
task	FFPA/SDPA	speedup	FFPA/SDPA	speedup	FFPA/SDPA	speedup	FFPA/SDPA	speedup
self	1.91/2.40	1.26x ✓	3.01/5.12	1.70x ✓	8.12/10.52	1.30x ✓	21.71/64.74	2.98x ✓
causal	1.17/1.35	1.16x ✓	1.85/2.68	1.45x ✓	4.73/5.29	1.12x ✓	12.82/32.92	2.57x ∗
gqa	1.79/2.40	1.35x ✓	2.81/5.15	1.83x ✓	7.34/10.62	1.45x ✓	20.46/64.73	3.16x ✓
cross	0.39/0.40	1.01x ✓	0.68/0.82	1.21x ✓	1.83/1.39	0.76x ✓	4.59/8.73	1.90x ✓

† D=512 参照为 SDPA-MATH（FA2 只覆盖 $D \leq 256$）；∗ fp16 因已知 ESS causal 特性临界失败，bf16 2.56x 通过（第 三十七 章 30.7.1 节定性）。self/causal 行与第 三十六–三十七 章同一轮 bench（第 三十六 章 29.6 节实测表）；gqa/cross 行为本机补充实测（2026-08，单卡 PRO 5000），原始 stdout 未随书归档。

三个形态段：

- **D≤128 优势区**：1.26x–1.83x。量化 kernel 占满 tensor pipe，aux 链被主 kernel 遮蔽；
- **D=256 交叉点区**：fp8 persist-D 掉到 1.14x–1.45x（smem 压力 + fp4 家族在此开始占优——同点 fp4 2.11x，见表 40.5；第 三十七 章的 dispatch 三分在此切换）；cross 甚至 0.76x：Nq=1024 小 Q 场景下 **aux 链占比**反超主 kernel（整条前处理链的聚合吞吐上限 ≈1.04TB/s，第 三十五 章 28.6 节）；
- **D=512 大 D 区**：参照换 SDPA-MATH，2.98x–3.16x——数字大但要正确归因：一半来自 FFPA split-D 真加速（第 三十七 章），一半来自 MATH 参照自身的低效。

40.6.2 fp4 矩阵：D≥128 稳定 2x 级

表 40.5: fp4 CLI 矩阵（`--cuda-impl fp4`，同形状，fp16；16 格 allclose 全绿，tol dense 0.15 / causal 0.70）

	D=64		D=128		D=192		D=256	
task	FFPA/SDPA	speedup	FFPA/SDPA	speedup	FFPA/SDPA	speedup	FFPA/SDPA	speedup
self	1.61/2.40	1.49x ✓	2.52/5.11	2.03x ✓	3.48/7.14	2.05x ✓	4.97/10.46	2.11x ✓
causal	1.08/1.35	1.25x ✓	1.67/2.68	1.61x ✓	2.40/3.81	1.59x ✓	3.54/5.27	1.49x ✓
gqa	1.53/2.41	1.58x ✓	2.32/5.13	2.21x ✓	3.17/7.26	2.29x ✓	4.50/10.50	2.33x ✓
cross	0.36/0.40	1.10x ✓	0.68/0.82	1.21x ✓	1.02/1.16	1.14x ✓	1.41/1.40	0.99x ✓

self/causal 行取自第 三十八 章同轮 fp16 记录；gqa/cross 行为本机补充实测（2026-08，单卡 PRO 5000），原始 stdout 未随书归档。

与 fp8 对照，差别集中在三处：D≥128 后 fp4 稳定 2x 级且**随 D 单调上升**（blockscale MMA 无 fp8 的 smem 交叉点问题）；D=256 同点 fp4 比 fp8 快 1.64x（4.96 vs 8.13ms），对应第 三十七 章 dispatch 表；cross 小 Q 场景两者受同样限制（aux 链占比），fp4 也只有 0.99x–1.21x。

40.6.3 knob 矩阵：数值通路中性，粒度有代价

表 40.6 在 fp8 D=128 self（优势区中心）逐 knob 实测：

读数：优势区里 kernel 主体是 QK+PV tensor pipe（占 77%，第 三十六 章 29.6 节），数值通路 knob 全部落在 ±2.3% 内，**精度选择可以按需开，速度代价可忽略**（int8 QK 略慢是 s8 通路 scale 折叠多一层）。粒度 knob 则有真实方向：默认的 per-block 恰是最快档（量化 kernel 的向量化收益，第 三十五 章），升到 per-thread（Q/K）或 per-channel（V），粒度升级的净代价即这 1.7–2.7%。smooth-v 略快（+1.0%）是 V 值域压缩后 PV 溢出重缩放更少。knob 真正的杠杆在两端：causal 精度（hybrid）与大 D dispatch（第 三十七 章），不在优势区的数值通路。

表 40.6: fp8 knob 矩阵（D=128 self-attn，fp16，其余同 CLI 默认；基线即 CLI 默认配置：qk=fp8、pv=f32、Q/K/V 全 per-block）

配置	FFPA ms	TFLOPS	vs 基线
基线（CLI 默认）	3.01	365T	—
qk=int8	3.06	359T	−1.7%
pv-acc=f16	3.01	365T	±0%
qk=int8 + pv=f16	3.08	357T	−2.3%
q/k per-thread	3.10	355T	−2.7%
v per-channel	3.07	359T	−1.7%
smooth-v（fp8_smv）	2.98	369T	+1.0%

本表部分数值（int8/pv-f16 行）为本机补充实测（2026-08，单卡 PRO 5000），原始 stdout 未随书归档；基线与 per-thread/smooth-v 行同表 33.4 口径。

40.6.4　竞品对照

`bench_fp4.py`（16K, fp16, H32/Hkv8）同场四 backend：self 场景 FFPA-FP4 523T（2.45x）> Sage3 410T（1.92x）> FFPA-FP8 377T（1.77x）> SDPA 213T；causal 场景 Sage3 的 rel_err 0.59/max 1.0（早期行 e2m1 噪声无 hybrid 抑制，`bench_fp4.py` 头注释 L8–10 记录其量级 ∼1.2），FFPA 的 fp4-hybrid 把 max 压到 0.28；GQA 场景 Sage3 未支持（无数据）。fp8 侧 `bench_fp8.py` 与 Sage2 总体打平互有胜负（多数 ±1.5% 内，最差 non-aligned-dense 8K FFPA 快 2.6%），这正是 preset 公平化的结果：2026-08 之前 cachedit preset 默认开 hybrid+smooth-v，dense 场景「慢 3–8.7%」全是功能成本，对齐后反超。除表 40.1 的 5090 引用外，本章数字均出自本机 PRO 5000（sm_120）实测——**形态结论（谁快谁慢、随 D 怎么走）跨设备可迁移，绝对数字不可**，引用这类比值时必须带上设备名。
注：本节 Sage3 对照的 rel_err/max_abs（0.59/1.0 与 0.28/0.14）为本机补充实测（2026-08，单卡 PRO 5000），原始 stdout 未随书归档。

40.7　实践经验

选用决策树（dense 优先快、causal 先保精度）：

1. D≤192 且精度预算允许 ⇒ fp4（2x 级 + tol 0.15 内）；精度敏感行（早期行/高对比度）开 fp4-hybrid；
2. D=256 区间 ⇒ fp4（fp8 persist-D 在此有 smem 交叉点，1.3x vs fp4 的 2.1x）；
3. D>256 ⇒ fp8 split-D/M4N2（fp4 大 D 依赖 split-D 家族，按第 三十七 章 dispatch 表选）；
4. 小 Nq（decode/cross 小 Q）⇒ 先看 aux 链占比，量化的收益可能被前处理抵消——此时 fp16 persist-D 或 SDPA 反而合理；
5. causal 一律确认 hybrid 已开（CLI 默认自动），并按 bf16 口径复核 allclose。

Cache-DiT × FFPA。量化 attention 进入扩散模型管线要处理三件事。布局上，diffusers 上游产 NHD [B,N,H,D]，FFPA 的 `tensor_layout="NHD"` 走零拷贝 fast path（第 三十五 章 35.3.6 节），省掉两侧 permute。占比上，按式 40.3，attention 占比决定端到端收益上限，视频扩散（长序列、双流块）是最佳场景。cache-dit 还允许按层选 backend：因果块用 hybrid、双向块用纯 fp4，精度损失只在最敏感的位置修正。README 引用的 NeMo Automodel 案例（D=512 全注意力层 10/60，训练 e2e 1.4x–1.5x）是同一公式在训练侧的实例。

40.8　常见坑

1. **A/B 不校验 SDPA 参照**：两轮 bench 的 SDPA 时间差 > 1% 即作废重测（GPU 级污染会同时抬高 FFPA 与 SDPA，制造「全面 2x」假象）；
2. **小 D 静默回退**：D≤256 不设 `FFPA_CUDA_ALLOW_SMALL_D=1` 时 FFPA 静默走 SDPA fallback——看到 TFLOPS 只有 200T 级、profiler 里是 `flash_fwd_kernel` 就是静默回退的表现；
3. **功耗墙顺序偏置**：高功耗卡上单顺序 bench 谁排后面谁吃亏（实测差 466–491μs）；必须 pre-heat + 固定顺序，关键结论用匹配窗口交替复核；

4. **grad-on 测推理**：autograd 链每调用多 ~30μs Python 开销，且 fast path 永不命中——推理 bench 必须 `no_grad`；
5. **容差档位错配**：fp4 用 fp8 的 5e-2 去卡必然全红；反过来 fp8 用 0.7 会让真 bug（坏行、读错布局）漏检——tol 必须按 impl 分档（`_runner_fwd.py` L413–419 的分流就是为此）；
6. **D=512 speedup 的参照陷阱**：MATH 后端 materialize $N_q \times N_{kv}$ 矩阵，报告里必须注明参照换挡，否则「3x」里近半是参照自身的低效。

40.9 面试要点速查

1. min-of-N + no_grad + pre-heat + 固定顺序——推理 bench 四条规则，各自防什么坑；
2. speedup 口径三要素：参照后端（FA2/MATH 随 D 换挡）、min-of-N、E2E 摊销公式（占比 α 决定上界）；
3. 精度四指标：rel_err（能量）/max_abs（结构）/allclose （门禁）/tol 分档（fp8 5e-2、fp4 0.15/0.7，依据 e2m1 噪声地板）；
4. knob 空间三层：数值通路（QK/PV dtype）、量化粒度（per-block/channel/thread）、hybrid（前缀行 fp16 零冗余拼接）；优势区数值通路 knob 中性（±2.3%）、粒度升级 1.7–2.7%，杠杆在 dispatch 与 hybrid；
5. fp4 vs fp8 的性能形态差：blockscale MMA 让 fp4 无 smem 交叉点、全 D 段 2x 级；fp8 在 D=256 有交叉点、大 D 靠 split-D；
6. 竞品对照的公平性：preset 对齐（hybrid/smooth-v 是功能不是负担）、Sage3 对 GQA 未支持与 causal 噪声地板；
7. 小 Q 场景（decode/cross）的 aux 链占比问题：量化收益的边界在哪里。

40.10 最小测试与动手实验

```
# 实验 1: fp8 全 D 矩阵（表 33.4 口径）
export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1
for D in 64 128 256 512; do
  python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
    --tasks self-attn,causal,gqa,cross-attn --D $D --no-bwd --show-allclose
done
# 预期: D=128 self 1.70x; D=512 参照为 MATH（注意归因）

# 实验 2: fp4 全 D 矩阵（表 33.5 口径）
for D in 64 128 192 256; do
  python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp4 \
    --tasks self-attn,causal,gqa,cross-attn --D $D --no-bwd --show-allclose
done
# 预期: 16 格全绿; D=256 self 2.11x（对照 fp8 同点 1.30x）

# 实验 3: knob 矩阵（表 33.6 口径，fp8 D=128 self）
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
  --fp8-qk-mm-type int8 --tasks self-attn --D 128 --no-bwd
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
  --fp8-q-quant-method per-thread --fp8-k-quant-method per-thread \
  --tasks self-attn --D 128 --no-bwd
# 预期: 数值通路 knob 基线 ±2.3% 内; 粒度升 per-thread -2.7%

# 实验 4: 竞品对照 + plot 落盘（表 33.2/33.3 口径）
python bench/bench_fp8.py --dtype fp16   # 输出 bench/.tmp/*.png
python bench/bench_fp4.py --dtype fp16
# 预期: FFPA vs Sage 总体打平（最差差 2.6%）; fp4 causal 行
# Sage3 rel_err ~0.59 vs FFPA 0.14（hybrid 的作用）
```

延伸阅读

- `bench/bench_fp8.py` L1–20（口径 docstring）与 L344–347（pre-heat 顺序）、`bench/bench_fp4.py` L1–14（Sage3 对照口径）；
- `cli/_bench.py` L347–386（`--cuda-impl` 别名树）、L387–490（fp8/fp4 knob 全集）与 L648–694（`--cuda-impl` 解析出的默认量化配置）；
- `cli/_runner_fwd.py` L413–419（tol 分档依据）；
- ffpa-attn README「Performance / End-to-End Inference」节（Cache-DiT 集成入口）；
- 瓶颈归因方法见第 三十五–三十七 章（aux 链 1.04TB/s、tensor pipe 77%、dispatch 三分）。

第七部分

附录

附录 A　基础设施工具箱：common.cuh 解析

A.1　这个文件在全书的角色

正文的组织方式是「一章一个 kernel 家族，公共工具向 common.cuh 集中」：cp.async 协议宏、ldmatrix/HMMA 指令封装、XOR swizzle、TMA/mbarrier helpers、setmaxnreg 封装、WGMMA descriptor 编码——每一项都在某一章首次出现，之后被后续章节直接复用。本附录把这个头文件按工具族拆开，给出「地图 + 用法要点」：读正文时想确认某个宏的出处，或动手复现时想知道该 include 什么、能拿什么现成设施，都可以从这里查起。工具族与章节的对应关系见表 A.1；逐段的 permalink 源码区间见附录 D。

表 A.1: common.cuh 工具族地图（行号按当前源码树核对）

段	行区间	内容	主要使用章
1	L4–29	头文件、CUDART ≥13.0 门控（L21–23）、`INT4/FLOAT4/HALF2`（L25–27）、`kWarpSize`	3、8、9
2	L31–105	cp.async 宏（L41–49）、ldmatrix 宏（L56–72）、`HMMA16816/F32`（L81/L92）、`div_ceil`	9–12、16–18
3	L110–348	swizzle 家族：`permuted`（L158）、`swizzle_v1_impl`（L235）、`SwizzleBMS`（L287）、`swizzle_v2_impl`（L326）	12、14、23
4	L350–465	TMA/mbarrier helpers（双实现门控 L353–364）	13、14、17、18
5	L467–518	setmaxnreg 封装：注释（L467–498）、宏门控（L499–505）、两个模板（L507–517）	13、14
6	L520–712	WGMMA 家族：异步三件套（L539–543）、`SMEM_DESC_ENCODE`（L552）、`make_smem_desc`（L588）、指令宏（L644）、`WgmmaSMem`（L702）	13
7	L714–722	SM120 适配：`TmaMmaWSSMem`（L717）	14
8	L724–803	host 侧 TMA descriptor：`create_tensor_map`（L765）、`allocate_and_create_tensor_map`（L791）	13、14

第 4–8 段整体由编译宏门控：不定义 `NOTES_V2_ENABLE_WGMMA` 或 `NOTES_V2_ENABLE_TMA_MA_WS` 时，TMA/WGMMA 相关设施完全不参与编译——这是「基础章构建保持轻量」的机制保证（各宏与架构的组合见附录 C）。

A.2 搬运原语：向量化宏与 cp.async

`FLOAT4(v)`（L26）把任意左值重解释为 `float4` 引用，实现一条 128-bit 访存指令加载 4 个 float——第 四 章的向量化与第 九 章 GEMV 的核心手段。Ampere 时代的异步全局到共享内存搬运则由四个宏构成完整协议（第 十一 章双缓冲首次使用）：

common.cuh L41-49（cp.async 协议宏：commit/wait 语义与 cg 粒度）

```
#define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
#define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
#define CP_ASYNC_WAIT_GROUP(n)                                                 \
  asm volatile("cp.async.wait_group %0;\n" ::"n"(n))

// 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
#define CP_ASYNC_CG(dst, src, bytes)                                           \
  asm volatile(                                                                \
      "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" ::"r"(dst),       \
```

用法要点：`commit_group` 把此前未提交的 `cp.async` 归入一个 async-group（per-thread）；`wait_group N` 阻塞到「未完成 group 数 ≤ N」，N=0 即等全部落地——这与后面 WGMMA 的 `wgmma.wait_group`（L542）语义完全同构，是贯穿第 十一 、十二 、十四 章的同一套「分组-等待」机制。`cg` 只支持 16 字节粒度（`ca` 支持 4/8/16），与 `FLOAT4` 天然对齐。

A.3 Tensor Core 入口：ldmatrix 与 HMMA 宏

ldmatrix 家族（L56–72）把 smem 中 8×8 的矩阵片段按 Tensor Core fragment 布局装进寄存器：`LDMATRIX_X4` 一次装 4 个片段，`LDMATRIX_X2_T` 的 `.trans` 后缀做转置加载——FA 里 P@V 的 B 矩阵需要 col-major，而 V 在 smem 是 row-major，转置在这一步完成（第 十七 章）。计算侧是两个指令宏：`HMMA16816`（L81，f16 累加）与 `HMMA16816F32`（L92，f32 累加），对应 `mma.sync.aligned.m16n8k16` 的两个变体。寄存器口径：16×8=128 个输出元素由 32 线程分担，f16 变体每线程 2 个 `uint32`、f32 变体 4 个。使用者：hgemm 家族（第 十二 、十三 章）、FA2 （第 十七 章）；第 十五 章的 SM120 kernel 同样复用 ldmatrix + `HMMA16816F32`。

A.4 smem swizzle 家族

第 3 段（L110–348）是全书篇幅最大的工具族，动机、bank 分布表与 `Swizzle<B,M,S>` 位级语义的完整推导都在第 十三 章，这里只给接口速览。核心是按列宽模板化的 `permuted`——三档 TMA 硬件 swizzle 模式的手写 XOR 展开：

common.cuh L171-183（permuted：SWIZZLE_32B/64B/128B 三档 XOR 分支）

```
  } else if constexpr (kColStride == 16) {
    return (((j >> 3) ^ (i >> 2)) & 1) << 3;  // SWIZZLE_32B
  } else if constexpr (kColStride == 32) {
    const int chunk = (j >> 3) & 3;
    const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
    return (chunk ^ xor_mask) << 3;  // SWIZZLE_64B
  } else {                           // kColStride == 64
    const int chunk = (j >> 3) & 7;
    const int xor_mask =
        (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);
    return (chunk ^ xor_mask) << 3;  // SWIZZLE_128B
  }
}
```

族内四个实体的关系：`swizzle_v1_impl`（L235）就是 `permuted<kColStride,8>` 的别名；`SwizzleBMS<B,M,S>`（L287）把 cute 的 `Swizzle<B,M,S>::apply` 位级公式独立实现（L287–320），`swizzle_v2_impl`（L326）用它统一表达 16/32/64 三档，与 v1 bit-exact 等价（等价性证明注释在 L320–325，host 端 `test_swizzle_equiv` 可

验证）。第 二十四 章讲 CuTe swizzle 时，`SwizzleBMS` 正是「脱离 cute 框架理解位级语义」的对照实现。一个容易踩的坑：L344–348 的派发器注释描述的 `swizzle<kColStride>` 函数**实体在 hgemm.cuh**，common.cuh 只提供 v1/v2 两个实现与注释（第 十三 章考据框有记录）。

A.5　TMA/mbarrier helpers 与 setmaxnreg

第 4 段（L350–465）在 `NOTES_V2_FORCE_INLINE_ASYNC_PROXY` 门控下提供两套等价实现：默认走 raw `asm volatile` PTX；未定义该宏则走 `cuda::ptx` / `cuda::device` C++ 封装（L423 起）。两套并存的原因是 ptxas 对 setmaxnreg 的 C7506 丢弃问题（下文）。raw 形式的 2D TMA 加载：

common.cuh L385-407（tma_load_2d：cp.async.bulk.tensor.2d raw PTX 形式）

```
// NOTE: dst must be shared::cta, NOT shared::cluster. ptxas (CUDA 13.2) treats
// the shared::cluster form as an implicit extern-call boundary and silently
// drops setmaxnreg from the same kernel (C7506). cute's TMA copy ops emit
// shared::cta on SM120 targets (CUTE_ARCH_TMA_SM120_ENABLED gate in
// copy_sm90_tma.hpp; the sm_90 path still emits cluster), which is why CuTe
// WS kernels keep their register rebalancing. Semantics are identical here:
// the destination is always CTA-local smem. Verified by PTX surgery:
// cluster->cta flips C7506 off and USETMAXREG appears (both sm_120a/120f).
static __device__ __forceinline__ void tma_load_2d(
    void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
    cuda::barrier<cuda::thread_scope_block> &barrier) {
  uint64_t gmem_int_desc = reinterpret_cast<uint64_t>(tensor_map);
  uint32_t smem_int_ptr = notes_cast_smem_ptr_to_uint(dst);
  uint32_t smem_int_mbar =
      notes_cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
  asm volatile(
      "cp.async.bulk.tensor.2d.shared::cta.global.mbarrier::complete_tx::bytes"
      " [%0], [%1, {%3, %4}], [%2];"
      :
      : "r"(smem_int_ptr), "l"(gmem_int_desc), "r"(smem_int_mbar),
        "r"(minor_coord), "r"(major_coord)
      : "memory");
}
```

要点：目的地址与 mbarrier 都是 32-bit smem 地址（`__cvta_generic_to_shared` 转换）；数据到齐由硬件对 mbarrier 做 complete-tx 计数，配合 `tma_arrive_expect_tx`（声明期望字节数）构成第 十四 、十五 、十八 、十九 章流水线的 full/empty 双屏障协议。寄存器再分配封装则被 `NOTES_V2_ENABLE_SETMAXNREGS` 独立门控：

common.cuh L507-517（warpgroup_reg_dealloc/alloc：setmaxnreg 指令模板）

```
template <uint32_t kNumRegs>
__device__ __forceinline__ void warpgroup_reg_dealloc()
{
  asm volatile("setmaxnreg.dec.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
}

template <uint32_t kNumRegs>
__device__ __forceinline__ void warpgroup_reg_alloc()
{
  asm volatile("setmaxnreg.inc.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
}
```

用法要点（完整事实链见第 十五 章考据框与坑二取证）：两条指令都要求 warpgroup 内全部 warp 同步执行，且 WS kernel 须声明 `__launch_bounds__(N,1)`（否则 PTX 无 `.minnctapersm`，ptxas 以 C7508 丢弃）；sm_120 上能否保留则取决于 TMA 的目的地址空间——dst 为 `shared::cluster` 时 ptxas 把它当作隐式 call 边界、以 C7506 警告丢弃 setmaxnreg，dst 为 `shared::cta`（common.cuh 现行实现，cute 在 SM120 门控下同款）则 sm_120a/sm_120f 均保留生效（ffpa-attn 实测，详见附录 C 的目标选择表）。默认构建里宏展开为 `((void)0)`，调用点零成本。

A.6 WGMMA 家族与 host 侧 TensorMap

第 6 段（L520–712）是 Hopper 专用：`WGMMA_FENCE/COMMIT_GROUP/ WAIT_GROUP`（L539–543）延续 cp.async 的分组-等待语义；`make_smem_desc`（L588）把 smem 指针编码成 64-bit 矩阵描述符（位域布局与 K-Major + 128B swizzle 的几何推导在第 十四 章，不重复）；`WGMMA_M64N128K16_F16F16F16`（L644）封装 `wgmma.mma_async.m64n128k16`，一次发射计算 $64 \times 128 \times 16$。配套的 `WgmmaSMem`（L702）与 SM120 的 `TmaMmaWSSMem`（L717）只差 alignas：前者供 descriptor 硬件补偿 swizzle 相位，后者要求 consumer （ldmatrix）用软件公式感知相位（第 十五 章 14.4.2 节）。host 侧创建 TMA descriptor 的入口：

common.cuh L765-785（create_tensor_map 核心：shape/stride/box 的 (minor,major) 顺序）

```
__host__ static inline void create_tensor_map(CUtensorMap *tma_map,
                                              half *gmem_ptr,
                                              int blocks_height,
                                              int blocks_width) {
  void *gmem_address = (void *)gmem_ptr;
  uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
                                 (uint64_t)BlockMajorSize * blocks_height,
                                 1, 1, 1};
  uint64_t gmem_prob_stride[5] = {
      sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
  uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
                                uint32_t(BlockMajorSize), 1, 1, 1};
  uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
  CUresult result = cuTensorMapEncodeTiled(
      tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
      gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
      CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
      CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
  if (result != CUDA_SUCCESS)
    printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
}
```

两个高频易错点（第 十四 章展开）：TMA 的 shape 与 box 都按 (minor, major) 排列，row-major $[M, K]$ 矩阵要传 (K, M)；`gmem_prob_stride + 1` 跳过隐式的最内维 stride（元素类型宽度），API 只接收 rank−1 个 stride。swizzle 模式固定选 `CU_TENSOR_MAP_SWIZZLE_128B` 与 `make_smem_desc` 的 layout_type=1 匹配。

A.7 写一个新教学 kernel 的最小 checklist

从零加一个「正文可引用、测试可复现」的教学 kernel，按这条路径走：

1. **include**：kernel 头文件放 `kernels/interview/`，直接 `#include "common.cuh"`；需要归约/softmax/norm 等基础件则 `#include "base.cuh"`（测试侧以 `#include "../../base.cuh"` 相对路径引用，见 book/tests/README.md 约定）。

2. **按目标代际选工具族**：sm_89 走 `FLOAT4` + cp.async + ldmatrix + `HMMA16816`；sm_90a 追加 TMA helpers + `make_smem_desc` + WGMMA 宏；sm_120 用 TMA helpers + mma.sync 路径（WGMMA 不存在），swizzle 记得从 `<16>` 换 `<64>` 档。

3. **kernel 写作三件事**：`__launch_bounds__` 写满；128B swizzle 路径的 smem 基址对齐 1024B（第 十五 章坑三）；网格维度一律 `div_ceil`（L104）。

4. **对拍**：测试复用 `book/tests/common_test.h`——`book_fill_rand` 造确定性输入，CPU fp64 写参考，`book_check` 按三档容差判定（附录 C）；规模不超过 `BOOK_TEST_MAX_N`。

5. **arch 门控**：普通特性用 `book_require_sm` 打 SKIP；家族特性（如 WGMMA）必须精确判断家族——`>=` 挡不住「sm_120 ≥ sm_90 但无 WGMMA」的假阳性（第 十四 章教训）。

6. **落盘验证**：单文件写 `book/tests/chNN_<slug>.cu`，`./build_tests.sh --ch chNN` 跑到尾行 `ALL OK`。
7. **接回正文**：从 `chapters/_template.tex` 起步写章稿，源码区间登记 `scripts/anchors.yaml`，进附录 D 索引。

附录 B　性能数据汇总

B.1　口径与标尺

除单独标注外，本附录所有数字都是本书复测：RTX PRO 5000 72GB（sm_120a，110 SM，L2 96MB，CUDA 13.2），cudaEvent 计时，warmup 3–5 + 10–20 次取均值；roofline 标尺取自第 二 章（GDDR7 理论带宽 1344 GB/s、FP32 峰值估算 66.9 TFLOPS、ridge point ≈ 49.8 FLOP/B）。读表先记三件事：（1）带宽型算子分 L2/HBM 两口径，工作集小于 96MB L2 时实测带宽可以超过 HBM 理论值（第 四 章 1862 GB/s 的来源）；（2）TFLOPS 跨形状不可直接比较，「形状」列必须一起读；（3）FLOPS 统一按 GEMM $2MNK$、attention $4BHN^2D$ 折算。

B.2　全书核心演进主线

章	kernel	形状	耗时/成绩	相对基线
第 02	`dot` / `dot_vec4`	N=4.19 M	0.042 ms；806 / 808 GB/s	三版同为 42 μs：瓶颈是全局原子串行
第 04	`softmax`（online）	S=4096，N=256	0.0142 ms；591.6 GB/s	naive 0.0121 ms 最快但数值不安全
第 05	`merge_attn_states`	T=16384，H=32，D=128	0.785 ms；1031.4 GB/s	HBM 理论带宽的 77%
第 07	`mat_transpose_padded`	2048^2	0.0145 ms；2320 GB/s	2.40× vs naive（L2 口径）
第 08	`sgemv` k128	M=8192，K=4096	0.144 ms；934.2 GB/s	全书实测带宽参照；较 k32 +5.2%
第 09	`sgemm_vec4`	2048^3	0.661 ms；25.97 T	4.76× vs naive（3.144 ms，5.46 T）
第 10	`sgemm_tf32`	1024^3	28.19 T	换 Tensor Core 的第一步
第 11	`hgemm_mma_stages_tn`	1024^3	76.55 T（f16 acc）	2.71× vs 第 10 章 TF32
第 12	7b-1（BK=16）	4096^3	124.7 T	7b-3 反而 118.6–119.0 T
第 14	`hgemm_tma_mma_ws_tn`	2048^3	0.104 ms；164.64 T	1.57× vs 第 11 章同形状 104.66 T
第 16	FA2 split-Q（F32Acc）	B=1，H=32，N=4096，D=64	0.781 ms；175.88 T	F16Acc 166.56 T（F32 反快 +5.6%）
第 17	FA2 TMA+WS（F32Acc，Sk=2）	N=16384，D=128	195.2 T	同口径 F16Acc 183.8 T
第 18	FA3 双 consumer（F32Acc，Sk=1）	N=4096，D=128	194.2 T	D=64 上 159.8 vs FA2 的 141.8（+13%）
第 19	FFPA TMA WS	N=8192，D=512	110.7 T vs cuDNN 57.2（1.94×）	注释口径（未复测）；README 5090：182.6 vs 83.1（2.20×）

B.3　Part I：带宽型算子明细

有效带宽按各章声明的有效流量折算；L2/HBM 口径以工作集是否落入 96MB L2 划分。

章	kernel	形状	ms/iter	GB/s	口径与备注
第 02	`block_reduce_all`	N=4.19 M	0.0416	402.9	G=16384　次全局原子串行（2.6 ns/op）
第 02	`dot` / `dot_vec4`	同上	0.0416	806.2 / 807.5	vec4 零收益：原子串行
第 03	`relu`（_vec4）	N=4.19 M	0.0180	1862 / 1866	L2 口径（16 MB 工作集）
第 03	`elementwise_add`（_vec4）	同上	0.0190	2653.7 / 2611.9	L2 口径
第 03	`histogram` 256 bins	同上	0.6286	26.7	原子争用，慢两个数量级
第 04	softmax naive / safe / online	S=4096，N=256	0.0121 / 0.0146 / 0.0142	690.6 / 572.9 / 591.6	短 kernel，launch 开销占比大
第 05	`merge_attn_states`	T=16384，H=32，D=128	0.7849	1031.4	HBM 口径
第 06	`rms_norm`（_vec4）	N=131072，K=128	0.1289 / 0.1107	1041 / 1212	L2 口径，vec4 +16%
第 06	`layer_norm`（_vec4）	同上	0.1384 / 0.1118	970 / 1201	1-pass 稳定省约 7%
第 06	四 kernel（HBM）	N=524288	0.50–0.54	1000–1070	HBM 口径，vec4 收益被吸收
第 07	transpose naive → padded	2048^2	0.0347 → 0.0145	966 → 2320	L2 口径 2.40×
第 07	transpose padded	8192^2	0.5318	1010	HBM 口径，收益缩至 1.11×
第 08	`sgemv` k32 / k128	M=8192，K=4096	0.1512 / 0.1437	888.3 / 934.2	k128 = 带宽参照
第 08	`sgemv` k16	M=2^{20}，K=16	0.1078	661.4	参照的 71%（无 ILP 藏延迟）

B.4　Part II：GEMM 阶梯与 bank conflict

GEMM 的优化路线在三个形状上各取一段可受控对照的阶梯（T 为 TFLOPS）：

形状	配置	ms/iter	相对基线
1024^3	`sgemm_tf32`（WMMA，B_K=8）	28.19 T	基线（第 十一 章）
1024^3	`hgemm_mma_stages_tn`（HMMA，B_K=16，S=3）	76.55 T	2.71×
2048^3	`hgemm_mma_stages_tn`（cp.async，S=3）	0.1641；104.66 T	基线（第 十五 章对照）
2048^3	`hgemm_tma_mma_ws_tn`（TMA+WS，S=2）	0.1043；164.64 T	1.57×
4096^3	7b-1（BK=16，kS=3，无 swizzle）	124.7 T	基线（第 十三 章）
4096^3	7b-3（BK=64，kS=2，BS=0/1）	118.6 / 119.0 T	−4 ∼ 5%（4-way 冲突抵消同步收益）
2048×16384×4096	7b-3 BS=1 vs BS=0	122.9 vs 117.9 T	宽 N 形状 block swizzle +4.2%

swizzle 消除 bank conflict 的微观证据（第 十三 章 NCU 计数器，M=N=K=512）：BK=16 基线 98,304 次冲突/launch、17.6 μs；BK=64 无 swizzle 691,067 次、30.2 μs；BK=64 + swizzle 297,628 次、20.7 μs（8-way 降 4-way，1.46×）。

B.5 Part III：Attention 演进

除标注外均为 B=1, H=32 本书复测（T 为 TFLOPS）：

配置	形状		TFLOPS	备注
FA2 split-Q F16Acc	N=4096, D=64		166.56	与源码注释 166.6 逐位吻合
FA2 split-Q F32Acc	N=4096, D=64		175.88	F32 累加反而更快
FA2 compact XOR（消融）	N=4096, D=64		120.0	注释口径；写端 wavefront 2.26×
FA2 TMA WS Sk=2 F16Acc	D=128，N=4096 16384	/	172.5 / 183.8	Sk=2 即饱和（Sk=3 <0.5%）
FA2 TMA WS Sk=2 F32Acc	D=128，N=4096 16384	/	191.1 / 195.2	
FA2 cp.async → TMA WS	N=16384, D=128		158.7 → 190.4	原版辅助工具对照，+20%
FA3 Sk=1 F16Acc	D=128，N=4096 16384	/	190.4 / 185.2	双 consumer，smem 限 Sk=1
FA3 Sk=1 F32Acc	D=128，N=4096 16384	/	194.2 / 189.7	
FA3 vs FA2（D=64）	N=4096		159.8 vs 141.8	+13%（B_r=64 小 tile 两条流）
FFPA TMA WS vs cuDNN	N=8192, D=512		110.7 vs 57.2	注释口径（PRO 5000，1.94×）
FFPA Sk=Sv=1 → 2	N=16384, D=320		127.2 → 182.6	README 5090 口径（cuDNN 83.1）
FA2 论文锚点	A100 前向		230（73% 峰值）	论文口径，供量级对照

第 十九 章的 FA2/FA3 对照不是受控 A/B（B_r、Sk、网格形状均不同），读作「两条路线在同一张卡上的量级对照」。

B.6 Part IV：CuTe 对照口径

CuTe 篇的核心结论是**抽象零开销**，对照维度因此不是 TFLOPS 而是「同一份指令序列 + 更少的代码 + 可静态断言」（详见第 二十五 章）：

维度	手写	CuTe
swizzle kernel 代码量	235 行	118 行 + wrapper 88 行
HGEMM kernel 代码量	147（7b）/ 231（7b-3）行	118 行 + wrapper 86 行
寄存器（`ptxas -v`）	—	254（B_N=256/F16）/ 226（B_N=128/F32），均 0 spill
SASS 指令序列	LDGSTS + LDSM + HMMA	同左；kernel 内联汇编 0 条
swizzle 正确性	v1/v2 30,720 点 bit-exact	`Swizzle<3,3,3>` 256 点 0 差异

B.7 Part V：FP8 GEMM 明细

Part V（第 三十–三十一 章）是量化矩阵乘的教学链：基准是 cuBLAS BF16 GEMM（`CUBLAS_COMPUTE_32F` 累加），数字为 `notes_v2_sm120a.bin --bench --mnk M,N,K` 标准表复测（PRO 5000，2026-09-24；输入均匀随机 $[-1,1]$，warmup 2 / repeat 3）。kernel-only 行只循环 GEMM kernel，E2E 行含量化前处理（第 三十 章三个 kernel）。四种 scale 粒度组合差异 < 4%（2048^3），表取 per-row× per-col 代表，tile 取库默认档（$128\times256\times128$/s2，即 35.8.4 节的扫描最快档）。基线口径注意：cuBLAS 在 sm_120 的 32F-acc 档只有 sm_80 旧 kernel（`nsys` 取证见第 三十一 章 35.8.1 节，cublasLt 完整池遍历也只有 191.7 T），因此表中加速比是「库现状」口径；对手写 bf16 HGEMM 上限（≈243 T）的口径为 1.81×（4096^3）。

章	kernel 家族	形状	成绩	备注
第 34	fp8 量化前处理链	4096^3（E2E 与 kernel-only 之差）	≈424 μs/次	纯带宽工序；两遍读+staging 有 ∼2× 优化空间（35.8.2 习题）
第 35	fp8 GEMM CuTe NonWS	2048^3/4096^3/8192^3	244/441/484 T (1.77/2.70/2.96×)	kernel-only vs cuBLAS BF16；上限口径 1.81×（4096^3）；WS 版 +0∼0.9%
第 35	fp8 GEMM+Quant E2E	同上	76/187/315 T (0.55/1.14/1.93×)	量化开销随 shape 缩小占比放大；2048^3 反而更慢
第 35	fp8 GEMM+A-Quant E2E（B 离线）	同上	226/411/429 T (1.64/2.52/2.63×)	权重量化一次后落盘，每步只量化 A（$O(MK)$）；数值与全链路逐 bit 一致（35.8.3 节）；WS 版 +0.5∼1.1%
第 35	fp8 WS + setmaxnreg	4096^3	440.9 T（2.70×）	直调 cutlass::arch API；SASS 4 实例 × 2 条 USETMAXREG 实证；sm_120a/f 均可
第 35	fp8 tile/深度扫描（non-WS）	4096^3/8192^3	219∼442 / 240∼485 T	tile $\{64,128,256\}^2\times$ stages $\{2,3,4\}$：BN 比 BM 更有效（128×256 vs 256×128 两档都 ≈ +8.5%），s2/s3 差 ≤ 2%、s4 无收益，256×256 超 99 KB 装不下（35.8.4 节）
第 35	fp8 最优 tile（ws 128×256/s2）	4096^3/8192^3	443.7/487.8 T (2.72/2.99×)	即库默认档（同时是大形状全场最优）；比 128^3/s3 快 7.7%/5.9%；2048^3 上因波次量化反慢 17%；NCU：pipe_tensor 80.9%（elapsed）、寄存器 168/线程、bank conflict 0、smem 读波前 −5.6%（35.8.5 节）

B.8 Part VI：FP8/FP4 量化 attention 明细

Part VI（第 三十四–四十 章）的性能口径与前三篇不同：基准是 PyTorch SDPA（FA2 后端，$D > 256$ 自动落 MATH），数字为 `ffpa_attn.bench` CLI 复测（B1 H32 Hkv8 N=8192，fp16，PRO 5000，2026-09-16；tol 分档 fp8 5e-2 / fp4 dense 0.15、causal 0.70）。speedup 一律对 SDPA 同形状。列内数字取自各自所在章节的原始记录，跨章对比时注意口径。

章	kernel 家族	形状	成绩	备注
第 28	fp8 aux 链（quantize/smooth）	D=128, N=8192	聚合带宽 ≈1.04 TB/s	纯带宽接近峰值（HBM 78%）；smk/smv 零开销
第 29	fp8 persist-D	self D∈[64,256]	1.26–1.70×	D=192 正确性 bug 见 29.7.1；tensor pipe 77%
第 30	fp8 split-D / M4N2	D=320 self / causal	3.76× / 3.31×	D=512 2.98×（参照换 MATH，见 33.6.1 归因）；causal 大 D allclose 临界失败为已知 ESS 特性（hybrid 解法）
第 31	fp4 量化链（fused）	D=128 N=8192	fused 单 launch 1.4→0.3 ms	hadamard 零开销（bitwise 一致）
第 32	fp4 persist-D	self D∈[64,256]	1.49–2.11×	同点 D=256 比 fp8 快 1.64×（对照见 33.6.2）；mxfp8 PV 变体时间代价 +28–39%（精度兜底）
第 33	bench 矩阵与 knob	D=128 self	fp4 523 T（2.45×）	fp4 vs fp8 1.39×；粒度 knob 代价 1.7–2.7%；Sage3 对照 causal rel_err 0.59 vs hybrid 0.14（523T 为 `bench_fp4.py` 16K 口径，非 CLI N=8192）

非本地口径（ffpa-attn README，RTX 5090，引用不复测）：fp8 D=128 FFPA-FP8 441 T vs Sage2 394 T vs SDPA 291 T；fp4 D=128 FFPA-FP4 804 T（2.60×）。同 sm_120 微架构下 PRO 5000 绝对值低 ∼15%（fp8）/∼35%（fp4），形态结论可迁移、绝对数字不可（第 四十 章 33.6.4）。

B.9 四条性能主线

主线一：带宽（合并访问与向量化）。Part I 的全部算子 $AI \leq 0.625$，成绩就是有效带宽。这条线的三个阶段：先把原子/同步移出关键路径（第 三 章 42 μs 三行全同证明全局原子串行可以完全掩盖一切带宽优化）；再做合并访问与向量化（`sgemv` k128 拿到 934 GB/s 的全书带宽参照）；最后认清收益边界——vec4 只在 L2 口径显现（第 七 章 +16–24%），HBM 口径下被 DRAM 带宽吸收（第 四、八 章）。**先定层级，再选方法**。

主线二：计算（MMA 流水）。GEMM 从 CUDA Core 的 25.97 T（vec4 已是该路径的 87% 带宽上限）起步，换 HMMA 与 B_K=16 后到 76.55 T，加深流水与统一 tile 到 124.7 T，TMA+warp specialization 后 164.64 T。attention 走完同一条路：166.6 → 195.2 T。共同规律是**流水深度很快饱和**（FA2 的 Sk=2 即饱和、FA3 的 Sk=1 即饱和）——装载一旦退出关键路径，剩下的增益只能从结构上找。

主线三：结构（warp specialization 与 CuTe 抽象零开销）。第 十五 章把搬运端换成 TMA + mbarrier + 双角色 warp（计算指令一条不动）即得 1.57×；第 十八 章在 attention 上复现同样的 +20%。CuTe 把这套结构变成类型：layout/TiledCopy/TiledMMA 全部编译期折叠，SASS 与手写同构、寄存器 0 spill、代码量约减半——它不直接产生加速，但让 swizzle 档位、tile 形状、流水深度这些 **真正带来加速的算法级决策**更容易被表达、被检查、被复用。

主线四：量化（低位宽数据通路）。Part V 先在矩阵乘上打通「量化数学 → 前处理 kernel → 主 kernel 折叠 → BF16 输出」的完整链路（fp8 GEMM 对 cuBLAS BF16 最高 2.99×），Part VI 把 attention 的数据通路从 fp16 一路降到 fp8/fp4：fp8 persist-D 1.70×（D=128）起步，fp4 同点 2.04×、大 D 靠 split-D/M4N2 3× 级。这条线的规律与前三

条不同——收益不来自新结构，而来自**同样的数学量换更宽的通路**（blockscale MMA）；代价是精度管理（smooth、两级 P、hybrid），而度量本身也需要专门处理（第 四十 章）。

附录 C　构建、运行与最小测试指南

C.1　两套构建入口的分工

仓库里有两个构建入口，角色不同、不要混用：

- `kernels/interview/build.sh`——编译集成 bench 辅助工具（多 TU：`bench/bench_leetcuda.cu` 总装 + `bench/` 模块.cu，带 cuBLAS/cuDNN 对照与性能计时）。检测到 ccache 时走两段式：`ccache nvcc -c` 产 `.o`（可缓存），再裸 `nvcc` 链接产 `leetcuda_bench_<arch>.bin`（不缓存）。公共 flags：`-std=c++20 -O3 --expt-relaxed-constexpr --use_fast_math` + cutlass/cudnn-frontend include。
- `book/tests/build_tests.sh`——编译并运行每章最小正确性测试。本书的作用是**取代集成辅助工具的正确性验证角色**：对拍一律 CPU fp64，无 cuBLAS/cuDNN 依赖，可在任何机器复现；辅助工具保留为集成 bench，现为多 TU——`bench/bench_leetcuda.cu`（440 行）只剩 main 与 CLI 解析，host 侧 test/bench 拆到 `bench/` 下 base/sgemv/sgemm/hgemm/ fp8_gemm/fp4_gemm/flash_attn/ffpa_attn 八个模块.cu 与 `bench/utils.cu` 共享工具。

build.sh 的架构 × 编译宏矩阵（决定 common.cuh 哪些段参与编译，见附录 A）：

表 C.1: build.sh 架构配置与产物

`--arch`	编译宏	产物
sm_86	CUTE, CUDNN	leetcuda_bench_sm86.bin
sm_89	CUTE, CUDNN	leetcuda_bench_sm89.bin
sm_90a	WGMMA, CUTE, TMA_MMA_WS, CUDNN	leetcuda_bench_sm90a.bin
sm_120a	CUTE, TMA_MMA_WS, CUDNN	leetcuda_bench_sm120a.bin

C.2　ffpa-attn（Part VI 源码仓）安装与 bench

Part VI（第 三十四–四十 章）的源码在独立仓库 `ffpa-attn`（锚定 commit `861d75e`，与 LeetCUDA 同 checkout 于 monorepo）。两种安装形态：

```
# 仅 Python 侧（Triton/CuTeDSL 开发，跳过 CUDA kernel 编译）
pip uninstall ffpa-attn -y && pip install -e . --no-build-isolation

# 完整 CUDA kernel 编译（本地 PRO 5000 是 sm_120a，用 sm_120f family 目标
# 构建以覆盖整个 Blackwell 家族；setmaxnreg 的保留条件是 TMA dst 用
# shared::cta（cute 在 SM120 下即如此），见第~\ref{ch:14}~章坑二取证）
bash ./build.sh --arch sm_120f --headdim default --ext all --jobs 64
```

bench 验收用 `ffpa_attn.bench` CLI（第 四十 章的矩阵与 knob 口径）。小 headdim 必须先放开环境变量，否则 FFPA **静默回退 SDPA**（TFLOPS 掉到 200T 级说明已回退）：

```
export CUDA_VISIBLE_DEVICES=7 FFPA_CUDA_ALLOW_SMALL_D=1
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp8 \
  --tasks self-attn,causal --D 128 --no-bwd --show-allclose
```

```
python -m ffpa_attn.bench --fwd-backend cuda --cuda-impl fp4 \
  --tasks self-attn,causal --D 128 --no-bwd --show-allclose
# 竞品对照层（Sage2/3 同场，生成 TFLOPS plot）：
python bench/bench_fp8.py --dtype fp16    # 输出 bench/.tmp/*.png
python bench/bench_fp4.py --dtype fp16
```

C.3　book/tests/build_tests.sh 用法

```
cd kernels/interview/book/tests
./build_tests.sh --arch sm_120a --all   # 构建运行全部已存在测试
./build_tests.sh --ch ch04              # 单章
./build_tests.sh --ch ch13 --no-run     # 只编译不运行（远端无 H800 时）
```

行为细节（对应脚本 L13/L47–51/L58/L65）：`--arch` 默认 `sm_120a`（验收机 RTX PRO 5000），`NVCC` 可用环境变量覆盖；ch13（WGMMA）在非 `sm_90a` 目标下自动跳过并打印提示；每个测试用 `-std=c++20 -O2` + cutlass include 单文件编译；全部成功时末行 `ALL OK`、退出码 0，任一编译或运行失败输出 `FAILURES PRESENT` 并返回非 0。生成物在 `.build/` 下：与源文件同名的可执行文件（无扩展名，如 `.build/ch01_roofline`）与逐源的 `<name>.buildlog`。

C.4　公共对拍头 common_test.h

所有章测试共享 `book/tests/common_test.h`（72 行）：三档容差 `TOL_F32ACC`=1e-3 / `TOL_F16ACC`=5e-2 / `TOL_TF32`=1e-2（L14–16）；测试规模上限 `BOOK_TEST_MAX_N`=512（L19，CPU fp64 三重循环的时限约束）；`book_require_sm`（L32）arch 不在位时打印 `SKIP(chNN)` 并正常退出；`BOOK_CUDA_CHECK`（L39）统一 CUDA 错误拦截；`book_check`（L51）打印 `max_abs_err` 与首个超差下标；`book_fill_rand`（L69）以固定种子 0xA5A5 造确定性输入。

C.5　-arch 目标选择

表 C.2: -arch 目标选择表

目标	适用场景
sm_89	Ada Lovelace（RTX 40 / L20）。基础章至 GEMM 章的通用目标。
sm_90a	Hopper（H100/H200/H800）。WGMMA 章必需：build_tests.sh 在其它目标下自动跳过 ch13；完整编译须用 `-gencode arch=compute_90a,code=sm_90a` 形式（`-arch sm_90a` 会退化为 compute_90，第 十四 章 13.7 节勘误）。
sm_120a	Blackwell（RTX 5090 / PRO 5000/6000），build_tests.sh 与本书验收机的默认目标：ch14/ch17/ch18 的 TMA+mbarrier 路径直接可跑。
sm_120f	family-specific 目标（CUDA 13.2+），build.sh 的 setmaxnreg 实验目标（预设 `NOTES_V2_ENABLE_SETMAXNREGS` 与 raw-PTX/cta TMA 路径）。**保留 setmaxnreg 的真正条件是 TMA dst 用 `shared::cta`**：dst 为 `shared::cluster` 时 ptxas 以 C7506 静默丢弃（与 arch 后缀无关，sm_120a/sm_120f 都丢；修复后两者都 112/112 保留，第 十五 章坑二取证）。sm_120f 额外的好处是整个 Blackwell 家族二进制兼容。

工具链门槛：TMA 相关章硬性要求 CUDA ≥ 13.0（common.cuh L21–23 直接 `#error`）；sm_89/sm_90a/sm_120a 后缀目标与 sm_120f 建议 CUDA 13.2+。

C.6 常见编译与运行错误速查

以下报错均为真实编译器输出照录（探针最小复现，CUDA 13.2）：

```
# 1. WGMMA 用在非 sm_90a 目标上（忘了换 -arch，或误在 sm_89/sm_120 上编 ch13）
ptxas ... error : Instruction 'wgmma.commit_group' not supported on .target 'sm_89'
# -> WGMMA 仅 sm_90a 存在；sm_120 改用 mma.sync 路径（ch14）

# 2. smem 超过 sm_90a 单 block 上限（227 KB = 0x38c00 = 232448 B）
ptxas error : Entry function '_Z1kPf' uses too much shared data (0x40000 bytes, 0x38c00 max)
# -> 减 kStages 或缩小 tile；多级流水先算 smem 预算

# 3. smem 超过 sm_120a 单 block 上限（99 KB = 0x18c00 = 101376 B）
ptxas error : Entry function '_Z1kPf' uses too much shared data (0x28000 bytes, 0x18c00 max)
# -> SM120 统一 L1/smem 128 KB，其中 smem 每 SM 上限 100 KB（102400 B），
#    单 block opt-in 上限 101376 B（ch17 实测口径）

# 4. setmaxnreg 被 ptxas 静默丢弃（warning，不是 error）
#    ... C7506: ... ignored to maintain compatibility into 'extern' call ...
# -> TMA dst 写成了 shared::cluster（隐式 call 边界触发，与 arch 后缀
#    无关）；改成 shared::cta 即保留（第~\ref{ch:14}~章坑二取证）。
#    另一类触发：__launch_bounds__(N) 缺第二参数 -> C7508，补 (N, 1)。

# 5. CUDA Toolkit 版本不足（common.cuh L21--23 门控）
#error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
# -> 升级工具链，或改用 raw PTX 分支并降级宏组合
```

另一类「不报错但全错」的运行期坑（系统性错值而非随机数）见第 十五 章常见坑一节：smem 基址未对齐 1024B、swizzle 档位选错、期望字节数没传给 mbarrier。

C.7 全书 218 个测试：运行与期望输出

一条命令跑全书最小测试（验收机口径）：

```
cd kernels/interview/book/tests
./build_tests.sh --arch sm_120a --all
```

实测输出摘录（RTX PRO 5000，2026-09-24；全书 30 个测试可执行——ch13 在 sm_120a 下自动跳过、ch15 原理章无测试；含 ch26b/ch26c 与 ch34–37——共 218 条 `PASS`、0 条 `FAIL`，末行 `ALL OK`、退出码 0）：

```
(ch13 skipped: requires -arch sm_90a; see ch13 §13.10 for explicit build)
== ch01_roofline.cu (sm_120a)
PASS GEMM     FLOPs=1.3744e+11 Bytes=2.0133e+08 AI= 682.6667 (expect  682.6667, rel_err=4.9e-08)
PASS softmax  FLOPs=2.0480e+04 Bytes=3.2768e+04 AI=  0.6250 (expect    0.6250, rel_err=0.0e+00)
...（ch02--ch26 依次构建运行；各章输出行格式一致）...
PASS ffpa split-d cute D=128 N=128 Sk=2 Sv=2 B=1 H=2: max_abs_err=8.084e-05 (tol=5.0e-02)
ALL OK
```

判定行格式统一为 `PASS/FAIL <name>: max_abs_err=<e> (tol=<e>)`，超差时追加 `first_bad_index`；arch 不在位输出 `SKIP(chNN): requires sm_XX, device is sm_YY` 且退出码 0（不视为失败）。逐章测试的设计口径（规模、容差档、K-tile 回绕覆盖）见各章「最小测试与动手实验」节；ch13 的 Hopper 复现命令见第 十四 章 13.10 节。

附录 D　源码索引

正文关键代码段的完整实现入口。行区间与 `scripts/anchors.yaml` 冻结登记一致；链接为 GitHub commit permalink（防 branch 漂移）。

章	主题	源码区间（permalink）
A	common.cuh 工具箱	common.cuh L1–L803
第 01	GPU 架构/Roofline 速查	base.cuh L1–L86
第 02	warp/block 归约与 dot	base.cuh L87–L305
第 03	向量化与原子操作	base.cuh L306–L386
第 04	softmax 三级递进	base.cuh L520–L665
第 05	merge_attn_states	base.cuh L387–L519
第 06	RMSNorm/LayerNorm	base.cuh L666–L801
第 07	RoPE 与转置	base.cuh L802–L909
第 08	sgemv 三种划分	sgemv.cuh L1–L102
第 09	sgemm 阶梯一	sgemm.cuh L6–L183
第 10	sgemm TF32/WMMA	sgemm.cuh L184–L434
第 11	hgemm mma.sync	hgemm.cuh L3–L397
第 12	hgemm swizzle	hgemm.cuh L399–L716, common.cuh L110–L348
第 13	WGMMA/TMA/mbarrier	hgemm.cuh L1428–L1857, common.cuh L350–L803
第 14	SM120 TMA+WS	hgemm.cuh L1859–L2100, common.cuh L467–L517
第 15	FA 原理（头注释）	flash_attn.cuh L5–L110
第 16	FA2 split-Q+MMA	flash_attn.cuh L5–L790
第 17	FA2 TMA+WS	flash_attn.cuh L792–L1440
第 18	FA3 双 consumer	flash_attn.cuh L1441–L2192
第 19	FFPA split-D 全篇	ffpa_attn.cuh L1–L641
第 20	CuTe Layout 对照	hgemm.cuh L788–L820
第 21	CuTe TiledCopy 对照	hgemm.cuh L821–L1196
第 22	CuTe TiledMMA 对照	ffpa_attn.cuh L31–L79
第 23	CuTe SW128 atom 对照	hgemm.cuh L1232–L1270
第 24	CuTe HGEMM	hgemm.cuh L718–L1427
第 25	CuTe FA 三实现	flash_attn.cuh L2196–L3488
第 26	CuTe FFPA 双 TiledMMA	ffpa_attn.cuh L30–L65, ffpa_attn.cuh L79–L375, ffpa_attn.cuh L377–L640
集成辅助工具多 TU 拆分（host 侧 test/bench 接线，源码在 `bench/` 子目录；kernel 与 host 封装仍在上方各.cuh，构建与角色见附录 C）：		
H	main + CLI 总装	bench/bench_leetcuda.cu L1–L440
H	基础算子 test 家族	bench/base.cu L1–L730
H	各级 GEMM test/bench	bench/sgemv.cu L1–L117, bench/sgemm.cu L1–L145, bench/hgemm.cu L1–L1187

章	主题	源码区间（permalink）
H	FP8/FP4 GEMM test/bench/sweep	bench/fp8_gemm.cu L1–L466, bench/fp4_gemm.cu L1–L594
H	FA1/2/3 + persist-D test/bench	bench/flash_attn.cu L1–L2695
H	split-D test/bench	bench/ffpa_attn.cu L1–L415
H	跨模块共享 check/bench 参照工具	bench/utils.cu L1–L83
增补章（ch26b/ch26c/ch34–ch37）源码为本仓新增/追加区间（ffpa_attn.cuh L643–L1306、fp8_gemm.cuh L1–L958、fp4_gemm.cuh L55–L1297、集成辅助工具模块.cu），随增补 commit 在 anchors.yaml 冻结		
以下 Part VI（第 27–33 章）源码在 `ffpa-attn` 仓（commit `861d75e`），路径前缀以 `csrc/cuffpa/` 为主（bench 与 CLI 见第 33 章）		
第 27	量化注意力数学	cute/fp8/fp8_pscale.cuh L17–L37, L111–L138
第 28	fp8 量化前处理链	cute/fp8/smooth_k.cuh L12–L84, cute/fp8/quantize_fp8.cuh L29–L201, cute/hadamard.cuh L1–L92
第 29	fp8 persist-D	cute/fp8/sm_120/persist_d.cuh L37–L215, L583–L1031, cute/fp8/reg2reg_8b.cuh L14–L141
第 30	fp8 split-D/M4N2	cute/fp8/sm_120/split_d.cuh L39–L245, L446–L756, split_d_m4n2.cuh L300–L970
第 31	fp4 量化链	cute/fp4/quantize_fp4.cuh L32–L320, L322–L496, cute/fp4/delta_s.cuh L2–L161
第 32	fp4 persist-D	cute/fp4/sm_120/persist_d.cuh L1–L120, L722–L1066, cute/fp4/fp4_pscale.cuh L117–L346
第 33	bench 与 CLI	bench/bench_fp8.py L1–L140, src/ffpa_attn/cli/_bench.py L347–L490

基准 commit：LeetCUDA `6cd32de`（v4.0.0，2026-09-17 冻结）；`anchors.yaml` 现为 repo_commit `3b52c5c`、frozen_at 2026-09-29；Part VI 各章 ffpa-attn `861d75e`（2026-09-16 锚定）；Part V 四章（第 三十–三十三）与增补章（ch26b/ch26c）为本仓新增区间，随增补 commit 在 `anchors.yaml` 冻结。行号漂移由 `verify_anchors.py` 兜底：SHA256 + 首末行锚点双重校验。

附录 E　参考文献与延伸阅读

E.1　官方文档与仓库

- NVIDIA PTX ISA（*Parallel Thread Execution*）
  https://docs.nvidia.com/cuda/parallel-thread-execution/
  正文考据框的核查基准：mma/ldmatrix（§9.7.14.5）、mbarrier （§9.7.13.15）、cp.async.bulk.tensor 与 tensor map 位域（§9.7.9.25/§9.7.9.26）、WGMMA（§9.7.15.5）、setmaxnreg （第 十四、十五 章）；
- CUDA C++ Programming Guide
  https://docs.nvidia.com/cuda/cuda-c-programming-guide/
  shuffle 掩码语义（第 九 章坑二）、异步执行与内存模型；
- CUTLASS 功能文档（media/docs/cpp）
  https://github.com/NVIDIA/cutlass/tree/main/media/docs/cpp
  swizzle 模式、TMA descriptor 与 Blackwell/sm120 相关功能文档——第 十三、十五 章与 CuTe 篇（第 二十一–二十七 章）类型系统的上游依据；
- FlashAttention 仓库
  https://github.com/Dao-AILab/flash-attention
  FA2/FA3 参考实现（第 十七–十九 章的对照对象）；
- ffpa-attn 仓库（Part VI 代码链路，源码索引锚定 commit `861d75e`，permalink 见附录 D）
  https://github.com/vipshop/ffpa-attn
  SageAttention 2/3 思想的 CuTe 高效复现：fp8/fp4 量化前处理链 + persist-D/split-D/M4N2 kernel 家族（第 二十、二十七、三十四–四十 章）；
- Nsight Compute 文档
  https://docs.nvidia.com/nsight-compute/
  bank conflict 计数器与 stall 指标（第 十三、十七 章 profile 口径）；
- LeetCUDA 仓库（本书源码，冻结 commit 的 permalink 见附录 D）
  https://github.com/xlite-dev/LeetCUDA

E.2　论文

- T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022, arXiv:2205.14135 （第 十六 章）；
- T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691, 2023 （第 十六–十八 章）；
- J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, T. Dao. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. arXiv:2407.08608, 2024 （第 十六、十九、三十五 章）；
- A. Milakov, N. Gimelshein. Online normalizer calculation for softmax. arXiv:1805.02867, 2018（NVIDIA 内部技术报告；online softmax 递推与二元合并的原始出处，第 五 章；第 六 章主题相关——merge 公式同源）；

- J. Zhang et al. SageAttention: Accurate 8-bit Attention for Plug-and-play Inference Acceleration. arXiv:2410.02367, 2024（第一个端到端指标近无损（损失可忽略）的量化 attention：int8 QK + fp16 PV、smooth-K，第 三十四、三十五 章）；
- J. Zhang et al. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT8 Quantization. arXiv:2411.10958, 2024（int4/int8 QK + fp8 PV、per-thread 粒度对齐 MMA fragment，第 三十四–三十六 章）；
- J. Zhang et al. SageAttention2++: A More Efficient Implementation of SageAttention2. arXiv:2505.21136, 2025（fp8-f16acc MMA 与值域约束 $448 \times 2.25 \leq 2047$，第 三十四、三十六 章）；
- J. Zhang et al. SageAttention3: Microscaling FP4 Attention for Inference and An Exploration of 8-bit Training. arXiv:2505.11594, 2025（Blackwell FP4：NVFP4 microscaling + 两级 P 量化，第 三十四、三十八、三十九 章）；
- T. Zadouri, M. Hoehnerbach, J. Shah, et al. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv:2603.05451, 2026（lazy rescale / conditional rescaling 的原始出处，第 三十四、三十六、三十九 章）；
- FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv:2501.01005, 2025（`merge_attn_states` 的论文出处，第 六 章）；
- J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021（RoPE 旋转位置编码的原始出处，第 八 章）。

E.3 知乎系列（按主题）

正文各章末「延伸阅读」给出当章主参考；本节按主题汇总全集。系列文章以首篇 URL + 「系列」标注，避免逐篇罗列。

E.3.1 GPU 架构与指令集（第 1 章）

- @reed《GPU 指令集架构》系列八篇：前言 https://zhuanlan.zhihu.com/p/686198447、寄存器 https://zhuanlan.zhihu.com/p/688616037、Load 和 Cache https://zhuanlan.zhihu.com/p/692445145、浮点运算 https://zhuanlan.zhihu.com/p/695667044、整数运算 https://zhuanlan.zhihu.com/p/700921948、比特和逻辑操作 https://zhuanlan.zhihu.com/p/712356884、Warp 级和 Uniform 操作 https://zhuanlan.zhihu.com/p/712357647、程序控制和原子操作 https://zhuanlan.zhihu.com/p/712357443（第 1 章；Load/Cache 兼第 11 章、原子兼第 3 章）；
- @ 紫气东来《CUDA》系列三篇（编程基础/内存体系/GEMM 从入门到熟练）：https://zhuanlan.zhihu.com/p/645330027、https://zhuanlan.zhihu.com/p/654027980、https://zhuanlan.zhihu.com/p/657632577（第 1、9 章）。

E.3.2 基础算子优化（第 2–9 章）

- @ 懒蚂蚁呀不嘿《CUDA 算子详解》三篇：element-wise https://zhuanlan.zhihu.com/p/188863073 5520391519、transpose https://zhuanlan.zhihu.com/p/1899760505733756129、reduce https://zhuanlan.zhihu.com/p/1905661893739283464（第 3、7、2 章）；
- @ 紫气东来《CUDA 算子系列》：ops(1) LayerNorm https://zhuanlan.zhihu.com/p/694974164、ops(2) SoftMax https://zhuanlan.zhihu.com/p/695307283、ops(5) 激活与残差 https://zhuanlan.zhihu.com/p/695703671、ops(7)(8) self-attention 上/下 https://zhuanlan.zhihu.com/p/695898274、https://zhuanlan.zhihu.com/p/696197013（第 6、4、3、15 章）；
- 《[CUDA 优化实战] RoPE：手写算子的作用之 kernel fusion》https://zhuanlan.zhihu.com/p/201104557 9652895890（第 7 章）；

- @ 有了琦琦的棍子《深入浅出 GPU 优化系列：gemv 优化》https://zhuanlan.zhihu.com/p/494144694（第 8 章）；
- @ 白牛《CUDA 入门的正确姿势：how-to-optimize-gemm》https://zhuanlan.zhihu.com/p/478846788（第 9 章，README 源）。

E.3.3　Tensor Core 与 MMA 编程（第 10–11 章）

- @ 木子知《Nvidia Tensor Core 初探》https://zhuanlan.zhihu.com/p/620185229、《WMMA API》https://zhuanlan.zhihu.com/p/620766588、《MMA PTX 编程入门》https://zhuanlan.zhihu.com/p/621855199（第 10、11 章）；
- @Anonymous《GEMM 细节分析 (一)：ldmatrix 的选择》https://zhuanlan.zhihu.com/p/702818267（第 11 章）；
- @WingEdge777《[CUDA 优化实战] hgemm：超越 cuBLAS 之 Tensor-core、cp.async、ldmatrix》https://zhuanlan.zhihu.com/p/2017717179840214041（第 11 章）；
- @nicholaswilde《CUDA Ampere Tensor Core HGEMM 优化》https://zhuanlan.zhihu.com/p/555339335（第 11 章）。

E.3.4　Swizzle 与 bank conflict（第 12、23 章）

- @frankshi《CUDA shared memory 避免 bank conflict 的 swizzling 机制解析》https://zhuanlan.zhihu.com/p/4746910252（第 12 章主参考，README 源）；
- @reed《cute 之 Swizzle》https://zhuanlan.zhihu.com/p/671419093；
- @melonedo《布局代数实战：Swizzle 自动推导》https://zhuanlan.zhihu.com/p/19413064426835150 68、《Swizzle 布局的直观解释和推导》https://zhuanlan.zhihu.com/p/1935799939990008359；
- @ 进击的 Killua《cute Swizzle 细谈》https://zhuanlan.zhihu.com/p/684250988；
- @Anonymous《GEMM 细节分析 (三)：Swizzle⟨B,M,S⟩ 参数取值》https://zhuanlan.zhihu.com/p/713 713957；
- @Titus《cutlass swizzle 机制解析（一）（二）》https://zhuanlan.zhihu.com/p/710337546、https://zhuanlan.zhihu.com/p/711398930；
- @JoeNomad《cutlass block swizzle 和 tile iterator》https://zhuanlan.zhihu.com/p/679929705、《cutlass bank conflict free 的 smem layout》https://zhuanlan.zhihu.com/p/681966685；
- @ 皓月争曦《动手 Attention 优化 3：理解 Bank Conflict 及 Cutlass Swizzle》https://zhuanlan.zhihu.com/p/9840919069；
- @ 德布劳钦《从一个反直觉的 Case 说起：A Deep Dive into CUDA Shared Memory Bank Conflict》https://zhuanlan.zhihu.com/p/1990531281377833323；
- @zzk again《DeepGEMM transpose swizzle 推导》https://zhuanlan.zhihu.com/p/203177889515072 0504；
- @ 韭浪《Optimal Generic Swizzling》https://zhuanlan.zhihu.com/p/2005574080691185656；
- @Arthur《Bank Conflict 自动消除：Swizzle 技术原理解析》https://zhuanlan.zhihu.com/p/204204949 9263136652；
- @ 北漂小菜鸡《Swizzle 工作原理》https://zhuanlan.zhihu.com/p/1996740631024924636；
- @Hardy《CUDA Kernel：Tile、Layout 与 Swizzle》https://zhuanlan.zhihu.com/p/206584097309206 7294（兼第 20 章）。

E.3.5 异步搬运：cp.async / mbarrier / TMA / WGMMA（第 10、13、14、17 章）

- @Frank Wang《Async Copy 及 Memory Barrier 指令的功能与实现》https://zhuanlan.zhihu.com/p/685168850；
- @reed《cute 之 TMA Descriptor 编码与隐藏的第 21bit》https://zhuanlan.zhihu.com/p/2037200219700449995（第 13 章主参考）；
- @ 竹熙佳处《CuTe 教程：TMA Copy》https://zhuanlan.zhihu.com/p/2003198909405763007；
- @ 郑思泽《[施工中] 在 Hopper GPU 上实现 CuBLAS 90% 性能的 GEMM》https://zhuanlan.zhihu.com/p/695589046（兼第 24 章）。

E.3.6 CuTe：Layout 与代数（第 20 章）

- @reed《cute 之 Layout》https://zhuanlan.zhihu.com/p/661182311、《cute Layout 的代数和几何解释》https://zhuanlan.zhihu.com/p/662089556（第 20 章主参考）；
- @ 竹熙佳处《CuTe 教程：Layout Compose & Inverse》https://zhuanlan.zhihu.com/p/1962625273636845008、《Layout Product & Divide》https://zhuanlan.zhihu.com/p/1971945267294111573；
- @melonedo《Cute 布局代数实战：除法》https://zhuanlan.zhihu.com/p/1970274785691936058、《从线性布局的视角看 CuTe 布局》https://zhuanlan.zhihu.com/p/1937977743569577920；
- @Anonymous《介绍一个关于 CuTe Layout 变换的小技巧》https://zhuanlan.zhihu.com/p/2006000375463961170；
- @ 朱小霖《cutlass cute 101》https://zhuanlan.zhihu.com/p/660379052（兼第 24 章）；
- @ 可怕的杰瑞《Cute 极简教程》https://zhuanlan.zhihu.com/p/1991248525925819668（第 20–22 章导览）；
- @DeclK《CUTLASS CUTE 系列 1–5》https://zhuanlan.zhihu.com/p/1983922984805742098（Layout 代数/MMA & COPY 抽象/GEMM 优化/Hopper TMA & WGMMA，第 20–24 章系统性读物）；
- @ 北漂小菜鸡《CuTe 教程：Layout》https://zhuanlan.zhihu.com/p/1997872253799519607。

E.3.7 CuTe：TiledCopy 与 TiledMMA（第 21、22 章）

- @ 竹熙佳处《写给大家看的 CuTe 教程：tiled copy》https://zhuanlan.zhihu.com/p/1930389542784964333（第 21 章主参考）、《写给大家看的 CuTe 教程：tiled mma》https://zhuanlan.zhihu.com/p/1937145378446226159（第 22 章主参考）；
- @reed《cute 之 Tensor》https://zhuanlan.zhihu.com/p/663093816、《cute 之 Copy 抽象》https://zhuanlan.zhihu.com/p/666232173、《cute 之 MMA 抽象》https://zhuanlan.zhihu.com/p/663092747；
- @Anonymous《GEMM 细节分析 (二)：TiledCopy 与 cp.async》https://zhuanlan.zhihu.com/p/703560147；
- @ 水木皇工仔《基于 CuTe 理解 swizzle, LDSM, MMA》https://zhuanlan.zhihu.com/p/934430036；
- @ 可怕的杰瑞《Cute TiledMMA 简单理解（一）》https://zhuanlan.zhihu.com/p/1991908850132088026、《Cute TiledMMA 极简理解（二）》https://zhuanlan.zhihu.com/p/1992328297888126320；
- @weishengying《cute swizzle》https://zhuanlan.zhihu.com/p/706796240（第 23 章）。

E.3.8　CuTe GEMM 与流水线（第 24、25 章）

- @reed《cute 之简单 GEMM 实现》https://zhuanlan.zhihu.com/p/667521327、《cute 之 GEMM 流水线》https://zhuanlan.zhihu.com/p/665082713、《cute 之高效 GEMM 实现》https://zhuanlan.zhihu.com/p/675308830（第 24 章主参考三篇）；
- @ 竹熙佳处《CuTe 笔记：permutationMNK 参数》https://zhuanlan.zhihu.com/p/19735267101054199 53、《写给大家看的 CuTe 教程：async pipeline》https://zhuanlan.zhihu.com/p/2024832248386528275（兼第 25 章）；
- @ 进击的 Killua《CUTLASS CuTe 实战（一）基础》https://zhuanlan.zhihu.com/p/690703999、《(二)应用》https://zhuanlan.zhihu.com/p/692078624；
- @BBuf《CUTLASS 2.x & 3.x Intro 学习笔记》https://zhuanlan.zhihu.com/p/710516489、《Hopper Mixed GEMM 的 CUTLASS 实现笔记》https://zhuanlan.zhihu.com/p/714378343；
- @ 水木皇工仔《基于 CUTE 的 GEMM 优化【2】——高效 GEMM 实现，超越 Cublas 20%》https://zhuanlan.zhihu.com/p/696028389；
- @ 哈密瓜《cutlass cute 中 GEMM 的理解和使用》https://zhuanlan.zhihu.com/p/189223706915108 5950；
- @JoeNomad《cutlass 多级流水线》https://zhuanlan.zhihu.com/p/687397095；
- @Titus《GEMM 流水线：single/multi-stage、pipeline》https://zhuanlan.zhihu.com/p/712451053（兼第 25 章）。

E.3.9　FlashAttention 与 FFPA（第 15–19、25、26 章）

- @DefTruth《图解：从 Online-Softmax 到 FlashAttention V1/V2/V3》https://zhuanlan.zhihu.com/p/6 68888063（第 15 章主参考，亦是全书行文风格基线之一）；
- @DefTruth《FFPA(Split-D)：FA2 无限 HeadDim 扩展》https://zhuanlan.zhihu.com/p/13975660308（第 19、26 章主参考）；
- @DefTruth《vLLM Triton Merge Attention States Kernel 详解》https://zhuanlan.zhihu.com/p/19049 37907703243110（第 5 章）；
- @DefTruth《LeetCUDA v3.0 大升级（项目自述）》https://zhuanlan.zhihu.com/p/19862356369（全书项目背景）；
- @66RING《使用 cutlass cute 复现 flash attention》https://zhuanlan.zhihu.com/p/696323042（第 25 章主参考）；
- @shengying.wei《FlashAttention 笔记：tiny-flash-attention 解读》https://zhuanlan.zhihu.com/p/70886 7810、《FlashAttention fp8 实现（ada 架构）》https://zhuanlan.zhihu.com/p/712314257（第 25 章）。

以上知乎条目的存在性分别于入册时经知乎开放平台检索接口核验（首批　2026-09-11；后补条目随 `zhihu-inventory.md` 增补时核验。标记「README 源」的两篇来自 LeetCUDA README 作者自校清单）；完整字段（对应章、核验状态、图片引用）见 `book/references/zhihu-inventory.md`。